

—— // STRYKE // ——

stryke – Language Reference

v0.17.47

LANGUAGE REFERENCE

every builtin · keyword · alias · extension



MENKE
TECHNOLOGIES

Copyright and Attribution

stryke — the parallel dynamic language on fusevm

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the Perl Foundation, the Ruby, Clojure, Racket, or Scala projects, the Rust Foundation, or any other project named in these pages.

The software this book describes

`stryke` is released under the MIT License, Copyright © 2026 MenkeTechnologies. The source is at github.com/MenkeTechnologies/strykelang.

stryke is an original language, not a port. Its syntax is a deliberate synthesis — Perl's sigils, Clojure's and Racket's threading forms, Scala's underscore, Ruby's block and interpolation notation — and where this book names one of those languages it is crediting an influence, not citing a source. The `--compat` mode is a compatibility surface, not a fork of any existing implementation.

The Perl modules under `vendor/perl/` in the stryke repository are minimal pure-Perl stubs written for this runtime, not copies of core Perl's own modules.

The engine underneath

stryke executes on `fusevm` — the bytecode virtual machine and three-tier Cranelift JIT it shares with `zshrs` and the rest of the language fleet. `fusevm` is released under the MIT License, Copyright © 2026 MenkeTechnologies, and its source is at github.com/MenkeTechnologies/fusevm.



MT PUBLISHING



MT OPEN SOURCE

Contents

Copyright and Attribution	1
Style Guide	8
Parallel Primitives	18
Shared State & Concurrency	24
Pipeline & Pipe-Forward	28
Streaming Iterators	32
List Operations	37
Operators	49
stryke Extensions	61
Data & Serialization	67
HTTP & Networking	76
Crypto & Encoding	81
Special Math Functions	102
Bessel / Airy / Hankel / Struve / Kelvin	104
Orthogonal Polynomials	106
Elliptic Integrals & Functions	108
Zeta / Polylog / Lerch	110
Hypergeometric	111
Modular Forms	112
Si / Ci / Ei / Li / Fresnel Integrals	113
Number Theory (Wolfram Parity)	114
Combinatorics (Wolfram Parity)	116
q-Series, Mittag-Leffler, Coulomb	117
Inverse Special Functions	118
Piecewise & Symbolic Primitives	119
Number Theory (Wolfram Parity II)	120
Combinatorial Sequences	122
Linear Algebra (Extended)	123
Polynomial Algebra	124

Distributions (Extended)	125
Mathieu / Heun	127
Wavelets	128
Graph Algorithms (Extended)	129
Number-Theoretic Sums & Constants	130
Vector Calculus	131
Optimization	132
Numerical Integration	133
Linear Algebra (Numeric)	134
Polynomial Helpers	135
Quaternions & 3D Rotations	136
Information Theory	137
Quantum Primitives	138
Statistical Mechanics	139
Optics	140
Astrodynamics	141
Time Series	142
Graph Centrality	143
Random Samplers (Extended)	144
2-D Geometry	145
Auto-Differentiation	146
Statistical Tests (R parity)	147
Distance Metrics (Distances.jl parity)	149
Multivariate / Non-Central Distributions	150
Matrix Functions	151
Adaptive ODE Solvers	152
Generalized Linear Models	153
Bootstrap & Resampling	154
Time Series Operators	155
Combinatorial Generators	156
Dynamic Programming	157
ML Metrics	158
DSP / Image Filters	159
Stochastic Processes	160

Compression / Information Complexity	161
Physics & Quantum (Extended)	162
Number Theory & Algebra (Extended)	163
CAS-Lite	164
Quadrature (Specialised)	165
Optimization (Heuristic)	166
Distributions (Tail / Mixture / Categorical)	167
Clustering & Dimensionality Reduction	169
Neural-Net Primitives	170
Time Series (Spectral)	171
Image Processing	172
Loss Functions	173
Spatial / Geographic	174
Integer Sequences	175
Graph Link & Cluster Metrics	176
3-D Geometry	177
Iterative Solvers	178
Crypto / Modular Algebra	179
Physics & Chemistry	180
MCMC & SDEs	181
Hidden Markov Models	182
Survival Analysis	183
Sequence Alignment	184
Chemistry	185
Control Theory	186
Game Theory	187
Operations Research	188
Numerical PDE	189
Bayesian Conjugate Updates	190
Quantum Computing Gates	191
Splines & Curves	192
Music & Audio	193
Astronomy & Cosmology	194
Fluid Dynamics	195

Distributions (Discrete / Circular)	196
Random Graph Generators	197
Color Science	198
Integer Sequences (Combinatorial)	199
Multiple-Testing Corrections	200
Probabilistic Distances	201
Distributions (Tail / Inflated)	202
Astrophysics & Radiation	203
Chemistry (Beer / Rate / Colligative)	204
Mixed-Strategy Game Theory	205
Computer Graphics	206
DSP (Hilbert / Cepstrum / Filter Design)	207
Image Processing (Edges / Bilateral)	208
Clustering Helpers	209
Combinatorics (Tableaux / Boustrophedon)	210
Number Theory (Tetration / Smoothness)	211
Network Coreness & Structure	212
Crypto Helpers	213
Quantum Entanglement	214
2-D Geometry (Polygons & Circles)	215
Time-Series Smoothing	216
Bioinformatics	217
Geographic / Map Projection	218
Fixed-Income Finance	219
Image Quality Metrics	220
Acoustics	221
Population Genetics	222
Epidemiology	223
Inequality Measures	224
APL / J Array Primitives	225
Plasma Physics	226
String Similarity	227
Rating Systems	228
Effect Sizes	229

Control Transient Response	230
Matrix Norms	231
Network Triad Analysis	232
Misc Inverses	233
Parallel I/O	234
CLI / Args	237
File I/O	238
Strings	249
Arrays & Hashes	256
Control Flow	261
Error Handling	266
AOP / Advice	269
Declarations	271
Cluster / Distributed	275
Scriptable Distributed Compute (congregation)	277
Provenance / Lineage	282
Datetime	284
Math	288
File System	291
Process	298
Pack / Binary	303
Logging	304
Charts (SVG)	306
Audio	309
Network Utilities	310
Markup / Web Scraping	321
Date Utilities	322
Git	323
System	325
Terminal Control	328
Testing	332
Test Runner (worker pool)	334
AI primitives (docs/AI_PRIMITIVES.md)	335
MCP (Model Context Protocol)	344

Web framework (stryke_web/README.md)	346
Stress / telemetry (docs/STRESS_TESTING.md)	358
PTY / expect (docs/expect-feature-idea.md)	362
Other	364

Style Guide

1 topics

style

stryke style guide

Authoritative idioms for stryke source. The principle behind every rule: **min chars win**. stryke is the power-user language; every design call defaults to the terser form, no newbie concessions, no optional verbosity.

Every example pair is **DO / DON'T** with one-line rationale. The DON'T side is valid stryke (so old Perl ports keep parsing) — but new code that uses it is wrong on style, and `--no-interop` rejects most of it at parse time as a forcing function.

0. The ten hard rules

Every other idiom is downstream of these. If a piece of stryke code violates any of them, it is wrong style, full stop.

1. **fn not sub**. `sub` is Perl 5 baggage and is rejected by `--no-interop`. 2. **p not print, never say**. `p` is the stryke verb, has the record separator built in, costs one character. `say` is rejected by `--no-interop`. 3. **Use `|> / ~>` over nested calls**. `f(g(h(x)))` reads inside-out; `x |> h |> g |> f` reads left-to-right and shows up at the call site without diving the cursor through three closing parens. **Always pipeline**. 4. **No semicolons at end of line**. stryke statement-terminates on newline. Trailing `;` is noise that visually weights every line. The only place `;` belongs is **between statements on the same line** (rare, e.g. C-style `for` headers): `for ($i = 0; $i < 10; $i++)`. 5. **Implicit closure parameters always**. Never write `sub { my $x = shift; ... }` or `fn { my ($a, $b) = @_; ... }`. Use `_` for slot 0 and `_N` for slot N. Named params are only correct when the closure is a real top-level `fn` whose arity > 2 and* the names carry meaning. 6. **Expression-bodied fn for one-liners**. `fn double = _ * 2 over fn double($a) { $a * 10 }` over `fn double { _ * 2 }`. No braces, no named param, implicit topic. The block form is for multi-statement bodies only. 7. **Implicit return — never return on the tail expression**. The last expression in any `fn` body, arrow-block, or expression-position block IS the return value. `return` is **only** correct as an early-return guard (`return undef if $err`). Writing `return $x` at the bottom of a body is noise. 8. **No parens on fn calls unless precedence demands them**. stryke calls take args list-operator-style. `p "hello", len @a, uc rev "abc"` — paren-less. Add parens **only** when the call is followed by an operator that would re-bind (`len(@a) > 1`), when nesting needs disambiguation, or when an empty arg list is the intent (`time()`). 9. **No trailing 1 (or any sentinel truthy value) at the end of a library / module file**. That's Perl 5's `package returns true` ritual — `require` there demanded the last evaluated expression be truthy, so every `.pm` ended in `1`; stryke `use/import` resolution does not consult the file's tail expression, so the `1` is pure noise inherited from CPAN muscle memory. Library files end at the last `fn` (or `class / const`) definition. `--no-interop` rejects a bare-integer tail expression in module position. 10. **val for immutable, var for mutable — never bare my in new code**. stryke flips Perl 5's mutable-by-default `my` into Scala/Kotlin-style immutable-by-default: `val` marks the binding *frozen* at compile time (reassignment, push/pop, `$x[i] = $h{k} = delete $h{k}` all error at compile time, not run time); `var` is the mutable counterpart. Both desugar to `my` in the AST — same scope, same bytecode, same slot allocation — so the cost is zero. Default to `val`; reach for `var` only when you actually mutate. `val` also accepts a type annotation (`val $port : Int = 8080`). `for val $x (LIST)` and `for var $x (LIST)` both work — use `for val` unless the loop body reassigns the iterator (almost never). Bare `my` stays valid for CPAN ports (it's what Perl 5 spells), but new stryke code wrong-on-style; `--no-interop` rejects it.

```
# RIGHT - fn, p, pipeline, no semicolons, implicit params, expression-body, implicit return
fn shout = uc(_) . "!"
@lines |> maps { trim(_) |> shout } |> greps { len(_) > 0 } |> ep

# WRONG - every rule violated
sub shout { my $s = shift; return uc($s) . "!" ; }
for my $line (@lines) {
  my $trimmed = trim($line);
  my $upper = shout($trimmed);
  if (length($upper) > 0) {
    print "$upper\n";
  }
}
```

The wrong version is 7× the characters and reads bottom-up because of the `for` ↔ `print` distance. Walk the violations: `sub` (rule 1), `print` (rule 2), nested `length(...)` no pipeline (rule 3), trailing `;` everywhere (rule 4), explicit `my $s = shift` not implicit `_` (rule 5), block-bodied `fn` for a one-liner (rule 6), explicit `return` on tail (rule 7). All seven, in one wrong shout-and-loop.

0a. Paren-less calls – list-operator style

stryke is a list-operator language: function args bind right of the function name without parens. `p "hello"`, `len @a`, `uc rev "abc"` are all preferred. Parens are noise unless they're earning their keep.

| DO | DON'T | Why | |---|---|---| | `p "hello"` | `p("hello")` | Single arg; parens add nothing. | | `p $x, $y, $z` | `p($x, $y, $z)` | Multi-arg works the same paren-less. | | `len @a` | `len(@a)` | Builtin call; LHS-binds. | | `uc rev "abc"` | `uc(rev("abc"))` | Right-to-left chain reads naturally; no escape parens. | | `len(@a) > 1` | `len @a > 1` | **Parens NEEDED** — without them, `>` rebinds and you get `len(@a > 1)` which is `len(bool) = 0`. | | `len(keys %h)` | `len keys %h` | Same — keys's list result needs to be sealed before `len` can act on it explicitly. | | `time()` | `time` | When you genuinely want zero args, `()` makes the call-vs-bareword intent obvious. |

Three legitimate reasons to add parens: 1. **Precedence** — when a binary operator follows the call: `len(@a) > 1`, `uc($s) . "!"`. 2. **Disambiguation** — when two adjacent unparenthesized list-ops would slurp each other's args ambiguously: `func1(arg1, arg2), func2(arg3)`. 3. **Zero-arg explicit** — `time()` over `time` when readers might mistake the bareword for a value lookup.

In a pipeline, parens are almost never needed because `|> / ->` already supply the staging — write `$x |> uc |> rev` not `rev(uc($x))` and not `($x) |> uc() |> rev()`.

1. Output

| DO | DON'T | Why | |---|---|---| | `p $x` | `print $x`, `print "$x\n"`, `say $x` | `p` is the canonical print-with-newline. One char vs five-to-seven. **say is rejected by --no-interop.** | | `warn "$msg"` | `print STDERR "$msg\n"` | `warn` is the stryke-and-Perl verb for stderr. Auto-appends `\n` if the message doesn't end with one. Catchable via `$$SIG{__WARN__}`. | | `@xs \> e p` | `for (@xs) { p $_ }` | Pipe-forward + `e` (each) collapses the loop to four chars after the pipe. | | `@xs \> ep` | `@xs \> e p` | `ep = e { p }` shorthand. **Goes to stdout, NOT stderr** — name is "each-print", not "err-print". | | `ddump $struct` | `use Data::Dumper; print Dumper($struct)` | Built-in pretty-printer; one bareword. |

1a. String interpolation – `#{ ... }` / `${ ... }` over `.` concat

stryke supports four interpolation forms inside double-quoted strings. **Always interpolate; never build strings with `.` concatenation when interpolation reaches.**

| Form | Example | Use when | |---|---|---| | `$name` | `"hello, $name!"` | Bare scalar; the cheapest form. | | `${name}` | `"file_${name}.txt"` | Bare scalar **with disambiguation** — when adjacent chars would extend the var name (`${name}_v2`, not `$name_v2` which looks for `$name_v2`). | | `#{ EXPR }` | `"x squared is #{ $x * $x }", "sum: #{sum @a}", "page #{1 + $i}"` | Ruby-style interpolation of an **arbitrary expression** — math, function calls, method calls, anything. The only form that's not just-a-variable. | | `@arr` | `"items: @arr"` | Array interpolation (joined by `$`, default `" "`). |

| DO | DON'T | Why | |---|---|---| | `"hello, $name!"` | `"hello, " . $name . "!"` | Three concatenations vs zero; reads as one literal. | | `"x squared is #{ $x * $x }"` | `"x squared is " . ($x * $x)` | `#{ }` lets the expression sit inline; no parens-and-dots dance. | | `"sum: #{sum @a}"` | `"sum: " . sum(@a)` | Function calls work in `#{ }`. | | `"file_${name}.txt"`

| "file_" . \$name . ".txt" | \${} disambiguates when the next char would extend the var name. | | "items: @arr" | "items: " . join(" ", @arr) | Array interpolation auto-joins with \$" (default space). |

. concat is **only** correct when both sides are non-string-literals (\$a . \$b joining two variables) or when computing a string lazily across statements. Inside a single string-literal context, always interpolate.

```
# RIGHT - interpolated
my $name = "world"
p "hello, $name!"
p "sum is #{sum @nums}"
p "page ${i}_of_${total}"

# WRONG - concatenated
p "hello, " . $name . "!"
p "sum is " . sum(@nums)
p "page " . $i . "_of_" . $total
```

2. Topic variables

The stryke topic system has four equivalent spellings for slot 0 — pick the shortest that reads in context.

| DO | DON'T | Why | |---|---|---| | _ (bare) | \$_ | Sigil-prefixed \$_ is Perl-ism only needed inside string interpolation; bare _ is the stryke calling card. | | _1, _2 | \$a, \$b (outside sort/reduce blocks) | \$a/\$b are sort/reduce-only magic; _N is universal positional. | | _< (outer) | my \$cap = _; ... | _<...< chains depth — one < per closure level outward. | | _[3] | substr(_, 3, 1) | _[N] is char-of-topic; alias for _!N!. |

Note: \$_[N] (with \$ and []) is the legacy Perl @_ first-positional-arg, kept for compat. Don't use in new code; use _N for positional or named params for clarity.

3. Function definition

Expression-bodied form (preferred)

For one-liner functions — anything that fits on one line — use the **expression-bodied** form: `fn name = <expr>`. No braces, no named params, implicit topic.

| DO | DON'T | Why | |---|---|---| | fn double = _ * 2 | fn double { _ * 2 } | No braces; expression body. Same length saved per definition. | | fn double = _ * 2 | fn double(\$a) { \$a * 10 } | Implicit _ over named \$a; brace-less form. | | fn add = _0 + _1 | fn add(\$a, \$b) { \$a + \$b } | _N over named for arity ≤ 3 unless names carry meaning. | | fn shout = uc(_) . "!" | fn shout { uc(shift) . "!" } | _ is the topic; shift is Perl-ism. | | fn area = _0 * _1 |> int | fn area(\$w, \$h) { return int(\$w * \$h) } | Pipeline + implicit + no return keyword (last expr is the value). |

The block-bodied form (`fn name { ... }`) is correct when:

- The body is multi-statement and won't fit on one line
- Names genuinely document the role (e.g. `fn parse_url($scheme, $host, $port, $path)`)
- You need explicit `my` declarations or control flow inside the function

For one-liners, **always prefer** `fn name = expr`.

Implicit return — never write return unless it's an early-return

The last expression in a `fn` body is the return value. **Never write return for the tail expression** — it's noise, and it visually hides the actual value the function produces.

| DO | DON'T | Why | |---|---|---| | fn area = _0 * _1 | fn area(\$w, \$h) { return \$w * \$h } | Tail position auto-returns. | | fn classify { ... last expr ... } | fn classify { ... return last_expr } | Same; block bodies too. | | fn find_first { for (@_) { return \$_ if /good/ } undef } | (no DON'T) | **Early-return is the only legitimate use of return**. Use it freely for guard clauses and early exits — never for the tail. |

```

# RIGHT - implicit return, expression-bodied where possible
fn double = _ * 2
fn shout = uc(_) . "!"
fn area = _0 * _1

fn classify($n) {
  return "zero"      if $n == 0      # early return: legit
  return "negative"  if $n < 0      # early return: legit
  $n < 10 ? "small" : "big"        # tail expression: NO return keyword
}

# WRONG - explicit return on tail
fn double($x) { return $x * 2 }
fn classify($n) {
  if ($n == 0) { return "zero" }
  if ($n < 0) { return "negative" }
  return $n < 10 ? "small" : "big"
}

```

The same rule applies inside arrow-blocks (`>{ ... }`), `if/else` arms, `match` arms, and any other expression-position block.

Other rules

| DO | DON'T | Why | |`fn double = _ * 2`| `sub double { $_[0] * 2 }` | `fn` opens the topic-var protocol; `sub` is rejected by `--no-interop`. Always `fn`. | | `fn Rosetta::Foo::helper = ...` | `fn helper = ...` (when `helper` is a builtin) | Never shadow a builtin; namespace under `Module::Sub::name`. |

Builtin-shadowing is a hard rule — `fn close`, `fn log`, `fn cos`, `fn open` etc. all will abort when overriding the runtime function. Use a namespace (`MyApp::close`) or pick a different name.

4. Ranges

| DO | DON'T | Why | |`1:10`| `1..10` | `:` is the stryke polymorphic range; works for ints, chars, dates, IPs. | | `'a': 'z'` | `'a'..'z'` | Same operator, type-inferred from literals. | | `1:100:5` | — | Optional step is the third colon. | | `'I'~'V'` | `'I': 'V'` | Roman ranges use `~`, the full-extension-range separator (see below). |

Roman ranges use `~`, not `:`. Roman digits (`I V X L C D M`) collide with Perl character ranges, so under `:` / `..` / `...` they stay char ranges — `'I': 'V'` is `I, J, K, ..., V` and `'C': 'M'` is `C, D, ..., M` (11 letters). The `~` "full-extension-range" separator disambiguates: `'I'~'V'` is `I, II, III, IV, V` and `'C'~'M'` is the 901-element roman span `C ... M`. Stepping works too: `'I'~'X':2` → `I, III, V, VII, IX`. `~` is the separator to reach for whenever a range type would otherwise collide with Perl's literal (char / numeric) ranges; non-colliding types (dates, IPs, hex) infer under either separator.

Eager `:` vs lazy `range()` — both valid, different semantics. The colon range is **eager**: `1:N` materializes the whole list at once. The `range()` builtin is the **lazy** counterpart: `range(start, end[, step])` yields values on demand, so `range(1, 1e9) |> take 3` is instant and constant-memory where `(1:1e9) |> take 3` builds a billion-element list first. They are two range forms with different evaluation, not a right/wrong pair — reach for `range()` when you want laziness (very large or streamed spans, early `take`, pipeline chains) and `:` for ordinary eager spans.

5. Length / count — `len` always

`len` is the **one** stryke verb for length/count. It works on strings, arrays, hashes, sets, iterators, and anything else with a measurable size. Never reach for `length`, `scalar`, or `0 + numeric coercion` — they're all Perl-isms with separate type-specific quirks that `len` collapses into one bareword.

| DO | DON'T | Why | |`len($s)`| `length($s)` | `length` is Perl 5; `len` is three chars and unifies with array/hash forms. | | `len(@arr)` | `scalar(@arr)` | `scalar` is a context-coercion verb, not a length verb — wrong tool. | | `len(keys %h)` | `scalar(keys %h)`, `0 + keys %h` | Same call shape, no implicit-numeric tricks. | | `len(%h)` | `scalar(keys %h)` | Hash → key count directly. | | `len($iter)` | manually drain the iterator and count | Lazy iterator's known-size case is one call. | | `unpack_first(...)` | `scalar(unpack(...))` | Use the explicit non-list builtin instead of forcing scalar context for first-element. |

Aliases: `cnt`, `count` — same builtin. Use `len` by default; `cnt/count` are accepted in code where "count" reads more naturally (`pgrep { ... } |> cnt`).

Both `length` and `scalar` are rejected by `--no-interop`. There's a stryke replacement for every Perl 5 use of either:

```
| Perl 5 | stryke | |---|---| | length $s | len($s) | | scalar @arr | len(@arr) | | scalar keys %h | len(keys %h) or len(%h) | | scalar(unpack ...) | unpack_first(...) | | scalar(@arr) > 0 | len(@arr) > 0, or just @arr in boolean context | | 0 + @arr | len(@arr) |
```

6. Reverse / rev

```
| DO | DON'T | Why | |---|---|---| | rev @xs | reverse @xs | Three chars vs seven. | | rev "abc" | reverse "abc" | Same; works on scalars too. |
```

6a. String / array slicing – `[N:M]` not `substr`

stryke uses bracket-indexing with the `:` range operator for substring extraction. **Never call `substr` on a stryke value.** `[]` slicing is one syntactic form for both strings and arrays.

```
| DO | DON'T | Why | |---|---|---| | $s[1] | substr($s, 1, 1) | Single-char index. | | $s[1:3] | substr($s, 1, 3) | Slice from index 1 through index 3 inclusive. | | $s[-1] | substr($s, -1, 1) | Last char. Negative indexing built in. | | $s[-3:-1] | substr($s, -3, 3) | Last three chars. | | @arr[2:5] | @arr[2..5] | Array slicing uses the same : range op as everywhere else. |
```

Important semantic difference vs Perl `substr`: stryke's `[N:M]` is **inclusive on both ends**. `$s[1:3]` returns 3 chars (indices 1, 2, 3). Perl's `substr($s, 1, 3)` returns 3 chars starting at index 1 (length-based). Same result for that example, but the mental model is different and they diverge for any range that crosses an end:

```
my $s = "hello"
p $s[1:3]          # "ell" - chars 1, 2, 3 inclusive
p substr($s, 1, 3) # "ell" - start=1, len=3 (Perl-style; works in compat mode)
p $s[1:99]         # "ello" - clamps to end gracefully; Perl would warn
```

Open-ended slices are supported, so `$s[2:]` reads as "from index 2 to end" and `$s[0:N]` reads as "from start to N":

```
my $s = "hello"
p $s[2:]          # "llo" - index 2 through last
p $s[0:2]         # "hel" - start through index 2
p $s[-3:]         # "llo" - last 3 chars
```

For arrays, the open-ended bracket forms also work directly:

```
my @a = (10, 20, 30, 40, 50)
p @a[2:]          # 30, 40, 50 - from index 2 to end
p @a[:2]          # 10, 20, 30 - start through index 2
```

6b. No nested calls – pipeline everything

Inside-out reading is the single biggest readability tax in mainstream languages. stryke's `|>` and `->` collapse the indirection: every transformation reads top-to-bottom, left-to-right, in the order it executes.

```
| DO | DON'T | Why | |---|---|---| | $x \> trim \> uc \> first_word | first_word(uc(trim($x))) | Pipeline reads in execution order. | | @xs \> sort \> rev \> head(3) | head(rev(sort(@xs)), 3) | The head arg-position requires recoding when head becomes nth. Pipeline is rewrite-stable. | | $json \> from_json \> ->{ users } \> maps { _->{name} } | [map { $_->{name} } @{(from_json($json))->{users}}] | Six chars per stage. Each stage standalone debuggable. | | $h -> $key -> $default (Racket-style fallback) | defined $h->{$key} ? $h->{$key} : $default | -> short-circuits on undef. |
```

Rule: if you wrote two open parens in a row (`f(g())`), refactor to a pipeline. The only exception is constructor-style call sites where the arg is a literal struct (`Point(1, 2)`).

The threading-operator family is stryke's universal-access protocol — every value, every callable, and every reflection-metadata access flows through `->` `-p>` `-p>` `-s>` `-d>` without adapter code.

7. Pipeline / threading operators

stryke ships distinct threading operators. Each has its own syntactic shape and execution semantics — they are not interchangeable.

7.1 `|>` — pipe-forward (one stage at a time)

Single-stage pipe. LHS becomes the **first** argument of the RHS callable. Stages are separated by `|>`. Cheap, zero-runtime-cost desugaring.

```
"hello" |> uc |> rev |> p           # OLLEH
1:10 |> grep _ > 5 |> map _ * 2 |> e p
$url |> fetch_json |> json_jq '.name' |> p
"hello world" |> s/world/perl/ |> p # hello perl
```

Use `|>` when each stage is a single function call and you want the most explicit, line-by-line readable form. It's the day-to-day default.

7.2 `->` — thread-first macro (Racket / Clojure `->`)

Multi-stage thread. Source goes immediately after `->`; stages are **space-separated** (not arrow-separated). The threaded value is injected as the **first** argument of each stage. `|>` terminates the macro and drops back to regular pipe-forward.

```
-> " hello " tm uc rv |> p           # "OLLEH" (trim → uc → reverse)
-> @data grep { _ > 5 } map { _ * 2 } sort { _0 <= _1 } |> p
-> 10 div(2) p                       # bare-fn stage: div(10, 2)
-> 10 div(_ , 2) p                   # explicit `_` placeholder: div(10, 2)
```

Stages can be: bare function name, `name(args)` with `_` placeholder, `>{ }` anonymous block, or `map { ... } / grep { ... } / sort { ... }`. Spellings: `->`, `thread`, `t`.

Use `->` when you have ≥ 3 stages and want to drop one operator-character per stage; the macro form pays for itself at three calls.

7.3 `->>` / `->>>` — thread-last (Clojure `->>`)

Same syntax as `->`, but the threaded value goes in as the **last** argument — the right shape for list-consuming functions like `map/grep/reduce` whose Perl-tradition arity is `(block, list)`.

```
fn div = _0 / _1
->> 10 div(2) p                     # div(2, 10) = 0.2 (10 → last arg)
->> 10 div(_ , 2) p                 # explicit `_` overrides: div(10, 2) = 5

->> (1,2,3,4,5) take(3) p           # take(3, list) = (1,2,3)
```

Use thread-last when the callee's natural last arg is your data — saves wrapping `_` placeholders.

7.4 `~s>` — per-item streaming pipeline

Each stage runs as an **independent worker thread** connected to the next via a bounded channel. Items flow through concurrently; intermediate arrays are never materialized.

```
~s> [1, 2, 3, 4, 5] map { _ * 10 } map { _ + 1 }
~s> @lines map { json_decode } grep { _->[ok] } map { _->[val] }
~s> gen { yield_lines "huge.log" } map { parse } grep { /ERROR/ }
```

Use `~s>` when stages have **uneven latencies** (I/O + CPU mix) — fast stages can pull ahead while slow ones catch up. Output order is non-deterministic. `~s>>` is the thread-last variant. The macro's value is the count of items the last stage emitted.

7.5 `~p>` — chunk-parallel pipeline

Splits the input into chunks, runs the **whole pipeline on each chunk** in parallel via rayon, auto-merges the per-chunk results. Equivalent to `par_reduce { stage1 |> stage2 |> ... } SRC`.

```

~p> c("**.rs") letters freq           # histogram across all .rs files
~p> @big map { _ * 2 } sum             # parallel map+sum, auto-summed across chunks
~p> $huge letters freq ||> values |> sum # ||> drops back to sequential for the merged whole
~p> $huge letters freq |then| values |> sum # english-spelled equivalent of ||>

```

Auto-merger picks strategy from the first chunk's type: hash-of-numbers merges key-wise with `+` (canonical histogram), numeric scalars `sum`, arrays `concat`, strings `concat`. Use `||>` or `|then|` to switch to sequential `~>` mode for stages that need the merged whole. `~p>>` is the thread-last variant.

Use `~p>` when each chunk is independent CPU work and you want all-cores utilization without writing `pmap/par_reduce` by hand.

7.6 Choosing between them

| Scenario | Pick | `||>` | `||>` | 1–2 stages, mostly readable as-is | `||>` | 3+ stages, no I/O, one-character-per-stage cost matters | `~>` | Last-arg-is-the-data style (Perl-tradition `map BLOCK LIST`) | `->>` / `~>>` | I/O-heavy stages with uneven latency, want pipeline parallelism | `~s>` | CPU-heavy stages on a big collection, want chunk parallelism | `~p>` |

7.7 Stage notation forms

Inside any `~>` / `~>>` / `~s>` / `~p>` stage, four notations are accepted:

```

~> $x uc           # bare fn: x → uc(x)
~> $x div(_, 2)    # placeholder: x → div(x, 2)
~> $x >{ _ * 2 + 1 } # arrow-block: x → (x*2+1)
~> @xs map { _ * 10 } sum # block-arg form: map { ... } @xs → sum(...)

```

Mix freely. The placeholder `_` is always available and overrides the implicit position when present.

8. Iteration shorthand

| DO | DON'T | Why | `||>` | `||>` | `@xs \> e p` | for my `$x (@xs) { p $x }` | `e` is each-call; `e p` = call `p` on each element. | `@xs \> ep` | `@xs \> e p` (when chained from a pipeline) | `ep` is shorthand for `e { p }` — each-then-print. **Equivalent to `e p`, both go to stdout.** | `pmaps { ... } @xs` | `[pmap { ... } @xs]` then iterate | `pmaps` is the streaming parallel iter — lazy, no intermediate vec. | `pgreps { ... } @xs` | `[pgrep { ... } @xs]` | Streaming parallel filter; same reason. | `flat_maps`, `pflat_maps` | `flat_map { ... } \> e` | Stream variant; no list materialization. |

Streaming variants (`pmaps`, `pgreps`, `pflat_maps`, `maps`, `greps`, `flat_maps`) are the default when chaining; the array-returning forms (`pmap`, `pgrep`, `flat_map`) are only correct when you genuinely need a vec.

9. File I/O

| DO | DON'T | Why | `||>` | `||>` | `c"path" | slurp("path")`, do { open my \$fh, "<", "path"; local \$/; <\$fh> } | `c"..."` is slurp-as-quote-prefix. Two extra chars over the path itself. | `spurt $path, $data` | open my \$fh, ">", \$path; print \$fh \$data; close \$fh | One builtin call. | `lines("path")` | open ... while <\$fh> | Lazy line iterator. | `glob("*.stk")` | `<*.stk>` | `<>` is reserved for stdin; use `glob` explicitly. |

10. Parallelism

| DO | DON'T | Why | `||>` | `||>` | `pmaps { f(_) } @xs` | spawn threads manually | `pmaps` rides rayon work-stealing; correct + faster. | `varsync $count = 0` | shared mutex by hand | `varsync` and `oursync` are the stryke-shared-state primitive; lockless reads, lock-on-write. | `pchannel` | hand-rolled mpvc | Built-in unbounded channel with select. | `barrier($n)` | hand-rolled CountdownLatch | One call. |

When calling parallel operations from `pmaps` workers, prefer per-thread idioms (`set_no_interop_mode_tls`) over global atomics — sibling workers don't race.

11. Sort / reduce blocks

`$a` and `$b` are **only valid inside** sort/reduce comparator blocks (the Perl-ism). Don't reference them anywhere else, and don't shadow them with `my $a / my $b` in regular code.

| DO | DON'T | Why | |---|---|---| | `sort { _0 cmp _1 } @xs` | `sort { $a cmp $b } @xs` | `_0/_1` is the universal positional protocol; `$a/$b` is comparator-block-only magic. | | `reduce { _0 + _1 } @xs` | `reduce { $a + $b } @xs` | Same. |

stryke accepts the `$a/$b` form for Perl 5 compat but `--no-interop` rejects it.

12. Implicit closure params

stryke's calling card. Never write named params for one-liner closures. Three syntactic homes, each with its own bracket form — pick by where the closure lives.

Block form `{ ... }` — for `map` / `grep` / `sort` / `reduce` and `|>` pipelines

The plain block is what list builtins accept. Use it when the closure is the block-arg of a builtin or any stage of a pipe-forward `|>` chain.

| DO | DON'T | Why | |---|---|---| | `map { _ * 2 } @xs` | `map { $_ * 2 } @xs` | `_` is the bare topic; `$_` is the Perl-ism (rejected by `--no-interop`). | | `grep { _ > 5 } @xs` | `grep { $_ > 5 } @xs` | Same. | | `sort { _0 <=> _1 } @xs` | `sort { $a <=> $b } @xs` | Universal positional, not sort-block magic. | | `@xs \> maps { _ * 2 }` | `@xs \> maps >{ _ * 2 }` | `| \> uses plain { }, not >{ }. >{ }` is reserved for `~>` thread macros. |

Arrow-block `>{ ... }` — only inside `~>` / `~>>` / `~s>` / `~p>` thread macros

`>{ }` is a syntactic primitive *of* the thread-macro family — it marks an anonymous-fn stage in `~>` chains. Outside threading it's a parse error ("Not a code reference").

| DO | DON'T | Why | |---|---|---| | `~> @xs >{ _ * 2 } sum` | `~> @xs { _ * 2 } sum` | `~>` stages need either `name`, `name(args)`, `>{ ... }`, or `name { ... }`. A bare `{ }` doesn't parse as a stage. | | `~> @xs map { _ * 2 } sum` | always-`>{ }` style | `map { ... }` is the block-arg stage form, equally valid in `~>`. Pick `>{ }` for standalone closures, `name { ... }` when calling a builtin. | | `val $f = fn { _ * 2 }` | `my $f = >{ _ * 2 }` | **Arrow-block is not a value.** Bare `>{ }` outside threading errors with "Not a code reference". |

`fn { ... }` / `fn name = ...` — anonymous and named function values

For storing a closure in a variable or returning one, use `fn`. Anonymous `fn { ... }` accepts the topic protocol exactly like the block forms above.

| DO | DON'T | Why | |---|---|---| | `val $f = fn { _ * 2 }` | `my $f = sub { $_[0] * 2 }` | `fn` opens the topic-var protocol; `sub` doesn't. | | `val $f = fn { _ * 2 }` | `my $f = >{ _ * 2 }` | `>{ }` is threading-only — error outside `~>`. | | `fn double = _ * 2` (top-level) | `fn double { return _ * 2 }` | Expression-bodied named fn (rule 6) + implicit return (rule 7). |

Outer chain `_<` — one `<` per closure level

`_<` reaches the topic of the *enclosing* closure; chain `<s` for deeper. Most useful in nested `~>` where each stage opens a fresh topic frame.

```
# Multiply each element by its row index across nested rows
~> @rows enumerate map { ~> _0 map { _ * _< } } # _< = outer-outer topic

# [capture-style: explicit `my` works but is the verbose form
my $outer = ...
@xs |> maps { _ + $outer } # explicit lexical capture
@xs |> maps { _ + _< } # `_<` reads cleaner once internalized
```

Quick reference — which bracket form?

| Context | Form | |---|---| | `map/grep/sort/reduce` block-arg | `{ ... }` | | Stage in `\>` pipeline (after a builtin that takes a block) | `{ ... }` | | Stage in `~>` / `~>>` / `~s>` / `~p>` thread macro | `{ ... }` (block-arg) or `>{ ... }` (arrow-block) | | Stored in a variable (anon fn) | `fn { ... }` | | Top-level named function | `fn name = ...` (one-liner) or `fn name { ... }` (multi-stmt) |

12b. No semicolons at end of line

stryke statement-terminates on newline. Trailing `;` is noise that adds a character per line and visually weights every statement equally with no payoff.

| DO | DON'T | Why | |---|---|---| | `val $x = 1` | `my $x = 1`; | Newline already terminates; `val` over bare `my` (rule 10). | | `p $x` | `p $x`; | Same. | | `for val $i (1:10) { p $i }` | `for my $i (1..10) { p $i; }` | Both sides; range too. |

The only legitimate `;` is separating statements on the same line — almost always inside a C-style `for` header:

```
for (my $i = 0; $i < 10; $i++) { ... }
```

Or, very rarely, a one-liner with two statements that genuinely belong glued:

```
$count++; last if $count > 100      # acceptable
```

Even that is usually wrong — split to two lines unless brevity is the actual goal. `use strict / --no-interop` will not flag trailing `;` (legal Perl 5), but new stryke code that ships with them is wrong on style and will fail review.

13. Multi-line expressions

stryke terminates statements at newline (Perl/shell-like), not at `;`. Multi-line operators silently produce wrong values without explicit continuation.

| DO | DON'T | Why | |---|---|---| | `$x = a()` | `$x = a()
b()
c()` | use `\
` at the end of each line | Pipelines auto-continue across newlines when starting with `\>` or `~>`. | | `$x = $a +` | `$x = $a +
$b
* $c` | `$x = $a + $b * $c` (one line if it fits) | Use `\` for binary-op-mid-expression continuation. |

If a pipeline starts a fresh line with `\>` or `~>`, the parser auto-extends the previous statement. Otherwise newlines terminate.

14. Hash declarations

For hashes with more than 3 keys, one `key => value` pair per line. Makes block editing (visual select, delete, reorder) cheap.

```
my %config = (  
  host    => "localhost",  
  port    => 8080,  
  timeout => 30,  
  retries => 3,  
)
```

Short hashes (≤ 3 keys) can stay on one line:

```
my %sm = (a => 1, b => 2)
```

15. No defensive code

| DO | DON'T | Why | |---|---|---| | `val $sum = sum(@xs)` | `my $sum = @xs ? sum(@xs) : 0` | Trust the builtin's empty-list contract. | | `die "bad input" if $n < 0` | check + return undef + caller checks for undef | `die` is cheap; failure paths are explicit. | | direct call | guard with `if defined $obj && ref $obj && $obj->can('method')` | Trust internal code. Validate at boundaries (user input, network), nowhere else. |

Don't add `my $foo` declarations inside loops just to be defensive — declare once at function top, assign inside the loop. (zsh-rule, but applies to stryke too: re-declaring `my` in a loop is wasteful and may surface as warnings under future strictness.)

16. Test runner

stryke's `test()` builtin and `s t` CLI default to the **worker pool** — pre-forked persistent stryke processes that fork-on-receive per test. Don't roll your own.

| DO | DON'T | Why | |---|---|---| | `s t t/` | `for f in t/*.stk; do stryke $f; done` | Pool reuses warm stryke; ~5–7× faster than the shell loop. | | `test("t/")` | from a stryke script | `system "stryke test t/"` | Builtin avoids the outer fork. | | `test_no_interop("t/")` | `system "stryke --no-interop test t/"` | Per-thread TLS no-interop, race-free. | | `s t -j 1 t/foo.stk` | to debug | guess-and-check at full parallelism | Serial mode pinpoints which file hangs. |

See [Test Runner — Worker Pool Architecture](./README.md#0x0c-test-test-runner--worker-pool-architecture) for the topology and wire protocol.

17. Strict / no-interop modes

stryke does **not** enforce `use strict`; by default — that's a Perl-ism. Topic vars (`_N`, `$_N`) and special vars (`@ARGV`, `%ENV`, ...) are **always** exempt regardless.

- `use strict`; — opt-in Perl-style undefined-var enforcement. Use only in modules you're porting from CPAN.
- `--no-interop` — bot firewall. Rejects every entry in this guide's DON'T column at parse time. Use in CI for new stryke code.
- `--compat` — locks stryke to stock Perl 5 syntax/builtins. Use for parity testing only.

18. Naming

| Convention | Where | Example | |---|---|---| | `snake_case` | function names, variables | `fn parse_config, $line_count` | | `Module::Sub::name` | namespaced functions | `fn Rosetta::Strings::reverse_words` | | `SCREAMING_SNAKE` | true constants | `const MAX_RETRIES = 3` | | `CamelCase` | class / struct / enum names | `class HttpClient, enum Color` | | short single-letter | only as block topic / loop index | `for $i (1:10), { _ }` |

Never name a function after a builtin. Always namespace if there's any chance of clash. Run `s 'p $stryke::all{your_name} // "free"'` to check.

19. When to break the rules

Three legitimate reasons:

1. **Porting CPAN code where line-by-line parity matters** — keep the original spelling, add `# perl-port` comment, don't mix idioms.
2. **Code that already exists** — don't refactor neighboring style mid-PR. Match the file's existing convention; queue the migration as a separate change.
3. **Performance critical paths verified by benchmark** — if `<<` resolution shows up hot in a profile, switching to explicit lexical capture is fine. Rarely needed; profile first.

Anywhere else: stryke idioms win. The DON'T column is for compat, not for new code.

Parallel Primitives

20 topics

pmap

Parallel `map` powered by rayon's work-stealing thread pool. Every element of the input list is processed concurrently across all available CPU cores, and the output order is guaranteed to match the input order. This is the primary workhorse for CPU-bound transforms in stryke — use it whenever you have a pure function and a large list. Pass `progress => 1` to get a live progress bar on STDERR for long-running jobs.

Two equivalent surface syntaxes: • Block form — `pmap BLOCK LIST` — element bound to `_` • Bare-fn form — `pmap FUNC, LIST` — single-arg function name as first argument

```
# Block form
var @out = pmap { _ * 2 } 1:1_000_000
var @hashes = pmap sha256 @blobs, progress => 1
1:100 |> pmap { fetch("https://api.example.com/item/${_}") } |> e p

# Bare-fn form (works for builtins and user-defined subs)
var @hashes = pmap sha256, @blobs, progress => 1
fn dbl = _ * 2
var @r = pmap dbl, (1:1_000_000)
```

pmaps

Streaming parallel `map` — returns a lazy iterator that processes items across all CPU cores using a persistent worker-thread pool. Unlike `pmap` which eagerly collects all results into an array, `pmaps` yields results as they complete through a bounded channel, so downstream consumers see output immediately. Output order is non-deterministic (completion order). Each worker thread reuses a single interpreter instance, making it faster than `pmap` for large inputs.

Best for: • Pipelines with `take/head` — avoids processing the full list • Very large inputs where holding all results in memory is impractical • Streaming pipelines where you want progressive output

```
range(0, 1e9) |> pmaps { _ * 2 } |> take 10 |> ep
t 1:1e6 pmaps { expensive(_) } ep
range(0, 1e6) |> pmaps { fetch("https://api/item/${_}") } |> pgreps { _->{ok} } |> ep
```

pmap_chunked

Parallel map that groups input into contiguous batches of N elements before distributing to threads. This reduces per-item scheduling overhead when the per-element work is very cheap (e.g. a few arithmetic ops). Each thread receives a slice of N consecutive items, processes them sequentially within the batch, then returns the batch result. Use this instead of `pmap` when profiling shows rayon overhead dominates the actual computation.

```
val @out = pmap_chunked 100, { _ ** 2 } 1:1_000_000
var @parsed = pmap_chunked 50, { json_decode } @json_strings
```

pgrep

Parallel `grep` that evaluates the filter predicate concurrently across all CPU cores using rayon. The result preserves the original input order, so it is a drop-in replacement for `grep` on large lists. Best suited for predicates that do meaningful work per element — if the predicate is trivial (e.g. a single regex on short strings), sequential `grep` may be faster due to lower scheduling overhead.

Two equivalent surface syntaxes: `pgrep { BLOCK } LIST` or `pgrep FUNC, LIST`.

```
# Block form
var @matches = pgrep { /complex_pattern/ } @big_list
var @primes = pgrep { is_prime } 2:1_000_000
@files |> pgrep { -s _ > 1024 } |> e p

# Bare-fn form
fn Demo::even = _ % 2 == 0
var @e = pgrep Demo::even, 1:10 # (2,4,6,8,10)
```

pgreps

Streaming parallel `grep` — returns a lazy iterator that filters items across all CPU cores. Unlike `pgrep` which eagerly collects all matching items, `pgreps` yields matches as they are found through a bounded channel. Output order is non-deterministic (completion order). Each worker thread reuses a single interpreter instance.

```
range(0, 1e6) |> pgreps { is_prime(_) } |> take 100 |> ep
t 1:1e9 pgreps { _ % 7 == 0 } take 10 ep
```

pfor

Parallel `foreach` that executes a side-effecting block across all CPU cores with no return value. `pfor` and `pforeach` are interchangeable keywords, same as `for/foreach`. Use this when you need to perform work for each element (writing files, sending requests, updating shared state) but don't need to collect results. The block receives each element as `_`. Iteration order is non-deterministic, so the block must be safe to run concurrently.

Two equivalent surface syntaxes: `pfor { BLOCK } LIST` or `pfor FUNC, LIST`.

```
# Block form
pfor { write_report } @records
pfor { compress_file } glob("*.log")
@urls |> pfor { fetch
  p "done: $_" }

# Bare-fn form
fn task = ep "did $_"
pfor task, (1, 2, 3)
```

ploop

Parallel `loop` — runs the body as an infinite loop on each of N rayon workers simultaneously (default: `thread_count()`). Desugared at parse time to `pfor { while (1) { BODY } } 1..N`, so there is no separate opcode. Inside the body `_` is the 1-based worker id and `last` exits *that worker's* loop; the construct finishes when every worker has exited. Use for worker-pool patterns: each worker pulls jobs from a shared queue until it drains.

```
# one worker per core, each drains a shared channel
ploop {
  val $job = recv_or_undef($ch)
  last unless defined $job
  handle($job)
}

# explicit worker count
ploop 4 {
  val $task = next_task()
  last unless defined $task
  run($task)
}
```

See also: `loop`, `pwhile`, `pfor`, `pchannel`.

pwhile

Parallel `while` — each of N rayon workers (default: `thread_count()`) runs `while (COND) { BODY }`, re-evaluating COND per worker per iteration. Desugared at parse time to `pfor { while (COND) { BODY } } 1..N` — no separate opcode.

Inside the body `_` is the 1-based worker id; a shared `varsync` flag or a draining queue in `COND` stops all workers, and `last` exits one worker early. The construct finishes when every worker's condition has gone false.

```
# all cores chew until the shared queue drains
mysync @queue = load_jobs()
pwhile (len(@queue)) {
  val $job = pop @queue
  handle($job) if defined $job
}

# explicit worker count with a shared stop flag
mysync $run = 1
pwhile 4 ($run) {
  $run = 0 if poll_shutdown()
  step()
}
```

See also: `while`, `ploop`, `pfor`, `varsync`.

psort

Parallel sort that uses rayon's parallel merge-sort algorithm. Accepts an optional comparator block; the two sort candidates are `_` and `_1`. For large lists (10k+ elements), this significantly outperforms the sequential `sort` by splitting the array, sorting partitions in parallel, and merging. The sort is stable — equal elements retain their relative order.

```
val @sorted = psort { _ <=> _1 } @big_list
var @by_name = psort { (_,_)>{name} cmp (_1)->{name} } @records
@nums |> psort { _ <=> _1 } |> e p
```

pcache

Parallel memoized map — each element is processed concurrently, but results are cached by the stringified value of `_` so duplicate inputs are computed only once. This is ideal when your input list contains many repeated values and the computation is expensive. The cache is a concurrent hash map shared across all threads, so there is no lock contention on reads after the first computation.

Two equivalent surface syntaxes: `pcache { BLOCK } LIST` or `pcache FUNC, LIST`.

```
# Block form
var @out = pcache { expensive_lookup } @list_with_dupes
var @resolved = pcache { dns_resolve } @hostnames

# Bare-fn form
var @resolved = pcache dns_resolve, @hostnames
```

preduce

Parallel tree-fold using rayon's `reduce` — splits the list into chunks, reduces each chunk independently, then merges partial results. The combining operation **must be associative** (e.g. `+`, `*`, `max`); non-associative ops will produce incorrect results. Much faster than sequential `reduce` on large numeric lists because the tree structure allows $O(\log n)$ merge depth across cores.

```
val $total = preduce { _ + _1 } @nums
var $biggest = preduce { _ > _1 ? _ : _1 } @vals
var $product = preduce { _ * _1 } 1:100
```

preduce_init

Parallel tree-fold with an explicit identity element. Same engine as `preduce` (rayon's `reduce_with`), but `INIT` seeds every chunk so empty / single-element inputs return a sensible value instead of `undef`. The combining operation **must be associative AND INIT** must be its identity (`INIT  $\oplus$  x == x`); otherwise different chunk boundaries produce different sums.

```

val $total = preduce_init 0, { _ + _1 } @nums # sum, identity 0
var $product = preduce_init 1, { _ * _1 } @nums # product, identity 1
var $biggest = preduce_init -inf, { _ > _1 ? _ : _1 } @vals

# Histogram merge - identity is the empty hash.
var %h = %{ preduce_init +{ }, { merge_hash(_, _1) } @per_chunk_hashes }

```

Without an identity, prefer `preduce` (no init) or `pmap_reduce` (fused mapper + reducer).

See also: `preduce`, `pmap_reduce`, `reduce` (sequential), `fold` (sequential with init).

pmap_reduce

Fused parallel map + tree reduce. Each input element is mapped in parallel via rayon; results are combined pairwise by the reducer block. The reducer must be associative ($(a \oplus b) \oplus c == a \oplus (b \oplus c)$) — order of pairwise combines is not fixed.

```

# Two-block form: pmap_reduce { MAPPER } { REDUCER } LIST
var $sum = pmap_reduce { _ * 2 } { _ + _1 } @list

# Chunked aggregation - mapper returns a per-chunk arrayref, reducer merges.
var $grads = @batches |> pmap_reduce {
  val $idxs =
    [ batch_grad($weights, $bias, $XS, $YS, $idxs), len(@$idxs) ]
} {
  [ _->[0] + _1->[0], _->[1] + _1->[1] ]
}

```

Caveats:

- **rand() in the mapper is parallel-correlated.** Per-task `rand()` calls across workers produce statistically correlated values; prefer chunked Monte-Carlo where each task runs its own inner loop.

See also: `pmap` (eager parallel map), `pmaps` (streaming), `par_reduce` (verbose builder).

pany

`pany { COND } @list` — parallel short-circuit **any**. Splits the list across cores via rayon, evaluates `COND` on each element, returns truthy as soon as **any** chunk reports a hit and cancels the rest. Returns the matching element (truthy) or `0` (no match).

```

# Found in O(N/cores), often much faster - early-stops once any worker hits.
if (pany { _ > 1_000_000 } @samples) { p "found a large sample" }
pany { is_prime(_) } 10**9 : 10**9 + 50 # "is there a prime in this range?"

```

See also: `pall` (parallel **all**), `pfirst` (return the matching value), **any** (sequential), `pgrep` (full filter).

pfirst

`pfirst { COND } @list` — parallel **first**. Like `pany` but returns the **matching element** rather than just a truthy flag. Workers run concurrently across chunks; the first hit (by *completion order*, not input order) wins and the remaining workers are cancelled. Returns `undef` if no element matches.

```

val $hit = pfirst { _->{status} eq "FAILED" } @records
var $prime = pfirst { is_prime(_) } 10**12 : 10**12 + 1000

```

Note: output is non-deterministic when multiple elements match — workers race. For first-by-input-order use sequential `first` or `pfirst_in_order` (where supported).

See also: `pany`, `first`, `pgrep`.

puniq

`puniq @list` — parallel deduplication. Returns the unique elements of `@list`, with the original input order preserved (first occurrence wins). Elements are keyed by stringification ("`$_"`"), so `1` and `"1"` deduplicate together. Backed by a concurrent hash set.

```
val @uniq    = puniq @log_lines
var @hosts   = puniq map { _->{host} } @records
var @hashes  = puniq pmap sha256, @files      # parallel hash + dedup
```

For exact-key (no stringification) dedup, build the set manually with `varsync %seen + pmaps`. For **count** of unique items use `hll_count` (HyperLogLog) which is O(1) memory.

See also: `uniq` (sequential), `dedup`, `hll_count`.

pflat_map

Parallel flat-map: map + flatten results. Each element produces zero or more output values via the block/function, and the outputs are concatenated in input order.

Two equivalent surface syntaxes: `pflat_map BLOCK LIST` or `pflat_map FUNC, LIST`.

```
# Block form
var @out = pflat_map expand @list

# Bare-fn form
fn expand = (_, _ * 10)
var @r = pflat_map expand, (1, 2, 3)      # (1, 10, 2, 20, 3, 30)
```

pflat_maps

Streaming parallel flat-map. Like `pmaps` but each block iteration may emit zero or more values, all of which are spliced into the downstream iterator (no intermediate vec). Runs on the persistent worker-thread pool, output order is non-deterministic, and downstream `take/head` cancels remaining work.

```
# One-to-many expansion in parallel
range(0, 1e6) |> pflat_maps { (_, _ * 10) } |> ep

# Drop and explode: yield only odd-derived pairs
1:1_000_000 |> pflat_maps { _ % 2 ? (_, _ * _) : () } |> take 20 |> ep

# Tokenize a big corpus in parallel
glob("*.txt") |> pflat_maps { slurp(_) |> split /\s+/ } |> pgreps { len _ } |> ep
```

For eager flat-map use `pflat_map` (returns a vec). For sequential lazy flat-map use `flat_maps`.

See also: `pmaps`, `pflat_map`, `flat_maps`, `flat_map`.

fan

Execute BLOCK or FUNC N times in parallel (`_` = index 0..N-1). With no count, defaults to the rayon pool size (`stryke -j`).

Two equivalent surface syntaxes: • Block form — `fan N { BLOCK }` or `fan { BLOCK }` • Bare-fn form — `fan N, FUNC` or `fan FUNC`

```
# Block form
fan 8 { work(_) }
fan { work(_) } progress => 1

# Bare-fn form
fn tick = ep "tick $_"
fan 8, tick
fan tick, progress => 1      # uses pool size
```

fan_cap

Like `fan` but captures return values in index order. Two surface syntaxes: `fan_cap N { BLOCK }` or `fan_cap N, FUNC`.

Shared State & Concurrency

12 topics

mysync

Declare shared variables for parallel blocks (`Arc<Mutex>`). `varsync` is the canonical spelling — it matches `var/val` replacing `my`. `mysync` is the legacy `my`-prefixed alias, kept for compatibility. Both parse to the same `StmtKind::MySync` AST node and produce byte-identical bytecode; prefer `varsync` in new code.

```
varsync $counter = 0      # canonical spelling
mysync  $counter = 0      # legacy alias - identical semantics
fan 10000 { $counter++ }  # always exactly 10000
varsync @results
varsync %histogram
```

Compound ops (`++`, `+=`, `-=`, `|=`, `&=`) are fully atomic on shared scalars. Both spellings reflect under `%k` with category `concurrency`.

See also: `oursync` (package-global counterpart), `var` / `val` (the `my` aliases that established the convention).

oursync

Package-global counterpart of `varsync` / `mysync`. Same `Arc<Mutex>` backing, but the binding is keyed by the package-qualified stash name (`Pkg::x`) so all packages and parallel workers share one cell.

```
package Counter
oursync $total = 0
fn bump { $total++ }
package main
fan 10000 { Counter::bump() }
p $Counter::total # 10000
```

Use `oursync` for cross-package shared state; use `varsync` for lexically scoped shared state.

async

`async { BLOCK }` — schedule a block for execution on a background worker thread and return a task handle immediately.

The block begins executing as soon as a thread is available in stryke's global rayon thread pool, while the calling code continues without blocking. To retrieve the result, pass the task handle to `await`, which blocks until the task completes and returns its value. If the block panics, the panic is captured and re-raised at the `await` call site. Use `async` for fire-and-forget background work, overlapping I/O with computation, or launching multiple independent tasks that you later join. For structured fan-out with index-based parallelism, prefer `fan` or `fan_cap` instead.

```
val $task = async { long_compute() }
do_other_work()
var $val = await $task

# Overlapping multiple fetches:
var @tasks = map { async { fetch("https://api.example.com/$_") } } 1:10;
var @results = map { await _ } @tasks
```

spawn

`spawn { BLOCK }` — Rust-style alias for `async`; schedules a block on a background thread and returns a joinable task handle.

This is identical to `async` in every respect — same thread pool, same semantics, same `await` for joining. The name exists for developers coming from Rust's `tokio::spawn` or `std::thread::spawn` who find `spawn` more natural. Use whichever reads better in your code; mixing `async` and `spawn` in the same program is perfectly fine since they share the same underlying pool.

```
val $task = spawn { expensive_io() }
var $val = await $task

var @handles = map { spawn { process } } @items;
var @out = map { await _ } @handles
```

await

`await $task` — block the current thread until an `async/spawn` task completes and return its result value.

If the background task has already finished by the time `await` is called, the result is returned immediately with no scheduling overhead. If the task panicked, `await` re-raises the panic as a die in the calling thread, preserving the original error message and backtrace. You can `await` a task exactly once; calling `await` on an already-joined handle is a fatal error. For waiting on multiple tasks, simply map over the handles — stryke does not yet provide a `join_all` primitive, but `map await @tasks` achieves the same effect.

```
val $task = async { 42 }
var $result = await $task           # 42

# Await with error handling:
var $t = spawn { die "oops" }
eval { await $t }                  # $@ eq "oops"
```

pchannel

`pchannel(N)` — create a bounded multi-producer multi-consumer (MPMC) channel with capacity N, returning a `($tx, $rx)` pair.

The transmitter `$tx` supports `->send($val)` which blocks if the channel is full (backpressure). The receiver `$rx` supports `->recv` which blocks until a value is available, and `->try_recv` which returns `undef` immediately if the channel is empty. Both ends can be cloned and shared across threads — clone `$tx` with `$tx->clone` to create additional producers, or clone `$rx` for additional consumers. When all transmitters are dropped, `->recv` on the receiver returns `undef` to signal completion. Use `pchannel` to build producer-consumer pipelines, rate-limited work queues, or to communicate between `async/spawn` tasks.

```
var ($tx, $rx) = pchannel(100)
async { $tx->send(_) for 1:1000
  undef $tx }
while (defined(val $val = $rx->recv)) {
  p $val
}

# Multiple producers:
var ($tx, $rx) = pchannel(50)
for val $i (1:4) {
  val $t = $tx->clone
  spawn { $t->send("from worker $i: _") for 1:100 }
}
undef $tx # drop original so channel closes when workers finish
```

pselect

`pselect(@receivers) / pselect(timeout_ms => N, @receivers)` — wait on multiple `pchannel` receivers; returns `($index, $value)` for the first receiver to produce a value. Equivalent to Go's `select { case v := <-ch1: ... case v := <-ch2: ... }`.

The optional `timeout_ms` clause returns `(-1, undef)` if no receiver delivers within the deadline. Closed channels (all senders dropped) return `(i, undef)` so the caller can detect end-of-stream per receiver.

```

var ($tx_a, $rx_a) = pchannel(8)
var ($tx_b, $rx_b) = pchannel(8)
spawn { $tx_a->send($_) for 1:100 }
spawn { $tx_b->send($_ * 10) for 1:100 }
while (1) {
  var ($idx, $val) = pselect $rx_a, $rx_b
  last unless defined $val
  p "from # $idx: $val"
}

# With deadline:
var ($idx, $val) = pselect timeout_ms => 500, $rx_a, $rx_b

```

See also: `pchannel`, `spawn`, `async`, `barrier`.

barrier

`barrier(N)` — create a synchronization barrier that blocks until exactly N threads have arrived at the wait point.

Each thread calls `$bar->wait` and is suspended until all N participants have reached the barrier, at which point all are released simultaneously. This is useful for coordinating phased parallel algorithms where all workers must complete step K before any worker begins step K+1. The barrier is reusable — after all threads are released, it resets and can be waited on again. Internally backed by a Rust `std::sync::Barrier` for zero-overhead synchronization.

```

val $bar = barrier(4)
for val $i (0:3) {
  spawn {
    setup_phase($i)
    $bar->wait           # all 4 threads sync here
    compute_phase($i)
  }
}

```

ppool

`ppool(N, fn { ... })` — create a persistent thread pool of N worker threads, each running the provided subroutine in a loop.

The pool is typically paired with a `pchannel`: workers pull items from the channel's receiver, process them, and optionally send results to an output channel. Unlike `pmap` which is a one-shot parallel transform, `ppool` keeps threads alive for the lifetime of the pool, making it ideal for long-running server-style workloads, background drain loops, or scenarios where thread startup cost would dominate short-lived `pmap` calls. Workers exit when their input channel is closed (all transmitters dropped). The pool object supports `->join` to block until all workers have finished.

```

var ($tx, $rx) = pchannel(100)
var $pool = ppool 4, fn {
  while (defined(val $job = $rx->recv)) {
    process($job)
  }
}
$tx->send(_) for @work_items
undef $tx           # signal completion
$pool->join          # wait for drain

```

deque

`deque LIST` — create a double-ended queue initialized with the given elements.

A deque supports efficient O(1) insertion and removal at both ends via `->push_front($val)`, `->push_back($val)`, `->pop_front`, and `->pop_back`. It also supports `->len` and iteration. Internally backed by a Rust `VecDeque`, it is ideal for sliding window algorithms, BFS traversals, or any scenario where you need fast access to both ends of a sequence.

```

val $dq = deque(1, 2, 3)
$dq->push_front(0)
$dq->push_back(4)
p $dq->pop_front      # 0

```

heap

heap LIST — create a min-heap (priority queue) from the given elements.

Elements are heapified on construction so that `->pop` always returns the smallest element in $O(\log n)$ time. `->push($val)` inserts a new element, also in $O(\log n)$. The heap supports `->peek` to inspect the minimum without removing it, and `->len` for the current size. Internally backed by a Rust [BinaryHeap](#) (inverted for min-heap semantics), it is the go-to structure for top-K queries, Dijkstra, and merge-K-sorted-lists problems.

```
val $h = heap(5, 3, 8, 1)
p $h->pop           # 1 (smallest first)
p $h->peek          # 3 (next smallest)
$h->push(0)
```

set

set LIST — create a set (unique unordered collection) from the given elements.

Duplicate values are collapsed on construction so the set contains each value exactly once. The set object provides `->contains($val)` for $O(1)$ membership testing, plus `->union($s)`, `->intersection($s)`, `->difference($s)`, and `->len` methods. Internally backed by a Rust [HashSet](#) for performance. Use `to_set` to convert an existing iterator into a set.

```
val $s = set(1, 2, 3, 2, 1)
p $s->contains(2)      # 1
p $s->len              # 3
var $both = $s->union(set(3, 4, 5))
```

Pipeline & Pipe-Forward

13 topics

|>

Pipe-forward operator — threads LHS as first argument of RHS call.

```
"hello" |> uc |> rev |> p # OLLEH
1:10 |> grep _> 5 |> map _ * 2 |> e p
$url |> fetch_json |> json jq '.name' |> p
"hello world" |> s/world/perl/ |> p # hello perl
```

Zero runtime cost (parse-time desugaring). Binds looser than `||`, tighter than `?:`.

thread

Thread-first macro — chain stages where the threaded value is injected as the **first** argument (like Clojure's `->`).

```
-> @data grep { _> 5 } map { _ * 2 } sort { _ <=> _1 } |> join ", " |> p
-> "hello" tm uc rv lc ufc sc cc kc tj p # short aliases
fn div = _0 / _1
-> 10 div(2) p # div(10, 2) = 5
-> 10 div(_, 2) p # explicit _ position: div(10, 2) = 5
```

Spellings: `->`, `thread`, `t`. Stages: bare function (`uc`, `tm`, ...), function with block (`map { ... }`, `grep { ... }`), `name(args)` call where `_` is the threaded-value placeholder, or `>{ }` anonymous block. `|>` terminates the macro.

~>>

Thread-last macro — chain stages where the threaded value is injected as the **last** argument (like Clojure's `->>`).

```
fn div = _0 / _1
->> 10 div(2) p # div(2, 10) = 0.2
->> 10 div(_, 2) p # explicit _ overrides: div(10, 2) = 5

# Thread-last is useful for list-consuming functions:
fn first_n { @_[0..$_[-1]-1] } # first_n(list, n)
->> (1,2,3,4,5) first_n(3) p # first_n(1,2,3,4,5, 3) = (1,2,3)
```

Spellings: `~>>`, `->>`. The explicit `_` placeholder always takes precedence.

~s>

Per-item streaming thread macro — each stage runs as an independent worker thread connected to the next via a bounded channel, so items flow through the pipeline concurrently without materializing intermediate arrays.

```
~s> [1, 2, 3, 4, 5] map { _ * 10 } map { _ + 1 }
~s> @lines map { json_decode } grep { _->{ok} } map { _->{val} }
~s> gen { yield_lines "huge.log" } map { parse } grep { /ERROR/ }
```

Use `~s>` when stages have differing latencies (I/O + CPU mix) so faster workers can pull ahead. Output order is non-deterministic. The macro's final value is the count of items the last stage emitted. `~s>>` is the thread-last variant.

See `~p>` for chunk-parallel (vs per-item) execution and `par_pipeline` for the verbose builder form.

`~p>`

Chunk-parallel thread macro — splits the input into chunks, runs the entire pipeline on each chunk in parallel via rayon, and auto-merges chunk results. Equivalent to `par_reduce { stage1 |> stage2 |> ... } SRC`.

```
~p> c("**.rs") letters freq # histogram across all .rs
~p> @big map { _ * 2 } sum # parallel map+sum
~p> $huge letters freq |> values |> sum # boundary: terse
~p> $huge letters freq |then| values |> sum # boundary: english
```

The auto-merger picks a strategy by inspecting the first chunk's result type: hash-of-numbers merges key-wise with `+` (canonical histogram), numeric scalars sum, arrays concat, strings concat. Use `|>` or `|then|` to switch back to sequential `->` mode for stages that operate on the merged whole. `|>` also terminates a `~p>` macro. `~p>>` is the thread-last variant.

`~d>`

Distributed thread macro — same chunk-block semantics as `~p>` but ships the pipeline to a remote stryke cluster rather than running locally. Syntax: `~d> on $cluster SOURCE stage1 stage2 ....` The `on EXPR` slot is mandatory — `EXPR` must evaluate to a value built by `cluster(...)` (see `cluster` builtin).

```
val $c = cluster(["host1", "host2", "host3:8022"])
~d> on $c c("**.parquet") map { parse_pq } map { transform } collect
~d> on $c @huge_urls map { fetch } grep { _->{status} == 200 } |> sum_bytes
```

Unlike `~p>`, captures from the enclosing scope are restricted: `varsync`/atomic captures are rejected (the value can't cross the wire safely). Errors on remote workers are surfaced as a single aggregated error. `~d>>` is the thread-last variant — threaded value injected as the last argument of each stage.

See `cluster` for cluster construction, `~p>` for local chunk-parallel, `pmap_on` for the verbose builder form.

par

Generic parallel-chunk evaluator. Splits the input list into chunks, runs `BLOCK` on each chunk in parallel with the chunk bound to `_` / `_0`, and flattens chunk outputs into a single list. Callable in three shapes:

```
# 1. Top-level prefix form - yields a flat list
par { BLOCK } LIST
var @r = par { letters } "Hello, World 123" # H e l l o W o r l d
var @s = par { map { _ ** 2 } _ } @huge # parallel squares

# 2. Postfix-statement form - side-effecting, no return binding
{ BLOCK } par LIST # runs BLOCK per chunk
{ log_chunk(_) } par @huge

# 3. As a `~>` / `~>>` thread-macro stage
~> "abc def" par { letters } # a b c d e f
~> @huge par { map { _ ** 2 } |> sum } |> sum p # parallel sum-of-squares
```

Each chunk gets a fresh sub-VM with the chunk bound to `_` and `_0`. List-returning ops keep their values; the runtime auto-flattens. Use `par_reduce` when you need an explicit merge step rather than flat concat.

par_reduce

Chunk-extract-merge thread-stage. The single-block form runs `extract` per chunk and auto-merges results by type (hash<num>→key-wise add, number→sum, array→concat, string→concat). The two-block form binds the two accumulators as `_1` in the explicit reducer.

```
~> "abc def abc" par_reduce { letters |> freq } # canonical histogram merge
~> @nums par_reduce { sum } # parallel sum
~> "abcdefgh" par_reduce { len(_) } { _ + _1 } # explicit pairwise reducer
```

Below the chunk threshold the runtime falls back to single-chunk eval; the result must still match the bare `extract` block's output. Multi-chunk paths require the merger (auto or explicit) to be associative and commutative for correct results.

`||>`

Boundary marker that terminates the chunk-parallel section of a `~p>` macro and switches the merged result into a sequential `~>` continuation.

```
~p> "abc def abc" letters freq ||> values |> sum      # 9
~p> "abc def abc" letters freq |then| values |> sum   # 9 - readable form
~p> @data map { _ * 2 } sum ||> sqrt p                # parallel sum, sequential sqrt
```

Both spellings are byte-equivalent; `||>` is terse, `|then|` reads in English. Use either when you need a post-merge stage that operates on the whole reduced value rather than per-chunk. `|>` also terminates a `~p>` macro.

pipeline

`pipeline(@list)` — wrap a list (or iterator) in a lazy pipeline object supporting chained `->map`, `->filter`, `->take`, `->skip`, `->flat_map`, `->tap`, and other transforms that execute zero work until a terminal method is called.

No intermediate lists are allocated between stages — each element flows through the full chain one at a time, making pipelines memory-efficient even on very large or infinite inputs. Terminal methods include `->collect` (materialize to list), `->reduce` { ... } (fold), `->for_each` { ... } (side-effect iteration), and `->count`. Pipelines compose naturally with stryke's `|>` operator: you can feed `pipeline(...)` output into further `|>` stages or vice versa. Use `pipeline` when you want explicit method-chaining style rather than the flat `|>` pipe syntax — both compile to the same lazy evaluation engine.

```
val @out = pipeline(@data)
  ->filter { _ > 0 }
  ->map { _ * 2 }
  ->take(10)
  ->collect

pipeline(1:1_000_000)
  ->filter { _ % 3 == 0 }
  ->map { _ ** 2 }
  ->take(5)
  ->for_each { p _ }           # 9 36 81 144 225

var $sum = pipeline(@scores)
  ->filter { _ >= 60 }
  ->reduce { _ + _1 }
```

par_pipeline

`par_pipeline(source => \@data, stages => [...], workers => N)` — build a multi-stage parallel pipeline where each stage's map/filter block runs concurrently across N worker threads.

Unlike `pmap` which parallelizes a single transform, `par_pipeline` lets you define a sequence of named stages — each with its own block — that execute in parallel while preserving input order in the final output. Internally, stryke partitions the source into chunks, distributes them across workers, and pipelines the stages so that stage 2 can begin processing a chunk as soon as stage 1 finishes it, overlapping computation across stages. This is ideal for multi-step ETL workloads where each step is CPU-bound and the data volume is large. The `workers` parameter defaults to the number of logical CPUs.

```
val @results = par_pipeline(
  source => \@raw_records,
  stages => [
    { name => "parse",    map => fn { json_decode } },
    { name => "transform", map => fn { enrich } },
    { name => "validate", filter => fn { _->{valid} } },
  ],
  workers => 8,
)
```

par_pipeline_stream

`par_pipeline_stream(source => ..., stages => [...], workers => N)` — streaming variant of `par_pipeline` that connects stages via bounded `pchannel` queues instead of materializing intermediate arrays.

Each stage runs as an independent pool of workers, pulling from an input channel and pushing to an output channel. This gives true pipelined parallelism: stage 1 workers produce items while stage 2 workers consume them concurrently, bounded by channel capacity to prevent memory blowup. The streaming design makes this suitable for infinite or very large data sources (file streams, network feeds) where materializing the full dataset between stages is impractical. Results are emitted in arrival order by default; pass `ordered => 1` to reorder them to match input order at the cost of buffering.

```
par_pipeline_stream(  
  source => fn { while (val $line = <STDIN>) { yield $line } },  
  stages => [  
    { name => "parse", map => fn { json_decode } },  
    { name => "score", map => fn { compute_score } },  
  ],  
  workers => 4,  
  on_item => fn { p _ },          # process results as they arrive  
)
```

collect

`collect` `ITERATOR` — materialize a lazy iterator or pipeline into a concrete list.

Pipeline stages in stryke are lazy: chaining `|> map {...} |> grep {...}` builds up a deferred computation without consuming any elements. Calling `collect` forces evaluation and returns all results as a regular Perl list. This is the standard way to terminate a lazy chain when you need the full result in an array. Without `collect`, the iterator is consumed element-by-element by `e`, `take`, or other streaming sinks.

```
val @out = range(1,5) |> map { _ * 2 } |> collect  
gen { yield _ for 1:3 } |> collect |> e p  
var @data = stdin |> grep /INFO/ |> collect
```


Streaming Iterators

24 topics

maps

`maps { BLOCK } LIST` — the lazy, streaming counterpart of `map`. Instead of materializing the entire output list, `maps` returns a pull iterator that evaluates the block on demand as downstream consumers request values. This makes it ideal for `|>` pipeline chains, especially when combined with `take`, `greps`, or `collect`. Use `maps` over `map` when working with large ranges, infinite sequences, or when you want to short-circuit processing early with `take`. Memory usage is constant regardless of input size.

```
1:10 |> maps { _ * 3 } |> take 4 |> e p
# 3 6 9 12
1:1_000_000 |> maps { _ ** 2 } |> greps { _ > 100 } |> take 3 |> e p
```

greps

`greps { BLOCK } LIST` — the lazy, streaming counterpart of `grep`. Returns a pull iterator that only evaluates the predicate as elements are requested downstream. This is the preferred filtering function in `|>` pipelines because it avoids materializing intermediate lists. Combine with `take` to short-circuit early, or with `maps` and `collect` for full lazy pipelines. The block receives `_` just like `grep`.

```
1:100 |> greps { _ % 7 == 0 } |> take 3 |> e p
# 7 14 21
@lines |> greps { /ERROR/ } |> maps { tm } |> e p
```

filter

`filter` (alias `fi`) `{ BLOCK } LIST` — stryke-native lazy filter that returns a pull iterator, functionally identical to `greps` but named for familiarity with Rust/Ruby/JS conventions. Use `filter` or `greps` interchangeably in `|>` chains; both are streaming and both set `_` in the block. The result must be consumed with `collect`, `e p`, `foreach`, or another terminal. Prefer `filter` when writing stryke-idiomatic code; prefer `grep/greps` when porting from Perl.

```
val @big = 1:1000 |> filter { _ > 990 } |> collect
@big |> e p # 991 992 993 994 995 996 997 998 999 1000
1:50 |> fi { _ % 2 } |> take 5 |> e p # 1 3 5 7 9
```

tap

`tap { BLOCK } LIST` — execute a side-effecting block for each element, then pass the element through unchanged.

The return value of the block is ignored; the original element is always forwarded to the next pipeline stage. This makes `tap` ideal for injecting logging, debugging, or metrics collection into the middle of a pipeline without altering the data flow. It is fully streaming and preserves element order.

```
1:5 |> tap { log_debug "saw: $_" } |> map { _ * 2 } |> e p
@files |> tap { p "processing: $_" } |> map slurp |> e p
```

peek

`peek ITERATOR` — inspect the next element of an iterator without consuming it.

The peeked value is buffered internally so that the next call to `->next` or pipeline pull returns the same element. This is useful for lookahead parsing, conditional branching on the next value, or implementing `take_while`-style logic manually. Calling

`peek` multiple times without advancing the iterator returns the same value each time. Works with any stryke iterator including `gen`, `range`, `stdin`, and pipeline results.

```
val $g = gen { yield _ for 1:5 }
p peek $g           # 1 (not consumed)
p $g->next          # 1
p peek $g           # 2
```

tee

`tee FILE, ITERATOR` — write each element to a file as a side effect while passing it through the pipeline.

Every element that flows through `tee` is appended as a line to the specified file, and the element itself continues downstream unchanged. The file is opened once on first element and closed when the iterator is exhausted. This is the stryke equivalent of the Unix `tee` command, useful for auditing or logging intermediate pipeline results to disk.

```
1:10 |> tee "/tmp/log.txt" |> map { _ * 2 } |> e p
stdin |> tee "/tmp/raw.log" |> grep /ERROR/ |> e p
```

take

`take N, LIST / head N, LIST / hd N, LIST` — return at most the first N elements.

In streaming mode, `take` pulls exactly N elements from the upstream iterator and then stops, so it short-circuits infinite or very large sources efficiently. This makes it safe to write `range(1, 1_000_000) |> take 5` without allocating a million-element list. If the source has fewer than N elements, all of them are returned. The `head` and `hd` aliases mirror Unix `head` semantics.

```
1:100 |> take 5 |> e p           # 1 2 3 4 5
var @top = hd 3, @sorted
stdin |> hd 10 |> e p           # first 10 lines
```

tail

`tail N, @list` — returns the last N elements of the list. If N exceeds the list length, the entire list is returned. The alias `tl` is also available. This is the complement of `head/take`. In stryke, it works both as a function call and in `|>` pipelines.

```
val @t = tail 2, 1:5
@t |> e p           # 4 5
1:100 |> tl 3 |> e p # 98 99 100
```

drop

`drop N, LIST / skip N, LIST / drp N, LIST` — skip the first N elements and return the rest.

This operation is fully streaming: when used in a pipeline, the first N elements are consumed and discarded without buffering, and all subsequent elements flow through to the next stage. If the list contains fewer than N elements, the result is empty. The `skip` and `drp` aliases exist for readability in different contexts — all three compile to the same internal op.

```
1:10 |> drop 3 |> e p           # 4 5 6 7 8 9 10
var @rest = drp 2, @data
stdin |> skip 1 |> e p          # skip header line
```

take_while

`take_while { COND } LIST` — emit leading elements while the predicate returns true, then stop.

The block receives each element as `_`. Elements are emitted as long as the predicate holds; the moment it returns false, the pipeline terminates immediately without consuming further input. This short-circuit behavior makes `take_while` efficient on infinite iterators and large streams. It is the complement of `drop_while`.

```
1:10 |> take_while { _ < 5 } |> e p # 1 2 3 4
stdin |> take_while { !/^END/ } |> e p
```

drop_while

`drop_while { COND } LIST` — skip leading elements while the predicate returns true, then emit everything after.

The block receives each element as `_`. Once the predicate returns false for the first time, that element and all remaining elements pass through unconditionally — the predicate is never consulted again. This is streaming: elements are tested one at a time without buffering. Useful for skipping headers, preamble, or sorted prefixes in data streams.

```
1:10 |> drop_while { _ < 5 } |> e p # 5 6 7 8 9 10
@log |> drop_while { _ ^ #/ } |> e p # skip comment header
```

reject

`reject { BLOCK } LIST` — stryke-native streaming inverse of `filter/greps`. It keeps only the elements for which the block returns false. This reads more naturally than `filter { !(...) }` when the condition describes what you want to exclude rather than what you want to keep. Like `filter` and `greps`, it returns a lazy iterator suitable for `|>` chains. The block receives `_` as the current element.

```
1:10 |> reject { _ % 3 == 0 } |> e p
# 1 2 4 5 7 8 10
@files |> reject { /\.bak$/ } |> e p # skip backups
```

compact

`compact` (alias `cpt`) — stryke-native streaming operator that removes `undef` and empty-string values from a list or iterator. This is a common data-cleaning step when dealing with parsed input, optional fields, or split results that produce empty segments. It is equivalent to `greps { defined(_) && _ ne "" }` but more concise and faster because the check is inlined in Rust. Numeric zero and the string `"0"` are preserved since they are defined and non-empty.

```
val @raw = (1, undef, "", 2, undef, 3)
@raw |> compact |> e p # 1 2 3
split(/./, "a,,b,,c") |> cpt |> e p # a b c
```

concat

`concat` (alias `chain`) — stryke-native streaming operator that concatenates multiple lists or iterators into a single sequential iterator. Pass array references and they will be yielded in order without copying. This is useful for merging data from multiple sources into a unified pipeline. The operation is lazy: each source is drained in turn, so memory usage stays proportional to the largest single element, not the total.

```
val @a = 1:3
var @b = 7:9
concat(@a, @b) |> e p # 1 2 3 7 8 9
var @c = ("x")
concat(@a, @b, @c) |> maps { uc } |> e p
```

enumerate

`enumerate ITERATOR / en ITERATOR` — yield `[$index, $item]` pairs from a streaming iterator.

Each element is wrapped as `[$index, $item]` where the index starts at 0 and increments for each element. Unlike `with_index` which returns `[item, index]`, `enumerate` uses the Rust/Python convention of `[index, item]`. This is a streaming operation: the index counter is maintained lazily as elements flow through the pipeline.

```
stdin |> en |> e { p "->[0]: _->[1]" }
1:5 |> en |> e { p "->[0]: _->[1]" }
@lines |> en |> grep { _->[0] < 10 } |> e { p _->[1] }
```

chunk

`chunk N, ITERATOR / chk N, ITERATOR` — group elements into N-sized arrayrefs as they stream through.

Elements are buffered until N are collected, then the group is emitted as a single arrayref. The final chunk may contain fewer than N elements if the source is not evenly divisible. This is fully streaming: only one chunk is held in memory at a time, making it safe for large or infinite iterators. Use `chunk` for batching work (e.g., bulk database inserts) or formatting output into rows.

```
1:9 |> chk 3 |> e { p join ", ", @_ }
# 1,2,3 4,5,6 7,8,9
stdin |> chk 100 |> e { bulk_insert(@_) }
```

dedup

`dedup` *ITERATOR* / `dup` *ITERATOR* — drop consecutive duplicate elements from a stream.

Only adjacent duplicates are removed: if the same value appears later after an intervening different value, it is emitted again. Comparison is string-based by default. This is a streaming operation that holds only the previous element in memory, so it works on infinite iterators. For global deduplication across the entire stream, use `distinct` instead.

```
1,1,2,2,3,1,1 |> dedup |> e p      # 1 2 3 1
@sorted |> dedup |> e p          # like uniq(1)
stdin |> dedup |> e p            # collapse repeated lines
```

distinct

`distinct` *LIST* — remove duplicate elements, preserving first-occurrence order (alias for `uniq`).

Each element is compared as a string; the first time a value is seen it is emitted, and all subsequent occurrences are silently dropped. When used in a pipeline the deduplication state is maintained across streamed chunks, so `distinct` works correctly on lazy iterators and generators. This is a stryke built-in backed by a hash set internally, so it runs in $O(n)$ time regardless of list size.

```
val @u = distinct(3,1,2,1,3)      # (3,1,2)
1,2,2,3,3,3 |> distinct |> e p    # 1 2 3
stdin |> distinct |> e p          # unique lines
```

flatten

`flatten` *LIST* / `fl` *LIST* — recursively flatten nested arrayrefs into a single flat list.

Flatten walks every element: scalars pass through unchanged, arrayrefs are opened and their contents are recursively flattened, so arbitrarily deep nesting is handled in one call. In pipeline mode (`|> fl`) it streams element-by-element, so you can chain it with `map`, `grep`, or `take` without materializing the entire intermediate list. The `fl` alias keeps pipeline chains concise.

```
val @flat = flatten([1,[2,3]],[4]) # (1,2,3,4)
[1,[2,[3,4]]] |> fl |> e p          # 1 2 3 4
@nested |> fl |> grep { _->0 } |> e p
```

with_index

`with_index` *LIST* / `wi` *LIST* — pair each element with its 0-based index as `[$item, $index]`.

Each element is wrapped in a two-element arrayref where `_->[0]` is the original value and `_->[1]` is its position. This is useful when you need positional information during a map or grep without maintaining a manual counter. Note the order is `[item, index]`, which differs from `enumerate` which yields `[index, item]`.

```
qw(a b c) |> wi |> e { p "_->[1]: _->[0]" }
# 0: a 1: b 2: c
@data |> wi |> grep { _->[1] % 2 == 0 } |> e { p _->[0] }
```

first_or

`first_or` *DEFAULT*, *LIST* — return the first element of the list, or *DEFAULT* if the list is empty.

This is a streaming terminal: it pulls exactly one element from the upstream iterator and returns it, or returns the default value if the iterator is exhausted. It never buffers the entire list. This is especially useful at the end of a `grep` or `map` pipeline where you need a safe fallback instead of `undef` when no match is found.

```
val $v = first_or 0, @maybe_empty
var $x = grep { _ > 99 } @nums |> first_or -1
stdin |> grep /^ERROR/ |> first_or "(none)" |> p
```

range

`range(START, END [, STEP])` — create a lazy integer iterator from START to END with an optional step.

The range is inclusive on both ends. When `START > END` and no step is given, stryke automatically counts downward. An explicit STEP controls the increment and must be negative when counting down, or the range will be empty. The iterator is fully lazy: no list is allocated, and elements are generated on demand, making `range(1, 1_000_000)` as cheap to create as `range(1, 5)`. Combine with `|>` to feed into streaming pipelines.

```
range(1, 5) |> e p      # 1 2 3 4 5
range(5, 1) |> e p      # 5 4 3 2 1
range(0, 10, 2) |> e p   # 0 2 4 6 8 10
range(10, 0, -2) |> e p  # 10 8 6 4 2 0
```

stdin

`stdin` — return a streaming iterator over lines read from standard input.

Each call to the iterator reads one line from STDIN, strips the trailing newline, and yields it. The iterator terminates at EOF. Because it is lazy, combining `stdin` with `take`, `grep`, or `first_or` processes only as many lines as needed — the rest of STDIN is left unconsumed. This is the idiomatic stryke way to build Unix-style filters.

```
stdin |> grep /error/i |> e p
stdin |> take 5 |> e p
stdin |> en |> e { p "_->[0]: _->[1]" }
```

nth

`nth N, LIST` — return the Nth element using 0-based indexing.

When used in a pipeline, `nth` consumes and discards the first N elements, returns the next one, and stops — so it short-circuits on infinite iterators. On a plain list, it is equivalent to `$list[N]` but works as a function call for pipeline composition. Returns `undef` if the list has fewer than N+1 elements.

```
val $third = nth 2, @data
1:10 |> nth 4 |> p      # 5
stdin |> nth 0 |> p     # first line
```

List Operations

62 topics

map

map **BLOCK** **LIST** — evaluates the block for each element of the list, setting `_` to the current element, and returns a new list of all the block's return values. This is the eager version: it consumes the entire input list and produces the entire output list before returning. Use **map** when you need the full result array or when the input is small. For large or infinite sequences, prefer **maps** (the streaming variant). The block can return zero, one, or multiple values per element, making **map** useful for both transformation and flattening. Use `{ |$var| body }` to name the block parameter instead of `_`.

```
val @sq = map { _ ** 2 } 1:5
@sq |> e p    # 1 4 9 16 25
map { |$n| $n * $n }, 1:5    # named param
var @pairs = map { (_, _ * 10) } 1:3
@pairs |> e p    # 1 10 2 20 3 30
```

grep

grep `{ BLOCK }` **LIST** — filters the list, returning only elements for which the block evaluates to true. The current element is available as `_`. This is the eager version: it processes the entire list and returns a new list. It is Perl-compatible and works exactly like Perl's builtin **grep**. For streaming/lazy filtering in `|>` pipelines, use **greps** or **filter** instead. Use `{ |$var| body }` to name the block parameter instead of `_`.

```
val @evens = grep { _ % 2 == 0 } 1:10
@evens |> e p    # 2 4 6 8 10
grep { |$x| $x > 3 }, 1:6    # named param
var @long = grep { len > 3 } @words
```

sort

sort [**BLOCK**] **LIST** — returns a new list sorted in ascending order. Without a block, **sort** compares elements as strings (lexicographic). Pass a comparator block; the two candidates are `_` and `_1`. Use `<=>` for numeric and **cmp** for string comparison. The sort is stable in stryke, meaning equal elements preserve their original relative order. For descending order, reverse the operands in the comparator. Use `{ |$x, $y| body }` to name the two comparator params. Works naturally in `|>` pipelines.

```
val @nums = {3, 1, 4, 1, 5}
var @asc = sort { _ <=> _1 } @nums
@asc |> e p    # 1 1 3 4 5
sort { |$x, $y| $y <=> $x }, @nums    # named params
var @desc = sort { _1 <=> _ } @nums
@desc |> e p    # 5 4 3 1 1
@nums |> sort |> e p    # string sort in pipeline
```

reverse

reverse **LIST** — in list context, return the elements in reverse order; in scalar context, reverse the characters of a string.

The dual nature of **reverse** is context-dependent: **reverse** `@array` flips element order, while **scalar reverse** `$string` (or just **reverse** `$string` in scalar context) reverses character order. In stryke, the string form is Unicode-aware, correctly reversing multi-byte characters rather than individual bytes. In `|>` pipelines, use the **t rev** shorthand for a concise streaming string reverse. **reverse** does not modify the original — it always returns a new list or string.

```
p reverse "hello"      # olleh
var @r = reverse 1:5    # (5,4,3,2,1)
@r |> e p              # 5 4 3 2 1
"abc" |> t rev |> p     # cba
```

reduce

`reduce { _ OP _1 } @list` — performs a sequential left fold over the list. The first two elements are `_` and `_1`; each iteration's result becomes the next `_`. Use `{ |$acc, $val| body }` to name the two params. Returns `undef` for an empty list and the single element for a one-element list. For a fold with an explicit initial value, see `fold`.

```
val $fac = reduce { _ * _1 } 1:6
p $fac    # 720
reduce { |$acc, $val| $acc + $val }, 1:10 # 55 (named params)
var $longest = reduce { length(_) > length(_1) ? _ : _1 } @words
```

fold

`fold { _ OP _1 } INIT, @list` — left fold with an explicit initial accumulator value. The initial value is the first `_`, and each list element is `_1`. Unlike `reduce`, `fold` never returns `undef` for an empty list — it returns the initial value instead. Use `fold` when you need a guaranteed starting point for the accumulation.

```
val $total = fold { _ + _1 } 100, 1:5
p $total    # 115
var $csv = fold { _ . ", " . _1 } "", @fields
```

reductions

`reductions { _ OP _1 } @list` — returns the running (cumulative) results of a left fold, also known as a scan or prefix-sum. Each element of the output is the accumulator state after processing the corresponding input element. The output list has the same length as the input.

```
val @pfx = reductions { _ + _1 } 1:4
@pfx |> e p    # 1 3 6 10
1:5 |> reductions { _ * _1 } |> e p # 1 2 6 24 120
```

all

`all { COND } @list` — returns true (1) if every element in the list satisfies the predicate, false ("") otherwise. The block receives each element as `_` and should return a boolean. Short-circuits on the first failing element, so it never evaluates more than necessary. Works with `|>` pipelines and accepts bare lists or array variables.

```
val @nums = 2, 4, 6, 8
p all { _ % 2 == 0 } @nums # 1
1:100 |> all { _ > 0 } |> p # 1
```

any

`any { COND } @list` — returns true (1) if at least one element satisfies the predicate, false ("") if none do. The block receives each element as `_`. Short-circuits on the first match, making it efficient even on large lists. `any` is a stryke global builtin and can be used in `|>` pipelines.

```
val @vals = 1, 3, 5, 8
p any { _ > 7 } @vals # 1
1:1000 |> any { _ == 42 } |> p # 1
```

none

`none { COND } @list` — returns true (1) if no element satisfies the predicate, false (0) if any element matches. Logically equivalent to `!any { COND } @list` but reads more naturally in guard clauses. Short-circuits on the first match. Useful for validation checks where you want to assert the absence of a condition.

```
val @words = ("cat", "dog", "bird")
p none { /z/ } @words # 1
p "all non-negative" if none { _ < 0 } @vals
```

first

`first { COND } @list` — returns the first element for which the block returns true, or `undef` if no element matches. The alias `fst` is also available. Short-circuits immediately upon finding a match, so only the minimum number of elements are tested. Ideal for searching sorted or unsorted lists when you need a single result.

```
val $f = first { _ > 10 } 3, 7, 12, 20
p $f # 12
1:1_000_000 |> first { _ % 9999 == 0 } |> p # 9999
```

min

`min @list` — returns the numerically smallest value from a list. Compares all elements using numeric (`<=>`) comparison, so stringy values are coerced to numbers. Returns `undef` for an empty list. In stryke, `min` is a global built-in (no import needed) and works directly in `|>` pipelines.

```
p min(5, 3, 9, 1) # 1
var @temps = (72.1, 68.5, 74.3)
@temps |> min |> p # 68.5
```

max

`max @list` — returns the numerically largest value from a list. Compares all elements using numeric (`<=>`) comparison. Returns `undef` for an empty list. Like `min`, this is a stryke built-in available without imports and works in `|>` pipelines. Combine with `map` to extract max values from complex structures.

```
p max(5, 3, 9, 1) # 9
1:100 |> map { _ ** 2 } |> max |> p # 10000
```

sum

`sum @list` returns the numeric sum of all elements. Returns `undef` for an empty list. `sum0` is identical except it returns `0` for an empty list, which avoids the need for a fallback `// 0` guard. Both are stryke built-ins that work in `|>` pipelines. Use `sum0` in contexts where an empty input is expected and you need a safe numeric default.

```
p sum(1:100) # 5050
p sum0() # 0
@prices |> map { _->{amount} } |> sum0 |> p
```

product

`product @list` — returns the product of all numeric elements in the list. Returns `undef` for an empty list. Useful for computing factorials, combinatoric products, and compound multipliers. In stryke this is a built-in that composes naturally with `|>` pipelines and `range`.

```
p product(1:5) # 120
range(1, 10) |> product |> p # 3628800
```

mean

`mean @list` — returns the arithmetic mean (average) of a numeric list. Computed as `sum / count` in a single pass. Returns `undef` for an empty list. The result is always a floating-point value even if all inputs are integers. Combine with `map` to compute averages over extracted fields.


```
p mean(2, 4, 6, 8)    # 5
@students |> map { _->{score} } |> mean |> p
```

median

median *@list* — returns the median value of a numeric list. For odd-length lists, this is the middle element after sorting. For even-length lists, it is the arithmetic mean of the two middle elements. The input list does not need to be pre-sorted. Returns **undef** for an empty list.

```
p median(1, 3, 5, 7, 9)    # 5
p median(1, 3, 5, 7)      # 4
1:100 |> median |> p      # 50.5
```

mode

mode *@list* — returns the most frequently occurring value in the list. If there is a tie, the value that appears first in the list wins. Returns **undef** for an empty list. Works with both numeric and string values — comparison is done by stringification. Useful for finding the dominant category in a dataset.

```
p mode(1, 2, 2, 3, 3, 3)    # 3
var @logs = qw(INFO WARN INFO ERROR INFO)
p mode(@logs)              # INFO
```

stddev

stddev *@list* (alias **std**) — returns the population standard deviation of a numeric list. This is the square root of the population variance, measuring how spread out values are from the mean. Uses N (not N-1) in the denominator, so it computes the population statistic rather than the sample statistic. Returns **undef** for an empty list.

```
p stddev(2, 4, 4, 4, 5, 5, 7, 9)    # 2
1:10 |> std |> p
```

variance

variance *@list* — returns the population variance of a numeric list, computed as the mean of the squared deviations from the mean. Like **stddev**, this uses N (not N-1) as the divisor. The variance is **stddev** ****** 2. Returns **undef** for an empty list. Useful for statistical analysis and as a building block for higher-order stats.

```
p variance(2, 4, 4, 4, 5, 5, 7, 9)    # 4
var @samples = map { rand(100) } 1:1000
p variance(@samples)
```

sample

sample *N*, *@list* — returns a random sample of N elements drawn without replacement from the list. The returned elements are in random order. If N exceeds the list length, the entire list is returned (shuffled). Uses a Fisher-Yates partial shuffle internally for efficiency. Each call produces a different result due to random selection.

```
val @pick = sample 3, 1:100
@pick |> e p    # 3 random values
var @test_cases = sample 10, @all_cases
```

shuffle

shuffle *@list* — returns a new list with all elements in random order using a Fisher-Yates shuffle. The original list is not modified. Every permutation is equally likely. In stryke this is a global built-in; no import is needed. The alias **shuf** is also available. Commonly used with **|>** and **take** to draw random subsets.

```
val @deck = shuffle 1:52
@deck |> take 5 |> e p    # 5 random cards
var @randomized = shuffle @questions
```

uniq

`uniq @list` — removes duplicate elements from a list, preserving the order of first occurrence. Comparison is done by string equality. The alias `uq` is also available. This is eager (not streaming) and returns a new list. For streaming deduplication in `|>` pipelines, use `distinct`. For type-specific comparison, see `uniqnum`, `uniqstr`, and `uniqint`.

```
val @u = uniq 1, 2, 2, 3, 1, 3
@u |> e p # 1 2 3
var @hosts = uniq @all_hosts
```

uniqint

`uniqint @list` — removes duplicate elements comparing values as integers. Each element is truncated to its integer part before comparison, so `1.1` and `1.9` are considered equal (both become `1`). The first occurrence is kept. This is useful when you have floating-point data but care only about the integer portion for uniqueness.

```
val @u = uniqint 1, 1.1, 1.9, 2
@u |> e p # 1 2
var @distinct_ids = uniqint @raw_ids
```

uniqnum

`uniqnum @list` — removes duplicate elements comparing values as floating-point numbers. Unlike `uniq` (which compares as strings), `uniqnum` treats `1.0` and `1.00` as equal because they have the same numeric value. The first occurrence of each numeric value is preserved. Use this when your data contains numbers that may have different string representations.

```
val @u = uniqnum 1.0, 1.00, 2.5, 2.50
@u |> e p # 1 2.5
var @prices = uniqnum @all_prices
```

uniqstr

`uniqstr @list` — removes duplicate elements comparing values strictly as strings. This is the same comparison as `uniq` but makes the intent explicit. Numeric values `1` and `1.0` are considered different because their string representations differ. Use `uniqstr` when you want to be explicit that string semantics are intended.

```
val @u = uniqstr "a", "b", "a", "c"
@u |> e p # a b c
var @tags = uniqstr @all_tags
```

zip

`zip(\@a, \@b, ...)` — combines multiple arrays element-wise into a list of arrayrefs. Each output arrayref contains one element from each input array at the corresponding index. Accepts two or more array references. The alias `zp` is also available. By default, `zip` pads shorter arrays with `undef` (equivalent to `zip_longest`). For truncating behavior, use `zip_shortest`.

```
val @a = 1:3
var @b = ("a", "b", "c")
zip(\@a, \@b) |> e p # [1,a] [2,b] [3,c]
var @matrix = zip(\@xs, \@ys, \@zs)
```

zip_longest

`zip_longest(\@a, \@b, ...)` — combines arrays element-wise, padding shorter arrays with `undef` to match the longest. This is the explicit version of the default `zip` behavior. Every input array contributes exactly one element per output tuple; missing elements become `undef`. Useful when you need to process all data from unequal-length sources.

```
val @a = 1:3
var @b = ("x")
zip_longest(\@a, \@b) |> e p # [1,x] [2,undef] [3,undef]
```

zip_shortest

`zip_shortest(\@a, \@b, ...)` — combines arrays element-wise, stopping at the shortest input array. No `undef` padding is produced; the output length equals the minimum input length. Use this when you only want complete tuples and extra trailing elements should be discarded.

```
val @a = 1:5
var @b = ("x","y")
zip_shortest(\@a, \@b) |> e p # [1,x] [2,y]
var @paired = zip_shortest(\@keys, \@values)
```

chunked

`chunked N, @list` — splits a list into non-overlapping chunks of N elements each. Each chunk is an arrayref. The final chunk may contain fewer than N elements if the list length is not evenly divisible. The alias `chk` is also available. In `stryke`, `chunked` is eager and returns a list of arrayrefs; for streaming chunk behavior in pipelines, use `chunk`.

```
val @ch = chunked 3, 1:7
@ch |> e p # [1,2,3] [4,5,6] [7]
1:12 |> chunked 4 |> e { p join ", ", @_ }
```

windowed

`windowed N, @list` — returns a sliding window of N consecutive elements over the list. Each window is an arrayref. The output contains `len - N + 1` windows. Unlike `chunked`, windows overlap: each successive window advances by one element. The alias `win` is also available. Useful for computing moving averages, detecting patterns in sequences, or n-gram extraction.

```
val @w = windowed 3, 1:5
@w |> e p # [1,2,3] [2,3,4] [3,4,5]
var @deltas = windowed(2, @vals) |> map { _->[1] - _->[0] }
```

pairs

`pairs @list` — takes a flat list and groups consecutive elements into pairs, returning a list of two-element arrayrefs (`[$k, $v], ...`). The input list must have an even number of elements. Each pair can be accessed via array indexing (`_->[0]`, `_->[1]`). This is the inverse of `unpairs` and is commonly used to iterate over hash-like flat lists in a structured way.

```
val @p = pairs "a", 1, "b", 2
@p |> e { p "_->[0]=_->[1]" } # a=1 b=2
var @entries = pairs %hash
```

unpairs

`unpairs @list_of_pairs` — flattens a list of two-element arrayrefs back into a flat key-value list. This is the inverse of `pairs`. Each arrayref `[$k, $v]` becomes two consecutive elements in the output. Useful for converting structured pair data back into a format suitable for hash assignment or flat list processing.

```
val @flat = unpairs ["a",1], ["b",2]
@flat |> e p # a 1 b 2
var %h = unpairs @filtered_pairs
```

pairkeys

`pairkeys @list` — extracts the keys (even-indexed elements) from a flat pairlist. Given a list like `("a", 1, "b", 2, "c", 3)`, returns `("a", "b", "c")`. This is equivalent to `map { _->[0] } pairs @list` but more concise and efficient. Useful for extracting just the key side of a key-value flat list without constructing intermediate pair objects.

```
val @k = pairkeys "a", 1, "b", 2, "c", 3
@k |> e p # a b c
var @config_pairs = (host => "localhost", port => 8080)
var @names = pairkeys @config_pairs
```

pairvalues

`pairvalues @list` — extracts the values (odd-indexed elements) from a flat pairlist. Given `("a", 1, "b", 2)`, returns `(1, 2)`. This is equivalent to `map { _->[1] } pairs @list` but more concise. Pair it with `pairkeys` to split a flat key-value list into separate key and value arrays.

```
val @v = pairvalues "a", 1, "b", 2
@v |> e p    # 1 2
var @defaults = (timeout => 30, retries => 3)
var @settings = pairvalues @defaults
```

pairmap

`pairmap BLOCK @list` — maps over consecutive pairs in a flat list, passing the key as `_` and the value as `_1`. The block can return any number of elements. This is the pair-aware equivalent of `map` and is ideal for transforming hash-like flat lists. The result is a flat list of whatever the block returns.

```
val @out = pairmap { _ . "=" . _1 } "a", 1, "b", 2
@out |> e p    # a=1 b=2
var @cfg_pairs = (host => "x", port => 80)
var @upper = pairmap { uc(_), _1 } @cfg_pairs
```

pairgrep

`pairgrep { BLOCK } @list` — filters consecutive pairs from a flat list, keeping only those where the block returns true. The key is `_` and the value is `_1`. Returns a flat list of the matching key-value pairs. This is the pair-aware equivalent of `grep` and is useful for filtering hash-like data by both key and value simultaneously.

```
val @big = pairgrep { _1 > 5 } "a", 3, "b", 9, "c", 1
@big |> e p    # b 9
var @alert_pairs = (info => 1, critical_a => 9, critical_b => 7)
var @important = pairgrep { _ =~ /^critical/ } @alert_pairs
```

pairfirst

`pairfirst { BLOCK } @list` — returns the first pair from a flat list where the block returns true, as a two-element list `($key, $value)`. The key is `_` and the value is `_1`. Short-circuits on the first match. Returns an empty list if no pair matches. This is the pair-aware equivalent of `first`.

```
val @hit = pairfirst { _1 > 5 } "x", 2, "y", 8
p "@hit"  # y 8
var @flags = (info => 1, debug => 7, trace => 0)
var ($k, $v) = pairfirst { _ eq "debug" } @flags
```

mesh

`mesh(\@a, \@b, ...)` — interleaves multiple arrays into a flat list rather than arrayrefs. While `zip` returns `[[1,"a"], [2,"b"]]`, `mesh` returns `(1, "a", 2, "b")`. This makes it ideal for constructing hashes from parallel key and value arrays. The result is a flat list suitable for direct hash assignment.

```
val @k = ("a", "b")
var @v = (1, 2)
var %h = mesh(\@k, \@v)
p $h{a}  # 1
var %lookup = mesh(\@ids, \@names)
```

mesh_longest

`mesh_longest(\@a, \@b, ...)` — interleaves arrays into a flat list, padding shorter arrays with `undef` to match the longest. Like `mesh`, the output is flat (not arrayrefs). Missing elements become `undef` in the output sequence. Use this when building a flat interleaved list from arrays of unequal length where you need every element represented.

```
val @a = 1:3
var @b = ("x")
var @r = mesh_longest(\@a, \@b)
@r |> e p    # 1 x 2 undef 3 undef
```

mesh_shortest

`mesh_shortest(\@a, \@b, ...)` — interleaves arrays into a flat list, stopping at the shortest input array. No `undef` padding is produced; trailing elements from longer arrays are silently dropped. Use this when you only want complete interleaved groups and partial data should be discarded.

```
val @a = 1:3
var @b = ("x", "y")
var @r = mesh_shortest(\@a, \@b)
@r |> e p    # 1 x 2 y
```

partition

`partition` — operation `partition`. Alias for `take_while`. Category: **functional / iterator**.

```
p partition $x
```

frequencies

`frequencies` (aliases `freq`, `frq`) counts how many times each distinct element appears in a list and returns a hashref mapping each value to its count. This is the stryke equivalent of a histogram or counter — useful for analyzing log files, counting word occurrences, tallying categorical data, or finding duplicates. The input list is flattened, so you can pass arrays directly. The returned hashref can be fed into `dd` for inspection or `to_json` for serialization.

```
val @words = qw(apple banana apple cherry banana apple)
var $counts = freq @words
p $counts    # {apple => 3, banana => 2, cherry => 1}
rl("access.log") |> map { /\^(S+)/ 66 $1 } |> freq |> dd
var @rolls = map { 1 + int(rand 6) } 1:1000
frq(@rolls) |> to_json |> p
```

tally

`tally` counts how many times each distinct element appears in a list and returns a hash ref mapping element → count. This is the same as Ruby's `Enumerable#tally` or Python's `Counter`. Similar to `frequencies` but follows the Ruby naming convention.

```
val $t = tally("a", "b", "a", "c", "a", "b")
p $t->{a}    # 3
p $t->{b}    # 2
qw(red blue red green blue red) |> tally |> dd
```

interleave

`interleave` (alias `il`) merges two or more arrays by alternating their elements: first element of each array, then second element of each, and so on. If the arrays have different lengths, shorter arrays contribute `undef` for their missing positions. This is useful for building key-value pair lists from separate key and value arrays, creating round-robin schedules, or weaving parallel data streams together.

```
val @keys = qw(name age city)
var @vals = ("Alice", 30, "NYC")
var @pairs = il \@keys, \@vals
p @pairs    # name, Alice, age, 30, city, NYC
var %h = il \@keys, \@vals
p $h{name} # Alice
var @rgb = il [255,0,0], [0,255,0], [0,0,255]
```

pluck

pluck *KEY*, *LIST_OF_HASHREFS* — extract a single key from each hashref in the list.

For each element, **pluck** dereferences it as a hashref and returns the value at the given key. Elements where the key is missing yield **undef**. This is a streaming operation: in a pipeline, each hashref is processed and the extracted value is emitted immediately. It is the stryke equivalent of `map { _->{KEY} }` but more readable and optimized internally.

```
@users |> pluck "name" |> e p
var @ids = pluck "id", @records
@rows |> pluck "email" |> distinct |> e p
```

grep_v

grep_v *PATTERN*, *LIST* — inverse grep: reject elements that match the pattern, keep the rest.

This is the complement of **grep** — any element where the pattern matches is dropped, and non-matching elements pass through. It accepts a regex, a string, or a code block as the pattern. In streaming mode, each element is tested and either forwarded or discarded without buffering. This is a stryke built-in that avoids the awkward `grep { !/pattern/ }` double-negation.

```
@words |> grep_v /^ #/ |> e p          # drop comments
var @clean = grep_v qr/tmp/, @files
stdin |> grep_v /\s*$/ |> e p         # drop blank lines
```

select_keys

select_keys — operation **select keys**. Category: *pipeline / string helpers*.

```
p select_keys $x
```

clamp

clamp (alias **clp**) constrains each value in a list to lie within a specified minimum and maximum range. Values below the minimum are raised to it; values above the maximum are lowered to it; values already in range pass through unchanged. This is essential for sanitizing user input, bounding computed values before display, or enforcing physical constraints in simulations. When given a single scalar, it returns a single clamped value.

```
val @scores = (105, -3, 42, 99, 200)
var @clamped = clp 0, 100, @scores
p @clamped    # 100, 0, 42, 99, 100
var $val = clp 0, 255, $input    # bound to byte range
var @pct = map { clp 0.0, 1.0, _ } @raw_ratios
```

normalize

normalize (alias **nrm**) rescales a list of numeric values so that the minimum maps to 0 and the maximum maps to 1, using min-max normalization. This is a standard preprocessing step for machine learning features, data visualization (mapping values to color gradients or bar heights), and statistical analysis. If all values are identical, the result is a list of zeros to avoid division by zero. The output preserves the relative ordering of the input.

```
val @temps = (32, 68, 100, 212)
var @normed = nrm @temps
p @normed    # 0, 0.2, 0.377..., 1
var @pixels = nrm @raw_intensities
@pixels |> map { int(_ * 255) } |> e p    # scale to 0-255
```

compose

compose (alias **comp**) creates a right-to-left function composition. Given `compose(\&f, \&g)`, calling the result with *x* computes `f(g(x))`. Chain any number of functions — they apply from right to left (last argument first). This is the standard mathematical function composition found in Haskell, Clojure, and Ramda. The returned code ref can be stored, passed around, or used in pipelines.

```

val $double = fn { _0 * 2 }
var $inc     = fn { _0 + 1 }
var $f = compose($inc, $double)
p $f->(5)    # 11 (double 5 → 10, inc 10 → 11)

var $pipeline = compose(
  fn { join ", ", @[_0] },
  fn { [sort @[_0]] },
  fn { [grep { _ > 2 } @[_0]] },
)
p $pipeline->([3,1,4,1,5]) # 3,4,5

```

partial

partial returns a partially applied function — the bound arguments are prepended to any arguments supplied at call time. `partial(\&f, @bound)->(x)` is equivalent to `f(@bound, x)`. This is the standard partial application from functional programming, useful for creating specialized versions of general functions without closures.

```

val $add = fn { _0 + _1 }
var $add5 = partial($add, 5)
p $add5->(3) # 8

var $log = fn { p "[#{_0}] #{_1}" }
var $warn_log = partial($log, "WARN")
$warn_log->("disk full") # [WARN] disk full

```

curry

curry auto-curries a function with a given arity. The curried function accumulates arguments across calls and invokes the original only when enough have been collected. `curry(\&f, N)->(a)->(b)` calls `f(a, b)` when $N=2$. If all arguments are supplied at once, it calls immediately.

```

val $add = curry(fn { _0 + _1 }, 2)
var $add5 = $add->(5)
p $add5->(3) # 8
p $add->(10, 20) # 30 (enough args - calls immediately)

```

memoize

memoize (alias **memo**) wraps a function so that repeated calls with the same arguments return a cached result instead of re-executing the function. Arguments are stringified and joined as the cache key. This is essential for expensive pure functions like recursive algorithms, API lookups with stable results, or any computation where the same inputs always produce the same output.

```

val $fib = memoize(fn {
  val $n = _0
  $n < 2 ? $n : $fib->($n-1) + $fib->($n-2)
})
p $fib->(30) # instant (without memoize: ~1B calls)

var $fetch_user = memo(fn { fetch_json("https://api/users/#{_0}") })
$fetch_user->(42) # hits API
$fetch_user->(42) # returns cached

```

once

once wraps a function so it is called at most once. The first invocation executes the function and caches the result; all subsequent calls return the cached value without re-executing. This is ideal for lazy initialization, one-time setup, or singleton patterns.

```
val $init = once(fn { p "initializing..."
  42 })
p $init->() # prints "initializing..." → 42
p $init->() # 42 (no print – cached)
p $init->() # 42 (still cached)
```

constantly

constantly (alias **const**) returns a function that ignores all arguments and always returns the given value. Useful as a default callback, a stub in higher-order function pipelines, or anywhere a function is required but a fixed value suffices.

```
val $zero = constantly(0)
p $zero->("anything") # 0
var @defaults = map { constantly(0)->() } 1:5 # [0,0,0,0,0]
```

complement

complement (alias **compl**) wraps a predicate function and returns a new function that negates its boolean result. **complement(\&even?)->(3)** returns true. This is the functional equivalent of **!f(x)** without creating a closure.

```
val $even = fn { _0 % 2 == 0 }
var $odd = complement($even)
p $odd->(3) # 1
p $odd->(4) # 0
1:10 |> grep { complement($even)->( _) } |> e p # 1 3 5 7 9
```

juxt

juxt (juxtapose) takes multiple functions and returns a new function that calls each one with the same arguments and collects the results into an array. This is useful for computing multiple derived values from the same input in a single pass.

```
val $stats = juxt(fn { min @_ }, fn { max @_ }, fn { avg @_ })
var @r = $stats->(3, 1, 4, 1, 5)
p "@r" # 1 5 2.8
```

fnil

fnil wraps a function so that any **undef** arguments are replaced with the given defaults before the function is called. This eliminates repetitive **// \$default** patterns inside function bodies.

```
val $greet = fnil(fn { "Hello, #{_0}!" }, "World")
p $greet->(undef) # Hello, World!
p $greet->("Alice") # Hello, Alice!
```

deep_clone

deep_clone (alias **dclone**) performs a recursive deep copy of a nested data structure. Array refs, hash refs, and scalar refs are cloned recursively so that the result shares no references with the original. Modifications to the clone never affect the source. This is the stryke equivalent of JavaScript's **structuredClone** or Perl's **Storable::dclone**.

```
val $orig = {users => [{name => "Alice"}], meta => {v => 1}}
var $copy = deep_clone($orig)
$copy->{users}[0]{name} = "Bob"
p $orig->{users}[0]{name} # Alice (unchanged)
```

deep_merge

deep_merge (alias **dmerge**) recursively merges two hash references. When both sides have a hash ref for the same key, they are merged recursively; otherwise the right-hand value wins. This is the standard deep merge from **Lodash**, **Ruby's deep_merge**, and **config-file overlay patterns**. Returns a new hash ref — neither input is modified.


```

val $defaults = {db => {host => "localhost", port => 5432}, debug => 0}
var $overrides = {db => {port => 3306}, debug => 1}
var $cfg = deep_merge($defaults, $overrides)
p $cfg->{db}{host}    # localhost (from defaults)
p $cfg->{db}{port}    # 3306 (overridden)
p $cfg->{debug}       # 1 (overridden)

```

deep_equal

deep_equal (alias **deq**) performs structural equality comparison of two values, recursively descending into array refs, hash refs, and scalar refs. Returns 1 if the structures are identical, 0 otherwise. This is the stryke equivalent of Node's **assert.deepStrictEqual**, Lodash **isEqual**, or Python's **==** on nested dicts/lists.

```

p deep_equal([1, {a => 2}], [1, {a => 2}]) # 1
p deep_equal([1, {a => 2}], [1, {a => 3}]) # 0
p deq({x => [1,2]}, {x => [1,2]})          # 1

```

Operators

66 topics

->

Arrow operator — method invocation on objects/classes and dereference of refs. The single most-used operator in Perl OOP and any code touching references.

```
# Method call (object or class):
var $p = Point->new(x => 1, y => 2)
p $p->x                                # 1
p Math::Utils->compute(@args)

# Hash-ref dereference:
var $cfg = { host => "localhost", port => 5432 }
p $cfg->{host}
$cfg->{port} = 6543

# Array-ref dereference:
var $rows = [[1,2,3], [4,5,6]]
p $rows->[1][2]                        # 6 (chained - implicit ->)
var $val = $rows->[0]->[2]              # explicit form: 3

# Code-ref invocation:
var $f = fn ($x) { $x * 2 }
p $f->(21)                             # 42

# Postfix-deref (modern):
var @copy = $rows->@*                  # all elements
var %env = %ENV->%*                    # entire hash via slice
```

Chained arrow elision — between subscripts (`[...]`, `{...}`), `->` is optional: `$rows->[1][2]` $\equiv$ `$rows->[1]->[2]`. The first arrow after a scalar variable is *required*; intermediate arrows can be omitted for chained subscripting only.

See also: `->>` / `->>>` (thread-last macros — distinct operator), `bless`, `class`, `new`, `ref`.

new

new — the default constructor automatically provided for every `class` and `struct`. Called as `ClassName->new(field1 => v1, field2 => v2, ...)`. Returns a blessed object whose attribute storage is initialized from the named arguments. If the class defines a `BUILD` method, `new` runs it *after* attribute initialization but *before* returning, so `BUILD` can validate, derive fields, or attach external resources. A class never overrides `new` itself in stryke — to customize construction, define `BUILD` (post-init hook) instead.

Implementation: `vm_helper.rs:10686` — when `->new` is called and no user-defined `new` exists, the runtime dispatches to `builtin_new` which allocates the object, populates fields from the arglist, then calls `BUILD` via the C3 MRO if present.

```

class Point {
  has 'x' => (is => 'ro', default => 0)
  has 'y' => (is => 'ro', default => 0)
}
var $p = Point->new(x => 1.5, y => 2.0)
p $p->x      # 1.5

class Connection {
  has 'host' => (is => 'ro', required => 1)
  has 'port' => (is => 'ro', default => 5432)
  fn BUILD($self, $args) {
    die "host required" unless $self->host
  }
}
var $c = Connection->new(host => "db.example.com")

```

See also: [BUILD](#) (post-init hook), [DESTROY](#) (destructor), [class](#), [struct](#), [bless](#), [@ISA](#).

\$

Scalar sigil. [\\$name](#) accesses a scalar variable. [\\$ref](#) holds references. [\\$h{key}](#) accesses one hash value; [\\$arr\[idx\]](#) accesses one array element (sigil reflects the *value* being accessed, not the container). For postfix-deref: [\\$ref->\\$*](#) (in stryke contexts that accept it).

```

val $x = 42          # scalar declaration
var @arr = (1, 2, 3)
p $arr[0]            # 1 - scalar context on @arr
var %h = (k => "v")
p $h{k}              # one value
var $ref = \@arr     # reference, scalar holds it

```

See also: [@](#) (array sigil), [%](#) (hash sigil), [\](#) (reference), [->](#).

@

Array sigil. [@name](#) accesses an entire array. [@h{@keys}](#) is a hash slice (multiple values by key list). [@a\[@idx\]](#) is an array slice. [@_](#) is the implicit sub argument array. Stringified in double quotes by joining elements with [\\$"](#) (space by default).

```

val @nums = (1, 2, 3)
p scalar @nums      # 3 - length in scalar context
p "@nums"           # 1 2 3 - joined with "$"
var @slice = @arr[1..3] # array slice
var @vals = @h{qw(a b c)} # hash slice

```

See also: [\\$](#) (scalar sigil), [%](#) (hash sigil), [@_](#), [\\$"](#), postfix-deref [->@*](#).

%

Hash sigil / modulo operator — same character, two meanings disambiguated by context.

As sigil (preceding a name): [%h](#) accesses the whole hash. [%ENV](#) is the env hash, [%h = \(k1 => v1, k2 => v2\)](#) declares one, [keys %h](#) / [values %h](#) / [each %h](#) iterate. The sigil reflects the *whole hash*; one value is [\\$h{k}](#) (scalar).

As operator (between expressions): [\\$x % \\$y](#) is integer modulo (remainder); result takes the sign of the right operand.

```

val %cfg = (host => "localhost", port => 5432)
p keys %cfg      # hash iteration
p 10 % 3         # 1 - modulo
p -7 % 3         # 2 - Perl semantics: sign matches RHS

```

See also: [\\$](#) (scalar), [@](#) (array), [%=", %ENV, %SIG](#).

+

Addition / numeric coercion. [\\$x + \\$y](#) adds two numbers. Operands are coerced to numeric context — strings parse as [\d+\(\.\d+\)?](#) until a non-numeric character, so ["5 apples" + 3 == 8](#). Use [.](#) (dot) for string concatenation; using [+](#) on

intended-strings is a common bug since the strings parse to 0 silently under `no warnings`. As a unary prefix, `+EXPR` forces numeric context without changing value.

```
p 2 + 3          # 5
p "10x" + 5      # 15 - leading digits parse
p "x10" + 5      # 5 - leading non-digit → 0
var $sum = sum(1:100) # 5050
```

See also: `-`, `*`, `/`, `+=` (compound assign), `.` (string concat).

-

Subtraction / unary minus / file-test operator. Binary: `$x - $y` subtracts numbers (same coercion rules as `+`). Unary: `-EXPR` negates. Special: `-X FILE` where `X` is a single letter is a file-test operator (`-e` exists, `-f` regular file, `-d` directory, `-r` readable, `-w` writable, `-x` executable, `-s` size in bytes, `-M` age in days, etc.).

```
p 10 - 3          # 7
p -5 * 2          # -10
if (-e $path) { p "exists" }
if (-f $path && -r _) { p "readable file" } # _ reuses last stat
```

See also: `+`, `-=`, file-test reference.

*

Multiplication / glob deref. Binary: `$x * $y` multiplies numbers. In expression context (`*name`), references the typeglob for that name — rarely used directly; assigning `*alias = \&func` installs an alias for the sub.

```
p 6 * 7          # 42
var $area = $w * $h
*shortcut = \&long_function_name # sub alias
```

See also: `*=`, `**` (exponent), `x` (string/list repeat — NOT multiplication).

/

Division / regex delimiter. Binary: `$x / $y` divides as floats. Stryke uses true division — `5 / 2 == 2.5`, not 2. Use `int($x / $y)` or `intdiv($x, $y)` for integer division. In expression context, `/PATTERN/` opens a regex — context-dependent: after an operator or at statement start, `/` parses as regex; after a variable or value, it's division.

```
p 10 / 4          # 2.5
p int(10 / 4)     # 2
if ($s =~ /\d+$/) { p "all digits" }
```

See also: `*`, `%` (modulo), `/=`, regex syntax, `intdiv`.

**

Exponentiation. `$x ** $y` raises `$x` to the `$y`-th power. Floating-point throughout — `2 ** 10 == 1024`, `2 ** 0.5 == 1.414...`. Right-associative: `2 ** 3 ** 2 == 2 ** 9 == 512`. Negative bases with fractional exponents return NaN.

```
p 2 ** 10        # 1024
p 2 ** 0.5       # 1.4142135623731
p 2 ** 3 ** 2    # 512 (right-assoc)
var $sq = $r ** 2
```

See also: `**=`, `sqrt`, `exp`, `log`.

++

Increment. Postfix `$x++` returns the value and `*then*` increments; prefix `++$x` increments first and returns the new value. Works on integers, floats, and **magic strings** (`"aa"++ == "ab"`, `"az"++ == "ba"`, `"Az"++ == "Ba"`). String increment follows Perl's letter-roll rules — preserves case, rolls letters with carry into the next position.

```

val $n = 5
p $n++          # 5 (post: returns old)
p $n            # 6
p ++$n          # 7 (pre: returns new)

var $s = "Az"
$s++; p $s      # Ba - magic string increment

```

See also: `--`, atomic `++` on `varsync` shared scalars (always atomic).

--

Decrement. Postfix `$x--` returns the value and *then* decrements; prefix `--$x` decrements first. Numeric-only — there is no magic-string **decrement** in Perl. On non-numeric strings, `--` coerces to 0 first then decrements.

```

val $n = 5
p $n--          # 5
p $n            # 4
p --$n          # 3

```

See also: `++`, atomic `--` on `varsync` shared scalars.

.

String concatenation. `$x . $y` glues two strings together. Both operands are coerced to string context. For multi-arg joins prefer `join ""`, `@parts` (faster, less typing); for building large strings prefer interpolation or `sprintf`. The `.` operator does **not** insert separators — `"hello" . "world" eq "helloworld"`.

```

val $name = "world"
p "hello " . $name . "!" # hello world!
var $full = $first . " " . $last
var $line = sprintf "%-20s %d", $name, $count # often clearer

```

See also: `.=` (append assign), `x` (repeat), `join`, `sprintf`, interpolation.

x

Repeat operator. `STR x N` repeats a string N times; `(LIST) x N` repeats a list. The context of the LHS decides: scalar LHS → string repeat; list LHS (must be in parens) → list repeat. N is truncated to integer; negative or zero N produces empty.

```

p " " x 20          # =====
p "ab" x 3           # ababab
var @z = (0) x 10     # ten zeros
var @grid = ([0,0,0]) x 5 # five distinct rows (each is a new arrayref via shallow copy)

```

See also: `.` (concat), `x=` (compound), `repeat`.

==

Numeric equality. Coerces both operands to numbers and compares. Use `eq` for strings — `"10" == "10.0"` is true, but `"10" eq "10.0"` is false. Float comparison with `==` is dangerous due to rounding; for tolerances use `abs($x - $y) < $epsilon`.

```

p 5 == 5.0          # 1
p "10" == 10         # 1 (numeric coercion)
p "foo" == "bar"     # 1 (both → 0) - common bug, use eq!
if ($n == 0) { ... }

```

See also: `!=`, `<=>`, `eq` (string equality), `near_eq` builtin for float tolerance.

!=

Numeric inequality. Inverse of `==`. Use `ne` for strings.

```
p 5 != 6 # 1
while ($i != $target) { ... }
```

See also: `==`, `ne`, `<=>`.

<

Numeric less-than. Coerces to numbers. Use `lt` for strings.

```
p 3 < 5 # 1
if ($age < 18) { ... }
var @small = grep { _ < 10 } @nums
```

Stryke supports **chained comparisons** Raku-style: `1 < $x < 10` $\equiv$ `(1 < $x) && ($x < 10)`. See `chained_comparison`.

See also: `>`, `<=`, `<=>`, `lt`.

>

Numeric greater-than. Coerces to numbers. Use `gt` for strings.

```
p 7 > 5 # 1
var @big = grep { _ > 100 } @vals
```

Also the redirect operator in `open` (`open var $fh, '>', $path`) and the diamond closer (`<$fh>`).

See also: `<`, `>=`, `<=>`, `gt`, `chained_comparison`.

<=

Numeric less-than-or-equal. Use `le` for strings. Participates in chained comparisons: `0 <= $i < @arr`.

```
if (0 <= $i < scalar @arr) { p $arr[$i] }
```

See also: `<`, `>=`, `<=>`, `le`, `chained_comparison`.

>=

Numeric greater-than-or-equal. Use `ge` for strings.

```
return if $count >= $max
```

See also: `>`, `<=`, `<=>`, `ge`, `chained_comparison`.

<=>

Spaceship operator — numeric three-way compare. Returns -1, 0, or 1. The standard `sort` comparator for numeric sort.

```
val @sorted = sort { _ <=> _1 } @nums # ascending
var @desc = sort { _1 <=> _ } @nums # descending
p 5 <=> 10 # -1
p 10 <=> 10 # 0
p 15 <=> 10 # 1
```

See also: `cmp` (string version), `sort`, `$a`, `$b`.

eq

String equality. Compares operands as strings. `"10" eq "10.0"` is false. The string-context counterpart of `==`.

```
if ($cmd eq "start") { ... }
p "abc" eq "abc" # 1
p "abc" eq "ABC" # "" (case matters)
```

See also: `ne`, `cmp`, `==` (numeric), `lc/uc` for case-insensitive compare.

ne

String inequality. Inverse of `eq`.

```
next if $line ne ""
```

See also: `eq`, `cmp`, `!=` (numeric).

lt

String less-than — lexicographic comparison. Use `<` for numeric.

```
p "apple" lt "banana"          # 1
var @sorted = sort { _ lt _1 ? -1 : 1 } @words
```

See also: `gt`, `le`, `cmp`, `<` (numeric), `chained_comparison`.

le

String less-than-or-equal — lexicographic. Use `<=` for numeric. Participates in chained comparisons.

See also: `lt`, `ge`, `cmp`.

gt

String greater-than — lexicographic.

```
p "banana" gt "apple"          # 1
```

See also: `lt`, `ge`, `cmp`, `>` (numeric).

ge

String greater-than-or-equal — lexicographic. Use `>=` for numeric.

See also: `gt`, `le`, `cmp`.

cmp

String three-way compare — returns -1, 0, or 1. The standard `sort` comparator for string sort.

```
val @sorted = sort { _ cmp _1 } @words      # case-sensitive
var @ci      = sort { lc(_) cmp lc(_1) } @words # case-insensitive
p "alice" cmp "bob"                        # -1
```

See also: `<=>` (numeric three-way), `eq`, `lt`, `gt`, `sort`.

&&

Logical AND (high-precedence). Short-circuit: returns the **first falsy operand**, or the **last operand if all truthy**. Stryke (like Perl) doesn't return boolean — it returns the actual value, which makes idioms like `$default = $val && transform($val)` work.

```
if ($x > 0 && $x < 100) { ... }
var $first = $x && $y && $c      # first truthy or last falsy
```

Lower-precedence variant `and` binds looser than `=` so use it for control flow: `open $fh, $f and read_loop()`. See also: `||`, `//`, `and`, `!`.

||

Logical OR (high-precedence). Short-circuit: returns the **first truthy operand**, or the **last operand if all falsy**. The classic Perl default idiom: `var $x = $arg || "default"` — but watch out: `0`, `"`, `"0"`, and `undef` are all falsy, so `||` falls through for legitimate zero values. Use `//` (defined-or) when only `undef` should trigger the fallback.

```
val $name = $user || "guest"
var $val = $count // 0      # better: only undef triggers fallback
open $fh, $f or die $!      # `or` is the low-precedence form
```

See also: `&&`, `//`, `or`, `defined`.

//

Defined-or operator. `??` is an interchangeable stryke alias (C#/Swift spelling); `??=` likewise aliases `//=`. Returns the LHS if it is defined (even if `0` or `"`), else the RHS. Distinct from `||` which treats all falsy values as "use the fallback". Use `//` whenever you have a numeric default or want to distinguish absent values from explicit zero.

```
val $count = $opts{count} // 10      # 10 only if missing - 0 is allowed
var $tag = $row->{label} // "untagged"
return $cache{$key} // compute($key) # //= : assign only if undef
```

See also: `||`, `defined`, `//=`.

!

Logical NOT (high-precedence). Returns 1 (true) if the operand is falsy, `"` (false) if truthy. Lower-precedence variant `not` reads English-y; prefer `!` in expressions, `not` in conditions.

```
if (!$ok) { die "failed" }
return unless not @items    # avoid double-negative - prefer `unless @items`
var $inverse = !$truthy     # !! coerces to canonical 1 / ""
```

See also: `not`, `unless`, `&&`, `||`.

and

Low-precedence logical AND. Same short-circuit behaviour as `&&` but binds looser than `=`, `,`, and most other operators. Use for control-flow expressions where you'd otherwise need parens: `$ok = read_file($path) and parse(...)` — but note this assigns to `$ok` ONLY the return of `read_file` (because `=` binds tighter than `and`). For value-binding use `&&`.

```
open $fh, '<', $f and read_loop($fh) # control flow
```

See also: `&&`, `or`, `not`.

or

Low-precedence logical OR. Same short-circuit behaviour as `||` but binds looser than `=`. The canonical idiom for `open ... or die`: `open var $fh, $path or die "open: $!"`. The whole `open` expression is evaluated, then `or die` runs only if `open` returned false.

```
open val $fh, '<', $f or die "open: $!"
var $val = read_line() or last      # WRONG: assigns the read, not the fallback
var $val = read_line() || last      # WRONG too - falsy values fall through
# Use defined-or for value defaults:
var $val = read_line() // last
```

See also: `||`, `//`, `and`.

not

Low-precedence logical negation — returns true if the expression is false, and false if it is true. Functionally identical to `!` but binds looser than almost everything, so `not $x == $y` is `not($x == $y)` rather than `(!$x) == $y`. Useful for readable boolean conditions.


```

if (not defined $val) {
    p "val is undef"
}
var @missing = grep { not -e _ } @files
@missing |> e p
p not 0      # 1
p not 1      # (empty string)

```

XOR

Low-precedence logical XOR. Returns true exactly when one operand is truthy and the other is falsy. No short-circuit (both sides always evaluated). Rarely used; if you find yourself wanting `xor` consider `($x ? 1 : 0) != ($y ? 1 : 0)` for clarity.

```

if ($admin xor $guest) { p "exactly one role" }

```

See also: `&&`, `||`, `^` (bitwise xor).

=~

Regex bind — applies the right-side regex/transliteration/substitution to the left-side scalar. Returns success/match data in scalar context, captured groups in list context. Without `=~`, regex ops default to operating on `$_`.

```

if ($line =~ /^s*#/ ) { ... }           # match
var ($host) = $url =~ m{^https?://([^/]+)} # extract
$text =~ s/foo/bar/g                    # substitute
$str =~ tr/a-z/A-Z/                     # transliterate

```

See also: `!~`, `m//`, `s///`, `tr///`, named captures `((?<name>...), %+)`.

!~

Negated regex bind. True if the regex does NOT match.

```

next if $line !~ /^s*\w/                # skip lines that don't start with a word

```

See also: `=~`, `m//`.

..

Range operator. In list context, generates an inclusive integer or magic-string sequence: `1..5 == (1,2,3,4,5)`. In scalar/boolean context (rare — only in `while/if`), acts as a flip-flop state machine.

```

for (1..10) { p _ }
var @letters = ('a'..'z')           # magic string range
var @rev = reverse 1..100

```

Stryke convention: prefer `1:5` (colon-range, fully equivalent) since it doesn't visually clash with method/property access chains.

See also: `...` (three-dot range), `:` colon range, flip-flop in `while`.

...

Exclusive range / yada-yada. As an operator: `1...5` is the **exclusive** range (does not include the endpoint in some contexts) and the **flip-flop** operator with slightly different reset semantics than `..`. As a statement: `...` alone (the "yada-yada") is a placeholder that compiles fine but dies at runtime — useful for sketching APIs.

```

fn not_yet_implemented { ... }      # yada-yada - dies if called

```

See also: `..`, yada-yada placeholder.

=>

Fat comma — variant of `,` that auto-quotes the left operand if it looks like a bareword identifier. The canonical syntax for hash entries and named arguments since it self-documents which side is the key.

```
val %cfg = (
  host  => "localhost",    # "host" auto-quoted
  port  => 5432,
  debug => 1,
)
open val $fh, '<', $path or die
class->new(name => "alice", age => 30)
```

With strict syntax, `,` and `=>` are semantically equivalent EXCEPT for the auto-quoting. `1 => 2` is the same as `1 , 2`; the magic only fires on bareword LHS.

See also: `,`, hash declaration, named parameters.

`,`

Comma operator / list separator. In list context, separates list elements. In scalar context (right-to-left), evaluates each expression for side effects and returns the rightmost (like C). The fat-comma `=>` is a variant that auto-quotes bareword LHS.

```
val @x = (1, 2, 3)
var %h = (a => 1, b => 2)
for (val $i = 0, val $j = 10; $i < 5; $i++, $j--) { ... } # scalar context
```

See also: `=>`, list operators (`push`, `unshift`, ...).

`?:`

Ternary conditional — `COND ? IF_TRUE : IF_FALSE`. Right-associative for chaining. Returns one of the two branches as an expression value, unlike `if/else` which is a statement. Stryke also accepts the `if/else` block form as an expression: `var $x = do { if (cond) { 1 } else { 2 } }`.

```
val $sign = $n > 0 ? "+" : $n < 0 ? "-" : "0"
p $count == 1 ? "$count item" : "$count items"
```

Binds looser than most operators; use parens around complex conditions for readability. See also: `if/else`, `match`.

`\`

Reference operator — takes a reference to a variable, sub, or value. `\$scalar`, `\@array`, `\%hash`, `\&sub`, `\(...)`. References are scalars; deref with `$$ref` / `@$ref` / `_%$ref` / `&$ref` / `$ref->[i]` / `$ref->{k}` / `$ref->(...)`.

```
val @nums = (1, 2, 3)
var $aref = \@nums
p $$aref[0]           # 1 (sigil-deref)
p $aref->[0]          # 1 (arrow-deref - preferred)
p join ", ", @$aref   # 1,2,3

var $code = \&my_function
$code->(42)
```

Also `\(LIST)` returns a list of references — `\(@a, @b)` gives one ref per element, not a single ref to the list.

See also: `->`, `ref`, deref syntax, `bless`.

`&`

Bitwise AND (and prototype/sub-deref). On numbers, `$x & $y` computes the bitwise AND. As a sigil prefix `&name` calls a sub (in non-bareword contexts) or takes a coderef. With `\` makes a coderef: `\&sub`.

```
p 0b1100 & 0b1010      # 8 (0b1000)
var $mask = $flags & 0xFF
var $code = \&my_sub     # coderef
```

See also: `|`, `^`, `~`, `&&`, `\&` (coderef).

|

Bitwise OR. `$x | $y` ORs the bits. Also a regex alternation inside `/.../`. As a stream operator `||` is logical OR, `|>` is pipe-forward.

```
p 0b1100 | 0b1010      # 14 (0b1110)
var $flags = OPEN_READ | OPEN_NONBLOCK
```

See also: `&`, `^`, `~`, `||`, `|>`, regex alternation.

^

Bitwise XOR. `$x ^ $y` XORs the bits.

```
p 0b1100 ^ 0b1010      # 6 (0b0110)
var $toggled = $bits ^ $mask
```

See also: `&`, `|`, `~`, `xor` (logical).

~

Bitwise NOT (unary). Flips every bit; works on integers. Also the prefix of stryke thread macros (`~>`, `~p>`, `~d>`, etc.) — those are distinct multi-char operators, not `~` followed by something.

```
p ~0                    # -1 (all bits set on signed int)
p ~0xFF                 # ...higher bits set
var $inverted = ~$mask & 0xFFFF
```

See also: `&`, `|`, `^`, `!` (logical NOT), `~>` / `~>>` / `~p>` (distinct ops).

<<

Left bit-shift / heredoc. As `$x << $y`, shifts bits left by `$y` positions. As `<<TAG`, opens a heredoc (multi-line string literal terminated by `TAG` at start of line).

```
p 1 << 10                # 1024
var $kb = $bytes << 10
var $sql = <<END
SELECT *
FROM users
END
```

See also: `>>`, `<<=`, heredoc syntax.

>>

Right bit-shift / file redirect. As `$x >> $y`, shifts bits right by `$y` positions. As a mode in `open`, `>>` opens for append.

```
p 1024 >> 2              # 256
var $exit_code = $? >> 8  # extract exit code from $?
open val $fh, '>>', 'log.txt' or die
```

See also: `<<`, `>>=`, `open` modes.

=

Assignment. Returns the assigned value (so chained `$x = $y = 5` works). Context matters: `var @a = $scalar` copies the scalar's value into a one-element list; `var @a = @b` does list copy; `var $s = @a` gives the array's length in scalar context. The single most-used operator after `;`.

```
val $x = 42
var ($x, $y, $c) = (1, 2, 3)      # list-destructure
var ($first, @rest) = split /,/
($x, $y) = ($y, $x)               # swap
var $len = @items                  # scalar context - count
```

See also: `+=`, `-=`, `.=`, `//=`, `||=`, `&&=`, `my`, `our`, `local`, `state`.

`+=`

Add-assign. `$x += $y` $\equiv$ `$x = $x + $y`. Atomic on `varsync` shared scalars.

```
val $total = 0
$total += $_ for @nums
mysync $sum = 0; fan 10000 { $sum += 1 } # atomic - always 10000
```

See also: `+`, `-=`, `*=`, etc.

`-=`

Subtract-assign. `$x -= $y` $\equiv$ `$x = $x - $y`. Atomic on `varsync` scalars.

See also: `-`, `+=`.

`*=`

Multiply-assign. `$x *= $y` $\equiv$ `$x = $x * $y`.

See also: `*`, `**=`.

`/=`

Divide-assign. `$x /= $y` $\equiv$ `$x = $x / $y`.

See also: `/`.

`**=`

Power-assign. `$x **= $y` $\equiv$ `$x = $x ** $y`.

See also: `**`.

`.=`

Append-assign (string concat). `$s .= $more` $\equiv$ `$s = $s . $more` but more efficient for large-string accumulators (single buffer growth, not realloc per concat). Atomic on `varsync` scalars.

```
val $buf = ""
$buf .= $line for @lines
mysync $log = ""; fan 100 { $log .= "event\n" } # atomic
```

See also: `.`, `+=`.

`x=`

Repeat-assign (string). `$s x= 3` $\equiv$ `$s = $s x 3`.

```
val $bar = "="; $bar x= 40 # forty equals signs
```

See also: `x`.

`||=`

Or-assign. `$x ||= $default` $\equiv$ `$x = $x || $default` — assigns the default only if `$x` is falsy. Watch out for legitimate falsy values like `0` and `""`; prefer `//=` for those.

```
$opts{verbose} ||= 0
$cache{$key} ||= compute($key) # but: returns wrong thing if 0 is valid
```

See also: `||`, `||=`.

`||=`

Defined-or assign. `??=` is an interchangeable stryke alias (C#/Swift spelling). `$x ||= $default` $\equiv$ `$x = $x || $default` — assigns the default only if `$x` is `undef`. The right tool for memoization/cache-fill patterns.

```
$cache{$key} ||= expensive_compute($key)
$count{$word} ||= 0; $count{$word}++
```

See also: `||`, `||=`.

`&&=`

And-assign. `$x &&= $val` $\equiv$ `$x = $x && $val` — assigns `$val` only if `$x` is truthy. Rare.

See also: `&&`, `||=`.

`|=`

Bitwise OR-assign. `$x |= 0x80` sets a bit. Atomic on `varsync` scalars.

See also: `|`.

`&=`

Bitwise AND-assign. `$x &= 0xFF` masks to low byte.

See also: `&`.

`^=`

Bitwise XOR-assign. `$x ^= $bit` toggles bits.

See also: `^`.

`<<=`

Left-shift assign. `$x <<= 3` $\equiv$ multiply by 8.

See also: `<<`.

`>>=`

Right-shift assign. `$x >>= 3` $\equiv$ integer-divide by 8.

See also: `>>`.

stryke Extensions

19 topics

fn

Define a named or anonymous function with optional typed parameters and default values. Named functions are installed into the current package; anonymous functions (closures) capture their enclosing lexical scope. Functions are first-class values — assign them to scalars, store in arrays, pass as callbacks. The last expression evaluated is the implicit return value unless an explicit `return` is used.

```
fn dbl($x) { $x * 2 }
var $f = fn { _ * 2 }
var $add = fn ($x: Int, $y: Int) { $x + $y }

# Default parameter values:
fn greet($name = "world") { p "hello $name" }
greet()           # hello world
greet("Alice")    # hello Alice

fn range_check($x, $min = 0, $max = 100) { $min <= $x <= $max }
fn with_list(@items = (1, 2, 3)) { join "-", @items }
fn with_hash(%opts = (debug => 0)) { $opts{debug} }
```

Default values are evaluated at call time if the argument is not provided.

struct

`struct` declares a named record type with typed fields, giving stryke lightweight struct semantics similar to Rust structs or Python dataclasses. Structs support multiple construction syntaxes, default values, field mutation, user-defined methods, functional updates, and structural equality.

Declaration:

```
struct Point { x => Float, y => Float }           # typed fields
struct Point { x => Float = 0.0, y => Float = 0.0 } # with defaults
struct Pair { key, value }                       # untyped (Any)
```

Construction:

```
val $p = Point(x => 1.5, y => 2.0) # function-call with named args
var $p = Point(1.5, 2.0)          # positional (declaration order)
var $p = Point->new(x => 1.5, y => 2.0) # traditional OO style
var $p = Point()                  # uses defaults if defined
```

Field access (getter/setter):

```
p $p->x      # getter (0 args)
$p->x(3.0)    # setter (1 arg)
```

User-defined methods:

```
struct Circle {
  radius => Float,
  fn area { 3.14159 * $self->radius ** 2 }
  fn scale($factor: Float) {
    Circle(radius => $self->radius * $factor)
  }
}
var $c = Circle(radius => 5)
p $c->area      # 78.53975
p $c->scale(2)  # Circle(radius => 10)
```

Built-in methods:

```
val $q = $p->with(x => 5) # functional update - new instance
var $h = $p->to_hash      # { x => 1.5, y => 2.0 }
var @f = $p->fields       # (x, y)
var $c = $p->clone        # deep copy
```

Smart stringify:

```
p $p # Point(x => 1.5, y => 2)
```

Structural equality:

```
val $x = Point(1, 2)
var $y = Point(1, 2)
p $x == $y # 1 (compares all fields)
```

Note: field type is checked at construction and mutation; unknown field names are fatal errors.

typed

typed adds optional runtime type annotations to lexical variables and subroutine parameters. When a **typed** declaration is in effect, stryke inserts a lightweight check at assignment time that verifies the value matches the declared type (**Int**, **Str**, **Float**, **Bool**, **ArrayRef**, **HashRef**, or a user-defined **struct** name). This is especially useful for catching accidental type mismatches at function boundaries in larger programs. The annotation is purely a runtime guard — it has zero impact on pipeline performance because the check is only performed once at the point of assignment, not on every read.

```
typed val $x : Int = 42
typed val $name : Str = "hello"
typed val $pi : Float = 3.14
var $add = fn ($x: Int, $y: Int) { $x + $y }
p $add->(3, 4) # 7
```

Note: assigning a value of the wrong type raises a runtime exception immediately.

You can mix typed and untyped variables freely in the same scope, so adopting **typed** is incremental — annotate the variables that matter and leave the rest dynamic. Subroutine parameters declared with type annotations in **fn** are checked on every call, giving you contract-style validation at function boundaries without a separate assertion library.

```
typed val @nums : Int = (1, 2, 3)
typed val %cfg : Str = (host => "localhost", port => "8080")
```

match

Algebraic pattern matching (stryke extension).

```
match ($val) {
  /^d+$/ => p "number: $val",
  [1, 2, _] => p "array starting with 1,2",
  { name => $n } => p "name is $n",
  _ => p "default",
}
```

Patterns: regex, array, hash, literal, wildcard **_**. Optional **if** guard per arm.

Caveat — topic shadowing inside arms: inside each arm, **_** rebinds to the **matched** value (a regex string, an array element, a hash slot), **not** the outer function's **_** topic. If you need both the matched value and the outer topic, capture the outer one before the **match**:

```
fn process_event {
  val $e = _ # capture outer topic
  match ($e->{type}) {
    "LOGIN" => "user #{ $e->{user} } logged in", # ← $e, not _
    _ => "unknown",
  }
}
```

frozen

Declare an immutable lexical variable. `const my` and `frozen my` are interchangeable spellings; `const` reads more naturally for engineers coming from other languages.

```
const val $pi = 3.14159
# $pi = 3 # ERROR: cannot assign to frozen variable

frozen val @primes = (2, 3, 5, 7, 11)
```

fore

Side-effect-only list iterator (like `map` but void, returns item count).

```
qw(a b c) |> e p      # prints a, b, c; returns 3
1:5 |> map _ * 2 |> e p # prints 2,4,6,8,10
```

ep

`ep` — shorthand for `e { p }` (foreach + print). Iterates the list and prints each element.

```
qw(a b c) |> ep      # prints a, b, c (one per line)
filef |> sorted |> ep # print sorted file list
1:5 |> map _ * 2 |> ep # prints 2,4,6,8,10
```

p

`p` — **the stryke print-with-newline verb**. Costs one character, has the record separator built in, and is the canonical output primitive: every example, hover doc, and rosetta program in the corpus uses `p`. With multiple args, joins them with `$`, (default empty) and appends `$\` (default `\n`). Stringifies refs as `HASH(0x...)` / `ARRAY(0x...)`; use `ep` / `ddump` / `to_json` for structured dumps.

```
p "hello"      # hello\n
p 42           # 42\n
p "x=", $x, " y=", $y      # multi-arg, joined by $,
1:5 |> e p      # iterate + p - prints one per line
@xs |> ep      # shorthand for `|> e p`
p "#{ sum @arr }"      # interpolation
```

`p` is preferred over Perl 5's `print` and `say` in every stryke context:

| Form | Status | |---|---| | `p "x"` | **stryke canonical** — one char, includes newline, no flags | | `print "x\n"` | Perl 5 compat — keeps working under `--compat`, but reads as legacy | | `say "x"` | **Rejected by --no-interop**. Perl 5.10 keyword; `p` exists precisely to replace it |

`p` is NOT "an alias for `say`" — it's the **primary** spelling. `say` is the rejected Perl 5 form. Use `p`.

See also: `ep` (each-then-p), `pp` (pretty-print), `ddump` (full structure dump), `warn` (stderr equivalent of `p`).

gen

Create a generator — lazy `yield` values on demand.

```
val $g = gen { yield for 1:5 }
var ($val, $more) = @{$g->next}
```

yield

Yield a value from inside a `gen { }` generator block, suspending the generator until the consumer calls `->next` again. Each `yield` produces one element in the lazy sequence. When the block finishes without yielding, the generator signals exhaustion. This is the stryke equivalent of Python's `yield` or Rust's `Iterator::next`.


```

val $fib = gen {
  var ($x, $y) = (0, 1)
  while (1) {
    yield $x
    ($x, $y) = ($y, $x + $y)
  }
}
for (1:10) {
  var ($val) = @{$fib->next}
  p $val
}

```

trace

trace { BLOCK } — enable shared-state mutation tracing for the duration of **BLOCK**. Every read/write of a **varsync/oursync** variable (including compound ops like **++**, **+=**, **.=**) is logged to **STDERR** with the worker index, value before, and value after. Invaluable for debugging races, dropped updates, or compound-op atomicity violations.

Outside **trace**, the same operations run with zero overhead — **trace** is the only way to switch the instrumentation on.

```

mysync $counter = 0
trace {
  fan 10 { $counter++ }      # 10 workers, each prints "<w> ++ : N → N+1"
}
p $counter                  # 10

# Diagnose a missed update:
mysync %hits = ()
trace { fan 100 { $hits[rand(10)]++ } }

```

For function-call tracing (every named-fn entry/exit) use **--profile** or AOP **before/after**. For low-level VM op tracing use **--trace-ops**.

See also: **varsync**, **oursync**, **fan**, AOP **before/after**.

timer

timer { BLOCK } — measure the wall-clock elapsed time of **BLOCK** and return the duration in **milliseconds** (**f64**). The block's own return value is discarded; use **timed** if you need both. Internally **std::time::Instant::now** brackets the block, so resolution is the host's monotonic clock (sub-microsecond on modern CPUs).

```

val $ms = timer { sort_huge_list() }
printf "sort took %.2f ms\n", $ms

# Compare two implementations:
var $ta = timer { algorithm_a(@data) }
var $tb = timer { algorithm_b(@data) }
printf "speedup: %.2fx\n", $ta / $tb

```

For statistical benchmarking (warmup, min / mean / p99), use **bench** instead. For per-operation profiling use **--profile** or **trace**.

See also: **bench**, **eval_timeout**, **time**, **now_ns**.

bench

Benchmark a block N times; returns **"min/mean/p99"**.

```

val $report = bench { work() } 1000

```

eval_timeout

Run a block with a wall-clock timeout (seconds).

```

eval_timeout 5 { slow_operation() }

```

retry

Retry a block on failure.

```
retry { http_call() } times => 3, backoff => 'exponential'
```

rate_limit

`rate_limit(N, "WINDOW") { BLOCK }` — token-bucket throttle: run the body at most `N` times per `WINDOW`. Excess calls block until a token is available rather than failing — guarantees the rate without dropping work. `WINDOW` accepts `"500ms"`, `"1s"`, `"5m"`, `"1h"`, or a bare number of seconds. The throttle state is per-call-site, so multiple `rate_limit` blocks at different sites have independent buckets.

```
# Cap API calls at 10/sec - the 11th waits until a token frees up.
for val $url (@urls) {
  rate_limit(10, "1s") { fetch($url) }
}

# Long window - at most 1 nightly job per hour even under retries.
rate_limit(1, "1h") { nightly_compaction() }

# Inside a parallel iter (varsync bucket - actually shared across workers).
pfor { rate_limit(50, "1s") { ingest($_) } } @records
```

For non-blocking probe-style throttling (test if a token is available, drop instead of waiting) use `try_acquire`. For bulk burst handling use a `pchannel` of capacity `N` as a hand-rolled bucket.

See also: `every`, `retry`, `circuit_breaker`, `semaphore`.

every

`every "INTERVAL" { BLOCK }` — run `BLOCK` repeatedly with at least `INTERVAL` between iterations. `INTERVAL` accepts `"500ms"`, `"1s"`, `"5m"`, etc. Runs `*forever*` on the calling thread by default; combine with `last/die` from inside the block to exit, or with `spawn { every ... }` to run in the background.

```
# Tick the heartbeat every second.
every "1s" { p "alive at " . now }

# Background poll - quits cleanly when state.done.
spawn {
  every "500ms" {
    last if $shared->{done}
    update_dashboard()
  }
}

# Fixed-delta scheduler - actually-tick exactly every N ms even if BLOCK is slow.
every "100ms" { check_health() } # next tick = previous start + 100ms
```

For one-shot delayed execution use `sleep` + a normal call. For wall-clock-aligned ticks (every minute at :00) use `cron`.

See also: `sleep`, `rate_limit`, `eval_timeout`, `cron`.

watch

`watch PATH, fn { ... }` — register a filesystem watcher on `PATH` (file or directory) and invoke the callback whenever the path is modified. Inside the callback, `_` is the absolute path of the changed file. Blocks the calling thread; combine with `spawn` to watch in the background.

Uses the OS-native facility (`FSEvents` on macOS, `inotify` on Linux). Recursive on directories; not recursive on files. The callback fires for every event, so debounce inside the body if you only want a single rebuild after a burst.

```
# Live recompile loop
watch "src/", fn {
  next unless _ =~ /\.rs$/
  p "changed: $_"
  system "cargo", "build"
}

# Background watcher
spawn {
  watch "config.toml", fn { reload_config() }
}

# Debounce trailing edge - coalesce bursts into one rebuild every 250ms.
mysync $last = 0
watch ".", fn {
  val $now = now_ns()
  if ($now - $last > 250_000_000) {
    $last = $now
    rebuild()
  }
}
```

For parallel multi-path watching use `par_watch`. For poll-based watching that doesn't need OS support use `every "1s" { stat ... }`.

See also: `par_watch`, `every`, `spawn`.

capture

Run a command and capture structured output. Returns an object with `->stdout`, `->stderr`, `->exitcode`, and `->failed` methods. When piped to `lines` or `words`, stdout is auto-extracted so you can chain directly.

```
val $r = capture("ls -la")
p $r->stdout, $r->stderr, $r->exitcode
capture("ps aux") |> lines |> map { columns } |> tbl |> p
```

Data & Serialization

45 topics

json_encode

`json_encode` serializes any stryke data structure — hashrefs, arrayrefs, nested combinations, numbers, strings, booleans, and undef — into a compact JSON string. Native bare-name builtin backed by a Rust serializer; no module to import. The output is always valid UTF-8 JSON suitable for writing to files, sending over HTTP, or piping to other tools. Use `json_decode` to round-trip back.

```
val %cfg = (debug => 1, paths => ["/tmp", "/var"])
var $j = json_encode(\%cfg)
p $j    # {"debug":1,"paths":["/tmp","/var"]}
$j |> spurt "/tmp/cfg.json"$1
```

Note: undef becomes JSON `'null'`; booleans serialize as `'true'/'false'`.

json_decode

`json_decode` parses a JSON string and returns the corresponding Perl data structure — hashrefs for objects, arrayrefs for arrays, and native scalars for strings/numbers/booleans. It is strict by default: malformed JSON raises an exception rather than returning partial data. This makes it safe to use in pipelines where corrupt input should halt processing. The Rust parser underneath handles large documents efficiently and supports full Unicode.

```
var $data = json_decode('{"name":"stryke","ver":1}')
p $data->{name}    # stryke
slurp("data.json") |> json_decode |> dd$1
```

Note: JSON `'null'` becomes Perl `'undef'`; trailing commas and comments are not allowed.

json_jq

`json_jq` applies a jq-style query expression to a stryke data structure and returns the matched value. This brings the power of the `jq` command-line JSON processor directly into stryke without shelling out. Dot notation traverses hash keys, bracket notation indexes arrays, and nested paths are separated by dots. It is ideal for extracting deeply nested values from API responses or config files without chains of hash dereferences.

```
val $data = json_decode(slurp "api.json")
p json_jq($data, ".results[0].name")
var $cfg = rj "config.json"
p json_jq($cfg, ".database.host") # deep extract
fetch_json("https://api.example.com/users") |> json_jq(".data[0].email") |> p
```

to_json

`to_json` (alias `tj`) converts a stryke data structure into a JSON string, functioning as a convenient shorthand for `json_encode`. It accepts hashrefs, arrayrefs, scalars, and nested combinations, producing compact JSON output suitable for APIs, config files, or inter-process communication. The alias `tj` is particularly useful at the end of a pipeline to serialize the final result. The Rust-backed serializer handles large structures efficiently and always produces valid UTF-8.

```
val %user = (name => "Bob", age => 30)
p tj \%user    # {"age":30,"name":"Bob"}
var @items = map { {id => _, val => _ * 2} } 1:3
p tj \@items   # [{"id":1,"val":2},{ "id":2,"val":4},{ "id":3,"val":6}]
@data |> tj |> spurt "out.json"
```

to_csv

`to_csv` (alias `tc`) serializes a list of hashrefs or arrayrefs into a CSV-formatted string, complete with a header row derived from hash keys when given hashrefs. This is the fastest way to produce spreadsheet-ready output from structured data. Fields containing commas, quotes, or newlines are automatically escaped according to RFC 4180. The alias `tc` keeps one-liners terse when piping query results or API responses straight to CSV.

```
val @rows = ({name => "Alice", age => 30}, {name => "Bob", age => 25})
p tc \@rows    # name,age\nAlice,30\nBob,25
tc(\@rows) |> spurt "people.csv"
var @grid = ([1, 2, 3], [4, 5, 6])
p tc \@grid    # 1,2,3\n4,5,6
```

to_toml

`to_toml` (alias `tt`) serializes a stryke hashref into a TOML-formatted string. TOML is a popular configuration format that maps cleanly to hash structures, making `tt` ideal for generating config files programmatically. Nested hashes become TOML sections, arrays become TOML arrays, and scalar values are serialized with their natural types. The output is always valid TOML that can be parsed back with `toml_decode`.

```
val %cfg = (database => {host => "localhost", port => 5432}, debug => 1)
p tt \%cfg
# [database]
# host = "localhost"
# port = 5432
# debug = 1
tt(\%cfg) |> spurt "config.toml"
```

to_yaml

`to_yaml` (alias `ty`) serializes a stryke data structure into a YAML-formatted string. YAML is widely used for configuration and data exchange where human readability matters. Nested structures are represented with indentation, arrays with leading dashes, and strings are quoted only when necessary. The output is valid YAML 1.2 that round-trips cleanly through `yaml_decode`. The alias `ty` is convenient for quick inspection of complex data.

```
val %app = (name => "myapp", deps => ["tokio", "serde"], version => "1.0")
p ty \%app
# name: myapp
# version: "1.0"
# deps:
#   - tokio
#   - serde
ty(\%app) |> spurt "app.yaml"
```

to_xml

`to_xml` (alias `tx`) serializes a stryke data structure into an XML string. Hash keys become element names, array values become repeated child elements, and scalar values become text content. This is useful for generating XML payloads for SOAP APIs, RSS feeds, or configuration files that require XML format. The output is well-formed XML that can be parsed back with `xml_decode`.

```
val %doc = (root => {title => "Hello", items => ["a", "b", "c"]})
p tx \%doc
# <root><title>Hello</title><items>a</items><items>b</items><items>c</items></root>
tx(\%doc) |> spurt "doc.xml"
```

to_html

`to_html` (alias `th`) serializes a stryke data structure into a self-contained HTML document with cyberpunk styling (dark background, neon cyan/magenta accents, monospace fonts). Arrays of hashrefs render as full tables with headers, plain arrays as bullet lists, single hashes as key-value tables, and scalars as styled text blocks. Pipe to a file and open in a browser for instant data visualization.

```
fr |> map +{name => _, size => format_bytes(size)} |> th |> to_file("report.html")
var @rows = ({name => "Alice", age => 30}, {name => "Bob", age => 25})
@rows |> th |> to_file("people.html")
th({host => "localhost", port => 5432}) |> p
```

to_markdown

`to_markdown` (aliases `to_md`, `tmd`) serializes a stryke data structure into Markdown text. Arrays of hashrefs render as GFM tables with headers and separator rows, plain arrays as bullet lists, single hashes as 2-column key-value tables, and scalars as plain text. The output is valid GitHub-Flavored Markdown suitable for README files, issue comments, or any Markdown renderer.

```
fr |> map +{name => _, size => format_bytes(size)} |> tmd |> to_file("report.md")
var @rows = ({name => "Alice", age => 30}, {name => "Bob", age => 25})
@rows |> tmd |> p
# | name | age |
# | --- | --- |
# | Alice | 30 |
# | Bob | 25 |
```

from_json

`from_json` (alias `fj`) parses a JSON string into a stryke data structure (hashref, arrayref, scalar). This is the inverse of `to_json` and the idiomatic way to decode JSON responses from APIs, config files, or inter-process communication. The Rust-backed parser handles large documents efficiently.

```
val $json = '{"name": "Alice", "age": 30}'
var $data = from_json($json)
p $data->{name} # Alice
var $arr = fj '[1, 2, {"x": 3}]'
p $arr->[2]{x} # 3
slurp("config.json") |> fj
```

from_yaml

`from_yaml` (alias `fy`) parses a YAML string into a stryke data structure. YAML is commonly used for configuration files due to its human-readable syntax. Handles scalars, arrays, hashes, and nested combinations. The parser supports common YAML features like multi-line strings and bare unquoted keys.

```
val $yaml = "name: Alice\nage: 30"
var $data = from_yaml($yaml)
p $data->{name} # Alice
slurp("config.yaml") |> fy |> p _->{database}{host}
```

from_toml

`from_toml` (alias `ftoml`) parses a TOML string into a stryke hashref. TOML is popular for Rust, Python, and other config files due to its explicit syntax. Sections become nested hashes, arrays stay arrays, and typed values (integers, floats, strings, booleans) are preserved.

```
val $toml = "[package]\nname = \"myapp\"\nversion = \"1.0\""
var $data = from_toml($toml)
p $data->{package}{name} # myapp
slurp("Cargo.toml") |> ftoml |> p _->{package}{version}
```

from_xml

`from_xml` (alias `fx`) parses an XML string into a stryke hashref. Element names become hash keys, text content becomes string values, nested elements become nested hashes. The parser handles the XML declaration and basic structures.

```
val $xml = '<root><name>Alice</name><age>30</age></root>'
var $data = from_xml($xml)
p $data->{root}{name} # Alice
slurp("config.xml") |> fx |> p _->{config}{database}{host}
```

from_csv

`from_csv` (alias `fcsv`) parses a CSV string into an arrayref of hashrefs, treating the first line as headers. This is the inverse of `to_csv` and handles quoted fields containing commas according to RFC 4180.

```
val $csv = "name,age\nAlice,30\nBob,25"
var $rows = from_csv($csv)
p $rows->[0]{name} # Alice
p $rows->[1]{age} # 25
slurp("data.csv") |> fcsv |> grep { _->{age} > 25 }
```

xopen

`xopen` (alias `xo`) opens a file or URL with the system default handler — `open` on macOS, `xdg-open` on Linux, `start` on Windows. Returns the path unchanged so it can sit transparently in a pipeline. This is the missing link between generating output files and viewing them: pipe the path through `xopen` and the OS opens it in the appropriate app (browser for HTML, viewer for PDF, editor for text).

```
fr |> map +{name => _, size => format_bytes(size)} |> th |> to_file("report.html") |> xopen
xopen "https://example.com"
xopen "output.pdf"
```

clip

`clip` (aliases `clipboard`, `pbcopy`) copies text to the system clipboard (`pbcopy` on macOS, `xclip/xsel` on Linux). Returns the text unchanged for pipeline chaining. This is the missing link between generating output and sharing it — pipe any serializer's output through `clip` and paste directly into Slack, GitHub, docs, or email.

```
fr |> map +{name => _, size => format_bytes(size)} |> tmd |> clip # markdown table → clipboard
qw(a b c) |> join "," |> clip |> p # also prints
"some text" |> clip # just copy
```

paste

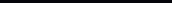
`paste` (alias `pbpaste`) reads text from the system clipboard (`pbpaste` on macOS, `xclip/xsel` on Linux). Returns the clipboard contents as a string, ready for pipeline processing.

```
p paste # print clipboard contents
paste |> lines |> grep /error/i |> p # search clipboard for errors
paste |> wc |> p # word count of clipboard
```

to_table

`to_table` (aliases `table`, `tbl`) renders data as a plain-text column-aligned table with Unicode box-drawing borders. Arrays of hashrefs render as full tables with headers, plain arrays as numbered rows, single hashes as key-value tables sorted by numeric value descending (ties by key), matching `freq` → `tbl` expectations. The output is fixed-width text suitable for terminal display, log files, or any monospace context.

sparkline

sparkline (alias **spark**) renders a list of numbers as a compact Unicode sparkline string using block characters (). Each value maps to a bar height proportional to the min/max range. Ideal for inline data visualization in terminal output, dashboards, or log summaries.

```
(3,7,1,9,4,2,8,5) |> spark |> p # 
@daily_sales |> spark |> p # quick trend line
fr |> map { size } |> spark |> p # file size distribution
```

bar_chart

bar_chart (alias **bars**) renders a hashref as a colored horizontal bar chart in the terminal. Each key becomes a label, each value a proportional bar. Colors cycle automatically. Pairs naturally with **freq** for instant word/event counting visualization.

```
qwl(a b a c a b) |> freq |> bars |> p  
# a | ██████████ 3  
# b | ████████ 2  
# c | █████ 1  
bars({cpu => 73, mem => 45, disk => 91}) |> p
```

flame

flame (alias **flamechart**) renders a hierarchical hashref as a terminal flamechart with colored stacked bars. Nested hashes become child rows; widths are proportional to leaf values (weights). Useful for visualizing call stacks, cost breakdowns, or any tree-structured data.

histo

histo renders a vertical histogram in the terminal. Given a hashref (label → count), it draws colored vertical bars. Given a flat array of numbers, it auto-bins into 10 buckets and shows the distribution. Pairs with **freq** for categorical data.

gauge

gauge renders a single value as a horizontal gauge bar with color coding (green $\geq 80\%$, yellow $\geq 50\%$, magenta $\geq 25\%$, red below). Accepts a fraction (0.0–1.0) or a value with max: `gauge(45, 100)`.

```
p_gauge(0.73)      # [██████████░░░░░░░░░░░░░░░░] 73%
p_gauge(45, 100)   # [██████████░░░░░░░░░░░░░░░░] 45%
```

spinner

spinner shows an animated braille spinner on stderr while a block executes, then clears the spinner and returns the block's result. Useful for long-running one-liners where you want visual feedback without polluting stdout.

```
val $r = spinner "loading" { fetch_json("https://api.example.com/data") }
spinner { sleep 2
  42 } # default message "working"
```

csv_read

csv_read (alias **cr**) reads CSV data from a file path or string and returns an array of arrayrefs, where each inner arrayref represents one row. The parser handles RFC 4180 CSV correctly — quoted fields with embedded commas, newlines inside quotes, and escaped double quotes are all supported. The first row is treated as data (not a header) unless you process it separately. This is the fastest way to ingest tabular data in stryke.

```
val @rows = csv_read "data.csv"
p $rows[0] # first row as arrayref
@rows |> e { p _->[0] } # print first column
var @inline = csv_read "a,b,c\n1,2,3\n4,5,6"
p scalar @inline # 3 rows (including header)
```

csv_write

csv_write (alias **cw**) serializes an array of arrayrefs into a CSV-formatted string. Each inner arrayref becomes one row, and fields are automatically quoted when they contain commas, quotes, or newlines. This is the complement of **csv_read** and produces output conforming to RFC 4180. Use it to generate CSV files from computed data, export database query results, or prepare data for spreadsheet import.

```
val @data = [{"name", "age"}, ["Alice", 30], ["Bob", 25]]
csv_write(@data) |> spurt "people.csv"
var @report = map { [_->{id}, _->{score}] } @results
p csv_write @report
```

dataframe

dataframe (alias **df**) creates a columnar dataframe from tabular data, providing a structured way to work with rows and columns. You can construct a dataframe from an array of hashrefs, an array of arrayrefs with a header row, or from a CSV file. The dataframe supports column selection, filtering, sorting, and aggregation operations. This is stryke's answer to Python's pandas DataFrame — lightweight but sufficient for common data manipulation tasks.

```
val $df = df [{name => "Alice", age => 30}, {name => "Bob", age => 25}]
p $df
var $df2 = df csv_read "data.csv"
var @ages = $df->{age}
p @ages # 30, 25
```

sqlite

sqlite (alias **sql**) executes a SQL statement against an SQLite database file and returns the results. For SELECT queries, it returns an array of hashrefs where each hashref represents one row with column names as keys. For INSERT, UPDATE, and DELETE statements, it returns the number of affected rows. Bind parameters prevent SQL injection and handle quoting automatically. The database file is created if it does not exist, making **sqlite** a zero-setup embedded database.

```
sqlite("app.db", "CREATE TABLE users (name TEXT, age INT)")
sqlite("app.db", "INSERT INTO users VALUES (?, ?)", "Alice", 30)
var @rows = sqlite("app.db", "SELECT * FROM users WHERE age > ?", 20)
@rows |> e { p _->{name} }
sqlite("app.db", "SELECT count(*) as n FROM users") |> dd
```

digits

digits (alias **dg**) extracts all digit characters from a string and returns them as a list. This is a quick way to pull numbers out of mixed text without writing a regex. Pairs naturally with **join** to reconstruct the numeric string, or **freq** for digit distribution analysis.

```
p join "", digits("phone: 555-1234") # 5551234
"abc123def456" |> digits |> cnt |> p # 6
cat("log.txt") |> digits |> freq |> bars |> p
```

letters

letters (alias **lts**) extracts all alphabetic characters from a string and returns them as a list. Filters out digits, punctuation, whitespace — keeps only letters.

```
p join "", letters("h3ll0 w0rld!") # hllwrld
"abc123DEF" |> letters |> cnt |> p # 6
```

sentences

sentences (alias **sents**) splits text on sentence boundaries (. ! ? followed by whitespace or end of string). Returns a list of trimmed sentences. Useful for NLP pipelines, text analysis, or splitting prose into processable units.

```
"Hello world. How are you? Fine!" |> sentences |> e p
# Hello world.
# How are you?
# Fine!
cat("essay.txt") |> sentences |> cnt |> p # sentence count
```

paragraphs

paragraphs (alias **paras**) splits text on blank lines (one or more consecutive newlines). Returns a list of trimmed paragraph strings. Useful for processing structured documents, README files, or any text with paragraph breaks.

```
cat("README.md") |> paragraphs |> cnt |> p # paragraph count
cat("essay.txt") |> paragraphs |> map { sentences |> cnt } |> spark |> p # sentences per paragraph
```

sections

sections (alias **sects**) splits text on markdown-style headers (# ..., ## ...) or lines of `===/---`. Returns a list of arrayrefs [**heading**, **body**] where each section's heading and body are separated. Useful for parsing structured documents.

```
cat("README.md") |> sections |> cnt |> p # section count
cat("README.md") |> sections |> map { _->[0] } |> e p # list all headings
```

numbers

numbers (alias **nums**) extracts all numbers (integers and floats, including negatives) from a string and returns them as numeric values. Unlike **digits** which returns individual digit characters, **numbers** returns actual parsed values. Useful for pulling measurements, scores, or IDs out of mixed text.

```
p join ", ", numbers("temp 98.6F, -20C, ver 3") # 98.6,-20,3
cat("log.txt") |> numbers |> avg |> p # average of all numbers in file
"price: $12.99 qty: 5" |> numbers |> e p # 12.99, 5
```

graphemes

graphemes (alias **grs**) splits a string into Unicode grapheme clusters. Unlike **chars** which splits on code points, **graphemes** keeps combining marks and emoji sequences together as single visual units. `"café\u{0301}"` gives 4 graphemes (not 5 code points). Essential for correct text processing of accented characters, emoji, and non-Latin scripts.

```
p cnt graphemes("café\u{0301}") # 4 (not 5)
graphemes("hello") |> e p # h, e, l, l, o
```

columns

`columns` (alias `cols`) splits fixed-width columnar text into fields. Without a `widths` argument, it auto-detects columns by splitting on runs of 2+ whitespace (ideal for `ps aux`, `ls -l`, `df` output). With an arrayref of widths, it splits at exact fixed positions. Pairs with `lines` for processing tabular command output.

```
val @fields = columns("USER  PID  %CPU")           # auto-detect: [USER, PID, %CPU]
var @fixed = columns("John  Doe   30", [7, 7, 3])   # fixed-width: [John, Doe, 30]
capture("ps aux") |> lines |> map { columns } |> tbl |> p
```

stringify

`stringify` (alias `str`) converts any stryke value — scalars, array refs, hash refs, nested structures, undef — into a string representation that is a valid stryke literal. The output is designed for round-tripping: you can `eval` the returned string to reconstruct the original data structure. This makes it ideal for serializing state to a file in a Perl-native format, generating code fragments, or building reproducible test fixtures. Unlike `dd`, which targets human readability, `str` prioritizes parseability.

```
val $s = str {a => 1, b => [2, 3]}
p $s           # {a => 1, b => [2, 3]}
var $copy = eval $s # round-trip back to hashref
var @list = (1, "hello", undef)
p str \@list     # [1, "hello", undef]
```

Note: references are serialized recursively; circular references will cause infinite recursion.

ddump

`ddump` (alias `dd`) pretty-prints any stryke data structure to stderr in a human-readable, indented format. Native bare-name builtin — no module to import. It is the go-to tool for quick debugging — drop a `dd` call anywhere in a pipeline to inspect intermediate values without disrupting the data flow. The output is colorized when stderr is a terminal. Unlike `str`, the output is not meant for `eval` round-tripping; it prioritizes clarity over parseability. `dd` returns its argument unchanged, so it can be inserted into pipelines transparently.

```
val %h = (name => "Alice", scores => [98, 87, 95])
dd \%h           # pretty-prints to stderr
var @result = @data |> dd |> grep { _->{active} } |> dd
dd [1, {a => 2}, [3, 4]] # nested structure
```

Note: `dd` writes to stderr, not stdout, so it never contaminates pipeline output.

toml_decode

`toml_decode` (alias `td`) parses a TOML-formatted string and returns the corresponding stryke hash structure. TOML sections become nested hashrefs, arrays map to arrayrefs, and typed scalars (integers, floats, booleans, datetime strings) are preserved. This is useful for reading configuration files, parsing Cargo.toml manifests, or processing any TOML input. Malformed TOML raises an exception.

```
val $cfg = toml_decode(slurp "config.toml")
p $cfg->{database}{host} # localhost
var $cargo = toml_decode(slurp "Cargo.toml")
p $cargo->{package}{version}
slurp("settings.toml") |> toml_decode |> dd
```

toml_encode

`toml_encode` (alias `te`) serializes a stryke hashref into a valid TOML string. Nested hashes become TOML sections with `[section]` headers, arrays become TOML arrays, and scalars are serialized with appropriate quoting. This is the inverse of `toml_decode` and is useful for generating or updating configuration files programmatically. The output is human-readable and can be written directly to a `.toml` file.

```
val %cfg = (server => {host => "0.0.0.0", port => 8080}, debug => 0)
toml_encode(\%cfg) |> spurt "server.toml"
var $round = toml_decode(toml_encode(\%cfg))
p $round->{server}{port} # 8080
```

yaml_decode

`yaml_decode` (alias `yd`) parses a YAML string and returns the corresponding stryke data structure. Mappings become hashrefs, sequences become arrayrefs, and scalars are coerced to their natural Perl types. This handles YAML 1.2 including multi-document streams, anchors/aliases, and flow notation. It is the go-to function for reading YAML configuration files, Kubernetes manifests, or CI pipeline definitions. Invalid YAML raises an exception.

```
val $cfg = yaml_decode(slurp "docker-compose.yml")
p $cfg->{services}{web}{image}
var $ci = yaml_decode(slurp ".github/workflows/ci.yml")
dd $ci->{jobs}
slurp("values.yaml") |> yaml_decode |> json_encode |> p
```

yaml_encode

`yaml_encode` (alias `ye`) serializes a stryke data structure into a valid YAML string. Hashes become YAML mappings, arrays become sequences with dash prefixes, and scalars are quoted only when necessary for disambiguation. The output is human-readable and suitable for writing config files, generating Kubernetes resources, or producing YAML-based API payloads. It is the inverse of `yaml_decode`.

```
val %svc = (name => "api", replicas => 3, ports => [80, 443])
yaml_encode(\%svc) |> spurt "service.yaml"
var $round = yaml_decode(yaml_encode(\%svc))
p $round->{replicas} # 3
rj("config.json") |> yaml_encode |> p # JSON to YAML
```

xml_decode

`xml_decode` (alias `xd`) parses an XML string and returns a stryke data structure. Elements become hash keys, text content becomes scalar values, repeated child elements become arrayrefs, and attributes are accessible through a conventions-based mapping. This is useful for consuming SOAP responses, RSS feeds, SVG files, or any XML-based API. Malformed XML raises an exception rather than returning partial data.

```
val $doc = xml_decode('<root><name>stryke</name><ver>1</ver></root>')
p $doc->{root}{name} # stryke
slurp("feed.xml") |> xml_decode |> dd
var $svg = xml_decode(fetch("https://example.com/image.svg"))
p $svg->{svg}
```

xml_encode

`xml_encode` (alias `xe`) serializes a stryke data structure into a well-formed XML string. Hash keys become element names, scalar values become text content, and arrayrefs become repeated sibling elements. This is useful for generating XML payloads for SOAP APIs, creating RSS or Atom feeds, or producing configuration files in XML format. The output is the inverse of `xml_decode` and round-trips cleanly.

```
val %data = (root => {title => "Test", items => [{id => 1}, {id => 2}]})
p xml_encode \%data
xml_encode(\%data) |> spurt "output.xml"
var $payload = xml_encode {request => {action => "query", id => 42}}
http_request(method => "POST", url => $endpoint, body => $payload)
```

HTTP & Networking

23 topics

fetch

`fetch` (alias `ft`) performs a blocking HTTP GET request to the given URL and returns the response body as a string. It follows redirects automatically and raises an exception on network errors or non-2xx status codes. This is the simplest way to retrieve content from the web in stryke — no need to configure a client or parse response objects. For JSON APIs, prefer `fetch_json` which additionally decodes the response.

```
val $html = fetch "https://example.com"
p $html
var $ip = fetch "https://api.ipify.org"
p "My IP: $ip"
fetch("https://example.com/data.txt") |> spurt "local.txt"$1

Note: for POST, PUT, DELETE, or custom headers, use `http_request` instead.
```

fetch_json

`fetch_json` (alias `ftj`) performs a blocking HTTP GET request and automatically decodes the JSON response body into a stryke data structure. This combines `fetch` and `json_decode` into a single call, which is the common case when consuming REST APIs. It raises an exception if the response is not valid JSON or the request fails. The returned value is typically a hashref or arrayref ready for immediate use.

```
var $data = fetch_json "https://api.github.com/repos/stryke/stryke"
p $data->{stargazers_count}
fetch_json("https://jsonplaceholder.typicode.com/todos") |> e { p _->{title} }
var $weather = fetch_json "https://wttr.in/?format=j1"
p $weather->{current_condition}[0]{temp_C}
```

fetch_async

`fetch_async` (alias `fta`) initiates a non-blocking HTTP GET request and returns a task handle that can be awaited later. This allows you to fire off multiple HTTP requests concurrently and wait for them all to complete, dramatically reducing total latency when fetching from several endpoints. The task resolves to the response body as a string, just like `fetch`. Use this when you need to parallelize network I/O without threads.

```
val $t1 = fetch_async "https://api.example.com/users"
var $t2 = fetch_async "https://api.example.com/posts"
var $users = await $t1
var $posts = await $t2
p "Got users and posts concurrently"
```

fetch_async_json

`fetch_async_json` (alias `ftaj`) initiates a non-blocking HTTP GET request that automatically decodes the JSON response when awaited. This combines `fetch_async` with `json_decode`, making it the ideal choice for concurrent API calls where every response is JSON. The task resolves to the decoded stryke data structure — a hashref, arrayref, or scalar depending on the JSON content.

```
val @urls = map { "https://api.example.com/item/$_" } 1:10
var @tasks = map fetch_async_json @urls
var @results = map await @tasks
@results |> e { p _->{name} }
```

http_request

`http_request` (alias `hr`) performs a fully configurable HTTP request with control over method, headers, body, and timeout. Unlike `fetch` which is limited to GET, `http_request` supports POST, PUT, PATCH, DELETE, and any other HTTP method. Pass named parameters for configuration. The response is returned as a hashref containing `status`, `headers`, and `body` fields, giving you full access to the HTTP response. This is the right tool when you need to send data, set authentication headers, or inspect status codes.

```
val $res = http_request(method => "POST", url => "https://api.example.com/users",
  headers => {"Content-Type" => "application/json", Authorization => "Bearer $token"},
  body => tj {name => "Alice"})
p $res->{status} # 201
var $data = json_decode $res->{body}
var $del = http_request(method => "DELETE", url => "https://api.example.com/users/42")
```

par_fetch

`par_fetch @urls` — fetch a list of URLs in parallel using async HTTP, returning an array of response bodies in input order.

Under the hood, `stryke` spawns concurrent HTTP GET requests across a connection pool with keep-alive and automatic retry on transient failures (5xx, timeouts). The degree of parallelism is bounded by the connection pool size (default 64) to avoid overwhelming the target server. For heterogeneous HTTP methods or custom headers, use `http_request` inside `pmap` instead. `par_fetch` is the right tool when you have a homogeneous list of URLs and just need the bodies — it handles connection reuse, DNS caching, and TLS session resumption automatically for maximum throughput.

```
val @bodies = par_fetch @urls

# Fetch and decode JSON in one shot:
var @data = par_fetch(@api_urls) |> map json_decode

# Download pages with progress:
var @pages = par_fetch @urls, progress => 1
```

serve

Start a blocking HTTP server.

```
serve 8080, fn ($req) {
  # $req = { method, path, query, headers, body, peer }
  { status => 200, body => "hello" }
}

serve 3000, fn ($req) {
  val $data = { name => "stryke", version => "0.4" }
  { status => 200, body => json_encode($data) }
}, { workers => 8 }$1

Handler returns: hashref `{ status, body, headers }`, string (200 OK), or undef (404).
JSON content-type auto-detected when body starts with `{` or `[`.
```

socket

Create a network socket with the specified domain, type, and protocol. The socket handle is stored in the first argument and can then be used with `bind`, `connect`, `send`, `recv`, and other socket operations. Domain constants include `AF_INET` (IPv4) and `AF_INET6` (IPv6); type constants include `SOCK_STREAM` (TCP) and `SOCK_DGRAM` (UDP).

```
socket(val $sock, AF_INET, SOCK_STREAM, 0)
var $addr = sockaddr_in(8080, inet_aton("127.0.0.1"))
connect($sock, $addr)
send($sock, "GET / HTTP/1.0\r\n\r\n", 0)
```

bind

Bind a socket to a local address so it can accept connections or receive datagrams on that address. The address is a packed `sockaddr_in` or `sockaddr_in6` structure. Binding is required before calling `listen` on a server socket. Dies if the address

is already in use unless `SO_REUSEADDR` is set.

```
socket(val $srv, AF_INET, SOCK_STREAM, 0)
setsockopt($srv, SOL_SOCKET, SO_REUSEADDR, 1)
bind($srv, sockaddr_in(8080, INADDR_ANY)) or die "bind: $!"
listen($srv, 5)
```

listen

Mark a bound socket as passive, ready to accept incoming connections. The backlog argument specifies the maximum number of pending connections the OS will queue before refusing new ones. This is only meaningful for stream (TCP) sockets. Call `accept` in a loop after `listen` to handle clients.

```
socket(val $srv, AF_INET, SOCK_STREAM, 0)
bind($srv, sockaddr_in(9000, INADDR_ANY))
listen($srv, 128) or die "listen: $!"
while (accept(val $client, $srv)) {
    send($client, "hello\n", 0)
}
```

accept

Accept a pending connection on a listening socket and return a new connected socket handle. The new handle is used for communication with that specific client while the original listening socket continues accepting others. Returns the packed remote address on success, false on failure.

```
listen($srv, 5)
while (val $remote = accept(val $client, $srv)) {
    var ($port, $ip) = sockaddr_in($remote)
    p "connection from " . inet_ntoa($ip) . " :$port"
    send($client, "welcome\n", 0)
}
```

connect

Initiate a connection from a socket to a remote address. For TCP sockets this performs the three-way handshake; for UDP it sets the default destination so subsequent `send` calls do not need an address. Dies or returns false if the connection is refused or times out.

```
socket(val $sock, AF_INET, SOCK_STREAM, 0)
var $addr = sockaddr_in(80, inet_aton("example.com"))
connect($sock, $addr) or die "connect: $!"
send($sock, "GET / HTTP/1.0\r\n\r\n", 0)
recv($sock, val $buf, 4096, 0)
p $buf
```

send

Send data through a connected socket. The flags argument controls behavior — use `0` for normal sends. For UDP sockets you can supply a destination address as a fourth argument to send to a specific peer without calling `connect` first. Returns the number of bytes sent, or `undef` on error.

```
send($sock, "hello world\n", 0)
var $n = send($sock, $payload, 0)
p "sent $n bytes"
# UDP to specific peer
send($udp, $msg, 0, sockaddr_in(5000, inet_aton("10.0.0.1")))
```

recv

Receive data from a socket into a buffer. The length argument specifies the maximum number of bytes to read. For stream sockets a short read is normal — loop until you have all expected data. For datagram sockets each call returns exactly one datagram. Returns the sender address for UDP, or empty string for TCP.

```
recv($sock, val $buf, 4096, 0) or die "recv: $!"
p $buf
var $data = ""
while (recv($sock, val $chunk, 8192, 0) && length($chunk)) {
    $data .= $chunk
}
```

shutdown

Shut down part or all of a socket connection without closing the file descriptor. The `how` argument controls direction: `0` stops reading, `1` stops writing (sends FIN to peer), `2` stops both. This is useful for signaling end-of-data to the remote side while still reading its response.

```
send($sock, $request, 0)
shutdown($sock, 1) # done writing
recv($sock, val $resp, 65536, 0) # still read response
shutdown($sock, 2) # fully close
```

setsockopt

Set an option on a socket at the specified protocol level. Common uses include enabling `SO_REUSEADDR` to allow immediate rebinding after a server restart, setting `TCP_NODELAY` to disable Nagle's algorithm, or adjusting buffer sizes. The value is typically a packed integer.

```
setsockopt($srv, SOL_SOCKET, SO_REUSEADDR, 1)
setsockopt($sock, IPPROTO_TCP, TCP_NODELAY, 1)
setsockopt($sock, SOL_SOCKET, SO_RCVBUF, pack("I", 262144))
```

getsockopt

Retrieve the current value of a socket option at the specified protocol level. Returns the option value as a packed binary string — use `unpack` to interpret it. Useful for inspecting buffer sizes, checking whether `SO_REUSEADDR` is set, or reading OS-assigned values.

```
val $val = getsockopt($sock, SOL_SOCKET, SO_RCVBUF)
p unpack("I", $val) # e.g. 131072
var $reuse = getsockopt($srv, SOL_SOCKET, SO_REUSEADDR)
p unpack("I", $reuse) # 1 or 0
```

getsockname

Return the packed socket address of the local end of a socket. This is useful when the socket was bound to `INADDR_ANY` or port `0` (OS-assigned) and you need to discover the actual address and port the OS chose. Unpack the result with `sockaddr_in`.

```
bind($sock, sockaddr_in(0, INADDR_ANY)) # OS picks port
var $local = getsockname($sock)
var ($port, $ip) = sockaddr_in($local)
p "listening on port $port"
```

getpeername

Return the packed socket address of the remote end of a connected socket. Use `sockaddr_in` or `sockaddr_in6` to unpack it into a port and IP address. This is how a server discovers which client it is talking to after `accept`, or how a client confirms the peer address after `connect`.

```
val $packed = getpeername($client)
var ($port, $ip) = sockaddr_in($packed)
p "peer: " . inet_ntoa($ip) . ":" . $port
```


gethostbyname

Resolve a hostname to its network addresses using the system resolver. Returns (`$name`, `$aliases`, `$addrtype`, `$length`, `@addrs`). The addresses are packed binary — pass them through `inet_ntoa` to get dotted-quad strings. This is the classic DNS forward-lookup function.

```
val @info = gethostbyname("example.com")
var @addrs = @info[4..$ #info]
@addrs |> map inet_ntoa |> e p
var $ip = inet_ntoa((gethostbyname("localhost"))[4])
p $ip # 127.0.0.1
```

gethostbyaddr

Perform a reverse DNS lookup — given a packed binary IP address and address family, return the hostname associated with that address. Returns (`$name`, `$aliases`, `$addrtype`, `$length`, `@addrs`) on success, or empty list if no PTR record exists.

```
val $packed = inet_aton("8.8.8.8")
var $name = (gethostbyaddr($packed, AF_INET))[0]
p $name # dns.google
```

getprotobyname

Look up a network protocol by its name and return protocol information. Returns (`$name`, `$aliases`, `$proto_number`). The protocol number is what you pass to `socket` as the protocol argument. Common names include `tcp`, `udp`, and `icmp`.

```
var ($name, $aliases, $proto) = getprotobyname("tcp")
p "$name = protocol $proto" # tcp = protocol 6
socket(val $raw, AF_INET, SOCK_RAW, (getprotobyname("icmp"))[2])
```

getservbyname

Look up a network service by its name and protocol, returning the port number and related information. Returns (`$name`, `$aliases`, `$port`, `$proto`). The port is in host byte order. This resolves well-known service names like `http`, `ssh`, or `smtp` to their port numbers portably.

```
var ($name, $aliases, $port) = getservbyname("http", "tcp")
p "$name => port $port" # http => port 80
var $ssh_port = (getservbyname("ssh", "tcp"))[2]
p $ssh_port # 22
```

Crypto & Encoding

138 topics

sha256

`sha256` (alias `s256`) computes the SHA-256 cryptographic hash of the input data and returns it as a 64-character lowercase hexadecimal string. SHA-256 is the most widely used hash function for data integrity verification, content addressing, and digital signatures. The Rust implementation is significantly faster than pure-Perl alternatives. Accepts strings or byte buffers.

```
p sha256 "hello world" # b94d27b9934d3e08...
var $checksum = sha256 slurp "release.tar.gz"
p $checksum
rl("passwords.txt") |> map sha256 |> e p
```

sha224

`sha224` (alias `s224`) computes the SHA-224 cryptographic hash and returns a 56-character hex string. SHA-224 is a truncated variant of SHA-256 that produces a shorter digest while maintaining strong collision resistance. It is sometimes preferred when storage or bandwidth for the hash value is constrained, such as in compact data structures or short identifiers.

```
p sha224 "hello world" # 2f05477fc24bb4fa...
var $h = sha224(tj {key => "value"})
p $h
```

sha384

`sha384` (alias `s384`) computes the SHA-384 cryptographic hash and returns a 96-character hex string. SHA-384 is a truncated variant of SHA-512 that offers a middle ground between SHA-256 and SHA-512 in both digest length and security margin. It is commonly used in TLS certificate fingerprints and government security standards that require larger-than-256-bit digests.

```
p sha384 "hello world" # fdbd8e75a67f29f7...
var $sig = sha384(slurp "document.pdf")
spurt "document.pdf.sha384", $sig
```

sha512

`sha512` (alias `s512`) computes the SHA-512 cryptographic hash and returns a 128-character hex string. SHA-512 provides the largest digest size in the SHA-2 family and is the strongest option when maximum collision resistance is needed. On 64-bit systems, SHA-512 is often faster than SHA-256 because it operates on 64-bit words natively. Use this for high-security applications or when you need a longer hash.

```
p sha512 "hello world" # 309ecc489c12d6eb...
var $hash = sha512(slurp "firmware.bin");
p $hash;
var @hashes = map { sha512 } @files
```

sha1

`sha1` (alias `s1`) computes the SHA-1 hash and returns a 40-character hex string. SHA-1 is considered cryptographically broken for collision resistance and should not be used for security-sensitive applications. However, it remains widely used for non-security purposes such as Git object IDs, cache keys, and deduplication checksums where collision attacks are not a concern.

```
p sha1 "hello world" # 2aae6c35c94fcfb4...
var $git_id = sha1("blob " . length($content) . "\0" . $content)
p $git_id$1
```

Note: prefer SHA-256 for any security-related use case.

crc32

crc32 computes the CRC-32 checksum of the input data and returns it as an unsigned 32-bit integer. CRC-32 is not a cryptographic hash — it is a fast error-detection code used in network protocols (Ethernet, ZIP, PNG), file integrity checks, and hash table bucketing. It is extremely fast compared to SHA functions, making it suitable for high-throughput deduplication or quick change detection where collision resistance is not required.

```
p crc32 "hello world" # 222957957
var $chk = crc32(slurp "archive.zip");
p sprintf "0x%08x", $chk; # hex representation
var @checksums = map { crc32 } @chunks
```

hmac_sha256

hmac_sha256 (alias **hmac**) computes an HMAC-SHA256 message authentication code using the given data and secret key, returning a hex string. HMAC combines a cryptographic hash with a secret key to produce a signature that verifies both data integrity and authenticity. This is the standard mechanism for signing API requests (AWS, Stripe, GitHub webhooks), generating secure tokens, and verifying message authenticity.

```
val $sig = hmac_sha256 "request body", "my-secret-key"
p $sig
var $webhook_sig = hmac("POST /hook\n$body", $secret)
p $webhook_sig eq $expected ? "valid" : "tampered"
```

blake2b

blake2b (alias **b2b**) computes the BLAKE2b-512 cryptographic hash and returns a 128-character hex string. BLAKE2b is faster than SHA-256 while being at least as secure. It is used in Argon2 password hashing, libsodium, WireGuard, and many modern cryptographic protocols. Prefer BLAKE2b or BLAKE3 for new projects over SHA-256.

```
p blake2b "hello world" # 021ced8799...
var $hash = b2b(slurp "large-file.bin");
var @hashes = map { blake2b } @messages
```

blake2s

blake2s (alias **b2s**) computes the BLAKE2s-256 cryptographic hash and returns a 64-character hex string. BLAKE2s is optimized for 8-32 bit platforms while BLAKE2b targets 64-bit. Use BLAKE2s when targeting embedded systems, WebAssembly, or when a 256-bit digest suffices.

```
p blake2s "hello world" # 9aec6806...
var $checksum = b2s($firmware_blob)
```

blake3

blake3 (alias **b3**) computes the BLAKE3 cryptographic hash and returns a 64-character hex string (256-bit). BLAKE3 is the latest evolution — it is parallelizable, even faster than BLAKE2, and suitable for hashing large files at maximum speed. It is the recommended hash function for new projects where legacy compatibility is not required.

```
p blake3 "hello world" # d74981efa...
var $hash = b3(slurp "gigabyte-file.bin") # fast!
```

argon2_hash

argon2_hash (alias **argon2**) hashes a password using Argon2id, the winner of the Password Hashing Competition. Returns a PHC-format string containing algorithm parameters, salt, and hash. Argon2 is memory-hard and resistant to GPU/ASIC attacks. Use this for user password storage — never use MD5/SHA for passwords.

```
val $hash = argon2_hash("user-password")
to_file("user.hash", $hash)
# later...
if (argon2_verify("user-password", $hash)) {
    p "login successful"
}
```

argon2_verify

argon2_verify verifies a password against an Argon2 PHC hash string. Returns 1 if the password matches, 0 otherwise. The verification automatically extracts parameters from the stored hash, so changing Argon2 settings for new hashes does not break existing ones.

```
val $stored = rl("user.hash")
if (argon2_verify($user_input, $stored)) {
    p "access granted"
} else {
    p "invalid password"
}
```

bcrypt_hash

bcrypt_hash (alias **bcrypt**) hashes a password using the bcrypt algorithm, returning a standard **\$2b\$...** format string. Bcrypt has been the industry standard for password hashing for over 20 years. While Argon2 is now preferred for new systems, bcrypt remains secure and widely deployed.

```
val $hash = bcrypt_hash("my-password")
p $hash # $2b$12$...
if (bcrypt_verify("my-password", $hash)) {
    p "correct"
}
```

bcrypt_verify

bcrypt_verify verifies a password against a bcrypt hash string. Returns 1 if correct, 0 otherwise. The cost factor and salt are extracted from the stored hash automatically.

```
if (bcrypt_verify($password, $stored_hash)) {
    grant_access()
}
```

scrypt_hash

scrypt_hash (alias **scrypt**) hashes a password using the scrypt algorithm, returning a PHC-format string. Scrypt is memory-hard like Argon2 and is used in cryptocurrency (Litecoin) and some enterprise systems. It predates Argon2 but remains secure.

```
val $hash = scrypt_hash("password123")
if (scrypt_verify("password123", $hash)) {
    p "verified"
}
```

scrypt_verify

scrypt_verify verifies a password against a scrypt PHC hash. Returns 1 on match, 0 otherwise.

```
if (scrypt_verify($input, $stored)) {
    p "valid"
}
```

pbkdf2

pbkdf2 (alias **pbkdf2_derive**) derives a cryptographic key from a password using PBKDF2-HMAC-SHA256. Returns a 64-character hex string (32 bytes). Takes password, salt, and optional iteration count (default 100,000). Use this when you need a fixed-length key from a password — for encryption keys, not password storage (use Argon2/bcrypt for that).

```
val $key = pbkdf2("password", "random-salt")
p $key # 64 hex chars = 32 bytes
var $strong = pbkdf2("pass", $salt, 200_000) # more iterations
```

random_bytes

random_bytes (alias **randbytes**) generates cryptographically secure random bytes using the OS CSPRNG. Returns a byte buffer of the specified length. Use for encryption keys, nonces, salts, and any security-sensitive randomness. Never use **rand()** for cryptographic purposes.

```
val $key = random_bytes(32) # 256-bit key
var $nonce = random_bytes(12) # 96-bit nonce for AES-GCM
p hex_encode($key)
```

random_bytes_hex

random_bytes_hex (alias **randhex**) generates cryptographically secure random bytes and returns them as a hex string. Convenient when you need a random hex token or key without a separate **hex_encode** call.

```
val $token = random_bytes_hex(16) # 32 hex chars
p $token # 7a3f9b2c1d...
var $api_key = "sk_" . randhex(24)
```

aes_encrypt

aes_encrypt (alias **aes_enc**) encrypts plaintext using AES-256-GCM authenticated encryption. Takes a 32-byte key (use **random_bytes(32)** or **pbkdf2**). Returns a base64 string containing the nonce and ciphertext. AES-GCM provides both confidentiality and integrity — tampering is detected on decryption.

```
val $key = random_bytes(32)
var $cipher = aes_encrypt($key, "secret message")
p $cipher # base64 encoded
var $plain = aes_decrypt($key, $cipher)
p $plain # secret message
```

aes_decrypt

aes_decrypt (alias **aes_dec**) decrypts AES-256-GCM ciphertext produced by **aes_encrypt**. Takes the same 32-byte key and the base64 ciphertext. Dies if the key is wrong or the ciphertext was tampered with (authentication failure).

```
val $plain = aes_decrypt($key, $ciphertext)
p $plain
# wrong key or tampered data raises exception
```

chacha_encrypt

chacha_encrypt (alias **chacha_enc**) encrypts plaintext using ChaCha20-Poly1305 authenticated encryption. Takes a 32-byte key. Returns base64(nonce || ciphertext || tag). ChaCha20-Poly1305 is the modern alternative to AES-GCM — faster in software, constant-time, and used in TLS 1.3, WireGuard, and SSH.

```
val $key = random_bytes(32)
var $cipher = chacha_encrypt($key, "secret data")
var $plain = chacha_decrypt($key, $cipher)
p $plain
```

chacha_decrypt

chacha_decrypt (alias **chacha_dec**) decrypts ChaCha20-Poly1305 ciphertext. Takes the same 32-byte key and base64 ciphertext. Dies on wrong key or tampering.

```
val $plain = chacha_decrypt($key, $cipher)
p $plain
```

ed25519_keygen

ed25519_keygen (alias **ed_keygen**) generates an Ed25519 signing keypair. Returns [private_key_hex, public_key_hex]. Ed25519 is the modern standard for digital signatures — fast, secure, and used in SSH, GPG, and cryptocurrency. The private key is 32 bytes (64 hex), the public key is also 32 bytes.

```
var ($priv, $pub) = @{ ed25519_keygen() }
p "Private: $priv"
p "Public: $pub"
var $sig = ed25519_sign($priv, "message")
```

ed25519_sign

ed25519_sign (alias **ed_sign**) signs a message with an Ed25519 private key. Takes the private key (hex) and message. Returns a 128-character hex signature (64 bytes). Signatures are deterministic — the same key+message always produces the same signature.

```
val $sig = ed25519_sign($private_key, "hello world")
p $sig # 128 hex chars
```

ed25519_verify

ed25519_verify (alias **ed_verify**) verifies an Ed25519 signature. Takes public_key_hex, message, signature_hex. Returns 1 if valid, 0 if invalid. Never trust unsigned data in security contexts.

```
if (ed25519_verify($pub, "hello world", $sig)) {
  p "signature valid"
} else {
  p "FORGED!"
}
```

x25519_keygen

x25519_keygen (alias **x_keygen**) generates an X25519 key-exchange keypair. Returns [private_key_hex, public_key_hex]. X25519 is the modern Diffie-Hellman — used in TLS 1.3, Signal, WireGuard. Both parties generate keypairs and exchange public keys to derive a shared secret.

```
var ($my_priv, $my_pub) = @{ x25519_keygen() }
# send $my_pub to peer, receive $their_pub
var $shared = x25519_dh($my_priv, $their_pub)
```

x25519_dh

x25519_dh (alias **x_dh**) performs X25519 Diffie-Hellman key exchange. Takes your private key and their public key. Returns a 64-character hex shared secret. Both parties derive the same secret, which can be used as an encryption key.

```
val $shared = x25519_dh($my_private, $their_public)
var $key = hex_decode(substr($shared, 0, 64)) # use as AES key
```

base64_encode

base64_encode (alias **b64e**) encodes a string or byte buffer as a Base64 string using the standard alphabet (A-Z, a-z, 0-9, +, /). Base64 is the standard way to embed binary data in text-based formats like JSON, XML, email (MIME), and data URIs. The output length is always a multiple of 4, padded with = as needed. Use **base64_decode** to reverse the encoding.

```
val $encoded = base64_encode "hello world"
p $encoded    # aGVsbG8gd29ybGQ=
var $img_data = slurp "photo.png"
var $data_uri = "data:image/png;base64," . base64_encode($img_data)
p base64_decode(base64_encode("round trip")) # round trip
```

base64_decode

base64_decode (alias **b64d**) decodes a Base64-encoded string back to its original bytes. It accepts standard Base64 with padding and is tolerant of line breaks within the input. This is essential for processing email attachments, decoding JWT payloads, extracting embedded images from data URIs, or reading any Base64-encoded field from an API response. Raises an exception on invalid Base64 input.

```
val $decoded = base64_decode "aGVsbG8gd29ybGQ="
p $decoded    # hello world
var $img = base64_decode($api_response->{avatar_b64})
spurt "avatar.png", $img
var $json = base64_decode($jwt_parts[1])
dd json_decode $json
```

hex_encode

hex_encode (alias **hxe**) converts a string or byte buffer into its lowercase hexadecimal representation, with two hex characters per input byte. This is useful for displaying binary data in a human-readable format, generating hex-encoded keys or IDs, logging raw bytes, or preparing data for protocols that use hex encoding. The output is always an even number of characters.

```
p hex_encode "hello"    # 68656c6c6f
var $raw = slurp "key.bin"
p hex_encode $raw
var $color = hex_encode chr(255) . chr(128) . chr(0)
p "#$color"             # #ff8000
```

hex_decode

hex_decode (alias **hxd**) converts a hexadecimal string back to its original bytes, interpreting every two hex characters as one byte. This is the inverse of **hex_encode** and is useful for parsing hex-encoded binary data from config files, network protocols, or cryptographic outputs. The input must have an even number of valid hex characters (0-9, a-f, A-F) or an exception is raised.

```
val $bytes = hex_decode "68656c6c6f"
p $bytes    # hello
var $key = hex_decode $env_hex_key
var $mac = hmac_sha256($data, hex_decode($secret_hex))
p hex_decode(hex_encode("round trip")) # round trip
```

hide

hide(CARRIER, SECRET [, KEY]) — polymorphic steganography. Embeds SECRET inside CARRIER and returns the modified carrier. Carrier kind is detected from the input bytes: data starting with the PNG magic (**\x89PNG**) uses LSB embedding on the R/G/B channels (alpha skipped so transparent pixels can't leak); anything else is treated as text and SECRET bits are encoded as zero-width chars (U+200B for 0, U+200C for 1) inserted between visible characters. Optional KEY enables SHA-256(key||counter)-derived XOR masking of the secret prior to embedding.

Wire format (so **reveal** knows where to stop): **[4-byte BE length][secret bytes][4-byte BE CRC32-IEEE]**. A CRC mismatch on **reveal** means the carrier was re-encoded, truncated, or never contained a payload.

```
# Text carrier - invisible in rendered output, survives copy-paste in most channels.
var $stego = hide("lorem ipsum dolor sit amet " x 6, "secret message")
reveal($stego) eq "secret message" # 1

# PNG carrier - LSB embed survives a re-save through any lossless tool.
"carrier.png" |> slurp |> hide("hidden data") |> spew("out.png")
slurp("out.png") |> reveal |> p # hidden data

# Key-XOR mode - wrong key still passes CRC but recovers garbage.
hide($png_bytes, $secret, "shared-passphrase") |> spew("out.png")
reveal(slurp("out.png"), "shared-passphrase")
```

World-first: no other major scripting language ships polymorphic steganography (text + image) as a single first-class builtin. Python needs `stegano` + `stegclook`, Perl needs `Image::PNG::Libpng` + hand-rolled bit twiddling.

See also: `reveal`, `hide_capacity`.

reveal

`reveal(STEGO [, KEY])` — extract the secret embedded by `hide`. Auto-detects carrier kind the same way `hide` does (PNG magic vs text). Returns raw bytes; wrap with `decode_utf8(...)` if you know the secret is text.

Errors with a `corrupt` or `absent payload` message on:

- CRC32 mismatch (carrier re-encoded by a lossy tool, or no payload was ever embedded)
- declared length exceeding the carrier's embedded bit capacity
- carrier too small to even hold a 4-byte length prefix

```
val $secret = reveal($stego) # bytes
p $secret # auto-stringifies via UTF-8 decode

var $hidden = slurp("image.png") |> reveal
var $keyed = reveal($stego, "shared-passphrase")

# Detect tampering: anything that mutates the embedded bits fails the CRC32.
try { reveal($maybe_stego) } catch ($e) {
  warn "no valid payload: $e"
}
```

See also: `hide`, `hide_capacity`.

hide_capacity

`hide_capacity(CARRIER)` — return how many bytes of secret payload CARRIER can hold, accounting for the 8-byte envelope overhead (4-byte length + 4-byte CRC32). Useful for sizing before calling `hide`.

For PNG carriers: $(width * height * 3) / 8 - 8$ (R+G+B LSBs, alpha skipped). For text carriers: $(visible_chars / 8) - 8$ (one bit per visible code point).

```
val $cap = hide_capacity(slurp("carrier.png"))
p "can hide up to $cap bytes"
if (length($secret) > $cap) {
  die "secret too large for this carrier"
}
hide_capacity("x" x 100) # 4
```

See also: `hide`, `reveal`.

uuid

`uuid` generates a cryptographically random UUID version 4 string in the standard 8-4-4-4-12 hyphenated format. Each call produces a unique identifier suitable for database primary keys, correlation IDs, session tokens, temporary file names, or any situation requiring a globally unique identifier without coordination. The randomness comes from the OS CSPRNG.


```
val $id = uuid()
p $id # e.g., 550e8400-e29b-41d4-a716-446655440000
var %record = {id => uuid(), name => "Alice", created => time}
var @ids = map { uuid() } 1:10
```

jwt_encode

`jwt_encode` creates a signed JSON Web Token from a payload hashref and a secret key. The default algorithm is HS256 (HMAC-SHA256), but you can specify an alternative as the third argument. JWTs are the standard for stateless authentication tokens, API authorization, and secure inter-service communication. The returned string contains the base64url-encoded header, payload, and signature separated by dots.

```
val $token = jwt_encode({sub => "user123", exp => time + 3600}, "my-secret")
p $token
var $admin = jwt_encode({role => "admin", iat => time}, $secret, "HS512")
# send as Authorization header
http_request(method => "GET", url => $api_url,
  headers => {Authorization => "Bearer $token"})
```

jwt_decode

`jwt_decode` verifies the signature of a JSON Web Token using the provided secret key and returns the decoded payload as a hashref. If the signature is invalid, the token has been tampered with, or it has expired (when an `exp` claim is present), the function raises an exception. This is the secure way to validate incoming JWTs from clients or other services — always use this over `jwt_decode_unsafe` in production.

```
val $payload = jwt_decode($token, "my-secret")
p $payload->{sub} # user123
var $claims = jwt_decode($bearer_token, $secret)
if ($claims->{role} eq "admin") {
  p "admin access granted"
}
```

Note: raises an exception on expired tokens, invalid signatures, or malformed input.

jwt_decode_unsafe

`jwt_decode_unsafe` decodes a JSON Web Token and returns the payload as a hashref without verifying the signature. This is intentionally insecure and should only be used for debugging, logging, or inspecting token contents in development environments. Never use this to make authorization decisions in production — an attacker can stryke arbitrary payloads. The function still parses the JWT structure and base64-decodes the payload, but skips all cryptographic checks.

```
# debugging only - never use for auth
var $claims = jwt_decode_unsafe($token)
dd $claims
p $claims->{sub} # inspect without needing the secret
var $exp = $claims->{exp}
p "Expires: " . datetime_from_epoch($exp)$1

Note: this function exists for debugging. Use `jwt_decode` with a secret for any security-relevant
    validation.
```

url_encode

Percent-encode a string so it is safe to embed in a URL query parameter or path segment. Unreserved characters (alphanumeric, `-`, `_`, `.`, `~`) are left as-is; everything else becomes `%XX`. The alias `uri_escape` matches the classic `URI::Escape` name for Perl muscle-memory.

```
val $q = "hello world & friends"
var $safe = url_encode($q)
p $safe # hello%20world%20%26%20friends
var $url = "https://example.com/search?q=" . url_encode($q)
p $url$1
```

Note: does not encode the full URL structure – encode individual components, not the whole URL.

url_decode

Decode a percent-encoded string back to its original form, converting `%XX` sequences to the corresponding bytes and `+` to space. The alias `uri_unescape` matches `URI::Escape` conventions. Use this when parsing query strings from incoming URLs or reading URL-encoded form data.

```
var $encoded = "hello%20world%20%26%20friends"
p url_decode($encoded) # hello world & friends
# round-trip
var $orig = "café ☹️"
p url_decode(url_encode($orig)) eq $orig # 1
```

gzip

Compress a string or byte buffer using the gzip (RFC 1952) format and return the compressed bytes. Useful for shrinking data before writing to disk or sending over the network. Pairs with `gunzip` for decompression. The compression level is chosen automatically for a good speed/size tradeoff.

```
val $raw = "hello world" x 1000
var $gz = gzip($raw)
to_file("data.gz", $gz)
p length($gz) # much smaller than original
p gunzip($gz) eq $raw # 1
```

gunzip

Decompress gzip-compressed data (RFC 1952) and return the original bytes. Dies if the input is not valid gzip. Use this to read `.gz` files or decompress data received from HTTP responses with `Content-Encoding: gzip`. Always the inverse of `gzip`.

```
val $compressed = rl("archive.gz")
var $text = gunzip($compressed)
p $text
# round-trip in a pipeline
"payload" |> gzip |> gunzip |> p # payload
```

zstd

Compress a string or byte buffer using the Zstandard algorithm and return the compressed bytes. Zstandard offers significantly better compression ratios and speed compared to gzip, making it ideal for large datasets, IPC buffers, and caching. Pairs with `zstd_decode` for decompression.

```
val $big = "x]" x 100_000
var $compressed = zstd($big)
p length($compressed) # fraction of original
to_file("data.zst", $compressed)
p zstd_decode($compressed) eq $big # 1
```

zstd_decode

Decompress Zstandard-compressed data and return the original bytes. Dies if the input is not valid Zstandard. This is the inverse of `zstd`. Use it to read `.zst` files or decompress cached buffers that were compressed with `zstd`.

```
val $packed = zstd("important data\n" x 500)
var $original = zstd_decode($packed)
p $original
# file round-trip
to_file("cache.zst", zstd($payload))
p zstd_decode(rl("cache.zst"))
```

sha3_256

sha3_256 (alias **s3_256**) computes the SHA3-256 hash (NIST FIPS 202) and returns a 64-character hex string. SHA-3 is the newest NIST-approved hash family, designed as a backup if SHA-2 is ever compromised. It uses the Keccak sponge construction which is fundamentally different from SHA-2.

```
p sha3_256 "hello world" # 644bcc7e...
var $h = s3_256(slurp "file.bin")
```

sha3_512

sha3_512 (alias **s3_512**) computes the SHA3-512 hash and returns a 128-character hex string. Provides a 512-bit digest with the SHA-3 sponge construction.

```
p sha3_512 "hello world" # 840006...
```

shake128

shake128 is a SHA-3 extendable-output function (XOF). Unlike fixed-length hashes, SHAKE can produce arbitrary-length output. Takes data and output length in bytes, returns hex. Useful for key derivation or when you need a variable-length digest.

```
p shake128("seed", 32) # 64 hex chars (32 bytes)
p shake128("seed", 64) # 128 hex chars
```

shake256

shake256 is a SHA-3 XOF with higher security margin than SHAKE128. Takes data and output length in bytes, returns hex.

```
p shake256("seed", 32)
p shake256("key material", 64)
```

ripemd160

ripemd160 (alias **rmd160**) computes the RIPEMD-160 hash and returns a 40-character hex string. RIPEMD-160 is used in Bitcoin addresses (Hash160 = RIPEMD160(SHA256(pubkey))) and some legacy systems. Not recommended for new designs but essential for Bitcoin/crypto compatibility.

```
p ripemd160 "hello world" # 98c615784c...
var $hash160 = ripemd160(hex_decode(sha256($pubkey)))
```

siphash

siphash computes SipHash-2-4 with default keys (0,0) and returns a 16-character hex string (64-bit). SipHash is designed for hash table DoS resistance — it's fast and keyed, preventing attackers from crafting collisions. Used in Rust's HashMap, Python dict, and many other hash tables.

```
p siphash "key" # 16 hex chars
```

hmac_sha1

hmac_sha1 computes HMAC-SHA1 and returns a 40-character hex string. Used in OAuth 1.0, TOTP (Google Authenticator), and legacy APIs. Prefer HMAC-SHA256 for new designs.

```
p hmac_sha1("secret", "message") # 40 hex chars
```

hmac_sha384

`hmac_sha384` computes HMAC-SHA384 and returns a 96-character hex string. Middle ground between SHA-256 and SHA-512.

```
p hmac_sha384("key", "data") # 96 hex chars
```

hmac_sha512

`hmac_sha512` computes HMAC-SHA512 and returns a 128-character hex string. Maximum security margin in the SHA-2 family.

```
p hmac_sha512("key", "data") # 128 hex chars
```

hmac_md5

`hmac_md5` computes HMAC-MD5 and returns a 32-character hex string. Legacy only — MD5 is broken for collision resistance but HMAC-MD5 is still considered safe for authentication. Only use for compatibility with old systems.

```
p hmac_md5("key", "data") # 32 hex chars
```

hkdf_sha256

`hkdf_sha256` (alias `hkdf`) is HKDF key derivation (RFC 5869) using HMAC-SHA256. Extracts entropy from input key material and expands it to the desired length. Used to derive encryption keys from shared secrets (after ECDH/X25519). Args: `ikm`, `salt`, `info`, `output_length`.

```
val $key = hkdf("shared_secret", "salt", "context", 32) # 64 hex
var $enc_key = hex_decode($key) # 32 bytes for AES-256
```

hkdf_sha512

`hkdf_sha512` is HKDF using HMAC-SHA512. Higher security margin than SHA256 variant.

```
val $key = hkdf_sha512("ikm", "salt", "info", 64) # 128 hex chars
```

poly1305

`poly1305` computes a Poly1305 one-time MAC. Takes a 32-byte key and message, returns a 32-character hex tag (128-bit). Poly1305 is used with ChaCha20 in TLS 1.3. CRITICAL: each key must only be used once — reusing a key completely breaks security.

```
val $key = random_bytes(32)
p poly1305($key, "message") # 32 hex chars
```

rsa_keygen

`rsa_keygen` generates an RSA keypair. Takes key size in bits (2048, 3072, or 4096). Returns [`private_key_pem`, `public_key_pem`]. RSA is the most widely used asymmetric algorithm — essential for TLS, SSH, and JWT RS256 signing.

```
val @kp = rsa_keygen(2048)
var ($priv, $pub) = @kp
spurt("private.pem", $priv)
spurt("public.pem", $pub)
```

rsa_encrypt

rsa_encrypt (alias **rsa_enc**) encrypts data with RSA-OAEP-SHA256. Takes **public_key_pem** and plaintext. Returns base64 ciphertext. Message size is limited to **key_size/8 - 66** bytes (e.g. 190 bytes for 2048-bit key).

```
val $cipher = rsa_encrypt($pub_pem, "secret")
var $plain = rsa_decrypt($priv_pem, $cipher)
```

rsa_decrypt

rsa_decrypt (alias **rsa_dec**) decrypts RSA-OAEP ciphertext. Takes **private_key_pem** and base64 ciphertext.

```
val $plain = rsa_decrypt($priv, $ciphertext)
p $plain
```

rsa_sign

rsa_sign signs a message with RSA-PKCS1v15-SHA256. Takes **private_key_pem** and message. Returns base64 signature. This is the RS256 algorithm used in JWT.

```
val $sig = rsa_sign($priv, "message")
if (rsa_verify($pub, "message", $sig)) {
  p "valid"
}
```

rsa_verify

rsa_verify verifies an RSA-PKCS1v15-SHA256 signature. Takes **public_key_pem**, message, and base64 signature. Returns 1 if valid, 0 if not.

```
if (rsa_verify($pub_pem, $msg, $sig)) {
  p "signature valid"
}
```

ecdsa_p256_keygen

ecdsa_p256_keygen (alias **p256_keygen**) generates an ECDSA P-256 (secp256r1/prime256v1) keypair. Returns [**private_hex**, **public_hex_compressed**]. P-256 is the NIST curve used in TLS, ES256 JWT, and WebAuthn.

```
val @kp = ecdsa_p256_keygen()
var ($priv, $pub) = @kp
```

ecdsa_p256_sign

ecdsa_p256_sign (alias **p256_sign**) signs a message with ECDSA P-256. Takes **private_key_hex** and message. Returns DER-encoded signature as hex.

```
val $sig = ecdsa_p256_sign($priv, "hello")
```

ecdsa_p256_verify

ecdsa_p256_verify (alias **p256_verify**) verifies an ECDSA P-256 signature. Returns 1 if valid.

```
if (ecdsa_p256_verify($pub, "hello", $sig)) {
  p "valid"
}
```

ecdsa_p384_keygen

ecdsa_p384_keygen generates an ECDSA P-384 keypair. P-384 offers more security margin than P-256.

```
val @kp = ecdsa_p384_keygen()
```

ecdsa_p384_sign

`ecdsa_p384_sign` signs with ECDSA P-384.

```
val $sig = ecdsa_p384_sign($priv, $msg)
```

ecdsa_p384_verify

`ecdsa_p384_verify` verifies an ECDSA P-384 signature.

```
p ecdsa_p384_verify($pub, $msg, $sig)
```

ecdsa_secp256k1_keygen

`ecdsa_secp256k1_keygen` generates an ECDSA secp256k1 keypair. This is the Bitcoin/Ethereum curve — different from P-256.

```
val @kp = ecdsa_secp256k1_keygen()
```

ecdsa_secp256k1_sign

`ecdsa_secp256k1_sign` signs with ECDSA secp256k1.

```
val $sig = ecdsa_secp256k1_sign($priv, $msg)
```

ecdsa_secp256k1_verify

`ecdsa_secp256k1_verify` verifies an ECDSA secp256k1 signature.

```
p ecdsa_secp256k1_verify($pub, $msg, $sig)
```

ecdh_p256

`ecdh_p256` (alias `p256_dh`) performs ECDH key exchange on P-256. Takes `my_private_hex` and `their_public_hex`, returns `shared_secret_hex`. Use HKDF to derive encryption keys from the shared secret.

```
val @alice = ecdsa_p256_keygen()
val @bob = ecdsa_p256_keygen()
val $shared_a = ecdh_p256($alice[0], $bob[1])
val $shared_b = ecdh_p256($bob[0], $alice[1])
p $shared_a eq $shared_b # 1 - same secret
```

ecdh_p384

`ecdh_p384` performs ECDH key exchange on P-384.

```
val $shared = ecdh_p384($my_priv, $their_pub)
```

base32_encode

`base32_encode` (alias `b32e`) encodes data as RFC 4648 Base32. Used in TOTP secrets, onion addresses, and Bech32. Returns uppercase with padding.

```
p base32_encode("hello") # NBSWY3DP
var $secret = base32_encode(random_bytes(20))
```

base32_decode

`base32_decode` (alias `b32d`) decodes RFC 4648 Base32 back to bytes. Accepts with or without padding.

```
p base32_decode("NBSWY3DP") # hello
```

base58_encode

`base58_encode` (alias `b58e`) encodes data using Bitcoin's Base58 alphabet (no 0, O, I, l to avoid confusion). Used in Bitcoin addresses, IPFS CIDs.

```
p base58_encode("hello") # Cn8eVZg
```

base58_decode

`base58_decode` (alias `b58d`) decodes Base58 back to bytes.

```
p base58_decode("Cn8eVZg") # hello
```

totp

`totp` (alias `totp_generate`) generates a TOTP code (RFC 6238) for 2FA. Takes base32-encoded secret, optional digits (default 6), optional period (default 30s). Compatible with Google Authenticator, Authy, etc.

```
val $secret = base32_encode(random_bytes(20))
var $code = totp($secret)
p $code # 6-digit code
# Custom: 8 digits, 60s period
var $code8 = totp($secret, 8, 60)
```

totp_verify

`totp_verify` verifies a TOTP code with optional time window (default ± 1 period). Returns 1 if valid.

```
if (totp_verify($secret, $user_code)) {
  p "2FA valid"
}
# Wider window:  $\pm 2$  periods
totp_verify($secret, $code, 2)
```

hotp

`hotp` (alias `hotp_generate`) generates an HOTP code (RFC 4226) using a counter. Takes base32 secret, counter value, optional digits.

```
val $code = hotp($secret, 42) # counter=42
```

aes_cbc_encrypt

`aes_cbc_encrypt` (alias `aes_cbc_enc`) encrypts with AES-256-CBC and PKCS7 padding. Takes 32-byte key, plaintext, optional 16-byte IV (auto-generated if omitted). Returns base64(iv || ciphertext). Legacy mode — prefer `aes_encrypt` (GCM) for new code.

```
val $key = random_bytes(32)
var $ct = aes_cbc_encrypt($key, "secret")
var $pt = aes_cbc_decrypt($key, $ct)
```

aes_cbc_decrypt

`aes_cbc_decrypt` (alias `aes_cbc_dec`) decrypts AES-256-CBC. Takes 32-byte key and base64(iv || ciphertext).

```
val $pt = aes_cbc_decrypt($key, $ciphertext)
```

qr_ascii

`qr_ascii` (alias `qr`) generates a QR code as ASCII art. Perfect for terminal output.

```
p qr("https://example.com")
p qr("otpauth://totp/App:user?secret=$secret&issuer=App")
```

qr_png

`qr_png` generates a QR code as PNG image data (base64 encoded). Optional size parameter. Save to file or embed in HTML.

```
val $png = qr_png("https://example.com")
spurt("qr.png", base64_decode($png))
# Larger QR
var $big = qr_png($url, 16)
```

controller

`controller([bind, [port]])` — drop this stryke process into **controller mode**: bind a TCP listener at `bind:port`, accept agent connections, and run the interactive REPL on stdin so commands like `status`, `@CODE`, `eval CODE`, `fire`, `terminate`, and `shutdown` can be typed at the prompt. Blocking; returns the exit code (0 on a clean `shutdown`, 1 if bind fails). Defaults: `bind="0.0.0.0"`, `port=9999`.

Equivalent to the `stryke controller` CLI subcommand but invocable from any stryke source file. Combine with `agent()` and `spawn` to run controller + local agent in a single process for testing.

```
# Bare invocation - bind 0.0.0.0:9999, REPL on stdin
controller()

# Explicit
exit controller("127.0.0.1", 8888)

# Controller + local agent in one process (testing pattern)
spawn {
  sleep 1
  agent("localhost:9999", "local-worker")
}
controller("127.0.0.1", 9999)
```

See also: `agent`.

agent

`agent([controller_addr, [name]])` — drop this stryke process into **agent mode**: connect to `controller_addr` (default `localhost:9999`), send `AGENT_HELLO`, and enter the persistent frame loop handling `FIRE` / `EVAL` / `TERMINATE` / `STATUS` / `SHUTDOWN` from the controller. EVAL frames carry arbitrary stryke source that runs against a persistent per-agent `VMHelper`, so `sub` definitions and `$main::name` globals survive across calls — the controller becomes a remote REPL.

`controller_addr` may be a bare host (port defaults to 9999) or `host:port`. `name` is what shows in the controller's `status` table — defaults to the local hostname if omitted.

Blocking; returns the exit code (0 clean disconnect, 1 connection or handshake failure). Wrap in `spawn { agent(...) }` to run an agent in the background.

```
#!/usr/bin/env stryke
# Become an agent on boot
exit agent(${ENV{CONTROLLER_ADDR}} // "ctl.local:9999", ${ENV{HOSTNAME}})

# Multi-agent test fixture in one process
spawn { agent("localhost:9999", "worker-1") }
spawn { agent("localhost:9999", "worker-2") }
controller("127.0.0.1", 9999)
```

See also: `controller`.

qr_svg

`qr_svg` generates a QR code as SVG string. Scalable vector graphics, ideal for web.

```
val $svg = qr_svg("https://example.com")
spurt("qr.svg", $svg)
```

barcode_code128

`barcode_code128` (alias `code128`) generates a Code 128 barcode as ASCII. Code 128 supports alphanumeric data and is widely used in shipping labels.

```
p code128("ABC-123")
```

barcode_code39

`barcode_code39` (alias `code39`) generates a Code 39 barcode. Supports uppercase, digits, and some symbols. Used in automotive and defense.

```
p code39("HELLO123")
```

barcode_ean13

`barcode_ean13` (alias `ean13`) generates an EAN-13 barcode (European Article Number). Standard retail barcode — requires exactly 12-13 digits.

```
p ean13("5901234123457")
```

barcode_svg

`barcode_svg` generates a barcode as SVG. Second argument specifies type: `code128`, `code39`, `ean13`, `upca`.

```
val $svg = barcode_svg("ABC-123", "code128")
spurt("barcode.svg", $svg)
var $retail = barcode_svg("012345678905", "upca")
```

brotnli

`brotnli` (alias `br`) compresses data using the Brotli algorithm (RFC 7932). Excellent compression ratio, used in HTTP compression. Returns compressed bytes.

```
val $compressed = brotnli($data)
p length($data) . " -> " . length($compressed)
```

brotnli_decode

`brotnli_decode` (alias `ubr`) decompresses Brotli data.

```
val $original = brotnli_decode($compressed)
```

xz

`xz` (alias `lzma`) compresses data using XZ/LZMA2. Best compression ratio, slower. Returns compressed bytes.

```
val $compressed = xz($data)
spurt("file.xz", $compressed)
```

xz_decode

`xz_decode` (aliases `unxz`, `unlzma`) decompresses XZ/LZMA data.

```
val $original = xz_decode(slurp("file.xz"))
```

bzip2

`bzip2` (alias `bz2`) compresses data using bzip2. Good compression, moderate speed. Returns compressed bytes.

```
val $compressed = bzip2($data)
```

bzip2_decode

`bzip2_decode` (aliases `bunzip2`, `ubz2`) decompresses bzip2 data.

```
val $original = bunzip2($compressed)
```

lz4

`lz4` compresses data using LZ4. Very fast compression/decompression, moderate ratio. Ideal for real-time compression.

```
val $compressed = lz4($data)
```

lz4_decode

`lz4_decode` (alias `unlz4`) decompresses LZ4 data.

```
val $original = lz4_decode($compressed)
```

snappy

`snappy` (alias `snp`) compresses data using Snappy. Fastest compression, used in databases and RPC. Returns compressed bytes.

```
val $compressed = snappy($data)
```

snappy_decode

`snappy_decode` (alias `unsnappy`) decompresses Snappy data.

```
val $original = snappy_decode($compressed)
```

lzw

`lzw` compresses data using LZW (GIF/TIFF style). Classic algorithm.

```
val $compressed = lzw($data)
```

lzw_decode

`lzw_decode` (alias `unlzw`) decompresses LZW data.

```
val $original = lzw_decode($compressed)
```

tar_create

`tar_create` (alias `tar`) creates a tar archive from a directory. Returns tar bytes.

```
val $archive = tar_create("./src")
spurt("backup.tar", $archive)
```

tar_extract

`tar_extract` (alias `untar`) extracts a tar archive to a directory.

```
tar_extract(slurp("backup.tar"), "./restored")
```

tar_list

`tar_list` lists files in a tar archive. Returns array of paths.

```
val @files = @{tar_list(slurp("backup.tar"))}
@files |> e p
```

tar_gz_create

`tar_gz_create` (alias `tgz`) creates a gzipped tar archive. Convenience for tar + gzip.

```
val $tgz = tgz("./project")
spurt("project.tar.gz", $tgz)
```

tar_gz_extract

`tar_gz_extract` (alias `untgz`) extracts a .tar.gz archive.

```
untgz(slurp("project.tar.gz"), "./extracted")
```

zip_create

`zip_create` creates a ZIP archive from a directory. Returns zip bytes.

```
val $archive = zip_create("./docs")
spurt("docs.zip", $archive)
```

zip_extract

`zip_extract` extracts a ZIP archive to a directory.

```
zip_extract(slurp("docs.zip"), "./extracted")
```

zip_list

`zip_list` lists files in a ZIP archive. Returns array of paths.

```
val @files = @{zip_list(slurp("archive.zip"))}
```

md4

`md4` computes the MD4 hash and returns a 32-character hex string. MD4 is completely broken — only use for legacy NTLM compatibility or historical systems.

```
p md4("hello") # 32 hex chars
```

xxh32

`xxh32` (alias `xxhash32`) computes xxHash32 — extremely fast non-cryptographic hash. Optional seed parameter. Returns 8 hex chars.

```
p xxh32("hello")      # default seed 0
p xxh32("hello", 42)   # with seed
```

xxh64

`xxh64` (alias `xxhash64`) computes xxHash64 — fast 64-bit hash. Returns 16 hex chars.

```
p xxh64("hello")
```

xxh3

`xxh3` (alias `xxhash3`) computes xxHash3-64 — newest xxHash variant, fastest on modern CPUs. Returns 16 hex chars.

```
p xxh3("hello")
```

xxh3_128

`xxh3_128` computes xxHash3-128. Returns 32 hex chars.

```
p xxh3_128("hello")
```

murmur3

`murmur3` (alias `murmur3_32`) computes MurmurHash3 32-bit. Fast non-cryptographic hash, widely used in hash tables and bloom filters. Optional seed. Returns 8 hex chars.

```
p murmur3("hello")    # default seed 0
p murmur3("hello", 42) # with seed
```

murmur3_128

`murmur3_128` computes MurmurHash3 128-bit (x64 variant). Returns 32 hex chars.

```
p murmur3_128("hello")
```

blowfish_encrypt

`blowfish_encrypt` (alias `bf_enc`) encrypts with Blowfish-CBC. Key=4-56 bytes, optional 8-byte IV. Legacy cipher — use AES for new code.

```
val $ct = blowfish_encrypt($key, "secret")
var $pt = blowfish_decrypt($key, $ct)
```

blowfish_decrypt

`blowfish_decrypt` (alias `bf_dec`) decrypts Blowfish-CBC.

```
val $pt = blowfish_decrypt($key, $ciphertext)
```

des3_encrypt

`des3_encrypt` (aliases `3des_enc`, `tdes_enc`) encrypts with Triple DES (3DES) CBC. Key=24 bytes (three 8-byte DES keys). Legacy cipher for PCI-DSS compliance.

```
val $key = random_bytes(24)
var $ct = des3_encrypt($key, "secret")
var $pt = des3_decrypt($key, $ct)
```

des3_decrypt

`des3_decrypt` (aliases `3des_dec`, `tdes_dec`) decrypts Triple DES (3DES) CBC.

```
val $pt = des3_decrypt($key, $ciphertext)
```

twofish_encrypt

`twofish_encrypt` (alias `tf_enc`) encrypts with Twofish-CBC. Key=16/24/32 bytes. AES finalist, still secure.

```
val $key = random_bytes(32)
var $ct = twofish_encrypt($key, "secret")
```

twofish_decrypt

`twofish_decrypt` (alias `tf_dec`) decrypts Twofish-CBC.

```
val $pt = twofish_decrypt($key, $ct)
```

camellia_encrypt

`camellia_encrypt` (alias `cam_enc`) encrypts with Camellia-CBC. Key=16/24/32 bytes. Japanese/EU standard, equivalent security to AES.

```
val $ct = camellia_encrypt($key, "secret")
```

camellia_decrypt

`camellia_decrypt` (alias `cam_dec`) decrypts Camellia-CBC.

```
val $pt = camellia_decrypt($key, $ct)
```

cast5_encrypt

`cast5_encrypt` encrypts with CAST5-CBC. Key=5-16 bytes. Used in PGP.

```
val $ct = cast5_encrypt($key, "secret")
```

cast5_decrypt

`cast5_decrypt` decrypts CAST5-CBC.

```
val $pt = cast5_decrypt($key, $ct)
```

salsa20

`salsa20` encrypts with Salsa20 stream cipher. Key=32 bytes, nonce auto-generated. Fast, secure stream cipher.

```
val $ct = salsa20($key, "data")
var $pt = salsa20_decrypt($key, $ct)
```

salsa20_decrypt

salsa20_decrypt decrypts Salsa20.

```
val $pt = salsa20_decrypt($key, $ct)
```

xsalsa20

xsalsa20 encrypts with XSalsa20 (extended 24-byte nonce). Safer for random nonces.

```
val $ct = xsalsa20($key, "data")
```

xsalsa20_decrypt

xsalsa20_decrypt decrypts XSalsa20.

```
val $pt = xsalsa20_decrypt($key, $ct)
```

secretbox

secretbox (alias `secretbox_seal`) is NaCl's symmetric authenticated encryption (XSalsa20-Poly1305). Key=32 bytes. Simple, secure, fast.

```
val $key = random_bytes(32)
var $ct = secretbox($key, "message")
var $pt = secretbox_open($key, $ct)
```

secretbox_open

secretbox_open decrypts and authenticates NaCl secretbox.

```
val $pt = secretbox_open($key, $ct)
```

nacl_box_keygen

nacl_box_keygen generates a NaCl box keypair (X25519). Returns [secret_key_hex, public_key_hex].

```
val @kp = nacl_box_keygen()
var ($sk, $pk) = @kp
```

nacl_box

nacl_box is NaCl's asymmetric authenticated encryption. Takes recipient's public key, sender's secret key, plaintext.

```
val @alice = nacl_box_keygen()
var @bob = nacl_box_keygen()
var $ct = nacl_box($bob[1], $alice[0], "hello") # to Bob from Alice
var $pt = nacl_box_open($alice[1], $bob[0], $ct) # Bob decrypts
```

nacl_box_open

nacl_box_open decrypts NaCl box. Takes sender's public key, recipient's secret key, ciphertext.

```
val $pt = nacl_box_open($sender_pk, $my_sk, $ct)
```

Special Math Functions

12 topics

erf

`erf` computes the error function, which arises in probability, statistics, and solutions to the heat equation. $\text{erf}(x)$ is the probability that a standard normal random variable falls in $[-x\sqrt{2}, x\sqrt{2}]$. Returns a value in $(-1, 1)$.

```
p erf(0)      # 0
p erf(1)      # 0.8427...
p erf(10)     # ~1
```

erfc

`erfc` computes the complementary error function $\text{erfc}(x) = 1 - \text{erf}(x)$. Numerically stable for large x where $\text{erf}(x) \approx 1$. Used in computing tail probabilities of normal distributions.

```
p erfc(0)     # 1
p erfc(3)     # 0.0000220...
```

gamma

`gamma` (alias `tgamma`) computes the gamma function $\Gamma(x)$, the extension of factorial to real numbers. $\Gamma(n) = (n-1)!$ for positive integers. Used throughout statistics, physics, and combinatorics.

```
p gamma(5)    # 24 (= 4!)
p gamma(0.5)  #  $\sqrt{\pi} \approx 1.7724...$ 
p gamma(1)    # 1
```

lgamma

`lgamma` (alias `ln_gamma`) computes the natural logarithm of the gamma function: $\ln(\Gamma(x))$. Avoids overflow for large arguments where $\Gamma(x)$ would be astronomical. Essential for computing log-probabilities in statistics.

```
p lgamma(100) # 359.13...
p lgamma(1000) # 5905.22...
```

digamma

`digamma` (alias `psi`) computes the digamma function $\psi(x) = d/dx \ln(\Gamma(x))$, the logarithmic derivative of gamma. Appears in Bayesian statistics (expected log of Dirichlet variables), optimization, and special function theory.

```
p digamma(1)  #  $-\gamma \approx -0.5772$  (Euler-Mascheroni)
p digamma(2)  #  $1 - \gamma \approx 0.4228$ 
```

beta_fn

`beta_fn` computes the beta function $B(a,b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$. The beta function is the normalizing constant of the Beta distribution and appears throughout Bayesian statistics and combinatorics.

```
p beta_fn(2, 3) # 0.0833...
p beta_fn(0.5, 0.5) #  $\pi$ 
```

lbeta

lbeta (alias **ln_beta**) computes $\ln(B(a,b))$, the log of the beta function. Avoids overflow for small a or b where $B(a,b)$ is very large.

```
p lbeta(0.01, 0.01) # large positive
```

betainc

betainc (alias **beta_reg**) computes the regularized incomplete beta function $I_x(a,b)$, the CDF of the Beta distribution. Takes (x, a, b) . Essential for computing p-values of F-tests, t-tests, and binomial probabilities.

```
p betainc(0.5, 2, 3) # P(Beta(2,3) < 0.5)
```

gammainc

gammainc (alias **gamma_li**) computes the lower incomplete gamma function $\gamma(a,x) = \int_0^x t^{a-1} e^{-t} dt$. Used in computing CDFs of gamma and chi-squared distributions.

```
p gammainc(2, 1) #  $\gamma(2, 1)$ 
```

gammaincc

gammaincc (alias **gamma_ui**) computes the upper incomplete gamma function $\Gamma(a,x) = \int_x^\infty t^{a-1} e^{-t} dt = \Gamma(a) - \gamma(a,x)$. Useful for tail probabilities.

```
p gammaincc(2, 1) #  $\Gamma(2, 1)$ 
```

gammainc_reg

gammainc_reg (alias **gamma_lr**) computes the regularized lower incomplete gamma $P(a,x) = \gamma(a,x)/\Gamma(a)$, the CDF of the gamma distribution $\text{Gamma}(a, 1)$.

```
p gammainc_reg(2, 1) # P(Gamma(2,1) < 1)
```

gammaincc_reg

gammaincc_reg (alias **gamma_ur**) computes the regularized upper incomplete gamma $Q(a,x) = 1 - P(a,x)$, the survival function of the gamma distribution.

```
p gammaincc_reg(2, 1) # P(Gamma(2,1) > 1)
```


Bessel / Airy / Hankel / Struve / Kelvin

19 topics

bessel_j0

`bessel_j0` (alias `j0`) computes the Bessel function of the first kind, order 0.

```
p j0(0)      # 1.0
p j0(2.4)    # = 0 (first zero)
```

bessel_j1

`bessel_j1` (alias `j1`) computes the Bessel function of the first kind, order 1.

```
p j1(0)      # 0.0
p j1(1.84)    # = 0.582 (first max)
```

bessel_j

`bessel_j N, X` — Bessel function of the first kind $J_n(x)$ for integer n . Generalises `bessel_j0` / `bessel_j1`.

```
p bessel_j(2, 5)    # = 0.0466
p bessel_j(0, 0)    # 1
```

bessel_y

`bessel_y N, X` — Bessel function of the second kind $Y_n(x)$. Singular at $x = 0$.

```
p bessel_y(0, 1)    # = 0.0883
p bessel_y(1, 5)    # = 0.1479
```

bessel_i

`bessel_i N, X` — Modified Bessel function $I_n(x)$. Grows exponentially for large x .

```
p bessel_i(0, 1)    # = 1.266
p bessel_i(2, 3)    # = 2.245
```

bessel_k

`bessel_k N, X` — Modified Bessel function $K_n(x)$. Decays exponentially for large x ; singular at 0.

```
p bessel_k(0, 1)    # = 0.4210
p bessel_k(1, 1)    # = 0.6019
```

hankel_h1

`hankel_h1 N, X` returns the complex Hankel $H_n^{(1)}(x) = J_n(x) + i Y_n(x)$ as `[Re, Im]`.

```
var ($re, $im) = hankel_h1(0, 1)
p "$re + i*$im"
```

hankel_h2

`hankel_h2 N, X` returns $H_n^{(2)}(x) = J_n(x) - i Y_n(x)$ as `[Re, Im]`.

bessel_j_zero

`bessel_j_zero N, K` computes the Kth positive zero of J_n . Uses McMahon expansion + Newton refinement.

```
p bessel_j_zero(0, 1) # 2.4048...
p bessel_j_zero(1, 1) # 3.8317...
```

airy_ai

`airy_ai X` — Airy function $Ai(x)$ (decaying solution of $y'' = x y$).

```
p airy_ai(0) # ≈ 0.3550
p airy_ai(2) # ≈ 0.0349
```

airy_bi

`airy_bi X` — Airy function $Bi(x)$ (growing solution).

airy_ai_prime

`airy_ai_prime X` — derivative $Ai'(x)$.

airy_bi_prime

`airy_bi_prime X` — derivative $Bi'(x)$.

spherical_bessel_j

`spherical_bessel_j N, X` — $j_n(x) = \sqrt{\pi/2x} J_{\{n+1/2\}}(x)$.

```
p spherical_bessel_j(0, 1) # sin(1)
```

spherical_bessel_y

`spherical_bessel_y N, X` — $y_n(x) = \sqrt{\pi/2x} Y_{\{n+1/2\}}(x)$.

struve_h

`struve_h N, X` — Struve function $H_n(x)$ (solution of an inhomogeneous Bessel equation).

struve_l

`struve_l N, X` — Modified Struve $L_n(x)$.

kelvin_ber

`kelvin_ber X` — Real part of $J_0(x e^{(3\pi i/4)})$ (Kelvin function ber_0).

kelvin_bei

`kelvin_bei X` — Imaginary part of $J_0(x e^{(3\pi i/4)})$ (Kelvin function bei_0).

Orthogonal Polynomials

13 topics

legendre_p

`legendre_p N, X` — Legendre polynomial $P_n(x)$. Uses the stable Bonnet recurrence $(2n+1) x P_n = (n+1) P_{n+1} + n P_{n-1}$.

```
p legendre_p(2, 0.5) # -0.125
p legendre_p(5, 1)   # 1
```

legendre_q

`legendre_q N, X` — Legendre $Q_n(x)$, the second-kind solution. NaN for $|x| \geq 1$.

assoc_legendre_p

`assoc_legendre_p N, M, X` — Associated Legendre $P_n^m(x)$. Building block for spherical harmonics.

hermite_h

`hermite_h N, X` — Physicist's Hermite $H_n(x)$: $H_0=1$, $H_1=2x$, $H_{n+1}=2x H_n - 2n H_{n-1}$.

```
p hermite_h(3, 1) # -4
```

hermite_he

`hermite_he N, X` — Probabilist's Hermite $He_n(x)$: $He_0=1$, $He_1=x$, $He_{n+1}=x He_n - n He_{n-1}$.

laguerre_l

`laguerre_l N, X` — Laguerre polynomial $L_n(x)$.

assoc_laguerre_l

`assoc_laguerre_l N, ALPHA, X` — Generalised Laguerre $L_n^\alpha(x)$.

jacobi_p

`jacobi_p N, ALPHA, BETA, X` — Jacobi polynomial $P_n^{\alpha, \beta}(x)$. Generalises Legendre/Chebyshev/Gegenbauer.

gegenbauer_c

`gegenbauer_c N, ALPHA, X` — Gegenbauer $C_n^\alpha(x)$. Reduces to Chebyshev/Legendre at specific α .

chebyshev_t

`chebyshev_t N, X` — Chebyshev polynomial of the first kind $T_n(x)$. Recurrence $T_{n+1} = 2x T_n - T_{n-1}$.

```
p chebyshev_t(3, 0.5) # -0.5
```

`chebyshev_u`

`chebyshev_u` N , X — Chebyshev polynomial of the second kind $U_n(x)$.

`spherical_harmonic_y`

`spherical_harmonic_y` L , M , $THETA$, PHI — Spherical harmonic $Y_l^m(\theta, \varphi)$ returned as $[Re, Im]$.

`zernike_r`

`zernike_r` N , M , R — Radial Zernike polynomial $R_n^m(r)$. 0 if $(n - m)$ is odd or $m > n$.

Elliptic Integrals & Functions

18 topics

elliptic_k

`elliptic_k M` — Complete elliptic integral $K(m)$, $m = k^2$. Computed via Carlson R_F .

```
p elliptic_k(0)      #  $\pi/2$ 
p elliptic_k(0.5)    #  $\approx 1.8541$ 
```

elliptic_e

`elliptic_e M` — Complete elliptic integral of the second kind $E(m)$.

elliptic_pi

`elliptic_pi N, M` — Complete elliptic integral of the third kind $\Pi(n | m)$.

elliptic_f

`elliptic_f PHI, M` — Incomplete $F(\varphi, m)$.

elliptic_e_inc

`elliptic_e_inc PHI, M` — Incomplete $E(\varphi, m)$.

elliptic_pi_inc

`elliptic_pi_inc N, PHI, M` — Incomplete $\Pi(n; \varphi | m)$.

carlson_rf

`carlson_rf X, Y, Z` — Carlson symmetric form R_F . Building block for the elliptic family.

carlson_rd

`carlson_rd X, Y, Z` — Carlson R_D .

carlson_rj

`carlson_rj X, Y, Z, P` — Carlson R_J .

jacobi_sn

`jacobi_sn U, M` — Jacobi elliptic $\text{sn}(u, m)$. Reduces to $\sin(u)$ at $m = 0$ and $\tanh(u)$ at $m = 1$.

```
p jacobi_sn(1, 0.5) #  $\approx 0.8030$ 
```

jacobi_cn

`jacobi_cn U, M` — Jacobi elliptic $\text{cn}(u, m)$. Reduces to $\cos(u)$ at $m = 0$.

`jacobi_dn`

`jacobi_dn U, M` — Jacobi elliptic $\operatorname{dn}(u, m)$. Reduces to 1 at $m = 0$.

`jacobi_am`

`jacobi_am U, M` — Jacobi amplitude φ such that $\operatorname{sn}(u, m) = \sin(\varphi)$.

`elliptic_theta`

`elliptic_theta J, Z, Q` — Jacobi theta $\theta_j(z, q)$ for $j \in \{1, 2, 3, 4\}$, $|q| < 1$.

`weierstrass_p`

`weierstrass_p Z, G2, G3` — Weierstrass $\wp(z; g_2, g_3)$ via Laurent series. Pole at $z = 0$.

`weierstrass_zeta`

`weierstrass_zeta Z, G2, G3` — Weierstrass $\zeta(z; g_2, g_3)$.

`weierstrass_sigma`

`weierstrass_sigma Z, G2, G3` — Weierstrass $\sigma(z; g_2, g_3)$.

`inverse_jacobi_sn`

`inverse_jacobi_sn X, M` — inverse of `jacobi_sn`: returns u with $\operatorname{sn}(u, m) = x$.

Zeta / Polylog / Lerch

9 topics

zeta

`zeta S` — Riemann $\zeta(s)$ via Euler-Maclaurin ($s > 0.5$) or reflection identity.

```
p zeta(2)      #  $\pi^2/6 \approx 1.6449$ 
p zeta(4)      #  $\approx 1.0823$ 
p zeta(-1)     #  $-1/12$ 
```

hurwitz_zeta

`hurwitz_zeta S, A` — Hurwitz $\zeta(s, a)$. `hurwitz_zeta(s, 1)  $\equiv$  zeta(s).`

polylog

`polylog N, Z` — Polylogarithm $\text{Li}_n(z) = \sum z^k / k^n$ for $|z| \leq 1$.

```
p polylog(2, 1)      #  $\pi^2/6$ 
```

dilog

`dilog Z` — Dilogarithm $\text{Li}_2(z)$. Convenience alias for `polylog(2, z)`.

lerch_phi

`lerch_phi Z, S, A` — Lerch transcendent $\Phi(z, s, a) = \sum z^k / (a+k)^s$.

riemann_siegel_z

`riemann_siegel_z T` — Riemann-Siegel $Z(t)$ on the critical line.

riemann_siegel_theta

`riemann_siegel_theta T` — Riemann-Siegel $\theta(t)$ (asymptotic Stirling form).

dirichlet_eta

`dirichlet_eta S` — Dirichlet $\eta(s) = (1 - 2^{1-s}) \zeta(s)$.

dirichlet_beta

`dirichlet_beta S` — Dirichlet $\beta(s) = \sum (-1)^k / (2k+1)^s$.

```
p dirichlet_beta(1)  #  $\pi/4$  (Catalan-like)
p dirichlet_beta(2)  # Catalan's constant  $\approx 0.9160$ 
```

Hypergeometric

5 topics

hypergeometric_2f1

`hypergeometric_2f1 A, B, C, Z` — ${}_2F_1(a, b; c; z)$ Gauss hypergeometric. Series convergent for $|z| < 1$.

```
p hypergeometric_2f1(0.5, 0.5, 1, 0.5) # ≈ 1.1803
```

hypergeometric_1f1

`hypergeometric_1f1 A, B, Z` — ${}_1F_1(a; b; z)$ Kummer / confluent hypergeometric.

hypergeometric_0f1

`hypergeometric_0f1 B, Z` — ${}_0F_1(; b; z)$. Related to Bessel: $J_\nu(x) = (x/2)^\nu / \Gamma(\nu+1) \cdot {}_0F_1(; \nu+1; -x^2/4)$.

hypergeometric_pfq

`hypergeometric_pfq AS, BS, Z` — generalised ${}_pF_q$. AS, BS are arrays of upper/lower parameters.

hypergeometric_u

`hypergeometric_u A, B, Z` — Tricomi's confluent $U(a, b, z)$.

Modular Forms

4 topics

dedekind_eta

`dedekind_eta` Y — Dedekind $\eta(iy)$ on the imaginary axis. Real-valued q -series with $q = \exp(-2\pi y)$.

klein_j

`klein_j` Y — Klein j -invariant $j(iy) = E_4^3 / \Delta$.

modular_lambda

`modular_lambda` Y — Modular $\lambda(iy) = \theta_2(0, q)^4 / \theta_3(0, q)^4$.

ramanujan_tau

`ramanujan_tau` N — $\tau(n)$, the n th coefficient of $\Delta(\tau) = \eta(\tau)^{24}$. Returns an integer.

```
p ramanujan_tau(1)    # 1
p ramanujan_tau(2)    # -24
p ramanujan_tau(3)    # 252
```

Si / Ci / Ei / Li / Fresnel Integrals

9 topics

sin_integral

`sin_integral` X — $\text{Si}(x) = \int_0^x \sin(t)/t \, dt$. $\text{Si}(\infty) = \pi/2$.

```
p sin_integral(1)      # ≈ 0.9461
p sin_integral(10)     # ≈ 1.6583
```

cos_integral

`cos_integral` X — $\text{Ci}(x) = \gamma + \ln(x) + \int_0^x (\cos(t)-1)/t \, dt$. NaN for $x \leq 0$.

sinh_integral

`sinh_integral` X — $\text{Shi}(x) = \int_0^x \sinh(t)/t \, dt$.

cosh_integral

`cosh_integral` X — $\text{Chi}(x) = \gamma + \ln(x) + \int_0^x (\cosh(t)-1)/t \, dt$.

exp_integral_e

`exp_integral_e` N, X — generalised exponential integral $E_{-n}(x) = \int_{-1}^{\infty} e^{-x t} / t^n \, dt$.

exp_integral_ei

`exp_integral_ei` X — $\text{Ei}(x)$. For $x > 0$ uses series + asymptotic split.

log_integral

`log_integral` X — $\text{li}(x) = \text{Ei}(\ln x)$. Approximates the prime-counting function near `prime_pi`.

```
p log_integral(1000)   # ≈ 178
```

fresnel_s

`fresnel_s` X — Fresnel $S(x) = \int_0^x \sin(\pi t^2 / 2) \, dt$.

fresnel_c

`fresnel_c` X — Fresnel $C(x) = \int_0^x \cos(\pi t^2 / 2) \, dt$.

Number Theory (Wolfram Parity)

13 topics

`jacobi_symbol`

`jacobi_symbol` A , N — Jacobi symbol (a/n) for positive odd n . Returns -1, 0, or 1.

```
p jacobi_symbol(2, 7) # 1
```

`kronecker_symbol`

`kronecker_symbol` A , N — Extension of the Jacobi symbol to all integers n .

`primitive_root`

`primitive_root` P — Smallest primitive root mod p (p prime). `undef` if none.

`multiplicative_order`

`multiplicative_order` A , N — smallest $k > 0$ with $a^k \equiv 1 \pmod{n}$. `undef` if $\gcd(a, n) \neq 1$.

`mangoldt_lambda`

`mangoldt_lambda` N — von Mangoldt $\Lambda(n) = \ln(p)$ if $n = p^k$, else 0.

`carmichael_lambda`

`carmichael_lambda` N — Carmichael $\lambda(n)$, the exponent of $(\mathbb{Z}/n\mathbb{Z})^*$.

```
p carmichael_lambda(15) # 4
```

`squares_r`

`squares_r` K , N — number of representations of n as a sum of k squares (signed, ordered).

```
p squares_r(2, 5) # 8 (e.g.  $\pm 1^2 + \pm 2^2$ )
```

`thue_morse`

`thue_morse` N — Thue-Morse $t(n) = \text{popcount}(n) \bmod 2$.

`rudin_shapiro`

`rudin_shapiro` N — Rudin-Shapiro sequence, $\pm 1 = (-1)^{a(n)}$ where $a(n)$ counts "11" patterns in $\text{binary}(n)$.

`farey_sequence`

`farey_sequence` N — Farey sequence F_n . Returns array of $[a, b]$ pairs.

```
p farey_sequence(3) # [[0,1],[1,3],[1,2],[2,3],[1,1]]
```

frobenius_number

`frobenius_number COINS` — largest amount NOT representable as a non-negative combination of `COINS`. Closed form for two coprime denominations.

```
p frobenius_number([3, 5]) # 7
```

frobenius_solve

`frobenius_solve COINS, N` — number of ways to make change for N using `COINS`.

stern_brocot

`stern_brocot N` — Stern-Brocot tree node N (1-indexed) as `[a, b]` (Calkin-Wilf order).

Combinatorics (Wolfram Parity)

8 topics

stirling_s1

`stirling_s1` N , K — unsigned Stirling number of the first kind $|s(n,k)|$.

bell_polynomial_b

`bell_polynomial_b` N , K , $X5$ — partial Bell polynomial $B_{\{n,k\}}(x_1, \dots, x_{\{n-k+1\}})$.

clebsch_gordan

`clebsch_gordan` $J1$, $J2$, J , $M1$, $M2$ [$, M$] — Clebsch-Gordan coefficient. M defaults to $M1 + M2$.

three_j_symbol

`three_j_symbol` $J1$, $J2$, $J3$, $M1$, $M2$, $M3$ — Wigner 3-j symbol.

six_j_symbol

`six_j_symbol` $J1$ $J2$ $J3$ $J4$ $J5$ $J6$ — Wigner 6-j symbol.

nine_j_symbol

`nine_j_symbol` (9 args) — Wigner 9-j as a single sum over 6-j.

debruijn_sequence

`debruijn_sequence` K , N — De Bruijn $B(k, n)$ as an array of k^n integers.

wigner_d

`wigner_d` J , $M1$, $M2$, $BETA$ — small Wigner d-matrix $d^J_{\{m1, m2\}}(\beta)$.

q-Series, Mittag-Leffler, Coulomb

7 topics

`q_pochhammer`

`q_pochhammer A, Q [, N]` — $(a; q)_n$. Omit N for the infinite product ($|q| < 1$).

`q_factorial`

`q_factorial N, Q` — q-factorial $[n]_q!$.

`q_binomial`

`q_binomial N, K, Q` — Gaussian binomial $[n \text{ choose } k]_q$.

`q_hypergeometric_pfq`

`q_hypergeometric_pfq A5, B5, Q, Z` — basic-hypergeometric series.

`mittag_leffler_e`

`mittag_leffler_e ALPHA, BETA, Z` — $E_{\{\alpha, \beta\}}(z) = \sum z^k / \Gamma(\alpha k + \beta)$. Generalises exp / cosh / Mittag-Leffler.

`coulomb_wave_f`

`coulomb_wave_f L, ETA, RHO` — regular Coulomb wave $F_L(\eta, \rho)$ via the confluent-hypergeometric representation.

`coulomb_wave_g`

`coulomb_wave_g L, ETA, RHO` — irregular partner G_L . Currently returns NaN; faithful Bardin/Goesnig port pending.

Inverse Special Functions

4 topics

`inverse_erf`

`inverse_erf` Y — inverse error function. Solves $\text{erf}(x) = y$.

```
p inverse_erf(0.5)  # ≈ 0.4769
```

`inverse_erfc`

`inverse_erfc` Y — inverse complementary error function.

`inverse_gamma_regularized`

`inverse_gamma_regularized` A, Y — solves $P(a, x) = y$ for x .

`inverse_beta_regularized`

`inverse_beta_regularized` A, B, Y — solves $I_x(a, b) = y$ for x via bisection.

Piecewise & Symbolic Primitives

8 topics

`dirac_delta`

`dirac_delta X [, EPS]` — finite-width discrete approximation: $1/\text{eps}$ if $|x| < \text{eps}/2$ else 0. EPS defaults to $1e-3$.

`heaviside_theta`

`heaviside_theta X` — Heaviside step function: 1 ($x > 0$), 0 ($x < 0$), 0.5 ($x = 0$).

`unit_box`

`unit_box X` — 1 if $|x| \leq 1/2$ else 0.

`unit_triangle`

`unit_triangle X` — $\max(1 - |x|, 0)$.

`square_wave`

`square_wave X [, PERIOD]` — alternates $+1/-1$ with the given period (default 1).

`triangle_wave`

`triangle_wave X [, PERIOD]` — periodic triangle wave on $[-1, 1]$.

`sawtooth_wave`

`sawtooth_wave X [, PERIOD]` — periodic sawtooth on $[-1, 1]$.

`dirac_comb`

`dirac_comb X, T [, EPS]` — sum of Dirac deltas at multiples of T (discrete approximation).

Number Theory (Wolfram Parity II)

14 topics

liouville_lambda

`liouville_lambda` N — Liouville $\lambda(n) = (-1)^{\Omega(n)}$.

jordan_totient

`jordan_totient` K, N — Jordan totient $J_k(n) = n^k \prod_{p \mid n} (1 - p^{-k})$.

ramanujan_sum

`ramanujan_sum` Q, N — $c_q(n) = \sum_{d \mid \gcd(q,n)} \mu(q/d) d$.

cyclotomic_polynomial

`cyclotomic_polynomial` N — coefficient list of $\Phi_n(x)$.

```
p cyclotomic_polynomial(6)  # [1, -1, 1] = x^2-x+1
```

legendre_symbol

`legendre_symbol` A, P — Legendre symbol (a/p) for prime p . -1, 0, or 1.

pythagorean_triple_q

`pythagorean_triple_q` A, B, C — 1 if $a^2 + b^2 = c^2$ (any order/sign).

gen_pythagorean_triple

`gen_pythagorean_triple` M, N — Euclid formula: returns $[m^2-n^2, 2mn, m^2+n^2]$.

sophie_germain_q

`sophie_germain_q` P — 1 if both p and $2p+1$ are prime.

mersenne_q

`mersenne_q` N — 1 if $2^n - 1$ is a Mersenne prime (Lucas-Lehmer).

lucas_lehmer_test

`lucas_lehmer_test` P — Lucas-Lehmer primality test for $2^p - 1$.

continued_fraction

`continued_fraction` $X [, N]$ — first N coefficients of the simple continued-fraction expansion. N defaults to 12.

`from_continued_fraction`

`from_continued_fraction COEFFS` — evaluate $[a_0; a_1, a_2, \dots]$ back to a real number.

`convergents`

`convergents X [, N]` — N successive $[\text{num}, \text{denom}]$ convergents.

`best_rational_approximation`

`best_rational_approximation X, MAX_DENOM` — closest fraction p/q with $q \leq \text{MAX_DENOM}$. Returns $[p, q]$.

Combinatorial Sequences

12 topics

motzkin_number

motzkin_number N — # paths on $(0..n)$ using $\rightarrow \nearrow \searrow$ steps (no crossings).

narayana_number

narayana_number N, K — $N(n, k) = (1/n) C(n, k) C(n, k-1)$.

delannoy_number

delannoy_number M, N — # paths in $(m+1) \times (n+1)$ grid using $\rightarrow \uparrow \nearrow$.

schroder_number

schroder_number N — large Schröder number.

small_schroder_number

small_schroder_number N — small Schröder $s_n = S_n / 2$ ($n \geq 1$).

eulerian_number

eulerian_number N, K — $\langle n \ k \rangle$, # permutations of $[n]$ with k ascents.

bernoulli_polynomial

bernoulli_polynomial N, X — $B_n(x)$.

euler_polynomial

euler_polynomial N, X — $E_n(x)$.

pell_number

pell_number N — Pell number P_n .

pell_lucas_number

pell_lucas_number N — companion Pell-Lucas Q_n .

perrin_number

perrin_number N — Perrin sequence $P(n) = P(n-2) + P(n-3)$.

padovan_number

padovan_number N — Padovan sequence (same recurrence, different seeds).

Linear Algebra (Extended)

8 topics

`kronocker_product`

`kronocker_product A, B` — $A \otimes B$ (block-by-block outer product).

`tensor_product`

`tensor_product A, B` — outer product of two flat vectors $\rightarrow$ matrix $A_i B_j$.

`tensor_contract`

`tensor_contract A, B` — Frobenius inner product of two matrices.

`matrix_rank`

`matrix_rank A` — rank via Gaussian elimination with partial pivoting.

`companion_matrix`

`companion_matrix [a0, a1, ..., a_{n-1}]` — companion of $x^n + a_{n-1} x^{n-1} + \dots + a_0$.

`characteristic_polynomial`

`characteristic_polynomial A` — coefficients $[a_0, \dots, a_n]$ of $\det(xI - A)$ via Faddeev-LeVerrier.

`singular_values`

`singular_values A` — sorted SVDs of A (eigenvalues of $A^T A$, square-rooted).

`nullspace`

`nullspace A` — basis vectors of A 's null space (zero singular values of $A^T A$).

Polynomial Algebra

6 topics

`polynomial_gcd`

`polynomial_gcd P, Q` — Euclidean GCD over real coefficients (monic).

```
p polygcd([-1, 0, 1], [-1, 1])  #  $x - 1$ 
```

`polynomial_quotient`

`polynomial_quotient P, Q` — quotient of synthetic division.

`polynomial_remainder`

`polynomial_remainder P, Q` — remainder of synthetic division.

`polynomial_resultant`

`polynomial_resultant P, Q` — Sylvester-matrix determinant.

`polynomial_discriminant`

`polynomial_discriminant P` — $\text{disc}(p) = (-1)^{\{n(n-1)/2\}} / a_n \cdot \text{res}(p, p')$.

`polynomial_roots`

`polynomial_roots P` — real roots via deflation power iteration on companion matrix.

Distributions (Extended)

18 topics

`gumbel_pdf`

`gumbel_pdf` X , MU , $BETA$ — Gumbel (Type I) extreme-value PDF.

`gumbel_cdf`

`gumbel_cdf` X , MU , $BETA$ — Gumbel CDF.

`gumbel_quantile`

`gumbel_quantile` P , MU , $BETA$ — Gumbel quantile.

`frechet_pdf`

`frechet_pdf` X , $ALPHA$, S — Fréchet (Type II) PDF.

`frechet_cdf`

`frechet_cdf` X , $ALPHA$, S — Fréchet CDF.

`frechet_quantile`

`frechet_quantile` P , $ALPHA$, S — Fréchet quantile.

`logistic_pdf`

`logistic_pdf` X , MU , S — Logistic PDF.

`logistic_cdf`

`logistic_cdf` X , MU , S — Logistic CDF.

`logistic_quantile`

`logistic_quantile` P , MU , S — Logistic quantile.

`rayleigh_pdf`

`rayleigh_pdf` X , $SIGMA$ — Rayleigh PDF ($x \geq 0$).

`rayleigh_cdf`

`rayleigh_cdf` X , $SIGMA$ — Rayleigh CDF.

`rayleigh_quantile`

`rayleigh_quantile` P , $SIGMA$ — Rayleigh quantile.

`inverse_gamma_pdf`

`inverse_gamma_pdf` X , ALPHA , BETA — inverse-gamma PDF.

`inverse_gamma_cdf`

`inverse_gamma_cdf` X , ALPHA , BETA — inverse-gamma CDF.

`inverse_gamma_quantile`

`inverse_gamma_quantile` P , ALPHA , BETA — inverse-gamma quantile.

`kumaraswamy_pdf`

`kumaraswamy_pdf` X , A , B — Kumaraswamy PDF on $(0, 1)$.

`kumaraswamy_cdf`

`kumaraswamy_cdf` X , A , B — Kumaraswamy CDF.

`kumaraswamy_quantile`

`kumaraswamy_quantile` P , A , B — Kumaraswamy quantile.

Mathieu / Heun

4 topics

`mathieu_a`

`mathieu_a Q, N` — Mathieu characteristic value $a_n(q)$ (Hill-matrix eigenvalue).

`mathieu_ce`

`mathieu_ce N, X, Q` — even Mathieu function $ce_n(x, q)$ (perturbative form for small q).

`mathieu_se`

`mathieu_se N, X, Q` — odd Mathieu function $se_n(x, q)$.

`heun_g`

`heun_g Z, A, Q, ALPHA, BETA, GAMMA, DELTA` — general Heun H_I via Frobenius series. Requires $|z| < \min(1, |a|)$.

Wavelets

4 topics

haar_transform

`haar_transform X5` — single-level Haar DWT. Returns [approx..., detail...].

haar_inverse

`haar_inverse X5` — inverse single-level Haar.

daubechies_db4

`daubechies_db4 X5` — single-level Daubechies-4 DWT (length must be even ≥ 4).

daubechies_db4_inverse

`daubechies_db4_inverse X5` — inverse single-level db4.

Graph Algorithms (Extended)

8 topics

`topo_sort_adj`

`topo_sort_adj ADJ` — Kahn's algorithm on adjacency-list graph. Returns ordering or `[]` on cycle. (Edge-list input → use `topological_sort` instead.)

`scc_tarjan`

`scc_tarjan ADJ` — Tarjan strongly-connected components. Returns array of components.

`bipartite_q`

`bipartite_q ADJ` — 1 if graph is 2-colourable (BFS), 0 otherwise.

`max_flow_edmonds_karp`

`max_flow_edmonds_karp CAP, S, T` — max flow on capacity matrix from S to T.

`min_cut`

`min_cut CAP, S, T` — min-cut value (= max-flow).

`eccentricity`

`eccentricity ADJ, V` — max BFS distance from V (unweighted).

`graph_diameter`

`graph_diameter ADJ` — max eccentricity over all vertices.

`graph_radius`

`graph_radius ADJ` — min eccentricity over all vertices.

Number-Theoretic Sums & Constants

5 topics

`stieltjes_constant`

`stieltjes_constant` K — k th Stieltjes constant γ_k . $\gamma_0 = \gamma$ (Euler-Mascheroni).

`gauss_sum`

`gauss_sum` A, P — quadratic Gauss sum $G(a, p)$ as `[Re, Im]`.

`kloosterman_sum`

`kloosterman_sum` A, B, M — $K(a, b; m)$ as `[Re, Im]`.

`eta_quotient`

`eta_quotient` `PAIRS, Y` — $\prod \eta(d \cdot iy)^{\{r_d\}}$ for `[[d, r_d], ...]`.

`root_approximant`

`root_approximant` X `[, MAX_DENOM]` — best rational approximant (alias for `best_rational_approximation`).

Vector Calculus

6 topics

numerical_gradient

`numerical_gradient F, POINT [ , H]` — central-difference ∇f . `F` receives a single arrayref `_0`.

```
fn quad { val @y = @[_0]; $y[0]**2 + 2*$y[1]**2 }  
var @g = ngrad(\6quad, [3, 4])      # (6, 16)
```

numerical_jacobian

`numerical_jacobian F, POINT [ , H]` — Jacobian of vector-valued `F`.

numerical_hessian

`numerical_hessian F, POINT [ , H]` — symmetric Hessian via mixed second differences.

numerical_divergence

`numerical_divergence F, POINT [ , H]` — div of vector field `F`.

numerical_curl

`numerical_curl F, POINT [ , H]` — 3-D curl returned as `[c_x, c_y, c_z]`.

numerical_laplacian

`numerical_laplacian F, POINT [ , H]` — $\nabla^2 f$.

Optimization

6 topics

nelder_mead

`nelder_mead F, X0 [, MAX_ITER, TOL]` — Nelder-Mead simplex minimisation. Returns $[x^*, f(x^*)]$.

gradient_descent

`gradient_descent F, GRAD, X0 [, STEP, ITERS]` — fixed-step descent.

bfgs_minimize

`bfgs_minimize F, GRAD, X0 [, MAX_ITER, TOL]` — BFGS quasi-Newton with Armijo line search.

levenberg_marquardt

`levenberg_marquardt F_RESID, X0 [, MAX_ITER, TOL]` — nonlinear LSQ. `F` returns the residual vector.

conjugate_gradient

`conjugate_gradient A, B` — CG for symmetric positive definite $Ax = b$.

least_squares

`least_squares A, B` — solves $A^T A x = A^T b$.

Numerical Integration

4 topics

romberg

`romberg F, A, B [, LEVELS]` — Romberg integration. Doubles trapezoid then Richardson-extrapolates.

gauss_legendre_quad

`gauss_legendre_quad F, A, B [, N]` — N-point Gauss-Legendre quadrature on [a, b] (default N=10).

monte_carlo_integrate

`monte_carlo_integrate F, A, B [, N]` — uniform Monte Carlo over [a, b] with N samples (default 10 000).

adaptive_simpson

`adaptive_simpson F, A, B [, TOL, MAX_DEPTH]` — adaptive Simpson with Richardson tail.

Linear Algebra (Numeric)

7 topics

`lu_decompose`

`lu_decompose A` $\rightarrow [L, U, P]$ with $PA = LU$; P stored as a row-permutation vector.

`qr_decompose`

`qr_decompose A` $\rightarrow [Q, R]$ via Gram-Schmidt.

`householder_reflector`

`householder_reflector V` $\rightarrow I - 2vv^T / (v \cdot v)$.

`givens_rotation`

`givens_rotation A, B` $\rightarrow [c, s]$ such that the rotation $[c \ -s; s \ c]$ zeros the second component.

`forward_substitute`

`forward_substitute L, B` — solve $Lx = b$ for lower-triangular L .

`back_substitute`

`back_substitute U, B` — solve $Ux = b$ for upper-triangular U .

`hessenberg_reduce`

`hessenberg_reduce A` — orthogonal Hessenberg form via Householder reflections.

Polynomial Helpers

5 topics

`poly_derivative`

`poly_derivative P` — coefficient-list derivative of polynomial `P` (low-to-high order).

`poly_integrate`

`poly_integrate P [, C]` — antiderivative with constant `C` (default 0).

`poly_compose`

`poly_compose P, Q` → coefficients of $P(Q(x))$.

`poly_eval_horner`

`poly_eval_horner P, X` — Horner-rule polynomial evaluation.

`pade_approximant`

`pade_approximant TAYLOR, M, N` → `[num, den]` Padé approximation.

Quaternions & 3D Rotations

12 topics

`quat_mul`

`quat_mul A, B` — Hamilton product (w, x, y, z).

`quat_conj`

`quat_conj Q` — conjugate (w, -x, -y, -z).

`quat_norm`

`quat_norm Q` — Euclidean norm.

`quat_inv`

`quat_inv Q` — inverse $Q^* / |Q|^2$.

`quat_from_axis_angle`

`quat_from_axis_angle AXIS, THETA` — unit quaternion encoding rotation about AXIS by THETA radians.

`quat_to_axis_angle`

`quat_to_axis_angle Q` → [axis, theta].

`quat_to_matrix`

`quat_to_matrix Q` → 3×3 rotation matrix.

`quat_from_matrix`

`quat_from_matrix R` — Shepherd's algorithm.

`quat_slerp`

`quat_slerp A, B, T` — spherical-linear interpolation between two unit quaternions.

`euler_zyx_to_matrix`

`euler_zyx_to_matrix YAW, PITCH, ROLL` — intrinsic Z-Y-X Euler rotation.

`matrix_to_euler_zyx`

`matrix_to_euler_zyx R` → [yaw, pitch, roll].

`rotate_3d_vec`

`rotate_3d_vec Q, V` — apply quaternion rotation to a 3-vector.

Information Theory

6 topics

`kl_divergence`

`kl_divergence P, Q` — $D_{\text{KL}}(P \parallel Q)$ in nats.

`js_divergence`

`js_divergence P, Q` — Jensen-Shannon divergence.

`mutual_information`

`mutual_information JOINT` — $I(X; Y)$ from a joint-probability matrix.

`cross_entropy_arr`

`cross_entropy_arr P, Q` — $H(P, Q) = -\sum p \log q$.

`renyi_entropy`

`renyi_entropy P [, ALPHA]` — Rényi entropy of order α (default 2).

`tsallis_entropy`

`tsallis_entropy P [, Q]` — Tsallis entropy of order q (default 2).

Quantum Primitives

11 topics

`pauli_x`

`pauli_x` — 2×2 σ_x matrix.

`pauli_y`

`pauli_y` — 2×2 σ_y matrix encoded with [Re, Im] entries (since σ matrices are real).

`pauli_z`

`pauli_z` — 2×2 σ_z matrix.

`pauli_id`

`pauli_id` — 2×2 identity.

`ket_bra`

`ket_bra` `PSI`, `PHI` → outer product $|\psi\rangle\langle\phi|$ of two real vectors.

`density_matrix`

`density_matrix` `PSI` → $\rho = |\psi\rangle\langle\psi|$ (real-valued).

`expectation_value`

`expectation_value` `OP`, `PSI` → $\langle\psi|O|\psi\rangle$.

`commutator`

`commutator` `A`, `B` — $[A, B] = AB - BA$.

`anticommutator`

`anticommutator` `A`, `B` — $\{A, B\} = AB + BA$.

`partial_trace`

`partial_trace` `RHO`, `DIMS`, `KEEP` — partial trace over subsystems whose indices are not in `KEEP`.

`von_neumann_entropy`

`von_neumann_entropy` `RHO` — $S(\rho) = -\text{tr}(\rho \log \rho)$ for symmetric ρ .

Statistical Mechanics

7 topics

bose_einstein

bose_einstein E , μ , KT — $1 / (\exp((E - \mu)/kT) - 1)$.

fermi_dirac

fermi_dirac E , μ , KT — $1 / (\exp((E - \mu)/kT) + 1)$.

maxwell_boltzmann_speed

maxwell_boltzmann_speed V , M , KT — Maxwell-Boltzmann speed PDF.

partition_function

partition_function $ENERGIES$, KT — $Z = \sum \exp(-E_i / kT)$.

helmholtz_free_energy

helmholtz_free_energy Z , KT — $F = -kT \ln Z$.

boltzmann_factor

boltzmann_factor E , KT — $\exp(-E / kT)$.

einstein_specific_heat

einstein_specific_heat T , $THETA_E$ — Einstein heat-capacity model (J / mol·K).

Optics

7 topics

fresnel_reflection_te

fresnel_reflection_te THETA_I, N1, N2 — TE (s-polarisation) reflection coefficient.

fresnel_reflection_tm

fresnel_reflection_tm THETA_I, N1, N2 — TM (p-polarisation) reflection.

fresnel_transmission_te

fresnel_transmission_te THETA_I, N1, N2 — TE transmission.

fresnel_transmission_tm

fresnel_transmission_tm THETA_I, N1, N2 — TM transmission.

abcd_thin_lens

abcd_thin_lens F — ABCD ray matrix for a thin lens of focal length F.

abcd_free_space

abcd_free_space D — ABCD matrix for free-space propagation D.

gaussian_beam_q

gaussian_beam_q Z, W0, LAMBDA $\rightarrow [\text{Re}(q), \text{Im}(q)]$ for a Gaussian beam at distance Z.

Astrodynamics

9 topics

kepler_solve

`kepler_solve` M , E — solve Kepler's equation $M = E - e \sin(E)$ by Newton iteration.

true_to_eccentric

`true_to_eccentric` NU , E → eccentric anomaly E .

eccentric_to_mean

`eccentric_to_mean` E , ECC → mean anomaly M .

julian_date

`julian_date` Y , M , D , H , MN , S — Julian date (Meeus formula).

jd_to_gregorian

`jd_to_gregorian` JD → $[Y, M, D.dd]$.

sidereal_time_gmst

`sidereal_time_gmst` JD — Greenwich Mean Sidereal Time in radians.

vis_viva

`vis_viva` R , A , MU — orbital speed from vis-viva equation.

orbital_period_kepler

`orbital_period_kepler` A , MU — $T = 2\pi \sqrt{a^3 / \mu}$.

orbital_elements_to_state

`orbital_elements_to_state` A , E , I , OM , W , NU , MU → $[r_x, r_y, r_z, v_x, v_y, v_z]$.

Time Series

4 topics

`kalman_step`

`kalman_step X, P, F, H, Q, R, Z` $\rightarrow$ `[X', P']` — single Kalman predict-and-update step.

`exponential_smoothing`

`exponential_smoothing XS, ALPHA` — first-order EMA.

`holt_winters`

`holt_winters XS, ALPHA, BETA, GAMMA, PERIOD` — additive triple exponential smoothing.

`arma_yw_fit`

`arma_yw_fit XS, P` — Yule-Walker AR(p) coefficients via Levinson-Durbin.

Graph Centrality

6 topics

pagerank

`pagerank ADJ [, DAMPING, ITERS]` — PageRank via power iteration. Default damping 0.85, 100 iters.

betweenness centrality

`betweenness centrality ADJ` — Brandes' algorithm (unweighted).

closeness centrality

`closeness centrality ADJ` — closeness centrality. 0 for disconnected vertices.

eigenvector centrality

`eigenvector centrality ADJ` — power iteration on the adjacency matrix.

degree centrality

`degree centrality ADJ` — $\text{degree} / (n - 1)$.

triangle count

`triangle count ADJ` — number of triangles incident at each vertex.

Random Samplers (Extended)

6 topics

`rgumbel`

`rgumbel MU, BETA` — Gumbel sample.

`rfrechet`

`rfrechet ALPHA, S` — Fréchet sample.

`rrayleigh`

`rrayleigh SIGMA` — Rayleigh sample.

`rlogistic`

`rlogistic MU, S` — logistic sample.

`rkumaraswamy`

`rkumaraswamy A, B` — Kumaraswamy sample on $(0, 1)$.

`rinverse_gamma`

`rinverse_gamma ALPHA, BETA` — inverse-gamma sample (Marsaglia-Tsang).

2-D Geometry

3 topics

`graham_scan`

`graham_scan` `POINTS` — 2-D convex hull (CCW).

`line_line_intersect_2d`

`line_line_intersect_2d` `P1`, `P2`, `P3`, `P4` → intersection `[x, y]` or `undef` when parallel.

`point_segment_distance`

`point_segment_distance` `P`, `A`, `B` — Euclidean distance from `P` to segment `AB`.

Auto-Differentiation

2 topics

`forward_diff`

`forward_diff F, X` — derivative of scalar f at scalar x (Julia ForwardDiff style; central-difference fallback).

`forward_diff_grad`

`forward_diff_grad F, X_VEC` — gradient at vector x ; alias for `numerical_gradient`.

Statistical Tests (R parity)

12 topics

`bartlett_test`

`bartlett_test GROUPS` — Bartlett's k-sample variance equality test. Returns `[chi2, df, p]`.

`levene_test`

`levene_test GROUPS` — Levene's variance-equality test. Returns `[F, df1, df2, p]`.

`fishers_exact_test_2x2`

`fishers_exact_test_2x2 [[a, b], [c, d]]` — two-sided p-value.

`mcnemar_test`

`mcnemar_test [[a, b], [c, d]]` — paired 2×2 table with continuity correction. Returns `[chi2, p]`.

`runs_test`

`runs_test SEQ` — Wald-Wolfowitz runs test on a binary sequence. Returns `[Z, p]`.

`friedman_test`

`friedman_test BLOCKS_BY_TREATMENTS` — non-parametric ANOVA on ranks. Returns `[Q, df, p]`.

`kruskal_wallis_test`

`kruskal_wallis_test GROUPS` — non-parametric ANOVA. Returns `[H, df, p]`.

`sign_test`

`sign_test PAIRS` — paired sign test. Returns `[N+, p]`.

`anderson_darling_normality`

`anderson_darling_normality X5` — A² statistic. > 0.752 → reject normality at α=0.05.

`jarque_bera_test`

`jarque_bera_test X5` — normality test from sample skewness + kurtosis.

`ljung_box_test`

`ljung_box_test X5 [, H]` — autocorrelation test. Returns `[Q, h, p]`.

`durbin_watson_stat`

`durbin_watson_stat RESID` — DW autocorrelation statistic on residuals.

Distance Metrics (Distances.jl parity)

6 topics

`mahalanobis_distance`

`mahalanobis_distance` X , MU , $SIGMA$ — $\sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$.

`cosine_distance`

`cosine_distance` A , B — $1 - \cos(A, B)$.

`canberra_distance`

`canberra_distance` A , B — $\sum |a-b| / (|a|+|b|)$.

`bray_curtis_distance`

`bray_curtis_distance` A , B — $\sum |a-b| / \sum(a+b)$.

`l1_distance`

`l1_distance` A , B — Manhattan / taxi-cab distance.

`chi_squared_distance`

`chi_squared_distance` A , B — $\frac{1}{2} \sum (a-b)^2 / (a+b)$.

Multivariate / Non-Central Distributions

9 topics

`multivariate_normal_pdf`

`multivariate_normal_pdf` X , MU , SIGMA — multivariate Gaussian PDF.

`multivariate_normal_sample`

`multivariate_normal_sample` MU , SIGMA — sample from MVN via Cholesky.

`dirichlet_pdf`

`dirichlet_pdf` X , ALPHA — Dirichlet PDF on the simplex.

`dirichlet_sample`

`dirichlet_sample` ALPHA — sample via independent Gamma draws.

`skellam_pmf`

`skellam_pmf` K , MU1 , MU2 — difference of two independent Poisson(μ) variates.

`inverse_gaussian_pdf`

`inverse_gaussian_pdf` X , MU , LAMBDA — Wald PDF.

`inverse_gaussian_cdf`

`inverse_gaussian_cdf` X , MU , LAMBDA — Wald CDF.

`inverse_gaussian_sample`

`inverse_gaussian_sample` MU , LAMBDA — Michael-Schucany-Haas.

`non_central_chi2_pdf`

`non_central_chi2_pdf` X , K , LAMBDA — Series-form non-central χ^2 PDF.

Matrix Functions

5 topics

`matrix_exp`

`matrix_exp A` — e^A via 13-term Padé + scaling-and-squaring.

`matrix_log`

`matrix_log A` — $\ln A$ via inverse scaling (Denman-Beavers SQRT) + Padé log.

`matrix_sqrt`

`matrix_sqrt A` — $A^{(1/2)}$ via Denman-Beavers iteration.

`matrix_sin`

`matrix_sin A` — series form $\sum (-1)^k A^{(2k+1)} / (2k+1)!$.

`matrix_cos`

`matrix_cos A` — series form $\sum (-1)^k A^{(2k)} / (2k)!$.

Adaptive ODE Solvers

4 topics

`rk45_dormand_prince`

`rk45_dormand_prince F, Y0, T0, T1, H0 [, TOL]` — adaptive Dormand-Prince. `F` receives `[t, y0, ...]`.

`midpoint_step`

`midpoint_step F, Y, T, H` — single explicit-midpoint step.

`heun_step`

`heun_step F, Y, T, H` — improved-Euler / Heun step.

`verlet_step`

`verlet_step ACCEL, Q, P, T, H` — velocity-Verlet symplectic step.

Generalized Linear Models

4 topics

`logistic_regression`

`logistic_regression X, Y` — IRLS (no intercept added — supply column of ones).

`poisson_regression`

`poisson_regression X, Y` — IRLS with log link.

`ridge_regression`

`ridge_regression X, Y [, LAMBDA]` — Tikhonov-regularised LSQ.

`lasso_coord`

`lasso_coord X, Y [, LAMBDA, MAX_ITER]` — coordinate-descent LASSO.

Bootstrap & Resampling

3 topics

`bootstrap_mean_ci`

`bootstrap_mean_ci DATA [, B, ALPHA]` — percentile CI for the mean.

`jackknife_estimate`

`jackknife_estimate DATA` — leave-one-out SE of the mean.

`permutation_test_diff`

`permutation_test_diff A, B [, N_ITER]` — two-sided p of mean difference.

Time Series Operators

4 topics

`acf_at_lag`

`acf_at_lag X5, LAG` — autocorrelation at one lag.

`diff_op`

`diff_op X5 [, LAG]` — $y_t - y_{t-lag}$.

`lag_op`

`lag_op X5 [, K]` — shift by k (NaN-pad).

`decompose_classical`

`decompose_classical X5 [, PERIOD]` — additive [trend, season, residual].

Combinatorial Generators

4 topics

`combinations_list`

`combinations_list N, K` — all k-subsets of 0..n in lex order.

`permutations_list`

`permutations_list N` — all permutations of 0..n via Heap's algorithm.

`cyclic_permutations`

`cyclic_permutations ARR` — all rotations.

`subsets_of_size`

`subsets_of_size ARR, K` — all k-subsets of an arbitrary array.

Dynamic Programming

5 topics

`longest_increasing_subseq`

`longest_increasing_subseq` XS — length only ($O(n \log n)$).

`knapsack_01`

`knapsack_01` $ITEMS$, $CAPACITY$ — items as $[[w, v], \dots]$.

`subset_sum_target`

`subset_sum_target` ARR , T — 1 if any subset sums to T .

`coin_change_min`

`coin_change_min` $COINS$, N — min number of coins to make N (-1 if impossible).

`edit_distance_levenshtein`

`edit_distance` A , B — Levenshtein distance.

ML Metrics

5 topics

one_hot_encode

`one_hot_encode LABELS [, N_CLASSES]` → $n \times K$ matrix.

label_encode

`label_encode ARR` → `[indices, classes]`.

categorical_crossentropy

`categorical_crossentropy Y_TRUE, Y_PRED` — averaged over batch.

classification_metrics

`classification_metrics TP, FP, FN, TN` → `[acc, prec, recall, f1]`.

roc_auc

`roc_auc SCORES, LABELS` — Mann-Whitney U / $(n+ n-)$.

DSP / Image Filters

6 topics

`gaussian_blur_kernel`

`gaussian_blur_kernel SIGMA [ , RADIUS]` — 1-D normalized kernel.

`sobel_x`

`sobel_x` — horizontal Sobel kernel.

`sobel_y`

`sobel_y` — vertical Sobel kernel.

`prewitt_x`

`prewitt_x` — horizontal Prewitt.

`prewitt_y`

`prewitt_y` — vertical Prewitt.

`laplacian_of_gaussian`

`laplacian_of_gaussian SIGMA` — 2-D LoG kernel.

Stochastic Processes

4 topics

`brownian_path`

`brownian_path T, N` — discrete Wiener path on $[0, T]$.

`geometric_brownian_path`

`geometric_brownian_path S0, MU, SIGMA, T, N` — GBM sample path.

`poisson_process`

`poisson_process LAMBDA, T` — homogeneous Poisson arrival times.

`random_walk_1d`

`random_walk_1d N [, P]` — ± 1 walk over N steps.

Compression / Information Complexity

3 topics

`lempel_ziv_complexity`

`lempel_ziv_complexity` SEQ — LZ76 production count.

`huffman_code_lengths`

`huffman_code_lengths` FREQS — code-length per symbol.

`shannon_entropy_rate`

`shannon_entropy_rate` SEQ [, M] — $H_m - H_{\{m-1\}}$ block estimate.

Physics & Quantum (Extended)

5 topics

`planck_blackbody`

`planck_blackbody` `LAMBDA`, `T` — spectral radiance $B(\lambda, T)$.

`rayleigh_jeans`

`rayleigh_jeans` `LAMBDA`, `T` — long-wavelength approx.

`compton_shift`

`compton_shift` `THETA` — photon wavelength shift $\Delta\lambda = (h/m_e c)(1-\cos\theta)$.

`rydberg_energy`

`rydberg_energy` `N` — hydrogen level energy $-13.605693/n^2$ eV.

`hydrogen_radial_wavefunction`

`hydrogen_radial_wavefunction` `N`, `L`, `R` — atomic-units $R_{\{n,l\}}(r)$.

Number Theory & Algebra (Extended)

5 topics

`integer_log`

`integer_log N [, BASE]` — largest k with $\text{base}^k \leq n$.

`aks_primality`

`aks_primality N` — deterministic primality (perfect-power + small-factor + Miller-Rabin fallback).

`elliptic_curve_add`

`elliptic_curve_add P, Q, A, B` — group law on $y^2 = x^3 + a x + b$. Identity = `[NaN, NaN]`.

`berlekamp_massey`

`berlekamp_massey SEQ` — minimal-LFSR connection-polynomial coefficients.

`bezout_coefficients`

`bezout_coefficients A, B` → `[gcd, x, y]` with $a x + b y = \text{gcd}$.

CAS-Lite

3 topics

`factor_quadratic`

`factor_quadratic A, B, C` — real roots of $a x^2 + b x + c = 0$.

`complete_square`

`complete_square A, B, C` $\rightarrow [h, k]$ for a $(x-h)^2 + k$.

`partial_fraction_simple`

`partial_fraction_simple NUM, ROOTS` — Heaviside cover-up residues.

Quadrature (Specialised)

7 topics

`gauss_chebyshev_quad`

`gauss_chebyshev_quad F [ , N]` — $\int_{-1}^1 f(x)/\sqrt{1-x^2} \, dx$ (default $N=16$).

`gauss_hermite_quad`

`gauss_hermite_quad F [ , N]` — $\int_{-\infty}^{\infty} f(x) e^{-x^2} \, dx$ (default $N=20$).

`gauss_laguerre_quad`

`gauss_laguerre_quad F [ , N]` — $\int_0^{\infty} f(x) e^{-x} \, dx$ (default $N=20$).

`clenshaw_curtis_quad`

`clenshaw_curtis_quad F, A, B [ , N]` — Chebyshev-node quadrature.

`tanh_sinh_quad`

`tanh_sinh_quad F, A, B [ , LEVELS]` — double-exponential quadrature.

`gauss_legendre_2d`

`gauss_legendre_2d F, AX, BX, AY, BY [ , N]` — Cartesian-product GL.

`monte_carlo_2d`

`monte_carlo_2d F, AX, BX, AY, BY [ , N]` — uniform Monte Carlo on a rectangle.

Optimization (Heuristic)

3 topics

`simulated_annealing`

`simulated_annealing` F , $X0$ [, $T0$, $COOL$, $ITERS$, $STEP$] — Metropolis-Hastings minimisation.

`simplex_lp`

`simplex_lp` C , A , B — $\max c^T x$ s.t. $Ax \leq b$, $x \geq 0$. Returns [x , $value$].

`particle_swarm`

`particle_swarm` F , $BOUNDS$, $SWARM$, $ITERS$ — constriction-coefficient PSO.

Distributions (Tail / Mixture / Categorical)

14 topics

`gev_pdf`

`gev_pdf X, MU, SIGMA, XI` — Generalised Extreme Value PDF.

`gev_cdf`

`gev_cdf X, MU, SIGMA, XI` — GEV CDF.

`gev_sample`

`gev_sample MU, SIGMA, XI` — GEV sample.

`gen_pareto_pdf`

`gen_pareto_pdf X, MU, SIGMA, XI` — Generalised Pareto PDF.

`gen_pareto_cdf`

`gen_pareto_cdf X, MU, SIGMA, XI` — GP CDF.

`gen_pareto_sample`

`gen_pareto_sample MU, SIGMA, XI` — GP sample.

`skew_normal_pdf`

`skew_normal_pdf X, XI, OMEGA, ALPHA` — Azzalini skew-normal PDF.

`skew_normal_cdf`

`skew_normal_cdf X, XI, OMEGA, ALPHA` — skew-normal CDF (numeric integration).

`mixture_normal_pdf`

`mixture_normal_pdf X, WEIGHTS, MEANS, STDS` — Gaussian-mixture PDF.

`categorical_sample`

`categorical_sample PROBS` — sample category index.

`multinomial_pmf`

`multinomial_pmf COUNTS, PROBS` — multinomial PMF.

`multinomial_sample`

`multinomial_sample N, PROBS` — multinomial counts vector.

`truncated_normal_pdf`

`truncated_normal_pdf` X , MU , SIGMA , LO , HI — truncated-normal PDF.

`truncated_normal_sample`

`truncated_normal_sample` MU , SIGMA , LO , HI — rejection sampler.

Clustering & Dimensionality Reduction

7 topics

`dbscan`

`dbscan POINTS, EPS, MIN_PTS` — density-based clustering. Returns label per point (-1 = noise).

`gmm_em_1d`

`gmm_em_1d DATA, K [, MAX_ITER]` — 1-D Gaussian-mixture EM. Returns $[\pi, \mu, \sigma]$.

`silhouette_score`

`silhouette_score POINTS, LABELS` — mean silhouette coefficient.

`davies_bouldin_index`

`davies_bouldin_index POINTS, LABELS` — lower is better.

`calinski_harabasz_index`

`calinski_harabasz_index POINTS, LABELS` — higher is better.

`mds_2d`

`mds_2d D` — classical multidimensional scaling on a distance matrix → 2-D coords.

`mean_shift`

`mean_shift POINTS [, BANDWIDTH, MAX_ITER]` — non-parametric mode-seeking.

Neural-Net Primitives

11 topics

batch_norm

`batch_norm XS [, EPS]` — zero-mean unit-variance normalisation.

layer_norm

`layer_norm XS [, EPS]` — alias for `batch_norm` (per-sample reuse).

dropout_mask

`dropout_mask N [, P]` — Bernoulli dropout mask scaled by $1/(1-p)$.

max_pool_1d

`max_pool_1d XS [, WIN]` — non-overlapping max-pool.

avg_pool_1d

`avg_pool_1d XS [, WIN]` — non-overlapping average-pool.

attention_softmax

`attention_softmax XS` — numerically stable softmax.

positional_encoding

`positional_encoding LENGTH, D_MODEL` — sinusoidal Transformer encoding.

glorot_init

`glorot_init FAN_IN, FAN_OUT` — Glorot uniform.

he_init

`he_init FAN_IN, FAN_OUT` — Gaussian ($\text{std} = \sqrt{2/\text{fan_in}}$).

adam_step

`adam_step PARAM, GRAD, M, V, LR, B1, B2, EPS, T` → `[param', m', v']`.

rmsprop_step

`rmsprop_step PARAM, GRAD, V, LR, DECAY, EPS` → `[param', v']`.

Time Series (Spectral)

5 topics

`ewma`

`ewma XS [, ALPHA]` — exponentially-weighted moving average.

`ccf`

`ccf XS, YS [, MAX_LAG]` — sample cross-correlation function over $-L..L$ lags.

`periodogram`

`periodogram XS` — squared-magnitude DFT / N (one-sided spectrum).

`welch_psd`

`welch_psd XS [, SEG_LEN, OVERLAP]` — Welch's PSD via Hann segments.

`lag_features`

`lag_features XS, P` — design matrix of P lagged features.

Image Processing

5 topics

median_filter_2d

`median_filter_2d IMG [, RADIUS]` — denoising median filter.

threshold_otsu

`threshold_otsu IMG` — Otsu's optimal threshold for 0..255 grayscale.

histogram_equalize

`histogram_equalize IMG` — global histogram equalisation.

erode_2d

`erode_2d IMG [, RADIUS]` — morphological erosion (square structuring element).

dilate_2d

`dilate_2d IMG [, RADIUS]` — morphological dilation.

Loss Functions

3 topics

`mse_loss`

`mse_loss A, B` — mean-squared error.

`mae_loss`

`mae_loss A, B` — mean-absolute error.

`huber_loss`

`huber_loss A, B [, DELTA]` — Huber loss (default $\delta=1$).

Spatial / Geographic

3 topics

`vincenty_distance`

`vincenty_distance` LAT1, LON1, LAT2, LON2 — geodesic distance on WGS-84.

`mercator_project`

`mercator_project` LAT, LON [, R] → [x, y] Mercator metres.

`destination_from_bearing`

`destination_from_bearing` LAT, LON, BEARING_DEG, DIST_M [, R] — spherical great-circle target.

Integer Sequences

6 topics

`recaman`

`recaman N` — first N terms of Recamán's sequence.

`sylvester`

`sylvester N` — Sylvester's sequence ($a_n = a_{n-1}^2 - a_{n-1} + 1$).

`happy_q`

`happy_q N` — 1 if N is a happy number.

`amicable_pair_q`

`amicable_pair_q A, B` — 1 if (a, b) is an amicable pair.

`aliquot_sequence`

`aliquot_sequence N [, MAX_STEPS]` — sum-of-proper-divisors trajectory.

`magic_constant`

`magic_constant N` — $n(n^2+1)/2$ (sum of any line of an $n \times n$ magic square).

Graph Link & Cluster Metrics

7 topics

`clustering_coefficient_local`

`clustering_coefficient_local` `ADJ` — local CC per vertex.

`clustering_coefficient_global`

`clustering_coefficient_global` `ADJ` — mean local CC.

`assortativity`

`assortativity` `ADJ` — degree-degree correlation coefficient.

`common_neighbors`

`common_neighbors` `ADJ`, `U`, `V` — count.

`jaccard_neighbors`

`jaccard_neighbors` `ADJ`, `U`, `V` — $|\Gamma(u) \cap \Gamma(v)| / |\Gamma(u) \cup \Gamma(v)|$.

`adamic_adar`

`adamic_adar` `ADJ`, `U`, `V` — $\sum 1/\log(\deg(w))$ over common neighbours `w`.

`preferential_attachment_score`

`preferential_attachment_score` `ADJ`, `U`, `V` — $\deg(u) \cdot \deg(v)$.

3-D Geometry

8 topics

`triangle_3d_normal`

`triangle_3d_normal P1, P2, P3` — unit normal.

`triangle_3d_area`

`triangle_3d_area P1, P2, P3` — area via cross product.

`tetrahedron_volume`

`tetrahedron_volume A, B, C, D` — $|\det(\mathbf{B}-\mathbf{A}, \mathbf{C}-\mathbf{A}, \mathbf{D}-\mathbf{A})|/6$.

`plane_from_3_points`

`plane_from_3_points P1, P2, P3` → `[a, b, c, d]` ($a x + b y + c z + d = 0$).

`point_to_plane_distance`

`point_to_plane_distance P, [a, b, c, d]`.

`ray_triangle_intersect`

`ray_triangle_intersect O, D, A, B, C` — Möller-Trumbore. Returns `t` or `NaN`.

`ray_sphere_intersect`

`ray_sphere_intersect O, D, CENTER, RADIUS` — nearest non-negative `t`.

`aabb_overlap`

`aabb_overlap A_MIN, A_MAX, B_MIN, B_MAX` — 1 if 3-D AABBs overlap.

Iterative Solvers

6 topics

`gauss_seidel`

`gauss_seidel A, B [, MAX_ITER, TOL]` — Gauss-Seidel iterative solver.

`jacobi_iteration`

`jacobi_iteration A, B [, MAX_ITER, TOL]` — Jacobi iteration.

`sor_solve`

`sor_solve A, B [, OMEGA, MAX_ITER, TOL]` — Successive Over-Relaxation.

`thomas_tridiag_solve`

`thomas_tridiag_solve A_SUB, B_MAIN, C_SUP, D` — tridiagonal direct solve.

`richardson_extrapolation`

`richardson_extrapolation F, H0 [, LEVELS]` — Romberg-style extrapolation.

`finite_difference_5pt`

`finite_difference_5pt F, X [, H]` — five-point central derivative.

Crypto / Modular Algebra

5 topics

`tonelli_shanks_sqrt`

`tonelli_shanks_sqrt N, P` — modular square root mod prime p.

`baby_step_giant_step`

`baby_step_giant_step G, H, P` — discrete logarithm mod p.

`pollard_rho_factor`

`pollard_rho_factor N` — return a non-trivial factor of N.

`modular_lcm`

`modular_lcm XS` — LCM via gcd.

`crt_general`

`crt_general REMAINDERS, MODULI` — CRT over arbitrary (not coprime) moduli; -1 if infeasible.

Physics & Chemistry

5 topics

van_der_waals_p

van_der_waals_p N , T , V , A , B — pressure from Van der Waals equation.

nernst_equation

nernst_equation E^0 , N , T , Q — half-cell potential.

arrhenius_rate

arrhenius_rate A , E_a , T — $k = A \exp(-E_a / RT)$.

reduced_mass

reduced_mass M_1 , M_2 — $\mu = m_1 m_2 / (m_1 + m_2)$.

ph_to_concentration

ph_to_concentration PH — $[H^+] = 10^{-pH}$.

MCMC & SDEs

5 topics

metropolis_hastings

`metropolis_hastings LOG_PI, X0, SIGMA, ITERS [, BURN_IN]` — random-walk MH.

gibbs_sampler_step

`gibbs_sampler_step COND_SAMPLERS, X` — one Gibbs sweep over coordinates.

euler_maruyama

`euler_maruyama MU, SIGMA, X0, T0, T_END, N_STEPS` — Euler-Maruyama SDE integrator.

milstein

`milstein MU, SIGMA, SIGMA_X, X0, T0, T_END, N_STEPS` — Milstein scheme.

ornstein_uhlenbeck_path

`ornstein_uhlenbeck_path THETA, MU, SIGMA, X0, T, N` — exact OU sample path.

Hidden Markov Models

3 topics

`hmm_forward`

`hmm_forward PI, A, B, OBS` — $\log P(O | \lambda)$.

`hmm_viterbi`

`hmm_viterbi PI, A, B, OBS` — most-likely state path.

`hmm_backward`

`hmm_backward PI, A, B, OBS` — backward β matrix.

Survival Analysis

2 topics

`kaplan_meier`

`kaplan_meier TIMES, EVENTS` → matrix of $[t, S(t)]$.

`log_rank_test`

`log_rank_test T1, E1, T2, E2` → $[\chi^2, p]$.

Sequence Alignment

2 topics

`needleman_wunsch`

`needleman_wunsch A, B [, MATCH, MISMATCH, GAP]` — global alignment score.

`smith_waterman`

`smith_waterman A, B [, MATCH, MISMATCH, GAP]` — best local-alignment score.

Chemistry

4 topics

`gibbs_free_energy`

`gibbs_free_energy` ΔH [, T , ΔS] — $\Delta G = \Delta H - T\Delta S$.

`henderson_hasselbalch`

`henderson_hasselbalch` pK_a , A_{CONJ} , HA — $pH = pK_a + \log_{10}([A^-]/[HA])$.

`radioactive_decay`

`radioactive_decay` N_0 , λ , T — $N(t) = N_0 e^{-\lambda t}$.

`half_life_to_constant`

`half_life_to_constant` T_{HALF} — $\lambda = \ln 2 / t_{1/2}$.

Control Theory

5 topics

pid_step

`pid_step STATE, KP, KI, KD, E, DT` → `[new_state, u]` (PID controller).

transfer_function_eval

`transfer_function_eval NUM, DEN, OMEGA` → `[Re, Im]` of $H(j\omega)$.

bode_magnitude_db

`bode_magnitude_db NUM, DEN, OMEGA` — $20 \log_{10} |H(j\omega)|$.

bode_phase_deg

`bode_phase_deg NUM, DEN, OMEGA` — phase of $H(j\omega)$ in degrees.

lqr_2x2

`lqr_2x2 A, B, Q, R` — closed-form 2×2 continuous LQR. Returns `[K, P]`.

Game Theory

3 topics

nash_eq_2x2

nash_eq_2x2 P1, P2 — pure Nash equilibria of a 2×2 bimatrix game.

shapley_value

shapley_value N, V_TABLE — Shapley values from characteristic-function vector.

expected_utility

expected_utility PROBS, PAYOFFS — $\sum p \cdot u$.

Operations Research

3 topics

hungarian_assignment

`hungarian_assignment COST_MATRIX` — assignment + total cost. (Greedy heuristic; reduces to Hungarian for small instances.)

tsp_nearest_neighbor

`tsp_nearest_neighbor DIST_MATRIX` → [tour, length].

vertex_cover_2approx

`vertex_cover_2approx ADJ` — 2-approximation greedy.

Numerical PDE

3 topics

`heat_eq_1d`

`heat_eq_1d F, A, B, NX, T, NT, ALPHA` — explicit FTCS heat-equation solver.

`wave_eq_1d`

`wave_eq_1d F, A, B, NX, T, NT, C` — explicit wave-equation solver.

`laplace_2d_jacobi`

`laplace_2d_jacobi GRID [, MAX_ITER, TOL]` — Jacobi smoother on a Dirichlet grid.

Bayesian Conjugate Updates

4 topics

beta_binomial_update

`beta_binomial_update ALPHA, BETA, N, K` → posterior $[\alpha', \beta']$.

normal_normal_update

`normal_normal_update MU0, VAR0, N, YBAR, VAR_DATA` → $[\mu_{\text{post}}, \sigma^2_{\text{post}}]$.

gamma_poisson_update

`gamma_poisson_update ALPHA, BETA, N, TOTAL_EVENTS` → $[\alpha', \beta']$.

dirichlet_multinomial_update

`dirichlet_multinomial_update ALPHA, COUNTS` → $\alpha + \text{counts}$.

Quantum Computing Gates

8 topics

hadamard_gate

hadamard_gate — 2×2 Hadamard.

cnot_gate

cnot_gate — 4×4 controlled-NOT.

swap_gate

swap_gate — 4×4 SWAP.

cz_gate

cz_gate — 4×4 controlled-Z.

qft_matrix

qft_matrix $N \rightarrow [\text{Re}, \text{Im}]$ of $N \times N$ QFT matrix.

phase_gate

phase_gate $\text{PHI} \rightarrow [\text{Re}, \text{Im}]$ of $\text{diag}(1, e^{i\phi})$.

s_gate

s_gate — $\text{phase}(\pi/2)$.

t_gate

t_gate — $\text{phase}(\pi/4)$.

Splines & Curves

4 topics

`bezier_eval`

`bezier_eval` `CONTROL_PTS`, `T` — de Casteljau evaluation.

`catmull_rom_eval`

`catmull_rom_eval` `P0`, `P1`, `P2`, `P3`, `T` — uniform Catmull-Rom.

`cubic_hermite_eval`

`cubic_hermite_eval` `P0`, `M0`, `P1`, `M1`, `T` — Hermite basis interpolation.

`bspline_basis`

`bspline_basis` `I`, `K`, `T`, `KNOTS` — Cox-de Boor B-spline basis.

Music & Audio

4 topics

`freq_to_midi`

`freq_to_midi` F — MIDI note number from frequency.

`midi_to_freq`

`midi_to_freq` M — frequency from MIDI note.

`equal_temperament_freq`

`equal_temperament_freq` `SEMITONES_ABOVE_A4` — 12-TET frequency.

`cents_difference`

`cents_difference` $F1$, $F2$ — $1200 \log_2(F2/F1)$.

Astronomy & Cosmology

3 topics

redshift_z

redshift_z LAMBDA_OBS, LAMBDA_EMIT — $z = \lambda_{\text{obs}}/\lambda_{\text{emit}} - 1$.

hubble_distance

hubble_distance H0 — c / H_0 in Mpc.

luminosity_distance

luminosity_distance Z, H0, OMEGA_M, OMEGA_L — flat Λ CDM (numeric).

Fluid Dynamics

4 topics

reynolds_number

reynolds_number $\rho, U, L, \mu \rightarrow \rho U L / \mu$.

mach_number

mach_number $U, c \rightarrow U / c$.

prandtl_number

prandtl_number $C_p, \mu, k \rightarrow c_p \mu / k$.

bernoulli_velocity

bernoulli_velocity $P_1, P_2, \rho \rightarrow \sqrt{(2(P_1 - P_2) / \rho)}$.

Distributions (Discrete / Circular)

4 topics

`negative_binomial_pmf`

`negative_binomial_pmf` K , R , P — failures-before-rth-success PMF.

`hypergeometric_pmf`

`hypergeometric_pmf` N , K_{POP} , N_{DRAWS} , K_{OBS} — finite-population PMF.

`beta_binomial_pmf`

`beta_binomial_pmf` K , N , ALPHA , BETA — beta-binomial PMF.

`von_mises_pdf`

`von_mises_pdf` THETA , MU , KAPPA — circular Gaussian.

Random Graph Generators

3 topics

`erdos_renyi_random`

`erdos_renyi_random N, P` — $G(n, p)$ Bernoulli edge model.

`barabasi_albert_random`

`barabasi_albert_random N, M` — preferential-attachment model.

`watts_strogatz_random`

`watts_strogatz_random N, K, P` — small-world rewiring.

Color Science

3 topics

`rgb_to_lab`

`rgb_to_lab` R, G, B — sRGB $\rightarrow$ CIE Lab.

`lab_to_rgb`

`lab_to_rgb` L, A, B — CIE Lab $\rightarrow$ sRGB.

`kelvin_to_rgb`

`kelvin_to_rgb` K — black-body sRGB approximation (Tanner Helland).

Integer Sequences (Combinatorial)

4 topics

`bell_triangle`

`bell_triangle` N — first $N+1$ rows of the Bell / Aitken triangle.

`surjection_count`

`surjection_count` N, K — $k! \cdot S(n, k)$.

`distinct_partition_count`

`distinct_partition_count` N — partitions of N into distinct parts.

`fibonacci_q`

`fibonacci_q` N — 1 if N is a Fibonacci number.

Multiple-Testing Corrections

3 topics

`bonferroni_correction`

`bonferroni_correction` `PVALS` — Bonferroni-adjusted p-values (capped at 1).

`benjamini_hochberg`

`benjamini_hochberg` `PVALS` — BH FDR q-values.

`tukey_hsd`

`tukey_hsd` `ALPHA`, `K`, `DF`, `MSE`, `N` — Tukey honestly-significant-difference critical bound.

Probabilistic Distances

3 topics

hellinger_distance

`hellinger_distance` $P, Q \rightarrow \sqrt{1/2 \sum (\sqrt{p} - \sqrt{q})^2}$.

wasserstein_1d

`wasserstein_1d` A, B — earth-mover distance for sorted samples.

chi_squared_divergence

`chi_squared_divergence` $P, Q \rightarrow \sum (p - q)^2 / q$.

Distributions (Tail / Inflated)

3 topics

`beta_geometric_pmf`

`beta_geometric_pmf` K , ALPHA , BETA — beta-geometric PMF.

`generalized_gamma_pdf`

`generalized_gamma_pdf` X , A , D , P — generalised-gamma PDF.

`zip_pmf`

`zip_pmf` K , PI , LAMBDA — zero-inflated Poisson.

Astrophysics & Radiation

9 topics

stefan_boltzmann_luminosity

stefan_boltzmann_luminosity R , T — $L = 4\pi R^2 \sigma T^4$.

photon_momentum

photon_momentum $LAMBDA$ — h / λ .

photon_energy_ev

photon_energy_ev $LAMBDA$ — hc / λ in eV.

dipole_radiation_power

dipole_radiation_power Q , A — Larmor formula.

parallax_to_distance

parallax_to_distance P_ARCSEC — parsecs from arcseconds.

hawking_temperature

hawking_temperature M_KG — black-hole temperature.

roche_limit

roche_limit R , $RHO_PRIMARY$, RHO_SAT — rigid-body Roche limit.

apparent_magnitude

apparent_magnitude ABS_M , D_PC — $m = M + 5 \log_{10}(d/10)$.

distance_modulus

distance_modulus D_PC — $\mu = 5 \log d - 5$.

Chemistry (Beer / Rate / Colligative)

3 topics

beer_lambert

beer_lambert EPSILON, L, C — $A = \epsilon \cdot l \cdot c$.

rate_law_n

rate_law_n A0, K, T, N — concentration after time t for nth-order reaction.

freezing_point_depression

freezing_point_depression KF, MOLALITY, I — $\Delta T = K_f m i$.

Mixed-Strategy Game Theory

2 topics

`mixed_nash_2x2`

`mixed_nash_2x2` $P_1, P_2 \rightarrow [p, q]$ mixed-strategy probabilities.

`minimax_2x2`

`minimax_2x2` M — security level for the row player.

Computer Graphics

4 topics

`barycentric_coords_2d`

`barycentric_coords_2d` $P, A, B, C \rightarrow [u, v, w]$.

`bresenham_line`

`bresenham_line` $X0, Y0, X1, Y1 \rightarrow$ array of integer pixels.

`bilinear_interp_2d`

`bilinear_interp_2d` $F00, F10, F01, F11, U, V$ — unit-square bilinear.

`point_in_polygon_2d`

`point_in_polygon_2d` $P, POLY$ — ray-casting test.

DSP (Hilbert / Cepstrum / Filter Design)

5 topics

`hilbert_transform`

`hilbert_transform` `X5` — discrete Hilbert via DFT.

`cepstrum`

`cepstrum` `X5` — real cepstrum `ifft(log|fft|)`.

`butterworth_lowpass_coeffs`

`butterworth_lowpass_coeffs` `ORDER`, `CUTOFF` → `[b, a]` digital biquad-cascade.

`savitzky_golay_coeffs`

`savitzky_golay_coeffs` `NL`, `NR`, `M` — SG filter coefficients.

`savitzky_golay_filter`

`savitzky_golay_filter` `X5`, `COEFFS` — apply pre-computed SG coefficients.

Image Processing (Edges / Bilateral)

2 topics

`canny_edge_intensity`

`canny_edge_intensity` `IMG` — Sobel-magnitude edge map (caller thresholds).

`bilateral_filter_basic`

`bilateral_filter_basic` `IMG`, `RADIUS`, `SIGMA_S`, `SIGMA_R` — edge-preserving smoother.

Clustering Helpers

2 topics

kmeans_pp_init

`kmeans_pp_init POINTS, K` — k-means++ seed centroids.

elbow_score

`elbow_score POINTS, LABELS` — within-cluster sum of squares.

Combinatorics (Tableaux / Boustrophedon)

4 topics

`young_tableaux_count`

`young_tableaux_count` `LAMBDA` — hook-length formula.

`euler_alt_permutation`

`euler_alt_permutation` `N` — Euler / zigzag number A000111.

`genocchi_number`

`genocchi_number` `N` — G_n via Bernoulli identity.

`lattice_paths_count`

`lattice_paths_count` `M`, `N` — $C(m+n, n)$ east/north paths.

Number Theory (Tetration / Smoothness)

4 topics

`tetration`

`tetration A, N` — N-fold a-tower (overflow $\rightarrow \infty$).

`ackermann_limited`

`ackermann_limited M, N` — depth-limited Ackermann (refuses past $M=3$ $N=14$).

`perfect_power_q`

`perfect_power_q N` — 1 if $N = a^b$ for integers $a, b \geq 2$.

`b_smooth_q`

`b_smooth_q N, B` — 1 if every prime factor of N is $\leq B$.

Network Coreness & Structure

2 topics

`k_core`

`k_core` `ADJ` — vertex coreness.

`rich_club_coefficient`

`rich_club_coefficient` `ADJ`, `K` — fraction of edges among nodes with $\text{deg} > k$.

Crypto Helpers

3 topics

`rsa_basic_encrypt`

`rsa_basic_encrypt M, E, N` — $m^e \bmod n$.

`rsa_basic_decrypt`

`rsa_basic_decrypt C, D, N` — $c^d \bmod n$.

`dh_shared_secret`

`dh_shared_secret PEER_PUB, PRIVATE, P` — Diffie-Hellman.

Quantum Entanglement

4 topics

`bell_state_phi_plus`

`bell_state_phi_plus` — Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$.

`bell_state_psi_minus`

`bell_state_psi_minus` — singlet $(|01\rangle - |10\rangle)/\sqrt{2}$.

`density_matrix_purity`

`density_matrix_purity` ρ — $\text{tr}(\rho^2)$.

`concurrence_2qubit`

`concurrence_2qubit` ψ — $2|ad - bc|$ in real basis.

2-D Geometry (Polygons & Circles)

4 topics

`point_in_circle`

`point_in_circle` *P*, *CENTER*, *R* — 1 if inside.

`circle_circle_intersect_2d`

`circle_circle_intersect_2d` *C1*, *R1*, *C2*, *R2* — 0/1/2 intersection points.

`polygon_centroid`

`polygon_centroid` *POLY* — area-weighted centroid.

`sutherland_hodgman_clip`

`sutherland_hodgman_clip` *SUBJECT*, *CLIP* — clip against convex CCW polygon.

Time-Series Smoothing

1 topics

`kalman_rts_smoother`

`kalman_rts_smoother` `X_HAT`, `P`, `X_PRED`, `P_PRED`, `F` — RTS smoothed means.

Bioinformatics

6 topics

`gc_content`

`gc_content SEQ` — fraction of G + C in DNA / RNA string.

`codon_to_aa`

`codon_to_aa CODON` — single-letter amino-acid (or '*' / 'X').

`reverse_complement_dna`

`reverse_complement_dna SEQ` — DNA reverse complement.

`hamming_dna`

`hamming_dna A, B` — case-insensitive Hamming distance.

`blosum62_pair_score`

`blosum62_pair_score A, B` — BLOSUM62 substitution score.

`kmer_count`

`kmer_count SEQ, K` — total count of length-K windows.

Geographic / Map Projection

4 topics

`great_circle_bearing`

`great_circle_bearing` LAT1, LON1, LAT2, LON2 — initial bearing in degrees.

`midpoint_lat_lon`

`midpoint_lat_lon` LAT1, LON1, LAT2, LON2 — great-circle midpoint.

`utm_zone_for`

`utm_zone_for` LON — UTM zone (1..60).

`area_polygon_lat_lon`

`area_polygon_lat_lon` POLY [, R] — spherical-excess polygon area in m².

Fixed-Income Finance

5 topics

`crr_binomial_option`

`crr_binomial_option S0, K, T, R, SIGMA, N, IS_PUT` — Cox-Ross-Rubinstein price.

`bond_price_clean`

`bond_price_clean FACE, COUPON, N, PP_Y, YIELD` — present value.

`bond_yield_to_maturity`

`bond_yield_to_maturity PRICE, FACE, COUPON, N, PP_Y` — solve YTM by bisection.

`modified_duration_bond`

`modified_duration_bond FACE, COUPON, N, PP_Y, YIELD` — modified duration.

`convexity_bond`

`convexity_bond FACE, COUPON, N, PP_Y, YIELD` — bond convexity measure.

Image Quality Metrics

3 topics

`ssim`

`ssim A, B [, L]` — Structural Similarity Index (single window).

`psnr`

`psnr A, B [, MAX_VAL]` — peak SNR in dB.

`mssim`

`mssim A, B [, WIN, L]` — mean SSIM across non-overlapping windows.

Acoustics

4 topics

`db_spl_from_pa`

`db_spl_from_pa` `P_PA` — sound pressure level dB SPL.

`a_weighting_factor`

`a_weighting_factor` `F_HZ` — IEC 61672 A-weighting amplitude.

`octave_band_center`

`octave_band_center` `BAND` — $1 \text{ kHz} \cdot 2^{\text{band centre frequency}}$.

`semitone_ratio`

`semitone_ratio` — 12-TET ratio $2^{(1/12)}$.

Population Genetics

4 topics

hardy_weinberg

hardy_weinberg P — $(p^2, 2pq, q^2)$ genotype frequencies.

expected_heterozygosity

expected_heterozygosity P_VEC — $1 - \sum p_i^2$.

fst_simple

fst_simple $P1, P2, N1, N2$ — Wright's F_{ST} between two populations.

allele_frequencies

allele_frequencies $GENOTYPES$ — p, q from 0/1/2 genotype counts.

Epidemiology

3 topics

`sir_step`

`sir_step` S , I , R , $BETA$, $GAMMA$, DT — Euler step of the SIR model.

`sir_r0`

`sir_r0` $BETA$, $GAMMA$ — basic reproduction number β/γ .

`doubling_time`

`doubling_time` R — $t_2 = \ln 2 / r$.

Inequality Measures

4 topics

`theil_index`

`theil_index X5` — Theil T inequality index.

`herfindahl_hirschman`

`herfindahl_hirschman SHARES` — $\sum s_i^2$.

`atkinson_index`

`atkinson_index X5 [, EPS]` — Atkinson inequality with parameter ϵ .

`lorenz_curve_points`

`lorenz_curve_points X5` — matrix of [cumulative-pop, cumulative-income].

APL / J Array Primitives

4 topics

`iota_range`

`iota_range` N — $0..N$ (APL convention).

`reshape_array`

`reshape_array` $ROWS$, $COLS$, $FLAT$ — APL reshape (cycles input).

`grade_up`

`grade_up` XS — index permutation that sorts ascending.

`grade_down`

`grade_down` XS — index permutation that sorts descending.

Plasma Physics

4 topics

`plasma_frequency`

`plasma_frequency` `N_E` — electron plasma angular frequency (rad/s).

`debye_length`

`debye_length` `T_K`, `N_E` — Debye screening length.

`cyclotron_frequency`

`cyclotron_frequency` `B` [`,` `Q`, `M`] — qB/m (electron defaults).

`larmor_radius`

`larmor_radius` `V_PERP`, `B` [`,` `Q`, `M`] — gyroradius (electron defaults).

String Similarity

2 topics

`jaro_winkler_similarity`

`jaro_winkler_similarity A, B` — JW string similarity.

`metaphone_simple`

`metaphone_simple S` — abridged Metaphone consonant skeleton.

Rating Systems

3 topics

`elo_rating_update`

`elo_rating_update` $R_A, R_B, SCORE_A$ $[, K] \rightarrow [R'_A, R'_B]$.

`glicko_rating_update`

`glicko_rating_update` $R, RD, OPP_R, OPP_RD, SCORE \rightarrow [R', RD']$.

`dice_sum_pmf`

`dice_sum_pmf` $N, S, TARGET$ — probability of N s -sided dice summing to $TARGET$.

Effect Sizes

3 topics

`cohens_d`

`cohens_d A, B` — pooled-SD standardised mean difference.

`cliff_delta`

`cliff_delta A, B` — non-parametric effect size in $[-1, 1]$.

`vargha_delaney_a12`

`vargha_delaney_a12 A, B` — $P(X > Y) + 0.5 P(X = Y)$.

Control Transient Response

2 topics

`step_response_2nd_order`

`step_response_2nd_order` ZETA, OMEGA_N, T — $y(t)$.

`overshoot_2nd_order`

`overshoot_2nd_order` ZETA — peak overshoot %.

Matrix Norms

3 topics

`frobenius_norm`

`frobenius_norm M` — $\sqrt{(\sum m_{ij}^2)}$.

`spectral_norm`

`spectral_norm M` — largest singular value.

`trace_matrix`

`trace_matrix M` — $\sum m_{ii}$.

Network Triad Analysis

3 topics

homophily_index

`homophily_index` `ADJ`, `LABELS` — fraction of same-group edges.

dyad_census

`dyad_census` `ADJ` — [mutual, asymmetric, null] dyads.

triad_census

`triad_census` `ADJ` — [empty, edge, path, triangle] counts.

Misc Inverses

1 topics

`sigmoid_inverse`

`sigmoid_inverse` $X = \ln(x / (1-x))$.

Parallel I/O

8 topics

par_lines

`par_lines PATH, { code }` — memory-map a file and scan its lines in parallel across all available CPU cores.

The file is `mmap`'d into memory rather than read sequentially, and line boundaries are detected in parallel chunks. Each line is passed to the callback as `_`. Because the file is memory-mapped, there is no read-buffer overhead and the OS kernel pages data in on demand, making `par_lines` extremely efficient for multi-gigabyte log files or CSV data. Line order within the callback is not guaranteed (lines run in parallel), so the callback should be a self-contained side-effecting operation (accumulate into a shared structure via `pchannel`, write to a file, etc.) or use `par_lines` with a reducer. For ordered processing, use `read_lines` with `pmap` instead.

```
par_lines "data.txt", fn { process }

# Count matching lines in a large log:
var $count = 0
par_lines "/var/log/syslog", fn { $count++ if /ERROR/ }
p $count

# Feed lines into a channel for downstream processing:
var ($tx, $rx) = pchannel(1000)
async { par_lines "huge.csv", fn { $tx->send(_) } }
undef $tx }
```

par_walk

`par_walk PATH, { code }` — recursively walk a directory tree in parallel, invoking the callback for every file path found.

Directory traversal is parallelized using a work-stealing thread pool: multiple directories are read concurrently, and the callback fires as each file is discovered. The path is passed as `_` (absolute). This is significantly faster than a sequential `find`-style walk on SSDs and networked filesystems where directory `readdir` latency dominates. Symlinks are not followed by default to avoid cycles. The walk visits files only — directories themselves are not passed to the callback unless you pass `dirs => 1`. Combine with `pmap` for a two-phase pattern: first collect paths with `par_walk`, then process file contents in parallel.

```
par_walk "./src", fn { p _ if /\.rs$/ }

# Collect all JSON files under a directory:
var @json_files
par_walk "/data", fn { push @json_files, _ if /\.json$/ }
p scalar @json_files

# Parallel content search:
par_walk ".", fn {
  if ( /\.log$/ ) {
    val @hits = grep /FATAL/, rl
    p "$_: ", scalar @hits, " fatals" if @hits
  }
}
```

par_sed

`par_sed PATTERN, REPLACEMENT, @files` — perform an in-place regex substitution across multiple files in parallel.

Each file is processed by a separate worker thread: the file is read into memory, all matches of `PATTERN` are replaced with `REPLACEMENT`, and the result is written back atomically (via a temp file + rename, so readers never see a partially-written file). This is the `stryke` equivalent of `sed -i` but parallelized across the file list — ideal for codebase-wide refactors, log scrubbing,

or bulk config updates. The pattern uses stryke regex syntax (PCRE-style). Returns the total number of substitutions made across all files.

```
par_sed qr/oldFunc/, "newFunc", glob("src/*.pl")

# Case-insensitive replace across a project:
par_sed qr/TODO/i, "DONE", par_walk(".", fn { _ if /\.rs$/ })

# Scrub sensitive data from logs:
var $n = par_sed qr/\b\d{3}-\d{2}-\d{4}\b/, "XXX-XX-XXXX", @log_files
p "redacted $n occurrences"
```

par_find_files

Recursively search a directory tree in parallel for files matching a glob pattern. Unlike `glob_par` which takes a single pattern string, `par_find_files` separates the root directory from the pattern, making it convenient when the search root is a variable. Returns a list of absolute paths to matching files.

```
val @src = par_find_files("src", "*.rs")
p scalar @src # count of Rust files under src/
var @tests = par_find_files(".", "*_test.pl")
@tests |> e p
var @configs = par_find_files("/etc", "*.conf")
```

par_line_count

Count lines across multiple files in parallel, returning the total line count. Each file is read and counted by a separate thread, making this dramatically faster than sequential `wc -l` for large file sets. Useful for codebase metrics, log analysis, or validating data pipeline output.

```
val @files = glob("src/**/*.rs")
var $total = par_line_count(@files)
p "$total lines of Rust"
var $logs = par_line_count(glob("/var/log/*.log"))
p "$logs log lines"
```

par_csv_read

`par_csv_read @files` — read multiple CSV files in parallel, returning an array of parsed datasets (one per file).

Each file is read and parsed by a separate worker thread using a fast Rust CSV parser that handles quoting, escaping, and UTF-8 correctly. Headers are auto-detected from the first row of each file, and each row is returned as a hashref keyed by header names. This is dramatically faster than sequential CSV parsing when you have many files — common in data engineering pipelines where data arrives as daily/hourly CSV partitions. For a single large CSV file, prefer `par_lines` with manual splitting, since `par_csv_read` parallelizes across files, not within a single file.

```
var @datasets = par_csv_read glob("data/*.csv")
for var $ds (@datasets) {
  p scalar @$ds, "rows"
}

# Merge all CSVs into one list:
var @all_rows = par_csv_read(@files) |> flat
p $all_rows[0]->{name} # access by header

# Filter and aggregate:
var @sales = par_csv_read(glob("sales/*.csv")) |> flat
|> grep { _->{region} eq "US" }
```

glob_par

Parallel recursive file-system glob using rayon across multiple patterns. Backed by the same zshrs glob engine as `glob`, so every zsh qualifier (`((/))`, `(. )`, `(L+1024)`, `(om[1])`, ...) is supported here too. Each pattern expands single-threaded but the patterns run concurrently — ideal for many independent searches. For full qualifier reference see `glob`.

```

val @logs = glob_par("**/*.log")
p scalar @logs # count of log files
"**/*.rs" |> glob_par |> e p # print all Rust files
var @imgs = glob_par("assets/**/*.{png,jpg,webp}")
var @big_recent = glob_par("**(.L+1m om[1,5])") # 5 newest >1MB files

```

pwatch

pwatch(PATTERN, fn { ... }) — watch a path or glob pattern for filesystem changes and invoke the callback on each event.

The watcher runs on a background thread using OS-native notifications (FSEvents on macOS, inotify on Linux) so it consumes near-zero CPU while idle. The callback receives the event path in `_`. The watcher continues until the returned handle is dropped or the program exits. Useful for live-reload dev servers, file-triggered pipelines, or audit logs. Combine with [debounce](#) or a [pchannel](#) if the callback is expensive and rapid bursts of events need to be coalesced.

The initial expansion goes through the full zshrs glob engine, so zsh qualifiers (`(.)`, `(/)`, `(L+1m)`, ...) work for picking what to watch. The runtime event matcher uses the qualifier-stripped pattern shape — stat-based qualifiers can't apply to a single fired event, but they did apply to the watch-set selection upfront.

```

val $w = pwatch "./src/**/*.rs", fn {
  p "changed: $_"
  rebuild()
}

var $logs = pwatch "logs/(.)", fn { p "log updated: $_" }
var $cfg = pwatch "./config", fn { reload_config() }
sleep # block forever

```

CLI / Args

1 topics

getopts

`getopts(SPECS [, META])` — Getopt::Long-style CLI flag parser. By default operates on `@ARGV` directly (no `\@ARGV` needed), returning a hashref of canonical-name → value and leaving only the leftover positionals in `@ARGV`. Use `getopts(\@argv, SPECS [, META])` to parse a list other than `@ARGV`.

Spec language (subset of Perl's `Getopt::Long`):

- `name` — bool flag (present = 1)
- `name|alias|alias2` — same option, multiple names; first is canonical
- `name=s / name=i / name=f` — required string / int / float
- `name:s / name:i / name:f` — optional arg (defaults to `" / 0 / 0.0`)
- `name=s@` — repeatable → arrayref (also `=i@, =f@`)
- `name=s%` — `--name key=val` → hashref (also `=i%, =f%`)
- `name!` — negatable bool; `--no-name` sets 0
- `name+` — counter: each occurrence increments

```
val %opts = %{ getopts([
    "verbose|v",
    "file|f=s",
    "count|n=i",
    "tag|t=s@"
]) };

# Hash form lets each spec carry a default:
var %opts = %{ getopts({
    "verbose|v" => 0,
    "count|n=i" => 10,
    "tag|t=s@"  => [],
}) };

# Explicit array ref (parse a list other than @ARGV):
var %opts = %{ getopts(\@args, [ "verbose|v" ]) };
```

Parsing rules: `--name`, `--name=value`, `--name value`, `-n`, `-n value`, `-nvalue` all accepted. Bundling: `-vDR = -v -D -R`; if a char takes an arg, the rest of the token is its value (`-vfx.txt` → `-v -f x.txt`). `--` terminates options; first non-option positional stops parsing. Unknown options or type mismatches raise a runtime error.

Per-option metadata + auto-`--help`: hash values may themselves be hashrefs carrying `{ help, default, required, metavar }`. When any spec has `help` text and the user hasn't claimed `--help/-h`, `getopts` intercepts `--help/-h`, prints a formatted usage block, and `exit(0)`s. Optional `META` arg: `{ prog, desc, epilog }` controls the banner.

```
val %opts = %{ getopts({
    "verbose|v" => { help => "enable verbose" },
    "file|f=s"  => { help => "output path", default => "out.txt" },
    "count|n=i" => { help => "iterations", required => 1 },
}, { prog => "myscript", desc => "do a thing" }) };
```

File I/O

38 topics

open

Open a filehandle for reading, writing, appending, or piping. The three-argument form `open var $fh, MODE, EXPR` is strongly preferred for safety — it prevents shell injection and makes the mode explicit. Modes include `<` (read), `>` (write/truncate), `>>` (append), `+<` (read-write), `|<` (pipe to command), and `|>` (pipe from command). In stryke, always use lexical filehandles (`var $fh`) rather than bareword globals. PerlIO layers can be specified in the mode: `<:utf8`, `<:raw`, `<:encoding(UTF-16)`. Always check the return value — `open ... or die "...: $!"` is idiomatic. The `$!` variable contains the OS error message on failure. Forgetting to check `open` is one of the most common bugs in Perl code.

```
open val $fh, '<', 'data.txt' or die "open: $!"
var @lines = <$fh>
close $fh

open val $out, '>>', 'log.txt' or die "append: $!"
print $out tm "event happened\n"
close $out

open val $pipe, '|<', 'ls', '-la' or die "pipe: $!"
while (<$pipe>) { p tm _ }
```

close

Close a filehandle, flushing any buffered output and releasing the underlying OS file descriptor. Returns true on success, false on failure — and failure is more common than you might think. For write handles, `close` is where buffered data actually hits disk, so a full disk or network error may only surface at `close` time. Always check the return value when writing: `close $fh or die "close: $!"`. For pipe handles opened with `|<` or `|>`, `close` waits for the child process to exit and sets `$?` to the child's exit status. In stryke, lexical filehandles are automatically closed when they go out of scope, but explicit `close` is clearer and lets you handle errors.

```
open val $fh, '>', 'out.txt' or die $!
print $fh "done\n"
close $fh or die "write failed: $!"

open val $p, '|<', 'gzip', '-c' or die $!
print $p $data
close $p
p "gzip exited: $?" if $?
```

read

Read a specified number of bytes from a filehandle into a scalar buffer. The signature is `read FH, SCALAR, LENGTH [, OFFSET]`. Returns the number of bytes actually read (which may be less than requested at EOF or on partial reads), 0 at EOF, or `undef` on error. The optional `OFFSET` argument lets you append to an existing buffer at a given position, which is useful for accumulating data in a loop. For text files, the bytes are decoded according to the handle's PerlIO layer — use `<:raw` for binary data to avoid encoding transforms. In stryke, `read` works identically to Perl 5. For line-oriented input, prefer `<$fh>` or `readline` instead.

```
open val $fh, '<:raw', $path or die $!
var $buf = ''
while (read $fh, val $chunk, 4096) {
    $buf .= $chunk
}
p "total: " . length($buf) . " bytes"
close $fh
```

readline

Read one line (or all remaining lines in list context) from a filehandle. The angle-bracket operator `<$fh>` is syntactic sugar for `readline($fh)`. In scalar context, returns the next line including the trailing newline (or `undef` at EOF). In list context, returns all remaining lines as a list. The line ending is determined by `$/` (input record separator, default `\n`). Set `$/ = undef` to slurp the entire file in one read. In stryke, `readline` integrates with the pipe operator — you can pipe filehandle lines through `maps`, `greps`, and other streaming combinators. Always `chomp` after reading if you don't want trailing newlines.

```
while (val $line = <$fh>) {
  chomp $line
  p $line
}

# Slurp entire file
local $/
var $content = <$fh>
p length $content
```

eof

Test whether a filehandle has reached end-of-file. Returns 1 if the next read on the handle would return EOF, 0 otherwise. Called without arguments, `eof()` (with parens) checks the last file in the `<> / ARGV` stream. Called with no parens as `eof`, it tests whether the current ARGV file is exhausted but more files may follow. In stryke, `eof` is typically used in `until` loops or as a guard before `read` calls. Note that `eof` may trigger a blocking read on interactive handles (like STDIN from a terminal) to determine if data is available, so avoid calling it speculatively on interactive input. For most line-processing, `while (<$fh>)` is simpler and implicitly handles EOF.

```
until (eof $fh) {
  val $line = <$fh>
  p tm $line
}

# Process multiple files via ARGV
while (<>) {
  p "new file: $ARGV" if eof()
}
```

seek

Reposition a filehandle to an arbitrary byte offset. The signature is `seek FH, POSITION, WHENCE` where `WHENCE` is 0 (absolute from start), 1 (relative to current position), or 2 (relative to end of file). Use the `Fcntl` constants `SEEK_SET`, `SEEK_CUR`, `SEEK_END` for clarity. Returns 1 on success, 0 on failure. `seek` is essential for random-access file I/O — re-reading headers, skipping to known offsets in binary formats, or rewinding a file for a second pass. In stryke, `seek` flushes the PerlIO buffer before repositioning. Do not mix `seek/tell` with `sysread/syswrite` — they use separate buffering layers.

```
seek $fh, 0, 0      # rewind to start
var $header = <$fh>

seek $fh, -100, 2   # last 100 bytes
read $fh, val $tail, 100
p $tail
```

tell

Return the current byte offset of a filehandle's read/write position. Returns a non-negative integer on success, or -1 if the handle is invalid or not seekable (e.g., pipes, sockets). Useful for bookmarking a position before a speculative read so you can `seek` back if the data doesn't match expectations. In stryke, `tell` reflects the PerlIO buffered position, not the raw OS file descriptor position — so it correctly accounts for encoding layers and buffered reads. Pair with `seek` for random-access patterns.


```

val $pos = tell $fh
p "at byte $pos"

# Bookmark and restore
var $mark = tell $fh
var $line = <$fh>
unless ($line =~ /^HEADER/) {
    seek $fh, $mark, 0 # rewind to before the read
}

```

print

Write operands to the selected output handle (default `STDOUT`) without appending a newline. The output field separator `$,` is inserted between arguments, and `$\` is appended at the end — both default to empty string. You can direct output to a specific handle by passing it as the first argument with no comma: `print STDERR "msg"`. In stryke, `print` behaves identically to Perl 5 and is useful when you need precise control over output formatting, such as building progress bars, writing binary data, or emitting partial lines. For most line-oriented output, prefer `p` (say) instead since it handles the newline automatically.

```

print "no newline"
print STDERR "error msg\n"
print $fh "data to filehandle\n"
for val $pct (0:100) {
    printf "\rprogress: %3d%%", $pct
}
print "\n"

```

say

Print operands followed by an automatic newline to `STDOUT`. In stryke, `say` is always available without `-E` or `use feature 'say'` — it is a first-class builtin. The shorthand `p` is an alias for `say` and is preferred in most stryke code. When given a list, `say` joins elements with `$,` (output field separator, empty by default) and appends `$\` plus a newline. For streaming output over pipelines, combine with `e` (each) to print one element per line. Gotcha: `say` always adds a newline — if you need raw output without one, use `print` instead.

```

p "hello world"
var @names = ("alice", "bob", "eve")
@names |> e p
1:5 |> maps { _ * 2 } |> e p

```

printf

Formatted print to a filehandle (default `STDOUT`), using C-style format specifiers. The first argument is the format string with `%s` (string), `%d` (integer), `%f` (float), `%x` (hex), `%o` (octal), `%e` (scientific), `%g` (general float), and `%%` (literal percent). Width and precision modifiers work as in C: `%-10s` left-aligns in a 10-char field, `%05d` zero-pads to 5 digits, `%.2f` gives 2 decimal places. In stryke, `printf` supports all standard Perl 5 format specifiers including `%v` (version strings) and `%n` is disabled for safety. Direct output to a handle by placing it before the format: `printf STDERR "...", @args`. Unlike `sprintf`, `printf` outputs directly and returns a boolean indicating success.

```

printf "%-10s %5d\n", $name, $score
printf STDERR "error %d: %s\n", $code, $msg
printf "%08.2f\n", 3.14159 # 00003.14
printf "%s has %d item%s\n", $user, $n, $n == 1 ? "" : "s"

```

sprintf

Return a formatted string without printing it, using the same C-style format specifiers as `printf`. This is the go-to function for building formatted strings for later use — constructing log messages, building padded table columns, converting numbers to hex or binary representations, or assembling strings that will be passed to other functions. In stryke, `sprintf` is often combined with the pipe operator: `$val |> t { sprintf "0x%04x", _ } |> p`. The return value is always a string. All format specifiers from `printf` apply here.

```
val $hex = sprintf "0x%04x", 255
var $padded = sprintf "%08d", $id
var $msg = sprintf "%-20s: %s", $key, $value
var @rows = map { sprintf "%3d. %s", _, $names[_] } 0..$ #names
@rows |> e p
```

slurp

Read an entire file into memory as a **byte string** (Perl-default semantics — strings are bytes), or — when the path is a glob pattern — concatenate the bytes of every matching regular file. Aliases: `sl`, `cat`, `c`. Dies if the file is missing or unreadable.

Binary-safe. The result is a byte buffer that stringifies via UTF-8-or-Latin-1 decoding, so `eq/regex/substr` still match on text content, but `length()` reports the byte count and writing back via `spew` round-trips the exact bytes. Use directly on binary files (images, archives, etc.) — no separate `read_bytes/slurp_raw` call needed. `read_bytes/slurp_raw` remain available as explicit-intent aliases.

When the path contains glob metacharacters (`* ? [ **`) or a trailing zsh qualifier suffix (`...`), it is expanded by the zshrs glob engine and the results are concatenated. A **plain wildcard sweep** (`c("**.rs")`) is broad and incidental, so it skips matches it can't read as a regular file — a dangling symlink, a vanished file, a directory caught by the pattern — the way `grep -r` does, instead of aborting the whole sweep on one bad entry (skips are logged at debug). A **qualifier glob** is an explicit selection, so it stays strict: **slurp hard-fails if the qualifier yields any non-regular file** — `slurp "**(/)"` errors `slurp: not a regular file: ./sub` because slurping a directory is meaningless. To list directory paths, use `glob "**(/)"`. See `glob` for the full qualifier reference.

```
val $text = slurp("config.yaml")
var $png = slurp("logo.png")           # binary-safe; bytes preserved
spew("copy.png", $png)                 # round-trips exactly
var $all = c("**(.)")                  # every regular file recursively, concatenated bytes
var $logs = c("logs/*.log")            # all logs in one go
var $json = sl("data.json")
var $data = decode_json($json)
"README.md" |> sl |> length |> p       # byte count (Perl-default)
```

Per-file hash sibling: see `swallow/swa` — same glob engine but returns `{ canonical_abspath => bytes }` instead of concatenating.

slurp_raw

Read an entire file into memory as raw bytes without any encoding interpretation. Unlike `slurp`, which returns a decoded UTF-8 string, `read_bytes` preserves the exact byte content — useful for binary files like images, compressed archives, or protocol buffers. The alias `slurp_raw` emphasizes the raw nature.

```
val $png = read_bytes("logo.png")
p length($png)                       # byte count
var $gz = slurp_raw("data.gz")
var $text = gunzip($gz)
p $text
```

swallow

swallow PATTERN — expand a zsh-style glob and return a hash `{ canonical_abspath => raw_bytes }`. Aliases: `swa`. The per-file body never decodes, so text and binary files round-trip uniformly — wrap with `decode_utf8(...)` when you want a string.

Symmetry table:

| | concat / single value | per-file hash | streaming iterator | |---|---|---| | text (UTF-8) | `slurp` / `sl` / `cat` / `c` | — | — | | bytes (raw) | `read_bytes` / `slurp_raw` / `rb` | `swallow` / `swa` | `ingest` / `ing` |

Uses the **same** zshrs glob engine as `glob` and `slurp`, so every qualifier (`(N)`, `(.)`, `(/)`, `(L+1024)`, `(om[1])`, `(f644)`, ...) is supported. Hash keys are absolute paths produced by `fs::canonicalize`, which flattens symlinks to their real targets — a symlink farm collapses to the underlying files.

Same match policy as `slurp/c`: a **plain wildcard sweep** skips matches it can't read as a regular file — a dangling symlink, a vanished file, a directory caught by the pattern — so a bad reference is skipped, not fatal; a **qualifier glob** is an explicit

selection and stays strict (`swallow: not a regular file: ./sub`). Opt into silence on a no-match with the `(N)` null-glob qualifier (`swallow "missing*(N)"` returns an empty hash).

```
val %src = swallow("src/**/*.rs")           # every Rust file, raw bytes
swallow("**(.)") |> keys |> p                 # every regular file's abs path
var %imgs = swa("assets/**/*.{png,jpg}")     # binary files work uniformly
for var ($path, $bytes) (swallow "*.log") {
  p "$path: ", length($bytes), " bytes"
}
var %recent = swallow("**(.om[1,5])")        # 5 newest files (zsh qualifier)
var %safe = swallow("missing*(N)")           # empty hash, no error
```

For giant trees where every file's bytes don't fit in RAM at once, use `ingest` — it streams `[path, bytes]` one file at a time instead of materialising the full hash.

See also: `ingest` (streaming variant), `slurp` (concat text), `read_bytes` (single-file bytes), `glob` (paths only), `glob_par` (parallel paths).

ingest

`ingest PATTERN` — streaming variant of `swallow`. Returns a lazy iterator that yields `[canonical_abspath, raw_bytes]` one file at a time. Aliases: `ing`. Only one file's bytes are resident at any moment — usable on directory trees whose total contents won't fit in memory.

Pipeline shape: path discovery + stat + canonicalize run eagerly up front (so qualifier support is identical to `glob/slurp/swallow`, and a plain wildcard sweep skips matches it can't stat as a regular file while a qualifier glob stays strict). File content reads are deferred to each `next_item()` call.

Iteration models:

- `for var ($path, $bytes) (ingest "...") { ... }` — streams (the foreach loop pulls one pair at a time from the iterator, never materialising a list).
- `ingest "..." |> e { ... }` — streams through stryke's pipe terminals.
- `var $it = ingest "..."; while (var $pair = $it->next->[0]) { ... }` — explicit driving via the standard iterator `->next` protocol.

A plain sweep skips matches it can't stat as a regular file; a qualifier glob stays strict (`ingest: not a regular file: ./sub`). Opt into silence on a no-match with the `(N)` null-glob qualifier.

```
# Hash every file in a tree without ever loading two files' bytes at once.
use Digest::SHA qw(sha256_hex);
for var ($path, $bytes) (ingest "data/**/*.bin") {
  p "$path ", sha256_hex($bytes)
}

# Streaming pipe - counts total bytes without materialising any file twice.
var $total = 0;
for var ($p, $bytes) (ingest "logs/**/*.log") { $total += len($bytes) }
p "$total bytes across ", scalar(glob "logs/**/*.log"), " files"

# Explicit driving + early termination.
var $it = ingest "src/**/*.rs";
while (val $pair = $it->next->[0]) {
  var ($path, $bytes) = @$pair;
  last if length($bytes) > 100_000;
}
```

See also: `swallow` (eager hash variant), `slurp` (concat), `read_bytes` (single-file).

burp

`burp HASH` — inverse of `swallow`. Walk a `{ path => content }` hash, write each entry to disk (parent directories created automatically), and return the integer count of files written.

Values that are byte buffers (`$StrykeValue::bytes` — the type returned by `swallow/read_bytes/ingest`) write raw bytes. Any other scalar stringifies through `to_string()` first (matches `spew/spurt` conventions). Pass the hash by reference: `burp %h` or `burp { ... }` (inline hashref). The bare `burp %h` form is not supported — the parser binds it as a scalar, not a hash.

Hard-fails on the first I/O error, so a half-applied burp surfaces immediately instead of silently dropping entries. Empty hash returns `0` without touching the filesystem.

```
# Canonical swallow → modify → burp round-trip
var %src = swallow "src/**/*.rs";
for val $path (keys %src) {
  $src{$path} = uc $src{$path};      # bytes are mutable scalars in stryke
}
var $n = burp \%src;
p "$n files updated";

# Snapshot-restore: copy an entire tree under a new root.
var %tree = swallow "old/**/*.rs";
var %dest = map { val $k = $_; $k =~ s{^/abs/old}{/abs/new}; ($k => $tree{$k}) } keys %tree;
burp %dest;                          # parent dirs in /abs/new/... created on the fly

# Project scaffold from inline literals.
burp {
  "/out/README.md" => "# Hello\n",
  "/out/src/main.rs" => "fn main() { println!(\"hi\"); }\n",
};
```

See also: `swallow` (read side), `spew/spurt` (single-file write), `ingest` (streaming read).

input

Slurp all of stdin (or a filehandle) as one string.

```
val $all = input          # slurp stdin
var $fh_data = input($fh) # slurp filehandle
```

read_lines

Read a file and return its contents as a list of lines with trailing newlines stripped. This is the idiomatic way to slurp a file line-by-line in stryke without manually opening a filehandle. The short alias `rl` keeps one-liners concise. If the file does not exist, the program dies with an error message.

```
var @lines = rl("data.txt")
p scalar @lines      # line count
@lines |> grep /ERROR/ |> e p # print error lines
var $first = (rl "config.ini")[0]$1
```

Note: returns an empty list for an empty file.

append_file

Append a string to the end of a file, creating it if it does not exist. This is the safe way to add content without overwriting — useful for log files, CSV accumulation, or incremental output. The short alias `af` is convenient in pipelines. The file is opened, written, and closed atomically per call.

```
af("log.txt", "started at " . datetime_utc() . "\n")
1:5 |> e { af("nums.txt", "$_\n") }
var @data = ("a", "b", "c")
@data |> e { af "out.txt", "$_\n" }
```

to_file

Write a string to a file, truncating any existing content. Unlike `append_file`, this replaces the file entirely. Returns the written content so it can be used in a pipeline — write to disk and continue processing in one expression. Creates the file if it does not exist.

```
val $csv = "name,age\nAlice,30\nBob,25"
$csv |> to_file("people.csv") |> p
to_file("empty.txt", "") # truncate a file
```

Note: the return-value-for-piping behavior distinguishes this from a plain write.

write

Output a formatted record to a filehandle using a `format` declaration. `write` looks up the format associated with the current (or specified) filehandle, evaluates the format's picture lines against the current variables, and outputs the result. This is Perl's original report-generation mechanism, predating modules like `Text::Table` and template engines. In `stryke`, `write` and `format` are supported for backward compatibility but are rarely used in new code — `printf/sprintf` or template strings are more flexible. The format name defaults to the filehandle name (e.g., format `STDOUT` is used by `write STDOUT`).

```
format STDOUT =
@<<<<<<<< @>>>>>>>
$name,      $score
.

var ($name, $score) = ("alice", 42)
write # outputs: alice      42
```

write_file

Write a string to a file, creating it if it does not exist or truncating it if it does. This is the complement of `slurp` — together they form a read/write pair for whole-file operations. The short alias `wf` is convenient for one-liners. The file is opened, written, and closed in a single call.

```
spurt("hello.txt", "Hello, world!\n")
wf("nums.txt", join("\n", 1:10))
"generated content" |> wf("out.txt")
var $data = slurp("in.txt")
wf("copy.txt", $data)
```

write_json

Serialize a `stryke` data structure (hash ref or array ref) as pretty-printed JSON and write it to a file. Creates or overwrites the target file. The short alias `wj` pairs with `rj` for round-trip JSON workflows. Useful for persisting configuration, caching API responses, or generating fixture data.

```
var %data = (name => "Alice", scores => [98, 87, 95])
wj("out.json", \%data)
var $back = rj("out.json")
p $back->{name} # Alice
```

read_json

Read a JSON file from disk and parse it into a `stryke` data structure (hash ref or array ref). The short alias `rj` keeps JSON-config one-liners terse. Dies if the file does not exist or contains malformed JSON. This is the complement of `write_json/wj`.

```
val $cfg = rj("config.json")
p $cfg->{database}{host}
var @items = @{ rj("list.json") }
@items |> e { p _->{name} }$l
```

Note: numeric strings remain strings; use ``+0`` to coerce if needed.

tempfile

Create a temporary file in the system temp directory and return its absolute path as a string. The file is created with a unique name and exists on disk immediately. Use `tf` as a short alias for quick scratch files in one-liners. The caller is responsible for cleanup, though OS temp-directory reaping will eventually reclaim it.

```
val $tmp = tf()
to_file($tmp, "scratch data\n")
p il($tmp) # scratch data
var @all = map { tf() } 1:3 # three temp files
```

tempdir

Create a temporary directory in the system temp directory and return its absolute path. The directory is created with a unique name and is ready for use immediately. The short alias `tdr` mirrors `tf` for files. Useful for isolating multi-file operations like test fixtures, build artifacts, or staged output.

```
val $dir = tdr()
to_file("$dir/a.txt", "hello")
to_file("$dir/b.txt", "world")
var @files = glob("$dir/*.txt")
p scalar @files # 2
```

binmode

Set the I/O layer on a filehandle, controlling how bytes are translated during reads and writes. Without a layer argument, `binmode $fh` switches the handle to raw binary mode (no CRLF translation on Windows, no encoding transforms). With a layer, `binmode $fh, ':utf8'` enables UTF-8 decoding, `binmode $fh, ':raw'` strips all layers for pure byte I/O, and `binmode $fh, ':encoding(ISO-8859-1)'` sets a specific encoding. In stryke, encoding layers can also be specified directly in `open`: `open var $fh, '<:utf8', $path`. Call `binmode` before any I/O on the handle — calling it mid-stream may produce garbled output. For binary file processing (images, archives, network protocols), always use `:raw`.

```
open val $fh, '<', $path or die $!
binmode $fh, ':utf8'
var @lines = <$fh>

open val $bin, '<', $img_path or die $!
binmode $bin, ':raw'
read $bin, val $header, 8
p sprintf "magic: %s", unpack("H*", $header)
```

fileno

Return the underlying OS file descriptor number for a filehandle, or `undef` if the handle is not connected to a real file descriptor (e.g., tied handles, in-memory handles opened to scalar refs). File descriptor numbers are small non-negative integers managed by the OS kernel: 0 is STDIN, 1 is STDOUT, 2 is STDERR. This function is mainly useful for interfacing with system calls that require raw fd numbers, checking whether two handles share the same underlying fd, or passing descriptors to child processes. In stryke, `fileno` is rarely needed in everyday code but is essential for low-level I/O multiplexing and process management.

```
val $fd = fileno STDOUT
p "stdout fd: $fd" # 1

if (defined fileno $fh) {
  p "handle is backed by fd " . fileno($fh)
} else {
  p "not a real file descriptor"
}
```

flock

Advisory file locking for coordinating access between processes. `flock $fh, OPERATION` where OPERATION is `LOCK_SH` (shared/read lock), `LOCK_EX` (exclusive/write lock), or `LOCK_UN` (unlock). Add `LOCK_NB` for non-blocking mode: `LOCK_EX | LOCK_NB` returns false immediately if the lock is held rather than waiting. Import constants from `Fcntl`. Advisory locks are cooperative — they only work if all processes accessing the file use `flock`. In stryke, `flock` is the standard mechanism for safe concurrent file access in multi-process scripts, cron jobs, and daemons. Always unlock explicitly or let the handle close (which releases the lock). Gotcha: `flock` does not work on NFS on many systems.

```

use Fcntl ':flock'
open val $fh, '>>', 'shared.log' or die $!
flock $fh, LOCK_EX or die "lock: $!"
print $fh tm "pid $$ wrote this\n"
flock $fh, LOCK_UN
close $fh

unless (flock $fh, LOCK_EX | LOCK_NB) {
  p "file is locked by another process"
}

```

getc

Read a single character from a filehandle (default `STDIN`). Returns `undef` at EOF. The character returned respects the handle's encoding layer — under `:utf8`, `getc` returns a full Unicode character which may be multiple bytes on disk. This function is useful for interactive single-key input, parsing binary formats one byte at a time, or implementing character-level tokenizers. In stryke, `getc` blocks until a character is available on the handle. For terminal input, note that most terminals are line-buffered by default, so `getc STDIN` won't return until the user presses Enter unless you put the terminal into raw mode.

```

val $ch = getc STDIN
p "you pressed: $ch"

open val $fh, '<:utf8', $path or die $!
while (defined(val $c = getc $fh)) {
  p "char: $c (ord " . ord($c) . ") "
}

```

select

In its one-argument form, `select HANDLE` sets the default output handle for `print`, `say`, and `write`, returning the previously selected handle. This is useful for temporarily redirecting output — for example, sending diagnostics to `STDERR` while a report goes to a file. In its four-argument form, `select RBITS, WBITS, EBITS, TIMEOUT` performs POSIX `select(2)` I/O multiplexing, waiting for one or more file descriptors to become ready for reading, writing, or to report exceptions. The four-argument form is low-level and rarely used directly in stryke — prefer `IO::Select` or async I/O patterns for multiplexing. `select with $|` is also the classic way to enable autoflush on a handle.

```

val $old = select STDERR
p "this goes to stderr"
select $old

# Enable autoflush on STDOUT
select STDOUT; $| = 1
print "immediately flushed"

```

truncate

Truncate a file to a specified byte length. Accepts either a filehandle or a filename as the first argument, and the desired length as the second. `truncate $fh, 0` empties the file entirely — a common pattern when rewriting a file in place. `truncate $fh, $len` discards everything beyond byte `$len`. Returns true on success, false on failure (check `$!` for the error). In stryke, `truncate` works on any seekable filehandle. When using `truncate` to rewrite a file, remember to also `seek $fh, 0, 0` to rewind the write position — truncating does not move the file pointer. Gotcha: truncating a file opened read-only will fail.

```

open val $fh, '+<', 'data.txt' or die $!
truncate $fh, 0      # empty the file
seek $fh, 0, 0      # rewind to start
print $fh "fresh content\n"
close $fh

```

sysopen

Low-level open using POSIX flags for precise control over how a file is opened. The signature is `sysopen FH, FILENAME, FLAGS [, PERMS]`. Flags are bitwise-OR combinations from `Fcntl`: `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREAT`, `O_EXCL`, `O_TRUNC`, `O_APPEND`, `O_NONBLOCK`, and others. The optional `PERMS` argument (e.g., `0644`) sets the file mode when `O_CREAT` creates a new file, subject to the process umask. `sysopen` is the right tool when you need `O_EXCL` for atomic file creation (lock

files, temp files), `O_NONBLOCK` for non-blocking I/O, or other flags that `open` cannot express. In stryke, prefer three-argument `open` for routine file access and reserve `sysopen` for cases requiring specific POSIX semantics.

```
use Fcntl
# Atomic create - fails if file already exists
sysopen val $lock, '/tmp/my.lock', O_WRONLY|O_CREAT|O_EXCL, 0644
  or die "already running: $!"
print $lock $$

sysopen val $log, 'app.log', O_WRONLY|O_APPEND|O_CREAT, 0644
  or die "open log: $!"
```

sysread

Low-level unbuffered read directly from a file descriptor, bypassing PerlIO buffering layers. The signature is `sysread FH, SCALAR, LENGTH [, OFFSET]`. Returns the number of bytes read, 0 at EOF, or `undef` on error. Unlike `read`, `sysread` issues a single `read(2)` system call and may return fewer bytes than requested (short read). This is the right choice for sockets, pipes, and non-blocking I/O where you need precise control over how many system calls occur and cannot tolerate buffering. In stryke, never mix `sysread/syswrite` with buffered I/O (`print`, `read`, `<$fh>`) on the same handle — the buffered and unbuffered positions will diverge and produce corrupted reads.

```
open val $fh, '<:raw', $path or die $!
var $buf = ''
while (val $n = sysread $fh, $buf, 4096, length($buf)) {
  p "read $n bytes (total: " . length($buf) . ")"
}
close $fh
```

syswrite

Low-level unbuffered write directly to a file descriptor, bypassing PerlIO buffering layers. The signature is `syswrite FH, SCALAR [, LENGTH [, OFFSET]]`. Returns the number of bytes actually written, which may be less than requested on non-blocking handles or when writing to pipes/sockets (short write). Returns `undef` on error. Like `sysread`, this issues a single `write(2)` system call and must not be mixed with buffered I/O on the same handle. In stryke, `syswrite` is essential for socket programming, IPC, and performance-critical binary output where you need to avoid double-buffering. Always check the return value and handle short writes in a loop for robust code.

```
val $data = "hello, world"
var $n = syswrite $fh, $data
p "wrote $n bytes"

# Robust write loop for sockets
var $off = 0
while ($off < length $data) {
  val $written = syswrite $fh, $data, length($data) - $off, $off
  die "syswrite: $!" unless defined $written
  $off += $written
}
```

sysseek

Low-level seek on a file descriptor, bypassing PerlIO buffering. The signature is `sysseek FH, POSITION, WHENCE` with the same `WHENCE` values as `seek` (0=start, 1=current, 2=end). Returns the new position as a true value, or `undef` on failure. Unlike `seek`, `sysseek` does not flush PerlIO buffers — it operates directly on the underlying OS file descriptor. Use `sysseek` when working with `sysread/syswrite` for consistent positioning. In stryke, `sysseek` with `SEEK_CUR` and position 0 is an idiom for querying the current fd position without moving: `var $pos = sysseek $fh, 0, 1`.

```
sysseek $fh, 0, 0 # rewind to start
sysread $fh, val $buf, 512

var $pos = sysseek $fh, 0, 1
p "fd position: $pos"
```


format

Declare a picture-format template for generating fixed-width text reports. The syntax is `format NAME = ...` where each line alternates between picture lines (containing field placeholders) and argument lines (listing the variables to fill in). Placeholders include `@<<<<` (left-align), `@>>>>` (right-align), `@| | |` (center), `@###.##` (numeric with decimal), and `@*` (multiline block fill). The format ends with a lone `.` on its own line. In Strike, formats are a legacy feature preserved for compatibility with Perl 5 code — for new reports, prefer `printf/sprintf` for simple alignment or a templating module for complex layouts. Formats interact with the special variables `$~` (current format name), `$^` (header format), and `$=` (lines per page).

```
format REPORT =  
@<<<<<<<<<<< @>>>>> @ ###.##  
$item,          $qty,   $price  
.  
  
var ($item, $qty, $price) = ("Widget", 100, 9.99)  
var Sold = select REPORT  
$~ = 'REPORT'  
write REPORT  
select Sold
```

formline

Format a line of output according to a picture template, appending the result to the `$_A` (format accumulator) variable. This is the low-level engine behind Perl's `format/write` report-generation system. Template characters like `@<<<` (left-justify), `@>>>` (right-justify), and `@###.##` (numeric) control field placement.

```
$^A = ""
formline("@<<<< @>>>>\n", "Name", "Score")
formline("@<<<< @>>>>\n", "Alice", 98)
formline("@<<<< @>>>>\n", "Bob", 85)
p $^A
```

Strings

28 topics

template

Render a template string at runtime against a hashref of variables — one EJS-shaped engine with two tag families. DATA (Mustache): `{{key}}` interpolates (empty if missing); `{{#key}}...{{/key}}` is a section (array repeats the block with `{{.}}` as the element; truthy hashref renders once in its scope; falsy/empty skips); `{{^key}}...{{/key}}` is inverted; `{{!...}}` is a comment. CODE (stryke): `<% stmt %>` runs (no output), `<%= expr %>` evaluates into the output, `<%- ... -%>` trims surrounding whitespace, `<%# ... %>` is a comment. Both families work in the same template; the variables' top-level keys are also visible as `$scalars` to `<% %>` code. `<%=` is raw (not HTML-escaped) — call `web_h(...)` to escape. Same engine the web view layer uses, available in any string (codegen, config, reports, email bodies).

```
p template("Hi {{name}}!", {name => "Ada"}) # Hi Ada!
val $rows = [{n=>"a"}, {n=>"b"}]
p template("{{#rows}}- {{n}}\n{{/rows}}", {rows => $rows})
# - a
# - b
p template("{{^items}}empty{{/items}}", {items => []}) # empty
p template("<% for val $i (1:3) { %>[<%= $i %>]<% } %>', {}} # [1][2][3]
p template('total: <%= $a + $b %>', {a => 2, b => 3}) # total: 5
# single-quote templates that contain ` $vars ` so the outer interpreter
# does not interpolate them before `template` runs.
```

deburrr

Fold accented Latin letters to their basic-ASCII forms and strip combining diacritical marks — `déjà vu` becomes `deja vu`, `Œuvre` becomes `Oeuvre`, `straße` becomes `strasse`. Covers the Latin-1 Supplement and Latin Extended-A blocks (the same table as lodash `deburrr`). Use it to normalize text before slugifying, indexing, or fuzzy-matching.

```
p deburr("déjà vu") # deja vu
p deburr("Crème brûlée") # Creme brulee
p deburr("Œuvre, Æsop, Beta") # Oeuvre, Aesop, sseta
val $slug = deburr($title) |> lc |> s/\s+/-/gi
```

chomp

chomp *STRING* — remove the trailing record separator (usually `\n`) from a string in place and return the number of characters removed.

chomp is the idiomatic way to strip newlines after reading input; unlike **chop**, it only removes the value of `$/` (the input record separator), so it is safe to call on strings that do not end with a newline — it simply does nothing. You can also **chomp** an entire array to strip every element at once. In stryke, **chomp** operates on UTF-8 strings and respects multi-byte `$/` values. Prefer **chomp** over **chop** in virtually all input-processing code; **chop** is only for when you truly need to remove an arbitrary trailing character regardless of what it is.

```
val $line = <STDIN>
chomp $line
p $line
chomp(val @lines = <$fh>) # strip all at once
p scalar @lines
```

chop

chop *STRING* — remove and return the last character of a string, modifying the string in place.

Unlike `chomp`, `chop` unconditionally removes whatever the final character is — newline, letter, digit, or even a multi-byte UTF-8 codepoint in stryke. The return value is the removed character. This makes `chop` useful for peeling off known trailing delimiters or building parsers that consume input character-by-character, but dangerous for general newline removal because it will silently eat a real character if the string does not end with `\n`. When called on an array, `chop` removes the last character of every element and returns the last one removed.

```
val $s = "hello!"
var $last = chop $s # $last = "!", $s = "hello"
p $s
var @words = ("foo\n", "bar\n")
chop @words # strips trailing newline from each
@words |> e p
```

length

`length STRING` — return the **byte** length of a string. Perl 5 builtin; pairs with `index` / `rindex` / `substr` (all byte-based) so positions and lengths share a coordinate system.

`length "hello"` is 5; `length "—"` is 3 (the box-drawing char takes 3 bytes in UTF-8). For codepoint count, use `len` (stryke extension), which mirrors `[i]` / `[a:b]` slice semantics.

```
p length "hello"      # 5
p length "—"          # 3 (bytes)
p len "—"              # 1 (codepoint)
p length "\x{1F600}"  # 4 (bytes - emoji is 4-byte UTF-8)
p len "\x{1F600}"     # 1 (codepoint)
```

substr

`substr STRING, OFFSET [, LENGTH [, REPLACEMENT]]` — extract or replace a substring.

`substr` is stryke's Swiss-army knife for positional string manipulation. With two arguments it extracts from `OFFSET` to end; with three it extracts `LENGTH` characters; with four it replaces that range with `REPLACEMENT` and returns the original extracted portion. Negative `OFFSET` counts from the end of the string (`substr $s, -3` gives the last three characters). In stryke, offsets are UTF-8 character positions, making it safe for multi-byte text. `substr` as an lvalue (`substr($s, 0, 1) = "X"`) is also supported for in-place mutation. Prefer `substr` over regex when you know exact positions — it is faster and clearer.

```
val $s = "hello world"
p substr $s, 0, 5      # hello
p substr $s, -5        # world
substr $s, 6, 5, "stryke" # $s = "hello stryke"
p $s
```

index

`index STRING, SUBSTRING [, POSITION]` — return the zero-based **byte** position of the first occurrence of `SUBSTRING` within `STRING`, or -1 if not found. Pairs with `substr` and `length` (both byte-based) for Perl 5 compat.

The optional `POSITION` argument lets you start the search at a given offset, which is essential for scanning forward through a string in a loop (call `index` repeatedly, advancing `POSITION` past each hit). For multi-byte strings where you want **codepoint** positions paired with `[i]` / `[a:b]` slice operators, use `cindex` (stryke extension) instead — `index` is byte-indexed because `length` and `substr` are, and changing it would break Perl 5 binary-protocol code.

```
val $i = index "hello world", "world" # 6
p $i
var $s = "a.b.c.d"
var $pos = 0
while (($pos = index $s, ".", $pos) != -1) {
  p "dot at $pos"
  $pos++
}
```

rindex

rindex *STRING*, *SUBSTRING* [*,* *POSITION*] — return the zero-based **byte** position of the last occurrence of *SUBSTRING* within *STRING*, searching backward from *POSITION* (or the end). Mirror of **index**; pairs with **substr** and **length** for Perl 5 compat.

rindex searches from right to left, making it ideal for extracting file extensions, final path components, or the last delimiter in a string. The optional *POSITION* argument caps how far right the search begins — characters after *POSITION* are ignored. Returns -1 when the substring is not found. For codepoint positions paired with **[]** slice operators, use **crindex**.

```
val $path = "foo/bar/baz.tar.gz"
var $i = rindex $path, "/"      # 7
p substr $path, $i + 1         # baz.tar.gz
var $ext = rindex $path, "."    # 15
p substr $path, $ext + 1       # gz
```

split

split */PATTERN/*, *STRING* [*,* *LIMIT*] — divide *STRING* into a list of substrings by splitting on each occurrence of *PATTERN*.

split is one of the most-used string functions. The *PATTERN* is a regex, so you can split on character classes, alternations, or lookaheads. A *LIMIT* caps the number of returned fields; the final field contains the unsplit remainder. The special pattern " " (a single space string, not regex) mimics **awk**-style splitting: it strips leading whitespace and splits on runs of whitespace. In **stryke**, **split** integrates with **|>** pipelines, accepting piped-in strings for ergonomic one-liners. Trailing empty fields are removed by default; pass -1 as *LIMIT* to preserve them.

```
val @parts = split /\./, "a,b,c"
"one:two:three" |> split /\:/ |> e p  # one two three
var ($user, $domain) = split /\@/, $email, 2
p "$user at $domain"
```

join

join *SEPARATOR*, *LIST* — concatenate all elements of *LIST* into a single string, placing *SEPARATOR* between each pair of adjacent elements.

join is the inverse of **split**. It stringifies each element before joining, so mixing numbers and strings is fine. An empty separator (**join** "", @list) concatenates without gaps. In **stryke**, **join** works naturally with **|>** pipelines: a range or filtered list can be piped directly into **join** with a separator argument. This is the standard way to build CSV lines, path strings, or human-readable lists from arrays. **join** never adds a trailing separator — if you need one, append it yourself.

```
val $csv = join ", ", @fields
1:5 |> join "-" |> p  # 1-2-3-4-5
var @parts = ("usr", "local", "bin")
p join "/", "", @parts  # /usr/local/bin
```

uc

uc *STRING* — return a fully uppercased copy of the string.

uc performs Unicode-aware uppercasing in **stryke**, correctly handling multi-byte characters and locale-sensitive transformations. It returns a new string without modifying the original. Use **uc** for normalizing strings before comparison, formatting headers, or transforming pipeline output. In **stryke** **|>** chains, combine with **maps** or **t** for streaming uppercase transforms. For uppercasing only the first character (e.g., sentence capitalization), use **ucfirst** instead.

```
p uc "hello"  # HELLO
@words |> maps { uc } |> e p
"whisper" |> t uc |> p  # WHISPER
```

lc

lc *STRING* — return a fully lowercased copy of the string.

lc performs Unicode-aware lowercasing in **stryke**, so **lc** "\xc4" (capital A with diaeresis) correctly returns the lowercase form. It does not modify the original string — it returns a new one. This is the standard way to normalize strings for case-

insensitive comparison: `if (lc $x eq lc $y)`. In stryke `|>` pipelines, wrap `lc` with `t` to apply it as a streaming transform. For lowercasing only the first character, use `lcfirst` instead.

```
p lc "HELLO" # hello
"SHOUT" |> t lc |> t rev |> p # tuohs
var @norm = map lc, @words
@norm |> e p
```

ucfirst

ucfirst *STRING* — return a copy of the string with only the first character uppercased, leaving the rest unchanged.

This is the standard way to capitalize the first letter of a word for display, title-casing, or converting camelCase to PascalCase. In stryke, `ucfirst` is Unicode-aware and correctly handles multi-byte leading characters. It returns a new string and does not modify the original. For full uppercasing, use `uc`. A common idiom is `ucfirst lc $word` to normalize a word to "Title" form.

```
p ucfirst "hello" # Hello
p ucfirst lc "hELLO" # Hello
var @titled = map { ucfirst lc } @raw
@titled |> join " " |> p
```

lcfirst

lcfirst *STRING* — return a copy of the string with only the first character lowercased, leaving the rest unchanged.

This is useful for converting PascalCase identifiers to camelCase, or for formatting output where only the initial letter matters. In stryke, `lcfirst` is Unicode-aware, so it handles accented capitals and multi-byte first characters correctly. Like `lc`, it returns a new string rather than modifying in place. If you need the entire string lowercased, use `lc` instead.

```
p lcfirst "Hello" # hello
p lcfirst "XMLParser" # xMLParser
var @camel = map lcfirst, @PascalNames
@camel |> e p
```

chr

chr *NUMBER* — return the character represented by the given ASCII or Unicode code point.

`chr` is the inverse of `ord`: `chr(ord($c)) eq $c` always holds. It accepts any non-negative integer and returns a single-character string. In stryke, values above 127 produce valid UTF-8 characters, so `chr 0x1F600` gives you a smiley emoji with no special encoding gymnastics. For string literals, stryke supports all standard escapes: `\x{hex}`, `\u{hex}`, `\o{oct}`, `\NNN` (octal), `\cX` (control), `\N{U+hex}`, `\N{UNICODE NAME}`, plus case modifiers `\U. .\E`, `\L. .\E`, `\u`, `\l`, `\Q. .\E`.

```
p chr 65 # A
p chr 0x1F600 # smiley emoji
p "\u{0301}" # combining acute accent
p "\N{SNOWMAN}" # ❄
var @abc = map { chr(_ + 64) } 1:26
@abc |> join " " |> p # ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

ord

ord *STRING* — return the numeric (Unicode code point) value of the first character of the string.

`ord` is the inverse of `chr`: `ord(chr($n)) == $n` always holds. When passed a multi-character string, only the first character is examined — the rest are ignored. In stryke, `ord` returns the full Unicode code point, not just 0-255, so `ord "\u{1F600}"` returns 128512. This is useful for character classification, building lookup tables, or implementing custom encodings. For ASCII checks, `ord($c) >= 32 && ord($c) <= 126` tests printability.

```
p ord "A" # 65
p ord "\n" # 10
p ord "\u{1F600}" # 128512
var @codes = map ord, split //, "hello"
@codes |> e p # 104 101 108 108 111
```

hex

hex *STRING* — interpret a hexadecimal string and return its numeric value.

The leading **0x** prefix is optional: both **hex** `"ff"` and **hex** `"0xff"` return 255. The string is case-insensitive, so **hex** `"DeAdBeEf"` works fine. If the string contains non-hex characters, **hex** warns and converts up to the first invalid character. This is the standard way to parse hex-encoded values from config files, color codes, or protocol dumps. For the reverse operation (number to hex string), use **sprintf** `"%x"`. Note that **hex** always returns a number, never a string — arithmetic is immediate.

```
val $n = hex "deadbeef"
printf "0x%x = %d\n", $n, $n
p hex "ff" # 255
"cafe" |> t { hex } |> p # 51966
```

oct

oct *STRING* — interpret an octal, hexadecimal, or binary string and return its numeric value.

oct is the multi-base cousin of **hex**: it auto-detects the base from the prefix. Strings starting with **0b** are binary, **0x** are hex, and bare digits or **0**-prefixed digits are octal. This makes it the go-to for parsing Unix file permissions (**oct** `"0755"` gives 493), binary literals, or any string where the radix is embedded in the value. In **stryke**, **oct** handles arbitrarily large integers via the same big-number pathway as other arithmetic. A common gotcha: **oct** `"8"` warns because 8 is not a valid octal digit — use **hex** or a bare numeric literal instead.

```
p oct "0755" # 493
p oct "0b1010" # 10
p oct "0xff" # 255
var $perms = oct "644"
p sprintf "%04o", $perms # 0644
```

quotemeta

quotemeta *STRING* — escape all non-alphanumeric characters with backslashes, returning a string safe for interpolation into a regex.

This is essential when building dynamic regexes from user input: without **quotemeta**, characters like `.`, `*`, `(`, `)`, `[`, and `\` would be interpreted as regex metacharacters, leading to either broken patterns or security vulnerabilities (ReDoS). The equivalent inline syntax is `\Q. \E` inside a regex. In **stryke**, **quotemeta** handles the full Unicode range, escaping any character that is not `[A-Za-z0-9_]`. Use it liberally whenever you splice user-provided strings into patterns.

```
val $input = "file (1).txt"
var $safe = quotemeta $input
p $safe # file\ \(1\)\.txt
var $found = ("file (1).txt" =~ /^$safe$/)
p $found # 1
```

trim

trim *STRING* or **trim** *LIST* — strip leading and trailing ASCII whitespace.

When given a single string, **trim** returns the stripped result. When given a list or used in a pipeline, it operates in streaming mode, trimming each element individually as it flows through. Whitespace includes spaces, tabs, carriage returns, and newlines. The **tm** alias is convenient in pipeline chains where brevity matters.

```
" hello " |> tm |> p # "hello"
@raw |> tm |> e p
slurp("data.csv") |> ln |> tm |> e p
```

lines

lines *STRING* / **ln** *STRING* — split a string on newline boundaries, returning a streaming iterator of lines.

Each line is yielded without the trailing newline character. When piped from **slurp**, this gives you a lazy line-by-line view of a file without loading all lines into memory at once. Both `\n` and `\r\n` line endings are handled. The **ln** alias keeps pipelines

compact.

```
slurp("data.txt") |> lines |> e p
var @rows = lines $multiline_str
$body |> ln |> grep /TODO/ |> e p
```

words

words (alias **wd**) splits a string on whitespace boundaries and returns the resulting list of words. It handles leading, trailing, and consecutive whitespace gracefully — unlike a naive **split** `/ /`, it never produces empty strings. This is the idiomatic way to tokenize a line of text in **stryke**, and the short alias **wd** keeps pipelines compact. It is equivalent to Perl's **split** `' '` behavior.

```
val @w = wd " hello world "
p @w      # hello, world
var $line = " foo bar baz "
var $count = scalar wd $line
p $count  # 3
rl("data.txt") |> map { scalar wd _ } |> e p
```

chars

chars **STRING** / **ch** **STRING** — split a string into individual characters, returning a streaming iterator.

Each Unicode grapheme cluster is yielded as a separate string element. This is useful for character-level processing such as frequency counting, transliteration, or building character n-grams. In pipeline mode the characters stream one at a time, so you can chain with **take**, **grep**, or **with_index** without materializing the full character array.

```
"hello" |> chars |> e p      # h e l l o
var @c = chars "abc"
"emoji: \x{1F600}" |> ch |> e p
```

snake_case

snake_case (alias **sc**) converts a string from any common casing convention — camelCase, PascalCase, kebab-case, or mixed — into snake_case, where words are lowercase and separated by underscores. This is the standard naming convention for Perl and Rust variables and function names. Consecutive uppercase letters in acronyms are handled intelligently (e.g., **parseHTTPResponse** becomes **parse_http_response**).

```
p sc "camelCase"      # camel_case
p sc "PascalCase"     # pascal_case
p sc "kebab-case"     # kebab_case
p sc "parseHTTPResponse" # parse_http_response
var @methods = qw(getUserName setEmailAddr)
@methods |> map sc |> e p
```

camel_case

camel_case (alias **cc**) converts a string from any casing convention into camelCase, where the first word is lowercase and subsequent words are capitalized with no separators. This is the standard naming convention for JavaScript variables and Java methods. The function handles underscores, hyphens, and spaces as word boundaries and strips them during conversion.

```
p cc "snake_case"      # snakeCase
p cc "kebab-case"      # kebabCase
p cc "PascalCase"     # pascalCase
p cc "hello world"    # helloWorld
var @cols = qw(first_name last_name email_addr)
@cols |> map cc |> e p
```

kebab_case

kebab_case (alias **kc**) converts a string from any casing convention into kebab-case, where words are lowercase and separated by hyphens. This is the standard naming convention for CSS classes, URL slugs, and CLI flag names. Like **snake_case**, it intelligently handles acronyms and mixed-case input. The function treats underscores, spaces, and case transitions as word boundaries.

```

p kc "camelCase"      # camel-case
p kc "PascalCase"    # pascal-case
p kc "snake_case"    # snake-case
p kc "parseHTTPResponse" # parse-http-response
var $slug = kc $title # URL-safe slug

```

study

study *STRING* — hint to the regex engine that the given string will be matched against multiple patterns, allowing it to build an internal lookup table for faster matching.

In classic Perl, **study** builds a linked list of character positions so subsequent regex matches can skip impossible starting points. In practice, modern regex engines (including stryke's Rust-based engine) already perform these optimizations internally, so **study** is effectively a no-op in stryke — calling it is harmless but provides no measurable speedup. It exists for compatibility with Perl code that uses it. You only need **study** when porting legacy scripts that call it; do not add it to new stryke code expecting a performance benefit.

```

study $text # no-op in stryke, kept for compatibility
var @hits = grep { /$pattern/ } @lines
p scalar @hits

```

pos

pos *SCALAR* — gets or sets the position where the next **m//g** (global match) will start searching in the given string. After a successful **m//g** match, **pos** returns the offset just past the end of the match. You can assign to **pos(\$s)** to manually reposition the search. If the last **m//g** failed, **pos** resets to **undef**. This is essential for writing lexers or tokenizers that consume a string incrementally with **\G**-anchored patterns. In stryke, **pos** tracks per-scalar state just like Perl.

```

val $s = "abcabc"
while ($s =~ /a/g) { p pos($s) }
# 1 4
pos($s) = 0 # reset to scan again
p pos($s)   # 0

```


Arrays & Hashes

20 topics

push

push *@array*, *LIST* — appends one or more elements to the end of an array and returns the new length. This is the primary way to grow arrays in stryke and works identically to Perl's builtin. You can push scalars, lists, or even the result of a pipeline. In stryke, **push** is O(1) amortized thanks to the underlying Rust **Vec**.

```
var @q
push @q, 1:3
push @q, "four", "five"
p scalar @q      # 5
@q |> e p        # 1 2 3 four five
```

Returns the new element count, so **var \$len = push @arr, \$val;** is valid.

pop

pop *@array* — removes and returns the last element of an array, or **undef** if the array is empty. This is the complement of **push** and together they give you classic stack (LIFO) semantics. In stryke the operation is O(1) because the underlying Rust **Vec::pop** simply decrements the length. When called without an argument inside a subroutine, **pop** operates on **@_**; at file scope it operates on **@ARGV**, matching standard Perl behavior.

```
var @stk = 1:5
var $top = pop @stk
p $top      # 5
p scalar @stk # 4
@stk |> e p  # 1 2 3 4
```

shift

shift *@array* — removes and returns the first element of an array, sliding all remaining elements down by one index. Like **pop**, it returns **undef** on an empty array. Without an explicit argument it defaults to **@_** inside subroutines and **@ARGV** at file scope. Because every element must be moved, **shift** is O(n); if you only need LIFO access, prefer **push/pop**. Use **shift** when processing argument lists or implementing FIFO queues.

```
var @args = @ARGV
var $cmd = shift @args
p $cmd
@args |> e p # remaining arguments
```

unshift

unshift *@array*, *LIST* — prepends one or more elements to the beginning of an array and returns the new length. It is the counterpart of **push** for the front of the array. Like **shift**, this is O(n) because existing elements must be moved to make room. When you need to build an array in reverse order or maintain a priority queue with newest items first, **unshift** is the idiomatic choice. You can pass multiple values and they will appear in the same order they were given.

```
val @log = ("b", "c")
unshift @log, "a"
@log |> e p # a b c
var $len = unshift @log, "x", "y"
p $len      # 5
@log |> e p  # x y a b c
```

splice

`splice @array, OFFSET [, LENGTH [, LIST]]` — the Swiss-army knife for array mutation. It can insert, remove, or replace elements at any position in a single call. With just an offset it removes everything from that point to the end. With offset and length it removes that many elements. With a replacement list it inserts those elements in place of the removed ones. The return value is the list of removed elements, which is useful for saving what you cut. In stryke this compiles down to Rust's `Vec::splice` and `Vec::drain`.

```
val @a = 1:5
var @removed = splice @a, 1, 2, 8, 9
@a |> e p      # 1 8 9 4 5
@removed |> e p # 2 3
splice @a, 2    # remove from index 2 onward
@a |> e p      # 1 8
```

splice_last

`splice_last @array, OFFSET [, LENGTH [, LIST]]` — same shape as `splice`, but returns the **last** removed element instead of the full removed list (or `undef` if nothing was removed). The stryke spelling of Perl's `scalar splice(@a, off, n)` for `--no-interop` mode where `scalar` is rejected at parse time. The array is still mutated in place, exactly like `splice`. Aliases: `splice1`, `spl_last`. Desugars to `tail(splice(...))` so any list-context property of `splice` is preserved.

```
val @a = 10:50:10
var $last = splice_last @a, 1, 2 # removes (20, 30), returns 30
@a |> e p      # 10 40 50
var $popped = splice_last @b, 0, 0 # removes nothing, returns undef
```

keys

`keys %hash` — returns the list of all keys in a hash in no particular order. When called on an array, it returns the list of valid indices (0 to `$#array`). In scalar context, `keys` returns the number of keys. Calling `keys` resets the `each` iterator on that hash, which is the standard way to restart iteration. In stryke, this calls Rust's `HashMap::keys()` and collects into a `Vec`. For sorted output, chain with `sort` via the pipe operator.

```
val %env = (HOME => "/root", USER => "me")
keys(%env) |> sort |> e p # HOME USER
p scalar keys %env      # 2
var @a = (10, 20, 30)
keys(@a) |> e p          # 0 1 2
```

values

`values %hash` — returns the list of all values in a hash in no particular order (matching the order of `keys` for the same hash state). When called on an array, it returns the array elements themselves. In scalar context, returns the count of values. Like `keys`, calling `values` resets the `each` iterator. In stryke this maps to Rust's `HashMap::values()`. Combine with `sum`, `sort`, or pipeline operators for common aggregation patterns.

```
val %scores = (alice => 90, bob => 85)
p sum(values %scores) # 175
values(%scores) |> sort { _1 <=> _ } |> e p # 90 85
```

each

`each %hash` — returns the next (key, value) pair from a hash as a two-element list, or an empty list when the iterator is exhausted. Each hash has its own internal iterator, which is reset when you call `keys` or `values` on the same hash. This is memory-efficient for large hashes because it does not build the full key list. Gotcha: do not add or delete keys while iterating with `each`; that can cause skipped or duplicated entries. In stryke, the iteration order is non-deterministic (Rust `HashMap` order).

```
val %h = (a => 1, b => 2, c => 3)
while (var ($k, $v) = each %h) { p "$k=$v" }
# output order varies: a=1 b=2 c=3
```

delete

delete *EXPR* — removes a key-value pair from a hash or an element from an array and returns the removed value (or **undef** if it did not exist). For hashes this is the only way to truly remove a key; assigning **undef** to `$h{key}` leaves the key in place. For arrays, **delete** sets the element to **undef** but does not shift indices (use **splice** if you need to close the gap). In stryke, hash deletion maps to Rust's `HashMap::remove`. You can delete multiple keys at once with a hash slice: **delete** `@h{@keys}`.

```
var %h = (x => 1, y => 2, z => 3)
var $old = delete $h{x}
p $old      # 1
p exists $h{x} ? "yes" : "no"    # no
delete @h{"y", "z"}             # delete multiple keys
```

exists

exists *EXPR* — tests whether a specific key is present in a hash or an index is present in an array, regardless of whether the associated value is **undef**. This is different from **defined**: a key can exist but hold **undef**. Use **exists** to check hash membership before accessing a value to avoid autovivification side effects. In stryke, **exists** on a hash compiles to Rust's `HashMap::contains_key`, and on an array it checks bounds. You can also use it with nested structures: **exists** `$h{a}{b}` only checks the final level.

```
val %h = (a => 1, b => undef)
p exists $h{b} ? "yes" : "no"    # yes (key present, value undef)
p exists $h{c} ? "yes" : "no"    # no
var @a = (10, 20, 30)
p exists $x[5] ? "yes" : "no"    # no
```

scalar

scalar *EXPR* — forces the expression into scalar context. The most common use is **scalar** `@array` to get the element count instead of the list of elements. In stryke, scalar context on an array returns its length as a Rust **usize**. You can also use **scalar** on a hash to get the number of key-value pairs, or on a function call to force it to return a single value. This is essential when you want a count inside string interpolation or as a function argument where list context would be ambiguous.

```
val @items = ("a", "b", "c")
p scalar @items      # 3
p "count: " . scalar @items    # count: 3
var %h = (x => 1, y => 2)
p scalar %h          # 2
```

defined

defined *EXPR* — returns true if the expression has a value that is not **undef**. This is the canonical way to distinguish between "no value" and "a value that happens to be false" (such as `0`, `"`, or `"0"`). Use **defined** before dereferencing optional values or checking return codes from functions that return **undef** on failure. In stryke, **defined** compiles to a Rust `Option::is_some()` check internally. Note that **defined** on an aggregate (hash or array) is deprecated in Perl 5 and is a no-op in stryke.

```
val $x = undef
p defined($x) ? "yes" : "no"    # no
$x = 0
p defined($x) ? "yes" : "no"    # yes
var $val = fn_that_may_fail()
p $val if defined $val
```

undef

undef — the undefined value, representing the absence of a value. **null** is an interchangeable stryke alias (JS/SQL spelling). As a function, **undef** `$var` explicitly undefines a variable, freeing any value it held. **undef** is falsy in boolean context and triggers "use of uninitialized value" warnings under **use warnings**. In stryke, **undef** maps to Rust's **None** in an **Option** type internally. Use **undef** to reset variables, signal missing return values, or clear hash entries without deleting the key.

```

val $x = 42
undef $x
p defined($x) ? "def" : "undef" # undef
fn maybe { return undef if !@_
    return $_[0] }
p defined(maybe()) ? "got" : "nothing" # nothing

```

ref

ref *EXPR* — returns a string indicating the reference type of the value, or an empty string if it is not a reference. Common return values are **SCALAR**, **ARRAY**, **HASH**, **CODE**, **REF**, and **Regexp**. For blessed objects it returns the class name. Use **ref** to dispatch on data type or validate arguments in polymorphic functions. In stryke, **ref** inspects the Rust enum variant of the underlying value. Note that **ref** does not recurse; it only tells you the top-level type.

```

val $r = [1, 2, 3]
p ref($r) # ARRAY
p ref(%ENV) # HASH
p ref(&main) # CODE
p ref(42) # (empty string)

```

bless

Associate a reference with a package name, turning it into an object of that class. The blessed reference can then have methods called on it via **->**. The second argument defaults to the current package. This is the foundation of Perl's object system.

```

fn new($class, %args) {
    bless { %args }, $class
}
var $obj = new("Dog", name => "Rex", breed => "Lab")
p ref($obj) # Dog
p $obj->{name} # Rex

```

tie

Bind a variable to an implementing class so that all accesses (read, write, delete, etc.) are intercepted by methods on that class. This is Perl's mechanism for transparent object-backed variables — tied hashes can be backed by a database, tied scalars can validate on assignment, etc. Use **untie** to remove the binding.

```

tie var %db, 'DB_File', 'cache.db'
$db{key} = "value" # writes to disk
p $db{key} # reads from disk
untie %db

```

prototype

Return the prototype string of a named function, or **undef** if the function has no prototype. Prototypes control how arguments are parsed at compile time — they influence context and reference-passing behavior. Useful for introspection and metaprogramming.

```

p prototype("CORE::push") # \@@
p prototype("CORE::map") # &@
fn greet($name) { p "hi $name" }
p prototype(&greet) # undef (signatures, no proto)

```

wantarray

wantarray() — returns true if the current subroutine was called in list context, false in scalar context, and **undef** in void context. This lets a function adapt its return value to the caller's expectations. A common pattern is returning a list in list context and a count or reference in scalar context. In stryke, use **fn** to define subroutines and **wantarray()** inside them just like in Perl. Note that **wantarray** only reflects the immediate call site, not nested contexts.

```

fn ctx { wantarray() ? "list" : "scalar" }
var @r = ctx()
p $r[0] # list
var $r = ctx()
p $r # scalar
fn flexible { wantarray() ? {1, 2, 3} : 3 }
var @all = flexible()
p scalar @all # 3
var $cnt = flexible()
p $cnt # 3

```

caller

caller [LEVEL] — returns information about the calling subroutine's context. In list context it returns (**package**, **filename**, **line**) for the given call-stack level (default 0, the immediate caller). With higher levels you can walk up the call stack for debugging or generating stack traces. In scalar context, **caller** returns just the package name. In **stryke**, **caller** works with **fn**-defined subroutines and integrates with the runtime's frame tracking. This is invaluable for building custom error reporters or trace utilities.

```

fn here {
  var ($pkg, $f, $ln) = caller()
  p "$f:$ln"
}
here() # prints current file:line
fn deep {
  var ($pkg, $f, $ln) = caller(1)
  p "grandparent: $f:$ln"
}

```

Control Flow

17 topics

if

The fundamental conditional construct. Evaluates its condition in boolean context and executes the block if true. `stryke` supports both the block form `if (COND) { BODY }` and the postfix form `EXPR if COND`, which is idiomatic for single-statement guards. The condition can be any expression — numbers (0 is false), strings (empty and `"0"` are false), `undef` (false), and references (always true). Postfix `if` cannot have `elsif/else` — use the block form for multi-branch logic.

```
val $n = 42
if ($n > 0) { p "positive" }
p "big" if $n > 10          # postfix - clean one-liner
var $label = "even" if $n % 2 == 0
p "got: $n" if defined $n   # guard against undef
```

elsif

Chain additional conditions after an `if` block without nesting. Each `elsif` is tested in order; the first one whose condition is true has its block executed, and the rest are skipped. There is no limit on the number of `elsif` branches. In `stryke`, prefer `match` for complex multi-branch dispatch since it supports pattern matching, destructuring, and guards — but `elsif` remains the right tool for simple linear condition chains. Note: it is `elsif`, not `elseif` or `else if` — the latter is a syntax error.

```
fn classify($n) {
  if ($n < 0) { "negative" }
  elsif ($n == 0) { "zero" }
  elsif ($n < 10) { "small" }
  elsif ($n < 100) { "medium" }
  else { "large" }
}
1:200 |> map classify |> frequencies |> dd
```

else

The final fallback branch of an `if/elsif` chain, executed when no preceding condition was true. Every `if` can have at most one `else`, and it must come last. For ternary-style expressions, use `COND ? A : B` instead of an `if/else` block — it composes better in `|>` pipes and assignments.

```
if ($n % 2 == 0) { p "even" }
else { p "odd" }

# Ternary is often cleaner in pipes:
1:10 |> map { _ % 2 == 0 ? "even" : "odd" } |> e p
```

unless

A negated conditional — executes the block when the condition is `*false*`. This reads more naturally than `if (!COND)` for guard clauses and early returns. `stryke` supports both block and postfix forms. Convention: use `unless` for simple negative guards; avoid `unless` with complex compound conditions, as double-negatives hurt readability. There is no `unlessif` — use `if/elsif` chains for multi-branch logic.

```
unless ($ENV{QUIET}) { p "verbose output" }
p "missing!" unless -e $path # postfix guard
die "no input" unless @ARGV
return unless defined $val   # early return pattern
```

for

Iterate over a list, binding each element to a loop variable (or `_` by default). `for` and `foreach` are interchangeable keywords. The loop variable is automatically localized and aliases the original element — modifications to `_` inside the loop mutate the list in-place. In `stryke`, `for` loops work with ranges, arrays, hash slices, and iterator results. For parallel iteration, see `pfor`; for pipeline-style processing, prefer `|> e` or `|> map`. C-style `for (INIT; COND; STEP)` is also supported.

```
for val $f (glob "*.txt") { p $f }
for (1:5) { p _ * 2 }           # _ is default
var @names = qw(alice bob carol)
for (@names) { _ = uc _ }      # mutates in-place
p join ", ", @names            # ALICE, BOB, CAROL
```

while

Loop that re-evaluates its condition before each iteration and continues as long as it is true. Commonly used for reading input line-by-line, polling, and indefinite iteration. The condition is tested in boolean context. `while` integrates naturally with the diamond operator `<>` for reading filehandles. The loop variable can be declared in the condition with `my`, scoping it to the loop body. Postfix form is also supported: `EXPR while COND;`

```
while (val $line = <STDIN>) {
    $line |> tm |> p           # trim + print each line
}
var $i = 0
while ($i < 5) { p $i++ }     # counted loop
while (1) { last if done() }  # infinite loop with break
```

until

Loop that continues as long as its condition is `*false*` — the logical inverse of `while`. Useful when the termination condition is more naturally expressed as a positive assertion (“keep going until X happens”). Supports both block and postfix forms. Prefer `while` with a negated condition if `until` makes the logic harder to read.

```
val $n = 1
until ($n > 1000) { $n *= 2 }
p $n      # 1024

var $tries = 0
until (connected()) {
    $tries++
    sleep 1
}
p "connected after $tries tries"
```

loop

Rust-style infinite loop — exactly equivalent to `while (1) { ... }` and desugared to one at parse time. Use when the only sane exit is `last` / `return` / `die` / a signal: the omitted condition makes the intent obvious to a reader instead of obscuring it behind a `1` literal.

Labels, `last LABEL`, `next LABEL`, and `continue { ... }` all work because the desugar shares the existing `while` runtime — there is no separate `loop` opcode.

```

loop {
  val $msg = recv_or_undef()
  last unless defined $msg
  handle($msg)
}

# polling pattern
loop {
  val $line = readline(STDIN)
  last if !defined $line
  next if $line =~ /^s*$/
  process($line)
}

# labeled loops compose
OUTER: loop {
  for val $job (@queue) {
    last OUTER if $job->{shutdown}
    run($job)
  }
  sleep(1)
}

```

See also: `while`, `until`, `last`, `next`.

do

Execute a block and return its value, or execute a file. As a block, `do { ... }` creates an expression scope — the last expression in the block is the return value, making it useful for complex initializations. As a file operation, `do "file.pl"` executes the file in the current scope and returns its last expression. Unlike `require`, `do` does not cache and re-executes on each call. `do { ... } while (COND)` creates a loop that always runs at least once.

```

val $val = do { val $x = 10
  $x ** 2 } # 100
p $val
var $cfg = do "config.pl" # load config
# do-while: body runs at least once
var $input
do { $input = readline(STDIN) |> tm } while ($input eq "")

```

last

Immediately exit the innermost enclosing loop (equivalent to `break` in C/Rust). Execution continues after the loop. `last LABEL` can target a labeled outer loop to break out of nested loops. Works in `for`, `foreach`, `while`, `until`, and `do-while`. Does *not* work inside `map`, `grep`, or `|>` pipeline stages — use `take`, `first`, or `take_while` for early termination in functional contexts.

```

for (1:1_000_000) {
  last if _ > 5
  p _
} # prints 1 2 3 4 5

OUTER: for val $i (1:10) {
  for val $j (1:10) {
    last OUTER if $i * $j > 50 # break both loops
  }
}

```

For pipeline early-exit: `1:1000 |> take_while { _ < 50 } |> e p`.

next

Skip the rest of the current loop iteration and jump to the next one. The loop condition (for `while/until`) or the next element (for `for/foreach`) is evaluated immediately. Like `last`, `next` supports labeled loops with `next LABEL` for skipping in nested loops. This is the primary tool for filtering within imperative loops. In `stryke`, consider `grep/filter` or `|> reject` for functional-style filtering instead.


```

for (1:10) {
  next if _ % 2      # skip odds
  p _
} # 2 4 6 8 10

for val $file (glob "**") {
  next unless -f $file # skip non-files
  next if $file =~ /^\. / # skip hidden
  p $file
}

```

redo

Restart the current loop iteration from the top of the loop body *without* re-evaluating the loop condition or advancing to the next element. The loop variable retains its current value. This is a niche but powerful tool for retry logic within loops — when an iteration fails, **redo** lets you try again with the same input. Use sparingly, as it can create infinite loops if the retry condition never resolves. Always pair with a guard or counter. For automated retry with backoff, prefer `retry { ... } times => N, backoff => 'exponential'`.

```

for val $url (@urls) {
  val $body = eval { fetch($url) }
  if ($?) {
    warn "retry $url: $@"
    sleep 1
    redo # try same URL again
  }
  p length($body)
}

```

continue

A block attached to a `for/foreach/while` loop that executes after each iteration, even when `next` is called. Analogous to the increment expression in a C-style `for` loop. The `continue` block does *not* run when `last` or `redo` is used. Useful for unconditional per-iteration bookkeeping like incrementing counters, logging progress, or flushing buffers. Rarely used but fully supported in stryke.

```

val $count = 0
for val $item (@work) {
  next if $item->{skip}
  process($item)
} continue {
  $count++
  p "processed $count so far" if $count % 100 == 0
}

```

given

A switch-like construct that evaluates an expression and dispatches to `when` blocks via smartmatch semantics. The topic variable `_` is set to the `given` expression for the duration of the block. Each `when` clause is tested in order; the first match executes its block and control passes out of the `given` (implicit break). A `default` block handles the no-match case. In stryke, prefer the `match` keyword for new code — it offers pattern destructuring, typed patterns, array/hash shape matching, and `if` guards that `given/when` cannot express.

```

given ($cmd) {
  when ("start") { p "starting up" }
  when ("stop") { p "shutting down" }
  when (/^re/) { p "restarting" }
  default { p "unknown: $cmd" }
}

```

See `match` for stryke-native pattern matching with destructuring.

when

A case clause inside a `given` block. The expression is matched against the topic `_` using smartmatch semantics: strings match exactly, regexes match against `_`, arrayrefs check membership, coderefs are called with `_` as argument, and numbers compare numerically. When a `when` clause matches, its block executes and control exits the enclosing `given` (implicit break). Multiple `when` clauses are tried in order until one matches.

```
given ($val) {
  when (/^\d+$/ ) { p "number" }
  when (["a","b","c"]) { p "early letter" }
  when (42) { p "the answer" }
  default { p "something else" }
}
```

In `stryke`, the `match` keyword provides more powerful pattern matching.

default

The fallback clause in a `given` block, executed when no `when` clause matched. Every `given` should have a `default` to handle unexpected values, similar to `else` in an `if` chain or the wildcard `_` arm in `stryke match`. If no `default` is present and nothing matches, execution simply continues after the `given` block. In `stryke match`, use `_ => ...` for the default arm instead.

```
given ($exit_code) {
  when (0) { p "success" }
  when (1) { p "general error" }
  when (2) { p "misuse" }
  default { p "unknown exit code: $exit_code" }
}
```

return

Return a value from a subroutine.

```
fn pin_val($v, $lo, $hi) {
  return $lo if $v < $lo
  return $hi if $v > $hi
  $v
}
```

Error Handling

8 topics

try

Structured exception handling that cleanly separates the happy path from error recovery. The `try` block runs the code; if it throws (via `die`), execution jumps to the `catch` block with the exception bound to the declared variable. An optional `finally` block runs unconditionally afterward — ideal for cleanup like closing filehandles or releasing locks. Unlike `eval`, `try/catch` is a first-class statement with proper scoping and no `$@` pollution. In stryke, `try` integrates with all exception types including string messages, hashrefs, and objects.

```
try {
  val $data = fetch_json($url)
  p $data->{name}
} catch ($e) {
  warn "request failed: $e"
  return fallback()
} finally {
  log_info("fetch attempt complete")
}
```

Prefer `try/catch` over `eval { }` for new code — it reads better and avoids `$@` clobbering races.

catch

The error-handling clause that follows a `try` block. When the `try` block throws an exception, execution transfers to `catch` with the exception value bound to the declared variable (e.g. `$e`). The catch variable is lexically scoped to the catch block. You can inspect the exception — it may be a string, a hashref with structured error info, or an object with methods. Multiple error types can be differentiated with `ref` or `match` inside the catch body. If the catch block itself throws, the exception propagates upward (the `finally` block still runs first, if present).

```
try { die { code => 404, msg => "not found" } }
catch ($e) {
  if (ref $e eq 'HASH') {
    p "error $e->{code}: $e->{msg}"
  } else {
    p "caught: $e"
  }
}
```

finally

A cleanup block that runs after `try/catch` regardless of whether an exception was thrown or not. This guarantees resource cleanup even if the `try` block throws or the `catch` block re-throws. The `finally` block cannot change the exception or the return value — it is strictly for side effects like closing filehandles, releasing locks, or logging. If `finally` itself throws, that exception replaces the original one (avoid throwing in finally). `finally` is optional — you can use `try/catch` without it.

```
val $fh
try {
  open $fh, '<', $path or die $!
  process(<$fh>)
} catch ($e) {
  log_error("failed: $e")
} finally {
  close $fh if $fh # always cleanup
}
```

eval

The classic Perl exception-catching mechanism. `eval { BLOCK }` executes the block in an exception-trapping context: if the block throws (via `die`), execution continues after the `eval` with the error stored in `$@`. `eval "STRING"` compiles and executes Perl code at runtime — powerful but dangerous (code injection risk). In stryke, prefer `try/catch` for exception handling as it avoids the `$@` clobbering pitfalls and reads more clearly. `eval` remains useful for dynamic code evaluation and backward compatibility with Perl 5 idioms.

```
eval { die "oops" }
if ($@) { p "caught: $@" }

# Eval string – dynamic code execution
var $expr = "2 + 2"
var $result = eval $expr
p $result    # 4
```

Caveat: `$@` can be clobbered by intervening `evals` or destructors — `try/catch` avoids this.

die

Raise an exception, immediately unwinding the call stack until caught by `try/catch` or `eval`. The argument can be a string (most common), a reference (hashref for structured errors), or an object. If uncaught, the program terminates and the message is printed to STDERR. In stryke, `die` works identically to Perl 5 and integrates with `try/catch`, `eval`, and the `$@` mechanism. Convention: end die messages with `\n` to suppress the automatic "at FILE line LINE" suffix, or omit it to get location info for debugging.

```
fn divide($x, $y) {
  die "division by zero\n" unless $y
  $x / $y
}

# Structured error
die { code => 400, msg => "bad request", field => "email" }

# With automatic location
die "something broke"    # prints: something broke at script.pl line 5.
```

warn

Print a warning message to STDERR without terminating the program. Behaves like `die` but only emits the message instead of throwing an exception. If the message does not end with `\n`, Perl appends the current file and line number. Warnings can be intercepted with `$SIG{__WARN__}` or suppressed with `no warnings`. In stryke, `warn` is useful for non-fatal diagnostics; for structured logging, use `log_warn` instead.

```
warn "file not found: $path" unless -e $path
warn "deprecated: use fetch_json instead\n" # no line number

# Intercept warnings
local $SIG{__WARN__} = fn ($msg) { log_warn($msg) }
warn "redirected to logger"
```

croak

Die from the caller's perspective — the error message reports the file and line of the `*caller*`, not the function that called `croak`. This is the right choice for library/module functions where the error is the caller's fault (bad arguments, misuse). Without `croak`, the user sees an error pointing at library internals, which is unhelpful. In stryke, `croak` is available as a builtin without `use Carp`. For debugging deep call chains, use `confess` instead to get a full stack trace.

```

fn parse($s) {
  croak "parse: empty input" unless length $s
  croak "parse: not JSON" unless $s =~ /^[\{\[\]/
  json_decode($s)
}

# Error will point at the call site, not inside parse()
var $data = parse("") # dies: "parse: empty input at caller.pl line 5"

```

confess

Die with a full stack trace from the point of the error all the way up through every caller. This is invaluable for debugging deep call chains where `die` or `croak` only show one frame. Each frame includes the package, file, and line number. In `stryke`, `confess` is available as a builtin without `use Carp`. Use `confess` during development for maximum diagnostic info; switch to `croak` in production-facing libraries where the trace would confuse end users.

```

fn validate($data) {
  confess "missing required field 'name'" unless $data->{name}
}
fn process($input) { validate($input) }
fn main { process({}) } # full trace: main -> process -> validate

```

The trace output shows: `missing required field 'name' at script.pl line 2.\n\tmain::validate called at line 4\n\t...`

AOP / Advice

7 topics

before

`before "<glob>" { ... }` — register advice that runs *before* every call to a sub whose name matches the glob pattern. Inside the body, `$INTERCEPT_NAME` holds the called sub's name and `@INTERCEPT_ARGS` holds the args.

The leading keyword only commits to advice parsing when followed by a string literal, so `before(...)` as a normal sub call still parses normally. Multiple `before` advices on the same name all fire in registration order. The advice cannot suppress the call (use `around` for that) and its return value is discarded.

```
before "fetch" { warn "calling fetch with @INTERCEPT_ARGS" }
before "log_*" { $log_count++ }      # glob: any sub starting with log_
before "*"     { trace($INTERCEPT_NAME) } # every sub call
```

Bodies are lowered to bytecode and dispatched through the VM (`run_block_region`), so `our $x` and other compile-time name resolutions work the same inside advice as outside it. The body's final statement must be an expression (same constraint as `map { }` block lowering); a literal `for/while/if` block or a literal `return` will be rejected at firing time.

after

`after "<glob>" { ... }` — register advice that runs *after* every call to a sub whose name matches the glob pattern. The body sees the call's return in `$INTERCEPT_RESULT`, the wall-clock duration in `$INTERCEPT_MS` (millis, float) and `$INTERCEPT_US` (micros, integer), the called name in `$INTERCEPT_NAME`, and the args in `@INTERCEPT_ARGS`.

Multiple `after` advices on the same name all fire in registration order. The advice's return value is discarded — the call's return is whatever the original sub (or a matching `around`) produced.

```
after "fetch" { warn "fetch returned $INTERCEPT_RESULT in ${INTERCEPT_MS}ms" }
after "*"     { log_call($INTERCEPT_NAME, $INTERCEPT_US) }
```

around

`around "<glob>" { ... }` — register advice that *wraps* every call to a sub whose name matches the glob pattern. Use `proceed()` to invoke the original; the around block's evaluated value is the call's return (AspectJ-style).

The first matching `around` on a given name wraps; later `around` matches are skipped (mirrors zshrs `run_intercepts`). If `proceed()` is not called, the original sub never runs and the around block's value replaces it. The block can transform (`proceed() + 100`), forward (just `proceed()`), or replace (omit `proceed()` and emit a value).

```
around "expensive" {
  val $cached = cache_get($INTERCEPT_ARGS[0])
  return $cached if defined $cached
  val $r = proceed()
  cache_put($INTERCEPT_ARGS[0], $r)
  $r
}

around "flaky" { val $r = eval { proceed() }; $@ ? retry_default() : $r }
```

Recursion guard: calling the advised sub from inside its own advice runs the original directly without re-firing advice — no infinite loop.

proceed

`proceed()` — invoke the original sub with the saved args from inside an `around` advice block. Returns the original's value. Calling `proceed` outside an `around` block is a runtime error.

The re-entrancy guard ensures `proceed` runs the original directly without re-firing any matching `around` advice for the same name, so transformation chains can't infinite-loop.

```
around "target" {
    val $r = proceed()
    $r * 2      # double whatever target returned
}

around "target" { proceed() // "default" }    # forward, with fallback
```

intercept_list

`intercept_list()` — return the registered AOP advice as an array of `[id, kind, pattern]` triples (each element is an arrayref).

```
before "foo" { ... }
after  "bar*" { ... }
for val $row (intercept_list()) {
    var ($id, $kind, $pat) = @$row
    p "id=$id kind=$kind pattern=$pat"
}
```

intercept_remove

`intercept_remove($id)` — remove a single AOP advice by its id (from `intercept_list`). Returns the count removed (0 or 1).

```
before "foo" { ... }
var ($id, $kind, $pat) = @{ (intercept_list())[0] }
intercept_remove($id)
p scalar intercept_list()    # 0
```

intercept_clear

`intercept_clear()` — drop all registered AOP advice. Returns the count cleared.

```
before "a" { ... }
after  "b" { ... }
p intercept_clear()        # 2
p scalar intercept_list()  # 0
```

Declarations

12 topics

my

Declare a lexically scoped variable visible only within the enclosing block or file. `my` is the workhorse of Perl variable declarations — use it for scalars (`$`), arrays (`@`), and hashes (`%`). Variables declared with `my` are invisible outside their scope, which prevents accidental cross-scope mutation. In `stryke`, `my` variables participate in pipe chains and can be destructured in list context. Uninitialized `my` variables default to `undef`.

```
val $name = "world"
var @nums = 1:5
var %cfg = (debug => 1, verbose => 0)
p "hello $name"
@nums |> grep _ > 2 |> e p$1

Use `my` everywhere unless you specifically need `our` (package global), `local` (dynamic scope), or
`state` (persistent). Aliases: `var` (Kotlin/Scala/Java spelling) and `val` (immutable shorthand for
`const my`).
```

var

Kotlin/Scala/Java-style synonym for `my` — declares a lexically scoped, **mutable** variable. Parses identically to `my` (same `StmtKind::My` AST, same scoping rules, same bytecode); pick whichever spelling reads better at the call site. For an immutable binding use `val` (the `const my` equivalent).

```
var $name = "world"
var @nums = 1:5
var %cfg = (debug => 1)
$name = "stryke" # ok - var is mutable
```

See also: `my` (the canonical spelling), `val` (immutable), `const my`, `our`, `state`.

val

Kotlin/Scala-style synonym for `const my` — declares a lexically scoped, **frozen (immutable)** variable. Reassignment is a compile-time error.

```
val $PI = 3.14159
val $config = { host => "localhost", port => 8080 }
$PI = 3.14 # COMPILER ERROR: cannot assign to frozen variable `$PI`
```

Type annotations are accepted the same way as `const my`:

```
val $count : Int = 0
val @names : Array[Str] = ("alice", "bob")
```

Use `val` for constants (configuration, derived expressions, IDs you compute once and reference many times). For mutable bindings use `var` or `my`.

See also: `var` (mutable), `my`, `const my` / `frozen my` (same semantics), `typed`.

our

Declare a package-global variable accessible as a lexical alias in the current scope. Unlike `my`, `our` variables are visible across the entire package and can be accessed from other packages via their fully qualified name (e.g. `$Counter::total`). Useful for package-level configuration, shared counters, or variables that need to survive across file boundaries.

In stryke, `our` variables are NOT mutex-protected. Mutating an `our` variable from inside `fan/pmap/pfor` is rejected at runtime with a directive to use `oursync` (the package-global counterpart of `varsync` — same `Arc<Mutex>` backing, keyed by `Pkg::x`).

```
package Counter
our $total = 0          # serial use only
fn bump { $total++ }
package main
Counter::bump() for 1:5
p $Counter::total      # 5

# For parallel mutation, use oursync:
package C
oursync $shared = 0
fn bump { $shared++ }
package main
fan 10000 { C::bump() } # always exactly 10000
```

Prefer `my` for local state; `our` for cross-package read-mostly globals; `oursync` for cross-package mutation under parallelism.

local

Temporarily override a global variable's value for the duration of the current dynamic scope. When the scope exits, the original value is automatically restored. This is essential for modifying Perl's special variables (like `$/`, `$\`, `$,`) without permanently altering global state. Unlike `my` which creates a new variable, `local` saves and restores an existing global. In stryke, `local` works with all special variables and respects the same restoration semantics during exception unwinding.

```
local $/ = undef        # slurp mode
open val $fh, '<', 'data.txt' or die $!
var $body = <$fh>       # reads entire file
close $fh
# $/ restored to "\n" when scope exits
```

Common patterns: `local $/ = undef` (slurp), `local $, = ", "` (join print args), `local %ENV` (temporary env).

state

Declare a persistent lexical variable that retains its value across calls to the enclosing subroutine. Unlike `my`, which reinitializes on each call, `state` initializes only once — the first time execution reaches the declaration — and preserves the value for all subsequent calls. This is perfect for counters, caches, and memoization without resorting to globals or closures over external variables. In stryke, `state` variables are per-thread when used inside `~> { }` or `fan` blocks; they are not shared across workers.

```
fn ctr {
  state $n = 0
  ++$n
}
p ctr() for 1:5 # 1 2 3 4 5
fn cache_lookup($x) {
  state %cache
  $cache{$x} //= expensive($x)
}
```

Requires `use feature 'state'` in standard Perl, but is always available in stryke.

package

Set the current package namespace for all subsequent declarations. Package names are conventionally `CamelCase` with `::` separators (e.g. `Math::Utils`). All unqualified function and `our` variable names are installed into the current package. In stryke, packages work identically to standard Perl — they provide namespace isolation but are not classes by themselves (use `struct` or `bless` for OOP). Switching packages mid-file is allowed but discouraged; prefer one package per file.

```
package Math::Utils
fn calc_fact($n) { $n <= 1 ? 1 : $n * calc_fact($n - 1) }
fn calc_fib($n) { $n < 2 ? $n : calc_fib($n - 1) + calc_fib($n - 2) }
package main
p Math::Utils::calc_fact(10) # 3628800
```

use

Load and import a user module at compile time: `use Module qw(func);` (or the alias `import Module qw(func);`). Stryke list utilities (`sum`, `max`, `uniq`, `reduce`, ...), error helpers (`croak`, `confess`, `carp`), and serialization (`json_encode`, `json_decode`, `ddump`) are all native bare-name builtins — no import needed.

```
use My::Project::Helpers qw(normalize_path canonicalize)
import My::Project::Helpers qw(normalize_path canonicalize) # same statement

var @vals = 1:10
p sum(@vals) # 55
p max(@vals) # 10
p normalize_path("/tmp/./etc")
```

Per-module version pin — `use Module VERSION` follows Perl 5's `use Module VERSION LIST` grammar. The numeric first arg is captured as the package-version pin and routes resolution directly to `~/.stryke/store/<name>@<VERSION>/lib/...`. A trailing `, LIST` is the import list as usual.

```
use Some::Module 2.13 # pin to ~/.stryke/store/some@2.13/
use Some::Module 1.0, 'foo', 'bar' # pin + import LIST
```

Resolution order for plain `use Foo::Bar`:

1. Project-local `lib/Foo/Bar.stk` — live edits always win.
2. **Inside a project** (any `stryke.toml` ancestor): the `stryke.lock` pin is authoritative. If the lockfile pins `foo@1.0` but the store doesn't have that extraction, the runtime errors — it never silently substitutes a different version.
3. **Outside a project**: scan `~/.stryke/store/foo@*` and pick the HIGHEST semver. `2.0` beats `1.99`; `0.10.0` beats `0.3.0`. The store-scan honors the namespace bridge (`use GUI` finds `stryke-gui@*`).
4. `@INC` system paths.

Version-respect is strict. Use-site pin OR lockfile pin missing in the store → hard error, not a silent fall-through. Run `s install` to populate the missing extraction.

no

Unimport a module or pragma: `no strict 'refs';`.

```
no warnings 'experimental'
given ($x) { when (1) { p "one" } }
```

require

Load a user module at runtime: `require Module;`. JSON / YAML / TOML decoding is built in (`json_decode`, `yaml_decode`, `toml_decode`) — no module load needed.

```
require My::Plugin::Loader
var $loader = My::Plugin::Loader::init()
p $loader->status

# Built-in: no require needed
var $data = json_decode($text)
p $data->{name}
```

BEGIN

`BEGIN { }` — runs at compile time, before the rest of the program.

```
BEGIN { p "compiling..." }
p "running"
# output: compiling... then running
```

END

`END { }` — runs after the program finishes (or on exit).

Cluster / Distributed

4 topics

cluster

`cluster(["host1:N", "host2:M", ...])` — build an SSH-backed worker pool for distributing stryke workloads across multiple machines.

Each entry in the list is a hostname (or `user@host`) followed by a colon and the number of worker slots to allocate on that host. Under the hood, stryke opens persistent SSH multiplexed connections to each host, deploys lightweight `stryke --remote-worker` processes, and manages a work-stealing scheduler across the entire cluster. The cluster object is then passed to distributed primitives like `pmap_on` and `pflat_map_on`. Workers must have `stryke` installed and accessible on `$PATH`. If a host becomes unreachable mid-computation, its in-flight tasks are automatically re-queued to surviving hosts.

```
val $cl = cluster(["server1:8", "server2:4", "gpu-box:16"])
var @results = pmap_on $cl, { heavy_compute } @jobs

# Single-machine cluster for testing:
var $local = cluster(["localhost:4"])
```

pmap_on

`pmap_on $cluster, { BLOCK } @list` — distributed parallel map that fans work across a `cluster` of remote machines.

Elements from `@list` are serialized, shipped to remote `stryke --remote-worker` processes over SSH, executed in parallel across every worker slot in the cluster, and the results are gathered back in input order. This is the distributed equivalent of `pmap`: same interface, same order guarantee, but the thread pool spans multiple hosts instead of local cores. Use this when a single machine's CPU count is the bottleneck. The block must be self-contained — it cannot close over local file handles or database connections, since it executes in a separate process on a remote host. Large closures are serialized once and cached on each worker for the lifetime of the cluster.

```
val $cl = cluster(["host1:8", "host2:8"])
var @hashes = pmap_on $cl, { sha256(slurp) } @file_paths
var @results = pmap_on $cl, { fetch("https://api.example.com/$_") } 1:10_000
```

pflat_map_on

`pflat_map_on(CLUSTER, BLOCK, LIST)` — distributed parallel flat-map: ships each input item over the wire to a worker in `CLUSTER` (a list of `"host:port"` strings registered with `cluster_init`), evaluates the block remotely, and streams flattened results back. Output is concatenated by completion order across workers. Drop-in distributed analog of local `pflat_map`.

The cluster handle is a normal stryke value, so you can have several pools targeting different host sets. Each remote eval runs in the worker's own interpreter, so be explicit about closure values — they're serialized via the wire protocol, not captured by reference. Builtins are available remotely; user-defined `fns` must be installed on the worker via `cluster_install_code`.

```
val $cluster = cluster_init("worker01:7878", "worker02:7878", "worker03:7878")

var @hashes = pflat_map_on $cluster, { sha256(slurp(_)) } @big_files

fn explode = (_, _ * 10, _ * 100)
cluster_install_code($cluster, \Gexplode)
var @out = pflat_map_on $cluster, explode, 1:100
```

For distributed `map` (1:1) use `pmap_on`. For local flat-map use `pflat_map`. For streaming variant use `pflat_maps_on`.

See also: `pflat_map`, `pmap_on`, `cluster_init`, `cluster_install_code`.

ssh

`ssh($host, $command)` — execute a shell command on a remote host via SSH and return its stdout as a string.

This is a simple synchronous wrapper around an SSH invocation. The command is run in the remote user's default shell, and stdout is captured and returned. If the remote command exits non-zero, `stryke` dies with the stderr output. For bulk remote work, prefer `cluster` + `pmap_on` which manages connection pooling and parallelism automatically. `ssh` is best for one-off administrative commands, health checks, or bootstrapping a remote environment before building a cluster.

```
val $uptime = ssh("server1", "uptime")
p ssh("deploy@prod", "cat /etc/hostname")
var $free = ssh("gpu-box", "nvidia-smi --query-gpu=memory.free --format=csv,noheader")
```

Scriptable Distributed Compute (congregation)

29 topics

congregation

`congregation($n)` — fork N stryke worker children locally, wait for them to register with an auto-spawned controller on `127.0.0.1`, return an array of N worker handles (the agent `session_ids`). Sets the current controller so subsequent `pray/annex` calls don't need an explicit handle.

```
val @workers = congregation(4)
var %results = harvest "compute()", @workers
excommunicate(@workers)
```

See also: `anoint`, `ordain`, `pray`, `harvest`, `excommunicate`.

ordain

`ordain([$name, [$bind, [$port]]])` — explicit lower-level form of `congregation`. Spawns a controller (no agent processes), registers `$name` → `endpoint` in the in-process cathedral so subsequent `profess($name)` calls can resolve. Returns controller handle ID; sets the current controller.

```
ordain "renderfarm", "0.0.0.0", 9999
welcome 4, 30_000 # wait for 4 slaves to profess
```

See also: `congregation`, `anoint`, `profess`, `cathedral`.

muster

`muster([$controller_id])` — return the current congregation's worker handles as an array of session-ids (sorted ascending). With no argument, uses the current controller (set by the most recent `congregation` or `ordain`).

See also: `congregation`, `welcome`.

pray

`pray($code, @agent_handles)` — scatter `$code` to every agent in `@agent_handles`, return a divination ID for `annex` to consume. Agents start executing immediately in parallel; this call returns as soon as the EVAL frame is on every wire.

`$code` accepts either a string OR a coderef. Coderef body is deparsed via `crate::deparse::deparse_block` before shipping. Closure captures are NOT supported in v1 — body must be self-contained.

```
val $div = pray fn { 2 + _ }, @workers # coderef form
var $div = pray "2 + 3", @workers      # string form
var %results = annex $div
```

See also: `annex`, `harvest`, `chant`.

annex

`annex($divination_id [, $timeout_ms])` — block until every agent that received the prayer has replied (or per-agent timeout, default 30s). Returns a hash keyed by stringified session-id; values are each agent's eval output. Consumes the divination — calling `annex` twice on the same ID is an error.

Agents that timed out or disconnected are silently omitted from the result hash.

See also: `pray`, `harvest`, `lick` (non-destructive peer alternative).

harvest

`harvest($code, @agent_handles [, $timeout_ms])` — fused `pray` + `annex`. Scatters `$code` to every handle, blocks for replies up to `$timeout_ms` (default 30s), returns the result hash directly. No divination id leaked. The common scatter-gather pattern in one call.

Last arg is treated as a timeout if it's a bare integer ≥ 100 (heuristic; for unambiguous form pass `[@cong]` as an array literal).

```
val %frames = harvest "render_frame()", @workers, 60_000
```

See also: `pray`, `annex`.

excommunicate

`excommunicate(@agent_handles)` — send `SHUTDOWN` frames to every named agent. Each agent exits its loop and its TCP connection drops. Returns the count of agents successfully notified. Use for clean cleanup at end of a congregation's life.

See also: `smite` (state-only reset, agent stays connected), `apostatize` (cathedral cleanup).

smite

`smite(@agent_handles)` — destroy each agent's `%soul` and `%gift` hashes (volatile worker state) without disconnecting. Implemented as a synchronous EVAL that clears the conventional `our %soul` / `our %gift` then returns. Use when you want to wipe worker state without losing the connection.

Returns the count of agents that responded to the smiting.

See also: `excommunicate` (full disconnect), `recant` (slave-side partial erasure).

bestow

`bestow(\%hash, @agent_handles)` — push `%hash` to every named agent's `our %gift` hash. JSON-serialized on master, decoded on agent via `from_json`. The agent's existing `%gift` is replaced (not merged) — for merge semantics the agent's `divine` handler would pull from `%gift` and combine explicitly.

Returns the count of agents that accepted the bestowal.

See also: `smite` (destroy `%gift`), `lick` (read `%soul` back).

enshrine

`enshrine(\%hash, $path)` — write the souls hash to disk as JSON for later `exhume`. Format: `{ "session_id_str": "output_str", ... }`. Returns the number of bytes written.

See also: `exhume`, `martyr`, `resurrect`.

exhume

`exhume($path)` — read an `enshrined` hash back from disk. Returns the hash with the same shape it was written with (string keys, string values). Errors if file is missing or not a valid JSON object.

See also: `enshrine`, `resurrect`.

smother

`smother(\%hash)` — securely zero a local hash. Walks every key and overwrites the value with an empty string, then clears the hash. Returns the count of entries scrubbed. Local-only — no network.

Most useful with a `hash_ref` (mutates caller's hash in place). Pass-by-value hash returns the count but can't actually scrub the caller's namespace.

```
val %souls = harvest "compute()", @workers
use_results(\%souls)
smother(\%souls)    # secure-erase before drop
```

amen

amen(\$id) — release a pending divination OR stop an active chant. Tries divination registry first, then chant registry. For chants, also stops the rescatter at the new-joiner path so late joiners stop receiving the prayer.

Returns 1 if the ID was found (in either registry), 0 if already consumed or never existed.

See also: [pray](#), [chant](#).

anoint

anoint(\$n) — like [congregation\(\\$n\)](#) but does NOT take over as the current controller. Use when you want to keep a primary congregation as the implicit target while spawning a secondary one. Returns array of session-ids; controller handle is registered but callers manage it explicitly.

See also: [congregation](#), [ordain](#).

welcome

welcome(\$n [, \$timeout_ms]) — block until at least \$n agents have joined the current congregation, or \$timeout_ms elapses (default 5s). Returns the actual agent count at return time. Useful for "wait for the slow joiner" patterns after [ordain](#).

```
ordain "renderfarm"
welcome 4, 30_000
var @workers = muster()
```

See also: [congregation](#), [muster](#).

pilgrimage

pilgrimage(\$code, @agent_handles [, \$timeout_ms]) — sync barrier. Scatter \$code to every named agent and block until every dispatched agent has replied, or \$timeout_ms elapses (default 10s). Returns 1 if every agent rendezvoused in time, 0 otherwise.

For a pure rendezvous (no work), pass "1" as the prayer — the agent's reply is the synchronization signal.

See also: [pray](#), [harvest](#).

bow

bow() — drop this stryke process into agent mode and enter the receive loop. Slave-side alias for [agent\(\)](#). Completes the master/slave verb pair (master [prays](#), slave [bows](#) and [divines](#)).

See also: [agent](#), [divine](#), [profess](#).

lick

lick(@agent_handles) — non-destructive snapshot of every worker's %soul. Scatters [to_json\(\%soul\)](#) and gathers; workers retain their %soul intact. Returns hash keyed by stringified session-id; values are each worker's %soul rehydrated from JSON.

For destructive harvest (read then wipe): pair with [smite](#):

```
val %snapshot = lick @workers
smite @workers    # zero workers' %soul and %gift
```

See also: [peruse](#) (deeper walk), [annex](#) (destructive on pray-result), [smite](#).

peruse

`peruse(@agent_handles)` — deep non-destructive `%soul` walk. Tier 3 ships as an alias for `lick`; Tier 5 will replace the body with a `god`-style recursive dump that captures blessed objects, references, and cycles (vs `lick`'s flat JSON serialization).

See also: `lick`.

chant

`chant($code, @agent_handles)` — register an ongoing prayer that fires at every current agent AND every new agent that joins later (via the accept-loop hook). Returns a `chant_id` used by `amen` to stop the rescatter. Fire-and-forget — no gather.

Use for state distribution — a `bestow`-like push to current AND future workers without a master-side polling loop.

```
val $vigil = chant "our %config = (max_depth => 8)", @workers
# new workers can profess() and auto-receive the chant on join
amen $vigil
```

See also: `pray`, `amen`, `bestow`.

profess

`profess($congregation_name)` — slave-side: look up the congregation in the in-process cathedral registry, drop this process into agent mode pointed at the resolved endpoint. Blocks for the lifetime of the agent session.

Tier 4: cathedral is in-process only — `profess` only succeeds when the master ran `ordain($name, ...)` in the same OS process. Tier 5+ promotes the cathedral to a standalone `stryked` daemon for cross-host name resolution.

See also: `ordain`, `cathedral`, `apostatize`, `bow`.

apostatize

`apostatize($congregation_name)` — master-side: unregister the named congregation from the cathedral so future `profess` calls won't find it. Does NOT excommunicate currently-connected agents (use `excommunicate(muster())` for that). Returns 1 if the name was registered, 0 if it wasn't.

See also: `ordain`, `excommunicate`.

cathedral

`cathedral()` — return an array of currently-registered congregation names. Diagnostic helper for inspecting the cathedral registry.

See also: `ordain`, `profess`, `apostatize`.

cloister

`cloister($token)` — turn the current controller's `:cloistered` mode on (when called with a non-empty token) or off (when called with undef/empty). In `:cloistered` mode, agents must send an `AGENT_AUTH` frame carrying `$token` within 500ms of HELLO or they're rejected. Open mode (default) skips the AUTH read entirely.

Slaves opt into AUTH by setting `STRYKE_AGENT_TOKEN` env var before calling `agent()` / `bow()` / `profess()`.

```
ordain "audit_pool"
cloister "s3cret-t0ken"
```

See also: `ordain`, `welcome`.

recant

`recant(@keys)` — partial self-erasure of `%soul`: delete the named keys from the package-global `%soul`. Slave-side primitive (runs in the agent's persistent VM). Returns the count of keys requested.

Tier 4 limitation: returns the intended-delete count; caller wraps with `for var $k (@keys) { delete $main::soul{$k} }` to actually mutate. Tier 5 wires a Rust interpreter handle for atomic delete.

For full erasure, use `smite(@me)` from the master side.

See also: `smite`, `martyr`.

martyr

`martyr($path)` — slave-side: enshrine current `%soul` to `$path` then `exit(0)`. Returns only if enshrine fails; on success the process terminates.

Tier 4 limitation: caller must call `enshrine(\%soul, $path)` first; `martyr` is just the "exit after enshrine" half. Tier 5 fuses them.

```
enshrine \%soul, "/tmp/soul.json"
martyr "/tmp/soul.json"      # never returns
```

See also: `enshrine`, `resurrect`.

resurrect

`resurrect($enshrine_path)` — master-side: read enshrined state, anoint one new agent, bestow the state to it. Returns the new agent's session-id. Composable: just `exhume + anoint + bestow` in one call.

See also: `exhume`, `anoint`, `bestow`, `martyr`.

divine

`divine($handler)` — slave-side: register a closure handler for incoming petitions. The handler is stored in a global slot; the caller wraps the agent's EVAL dispatch to call this handler instead of evaluating arbitrary code.

Tier 4 ships divine as a closure-registration primitive only; full dispatch integration with the agent's EVAL loop is Tier 5 work (requires splitting the agent's frame handler).

See also: `bow`, `pray`.

interrogate

`interrogate($pid_or_handles)` — polymorphic process-state dump.

- `interrogate($pid)` (single scalar): OS-level dump for an OS PID via the `sysinfo` crate. Returns a hash_ref with `pid`, `ppid`, `name`, `cmd`, `exe`, `cwd`, `uid`, `memory_bytes`, `virtual_memory_bytes`, `cpu_usage`, `status`, `start_time`, `run_time_secs`. Works for ANY visible OS process — not just stryke agents. Returns `undef` if the PID isn't visible (terminated, permission denied, or invalid).
- `interrogate(@agent_handles)` (array / multi-arg): agent VM state dump via the scatter-EVAL path. Returns hash keyed by session-id; values are each agent's `{pid, time, soul, gift}`.

Dispatch heuristic: single scalar = OS PID; array/multi-arg = agent handles.

```
val $h = interrogate($$)          # self-introspection
p $h->{name}, " using ", $h->{memory_bytes}, " bytes"
```

See also: `lick`, `peruse`.

Provenance / Lineage

3 topics

mark

`mark($val)` — tag a heap value's `Arc` pointer so subsequent builtin calls that consume it accumulate a lineage record in the provenance ledger. Returns `$val` unchanged, so it composes inline:

```
val $config = mark({ host => "prod", retries => 3 })
```

Value-lineage tracking as a first-class scripting-language builtin is **new ground** — no major scripting language (Perl, Python, Ruby, JavaScript, Lua, PHP) ships this. Closest analogs are research dataflow languages (LIO, Adapton) which surface it as a type-system feature, never as a `mark/provenance` call pair.

Cost model is zero when unused: a process-global `LEDGER_ACTIVE` flag flips to `true` on the first `mark(...)` call. Until then, the post-dispatch hook in every builtin call elides via a single inlined `AtomicBool::Load(Relaxed)` branch.

Marks only apply to **heap values** — hashes, arrays, byte buffers, atomics, generators, pipelines. Immediates (integers, floats, undef) have no stable `Arc` to key on and return `$val` unchanged from `mark` (silently no-op). Wrap a number in a one-element array if you need to track it.

```
val $payload = mark({ user => "jamenk", roles => ["admin"] })
var $jwt      = jwt_encode($payload, $secret)           # tracked op
var $cookie   = base64_encode($jwt)                   # tracked op
p provenance($cookie)
# { origin      => "HASH entries=2",
#   origin_line => 1,
#   ops        => [ { op => "jwt_encode",    args => ["HASH entries=2", ...], line => 2 },
#                 { op => "base64_encode",  args => ["BYTES len=...",   line => 3 } ] }
```

See also: `provenance`, `unmark`, `god` (for runtime introspection without lineage).

provenance

`provenance($val)` — return the lineage hash for a marked value, or `undef` if the value was never tagged via `mark($val)`. Schema:

```
{
  origin      => "HASH entries=3",    # god-style summary at mark time
  origin_line => 7,                   # call-site line of mark($val)
  ops         => [
    { op => "to_array", args => ["HASH entries=3"], line => 8 },
    { op => "pack",    args => ["ARRAY len=3"],   line => 9 },
    ...
  ],
}
```

The `ops` array is append-only — each entry records one builtin call site that consumed a marked value (or a value whose lineage extends from one). Origin and `origin_line` propagate through the entire chain so you can trace the most-transformed value back to where it started.

Use cases:

- Debugging "where did this bad value come from?" — `p provenance($corrupt)` shows the op chain.
- Audit trails — capture the lineage of a credential or signed payload alongside it.
- Test introspection — assert that a value was produced via the expected sequence of ops without coupling to intermediate state.

Ops performed against an unmarked value do NOT add to the ledger. Mark the leaf value at the START of the chain you care about; downstream transformations inherit and extend automatically.

```

val $start = mark([1, 2, 3])
var $sum    = reduce { _ + _1 } @$start
p provenance($sum)      # undef - sum is an INTEGER (immediate), not heap-Arc'd
p provenance($start)    # { origin => "ARRAY len=3", ops => [{ op => "reduce", ... }] }

```

See also: [mark](#), [unmark](#), [god](#).

unmark

[unmark\(\\$val\)](#) — drop [\\$val](#)'s entry from the provenance ledger. Returns [\\$val](#) unchanged. Idempotent — [unmark](#) on an unmarked value is a no-op.

Use to bound ledger growth in long-running scripts that don't want a value's lineage tracked past a known point (e.g. once a value has been serialized to a sink). The ledger is process-global and v1 has no automatic sweep, so explicit [unmark](#) is the only way to reclaim entries until v2 ships Arc-weak-ref-based GC.

```

val $req = mark({ url => $url })
var $body = curl_get($req->{url}) # lineage tracked
# ... downstream code ...
unmark($req)                      # we're done tracing; drop the entry

```

See also: [mark](#), [provenance](#).

Datetime

18 topics

datetime_utc

Return the current UTC date and time as an ISO 8601 string (e.g. `2026-04-15T12:30:00Z`). This is the simplest way to get a portable, unambiguous timestamp for logging, file naming, or serialization. The returned string always ends with `Z` indicating UTC, so there is no timezone ambiguity.

```
val $now = datetime_utc()
p $now # 2026-04-15T12:30:00Z
af("audit.log", "$now: started\n")
var %event = (ts => datetime_utc(), action => "deploy")
wj("event.json", \"%event")
```

datetime_from_epoch

Convert a Unix epoch timestamp (seconds since 1970-01-01 00:00:00 UTC) into an ISO 8601 datetime string. This is useful when you have raw epoch values from `time()`, file modification times, or external APIs and need a human-readable representation. Fractional seconds are truncated.

```
val $ts = 1700000000
p datetime_from_epoch($ts) # 2023-11-14T22:13:20Z
var $born = datetime_from_epoch(0)
p $born # 1970-01-01T00:00:00Z
var @epochs = (1e9, 1.5e9, 2e9)
@epochs |> e { p datetime_from_epoch }
```

datetime_strftime

Format an epoch timestamp using a `strftime`-style format string, giving full control over the output representation. The first argument is the format pattern and the second is the epoch value. Supports all standard specifiers: `%Y` (4-digit year), `%m` (month), `%d` (day), `%H` (hour), `%M` (minute), `%S` (second), `%A` (weekday name), and more.

```
val $t = time()
p datetime_strftime("%Y-%m-%d", $t) # 2026-04-15
p datetime_strftime("%H:%M:%S", $t) # 14:23:07
p datetime_strftime("%A, %B %d", $t) # Wednesday, April 15
var $log_ts = datetime_strftime("%Y%m%d_%H%M%S", $t)
to_file("backup_$log_ts.sql", $data)
```

datetime_now_tz

Return the current date and time in a specified IANA timezone as a formatted string. Pass a timezone name like `America/New_York`, `Europe/London`, or `Asia/Tokyo`. This avoids manual UTC-offset arithmetic and handles daylight saving transitions correctly. Dies if the timezone name is not recognized.

```
p datetime_now_tz("America/New_York") # 2026-04-15 08:30:00 EDT
p datetime_now_tz("Asia/Tokyo") # 2026-04-15 21:30:00 JST
p datetime_now_tz("UTC") # same as datetime_utc
var @offices = ("US/Pacific", "Europe/Berlin", "Asia/Kolkata")
@offices |> e { p "$_: " . datetime_now_tz }
```

datetime_format_tz

Format an epoch timestamp in a specific IANA timezone, combining the capabilities of `datetime_strftime` and `datetime_now_tz`. This lets you render a historical or future timestamp as it would appear on the wall clock in any timezone. Handles DST transitions automatically.

```
val $epoch = 1700000000
p datetime_format_tz($epoch, "America/Chicago")
# 2023-11-14 16:13:20 CST
p datetime_format_tz($epoch, "Europe/London")
# 2023-11-14 22:13:20 GMT
p datetime_format_tz(time(), "Australia/Sydney")
```

datetime_parse_local

Parse a local datetime string (without timezone info) into a Unix epoch timestamp, interpreting it in the system's local timezone. Accepts common formats like `2026-04-15 14:30:00` or `2026-04-15T14:30:00`. Dies if the string cannot be parsed. This is the inverse of formatting with `localtime`.

```
val $epoch = datetime_parse_local("2026-04-15 14:30:00")
p $epoch # Unix timestamp
p datetime_from_epoch($epoch) # back to ISO 8601
var $midnight = datetime_parse_local("2026-01-01 00:00:00")
p time() - $midnight # seconds since New Year
```

datetime_parse_rfc3339

Parse an RFC 3339 / ISO 8601 datetime string (with timezone offset or `Z` suffix) into a Unix epoch timestamp. This is the standard format used by JSON APIs, RSS feeds, and git timestamps. Accepts strings like `2026-04-15T14:30:00Z` or `2026-04-15T14:30:00+05:30`. Dies on malformed input.

```
val $epoch = datetime_parse_rfc3339("2026-04-15T12:00:00Z")
p $epoch
var $with_tz = datetime_parse_rfc3339("2026-04-15T08:00:00-04:00")
p $epoch == $with_tz # 1 (same instant)
# parse API response timestamps
var $created = $response->{created_at}
var $age = time() - datetime_parse_rfc3339($created)
p "created ${age}s ago"
```

datetime_add_seconds

Add (or subtract) a number of seconds to an ISO 8601 datetime string and return the resulting ISO 8601 string. This performs calendar-aware arithmetic, correctly crossing day, month, and year boundaries. Pass a negative number to subtract time. Useful for computing deadlines, expiration times, or time windows.

```
val $now = datetime_utc()
var $later = datetime_add_seconds($now, 3600) # +1 hour
p $later
var $yesterday = datetime_add_seconds($now, -86400) # -1 day
p $yesterday
var $deadline = datetime_add_seconds($now, 7 * 86400) # +1 week
p "due by $deadline"
```

strftime

Parse a datetime string against an arbitrary `strftime`-style format and return UTC epoch seconds. The inverse of `datetime_strftime`. Unlike `datetime_parse_local` / `datetime_parse_rfc3339`, you supply the format explicitly, so it handles non-ISO inputs — web-server log stamps, `date(1)` output, custom report dates. If the format carries no time component the time is `00:00:00`; the instant is read as UTC.

```
p strftime("15/Jun/2026:14:30:00", "%d/%b/%Y:%H:%M:%S") # 1781274600
p strftime("2026-06-15", "%Y-%m-%d") # midnight UTC
# parse nginx access-log timestamps
val $epoch = strftime($stamp, "%d/%b/%Y:%H:%M:%S %z")
p datetime_strftime("%Y-%m-%d", $epoch)
```

time_ago

Render a Unix timestamp relative to the current time: "3 hours ago", "in 2 days" (future), or "just now" (under 10s). `from_now` is an alias. Buckets by second/minute/hour/day/week/month/year with singular/plural agreement.

```
p time_ago(time() - 7200) # 2 hours ago
p time_ago(time() + 86400) # in 1 day
p time_ago(strftime($stamp, "%Y-%m-%d %H:%M:%S"))
```

from_now

Alias for `time_ago` — render a Unix timestamp relative to now ("in 2 days" / "3 hours ago").

elapsed

Return the number of seconds elapsed since the stryke process started, using a monotonic clock that is immune to system clock adjustments. The short alias `el` keeps benchmarking one-liners terse. Returns a floating-point value with sub-millisecond precision. Useful for timing operations, profiling hot loops, or adding relative timestamps to log output.

```
val $t0 = el()
var @sorted = sort @big_array
var $dur = el() - $t0
p "sort took ${dur}s"
# progress logging
1:100 |> e { do_work
    log_info("step $_ at " . el() . "s") }
```

time

Return the current Unix epoch as an integer — the number of seconds since 1970-01-01 00:00:00 UTC. This is the standard wall-clock timestamp used for file times, database records, and interop with external systems. For monotonic timing of code sections, prefer `elapsed/el` instead since `time` can jump if the system clock is adjusted.

```
val $start = time()
sleep(2)
p time() - $start # ~2
var $ts = time()
wj("stamp.json", { created => $ts })
p datetime_from_epoch($ts) # human-readable form
```

times

Return the accumulated CPU times for the process as a four-element list: (`$user`, `$system`, `$child_user`, `$child_system`). User time is CPU spent executing your code; system time is CPU spent in kernel calls on your behalf. Child times cover subprocesses. Values are in seconds (floating point). Useful for profiling whether a script is CPU-bound or I/O-bound.

```
var ($u, $s, $cu, $cs) = times()
p "user=${u}s sys=${s}s"
# after heavy computation
var ($u2, $s2) = times()
p "used " . ($u2 - $u) . "s of CPU"
p "total CPU: " . ($u2 + $s2) . "s"
```

localtime

Convert a Unix epoch timestamp to a nine-element list of broken-down local time components: (*\$sec*, *\$min*, *\$hour*, *\$mday*, *\$mon*, *\$year*, *\$yday*, *\$yday*, *\$isdst*). Follows the Perl convention where *\$mon* is 0-based (January=0) and *\$year* is years since 1900. When called without arguments, uses the current time. Use *gmtime* for the UTC equivalent.

```
val @t = localtime(time())
p "$t[2]:$t[1]:$t[0]"           # HH:MM:SS
var $year = $t[5] + 1900
var $mon  = $t[4] + 1
p "$year-$mon-$t[3]"           # YYYY-M-D
var @days = qw(Sun Mon Tue Wed Thu Fri Sat)
p $days[$t[6]]                # day of week
```

gmtime

Convert a Unix epoch timestamp to a nine-element list of broken-down UTC time components, identical in structure to *localtime* but always in the UTC timezone. The fields are (*\$sec*, *\$min*, *\$hour*, *\$mday*, *\$mon*, *\$year*, *\$yday*, *\$yday*, *\$isdst*) where *\$isdst* is always 0. When called without arguments, uses the current time.

```
val @utc = gmtime(time())
var $year = $utc[5] + 1900
var $mon  = $utc[4] + 1
p sprintf("%04d-%02d-%02dT%02d:%02d:%02dZ",
    $year, $mon, @utc[3,2,1,0])
# compare local vs UTC
var @loc = localtime(time())
p "UTC hour=$utc[2] local hour=$loc[2]"
```

sleep

Pause execution for the specified number of seconds. Accepts both integer and fractional values for sub-second sleeps (e.g. *sleep 0.1* for 100ms). The process yields the CPU during the sleep, so it is safe to use in polling loops without burning cycles. Returns the unslept time (always 0 unless interrupted by a signal).

```
p "waiting..."
sleep(2)
p "done"
# polling loop
while (!-e "done.flag") {
    sleep(0.5)
}
# rate limiting
var @urls = @targets
@urls |> e { fetch
    sleep(0.1) }
```

alarm

Schedule a *SIGALRM* signal to be delivered to the process after the specified number of seconds. Calling *alarm(0)* cancels any pending alarm. Only one alarm can be active at a time — setting a new alarm replaces the previous one. Returns the number of seconds remaining on the previous alarm (or 0 if none was set). Combine with *eval* and *\$\$SIG{ALRM}* to implement timeouts around potentially hanging operations.

```
eval {
    local $$SIG{ALRM} = fn { die "timeout\n" }
    alarm(5)           # 5 second deadline
    val $data = slow_network_call()
    alarm(0)          # cancel on success
}
if ($@ == /timeout/) {
    log_error("operation timed out")
}
```


Math

13 topics

abs

Return the absolute value of a number, stripping any negative sign. Returns the argument unchanged if it is already non-negative. Works on both integers and floating-point numbers.

```
p abs(-42)      # 42
p abs(3.14)     # 3.14
var $diff = abs($x - $y)
p "distance: $diff"
1:5 |> map { _ - 3 } |> map abs |> e p # 2 1 0 1 2
```

int

Truncate a floating-point number toward zero, discarding the fractional part. This is not rounding — `int(1.9)` is `1` and `int(-1.9)` is `-1`. Use `sprintf("%.0f", $n)` or `POSIX::round` for proper rounding. Commonly paired with `rand` to generate random integers.

```
p int(3.7)      # 3
p int(-3.7)     # -3
p int(rand(6)) + 1 # dice roll 1:6
1:10 |> map { _ / 3 } |> map int |> e p
```

sqrt

Return the square root of a non-negative number. Dies if the argument is negative — use `abs` first or check the sign. For the inverse operation, use `squared/sq` or the `**` operator.

```
p sqrt(144)     # 12
p sqrt(2)       # 1.41421356...
var $hyp = sqrt($x**2 + $y**2) # Pythagorean theorem
1:5 |> map sqrt |> e { p sprintf("%.3f", _) }
```

squared

Return the square of a number (`N * N`). This is a stryke convenience function — clearer than writing `$n ** 2` or `$n * $n` in pipelines. The aliases `sq` and `square` are interchangeable.

```
p squared(5)    # 25
p sq(12)        # 144
1:5 |> map sq |> e p # 1 4 9 16 25
var $hyp = sqrt(sq($x) + sq($y)) # Pythagorean theorem
```

cubed

Return the cube of a number (`N * N * N`). This is a stryke convenience function for the common `$n ** 3` operation, useful in math-heavy pipelines. The aliases `cb` and `cube` are interchangeable.

```
p cubed(3)      # 27
p cb(10)        # 1000
1:4 |> map cb |> e p # 1 8 27 64
var $vol = cb($side) # volume of a cube
```

expt

Raise a base to an arbitrary exponent and return the result. This is the function form of the ****** operator. Accepts integer and floating-point exponents, including negative values for reciprocals and fractional values for roots.

```
p expt(2, 10)      # 1024
p expt(27, 1/3)    # 3.0 (cube root)
p expt(10, -2)     # 0.01
1:8 |> map { expt(2, _) } |> e p # 2 4 8 16 32 64 128 256
```

exp

Return Euler's number *e* raised to the given power. **exp(0)** is 1, **exp(1)** is approximately 2.71828. This is the inverse of **log**. Useful for exponential growth/decay calculations, probability distributions, and converting between logarithmic and linear scales.

```
p exp(1)           # 2.71828182845905
p exp(0)           # 1
var $growth = $initial * exp($rate * $time)
1:5 |> map exp |> e { p sprintf("%.4f", _) }
```

log

Return the natural (base-*e*) logarithm of a positive number. Dies if the argument is zero or negative. For base-10 logarithms, divide by **log(10)**. For base-2, divide by **log(2)**. This is the inverse of **exp**.

```
p log(exp(1))      # 1.0
p log(100) / log(10) # 2.0 (log base 10)
var $bits = log($n) / log(2) # log base 2
1:5 |> map log |> e { p sprintf("%.3f", _) }
```

sin

Return the sine of an angle given in radians. The result ranges from -1 to 1. For degrees, convert first: **sin(\$deg * 3.14159265 / 180)**. Use **atan2** to go in the reverse direction.

```
p sin(0)           # 0
p sin(3.14159265 / 2) # 1.0
var @wave = map { sin(_ * 0.1) } 0:62
@wave |> e { p sprintf("%.3f", _) }
```

cos

Return the cosine of an angle given in radians. The result ranges from -1 to 1. **cos(0)** is 1. Like **sin**, convert degrees to radians before calling.

```
p cos(0)           # 1
p cos(3.14159265)   # -1.0
var $x = $radius * cos($theta)
var $y = $radius * sin($theta)
p "($x, $y)"
```

atan2

Return the arctangent of *Y/X* in radians, using the signs of both arguments to determine the correct quadrant. The result ranges from -**pi** to **pi**. This is the standard way to compute angles from Cartesian coordinates and is more robust than **atan(Y/X)** because it handles *X=0* correctly.

```
val $pi = atan2(0, -1) # 3.14159265...
p atan2(1, 1)          # 0.785... (pi/4)
var $angle = atan2($dy, $dx)
p sprintf("%.1f degrees", $angle * 180 / $pi)
```

rand

Return a pseudo-random floating-point number in the range `[0, N)`. If `N` is omitted it defaults to `1`. The result is never exactly equal to `N`. For integer results, combine with `int`. Seed the generator with `srand` for reproducible sequences.

```
p rand()                # e.g. 0.7342...
p int(rand(100))         # random int 0:99
var @deck = 1:52
var @shuffled = sort { rand() <=> rand() } @deck # poor shuffle
var $coin = rand() < 0.5 ? "heads" : "tails"
p $coin
```

srand

Seed the pseudo-random number generator used by `rand`. Calling `srand` with a specific value produces a reproducible sequence, which is useful for testing. Without arguments, Perl seeds from a platform-specific entropy source. You rarely need to call this explicitly — Perl auto-seeds on first use of `rand`.

```
srand(42)                # reproducible sequence
p int(rand(100))          # always the same value
srand(42)
p int(rand(100))          # same value again
srand()                  # re-seed from entropy
```

File System

35 topics

basename

Extract the filename component from a path, stripping all leading directory segments. The short alias `bn` keeps one-liner pipelines terse. If an optional suffix argument is provided, that suffix is also stripped from the result, which is handy for removing extensions.

```
p basename("/usr/local/bin/stryke")      # stryke
p bn("/tmp/data.csv", ".csv")           # data
"/etc/nginx/nginx.conf" |> bn |> p      # nginx.conf
```

dirname

Return the directory portion of a path, stripping the final filename component. The short alias `dn` mirrors `bn`. This is a pure string operation — it does not touch the filesystem, so it works on paths that do not exist yet. Useful for deriving output directories from input file paths.

```
p dirname("/usr/local/bin/stryke")      # /usr/local/bin
p dn("/tmp/data.csv")                   # /tmp
var $dir = dn($0)                       # directory of current script
```

fileparse

Split a path into its three logical components: the filename, the directory prefix, and a suffix that matches one of the supplied patterns. This mirrors Perl's `File::BaseName::fileparse` and is the most flexible path decomposition available. When no suffix patterns are given the suffix is empty.

```
var ($name, $dir, $sfx) = fileparse("/home/user/report.txt", qr/\..txt/)
p "$dir | $name | $sfx" # /home/user/ | report | .txt
var ($n, $d) = fileparse("./lib/Foo/Bar.pm")
p "$d$n" # ./lib/Foo/Bar.pm
```

realpath

Resolve a path to its absolute canonical form by following all symbolic links and eliminating `.` and `..` segments. Unlike `canonpath`, this hits the filesystem and will die if any component does not exist. The short alias `rp` is convenient in pipelines. Use this when you need a guaranteed unique path for deduplication or comparison.

```
p realpath(".") # /home/user/project
p rp("../sibling") # /home/user/sibling
var $canon = rp($0) # absolute path of current script
"." |> rp |> p
```

canonpath

Clean up a file path by collapsing redundant separators, resolving `.` and `..` segments, and normalizing trailing slashes — all without touching the filesystem. Unlike `realpath`, this is a purely lexical operation so it works on paths that do not exist. Use it to normalize user-supplied paths before comparison or storage.

```
p canonpath("/usr/./local/./local/bin/") # /usr/local/bin
p canonpath("a/b/./c") # a/c
var $clean = canonpath($ENV{HOME} . "/./docs/./docs")
p $clean
```

getcwd

Return the current working directory as an absolute path string. This calls the underlying OS `getcwd` function, so it always reflects the real directory even if the process changed directories via `chdir`. The alias `pwd` matches the familiar shell command. Often used to save and restore directory context around `chdir` calls.

```
val $orig = pwd()
chdir("/tmp")
p pwd()      # /tmp
chdir($orig)
p getcwd()   # back to original
```

cd

Set the interpreter working directory used to resolve **relative** paths for Stryke builtins (`slurp`, `spurt`, `open`, `glob`, `stat`, `rename`, `ls`, `-e`, ...). Does **not** call the OS `chdir` by itself — `getcwd/pwd` stay unchanged until you call `chdir`. A successful `chdir` **does** refresh this logical base from the new OS cwd. Relative `cd` targets are resolved against the current logical directory first. With no arguments, uses `$HOME / %USERPROFILE%` when set. Returns `1` on success, `0` on failure and sets `$!`.

```
cd("/tmp")
spurt("x.txt", "hi")      # writes /tmp/x.txt
```

which

Search the `PATH` environment variable for the first executable matching the given command name and return its absolute path, or `undef` if not found. This is the programmatic equivalent of the shell `which` command. Useful for checking tool availability before calling `system` or `exec`.

```
val $gcc = which("gcc") // die "gcc not found"
p $gcc                  # /usr/bin/gcc
if (which("rg")) {
    system("rg pattern file")
} else {
    system("grep pattern file")
}
```

glob

Expand a zsh-style glob pattern against the filesystem and return a list of matching paths. World-first: stryke supports **every** zsh glob qualifier from `man zshexpn _Filename Generation > Glob Qualifiers` (no other scripting language does this). Backed by `zsh::glob` from `zshrs` — single source of truth, zero stryke-side reimplementations. Patterns: `*`, `?`, `[abc]`, `{a,b}`, `**` recursive, plus the `(...)` qualifier suffix.

Qualifier reference (full set)

File type: `(/)` dir · `(.)` regular file · `(@)` symlink · `(=)` socket · `(p)` FIFO · `(%b)` block dev · `(%c)` char dev · `(%)` any device · `(*)` executable file.

Permission: `(r)` `(w)` `(x)` EUID `r/w/x` · `(R)` `(W)` `(X)` world · `(A)` `(I)` `(E)` group · `(s)` setuid · `(S)` setgid · `(t)` sticky · `(f<bits>)` exact mode-bit match (e.g. `(f644)`).

Ownership / device: `(U)` owned by EUID · `(G)` owned by EGID · `(u<N>)` uid `N` · `(g<N>)` gid `N` · `(d<N>)` match by device number.

Numeric (each accepts `+N` greater, `-N` less, `N` equal): `(L[unit]±N)` size — `p/k/m/g/t` units, default bytes · `(l±N)` link count · `(a[unit]±N)` atime · `(m[unit]±N)` mtime · `(c[unit]±N)` ctime — time units `s/m/h/d/w/M`.

Sort: `(on)` `(oL)` `(oI)` `(oa)` `(om)` `(oc)` `(od)` ascending; capital `0*` descending; `(oN)` no sort.

Selection: `([N])` Nth match · `([N,M])` slice. Combine with sort: `(om[1])` newest single match.

Flags: `(N)` NULL_GLOB — empty list on no match · `(D)` include dotfiles · `(F)` non-empty directory · `(M)` mark-dirs (append `/`) · `(T)` list-types (append type char) · `(n)` numeric sort.

Eval / functions: `(e 'CMD')` shell-eval predicate · `(+func)` function-as-test (predicate sub).

Join words: (P...) prefix words · (Q...) postfix words around each match.

Colon modifiers: (:s/PAT/REPL/) sed substitution on each path · (:e) extension · (:r) strip extension · (:t) tail · (:h) head · all of zsh's history-style modifiers.

Combinators: prefix ^ negates · prefix - toggles symlink-following · , separates OR alternatives · chained qualifiers AND.

```
val @scripts = glob("*.pl")
var @all_rs = glob("src/**/*_rs")
var @dirs = glob("**(/)")           # directories only, recursive
var @big = glob("**(L+1024)")       # files larger than 1024 bytes
var @recent = glob("**(om[1])")    # most recently modified, take 1
var @safe = glob("missing*(N)")     # null-glob, never errors
var @mode = glob("**(f644)")        # files with exactly mode 644
var @stems = glob("*.txt(:r)")      # all .txt files, extension stripped
```

glob_match

Test whether a string matches a shell-style glob pattern. Returns true (1) on match, false (empty string) otherwise. Supports *, ?, [abc], and {a,b}. This is a pure string match — it does not read the filesystem, so it works for filtering lists of paths you already have. Note: zsh glob qualifiers like (/) or (.) are stat-based filters that only apply at filesystem-expansion time (glob/glob_par/slurp); glob_match ignores any trailing qualifier.

```
p glob_match("*.pl", "script.pl")      # 1
p glob_match("*.pl", "script.py")      # (empty)
var @perl = grep { glob_match("*.{pl,pm}", _) } @files
@perl |> e p
```

copy

Copy a file from a source path to a destination path. The destination can be a file path or a directory (in which case the source filename is preserved). Dies on failure. This is the programmatic equivalent of cp and avoids shelling out. Metadata such as permissions is preserved where possible.

```
copy("src/config.yaml", "/tmp/config.yaml")
copy("report.pdf", "/backup/")
var $tmp = tf()
copy($0, $tmp) # back up the current script
p slurp($tmp)
```

move

Move or rename a file from source to destination. If the source and destination are on the same filesystem this is an atomic rename; otherwise it falls back to copy-then-delete. Dies on failure. The short alias mv mirrors the shell command.

```
move("draft.txt", "final.txt")          # rename in place
mv("output.csv", "/archive/output.csv") # move across dirs
var $tmp = tf()
spurt($tmp, "data")
mv($tmp, "data.txt")
```

rename

Rename a file or directory from an old name to a new name. This is an atomic operation on the same filesystem. If the destination already exists it is silently replaced. Dies on failure. Unlike move/mv, this does not fall back to copy-then-delete across filesystems.

```
rename("draft.md", "final.md")
rename("output", "output_v2")           # works on dirs too
var $bak = "config.yaml.bak"
rename("config.yaml", $bak)
spurt("config.yaml", $new_config)
```

unlink

Delete one or more files from the filesystem. Returns the number of files successfully removed. Does not remove directories — use `rmdir` for those. Dies on permission errors. Accepts a list of paths, making it convenient for batch cleanup.

```
unlink("temp.log")
var $n = unlink("a.tmp", "b.tmp", "c.tmp")
p "removed $n files"
var @old = glob("*.bak")
unlink(@old)
```

mkdir

Create a directory at the given path. An optional second argument specifies the permission mode as an octal number (default `0777`, modified by the current `umask`). Dies if the directory cannot be created. Only creates one level — use `make_path` or shell out for recursive creation.

```
mkdir("output")
mkdir("/tmp/secure", 0700)
var $dir = tdr() . "/sub"
mkdir($dir)
p -d $dir # 1
```

rmdir

Remove an empty directory. Dies if the directory does not exist, is not empty, or cannot be removed due to permissions. This only removes a single directory — it will not recursively delete contents. Remove files with `unlink` first, then call `rmdir`.

```
mkdir("scratch")
rmdir("scratch")
p -d "scratch" # (empty, dir is gone)
unlink("tmp/file.txt")
rmdir("tmp")
```

chmod

Change the permission bits of one or more files. The mode is specified as an octal number. Returns the number of files successfully changed. Does not follow symlinks on some platforms. Use `stat` to read the current mode before modifying.

```
chmod(0755, "script.pl")
chmod(0644, "config.yaml", "data.json")
var $n = chmod(0600, glob("*.key"))
p "secured $n key files"
```

chown

Change the owner and group of one or more files, specified as numeric UID and GID. Pass `-1` for either to leave it unchanged. Returns the number of files successfully changed. Typically requires root privileges.

```
chown(1000, 1000, "app.log")
chown(-1, 100, "shared.txt") # change group only
var $uid = (getpwnam("deploy"))[2]
chown($uid, -1, "release.tar")
```

chdir

Change the current working directory of the process. On success, also refreshes the interpreter logical directory used for relative Stryke builtins (same as after a successful `cd` to that path). Returns `0/1` (does not throw). Pair with `getcwd/pwd` to save and restore the original directory.

```
val $orig = pwd()
chdir("/tmp")
spurt("scratch.txt", "hello")
chdir($orig) # return to original
```

stat

Return a 13-element list of file status information for a path or filehandle: (*\$dev*, *\$ino*, *\$mode*, *\$nlink*, *\$uid*, *\$gid*, *\$rdev*, *\$size*, *\$atime*, *\$mtime*, *\$ctime*, *\$blksize*, *\$blocks*). This calls the POSIX *stat* system call. Use it to check file size, modification time, permissions, and other metadata without reading the file.

```
val @st = stat("data.bin")
p "size: $st[7] bytes"
p "modified: " . datetime_from_epoch($st[9])
var ($mode) = (stat($0))[2]
p sprintf("perms: %04o", $mode & 07777)
```

link

Create a hard link — a new directory entry pointing to the same underlying inode as the original file. Both names are indistinguishable and share the same data; removing one does not affect the other. Hard links cannot cross filesystem boundaries and typically cannot link directories.

```
link("data.csv", "data_backup.csv")
var @st1 = stat("data.csv")
var @st2 = stat("data_backup.csv")
p $st1[1] == $st2[1] # 1 — same inode
```

symlink

Create a symbolic (soft) link that points to a target path. Unlike hard links, symlinks can cross filesystems and can point to directories. The link stores the target as a string, so it can dangle if the target is later removed. Use *readlink* to inspect where a symlink points.

```
symlink("/usr/local/bin/stryke", "pe_link")
p readlink("pe_link") # /usr/local/bin/stryke
p -l "pe_link" # 1 (is a symlink)
symlink("../lib", "lib_link") # relative target
```

readlink

Return the target path that a symbolic link points to, without following the link further. Returns *undef* if the path is not a symlink. This is useful for inspecting symlink chains, verifying link targets, or resolving one level of indirection at a time.

```
symlink("real.conf", "link.conf")
var $target = readlink("link.conf")
p $target # real.conf
if (defined readlink($path)) {
    p "$path is a symlink"
}
```

utime

Set the access time and modification time of one or more files. Times are specified as epoch seconds. Pass *undef* for either time to set it to the current time. Returns the number of files successfully updated. Useful for cache invalidation, build systems, or preserving timestamps after transformations.

```
utime(time(), time(), "output.txt") # touch to now
var $epoch = 1700000000
utime($epoch, $epoch, @files) # backdate files
utime(undef, undef, "marker.flag") # equivalent of touch
```


umask

Get or set the file creation mask, which controls the default permissions for newly created files and directories. When called with an argument, sets the new mask and returns the previous one. When called without arguments, returns the current mask. The mask is subtracted from the requested permissions in `mkdir`, `open`, etc.

```
val $old = umask(0077)      # restrict: owner-only
mkdir("private")            # created with 0700
umask($old)                 # restore previous mask
p sprintf("%04o", umask())   # print current mask
```

uname

Return system identification as a five-element list: (`$sysname`, `$nodename`, `$release`, `$version`, `$machine`). This calls the POSIX `uname` system call and is useful for platform-specific logic, logging system info, or generating diagnostic reports without shelling out.

```
var ($sys, $node, $rel, $ver, $arch) = uname()
p "$sys $rel ($arch)"        # Linux 6.1.0 (x86_64)
if ($sys eq "Darwin") {
  p "running on macOS"
}
log_info("host: $node, kernel: $rel")
```

gethostname

Return the hostname of the current machine as a string. This calls the POSIX `gethostname` system call. The short alias `hn` is useful in log prefixes, temp-file naming, or distributed-system identifiers where you need to tag output by machine.

```
p gethostname()              # myhost.local
var $log_prefix = hn() . ":" . $$ # myhost.local:12345
log_info("running on " . hn())
```

opendir

Open a directory handle for reading its entries. Returns a directory handle that can be passed to `readdir`, `seekdir`, `telldir`, `rewinddir`, and `closedir`. Dies if the directory does not exist or cannot be opened. For most use cases `glob` or `readdir` with a path is simpler.

```
opendir(val $dh, "/tmp") or die "cannot open: $!"
var @entries = readdir($dh)
closedir($dh)
@entries |> grep { _ !~ /^\.\/ } |> e p # skip dotfiles
```

readdir

Read entries from a directory handle opened with `opendir`. In list context, returns all remaining entries. In scalar context, returns the next single entry or `undef` when exhausted. Entries include `.` and `..` so you typically filter them out.

```
opendir(val $dh, ".") or die $!
while (val $entry = readdir($dh)) {
  next if $entry =~ /^\.\/
  p $entry
}
closedir($dh)
```

closedir

Close a directory handle previously opened with `opendir`, releasing the underlying OS resource. While directory handles are closed automatically when they go out of scope, explicit `closedir` is good practice in long-running programs or loops that open many directories.

```
opendir(val $dh, "/var/log") or die $!
var @logs = readdir($dh)
closedir($dh)
@logs |> grep { /\.log$/ } |> e p
```

seekdir

Set the current position in a directory handle to a location previously obtained from `telldir`. This allows you to revisit directory entries without closing and reopening the handle. Rarely needed in practice, but useful for multi-pass directory scanning.

```
opendir(val $dh, ".") or die $!
var $pos = telldir($dh)
var @first_pass = readdir($dh)
seekdir($dh, $pos) # rewind to saved position
var @second_pass = readdir($dh)
closedir($dh)
```

telldir

Return the current read position within a directory handle as an opaque integer. The returned value can be passed to `seekdir` to return to that position later. This is the directory-handle equivalent of `tell` for file handles.

```
opendir(val $dh, "/tmp") or die $!
readdir($dh) # skip .
readdir($dh) # skip ..
var $pos = telldir($dh) # save position after . and ..
var @real = readdir($dh)
seekdir($dh, $pos) # go back
```

rewinddir

Reset a directory handle back to the beginning so that the next `readdir` returns the first entry again. This is equivalent to closing and reopening the directory but more efficient. Useful when you need to iterate a directory multiple times.

```
opendir(val $dh, "src") or die $!
var $count = scalar readdir($dh)
rewinddir($dh)
var @entries = readdir($dh)
closedir($dh)
p "$count entries"
```

du

`du` (alias `dir_size`) — compute the total size of a directory tree in bytes, recursively walking all files.

```
val $bytes = du("/usr/local")
p "$bytes bytes"
```

du_tree

`du_tree` (alias `dir_sizes`) — return directory sizes as an array of hashrefs `{path, size}`, sorted descending by size. Each entry is an immediate child directory.

```
val @dirs = du_tree("/usr/local")
for (@dirs) { p "->{path}: _->{size}" }
```

Process

24 topics

shell_quote

Quote a string so it survives unchanged through a POSIX shell (Python `shlex.quote`). Safe values pass through bare; anything with spaces or metacharacters is single-quoted with embedded `'` escaped as `'\''`. Empty string becomes `'`. Use it to build shell command lines safely from untrusted data.

```
p shell_quote("plain")           # plain
p shell_quote("a b")             # 'a b'
p shell_quote("it's")            # 'it'\''s'
val $cmd = "grep " . shell_quote($pat) . " file.txt"
```

shell_split

Split a command line into words honoring single quotes, double quotes, and backslash escapes (POSIX `shlex.split`); `shellwords` is an alias. Returns an array. Dies on an unterminated quote. Performs no expansion — `$VAR`, globs, and backticks are literal.

```
p shell_split(q{cmd -x 'a b' "c d"}) # (cmd, -x, a b, c d)
val @argv = shell_split($line)
```

shellwords

Alias for `shell_split` — POSIX word-split a command line into an array, honoring quotes and backslash escapes.

system

Execute a command in a subshell and wait for it to complete, returning the exit status. A return value of `0` means success; non-zero indicates failure. The exit status is encoded as `$? >> 8` for the actual exit code. Use backticks or `capture` if you need the command's output.

```
val $rc = system("make", "test")
p "exit code: " . ($rc >> 8)
system("cp data.csv /backup/") == 0 or die "copy failed"
if (system("which rg >/dev/null 2>&1") == 0) {
    p "ripgrep is installed"
}
```

exec

Replace the current process image entirely with a new command. This never returns on success — the new program takes over. If `exec` fails (command not found, permission denied) it returns false and execution continues. Use `system` instead if you want to run a command and keep the current process alive.

```
exec("ls", "-la", "/tmp") or die "exec failed: $!"
# code here only runs if exec fails

# common fork+exec pattern
if (fork() == 0) {
    exec("worker", "--daemon")
}
```

fork

Fork the current process, creating a child that is an exact copy of the parent. Returns the child's PID to the parent process and `0` to the child, allowing each side to branch. Returns `undef` on failure. Always pair with `wait/waitpid` to reap the child and avoid zombies.

```
val $pid = fork()
if ($pid == 0) {
    p "child $$"
    exit(0)
}
p "parent $$, child is $pid"
waitpid($pid, 0)
```

wait

Wait for any child process to terminate and return its PID. The exit status is stored in `$?`. If there are no child processes, returns `-1`. This is the simplest reaping function — use `waitpid` when you need to wait for a specific child or use non-blocking flags.

```
for (1:3) {
    fork() or do { sleep(1)
    exit(0) }
}
while ((val $pid = wait()) != -1) {
    p "child $pid exited with " . ($? >> 8)
}
```

waitpid

Wait for a specific child process identified by PID to change state. The flags argument controls behavior — use `0` for blocking wait, or `WNOHANG` for non-blocking (returns `0` if child is still running). Returns the PID on success, `-1` if the child does not exist. Exit status is in `$?`.

```
val $pid = fork() // die "fork: $!"
if ($pid == 0) { sleep(2)
    exit(42) }
waitpid($pid, 0)
p "child exited: " . ($? >> 8) # 42

# non-blocking poll
while (waitpid($pid, WNOHANG) == 0) {
    p "still running..."
    sleep(1)
}
```

kill

Send a signal to one or more processes by PID. The signal can be specified as a number (`9`) or a name ("`TERM`"). Sending signal `0` tests whether the process exists without actually sending anything. Returns the number of processes successfully signaled.

```
kill("TERM", $child_pid)
kill(9, @worker_pids) # SIGKILL
if (kill(0, $pid)) {
    p "process $pid is alive"
}
var $n = kill("HUP", @daemons)
p "signaled $n processes"
```

exit

Terminate the program immediately with the given exit status code. An exit code of `0` conventionally means success; any non-zero value indicates an error. `END` blocks and object destructors are still run. Use `POSIX::_exit` to skip cleanup entirely.

```

exit(0)           # success
exit(1) if $error  # failure

# conditional exit in a pipeline
var $ok = system("make test")
exit($ok >> 8) if $ok

```

getlogin

Return the login name of the user who owns the current terminal session. This reads from the system's utmp/utmpx database and may return `undef` for processes without a controlling terminal (cron jobs, daemons). For a more reliable alternative, use `getpwuid($<)` which looks up the effective UID.

```

val $user = getlogin() // (getpwuid($<))[0]
p "running as $user"
log_info("session started by " . getlogin())

```

getpwnam

Look up user account information by username string. Returns the same 9-element list as `getpwuid`: (`$name`, `$passwd`, `$uid`, `$gid`, `$quota`, `$comment`, `$gcos`, `$dir`, `$shell`). Returns empty list if the user does not exist. Useful for resolving a username to a UID before calling `chown`.

```

val @info = getpwnam("deploy")
p "uid=${info[2]} home=${info[7]}"
var $uid = (getpwnam("www-data"))[2]
chown($uid, -1, "public/index.html")

```

getpwuid

Look up user account information by numeric UID. Returns (`$name`, `$passwd`, `$uid`, `$gid`, `$quota`, `$comment`, `$gcos`, `$dir`, `$shell`) on success, or empty list if the UID does not exist. This is the reliable way to map a UID to a username and home directory.

```

var ($name, undef, undef, undef, undef, undef, undef, $home) = getpwuid($<)
p "user: $name, home: $home"
var $root_shell = (getpwuid(0))[8]
p $root_shell # /bin/bash or /bin/zsh

```

getpwent

Read the next entry from the system password database, iterating through all user accounts. Returns (`$name`, `$passwd`, `$uid`, `$gid`, `$quota`, `$comment`, `$gcos`, `$dir`, `$shell`) for each user, or empty list when exhausted. Call `setpwent` to rewind and `endpwent` to close.

```

while (val @pw = getpwent()) {
  p "$pw[0]: uid=$pw[2] home=$pw[7]"
}
endpwent()

```

getgrgid

Look up group information by numeric GID. Returns (`$name`, `$passwd`, `$gid`, `$members`) where `members` is a space-separated string of usernames belonging to the group. Returns empty list if the GID does not exist.

```

var ($name, undef, undef, $members) = getgrgid(0)
p "group $name: $members"
var $gname = (getgrgid((stat("file.txt"))[5]))[0]
p "file group: $gname"

```

getgrnam

Look up group information by group name string. Returns (*\$name*, *\$passwd*, *\$gid*, *\$members*). Useful for resolving a group name to a GID before calling `chown`, or for checking group membership.

```
var ($name, undef, $gid, $members) = getgrnam("staff")
p "gid=$gid members=$members"
chown(-1, $gid, "shared_dir")
if ((getgrnam("admin"))[3] =~ /\b$user\b/) {
    p "$user is an admin"
}
```

getgrent

Read the next entry from the system group database, iterating through all groups. Returns (*\$name*, *\$passwd*, *\$gid*, *\$members*) for each group, or empty list when exhausted. The members field is a space-separated string of usernames. Call `endgrent` when done.

```
while (val @gr = getgrent()) {
    p "$gr[0]: gid=$gr[2] members=$gr[3]"
}
endgrent()
```

getppid

Return the process ID of the parent process. This is useful for detecting whether the process has been orphaned (parent PID becomes 1 on Unix when the original parent exits), or for logging the process hierarchy. Always returns a valid PID.

```
p "my pid: $$, parent: " . getppid()
if (getppid() == 1) {
    log_warn("parent process has exited, we are orphaned")
}
```

getpgrip

Return the process group ID of the current process (or of the specified PID). Processes in the same group receive signals together — for example, Ctrl-C sends SIGINT to the entire foreground process group. Use `setpgrip` to move a process into a different group.

```
p "process group: " . getpgrip()
var $pg = getpgrip($$)
p "$pg == $$ ? "group leader" : "group member"
```

setpgrip

Set the process group ID of a process. Call `setpgrip(0, 0)` to make the current process a new group leader, which is useful for daemonization or isolating a subprocess from the terminal's signal group. Takes (*PID*, *PGID*) — use 0 for the current process.

```
setpgrip(0, 0) # become process group leader
if (fork() == 0) {
    setpgrip(0, 0) # child gets its own group
    exec("worker")
}
```

getpriority

Get the scheduling priority (nice value) of a process, process group, or user. The `which` argument selects the target type: `PRIO_PROCESS`, `PRIO_PGRP`, or `PRIO_USER`. Lower values mean higher priority. The default nice value is 0; range is typically -20 to 19.

```
val $nice = getpriority(0, $$) # PRIO_PROCESS, current PID
p "nice value: $nice"
var $user_prio = getpriority(2, $<) # PRIO_USER, current user
p "user priority: $user_prio"
```

setpriority

Set the scheduling priority (nice value) of a process, process group, or user. Lowering the nice value (higher priority) typically requires root privileges. Raising it (lower priority) is always allowed. Use this to deprioritize background batch work or boost latency-sensitive tasks.

```
setpriority(0, $$, 10) # lower priority for batch work
if (fork() == 0) {
  setpriority(0, 0, 19) # lowest priority for child
  exec("batch-job")
}
```

syscall

`syscall(NUM [, ARG ...])` — raw `syscall(2)` entry point. Calls the kernel with the platform-specific syscall number `NUM` and up to six arguments. Integer args pass as `c_long`; string args pass as pointer-to-bytes (so `read`-style scratch buffers work). Returns the kernel return value (or `-1` on error).

Syscall numbers are **not normalized across platforms** — pass the exact number for the host kernel (see `<sys/syscall.h>` for the `SYS_*` constants). Wrong numbers can crash the process; this is an escape hatch, not a portability layer. Use the wrapper builtins (`getpid`, `kill`, `fork`, `system`, file ops) for portable code.

```
# macOS aarch64: SYS_getpid = 20
var $pid = syscall(20)
p $pid == $$ ? "matches \$\$" : "mismatch"

# Linux x86_64: SYS_getpid = 39
# val $pid = syscall(39)

# Linux: SYS_sync = 162 - flush filesystem caches.
# syscall(162)
```

On non-Unix platforms (Windows) `syscall` returns `-1` and is otherwise inert.

See also: `getpid`, `getppid`, `kill`, `fork`, `system`, `$!`.

process_list

`process_list` (aliases `ps`, `procs`) — list running processes as an array of hashrefs with `{pid, name, uid}` keys.

```
var @procs = ps()
for (@procs) { p "_->{pid} _->{name}" }
```

Pack / Binary

4 topics

pack

Convert a list of values into a binary string according to a template. Each template character specifies how one value is encoded: **N** for 32-bit big-endian unsigned, **n** for 16-bit, **a** for raw bytes, **Z** for null-terminated string, etc. This is essential for constructing binary protocols, file formats, and **sockaddr** structures.

```
val $bin = pack("NnA4", 0xDEADBEEF, 8080, "test")
p length($bin)           # 10 bytes
var $header = pack("A8 N N", "MAGIC01\0", 1, 42)
spurt("data.bin", $header)
```

unpack

Decode a binary string into a list of values according to a template, performing the inverse of **pack**. The template characters must match how the data was packed. Use this for parsing binary file formats, network protocol headers, or any structured binary data.

```
var ($magic, $version, $count) = unpack("A8 N N", $header)
p "v$version, $count records"
var ($port, $addr) = unpack("n a4", $sockaddr)
p inet_ntoa($addr) . ":$port"
```

unpack_first

unpack_first(FMT, STR) — decode the binary string and return the **first** decoded element (or **undef** if the format yields nothing). The stryke spelling of Perl's **scalar unpack(FMT, STR)** for **--no-interop** mode where **scalar** is rejected at parse time. Aliases: **unpack1**, **up1**. Equivalent to **head(unpack(FMT, STR))** but spelled as a single verb.

```
val $first_byte = unpack_first("C", $bytes)
var $port       = unpack1("n", $sockaddr)
var $magic_word = up1("N", $header)
```

vec

Treat a string as a bit vector and get or set individual elements at a specified bit width. The first argument is the string, the second is the element offset, and the third is the bit width (1, 2, 4, 8, 16, or 32). As an lvalue, **vec** modifies the string in place. Useful for compact boolean arrays and bitmap manipulation.

```
val $bits = ""
vec($bits, 0, 1) = 1 # set bit 0
vec($bits, 7, 1) = 1 # set bit 7
p vec($bits, 0, 1)   # 1
p vec($bits, 3, 1)   # 0
p unpack("B8", $bits) # 10000001
```


Logging

7 topics

log_info

Log a message at INFO level to stderr with a timestamp prefix. INFO is the default visible level and is appropriate for normal operational messages — startup notices, progress milestones, summary statistics. Messages are suppressed if the current log level is set higher than INFO.

```
log_info("server started on port $port")
var @rows = rl("data.csv")
log_info("loaded " . scalar(@rows) . " rows")
1:5 |> e { log_info("processing item $_") }
```

log_warn

Log a message at WARN level to stderr. Warnings indicate unexpected but recoverable situations — missing optional config, deprecated usage, slow operations. WARN messages appear at the default log level and are visually distinct from INFO in structured log output.

```
log_warn("config file not found, using defaults")
log_warn("query took ${elapsed}s, exceeds threshold")
unless (-e $path) {
  log_warn("$path missing, skipping")
}
```

log_error

Log a message at ERROR level to stderr. Use this for failures that prevent an operation from completing but do not necessarily terminate the program — failed network requests, invalid input, permission errors. ERROR is always visible regardless of log level.

```
log_error("failed to connect to $host: $!")
eval { rj("bad.json") }
log_error("parse failed: $@" ) if $@
log_error("missing required field 'name'")
```

log_debug

Log a message at DEBUG level to stderr. Debug messages are hidden by default and only appear when the log level is lowered to DEBUG or TRACE via `log_level`. Use for detailed internal state that helps during development — variable dumps, branch decisions, intermediate values.

```
log_level("debug")
log_debug("cache key: $key")
var $result = compute($x)
log_debug("compute($x) => $result")
@items |> e { log_debug("item: $_") }
```

log_trace

Log a message at TRACE level to stderr. This is the most verbose level, producing very fine-grained output — loop iterations, function entry/exit, raw payloads. Only visible when `log_level("trace")` is set. Use sparingly in production code; primarily for deep debugging sessions.

```
log_level("trace")
fn process($x) {
  log_trace("entering process($x)")
  val $r = $x * 2
  log_trace("leaving process => $r")
  $r
}
1:3 |> map process |> e p
```

log_json

Emit a structured JSON log line to stderr containing the message plus any additional key-value metadata. This is designed for machine-parseable logging pipelines — centralized log collectors, JSON-based monitoring, or [jq](#)-friendly output. Each call emits exactly one JSON object per line.

```
log_json("request", method => "GET", path => "/api")
log_json("metric", name => "latency_ms", value => 42)
log_json("error", msg => $@, file => $0)$1
```

Note: all values are serialized as JSON strings.

log_level

Get or set the current minimum log level. When called with no arguments, returns the current level as a string. When called with a level name, sets it for all subsequent log calls. Valid levels from most to least verbose: [trace](#), [debug](#), [info](#), [warn](#), [error](#). The default level is [info](#).

```
p log_level()           # info
log_level("debug")      # enable debug output
log_debug("now visible")
log_level("error")      # suppress everything below error
log_info("hidden")      # not printed
```

Charts (SVG)

19 topics

scatter_svg

`scatter_svg` (alias `scatter_plot`) — generate an SVG scatter plot. Args: `xs`, `ys` [, `title`]. Dark theme, auto-scaled axes.

```
scatter_svg([1,2,3,4], [1,4,9,16], "Squares") |> to_file("scatter.svg")
```

line_svg

`line_svg` (alias `line_plot`) — generate an SVG line plot. Args: `xs`, `ys` [, `title`].

```
val @x = map { _ * 0.1 } 0:100
var @y = map { sin(_) } @x
line_svg(@x, @y, "Sine") |> to_file("sine.svg")
```

plot_svg

`plot_svg` — SVG line plot with auto X axis (0..n-1). Args: `ys` [, `title`].

```
plot_svg([map { _ ** 2 } 0:50], "Parabola") |> to_file("plot.svg")
```

hist_svg

`hist_svg` (alias `histogram_svg`) — SVG histogram with auto-binning (sqrt rule). Args: `data` [, `bins`, `title`].

```
rnorm(1000) |> hist_svg(30, "Normal") |> to_file("hist.svg")
```

boxplot_svg

`boxplot_svg` (alias `box_plot`) — SVG box-and-whisker plot with IQR whiskers and outlier detection. Args: `groups` [, `title`]. Groups is array of arrays.

```
boxplot_svg([[1,2,3,4,5], [3,4,5,6,20]], "Compare") |> to_file("box.svg")
```

bar_svg

`bar_svg` (alias `barchart_svg`) — SVG bar chart with labeled bars and value annotations. Args: `labels`, `values` [, `title`].

```
bar_svg(["Rust", "Go", "C++"], [45,30,25], "Languages") |> to_file("bar.svg")
```

pie_svg

`pie_svg` (alias `pie_chart`) — SVG pie chart with percentage labels. Args: `labels`, `values` [, `title`].

```
pie_svg(["A", "B", "C"], [50,30,20]) |> to_file("pie.svg")
```

heatmap_svg

`heatmap_svg` (alias `heatmap`) — SVG heatmap with blue-cyan-yellow-red colormap. Args: `matrix` [, `title`].

```
heatmap_svg(cor_mat($data), "Correlation") |> to_file("heat.svg")
```

donut_svg

`donut_svg` (alias `donut`) — SVG donut chart with percentage labels. Accepts labels+values or a hashref.

```
donut_svg(["A","B","C"], [50,30,20], "Share") |> to_file("donut.svg")  
{Rust => 45, Go => 30} |> donut("Languages") |> to_file("donut.svg")
```

area_svg

`area_svg` (alias `area_chart`) — SVG filled area chart. Args: xs, ys [, title].

```
val @x = 0:20  
var @y = map { sin(_ * 0.3) } @x  
area_svg(@x, @y, "Wave") |> to_file("area.svg")
```

hbar_svg

`hbar_svg` (alias `hbar`) — SVG horizontal bar chart. Accepts labels+values or a hashref.

```
hbar_svg(["Rust","Go","C++"], [45,30,25], "Speed") |> to_file("hbar.svg")  
{alpha => 10, beta => 20} |> hbar |> to_file("hbar.svg")
```

radar_svg

`radar_svg` (aliases `radar`, `spider`) — SVG radar/spider chart. Args: labels, values [, title].

```
radar_svg(["Atk","Def","Spd","HP","MP"], [8,6,9,5,7], "Stats") |> to_file("radar.svg")
```

candlestick_svg

`candlestick_svg` (aliases `candlestick`, `ohlc`) — SVG candlestick OHLC chart. Args: array of [open,high,low,close] [, title].

```
candlestick_svg([[100,110,95,105],[105,115,100,112]], "Stock") |> to_file("ohlc.svg")
```

violin_svg

`violin_svg` (alias `violin`) — SVG violin plot from array of arrays showing distribution shape.

```
violin_svg([rnorm(200), rnorm(200, 2)], "Distributions") |> to_file("violin.svg")
```

cor_heatmap

`cor_heatmap` (alias `cor_matrix_svg`) — SVG correlation matrix heatmap. Computes pairwise correlations and renders as heatmap.

```
cor_heatmap([rnorm(100), rnorm(100), rnorm(100)], "Correlations") |> to_file("cor.svg")
```

stacked_bar_svg

`stacked_bar_svg` (alias `stacked_bar`) — SVG stacked bar chart. Args: labels, series (array of arrays) [, title].

```
stacked_bar_svg(["Q1","Q2","Q3"], [[10,20,30],[5,15,25]], "Revenue") |> to_file("stacked.svg")
```

wordcloud_svg

`wordcloud_svg` (aliases `wordcloud`, `wcloud`) — SVG word cloud from frequency hashref. Word size scales with frequency.

```
{rust => 50, go => 30, python => 40} |> wordcloud("Languages") |> to_file("cloud.svg")
```

treemap_svg

`treemap_svg` (alias `treemap`) — SVG treemap from frequency hashref. Area proportional to values.

```
{src => 500, tests => 200, docs => 100} |> treemap("Codebase") |> to_file("tree.svg")
```

preview

`preview` (alias `pvw`) — wrap SVG/HTML content in a cyberpunk-styled page and open in the default browser.

```
scatter_svg([1,2,3], [1,4,9]) |> preview  
bar_svg(["A","B"], [10,20]) |> pvw
```

Audio

4 topics

audio_convert

`audio_convert` (alias `aconv`) — convert audio files between WAV, FLAC, AIFF, and MP3 formats.

```
audio_convert("song.flac", "song.mp3")
aconv("input.wav", "output.mp3")
```

audio_info

`audio_info` (alias `ainfo`) — get audio file metadata (duration, sample rate, channels, format) as a hashref.

```
val $info = ainfo("song.mp3")
p $info->{duration}      # seconds
p $info->{sample_rate}    # e.g. 44100
```

id3_read

`id3_read` (alias `id3`) — read ID3 tags from an MP3 file as a hashref (title, artist, album, year, etc.).

```
val $tags = id3("song.mp3")
p "$tags->{artist} - $tags->{title}"
```

id3_write

`id3_write` (alias `id3w`) — write ID3 tags to an MP3 file from a hashref.

```
id3w("song.mp3", {title => "New Title", artist => "Artist", year => "2026"})
```

Network Utilities

43 topics

kick

`kick($host, $port [, $timeout_ms])` — TCP liveness probe. Returns `1` if a TCP connect to `$host:$port` succeeds within the timeout, `0` otherwise. Default timeout 1000ms (matches `nc -zw1` convention).

Resolves `$host` via the OS resolver — accepts hostnames, IPv4 / IPv6 literals, `localhost`. When DNS returns multiple addresses, each is tried in turn until one succeeds OR all fail within the timeout window.

Does not raise on failure — wrong host, closed port, refused connection, DNS failure, and timeout all return `0`, so callers can write boolean-style code:

```
if (kick("db.prod", 5432, 500)) {
    p "postgres is up"
} else {
    warn "postgres unreachable"
}

# Quick service-mesh health sweep:
for val $pair ([["web", 80], ["api", 8080], ["db", 5432]]) {
    printf " %-6s :%d %s\n", $pair->[0], $pair->[1],
        kick($pair->[0], $pair->[1]) ? "up" : "down"
}
```

This is a **convenience builtin** — the underlying capability (TCP connect with timeout) is in every language's stdlib. Stryke ships it as a bare builtin so script-level service probes don't need a socket import.

See also: `punch` (UDP send), `net_port_open` (older alias-style probe).

tcp_probe

`tcp_probe($host, $port [, $timeout_ms=1000])` — TCP connect probe returning RTT, not just bool. Returns `{ alive, latency_ms }`. Use `kick` for fast 1/0 sweeps, `tcp_probe` when you want latency, `tcp_banner` for service fingerprinting.

```
val $r = tcp_probe("db.prod", 5432, 500)
printf "postgres up - %dms\n", $r->{latency_ms} if $r->{alive}
```

See also: `kick`, `tcp_banner`, `whois_query`.

tcp_banner

`tcp_banner($host, $port [, $timeout_ms=1000, $max_bytes=512])` — connect + read first `$max_bytes` bytes the server sends. Classic service-fingerprint pattern.

Returns `{ alive, latency_ms, banner }`. Servers that don't greet (HTTP) return `banner => ""` with `alive=1`. Read timeout hardcoded to 200ms after connect.

```
val $r = tcp_banner("web1", 22)
printf "SSH %s\n", $1 if $r->{alive} && $r->{banner} =~ /^SSH-(\S+)/
```

See also: `tcp_probe`, `kick`.

whois_query

`whois_query($domain [, $server="whois.iana.org", $timeout_ms=5000])` — RFC 3912 WHOIS query. Connects `$server:43`, sends `$domain\r\n`, reads response until close. Returns string, `undef` on failure.

Default server is IANA's referral — chase the `Refer:` line for definitive registry results. 256 KB response cap.

```
val $iana = whois_query("example.com")
var ($registry) = $iana =~ /^refer:\s+(\S+)/m
var $detail = whois_query("example.com", $registry // "whois.verisign-grs.com")
```

Common registries: `.com/.net` → `whois.verisign-grs.com`, `.io` → `whois.nic.io`, IP/ASN → `whois.arin.net` / `whois.ripe.net` / `whois.apnic.net`.

See also: `tcp_probe`, `tcp_banner`.

udp_send

`udp_send($host, $port, $payload [, $retries=1, $interval_ms=20])` — send `$payload` as a UDP datagram to `$host:$port`. Returns the number of datagrams successfully sent (0 = total failure, `$retries` = full success).

`$payload` can be a string (sent as UTF-8 bytes) or a byte buffer (sent as-is). When `$retries > 1`, the same datagram is sent that many times at `$interval_ms` apart — useful for Wake-on-LAN magic packets (commonly sent 3× for reliability), NAT keepalives, and best-effort IoT discovery broadcasts. Broadcast addresses (`255.255.255.255`, subnet broadcasts) are enabled automatically.

```
# Wake-on-LAN: 6× 0xFF + target MAC × 16 = 102-byte magic packet on UDP/9.
var $mac = "aa:bb:cc:dd:ee:ff"
var @bytes = (0xff) x 6
push @bytes, (map { hex $_ } split /\:/, $mac) for 1:16
var $packet = pack("C*", @bytes)
udp_send("255.255.255.255", 9, $packet, 3) # send 3× for reliability

# NAT keepalive.
udp_send("203.0.113.45", 51820, "", 5, 100) # 5 shots × 100ms apart

# IoT discovery (SSDP).
udp_send("239.255.255.250", 1900, $ssdp_query)
```

For peer-to-peer NAT traversal where both endpoints are behind NAT and need STUN discovery + bidirectional flow establishment, see `punch` (forthcoming).

See also: `kick` (TCP probe).

udp_open

`udp_open([$bind_host="0.0.0.0", $bind_port=0])` — bind a UDP socket and return an opaque integer handle (0 on bind failure). The socket lives in a process-global pool until `udp_close($id)`.

`$bind_port = 0` lets the kernel assign an ephemeral port — the normal case. Passing a fixed port is useful when you want a stable inbound address.

Returned handle plugs into the persistent-socket family: `udp_send_to`, `udp_recv`, `udp_close`, `stun`, `punch`. The same handle MUST be used across a STUN-discover → punch → peer-to-peer-traffic flow because the NAT mapping the STUN response reveals is tied to that socket's (`local_ip`, `local_port`) — using a different socket invalidates the mapping.

```
val $sock = udp_open() # ephemeral inbound
var $info = stun($sock) # discover public ip:port
udp_send_to($sock, $peer_ip, $peer_port, "hi") # send via same socket
var $reply = udp_recv($sock, 1000) # receive on same socket
udp_close($sock)
```

See also: `udp_send` (fire-and-forget one-shot), `stun`, `punch`.

udp_send_to

`udp_send_to($id, $host, $port, $payload)` — send `$payload` to `$host:$port` via the pool socket `$id`. Returns bytes sent on success, 0 on any failure (unknown id, DNS failure, send error).

`$payload` is sent verbatim — pass strings as UTF-8 or byte buffers as-is. For broadcast (255.255.255.255, subnet broadcasts) the socket's `SO_BROADCAST` is already enabled by `udp_open`.

This is the **persistent-socket** counterpart to `udp_send` — use this when you need to send from a specific local port (e.g., one whose NAT mapping was discovered via STUN). Use `udp_send` for fire-and-forget sends where the local port doesn't matter.

```
val $sock = udp_open("0.0.0.0", 51820)
udp_send_to($sock, "203.0.113.45", 51820, $wg_heartbeat_pkt)
```

See also: `udp_open`, `udp_recv`, `udp_send` (one-shot variant).

udp_recv

`udp_recv($id [, $timeout_ms=1000])` — read one datagram from socket `$id`. Returns the payload as a UTF-8 string if valid, otherwise as raw bytes the caller can `unpack`. Returns `undef` on timeout, unknown id, or recv error.

Default timeout is 1000 ms. Pass `0` to block indefinitely (useful for daemon-style listeners). The blocked thread is the calling stryke task; for concurrent recv loops, wrap in `spawn { ... }`.

```
val $sock = udp_open("0.0.0.0", 5683) # CoAP port for IoT inbound
while (defined(val $payload = udp_recv($sock, 0))) {
  p "received " . length($payload) . " bytes"
  handle($payload)
}
```

See also: `udp_open`, `udp_send_to`.

udp_recv_from

`udp_recv_from($id [, $timeout_ms=1000])` — read one datagram from socket `$id` and surface the source address alongside the payload. Returns a hashref:

```
{ payload => "hello",          # string if UTF-8 valid, else bytes
  src_ip   => "203.0.113.45",
  src_port => 51234 }
```

Or `undef` on timeout / unknown id / recv error. Default timeout 1000 ms; `0` blocks indefinitely.

`src_ip` / `src_port` are exactly what the kernel reported on `recvfrom(2)` — for a peer behind NAT this is the peer's PUBLIC address as seen from this side, not their private LAN address. That's the address you want to pair with `udp_send_to($id, $src_ip, $src_port, $reply)` to answer back through the same NAT pinhole the peer punched open.

This is the bidirectional-chat counterpart to `udp_recv` — use this when you need to reply to whoever sent the datagram. Use `udp_recv` when you just want the payload and don't care about the source.

```
val $sock = udp_open("0.0.0.0", 5683)
while (defined(val $msg = udp_recv_from($sock, 0))) {
  p "from $msg->{src_ip}:$msg->{src_port}: $msg->{payload}"
  udp_send_to($sock, $msg->{src_ip}, $msg->{src_port}, "ack")
}
```

See also: `udp_recv` (payload-only variant), `udp_open`, `udp_send_to`, `punch`.

udp_close

`udp_close($id)` — release the socket handle `$id` from the pool. Returns `1` if the handle was present, `0` if unknown. Idempotent — calling on an already-closed handle is safe.

Long-running scripts should close sockets they no longer need to bound the pool size and free the underlying file descriptor. Process exit also releases all sockets, so short-lived scripts can skip the close.

See also: `udp_open`.

stun

`stun($id [, $stun_host="stun.l.google.com", $stun_port=19302, $timeout_ms=2000])` — query a STUN server via socket `$id` and return the public (`ip`, `port`) the server reports observing the request from. Returns:

```
{ public_ip => "203.0.113.45", public_port => 51234 }
```

Or `undef` on timeout / parse failure / unknown id. The STUN protocol (RFC 8489) is implemented in-tree — no external crate dependency.

Critical: subsequent `punch($id, ...)` and `udp_send_to($id, ...)` calls MUST use the same socket `$id`. The NAT mapping the STUN response reveals is tied to that socket's local (`ip`, `port`); using a different socket gets you a different public port (or none at all) and the punch will fail.

Default STUN server is `stun.l.google.com:19302` — Google's public, free, and reliable. Other public options: `stun.cloudflare.com:3478`, `stun.nextcloud.com:443`. Hosted internally for privacy-sensitive deployments.

```
val $sock = udp_open()
if (defined(val $info = stun($sock))) {
    p "I'm at $info->{public_ip}:$info->{public_port}"
    # Now tell the peer this address via signaling (out-of-band - email,
    # paste, shared file, IRC, anything that gets bytes from A to B).
    # The peer does the same, then both call punch() pointing at each
    # other's discovered ip:port.
}
```

See also: `punch`, `udp_open`.

stun_classify

`stun_classify($id [, $opts])` — query multiple STUN servers via the SAME socket and classify the local NAT based on whether the reported public ports match across servers. Returns a hashref:

```
{ nat_type    => "cone" | "symmetric" | "unknown",
  public_ip   => "203.0.113.45" | undef,
  queried     => 3,
  succeeded   => 3,
  observations => [
    { server => "stun.l.google.com:19302", ok => 1,
      public_ip => "203.0.113.45", public_port => 51234 },
    { server => "stun.cloudflare.com:3478", ok => 1,
      public_ip => "203.0.113.45", public_port => 51234 },
    ... ] }
```

`nat_type` interpretation:

- `"cone"` — every responding server reported the SAME port → NAT uses one public mapping per source socket regardless of destination. `punch` will work to any peer.
- `"symmetric"` — servers reported DIFFERENT ports → NAT allocates a fresh public port per destination. `punch` will FAIL because the port the peer punches at (learned from STUN server X) won't be the port your traffic to the peer uses. The only recovery is a TURN relay (not yet shipped).
- `"unknown"` — fewer than 2 servers responded → can't classify.

`$opts` (hashref, all optional):

- `servers` (arrayref of `["host", port]` pairs) — defaults to a diverse set: Google, Cloudflare, Nextcloud STUN. Diversity matters — some symmetric NATs bucket by destination /24, so same-provider servers can falsely produce matching ports. Three different ASNs catches more cases.
- `timeout_ms` (default 2000) — per-server timeout.

Recommended pattern — classify BEFORE calling `punch`, bail early on symmetric to give the user a useful error rather than a 30s `punch` timeout:

```
val $sock = udp_open()
var $nat = stun_classify($sock)
printf "NAT type: %s (succeeded %d/%d)\n",
    $nat->{nat_type}, $nat->{succeeded}, $nat->{queried}
if ($nat->{nat_type} eq "symmetric") {
    die "behind symmetric NAT - hole-punching impossible without TURN relay"
}
if ($nat->{nat_type} eq "unknown") {
    warn "only $nat->{succeeded} STUN servers reached - proceeding cautiously"
}
# ... continue with stun() + signaling + punch() ...
```

See also: `stun` (single-server query), `punch`.

punch

`punch($id, $peer_ip, $peer_port [, $opts])` — UDP NAT hole-punching state machine. Bombards `$peer_ip:$peer_port` via socket `$id` at `$opts{interval_ms}` apart, listens for any inbound datagram, returns when bidirectional flow is established OR the timeout fires.

`$opts` (hashref, all keys optional):

- `timeout_ms` (default 5000) — wall-clock punch budget
- `interval_ms` (default 50) — gap between bombards
- `payload` (default `"stryke-punch"`) — probe payload sent each round

Returns:

```
{ established => 1,          # 0 on timeout
  latency_ms  => 380,        # time to first reply
  bombards    => 8,          # count of datagrams we sent
  peer_msg    => "hi from b", # first received payload (or undef)
  peer_addr   => "203.0.113.45:51234" } # source addr (or undef)
```

The full P2P flow: both peers run `udp_open` → `stun` → exchange their discovered (`ip`, `port`) via an out-of-band signaling channel (email, paste, shared file, anything) → simultaneously call `punch` pointing at each other's discovered address. The first few bombards from each side are dropped by the other's NAT (no inbound mapping yet); the act of sending establishes the mapping, and subsequent packets get through.

Real-world success rate is ~70-80%. Failures (with no language-level workaround short of allocating a TURN relay):

- Symmetric NATs — some mobile carriers, some corporate firewalls — assign a different public port per destination, so the port revealed by STUN ≠ the port the peer's traffic actually arrives on
- UDP-blocking firewalls — drop all UDP outright
- Misaligned timing — if one peer punches 30s before the other, the first peer's NAT mapping may time out before the second peer starts

For a guaranteed-delivery fallback you need TURN (relay every packet through a public server) — out of scope for stryke's builtin set; if you need it, run a coturn server and use `udp_send_to` to relay manually.

```
# Side A:
var $sock = udp_open()
var $me = stun($sock)
printf "Tell peer to: stryke peer.stk %s %d\n",
    $me->{public_ip}, $me->{public_port}
# ... peer signals back their (ip, port) via email/paste ...
var $peer_ip  = "203.0.113.45"
var $peer_port = 51234
var $r = punch($sock, $peer_ip, $peer_port, { timeout_ms => 10_000 })
if ($r->{established}) {
    p "connected after $r->{bombards} bombards (" . $r->{latency_ms} . "ms)"
    p "first message from peer: $r->{peer_msg}"
    udp_send_to($sock, $peer_ip, $peer_port, "reply from A")
    p "reply: " . udp_recv($sock, 5000)
}
udp_close($sock)
```

See also: `stun`, `udp_open`, `udp_send_to`, `udp_recv`.

turn_allocate

`turn_allocate($id, $server, $port, $username, $password [, $timeout_ms=3000])` — drive the TURN (RFC 8656) two-roundtrip allocation flow on socket `$id`. Returns a hashref on success, `undef` on auth / network / parse failure.

```
{ relay_ip      => "203.0.113.99", # the public address the TURN server allocated to us
  relay_port    => 49999,
  lifetime_secs => 600,           # how long the allocation lives before needing refresh
  realm         => "turn.example", # server-supplied auth realm
  nonce_len     => 13 }           # length of opaque nonce stored in pool
```

The allocation is registered in a process-global pool keyed by `$id` so subsequent `turn_permission`, `turn_send`, `turn_recv`, `turn_refresh` all find it via the same socket id. **The socket must be the one you intend to use for the full TURN session** — TURN's NAT mapping is tied to the source (ip, port) of the original Allocate request.

Protocol: client sends Allocate (no auth) → server responds 401 with REALM + NONCE → client retries with USERNAME + REALM + NONCE + MESSAGE-INTEGRITY (HMAC-SHA1 of MD5(user:realm:pass) over the message bytes) → server replies success with XOR-RELAYED-ADDRESS.

```
val $sock = udp_open()
var $alloc = turn_allocate($sock, "turn.example.com", 3478, "alice", "hunter2")
if (!defined $alloc) { die "TURN allocation failed" }
printf "Relay address: %s:%d (lifetime %ds)\n",
    $alloc->{relay_ip}, $alloc->{relay_port}, $alloc->{lifetime_secs}
# Share $alloc->{relay_ip}:$alloc->{relay_port} with the peer via signaling.
# They install a permission for our relay address and send to it; the TURN
# server forwards their packets to us as DATA-INDICATION wrappers.
```

See also: `turn_permission`, `turn_send`, `turn_recv`, `turn_refresh`, `punch` (direct hole-punching, faster when it works), `stun_classify` (detect when TURN is necessary).

turn_permission

`turn_permission($id, $peer_ip [, $timeout_ms=2000])` — install a TURN CreatePermission for `$peer_ip` on the allocation associated with socket `$id`. Required before the TURN server will accept SEND-INDICATIONs targeting that peer's address (per RFC 8656 §9 — the server uses permissions to prevent generic relay abuse).

Returns `1` on CreatePermission_Success, `0` on failure (no allocation, network error, server rejection).

Permissions last ~5 minutes by default; call `turn_permission` again to refresh. Permission is keyed by peer IP only — any port from that IP is accepted, but a different IP requires a separate permission.

```
val $sock = udp_open()
var $alloc = turn_allocate($sock, $turn_host, 3478, $user, $pass) or die
if (!turn_permission($sock, "203.0.113.45")) {
    die "could not install permission for peer"
}
turn_send($sock, "203.0.113.45", 51820, "hello via relay")
```

See also: `turn_allocate`, `turn_send`, `turn_recv`.

turn_send

`turn_send($id, $peer_ip, $peer_port, $payload)` — wrap `$payload` in a TURN SEND-INDICATION and ship via the allocation associated with socket `$id`. The TURN server forwards the payload to `$peer_ip:$peer_port` from the relay address.

Returns bytes written to the TURN server (NOT bytes confirmed at the peer — peer receipt is best-effort UDP; confirmation comes only when a DATA-INDICATION reply arrives, surfaced via `turn_recv`).

A permission for `$peer_ip` MUST be installed via `turn_permission` first, or the server silently drops the send.

```
turn_send($sock, $peer_ip, $peer_port, "control message")
turn_send($sock, $peer_ip, $peer_port, pack("V", $frame_number))
```

See also: `turn_allocate`, `turn_permission`, `turn_recv`.

turn_recv

`turn_recv($id [, $timeout_ms=1000])` — wait for the next DATA-INDICATION on the TURN allocation's socket. Returns:

```
{ payload => "reply from peer",      # string if UTF-8 valid, else bytes
  peer_ip => "203.0.113.45",         # the peer's address as the TURN server saw it
  peer_port => 51820 }
```

Or `undef` on timeout, if the next packet isn't a DATA-INDICATION (e.g. a late CreatePermission_Success), or if no allocation exists for `$id`. Caller should call again to drain non-data frames.

Default timeout 1000 ms; `0` blocks indefinitely.

```
while (defined(val $msg = turn_recv($sock, 0))) {
    printf "from %s:%d: %s\n", $msg->{peer_ip}, $msg->{peer_port}, $msg->{payload}
    turn_send($sock, $msg->{peer_ip}, $msg->{peer_port}, "ack")
}
```

See also: `turn_allocate`, `turn_send`, `udp_recv_from` (direct-UDP counterpart).

turn_refresh

`turn_refresh($id [, $lifetime_secs=600, $timeout_ms=2000])` — send a Refresh request to extend the TURN allocation's lifetime. Pass `$lifetime_secs=0` to release the allocation immediately.

Returns the new lifetime on success, `0` on failure. Call before the previous `lifetime_secs` elapses — typical cadence is refresh at ~2/3 of the current lifetime so a single network hiccup doesn't drop the allocation.

```
# Background keepalive: refresh every 5 minutes (allocation has 10-min lifetime).
spawn {
    while (1) {
        sleep 300
        last unless turn_refresh($sock)
    }
}
```

Releasing on shutdown is good citizenship — it frees the TURN server's relay port immediately instead of waiting for the timeout:

```
turn_refresh($sock, 0)
udp_close($sock)
```

See also: `turn_allocate`.

teleport

`teleport($val, @receiver_pids [, { hold_ms => 500 }])` — broadcast a value to N stryke processes via POSIX shared memory + per-receiver UDS notification. Returns the count of receivers whose notification landed (their UDS socket exists + accepted the notify).

Data-first signature so the prefix thread macro composes naturally:

```
-> $huge teleport(@worker_pids) # prefix ~>: threads $huge as arg 0
teleport($payload, $pid1, $pid2) # standalone, literal PIDs
teleport($payload, [@pids]) # arrayref also flattens
teleport($payload, $pid, { hold_ms => 1500 }) # opts hash for hold window
```

Receivers must be stryke processes running an `arrive()` loop. Non-stryke processes (or stryke processes without an `arrive` call) have no bound UDS socket and count as unreachable.

`hold_ms` (default 500) is how long the sender holds the SHM segment alive after notifying so receivers can `shm_open` + `mmap` + `read`. Receivers slower than `hold_ms` miss the window.

Wire path: `StrykeValue` → `serde_json::Value` → `JSON bytes` → `SHM segment named /stryke_tp_PID_SEQ` → `40-byte notify over UDS to /tmp/stryke_teleport_PID.sock` **per receiver**. Receivers reverse this. Same lossy-but-portable shape as `cluster` / `dist_thread` IPC — closures and blessed objects don't round-trip; scalars, arrays, hashes, and nested combos do. JSON beats bincode here because bincode can't deserialize a free-standing `serde_json::Value` (Value needs `deserialize_any`).

Top-level convention: arrays / hashes always arrive as refs (`@a` → arrayref, `%h` → hashref), matching `decode_json`. So `$msg->{key}` and `$msg->[i]` Just Work without re-wrapping.

World-first capability: no other scripting language ships zero-copy(ish) multi-target value teleportation as a single builtin. Closest analogs (Python `multiprocessing.shared_memory`, Perl `IPC::Shareable`) require manual segment lifecycle + per-process attach plumbing, single-target only, no notification primitive bundled in.

```
# Parent forks workers, then teleports a 100MB hashref to all of them.
var @worker_pids = map { fork_worker() } 1:8
var $huge = build_corpus() # 100MB hashref
teleport($huge, @worker_pids, { hold_ms => 1500 })
```

See also: `arrive`, `fork`, `cluster`.

arrive

`arrive([$timeout_ms=5000])` — block up to `$timeout_ms` for the next `teleport` notification from any sender, then read + return the value. Returns `undef` on timeout, malformed notification, or stale SHM (sender unlinked before this receiver got around to reading).

First call lazy-binds the per-process UDS socket at `/tmp/stryke_teleport_$$sock`; subsequent calls reuse it. The socket file is best-effort cleaned up on bind (stale file from a prior crashed instance gets unlinked).

Idiomatic receiver loop:

```
while (1) {
  val $msg = arrive(1000)
  last unless defined $msg
  process($msg)
}
```

Returned value is whatever the sender passed to `teleport($val, ...)` after a JSON → bincode → SHM → bincode → JSON → StrykeValue round-trip. Numbers, strings, arrays, hashes round-trip exactly; coderefs and blessed objects do not.

See also: `teleport`, `cluster_recv`.

turnbuckle

`turnbuckle($peer_pid [, { interval_ms => 50, timeout_ms => 200 }])` — open a 1:1 peer-pair keepalive against another stryke process. Spawns a background thread that exchanges single-byte heartbeats with the peer over symmetric UDS sockets:

```
this side binds: /tmp/stryke_turnbuckle_<my_pid>_<peer_pid>.sock
this side sends: /tmp/stryke_turnbuckle_<peer_pid>_<my_pid>.sock
```

Returns a handle hashref `{ _tb_id => N, peer_pid => P }` for use with `tb_alive` / `tb_ping` / `tb_close`. Returns `undef` on bind failure or non-positive `$peer_pid`.

Companion to `teleport` / `arrive`: where `teleport` is 1:N value broadcast, `turnbuckle` is a 1:1 liveness primitive. Carries no payload — use `teleport` to send data through the pair.

`interval_ms` (default 50) is how often the background thread heartbeats; `timeout_ms` (default 200) is the liveness window. `timeout_ms` is auto-bumped to `2 * interval_ms` if the caller specifies a tighter value, to absorb a single missed heartbeat without flapping.

```
# Each side opens against the other.
var $tb = turnbuckle($peer_pid, { interval_ms => 100, timeout_ms => 500 })
while (tb_alive($tb)) { do_work() }
tb_close($tb)
```

See also: `tb_alive`, `tb_ping`, `tb_close`, `teleport`.

tb_alive

`tb_alive($handle)` — return 1 if the peer has heartbeated within the configured `timeout_ms` window, else 0. Non-blocking. The handle is the hashref returned by `turnbuckle($peer_pid)`. Unknown handles (already-closed or never-opened) return 0.

```
val $tb = turnbuckle($peer_pid)
sleep_ms(150)           # let one heartbeat exchange happen
p tb_alive($tb)         # 1 if peer is up, 0 otherwise
```

tb_ping

`tb_ping($handle)` — send one extra heartbeat to the peer right now, in addition to the background cadence. Returns 1 on send success, 0 otherwise. Useful when you want to pre-warm the peer's `last_heard` timestamp ahead of a deadline rather than waiting for the next `interval_ms` tick.

```
tb_ping($tb)           # peer should see us alive within ~1 RTT
```

tb_close

tb_close(\$handle) — tear down the pair: signal the heartbeat thread to exit, join it, unbind the UDS socket file, drop the registry entry. Returns 1 if the handle was known, 0 if it was already closed or never opened. Idempotent.

```
tb_close($tb)
```

weep

weep(@source, \$interval_ms) — slow-trickle emitter. The LAST argument is always **interval_ms**; everything before it is the source. Wraps the source into a streaming iterator that emits each item paced at **\$interval_ms** between emissions. The FIRST item emits immediately; only the gap between successive items is throttled. Zero interval = passthrough (no sleep).

Call shapes:

```
weep(1:5, 100)          # range flattens → 5 items, throttle 100ms
weep([1, 2, 3], 100)     # arrayref → 3 items, throttle 100ms
weep(@list, 100)         # @list flattens → N items, throttle 100ms
weep($iter, 100)         # iterator (e.g. from another builtin) → use as-is
weep("a", "b", "c", 100) # 3 string items, throttle 100ms
```

Single-arg form **weep(\$x)** is source-only (no throttle). To pass a 1-item numeric source where the value would otherwise be parsed as the interval, wrap it: **weep([42], 0)**.

Lazy by construction — the sleep happens at each consumer pull, not up-front. The throttle counts wall-clock time SINCE THE LAST EMISSION, so if a downstream consumer is already slow, weep adds nothing.

```
# Drip every 100ms.
for val $item (weep([1, 2, 3, 4, 5], 100)) { p $item }

# lazy chain - take(3) pulls only 3 items, paying for only 2 sleeps.
~> 1:1000 weep(50) take(10) collect()
```

See also: **take**, **map**, **tap**, **turnbuckle**.

net_interfaces

net_interfaces (aliases **net_ifs**, **ifconfig**) — list all network interfaces with name, IPs, MAC, and status.

```
var @ifs = net_ifs()
for (@ifs) { p "_->{name}: _->{ipv4}" }
```

net_ipv4

net_ipv4 (aliases **myip**, **myip4**) — return the first non-loopback IPv4 address as a string.

```
p myip() # e.g. 192.168.1.42
```

net_ipv6

net_ipv6 (alias **myip6**) — return the first non-loopback IPv6 address as a string.

```
p myip6() # e.g. fe80::1
```

net_mac

net_mac (alias **mymac**) — return the first non-loopback MAC address.

```
p mymac() # e.g. aa:bb:cc:dd:ee:ff
```

net_public_ip

`net_public_ip` (aliases `pubip`, `extip`) — fetch your public IP address via HTTP.

```
p pubip() # e.g. 203.0.113.42
```

net_dns

`net_dns` (aliases `dns_resolve`, `resolve`) — resolve a hostname to IP addresses.

```
val @ips = resolve("github.com")
p @ips
```

net_reverse_dns

`net_reverse_dns` (alias `rdns`) — reverse DNS lookup via UDP PTR query.

```
p rdns("8.8.8.8") # dns.google
```

net_ping

`net_ping` (alias `ping`) — TCP connect ping to host:port. Returns array of RTT measurements in ms.

```
val @rtt = ping("google.com", 443, 5) # 5 pings
p mean(@rtt)
```

net_port_open

`net_port_open` (alias `port_open`) — check if a TCP port is open on a host. Returns boolean.

```
p port_open("localhost", 8080) # 1 or 0
```

net_ports_scan

`net_ports_scan` (aliases `port_scan`, `portscan`) — scan a range of TCP ports on a host. Returns list of open ports.

```
val @open = port_scan("localhost", 80, 443)
p @open # e.g. 80, 443
```

net_latency

`net_latency` (alias `tcplat`) — measure TCP connect latency to host:port in milliseconds.

```
p tcplat("google.com", 443) # e.g. 12.5
```

net_download

`net_download` (aliases `download`, `wget`) — download a URL to a local file via ureq.

```
wget("https://example.com/data.csv", "data.csv")
```

net_headers

`net_headers` (alias `http_headers`) — fetch HTTP response headers as a hashref.

```
val $h = http_headers("https://example.com")
p $h->{"content-type"}
```


net_dns_servers

`net_dns_servers` (alias `dns_servers`) — return the system's configured DNS server addresses.

```
val @dns = dns_servers()
p @dns    # e.g. 8.8.8.8, 8.8.4.4
```

net_gateway

`net_gateway` (alias `gateway`) — return the default gateway IP address.

```
p gateway()    # e.g. 192.168.1.1
```

net_whois

`net_whois` (alias `whois`) — WHOIS lookup via raw TCP connection on port 43.

```
p whois("example.com")
```

net_hostname

`net_hostname` — return the system hostname.

```
p net_hostname()    # e.g. my-macbook.local
```

smtp_send

`smtp_send` (aliases `send_email`, `email`) — send an email via SMTP with TLS. Takes a hashref with to, from, subject, body, and SMTP connection fields.

```
smtp_send({to => "bob@ex.com", from => "me@ex.com", subject => "Hi",
  body => "Hello!", smtp_host => "smtp.ex.com", smtp_port => 587,
  smtp_user => "me@ex.com", smtp_pass => $pass})
```

Markup / Web Scraping

4 topics

html_parse

`html_parse` (alias `parse_html`) — parse an HTML string into an array of element hashrefs with tag, text, and attrs.

```
val @els = html_parse(slurp("page.html"))
for (@els) { p "->{tag}: _->{text}" }
```

css_select

`css_select` (aliases `css`, `qs`, `query_selector`) — query parsed HTML with a CSS selector. Returns matching elements as hashrefs with tag, text, attrs, and html.

```
val @links = css_select(slurp("page.html"), "a.nav")
for (@links) { p "->{tag} _->{attrs}{href}" }
```

xml_parse

`xml_parse` (alias `parse_xml`) — parse an XML string into a nested hashref tree with tag, text, attrs, and children.

```
val $root = xml_parse(slurp("data.xml"))
p $root->{tag}
for (@{$root->{children}}) { p _->{text} }
```

xpath

`xpath` (alias `xml_select`) — query XML with XPath-like expressions (`//tag`, `//tag[@attr='val']`). Returns matching nodes as hashrefs.

```
val @items = xpath(slurp("feed.xml"), "//item")
for (@items) { p _->{text} }
```

Date Utilities

4 topics

dateseq

dateseq (alias **dseq**) — generate a sequence of dates. Args: start, end [, step]. Step defaults to 1 day.

```
val @days = dseq("2026-01-01", "2026-01-07")
p @days
```

dategrep

dategrep (alias **dgrep**) — filter a list of date strings by a pattern or range.

```
val @jan = dgrep("2026-01", @dates)
```

dateround

dateround (alias **dround**) — round date strings to a unit (day, hour, month, etc.).

```
p dround("2026-04-19T14:35:22", "hour")    # 2026-04-19T15:00:00
```

datesort

datesort (alias **dsort**) — sort date strings chronologically.

```
val @sorted = dsort("2026-03-01", "2026-01-15", "2026-02-10")
p @sorted
```

Git

10 topics

git_log

`git_log` (alias `glog`) — get the last N commits as an array of hashrefs (hash, author, date, message).

```
var @commits = glog(10)
for (@commits) { p "->{hash} _->{message}" }
```

git_status

`git_status` (alias `gst`) — get working tree status as an array of hashrefs (path, status).

```
var @st = gst()
for (@st) { p "->{status} _->{path}" }
```

git_diff

`git_diff` (alias `gdiff`) — get the current diff as a string.

```
p gdiff()
gdiff() |> lines |> grep /^\/+ / |> e p
```

git_branches

`git_branches` (alias `gbr`) — list all branches as an array of strings.

```
val @br = gbr()
p @br
```

git_tags

`git_tags` (alias `gtags`) — list all tags as an array of strings.

```
val @tags = gtags()
p @tags
```

git_blame

`git_blame` (alias `gblame`) — blame a file, returning per-line annotation.

```
p gblame("src/main.rs")
```

git_authors

`git_authors` (alias `gauthors`) — list unique authors sorted by commit count as hashrefs.

```
var @authors = gauthors()
for (@authors) { p "->{name}: _->{count}" }
```

git_files

`git_files` (alias `gfiles`) — list all tracked files in the repo.

```
val @files = gfiles()
p scalar @files    # total tracked files
```

git_show

`git_show` (alias `gshow`) — show details of a commit (message, diff, author).

```
p gshow("HEAD")
p gshow("abc1234")
```

git_root

`git_root` (alias `groot`) — return the repository root path.

```
p groot()    # e.g. /home/user/project
```

System

13 topics

mounts

`mounts` (aliases `disk_mounts`, `filesystems`) — list mounted filesystems with usage (total, used, available).

```
var @m = mounts()
for (@m) { p "_->{mount}: _->{used}/_->{total}" }
```

thread_count

`thread_count` (alias `nthreads`) — return the rayon thread pool size.

```
p nthreads() # e.g. 8
```

pool_info

`pool_info` (alias `par_info`) — thread-pool + host metadata as a hashref. Useful for sizing parallel work, gating on architecture, and reporting in benchmarks.

Keys (always present):

| key | type | meaning | |---|---|---| | `cpus` | Int | logical-CPU count from `available_parallelism` (hyperthreads counted) | | `rayon_threads` | Int | current rayon worker-pool size (what `pmap` / `pfor` / `pgrep` use) | | `arch` | Str | build target architecture: `aarch64`, `x86_64`, `riscv64`, ... | | `os` | Str | build target OS: `macos`, `linux`, `freebsd`, `windows`, ... |

macOS-only keys (populated via `sysctlbyname` on Apple Silicon / Intel Macs):

| key | type | meaning | |---|---|---| | `perf_cores` | Int | performance-core count (`hw.perflevel0.physicalcpu`) | | `eff_cores` | Int | efficiency-core count (`hw.perflevel1.physicalcpu`) |

```
val $info = pool_info()
p $info->{cpus}          # 12 (e.g. M3 Pro)
p $info->{rayon_threads} # 12
p $info->{arch}          # "aarch64"
p $info->{os}            # "macos"
p $info->{perf_cores}    # 6      (Apple Silicon only)
p $info->{eff_cores}     # 4      (Apple Silicon only)

# Sizing parallel work to perf cores when available:
var $n = $info->{perf_cores} // $info->{rayon_threads}
@inputs |> pmap { heavy(_) }
```

See also: `thread_count`, `par_bench`, `pmap`, `pfor`, `pgrep`, `set_pool_size`.

par_bench

`par_bench` (alias `pbench`) — run a parallel throughput benchmark and return results.

```
val $result = pbench(1000000)
p $result->{ops_per_sec}
```

to_pdf

`to_pdf` — generate a PDF from text, SVG, or structured data. Returns raw PDF bytes.

```
"Hello World" |> to_pdf |> to_file("out.pdf")
scatter_svg([1,2,3], [1,4,9]) |> to_pdf |> to_file("plot.pdf")
```

pdf_text

`pdf_text` (aliases `pdf_read`, `pdf_extract`) — extract all text content from a PDF file and return it as a string.

```
val $text = pdf_text("report.pdf")
p $text |> lines |> cnt # number of lines
```

pdf_pages

`pdf_pages` — return the number of pages in a PDF file.

```
p pdf_pages("report.pdf") # e.g. 42
```

jq

`jq` — alias for `json_jq`. Query JSON data with jq-style expressions.

```
val $data = json_decode(slurp("data.json"))
p jq($data, ".items[].name")
```

stress_cpu

`stress_cpu` (alias `scpu`) — CPU stress test: run SHA256 hashing in a tight loop for the specified duration (default 1 second). Returns number of hashes computed. Use for load testing and thermal validation.

```
val $hashes = stress_cpu(10) # 10 seconds of CPU stress
p "Computed $hashes hashes"

# Distributed across cluster
var $c = cluster([{"node1:8", "node2:8"}])
1:16 |> pmap_on $c { stress_cpu(60) } # 60s stress on all workers
```

stress_mem

`stress_mem` (alias `smem`) — Memory stress test: allocate and touch the specified number of bytes (default 100MB). Returns bytes allocated. Use for memory pressure testing and OOM validation.

```
val $bytes = stress_mem(1e9) # Allocate 1GB
p "Allocated $bytes bytes"

# Memory pressure across cluster
1:16 |> pmap_on $cluster { stress_mem(4e9) } # 4GB per worker
```

stress_io

`stress_io` (alias `sio`) — IO stress test: write and read temp files in the specified directory (default /tmp) for N iterations (default 100). Returns total bytes written. Use for storage IOPS testing.

```
val $bytes = stress_io("/tmp", 1000) # 1000 iterations
p "Wrote $bytes bytes"

# IO stress across cluster
1:16 |> pmap_on $cluster { stress_io("/data", 500) }
```

stress_test

`stress_test` (alias `st`) — Combined CPU/memory/IO stress test. If cluster provided, distributes across all workers. Returns hashref with stats: {cpu_hashes, mem_bytes, io_bytes, workers, duration}.

```
# Local stress test (10 seconds)
var $r = stress_test(10)
p "CPU: $r->{cpu_hashes}, Workers: $r->{workers}"

# Distributed stress test
var $c = cluster(["node1:8", "node2:8", "node3:8"])
var $r = stress_test($c, 60) # 60s across 24 workers
p "Hashes: $r->{cpu_hashes}, Duration: $r->{duration}s"
```

heat

heat — Maximum thermal stress. Pins ALL cores to 100% TDP. The hottest function in any programming language. Use for infrastructure burn-in, cooling validation, and power testing.

```
heat(60) # 60 seconds of maximum heat
heat()   # default 60 seconds

# Output:
# 🔥 HEAT: Pinning N cores to 100% TDP for 60s
# 🔥 HEAT: 3,116,320,000 hashes in 60.00s (51.9M/s across N cores)
```


Terminal Control

24 topics

clear

`clear` / `cls` — emit ANSI `\x1b[2J\x1b[H` (clear screen + cursor home). Lighter than `reset` (which re-init's the terminal); `clear` is the shell muscle-memory key for "give me a blank prompt."

```
clear()                                # blank the screen
```

set_title

`set_title(TITLE)` — emit the ANSI OSC sequence `\x1b]2;TITLE\x07` (sets the terminal window title in most modern terminals: macOS Terminal, iTerm2, Alacritty, kitty, GNOME Terminal, Windows Terminal). Uses `OSC 2` rather than `OSC 0` so the icon is left untouched. Returns `1`.

```
set_title("stryke - $script_name")
```

beep

`beep` / `ring_bell` — emit `\x07` (BEL — ASCII 7), which most terminals interpret as an audible or visual bell depending on settings. Returns `1`.

seizure

`seizure(count=24, delay_ms=40)` / `strobe` — strobe the terminal: blank the screen and cycle the background through bright ANSI colors `count` times, pausing `delay_ms` ms between frames. Always restores a clean screen on exit. Returns the number of frames flashed. Cosmetic REPL party-trick in the same family as `beep` / `clear`; `count` clamps to `[0, 10000]`, `delay_ms` to `[0, 5000]`.

```
seizure()                             # 24 flashes, ~1s
seizure(60, 20)                       # faster, longer
```

flash

`flash(ms=80)` / `visual_bell` — the proper visual bell: invert the whole screen (`\x1b[?5h`, DECSCNM reverse-video), pause `ms` milliseconds, then restore (`\x1b[?5l`). One clean blink with no color churn — the tame cousin of `seizure`. `ms` clamps to `[0, 5000]`. Returns `1`.

```
flash()                               # quick screen blink
flash(200)                            # longer, more emphatic
```

hide_cursor

`hide_cursor` / `cursor_hide` — emit `\x1b[?25l` (DECTCEM off) to hide the text cursor. Pair with `show_cursor` around any animation so the blinking caret doesn't flicker through the frames. Returns `1`.

```
hide_cursor()
# ... draw an animation ...
show_cursor()
```

show_cursor

`show_cursor` / `cursor_show` — emit `\x1b[?25h` (DECTCEM on) to restore the text cursor hidden by `hide_cursor`. Returns 1.

cursor_to

`cursor_to(row=1, col=1)` / `goto_xy` — move the cursor to a 1-based `(row, col)` via `\x1b[{row}]{col}H` (CUP). Row 1 / col 1 is the top-left corner; out-of-range coordinates are clamped by the terminal. Returns 1.

```
hide_cursor()
cursor_to(1, 1)
print "top-left"
```

clear_line

`clear_line` / `erase_line` — erase the entire line the cursor is on (`\x1b[2K`) without moving the cursor. Pair with `\r` to rewrite a status line in place. Returns 1.

```
for my $i (0..100) { print "\r"; clear_to_eol(); print progress($i/100) }
```

clear_to_eol

`clear_to_eol` / `clear_eol` — erase from the cursor to the end of the line (`\x1b[0K`), leaving everything before the cursor intact. Returns 1.

scroll_up

`scroll_up(n=1)` — scroll the whole display up `n` lines (`\x1b[{n}S`), opening `n` blank lines at the bottom. Returns 1.

scroll_down

`scroll_down(n=1)` — scroll the whole display down `n` lines (`\x1b[{n}T`), opening `n` blank lines at the top. Returns 1.

alt_screen

`alt_screen` / `enter_alt_screen` — switch to the terminal's alternate screen buffer (`\x1b[?1049h`), the same full-screen takeover `vim` and `less` use. The primary buffer and scrollbar are saved untouched; call `restore_screen` to return exactly where the user left off. Returns 1.

```
alt_screen()
clear()
# ... run a full-screen TUI ...
restore_screen()           # scrollbar comes back intact
```

restore_screen

`restore_screen` / `exit_alt_screen` — leave the alternate screen buffer (`\x1b[?1049l`) entered by `alt_screen`, restoring the primary buffer and scrollbar. Returns 1.

notify

`notify(TITLE, BODY="")` / `desktop_notify` — raise an OS desktop notification. macOS shells out to `osascript` (`display notification`), Linux to `notify-send`; other platforms are a no-op. Returns 1 on a successful spawn, 0 otherwise. A natural companion to `beep` when a long task finishes off-screen.

```
long_running_job()
notify("Build done", "all green ✓") || beep()
```


`term_width`

`term_width` → integer column count of the controlling terminal (0 off-TTY).

`term_height`

`term_height` → integer row count of the controlling terminal (0 off-TTY).

Testing

15 topics

assert_eq

assert_eq (alias **aeq**) — assert that two values are equal as strings. Takes A, B, and an optional message. Fails the test with a diff if A ne B.

```
assert_eq $got, $expected, "username matches"
aeq length(@arr), 3
```

assert_ne

assert_ne (alias **ane**) — assert that two values are not equal as strings. Takes A, B, and an optional message. Fails if A eq B.

```
assert_ne $token, "", "token must not be empty"
ane $x, $y
```

assert_ok

assert_ok (alias **aok**) — assert that a value is truthy (defined and non-zero/non-empty). Fails if the value is falsy or undef.

```
assert_ok $result, "fetch returned data"
aok $user->{active}
```

assert_err

assert_err — assert that a value is falsy or undef. The inverse of **assert_ok**. Useful for verifying error conditions or absent values.

```
assert_err $deleted_user, "user should be gone"
assert_err 0
```

assert_true

assert_true (alias **atrue**) — alias for **assert_ok**. Assert that a value is truthy.

```
assert_true $connected, "should be connected"
atrue $flag
```

assert_false

assert_false (alias **afalse**) — alias for **assert_err**. Assert that a value is falsy or undef.

```
assert_false $error, "no error expected"
afalse $disabled
```

assert_gt

assert_gt — assert that the first numeric value is strictly greater than the second. Fails with the actual values on mismatch.

```
assert_gt $elapsed, 0, "elapsed must be positive"
assert_gt scalar(@results), 10
```

assert_lt

`assert_lt` — assert that the first numeric value is strictly less than the second.

```
assert_lt $latency, 100, "latency under 100ms"
assert_lt $errors, $threshold
```

assert_ge

`assert_ge` — assert that the first numeric value is greater than or equal to the second.

```
assert_ge $count, 1, "at least one result"
assert_ge $version, 2
```

assert_le

`assert_le` — assert that the first numeric value is less than or equal to the second.

```
assert_le $memory, $limit, "within memory budget"
assert_le length($buf), 4096
```

assert_match

`assert_match` (alias `amatch`) — assert that a string matches a regex pattern. Fails with the actual string and pattern on mismatch.

```
assert_match $email, qr/\@/, "must contain @"
amatch $line, qr/^OK/
```

assert_contains

`assert_contains` (alias `icontains`) — assert that a string contains a given substring. Fails with both values on mismatch.

```
assert_contains $html, "<title>", "page has title"
icontains $log, "SUCCESS"
```

assert_near

`assert_near` (alias `anear`) — assert that two floats are approximately equal within an epsilon tolerance (default 1e-9). Essential for floating-point comparisons.

```
assert_near 0.1 + 0.2, 0.3, 1e-10, "float add"
anear $pi, 3.14159, 1e-5
```

assert_dies

`assert_dies` (alias `adies`) — assert that a block throws an error. Passes if the block dies, fails if it returns normally.

```
assert_dies { die "boom" } "should throw"
adies { 1 / 0 }
```

test_run

`test_run` (alias `run_tests`) — print a test summary with pass/fail counts and exit with code 1 if any test failed. Call at the end of a test file to report results.

```
# ... assertions above ...
test_run # prints summary, exits 1 on failure
```

Test Runner (worker pool)

4 topics

test

`test(FILE_OR_DIR, ...)` — run every `test_*.stk / t_*.stk` under each path through the **worker pool**. Same engine as `s t` on the CLI: pre-forks N persistent stryke `--test-worker` processes, dispatches paths via JSON over stdin/stdout, each worker `fork()`s a grandchild per test for full isolation, no per-test dyld cost. Returns the exit code (0 on all-pass, 1 on any failure). Optional trailing hashref selects mode: `{ fork => 1 }` for the legacy `posix_spawn`-per-test path, `{ inproc => 1 }` for single-process VM-per-test (fastest, hermetic only), `{ quiet => 1 }, { no_interop => 1 }`.

```
test("t/") # default: worker pool
test(["t/", "examples/"], { quiet => 1 })
test("t/test_foo.stk", { fork => 1 }) # opt back into per-test fork
var $rc = test(@dirs) # array flattens; rc = 0/1
```

test_no_interop

`test_no_interop(FILE_OR_DIR, ...)` (alias `test_ni`) — same as `test`, but pins each worker thread's TLS no-interop flag for the duration of the test (`set_no_interop_mode_tls`). Sibling `pmaps` workers don't race on a shared atomic. Forwards `--no-interop` to every spawned grandchild so each test runs under stryke's bot-firewall mode (rejects Perl-isms, forces stryke idioms).

```
test_no_interop("t/")
test_ni(["t/a", "t/b"], { quiet => 1 })
```

check

`check(FILE, ...)` — in-process equivalent of `stryke check FILE...`. Parses, compiles, and lints each path without executing it. Returns the exit code (0 on success, 1 on errors). Optional trailing hashref: `{ quiet => 1 }, { json => 1 }, { no_interop => 1 }`.

Faster than `system "stryke check $f"` because it skips the binary fork and reuses the parent process's parser/compiler. The lint pass is **not** a strict-vars enforcer — undefined-variable errors only fire on files that themselves do `use strict`;

```
val $rc = check("foo.stk") # 0 = ok, 1 = errors
check(@files, { quiet => 1 }) # arrayref + opts
for val $f (glob("src/*.stk")) { check($f) or warn "$f failed" }
```

check_no_interop

`check_no_interop(FILE, ...)` (alias `check_ni`) — same as `check`, but pins **this thread** to `--no-interop` for the duration of the check via an RAII guard on the per-thread no-interop flag. Sibling threads are unaffected, so the builtin is safe to call from `pmaps` workers in parallel — no global-atomic race.

```
check_no_interop("foo.stk") # rejects sub/say/$x/$y
pmaps { check_ni(_, { quiet => 1 }) } @files # parallel-safe, no race
```

AI primitives (docs/AI_PRIMITIVES.md)

79 topics

ai

`ai $prompt, opts...` — single-shot LLM call (or auto-routes to the agent loop when `tools => [...]/tool fns` are in scope, vision when `image => $url|$path`, structured output when `schema => +{...}`, document input when `pdf => $path`).

```
val $r = ai "summarize this", $document
var $r = ai "weather in SF" # auto-uses any `tool fn` defined in scope
var $r = ai "describe", image => "/photo.jpg"
var $r = ai "extract user", schema => +{ name => "string", age => "int" }
```

Providers: anthropic (default), openai. Mock with `ai_mock_install`. Cost via `ai_cost`.

stream_prompt

`stream_prompt $prompt, on_chunk => fn { ... }` — Anthropic SSE streaming. With `on_chunk`, fires once per text-delta. Without, returns a `PerlIterator`-backed handle: `for var $chunk in stream_prompt("...") { p $chunk }`.

```
val $state = +{ buf => "" };
stream_prompt("write a haiku",
  on_chunk => fn { $state->{buf} .= _0; p _0 });
```

chat

`chat \@messages, opts...` — explicit message-list call. Each entry is `+{role => "user"|"assistant"|"system", content => ...}`.

```
chat([
  +{ role => "system", content => "Be concise" },
  +{ role => "user", content => "hi" },
])
```

For multi-turn with auto-tracking, use `ai_session_*` instead.

embed

`embed $text` or `embed \@texts` — vector embedding via Voyage AI (default) or OpenAI. Returns arrayref of floats (single text) or arrayref-of-arrayref (batch). Routes via `ai_routing_set("embed", ...)`.

```
val @vec = @{ embed("hello world") }
var @vecs = @{ embed(["a", "b", "c"]) }
```

tokens_of

`tokens_of` — operation `tokens of`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p tokens_of $x
```

ai_estimate

`ai_estimate` — operation `ai estimate`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_estimate $x
```

`ai_cost`

`ai_cost()` — running spend / token / cache stats hashref: `+{usd, input_tokens, output_tokens, embed_tokens, cache_creation_tokens, cache_read_tokens, cache_hits, cache_misses}`.

`ai_history`

`ai_history` — operation `ai history`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_history $x
```

`ai_history_clear`

`ai_history_clear` — operation `ai history clear`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_history_clear $x
```

`ai_cache_clear`

`ai_cache_clear` — operation `ai cache clear`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_cache_clear $x
```

`ai_cache_size`

`ai_cache_size` — operation `ai cache size`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_cache_size $x
```

`ai_mock_install`

`ai_mock_install` — operation `ai mock install`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_mock_install $x
```

`ai_mock_clear`

`ai_mock_clear` — operation `ai mock clear`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_mock_clear $x
```

`ai_config_get`

`ai_config_get` — operation `ai config get`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_config_get $x
```

`ai_config_set`

`ai_config_set` — operation `ai config set`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_config_set $x
```

ai_routing_get

`ai_routing_get` — operation `ai routing get`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_routing_get $x
```

ai_routing_set

`ai_routing_set` — operation `ai routing set`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_routing_set $x
```

ai_register_tool

`ai_register_tool($name, $description, +{params}, fn { ... })` — register a tool that the bare `ai($prompt)` call auto-attaches to its agent loop. Idempotent re-register on same name.

```
ai_register_tool("weather", "Get weather",
  +{ city => "string" },
  fn { fetch "https://api.weather.com/" . uri_encode(_->{city}) });
ai_clear_tools();
```

For parser-level sugar, use `tool fn` instead.

ai_unregister_tool

`ai_unregister_tool` — operation `ai unregister tool`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_unregister_tool $x
```

ai_clear_tools

`ai_clear_tools` — operation `ai clear tools`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_clear_tools $x
```

ai_tools_list

`ai_tools_list` — operation `ai tools list`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_tools_list $x
```

ai_filter

`ai_filter \@items, "criterion"` — keep items the model judges as matching the criterion. Single batched LLM call; cheap by construction.

ai_map

`ai_map \@items, "instruction"` — apply instruction to each item, single batched LLM call. Returns arrayref of strings.

ai_classify

`ai_classify \@items, "hint", into => ["a", "b", "c"]` — assign one of the labels to each item.

ai_match

`ai_match` — operation `ai match`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_match $x
```

ai_sort

`ai_sort` — operation `ai sort`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_sort $x
```

ai_dedupe

`ai_dedupe` — operation `ai dedupe`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_dedupe $x
```

ai_extract

`ai_extract` "prompt", schema => +{ name => "string", age => "int" } — structured JSON output; returns a typed hashref.

ai_summarize

`ai_summarize` \$text, words => 50 — convenience wrapper that asks the model for a concise summary at target length.

ai_translate

`ai_translate` \$text, to => "Spanish" — convenience wrapper for translation.

ai_template

`ai_template` "hi {name}", name => "world" — pure string substitution. `{{` escapes to a literal `{`. Missing keys pass through unchanged.

ai_transcribe

`ai_transcribe` "audio.mp3", model => "whisper-1", language => "en" — OpenAI Whisper transcription. Multi-part upload to `/v1/audio/transcriptions`. Returns the transcript string. Defaults to `whisper-1`.

ai_speak

`ai_speak` \$text, voice => "alloy", model => "tts-1", format => "mp3", output => "out.mp3" — OpenAI TTS. Returns raw audio bytes; pass `output =>` to also write to disk. Voices: alloy, echo, fable, onyx, nova, shimmer.

ai_image

`ai_image` \$prompt, model => "dall-e-3", size => "1024x1024", quality => "hd", output => "out.png", n => 1 — image generation (OpenAI Images). Returns raw PNG bytes for `n=1`, or arrayref of bytes for `n>1`. Models: `dall-e-3`, `gpt-image-1`. `output` writes to disk (with `_1/_2/...` suffix for `n>1`).

ai_image_edit

`ai_image_edit` \$prompt, image => "in.png", mask => "mask.png", output => "out.png" — image-to-image edit via OpenAI `/v1/images/edits`. Optional mask is a PNG with transparent regions marking where to edit. Returns PNG bytes (or arrayref for `n>1`).

ai_image_variation

`ai_image_variation image => "in.png", n => 4, size => "1024x1024"` — generate variations of an existing image (OpenAI `/v1/images/variations`, DALL-E 2). Returns PNG bytes for `n=1` or arrayref for `n>1`.

ai_models

`ai_models($provider)` → arrayref of model IDs from the live catalog. Providers: `openai`, `anthropic`, `ollama`, `gemini`. Useful for tooling that surfaces the available models without hardcoding lists.

ai_citations

`ai_citations()` — arrayref of citations from the most recent Anthropic call (when `citations => 1` was passed). Each entry is a hashref with `text` (cited string), `title`, `start/end` char offsets or `start_page/end_page` page numbers, and `document_index`.

ai_dashboard

`ai_dashboard()` → ANSI-colored multi-line summary (cost, tokens in/out, embed tokens, prompt-cache write/read, result-cache hit ratio). Drop at the end of a script or during a session: `print ai_dashboard()`.

ai_pricing

`ai_pricing("claude-opus-4-7")` → hashref `{ model, input, output, input_per_1m, output_per_1m }` with the per-1K-token pricing the runtime uses for cost tracking. Useful for pre-flight cost estimates.

ai_describe

`ai_describe("image.png", style => "concise")` — vision wrapper that asks the model to describe an image. Styles: `concise` (one sentence), `detailed` (paragraph), `alt` (HTML alt-text ≤125 chars), or any custom suffix.

ai_grounded

`ai_grounded($prompt, documents => [@paths_or_strings], titles => [@titles])` — Anthropic-only multi-document grounding. Each entry can be a PDF path, a text-file path, or an inline string. Citations are auto-enabled; retrieve via `ai_citations()` after the call.

ai_moderate

`ai_moderate($text, model => "omni-moderation-latest")` → `{ flagged, categories, scores }`. OpenAI content-safety classifier. Free endpoint — no token cost. Use it before sending user-generated content to a generative model.

ai_chunk

`ai_chunk($text, max_tokens => 500, overlap => 50, by => "tokens"|"chars"|"sentences")` → arrayref of strings. Pure local logic — no API call. Sentence mode splits on `.!?` and greedily packs.

ai_warm

`ai_warm(model => ..., provider => ...)` → `{ ok, latency_ms, error, model, provider }`. Sends a 1-token ping so auth/network failures surface at script start instead of mid-flow.

ai_compare

`ai_compare($x, $y, criteria => "factual accuracy", scale => 5)` → `{ winner, reason, scores => {a, b}, raw }`. Single-call structured comparison. `winner` is `"a"`, `"b"`, or `"tie"`.

ai_session_export

`ai_session_export($handle)` → JSON string capturing system, model, provider, and full message log. Pair with `ai_session_import($json)` to restore.

ai_session_import

`ai_session_import($json)` → fresh session handle hashref. Inverse of `ai_session_export`. Allocates a new id and populates the session from the JSON body.

ai_file_upload

`ai_file_upload("path", purpose => "user_data")` → metadata hashref with `id`, `bytes`, `filename`, `purpose`, `created_at`. OpenAI Files API — `file_id` can be referenced by Whisper/Vision/Batch/Assistants.

ai_file_list

`ai_file_list(purpose => "...")` → arrayref of file metadata hashrefs from `/v1/files`.

ai_file_get

`ai_file_get($file_id)` → metadata hashref for a single file.

ai_file_delete

`ai_file_delete($file_id)` → 1 if deleted, 0 otherwise. Removes the file from OpenAI's storage.

ai_file_anthropic_upload

`ai_file_anthropic_upload("path/to/file.pdf")` → metadata hashref with `id`, `filename`, `mime_type`, `size_bytes`, `created_at`. Anthropic's beta `/v1/files` endpoint — `file_id` can be referenced from batch jobs and document blocks.

ai_file_anthropic_list

`ai_file_anthropic_list()` → arrayref of file-metadata hashrefs from Anthropic's `/v1/files`.

ai_file_anthropic_delete

`ai_file_anthropic_delete($file_id)` → 1 on success, 0 otherwise. Removes the file from Anthropic's storage.

ai_session_new

`ai_session_new(system => "...", model => "...")` — start a multi-turn chat session. Use `ai_session_send($s, "msg")` to advance, `ai_session_history($s)` to inspect, `ai_session_close($s)` to drop.

ai_session_send

`ai_session_send` — send of `ai session`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_session_send $x
```

ai_session_history

`ai_session_history` — operation `ai session history`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_session_history $x
```

ai_session_close

`ai_session_close` — close handle of `ai session`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_session_close $x
```

ai_session_reset

`ai_session_reset` — operation `ai session reset`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_session_reset $x
```

ai_memory_save

`ai_memory_save("id", "content", $metadata?, $path?)` — embed + persist a doc in the sqlite-backed memory store.

ai_memory_recall

`ai_memory_recall("query", top_k => 5)` — re-embed query, return top-k matches by cosine similarity. Each entry: `+{id, content, score, metadata}`.

ai_memory_forget

`ai_memory_forget` — operation `ai memory forget`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_memory_forget $x
```

ai_memory_count

`ai_memory_count` — count of `ai memory`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_memory_count $x
```

ai_memory_clear

`ai_memory_clear` — operation `ai memory clear`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_memory_clear $x
```

ai_vision

`ai_vision` — operation `ai vision`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_vision $x
```

ai_pdf

`ai_pdf` — operation `ai pdf`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_pdf $x
```

ai_budget

`ai_budget($usd_cap, fn { ... })` — runs the block; raises a runtime error if AI spend during the block exceeds `$usd_cap`. Restores prior global cap on exit.

ai_last_thinking

`ai_last_thinking` — operation `ai last thinking`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_last_thinking $x
```

ai_batch

`ai_batch` — operation `ai batch`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_batch $x
```

ai_pmap

`ai_pmap` — operation `ai pmap`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p ai_pmap $x
```

vec_cosine

`vec_cosine` `\@a`, `\@b` — cosine similarity in `[-1, 1]`. Both args are arrayrefs of floats (typical embedding shape).

vec_search

`vec_search` `\@query`, `\@candidates`, `top_k => N` — return top-k matches as `+{idx, score}`.

vec_topk

`vec_topk` — operation `vec topk`. Category: `*ai primitives (docs/ai_primitives.md)*`.

```
p vec_topk $x
```

web_search_tool

`web_search_tool()` — built-in tool spec for `tools => [...]`. Brave Search via `$BRAVE_SEARCH_API_KEY`, falls back to DuckDuckGo HTML scrape.

fetch_url_tool

`fetch_url_tool()` — built-in tool spec that fetches a URL and returns text.

read_file_tool

`read_file_tool()` — built-in tool spec that reads a file from disk.

run_code_tool

`run_code_tool()` — built-in tool spec that runs Python code in a subprocess (10s timeout, returns stdout+stderr).

tool

`tool fn NAME($p1: type, $p2: type) -> Type "docstring" { body }` — declare an agent-callable tool. Param types become the JSON Schema. Auto-registers via `ai_register_tool` so bare `ai($prompt)` calls see it.

MCP (Model Context Protocol)

13 topics

mcp_connect

`mcp_connect "stdio:CMD ARGS"` or `mcp_connect "https://server/mcp"` — connect to an MCP server. Speaks JSON-RPC over stdio (subprocess) or streamable HTTP. Returns a handle.

mcp_tools

`mcp_tools $h` — list the server's tools. Cached after first call.

mcp_resources

`mcp_resources` — operation `mcp resources`. Category: *mcp (model context protocol)*.

```
p mcp_resources $x
```

mcp_prompts

`mcp_prompts` — operation `mcp prompts`. Category: *mcp (model context protocol)*.

```
p mcp_prompts $x
```

mcp_call

`mcp_call $h, "tool_name", +{...args}` — invoke a tool on the connected server.

mcp_resource

`mcp_resource` — operation `mcp resource`. Category: *mcp (model context protocol)*.

```
p mcp_resource $x
```

mcp_prompt

`mcp_prompt` — operation `mcp prompt`. Category: *mcp (model context protocol)*.

```
p mcp_prompt $x
```

mcp_close

`mcp_close` — close handle of `mcp`. Category: *mcp (model context protocol)*.

```
p mcp_close $x
```

mcp_attach_to_ai

`mcp_attach_to_ai $h` — register a connected MCP server so subsequent bare `ai($prompt)` calls auto-include its tools.

mcp_detach_from_ai

`mcp_detach_from_ai` — operation `mcp detach from ai`. Category: `*mcp (model context protocol)*`.

```
p mcp_detach_from_ai $x
```

mcp_attached

`mcp_attached` — operation `mcp attached`. Category: `*mcp (model context protocol)*`.

```
p mcp_attached $x
```

mcp_server_start

`mcp_server_start` "name", `+{ tools => [...] }` — run a stdio JSON-RPC MCP server in-process. Pair with `s_web` build to ship a self-contained MCP server binary.

For parser-level sugar, use `mcp_server` "name" `{ tool foo($x) "..." {...} }`.

mcp_server

`mcp_server` "name" `{ tool NAME(args) "doc" { body } ... }` — declarative MCP server block. Compiles to `mcp_server_start("name", +{ tools => [...] })` with private helper fns for each tool body.

```
mcp_server "filesystem" {  
  tool read_file($path: string) "Read file contents" { slurp $path }  
  tool list_dir($path: string) "list dir entries" { join("\n", readdir $path) }  
}
```

Web framework (stryke_web/README.md)

112 topics

web_route

`web_route "VERB /path", "controller#action"` — register a route in the global router.

```
web_route "GET /posts", "posts#index"
web_route "POST /posts", "posts#create"
```

web_resources

`web_resources "posts"` — register the 7-route REST scaffold (index/show/new/create/edit/update/destroy) for a resource.

web_root

`web_root "posts#index"` — shortcut for `web_route "GET /"`.

web_application_config

`web_application_config` — operation `web application config`. Category: *web framework (docs/web_framework.md)*.

```
p web_application_config $x
```

web_render

`web_render html => "..."/text => .../json => .../template => "posts/index", locals => +{...}` — emit a response. Templates resolve `app/views/...html.erb`, run through the ERB engine, wrap in the application layout.

web_render_partial

`web_render_partial` — operation `web render partial`. Category: *web framework (docs/web_framework.md)*.

```
p web_render_partial $x
```

web_redirect

`web_redirect "/path", 302` — issue a 3xx redirect with `Location` header.

web_json

`web_json` — operation `web json`. Category: *web framework (docs/web_framework.md)*.

```
p web_json $x
```

web_text

`web_text` — operation `web text`. Category: *web framework (docs/web_framework.md)*.

```
p web_text $x
```

web_params

`web_params()` — hashref of query + form-urlencoded body + JSON body + path captures.

web_request

`web_request` — operation `web request`. Category: *web framework (docs/web_framework.md)*.

```
p web_request $x
```

web_routes_table

`web_routes_table` — lookup table of `web routes`. Category: *web framework (docs/web_framework.md)*.

```
p web_routes_table $x
```

web_boot_application

`web_boot_application` — operation `web boot application`. Category: *web framework (docs/web_framework.md)*.

```
p web_boot_application $x
```

web_db_connect

`web_db_connect "sqlite://path"` — open and cache the global SQLite connection. Required before any `web_model_*` call.

web_db_execute

`web_db_execute` — operation `web db execute`. Category: *web framework (docs/web_framework.md)*.

```
p web_db_execute $x
```

web_db_query

`web_db_query` — operation `web db query`. Category: *web framework (docs/web_framework.md)*.

```
p web_db_query $x
```

web_jobs_init

`web_jobs_init()` — create the SQLite `jobs` table if it doesn't exist. Idempotent.

web_job_enqueue

`web_job_enqueue("name", +{...args}, queue => "...", priority => N, max_attempts => N)` → job id (i64). Status starts as `pending`.

web_job_dequeue

`web_job_dequeue(queue => "...")` → `+{id, name, args_json, args, attempts, max_attempts}` or `undef`. Atomically marks the picked job as `running` and increments `attempts`.

web_job_complete

`web_job_complete($id)` — mark a running job as `done`.

web_job_fail

`web_job_fail($id, error => "msg")` — if `attempts < max_attempts`, returns to `pending` for retry; otherwise marked failed. Returns the new status.

web_jobs_list

`web_jobs_list(queue => "...", status => "...", limit => 50)` → arrayref of job hashrefs.

web_jobs_stats

`web_jobs_stats()` → `+{pending, running, done, failed}`.

web_job_purge

`web_job_purge(status => "done")` → number of rows deleted.

web_db_begin

`web_db_begin` — operation `web db begin`. Category: *web framework (docs/web_framework.md)*.

```
p web_db_begin $x
```

web_db_commit

`web_db_commit` — operation `web db commit`. Category: *web framework (docs/web_framework.md)*.

```
p web_db_commit $x
```

web_db_rollback

`web_db_rollback` — operation `web db rollback`. Category: *web framework (docs/web_framework.md)*.

```
p web_db_rollback $x
```

web_model_all

`web_model_all "posts"` — `SELECT * FROM posts ORDER BY id`. Returns arrayref of hashrefs.

web_model_find

`web_model_find "posts", $id` — find by id; returns hashref or undef.

web_model_where

`web_model_where` — operation `web model where`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_where $x
```

web_model_create

`web_model_create "posts", +{title => "...", body => "..."} — INSERT + return new row with id and auto-set created_at/updated_at.`

web_model_update

`web_model_update` — update step of `web model`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_update $x
```

web_model_destroy

`web_model_destroy` — operation `web model destroy`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_destroy $x
```

web_model_soft_destroy

`web_model_soft_destroy` — operation `web model soft destroy`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_soft_destroy $x
```

web_model_paginate

`web_model_paginate` — operation `web model paginate`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_paginate $x
```

web_model_search

`web_model_search` — operation `web model search`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_search $x
```

web_model_count

`web_model_count` — count of `web model`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_count $x
```

web_model_first

`web_model_first` — operation `web model first`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_first $x
```

web_model_last

`web_model_last` — operation `web model last`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_last $x
```

web_model_increment

`web_model_increment` — operation `web model increment`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_increment $x
```

web_model_with

`web_model_with` — operation `web model with`. Category: *web framework (docs/web_framework.md)*.

```
p web_model_with $x
```

web_create_table

`web_create_table` — lookup table of `web create`. Category: *web framework (docs/web_framework.md)*.

```
p web_create_table $x
```

web_drop_table

`web_drop_table` — lookup table of `web drop`. Category: *web framework (docs/web_framework.md)*.

```
p web_drop_table $x
```

web_add_column

`web_add_column` — operation `web add column`. Category: *web framework (docs/web_framework.md)*.

```
p web_add_column $x
```

web_remove_column

`web_remove_column` — operation `web remove column`. Category: *web framework (docs/web_framework.md)*.

```
p web_remove_column $x
```

web_migrate

`web_migrate()` — apply every `up` block from `db/migrate/*.stk` whose version isn't yet in `schema_migrations`.

web_rollback

`web_rollback` — operation `web rollback`. Category: *web framework (docs/web_framework.md)*.

```
p web_rollback $x
```

web_validate

`web_validate $attrs, +{title => "presence,length:1..100", email => "format:^.+@.+$"} — returns +{ok => 1} or +{ok => 0, errors => +{...}}. Validators: presence, length:MIN..MAX, format:REGEX, numericality, inclusion:a|b|c, confirmation:other_field.`

web_permit

`web_permit` — operation `web permit`. Category: *web framework (docs/web_framework.md)*.

```
p web_permit $x
```

web_password_hash

`web_password_hash $pw` returns a `web1$saltHex$digest` shaped string. `web_password_verify $pw, $stored` returns 1/0. SHA-256 + 16-byte salt; constant-time compare.

web_session

`web_session` — operation `web session`. Category: *web framework (docs/web_framework.md)*.

```
p web_session $x
```

web_session_set

Server-side session storage backed by a signed cookie. `web_session_set("user_id", 42)/web_session_get("user_id")`. Cookie value is base64-JSON; HttpOnly + SameSite=Lax by default.

web_session_clear

`web_session_clear` — operation `web session clear`. Category: *web framework (docs/web_framework.md)*.

```
p web_session_clear $x
```

web_set_cookie

`web_set_cookie` — operation `web set cookie`. Category: *web framework (docs/web_framework.md)*.

```
p web_set_cookie $x
```

web_cookies

`web_cookies` — operation `web cookies`. Category: *web framework (docs/web_framework.md)*.

```
p web_cookies $x
```

web_flash

`web_flash` — operation `web flash`. Category: *web framework (docs/web_framework.md)*.

```
p web_flash $x
```

web_flash_set

`web_flash_set` — operation `web flash set`. Category: *web framework (docs/web_framework.md)*.

```
p web_flash_set $x
```

web_flash_get

`web_flash_get` — operation `web flash get`. Category: *web framework (docs/web_framework.md)*.

```
p web_flash_get $x
```

web_jwt_encode

`web_jwt_encode +{user_id => 1, role => "admin"}` — HS256-sign a JWT using `secret_key_base` from `web_application_config`.

web_jwt_decode

`web_jwt_decode $token` — verify + decode an HS256 JWT. Returns the payload hashref or undef on bad signature.

web_bearer_token

`web_bearer_token` — operation `web bearer token`. Category: *web framework (docs/web_framework.md)*.

```
p web_bearer_token $x
```


web_otp_secret

TOTP / 2FA — RFC 6238, 30-second window, ± 1 step skew tolerance. `web_otp_secret()` returns a base32 secret to share with the authenticator app; `web_otp_generate($secret)` returns the current 6-digit code; `web_otp_verify($secret, $code)` returns 1/0.

web_token_for

`web_token_for` — operation `web token for`. Category: *web framework (docs/web_framework.md)*.

```
p web_token_for $x
```

web_token_consume

`web_token_consume` — operation `web token consume`. Category: *web framework (docs/web_framework.md)*.

```
p web_token_consume $x
```

web_can

`web_can` — operation `web can`. Category: *web framework (docs/web_framework.md)*.

```
p web_can $x
```

web_signed

`web_signed` — operation `web signed`. Category: *web framework (docs/web_framework.md)*.

```
p web_signed $x
```

web_unsigned

`web_unsigned` — operation `web unsigned`. Category: *web framework (docs/web_framework.md)*.

```
p web_unsigned $x
```

web_cache_get

`web_cache_get` — operation `web cache get`. Category: *web framework (docs/web_framework.md)*.

```
p web_cache_get $x
```

web_cache_set

`web_cache_set` — operation `web cache set`. Category: *web framework (docs/web_framework.md)*.

```
p web_cache_set $x
```

web_cache_delete

`web_cache_delete` — operation `web cache delete`. Category: *web framework (docs/web_framework.md)*.

```
p web_cache_delete $x
```

web_cache_clear

`web_cache_clear` — operation `web cache clear`. Category: *web framework (docs/web_framework.md)*.

```
p web_cache_clear $x
```

web_t

`web_t` — operation `web t`. Category: *web framework (docs/web_framework.md)*.

```
p web_t $x
```

web_load_locale

`web_load_locale` — operation `web load locale`. Category: *web framework (docs/web_framework.md)*.

```
p web_load_locale $x
```

web_log

`web_log` — operation `web log`. Category: *web framework (docs/web_framework.md)*.

```
p web_log $x
```

web_set_header

`web_set_header` — operation `web set header`. Category: *web framework (docs/web_framework.md)*.

```
p web_set_header $x
```

web_status

`web_status` — operation `web status`. Category: *web framework (docs/web_framework.md)*.

```
p web_status $x
```

web_uuid

`web_uuid` — operation `web uuid`. Category: *web framework (docs/web_framework.md)*.

```
p web_uuid $x
```

web_now

`web_now` — operation `web now`. Category: *web framework (docs/web_framework.md)*.

```
p web_now $x
```

web_security_headers

`web_security_headers` — operation `web security headers`. Category: *web framework (docs/web_framework.md)*.

```
p web_security_headers $x
```

web_openapi

`web_openapi` — operation `web openapi`. Category: *web framework (docs/web_framework.md)*.

```
p web_openapi $x
```

web_etag

`web_etag` — operation `web etag`. Category: *web framework (docs/web_framework.md)*.

```
p web_etag $x
```

web_csv

`web_csv` — operation `web csv`. Category: *web framework (docs/web_framework.md)*.

```
p web_csv $x
```

web_markdown

`web_markdown` — operation `web markdown`. Category: *web framework (docs/web_framework.md)*.

```
p web_markdown $x
```

web_md

`web_md` — operation `web md`. Alias for `web_markdown`. Category: *web framework (docs/web_framework.md)*.

```
p web_md $x
```

web_content_for

`web_content_for` — operation `web content for`. Category: *web framework (docs/web_framework.md)*.

```
p web_content_for $x
```

web_yield_content

`web_yield_content` — operation `web yield content`. Category: *web framework (docs/web_framework.md)*.

```
p web_yield_content $x
```

web_jsonapi_resource

`web_jsonapi_resource` — operation `web jsonapi resource`. Category: *web framework (docs/web_framework.md)*.

```
p web_jsonapi_resource $x
```

web_jsonapi_collection

`web_jsonapi_collection` — operation `web jsonapi collection`. Category: *web framework (docs/web_framework.md)*.

```
p web_jsonapi_collection $x
```

web_jsonapi_error

`web_jsonapi_error` — operation `web jsonapi error`. Category: *web framework (docs/web_framework.md)*.

```
p web_jsonapi_error $x
```

web_rate_limit

`web_rate_limit` — operation `web rate limit`. Category: *web framework (docs/web_framework.md)*.

```
p web_rate_limit $x
```

web_before_action

`web_before_action` — operation `web before action`. Category: *web framework (docs/web_framework.md)*.

```
p web_before_action $x
```

web_after_action

web_after_action — operation `web after action`. Category: *web framework (docs/web_framework.md)*.

```
p web_after_action $x
```

web_h

web_h — operation `web h`. Category: *web framework (docs/web_framework.md)*.

```
p web_h $x
```

web_link_to

web_link_to — operation `web link to`. Category: *web framework (docs/web_framework.md)*.

```
p web_link_to $x
```

web_button_to

web_button_to — operation `web button to`. Category: *web framework (docs/web_framework.md)*.

```
p web_button_to $x
```

web_form_with

web_form_with — operation `web form with`. Category: *web framework (docs/web_framework.md)*.

```
p web_form_with $x
```

web_form_close

web_form_close — close handle of `web form`. Category: *web framework (docs/web_framework.md)*.

```
p web_form_close $x
```

web_text_field

web_text_field — operation `web text field`. Category: *web framework (docs/web_framework.md)*.

```
p web_text_field $x
```

web_text_area

web_text_area — operation `web text area`. Category: *web framework (docs/web_framework.md)*.

```
p web_text_area $x
```

web_check_box

web_check_box — operation `web check box`. Category: *web framework (docs/web_framework.md)*.

```
p web_check_box $x
```

web_csrf_meta_tag

web_csrf_meta_tag — operation web csrf meta tag. Category: *web framework (docs/web_framework.md)*.

```
p web_csrf_meta_tag $x
```

web_stylesheet_link_tag

web_stylesheet_link_tag — operation web stylesheet link tag. Category: *web framework (docs/web_framework.md)*.

```
p web_stylesheet_link_tag $x
```

web_javascript_link_tag

web_javascript_link_tag — operation web javascript link tag. Category: *web framework (docs/web_framework.md)*.

```
p web_javascript_link_tag $x
```

web_image_tag

web_image_tag — operation web image tag. Category: *web framework (docs/web_framework.md)*.

```
p web_image_tag $x
```

web_truncate

web_truncate — operation web truncate. Category: *web framework (docs/web_framework.md)*.

```
p web_truncate $x
```

web_pluralize

web_pluralize — operation web pluralize. Category: *web framework (docs/web_framework.md)*.

```
p web_pluralize $x
```

web_time_ago_in_words

web_time_ago_in_words — operation web time ago in words. Category: *web framework (docs/web_framework.md)*.

```
p web_time_ago_in_words $x
```

web_faker_name

web_faker_name — operation web faker name. Category: *web framework (docs/web_framework.md)*.

```
p web_faker_name $x
```

web_faker_email

web_faker_email — operation web faker email. Category: *web framework (docs/web_framework.md)*.

```
p web_faker_email $x
```

web_faker_sentence

`web_faker_sentence` — operation `web faker sentence`. Category: *web framework (docs/web_framework.md)*.

```
p web_faker_sentence $x
```

web_faker_paragraph

`web_faker_paragraph` — operation `web faker paragraph`. Category: *web framework (docs/web_framework.md)*.

```
p web_faker_paragraph $x
```

web_faker_int

`web_faker_int` — operation `web faker int`. Category: *web framework (docs/web_framework.md)*.

```
p web_faker_int $x
```

Stress / telemetry (docs/STRESS_TESTING.md)

42 topics

stress_fp

`stress_fp $secs` — float matrix-multiply pinning every core. Returns total FLOP count.

stress_int

`stress_int $secs` — integer pipeline pinning every core.

stress_cache

`stress_cache $secs, $kb` — pound a working set of `kb` KiB per core in random order. Default 1024 (~ L2/L3 boundary).

stress_branch

`stress_branch` — operation `stress branch`. Category: *stress / telemetry extensions*.

```
p stress_branch $x
```

stress_sort

`stress_sort` — operation `stress sort`. Category: *stress / telemetry extensions*.

```
p stress_sort $x
```

stress_alloc

`stress_alloc` — operation `stress alloc`. Category: *stress / telemetry extensions*.

```
p stress_alloc $x
```

stress_mmap

`stress_mmap` — operation `stress mmap`. Category: *stress / telemetry extensions*.

```
p stress_mmap $x
```

stress_disk

`stress_disk $path, $secs, $mb_per_write` — sustained sequential write + fsync + read across all cores.

stress_iops

`stress_iops $path, $secs, $block_kb` — small random reads/writes for IOPS testing. Default 4KB blocks.

stress_net

`stress_net "host:port", $secs, $conns` — open `conns` TCP connections per core, pump bytes.

stress_http

`stress_http $url, $secs, $conns` — HTTP GET storm. Counts 2xx responses.

stress_dns

`stress_dns` — operation `stress dns`. Category: *stress / telemetry extensions*.

```
p stress_dns $x
```

stress_thread

`stress_thread` — operation `stress thread`. Category: *stress / telemetry extensions*.

```
p stress_thread $x
```

stress_fork

`stress_fork` — operation `stress fork`. Category: *stress / telemetry extensions*.

```
p stress_fork $x
```

stress_aes

`stress_aes $secs` — AES-128 encrypt round-trip across cores.

stress_compress

`stress_compress $secs, $kb` — gzip compress+decompress payload of `kb` KiB per core.

stress_regex

`stress_regex` — operation `stress regex`. Category: *stress / telemetry extensions*.

```
p stress_regex $x
```

stress_json

`stress_json` — operation `stress json`. Category: *stress / telemetry extensions*.

```
p stress_json $x
```

stress_burst

`stress_burst` — operation `stress burst`. Category: *stress / telemetry extensions*.

```
p stress_burst $x
```

stress_ramp

`stress_ramp` — operation `stress ramp`. Category: *stress / telemetry extensions*.

```
p stress_ramp $x
```


stress_oscillate

`stress_oscillate` — operation `stress oscillate`. Category: *stress / telemetry extensions*.

```
p stress_oscillate $x
```

stress_all

`stress_all $secs` — run every stress kernel above in parallel for `secs`. Returns a hashref of per-kernel counts.

stress_cores

`stress_cores` — operation `stress cores`. Category: *stress / telemetry extensions*.

```
p stress_cores $x
```

stress_temp

Telemetry helpers: `stress_temp()` reads CPU temp from `/sys/class/thermal` (Linux), `stress_freq()` averages cpufreq across cores, `stress_throttled()` returns 1 when freq < 80% of max AND temp > 80°C, `stress_load()` returns `+{m1, m5, m15}`, `stress_meminfo()` returns total/used/free/swap bytes.

stress_thermal_zones

`stress_thermal_zones` — operation `stress thermal zones`. Category: *stress / telemetry extensions*.

```
p stress_thermal_zones $x
```

stress_arm_kill_switch

`stress_arm_kill_switch` — operation `stress arm kill switch`. Category: *stress / telemetry extensions*.

```
p stress_arm_kill_switch $x
```

stress_killed

`stress_killed` — operation `stress killed`. Category: *stress / telemetry extensions*.

```
p stress_killed $x
```

stress_disarm_kill_switch

`stress_disarm_kill_switch` — operation `stress disarm kill switch`. Category: *stress / telemetry extensions*.

```
p stress_disarm_kill_switch $x
```

stress_metrics_record

`stress_metrics_record("name", $value, k1 => v1, ...)` — append a sample to the metrics history. Labels are stringified.

stress_metrics_clear

`stress_metrics_clear()` — wipe the metrics history. Pair with `stress_metrics_count()` before/after to verify.

stress_metrics_count

`stress_metrics_count()` → current number of samples in the history.

stress_metrics_export

`stress_metrics_export("path", format => "csv"|"json"|"prom")` → number of samples written.

stress_metrics_prometheus

`stress_metrics_prometheus()` → string in Prometheus text exposition format. Latest value per (name, labels) wins. Drop into a `/metrics` HTTP handler.

stress_metrics_json

`stress_metrics_json()` → JSON string of the entire metrics history.

stress_metrics_csv

`stress_metrics_csv()` → CSV string with `ts_ms,value,labels` columns.

stress_metrics_watch

`stress_metrics_watch(field => "stress_temp", interval_ms => 1000, max_ticks => 60)` → arrayref of values sampled at the requested cadence. Polls the metrics history for new entries matching `field`.

audit_log

`audit_log("event_name", k1 => v1, ...)` — append one JSONL line to `~/.stryke/audit.log` (or `$STRYKE_AUDIT_LOG`). Never throws; returns 1/0.

audit_log_path

`audit_log_path()` → absolute path the next `audit_log` call will write to.

secrets_encrypt

`secrets_encrypt($plain, key => $key)` → base64 envelope (nonce(12) || ciphertext || tag(16)). AES-256-GCM. Key must be 32 raw bytes or 32-byte base64.

secrets_decrypt

`secrets_decrypt($envelope, key => $key)` → plaintext, or undef on auth failure.

secrets_random_key

`secrets_random_key()` → fresh 32-byte AES-256 key as base64. Reads `/dev/urandom` on Unix.

secrets_kdf

`secrets_kdf($password, salt => "...", iterations => 600_000)` → 32-byte key as base64. PBKDF2-HMAC-SHA256.

PTY / expect (docs/expect-feature-idea.md)

14 topics

pty_spawn

`pty_spawn "cmd" args` or `pty_spawn("cmd", "arg1", "arg2")` — allocate a PTY, fork+exec the child, return a handle. Master fd is non-blocking.

```
val $h = pty_spawn("ssh user@host");
pty_expect($h, qr/password:/, 30);
pty_send($h, "hunter2\n");
pty_close($h);
```

pty_send

`pty_send $h, "text"` — write to the PTY master. Handles EAGAIN/EINTR; returns bytes written.

pty_read

`pty_read` — read of `pty`. Category: *mcp (model context protocol)*.

```
p pty_read $x
```

pty_expect

`pty_expect $h, qr/regex/, $timeout_secs` — loop reading until the regex matches the accumulated buffer or `timeout_secs` elapses. Returns the matched substring or undef on timeout/EOF. Buffer is consumed up to (and including) the match.

pty_expect_table

`pty_expect_table $h, [{re => qr/.../, do => fn {...}}, ...], $timeout` — multi-pattern dispatch (Tcl `expect { ... }` block). First match wins; the matched branch's `do` block is called.

pty_buffer

`pty_buffer` — operation `pty buffer`. Category: *mcp (model context protocol)*.

```
p pty_buffer $x
```

pty_alive

`pty_alive` — operation `pty alive`. Category: *mcp (model context protocol)*.

```
p pty_alive $x
```

pty_eof

`pty_eof` — operation `pty eof`. Category: *mcp (model context protocol)*.

```
p pty_eof $x
```

pty_close

`pty_close $h` — SIGTERM → 200ms grace → SIGKILL. Returns exit status.

pty_stream

`pty_stream $h, $timeout?` — non-interactive `expect eof`: echo the child's remaining output to stdout until it exits (or the optional timeout), then reap it and return the exit status. Use for running a spawned command to completion after answering its prompts.

pty_interact

`pty_interact $h` — raw-mode handoff: forwards stdin → master and master → stdout until EOF or Ctrl-]. Saves and restores tty state.

pty_strip_ansi

`pty_strip_ansi $text` → string with CSI/OSC/ESC sequences removed. Useful before regex-matching against PTY output that mixes color codes into prompts.

pty_after_eof

`pty_after_eof $h, "callback_name"` — register an EOF callback. Spawns a watcher thread that flips a fired flag once the PTY closes. Pair with `pty_pending_events()` in your main loop.

pty_pending_events

`pty_pending_events()` → arrayref of `+{handle_id, callback}` for any `pty_after_eof` registrations that have fired since the last call. Drains the fired list.

Other

8607 topics

\$!

`$!` — the OS error from the most recent system call, in stringified form (e.g. "No such file or directory") or as an `errno` integer in numeric context. Set by `open`, `close`, `chdir`, `read`, `write`, and other syscalls when they fail. The value persists until the next failing call, so always check it **immediately** after the failing operation — intermediate clean Perl code does not touch it, but any subsequent syscall will.

```
open val $fh, '<', $path or die "open $path: $!"
unless (chdir $dir) { warn "chdir: $!"; return }
# Numeric context for errno-based dispatch:
use Errno qw(ENOENT)
if ($! == ENOENT) { p "missing" }
```

See also: `$?` (child exit), `$@` (eval error), `$^E` (extended OS error).

\$"

`$"` — the list separator used when an array variable is interpolated inside a double-quoted string. Defaults to a single space. Setting it changes how `"@array"` renders — useful for ad-hoc CSV-like joining without `join`.

```
val @xs = (1, 2, 3)
p "@xs"          # 1 2 3
local $" = ", "
p "@xs"          # 1,2,3
local $" = " | "
p "row: @xs"     # row: 1 | 2 | 3
```

Only affects double-quoted interpolation, not bare `print @xs` (which uses `$,`). Use `local $"` to scope the change.

\$\$

`$$` — the process ID of the running stryke interpreter. Read-only, set once at startup. Useful for PID files, log filenames, and child/parent disambiguation after `fork`.

```
p "started, pid=$$"
open val $pf, '>', "/tmp/myapp.pid" or die $!
print $pf "$$\n"
close $pf

if (val $kid = fork()) {
  p "parent $$ spawned $kid"
} else {
  p "child $$ running"
}
```

See also: `getppid()` for the parent PID, `fork()` to spawn children.

\$&

`$&` — the entire string matched by the most recent successful regex match in the current scope. Reset on every match. In Perl 5 $\geq$ 5.20 this no longer carries the historical performance penalty (the pre-5.20 issue forced compilation of a slower regex engine globally); stryke's regex engine has no such penalty.

```
if ("hello world" =~ /\w+/) {
  p $&      # hello
}
var $s = "abc123def"
if ($s =~ /\d+/) { p $&; } # 123
```

See also: `$` (prematch), `$'` (postmatch), `$1..$9` (capture groups), `@-` / `@+` (offsets), `${^MATCH}` / `${^PREMATCH}` / `${^POSTMATCH}` (no-penalty equivalents under use re 'p').

`$'`

`$'` — the substring *following* the last successful regex match ("postmatch"). Combined with `$` (prematch) and `$&` (match), reconstructs the entire input: `"$ $&$"` eq `$original`. Reset on every match.

```
if ("hello world" =~ /\s/) {
  p "pre = $"      # pre = hello
  p "match= $&"    # match= (a space)
  p "post = $('"    # post = world
}
```

See also: `$` , `$&`, `${^POSTMATCH}` (cleaner under use re 'p').

`$(`

`$(` — the real group ID of the running process. In list context (or when stringified with whitespace) yields the primary GID followed by every supplementary group, space-separated. Use `getgrgid($(` to map a GID to its name. Assigning to `$(` requires appropriate privileges.

```
p "real gid: $( "      # 1000 1000 4 24 27 ...
var ($primary, @supp) = split ' ', $(
p "primary group: $primary"
use POSIX qw(getgrgid)
p scalar getgrgid($primary) || $primary
```

See also: `$)` (effective gid), `$<` (real uid), `$>` (effective uid).

`$)`

`$)` — the effective group ID of the running process. Like `$(`, returns the primary EGID plus supplementary groups space-separated in stringified form. Affects which permissions apply to filesystem operations. Assigning to `$)` requires privileges (typically root).

```
p "effective gid: $)"
if ($( != $)) {
  p "running setgid: real=$( effective=$)"
}
```

See also: `$(` (real gid), `$>` (effective uid).

`$+`

`$+` — the contents of the **last** parenthesised capture group that successfully matched in the most recent regex. When the regex has multiple (...) groups, this gives you the rightmost one that participated in the match — useful when alternation may match different groups depending on input.

```
if ("name=alice" =~ /^(\\w+)= (\\w+)$/) {
  p $+      # alice (last group)
}
if ("foo" =~ /(bar)|(foo)|(baz)/) {
  p $+      # foo (only the second group matched)
}
```

See also: `$1..$9` (positional captures), `%+` (named captures), `@+` (end-of-match offsets).

`$,`

`$,` — the output field separator inserted between arguments by `print` (default: empty string). Does *not* affect `printf/sprintf` (those use the format string) or `say/p` (which still respect it). Setting `$,` is the classic way to print comma-separated lists without an explicit `join`.

```
print 1, 2, 3, "\n"      # 123
local $, = ","
print 1, 2, 3, "\n"      # 1,2,3,
local $, = " | "
print qw(a b c), "\n"    # a | b | c |
```

See also: `$\` (output record separator), `$"` (list interpolation separator).

`$.`

`$.` — the current input line number of the most recently read filehandle, as tracked by `<>`, `<$fh>`, and `readline`. Counts lines as defined by `$/` (input record separator). Reset to 0 when a filehandle is closed (but **not** when `<>` rolls onto the next ARGV file — line numbers continue across files unless you reset it manually).

```
while (<>) {
  p "$ARGV:.$" if /error/i
  close ARGV if eof # reset $. between input files
}
```

See also: `$ARGV` (current filename), `eof` (file boundary detection).

`$/`

`$/` — the input record separator used by `<>/readline`. Default: `"\n"` (one line per read). Set to `undef` to slurp the whole file in one read; set to `" "` for **paragraph mode** (records separated by runs of blank lines); set to a reference to a positive integer for fixed-size block reads.

```
local $/ = undef          # slurp
var $whole = <$fh>

local $/ = " "            # paragraph mode
while (val $para = <$fh>) {
  p length $para
}

local $/ = \4096          # 4KB blocks
while (read $fh, val $blk, 4096) { ... }
```

Always `local`-ise the change so it doesn't leak. See also: `$\` (output record separator), `$,`.

`$0`

`$0` — the program name as the OS sees it. Initialised from `argv[0]`. Assignment to `$0` updates the visible process name on many Unix systems (Linux, BSDs), making `ps` / `top` show the new value — handy for long-running daemons that want to reflect what they're doing.

```
p "running as: $0"
$0 = "myapp worker [idle]"
sleep 5
$0 = "myapp worker [processing job 42]"
```

See also: `@ARGV` (the rest of the command line), `__FILE__` (compile-time source path).

`$1`

`$1` — the substring matched by the **first** parenthesised capture group of the most recent successful regex. Undef if the group didn't participate in the match or if the match failed.

```
if ("alice@example.com" =~ /^(\\w+)@(\\S+)$/) {
    p $1      # alice
    p $2      # example.com
}
"ip: 192.168.1.1" =~ /(\\d+\\.\\d+\\.\\d+\\.\\d+)/ and p $1
```

See also: `$2..$9, %+` (named captures via `(?<name>...)`), `@-` / `@+` (offsets), `$+` (last successful group).

\$2

`$2` — second capture group from the most recent regex match. See `$1` for full semantics.

\$3

`$3` — third capture group from the most recent regex match. See `$1` for full semantics.

\$4

`$4` — fourth capture group from the most recent regex match. See `$1` for full semantics.

\$5

`$5` — fifth capture group from the most recent regex match. See `$1` for full semantics.

\$6

`$6` — sixth capture group from the most recent regex match. See `$1` for full semantics.

\$7

`$7` — seventh capture group from the most recent regex match. See `$1` for full semantics.

\$8

`$8` — eighth capture group from the most recent regex match. See `$1` for full semantics.

\$9

`$9` — ninth capture group from the most recent regex match. Captures past `$9` work fine in stryke (no Perl 4 limit), but the conventional way to access many groups is named captures (`(?<name>...)` + `%+`) or `@-`/`@+` offsets.

\$;

`$;` — the multidimensional-array subscript separator. When you write `$h{a,b,c}` Perl joins the keys with `$;` (default: `\034`, the ASCII FS / file-separator control character) to produce a single string key. This emulates multi-dim arrays on top of the flat hash store. Rarely used in new code — prefer real nested data structures (`$h{a}{b}{c}` or arrayrefs).

```
$grid{1,2} = "x"      # really $grid{"1\034 2"} = "x"
p exists $grid{1,2}   # 1
local $; = "|"        # if you want readable keys for debugging
```

See also: prefer `$h{a}{b}{c}` autovivification for nested storage.

\$<

`$<` — the real user ID of the running process. Read-only on most systems unless the process has appropriate privileges. Use `getpwuid($<)` to map to a username.


```
p "real uid: $<"
use POSIX qw(getpwuid)
p "running as: " . (getpwuid($<))[0]
if ($< != $>) {
    p "setuid: real=$< effective=$>"
}
```

See also: `$>` (effective uid), `$( / $)` (gids).

`$>`

`$>` — the effective user ID of the running process. Determines filesystem permission checks. Setuid binaries start with `$<` (real) $\neq$ `$>` (effective). Assignment requires privileges; swapping `$<` and `$>` is a common pattern for setuid scripts that want to temporarily drop privileges.

```
p "effective uid: $>"
($<, $>) = ($>, $<) # drop to real uid (where supported)
```

See also: `$<`, `$( / $)`.

`$?`

`$?` — the status of the last child process: a backtick command, `system()` call, or piped-`open` close. The raw value packs the exit code, signal number, and core-dump flag into a single integer. Extract the exit code with `$? >> 8`; the signal with `$? & 127`; the core-dump flag with `$? & 128`.

```
val $out = `ls /missing 2>&1`
if ($?) {
    val $exit = $? >> 8
    val $sig = $? & 127
    die "ls failed: exit=$exit sig=$sig\n"
}

system('grep', $pat, $file) == 0 or warn "grep: $? >> 8"
```

See also: `$!` (syscall error), `system`, `exec`, `wait`.

`$@`

`$@` — the error message left by the most recent `eval { }` (or `die` inside one). Empty string when the eval succeeded; non-empty string when it caught an exception. For structured errors `died` as references, `$@` holds the reference itself, not a string.

Classic gotcha: any code between `eval` and `if ($@)` that runs another `eval` (including destructors and `END` blocks) can clobber `$@`. Save it locally if you need it later. In `stryke`, prefer the `try/catch` statement which gives you a properly scoped exception variable and avoids this hazard entirely.

```
eval { dangerous_call() }
if (val $err = $@) {
    warn "caught: $err"
}

# Modern: try/catch - no $@ clobbering
try { dangerous_call() } catch ($e) { warn "caught: $e" }
```

See also: `try`, `catch`, `eval`, `die`.

`$ARGV`

`$ARGV` — the name of the file currently being read via the diamond operator (`<>`) when iterating over `@ARGV`. Set automatically each time `<>` opens the next argv file. Useful inside loops that grep across many files to know which one produced a hit.

```
while (<>) {
    p "$ARGV:$.: $_" if /TODO/
    close ARGV if eof # reset $. between files
}
```

See also: `@ARGV` (the list of files), `$.` (line number).

\$NF

`$NF` — awk-compatible: the number of fields produced by the most recent autosplit of an input line into `@F` (`-a` flag). Equivalent to `scalar @F`. `stryke` exposes both for one-liner familiarity to awk users.

```
# stryke -F: -nae 'p "$NF fields: @F"' /etc/passwd
# Equivalent to awk '{ print NF" fields: "$0 }'
```

See also: `$NR`, `@F`, `-a` / `-F` command-line flags, awk-compatible one-liner mode.

\$NR

`$NR` — awk-compatible: the current input record number. Equivalent to `$.` (current line of the most recently read filehandle). `stryke` exposes both for one-liner familiarity to awk users. `$.`-based code and `$NR`-based code can be mixed in the same one-liner.

```
# stryke -nae 'p "$NR: $_" if /TODO/'
# Equivalent to awk '/TODO/ { print NR: "$0 }'
```

See also: `$.`, `$NF`, `@F`, `-a` / `-F`, `-n` / `-p`.

\$\

`$\` — the output record separator appended automatically by `print` (default: empty string). Pairs with `$.` (field separator between args). Useful for switching between Unix `\n` and Windows `\r\n` line endings without rewriting every `print`. Does not affect `say/p` (which always appends `\n`) or `printf` (which uses the format string).

```
local $\ = "\n"
print "hello"      # hello\n
print "world"      # world\n

local $\ = "\r\n"
print "crlf line"  # crlf line\r\n
```

See also: `$.` (output field separator), `$/` (input record separator).

\$^A

`$^A` — the accumulator for `format/write` output. Each `formline` call appends formatted text here; `write` flushes the accumulator to the selected filehandle and resets it. Direct manipulation is rare except when implementing custom report writers without `write`.

```
formline '@<<<<<<< @>>>>>' , "name", 42
p $^A      # "name", 42"
$^A = ''   # clear
```

See also: `format`, `formline`, `write`.

\$^C

`$^C` — true when Perl/stryke is running in `*compile-only*` mode (`-c` flag). Useful for guarding code that should be skipped during syntax checking (e.g. tests that would actually run, network calls). Read-only.

```
BEGIN { exit 0 if $^C } # don't run setup during -c
```

See also: `-c` command-line flag.

`$$D`

`$$D` — the debug flag bitmask (`-D` command-line flag in debug-enabled builds). Stryke debug builds expose the same flag set as Perl; production builds have it stubbed. Numeric, set from the `-D` digits or letters on the command line.

```
p "debug mask: $$D" if $$D
```

See also: `-D` flag, [use diagnostics](#).

`$$E`

`$$E` — extended OS error info beyond `$$!`. On Windows this carries Win32 error codes / messages distinct from the POSIX `errno` that `$$!` reports. On Unix it is usually identical to `$$!`. Useful for platform-specific diagnostics.

```
open val $fh, '<', $path or die "open: $$! ($$E)"
```

See also: `$$!`.

`$$F`

`$$F` — the maximum file descriptor considered "system" (default: 2). File descriptors at or below this number are *not* closed across `exec()` calls; higher fds are closed automatically. Raise it before `exec`-ing a child if you need extra fds to survive.

```
$$F = 10 # keep fds 0-10 across exec
open val $fh, '<', $path or die $$!
exec 'helper', fileno($fh)
```

See also: `exec`, `fcntl`.

`$$H`

`$$H` — the compile-time lexical hint bitmask (32 bits of integer flags). Pragmas like [use strict](#), [use warnings](#), [use utf8](#) flip bits here to communicate their state to the parser within a lexical scope. Reading at runtime is mostly useless (it reflects current-line compile state); manipulation is for pragma authors.

```
BEGIN { p sprintf "hint bits: 0x%08x", $$H }
```

See also: `$$H` (lexical hint hash), [use strict](#), [use warnings](#).

`$$I`

`$$I` — the in-place edit extension. Set by the `-i` command-line flag and controls `<>` rewriting: when `$$I` is defined, lines read from `@ARGV` files are written to a backup-named or in-place replacement. Empty string `""` means "no backup"; a non-empty value (e.g. `".bak"`) means "save original as `FILE.bak`".

```
# Equivalent to: stryke -i.bak -pe 's/foo/bar/g' file.txt
BEGIN { $$I = '.bak' }
while (<>) { s/foo/bar/g; print }
```

See also: `-i` flag, `-p` / `-n`.

`$$L`

`$$L` — the formfeed character emitted between pages by [format/write](#). Default: `"\f"`. Rarely changed; mainly useful when emitting paginated output to a printer-like destination.

```
local $$L = "\n---\n"
write # custom page break
```

See also: [format](#), [write](#).

`$$M`

`^M` — an emergency memory pool reserved for handling out-of-memory [dies](#). Pre-allocate a chunk early (e.g. `^M = 'x' x 65536`), and Perl will release it when allocation otherwise fails so cleanup code has room to run. Niche; only relevant in long-running daemons that might OOM.

```
BEGIN { ^M = 'x' x 65536 } # 64 KB emergency reserve
```

See also: [die](#), [END](#) block.

`^N`

`^N` — the substring matched by the most recently **closed** parenthesised group in the current regex match. Inside an embedded code assertion (`{ ... }`) this lets you grab the group that just closed without committing to a `$1/$2` number.

```
"abc" =~ /(.){}(.){}( ){ p ^N }/ # prints c
```

See also: `$+`, `$1..$9`, (`{ ... }`) embedded code.

`^O`

`^O` — the OS name string set at build time. Common values: [darwin](#) (macOS), [linux](#), [freebsd](#), [MSWin32](#), [cygwin](#). Use it for platform-specific branching without shelling out to [uname](#). Read-only.

```
if (^O eq 'darwin') {  
  p "on macOS"  
}  
elsif (^O eq 'linux') {  
  p "on linux"  
}  
elsif (^O =~ /MSWin32|cygwin/) {  
  p "on Windows"  
}
```

See also: [uname](#) (full system info), `Config:::%Config`.

`^P`

`^P` — the internal debugger control flag bitmask. Set to non-zero to enable the embedded debugger hooks; specific bits control which events (subroutine entry, line execution, file load, ...) trigger callbacks. Debugger / profiler authors only.

```
p "debugger flags: ^P"
```

See also: [stryke](#) debugger / profiler tooling.

`^R`

`^R` — the value returned by the most recent (`{ ... }`) or (`{?{ ... }`) regex code-assertion. Useful when building accumulators inside complex regex matches that compute as they go.

```
"42" =~ /(\d+){?{ ^R = $1 * 2 }/  
p ^R # 84
```

See also: (`{ ... }`), (`{?{ ... }`), `^N`.

`^S`

`^S` — the current exception-handling state: [undef](#) while compiling, [0](#) while executing the main program, [1](#) while executing inside an `eval { }`. Inside a `$SIG{__DIE__}` handler this tells you whether the die will be caught (`^S == 1`) or fatal (`^S == 0`).

```
$SIG{__DIE__} = fn ($msg) {  
  if (^S) { return } # let eval catch it  
  log_fatal($msg) # uncaught - log it  
}
```

See also: [eval](#), [die](#), `$SIG{__DIE__}`.

`$^T`

`$^T` — the time at which the program started, as a Unix epoch (seconds since 1970). Used by `-A`, `-M`, `-C` file-test operators (which return age in days relative to `$^T`). Read-write — set it to a different epoch to change what those tests mean.

```
p "started at: " . scalar(localtime $^T)
var @stale = grep { -M $_ > 30 } @files # older than 30 days
```

See also: `-M` / `-A` / `-C` file tests, `time()`, `localtime`.

`$^V`

`$^V` — the running stryke / Perl version as a version object (v-string). Compare with `$^V ge v5.20.0` style expressions for capability gating. Stringifies to `"v5.38.2"` etc.

```
p "running $^V"
die "need v5.20+" unless $^V ge v5.20.0
```

See also: `$stryke::VERSION`, `use VERSION`.

`$^W`

`$^W` — the global warnings flag (boolean). Set by the `-w` command-line flag or `BEGIN { $^W = 1 }`. The modern preferred mechanism is lexical `use warnings` / `no warnings`, which doesn't pollute distant scopes; reach for `$^W` only when matching legacy Perl 4 / early Perl 5 idioms.

```
local $^W = 0 # silence warnings in this scope (old style)
# Modern:
no warnings 'uninitialized'
```

See also: `use warnings`, `-w`, `${^WARNING_BITS}`.

`$^X`

`$^X` — the path used to invoke the stryke / Perl executable, as taken from `argv[0]` at startup. Useful for re-`exec`-ing the same interpreter from a child script or constructing `system($^X, '-e', ...)` calls that are guaranteed to use the same binary.

```
p "interpreter: $^X"
system $^X, '-e', 'p "hello from a fresh stryke"'
```

See also: `$0` (program name), `@ARGV`.

`$_`

`$_` — the default topic variable. Many builtins (`print`, `say/p`, `chomp`, `length`, `uc`, `lc`, `regex =~` without an explicit target, `-X` file tests) operate on `$_` when called without arguments. `for/foreach` loops and `while (<>)` automatically bind each element/line to `$_` unless you declare a loop variable. In stryke pipelines (`|>`), the unsigned `_` is the topic — pipe stages see the current value as `_`, and `$_` remains compatible for Perl 5 idioms.

```
for (1:5) { p $_ * 2 } # 2 4 6 8 10
while (<>) { chomp; p } # chomp $_; p $_
@names |> e { p uc } # uc $_

# Explicit aliasing – mutations propagate:
var @xs = (1, 2, 3)
for (@xs) { $_ *= 10 }
p "@xs" # 10 20 30
```

Use `local $_` to scope a topic change. In stryke, `_` (no sigil) is the preferred pipeline topic; `$_` is the Perl-5-compatible form.

`$``

`$`` — the substring **preceding** the last successful regex match ("prematch"). With `$&` and `$'`, reconstructs the full input. Reset on every successful match.

```
if ("hello world" =~ /\s/) {
  p $`      # hello
  p $&      # (space)
  p $'      # world
}
```

See also: `$&`, `$'`, `${^PREMATCH}` (cleaner under `use re 'p'`).

`$stryke::VERSION`

`$stryke::VERSION` — the stryke language version string (e.g. `"0.13.4"`). Matches the version baked into the binary at build time from `Cargo.toml`. Use it for feature gating and diagnostics.

```
p "stryke $stryke::VERSION"
die "need stryke 0.14+" if $stryke::VERSION lt "0.14"
```

See also: `$^V` (interpreter version), `--version` CLI flag.

`$|`

`$|` — the autoflush flag for the currently selected output filehandle. When true (any non-zero value), every `print/say` flushes the buffer immediately. Essential for progress bars, prompts that should appear before `<STDIN>`, and pipelines where downstream consumers need data without buffering delay. Affects only the *currently selected* handle — pair with `select` to set autoflush on a specific handle.

```
$| = 1          # autoflush STDOUT
for (1:10) {
  print "."     # appears immediately
  sleep 1
}

# Autoflush a specific handle:
var $old = select STDERR; $| = 1; select $old
```

See also: `select`, `IO::Handle::autoflush`.

`$~`

`$~` — the name of the current format used by `write`. Defaults to the name of the filehandle being written to. Set it to switch formats on the fly when the same handle should emit different report shapes.

```
format HEADER = ...
.
format BODY = ...
.

$~ = 'HEADER'; write
$~ = 'BODY';   write while @rows
```

See also: `$^` (top-of-page header format), `format`, `write`.

`%ENV`

`%ENV` — the process environment as a hash keyed by variable name. Reads return the current value (or `undef` for unset); assignments take effect immediately and propagate to child processes spawned afterwards via `system/exec/backticks`.

```
p $ENV{HOME}
p $ENV{PATH}
$ENV{MYAPP_DEBUG} = 1
system 'env' |> grep /MYAPP/ |> e p
delete $ENV{TMPDIR}      # unset
```

In stryke, `%ENV` is a live view of the OS environment — no caching. See also: `getenv`, `setenv`.

`%INC`

`%INC` — the registry of files loaded via `require` / `use`. Keys are the relative module path (e.g. `"My/Module.pm"`); values are the absolute path on disk. `require` consults `%INC` before loading: if the key already exists, the require is a no-op (this is how Perl avoids loading the same module twice). Delete a key to force a reload (advanced use, e.g. plugin reloading).

```
use File::Spec
p $INC{'File/Spec.pm'}      # absolute path to the loaded file
keys %INC |> sort |> e p     # every loaded module
```

See also: `@INC` (search path), `require`, `use`.

`%SIG`

`%SIG` — the signal handler table keyed by signal name (without the `SIG` prefix: `INT`, `TERM`, `HUP`, ...). Assign a coderef to install a handler; assign `'IGNORE'` to ignore; assign `'DEFAULT'` to restore default behavior. Two pseudo-keys hook the interpreter itself: `$SIG{__DIE__}` runs before any `die` (including caught), and `$SIG{__WARN__}` intercepts every `warn`.

```
$SIG{INT} = fn ($sig) { p "got SIGINT"; exit 0 }
$SIG{TERM} = 'IGNORE'
$SIG{__WARN__} = fn ($msg) { log_warn($msg) }

# Block a signal temporarily:
local $SIG{INT} = 'IGNORE'
critical_section()
```

See also: `kill`, `POSIX::sigaction`, `__DIE__` / `__WARN__` pseudo-signals.

`%^H`

`%^H` — the lexical hint hash, paired with `$$^H`. Pragma authors store per-scope state here, keyed by their package name. The contents are visible at compile time only — runtime code in the same scope sees an empty hash unless the pragma uses `$$^_RE_TRIE_MAXBUF` or similar magic.

```
BEGIN { $$^H{'My::Pragma'} = { strict_mode => 1 } }
```

Niche; almost exclusively used by `use` pragma implementations. See also: `$$^H`, `use`.

`%^HOOK`

`%^HOOK` — the runtime hook registry used by `stryke` / Perl extensions to register callbacks for compile-time events (function entry/exit, regex match, signal arrival). Keys are well-known hook names; values are coderefs. Modifying this directly is a sharp edge — usually you'd use a pragma or module that hooks for you (e.g. AOP intercepts in `zshrs`).

```
p keys %^HOOK      # registered hook names
```

See also: `stryke` AOP / intercept facilities.

`%a`

`%a` — short alias for `%stryke::aliases`. Maps every alias spelling to its canonical primary name. Keys are the 2nd-and-later names in each `try_builtin` match arm; values are the primary. For O(1) *reverse* lookup (primary → all its aliases), use `%p` / `%stryke::primaries`.

```
p $stryke::aliases{tj}      # to_json
p $stryke::aliases{bn}      # basename
p scalar keys %stryke::aliases      # total alias count
keys %stryke::aliases |> grep /^p/ |> sort |> e p      # every alias starting with p
```

See also: `%stryke::aliases`, `%p` / `%stryke::primaries`.

`%all`

`%all` — short alias for `%stryke::all`. Every callable *spelling* (primaries **and** aliases) → category. Aliases inherit their primary's category.

Use `%a` when you want "how many names can I type?" or want to look up an alias's category without hopping through `%a`. Use `%b` when you want unique operations only.

```
p scalar keys %aall # total callable-spellings count
p $aall{tj} # "serialization" (alias inherits)
p $aall{to_json} # "serialization"
keys %aall |> pager # browse every spelling
```

See also: `%stryke::all`, `%b / %stryke::builtins` (primaries-only), `%a / %stryke::aliases` (alias → primary).

%b

`%b` — short alias for `%stryke::builtins`. Maps every **primary** callable name to its category string. Primaries-only — aliases live in `%a / %stryke::aliases`. `scalar keys %b` is the clean unique-operation count of stryke's runtime.

```
p $stryke::builtins{pmap} # "parallel"
p $stryke::builtins{to_json} # "serialization"
p scalar keys %stryke::builtins # unique-op count
keys %stryke::builtins |> grep { $stryke::builtins{$_} eq 'parallel' } |> sort |> e p
```

See also: `%stryke::builtins`, `%c / %stryke::categories`, `%a / %stryke::aliases`.

%c

`%c` — short alias for `%stryke::categories`. Inverted index on `%b`: category string → arrayref of names in that category. Gives O(1) reverse-lookup for "list every op of kind X" queries that would otherwise be O(n) `greps`. Name lists are alphabetized.

```
$c{parallel} |> e p # every parallel op
p scalar @{$c{parallel}} # how many?
p join ", ", @{$c{"array / list"}} # joined roster
keys %c |> sort |> e p # every category name
```

See also: `%stryke::categories`, `%b / %stryke::builtins`.

%d

`%d` — short alias for `%stryke::descriptions`. Maps each callable name to its one-line summary (the first sentence of its LSP hover doc). Sparse — only names that have a hand-written hover doc appear, so `exists $d{$name}` doubles as "is this documented?".

```
p $d{pmap} # one-line summary of pmap
p $d{to_json} # "Serialize a StrykeValue to a JSON string."
p scalar keys %d # count of documented ops
keys %d |> grep { $d{$_} =~ /parallel/i } |> sort |> e p
```

See also: `%stryke::descriptions`, the LSP hover system.

%e

`%e` — short alias for `%stryke::extensions`. Stryke-only names, name → category. Disjoint from `%pc` (Perl 5 core). Everything `--compat` mode rejects at parse time, plus dispatch primaries (like `basename`, `ddump`) that are extensions at runtime even without a parser entry.

```
p $e{pmap} # "parallel"
p scalar keys %e # stryke-only name count
keys %e |> grep /^p/ |> sort |> e p # every p* parallel op
```

See also: `%stryke::extensions`, `%pc / %stryke::perl_comps`, `%b / %stryke::builtins`.

%k

`%k` — short alias for `%stryke::keywords`. Maps every stryke language keyword (`fn`, `match`, `class`, `struct`, `enum`, `varsync`, `frozen`, ...) to a one-line category/description. Use this to discover the full keyword set programmatically.


```
p scalar keys %k # total keyword count
keys %k |> sort |> e p # every keyword
keys %k |> grep { $k[_] =~ /control/i } |> e p
```

See also: `%stryke::keywords`, `%b / %stryke::builtins`.

%limits

`%limits` — reflection hash exposing stryke's hard and soft runtime limits (max thread count, max recursion depth, max string interpolation depth, regex backtrack budget, etc.). Keys are limit names; values are the numeric ceilings.

```
p keys %limits |> sort |> e p
p $limits{max_threads}
p $limits{regex_backtrack}
```

See also: stryke runtime / scheduler internals.

%o

`%o` — short alias for `%stryke::operators`. Maps every symbol operator stryke recognises (arithmetic, comparison, bitwise, pipeline arrows, assignment forms, ...) to its category string.

Key is the spelling as the lexer emits it ("`+`", "`<=>`", "`//`", "`->`", "`|>`", "`->`", "`==`", "`...`"); value is one of `arith`, `compare`, `logical`, `bitwise`, `assign`, `string`, `binding`, `pipeline`, `range`, `arrow`, `deref`, `inc`, `ternary`, `sigil`, `punct`. Word operators (`and`, `or`, `eq`, `cmp`, `x`, `not`) live in `%k` with category `"operator"`, not here.

```
p $o{"+"} # "arith"
p $o{"<=>"} # "compare"
p $o{"|>"} # "pipeline"
p $o{"=="} # "binding"
p scalar keys %o # total symbol-operator count
keys %o |> grep { $o[_] eq "pipeline" } |> sort |> e p
keys %o |> grep { $o[_] eq "assign" } |> sort |> e p
```

See also: `%stryke::operators`, `%k / %stryke::keywords` (word operators), `%v / %stryke::special_vars`.

%overload::

`%overload::` — the runtime symbol table of the `overload` package, exposing operator-overloading machinery (`overload::Method`, `overload::StrVal`, `overload::Overloaded`). Inspecting it tells you which packages have overloaded operators registered. Rarely accessed directly; use the `overload` pragma to register operators in your own classes.

```
p keys %overload::
```

See also: `use overload`, `overload::Method`, `overload::Overloaded`.

%p

`%p` — short alias for `%stryke::primaries`. Primary dispatcher name → arrayref of its aliases. Inverted `%a / %stryke::aliases`. Primaries with no aliases still have an entry (empty arrayref), so `exists $p{foo}` reliably answers "is foo a dispatch primary?" in O(1).

```
$p{to_json} |> e p # ["tj"]
p scalar @ { $p{basename} } # alias count for basename
# every primary that has at least one alias:
keys %p |> grep { scalar @ { $p[_] } } |> sort |> e p
```

Not to be confused with the `p` builtin (stryke print-with-newline) — `%p` is the *reflection hash*, `p "..."` is the *output verb*. The sigil disambiguates.

See also: `%stryke::primaries`, `%a / %stryke::aliases`, `%b / %stryke::builtins`.

%parameters

`%parameters` — reflection hash mapping each stryke builtin to its parameter signature (argument names, types, optionality, default values). Populated at build time from the function definitions in `strykelang/builtins*.rs`. Drives LSP signature help and type checking.

```
p $parameters{pmap}          # signature for pmap
keys %parameters |> sort |> e p  # every documented signature
keys %parameters |> grep { @{$parameters{$_}} > 3 } |> e p  # 4+ arg builtins
```

See also: `%b / %stryke::builtins`, LSP signature provider.

`%pc`

`%pc` — short alias for `%stryke::perl_compats`. Perl 5 core names only, name → category. Subset of `%b` restricted to names from `is_perl5_core`. Direct O(1) access for the “show me just Perl core” query.

```
p $pc{map}                    # "array / list"
p scalar keys %pc             # core-only count
keys %pc |> sort |> e p        # enumerate every Perl core name
keys %pc |> grep { $pc{$_} eq "array / list" } |> sort |> e p
```

See also: `%stryke::perl_compats`, `%e / %stryke::extensions`, `%b / %stryke::builtins`.

`%stryke::aliases`

`%stryke::aliases` (short: `%a`) — alias spelling → canonical primary.

Keys are the 2nd-and-later names in each `try_builtin` match arm. For O(1) *reverse* lookup (primary → all its aliases), use `%stryke::primaries / %p`.

```
p $stryke::aliases{tj}       # "to_json"
p $stryke::aliases{bn}       # "basename"
p scalar keys %stryke::aliases # total alias count
```

`%stryke::all`

`%stryke::all` (short: `%all`) — every callable *spelling* (primaries **and** aliases) → category. Aliases inherit their primary's category.

Use `%all` when you want “how many names can I type?” or want to look up an alias's category without hopping through `%aliases`. Use `%builtins` when you want unique operations.

```
p scalar keys %all           # total callable-spellings count
p $all{tj}                   # "serialization" (alias resolves via inheritance)
p $all{to_json}              # "serialization"
keys %all |> pager            # browse every spelling
```

`%stryke::builtins`

`%stryke::builtins` (short: `%b`) — every **primary** callable name → its category. Primaries-only, so `scalar keys %b` is a clean unique-operation count. For the “everything you can type” view (primaries + aliases), use `%stryke::all / %all`.

```
p $stryke::builtins{pmap}    # "parallel"
p $stryke::builtins{to_json} # "serialization"
p scalar keys %stryke::builtins # unique-op count
```

`%stryke::categories`

`%stryke::categories` (short: `%c`) — category string → arrayref of names in that category.

Inverted index on `%builtins`. Gives O(1) reverse-lookup for “list every op of kind X” queries that would otherwise be O(n) `greps`. Name lists are alphabetized.

```
$c{parallel} |> e p          # every parallel op
p scalar @{$c{parallel}}    # how many?
p join " ", " ", @{$c{"array / list"}} # joined roster
keys %c |> sort |> p          # all category names
```

%stryke::descriptions

%stryke::descriptions (short: %d) — name → one-line summary.

First sentence of each LSP hover doc (`doc_for_label_text`), harvested at build time. Sparse — only names that have a hover doc appear, so `exists $d{$name}` doubles as “is this documented?”.

```
p $d{pmap}                  # one-line summary
p $d{to_json}               # "Serialize a StrykeValue to a JSON string."
p scalar keys %d            # count of documented ops
keys %d |> grep { $d{$_} =~ /parallel/i } |> sort |> p
```

%stryke::extensions

%stryke::extensions (short: %e) — stryke-only names, name → category.

Disjoint from %perl_compts. Everything `--compat` mode rejects at parse time, plus dispatch primaries like `basename/ddump` that are extensions at runtime even without a parser entry.

```
p $e{pmap}                  # "parallel"
keys %e |> grep /^p/ |> sort |> p    # every p* parallel op
```

%stryke::keywords

%stryke::keywords (short: %k) — language keyword → one-line category/description. Includes `fn`, `match`, `class`, `struct`, `enum`, `varsync`, `frozen`, `async/await/spawn`, control-flow keywords, declaration keywords, etc.

```
p scalar keys %stryke::keywords
keys %stryke::keywords |> sort |> e p
```

See also: %k (short alias), %stryke::builtins (callable names rather than keywords).

%stryke::operators

%stryke::operators (short: %o) — symbol operator spelling → category. Built from the `OPERATORS` table in `builtins.rs`, which mirrors the operator token set in `token.rs` (every `Token::Plus` / `Token::Spaceship` / `Token::ThreadArrow` ...). Disjoint from %stryke::keywords: word operators (`and`, `or`, `eq`, `cmp`, `x`, `not`) appear in %k with category “operator”, while only symbol/punctuation operators appear here.

Categories: `arith`, `compare`, `logical`, `bitwise`, `assign`, `string`, `binding`, `pipeline`, `range`, `arrow`, `deref`, `inc`, `ternary`, `sigil`, `punct`.

```
p scalar keys %stryke::operators
keys %stryke::operators |> sort |> e p
keys %stryke::operators |> grep { $stryke::operators{$_} eq "pipeline" } |> sort |> e p
```

See also: %o (short alias), %stryke::keywords (word operators), %stryke::special_vars.

%stryke::perl_compts

%stryke::perl_compts (short: %pc) — Perl 5 core names only, name → category.

Subset of %builtins restricted to names from `is_perl5_core`. Direct O(1) access for the “show me just Perl core” query.

```
p $pc{map}                  # "array / list"
p scalar keys %pc           # core-only count
keys %pc |> sort |> p        # enumerate every Perl core name
```

%stryke::primaries

`%stryke::primaries` (short: `%p`) — primary dispatcher name → arrayref of its aliases.

Inverted `%aliases`. Primaries with no aliases still have an entry (empty arrayref), so `exists $p{foo}` reliably answers "is foo a dispatch primary?" O(1).

```
$p{to_json} |> e p          # ["tj"]
p scalar @{$p{basename}} } # how many aliases does basename have?
# find every primary that has at least one alias:
keys %p |> grep { scalar @{$p[_]} } |> sort |> p
```

%stryke::special_vars

`%stryke::special_vars` (short: `%v`) — special variable spelling → category. Every key carries its sigil ("`$!`", "`@ARGV`", "`%ENV`", "`$^X`", "`$stryke::VERSION`", "`__FILE__`") so a single hash covers scalar/array/hash/caret/compile-time forms uniformly. Every entry has a hand-written hover doc in `lsp.rs::doc_for_label_text`, so `exists $v{$name}` doubles as "is this documented?".

Categories partition by purpose: `topic`, `error`, `pid`, `regex-match`, `regex-capture`, `io`, `process`, `args`, `env`, `signal`, `script`, `caret`, `subscript`, `version`, `reflection`, `format`, `sort`.

```
p scalar keys %stryke::special_vars
p $stryke::special_vars{'$!'}          # "error"
p $stryke::special_vars{'@ARGV'}       # "args"
p $stryke::special_vars{'%ENV'}        # "env"
p $stryke::special_vars{'$^X'}         # "caret"
keys %stryke::special_vars |> grep { $stryke::special_vars[_] eq "regex-capture" } |> sort |> e p
```

See also: `%v` (short alias), `%stryke::operators`, `%stryke::keywords`, `%stryke::builtins`.

%term

`%term` — reflection hash describing the controlling terminal: width, height, color depth, capabilities (Unicode support, true-color, focus events, alternate screen). Populated lazily on first read by querying `tput` / `tcgetattr` / `TIOCGWINSZ`.

```
p $term{width}          # columns
p $term{height}         # rows
p $term{color}          # "truecolor" / "256" / "16" / "none"
p keys %term |> sort
```

See also: `%uname` (OS info), `term_width`/`term_height` builtins.

%v

`%v` — short alias for `%stryke::special_vars`. Maps every special variable spelling (sigil included) to its category.

One hash covers every kind of special variable: scalar (`$!`, `$_`, `$@`, `$1..$9`, `$^X`), array (`@_`, `@ARGV`, `@+`, `@-`, `@^CAPTURE`), hash (`%ENV`, `%INC`, `%SIG`, `%^H`), caret (`$^A..$^X`, `%^H`, `%^HOOK`), compile-time (`__FILE__`, `__LINE__`, `__PACKAGE__`, `__SUB__`), and version (`$stryke::VERSION`). Categories: `topic`, `error`, `pid`, `regex-match`, `regex-capture`, `io`, `process`, `args`, `env`, `signal`, `script`, `caret`, `subscript`, `version`, `reflection`, `format`, `sort`.

```
p $v{'$!'}              # "error"
p $v{'@ARGV'}           # "args"
p $v{'$1'}              # "regex-capture"
p scalar keys %v        # total special-var count
keys %v |> grep { $v[_] eq "caret" } |> sort |> e p
keys %v |> grep { $v[_] eq "regex-capture" } |> sort |> e p
```

Every key has a matching LSP hover entry, so `exists $v{$name}` doubles as "is this special var documented?".

See also: `%stryke::special_vars`, `%o` / `%stryke::operators`, `%k` / `%stryke::keywords`.

@+

`@+` — array of byte offsets *one past the end* of each match group in the most recent successful regex. `+[0]` is the end of the whole match; `+[1]` is the end of `$1`; etc. Pair with `@-` (start offsets) for `substr`-based slicing without re-running the regex.

```
val $s = "foo bar baz"
if ($s =~ /(\w+) (\w+)/) {
  p "match: [$-[0], $+[0])"          # [0, 7)
  p substr($s, $-[1], $+[1] - $-[1]) # foo
  p substr($s, $-[2], $+[2] - $-[2]) # bar
}
```

See also: `@-`, `$1..$9`, `${^MATCH}`.

@-

`@-` — array of byte offsets to the *start* of each match group in the most recent successful regex. `-[0]` is the start of the whole match; `-[1]` is the start of `$1`; etc. Pair with `@+` (end offsets).

```
val $s = "foo bar baz"
$s =~ /(\w+) (\w+)/
p "foo at: $-[1]..$+[1]"      # foo at: 0..3
p "bar at: $-[2]..$+[2]"      # bar at: 4..7
```

See also: `@+`, `$1..$9`.

@ARGV

`@ARGV` — the command-line argument list (excluding `$0`, the program name). `shift @ARGV` is the idiomatic way to peel off positional args one at a time. The diamond operator `<>` consumes `@ARGV` element by element, treating each as a filename to read (or `-` for stdin). For structured flag parsing prefer the `getopts` builtin.

```
val $cmd = shift @ARGV
var @files = @ARGV
while (<>) { p $_ }          # reads from every file in @ARGV

# Structured flags:
var %opts = %{ getopts(["verbose|v", "file|f=s"]) }
p "leftover positionals: @ARGV"
```

See also: `$ARGV` (current filename in `<>`), `getopts`, `<>` diamond operator.

@ENV

`@ENV` — array interpretation of the environment. Generally not useful — `%ENV` (hash) is the right interface. Mentioned here because the array form appears in completion words for symmetry with `%ENV / $ENV{...}`. Reading `@ENV` yields a list of `KEY=VALUE` strings on some systems, `undef` on others.

```
# Prefer %ENV:
for val $k (sort keys %ENV) {
  p "$k=$ENV{$k}"
}
```

See also: `%ENV`.

@F

`@F` — the autosplit field array, populated when `stryke` / `Perl` runs with the `-a` flag. Each input line read by `-n/-p` is automatically split on whitespace (or on the regex given to `-F`) into `@F`. AWK-like one-liners use this heavily.

```
# stryke -nae 'p $F[0] if $F[2] > 100'      # AWK '{ print $1 if $3 > 100 }'
# stryke -F: -nae 'p "user=$F[0] uid=$F[2]"' /etc/passwd
```

See also: `-a` / `-F` command-line flags, `split`, `-n` / `-p`.

@INC

`@INC` — the module search path. Each `require` / `use` walks `@INC` looking for the requested module. Defaults: the directories baked into the interpreter at build time plus `.` (current dir, removed in recent Perl for security). Modify with `BEGIN { unshift @INC, '/my/lib' }` to add directories before module loads run.

```
BEGIN { unshift @INC, "$ENV{HOME}/perl5/lib" }
use My::Module          # searched in @INC
p join "\n", @INC        # show search path
```

See also: `%INC` (loaded files), `require`, `use lib`.

@^CAPTURE

`@^CAPTURE` — array of substring captures from the most recent successful regex match, paralleling `$1..$9` but as a list. `@^CAPTURE[0]` is `$1`, `@^CAPTURE[1]` is `$2`, etc. Useful when the number of capture groups is dynamic.

```
if ("alice 30 engineer" =~ /^(\w+) (\d+) (\w+)/) {
  p "groups: @^CAPTURE" # alice 30 engineer
  p scalar @^CAPTURE    # 3
}
```

See also: `@^CAPTURE_ALL`, `$1..$9`, `%+` (named captures).

@^CAPTURE_ALL

`@^CAPTURE_ALL` — array-of-arrays of *every* capture from a `/g` (global) match. Each element is one match's captures. Useful for iterating over all matches in a single string without writing a `while ... =~` loop.

```
"a=1 b=2 c=3" =~ /(\w)=(\d)/g
for val $m (@^CAPTURE_ALL) {
  p "$m->[0] -> $m->[1]"
}
```

See also: `@^CAPTURE`, `/g` modifier.

@_

`@_` — the implicit argument array inside a subroutine. Every `fn` / `sub` receives its caller's arguments in `@_`, flattened (lists are spread, hash/array refs stay as single refs). `shift` and `pop` without an explicit array default to `@_` inside a sub. Modifying `$_[0]`, `$_[1]`, ... aliases back to the caller's variables — a sharp edge worth knowing about. In `stryke`, prefer named parameters (`fn f($x, $y) { ... }`) which destructure `@_` into proper lexicals; reach for raw `@_` only when you need variadic behavior or want to forward all arguments via `@_`.

```
fn sum_all {
  val $total = 0
  $total += _ for @_
  $total
}
p sum_all(1, 2, 3, 4) # 10

fn first_two {
  var ($x, $y) = @_ # destructure
  ($x, $y)
}

fn forward { other_fn(@_) } # pass through every arg
```

See also: `$_[N]` for positional access, `scalar @_` for arg count, `wantarray` for caller context.

@f

`@f` — short alias for `@fpath`. The list of directories searched for autoload functions (zsh-style). Mutating `@f` (push, unshift, splice) immediately changes which directories future autoload calls will search.

```
p "@f"
unshift @f, "$ENV{HOME}/.stryke/fpath" # prepend a directory
```

See also: `@fpath`, `autoload`.

@fpath

`@fpath` — the autoload search path: an ordered list of directories that stryke / zshrs scans for autoloadable function definitions when an unknown function name is called. Modeled on zsh's `$fpath`. Short alias: `@f`.

```
p "@fpath"                # current autoload search path
unshift @fpath, "./local-funcs" # prepend a directory
```

See also: `@f`, `autoload`, zsh `fpath`.

@path

`@path` — the executable search path as a Perl array, mirrored from `$ENV{PATH}` (or the OS-specific equivalent). Mutations to `@path` update `$ENV{PATH}` and propagate to child processes. Useful when prepending a directory for the rest of the script.

```
unshift @path, "$ENV{HOME}/bin"
system 'mytool' # finds it via the updated PATH
```

See also: `$ENV{PATH}`, `which`, `find_in_path`.

BUILD

`BUILD` — the constructor hook for stryke classes. When a class defines a `BUILD` method, it runs **after** attribute initialization but **before** the constructed object is returned from `new`. Use it for cross-attribute validation, derived-field computation, and external resource setup that depends on the attribute values.

Receives the freshly constructed `$self` and the original hashref of constructor arguments. Returning a value has no effect — `new` always returns the object. Throwing in `BUILD` aborts construction.

```
class Connection {
  has 'host' => (is => 'ro', required => 1)
  has 'port' => (is => 'ro', default => 5432)
  has 'dbh' => (is => 'rw')

  fn BUILD($self, $args) {
    die "host: invalid" unless $self->host =~ /^[\w.-]+$/
    $self->dbh(connect_to($self->host, $self->port))
  }
}
```

See also: `DESTROY` (destructor), `class`, `bless`.

CORE

`CORE::` — pseudo-namespace alias for every dispatch primary. Every callable bare name is also reachable as `CORE::name`, with identical semantics. The qualified spelling exists for tab-complete affordance and explicit-namespace style; stryke rejects redefining a builtin (`fn print { ... }` is a parse error), so there is nothing to shadow at runtime.

The set is built dynamically: any name in `keys %stryke::builtins` is a valid `CORE::name`. There's no separate `%CORE::` table — it's a virtual namespace resolved at dispatch time.

```
CORE::print "hello world\n"      # ≡ print "hello world\n"
var $r = CORE::sum(1, 2, 3)      # 6 - qualified call
map { CORE::sprintf("%d", _) } @nums # any builtin
```

The `s docs CORE::name` form resolves to the bare-name page (the alias and primary share docs). Aliases (`tj`, `bn`, `e`, ...) also support `CORE::tj` / `CORE::bn` / `CORE::e`.

CORE::pipe

Create a unidirectional pipe, returning a pair of connected filehandles: one for reading and one for writing. Data written to the write end can be read from the read end, making pipes the fundamental building block for inter-process communication. Commonly used with `fork` so the parent and child can exchange data.

```

pipe(val $rd, val $wr) or die "pipe: $"
if (fork() == 0) {
    close($rd)
    print $wr "hello from child\n"
    exit(0)
}
close($wr)
var $msg = <$rd>
p $msg    # hello from child

```

DESTROY

DESTROY — the destructor sub invoked when the last reference to a blessed object is dropped. Stryke runs **DESTROY** synchronously at the drop point (matching Perl semantics) by walking the class's C3 MRO child-first: the object's own **DESTROY** first, then each parent's. Destructors should be defensive — at interpreter shutdown `$_{^GLOBAL_PHASE}` is "DESTRUCT" and external state may already be torn down.

```

class FileHandle {
    has 'fh' => (is => 'rw')

    fn DESTROY($self) {
        return if $_{^GLOBAL_PHASE} eq 'DESTRUCT'
        close($self->fh) if $self->fh
    }
}

```

A pending-destroy queue (`pending_destroy.rs`) drains before the next VM op so destructors run in deterministic order.

See also: **BUILD**, **bless**, `$_{^GLOBAL_PHASE}`, **class**.

E

E — operation **E**. Alias for **euler_number**. Category: **math / numeric extras**.

```
p E $x
```

PI

PI — operation **PI**. Alias for **pi**. Category: **constants**.

```
p PI $x
```

TAU

TAU — operation **TAU**. Alias for **tau**. Category: **constants**.

```
p TAU $x
```

UNITCHECK

UNITCHECK { } — a compile-time phase block that runs after the **current compilation unit** (script, module, or eval string) finishes compiling, but before any code in it runs. Modules use this for setup that must observe the full compiled state of **just this file** (without seeing later-required modules). Runs in reverse declaration order, like **END**.

The four phase blocks in order: **BEGIN** (immediately at compile), **UNITCHECK** (end of this unit's compile), **CHECK** (end of program compilation), **INIT** (start of program execution), **END** (end of execution).

```

UNITCHECK { p "this file compiled" }
BEGIN     { p "compiling..." }
p "running"
# Output order:
# compiling...
# this file compiled
# running

```


See also: `BEGIN`, `CHECK`, `INIT`, `END`.

`__DATA__`

`__DATA__` — marks the end of compiled code and opens the rest of the file as the `DATA` filehandle. The compiler stops parsing at this line; every byte after the trailing newline is buffered and made readable via `<DATA>`. This is the canonical Perl pattern for shipping a script together with its embedded fixture data, lookup table, or test input — no separate file needed. The marker line must be `__DATA__` exactly (trailing whitespace tolerated).

In `stryke`, the data section is installed at startup (see `vm_helper.rs:install_data_handle`) so `<DATA>` and `readline(DATA)` work immediately. Setting `local $/ = undef` then reading from `DATA` slurps the whole section.

```
while (val $line = <DATA>) {
    chomp $line
    p "got: $line"
}
__DATA__
alpha
beta
gamma
```

See also: `__END__` (truncate only, no `DATA` handle).

`__END__`

`__END__` — marks the logical end of source code. Everything from this line to EOF is ignored by the compiler. This is the standard place to park POD documentation, embedded test data, or scratch notes in a `.pm` / `.stk` file — the parser stops at the marker so trailing prose never has to be valid syntax. The line must be `__END__` exactly (after trim); leading or trailing whitespace on the line itself is fine but no other characters.

In `stryke`, `__END__` truncates module compilation in `require` / `use` paths (see `strykelang/data_section.rs:strip_perl_end_marker`). Unlike standard Perl, `stryke` does **not** install a `DATA` filehandle for content after `__END__` — use `__DATA__` if you want the trailing bytes accessible via `<DATA>`.

```
p "running"
__END__
=pod

=head1 NAME

MyModule - does the thing.

=cut
```

See also: `__DATA__` (truncate + expose as `<DATA>`).

`__FILE__`

`__FILE__` — compile-time pseudo-token that evaluates to the path of the source file containing this token, exactly as the compiler saw it (the same string that shows up in `die/warn` location traces). Inside `-e` / `-E` one-liners it is `"-e"`; for stdin scripts it is `"-"`; for an executed file it is the path passed on the command line (relative or absolute, whichever the user typed).

Useful for self-locating modules, embedding the source path in log messages, or pinning a `do/require` relative to the current file's directory. The value is a normal string — pair with `dirname/abs_path` for filesystem operations.

```
p __FILE__                                # /tmp/script.stk
use File::Basename qw(dirname)
var $here = dirname(__FILE__)
p "loading config from $here/config.toml"
```

See also: `__LINE__`, `__PACKAGE__`, `__SUB__`.

`__LINE__`

`__LINE__` — compile-time pseudo-token that evaluates to the source line number of the token itself, as a plain integer. Each occurrence is rewritten at compile time to a literal — there is no runtime cost and no way to fool it with `eval`. The line number reflects the original source file, including any `#line` directive overrides.

Useful for embedding source location in custom log/error messages, asserting positions in generated code, or building self-describing diagnostics. For full caller context inside a sub, use `caller()` instead.

```
p "trace: " . __FILE__ . ":" . __LINE__
die "assertion failed at line " . __LINE__ unless $ok
```

See also: `__FILE__`, `caller`, `Carp::confess`.

`__PACKAGE__`

`__PACKAGE__` — compile-time pseudo-token that evaluates to the name of the currently enclosing `package`, as a string. At the top of a fresh program it is `"main"`; inside `package Foo::Bar { ... }` or after a `package Foo::Bar` declaration it tracks the active namespace. The substitution is lexical — switching packages mid-file changes what subsequent `__PACKAGE__` tokens see.

Useful for class methods that need to refer to their own package name without hard-coding it (so subclasses inherit the right value), building method-dispatch keys, and writing portable mixins.

```
package My::Logger
fn tag { __PACKAGE__ }
fn log_msg($msg) { p "[" . tag() . "] $msg" }

package main
My::Logger::log_msg("started")      # [My::Logger] started
p __PACKAGE__                       # main
```

See also: `package`, `caller`, `ref`.

`__SUB__`

`__SUB__` — a reference to the currently executing subroutine. Inside a `fn/sub` body, `__SUB__` returns the coderef of that very function. This is the clean way to write recursive anonymous subs without naming them, and to build self-referential closures that survive renaming.

In standard Perl this requires `use feature 'current_sub'` (or 5.16+); in stryke it is always available. Outside any sub, `__SUB__` is `undef`.

```
# Anonymous recursion without naming the sub:
var $fact = fn ($n) {
  return 1 if $n <= 1
  $n * __SUB__->($n - 1)
}
p $fact->(5)      # 120

fn me { __SUB__ }
p ref me()       # CODE
```

See also: `caller`, `__PACKAGE__`, `wantarray`.

`a0`

`bohr_radius` (alias `a0`) — $a_0 \approx 5.29 \times 10^{-11}$ m. Radius of hydrogen atom ground state.

```
p a0      # 5.29177210903e-11
```

`a256`

`a256` — operation `a256`. Alias for `ansi_256`. Category: `*color operations*`.

```
p a256 $x
```

`a3c_advantage_step`

`a3c_advantage_step` — single-step update of `a3c_advantage`. Category: `*electrochemistry, batteries, fuel cells*`.

```
p a3c_advantage_step $x
```

a85d

`base85_decode` (aliases `b85d`, `a85d`) decodes an Ascii85/Base85 encoded string.

```
p b85d(b85e("Hello")) # Hello
```

a85e

`base85_encode` (aliases `b85e`, `a85e`) encodes a string using Ascii85/Base85 encoding. More compact than Base64 (4:5 ratio vs 3:4).

```
p b85e("Hello") # encoded string
```

a_law_decode

`a_law_decode` — decode of `a_law`. Category: *signal processing*.

```
p a_law_decode $x
```

a_law_encode

`a_law_encode` — encode of `a_law`. Category: *signal processing*.

```
p a_law_encode $x
```

a_star_grid

`a_star_grid(grid, start, goal) → array` — A* on a 2D grid where non-zero cells are blocked. Manhattan heuristic, 4-connected.

```
p a_star_grid([[0,0,0],[1,1,0],[0,0,0]], [0,0], [2,2])
```

aabb_contains_point

`aabb_contains_point(aabb, point) → 0|1` — 1 if point lies inside (inclusive).

aabb_intersects

`aabb_intersects(a, b) → 0|1` — 1 if two AABBs overlap.

aabb_new

`aabb_new(min, max) → [min, max]` — pack a 3D axis-aligned bounding box as two `vec3`.

```
p aabb_new([0,0,0], [10,10,10])
```

aabb_union

`aabb_union(a, b) → aabb` — smallest AABB containing both.

aabb_volume

`aabb_volume(aabb) → number` — $(\text{max.x} - \text{min.x}) \cdot (\text{max.y} - \text{min.y}) \cdot (\text{max.z} - \text{min.z})$.

ab1_step

`ab1_step` — single-step update of `ab1`. Category: **ode advanced**.

```
p ab1_step $x
```

ab2_step

`ab2_step` — single-step update of `ab2`. Category: **ode advanced**.

```
p ab2_step $x
```

ab3_step

`ab3_step` — single-step update of `ab3`. Category: **ode advanced**.

```
p ab3_step $x
```

ab4

`ab4` — operation `ab4`. Alias for `adams_bashforth_4`. Category: **more extensions (2)**.

```
p ab4 $x
```

ab_coeff_sum

`ab_coeff_sum` — sum of `ab` `coeff`. Category: **ode advanced**.

```
p ab_coeff_sum @xs
```

abelianization_quotient

`abelianization_quotient` — operation `abelianization quotient`. Category: **econometrics**.

```
p abelianization_quotient $x
```

aberration_annual

`aberration_annual` — operation `aberration annual`. Category: **astronomy / astrometry**.

```
p aberration_annual $x
```

aberth_step

`aberth_step` — single-step update of `aberth`. Category: **more extensions (2)**.

```
p aberth_step $x
```

abs_ceil

`abs_ceil` — operation `abs ceil`. Category: **trivial numeric helpers**.

```
p abs_ceil $x
```

abs_diff

`abs_diff` — operation `abs diff`. Category: **trivial numeric / predicate builtins**.

```
p abs_diff $x
```

abs_each

`abs_each` — operation `abs each`. Category: **trivial numeric helpers**.

```
p abs_each $x
```

abs_floor

`abs_floor` — operation `abs floor`. Category: **trivial numeric helpers**.

```
p abs_floor $x
```

absolute_magnitude

`absolute_magnitude` — operation `absolute magnitude`. Category: **astronomy / music / color / units**.

```
p absolute_magnitude $x
```

absolute_vorticity_step

`absolute_vorticity_step` — single-step update of `absolute vorticity`. Category: **climate, fluids, atmospheric**.

```
p absolute_vorticity_step $x
```

absorbed_short_wave

`absorbed_short_wave` — operation `absorbed short wave`. Category: **climate, fluids, atmospheric**.

```
p absorbed_short_wave $x
```

abundant

`abundant` — operation `abundant`. Category: **math / number theory extras**.

```
p abundant $x
```

abundant_numbers

`abundant_numbers` — operation `abundant numbers`. Category: **number theory / primes**.

```
p abundant_numbers $x
```

abunnums

`abunnums` — operation `abunnums`. Alias for `abundant_numbers`. Category: **number theory / primes**.

```
p abunnums $x
```

abuse_test_voltage

`abuse_test_voltage` — operation `abuse test voltage`. Category: **electrochemistry, batteries, fuel cells**.

```
p abuse_test_voltage $x
```

accept_lang_pick

`accept_lang_pick` — operation `accept lang pick`. Category: **b81-misc-utility**.

```
p accept_lang_pick $x
```

acceptance_target

`acceptance_target` — operation `acceptance target`. Category: **more extensions (2)**.

```
p acceptance_target $x
```

accrint

`accrint` — operation `accrint`. Category: **b81-misc-utility**.

```
p accrint $x
```

accrued_interest

`accrued_interest` — operation `accrued interest`. Category: **excel/sheets + bond/loan financial**.

```
p accrued_interest $x
```

accum

`accum` — operation `accum`. Alias for `accumulate`. Category: **python/ruby stdlib**.

```
p accum $x
```

accumulate

`accumulate` — operation `accumulate`. Category: **python/ruby stdlib**.

```
p accumulate $x
```

acf

`acf_fn` (alias `acf`) — autocorrelation function. Returns ACF values for lags 0..`max_lag`. Like R's `acf()`.

```
val @a = @{acf([1,3,2,4,3,5,4,6], 5)}  
p $x[0] # 1.0 (lag 0 is always 1)  
p $x[1] # lag-1 autocorrelation
```

acf_basis

`acf_basis` — operation `acf basis`. Category: **statsmodels**.

```
p acf_basis $x
```

acirc

`acirc` — operation `acirc`. Alias for `area_circle`. Category: **geometry / physics**.

```
p acirc $x
```

acorr

`acorr` — operation `acorr`. Alias for `autocorrelation`. Category: **math / numeric extras**.

```
p acorr $x
```

acos

`acos` — operation `acos`. Category: **trig / math**.

```
p acos $x
```

acosh

`acosh` — operation `acosh`. Category: **trig / math**.

```
p acosh $x
```

acot

`acot` — operation `acot`. Category: **trig extensions**.

```
p acot $x
```

acro

`acro` — operation `acro`. Alias for `acronym`. Category: **string processing extras**.

```
p acro $x
```

acronym

`acronym` — operation `acronym`. Category: **string processing extras**.

```
p acronym $x
```

acsc

`acsc` — operation `acsc`. Category: **trig extensions**.

```
p acsc $x
```

adaboost_alpha

`adaboost_alpha` — operation `adaboost alpha`. Category: **ml extensions**.

```
p adaboost_alpha $x
```

adagrad_step

`adagrad_step` — single-step update of `adagrad`. Category: **more extensions (2)**.

```
p adagrad_step $x
```

adams_bashforth_4

`adams_bashforth_4` — operation `adams bashforth 4`. Category: **more extensions (2)**.

```
p adams_bashforth_4 $x
```

adaptive_threshold

`adaptive_threshold(image, window=11, c=2) → matrix` — local-mean adaptive threshold. Output is binary (0 or 255).

add_days

`add_days` — operation `add days`. Category: **extended stdlib**.

```
p add_days $x
```

add_hours

`add_hours` — operation `add hours`. Category: **extended stdlib**.

```
p add_hours $x
```

add_minutes

`add_minutes` — operation `add minutes`. Category: **extended stdlib**.

```
p add_minutes $x
```

add_seasonality

`add_seasonality(series, pattern) → array` — adds a repeating seasonal pattern element-wise (wraps with modulo on `pattern.len`).

```
p add_seasonality(\@xs, [1, -1, 1, -1])
```

addd

`addd` — operation `addd`. Alias for `add_days`. Category: **extended stdlib**.

```
p addd $x
```

addh

`addh` — operation `addh`. Alias for `add_hours`. Category: **extended stdlib**.

```
p addh $x
```

addm

`addm` — operation `addm`. Alias for `add_minutes`. Category: **extended stdlib**.

```
p addm $x
```

adf_test

`adf_test` — statistical test of `adf`. Category: **test runner**.

```
p adf_test $x
```

adf_test_stat

`adf_test_stat` — operation `adf test stat`. Category: **econometrics**.

```
p adf_test_stat $x
```


adfuller

`adfuller` — operation `adfuller`. Category: **statsmodels**.

```
p adfuller $x
```

adiabatic_lapse_rate_dry

`adiabatic_lapse_rate_dry` — operation `adiabatic lapse rate dry`. Category: **climate, fluids, atmospheric**.

```
p adiabatic_lapse_rate_dry $x
```

adiabatic_lapse_rate_moist

`adiabatic_lapse_rate_moist` — operation `adiabatic lapse rate moist`. Category: **climate, fluids, atmospheric**.

```
p adiabatic_lapse_rate_moist $x
```

adj_r2

`adj_r2` — operation `adj r2`. Alias for `adjusted_r_squared`. Category: **misc**.

```
p adj_r2 $x
```

adjacent_difference

`adjacent_difference` — operation `adjacent difference`. Category: **list helpers**.

```
p adjacent_difference $x
```

adjacent_pairs

`adjacent_pairs` — operation `adjacent pairs`. Category: **go/general functional utilities**.

```
p adjacent_pairs $x
```

adjp

`adjp` — operation `adjp`. Alias for `adjacent_pairs`. Category: **go/general functional utilities**.

```
p adjp $x
```

adjusted_r_squared

`adjusted_r_squared` — operation `adjusted r squared`. Category: **misc**.

```
p adjusted_r_squared $x
```

adjusted_winner_pct

`adjusted_winner_pct` — percentage of `adjusted winner`. Category: **climate, fluids, atmospheric**.

```
p adjusted_winner_pct $x
```

adl32

`adl32` — operation `adl32`. Alias for `adler32`. Category: **extended stdlib**.

```
p adl32 $x
```

adler32

`adler32` — operation `adler32`. Category: **extended stdlib**.

```
p adler32 $x
```

adler32_combine

`adler32_combine(a, b, blen) → int` — combine two Adler-32 checksums into one for concatenated streams.

adler32_hash

`adler32_hash` — hash function of `adler32`. Category: **misc**.

```
p adler32_hash $x
```

adm_mass_step

`adm_mass_step` — single-step update of `adm mass`. Category: **electrochemistry, batteries, fuel cells**.

```
p adm_mass_step $x
```

admm_dual_step

`admm_dual_step` — single-step update of `admm dual`. Category: **combinatorial optimization, scheduling**.

```
p admm_dual_step $x
```

admm_primal_step

`admm_primal_step` — single-step update of `admm primal`. Category: **combinatorial optimization, scheduling**.

```
p admm_primal_step $x
```

adobe_rgb_to_rgb

`adobe_rgb_to_rgb` — convert from `adobe rgb` to `rgb`. Category: **complex / geom / color / trig**.

```
p adobe_rgb_to_rgb $value
```

ads_metric_step

`ads_metric_step` — single-step update of `ads metric`. Category: **electrochemistry, batteries, fuel cells**.

```
p ads_metric_step $x
```

adsr_envelope

`adsr_envelope` — operation `adsr envelope`. Category: **extras**.

```
p adsr_envelope $x
```

adx

`adx(highs, lows, closes, period=14) → array` — Average Directional Index (Wilder). Returns the trend-strength scalar; pair with `plus_di` / `minus_di` for direction.

```
p adx(\@hi, \@lo, \@cl, 14)
```

aellip

`aellip` — operation `aellip`. Alias for `area_ellipse`. Category: **geometry / physics**.

```
p aellip $x
```

aerosol_optical_depth

`aerosol_optical_depth` — operation `aerosol optical depth`. Category: **climate, fluids, atmospheric**.

```
p aerosol_optical_depth $x
```

aes_inv_sbox_byte

`aes_inv_sbox_byte` — operation `aes inv sbox byte`. Category: **more extensions**.

```
p aes_inv_sbox_byte $x
```

aes_sbox_byte

`aes_sbox_byte` — operation `aes sbox byte`. Category: **more extensions**.

```
p aes_sbox_byte $x
```

affine_encrypt

`affine_encrypt` — encrypt of `affine`. Category: **cryptography deep**.

```
p affine_encrypt $x
```

african_monsoon_index

`african_monsoon_index` — index of `african monsoon`. Category: **climate, fluids, atmospheric**.

```
p african_monsoon_index $x
```

after_n

`after_n` — operation `after n`. Category: **functional combinators**.

```
p after_n $x
```

age_at_date

`age_at_date` — operation `age at date`. Category: **b82-misc-utility**.

```
p age_at_date $x
```

age_at_z

`age_at_z` — operation `age at z`. Category: **cosmology / gr / flrw**.

```
p age_at_z $x
```

age_from_birthdate

`age_from_birthdate` — operation `age from birthdate`. Category: **misc**.

```
p age_from_birthdate $x
```

age_in_years

`age_in_years` — operation `age in years`. Category: **extended stdlib**.

```
p age_in_years $x
```

ageyrs

`ageyrs` — operation `ageyrs`. Alias for `age_in_years`. Category: **extended stdlib**.

```
p ageyrs $x
```

ai_agent

`ai_agent` — operation `ai agent`. Category: **ai primitives (docs/ai_primitives.md)**.

```
p ai_agent $x
```

aic

`aic` — operation `aic`. Category: **misc**.

```
p aic $x
```

aig_simplify_step

`aig_simplify_step` — single-step update of `aig simplify`. Category: **logic, proof, sat/smt, type theory**.

```
p aig_simplify_step $x
```

ain

`ain` — operation `ain`. Alias for `assoc_in`. Category: **algebraic match**.

```
p ain $x
```

air_mass_kasten

`air_mass_kasten` — operation `air mass kasten`. Category: **more extensions**.

```
p air_mass_kasten $x
```

airy_disk_radius

`airy_disk_radius` — operation `airy disk radius`. Category: **astronomy / astrometry**.

```
p airy_disk_radius $x
```

airyai

`airyai` — operation `airyai`. Category: **numpy + scipy.special**.

```
p airyai $x
```

airybi

`airybi` — operation `airybi`. Category: **numpy + scipy.special**.

```
p airybi $x
```

aitken_delta_squared

`aitken_delta_squared` — operation `aitken delta squared`. Category: **more extensions**.

```
p aitken_delta_squared $x
```

akaike_info_crit

`akaike_info_crit` — operation `akaike info crit`. Category: **econometrics**.

```
p akaike_info_crit $x
```

aks_witness_count

`aks_witness_count` — count of `aks witness`. Category: **cryptanalysis & number theory deep**.

```
p aks_witness_count $x
```

albedo_blackbody_balance

`albedo_blackbody_balance` — operation `albedo blackbody balance`. Category: **climate, fluids, atmospheric**.

```
p albedo_blackbody_balance $x
```

albers_conic_project

`albers_conic_project` — operation `albers conic project`. Category: **more extensions**.

```
p albers_conic_project $x
```

alexander_polynomial_at_one

`alexander_polynomial_at_one` — operation `alexander polynomial at one`. Category: **econometrics**.

```
p alexander_polynomial_at_one $x
```

alfven_speed

`alfven_speed` — operation `alfven speed`. Category: **em / optics / relativity**.

```
p alfven_speed $x
```

algebraic_intersection_number

`algebraic_intersection_number` — operation `algebraic intersection number`. Category: **econometrics**.

```
p algebraic_intersection_number $x
```

aliquot_sum

`aliquot_sum` — sum of `aliquot`. Category: **math / numeric extras**.

```
p aliquot_sum @xs
```

all_distinct

`all_distinct` — operation `all distinct`. Category: **more list helpers**.

```
p all_distinct $x
```

all_eq

`all_eq` — operation `all eq`. Category: **more list helpers**.

```
p all_eq $x
```

all_equal

`all_equal` — operation `all equal`. Category: **iterator + string-distance extras**.

```
p all_equal $x
```

all_match

`all_match` — operation `all match`. Category: **predicates**.

```
p all_match $x
```

all_pass_filter

`all_pass_filter(signal, delay=441, gain=0.5) → array` — IIR all-pass: flat magnitude, shifted phase.

all_pay_auction_step

`all_pay_auction_step` — single-step update of `all pay auction`. Category: **climate, fluids, atmospheric**.

```
p all_pay_auction_step $x
```

all_unique

`all_unique` — operation `all unique`. Alias for `all_distinct`. Category: **more list helpers**.

```
p all_unique $x
```

allee_growth_step

`allee_growth_step` — single-step update of `allee growth`. Category: **biology / ecology**.

```
p allee_growth_step $x
```

allele_frequency

`allele_frequency` — operation `allele frequency`. Category: **bioinformatics deep**.

```
p allele_frequency $x
```

alpha_fs

`fine_structure_constant` (alias `alpha_fs`) — $\alpha \approx 1/137 \approx 0.00730$. Fundamental constant of electromagnetism.

```
p alpha_fs      # 0.0072973525693
p 1 / alpha_fs  # ~137.036
```

alphabet_value

`alphabet_value(leaves, depth) → number` — alpha-beta pruned minimax; same answer as `minimax_value`, fewer evaluations.

alt_az_to_ra_dec

`alt_az_to_ra_dec` — convert from `alt az` to `ra dec`. Category: **astronomy / astrometry**.

```
p alt_az_to_ra_dec $value
```

alternate_case

`alternate_case` — operation `alternate case`. Category: **string helpers**.

```
p alternate_case $x
```

alternating_seq_sum

`alternating_seq_sum` — sum of `alternating seq`. Category: **more extensions**.

```
p alternating_seq_sum @xs
```

always_false

`always_false` — operation `always false`. Category: **functional primitives**.

```
p always_false $x
```

always_true

`always_true` — operation `always true`. Category: **functional primitives**.

```
p always_true $x
```

am2_step

`am2_step` — single-step update of `am2`. Category: **ode advanced**.

```
p am2_step $x
```

am3_step

`am3_step` — single-step update of `am3`. Category: **ode advanced**.

```
p am3_step $x
```

am4_step

`am4_step` — single-step update of `am4`. Category: **ode advanced**.

```
p am4_step $x
```

am_synth

`am_synth(carrier, mod_freq, depth=0.5, sr=44100, dur=1) → array` — amplitude modulation: $(1 + \text{depth} \cdot \sin(2\pi \cdot \text{fm} \cdot t)) \cdot \sin(2\pi \cdot \text{fc} \cdot t)$.

amari_alpha_div

`amari_alpha_div` — operation `amari alpha div`. Category: **electrochemistry, batteries, fuel cells**.

```
p amari_alpha_div $x
```

amax

amax — operation [amax](#). Alias for [argmax](#). Category: *extended stdlib*.

```
p amax $x
```

amin

amin — operation [amin](#). Alias for [argmin](#). Category: *extended stdlib*.

```
p amin $x
```

amo_index_step

amo_index_step — single-step update of [amo index](#). Category: *climate, fluids, atmospheric*.

```
p amo_index_step $x
```

amort

[amortization_schedule\(\\$principal, \\$rate, \\$periods\)](#) (alias [amort](#)) — generates a full amortization schedule. Returns arrayref of hashrefs with period, payment, principal, interest, balance.

```
val $sched = amort(10000, 0.05, 12)
for val $row (@$sched) {
  p "Period $row->{period}: bal=$row->{balance}"
}
```

amortization_total_interest

amortization_total_interest — operation [amortization total interest](#). Category: *misc*.

```
p amortization_total_interest $x
```

amp_to_db

amp_to_db — convert from [amp](#) to [db](#). Category: *astronomy / music / color / units*.

```
p amp_to_db $value
```

amphoteric_check

amphoteric_check — operation [amphoteric check](#). Category: *chemistry & biochemistry*.

```
p amphoteric_check $x
```

amplitude_damping_channel

amplitude_damping_channel — operation [amplitude damping channel](#). Category: *more extensions*.

```
p amplitude_damping_channel $x
```

amplitude_damping_excited

amplitude_damping_excited — operation [amplitude damping excited](#). Category: *quantum mechanics deep*.

```
p amplitude_damping_excited $x
```


amu

amu — operation `amu`. Alias for `atomic_mass_unit`. Category: **physics constants**.

```
p amu $x
```

anagram_q

anagram_q — operation `anagram q`. Category: **misc**.

```
p anagram_q $x
```

anaphora_distance

anaphora_distance — distance / dissimilarity metric of `anaphora`. Category: **computational linguistics**.

```
p anaphora_distance $x
```

ancilla_alloc

ancilla_alloc — operation `ancilla alloc`. Category: **quantum**.

```
p ancilla_alloc $x
```

and_list

and_list — operation `and list`. Category: **additional missing stdlib functions**.

```
p and_list $x
```

anderson_darling

anderson_darling — operation `anderson darling`. Category: **r / scipy distributions and tests**.

```
p anderson_darling $x
```

anderson_step

anderson_step — single-step update of `anderson`. Category: **more extensions (2)**.

```
p anderson_step $x
```

andl

andl — operation `andl`. Alias for `and_list`. Category: **additional missing stdlib functions**.

```
p andl $x
```

andoyer_dist

andoyer_dist — operation `andoyer dist`. Category: **excel/sheets + bond/loan financial**.

```
p andoyer_dist $x
```

andrew_monotone_chain

andrew_monotone_chain — operation `andrew monotone chain`. Category: **more extensions (2)**.

```
p andrew_monotone_chain $x
```

andrew_monotone_hull

`andrew_monotone_hull(points)` → `array` — convex hull via Andrew's monotone chain. $O(n \log n)$.

angbet

`angbet` — operation `angbet`. Alias for `angle_between`. Category: **geometry (extended)**.

```
p angbet $x
```

angle_between

`angle_between($x1, $y1, $x2, $y2)` — computes the angle in radians between two 2D vectors from origin. Returns the angle using `atan2`.

```
val $angle = angle_between(1, 0, 0, 1)
p $angle # -1.5708 (π/2 radians = 90°)
```

angle_between_deg

`angle_between_deg` — operation `angle between deg`. Category: **math functions**.

```
p angle_between_deg $x
```

angle_bracket

`angle_bracket` — operation `angle bracket`. Category: **string helpers**.

```
p angle_bracket $x
```

angular_diameter_distance

`angular_diameter_distance` — distance / dissimilarity metric of `angular diameter`. Category: **cosmology / gr / flrw**.

```
p angular_diameter_distance $x
```

angular_velocity

`angular_velocity` — operation `angular velocity`. Category: **physics formulas**.

```
p angular_velocity $x
```

angvel

`angvel` — operation `angvel`. Alias for `angular_velocity`. Category: **physics formulas**.

```
p angvel $x
```

annuity_due_an

`annuity_due_an` — operation `annuity due an`. Category: **actuarial science**.

```
p annuity_due_an $x
```

annuity_future_value

`annuity_future_value` — value of `annuity future`. Category: **misc**.

```
p annuity_future_value $x
```

annuity_immediate_an

`annuity_immediate_an` — operation `annuity immediate an`. Category: **actuarial science**.

```
p annuity_immediate_an $x
```

annuity_present_value

`annuity_present_value` — value of `annuity present`. Category: **misc**.

```
p annuity_present_value $x
```

anova

`anova_oweway` (aliases `anova`, `anova1`) performs one-way ANOVA. Returns [F-statistic, df_between, df_within].

```
var ($F, $dfb, $dfw) = @{anova([1,2,3], [4,5,6], [7,8,9])}
```

anova_one_way

`anova_one_way(group1, group2, ...)` → {statistic, df, p_value} — one-way ANOVA F-test across ≥2 groups.

```
p anova_one_way([1,2,3], [10,11,12], [100,101,102])
```

ansi_256

`ansi_256` — operation `ansi 256`. Category: **color operations**.

```
p ansi_256 $x
```

ansi_black

`ansi_black` — operation `ansi black`. Category: **color / ansi**.

```
p ansi_black $x
```

ansi_blue

`ansi_blue` — operation `ansi blue`. Category: **color / ansi**.

```
p ansi_blue $x
```

ansi_bold

`ansi_bold` — operation `ansi bold`. Category: **color / ansi**.

```
p ansi_bold $x
```

ansi_cyan

`ansi_cyan` — operation `ansi cyan`. Category: **color / ansi**.

```
p ansi_cyan $x
```

ansi_dim

`ansi_dim` — operation `ansi dim`. Category: **color / ansi**.

```
p ansi_dim $x
```

ansi_green

`ansi_green` — operation `ansi green`. Category: **color / ansi**.

```
p ansi_green $x
```

ansi_magenta

`ansi_magenta` — operation `ansi magenta`. Category: **color / ansi**.

```
p ansi_magenta $x
```

ansi_off

`ansi_off` — operation `ansi off`. Alias for `red`. Category: **color / ansi**.

```
p ansi_off $x
```

ansi_red

`ansi_red` — operation `ansi red`. Category: **color / ansi**.

```
p ansi_red $x
```

ansi_reverse

`ansi_reverse` — operation `ansi reverse`. Category: **color / ansi**.

```
p ansi_reverse $x
```

ansi_truecolor

`ansi_truecolor` — operation `ansi truecolor`. Category: **color operations**.

```
p ansi_truecolor $x
```

ansi_underline

`ansi_underline` — operation `ansi underline`. Category: **color / ansi**.

```
p ansi_underline $x
```

ansi_white

`ansi_white` — operation `ansi white`. Category: **color / ansi**.

```
p ansi_white $x
```

ansi_yellow

`ansi_yellow` — operation `ansi yellow`. Category: **color / ansi**.

```
p ansi_yellow $x
```

anti_de_sitter_radius_step

`anti_de_sitter_radius_step` — single-step update of `anti de sitter radius`. Category: **electrochemistry, batteries, fuel cells**.

```
p anti_de_sitter_radius_step $x
```

antoine_p

`antoine_p` — operation `antoine p`. Category: **chemistry**.

```
p antoine_p $x
```

any_match

`any_match` — operation `any match`. Category: **predicates**.

```
p any_match $x
```

aov

`aov` — operation `aov`. Category: **r / scipy distributions and tests**.

```
p aov $x
```

ap_jonker_volgenant_step

`ap_jonker_volgenant_step` — single-step update of `ap jonker volgenant`. Category: **combinatorial optimization, scheduling**.

```
p ap_jonker_volgenant_step $x
```

aperture_stop_to_fnumber

`aperture_stop_to_fnumber(stops)` → `number` — $2^{(stops/2)}$. Converts stops from f/1 to f-number.

apery

`apery` — operation `apery`. Alias for `apery_constant`. Category: **math constants**.

```
p apery $x
```

apery_constant

`apery_constant` — operation `apery constant`. Category: **math constants**.

```
p apery_constant $x
```

apparent_horizon_step

`apparent_horizon_step` — single-step update of `apparent horizon`. Category: **electrochemistry, batteries, fuel cells**.

```
p apparent_horizon_step $x
```

append_elem

`append_elem` — operation `append elem`. Category: **list helpers**.

```
p append_elem $x
```

append_redis

`append_redis` — operation `append redis`. Category: **redis-flavour primitives**.

```
p append_redis $x
```

appl

`appl` — operation `appl`. Alias for `apply`. Category: **algebraic match**.

```
p appl $x
```

apply

`apply` — operation `apply`. Category: **algebraic match**.

```
p apply $x
```

apply_list

`apply_list` — operation `apply list`. Category: **functional combinators**.

```
p apply_list $x
```

apply_window

`apply_window(\@signal, \@window)` — element-wise multiplies signal by window function. Use before FFT to reduce spectral leakage.

```
val $w = hann(scalar @signal)
var $windowed = apply_window(\@signal, $w)
var $spectrum = dft($windowed)
```

applywin

`applywin` — operation `applywin`. Alias for `apply_window`. Category: **dsp / signal (extended)**.

```
p applywin $x
```

approval_voting_max

`approval_voting_max` — maximum of `approval voting`. Category: **climate, fluids, atmospheric**.

```
p approval_voting_max $x
```

approx

`approx_fn` (alias `approx`) — piecewise linear interpolation at query points. Like R's `approx()`.

```
val @y = @{approx([0,1,2], [0,10,0], [0.5, 1.0, 1.5])}
# [5, 10, 5]
```

approx_eq

`approx_eq` — operation `approx eq`. Category: **math functions**.

```
p approx_eq $x
```

approximate_entropy

`approximate_entropy` — operation `approximate_entropy`. Category: *cryptography deep*.

```
p approximate_entropy $x
```

apr_to_apy

`apr_to_apy` — convert from `apr` to `apy`. Category: *misc*.

```
p apr_to_apy $value
```

apsp

`floyd_warshall` (aliases `floydwarshall`, `apsp`) computes all-pairs shortest paths. Takes a distance matrix (use Inf for no edge).

```
val $d = apsp([[0,3,1e18],[1e18,0,1],[1e18,1e18,0]])
```

apy_to_apr

`apy_to_apr` — convert from `apy` to `apr`. Category: *misc*.

```
p apy_to_apr $value
```

ar_envelope

`ar_envelope` — operation `ar envelope`. Category: *extras*.

```
p ar_envelope $x
```

ar_model_likelihood

`ar_model_likelihood` — operation `ar model likelihood`. Category: *econometrics*.

```
p ar_model_likelihood $x
```

arange

`arange` — operation `arange`. Category: *matrix / linear algebra*.

```
p arange $x
```

arc_length

`arc_length($radius, $theta)` — computes the arc length of a circular arc. Theta is the central angle in radians. Returns `radius * theta`.

```
p arc_length(10, 3.14159) # ~31.4 (half circle)
p arc_length(5, 1.57)    # ~7.85 (quarter circle)
```

arch_lm

`arch_lm` — operation `arch lm`. Category: *statsmodels*.

```
p arch_lm $x
```

arch_lm_test

`arch_lm_test` — statistical test of `arch lm`. Category: *econometrics*.

```
p arch_lm_test $x
```

arclen

`arclen` — operation `arclen`. Alias for `arc_length`. Category: *geometry (extended)*.

```
p arclen $x
```

arctic_oscillation_step

`arctic_oscillation_step` — single-step update of `arctic oscillation`. Category: *climate, fluids, atmospheric*.

```
p arctic_oscillation_step $x
```

area_circle

`area_circle` — operation `area circle`. Category: *geometry / physics*.

```
p area_circle $x
```

area_ellipse

`area_ellipse` — operation `area ellipse`. Category: *geometry / physics*.

```
p area_ellipse $x
```

area_rectangle

`area_rectangle` — operation `area rectangle`. Category: *geometry / physics*.

```
p area_rectangle $x
```

area_trapezoid

`area_trapezoid` — operation `area trapezoid`. Category: *geometry / physics*.

```
p area_trapezoid $x
```

area_triangle

`area_triangle` — operation `area triangle`. Category: *geometry / physics*.

```
p area_triangle $x
```

arect

`arect` — operation `arect`. Alias for `area_rectangle`. Category: *geometry / physics*.

```
p arect $x
```

arellano_bond_step

`arellano_bond_step` — single-step update of `arellano bond`. Category: *econometrics*.

```
p arellano_bond_step $x
```


argc

`argc` — operation `argc`. Category: **process / env**.

```
p argc $x
```

argmax

`argmax` — operation `argmax`. Category: **extended stdlib**.

```
p argmax $x
```

argmin

`argmin` — operation `argmin`. Category: **extended stdlib**.

```
p argmin $x
```

argon2_block_mix

`argon2_block_mix` — operation `argon2 block mix`. Category: **cryptography deep**.

```
p argon2_block_mix $x
```

argpartition

`argpartition` — operation `argpartition`. Category: **numpy + scipy.special**.

```
p argpartition $x
```

argsort

`argsort` — operation `argsort`. Category: **extended stdlib**.

```
p argsort $x
```

ari

`ari` — operation `ari`. Alias for `automated_readability_index`. Category: **misc**.

```
p ari $x
```

arias_intensity

`arias_intensity` — operation `arias intensity`. Category: **geology, seismology, mineralogy**.

```
p arias_intensity $x
```

arima_diff

`arima_diff` — operation `arima diff`. Category: **more extensions (2)**.

```
p arima_diff $x
```

arima_fit

`arima_fit` — operation `arima fit`. Category: **statsmodels**.

```
p arima_fit $x
```

arma_forecast

`arma_forecast` — operation `arma forecast`. Category: **statsmodels**.

```
p arma_forecast $x
```

arithmetic_coding_interval

`arithmetic_coding_interval` — operation `arithmetic coding interval`. Category: **electrochemistry, batteries, fuel cells**.

```
p arithmetic_coding_interval $x
```

arithmetic_decode_interval

`arithmetic_decode_interval` — operation `arithmetic decode interval`. Category: **electrochemistry, batteries, fuel cells**.

```
p arithmetic_decode_interval $x
```

arithmetic_sequence

`arithmetic_sequence` — operation `arithmetic sequence`. Category: **misc**.

```
p arithmetic_sequence $x
```

arithmetic_series

`arithmetic_series($a1, $d, $n)` (alias `arithser`) — sum of n terms of arithmetic sequence starting at a_1 with common difference d . Formula: $n/2 \times (2a_1 + (n-1)d)$.

```
p arithser(1, 1, 100)    # 5050 (1+2+...+100)
p arithser(2, 3, 10)     # 155 (2+5+8+...+29)
```

arity_of

`arity_of` — operation `arity of`. Category: **functional combinators**.

```
p arity_of $x
```

arma_model_innovation

`arma_model_innovation` — operation `arma model innovation`. Category: **econometrics**.

```
p arma_model_innovation $x
```

arma_order_select

`arma_order_select` — operation `arma order select`. Category: **statsmodels**.

```
p arma_order_select $x
```

armstrong_q

`armstrong_q` — operation `armstrong q`. Category: **misc**.

```
p armstrong_q $x
```

arpad_predict

`arpad_predict(rating_a, rating_b) → number` — A's expected win rate via Elo formula: $1 / (1 + 10^{((rb-ra)/400)})$.

```
p arpad_predict(1900, 1500) # 0.909
```

arrayListOf

`arrayListOf` — operation `arrayListOf`. Alias for `listOf`. Category: *kotlin collections*.

```
p arrayOf $x
```

arrayOf

`arrayOf` — operation `arrayOf`. Alias for `listOf`. Category: *kotlin collections*.

```
p arrayOf $x
```

arrayOfNulls

`arrayOfNulls` — operation `arrayOfNulls`. Category: *kotlin collections*.

```
p arrayOfNulls $x
```

array_agg

`array_agg` — array helper `agg`. Category: *postgres sql strings, json, aggregates*.

```
p array_agg $x
```

array_difference

`array_difference` — array helper `difference`. Category: *collection*.

```
p array_difference $x
```

array_intersection

`array_intersection` — array helper `intersection`. Category: *collection*.

```
p array_intersection $x
```

array_position

`array_position` — array helper `position`. Category: *postgres sql strings, json, aggregates*.

```
p array_position $x
```

array_positions

`array_positions` — array helper `positions`. Category: *postgres sql strings, json, aggregates*.

```
p array_positions $x
```

array_remove

`array_remove` — array helper `remove`. Category: *postgres sql strings, json, aggregates*.

```
p array_remove $x
```

array_replace

`array_replace` — array helper `replace`. Category: *postgres sql strings, json, aggregates*.

```
p array_replace $x
```

array_to_hstore

`array_to_hstore` — convert from `array` to `hstore`. Category: *postgres sql strings, json, aggregates*.

```
p array_to_hstore $value
```

array_to_jsonb

`array_to_jsonb` — convert from `array` to `jsonb`. Category: *postgres sql strings, json, aggregates*.

```
p array_to_jsonb $value
```

array_to_string

`array_to_string` — convert from `array` to `string`. Category: *postgres sql strings, json, aggregates*.

```
p array_to_string $value
```

array_union

`array_union` — array helper `union`. Category: *collection*.

```
p array_union $x
```

arrhenius_k

`arrhenius_k` — operation `arrhenius k`. Category: *chemistry*.

```
p arrhenius_k $x
```

arrhenius_temp_q10

`arrhenius_temp_q10` — operation `arrhenius temp q10`. Category: *misc*.

```
p arrhenius_temp_q10 $x
```

arrow_impossibility_check

`arrow_impossibility_check` — operation `arrow impossibility check`. Category: *climate, fluids, atmospheric*.

```
p arrow_impossibility_check $x
```

arrow_independence

`arrow_independence` — operation `arrow independence`. Category: *economics + game theory*.

```
p arrow_independence $x
```

arrow_pareto_check

`arrow_pareto_check` — operation `arrow_pareto_check`. Category: **climate, fluids, atmospheric**.

```
p arrow_pareto_check $x
```

articulation_point

`articulation_point` — operation `articulation_point`. Category: **networkx graph algorithms**.

```
p articulation_point $x
```

articulation_points

`articulation_points` — operation `articulation_points`. Category: **misc**.

```
p articulation_points $x
```

ascent_parser_step

`ascent_parser_step` — single-step update of `ascent_parser`. Category: **compilers / parsing**.

```
p ascent_parser_step $x
```

ascii

`ascii` — operation `ascii`. Category: **postgres sql strings, json, aggregates**.

```
p ascii $x
```

ascii_chr

`ascii_chr` — ASCII text helper `chr`. Category: **string**.

```
p ascii_chr $x
```

ascii_ord

`ascii_ord` — ASCII text helper `ord`. Category: **string**.

```
p ascii_ord $x
```

ascii_table

`ascii_table` — ASCII text helper `table`. Alias for `format_table_simple`. Category: **misc**.

```
p ascii_table $x
```

ase_palette_extract

`ase_palette_extract` — operation `ase_palette_extract`. Category: **test runner**.

```
p ase_palette_extract $x
```

asec

`asec` — operation `asec`. Category: **trig extensions**.

```
p asec $x
```

asian_call_mc

`asian_call_mc` — operation `asian call mc`. Category: **financial pricing models**.

```
p asian_call_mc $x
```

asin

`asin` — operation `asin`. Category: **trig / math**.

```
p asin $x
```

asinh

`asinh` — operation `asinh`. Category: **trig / math**.

```
p asinh $x
```

asrt

`asrt` — operation `asrt`. Alias for `argsort`. Category: **extended stdlib**.

```
p asrt $x
```

assert_type

`assert_type` — assertion `type`. Category: **conversion / utility**.

```
p assert_type $x
```

assignment_lower_bound

`assignment_lower_bound` — operation `assignment lower bound`. Category: **combinatorial optimization, scheduling**.

```
p assignment_lower_bound $x
```

assoc

`assoc` — operation `assoc`. Category: **algebraic match**.

```
p assoc $x
```

assoc_fn

`assoc_fn` — operation `assoc fn`. Alias for `associate`. Category: **go/general functional utilities**.

```
p assoc_fn $x
```

assoc_in

`assoc_in` — operation `assoc in`. Category: **algebraic match**.

```
p assoc_in $x
```

associate

`associate` — operation `associate`. Category: **go/general functional utilities**.

```
p associate $x
```

astar_search

`astar_search` — operation `astar search`. Category: **networkx graph algorithms**.

```
p astar_search $x
```

astronomical_unit

`astronomical_unit` (alias `au`) — AU $\approx 1.496 \times 10^{11}$ m. Mean Earth-Sun distance.

```
p au          # 1.495978707e11 m
p au / 1000    # ~149.6 million km
```

at_content

`at_content` — operation `at content`. Category: **bioinformatics deep**.

```
p at_content $x
```

at_index

`at_index` — index of `at`. Category: **javascript array/object methods**.

```
p at_index $x
```

atan

`atan` — operation `atan`. Category: **trig / math**.

```
p atan $x
```

atan2_deg

`atan2_deg` — operation `atan2 deg`. Category: **complex / geom / color / trig**.

```
p atan2_deg $x
```

atan2_quadrant

`atan2_quadrant` — operation `atan2 quadrant`. Category: **complex / geom / color / trig**.

```
p atan2_quadrant $x
```

atanh

`atanh` — operation `atanh`. Category: **trig / math**.

```
p atanh $x
```

atbash

`atbash` — operation `atbash`. Category: **string processing extras**.

```
p atbash $x
```

atbash_cipher

`atbash_cipher` — operation `atbash cipher`. Category: **more extensions (2)**.

```
p atbash_cipher $x
```

atc

atc — operation **atc**. Alias for **ansi_truecolor**. Category: *color operations*.

```
p atc $x
```

ati

ati — operation **ati**. Alias for **at_index**. Category: *javascript array/object methods*.

```
p ati $x
```

atime

atime — operation **atime**. Alias for **file_atime**. Category: *file stat / path*.

```
p atime $x
```

atm_to_pa

atm_to_pa — convert from **atm** to **pa**. Category: *astronomy / music / color / units*.

```
p atm_to_pa $value
```

atomic_mass_unit

atomic_mass_unit — operation **atomic mass unit**. Category: *physics constants*.

```
p atomic_mass_unit $x
```

atomic_radius_pm

atomic_radius_pm — operation **atomic radius pm**. Category: *misc*.

```
p atomic_radius_pm $x
```

atr

atr(*highs*, *lows*, *closes*, *period*=14) → **array** — Average True Range (Wilder). True range = $\max(\text{high}-\text{low}, |\text{high}-\text{prev_close}|, |\text{low}-\text{prev_close}|)$; smoothed with Wilder's recursion $\text{ATR}[i] = (\text{ATR}[i-1] \cdot (p-1) + \text{TR}[i]) / p$, seeded by mean of first *p* TRs.

```
p atr(\@hi, \@lo, \@cl, 14)
```

atrap

atrap — operation **atrap**. Alias for **area_trapezoid**. Category: *geometry / physics*.

```
p atrap $x
```

atri

atri — operation **atri**. Alias for **area_triangle**. Category: *geometry / physics*.

```
p atri $x
```


attack_rate

`attack_rate` — rate of `attack`. Category: *epidemiology / public health*.

```
p attack_rate $x
```

attempt

`attempt` — operation `attempt`. Category: *functional combinators*.

```
p attempt $x
```

attributable_fraction_pop

`attributable_fraction_pop` — operation `attributable fraction pop`. Category: *epidemiology / public health*.

```
p attributable_fraction_pop $x
```

au_to_km

`au_to_km` — convert from `au` to `km`. Category: *astronomy / astrometry*.

```
p au_to_km $value
```

au_to_m

`au_to_m` — convert from `au` to `m`. Category: *astronomy / music / color / units*.

```
p au_to_m $value
```

audio_fade_in

`audio_fade_in` — operation `audio fade in`. Category: *extras*.

```
p audio_fade_in $x
```

audio_fade_out

`audio_fade_out` — operation `audio fade out`. Category: *extras*.

```
p audio_fade_out $x
```

audio_normalize

`audio_normalize` — operation `audio normalize`. Category: *extras*.

```
p audio_normalize $x
```

audio_to_mono

`audio_to_mono` — convert from `audio` to `mono`. Category: *extras*.

```
p audio_to_mono $value
```

audio_to_stereo

`audio_to_stereo` — convert from `audio` to `stereo`. Category: *extras*.

```
p audio_to_stereo $value
```

augmented_lagrangian_step

`augmented_lagrangian_step` — single-step update of `augmented_lagrangian`. Category: **combinatorial optimization, scheduling**.

```
p augmented_lagrangian_step $x
```

autocorrelation

`autocorrelation` — operation `autocorrelation`. Category: **math / numeric extras**.

```
p autocorrelation $x
```

automated_readability_index

`automated_readability_index` — index of `automated_readability`. Category: **misc**.

```
p automated_readability_index $x
```

average

`average` — operation `average`. Category: **additional missing stdlib functions**.

```
p average $x
```

averageif

`averageif` — operation `averageif`. Category: **excel/sheets + bond/loan financial**.

```
p averageif $x
```

averageifs

`averageifs` — operation `averageifs`. Category: **excel/sheets + bond/loan financial**.

```
p averageifs $x
```

avg

`avg` — operation `avg`. Category: **pipeline / string helpers**.

```
p avg $x
```

avg_degree

`avg_degree` — operation `avg_degree`. Category: **graph algorithms**.

```
p avg_degree $x
```

avogadro

`avogadro` — operation `avogadro`. Category: **math / physics constants**.

```
p avogadro $x
```

avogadro_constant

`avogadro_constant` — operation `avogadro_constant`. Category: **chemistry & biochemistry**.

```
p avogadro_constant $x
```

avogadro_count

avogadro_count — count of **avogadro**. Category: **chemistry**.

```
p avogadro_count $x
```

avogadro_number

avogadro_number — operation **avogadro number**. Category: **constants**.

```
p avogadro_number $x
```

avogadros_number

avogadros_number — operation **avogadros number**. Category: **misc**.

```
p avogadros_number $x
```

b2gray

b2gray — operation **b2gray**. Alias for **binary_to_gray**. Category: **base / gray code**.

```
p b2gray $x
```

b2i

b2i — operation **b2i**. Alias for **bool_to_int**. Category: **boolean combinators**.

```
p b2i $x
```

b64d

b64d — operation **b64d**. Alias for **base64_decode**. Category: **crypto / encoding**.

```
p b64d $x
```

b64e

b64e — operation **b64e**. Alias for **base64_encode**. Category: **crypto / encoding**.

```
p b64e $x
```

b_to_kb

b_to_kb — convert from **b** to **kb**. Alias for **bytes_to_kb**. Category: **unit conversions**.

```
p b_to_kb $value
```

b_wien

b_wien — operation **b wien**. Alias for **wien_constant**. Category: **physics constants**.

```
p b_wien $x
```

`babip`

`babip` — operation `babip`. Category: **astronomy / astrometry**.

```
p babip $x
```

`bac`

`bac` — operation `bac`. Alias for `bac_estimate`. Category: **finance**.

```
p bac $x
```

`bac_estimate`

`bac_estimate` — operation `bac estimate`. Category: **finance**.

```
p bac_estimate $x
```

`bachelier_call`

`bachelier_call` — operation `bachelier call`. Category: **financial pricing models**.

```
p bachelier_call $x
```

`backward_algorithm`

`backward_algorithm(obs, init, trans, emit) → number` — backward equivalent of `forward_algorithm` for Baum-Welch updates.

`backward_euler`

`backward_euler` — operation `backward euler`. Category: **more extensions (2)**.

```
p backward_euler $x
```

`badi_year_from_fixed`

`badi_year_from_fixed` — operation `badi year from fixed`. Category: **calendrical algorithms**.

```
p badi_year_from_fixed $x
```

`bahai_from_fixed`

`bahai_from_fixed` — operation `bahai from fixed`. Category: **calendrical algorithms**.

```
p bahai_from_fixed $x
```

`bairstow`

`bairstow` — operation `bairstow`. Alias for `lin_bairstow_step`. Category: **misc**.

```
p bairstow $x
```

`balanced_accuracy`

`balanced_accuracy` — operation `balanced accuracy`. Category: **ml extensions**.

```
p balanced_accuracy $x
```

bandpass_filter

`bandpass_filter(\@signal, $low, $high)` (alias `bpf`) — applies a band-pass filter passing frequencies between low and high cutoffs. Combines low-pass and high-pass.

```
val $filtered = bpf(\@signal, 0.1, 0.3)
```

bandwidth_format

`bandwidth_format` — operation `bandwidth format`. Category: `*network / ip / cidr*`.

```
p bandwidth_format $x
```

bandwidth_parse

`bandwidth_parse` — operation `bandwidth parse`. Category: `*network / ip / cidr*`.

```
p bandwidth_parse $x
```

banner

`banner()` — print the stryke ASCII logo + live-stats box + tagline (the same banner the REPL shows on startup and `stryke --help` displays). Every reflection count (`%b` builtins, `%a` aliases, `%k` keywords, `%o` operators, `%v` special vars, `%pc` perl5 core, `%e` stryke extensions, `%d` descriptions, `%c` categories, `%p` primaries, `%all` everything) is pulled at call time, so the values stay current after rebuilds.

With no args, prints to stdout and returns `undef`. With a truthy first arg (`banner(1)`), returns the rendered string instead of printing — useful for capturing into a variable, embedding in a larger report, or piping elsewhere. ANSI colors are emitted only when stdout is a TTY.

```
banner                # print to stdout
var $s = banner(1)    # capture as string
banner(1) |> spurt "/tmp/welcome.txt" # write to file (no ANSI when not a tty)
```

See also: `docs` (in-language doc browser), `%stryke::builtins`, `%stryke::aliases`.

banzhaf_index_two

`banzhaf_index_two` — operation `banzhaf index two`. Category: `*climate, fluids, atmospheric*`.

```
p banzhaf_index_two $x
```

bao_scale_today

`bao_scale_today` — operation `bao scale today`. Category: `*cosmology / gr / flrw*`.

```
p bao_scale_today $x
```

bar_to_pascals

`bar_to_pascals` — convert from `bar` to `pascals`. Category: `*file stat / path*`.

```
p bar_to_pascals $value
```

bargaining_set_check

`bargaining_set_check` — operation `bargaining set check`. Category: `*climate, fluids, atmospheric*`.

```
p bargaining_set_check $x
```

barrier_up_out_call

`barrier_up_out_call` — operation `barrier up out call`. Category: **financial pricing models**.

```
p barrier_up_out_call $x
```

barthann_w

`barthann_w` — operation `barthann w`. Category: **economics + game theory**.

```
p barthann_w $x
```

bartlett_window

`bartlett_window` — operation `bartlett window`. Category: **signal processing**.

```
p bartlett_window $x
```

base91_decode

`base91_decode(s)` → `string` — alternate spelling of `base91_decode`.

base91_encode

`base91_encode(s)` → `string` — alternate spelling of `base91_encode` (trademark capitalisation).

base58check_decode

`base58check_decode(s)` → `string` — verify the 4-byte checksum and return the payload; UNDEF if invalid.

```
p base58check_decode($encoded)
```

base58check_encode

`base58check_encode(payload)` → `string` — Bitcoin Base58Check: append `sha256d(payload)[..4]` then encode in Base58.

```
p base58check_encode("hello")
```

base64url_decode

Decode URL-safe base64 (`-/_` alphabet), accepting input with or without `=` padding. Inverse of `base64url_encode`.

```
p base64url_decode("aGk_Pg")      # hi?>
```

base64url_encode

RFC 4648 §5 URL-safe base64 with no `=` padding (`-/_` instead of `+//`). The encoding used in JWT segments and URL/path tokens. Pair with `base64url_decode`.

```
p base64url_encode("hi?>")      # aGk_Pg
val $tok = base64url_encode($json_header)
```

base91_decode

`base91_decode(s)` → `string` — inverse of `base91_encode`.

base91_encode

`base91_encode(s)` → `string` — basE91 encoding (more compact than Base64 due to 91-char alphabet).

```
p base91_encode("hello world")
```

base_convert

`base_convert` — operation `base convert`. Category: **base / gray code**.

```
p base_convert $x
```

batch

`batch` — operation `batch`. Category: **collection**.

```
p batch $x
```

batched

`batched` — operation `batched`. Category: **additional missing stdlib functions**.

```
p batched $x
```

batchelor_scale_step

`batchelor_scale_step` — single-step update of `batchelor scale`. Category: **climate, fluids, atmospheric**.

```
p batchelor_scale_step $x
```

battle_sexes_payoff

`battle_sexes_payoff` — operation `battle sexes payoff`. Category: **climate, fluids, atmospheric**.

```
p battle_sexes_payoff $x
```

bauer_furuta_step

`bauer_furuta_step` — single-step update of `bauer furuta`. Category: **econometrics**.

```
p bauer_furuta_step $x
```

bayes_factor

`bayes_factor(likelihood_h1, likelihood_h0)` → `number` — $P(D|H1) / P(D|H0)$. >3 substantial, >10 strong.

bayes_posterior

`bayes_posterior` — operation `bayes posterior`. Category: **bioinformatics deep**.

```
p bayes_posterior $x
```

bayesian_beta_update

`bayesian_beta_update(alpha, beta, successes, failures)` → `{alpha, beta}` — conjugate update of Beta prior with Bernoulli observations.

```
p bayesian_beta_update(1, 1, 5, 3) # {alpha: 6, beta: 4}
```

bayesian_info_crit

`bayesian_info_crit` — Bayesian conjugate update `info_crit`. Category: **econometrics**.

```
p bayesian_info_crit $x
```

bayesian_normal_update

`bayesian_normal_update(prior_mu, prior_var, data_mu, data_var, n) → {mean, variance}` — Normal-Normal conjugate posterior.

bayesian_step

`bayesian_step` — Bayesian conjugate update `step`. Category: **statsmodels**.

```
p bayesian_step $x
```

bbox

`bounding_box(@points)` (alias `bbox`) — computes the axis-aligned bounding box of 2D points. Returns `[min_x, min_y, max_x, max_y]`.

```
val @pts = (1,2, 3,4, 0,1, 5,3)
var $bb = bbox(@pts)
p @$bb   # (0, 1, 5, 4)
```

bbox_area

`bbox_area` — operation `bbox area`. Category: **extras**.

```
p bbox_area $x
```

bbox_center

`bbox_center` — operation `bbox center`. Category: **extras**.

```
p bbox_center $x
```

bbox_contains

`bbox_contains` — operation `bbox contains`. Category: **extras**.

```
p bbox_contains $x
```

bbox_from_points

`bbox_from_points` — operation `bbox from points`. Category: **geometry / topology**.

```
p bbox_from_points $x
```

bbox_intersect

`bbox_intersect` — operation `bbox intersect`. Category: **extras**.

```
p bbox_intersect $x
```


bbox_union

`bbox_union` — operation `bbox union`. Category: *extras*.

```
p bbox_union $x
```

bconv

`bconv` — operation `bconv`. Alias for `base_convert`. Category: *base / gray code*.

```
p bconv $x
```

bcount

`bcount` — operation `bcount`. Alias for `count_bytes`. Category: *extended stdlib*.

```
p bcount $x
```

bcp47_format

`bcp47_format` — operation `bcp47 format`. Category: *extras*.

```
p bcp47_format $x
```

bcp47_parse

`bcp47_parse(tag) → hash` — parse BCP-47 language tag into {`language`, `script`, `region`, `variants`, `extensions`}.

```
p bcp47_parse("zh-Hant-TW") # {language: zh, script: Hant, region: TW}
```

bcp47_validate

`bcp47_validate(tag) → 0|1` — 1 if tag is structurally well-formed.

bcrypt_cost_iters

`bcrypt_cost_iters` — operation `bcrypt cost iters`. Category: *cryptography deep*.

```
p bcrypt_cost_iters $x
```

bdd_apply

`bdd_apply` — operation `bdd apply`. Category: *logic, proof, sat/smt, type theory*.

```
p bdd_apply $x
```

bdd_quantify

`bdd_quantify` — operation `bdd quantify`. Category: *logic, proof, sat/smt, type theory*.

```
p bdd_quantify $x
```

bdd_restrict

`bdd_restrict` — operation `bdd restrict`. Category: *logic, proof, sat/smt, type theory*.

```
p bdd_restrict $x
```

bdf1_step

bdf1_step — single-step update of **bdf1**. Category: **ode advanced**.

```
p bdf1_step $x
```

bdf2_step

bdf2_step — single-step update of **bdf2**. Category: **ode advanced**.

```
p bdf2_step $x
```

bdf3_step

bdf3_step — single-step update of **bdf3**. Category: **ode advanced**.

```
p bdf3_step $x
```

bdf4_step

bdf4_step — single-step update of **bdf4**. Category: **ode advanced**.

```
p bdf4_step $x
```

bdf5_step

bdf5_step — single-step update of **bdf5**. Category: **ode advanced**.

```
p bdf5_step $x
```

bdf6_step

bdf6_step — single-step update of **bdf6**. Category: **ode advanced**.

```
p bdf6_step $x
```

bearing

bearing — operation **bearing**. Category: **geometry / physics**.

```
p bearing $x
```

beat_frequency

beat_frequency — operation **beat frequency**. Category: **astronomy / music / color / units**.

```
p beat_frequency $x
```

beat_to_seconds

beat_to_seconds — convert from **beat** to **seconds**. Category: **music theory**.

```
p beat_to_seconds $value
```

beaufort

beaufort — operation **beaufort**. Category: **cryptography deep**.

```
p beaufort $x
```

before_n

`before_n` — operation `before n`. Category: *functional combinators*.

```
p before_n $x
```

bekenstein_entropy_step

`bekenstein_entropy_step` — single-step update of `bekenstein entropy`. Category: *electrochemistry, batteries, fuel cells*.

```
p bekenstein_entropy_step $x
```

bell_inequality_chsh

`bell_inequality_chsh` — operation `bell inequality chsh`. Category: *more extensions*.

```
p bell_inequality_chsh $x
```

bell_number

`bell_number` — operation `bell number`. Category: *math / numeric extras*.

```
p bell_number $x
```

bell_state

`bell_state` — operation `bell state`. Category: *quantum*.

```
p bell_state $x
```

bell_state_index

`bell_state_index` — index of `bell state`. Category: *quantum mechanics deep*.

```
p bell_state_index $x
```

bellman_equation_step

`bellman_equation_step` — single-step update of `bellman equation`. Category: *electrochemistry, batteries, fuel cells*.

```
p bellman_equation_step $x
```

bellman_ford

`bellman_ford` (alias `bellmanford`) computes shortest paths from a source in a graph with negative weights. Takes edges `[[u,v,w],...]`, node count, source index.

```
val $d = bellmanford([[0,1,4],[0,2,5],[1,2,-3]], 3, 0)
```

bellman_ford_relax

`bellman_ford_relax` — operation `bellman ford relax`. Category: *networkx graph algorithms*.

```
p bellman_ford_relax $x
```

belln

`belln` — operation `belln`. Alias for `bell_number`. Category: *math / numeric extras*.

```
p belln $x
```

benders_decomposition_step

`benders_decomposition_step` — single-step update of `benders decomposition`. Category: *combinatorial optimization, scheduling*.

```
p benders_decomposition_step $x
```

berger_parker

`berger_parker` — operation `berger parker`. Category: *biology / ecology*.

```
p berger_parker $x
```

bergmann_adjust

`bergmann_adjust` — operation `bergmann adjust`. Category: *biology / ecology*.

```
p bergmann_adjust $x
```

bernoulli

`bernoulli_number` (alias `bernoulli`) computes the n th Bernoulli number. $B(0)=1$, $B(1)=-0.5$, odd $B(n>1)=0$.

```
p bernoulli(0) # 1
p bernoulli(2) # 0.1667
p bernoulli(4) # -0.0333
```

berry_phase_spin_half

`berry_phase_spin_half` — operation `berry phase spin half`. Category: *quantum mechanics deep*.

```
p berry_phase_spin_half $x
```

bertrand_eq

`bertrand_eq` — operation `bertrand eq`. Category: *economics + game theory*.

```
p bertrand_eq $x
```

bertrand_price_step

`bertrand_price_step` — single-step update of `bertrand price`. Category: *climate, fluids, atmospheric*.

```
p bertrand_price_step $x
```

bessel_in_general

`bessel_in_general` — operation `bessel in general`. Category: *special functions extra*.

```
p bessel_in_general $x
```

bessel_jn_general

`bessel_jn_general` — operation `bessel jn general`. Category: *special functions extra*.

```
p bessel_jn_general $x
```

bessel_lp

`bessel_lp` — operation `bessel lp`. Category: *economics + game theory*.

```
p bessel_lp $x
```

best_response_dynamic

`best_response_dynamic` — operation `best response dynamic`. Category: *climate, fluids, atmospheric*.

```
p best_response_dynamic $x
```

beta_function

`beta_function(a, b) → number` — Euler beta: $\Gamma(a) \cdot \Gamma(b) / \Gamma(a+b)$.

```
p beta_function(2, 3) # 1/12
```

beta_incomplete

`beta_incomplete(x, a, b) → number` — regularised incomplete beta function $I_x(a, b)$ via Lentz's continued fraction.

```
p beta_incomplete(0.5, 2, 3)
```

beta_pdf

`beta_pdf` (alias `betapdf`) evaluates the Beta distribution PDF at `x` with shape parameters `alpha` and `beta`.

```
p betapdf(0.5, 2, 5) # Beta(2,5) at x=0.5
```

betti_one

`betti_one` — operation `betti one`. Category: *econometrics*.

```
p betti_one $x
```

betti_two

`betti_two` — operation `betti two`. Category: *econometrics*.

```
p betti_two $x
```

betti_zero

`betti_zero` — operation `betti zero`. Category: *econometrics*.

```
p betti_zero $x
```

between

`between` — operation `between`. Category: *conversion / utility*.

```
p between $x
```

beylkin_wavelet_step

`beylkin_wavelet_step` — single-step update of `beylkin wavelet`. Category: **electrochemistry, batteries, fuel cells**.

```
p beylkin_wavelet_step $x
```

bfgs_h_update_1d

`bfgs_h_update_1d` — operation `bfgs h update 1d`. Category: **more extensions (2)**.

```
p bfgs_h_update_1d $x
```

bfield_solenoid

`bfield_solenoid` — operation `bfield solenoid`. Category: **em / optics / relativity**.

```
p bfield_solenoid $x
```

bfield_wire

`bfield_wire` — operation `bfield wire`. Category: **em / optics / relativity**.

```
p bfield_wire $x
```

bfs

`bfs` — operation `bfs`. Alias for `bfs_traverse`. Category: **extended stdlib**.

```
p bfs $x
```

bfs_count

`bfs_count` — count of `bfs`. Category: **networkx graph algorithms**.

```
p bfs_count $x
```

bfs_distances

`bfs_distances` — operation `bfs distances`. Category: **misc**.

```
p bfs_distances $x
```

bfs_traverse

`bfs_traverse` — operation `bfs traverse`. Category: **extended stdlib**.

```
p bfs_traverse $x
```

bg_black

`bg_black` — operation `bg black`. Alias for `red`. Category: **color / ansi**.

```
p bg_black $x
```

bg_blue

`bg_blue` — operation `bg blue`. Alias for `red`. Category: **color / ansi**.

```
p bg_blue $x
```

bg_bright_blue

bg_bright_blue — operation `bg bright blue`. Alias for `red`. Category: **color / ansi**.

```
p bg_bright_blue $x
```

bg_bright_cyan

bg_bright_cyan — operation `bg bright cyan`. Alias for `red`. Category: **color / ansi**.

```
p bg_bright_cyan $x
```

bg_bright_green

bg_bright_green — operation `bg bright green`. Alias for `red`. Category: **color / ansi**.

```
p bg_bright_green $x
```

bg_bright_magenta

bg_bright_magenta — operation `bg bright magenta`. Alias for `red`. Category: **color / ansi**.

```
p bg_bright_magenta $x
```

bg_bright_red

bg_bright_red — operation `bg bright red`. Alias for `red`. Category: **color / ansi**.

```
p bg_bright_red $x
```

bg_bright_white

bg_bright_white — operation `bg bright white`. Alias for `red`. Category: **color / ansi**.

```
p bg_bright_white $x
```

bg_bright_yellow

bg_bright_yellow — operation `bg bright yellow`. Alias for `red`. Category: **color / ansi**.

```
p bg_bright_yellow $x
```

bg_c256

bg_c256 — operation `bg c256`. Alias for `bg_color256`. Category: **color / ansi**.

```
p bg_c256 $x
```

bg_color256

bg_color256 — operation `bg color256`. Category: **color / ansi**.

```
p bg_color256 $x
```

bg_cyan

bg_cyan — operation `bg cyan`. Alias for `red`. Category: **color / ansi**.

```
p bg_cyan $x
```

bg_green

`bg_green` — operation `bg green`. Alias for `red`. Category: **color / ansi**.

```
p bg_green $x
```

bg_magenta

`bg_magenta` — operation `bg magenta`. Alias for `red`. Category: **color / ansi**.

```
p bg_magenta $x
```

bg_red

`bg_red` — operation `bg red`. Alias for `red`. Category: **color / ansi**.

```
p bg_red $x
```

bg_rgb

`bg_rgb` — operation `bg rgb`. Category: **color / ansi**.

```
p bg_rgb $x
```

bg_white

`bg_white` — operation `bg white`. Alias for `red`. Category: **color / ansi**.

```
p bg_white $x
```

bg_yellow

`bg_yellow` — operation `bg yellow`. Alias for `red`. Category: **color / ansi**.

```
p bg_yellow $x
```

bh_adjusted_p

`bh_adjusted_p` — operation `bh adjusted p`. Category: **bioinformatics deep**.

```
p bh_adjusted_p $x
```

bh_entropy

`bh_entropy` — operation `bh entropy`. Category: **cosmology / gr / flrw**.

```
p bh_entropy $x
```

bh_evaporation_time

`bh_evaporation_time` — operation `bh evaporation time`. Category: **cosmology / gr / flrw**.

```
p bh_evaporation_time $x
```

bhattacharyya_coefficient

`bhattacharyya_coefficient` — operation `bhattacharyya coefficient`. Category: **electrochemistry, batteries, fuel cells**.


```
p bhattacharyya_coefficient $x
```

bianchi_first_identity_check

`bianchi_first_identity_check` — operation `bianchi first identity check`. Category: **electrochemistry, batteries, fuel cells**.

```
p bianchi_first_identity_check $x
```

bianchi_second_identity_check

`bianchi_second_identity_check` — operation `bianchi second identity check`. Category: **electrochemistry, batteries, fuel cells**.

```
p bianchi_second_identity_check $x
```

bic

`bic` — operation `bic`. Category: **misc**.

```
p bic $x
```

biconnected_components

`biconnected_components` — operation `biconnected components`. Category: **networkx graph algorithms**.

```
p biconnected_components $x
```

bidirectional_bfs

`bidirectional_bfs(adj, s, t) → array` — search forward from `s` and backward from `t` simultaneously; meets in the middle.

```
p bidirectional_bfs($g, 0, 7)
```

bidirectional_dijkstra

`bidirectional_dijkstra` — operation `bidirectional dijkstra`. Category: **networkx graph algorithms**.

```
p bidirectional_dijkstra $x
```

bielliptic_total

`bielliptic_total` — operation `bielliptic total`. Category: **more extensions**.

```
p bielliptic_total $x
```

bignum_abs

`bignum_abs` — operation `bignum abs`. Category: **extras**.

```
p bignum_abs $x
```

bignum_add

`bignum_add` — operation `bignum add`. Category: **extras**.

```
p bignum_add $x
```

bignum_and

`bignum_and` — operation `bignum and`. Category: `*extras*`.

```
p bignum_and $x
```

bignum_bit_length

`bignum_bit_length` — operation `bignum bit length`. Category: `*extras*`.

```
p bignum_bit_length $x
```

bignum_clear_bit

`bignum_clear_bit` — operation `bignum clear bit`. Category: `*extras*`.

```
p bignum_clear_bit $x
```

bignum_compare

`bignum_compare` — operation `bignum compare`. Category: `*extras*`.

```
p bignum_compare $x
```

bignum_div

`bignum_div` — operation `bignum div`. Category: `*extras*`.

```
p bignum_div $x
```

bignum_factorial

`bignum_factorial` — operation `bignum factorial`. Category: `*extras*`.

```
p bignum_factorial $x
```

bignum_from_str

`bignum_from_str` — operation `bignum from str`. Category: `*extras*`.

```
p bignum_from_str $x
```

bignum_gcd

`bignum_gcd` — operation `bignum gcd`. Category: `*extras*`.

```
p bignum_gcd $x
```

bignum_is_negative

`bignum_is_negative` — operation `bignum is negative`. Category: `*extras*`.

```
p bignum_is_negative $x
```

bignum_is_prime

`bignum_is_prime` — operation `bignum is prime`. Category: `*extras*`.

```
p bignum_is_prime $x
```

bignum_is_zero

`bignum_is_zero` — operation `bignum is zero`. Category: **extras**.

```
p bignum_is_zero $x
```

bignum_lcm

`bignum_lcm` — operation `bignum lcm`. Category: **extras**.

```
p bignum_lcm $x
```

bignum_mod

`bignum_mod` — operation `bignum mod`. Category: **extras**.

```
p bignum_mod $x
```

bignum_modpow

`bignum_modpow` — operation `bignum modpow`. Category: **extras**.

```
p bignum_modpow $x
```

bignum_mul

`bignum_mul` — operation `bignum mul`. Category: **extras**.

```
p bignum_mul $x
```

bignum_negate

`bignum_negate` — operation `bignum negate`. Category: **extras**.

```
p bignum_negate $x
```

bignum_new

`bignum_new` — constructor of `bignum`. Category: **extras**.

```
p bignum_new $x
```

bignum_not

`bignum_not` — operation `bignum not`. Category: **extras**.

```
p bignum_not $x
```

bignum_or

`bignum_or` — operation `bignum or`. Category: **extras**.

```
p bignum_or $x
```

bignum_pow

`bignum_pow` — operation `bignum pow`. Category: **extras**.

```
p bignum_pow $x
```

bignum_random

`bignum_random` — operation `bignum random`. Category: *extras*.

```
p bignum_random $x
```

bignum_set_bit

`bignum_set_bit` — operation `bignum set bit`. Category: *extras*.

```
p bignum_set_bit $x
```

bignum_shl

`bignum_shl` — operation `bignum shl`. Category: *extras*.

```
p bignum_shl $x
```

bignum_shr

`bignum_shr` — operation `bignum shr`. Category: *extras*.

```
p bignum_shr $x
```

bignum_sign

`bignum_sign` — sign with private key of `bignum`. Category: *extras*.

```
p bignum_sign $x
```

bignum_sqrt

`bignum_sqrt` — operation `bignum sqrt`. Category: *extras*.

```
p bignum_sqrt $x
```

bignum_sub

`bignum_sub` — operation `bignum sub`. Category: *extras*.

```
p bignum_sub $x
```

bignum_test_bit

`bignum_test_bit` — operation `bignum test bit`. Category: *extras*.

```
p bignum_test_bit $x
```

bignum_to_int

`bignum_to_int` — convert from `bignum` to `int`. Category: *extras*.

```
p bignum_to_int $value
```

bignum_to_str

`bignum_to_str` — convert from `bignum` to `str`. Category: *extras*.

```
p bignum_to_str $value
```

bignum_xor

`bignum_xor` — operation `bignum xor`. Category: **extras**.

```
p bignum_xor $x
```

bigram

`bigram` — operation `bigram`. Alias for `bigrams`. Category: **string processing extras**.

```
p bigram $x
```

bigram_perplexity

`bigram_perplexity` — operation `bigram perplexity`. Category: **computational linguistics**.

```
p bigram_perplexity $x
```

bigrams

`bigrams` — operation `bigrams`. Category: **string processing extras**.

```
p bigrams $x
```

bilateral_filter_2d

`bilateral_filter_2d(image, size=5, sigma_s=2, sigma_r=25) → matrix` — edge-preserving smoothing. Spatial + range Gaussian weights.

bilinear_xform

`bilinear_xform` — operation `bilinear xform`. Category: **economics + game theory**.

```
p bilinear_xform $x
```

bilinear_zpk_xform

`bilinear_zpk_xform` — operation `bilinear zpk xform`. Category: **economics + game theory**.

```
p bilinear_zpk_xform $x
```

bin_of

`bin_of` — binary helper `of`. Alias for `to_bin`. Category: **base conversion**.

```
p bin_of $x
```

bin_packing_best_fit

`bin_packing_best_fit` — binary helper `packing best fit`. Category: **combinatorial optimization, scheduling**.

```
p bin_packing_best_fit $x
```

bin_packing_first_fit

`bin_packing_first_fit` — binary helper `packing first fit`. Category: **combinatorial optimization, scheduling**.

```
p bin_packing_first_fit $x
```

bin_packing_lower_bound_l1

`bin_packing_lower_bound_l1` — binary helper `packing lower bound l1`. Category: **combinatorial optimization, scheduling**.

```
p bin_packing_lower_bound_l1 $x
```

bin_packing_next_fit

`bin_packing_next_fit` — binary helper `packing next fit`. Category: **combinatorial optimization, scheduling**.

```
p bin_packing_next_fit $x
```

bin_to_gray

`bin_to_gray` — convert from `bin` to `gray`. Category: **misc**.

```
p bin_to_gray $value
```

binary_insert

`binary_insert` — operation `binary insert`. Category: **array / list operations extras**.

```
p binary_insert $x
```

binary_search

`binary_search` — operation `binary search`. Category: **collection**.

```
p binary_search $x
```

binary_to_gray

`binary_to_gray` — convert from `binary` to `gray`. Category: **base / gray code**.

```
p binary_to_gray $value
```

bincount

`bincount` — operation `bincount`. Category: **numpy + scipy.special**.

```
p bincount $x
```

binder_content_optimal

`binder_content_optimal` — operation `binder content optimal`. Category: **electrochemistry, batteries, fuel cells**.

```
p binder_content_optimal $x
```

binom

`binom` — operation `binom`. Alias for `binomial`. Category: **extended stdlib**.

```
p binom $x
```

binom_test

`binom_test` — exact binomial test. Returns two-sided p-value. Like R's `binom.test()`.

```
val $p = binomtest(7, 10, 0.5) # p-value for 7/10 successes vs p=0.5
```

binomial

binomial — operation **binomial**. Category: *extended stdlib*.

```
p binomial $x
```

binomial_test

binomial_test(*k*, *n*, *p*=0.5) → {*statistic*, *df*, *p_value*} — exact binomial test for *k* successes in *n* Bernoulli trials.

```
p binomial_test(7, 10, 0.5)
```

bininsert

bininsert — operation **bininsert**. Alias for **binary_insert**. Category: *array / list operations extras*.

```
p bininsert $x
```

biorthogonal_step

biorthogonal_step — single-step update of **biorthogonal**. Category: *electrochemistry, batteries, fuel cells*.

```
p biorthogonal_step $x
```

biot_number_step

biot_number_step — single-step update of **biot number**. Category: *climate, fluids, atmospheric*.

```
p biot_number_step $x
```

biquad_allpass_coeffs

biquad_allpass_coeffs — biquad IIR filter **allpass coeffs**. Category: *signal processing*.

```
p biquad_allpass_coeffs $x
```

biquad_bandpass

biquad_bandpass — biquad IIR filter **bandpass**. Category: *extras*.

```
p biquad_bandpass $x
```

biquad_bandpass_coeffs

biquad_bandpass_coeffs — biquad IIR filter **bandpass coeffs**. Category: *signal processing*.

```
p biquad_bandpass_coeffs $x
```

biquad_highpass

biquad_highpass — biquad IIR filter **highpass**. Category: *extras*.

```
p biquad_highpass $x
```

biquad_highpass_coeffs

biquad_highpass_coeffs — biquad IIR filter **highpass** coeffs. Category: *signal processing*.

```
p biquad_highpass_coeffs $x
```

biquad_highshelf_coeffs

biquad_highshelf_coeffs — biquad IIR filter **highshelf** coeffs. Category: *signal processing*.

```
p biquad_highshelf_coeffs $x
```

biquad_lowpass

biquad_lowpass — biquad IIR filter **lowpass**. Category: *extras*.

```
p biquad_lowpass $x
```

biquad_lowpass_coeffs

biquad_lowpass_coeffs — biquad IIR filter **lowpass** coeffs. Category: *signal processing*.

```
p biquad_lowpass_coeffs $x
```

biquad_lowshelf_coeffs

biquad_lowshelf_coeffs — biquad IIR filter **lowshelf** coeffs. Category: *signal processing*.

```
p biquad_lowshelf_coeffs $x
```

biquad_notch

biquad_notch — biquad IIR filter **notch**. Category: *extras*.

```
p biquad_notch $x
```

biquad_notch_coeffs

biquad_notch_coeffs — biquad IIR filter **notch** coeffs. Category: *signal processing*.

```
p biquad_notch_coeffs $x
```

biquad_peak_coeffs

biquad_peak_coeffs — biquad IIR filter **peak** coeffs. Category: *signal processing*.

```
p biquad_peak_coeffs $x
```

biquad_step

biquad_step — biquad IIR filter **step**. Category: *signal processing*.

```
p biquad_step $x
```

birnbaum_saunders_pdf

birnbaum_saunders_pdf — operation **birnbaum saunders pdf**. Category: *more extensions (2)*.

```
p birnbaum_saunders_pdf $x
```


bisect

`bisection` (alias `bisect`) finds a root of $f(x)=0$ in $[a,b]$ via the bisection method. Takes a code ref, `a`, `b`, and optional tolerance.

```
val $root = bisect(fn { _0**2 - 2 }, 1, 2) #  $\sqrt{2} \approx 1.4142$ 
```

bisimulation_step

`bisimulation_step` — single-step update of `bisimulation`. Category: **logic, proof, sat/smt, type theory**.

```
p bisimulation_step $x
```

bit_and

`bit_and` — bitwise op `and`. Category: **bit ops**.

```
p bit_and $x
```

bit_build

`bit_build` — bitwise op `build`. Alias for `fenwick_build`. Category: **misc**.

```
p bit_build $x
```

bit_clear

`bit_clear` — bitwise op `clear`. Category: **bit ops**.

```
p bit_clear $x
```

bit_clz

`bit_clz(x) → int` — count leading zeros in the 64-bit representation.

bit_count_ones

`bit_count_ones(x) → int` — population count: number of set bits.

```
p bit_count_ones(0b1011) # 3
```

bit_count_zeros

`bit_count_zeros(x) → int` — number of clear bits in the full 64-bit representation.

bit_ctz

`bit_ctz(x) → int` — count trailing zeros.

bit_extract

`bit_extract(x, start, len) → int` — extract `len` bits starting at bit position `start`.

```
p bit_extract(0xABCD, 4, 8) # 0xBC = 188
```

bit_first_clear

`bit_first_clear(x) → int` — position of the lowest unset bit.

bit_first_set

`bit_first_set(x)` → `int` — position of the lowest set bit (LSB=0); -1 if `x == 0`.

```
p bit_first_set(0b1100) # 2
```

bit_flip_prob

`bit_flip_prob` — bitwise op `flip prob`. Category: *quantum mechanics deep*.

```
p bit_flip_prob $x
```

bit_insert

`bit_insert(x, v, start, len)` → `int` — set `len` bits in `x` starting at `start` to the low bits of `v`.

```
p bit_insert(0, 0xFF, 8, 8) # 0xFF00
```

bit_last_clear

`bit_last_clear(x)` → `int` — position of the highest unset bit.

bit_last_set

`bit_last_set(x)` → `int` — position of the highest set bit; -1 if `x == 0`.

bit_length

`bit_length` — bitwise op `length`. Category: *base conversion*.

```
p bit_length $x
```

bit_log2_int

`bit_log2_int(x)` → `int` — $\lfloor \log_2(x) \rfloor$; -1 if `x == 0`.

```
p bit_log2_int(255) # 7
```

bit_not

`bit_not` — bitwise op `not`. Category: *bit ops*.

```
p bit_not $x
```

bit_or

`bit_or` — bitwise op `or`. Category: *bit ops*.

```
p bit_or $x
```

bit_parity

`bit_parity(x)` → `0|1` — even or odd number of set bits.

bit_query

`bit_query` — bitwise op `query`. Alias for `fenwick_query`. Category: `*misc*`.

```
p bit_query $x
```

bit_reverse

`bit_reverse` — bitwise op `reverse`. Alias for `bit_reverse_32`. Category: `*misc*`.

```
p bit_reverse $x
```

bit_reverse_32

`bit_reverse_32` — bitwise op `reverse` 32. Category: `*misc*`.

```
p bit_reverse_32 $x
```

bit_reverse_64

`bit_reverse_64` — bitwise op `reverse` 64. Category: `*cryptanalysis & number theory deep*`.

```
p bit_reverse_64 $x
```

bit_reverse_u16

`bit_reverse_u16(x) → int` — reverse the 16 lowest bits.

bit_reverse_u32

`bit_reverse_u32(x) → int` — reverse the 32 lowest bits.

```
p bit_reverse_u32(1) # 0x80000000
```

bit_reverse_u64

`bit_reverse_u64(x) → int` — reverse all 64 bits.

bit_reverse_u8

`bit_reverse_u8(x) → int` — reverse the 8 lowest bits.

```
p bit_reverse_u8(0x01) # 0x08 = 128
```

bit_rotate_left

`bit_rotate_left(x, n) → int` — cyclic left rotation by `n` bits (mod 64).

```
p bit_rotate_left(0x1, 4) # 0x10
```

bit_rotate_right

`bit_rotate_right(x, n) → int` — cyclic right rotation by `n` bits.

bit_set

`bit_set` — bitwise op `set`. Category: **bit ops**.

```
p bit_set $x
```

bit_swap_bytes

`bit_swap_bytes(x) → int` — reverse byte order (htonl/ntohl-style).

```
p bit_swap_bytes(0x12345678) # 0x78563412
```

bit_test

`bit_test` — bitwise op `test`. Category: **bit ops**.

```
p bit_test $x
```

bit_toggle

`bit_toggle` — bitwise op `toggle`. Category: **bit ops**.

```
p bit_toggle $x
```

bit_xor

`bit_xor` — bitwise op `xor`. Category: **bit ops**.

```
p bit_xor $x
```

bitcount

`bitcount` — operation `bitcount`. Category: **redis-flavour primitives**.

```
p bitcount $x
```

bitlen

`bitlen` — operation `bitlen`. Alias for `bit_length`. Category: **base conversion**.

```
p bitlen $x
```

bitop

`bitop` — operation `bitop`. Category: **redis-flavour primitives**.

```
p bitop $x
```

bitpos

`bitpos` — operation `bitpos`. Category: **redis-flavour primitives**.

```
p bitpos $x
```

bits_count

`bits_count` — count of `bits`. Category: **base conversion**.

```
p bits_count $x
```

bits_to_bytes

`bits_to_bytes` — convert from `bits` to `bytes`. Category: **unit conversions**.

```
p bits_to_bytes $value
```

bits_to_ip

`bits_to_ip` — convert from `bits` to `ip`. Category: **network / ip / cidr**.

```
p bits_to_ip $value
```

bitset_clear

`bitset_clear` — operation `bitset clear`. Category: **data structure helpers**.

```
p bitset_clear $x
```

bitset_new

`bitset_new` — constructor of `bitset`. Category: **data structure helpers**.

```
p bitset_new $x
```

bitset_set

`bitset_set` — operation `bitset set`. Category: **data structure helpers**.

```
p bitset_set $x
```

bitset_test

`bitset_test` — statistical test of `bitset`. Category: **data structure helpers**.

```
p bitset_test $x
```

bk_insert

`bk_insert(BK, WORD...)` → number of newly-inserted words (duplicates skipped).

bk_len

`bk_len(BK)` → number of stored words.

bk_query

`bk_query(BK, QUERY, MAX_DIST=2)` → list of `[word, distance]` arrayrefs within `MAX_DIST` edits, sorted by distance asc.

bk_tree

`bk_tree()` — Burkhard-Keller tree for fuzzy / typo retrieval. Uses Damerau-Levenshtein distance. Insert $O(\log n)$ avg; range queries $O(n^{(1-1/d)})$ for small `d`.

```
val $bk = bk_tree()
bk_insert($bk, qw(hello world help halo hells))
var @near = bk_query($bk, "hallo", 2) # [["hello",1], ["halo",1], ["hells",2]]
```

bkt

bkt — operation **bkt**. Alias for **bucket**. Category: *array / list operations extras*.

```
p bkt $x
```

bl

bl — operation **bl**. Alias for **butlast**. Category: *algebraic match*.

```
p bl $x
```

black

black — operation **black**. Alias for **red**. Category: *color / ansi*.

```
p black $x
```

black76_call

black76_call — operation **black76 call**. Category: *financial pricing models*.

```
p black76_call $x
```

black_hat_transform

black_hat_transform(image, size=3) → matrix — closing - image. Extracts dark details.

black_karasinski_drift

black_karasinski_drift — operation **black karasinski drift**. Category: *financial pricing models*.

```
p black_karasinski_drift $x
```

black_method_winner

black_method_winner — operation **black method winner**. Category: *climate, fluids, atmospheric*.

```
p black_method_winner $x
```

black_scholes_call

black_scholes_call(\$S, \$K, \$r, \$T, \$sigma) (alias **bscall**) — computes Black-Scholes price for a European call option. S=spot, K=strike, r=rate, T=time to expiry, sigma=volatility.

```
p bscall(100, 100, 0.05, 1, 0.2) # ~10.45
```

black_scholes_put

black_scholes_put(\$S, \$K, \$r, \$T, \$sigma) (alias **bsput**) — computes Black-Scholes price for a European put option.

```
p bsput(100, 100, 0.05, 1, 0.2) # ~5.57
```

blackman

window_blackman(\$n) (alias **blackman**) — generates a Blackman window of length n. Provides excellent sidelobe suppression at the cost of main lobe width.

```
val $w = blackman(1024)
```

blackman_harris_window

`blackman_harris_window` — operation `blackman harris window`. Category: **signal processing**.

```
p blackman_harris_window $x
```

blackman_w

`blackman_w` — operation `blackman w`. Category: **economics + game theory**.

```
p blackman_w $x
```

blackman_window

`blackman_window` — operation `blackman window`. Category: **signal processing**.

```
p blackman_window $x
```

blackscholes_call

`blackscholes_call` — operation `blackscholes call`. Alias for `bs_call`. Category: **financial pricing models**.

```
p blackscholes_call $x
```

blackscholes_put

`blackscholes_put` — operation `blackscholes put`. Alias for `bs_put`. Category: **financial pricing models**.

```
p blackscholes_put $x
```

blast_kmer_index

`blast_kmer_index` — index of `blast kmer`. Category: **test runner**.

```
p blast_kmer_index $x
```

blessed

`blessed` — operation `blessed`. Category: **list / aggregate**.

```
p blessed $x
```

blink

`blink` — operation `blink`. Alias for `red`. Category: **color / ansi**.

```
p blink $x
```

bloch_purity_check

`bloch_purity_check` — operation `bloch purity check`. Category: **quantum mechanics deep**.

```
p bloch_purity_check $x
```

bloch_sphere_x

`bloch_sphere_x` — operation `bloch sphere x`. Category: **quantum**.

```
p bloch_sphere_x $x
```

bloch_sphere_z

`bloch_sphere_z` — operation `bloch sphere z`. Category: **quantum**.

```
p bloch_sphere_z $x
```

bloch_to_density_real

`bloch_to_density_real` — convert from `bloch` to `density real`. Category: **quantum mechanics deep**.

```
p bloch_to_density_real $value
```

block_devices

`block_devices` — operation `block devices`. Category: **filesystem extensions**.

```
p block_devices $x
```

bloom

`bloom_filter(CAPACITY, FPR=0.01)` — Bloom filter sized for `CAPACITY` distinct items with target false-positive rate `FPR`. Bit count rounds up to a power of two; `k` (probe count) is computed for the (cap, fpr) pair. Probe indexing uses Kirsch–Mitzenmacher double-hashing over xxh3-128; no allocations on `bloom_add` / `bloom_contains`.

```
val $bf = bloom_filter(10_000, 0.001)
bloom_add($bf, "alice"); bloom_add($bf, "bob")
p bloom_contains($bf, "alice")      # 1
p bloom_contains($bf, "carol")     # 0
var $bytes = bloom_serialize($bf)
var $r = bloom_deserialize($bytes)
```

Family: `bloom_add`, `bloom_contains`, `bloom_len`, `bloom_clear`, `bloom_merge`, `bloom_fpr`, `bloom_bits`, `bloom_serialize`, `bloom_deserialize`.

bloom_add

`bloom_add(BF, KEY)` — insert `KEY` into a Bloom filter. Returns `1` if every probe bit was previously unset (newly inserted, no false positive collision on insert), `0` if all probe bits already `1` (key already present OR false positive). `bloom_len(BF)` tracks the unique-insert count.

bloom_bits

`bloom_bits(BF)` — total bit count of the underlying array (always a power of two ≥ 64).

bloom_clear

`bloom_clear(BF)` — zero the bit array and reset the insertion counter. Returns the same BF for chaining.

bloom_contains

`bloom_contains(BF, KEY)` — `1` if `KEY` may be present (no false negatives), `0` if definitely absent. Probe count `k` = `bloom_filter`'s (capacity, fpr)-derived constant.

bloom_count

`bloom_len(BF)` — unique-insert count seen by this filter. May exceed `capacity` if you over-insert; once it does, `bloom_fpr` will have climbed past target.

bloom_deserialize

`bloom_deserialize(BYTES)` → `BloomFilter` | `undef` — restore from `bloom_serialize` output. Returns `undef` on format mismatch.

bloom_fpr

`bloom_fpr(BF)` — current estimated false-positive rate from the running insertion count: $(1 - \exp(-k \cdot n/m))^k$. When this drifts above your target, rebuild with a larger capacity.

bloom_merge

`bloom_merge(BF, OTHER)` — bitwise OR union with another filter of identical geometry. Returns `1` on success, `0` if bit count or `k` differ. After merge, `bloom_contains` of any key inserted into either source returns `1`.

bloom_serialize

`bloom_serialize(BF)` → `BYTES` — versioned wire format (`STKBLOM\x01` magic + `log2_cap` + `k` + inserted + `capacity_hint` + `fpr_hint` + bit words). Pair with `bloom_deserialize`.

blossom_match_step

`blossom_match_step` — single-step update of `blossom match`. Category: **combinatorial optimization, scheduling**.

```
p blossom_match_step $x
```

blosum45_score

`blosum45_score` — score of `blosum45`. Category: **bioinformatics deep**.

```
p blosum45_score $x
```

blue

`blue` — operation `blue`. Alias for `red`. Category: **color / ansi**.

```
p blue $x
```

blue_bold

`blue_bold` — operation `blue bold`. Alias for `red`. Category: **color / ansi**.

```
p blue_bold $x
```

bm25_score

`bm25_score(query, doc, avg_dl, n_total, df, k1=1.5, b=0.75)` → `number` — Okapi BM25 retrieval score.

bm_horspool

`bm_horspool` — operation `bm horspool`. Alias for `horspool_search`. Category: **misc**.

```
p bm_horspool $x
```

bmi

bmi — operation `bmi`. Category: **finance**.

```
p bmi $x
```

bmi_calc

bmi_calc — operation `bmi calc`. Category: **physics formulas**.

```
p bmi_calc $x
```

bmp_header_read

bmp_header_read(hex_bytes) → hash — parse BMP header → {format, width, height, bits_per_pixel} or UNDEF.

```
p bmp_header_read("424D...")
```

bmr_harris_benedict_female

bmr_harris_benedict_female — operation `bmr harris benedict female`. Category: **misc**.

```
p bmr_harris_benedict_female $x
```

bmr_harris_benedict_male

bmr_harris_benedict_male — operation `bmr harris benedict male`. Category: **misc**.

```
p bmr_harris_benedict_male $x
```

bode_gain_margin

bode_gain_margin — operation `bode gain margin`. Category: **robotics & control**.

```
p bode_gain_margin $x
```

bode_phase

bode_phase — operation `bode phase`. Alias for `bode_phase_deg`. Category: **control theory**.

```
p bode_phase $x
```

bode_phase_margin

bode_phase_margin — operation `bode phase margin`. Category: **robotics & control**.

```
p bode_phase_margin $x
```

body_surface_area_dubois

body_surface_area_dubois — operation `body surface area dubois`. Category: **misc**.

```
p body_surface_area_dubois $x
```

body_wave_mb

`body_wave_mb` — operation `body wave mb`. Category: **geology, seismology, mineralogy**.

```
p body_wave_mb $x
```

bogacki_shampine_step

`bogacki_shampine_step` — single-step update of `bogacki shampine`. Category: **ode advanced**.

```
p bogacki_shampine_step $x
```

bohr_energy_ev

`bohr_energy_ev` — operation `bohr energy ev`. Category: **chemistry**.

```
p bohr_energy_ev $x
```

bohr_magneton

`bohr_magneton` — operation `bohr magneton`. Category: **physics constants**.

```
p bohr_magneton $x
```

bohr_radius_n

`bohr_radius_n` — operation `bohr radius n`. Category: **chemistry**.

```
p bohr_radius_n $x
```

bold

`bold` — operation `bold`. Alias for `red`. Category: **color / ansi**.

```
p bold $x
```

bold_blue

`bold_blue` — operation `bold blue`. Alias for `red`. Category: **color / ansi**.

```
p bold_blue $x
```

bold_cyan

`bold_cyan` — operation `bold cyan`. Alias for `red`. Category: **color / ansi**.

```
p bold_cyan $x
```

bold_green

`bold_green` — operation `bold green`. Alias for `red`. Category: **color / ansi**.

```
p bold_green $x
```

bold_magenta

`bold_magenta` — operation `bold magenta`. Alias for `red`. Category: **color / ansi**.

```
p bold_magenta $x
```

bold_red

bold_red — operation **bold red**. Alias for **red**. Category: **color / ansi**.

```
p bold_red $x
```

bold_white

bold_white — operation **bold white**. Alias for **red**. Category: **color / ansi**.

```
p bold_white $x
```

bold_yellow

bold_yellow — operation **bold yellow**. Alias for **red**. Category: **color / ansi**.

```
p bold_yellow $x
```

bollinger_lower

bollinger_lower(prices, period=20, mult=2) → array — $sma - mult \cdot stddev$ of the trailing window.

```
p bollinger_lower(\@closes, 20, 2)
```

bollinger_middle

bollinger_middle(prices, period=20) → array — alias for **sma**(prices, period); the central band of Bollinger bands.

```
p bollinger_middle(\@closes, 20)
```

bollinger_upper

bollinger_upper(prices, period=20, mult=2) → array — $sma + mult \cdot stddev$ of the trailing window.

```
p bollinger_upper(\@closes, 20, 2)
```

boltz

boltz — operation **boltz**. Alias for **boltzmann_constant**. Category: **constants**.

```
p boltz $x
```

boltzmann

boltzmann — operation **boltzmann**. Category: **math / physics constants**.

```
p boltzmann $x
```

boltzmann_choose

boltzmann_choose — operation **boltzmann choose**. Category: **test runner**.

```
p boltzmann_choose $x
```

boltzmann_const

`boltzmann_const` — operation `boltzmann const`. Category: **misc**.

```
p boltzmann_const $x
```

boltzmann_constant

`boltzmann_constant` — operation `boltzmann constant`. Category: **constants**.

```
p boltzmann_constant $x
```

boltzmann_softmax_action

`boltzmann_softmax_action` — operation `boltzmann softmax action`. Category: **electrochemistry, batteries, fuel cells**.

```
p boltzmann_softmax_action $x
```

bond_duration

`duration($face, $coupon_rate, $ytm, $periods)` (alias `bond_duration`) — computes the Macaulay duration of a bond, measuring interest rate sensitivity as weighted average time to receive cash flows.

```
p duration(1000, 0.05, 0.05, 10) # ~7.7 years
```

bond_price

`bond_price($face, $coupon_rate, $ytm, $periods)` — computes the present value (price) of a bond given face value, coupon rate, yield to maturity, and number of periods.

```
p bond_price(1000, 0.05, 0.04, 10) # ~1081 (premium)
p bond_price(1000, 0.05, 0.06, 10) # ~926 (discount)
```

bond_yield

`bond_yield($price, $face, $coupon_rate, $periods)` — computes the yield to maturity of a bond given its current price. Uses Newton-Raphson iteration.

```
p bond_yield(950, 1000, 0.05, 10) # ~5.7%
```

bondi_mass_step

`bondi_mass_step` — single-step update of `bondi mass`. Category: **electrochemistry, batteries, fuel cells**.

```
p bondi_mass_step $x
```

bondprc

`bondprc` — operation `bondprc`. Alias for `bond_price`. Category: **finance (extended)**.

```
p bondprc $x
```

bondyld

`bondyld` — operation `bondyld`. Alias for `bond_yield`. Category: **finance (extended)**.

```
p bondyld $x
```

bool_each

`bool_each` — boolean helper `each`. Category: **trivial numeric helpers**.

```
p bool_each $x
```

bool_to_int

`bool_to_int` — convert from `bool` to `int`. Category: **boolean combinators**.

```
p bool_to_int $value
```

boole_int

`boole_int` — operation `boole int`. Alias for `boole_rule`. Category: **more extensions (2)**.

```
p boole_int $x
```

boole_rule

`boole_rule` — operation `boole rule`. Category: **more extensions (2)**.

```
p boole_rule $x
```

bootstrap_resample

`bootstrap_resample(xs, n=len(xs), seed=0) → array` — `n` samples drawn with replacement from `xs` using a deterministic PRNG.

```
p bootstrap_resample([1,2,3,4,5], 1000, 42)
```

bootstrap_se_estimate

`bootstrap_se_estimate` — bootstrap resampling helper `se estimate`. Category: **econometrics**.

```
p bootstrap_se_estimate $x
```

borda_count_step

`borda_count_step` — single-step update of `borda count`. Category: **climate, fluids, atmospheric**.

```
p borda_count_step $x
```

borders_of_string

`borders_of_string` — operation `borders of string`. Category: **more extensions**.

```
p borders_of_string $x
```

bornhuetter_ferguson

`bornhuetter_ferguson` — operation `bornhuetter ferguson`. Category: **actuarial science**.

```
p bornhuetter_ferguson $x
```

boruvka_step

`boruvka_step` — single-step update of `boruvka`. Category: **networkx graph algorithms**.

```
p boruvka_step $x
```

bose_einstein_g

`bose_einstein_g` — operation `bose einstein g`. Category: **special functions extra**.

```
p bose_einstein_g $x
```

bose_einstein_pmf

`bose_einstein_pmf` — operation `bose einstein pmf`. Category: **quantum mechanics deep**.

```
p bose_einstein_pmf $x
```

boston_mechanism_step

`boston_mechanism_step` — single-step update of `boston mechanism`. Category: **climate, fluids, atmospheric**.

```
p boston_mechanism_step $x
```

both

`both` — operation `both`. Category: **boolean combinators**.

```
p both $x
```

bottom_k_max_heap

`bottom_k_max_heap` — operation `bottom k max heap`. Category: **iterator + string-distance extras**.

```
p bottom_k_max_heap $x
```

bottom_n_by

`bottom_n_by` — operation `bottom n by`. Category: **iterator + string-distance extras**.

```
p bottom_n_by $x
```

bowley_skewness

`bowley_skewness` — operation `bowley skewness`. Category: **more extensions**.

```
p bowley_skewness $x
```

box_blur_kernel

`box_blur_kernel` — convolution kernel of `box blur`. Category: **more extensions**.

```
p box_blur_kernel $x
```

box_pierce_test

`box_pierce_test` — statistical test of `box pierce`. Category: **econometrics**.

```
p box_pierce_test $x
```

boyer_lindquist_step

`boyer_lindquist_step` — single-step update of `boyer lindquist`. Category: **electrochemistry, batteries, fuel cells**.

```
p boyer_lindquist_step $x
```

boyer_moore_majority

`boyer_moore_majority` — operation `boyer moore majority`. Category: **iterator + string-distance extras**.

```
p boyer_moore_majority $x
```

boykov_kolmogorov_step

`boykov_kolmogorov_step` — single-step update of `boykov kolmogorov`. Category: **combinatorial optimization, scheduling**.

```
p boykov_kolmogorov_step $x
```

bp_elevation

`bp_elevation` — operation `bp elevation`. Category: **chemistry**.

```
p bp_elevation $x
```

bpm_to_midi_tick_us

`bpm_to_midi_tick_us` — convert from `bpm` to `midi tick us`. Category: **astronomy / music / color / units**.

```
p bpm_to_midi_tick_us $value
```

bpm_to_period

`bpm_to_period` — convert from `bpm` to `period`. Category: **misc**.

```
p bpm_to_period $value
```

bpm_to_spb

`bpm_to_spb` — convert from `bpm` to `spb`. Category: **astronomy / music / color / units**.

```
p bpm_to_spb $value
```

bracelet_count

`bracelet_count` — count of `bracelet`. Category: **misc**.

```
p bracelet_count $x
```

bracket

`bracket` — operation `bracket`. Category: **string helpers**.

```
p bracket $x
```

braille

`braille` — operation `braille`. Alias for `braille_encode`. Category: **encoding / phonetics**.


```
p braille $x
```

braille_encode

`braille_encode` — encode of `braille`. Category: *encoding / phonetics*.

```
p braille_encode $x
```

branch_and_bound_step

`branch_and_bound_step` — single-step update of `branch` and `bound`. Category: *combinatorial optimization, scheduling*.

```
p branch_and_bound_step $x
```

branch_length_substitutions

`branch_length_substitutions` — operation `branch length substitutions`. Category: *bioinformatics deep*.

```
p branch_length_substitutions $x
```

brans_dicke_step

`brans_dicke_step` — single-step update of `brans dicke`. Category: *electrochemistry, batteries, fuel cells*.

```
p brans_dicke_step $x
```

braycurtis_dist

`braycurtis_dist(a, b) → number` — $\sum |a_i - b_i| / \sum |a_i + b_i|$. Ecological-similarity classic.

brayton_efficiency

`brayton_efficiency` — operation `brayton efficiency`. Category: *misc*.

```
p brayton_efficiency $x
```

break_even

`break_even` — operation `break even`. Category: *finance*.

```
p break_even $x
```

break_even_price

`break_even_price(fixed_cost, qty, variable_cost) → number` — price needed at given quantity: `fixed/qty + variable`.

```
p break_even_price(10000, 1000, 15)
```

break_even_qty

`break_even_qty(fixed_cost, price, variable_cost) → number` — units to sell to cover fixed + variable: `fixed / (price - variable)`.

```
p break_even_qty(10000, 25, 15)
```

break_fn

`break_fn` — operation `break fn`. Category: **haskell list functions**.

```
p break_fn $x
```

breakeven

`breakeven` — operation `breakeven`. Alias for `break_even`. Category: **finance**.

```
p breakeven $x
```

brent

`brent` — operation `brent`. Alias for `brent_root`. Category: **more extensions (2)**.

```
p brent $x
```

brent_root

`brent_root` — operation `brent root`. Category: **more extensions (2)**.

```
p brent_root $x
```

breusch_godfrey

`breusch_godfrey` — operation `breusch godfrey`. Category: **statsmodels**.

```
p breusch_godfrey $x
```

breusch_godfrey_test

`breusch_godfrey_test` — statistical test of `breusch godfrey`. Category: **econometrics**.

```
p breusch_godfrey_test $x
```

breusch_pagan_lm

`breusch_pagan_lm` — operation `breusch pagan lm`. Category: **econometrics**.

```
p breusch_pagan_lm $x
```

breusch_pagan_test

`breusch_pagan_test` — statistical test of `breusch pagan`. Category: **econometrics**.

```
p breusch_pagan_test $x
```

brewster

`brewster` — operation `brewster`. Alias for `brewster_angle`. Category: **physics formulas**.

```
p brewster $x
```

brewster_angle

`brewster_angle` — operation `brewster angle`. Category: **physics formulas**.

```
p brewster_angle $x
```

bridge_edge

`bridge_edge` — operation `bridge edge`. Category: **networkx graph algorithms**.

```
p bridge_edge $x
```

bridges

`bridges` — operation `bridges`. Category: **graph algorithms**.

```
p bridges $x
```

bridges_edges

`bridges_edges` — operation `bridges edges`. Category: **misc**.

```
p bridges_edges $x
```

brier_score

`brier_score` — score of `brier`. Category: **ml extensions**.

```
p brier_score $x
```

bright_blue

`bright_blue` — operation `bright blue`. Alias for `red`. Category: **color / ansi**.

```
p bright_blue $x
```

bright_cyan

`bright_cyan` — operation `bright cyan`. Alias for `red`. Category: **color / ansi**.

```
p bright_cyan $x
```

bright_green

`bright_green` — operation `bright green`. Alias for `red`. Category: **color / ansi**.

```
p bright_green $x
```

bright_magenta

`bright_magenta` — operation `bright magenta`. Alias for `red`. Category: **color / ansi**.

```
p bright_magenta $x
```

bright_red

`bright_red` — operation `bright red`. Alias for `red`. Category: **color / ansi**.

```
p bright_red $x
```

bright_white

`bright_white` — operation `bright white`. Alias for `red`. Category: **color / ansi**.

```
p bright_white $x
```

bright_yellow

`bright_yellow` — operation `bright yellow`. Alias for `red`. Category: **color / ansi**.

```
p bright_yellow $x
```

brkf

`brkf` — operation `brkf`. Alias for `break_fn`. Category: **haskell list functions**.

```
p brkf $x
```

broadcast_channel_new

`broadcast_channel_new()` → `broadcast_channel` — fan-out channel: every subscriber receives every value.

broadcast_channel_publish

`broadcast_channel_publish` — operation `broadcast channel publish`. Category: **extras**.

```
p broadcast_channel_publish $x
```

broadcast_channel_subscribe

`broadcast_channel_subscribe(ch)` → `receiver` — create a new receiver subscribed to all future messages.

bron_kerbosch

`bron_kerbosch` — operation `bron kerbosch`. Category: **networkx graph algorithms**.

```
p bron_kerbosch $x
```

brotli_distance_code_count

`brotli_distance_code_count` — count of `brotli distance code`. Category: **electrochemistry, batteries, fuel cells**.

```
p brotli_distance_code_count $x
```

brotli_encode_meta

`brotli_encode_meta` — operation `brotli encode meta`. Category: **b81-misc-utility**.

```
p brotli_encode_meta $x
```

brotli_huffman_table

`brotli_huffman_table` — lookup table of `brotli huffman`. Category: **archive/encoding format primitives**.

```
p brotli_huffman_table $x
```

brotli_meta_block

`brotli_meta_block` — operation `brotli meta block`. Category: **archive/encoding format primitives**.

```
p brotli_meta_block $x
```

brown_york_quasilocal

`brown_york_quasilocal` — operation `brown york quasilocal`. Category: *electrochemistry, batteries, fuel cells*.

```
p brown_york_quasilocal $x
```

brunt_vaisala_full

`brunt_vaisala_full` — operation `brunt vaisala full`. Category: *climate, fluids, atmospheric*.

```
p brunt_vaisala_full $x
```

brzowski_derivative

`brzowski_derivative` — operation `brzowski derivative`. Category: *compilers / parsing*.

```
p brzowski_derivative $x
```

bs_call

`bs_call` — operation `bs call`. Category: *financial pricing models*.

```
p bs_call $x
```

bs_delta

`bs_delta` (aliases `bsdelta`, `option_delta`) computes the Black-Scholes delta ($\partial C / \partial S$). Args: S, K, T, r, sigma.

```
p bsdelta(100, 100, 1, 0.05, 0.2) # ≈ 0.64
```

bs_gamma

`bs_gamma` computes the Black-Scholes gamma ($\partial^2 C / \partial S^2$). Measures convexity of option value.

```
p bsgamma(100, 100, 1, 0.05, 0.2) # ≈ 0.019
```

bs_put

`bs_put` — operation `bs put`. Category: *financial pricing models*.

```
p bs_put $x
```

bs_rho

`bs_rho` computes the Black-Scholes rho ($\partial C / \partial r$). Sensitivity to interest rate.

```
p bsrho(100, 100, 1, 0.05, 0.2) # ≈ 46
```

bs_rho_call

`bs_rho_call` — operation `bs rho call`. Category: *financial pricing models*.

```
p bs_rho_call $x
```

bs_theta

`bs_theta` computes the Black-Scholes theta ($\partial C / \partial t$). Time decay per unit time.

```
p bstheta(100, 100, 1, 0.05, 0.2) # negative (time decay)
```

bs_theta_call

bs_theta_call — operation **bs theta call**. Category: *financial pricing models*.

```
p bs_theta_call $x
```

bs_vega

bs_vega computes the Black-Scholes vega ($\partial C / \partial \sigma$). Sensitivity to volatility.

```
p bsvega(100, 100, 1, 0.05, 0.2) # ≈ 37.5
```

bsa_dubois

bsa_dubois — operation **bsa dubois**. Alias for **body_surface_area_dubois**. Category: *misc*.

```
p bsa_dubois $x
```

bsearch

bsearch — operation **bsearch**. Alias for **binary_search**. Category: *collection*.

```
p bsearch $x
```

bsgs_discrete_log

bsgs_discrete_log — operation **bsgs discrete log**. Category: *cryptanalysis & number theory deep*.

```
p bsgs_discrete_log $x
```

bsize

bsize — operation **bsize**. Alias for **byte_size**. Category: *misc / utility*.

```
p bsize $x
```

btc_addr_check

btc_addr_check — operation **btc addr check**. Category: *b82-misc-utility*.

```
p btc_addr_check $x
```

btch

btch — operation **btch**. Alias for **batched**. Category: *additional missing stdlib functions*.

```
p btch $x
```

btrim

btrim — operation **btrim**. Category: *postgres sql strings, json, aggregates*.

```
p btrim $x
```

btu_to_joules

btu_to_joules — convert from **btu** to **joules**. Category: *astronomy / music / color / units*.

```
p btu_to_joules $value
```

bucket

`bucket` — operation `bucket`. Category: **array / list operations extras**.

```
p bucket $x
```

bucket_available

`rl_available(RL)` → tokens currently available (token-bucket) or remaining headroom (leaky-bucket).

bucket_try_take

`rl_try_take(RL, COST=1)` → 1 on success (capacity available / no overflow), 0 on rejection.

buffer_capacity_acid_base

`buffer_capacity_acid_base` — operation `buffer capacity acid base`. Category: **electrochemistry, batteries, fuel cells**.

```
p buffer_capacity_acid_base $x
```

bulirsch_stoer_step

`bulirsch_stoer_step` — single-step update of `bulirsch stoer`. Category: **ode advanced**.

```
p bulirsch_stoer_step $x
```

buoyancy_force

`buoyancy_force(fluid_density, submerged_volume, g=9.81)` → `number` — $\rho \cdot V \cdot g$ (Archimedes).

burr_xii_cdf

`burr_xii_cdf` — operation `burr xii cdf`. Category: **more extensions (2)**.

```
p burr_xii_cdf $x
```

burr_xii_pdf

`burr_xii_pdf` — operation `burr xii pdf`. Category: **more extensions (2)**.

```
p burr_xii_pdf $x
```

business_days_between

`business_days_between` — operation `business days between`. Category: **misc**.

```
p business_days_between $x
```

busy_beaver

`busy_beaver` — operation `busy beaver`. Category: **math / number theory extras**.

```
p busy_beaver $x
```

butlast

`butlast` — operation `butlast`. Category: **algebraic match**.

```
p butlast $x
```

butler_volmer_current

`butler_volmer_current` — operation `butler volmer current`. Category: **electrochemistry, batteries, fuel cells**.

```
p butler_volmer_current $x
```

butter_hp_mag

`butter_hp_mag` — operation `butter hp mag`. Category: **economics + game theory**.

```
p butter_hp_mag $x
```

butter_lp_re

`butter_lp_re` — operation `butter lp re`. Category: **economics + game theory**.

```
p butter_lp_re $x
```

butterworth_order

`butterworth_order` — operation `butterworth order`. Category: **signal processing**.

```
p butterworth_order $x
```

butterworth_prewarp

`butterworth_prewarp` — operation `butterworth prewarp`. Category: **signal processing**.

```
p butterworth_prewarp $x
```

bwt_decode

`bwt_decode` — Burrows-Wheeler transform `decode`. Category: **more extensions**.

```
p bwt_decode $x
```

bwt_encode

`bwt_encode` — Burrows-Wheeler transform `encode`. Category: **more extensions**.

```
p bwt_encode $x
```

bwt_invert

`bwt_invert(data, index) → string` — invert the Burrows-Wheeler transform.

```
p bwt_invert("nnbaaa", 3) # "banana"
```

bwt_transform

`bwt_transform(s) → {data, index}` — Burrows-Wheeler transform: returns last column + primary index.

```
p bwt_transform("banana") # {data: "nnbaaa", index: 3}
```


byte_length

`byte_length` — operation `byte length`. Category: **string helpers**.

```
p byte_length $x
```

byte_size

`byte_size` — operation `byte size`. Category: **misc / utility**.

```
p byte_size $x
```

bytecode_disasm_step

`bytecode_disasm_step` — single-step update of `bytecode_disasm`. Category: **compilers / parsing**.

```
p bytecode_disasm_step $x
```

bytes_to_bits

`bytes_to_bits` — convert from `bytes` to `bits`. Category: **unit conversions**.

```
p bytes_to_bits $value
```

bytes_to_gb

`bytes_to_gb` — convert from `bytes` to `gb`. Category: **unit conversions**.

```
p bytes_to_gb $value
```

bytes_to_hex_str

`bytes_to_hex_str` — convert from `bytes` to `hex str`. Category: **string helpers**.

```
p bytes_to_hex_str $value
```

bytes_to_ip

`bytes_to_ip` — convert from `bytes` to `ip`. Category: **network / ip / cidr**.

```
p bytes_to_ip $value
```

bytes_to_kb

`bytes_to_kb` — convert from `bytes` to `kb`. Category: **unit conversions**.

```
p bytes_to_kb $value
```

bytes_to_mac

`bytes_to_mac` — convert from `bytes` to `mac`. Category: **network / ip / cidr**.

```
p bytes_to_mac $value
```

bytes_to_mb

`bytes_to_mb` — convert from `bytes` to `mb`. Category: **unit conversions**.

```
p bytes_to_mb $value
```

bz2_encode_step

`bz2_encode_step` — single-step update of `bz2_encode`. Category: *b81-misc-utility*.

```
p bz2_encode_step $x
```

c256

`c256` — operation `c256`. Alias for `color256`. Category: *color / ansi*.

```
p c256 $x
```

c2s

`c2s` — operation `c2s`. Alias for `camel_to_snake`. Category: *string processing extras*.

```
p c2s $x
```

c_parallel

`c_parallel` — operation `c_parallel`. Alias for `capacitance_parallel`. Category: *misc*.

```
p c_parallel $x
```

c_series

`c_series` — operation `c_series`. Alias for `capacitance_series`. Category: *misc*.

```
p c_series $x
```

c_to_f

`c_to_f` — convert from `c` to `f`. Category: *unit conversions*.

```
p c_to_f $value
```

c_to_k

`c_to_k` — convert from `c` to `k`. Category: *unit conversions*.

```
p c_to_k $value
```

cabling_pair_signature

`cabling_pair_signature` — operation `cabling pair signature`. Category: *econometrics*.

```
p cabling_pair_signature $x
```

cache_clear

`cache_clear` — operation `cache clear`. Category: *caching*.

```
p cache_clear $x
```

cache_control_parse

`cache_control_parse` — operation `cache control parse`. Category: *b81-misc-utility*.

```
p cache_control_parse $x
```

cache_exists

cache_exists — operation `cache exists`. Category: **caching**.

```
p cache_exists $x
```

cache_stats

cache_stats — operation `cache stats`. Category: **caching**.

```
p cache_stats $x
```

cacheview

cacheview — operation `cacheview`. Category: **caching**.

```
p cacheview $x
```

caesar_shift

caesar_shift — operation `caesar shift`. Category: **string**.

```
p caesar_shift $x
```

cagr

`cagr(start_value, end_value, periods) → number` — Compound Annual Growth Rate:
 $(\text{end/start})^{(1/\text{periods})} - 1$.

```
p cagr(10000, 25000, 7)
```

cake_number

cake_number — operation `cake number`. Category: **misc**.

```
p cake_number $x
```

cal_to_joules

cal_to_joules — convert from `cal` to `joules`. Category: **file stat / path**.

```
p cal_to_joules $value
```

calmar_ratio

`calmar_ratio(returns) → number` — CAGR divided by max drawdown magnitude; higher is better.

```
p calmar_ratio(@returns)
```

calorimeter_dt

calorimeter_dt — operation `calorimeter dt`. Category: **chemistry**.

```
p calorimeter_dt $x
```

calorimetric_heat_battery

`calorimetric_heat_battery` — operation `calorimetric heat battery`. Category: **electrochemistry, batteries, fuel cells**.

```
p calorimetric_heat_battery $x
```

camel_to_snake

`camel_to_snake` — convert from `camel` to `snake`. Category: **string processing extras**.

```
p camel_to_snake $value
```

camel_words

`camel_words` — operation `camel words`. Category: **string helpers**.

```
p camel_words $x
```

canberra_dist

`canberra_dist(a, b) → number` — $\sum |a_i - b_i| / (|a_i| + |b_i|)$. Robust to outliers.

candlestick_pattern_doji

`candlestick_pattern_doji(open, high, low, close, tol=0.1) → 0|1` — 1 if `|close-open|` is within `tol` of the high-low range (indecision bar).

```
p candlestick_pattern_doji(100, 101, 99, 100.05)
```

candlestick_pattern_engulfing

`candlestick_pattern_engulfing(prev_open, prev_close, open, close) → -1|0|1` — bullish (1) / bearish (-1) engulfing.

```
p candlestick_pattern_engulfing(101, 100, 99, 102)
```

candlestick_pattern_evening_star

`candlestick_pattern_evening_star([open,high,low,close] × 3) → 0|1` — bearish counterpart of morning star.

candlestick_pattern_hammer

`candlestick_pattern_hammer(open, high, low, close) → 0|1` — bullish reversal: small body near top, long lower shadow $\geq 2 \times$ body.

```
p candlestick_pattern_hammer(100, 102, 95, 101)
```

candlestick_pattern_morning_star

`candlestick_pattern_morning_star([open,high,low,close] × 3) → 0|1` — bullish three-bar reversal: long red, small body, long green.

candlestick_pattern_three_black_crows

`candlestick_pattern_three_black_crows([open,high,low,close] × 3) → 0|1` — three consecutive long red bars; strong bearish continuation.

candlestick_pattern_three_white_soldiers

`candlestick_pattern_three_white_soldiers([open,high,low,close] × 3) → 0|1` — three consecutive long green bars; strong bullish continuation.

canny_edges_full

`canny_edges_full(image, low=50, high=150) → matrix` — full 4-stage Canny pipeline: 5×5 Gaussian blur ($\sigma=1.4$), Sobel gradient + atan2 angle (mod 180°), non-max suppression along gradient direction, double-threshold + 8-connected flood-fill hysteresis. Output is binary: 255 for strong/connected-to-strong pixels, 0 otherwise.

canny_edges_simple

`canny_edges_simple(image, low=50, high=150) → matrix` — simplified Canny: Sobel gradient + double threshold (no NMS).

canny_threshold_step

`canny_threshold_step` — single-step update of `canny threshold`. Category: *electrochemistry, batteries, fuel cells*.

```
p canny_threshold_step $x
```

cap

`cap` — operation `cap`. Alias for `capitalize`. Category: *trivial string ops*.

```
p cap $x
```

capacitance

`capacitance` — operation `capacitance`. Category: *physics formulas*.

```
p capacitance $x
```

capacitance_parallel

`capacitance_parallel` — operation `capacitance parallel`. Category: *misc*.

```
p capacitance_parallel $x
```

capacitance_parallel_sum

`capacitance_parallel_sum` — sum of `capacitance parallel`. Category: *em / optics / relativity*.

```
p capacitance_parallel_sum @xs
```

capacitance_series

`capacitance_series` — operation `capacitance series`. Category: *misc*.

```
p capacitance_series $x
```

capacitor_charge

`capacitor_charge` — operation `capacitor charge`. Category: *em / optics / relativity*.

```
p capacitor_charge $x
```

capacitor_energy

`capacitor_energy` — operation `capacitor energy`. Category: **physics formulas**.

```
p capacitor_energy $x
```

capenergy

`capenergy` — operation `capenergy`. Alias for `capacitor_energy`. Category: **physics formulas**.

```
p capenergy $x
```

capg

`capg` — operation `capg`. Alias for `capture_groups`. Category: **extended stdlib**.

```
p capg $x
```

capitalize

`capitalize` — operation `capitalize`. Category: **trivial string ops**.

```
p capitalize $x
```

capitalize_words

`capitalize_words` — operation `capitalize words`. Category: **misc**.

```
p capitalize_words $x
```

capm

`capm($risk_free, $beta, $market_return)` — computes expected return using the Capital Asset Pricing Model: $rf + \beta(rm - rf)$.

```
p capm(0.02, 1.2, 0.08) # 9.2% expected return
```

capm_expected_return

`capm_expected_return` — operation `capm expected return`. Category: **misc**.

```
p capm_expected_return $x
```

capture_groups

`capture_groups` — operation `capture groups`. Category: **extended stdlib**.

```
p capture_groups $x
```

carmichael_q

`carmichael_q` — operation `carmichael q`. Category: **misc**.

```
p carmichael_q $x
```

carnot_efficiency

carnot_efficiency — operation `carnot efficiency`. Category: **misc**.

```
p carnot_efficiency $x
```

carrying_capacity_from_data

carrying_capacity_from_data — operation `carrying capacity from data`. Category: **biology / ecology**.

```
p carrying_capacity_from_data $x
```

cart_n

cart_n — operation `cart n`. Alias for `cartesian_product_n`. Category: **misc**.

```
p cart_n $x
```

cartan_determinant_a2

cartan_determinant_a2 — operation `cartan determinant a2`. Category: **econometrics**.

```
p cartan_determinant_a2 $x
```

cartan_matrix_b2

cartan_matrix_b2 — operation `cartan matrix b2`. Category: **econometrics**.

```
p cartan_matrix_b2 $x
```

cartesian_power

cartesian_power — operation `cartesian power`. Category: **extended stdlib**.

```
p cartesian_power $x
```

cartesian_product

cartesian_product — operation `cartesian product`. Category: **python/ruby stdlib**.

```
p cartesian_product $x
```

cartesian_product_n

cartesian_product_n — operation `cartesian product n`. Category: **misc**.

```
p cartesian_product_n $x
```

cartesian_to_cylindrical

cartesian_to_cylindrical — convert from `cartesian` to `cylindrical`. Category: **complex / geom / color / trig**.

```
p cartesian_to_cylindrical $value
```

cartesian_to_polar

cartesian_to_polar — convert from `cartesian` to `polar`. Category: **complex / geom / color / trig**.

```
p cartesian_to_polar $value
```

cartesian_to_spherical

`cartesian_to_spherical` — convert from `cartesian` to `spherical`. Category: **complex / geom / color / trig**.

```
p cartesian_to_spherical $value
```

cartpow

`cartpow` — operation `cartpow`. Alias for `cartesian_power`. Category: **extended stdlib**.

```
p cartpow $x
```

cas_admm_lasso_step

`cas_admm_lasso_step` — single-step update of `cas admm lasso`. Category: **climate, fluids, atmospheric**.

```
p cas_admm_lasso_step $x
```

cas_alternating_projection

`cas_alternating_projection` — operation `cas alternating projection`. Category: **climate, fluids, atmospheric**.

```
p cas_alternating_projection $x
```

cas_arnoldi_iteration_step

`cas_arnoldi_iteration_step` — single-step update of `cas arnoldi iteration`. Category: **climate, fluids, atmospheric**.

```
p cas_arnoldi_iteration_step $x
```

cas_basis_pursuit_step

`cas_basis_pursuit_step` — single-step update of `cas basis pursuit`. Category: **climate, fluids, atmospheric**.

```
p cas_basis_pursuit_step $x
```

cas_block_lu_step

`cas_block_lu_step` — single-step update of `cas block lu`. Category: **climate, fluids, atmospheric**.

```
p cas_block_lu_step $x
```

cas_buchberger_step

`cas_buchberger_step` — single-step update of `cas buchberger`. Category: **climate, fluids, atmospheric**.

```
p cas_buchberger_step $x
```

cas_bunch_kaufman_step

`cas_bunch_kaufman_step` — single-step update of `cas bunch kaufman`. Category: **climate, fluids, atmospheric**.

```
p cas_bunch_kaufman_step $x
```

cas_burnside_count_step

`cas_burnside_count_step` — single-step update of `cas burnside count`. Category: **climate, fluids, atmospheric**.


```
p cas_burnside_count_step $x
```

cas_chebyshev_eval

cas_chebyshev_eval — operation `cas chebyshev eval`. Category: *climate, fluids, atmospheric*.

```
p cas_chebyshev_eval $x
```

cas_cholesky_step

cas_cholesky_step — single-step update of `cas cholesky`. Category: *climate, fluids, atmospheric*.

```
p cas_cholesky_step $x
```

cas_classical_gram_schmidt

cas_classical_gram_schmidt — operation `cas classical gram schmidt`. Category: *climate, fluids, atmospheric*.

```
p cas_classical_gram_schmidt $x
```

cas_companion_matrix_root

cas_companion_matrix_root — operation `cas companion matrix root`. Category: *climate, fluids, atmospheric*.

```
p cas_companion_matrix_root $x
```

cas_constrained_ls_step

cas_constrained_ls_step — single-step update of `cas constrained ls`. Category: *climate, fluids, atmospheric*.

```
p cas_constrained_ls_step $x
```

cas_continued_fraction_step

cas_continued_fraction_step — single-step update of `cas continued fraction`. Category: *climate, fluids, atmospheric*.

```
p cas_continued_fraction_step $x
```

cas_cosamp_step

cas_cosamp_step — single-step update of `cas cosamp`. Category: *climate, fluids, atmospheric*.

```
p cas_cosamp_step $x
```

cas_descartes_rule_count

cas_descartes_rule_count — count of `cas descartes rule`. Category: *climate, fluids, atmospheric*.

```
p cas_descartes_rule_count $x
```

cas_drazin_inverse_step

cas_drazin_inverse_step — single-step update of `cas drazin inverse`. Category: *climate, fluids, atmospheric*.

```
p cas_drazin_inverse_step $x
```

cas_dykstra_step

`cas_dykstra_step` — single-step update of `cas dykstra`. Category: *climate, fluids, atmospheric*.

```
p cas_dykstra_step $x
```

cas_eigenvalue_inverse_iteration

`cas_eigenvalue_inverse_iteration` — operation `cas eigenvalue inverse iteration`. Category: *climate, fluids, atmospheric*.

```
p cas_eigenvalue_inverse_iteration $x
```

cas_elastic_net_step

`cas_elastic_net_step` — single-step update of `cas elastic net`. Category: *climate, fluids, atmospheric*.

```
p cas_elastic_net_step $x
```

cas_expand_two_terms

`cas_expand_two_terms` — operation `cas expand two terms`. Category: *climate, fluids, atmospheric*.

```
p cas_expand_two_terms $x
```

cas_extended_euclid_step

`cas_extended_euclid_step` — single-step update of `cas extended euclid`. Category: *climate, fluids, atmospheric*.

```
p cas_extended_euclid_step $x
```

cas_factor_quadratic

`cas_factor_quadratic` — operation `cas factor quadratic`. Category: *climate, fluids, atmospheric*.

```
p cas_factor_quadratic $x
```

cas_gcd_polynomial_step

`cas_gcd_polynomial_step` — single-step update of `cas gcd polynomial`. Category: *climate, fluids, atmospheric*.

```
p cas_gcd_polynomial_step $x
```

cas_gegenbauer_eval

`cas_gegenbauer_eval` — operation `cas gegenbauer eval`. Category: *climate, fluids, atmospheric*.

```
p cas_gegenbauer_eval $x
```

cas_generalized_eigen

`cas_generalized_eigen` — operation `cas generalized eigen`. Category: *climate, fluids, atmospheric*.

```
p cas_generalized_eigen $x
```

cas_givens_rotation_apply

`cas_givens_rotation_apply` — operation `cas givens rotation apply`. Category: *climate, fluids, atmospheric*.

```
p cas_givens_rotation_apply $x
```

cas_groebner_lt_step

`cas_groebner_lt_step` — single-step update of `cas groebner lt`. Category: *climate, fluids, atmospheric*.

```
p cas_groebner_lt_step $x
```

cas_hermite_eval

`cas_hermite_eval` — operation `cas hermite eval`. Category: *climate, fluids, atmospheric*.

```
p cas_hermite_eval $x
```

cas_hermite_normal_step

`cas_hermite_normal_step` — single-step update of `cas hermite normal`. Category: *climate, fluids, atmospheric*.

```
p cas_hermite_normal_step $x
```

cas_householder_reflection

`cas_householder_reflection` — operation `cas householder reflection`. Category: *climate, fluids, atmospheric*.

```
p cas_householder_reflection $x
```

cas_iht_iteration

`cas_iht_iteration` — operation `cas iht iteration`. Category: *climate, fluids, atmospheric*.

```
p cas_iht_iteration $x
```

cas_indicator_simplex_proj

`cas_indicator_simplex_proj` — operation `cas indicator simplex proj`. Category: *climate, fluids, atmospheric*.

```
p cas_indicator_simplex_proj $x
```

cas_jacobi_eigen_step

`cas_jacobi_eigen_step` — single-step update of `cas jacobi eigen`. Category: *climate, fluids, atmospheric*.

```
p cas_jacobi_eigen_step $x
```

cas_jacobi_eval

`cas_jacobi_eval` — operation `cas jacobi eval`. Category: *climate, fluids, atmospheric*.

```
p cas_jacobi_eval $x
```

cas_kronecker_product_step

`cas_kronecker_product_step` — single-step update of `cas kronecker product`. Category: *climate, fluids, atmospheric*.

```
p cas_kronecker_product_step $x
```

cas_lagrange_interpolate

`cas_lagrange_interpolate` — operation `cas lagrange interpolate`. Category: *climate, fluids, atmospheric*.

```
p cas_lagrange_interpolate $x
```

cas_laguerre_eval

`cas_laguerre_eval` — operation `cas laguerre eval`. Category: *climate, fluids, atmospheric*.

```
p cas_laguerre_eval $x
```

cas_lanczos_iteration_step

`cas_lanczos_iteration_step` — single-step update of `cas lanczos iteration`. Category: *climate, fluids, atmospheric*.

```
p cas_lanczos_iteration_step $x
```

cas_lasso_soft_threshold

`cas_lasso_soft_threshold` — operation `cas lasso soft threshold`. Category: *climate, fluids, atmospheric*.

```
p cas_lasso_soft_threshold $x
```

cas_ldlt_step

`cas_ldlt_step` — single-step update of `cas ldlt`. Category: *climate, fluids, atmospheric*.

```
p cas_ldlt_step $x
```

cas_least_squares_solve

`cas_least_squares_solve` — operation `cas least squares solve`. Category: *climate, fluids, atmospheric*.

```
p cas_least_squares_solve $x
```

cas_legendre_eval

`cas_legendre_eval` — operation `cas legendre eval`. Category: *climate, fluids, atmospheric*.

```
p cas_legendre_eval $x
```

cas_lyapunov_continuous_step

`cas_lyapunov_continuous_step` — single-step update of `cas lyapunov continuous`. Category: *climate, fluids, atmospheric*.

```
p cas_lyapunov_continuous_step $x
```

cas_lyapunov_discrete_step

`cas_lyapunov_discrete_step` — single-step update of `cas lyapunov discrete`. Category: *climate, fluids, atmospheric*.

```
p cas_lyapunov_discrete_step $x
```

cas_macaulay_matrix_step

`cas_macaulay_matrix_step` — single-step update of `cas macaulay matrix`. Category: *climate, fluids, atmospheric*.

```
p cas_macaulay_matrix_step $x
```

cas_matrix_exp_pade

`cas_matrix_exp_pade` — operation `cas matrix exp pade`. Category: *climate, fluids, atmospheric*.

```
p cas_matrix_exp_pade $x
```

cas_matrix_function_step

`cas_matrix_function_step` — single-step update of `cas matrix function`. Category: *climate, fluids, atmospheric*.

```
p cas_matrix_function_step $x
```

cas_matrix_log_step

`cas_matrix_log_step` — single-step update of `cas matrix log`. Category: *climate, fluids, atmospheric*.

```
p cas_matrix_log_step $x
```

cas_matrix_pencil_step

`cas_matrix_pencil_step` — single-step update of `cas matrix pencil`. Category: *climate, fluids, atmospheric*.

```
p cas_matrix_pencil_step $x
```

cas_matrix_sqrt_step

`cas_matrix_sqrt_step` — single-step update of `cas matrix sqrt`. Category: *climate, fluids, atmospheric*.

```
p cas_matrix_sqrt_step $x
```

cas_minimal_polynomial

`cas_minimal_polynomial` — operation `cas minimal polynomial`. Category: *climate, fluids, atmospheric*.

```
p cas_minimal_polynomial $x
```

cas_modified_cholesky

`cas_modified_cholesky` — operation `cas modified cholesky`. Category: *climate, fluids, atmospheric*.

```
p cas_modified_cholesky $x
```

cas_modified_gram_schmidt

`cas_modified_gram_schmidt` — operation `cas modified gram schmidt`. Category: *climate, fluids, atmospheric*.

```
p cas_modified_gram_schmidt $x
```

cas_modular_inverse

`cas_modular_inverse` — inverse of `cas modular`. Category: *climate, fluids, atmospheric*.

```
p cas_modular_inverse $x
```

cas_moore_penrose_step

`cas_moore_penrose_step` — single-step update of `cas moore penrose`. Category: *climate, fluids, atmospheric*.

```
p cas_moore_penrose_step $x
```

cas_omp_step

`cas_omp_step` — single-step update of `cas omp`. Category: *climate, fluids, atmospheric*.

```
p cas_omp_step $x
```

cas_partial_fraction_simple

`cas_partial_fraction_simple` — operation `cas partial fraction simple`. Category: *climate, fluids, atmospheric*.

```
p cas_partial_fraction_simple $x
```

cas_pivoted_lu_step

`cas_pivoted_lu_step` — single-step update of `cas pivoted lu`. Category: *climate, fluids, atmospheric*.

```
p cas_pivoted_lu_step $x
```

cas_polar_decomposition

`cas_polar_decomposition` — operation `cas polar decomposition`. Category: *climate, fluids, atmospheric*.

```
p cas_polar_decomposition $x
```

cas_polya_enumeration_step

`cas_polya_enumeration_step` — single-step update of `cas polya enumeration`. Category: *climate, fluids, atmospheric*.

```
p cas_polya_enumeration_step $x
```

cas_polynomial_div_step

`cas_polynomial_div_step` — single-step update of `cas polynomial div`. Category: *climate, fluids, atmospheric*.

```
p cas_polynomial_div_step $x
```

cas_polynomial_gcd_step

`cas_polynomial_gcd_step` — single-step update of `cas polynomial gcd`. Category: *climate, fluids, atmospheric*.

```
p cas_polynomial_gcd_step $x
```

cas_polynomial_roots_kahan

`cas_polynomial_roots_kahan` — operation `cas polynomial roots kahan`. Category: *climate, fluids, atmospheric*.

```
p cas_polynomial_roots_kahan $x
```

cas_proj_box

`cas_proj_box` — operation `cas proj box`. Category: *climate, fluids, atmospheric*.

```
p cas_proj_box $x
```

cas_proj_exp_cone

cas_proj_exp_cone — operation `cas proj exp cone`. Category: *climate, fluids, atmospheric*.

```
p cas_proj_exp_cone $x
```

cas_proj_l1_ball

cas_proj_l1_ball — operation `cas proj l1 ball`. Category: *climate, fluids, atmospheric*.

```
p cas_proj_l1_ball $x
```

cas_proj_l2_ball

cas_proj_l2_ball — operation `cas proj l2 ball`. Category: *climate, fluids, atmospheric*.

```
p cas_proj_l2_ball $x
```

cas_proj_psd_cone

cas_proj_psd_cone — operation `cas proj psd cone`. Category: *climate, fluids, atmospheric*.

```
p cas_proj_psd_cone $x
```

cas_proj_soc_step

cas_proj_soc_step — single-step update of `cas proj soc`. Category: *climate, fluids, atmospheric*.

```
p cas_proj_soc_step $x
```

cas_proximal_l1_step

cas_proximal_l1_step — single-step update of `cas proximal l1`. Category: *climate, fluids, atmospheric*.

```
p cas_proximal_l1_step $x
```

cas_proximal_l2_step

cas_proximal_l2_step — single-step update of `cas proximal l2`. Category: *climate, fluids, atmospheric*.

```
p cas_proximal_l2_step $x
```

cas_proximal_l_inf_step

cas_proximal_l_inf_step — single-step update of `cas proximal l inf`. Category: *climate, fluids, atmospheric*.

```
p cas_proximal_l_inf_step $x
```

cas_pseudoinverse_step

cas_pseudoinverse_step — single-step update of `cas pseudoinverse`. Category: *climate, fluids, atmospheric*.

```
p cas_pseudoinverse_step $x
```

cas_qr_iteration_step

`cas_qr_iteration_step` — single-step update of `cas qr iteration`. Category: *climate, fluids, atmospheric*.

```
p cas_qr_iteration_step $x
```

cas_quasi_triangular

`cas_quasi_triangular` — operation `cas quasi triangular`. Category: *climate, fluids, atmospheric*.

```
p cas_quasi_triangular $x
```

cas_radical_simplify

`cas_radical_simplify` — operation `cas radical simplify`. Category: *climate, fluids, atmospheric*.

```
p cas_radical_simplify $x
```

cas_rank_revealing_qr

`cas_rank_revealing_qr` — operation `cas rank revealing qr`. Category: *climate, fluids, atmospheric*.

```
p cas_rank_revealing_qr $x
```

cas_regularized_lsq_tikhonov

`cas_regularized_lsq_tikhonov` — operation `cas regularized lsq tikhonov`. Category: *climate, fluids, atmospheric*.

```
p cas_regularized_lsq_tikhonov $x
```

cas_resultant_two

`cas_resultant_two` — operation `cas resultant two`. Category: *climate, fluids, atmospheric*.

```
p cas_resultant_two $x
```

cas_resultant_x_y

`cas_resultant_x_y` — operation `cas resultant x y`. Category: *climate, fluids, atmospheric*.

```
p cas_resultant_x_y $x
```

cas_riccati_continuous_step

`cas_riccati_continuous_step` — single-step update of `cas riccati continuous`. Category: *climate, fluids, atmospheric*.

```
p cas_riccati_continuous_step $x
```

cas_riccati_discrete_step

`cas_riccati_discrete_step` — single-step update of `cas riccati discrete`. Category: *climate, fluids, atmospheric*.

```
p cas_riccati_discrete_step $x
```


cas_root_isolate_step

`cas_root_isolate_step` — single-step update of `cas root isolate`. Category: *climate, fluids, atmospheric*.

```
p cas_root_isolate_step $x
```

cas_schur_decomposition_step

`cas_schur_decomposition_step` — single-step update of `cas schur decomposition`. Category: *climate, fluids, atmospheric*.

```
p cas_schur_decomposition_step $x
```

cas_simplify_term

`cas_simplify_term` — operation `cas simplify term`. Category: *climate, fluids, atmospheric*.

```
p cas_simplify_term $x
```

cas_singular_value_step

`cas_singular_value_step` — single-step update of `cas singular value`. Category: *climate, fluids, atmospheric*.

```
p cas_singular_value_step $x
```

cas_smith_normal_step

`cas_smith_normal_step` — single-step update of `cas smith normal`. Category: *climate, fluids, atmospheric*.

```
p cas_smith_normal_step $x
```

cas_solve_cubic

`cas_solve_cubic` — operation `cas solve cubic`. Category: *climate, fluids, atmospheric*.

```
p cas_solve_cubic $x
```

cas_solve_linear

`cas_solve_linear` — operation `cas solve linear`. Category: *climate, fluids, atmospheric*.

```
p cas_solve_linear $x
```

cas_solve_polynomial_n

`cas_solve_polynomial_n` — operation `cas solve polynomial n`. Category: *climate, fluids, atmospheric*.

```
p cas_solve_polynomial_n $x
```

cas_solve_quadratic

`cas_solve_quadratic` — operation `cas solve quadratic`. Category: *climate, fluids, atmospheric*.

```
p cas_solve_quadratic $x
```

cas_solve_quartic

`cas_solve_quartic` — operation `cas solve quartic`. Category: *climate, fluids, atmospheric*.

```
p cas_solve_quartic $x
```

cas_sturm_sequence_step

`cas_sturm_sequence_step` — single-step update of `cas sturm sequence`. Category: *climate, fluids, atmospheric*.

```
p cas_sturm_sequence_step $x
```

cas_subresultant_two

`cas_subresultant_two` — operation `cas subresultant two`. Category: *climate, fluids, atmospheric*.

```
p cas_subresultant_two $x
```

cas_sylvester_equation_step

`cas_sylvester_equation_step` — single-step update of `cas sylvester equation`. Category: *climate, fluids, atmospheric*.

```
p cas_sylvester_equation_step $x
```

cas_taylor_coefficient

`cas_taylor_coefficient` — operation `cas taylor coefficient`. Category: *climate, fluids, atmospheric*.

```
p cas_taylor_coefficient $x
```

cas_total_least_squares

`cas_total_least_squares` — operation `cas total least squares`. Category: *climate, fluids, atmospheric*.

```
p cas_total_least_squares $x
```

cas_truncated_lsq

`cas_truncated_lsq` — operation `cas truncated lsq`. Category: *climate, fluids, atmospheric*.

```
p cas_truncated_lsq $x
```

cas_truncated_svd_value

`cas_truncated_svd_value` — value of `cas truncated svd`. Category: *climate, fluids, atmospheric*.

```
p cas_truncated_svd_value $x
```

cas_vec_operator_step

`cas_vec_operator_step` — single-step update of `cas vec operator`. Category: *climate, fluids, atmospheric*.

```
p cas_vec_operator_step $x
```

cas_woodbury_identity

`cas_woodbury_identity` — operation `cas woodbury identity`. Category: *climate, fluids, atmospheric*.

```
p cas_woodbury_identity $x
```

case_alternating

`case_alternating(s)` → `string` — alternating case starting with lowercase.

```
p case_alternating("hello") # "hElLo"
```

case_constant

`case_constant(s)` → `string` — SCREAMING_SNAKE_CASE.

```
p case_constant("helloWorld") # "HELLO_WORLD"
```

case_dot

`case_dot(s)` → `string` — dot.case (lowercase, dot-separated).

case_pascal

`case_pascal(s)` → `string` — PascalCase: each word capitalised, no separators.

```
p case_pascal("hello_world") # "HelloWorld"
```

case_path

`case_path(s)` → `string` — lower/case/path/style.

case_sentence

`case_sentence(s)` → `string` — Sentence case (first word capitalised, rest lowercase).

case_swap

`case_swap(s)` → `string` — invert case of each ASCII letter.

```
p case_swap("Hello World") # "hELLO wORLD"
```

case_title_proper

`case_title_proper(s)` → `string` — Title Case with small-word exceptions (a/an/the/and/or/of/in/...).

case_train

`case_train(s)` → `string` — Train-Case (capitalised, hyphen-separated).

casimir_eigenvalue_su2

`casimir_eigenvalue_su2` — operation `casimir eigenvalue su2`. Category: **econometrics**.

```
p casimir_eigenvalue_su2 $x
```

catalan

`catalan_constant` (alias `catalan`) — Catalan's constant $G \approx 0.9159$. Appears in combinatorics and series.

```
p catalan # 0.9159655941772190
```

catalan_number

`catalan_number` — operation `catalan number`. Category: **math / number theory extras**.

```
p catalan_number $x
```

catn

`catn` — operation `catn`. Alias for `catalan`. Category: **extended stdlib**.

```
p catn $x
```

cauchy_pdf

`cauchy_pdf` (alias `cauchypdf`) evaluates the Cauchy distribution PDF at `x` with location `x0` and scale `gamma`.

```
p cauchypdf(0, 0, 1) # peak of standard Cauchy
```

caverphone

`caverphone(name)` → `string` — Caverphone 1.0 (6-char). Designed for matching New Zealand electoral roll names.

```
p caverphone("John")
```

caverphone2

`caverphone2(name)` → `string` — Caverphone 2.0 (10-char, padded with 1).

caverphone_2

`caverphone_2` — operation `caverphone 2`. Category: **computational linguistics**.

```
p caverphone_2 $x
```

cbind

`cbind` — bind matrices by columns (horizontal join). Like R's `cbind()`.

```
val $m = cbind([[1],[2]], [[3],[4]]) # [[1,3],[2,4]]
```

cblend

`cblend` — operation `cblend`. Alias for `color_blend`. Category: **color operations**.

```
p cblend $x
```

cbor_encode_str

`cbor_encode_str` — operation `cbor encode str`. Category: **archive/encoding format primitives**.

```
p cbor_encode_str $x
```

cbor_encode_uint

`cbor_encode_uint` — operation `cbor encode uint`. Category: **archive/encoding format primitives**.

```
p cbor_encode_uint $x
```

cbrt

`cbrt` — operation `cbrt`. Category: *trivial numeric / predicate builtins*.

```
p cbrt $x
```

cc_count

`cc_count` — count of `cc`. Category: *graph algorithms*.

```
p cc_count $x
```

cc_labels

`cc_labels` — operation `cc labels`. Category: *graph algorithms*.

```
p cc_labels $x
```

ccgraph

`ccgraph` — operation `ccgraph`. Alias for `connected_components_graph`. Category: *extended stdlib*.

```
p ccgraph $x
```

cci

`cci(highs, lows, closes, period=20) → array` — Commodity Channel Index: $(\text{typical_price} - \text{sma}) / (0.015 \cdot \text{mean_dev})$.

```
p cci(\@hi, \@lo, \@cl, 20)
```

ccomp

`continuous_compound($principal, $rate, $time)` (alias `ccomp`) — computes future value with continuous compounding: $P \times e^{(rt)}$.

```
p ccomp(1000, 0.05, 10) # -1648.72
```

ccompl

`ccompl` — operation `ccompl`. Alias for `color_complement`. Category: *color operations*.

```
p ccompl $x
```

ccount

`ccount` — operation `ccount`. Alias for `count_chars`. Category: *extended stdlib*.

```
p ccount $x
```

ccx_gate

`ccx_gate` — operation `ccx gate`. Alias for `toffoli_gate`. Category: *more extensions*.

```
p ccx_gate $x
```

cdarken

`cdarken` — operation `cdarken`. Alias for `color_darken`. Category: *color operations*.

```
p cdarken $x
```

cdist

`cdist` — operation `cdist`. Alias for `color_distance`. Category: *color operations*.

```
p cdist $x
```

cds_upfront

`cds_upfront` — operation `cds upfront`. Category: *financial pricing models*.

```
p cds_upfront $x
```

cech_zero_cohomology

`cech_zero_cohomology` — operation `cech zero cohomology`. Category: *econometrics*.

```
p cech_zero_cohomology $x
```

ceil

`ceil` — operation `ceil`. Category: *trivial numeric / predicate builtins*.

```
p ceil $x
```

ceil_div

`ceil_div` — operation `ceil div`. Category: *trig / math*.

```
p ceil_div $x
```

ceil_each

`ceil_each` — operation `ceil each`. Category: *trivial numeric helpers*.

```
p ceil_each $x
```

ceiling

`ceiling` — operation `ceiling`. Alias for `ceil`. Category: *trivial numeric / predicate builtins*.

```
p ceiling $x
```

center

`center` — operation `center`. Category: *trivial string ops*.

```
p center $x
```

center_of_mass_2d

`center_of_mass_2d(masses, positions)` → `[x, y]` — mass-weighted centroid of point particles in 2D.

center_of_mass_3d

`center_of_mass_3d(masses, positions)` → [x, y, z] — same in 3D.

center_text

`center_text` — operation `center text`. Category: *extended stdlib*.

```
p center_text $x
```

centered_hexagonal_number

`centered_hexagonal_number` — operation `centered hexagonal number`. Category: *misc*.

```
p centered_hexagonal_number $x
```

centered_pentagonal

`centered_pentagonal` — operation `centered pentagonal`. Category: *misc*.

```
p centered_pentagonal $x
```

centered_polygonal

`centered_polygonal(k, n)` → int — centered k-gonal number $1 + k \cdot n \cdot (n-1)/2$.

centered_square

`centered_square` — operation `centered square`. Category: *misc*.

```
p centered_square $x
```

centered_triangular

`centered_triangular` — operation `centered triangular`. Category: *misc*.

```
p centered_triangular $x
```

central_polygonal

`central_polygonal` — operation `central polygonal`. Category: *misc*.

```
p central_polygonal $x
```

centrality_betweenness

`centrality_betweenness` — operation `centrality betweenness`. Category: *networkx graph algorithms*.

```
p centrality_betweenness $x
```

centrality_closeness

`centrality_closeness` — operation `centrality closeness`. Category: *networkx graph algorithms*.

```
p centrality_closeness $x
```

centrality_degree

`centrality_degree` — operation `centrality_degree`. Category: **networkx graph algorithms**.

```
p centrality_degree $x
```

centrality_eigenvector

`centrality_eigenvector` — operation `centrality_eigenvector`. Category: **networkx graph algorithms**.

```
p centrality_eigenvector $x
```

centrality_katz

`centrality_katz` — operation `centrality_katz`. Category: **networkx graph algorithms**.

```
p centrality_katz $x
```

centrip

`centripetal_force($mass, $velocity, $radius)` (alias `centrip`) — $F = mv^2/r$ for circular motion.

```
p centrip(1000, 20, 50) # 8000 N (car turning)
```

centripetal_accel

`centripetal_accel` — operation `centripetal_accel`. Category: **misc**.

```
p centripetal_accel $x
```

centroid

`centroid(@points)` — computes the centroid (geometric center) of 2D points. Points as flat list [x1,y1,x2,y2,...]. Returns [\$cx, \$cy].

```
val @triangle = (0,0, 3,0, 0,3)
var $c = centroid(@triangle)
p @$c # (1, 1)
```

centroid_nd

`centroid_nd` — operation `centroid_nd`. Category: **geometry / topology**.

```
p centroid_nd $x
```

cents_between

`cents_between` — operation `cents_between`. Category: **astronomy / music / color / units**.

```
p cents_between $x
```

cents_between_freqs

`cents_between_freqs` — operation `cents_between_freqs`. Category: **music theory**.

```
p cents_between_freqs $x
```


ces_production

`ces_production` — operation `ces production`. Category: *economics + game theory*.

```
p ces_production $x
```

cfl_number

`cfl_number` — operation `cfl number`. Category: *ode advanced*.

```
p cfl_number $x
```

cfr_case_fatality

`cfr_case_fatality` — operation `cfr case fatality`. Category: *epidemiology / public health*.

```
p cfr_case_fatality $x
```

cg_beta_fr

`cg_beta_fr` — operation `cg beta fr`. Category: *more extensions (2)*.

```
p cg_beta_fr $x
```

cg_beta_pr

`cg_beta_pr` — operation `cg beta pr`. Category: *more extensions (2)*.

```
p cg_beta_pr $x
```

cg_simple

`cg_simple` — operation `cg simple`. Category: *quantum mechanics deep*.

```
p cg_simple $x
```

cgemm

`cgemm` — operation `cgemm`. Category: *blas / lapack*.

```
p cgemm $x
```

cgray

`cgray` — operation `cgray`. Alias for `color_grayscale`. Category: *color operations*.

```
p cgray $x
```

chacha20_ground

`chacha20_ground` — operation `chacha20 ground`. Category: *more extensions*.

```
p chacha20_ground $x
```

chain_from

`chain_from` — operation `chain from`. Category: *python/ruby stdlib*.

```
p chain_from $x
```

chain_ladder_step

`chain_ladder_step` — single-step update of `chain_ladder`. Category: *actuarial science*.

```
p chain_ladder_step $x
```

chain_rule_entropy

`chain_rule_entropy` — operation `chain rule entropy`. Category: *electrochemistry, batteries, fuel cells*.

```
p chain_rule_entropy $x
```

chandrasekhar_mass

`chandrasekhar_mass` — operation `chandrasekhar mass`. Category: *misc*.

```
p chandrasekhar_mass $x
```

channel_bounded

`channel_bounded(capacity)` → `channel` — bounded channel; senders block when full.

channel_close

`channel_close(ch)` → `0|1` — close channel; pending receivers get the channel-closed sentinel.

channel_drain

`channel_drain(ch)` → `array` — receive all pending values.

channel_is_closed

`channel_is_closed(ch)` → `0|1` — 1 if channel has been closed.

channel_recv_timeout

`channel_recv_timeout(ch, ms)` → `value` — try to receive with timeout.

channel_send_timeout

`channel_send_timeout(ch, value, ms)` → `0|1` — try to send with timeout.

channel_sync

`channel_sync()` → `channel` — rendezvous channel (capacity 0); send and recv complete in lockstep.

channel_try_recv

`channel_try_recv` — receive of `channel try`. Category: *extras*.

```
p channel_try_recv $x
```

channel_try_send

`channel_try_send` — send of `channel try`. Category: *extras*.

```
p channel_try_send $x
```

channel_unbounded

`channel_unbounded()` → `channel` — create an unbounded multi-producer multi-consumer channel.

chapman_estimator

`chapman_estimator` — operation `chapman_estimator`. Category: *biology / ecology*.

```
p chapman_estimator $x
```

char_at

`char_at` — single-character helper `at`. Category: *extended stdlib*.

```
p char_at $x
```

char_count

`char_count` — single-character helper `count`. Category: *trivial string ops*.

```
p char_count $x
```

char_devices

`char_devices` — single-character helper `devices`. Category: *filesystem extensions*.

```
p char_devices $x
```

char_frequencies

`char_frequencies` — single-character helper `frequencies`. Category: *string processing extras*.

```
p char_frequencies $x
```

char_length

`char_length` — single-character helper `length`. Category: *string helpers*.

```
p char_length $x
```

character_su2

`character_su2` — operation `character su2`. Category: *econometrics*.

```
p character_su2 $x
```

character_sun

`character_sun` — operation `character sun`. Category: *econometrics*.

```
p character_sun $x
```

characteristic_function

`characteristic_function` — operation `characteristic function`. Category: *climate, fluids, atmospheric*.

```
p characteristic_function $x
```

charcodes_to_string

`charcodes_to_string` — convert from `charcodes` to `string`. Category: **misc**.

```
p charcodes_to_string $value
```

charfreq

`charfreq` — operation `charfreq`. Alias for `char_frequencies`. Category: **string processing extras**.

```
p charfreq $x
```

charge_capacity_battery

`charge_capacity_battery` — operation `charge capacity battery`. Category: **electrochemistry, batteries, fuel cells**.

```
p charge_capacity_battery $x
```

charge_transfer_resistance

`charge_transfer_resistance` — operation `charge transfer resistance`. Category: **electrochemistry, batteries, fuel cells**.

```
p charge_transfer_resistance $x
```

chars_to_string

`chars_to_string` — convert from `chars` to `string`. Category: **string helpers**.

```
p chars_to_string $value
```

charset_convert

`iconv(STRING, FROM, TO)` → re-encode `STRING` between character sets. Supports `utf-8`, `latin-1` (aliases `iso-8859-1`, `iso8859-1`, `latin1`), and `ascii`. Lossy for unsupported chars (`?` substitute). Unknown FROM / TO returns input unchanged. For wider charset support, use a dedicated codec — `iconv` is the shell-grade convenience wrapper.

```
p iconv("café", "utf-8", "ascii")      # caf?
p iconv($latin_bytes, "latin-1", "utf-8")
```

chc_bound_makespan

`chc_bound_makespan` — operation `chc bound makespan`. Category: **combinatorial optimization, scheduling**.

```
p chc_bound_makespan $x
```

cheapest_insertion_step

`cheapest_insertion_step` — single-step update of `cheapest insertion`. Category: **combinatorial optimization, scheduling**.

```
p cheapest_insertion_step $x
```

cheby1_lp

`cheby1_lp` — operation `cheby1 lp`. Category: **economics + game theory**.

```
p cheby1_lp $x
```

cheby2_lp

`cheby2_lp` — operation `cheby2_lp`. Category: *economics + game theory*.

```
p cheby2_lp $x
```

chebyshev_distance

`chebyshev_distance` — distance / dissimilarity metric of `chebyshev`. Category: *math functions*.

```
p chebyshev_distance $x
```

chebyshev_norm

`chebyshev_norm(v)` → `number` — L^∞ norm $\max |v_i|$.

chebyt

`chebyt` — operation `chebyt`. Category: *numpy + scipy.special*.

```
p chebyt $x
```

chebyu

`chebyu` — operation `chebyu`. Category: *numpy + scipy.special*.

```
p chebyu $x
```

chem_arrhenius_k

`chem_arrhenius_k(A, Ea_J, T_K)` → `number` — $k = A \cdot e^{(-Ea / (R \cdot T))}$.

chem_avogadro

`chem_avogadro()` → `number` — Avogadro constant $6.022 \cdot 10^{23} \text{ mol}^{-1}$.

chem_balance_check

`chem_balance_check(lhs, rhs)` → `0|1` — 1 if both sides have identical element counts.

chem_boiling_point_elevation

`chem_boiling_point_elevation(Kb, molality, i=1)` → `number` — $\Delta T_b = K_b \cdot m \cdot i$.

chem_buffer_capacity

`chem_buffer_capacity([H+], Ca, Ka)` → `number` — Van Slyke buffer capacity β .

chem_celsius_to_fahrenheit

`chem_celsius_to_fahrenheit(C)` → `number` — $C \cdot 9/5 + 32$.

chem_celsius_to_kelvin

`chem_celsius_to_kelvin(C)` → `number` — $C + 273.15$.

chem_concentration_to_molarity

chem_concentration_to_molarity(mass_g, molar_mass, volume_L) → number — molarity = mass / (MM · V).

chem_dilution

chem_dilution(C1, V1, C2) → number — V2 such that C1V1 = C2V2.

```
p chem_dilution(1.0, 100, 0.1) # 1000
```

chem_fahrenheit_to_celsius

chem_fahrenheit_to_celsius(F) → number — $(F - 32) \cdot 5/9$.

chem_fahrenheit_to_kelvin

chem_fahrenheit_to_kelvin(F) → number — chain via Celsius.

chem_formula_parse

chem_formula_parse(formula) → array — parse chemical formula into [[element, count], ...].

```
p chem_formula_parse("C6H12O6")
```

chem_freezing_point_depression

chem_freezing_point_depression(Kf, molality, i=1) → number — $\Delta T_f = K_f \cdot m \cdot i$ (van't Hoff factor).

chem_h_from_ph

chem_h_from_ph(pH) → number — $[H^+] = 10^{-pH}$.

chem_henderson_hasselbalch

chem_henderson_hasselbalch(pKa, [A-], [HA]) → number — $pH = pKa + \log_{10}([A-]/[HA])$.

```
p chem_henderson_hasselbalch(4.76, 10, 1) # 5.76
```

chem_ideal_gas_volume

chem_ideal_gas_volume(moles, T_K=273.15, P_atm=1) → number — volume in liters from $PV = nRT$.

```
p chem_ideal_gas_volume(1) # 22.41 (STP)
```

chem_isoelectric_estimate

chem_isoelectric_estimate(pk1, pk2) → number — $(pk1 + pk2) / 2$; classic pI estimate for simple amino acids.

chem_kelvin_to_celsius

chem_kelvin_to_celsius(K) → number — $K - 273.15$.

chem_kelvin_to_fahrenheit

chem_kelvin_to_fahrenheit(K) → number — chain via Celsius.

chem_kelvin_to_rankine

`chem_kelvin_to_rankine(K)` → `number` — $K \cdot 9/5$.

chem_molality

`chem_molality(moles_solute, kg_solvent)` → `number` — moles per kg of solvent.

chem_molar_mass

`chem_molar_mass(formula)` → `number` — molar mass in g/mol using a built-in atomic weights table.

```
p chem_molar_mass("H2O") # 18.015
p chem_molar_mass("C6H12O6") # 180.156
```

chem_molarity_to_normality

`chem_molarity_to_normality(M, equiv_per_mole=1)` → `number` — $N = M \cdot \text{equiv_per_mole}$.

chem_partial_pressure

`chem_partial_pressure(total, mole_fraction)` → `number` — $\text{total} \cdot \text{mole_fraction}$ (Dalton's law).

chem_ph_from_h

`chem_ph_from_h(h_concentration)` → `number` — $\text{pH} = -\log_{10}([H+])$.

```
p chem_ph_from_h(1e-7) # 7
```

chem_pka_lookup

`chem_pka_lookup(name)` → `number` — pKa for common acid/base names: HCl, H2SO4, HNO3, CH3COOH, NH4+, H2CO3, etc.

```
p chem_pka_lookup("CH3COOH") # 4.76
```

chem_rankine_to_kelvin

`chem_rankine_to_kelvin(R)` → `number` — $R \cdot 5/9$.

chern_first_2d

`chern_first_2d` — operation `chern first 2d`. Category: *econometrics*.

```
p chern_first_2d $x
```

chern_pontryagin_4d_step

`chern_pontryagin_4d_step` — single-step update of `chern pontryagin 4d`. Category: *electrochemistry, batteries, fuel cells*.

```
p chern_pontryagin_4d_step $x
```

chern_simons_action

chern_simons_action — operation `chern simons action`. Category: **econometrics**.

```
p chern_simons_action $x
```

chezy_velocity

chezy_velocity — operation `chezy velocity`. Category: **misc**.

```
p chezy_velocity $x
```

chfr

chfr — operation `chfr`. Alias for `chain_from`. Category: **python/ruby stdlib**.

```
p chfr $x
```

chi2

`chi_square_stat(\@observed, \@expected)` (alias `chi2`) — computes the chi-squared test statistic comparing observed vs expected frequencies. Useful for goodness-of-fit tests. Returns the chi-squared value.

```
val @obs = (10, 20, 30)
var @exp = (15, 20, 25)
p chi2(\@obs, \@exp) # chi-squared statistic
```

chi2_pdf

chi2_pdf (alias `chi2pdf`) evaluates the chi-squared distribution PDF at `x` with `k` degrees of freedom.

```
p chi2pdf(3.84, 1) # p-value boundary for df=1
```

chi_square_goodness_fit

`chi_square_goodness_fit(observed, expected) → {statistic, df, p_value}` — Pearson χ^2 goodness-of-fit. `df = len - 1`.

```
p chi_square_goodness_fit([10,22,20], [20,20,20])
```

chi_square_independence

`chi_square_independence(contingency) → {statistic, df, p_value}` — χ^2 test of independence on an `r×c` contingency table.

```
p chi_square_independence([[10,20],[30,40]])
```

chicken_game_payoff

chicken_game_payoff — operation `chicken game payoff`. Category: **climate, fluids, atmospheric**.

```
p chicken_game_payoff $x
```

chinese_lunation_winter

chinese_lunation_winter — operation `chinese lunation winter`. Category: **calendrical algorithms**.

```
p chinese_lunation_winter $x
```


chinese_new_year

`chinese_new_year` — operation `chinese new year`. Category: **b82-misc-utility**.

```
p chinese_new_year $x
```

chinese_remainder

`chinese_remainder` (alias `crt`) solves a system of simultaneous congruences via the Chinese Remainder Theorem.

```
p crt([2,3,2], [3,5,7]) # 23 (x≡2 mod 3, x≡3 mod 5, x≡2 mod 7)
```

chinese_year_zodiac

`chinese_year_zodiac` — operation `chinese year zodiac`. Category: **calendrical algorithms**.

```
p chinese_year_zodiac $x
```

chinese_zodiac

`chinese_zodiac` — operation `chinese zodiac`. Category: **misc**.

```
p chinese_zodiac $x
```

chirp

`chirp` generates a linear chirp signal sweeping from `f0` to `f1` Hz. Args: `n_samples`, `f0`, `f1`, `sample_rate`.

```
val @sig = @{chirp(1000, 100, 1000, 8000)}
```

chirp_linear

`chirp_linear` — operation `chirp linear`. Category: **signal processing**.

```
p chirp_linear $x
```

chirp_mass

`chirp_mass` — operation `chirp mass`. Category: **cosmology / gr / flrw**.

```
p chirp_mass $x
```

chirp_mass_inspiral_step

`chirp_mass_inspiral_step` — single-step update of `chirp mass inspiral`. Category: **electrochemistry, batteries, fuel cells**.

```
p chirp_mass_inspiral_step $x
```

chisquare_metric

`chisquare_metric` — metric of `chisquare`. Category: **electrochemistry, batteries, fuel cells**.

```
p chisquare_metric $x
```

chkstr

`chkstr` — operation `chkstr`. Alias for `chunk_string`. Category: **string processing extras**.

```
p chkstr $x
```

chkw

`chkw` — operation `chkw`. Alias for `chunk_while`. Category: **ruby enumerable extras**.

```
p chkw $x
```

chlorine_radical_decay

`chlorine_radical_decay` — operation `chlorine radical decay`. Category: **climate, fluids, atmospheric**.

```
p chlorine_radical_decay $x
```

cholesky

`matrix_cholesky` (aliases `mchol`, `cholesky`) computes the Cholesky decomposition of a symmetric positive-definite matrix. Returns lower-triangular L where $M = L \cdot L^T$.

```
val $L = cholesky([[4,2],[2,3]])
```

chomp_str

`chomp_str` — operation `chomp str`. Category: **string helpers**.

```
p chomp_str $x
```

choose

`choose` — operation `choose`. Category: **excel/sheets + bond/loan financial**.

```
p choose $x
```

chop_str

`chop_str` — operation `chop str`. Category: **string helpers**.

```
p chop_str $x
```

chord_augmented

`chord_augmented(root)` → `array` — augmented triad (root, +4, +8).

chord_diminished

`chord_diminished(root)` → `array` — diminished triad (root, +3, +6).

chord_diminished7

`chord_diminished7(root)` → `array` — diminished 7th (root, +3, +6, +9).

chord_dominant7

`chord_dominant7(root)` → `array` — dominant 7th (root, +4, +7, +10).

```
p chord_dominant7(60)
```

chord_major

`chord_major(root)` → `array` — major triad MIDI notes (root, +4, +7). Accepts MIDI int or note-name string.

```
p chord_major(60) # [60, 64, 67] (C E G)
p chord_major("A4") # [69, 73, 76]
```

chord_major7

`chord_major7(root)` → `array` — major 7th (root, +4, +7, +11).

chord_minor

`chord_minor(root)` → `array` — minor triad (root, +3, +7).

```
p chord_minor(60) # [60, 63, 67]
```

chord_minor7

`chord_minor7(root)` → `array` — minor 7th (root, +3, +7, +10).

chord_notes

`chord_notes` — music-theory chord constructor `notes`. Category: *extras*.

```
p chord_notes $x
```

chord_quality_classify

`chord_quality_classify` — music-theory chord constructor `quality classify`. Category: *music theory*.

```
p chord_quality_classify $x
```

chord_root_inversion

`chord_root_inversion` — music-theory chord constructor `root inversion`. Category: *music theory*.

```
p chord_root_inversion $x
```

chord_to_freqs

`chord_to_freqs` — convert from `chord` to `freqs`. Category: *misc*.

```
p chord_to_freqs $value
```

chord_voicing_close

`chord_voicing_close` — music-theory chord constructor `voicing close`. Category: *music theory*.

```
p chord_voicing_close $x
```

chorus_simple

`chorus_simple(signal, depth_ms=8, rate_hz=0.5, sr=44100, mix=0.5)` → `array` — LFO-modulated short delay line mixed with the dry signal.

chow_test_stat

`chow_test_stat` — operation `chow test stat`. Category: **econometrics**.

```
p chow_test_stat $x
```

christoffel_first_kind_step

`christoffel_first_kind_step` — single-step update of `christoffel first kind`. Category: **electrochemistry, batteries, fuel cells**.

```
p christoffel_first_kind_step $x
```

christoffel_second_kind_step

`christoffel_second_kind_step` — single-step update of `christoffel second kind`. Category: **electrochemistry, batteries, fuel cells**.

```
p christoffel_second_kind_step $x
```

christoffel_symbol_normalize

`christoffel_symbol_normalize` — operation `christoffel symbol normalize`. Category: **electrochemistry, batteries, fuel cells**.

```
p christoffel_symbol_normalize $x
```

christoffel_word

`christoffel_word` — operation `christoffel word`. Category: **more extensions**.

```
p christoffel_word $x
```

christofides_ratio_bound

`christofides_ratio_bound` — operation `christofides ratio bound`. Category: **combinatorial optimization, scheduling**.

```
p christofides_ratio_bound $x
```

chroma_feature_step

`chroma_feature_step` — single-step update of `chroma feature`. Category: **electrochemistry, batteries, fuel cells**.

```
p chroma_feature_step $x
```

chromatic_adaptation

`chromatic_adaptation` — operation `chromatic adaptation`. Category: **complex / geom / color / trig**.

```
p chromatic_adaptation $x
```

chromatic_homology_rank

`chromatic_homology_rank` — operation `chromatic homology rank`. Category: **econometrics**.

```
p chromatic_homology_rank $x
```

chromatic_number_greedy

`chromatic_number_greedy` — operation `chromatic number greedy`. Category: **graph algorithms**.

```
p chromatic_number_greedy $x
```

chroot

`chroot` — operation `chroot`. Category: **process / system**.

```
p chroot $x
```

chsh_expectation

`chsh_expectation` — operation `chsh expectation`. Category: **quantum mechanics deep**.

```
p chsh_expectation $x
```

chull

`convex_hull(@points)` (alias `chull`) — computes the convex hull of a set of 2D points. Points given as flat list [x1,y1,x2,y2,...]. Returns the hull vertices in counterclockwise order. Uses Graham scan algorithm.

```
val @pts = (0,0, 1,1, 2,0, 1,0.5)
var $hull = chull(@pts)
p @$hull
```

chunk_by

`chunk_by` — operation `chunk by`. Category: **functional / iterator**.

```
p chunk_by $x
```

chunk_n

`chunk_n` — operation `chunk n`. Alias for `group_of_n`. Category: **collection**.

```
p chunk_n $x
```

chunk_string

`chunk_string` — operation `chunk string`. Category: **string processing extras**.

```
p chunk_string $x
```

chunk_while

`chunk_while` — operation `chunk while`. Category: **ruby enumerable extras**.

```
p chunk_while $x
```

church_numeral_n

`church_numeral_n` — operation `church numeral n`. Category: **logic, proof, sat/smt, type theory**.

```
p church_numeral_n $x
```

chvatal_gomory_cut

`chvatal_gomory_cut` — operation `chvatal gomory cut`. Category: *combinatorial optimization, scheduling*.

```
p chvatal_gomory_cut $x
```

ci

`confidence_interval` (alias `ci`) computes a confidence interval for the mean. Default 95%. Returns [lower, upper].

```
var ($lo, $hi) = @{ci([10,12,14,11,13])}
p "95%% CI: $lo to $hi"
var ($lo99, $hi99) = @{ci([10,12,14,11,13], 0.99)}
```

cidr_aggregate

`cidr_aggregate` — CIDR block op `aggregate`. Category: *network / ip / cidr*.

```
p cidr_aggregate $x
```

cidr_broadcast

`cidr_broadcast` — CIDR block op `broadcast`. Category: *network / ip / cidr*.

```
p cidr_broadcast $x
```

cidr_class

`cidr_class` — CIDR block op `class`. Category: *network / ip / cidr*.

```
p cidr_class $x
```

cidr_compare

`cidr_compare` — CIDR block op `compare`. Category: *network / ip / cidr*.

```
p cidr_compare $x
```

cidr_contains

`cidr_contains` — CIDR block op `contains`. Category: *network / ip / cidr*.

```
p cidr_contains $x
```

cidr_difference

`cidr_difference` — CIDR block op `difference`. Category: *network / ip / cidr*.

```
p cidr_difference $x
```

cidr_distance

`cidr_distance` — CIDR block op `distance`. Category: *network / ip / cidr*.

```
p cidr_distance $x
```

cidr_first_host

`cidr_first_host` — CIDR block op `first host`. Category: `*network / ip / cidr*`.

```
p cidr_first_host $x
```

cidr_format

`cidr_format` — CIDR block op `format`. Category: `*network / ip / cidr*`.

```
p cidr_format $x
```

cidr_hostmask

`cidr_hostmask` — CIDR block op `hostmask`. Category: `*network / ip / cidr*`.

```
p cidr_hostmask $x
```

cidr_hosts

`cidr_hosts` — CIDR block op `hosts`. Category: `*network / ip / cidr*`.

```
p cidr_hosts $x
```

cidr_intersection

`cidr_intersection` — CIDR block op `intersection`. Category: `*network / ip / cidr*`.

```
p cidr_intersection $x
```

cidr_is_aggregable

`cidr_is_aggregable` — CIDR block op `is aggregable`. Category: `*network / ip / cidr*`.

```
p cidr_is_aggregable $x
```

cidr_iterate

`cidr_iterate` — CIDR block op `iterate`. Category: `*network / ip / cidr*`.

```
p cidr_iterate $x
```

cidr_last_host

`cidr_last_host` — CIDR block op `last host`. Category: `*network / ip / cidr*`.

```
p cidr_last_host $x
```

cidr_minimum_covering

`cidr_minimum_covering` — CIDR block op `minimum covering`. Category: `*network / ip / cidr*`.

```
p cidr_minimum_covering $x
```

cidr_netmask

`cidr_netmask` — CIDR block op `netmask`. Category: `*network / ip / cidr*`.

```
p cidr_netmask $x
```

cidr_network

`cidr_network` — CIDR block op `network`. Category: `*network / ip / cidr*`.

```
p cidr_network $x
```

cidr_next

`cidr_next` — CIDR block op `next`. Category: `*network / ip / cidr*`.

```
p cidr_next $x
```

cidr_num_hosts

`cidr_num_hosts` — CIDR block op `num hosts`. Category: `*network / ip / cidr*`.

```
p cidr_num_hosts $x
```

cidr_overlaps

`cidr_overlaps` — CIDR block op `overlaps`. Category: `*network / ip / cidr*`.

```
p cidr_overlaps $x
```

cidr_parse

`cidr_parse` — CIDR block op `parse`. Category: `*network / ip / cidr*`.

```
p cidr_parse $x
```

cidr_prefix_len

`cidr_prefix_len` — CIDR block op `prefix len`. Category: `*network / ip / cidr*`.

```
p cidr_prefix_len $x
```

cidr_prev

`cidr_prev` — CIDR block op `prev`. Category: `*network / ip / cidr*`.

```
p cidr_prev $x
```

cidr_random_ip

`cidr_random_ip` — CIDR block op `random ip`. Category: `*network / ip / cidr*`.

```
p cidr_random_ip $x
```

cidr_size

`cidr_size` — CIDR block op `size`. Category: `*network / ip / cidr*`.

```
p cidr_size $x
```

cidr_sort

`cidr_sort` — CIDR block op `sort`. Category: `*network / ip / cidr*`.

```
p cidr_sort $x
```


cidr_split

`cidr_split` — CIDR block op `split`. Category: `*network / ip / cidr*`.

```
p cidr_split $x
```

cidr_subnet

`cidr_subnet` — CIDR block op `subnet`. Category: `*network / ip / cidr*`.

```
p cidr_subnet $x
```

cidr_subnets

`cidr_subnets` — CIDR block op `subnets`. Category: `*network / ip / cidr*`.

```
p cidr_subnets $x
```

cidr_summarize

`cidr_summarize` — CIDR block op `summarize`. Category: `*network / ip / cidr*`.

```
p cidr_summarize $x
```

cidr_supernet

`cidr_supernet` — CIDR block op `supernet`. Category: `*network / ip / cidr*`.

```
p cidr_supernet $x
```

cidr_to_netmask

`cidr_to_netmask` — convert from `cidr` to `netmask`. Category: `*network / ip / cidr*`.

```
p cidr_to_netmask $value
```

cidr_to_range

`cidr_to_range(cidr) → [first, last]` — network + broadcast addresses as strings.

```
p cidr_to_range("192.168.1.0/24") # ["192.168.1.0", "192.168.1.255"]
```

cidr_union

`cidr_union` — CIDR block op `union`. Category: `*network / ip / cidr*`.

```
p cidr_union $x
```

cidr_valid_subnet

`cidr_valid_subnet` — CIDR block op `valid subnet`. Category: `*network / ip / cidr*`.

```
p cidr_valid_subnet $x
```

cidr_wildcard

`cidr_wildcard` — CIDR block op `wildcard`. Category: `*network / ip / cidr*`.

```
p cidr_wildcard $x
```

`cie2000_color_distance`

`cie2000_color_distance([r,g,b], [r,g,b])` → `number` — CIEDE2000 ΔE . Most perceptually accurate of the CIE deltas.

`cie76_color_distance`

`cie76_color_distance([r,g,b], [r,g,b])` → `number` — CIE76 ΔE distance in Lab space (Euclidean).

```
p cie76_color_distance([255,0,0], [254,1,0])
```

`cie94_color_distance`

`cie94_color_distance([r,g,b], [r,g,b])` → `number` — CIE94 ΔE with chroma weighting.

`cindex`

`cindex` `STRING`, `SUBSTRING` [`,` `FROM`] — codepoint-indexed equivalent of Perl 5's `index`. Returns the zero-based codepoint position of the first occurrence, or -1 on miss. `FROM` is also a codepoint position. Pairs with `[i]` / `[a:b]` slice operators and `len` so search positions and slice bounds share one coordinate system.

Use `cindex` whenever your code mixes string search with `[]` slicing — Perl 5 `index` returns byte positions and silently misaligns when fed into `[]` (which is codepoint-based). `index` stays byte-indexed because `length` and `substr` are, and changing it would break Perl-5 binary-protocol code; `cindex` is the stryke-extension counterpart.

```
val $s = "hello - world"
p index $s, "world"      # 9 (bytes - '-' is 3 bytes)
p cindex $s, "world"     # 7 (codepoints)
var $i = cindex $s, "world"
p $s[$i : len($s) - 1]   # "world" - slice bounds line up
```

`cinv`

`cinv` — operation `cinv`. Alias for `count_inversions`. Category: **extended stdlib**.

```
p cinv $x
```

`cinvert`

`cinvert` — operation `cinvert`. Alias for `color_invert`. Category: **color operations**.

```
p cinvert $x
```

`cir_bond`

`cir_bond` — operation `cir bond`. Category: **financial pricing models**.

```
p cir_bond $x
```

`circ3`

`circle_from_three_points($x1, $y1, $x2, $y2, $x3, $y3)` (alias `circ3`) — finds the unique circle passing through three non-collinear points. Returns [`$cx`, `$cy`, `$radius`] or `undef` if collinear.

```
val $c = circ3(0, 0, 1, 0, 0.5, 0.866)
p "center=($c->[0], $c->[1]) r=$c->[2]"
```

`circ3pt`

`circ3pt` — operation `circ3pt`. Alias for `circle_from_three_points`. Category: **geometry (extended)**.

```
p circ3pt $x
```

`circle_3_points`

`circle_3_points` — circle geometry op `3 points`. Category: **more extensions (2)**.

```
p circle_3_points $x
```

`circle_area`

`circle_area` — circle geometry op `area`. Category: **complex / geom / color / trig**.

```
p circle_area $x
```

`circle_circumference`

`circle_circumference` — circle geometry op `circumference`. Category: **complex / geom / color / trig**.

```
p circle_circumference $x
```

`circle_intersects_circle`

`circle_intersects_circle` — circle geometry op `intersects circle`. Category: **complex / geom / color / trig**.

```
p circle_intersects_circle $x
```

`circle_intersects_line`

`circle_intersects_line` — circle geometry op `intersects line`. Category: **complex / geom / color / trig**.

```
p circle_intersects_line $x
```

`circle_of_fifths_step`

`circle_of_fifths_step` — circle geometry op `of fifths step`. Category: **misc**.

```
p circle_of_fifths_step $x
```

`circuit_depth`

`circuit_depth` — operation `circuit depth`. Category: **quantum**.

```
p circuit_depth $x
```

`circuit_width`

`circuit_width` — operation `circuit width`. Category: **quantum**.

```
p circuit_width $x
```

`circum`

`circum` — operation `circum`. Alias for `circumference`. Category: **geometry / physics**.

```
p circum $x
```

circumference

`circumference` — operation `circumference`. Category: **geometry / physics**.

```
p circumference $x
```

cityhash64

`cityhash64` — operation `cityhash64`. Category: **b81-misc-utility**.

```
p cityhash64 $x
```

clamp_array

`clamp_array` — operation `clamp array`. Category: **array / list operations extras**.

```
p clamp_array $x
```

clamp_each

`clamp_each` — operation `clamp each`. Category: **conversion / utility**.

```
p clamp_each $x
```

clamp_fn

`clamp_fn` — operation `clamp fn`. Category: **functional combinators**.

```
p clamp_fn $x
```

clamp_int

`clamp_int` — operation `clamp int`. Category: **extended stdlib**.

```
p clamp_int $x
```

clamp_list

`clamp_list` — operation `clamp list`. Category: **functional combinators**.

```
p clamp_list $x
```

clarr

`clarr` — operation `clarr`. Alias for `clamp_array`. Category: **array / list operations extras**.

```
p clarr $x
```

class

`class` declares a full object-oriented class with typed fields, inheritance, traits, instance methods, and static methods. Classes provide modern OOP semantics with a clean syntax.

Declaration:

```
class Animal {  
  name: Str  
  age: Int = 0  
  fn speak { p "Animal: " . $self->name }  
}
```

Inheritance with `extends`:

```
class Dog extends Animal {
  breed: Str = "Mixed"
  fn bark { p "Woof! I am " . $self->name }
  fn speak { p $self->name . " barks!" } # override
}
```

Construction (named or positional):

```
val $dog = Dog(name => "Rex", age => 5, breed => "Lab")
var $dog = Dog("Rex", 5, "Lab") # positional
```

Field access (getter/setter):

```
p $dog->name      # getter
$dog->age(6)      # setter
```

Static methods (fn `Self.name`):

```
class Math {
  fn Self.add($x, $y) { $x + $y }
  fn Self.pi { 3.14159 }
}
p Math::add(3, 4) # 7
p Math::pi()     # 3.14159
```

Traits (interfaces):

```
trait Printable { fn to_str }
class Item impl Printable {
  name: Str
  fn to_str { $self->name }
}
```

Multiple inheritance:

```
class C extends A, B { }
```

isa checks:

```
p $dog->isa("Dog")    # 1
p $dog->isa("Animal") # 1 (parent)
p $dog->isa("Cat")    # "" (false)
```

Built-in methods:

```
val @f = $dog->fields()      # (name, age, breed)
var $h = $dog->to_hash()     # { name => "Rex", ... }
var $d2 = $dog->with(age => 1) # functional update
var $d3 = $dog->clone()      # deep copy
```

Visibility (pub/priv):

```
class Secret {
  pub visible: Int = 1
  priv hidden: Int = 42
  pub fn get_hidden { $self->hidden } # internal access ok
}
```

Note: inherited fields come first in the values array; method lookup walks the inheritance chain.

`class_number_bound`

`class_number_bound` — operation `class number bound`. Category: *cryptanalysis & number theory deep*.

```
p class_number_bound $x
```

`clausen_cl2`

`clausen_cl2` — operation `clausen cl2`. Category: *special functions extra*.

```
p clausen_cl2 $x
```

clausius_clapeyron

clausius_clapeyron — operation `clausius clapeyron`. Category: **chemistry**.

```
p clausius_clapeyron $x
```

clausius_clapeyron_full

clausius_clapeyron_full — operation `clausius clapeyron full`. Category: **climate, fluids, atmospheric**.

```
p clausius_clapeyron_full $x
```

clean_price

clean_price — operation `clean price`. Category: **excel/sheets + bond/loan financial**.

```
p clean_price $x
```

clighten

clighten — operation `clighten`. Alias for `color_lighten`. Category: **color operations**.

```
p clighten $x
```

clique_count_3

clique_count_3 — operation `clique count 3`. Category: **more extensions**.

```
p clique_count_3 $x
```

clique_number_lower

clique_number_lower — operation `clique number lower`. Category: **combinatorial optimization, scheduling**.

```
p clique_number_lower $x
```

closed_pipe_harmonic

closed_pipe_harmonic — operation `closed pipe harmonic`. Category: **em / optics / relativity**.

```
p closed_pipe_harmonic $x
```

closest_pt_segment_2d

closest_pt_segment_2d — operation `closest pt segment 2d`. Category: **geometry / topology**.

```
p closest_pt_segment_2d $x
```

closing_2d

`closing_2d(image, size=3) → matrix` — dilation followed by erosion. Fills small dark holes.

clpi

`clpi` — operation `clpi`. Alias for `clamp_int`. Category: *extended stdlib*.

```
p clpi $x
```

cluster_attack_rate

`cluster_attack_rate` — rate of `cluster attack`. Category: *epidemiology / public health*.

```
p cluster_attack_rate $x
```

cluster_slots

`cluster_slots` — operation `cluster slots`. Category: *redis-flavour primitives*.

```
p cluster_slots $x
```

clustering_coefficient

`clustering_coefficient` — operation `clustering coefficient`. Category: *networkx graph algorithms*.

```
p clustering_coefficient $x
```

clutch_score

`clutch_score` — score of `clutch`. Category: *astronomy / astrometry*.

```
p clutch_score $x
```

cm_to_inches

`cm_to_inches` — convert from `cm` to `inches`. Category: *unit conversions*.

```
p cm_to_inches $value
```

cmap

`cmap` — operation `cmap`. Alias for `concat_map`. Category: *additional missing stdlib functions*.

```
p cmap $x
```

cmb_temperature

`cmb_temperature` — operation `cmb temperature`. Category: *cosmology / gr / flrw*.

```
p cmb_temperature $x
```

cmb_temperature_at_z

`cmb_temperature_at_z` — operation `cmb temperature at z`. Category: *cosmology / gr / flrw*.

```
p cmb_temperature_at_z $x
```

cmd_exists

`cmd_exists` — operation `cmd exists`. Category: *process / env*.

```
p cmd_exists $x
```

cmp_num

`cmp_num` — operation `cmp num`. Alias for `spaceship`. Category: `*file stat / path*`.

```
p cmp_num $x
```

cmp_str

`cmp_str` — operation `cmp str`. Category: `*file stat / path*`.

```
p cmp_str $x
```

cmp_r

`cmp_r` — operation `cmp_r`. Alias for `compress`. Category: `*python/ruby stdlib*`.

```
p cmp_r $x
```

cms

`cms(WIDTH=2048, DEPTH=5)` — Count-Min frequency sketch. `epsilon` $\approx e/\text{width}$ (over-estimation bound), `delta` $\approx 1/2^{\text{depth}}$ (failure probability). Defaults give 0.13% over-bound with 97% confidence. Counters are `u32` saturating; `cms_count` is always $\geq$ the true count.

```
val $c = cms(4096, 7)
cms_add($c, $_) for @stream
p cms_count($c, "frequent_key")
```

Family: `cms_add`, `cms_count`, `cms_merge`, `cms_clear`, `cms_serialize`, `cms_deserialize`.

cms_add

`cms_add(CMS, KEY, COUNT=1)` — add `COUNT` occurrences of `KEY` to the sketch.

cms_clear

`cms_clear` — operation `cms clear`. Category: `*probabilistic data structures*`.

```
p cms_clear $x
```

cms_count

`cms_count(CMS, KEY)` $\rightarrow$ estimated count (upper bound on truth).

cms_deserialize

`cms_deserialize` — operation `cms deserialize`. Category: `*probabilistic data structures*`.

```
p cms_deserialize $x
```

cms_from_bytes

`cms_from_bytes` — operation `cms from bytes`. Alias for `cms_deserialize`. Category: `*probabilistic data structures*`.

```
p cms_from_bytes $x
```

cms_merge

`cms_merge(CMS, OTHER)` — counter-wise sum (must share width and depth). Returns `1` on success.

cms_reset

`cms_reset` — operation `cms reset`. Alias for `cms_clear`. Category: *probabilistic data structures*.

```
p cms_reset $x
```

cms_serialize

`cms_serialize` — operation `cms serialize`. Category: *probabilistic data structures*.

```
p cms_serialize $x
```

cms_to_bytes

`cms_to_bytes` — convert from `cms` to `bytes`. Alias for `cms_serialize`. Category: *probabilistic data structures*.

```
p cms_to_bytes $value
```

cmyk_to_rgb

`cmyk_to_rgb` — convert from `cmyk` to `rgb`. Category: *b82-misc-utility*.

```
p cmyk_to_rgb $value
```

cn_coefficient

`cn_coefficient` — operation `cn coefficient`. Category: *ode advanced*.

```
p cn_coefficient $x
```

cn_ode

`cn_ode` — operation `cn ode`. Alias for `crank_nicolson_ode`. Category: *more extensions (2)*.

```
p cn_ode $x
```

cnf_dp11_branch

`cnf_dp11_branch` — operation `cnf dp11 branch`. Category: *logic, proof, sat/smt, type theory*.

```
p cnf_dp11_branch $x
```

cnf_pure_literal_elim

`cnf_pure_literal_elim` — operation `cnf pure literal elim`. Category: *logic, proof, sat/smt, type theory*.

```
p cnf_pure_literal_elim $x
```

cnf_unit_propagate

`cnf_unit_propagate` — operation `cnf unit propagate`. Category: *logic, proof, sat/smt, type theory*.

```
p cnf_unit_propagate $x
```

cnt

`count` (aliases `len`, `cnt`) returns the number of elements in a list, the **codepoint count** of a string, the number of key-value pairs in a hash, or the cardinality of a set. Universal length-like function that dispatches on the type of its argument. With **no arguments**, it is the same as one argument on `$_` (like Perl 5's bare `length` in `map/grep` blocks). Replaces `scalar @array`

/ `length $string` for stryke code; on strings it pairs with `[i]` / `[a:b]` and `cindex` / `crindex`, all of which use codepoint positions.

For byte length on a string, use Perl 5's `length` (kept byte-based for compat with `index` / `rindex` / `substr`). `size` is NOT a count alias — it returns a file's byte size.

```
val @arr = (1, 2, 3, 4, 5)
p cnt @arr          # 5
p len "hello"       # 5
p len "-"           # 1 (codepoint)
p length "-"        # 3 (bytes)
var %h = (a => 1, b => 2)
p cnt \%h           # 2
var @long = grep { len > 3 } qw(ab abcd x) # abcd
rl("file.txt") |> cnt |> p # line count
```

cntw

`cntw` — operation `cntw`. Alias for `count_while`. Category: **ruby enumerable extras**.

```
p cntw $x
```

co2_growth_rate_step

`co2_growth_rate_step` — single-step update of `co2 growth rate`. Category: **climate, fluids, atmospheric**.

```
p co2_growth_rate_step $x
```

coalesce

`coalesce` — operation `coalesce`. Category: **functional combinators**.

```
p coalesce $x
```

coalescent_expected_time

`coalescent_expected_time` — operation `coalescent expected time`. Category: **bioinformatics deep**.

```
p coalescent_expected_time $x
```

coalescent_tree_length

`coalescent_tree_length` — operation `coalescent tree length`. Category: **bioinformatics deep**.

```
p coalescent_tree_length $x
```

cobb_douglas

`cobb_douglas` — operation `cobb douglas`. Category: **economics + game theory**.

```
p cobb_douglas $x
```

code_point_at

`code_point_at` — operation `code point at`. Category: **extended stdlib**.

```
p code_point_at $x
```

codifferential_step

`codifferential_step` — single-step update of `codifferential`. Category: *electrochemistry, batteries, fuel cells*.

```
p codifferential_step $x
```

codon_adaptation_index

`codon_adaptation_index` — codon table / translation `adaptation index`. Category: *bioinformatics deep*.

```
p codon_adaptation_index $x
```

codon_optimize

`codon_optimize(protein)` → `string` — DNA back-translation using a high-frequency codon per amino acid (E. coli-style).

```
p codon_optimize("MKL")
```

codon_to_amino_acid

`codon_to_amino_acid(codon)` → `string` — single-letter amino acid code (* for stop, ? for unknown).

```
p codon_to_amino_acid("ATG") # M (Methionine)
p codon_to_amino_acid("TAA") # * (stop)
```

codon_usage_table

`codon_usage_table(dna)` → `hash` — count of each codon in a coding sequence; reads in 3-nt frames.

codon_usage_variance

`codon_usage_variance` — codon table / translation `usage variance`. Category: *bioinformatics deep*.

```
p codon_usage_variance $x
```

coeff_of_variation

`coeff_of_variation` — operation `coeff of variation`. Category: *math / numeric extras*.

```
p coeff_of_variation $x
```

cognitive_hierarchy_step

`cognitive_hierarchy_step` — single-step update of `cognitive hierarchy`. Category: *climate, fluids, atmospheric*.

```
p cognitive_hierarchy_step $x
```

cohen_d

`cohen_d` (alias `cohend`) computes Cohen's d effect size between two samples. Small=0.2, medium=0.5, large=0.8.

```
p cohend([1,2,3], [4,5,6]) # large effect
```

cohen_kappa

`cohen_kappa` — operation `cohen kappa`. Category: *ml extensions*.

```
p cohen_kappa $x
```

cohen_sutherland_clip

`cohen_sutherland_clip` — operation `cohen_sutherland_clip`. Category: **test runner**.

```
p cohen_sutherland_clip $x
```

coherence_xy

`coherence_xy` — operation `coherence_xy`. Category: **economics + game theory**.

```
p coherence_xy $x
```

coherent_mean_photons

`coherent_mean_photons` — operation `coherent_mean_photons`. Category: **quantum mechanics deep**.

```
p coherent_mean_photons $x
```

cohomology_rank

`cohomology_rank` — operation `cohomology_rank`. Category: **econometrics**.

```
p cohomology_rank $x
```

coif1_coeffs

`coif1_coeffs` — operation `coif1_coeffs`. Category: **more extensions**.

```
p coif1_coeffs $x
```

coiflet_wavelet_step

`coiflet_wavelet_step` — single-step update of `coiflet_wavelet`. Category: **electrochemistry, batteries, fuel cells**.

```
p coiflet_wavelet_step $x
```

coin_flip

`coin_flip` — operation `coin_flip`. Category: **random**.

```
p coin_flip $x
```

cointegration_eg

`cointegration_eg` — operation `cointegration_eg`. Category: **statsmodels**.

```
p cointegration_eg $x
```

cointegration_residual

`cointegration_residual` — operation `cointegration_residual`. Category: **econometrics**.

```
p cointegration_residual $x
```

colMeans

`col_means` (alias `colMeans`) — mean of each column. Like R's `colMeans()`.

```
p colMeans([[2,4],[6,8]]) # [4, 6]
```

colSums

`col_sums` (alias `colSums`) — sum of each column. Like R's `colSums()`.

```
p colSums([[1,2],[3,4]]) # [4, 6]
```

cole_cole_eis

`cole_cole_eis` — operation `cole cole eis`. Category: *electrochemistry, batteries, fuel cells*.

```
p cole_cole_eis $x
```

collapse_whitespace

`collapse_whitespace` — operation `collapse whitespace`. Category: *string processing extras*.

```
p collapse_whitespace $x
```

collatz

`collatz` — operation `collatz`. Alias for `collatz_length`. Category: *math / numeric extras*.

```
p collatz $x
```

collatz_length

`collatz_length` — operation `collatz length`. Category: *math / numeric extras*.

```
p collatz_length $x
```

collatz_sequence

`collatz_sequence` — operation `collatz sequence`. Category: *math / numeric extras*.

```
p collatz_sequence $x
```

collatz_steps

`collatz_steps` — operation `collatz steps`. Category: *math / number theory extras*.

```
p collatz_steps $x
```

collatzseq

`collatzseq` — operation `collatzseq`. Alias for `collatz_sequence`. Category: *math / numeric extras*.

```
p collatzseq $x
```

collect_into_btreemap

`collect_into_btreemap` — operation `collect into btreemap`. Category: *iterator + string-distance extras*.

```
p collect_into_btreemap $x
```

collect_into_btreeset

`collect_into_btreeset` — operation `collect into btreeset`. Category: *iterator + string-distance extras*.

```
p collect_into_btreeset $x
```

collect_into_hashmap

`collect_into_hashmap` — operation `collect into hashmap`. Category: **iterator + string-distance extras**.

```
p collect_into_hashmap $x
```

collect_into_hashset

`collect_into_hashset` — operation `collect into hashset`. Category: **iterator + string-distance extras**.

```
p collect_into_hashset $x
```

collect_into_string

`collect_into_string` — operation `collect into string`. Category: **iterator + string-distance extras**.

```
p collect_into_string $x
```

collision_response_2d

`collision_response_2d(v1, v2, normal, m1, m2) → [v1', v2']` — impulse-based elastic collision along contact normal.

collusion_payoff_step

`collusion_payoff_step` — single-step update of `collusion payoff`. Category: **climate, fluids, atmospheric**.

```
p collusion_payoff_step $x
```

collws

`collws` — operation `collws`. Alias for `collapse_whitespace`. Category: **string processing extras**.

```
p collws $x
```

color256

`color256` — operation `color256`. Category: **color / ansi**.

```
p color256 $x
```

color_blend

`color_blend` — color helper `blend`. Category: **color operations**.

```
p color_blend $x
```

color_blend_alpha

`color_blend_alpha` — color helper `blend alpha`. Category: **b82-misc-utility**.

```
p color_blend_alpha $x
```

color_blend_screen

`color_blend_screen` — color helper `blend screen`. Category: **complex / geom / color / trig**.

```
p color_blend_screen $x
```

color_blend_t

`color_blend_t` — color helper `blend t`. Category: **misc**.

```
p color_blend_t $x
```

color_complement

`color_complement` — color helper `complement`. Category: **color operations**.

```
p color_complement $x
```

color_darken

`color_darken` — color helper `darken`. Category: **color operations**.

```
p color_darken $x
```

color_distance

`color_distance` — color helper `distance`. Category: **color operations**.

```
p color_distance $x
```

color_grayscale

`color_grayscale` — color helper `grayscale`. Category: **color operations**.

```
p color_grayscale $x
```

color_interpolate_hsl

`color_interpolate_hsl` — color helper `interpolate hsl`. Category: **complex / geom / color / trig**.

```
p color_interpolate_hsl $x
```

color_interpolate_lab

`color_interpolate_lab` — color helper `interpolate lab`. Category: **complex / geom / color / trig**.

```
p color_interpolate_lab $x
```

color_interpolate_oklab

`color_interpolate_oklab` — color helper `interpolate oklab`. Category: **complex / geom / color / trig**.

```
p color_interpolate_oklab $x
```

color_interpolate_rgb

`color_interpolate_rgb` — color helper `interpolate rgb`. Category: **complex / geom / color / trig**.

```
p color_interpolate_rgb $x
```

color_invert

`color_invert` — color helper `invert`. Category: **color operations**.

```
p color_invert $x
```

color_lighten

`color_lighten` — color helper `lighten`. Category: *color operations*.

```
p color_lighten $x
```

color_temperature_kelvin

`color_temperature_kelvin` — color helper `temperature kelvin`. Category: *b82-misc-utility*.

```
p color_temperature_kelvin $x
```

color_temperature_to_rgb

`color_temperature_to_rgb` — convert from `color temperature` to `rgb`. Category: *complex / geom / color / trig*.

```
p color_temperature_to_rgb $value
```

columbic_capacity_lihalfcell

`columbic_capacity_lihalfcell` — operation `columbic capacity lihalfcell`. Category: *electrochemistry, batteries, fuel cells*.

```
p columbic_capacity_lihalfcell $x
```

column

`column(\@ITEMS)` → tabulate items into terminal-width-aware fixed-width columns. Falls back to 80 cols when terminal size is unavailable. Returns a single multi-line string suitable for `print`.

```
p column([sort keys %b]) # all builtins in columns
```

column_generation_step

`column_generation_step` — single-step update of `column generation`. Category: *combinatorial optimization, scheduling*.

```
p column_generation_step $x
```

comb

`comb` — Raku string decomposition. Four signatures:

- `comb($str)` — split into individual characters
- `comb($size: Int, $str [, $limit])` — fixed-size character chunks
- `comb($needle: Str, $str [, $limit])` — every literal occurrence of `$needle`
- `comb(qr/PAT/, $str [, $limit])` — every regex match (`m/.../g` semantics)

`$limit` caps the number of returned pieces; a negative limit means unlimited.

```
p comb("abc") # ("a","b","c")
p comb(3, "abcdefghijk") # ("abc","def","ghi","jk")
p comb("ab", "ababab") # ("ab","ab","ab")
p comb(qr/\w+/, "a;b;;c") # ("a","b","c")
p comb(qr/\d+/, "a1b22c333", 2) # ("1","22")
```

comb_filter

`comb_filter(signal, delay=441, gain=0.5)` → array — feedback comb: $y[n] = x[n] + g \cdot y[n - \text{delay}]$.

combinations

`combinations($n, @list)` — all n -element combinations of `@list` (Python's `itertools.combinations`). Returns an array of array-refs. Order within each combination preserves the list order.

```
p combinations(2, 1, 2, 3)      # [[1,2],[1,3],[2,3]]
p combinations(3, qw(a b c d)) # [[a,b,c],[a,b,d],[a,c,d],[b,c,d]]
```

combinations_rep

`combinations_rep` — operation `combinations rep`. Category: **additional missing stdlib functions**.

```
p combinations_rep $x
```

combinatorial_auction_step

`combinatorial_auction_step` — single-step update of `combinatorial auction`. Category: **climate, fluids, atmospheric**.

```
p combinatorial_auction_step $x
```

combrep

`combrep` — operation `combrep`. Alias for `combinations_rep`. Category: **additional missing stdlib functions**.

```
p combrep $x
```

comm

`comm(\@A, \@B)` — 3-column compare of two SORTED lists. Returns an arrayref [`$only_a`, `$only_b`, `$both`] where each entry is an arrayref of lines. Mirrors POSIX `comm`: input must already be sorted (lexical); use `sort` first if unsure.

```
val $r = comm([sort @set_a], [sort @set_b])
p "only in A: " . scalar @{$r->[0]}
p "only in B: " . scalar @{$r->[1]}
p "in both: "   . scalar @{$r->[2]}
```

common_kmers

`common_kmers` — operation `common kmers`. Category: **bioinformatics deep**.

```
p common_kmers $x
```

common_knowledge_iterations

`common_knowledge_iterations` — operation `common knowledge iterations`. Category: **climate, fluids, atmospheric**.

```
p common_knowledge_iterations $x
```

common_prefix

`common_prefix` — operation `common prefix`. Category: **extended stdlib**.

```
p common_prefix $x
```

common_suffix

`common_suffix` — operation `common suffix`. Category: **extended stdlib**.

```
p common_suffix $x
```

comoving_distance

`comoving_distance` — distance / dissimilarity metric of `comoving`. Category: *cosmology / gr / flrw*.

```
p comoving_distance $x
```

comoving_distance_approx

`comoving_distance_approx` — operation `comoving distance approx`. Category: *astronomy / music / color / units*.

```
p comoving_distance_approx $x
```

compare_versions

`compare_versions` — operation `compare versions`. Category: *file stat / path*.

```
p compare_versions $x
```

complex_abs

`complex_abs` — complex number op `abs`. Category: *complex / geom / color / trig*.

```
p complex_abs $x
```

complex_add

`complex_add` — complex number op `add`. Category: *complex / geom / color / trig*.

```
p complex_add $x
```

complex_angle

`complex_angle` — complex number op `angle`. Category: *complex / geom / color / trig*.

```
p complex_angle $x
```

complex_conjugate

`complex_conjugate` — complex number op `conjugate`. Category: *complex / geom / color / trig*.

```
p complex_conjugate $x
```

complex_cos

`complex_cos` — complex number op `cos`. Category: *complex / geom / color / trig*.

```
p complex_cos $x
```

complex_cosh

`complex_cosh` — complex number op `cosh`. Category: *complex / geom / color / trig*.

```
p complex_cosh $x
```

complex_div

`complex_div` — complex number op `div`. Category: *complex / geom / color / trig*.

```
p complex_div $x
```

complex_equal

`complex_equal` — complex number op `equal`. Category: *complex / geom / color / trig*.

```
p complex_equal $x
```

complex_exp

`complex_exp` — complex number op `exp`. Category: *complex / geom / color / trig*.

```
p complex_exp $x
```

complex_from_polar

`complex_from_polar` — complex number op `from polar`. Category: *complex / geom / color / trig*.

```
p complex_from_polar $x
```

complex_imag

`complex_imag` — complex number op `imag`. Category: *complex / geom / color / trig*.

```
p complex_imag $x
```

complex_log

`complex_log` — complex number op `log`. Category: *complex / geom / color / trig*.

```
p complex_log $x
```

complex_magnitude

`complex_magnitude` — complex number op `magnitude`. Category: *complex / geom / color / trig*.

```
p complex_magnitude $x
```

complex_mul

`complex_mul` — complex number op `mul`. Category: *complex / geom / color / trig*.

```
p complex_mul $x
```

complex_new

`complex_new` — complex number op `new`. Category: *complex / geom / color / trig*.

```
p complex_new $x
```

complex_phase

`complex_phase` — complex number op `phase`. Category: *complex / geom / color / trig*.

```
p complex_phase $x
```

complex_polar

`complex_polar` — complex number op `polar`. Category: *complex / geom / color / trig*.

```
p complex_polar $x
```

complex_pow

`complex_pow` — complex number op `pow`. Category: *complex / geom / color / trig*.

```
p complex_pow $x
```

complex_real

`complex_real` — complex number op `real`. Category: *complex / geom / color / trig*.

```
p complex_real $x
```

complex_sin

`complex_sin` — complex number op `sin`. Category: *complex / geom / color / trig*.

```
p complex_sin $x
```

complex_sinh

`complex_sinh` — complex number op `sinh`. Category: *complex / geom / color / trig*.

```
p complex_sinh $x
```

complex_sqrt

`complex_sqrt` — complex number op `sqrt`. Category: *complex / geom / color / trig*.

```
p complex_sqrt $x
```

complex_sub

`complex_sub` — complex number op `sub`. Category: *complex / geom / color / trig*.

```
p complex_sub $x
```

complex_tan

`complex_tan` — complex number op `tan`. Category: *complex / geom / color / trig*.

```
p complex_tan $x
```

complex_tanh

`complex_tanh` — complex number op `tanh`. Category: *complex / geom / color / trig*.

```
p complex_tanh $x
```

composition_count

`composition_count` — count of `composition`. Category: *misc*.

```
p composition_count $x
```

compositions_count

`compositions_count(n) → int` — $2^{(n-1)}$ compositions of n .

compound_interest

`compound_interest` — operation `compound interest`. Category: **physics formulas**.

```
p compound_interest $x
```

compound_interest_periods

`compound_interest_periods` — operation `compound interest periods`. Category: **misc**.

```
p compound_interest_periods $x
```

compress

`compress` — operation `compress`. Category: **python/ruby stdlib**.

```
p compress $x
```

compressibility_z

`compressibility_z` — operation `compressibility z`. Category: **chemistry**.

```
p compressibility_z $x
```

compton_wavelength

`compton_wavelength` — operation `compton wavelength`. Category: **quantum mechanics deep**.

```
p compton_wavelength $x
```

compute_residuals

`compute_residuals` — operation `compute residuals`. Alias for `residuals_compute`. Category: **misc**.

```
p compute_residuals $x
```

concat_map

`concat_map` — operation `concat map`. Category: **additional missing stdlib functions**.

```
p concat_map $x
```

concentration_dilute

`concentration_dilute` — operation `concentration dilute`. Category: **misc**.

```
p concentration_dilute $x
```

concordance_correlation

`concordance_correlation` — operation `concordance correlation`. Category: **more extensions**.

```
p concordance_correlation $x
```

cond

`matrix_cond` (aliases `mcond`, `cond`) estimates the condition number of a matrix (ratio of largest to smallest singular value). Large values indicate ill-conditioning.

```
p cond([[1,0],[0,1]]) # 1 (perfect)
p cond([[1,2],[2,4]]) # Inf (singular)
```

conditional_entropy

`conditional_entropy(joint_matrix)` → `number` — $H(X|Y)$ from joint distribution $P(X,Y)$.

conditional_entropy_step

`conditional_entropy_step` — single-step update of `conditional_entropy`. Category: *electrochemistry, batteries, fuel cells*.

```
p conditional_entropy_step $x
```

conditional_gradient_step

`conditional_gradient_step` — single-step update of `conditional_gradient`. Category: *combinatorial optimization, scheduling*.

```
p conditional_gradient_step $x
```

condorcet_winner_check

`condorcet_winner_check` — operation `condorcet winner check`. Category: *climate, fluids, atmospheric*.

```
p condorcet_winner_check $x
```

cone_surface

`cone_surface` — operation `cone surface`. Category: *geometry / topology*.

```
p cone_surface $x
```

cone_surface_area

`cone_surface_area` — operation `cone surface area`. Category: *complex / geom / color / trig*.

```
p cone_surface_area $x
```

cone_volume

`cone_volume` — operation `cone volume`. Category: *geometry / physics*.

```
p cone_volume $x
```

conevol

`conevol` — operation `conevol`. Alias for `cone_volume`. Category: *geometry / physics*.

```
p conevol $x
```

confint

`confint_lm` (alias `confint`) — confidence intervals for model coefficients. Returns hash with `intercept_lower/upper`, `slope_lower/upper`.

```
val $ci = confint(lm([1,2,3,4,5], [2,4,5,4,5]))
p $ci->{slope_lower}
```

conformal_compactification_step

`conformal_compactification_step` — single-step update of `conformal_compactification`. Category: **electrochemistry, batteries, fuel cells**.

```
p conformal_compactification_step $x
```

confusion_counts

`confusion_counts` — operation `confusion_counts`. Category: **ml extensions**.

```
p confusion_counts $x
```

conj

`conj` — operation `conj`. Category: **algebraic match**.

```
p conj $x
```

connected_components

`connected_components` — operation `connected_components`. Category: **misc**.

```
p connected_components $x
```

connected_components_graph

`connected_components_graph` — operation `connected_components_graph`. Category: **extended stdlib**.

```
p connected_components_graph $x
```

cons

`cons` — operation `cons`. Category: **algebraic match**.

```
p cons $x
```

consecutive_eq

`consecutive_eq` — operation `consecutive_eq`. Category: **collection more**.

```
p consecutive_eq $x
```

consecutive_pairs

`consecutive_pairs` — operation `consecutive_pairs`. Category: **list helpers**.

```
p consecutive_pairs $x
```

consistent_hash

`hash_ring(VNODES_PER_NODE=128)` — consistent-hash ring (Karger-style virtual nodes). Adds and removes nodes with minimal key remapping ($\approx 1/(N+1)$ of keys move when a node joins N existing nodes). xxh3-64 hashing; binary search lookup is $O(\log V)$.

```
val $hr = hash_ring(128)
hr_add($hr, "shard-a", "shard-b", "shard-c")
var $owner = hr_get($hr, "user-42")
```

const_fn

`const_fn` — operation `const fn`. Category: **functional primitives**.

```
p const_fn $x
```

constant_case

`constant_case` — operation `constant case`. Category: **string**.

```
p constant_case $x
```

constants_au_meters

`constants_au_meters()` → `number` — 1 AU = $1.495978707 \cdot 10^{11}$ m (exact).

constants_avogadro_n

`constants_avogadro_n()` → `number` — $N_A = 6.02214076 \cdot 10^{23}$ mol⁻¹ (exact by SI definition).

constants_bohr_radius

`constants_bohr_radius()` → `number` — $a_0 = 5.29177210903 \cdot 10^{-11}$ m.

constants_boltzmann_k

`constants_boltzmann_k()` → `number` — $k_B = 1.380649 \cdot 10^{-23}$ J/K (exact).

constants_earth_mass

`constants_earth_mass()` → `number` — $M_\oplus = 5.972 \cdot 10^{24}$ kg.

constants_earth_radius

`constants_earth_radius()` → `number` — $R_\oplus = 6.371 \cdot 10^6$ m (mean).

constants_electron_charge

`constants_electron_charge()` → `number` — $e = 1.602176634 \cdot 10^{-19}$ C (exact by SI definition).

constants_electron_mass

`constants_electron_mass()` → `number` — $m_e = 9.1093837015 \cdot 10^{-31}$ kg.

constants_faraday

`constants_faraday()` → `number` — $F = 96,485.33212$ C/mol.

`constants_gas_r`

`constants_gas_r()` → `number` — $R = 8.314462618 \text{ J/(mol}\cdot\text{K)}$.

`constants_gravitational_g`

`constants_gravitational_g()` → `number` — $G = 6.6743\cdot 10^{-11} \text{ m}^3/(\text{kg}\cdot\text{s}^2)$.

`constants_lightyear_meters`

`constants_lightyear_meters()` → `number` — $1 \text{ ly} = 9.4607304725808\cdot 10^{15} \text{ m}$.

`constants_neutron_mass`

`constants_neutron_mass()` → `number` — $m_n = 1.67492749804\cdot 10^{-27} \text{ kg}$.

`constants_parsec_meters`

`constants_parsec_meters()` → `number` — $1 \text{ pc} = 3.085677581\cdot 10^{16} \text{ m}$.

`constants_planck`

`constants_planck()` → `number` — Planck's constant $h = 6.62607015\cdot 10^{-34} \text{ J}\cdot\text{s}$.

`constants_planck_h`

`constants_planck_h()` → `number` — Planck constant (alias).

`constants_planck_hbar`

`constants_planck_hbar()` → `number` — Reduced Planck constant $\hbar = h/2\pi$.

`constants_proton_mass`

`constants_proton_mass()` → `number` — $m_p = 1.67262192369\cdot 10^{-27} \text{ kg}$.

`constants_rydberg`

`constants_rydberg()` → `number` — $R_\infty = 1.0973731568160\cdot 10^7 \text{ m}^{-1}$.

`constants_solar_mass`

`constants_solar_mass()` → `number` — $M_\odot = 1.989\cdot 10^{30} \text{ kg}$.

`constants_solar_radius`

`constants_solar_radius()` → `number` — $R_\odot = 6.957\cdot 10^8 \text{ m}$.

`constants_speed_of_light`

`constants_speed_of_light()` → `number` — $c = 299,792,458 \text{ m/s}$ (exact by SI definition).

`constants_stefan_boltzmann`

`constants_stefan_boltzmann()` → `number` — $\sigma = 5.670374419\cdot 10^{-8} \text{ W}/(\text{m}^2\cdot\text{K}^4)$.

constituency_cyk

`constituency_cyk` — operation `constituency cyk`. Category: *computational linguistics*.

```
p constituency_cyk $x
```

consumer_surplus

`consumer_surplus` — operation `consumer surplus`. Category: *economics + game theory*.

```
p consumer_surplus $x
```

contact_tracing_eff

`contact_tracing_eff` — operation `contact tracing eff`. Category: *epidemiology / public health*.

```
p contact_tracing_eff $x
```

contains

`contains` — operation `contains`. Category: *trivial string ops*.

```
p contains $x
```

contains_all

`contains_all` — operation `contains all`. Category: *string*.

```
p contains_all $x
```

contains_any

`contains_any` — operation `contains any`. Category: *string*.

```
p contains_any $x
```

contains_elem

`contains_elem` — operation `contains elem`. Category: *list helpers*.

```
p contains_elem $x
```

contcomp

`contcomp` — operation `contcomp`. Alias for `continuous_compound`. Category: *finance (extended)*.

```
p contcomp $x
```

content_negotiate

`content_negotiate` — operation `content negotiate`. Category: *b81-misc-utility*.

```
p content_negotiate $x
```

continued_fraction_sqrt

`continued_fraction_sqrt` — operation `continued fraction sqrt`. Category: *cryptanalysis & number theory deep*.

```
p continued_fraction_sqrt $x
```

contour_area

`contour_area(points)` → `number` — shoelace polygon area.

```
p contour_area([[0,0],[1,0],[1,1],[0,1]]) # 1
```

contour_centroid

`contour_centroid(points)` → `[x, y]` — area-weighted centroid of the closed polygon (M_{10}/M_{00} , M_{01}/M_{00} per OpenCV). Falls back to vertex mean if the polygon has zero signed area.

contour_find

`contour_find(image, thresh=128)` → `array` — boundary cells of foreground regions as `[[row, col], ...]`.

contour_perimeter

`contour_perimeter(points)` → `number` — perimeter of a polygon defined by an ordered point list.

contrast_ratio

`contrast_ratio` — ratio of `contrast`. Category: **b82-misc-utility**.

```
p contrast_ratio $x
```

contrast_ratio_wcag

`contrast_ratio_wcag` — operation `contrast ratio wcag`. Category: **misc**.

```
p contrast_ratio_wcag $x
```

conv

`conv` — operation `conv`. Alias for `convolution`. Category: **math / numeric extras**.

```
p conv $x
```

conv1d_apply

`conv1d_apply(signal, kernel)` → `array` — 1D convolution with zero-padding.

conv2d_apply

`conv2d_apply(image, kernel)` → `matrix` — true mathematical 2D convolution: kernel is **flipped** in both axes before the sum-of-products. Zero-padded; output has same dimensions as input. Use `correlate2d` for the cross-correlation form (no flip).

```
p conv2d_apply($img, gaussian_kernel(5, 1.0))
```

converge

`converge` — operation `converge`. Category: **functional combinators**.

```
p converge $x
```

convex_hull_3d

convex_hull_3d — operation `convex hull 3d`. Category: **test runner**.

```
p convex_hull_3d $x
```

convex_hull_3d_simple

convex_hull_3d_simple — operation `convex hull 3d simple`. Category: **test runner**.

```
p convex_hull_3d_simple $x
```

convex_hull_chan

convex_hull_chan — operation `convex hull chan`. Category: **excel/sheets + bond/loan financial**.

```
p convex_hull_chan $x
```

convex_hull_jarvis

convex_hull_jarvis — operation `convex hull jarvis`. Category: **geometry / topology**.

```
p convex_hull_jarvis $x
```

convexity

convexity(cashflows, rate) → number — second-order yield sensitivity: $\sum t \cdot (t+1) \cdot cf / (1+r)^{(t+2)} / price$.

```
p convexity([100,100,1100], 0.05)
```

convolution

convolution — operation `convolution`. Category: **math / numeric extras**.

```
p convolution $x
```

convolve_2d

convolve_2d — operation `convolve 2d`. Category: **pil/opencv image processing**.

```
p convolve_2d $x
```

convolve_full

convolve_full — operation `convolve full`. Category: **numpy + scipy.special**.

```
p convolve_full $x
```

convolve_valid

convolve_valid — operation `convolve valid`. Category: **numpy + scipy.special**.

```
p convolve_valid $x
```

cookie_domain_matches

cookie_domain_matches — operation `cookie domain matches`. Category: **network / ip / cidr**.

```
p cookie_domain_matches $x
```

cookie_format

cookie_format — operation `cookie format`. Category: `*network / ip / cidr*`.

```
p cookie_format $x
```

cookie_is_expired

cookie_is_expired — operation `cookie is expired`. Category: `*network / ip / cidr*`.

```
p cookie_is_expired $x
```

cookie_is_session

cookie_is_session — operation `cookie is session`. Category: `*network / ip / cidr*`.

```
p cookie_is_session $x
```

cookie_jar_add

cookie_jar_add — operation `cookie jar add`. Category: `*network / ip / cidr*`.

```
p cookie_jar_add $x
```

cookie_jar_get

cookie_jar_get — operation `cookie jar get`. Category: `*network / ip / cidr*`.

```
p cookie_jar_get $x
```

cookie_jar_new

cookie_jar_new — constructor of `cookie jar`. Category: `*network / ip / cidr*`.

```
p cookie_jar_new $x
```

cookie_parse

cookie_parse — operation `cookie parse`. Category: `*network / ip / cidr*`.

```
p cookie_parse $x
```

cookie_path_matches

cookie_path_matches — operation `cookie path matches`. Category: `*network / ip / cidr*`.

```
p cookie_path_matches $x
```

cookie_set_max_age

cookie_set_max_age — operation `cookie set max age`. Category: `*network / ip / cidr*`.

```
p cookie_set_max_age $x
```

coombs_runoff_step

coombs_runoff_step — single-step update of `coombs runoff`. Category: `*climate, fluids, atmospheric*`.

```
p coombs_runoff_step $x
```

cooperative_game_value

`cooperative_game_value` — value of `cooperative game`. Category: **climate, fluids, atmospheric**.

```
p cooperative_game_value $x
```

coordination_game_payoff

`coordination_game_payoff` — operation `coordination game payoff`. Category: **climate, fluids, atmospheric**.

```
p coordination_game_payoff $x
```

copeland_score_step

`copeland_score_step` — single-step update of `copeland score`. Category: **climate, fluids, atmospheric**.

```
p copeland_score_step $x
```

coppersmith_bound

`coppersmith_bound` — operation `coppersmith bound`. Category: **cryptanalysis & number theory deep**.

```
p coppersmith_bound $x
```

coptic_from_fixed

`coptic_from_fixed` — operation `coptic from fixed`. Category: **calendrical algorithms**.

```
p coptic_from_fixed $x
```

copy_sign

`copy_sign` — sign with private key of `copy`. Category: **extended stdlib**.

```
p copy_sign $x
```

copy_within

`copy_within` — operation `copy within`. Category: **additional missing stdlib functions**.

```
p copy_within $x
```

copysign

`copysign` — operation `copysign`. Category: **math functions**.

```
p copysign $x
```

coq_tactic_apply

`coq_tactic_apply` — operation `coq tactic apply`. Category: **logic, proof, sat/smt, type theory**.

```
p coq_tactic_apply $x
```

coq_unify_term

`coq_unify_term` — operation `coq unify term`. Category: **logic, proof, sat/smt, type theory**.

```
p coq_unify_term $x
```

cor_mat

`cor_matrix` (alias `cor_mat`) — correlation matrix from observations. Each row is an observation vector.

```
val $R = cor_mat([[1,2],[3,4],[5,6]]) # 2x2 correlation matrix
```

core_coalition

`core_coalition` — operation `core coalition`. Category: *economics + game theory*.

```
p core_coalition $x
```

core_membership_check

`core_membership_check` — operation `core membership check`. Category: *climate, fluids, atmospheric*.

```
p core_membership_check $x
```

coreference_singleton

`coreference_singleton` — operation `coreference singleton`. Category: *computational linguistics*.

```
p coreference_singleton $x
```

corner_subpix

`corner_subpix` — operation `corner subpix`. Category: *pil/opencv image processing*.

```
p corner_subpix $x
```

corr

`corr` — operation `corr`. Alias for `correlation`. Category: *extended stdlib*.

```
p corr $x
```

corr_agg

`corr_agg` — operation `corr agg`. Category: *postgres sql strings, json, aggregates*.

```
p corr_agg $x
```

correlate2d

`correlate2d(image, kernel) → matrix` — 2D cross-correlation (no kernel flip).

correlate_full

`correlate_full` — operation `correlate full`. Category: *numpy + scipy.special*.

```
p correlate_full $x
```

correlated_equilibrium_value

`correlated_equilibrium_value` — value of `correlated equilibrium`. Category: *climate, fluids, atmospheric*.

```
p correlated_equilibrium_value $x
```

correlation

`correlation` — operation `correlation`. Category: **extended stdlib**.

```
p correlation $x
```

cor_si_for

`cor_si_for` — operation `cor_si for`. Category: **astronomy / astrometry**.

```
p cor_si_for $x
```

cosh

`cosh` — operation `cosh`. Category: **trig / math**.

```
p cosh $x
```

cosh_minus1_over_x2

`cosh_minus1_over_x2` — operation `cosh minus1 over x2`. Category: **special functions extra**.

```
p cosh_minus1_over_x2 $x
```

cosine_integral_ci

`cosine_integral_ci` — operation `cosine integral ci`. Category: **special functions extra**.

```
p cosine_integral_ci $x
```

cosine_sim_sparse

`cosine_sim_sparse(a, b) → number` — cosine similarity for sparse vectors. Each input is `[[index, value], ...]`; dot product is taken over shared indices and L2 norms over each sparse set.

```
p cosine_sim_sparse([[0,1],[5,2]], [[0,1],[5,2]]) # 1
```

cosine_similarity

`cosine_similarity(a, b) → number` — cosine similarity of two same-length vectors.

```
p cosine_similarity([1,2,3], [2,4,6]) # 1.0
```

cosmic_distance_modulus_si

`cosmic_distance_modulus_si` — operation `cosmic distance modulus si`. Category: **cosmology / gr / flrw**.

```
p cosmic_distance_modulus_si $x
```

cosmological_constant

`cosmological_constant` — operation `cosmological constant`. Category: **cosmology / gr / flrw**.

```
p cosmological_constant $x
```


cosmological_constant_term

`cosmological_constant_term` — operation `cosmological_constant_term`. Category: **electrochemistry, batteries, fuel cells**.

```
p cosmological_constant_term $x
```

cot

`cot` returns the cotangent ($1/\tan$) of an angle in radians.

```
p cot(0.7854) # ≈ 1.0 (cot 45°)
```

coulomb

`coulomb_force($q1, $q2, $r)` (alias `coulomb`) — Coulomb's law: $F = kq_1q_2/r^2$. Charges in Coulombs, distance in meters.

```
p coulomb(1e-6, 1e-6, 0.1) # ~0.9 N (two 1μC charges 10cm apart)
```

coulomb_constant

`coulomb_constant` — operation `coulomb_constant`. Category: **physics constants**.

```
p coulomb_constant $x
```

coulomb_f0

`coulomb_f0` — operation `coulomb_f0`. Category: **special functions extra**.

```
p coulomb_f0 $x
```

coulombic_efficiency_cell

`coulombic_efficiency_cell` — operation `coulombic_efficiency_cell`. Category: **electrochemistry, batteries, fuel cells**.

```
p coulombic_efficiency_cell $x
```

count_by

`count_by` — operation `count_by`. Alias for `take_while`. Category: **functional / iterator**.

```
p count_by $x
```

count_bytes

`count_bytes` — operation `count_bytes`. Category: **extended stdlib**.

```
p count_bytes $x
```

count_char

`count_char` — operation `count_char`. Category: **string**.

```
p count_char $x
```

count_chars

count_chars — operation `count_chars`. Category: **extended stdlib**.

```
p count_chars $x
```

count_consonants

count_consonants — operation `count_consonants`. Category: **string**.

```
p count_consonants $x
```

count_digits

count_digits — operation `count_digits`. Category: **counters**.

```
p count_digits $x
```

count_elem

count_elem — operation `count_elem`. Category: **list helpers**.

```
p count_elem $x
```

count_eq

count_eq — operation `count_eq`. Category: **more list helpers**.

```
p count_eq $x
```

count_inversions

count_inversions — operation `count_inversions`. Category: **extended stdlib**.

```
p count_inversions $x
```

count_letters

count_letters — operation `count_letters`. Category: **counters**.

```
p count_letters $x
```

count_lines

count_lines — operation `count_lines`. Category: **extended stdlib**.

```
p count_lines $x
```

count_lower

count_lower — operation `count_lower`. Category: **counters**.

```
p count_lower $x
```

count_match

count_match — operation `count_match`. Category: **counters**.

```
p count_match $x
```

count_ne

`count_ne` — operation `count ne`. Category: **more list helpers**.

```
p count_ne $x
```

count_punctuation

`count_punctuation` — operation `count punctuation`. Category: **counters**.

```
p count_punctuation $x
```

count_regex_matches

`count_regex_matches` — operation `count regex matches`. Category: **file stat / path**.

```
p count_regex_matches $x
```

count_spaces

`count_spaces` — operation `count spaces`. Category: **counters**.

```
p count_spaces $x
```

count_substring

`count_substring` — operation `count substring`. Category: **file stat / path**.

```
p count_substring $x
```

count_upper

`count_upper` — operation `count upper`. Category: **counters**.

```
p count_upper $x
```

count_vowels

`count_vowels` — operation `count vowels`. Category: **string**.

```
p count_vowels $x
```

count_while

`count_while` — operation `count while`. Category: **ruby enumerable extras**.

```
p count_while $x
```

count_words

`count_words` — operation `count words`. Category: **extended stdlib**.

```
p count_words $x
```

counter

`counter` — operation `counter`. Category: **data structure helpers**.

```
p counter $x
```

counter_most_common

`counter_most_common` — operation `counter most common`. Category: **data structure helpers**.

```
p counter_most_common $x
```

countif

`countif` — operation `countif`. Category: **excel/sheets + bond/loan financial**.

```
p countif $x
```

countifs

`countifs` — operation `countifs`. Category: **excel/sheets + bond/loan financial**.

```
p countifs $x
```

country_code_alpha2

`country_code_alpha2(name_or_code)` → `string` — ISO 3166-1 alpha-2 code.

```
p country_code_alpha2("United States") # "US"
```

country_code_alpha3

`country_code_alpha3(name_or_code)` → `string` — ISO 3166-1 alpha-3 code.

country_code_numeric

`country_code_numeric(name_or_code)` → `string` — ISO 3166-1 numeric code (3 digits).

country_currency

`country_currency` — ISO 3166 country lookup `currency`. Category: **extras**.

```
p country_currency $x
```

country_languages

`country_languages(code)` → `array` — list of official language codes.

country_name

`country_name(code)` → `string` — country name for ISO code.

country_phone_prefix

`country_phone_prefix(code)` → `string` — international dialing prefix.

```
p country_phone_prefix("US") # "+1"
```

coupon_count

`coupon_count` — count of `coupon`. Category: **excel/sheets + bond/loan financial**.

```
p coupon_count $x
```

courant_friedrichs_lewy

`courant_friedrichs_lewy` — operation `courant_friedrichs_lewy`. Category: *climate, fluids, atmospheric*.

```
p courant_friedrichs_lewy $x
```

cournot_eq

`cournot_eq` — operation `cournot_eq`. Category: *economics + game theory*.

```
p cournot_eq $x
```

cournot_quantity_step

`cournot_quantity_step` — single-step update of `cournot_quantity`. Category: *climate, fluids, atmospheric*.

```
p cournot_quantity_step $x
```

cov

`cov` — operation `cov`. Alias for `covariance`. Category: *extended stdlib*.

```
p cov $x
```

cov2cor

`cov2cor` — convert covariance matrix to correlation matrix. Like R's `cov2cor()`.

```
val $cor = cov2cor(cov_mat($data))
```

cov_mat

`cov_matrix` (alias `cov_mat`) — covariance matrix from observations.

```
val $S = cov_mat([[1,2],[3,4]],[5,6]])
```

covar_pop

`covar_pop` — operation `covar_pop`. Category: *postgres sql strings, json, aggregates*.

```
p covar_pop $x
```

covar_samp

`covar_samp` — operation `covar_samp`. Category: *postgres sql strings, json, aggregates*.

```
p covar_samp $x
```

covariance

`covariance` — operation `covariance`. Category: *extended stdlib*.

```
p covariance $x
```

covariance_matrix_pts

`covariance_matrix_pts` — operation `covariance_matrix_pts`. Category: *geometry / topology*.

```
p covariance_matrix_pts $x
```

covariant_derivative_step

`covariant_derivative_step` — single-step update of `covariant derivative`. Category: **electrochemistry, batteries, fuel cells**.

```
p covariant_derivative_step $x
```

cp

Alias for `copy` — identical `FROM`, `TO`, and optional trailing hash (`preserve => 1`).

```
cp("a.txt", "b.txt")
```

cp_monatomic_ideal

`cp_monatomic_ideal` — operation `cp monatomic ideal`. Category: **chemistry**.

```
p cp_monatomic_ideal $x
```

cpat

`cpat` — operation `cpat`. Alias for `code_point_at`. Category: **extended stdlib**.

```
p cpat $x
```

cpfx

`cpfx` — operation `cpfx`. Alias for `common_prefix`. Category: **extended stdlib**.

```
p cpfx $x
```

cpl_w

`cpl_w` — operation `cpl w`. Category: **cosmology / gr / flrw**.

```
p cpl_w $x
```

cprod

`cprod` — operation `cprod`. Alias for `cartesian_product`. Category: **python/ruby stdlib**.

```
p cprod $x
```

cprod_acc

`cprod_acc` — operation `cprod acc`. Alias for `cumprod`. Category: **extended stdlib**.

```
p cprod_acc $x
```

cpsgn

`cpsgn` — operation `cpsgn`. Alias for `copy_sign`. Category: **extended stdlib**.

```
p cpsgn $x
```

cpu_count

`cpu_count` — count of `cpu`. Category: **more process / system**.

```
p cpu_count $x
```

cpyw

`cpyw` — operation `cpyw`. Alias for `copy_within`. Category: *additional missing stdlib functions*.

```
p cpyw $x
```

cr

`cr` — operation `cr`. Alias for `csv_read`. Category: *data / network*.

```
p cr $x
```

cramer_lundberg_step

`cramer_lundberg_step` — single-step update of `cramer lundberg`. Category: *actuarial science*.

```
p cramer_lundberg_step $x
```

cramer_rao_bound

`cramer_rao_bound` — operation `cramer rao bound`. Category: *quantum mechanics deep*.

```
p cramer_rao_bound $x
```

crank_nicolson_ode

`crank_nicolson_ode` — operation `crank nicolson ode`. Category: *more extensions (2)*.

```
p crank_nicolson_ode $x
```

crc10_atm

`crc10_atm(s) → int` — CRC-10/ATM AAL5 checksum.

crc12_dect

`crc12_dect(s) → int` — CRC-12/DECT (poly 0x80F).

crc16

`crc16(s) → int` — CRC-16/Modbus (poly 0xA001, init 0xFFFF, reflected). Verifies serial-protocol frames.

```
p crc16("123456789") # 0x4B37
```

crc16_ccitt

`crc16_ccitt` — CRC checksum variant 16 `ccitt`. Category: *misc*.

```
p crc16_ccitt $x
```

crc16_xmodem

`crc16_xmodem(s) → int` — CRC-16/XMODEM (poly 0x1021, init 0). Used in XMODEM file transfer.

```
p crc16_xmodem("123456789") # 0x31C3
```

crc24

`crc24(s) → int` — CRC-24/OpenPGP (poly 0x864CFB, init 0xB704CE).

```
p crc24("") # 0xB704CE
```

crc32_bzip2

`crc32_bzip2(s) → int` — CRC-32/BZIP2 (non-reflected).

crc32_jamcrc

`crc32_jamcrc(s) → int` — CRC-32/JAMCRC (reflected, xorout 0).

crc32_mpeg2

`crc32_mpeg2(s) → int` — CRC-32/MPEG-2 (non-reflected, xorout 0).

crc32_xfer

`crc32_xfer(s) → int` — CRC-32/XFER (XFER protocol).

crc32_zlib

`crc32_zlib(s) → int` — CRC-32/ISO-HDLC (zlib/gzip standard).

```
p crc32_zlib("123456789") # 0xCB43926
```

crc32c

`crc32c(s) → int` — CRC-32C (Castagnoli, poly 0x82F63B78). Used by iSCSI/Btrfs/SCTP.

```
p crc32c("123456789")
```

crc64_ecma

`crc64_ecma(s) → int` — CRC-64/ECMA-182 (poly 0x42F0E1EBA9EA3693).

```
p crc64_ecma("abc")
```

crc64_xz

`crc64_xz(s) → int` — CRC-64/XZ (reflected). Used in xz/lzma file integrity.

crc6_itu

`crc6_itu(s) → int` — CRC-6/ITU (poly 0x03, reflected).

crc8

`crc8(s) → int` — CRC-8 (polynomial 0x07, init 0). Common for sensor checksums.

```
p crc8("123456789")
```


credibility_buhlmann

`credibility_buhlmann` — operation `credibility_buhlmann`. Category: *actuarial science*.

```
p credibility_buhlmann $x
```

credible_interval_beta

`credible_interval_beta(alpha, beta, level=0.95) → [lo, hi]` — central credible interval from a Beta posterior.

credible_interval_normal

`credible_interval_normal(mean, sd, level=0.95) → [lo, hi]` — central credible interval from a Normal posterior.

crf_log_likelihood

`crf_log_likelihood` — operation `crf_log_likelihood`. Category: *computational linguistics*.

```
p crf_log_likelihood $x
```

crindex

`crindex STRING, SUBSTRING [, FROM]` — codepoint-indexed equivalent of Perl 5's `rindex`. Returns the last codepoint position where SUBSTRING occurs (at or before FROM, defaulting to end-of-string), or -1 on miss. Counterpart to `cindex`; pairs with `[]` slice operators and `len`.

```
val $path = "foo/bar-/baz.gz"
p rindex $path, "/"           # 13 (bytes)
p crindex $path, "/"          # 11 (codepoints)
var $i = crindex $path, "/"
p $path[$i + 1 : len($path) - 1] # "baz.gz"
```

critang

`critang` — operation `critang`. Alias for `critical_angle`. Category: *physics formulas*.

```
p critang $x
```

critical_angle

`critical_angle` — operation `critical_angle`. Category: *physics formulas*.

```
p critical_angle $x
```

critical_damping

`critical_damping(m, k) → number` — critical damping coefficient $c_c = 2 \cdot \sqrt{m \cdot k}$.

critical_density

`critical_density` — operation `critical_density`. Category: *astronomy / music / color / units*.

```
p critical_density $x
```

critical_density_si

`critical_density_si` — operation `critical_density_si`. Category: *cosmology / gr / flrw*.

```
p critical_density_si $x
```

cron_next_fire

`cron_next_fire` — operation `cron next fire`. Category: **b82-misc-utility**.

```
p cron_next_fire $x
```

crop_factor

`crop_factor(sensor_diag_mm)` → number — $43.27 / \text{sensor_diag}$. Full-frame = 1.

cross_correlation

`cross_correlation(\@a, \@b)` (alias `xcorr`) — computes the cross-correlation of two signals. Measures similarity as a function of time-lag. Peak location indicates time delay between signals.

```
val $xcor = xcorr(\@signal1, \@signal2)
var $lag = max_index(@$xcor) # find peak lag
```

cross_elasticity

`cross_elasticity` — operation `cross elasticity`. Category: **economics + game theory**.

```
p cross_elasticity $x
```

cross_entropy

`cross_entropy(p, q)` → number — $-\sum p \cdot \log q$ (natural log, alias `xentropy`).

cross_polarization_amp

`cross_polarization_amp` — operation `cross polarization amp`. Category: **electrochemistry, batteries, fuel cells**.

```
p cross_polarization_amp $x
```

cross_product

`cross_product` — operation `cross product`. Category: **extended stdlib**.

```
p cross_product $x
```

crossfade

`crossfade` — operation `crossfade`. Category: **extras**.

```
p crossfade $x
```

crossp

`crossp` — operation `crossp`. Alias for `cross_product`. Category: **extended stdlib**.

```
p crossp $x
```

crossprod

`crossprod` — cross product $t(M) * M$ (R's `crossprod`). Efficiently computes $M^T M$ without explicit transpose.

```
val $ata = crossprod([[1,2],[3,4]]) # [[10,14],[14,20]]
```

crr_american_call

`crr_american_call` — operation `crr american call`. Category: *financial pricing models*.

```
p crr_american_call $x
```

crr_american_put

`crr_american_put` — operation `crr american put`. Category: *financial pricing models*.

```
p crr_american_put $x
```

crustal_density_depth

`crustal_density_depth` — operation `crustal density depth`. Category: *geology, seismology, mineralogy*.

```
p crustal_density_depth $x
```

crystal_field_ligand

`crystal_field_ligand` — operation `crystal field ligand`. Category: *chemistry & biochemistry*.

```
p crystal_field_ligand $x
```

csc

`csc` returns the cosecant ($1/\sin$) of an angle in radians.

```
p csc(1.5708) # ≈ 1.0 (csc 90°)
```

csd_xy

`csd_xy` — operation `csd xy`. Category: *economics + game theory*.

```
p csd_xy $x
```

csfx

`csfx` — operation `csfx`. Alias for `common_suffix`. Category: *extended stdlib*.

```
p csfx $x
```

csiszar_phi_div

`csiszar_phi_div` — operation `csiszar phi div`. Category: *electrochemistry, batteries, fuel cells*.

```
p csiszar_phi_div $x
```

cspline

`cubic_spline` (aliases `cspline`, `spline`) performs natural cubic spline interpolation. Takes `xs`, `ys`, and a query point `x`.

```
p spline([0,1,2,3], [0,1,0,1], 1.5) # smooth interpolation
```

css_font_extract

`css_font_extract` — operation `css font extract`. Category: *extras*.

```
p css_font_extract $x
```

css_import_extract

css_import_extract — operation `css import extract`. Category: *extras*.

```
p css_import_extract $x
```

css_minify

css_minify — operation `css minify`. Category: *extras*.

```
p css_minify $x
```

css_parse

css_parse — operation `css parse`. Category: *extras*.

```
p css_parse $x
```

css_pretty

css_pretty — operation `css pretty`. Category: *extras*.

```
p css_pretty $x
```

css_property_get

css_property_get — operation `css property get`. Category: *extras*.

```
p css_property_get $x
```

css_property_set

css_property_set — operation `css property set`. Category: *extras*.

```
p css_property_set $x
```

css_rule_extract

css_rule_extract — operation `css rule extract`. Category: *extras*.

```
p css_rule_extract $x
```

css_selector_parse

css_selector_parse — operation `css selector parse`. Category: *extras*.

```
p css_selector_parse $x
```

css_specificity

css_specificity — operation `css specificity`. Category: *extras*.

```
p css_specificity $x
```

css_url_extract

css_url_extract — operation `css url extract`. Category: *extras*.

```
p css_url_extract $x
```

css_var_resolve

`css_var_resolve` — operation `css var resolve`. Category: *extras*.

```
p css_var_resolve $x
```

csum

`csum` — operation `csum`. Alias for `cumsum`. Category: *extended stdlib*.

```
p csum $x
```

cswap_gate

`cswap_gate` — operation `cswap gate`. Alias for `fredkin_gate`. Category: *more extensions*.

```
p cswap_gate $x
```

ctime

`ctime` — operation `ctime`. Alias for `file_ctime`. Category: *file stat / path*.

```
p ctime $x
```

ctxt

`ctxt` — operation `ctxt`. Alias for `center_text`. Category: *extended stdlib*.

```
p ctxt $x
```

cuban_number

`cuban_number` — operation `cuban number`. Category: *misc*.

```
p cuban_number $x
```

cube_fn

`cube_fn` — operation `cube fn`. Category: *trig / math*.

```
p cube_fn $x
```

cube_number

`cube_number(n)` $\rightarrow$ `int` — n^3 .

cube_root

`cube_root` — operation `cube root`. Category: *math functions*.

```
p cube_root $x
```

cubes_seq

`cubes_seq` — operation `cubes seq`. Category: *sequences*.

```
p cubes_seq $x
```

cullen_number

`cullen_number` — operation `cullen_number`. Category: **misc**.

```
p cullen_number $x
```

cumipmt

`cumipmt` — operation `cumipmt`. Category: **b81-misc-utility**.

```
p cumipmt $x
```

cummax

`cummax` — cumulative maximum. Each element is the max of all elements up to that position.

```
p cummax([3,1,4,1,5,9]) # [3,3,4,4,5,9]
```

cummin

`cummin` — cumulative minimum.

```
p cummin([9,5,1,4,3]) # [9,5,1,1,1]
```

cumprinc

`cumprinc` — operation `cumprinc`. Category: **b81-misc-utility**.

```
p cumprinc $x
```

cumprod

`cumprod` — operation `cumprod`. Category: **extended stdlib**.

```
p cumprod $x
```

cumsum

`cumsum` — operation `cumsum`. Category: **extended stdlib**.

```
p cumsum $x
```

cumtrapz

`cumtrapz` cumulative trapezoidal integration. Returns running integral array.

```
val @y = (1, 2, 3, 4)
var @F = @{cumtrapz(\@y)} # [0, 1.5, 4.0, 7.5]
```

cups_to_ml

`cups_to_ml` — convert from `cups` to `ml`. Category: **file stat / path**.

```
p cups_to_ml $value
```

curl_get

`curl_get(URL)` → response body as a string, or `undef` on failure. Shell-named wrapper over the existing `ureq` HTTP client. For full HTTP semantics (custom headers, redirects, JSON parsing), use the lower-level `fetch_val` / `fetch_async` / `http_*` family.

```
val $page = curl_get("https://example.com")
```

`curl_post`

`curl_post(URL, BODY)` → POST `BODY` to `URL`. `Content-Type` auto-detected: `application/json` when `BODY` parses as JSON, else `text/plain`. Returns response body as a string, or `undef` on failure.

`currency_code_to_symbol`

`currency_code_to_symbol(code)` → `string` — symbol like \$/€/¥ for ISO code.

`currency_convert`

`currency_convert` — currency / money helper `convert`. Category: `*extras*`.

```
p currency_convert $x
```

`currency_decimal_places`

`currency_decimal_places(code)` → `int` — minor-unit digits (2 for USD, 0 for JPY).

`currency_format`

`currency_format(amount, code)` → `string` — locale-aware money formatting.

`currency_iso_4217`

`currency_iso_4217(name_or_symbol)` → `string` — ISO 4217 alphabetic currency code.

```
p currency_iso_4217("dollar") # "USD"
```

`currency_parse`

`currency_parse(s)` → `{amount, code}` — parse formatted money string.

`currency_rate`

`currency_rate` — currency / money helper `rate`. Category: `*extras*`.

```
p currency_rate $x
```

`currency_round`

`currency_round(amount, code)` → `number` — round to currency precision.

`currency_split_thousands`

`currency_split_thousands(amount, sep=",")` → `string` — comma-separated thousands grouping.

`currency_symbol_to_code`

`currency_symbol_to_code(symbol)` → `string` — inverse of `currency_code_to_symbol`.

current_divider

`current_divider` — operation `current divider`. Category: **misc**.

```
p current_divider $x
```

curve25519_mul_simple

`curve25519_mul_simple` — operation `curve25519 mul simple`. Category: **more extensions (2)**.

```
p curve25519_mul_simple $x
```

cusip_check

`cusip_check` — operation `cusip check`. Category: **b82-misc-utility**.

```
p cusip_check $x
```

cut

`cut` — bin continuous values into intervals. Returns integer bin indices. Like R's `cut()`.

```
p cut([1.5, 3.2, 7.8], [0, 2, 5, 10]) # [1, 2, 3]
```

cutree

`cutree` — cut a dendrogram into k clusters. Takes merge list from `hclust` and k. Returns cluster assignments. Like R's `cutree()`.

```
val @clusters = @{cutree(hclust(dist_mat($data)), 3)}
```

cutting_plane_step

`cutting_plane_step` — single-step update of `cutting plane`. Category: **combinatorial optimization, scheduling**.

```
p cutting_plane_step $x
```

cv

`cv` — operation `cv`. Alias for `coeff_of_variation`. Category: **math / numeric extras**.

```
p cv $x
```

cv_monatomic_ideal

`cv_monatomic_ideal` — operation `cv monatomic ideal`. Category: **chemistry**.

```
p cv_monatomic_ideal $x
```

cvar_historical

`cvar_historical` — operation `cvar historical`. Category: **more extensions (2)**.

```
p cvar_historical $x
```

cw

`cw` — operation `cw`. Alias for `csv_write`. Category: **data / network**.


```
p cw $x
```

cwd

cwd — operation **cwd**. Category: *more process / system*.

```
p cwd $x
```

cwt_morlet

cwt_morlet — operation **cwt morlet**. Category: *economics + game theory*.

```
p cwt_morlet $x
```

cyan

cyan — operation **cyan**. Alias for **red**. Category: *color / ansi*.

```
p cyan $x
```

cyan_bold

cyan_bold — operation **cyan bold**. Alias for **red**. Category: *color / ansi*.

```
p cyan_bold $x
```

cyber_banner

cyber_banner (alias **neon_banner**) — large neon block-letter banner with gradient coloring and border. Args: text.

```
p cyber_banner("STRYKE")
p cyber_banner("HACK THE PLANET")
```

cyber_circuit

cyber_circuit — circuit board pattern with traces, intersections, and glowing nodes. Args: [width, height, seed].

```
p cyber_circuit()           # 60x20 default
p cyber_circuit(80, 30, 42)  # custom
```

cyber_city

cyber_city — procedural neon cityscape with buildings, windows, stars, and antennas. Args: [width, height, seed]. Output: ANSI-colored string for terminal.

```
p cyber_city()           # 80x24 default
p cyber_city(120, 40, 99) # custom size and seed
```

cyber_eye

cyber_eye — cyberpunk all-seeing eye motif with layered glow. Args: [size]. Size: "small" (default) or "large".

```
p cyber_eye()           # small eye
p cyber_eye("large")     # large eye
```

cyber_glitch

cyber_glitch (alias **glitch_text**) — glitch-distort text with ANSI corruption, screen tears, and neon color bleeding. Args: text [, intensity 1-10].

```
p cyber_glitch("SYSTEM BREACH", 7)
p cyber_glitch("hello world")
```

cyber_grid

cyber_grid — retro perspective grid with vanishing point and neon glow (Tron/synthwave style). Args: [width, height].

```
p cyber_grid()           # 80x24 default
p cyber_grid(120, 30)    # wider grid
```

cyber_rain

cyber_rain (alias **matrix_rain**) — matrix-style digital rain with Japanese katakana and green phosphor glow. Args: [width, height, seed].

```
p cyber_rain()           # 80x24 default
p cyber_rain(120, 40, 42) # custom
```

cyber_skull

cyber_skull — neon skull ASCII art with glitch effects. Args: [size]. Size: "small" (default) or "large".

```
p cyber_skull()          # small skull
p cyber_skull("large")    # large skull
```

cyc

cyc — operation **cyc**. Alias for **cycle**. Category: *algebraic match*.

```
p cyc $x
```

cycle

cycle — operation **cycle**. Category: *algebraic match*.

```
p cycle $x
```

cycle_n

cycle_n — operation **cycle** **n**. Alias for **repeat_list**. Category: *collection*.

```
p cycle_n $x
```

cyclic_homology_step

cyclic_homology_step — single-step update of **cyclic homology**. Category: *econometrics*.

```
p cyclic_homology_step $x
```

cylinder_surface

cylinder_surface — operation **cylinder** **surface**. Category: *geometry / topology*.

```
p cylinder_surface $x
```

cylinder_surface_area

`cylinder_surface_area` — operation `cylinder surface area`. Category: **complex / geom / color / trig**.

```
p cylinder_surface_area $x
```

cylinder_volume

`cylinder_volume` — operation `cylinder volume`. Category: **geometry / physics**.

```
p cylinder_volume $x
```

cylindrical_to_cartesian

`cylindrical_to_cartesian` — convert from `cylindrical` to `cartesian`. Category: **complex / geom / color / trig**.

```
p cylindrical_to_cartesian $value
```

cylvol

`cylvol` — operation `cylvol`. Alias for `cylinder_volume`. Category: **geometry / physics**.

```
p cylvol $x
```

d

`d` — operation `d`. Category: **filesystem extensions**.

```
p d $x
```

d2comp

`d2comp` — operation `d2comp`. Alias for `degrees_to_compass`. Category: **misc / utility**.

```
p d2comp $x
```

d2r

`d2r` — operation `d2r`. Alias for `deg_to_rad`. Category: **trivial numeric / predicate builtins**.

```
p d2r $x
```

dB_power

`dB_power` — operation `dB power`. Category: **misc**.

```
p dB_power $x
```

dB_voltage

`dB_voltage` — operation `dB voltage`. Category: **misc**.

```
p dB_voltage $x
```

dagum_pdf

`dagum_pdf` — operation `dagum pdf`. Category: **more extensions (2)**.

```
p dagum_pdf $x
```

daitch_mokotoff

`daitch_mokotoff` — operation `daitch mokotoff`. Category: *computational linguistics*.

```
p daitch_mokotoff $x
```

dalys_disability_weight

`dalys_disability_weight` — operation `dalys disability weight`. Category: *epidemiology / public health*.

```
p dalys_disability_weight $x
```

damerau_levenshtein

`damerau_levenshtein` — operation `damerau levenshtein`. Category: *bioinformatics deep*.

```
p damerau_levenshtein $x
```

damping_factor

`damping_factor(c, m, k) → number` — damping ratio $\zeta = c / (2 \cdot \sqrt{m \cdot k})$.

dantzig_wolfe_step

`dantzig_wolfe_step` — single-step update of `dantzig wolfe`. Category: *combinatorial optimization, scheduling*.

```
p dantzig_wolfe_step $x
```

darboux_frame_step

`darboux_frame_step` — single-step update of `darboux frame`. Category: *electrochemistry, batteries, fuel cells*.

```
p darboux_frame_step $x
```

dasum

`dasum` — operation `dasum`. Category: *blas / lapack*.

```
p dasum $x
```

data_processing_inequality

`data_processing_inequality` — operation `data processing inequality`. Category: *electrochemistry, batteries, fuel cells*.

```
p data_processing_inequality $x
```

date_add_days

`date_add_days(unix_or_iso, n) → int` — returns unix timestamp `n` days later.

```
p date_add_days("2024-01-01", 30)
```

date_add_months

`date_add_months(unix_or_iso, n) → int` — adds whole calendar months; clamps the day to the last valid day of the target month (so Jan 31 + 1mo → Feb 28/29, May 31 + 1mo → Jun 30).

```
p date_add_months("2024-01-31", 1)
```

date_add_years

`date_add_years(unix_or_iso, n) → int` — adds whole calendar years; clamps Feb 29 to Feb 28 in non-leap target years; otherwise preserves the original day.

```
p date_add_years("2024-02-29", 1)
```

date_business_days_between

`date_business_days_between(d1, d2) → int` — weekday count (Mon-Fri) strictly between two dates.

```
p date_business_days_between("2024-01-01", "2024-01-15")
```

date_day

`date_day(unix_or_iso) → int` — day of month 1-31.

date_dayofweek

`date_dayofweek(unix_or_iso) → int` — day of week 0-6, Sunday=0.

```
p date_dayofweek("2024-03-15") # 5 (Friday)
```

date_dayofyear

`date_dayofyear(unix_or_iso) → int` — ordinal day 1-366.

```
p date_dayofyear("2024-12-31") # 366
```

date_days_in_month

`date_days_in_month(year, month) → int` — days in given month accounting for leap years.

```
p date_days_in_month(2024, 2) # 29
```

date_diff_days

`date_diff_days(d1, d2) → int` — $d2 - d1$ in whole days.

```
p date_diff_days("2024-01-01", "2024-12-31")
```

date_diff_hours

`date_diff_hours(d1, d2) → int` — hours between two timestamps.

date_diff_minutes

`date_diff_minutes(d1, d2) → int` — minutes between two timestamps.

date_diff_seconds

`date_diff_seconds(d1, d2) → int` — seconds between two timestamps.

date_easter

`date_easter(year) → string` — Western Christian Easter Sunday date (Anonymous Gregorian algorithm).

```
p date_easter(2024) # "2024-03-31"
```

date_first_of_month

`date_first_of_month(unix_or_iso) → int` — unix timestamp of the first day of the same month at 00:00 UTC.

```
p date_first_of_month("2024-03-15")
```

date_hour

`date_hour(unix_or_iso) → int` — hour 0-23 (UTC).

date_is_leap

`date_is_leap(year) → 0|1` — Gregorian leap-year rule: divisible by 4, except by 100, except by 400.

```
p date_is_leap(2000) # 1
p date_is_leap(2100) # 0
```

date_is_weekend

`date_is_weekend(unix_or_iso) → 0|1` — 1 if Saturday or Sunday.

```
p date_is_weekend("2024-03-16") # 1
```

date_iso_format

`date_iso_format(unix_or_iso) → string` — RFC3339 representation.

```
p date_iso_format(0) # "1970-01-01T00:00:00+00:00"
```

date_iso_week

`date_iso_week(unix_or_iso) → string` — ISO 8601 week format `YYYY-Www`.

```
p date_iso_week("2024-03-15") # "2024-W11"
```

date_last_of_month

`date_last_of_month(unix_or_iso) → int` — unix timestamp of the last day of the same month.

```
p date_last_of_month("2024-02-15") # 2024-02-29
```

date_minute

`date_minute(unix_or_iso) → int` — minute 0-59.

date_month

`date_month(unix_or_iso) → int` — month 1-12.

date_quarter

`date_quarter(unix_or_iso)` → `int` — calendar quarter 1-4.

```
p date_quarter("2024-04-15") # 2
```

date_round_iso

`date_round_iso` — date / time helper `round iso`. Category: `*b82-misc-utility*`.

```
p date_round_iso $x
```

date_second

`date_second(unix_or_iso)` → `int` — second 0-59.

date_str_to_unix

`date_str_to_unix(s, fmt="%Y-%m-%d %H:%M:%S")` → `int` — parse a date string back to unix timestamp.

```
p date_str_to_unix("2024-01-01", "%Y-%m-%d")
```

date_unix_to_str

`date_unix_to_str(unix, fmt="%Y-%m-%d %H:%M:%S")` → `string` — strftime-format a unix timestamp.

```
p date_unix_to_str(1672531200, "%Y-%m-%d")
```

date_weekofyear

`date_weekofyear(unix_or_iso)` → `int` — ISO 8601 week number 1-53.

```
p date_weekofyear("2024-01-01")
```

date_year

`date_year(unix_or_iso)` → `int` — calendar year of a UTC date. Accepts unix timestamp or ISO string.

```
p date_year("2024-03-15") # 2024
```

dawsn

`dawsn` — operation `dawsn`. Category: `*numpy + scipy.special*`.

```
p dawsn $x
```

dawson_function

`dawson_function` — operation `dawson function`. Category: `*special functions extra*`.

```
p dawson_function $x
```

daxpy

`daxpy` — operation `daxpy`. Category: `*blas / lapack*`.

```
p daxpy $x
```

day_of_week_iso

day_of_week_iso — operation `day of week iso`. Category: **misc**.

```
p day_of_week_iso $x
```

day_of_week_zeller

day_of_week_zeller — operation `day of week zeller`. Category: **calendrical algorithms**.

```
p day_of_week_zeller $x
```

day_of_year

day_of_year — operation `day of year`. Category: **extended stdlib**.

```
p day_of_year $x
```

days_between

days_between — operation `days between`. Category: **misc**.

```
p days_between $x
```

days_in_month

days_in_month — operation `days in month`. Category: **date helpers**.

```
p days_in_month $x
```

days_in_month_fn

days_in_month_fn — operation `days in month fn`. Category: **extended stdlib**.

```
p days_in_month_fn $x
```

days_in_year

days_in_year — operation `days in year`. Category: **misc**.

```
p days_in_year $x
```

days_to_hours

days_to_hours — convert from `days` to `hours`. Category: **unit conversions**.

```
p days_to_hours $value
```

days_to_seconds

days_to_seconds — convert from `days` to `seconds`. Category: **unit conversions**.

```
p days_to_seconds $value
```

daysinmo

daysinmo — operation `daysinmo`. Alias for `days_in_month_fn`. Category: **extended stdlib**.

```
p daysinmo $x
```


db

`db` — operation `db`. Alias for `decibel_ratio`. Category: *physics formulas*.

```
p db $x
```

db4_coeffs

`db4_coeffs` — operation `db4 coeffs`. Category: *more extensions*.

```
p db4_coeffs $x
```

db4_wavelet_step

`db4_wavelet_step` — single-step update of `db4 wavelet`. Category: *electrochemistry, batteries, fuel cells*.

```
p db4_wavelet_step $x
```

db6_coeffs

`db6_coeffs` — operation `db6 coeffs`. Category: *more extensions*.

```
p db6_coeffs $x
```

db_anti_entropy_pull_step

`db_anti_entropy_pull_step` — single-step update of `db anti entropy pull`. Category: *database internals, distributed systems*.

```
p db_anti_entropy_pull_step $x
```

db_anti_entropy_step

`db_anti_entropy_step` — single-step update of `db anti entropy`. Category: *database internals, distributed systems*.

```
p db_anti_entropy_step $x
```

db_arc_cache_score

`db_arc_cache_score` — score of `db arc cache`. Category: *database internals, distributed systems*.

```
p db_arc_cache_score $x
```

db_async_commit_check

`db_async_commit_check` — operation `db async commit check`. Category: *database internals, distributed systems*.

```
p db_async_commit_check $x
```

db_avalanche_query_step

`db_avalanche_query_step` — single-step update of `db avalanche query`. Category: *database internals, distributed systems*.

```
p db_avalanche_query_step $x
```

db_b_tree_merge

db_b_tree_merge — operation `db b tree merge`. Category: *database internals, distributed systems*.

```
p db_b_tree_merge $x
```

db_b_tree_split

db_b_tree_split — operation `db b tree split`. Category: *database internals, distributed systems*.

```
p db_b_tree_split $x
```

db_block_cache_hit_rate

db_block_cache_hit_rate — rate of `db block cache hit`. Category: *database internals, distributed systems*.

```
p db_block_cache_hit_rate $x
```

db_bloom_filter_bit_index

db_bloom_filter_bit_index — index of `db bloom filter bit`. Category: *database internals, distributed systems*.

```
p db_bloom_filter_bit_index $x
```

db_buffer_pool_score

db_buffer_pool_score — score of `db buffer pool`. Category: *database internals, distributed systems*.

```
p db_buffer_pool_score $x
```

db_byzantine_quorum_size

db_byzantine_quorum_size — operation `db byzantine quorum size`. Category: *database internals, distributed systems*.

```
p db_byzantine_quorum_size $x
```

db_causal_consistency_check

db_causal_consistency_check — operation `db causal consistency check`. Category: *database internals, distributed systems*.

```
p db_causal_consistency_check $x
```

db_chao_estimator_step

db_chao_estimator_step — single-step update of `db chao estimator`. Category: *database internals, distributed systems*.

```
p db_chao_estimator_step $x
```

db_chord_finger_index

db_chord_finger_index — index of `db chord finger`. Category: *database internals, distributed systems*.

```
p db_chord_finger_index $x
```

db_chubby_lease_step

`db_chubby_lease_step` — single-step update of `db_chubby_lease`. Category: *database internals, distributed systems*.

```
p db_chubby_lease_step $x
```

db_circuit_breaker_step

`db_circuit_breaker_step` — single-step update of `db_circuit_breaker`. Category: *database internals, distributed systems*.

```
p db_circuit_breaker_step $x
```

db_clock_cache_hand

`db_clock_cache_hand` — operation `db_clock_cache_hand`. Category: *database internals, distributed systems*.

```
p db_clock_cache_hand $x
```

db_clock_skew_estimate

`db_clock_skew_estimate` — operation `db_clock_skew_estimate`. Category: *database internals, distributed systems*.

```
p db_clock_skew_estimate $x
```

db_compaction_score

`db_compaction_score` — score of `db_compaction`. Category: *database internals, distributed systems*.

```
p db_compaction_score $x
```

db_consensus_quorum_size

`db_consensus_quorum_size` — operation `db_consensus_quorum_size`. Category: *database internals, distributed systems*.

```
p db_consensus_quorum_size $x
```

db_consistent_hash_index

`db_consistent_hash_index` — index of `db_consistent_hash`. Category: *database internals, distributed systems*.

```
p db_consistent_hash_index $x
```

db_consistent_lookup_id

`db_consistent_lookup_id` — operation `db_consistent_lookup_id`. Category: *database internals, distributed systems*.

```
p db_consistent_lookup_id $x
```

db_count_min_sketch_bin

`db_count_min_sketch_bin` — operation `db_count_min_sketch_bin`. Category: *database internals, distributed systems*.

```
p db_count_min_sketch_bin $x
```

db_crdt_g_counter_merge

`db_crdt_g_counter_merge` — operation `db crdt g counter merge`. Category: *database internals, distributed systems*.

```
p db_crdt_g_counter_merge $x
```

db_crdt_lww_register_merge

`db_crdt_lww_register_merge` — operation `db crdt lww register merge`. Category: *database internals, distributed systems*.

```
p db_crdt_lww_register_merge $x
```

db_crdt_pn_counter_merge

`db_crdt_pn_counter_merge` — operation `db crdt pn counter merge`. Category: *database internals, distributed systems*.

```
p db_crdt_pn_counter_merge $x
```

db_crdt_set_or_merge

`db_crdt_set_or_merge` — operation `db crdt set or merge`. Category: *database internals, distributed systems*.

```
p db_crdt_set_or_merge $x
```

db_cuckoo_filter_fingerprint

`db_cuckoo_filter_fingerprint` — operation `db cuckoo filter fingerprint`. Category: *database internals, distributed systems*.

```
p db_cuckoo_filter_fingerprint $x
```

db_dd_sketch_bin

`db_dd_sketch_bin` — operation `db dd sketch bin`. Category: *database internals, distributed systems*.

```
p db_dd_sketch_bin $x
```

db_dht_replicate_factor

`db_dht_replicate_factor` — factor of `db dht replicate`. Category: *database internals, distributed systems*.

```
p db_dht_replicate_factor $x
```

db_distinct_estimate_hll

`db_distinct_estimate_hll` — operation `db distinct estimate hll`. Category: *database internals, distributed systems*.

```
p db_distinct_estimate_hll $x
```

db_distinct_estimate_lpc

`db_distinct_estimate_lpc` — operation `db distinct estimate lpc`. Category: *database internals, distributed systems*.

```
p db_distinct_estimate_lpc $x
```

db_eventual_consistency_check

`db_eventual_consistency_check` — operation `db eventual consistency check`. Category: *database internals, distributed systems*.

```
p db_eventual_consistency_check $x
```

db_freshness_score

`db_freshness_score` — score of `db freshness`. Category: *database internals, distributed systems*.

```
p db_freshness_score $x
```

db_gossip_fanout_step

`db_gossip_fanout_step` — single-step update of `db gossip fanout`. Category: *database internals, distributed systems*.

```
p db_gossip_fanout_step $x
```

db_group_commit_count

`db_group_commit_count` — count of `db group commit`. Category: *database internals, distributed systems*.

```
p db_group_commit_count $x
```

db_hash_join_cost

`db_hash_join_cost` — operation `db hash join cost`. Category: *database internals, distributed systems*.

```
p db_hash_join_cost $x
```

db_hinted_handoff_step

`db_hinted_handoff_step` — single-step update of `db hinted handoff`. Category: *database internals, distributed systems*.

```
p db_hinted_handoff_step $x
```

db_histogram_bucket_index

`db_histogram_bucket_index` — index of `db histogram bucket`. Category: *database internals, distributed systems*.

```
p db_histogram_bucket_index $x
```

db_honey_badger_step

`db_honey_badger_step` — single-step update of `db honey badger`. Category: *database internals, distributed systems*.

```
p db_honey_badger_step $x
```

db_hybrid_logical_clock

`db_hybrid_logical_clock` — operation `db hybrid logical clock`. Category: *database internals, distributed systems*.

```
p db_hybrid_logical_clock $x
```

db_hyperloglog_register_max

`db_hyperloglog_register_max` — maximum of `db hyperloglog register`. Category: *database internals, distributed systems*.

```
p db_hyperloglog_register_max $x
```

db_index_scan_cost

`db_index_scan_cost` — operation `db index scan cost`. Category: *database internals, distributed systems*.

```
p db_index_scan_cost $x
```

db_index_seek_cost

`db_index_seek_cost` — operation `db index seek cost`. Category: *database internals, distributed systems*.

```
p db_index_seek_cost $x
```

db_jaccard_minhash_estimate

`db_jaccard_minhash_estimate` — operation `db jaccard minhash estimate`. Category: *database internals, distributed systems*.

```
p db_jaccard_minhash_estimate $x
```

db_join_selectivity_step

`db_join_selectivity_step` — single-step update of `db join selectivity`. Category: *database internals, distributed systems*.

```
p db_join_selectivity_step $x
```

db_jump_hash_bucket

`db_jump_hash_bucket` — operation `db jump hash bucket`. Category: *database internals, distributed systems*.

```
p db_jump_hash_bucket $x
```

db_kademlia_xor_distance

`db_kademlia_xor_distance` — distance / dissimilarity metric of `db kademlia xor`. Category: *database internals, distributed systems*.

```
p db_kademlia_xor_distance $x
```

db_kll_quantile_step

`db_kll_quantile_step` — single-step update of `db kll quantile`. Category: *database internals, distributed systems*.

```
p db_kll_quantile_step $x
```

db_lamport_timestamp

`db_lamport_timestamp` — operation `db lamport timestamp`. Category: *database internals, distributed systems*.

```
p db_lamport_timestamp $x
```

db_leaky_bucket_step

`db_leaky_bucket_step` — single-step update of `db leaky bucket`. Category: *database internals, distributed systems*.

```
p db_leaky_bucket_step $x
```

db_levelled_compaction_step

`db_levelled_compaction_step` — single-step update of `db levelled compaction`. Category: *database internals, distributed systems*.

```
p db_levelled_compaction_step $x
```

db_lfu_cache_decay

`db_lfu_cache_decay` — operation `db lfu cache decay`. Category: *database internals, distributed systems*.

```
p db_lfu_cache_decay $x
```

db_linearizability_check

`db_linearizability_check` — operation `db linearizability check`. Category: *database internals, distributed systems*.

```
p db_linearizability_check $x
```

db_logical_clock_step

`db_logical_clock_step` — single-step update of `db logical clock`. Category: *database internals, distributed systems*.

```
p db_logical_clock_step $x
```

db_lru_cache_eviction_age

`db_lru_cache_eviction_age` — operation `db lru cache eviction age`. Category: *database internals, distributed systems*.

```
p db_lru_cache_eviction_age $x
```

db_lsm_compaction_step

`db_lsm_compaction_step` — single-step update of `db lsm compaction`. Category: *database internals, distributed systems*.

```
p db_lsm_compaction_step $x
```

db_maglev_hash_step

`db_maglev_hash_step` — single-step update of `db maglev hash`. Category: *database internals, distributed systems*.

```
p db_maglev_hash_step $x
```

db_merge_join_cost

`db_merge_join_cost` — operation `db merge join cost`. Category: *database internals, distributed systems*.

```
p db_merge_join_cost $x
```

db_merkle_node_hash

db_merkle_node_hash — hash function of db merkle node. Category: *database internals, distributed systems*.

```
p db_merkle_node_hash $x
```

db_merkle_path_verify

db_merkle_path_verify — verify with public key of db merkle path. Category: *database internals, distributed systems*.

```
p db_merkle_path_verify $x
```

db_min_hash_value

db_min_hash_value — value of db min hash. Category: *database internals, distributed systems*.

```
p db_min_hash_value $x
```

db_nested_loop_cost

db_nested_loop_cost — operation db nested loop cost. Category: *database internals, distributed systems*.

```
p db_nested_loop_cost $x
```

db_page_cache_eviction_age

db_page_cache_eviction_age — operation db page cache eviction age. Category: *database internals, distributed systems*.

```
p db_page_cache_eviction_age $x
```

db_partition_failure_check

db_partition_failure_check — operation db partition failure check. Category: *database internals, distributed systems*.

```
p db_partition_failure_check $x
```

db_partitions_for_n

db_partitions_for_n — operation db partitions for n. Category: *database internals, distributed systems*.

```
p db_partitions_for_n $x
```

db_pastry_routing_step

db_pastry_routing_step — single-step update of db pastry routing. Category: *database internals, distributed systems*.

```
p db_pastry_routing_step $x
```

db_paxos_propose_id

db_paxos_propose_id — operation db paxos propose id. Category: *database internals, distributed systems*.

```
p db_paxos_propose_id $x
```


db_pbft_view_change

db_pbft_view_change — operation db pbft view change. Category: *database internals, distributed systems*.

```
p db_pbft_view_change $x
```

db_power

db_power — operation db power. Alias for dB_power. Category: *misc*.

```
p db_power $x
```

db_quantile_estimate_p99

db_quantile_estimate_p99 — operation db quantile estimate p99. Category: *database internals, distributed systems*.

```
p db_quantile_estimate_p99 $x
```

db_query_cardinality

db_query_cardinality — operation db query cardinality. Category: *database internals, distributed systems*.

```
p db_query_cardinality $x
```

db_query_plan_cost_step

db_query_plan_cost_step — single-step update of db query plan cost. Category: *database internals, distributed systems*.

```
p db_query_plan_cost_step $x
```

db_quorum_intersection_check

db_quorum_intersection_check — operation db quorum intersection check. Category: *database internals, distributed systems*.

```
p db_quorum_intersection_check $x
```

db_quotient_filter_canonical

db_quotient_filter_canonical — operation db quotient filter canonical. Category: *database internals, distributed systems*.

```
p db_quotient_filter_canonical $x
```

db_raft_log_match_check

db_raft_log_match_check — operation db raft log match check. Category: *database internals, distributed systems*.

```
p db_raft_log_match_check $x
```

db_raft_term_advance

db_raft_term_advance — operation db raft term advance. Category: *database internals, distributed systems*.

```
p db_raft_term_advance $x
```

db_read_amplification

db_read_amplification — operation db read amplification. Category: *database internals, distributed systems*.

```
p db_read_amplification $x
```

db_read_repair_step

db_read_repair_step — single-step update of db read repair. Category: *database internals, distributed systems*.

```
p db_read_repair_step $x
```

db_rendezvous_hash_score

db_rendezvous_hash_score — score of db rendezvous hash. Category: *database internals, distributed systems*.

```
p db_rendezvous_hash_score $x
```

db_replica_lag_threshold

db_replica_lag_threshold — operation db replica lag threshold. Category: *database internals, distributed systems*.

```
p db_replica_lag_threshold $x
```

db_replication_lag_step

db_replication_lag_step — single-step update of db replication lag. Category: *database internals, distributed systems*.

```
p db_replication_lag_step $x
```

db_reservoir_sample_index

db_reservoir_sample_index — index of db reservoir sample. Category: *database internals, distributed systems*.

```
p db_reservoir_sample_index $x
```

db_seq_scan_cost

db_seq_scan_cost — operation db seq scan cost. Category: *database internals, distributed systems*.

```
p db_seq_scan_cost $x
```

db_simhash_bit

db_simhash_bit — operation db simhash bit. Category: *database internals, distributed systems*.

```
p db_simhash_bit $x
```

db_size_tiered_compaction

db_size_tiered_compaction — operation db size tiered compaction. Category: *database internals, distributed systems*.

```
p db_size_tiered_compaction $x
```

db_skiplist_height_pick

`db_skiplist_height_pick` — operation `db skiplist height pick`. Category: **database internals, distributed systems**.

```
p db_skiplist_height_pick $x
```

db_sort_cost_estimate

`db_sort_cost_estimate` — operation `db sort cost estimate`. Category: **database internals, distributed systems**.

```
p db_sort_cost_estimate $x
```

db_space_amplification

`db_space_amplification` — operation `db space amplification`. Category: **database internals, distributed systems**.

```
p db_space_amplification $x
```

db_split_brain_check

`db_split_brain_check` — operation `db split brain check`. Category: **database internals, distributed systems**.

```
p db_split_brain_check $x
```

db_strong_consistency_check

`db_strong_consistency_check` — operation `db strong consistency check`. Category: **database internals, distributed systems**.

```
p db_strong_consistency_check $x
```

db_synchronous_commit_check

`db_synchronous_commit_check` — operation `db synchronous commit check`. Category: **database internals, distributed systems**.

```
p db_synchronous_commit_check $x
```

db_t_digest_centroid

`db_t_digest_centroid` — operation `db t digest centroid`. Category: **database internals, distributed systems**.

```
p db_t_digest_centroid $x
```

db_three_phase_commit_step

`db_three_phase_commit_step` — single-step update of `db three phase commit`. Category: **database internals, distributed systems**.

```
p db_three_phase_commit_step $x
```

db_throttle_token_step

`db_throttle_token_step` — single-step update of `db throttle token`. Category: **database internals, distributed systems**.

```
p db_throttle_token_step $x
```

db_tinylfu_admit_score

db_tinylfu_admit_score — score of db tinylfu admit. Category: *database internals, distributed systems*.

```
p db_tinylfu_admit_score $x
```

db_to_amp

db_to_amp — convert from db to amp. Category: *astronomy / music / color / units*.

```
p db_to_amp $value
```

db_token_bucket_step

db_token_bucket_step — single-step update of db token bucket. Category: *database internals, distributed systems*.

```
p db_token_bucket_step $x
```

db_two_phase_commit_step

db_two_phase_commit_step — single-step update of db two phase commit. Category: *database internals, distributed systems*.

```
p db_two_phase_commit_step $x
```

db_universal_compaction_step

db_universal_compaction_step — single-step update of db universal compaction. Category: *database internals, distributed systems*.

```
p db_universal_compaction_step $x
```

db_vector_clock_merge

db_vector_clock_merge — operation db vector clock merge. Category: *database internals, distributed systems*.

```
p db_vector_clock_merge $x
```

db_voltage

db_voltage — operation db voltage. Alias for dB_voltage. Category: *misc*.

```
p db_voltage $x
```

db_w_tinylfu_freq

db_w_tinylfu_freq — operation db w tinylfu freq. Category: *database internals, distributed systems*.

```
p db_w_tinylfu_freq $x
```

db_wal_fsync_cost

db_wal_fsync_cost — operation db wal fsync cost. Category: *database internals, distributed systems*.

```
p db_wal_fsync_cost $x
```

db_write_amplification

`db_write_amplification` — operation `db write amplification`. Category: **database internals, distributed systems**.

```
p db_write_amplification $x
```

db_zab_epoch_step

`db_zab_epoch_step` — single-step update of `db zab epoch`. Category: **database internals, distributed systems**.

```
p db_zab_epoch_step $x
```

dbeta

`dbeta` — operation `dbeta`. Category: **extras**.

```
p dbeta $x
```

dbinom

`dbinom` — binomial PMF. $P(X = k)$ for $\text{Binom}(n, p)$. Args: `k`, `n`, `p`.

```
p dbinom(5, 10, 0.5) # P(exactly 5 heads in 10 flips)
```

dbmclose

`dbmclose` — operation `dbmclose`. Category: **ipc**.

```
p dbmclose $x
```

dbmopen

`dbmopen` — operation `dbmopen`. Category: **ipc**.

```
p dbmopen $x
```

dbsize

`dbsize` — operation `dbsize`. Category: **redis-flavour primitives**.

```
p dbsize $x
```

dcauchy

`dcauchy` — operation `dcauchy`. Category: **extras**.

```
p dcauchy $x
```

dchisq

`dchisq` — operation `dchisq`. Category: **r / scipy distributions and tests**.

```
p dchisq $x
```

dct

`dct` computes the Type-II Discrete Cosine Transform of a signal. Used in JPEG, MP3, and speech processing.

```
val @coeffs = @{dct([1,2,3,4])}
```

dde_euler_step

`dde_euler_step` — single-step update of `dde euler`. Category: **ode advanced**.

```
p dde_euler_step $x
```

ddot

`ddot` — operation `ddot`. Category: **blas / lapack**.

```
p ddot $x
```

ddpg_critic_loss_step

`ddpg_critic_loss_step` — single-step update of `ddpg critic loss`. Category: **electrochemistry, batteries, fuel cells**.

```
p ddpd_critic_loss_step $x
```

ddrot

`ddrot` — operation `ddrot`. Category: **blas / lapack**.

```
p ddrot $x
```

ddt

`ddt` — operation `ddt`. Alias for `dedent`. Category: **extended stdlib**.

```
p ddt $x
```

de_broglie

`de_broglie` — operation `de broglie`. Category: **chemistry**.

```
p de_broglie $x
```

de_broglie_wavelength

`de_broglie_wavelength($mass, $velocity)` (alias `debroglie`) — quantum wavelength $\lambda = h/(mv)$. Defaults to electron mass.

```
p debroglie(9.109e-31, 1e6) # ~7.3e-10 m (electron at 1 km/s)
```

de_broglie_wavelength_kg

`de_broglie_wavelength_kg` — operation `de broglie wavelength kg`. Category: **misc**.

```
p de_broglie_wavelength_kg $x
```

de_rham_zero

`de_rham_zero` — operation `de rham zero`. Category: **econometrics**.

```
p de_rham_zero $x
```

de_sitter_metric_step

`de_sitter_metric_step` — single-step update of `de sitter metric`. Category: **electrochemistry, batteries, fuel cells**.

```
p de_sitter_metric_step $x
```

de_sitter_radius_step

`de_sitter_radius_step` — single-step update of `de sitter radius`. Category: *electrochemistry, batteries, fuel cells*.

```
p de_sitter_radius_step $x
```

deadweight_loss

`deadweight_loss` — operation `deadweight loss`. Category: *economics + game theory*.

```
p deadweight_loss $x
```

deal_n_k

`deal_n_k` — operation `deal n k`. Category: *apl/j/k array primitives*.

```
p deal_n_k $x
```

debug_object

`debug_object` — operation `debug object`. Category: *redis-flavour primitives*.

```
p debug_object $x
```

debug_val

`debug_val` — operation `debug val`. Category: *functional combinators*.

```
p debug_val $x
```

debye_huckel

`debye_huckel` — operation `debye huckel`. Category: *chemistry*.

```
p debye_huckel $x
```

debye_huckel_activity

`debye_huckel_activity` — operation `debye huckel activity`. Category: *electrochemistry, batteries, fuel cells*.

```
p debye_huckel_activity $x
```

debye_huckel_screening_factor

`debye_huckel_screening_factor` — factor of `debye huckel screening`. Category: *electrochemistry, batteries, fuel cells*.

```
p debye_huckel_screening_factor $x
```

debye_length_electrolyte

`debye_length_electrolyte` — operation `debye length electrolyte`. Category: *electrochemistry, batteries, fuel cells*.

```
p debye_length_electrolyte $x
```

dec

`dec` — operation `dec`. Category: *pipeline / string helpers*.

```
p dec $x
```

dec_each

`dec_each` — operation `dec` `each`. Category: *trivial numeric helpers*.

```
p dec_each $x
```

decagonal_number

`decagonal_number(n) → int` — $n \cdot (4n - 3)$.

deceleration_q

`deceleration_q` — operation `deceleration` `q`. Category: *cosmology / gr / flrw*.

```
p deceleration_q $x
```

decibel_ratio

`decibel_ratio` — ratio of `decibel`. Category: *physics formulas*.

```
p decibel_ratio $x
```

decimate

`downsample(\@signal, $factor)` (alias `decimate`) — reduces sample rate by keeping every `n`th sample. Apply anti-aliasing filter first to avoid aliasing.

```
val $decimated = decimate(\@signal, 4) # keep every 4th sample
```

decimate_step

`decimate_step` — single-step update of `decimate`. Category: *economics + game theory*.

```
p decimate_step $x
```

decode_base

`decode_base` — operation `decode` `base`. Category: *apl/j/k array primitives*.

```
p decode_base $x
```

decoherence_time

`decoherence_time` — operation `decoherence` `time`. Category: *quantum mechanics deep*.

```
p decoherence_time $x
```

deconvolve_step

`deconvolve_step` — single-step update of `deconvolve`. Category: *economics + game theory*.

```
p deconvolve_step $x
```


decr

`decr` — operation `decr`. Category: **redis-flavour primitives**.

```
p decr $x
```

decrby

`decrby` — operation `decrby`. Category: **redis-flavour primitives**.

```
p decrby $x
```

dedent

`dedent` — operation `dedent`. Category: **extended stdlib**.

```
p dedent $x
```

dedent_text

`dedent_text` — operation `dedent text`. Category: **string processing extras**.

```
p dedent_text $x
```

deep_q_target

`deep_q_target` — operation `deep q target`. Category: **electrochemistry, batteries, fuel cells**.

```
p deep_q_target $x
```

default_to

`default_to` — operation `default to`. Category: **functional combinators**.

```
p default_to $x
```

defaultdict

`defaultdict` — operation `defaultdict`. Category: **data structure helpers**.

```
p defaultdict $x
```

deferred_acceptance

`deferred_acceptance` — operation `deferred acceptance`. Category: **economics + game theory**.

```
p deferred_acceptance $x
```

deferred_acceptance_step

`deferred_acceptance_step` — single-step update of `deferred acceptance`. Category: **climate, fluids, atmospheric**.

```
p deferred_acceptance_step $x
```

deficient

`deficient` — operation `deficient`. Category: **math / number theory extras**.

```
p deficient $x
```

deficient_numbers

`deficient_numbers` — operation `deficient numbers`. Category: *number theory / primes*.

```
p deficient_numbers $x
```

defined_count

`defined_count` — count of `defined`. Category: *counters*.

```
p defined_count $x
```

definums

`definums` — operation `definums`. Alias for `deficient_numbers`. Category: *number theory / primes*.

```
p definums $x
```

deflate_dynamic_huffman

`deflate_dynamic_huffman` — operation `deflate dynamic huffman`. Category: *archive/encoding format primitives*.

```
p deflate_dynamic_huffman $x
```

deflate_encode_huffman

`deflate_encode_huffman` — operation `deflate encode huffman`. Category: *b81-misc-utility*.

```
p deflate_encode_huffman $x
```

deflate_huffman_lit

`deflate_huffman_lit` — operation `deflate huffman lit`. Category: *electrochemistry, batteries, fuel cells*.

```
p deflate_huffman_lit $x
```

deflate_static_block

`deflate_static_block` — operation `deflate static block`. Category: *archive/encoding format primitives*.

```
p deflate_static_block $x
```

deg_to_rad

`deg_to_rad` — convert from `deg` to `rad`. Category: *trivial numeric / predicate builtins*.

```
p deg_to_rad $value
```

degrees

`degrees` — operation `degrees`. Category: *trig / math*.

```
p degrees $x
```

degrees_to_compass

`degrees_to_compass` — convert from `degrees` to `compass`. Category: *misc / utility*.

```
p degrees_to_compass $value
```

de la

`de la` — operation `de la`. Alias for `delete_at`. Category: **go/general functional utilities**.

```
p de la $x
```

delaunay_triangulate_2d

`delaunay_triangulate_2d(points) → array` — Bowyer-Watson Delaunay triangulation. Returns triangles as `[i, j, k]` index triples.

delby

`delby` — operation `delby`. Alias for `delete_by`. Category: **additional missing stdlib functions**.

```
p delby $x
```

delete_at

`delete_at` — operation `delete at`. Category: **go/general functional utilities**.

```
p delete_at $x
```

delete_by

`delete_by` — operation `delete by`. Category: **additional missing stdlib functions**.

```
p delete_by $x
```

delete_first

`delete_first` — operation `delete first`. Category: **additional missing stdlib functions**.

```
p delete_first $x
```

delfst

`delfst` — operation `delfst`. Alias for `delete_first`. Category: **additional missing stdlib functions**.

```
p delfst $x
```

delta_complex_count

`delta_complex_count` — delta encoding `complex count`. Category: **econometrics**.

```
p delta_complex_count $x
```

delta_decode

`delta_decode(deltas) → array` — running sum reconstruction of `delta_encode`.

delta_e

`delta_e` — delta encoding `e`. Alias for `delta_e_76`. Category: **misc**.

```
p delta_e $x
```

delta_e2000

delta_e2000 — delta encoding [e2000](#). Category: *b82-misc-utility*.

```
p delta_e2000 $x
```

delta_e76

delta_e76 — delta encoding [e76](#). Category: *b82-misc-utility*.

```
p delta_e76 $x
```

delta_e94

delta_e94 — delta encoding [e94](#). Category: *b82-misc-utility*.

```
p delta_e94 $x
```

delta_e_76

delta_e_76 — delta encoding [e 76](#). Category: *misc*.

```
p delta_e_76 $x
```

delta_e_94

delta_e_94 — delta encoding [e 94](#). Category: *astronomy / music / color / units*.

```
p delta_e_94 $x
```

delta_encode

delta_encode(ints) → array — keep first element, replace each subsequent with the difference from its predecessor.

```
p delta_encode([10,12,15,20]) # [10,2,3,5]
```

dema

dema(prices, period=10) → array — double EMA: $2 \cdot \text{EMA} - \text{EMA}(\text{EMA})$. Lower lag than EMA at the same period.

```
p dema(@prices, 14)
```

demorse

demorse — operation demorse. Alias for [morse_decode](#). Category: *encoding / phonetics*.

```
p demorse $x
```

denavit_hartenberg_h

denavit_hartenberg_h — operation [denavit hartenberg h](#). Category: *robotics & control*.

```
p denavit_hartenberg_h $x
```

dense_rank

dense_rank — operation [dense rank](#). Category: *extended stdlib*.

```
p dense_rank $x
```

density

density — kernel density estimation with Gaussian kernel and Silverman bandwidth. Returns `[[x_grid], [density_values]]`. Like R's `density()`.

```
var ($x, $y) = @{density([1,2,2,3,3,3,4,4,5])}  
# $x is grid of 512 points, $y is estimated density
```

density_altitude_full

density_altitude_full — operation `density altitude full`. Category: **climate, fluids, atmospheric**.

```
p density_altitude_full $x
```

density_altitude_m

density_altitude_m — operation `density altitude m`. Category: **misc**.

```
p density_altitude_m $x
```

density_matrix_purity_q

density_matrix_purity_q — operation `density matrix purity q`. Category: **quantum**.

```
p density_matrix_purity_q $x
```

depdbl

depdbl — operation `depdbl`. Alias for `depreciation_double`. Category: **math / numeric extras**.

```
p depdbl $x
```

dependency_parse_eisner

dependency_parse_eisner — operation `dependency parse eisner`. Category: **computational linguistics**.

```
p dependency_parse_eisner $x
```

dephasing_channel

dephasing_channel — operation `dephasing channel`. Category: **more extensions**.

```
p dephasing_channel $x
```

deplin

deplin — operation `deplin`. Alias for `depreciation_linear`. Category: **math / numeric extras**.

```
p deplin $x
```

depolarizing_channel

depolarizing_channel — operation `depolarizing channel`. Category: **more extensions**.

```
p depolarizing_channel $x
```

depolarizing_density_2x2

`depolarizing_density_2x2` — operation `depolarizing density 2x2`. Category: **quantum mechanics deep**.

```
p depolarizing_density_2x2 $x
```

depreciation_double

`depreciation_double` — operation `depreciation double`. Category: **math / numeric extras**.

```
p depreciation_double $x
```

depreciation_linear

`depreciation_linear` — operation `depreciation linear`. Category: **math / numeric extras**.

```
p depreciation_linear $x
```

depth_of_field_far

`depth_of_field_far(focal_mm, aperture, dist_mm, coc=0.03) → number` — far limit of DoF (∞ if subject is at/beyond hyperfocal).

depth_of_field_near

`depth_of_field_near(focal_mm, aperture, dist_mm, coc=0.03) → number` — near limit of DoF in mm.

derangement_count

`derangement_count(n) → int` — number of permutations with no fixed points: $D_n = (n-1) \cdot (D_{n-1} + D_{n-2})$.

```
p derangement_count(5) # 44
```

derangements

`derangements` counts the number of derangements (subfactorial $!n$) — permutations with no fixed points.

```
p derangements(4) # 9
p derangements(5) # 44
```

deroman

`deroman` — operation `deroman`. Alias for `roman_to_int`. Category: **roman numerals**.

```
p deroman $x
```

describe

`describe(@data)` — returns a hashref with comprehensive descriptive statistics: count, mean, std, min, 25%, 50%, 75%, max. Similar to `pandas DataFrame.describe()`.

```
val $stats = describe(1:100)
p $stats->{mean} # 50.5
p $stats->{std} # ~29.0
p $stats->{"50%"} # median
```

destination_lat_lon

`destination_lat_lon(lat, lon, bearing_deg, distance_m) → [lat, lon]` — destination point along a great circle from origin.

```
p destination_lat_lon(0, 0, 90, 111320) # ~1° east
```

destination_point

`destination_point` — operation `destination point`. Category: **excel/sheets + bond/loan financial**.

```
p destination_point $x
```

det

`matrix_det_general` (aliases `mdetg`, `det`) computes the determinant of any NxN matrix via LU decomposition.

```
p det([[1,2,3],[4,5,6],[7,8,0]]) # 27
```

detab

`detab(s, width=8) → string` — expand tab characters to spaces, respecting tab stops.

```
p detab("\tab", 4) # "    ab"
```

detect

`detect` — operation `detect`. Category: **functional / iterator**.

```
p detect $x
```

deterministic_prime

`deterministic_prime` — operation `deterministic prime`. Category: **cryptography deep**.

```
p deterministic_prime $x
```

detrend_linear

`detrend_linear(series) → array` — subtract OLS linear fit from each point, leaving the residuals.

```
p detrend_linear(@xs)
```

detrended_fluct_alpha

`detrended_fluct_alpha` — operation `detrended fluct alpha`. Category: **climate, fluids, atmospheric**.

```
p detrended_fluct_alpha $x
```

dew_point

`dew_point` — operation `dew point`. Category: **physics formulas**.

```
p dew_point $x
```

dew_point_magnus

dew_point_magnus — operation `dew point magnus`. Category: **misc**.

```
p dew_point_magnus $x
```

dewpoint_temperature_full

dewpoint_temperature_full — operation `dewpoint temperature full`. Category: **climate, fluids, atmospheric**.

```
p dewpoint_temperature_full $x
```

dexp

dexp — operation `dexp`. Category: **extras**.

```
p dexp $x
```

df

df — operation `df`. Alias for `dataframe`. Category: **data / network**.

```
p df $x
```

df_aggregate

df_aggregate — operation `df aggregate`. Category: **pandas dataframe ops**.

```
p df_aggregate $x
```

df_apply

df_apply — operation `df apply`. Category: **pandas dataframe ops**.

```
p df_apply $x
```

df_clip

df_clip — operation `df clip`. Category: **pandas dataframe ops**.

```
p df_clip $x
```

df_concat

df_concat — operation `df concat`. Category: **pandas dataframe ops**.

```
p df_concat $x
```

df_corr

df_corr — operation `df corr`. Category: **pandas dataframe ops**.

```
p df_corr $x
```

df_corrwith

df_corrwith — operation `df corrwith`. Category: **pandas dataframe ops**.

```
p df_corrwith $x
```


df_cov

df_cov — operation `df cov`. Category: *pandas dataframe ops*.

```
p df_cov $x
```

df_crosstab

df_crosstab — operation `df crosstab`. Category: *pandas dataframe ops*.

```
p df_crosstab $x
```

df_describe

df_describe — operation `df describe`. Category: *pandas dataframe ops*.

```
p df_describe $x
```

df_diff

df_diff — operation `df diff`. Category: *pandas dataframe ops*.

```
p df_diff $x
```

df_dist

df_dist — operation `df dist`. Category: *r / scipy distributions and tests*.

```
p df_dist $x
```

df_drop_duplicates

df_drop_duplicates — operation `df drop duplicates`. Category: *pandas dataframe ops*.

```
p df_drop_duplicates $x
```

df_dropna

df_dropna — operation `df dropna`. Category: *pandas dataframe ops*.

```
p df_dropna $x
```

df_duplicated

df_duplicated — operation `df duplicated`. Category: *pandas dataframe ops*.

```
p df_duplicated $x
```

df_eval

df_eval — operation `df eval`. Category: *pandas dataframe ops*.

```
p df_eval $x
```

df_ewm

df_ewm — operation `df ewm`. Category: *pandas dataframe ops*.

```
p df_ewm $x
```

df_expanding

`df_expanding` — operation `df expanding`. Category: *pandas dataframe ops*.

```
p df_expanding $x
```

df_explode

`df_explode` — operation `df explode`. Category: *pandas dataframe ops*.

```
p df_explode $x
```

df_fillna

`df_fillna` — operation `df fillna`. Category: *pandas dataframe ops*.

```
p df_fillna $x
```

df_filter

`df_filter` — filter op of `df`. Category: *pandas dataframe ops*.

```
p df_filter $x
```

df_get_dummies

`df_get_dummies` — operation `df get dummies`. Category: *pandas dataframe ops*.

```
p df_get_dummies $x
```

df_groupby

`df_groupby` — operation `df groupby`. Category: *pandas dataframe ops*.

```
p df_groupby $x
```

df_idxmax

`df_idxmax` — operation `df idxmax`. Category: *pandas dataframe ops*.

```
p df_idxmax $x
```

df_idxmin

`df_idxmin` — operation `df idxmin`. Category: *pandas dataframe ops*.

```
p df_idxmin $x
```

df_interpolate

`df_interpolate` — operation `df interpolate`. Category: *pandas dataframe ops*.

```
p df_interpolate $x
```

df_isnull

`df_isnull` — operation `df isnull`. Category: *pandas dataframe ops*.

```
p df_isnull $x
```

df_join

df_join — operation df join. Category: *pandas dataframe ops*.

```
p df_join $x
```

df_kurtosis

df_kurtosis — operation df kurtosis. Category: *pandas dataframe ops*.

```
p df_kurtosis $x
```

df_mad

df_mad — operation df mad. Category: *pandas dataframe ops*.

```
p df_mad $x
```

df_melt

df_melt — operation df melt. Category: *pandas dataframe ops*.

```
p df_melt $x
```

df_merge

df_merge — operation df merge. Category: *pandas dataframe ops*.

```
p df_merge $x
```

df_nlargest

df_nlargest — operation df nlargest. Category: *pandas dataframe ops*.

```
p df_nlargest $x
```

df_notnull

df_notnull — operation df notnull. Category: *pandas dataframe ops*.

```
p df_notnull $x
```

df_nsmallest

df_nsmallest — operation df nsmallest. Category: *pandas dataframe ops*.

```
p df_nsmallest $x
```

df_pct_change

df_pct_change — operation df pct change. Category: *pandas dataframe ops*.

```
p df_pct_change $x
```

df_pivot

df_pivot — operation df pivot. Category: *pandas dataframe ops*.

```
p df_pivot $x
```

df_pivot_table

`df_pivot_table` — lookup table of `df pivot`. Category: *pandas dataframe ops*.

```
p df_pivot_table $x
```

df_quantile

`df_quantile` — operation `df quantile`. Category: *pandas dataframe ops*.

```
p df_quantile $x
```

df_query

`df_query` — operation `df query`. Category: *pandas dataframe ops*.

```
p df_query $x
```

df_rank

`df_rank` — operation `df rank`. Category: *pandas dataframe ops*.

```
p df_rank $x
```

df_replace

`df_replace` — operation `df replace`. Category: *pandas dataframe ops*.

```
p df_replace $x
```

df_resample

`df_resample` — operation `df resample`. Category: *pandas dataframe ops*.

```
p df_resample $x
```

df_reset_index

`df_reset_index` — index of `df reset`. Category: *pandas dataframe ops*.

```
p df_reset_index $x
```

df_rolling

`df_rolling` — operation `df rolling`. Category: *pandas dataframe ops*.

```
p df_rolling $x
```

df_round

`df_round` — operation `df round`. Category: *pandas dataframe ops*.

```
p df_round $x
```

df_sample

`df_sample` — operation `df sample`. Category: *pandas dataframe ops*.

```
p df_sample $x
```

df_sem

df_sem — operation `df sem`. Category: *pandas dataframe ops*.

```
p df_sem $x
```

df_set_index

df_set_index — index of `df set`. Category: *pandas dataframe ops*.

```
p df_set_index $x
```

df_shift

df_shift — operation `df shift`. Category: *pandas dataframe ops*.

```
p df_shift $x
```

df_skew

df_skew — operation `df skew`. Category: *pandas dataframe ops*.

```
p df_skew $x
```

df_sort_values

df_sort_values — operation `df sort values`. Category: *pandas dataframe ops*.

```
p df_sort_values $x
```

df_stack

df_stack — operation `df stack`. Category: *pandas dataframe ops*.

```
p df_stack $x
```

df_to_datetime

df_to_datetime — convert from `df` to `datetime`. Category: *pandas dataframe ops*.

```
p df_to_datetime $value
```

df_to_numeric

df_to_numeric — convert from `df` to `numeric`. Category: *pandas dataframe ops*.

```
p df_to_numeric $value
```

df_to_timedelta

df_to_timedelta — convert from `df` to `timedelta`. Category: *pandas dataframe ops*.

```
p df_to_timedelta $value
```

df_transform

df_transform — transform of `df`. Category: *pandas dataframe ops*.

```
p df_transform $x
```

df_unstack

`df_unstack` — operation `df unstack`. Category: *pandas dataframe ops*.

```
p df_unstack $x
```

df_value_counts

`df_value_counts` — operation `df value counts`. Category: *pandas dataframe ops*.

```
p df_value_counts $x
```

dfa_minimize_hopcroft

`dfa_minimize_hopcroft` — operation `dfa minimize hopcroft`. Category: *compilers / parsing*.

```
p dfa_minimize_hopcroft $x
```

dfa_simulate_step

`dfa_simulate_step` — single-step update of `dfa simulate`. Category: *compilers / parsing*.

```
p dfa_simulate_step $x
```

dfact

`dfact` — operation `dfact`. Alias for `double_factorial`. Category: *math formulas*.

```
p dfact $x
```

dfs

`dfs` — operation `dfs`. Alias for `dfs_traverse`. Category: *extended stdlib*.

```
p dfs $x
```

dfs_postorder_done

`dfs_postorder_done` — operation `dfs postorder done`. Category: *networkx graph algorithms*.

```
p dfs_postorder_done $x
```

dfs_preorder

`dfs_preorder` — operation `dfs preorder`. Category: *misc*.

```
p dfs_preorder $x
```

dfs_traverse

`dfs_traverse` — operation `dfs traverse`. Category: *extended stdlib*.

```
p dfs_traverse $x
```

dft

`dft(\@signal)` — computes the Discrete Fourier Transform. Returns arrayref of complex numbers as [re, im] pairs. $O(n^2)$ reference implementation; for large n , use FFT libraries.

```
val @signal = map { sin(2 * 3.14159 * _ / 64) } 0:63
var $spectrum = dft(\@signal)
```

dg_from_k

dg_from_k — operation `dg_from_k`. Category: **chemistry**.

```
p dg_from_k $x
```

dgamma

dgamma — operation `dgamma`. Category: **extras**.

```
p dgamma $x
```

dgbsv

dgbsv — operation `dgbsv`. Category: **blas / lapack**.

```
p dgbsv $x
```

dgels

dgels — operation `dgels`. Category: **blas / lapack**.

```
p dgels $x
```

dgelzd

dgelzd — operation `dgelzd`. Category: **blas / lapack**.

```
p dgelzd $x
```

dgemm

dgemm — operation `dgemm`. Category: **blas / lapack**.

```
p dgemm $x
```

dgemm3m

dgemm3m — operation `dgemm3m`. Category: **blas / lapack**.

```
p dgemm3m $x
```

dgemv

dgemv — operation `dgemv`. Category: **blas / lapack**.

```
p dgemv $x
```

dgeom

dgeom — geometric PMF $P(X=k)$. Args: `k`, `prob`.

```
p dgeom(3, 0.5) # P(first success on 4th trial)
```

dgeqrf

`dgeqrf` — operation `dgeqrf`. Category: **blas / lapack**.

```
p dgeqrf $x
```

dgerqf

`dgerqf` — operation `dgerqf`. Category: **blas / lapack**.

```
p dgerqf $x
```

dgesv

`dgesv` — operation `dgesv`. Category: **blas / lapack**.

```
p dgesv $x
```

dgesvd

`dgesvd` — operation `dgesvd`. Category: **blas / lapack**.

```
p dgesvd $x
```

dgetrf

`dgetrf` — operation `dgetrf`. Category: **blas / lapack**.

```
p dgetrf $x
```

dh_compute_shared

`dh_compute_shared(private, public, modulus) → string` — Diffie-Hellman shared key: `public^private mod modulus`.

dh_secret

`dh_secret` — Diffie-Hellman key-exchange primitive `secret`. Category: **cryptanalysis & number theory deep**.

```
p dh_secret $x
```

dh_shared

`dh_shared` — Diffie-Hellman key-exchange primitive `shared`. Category: **cryptography deep**.

```
p dh_shared $x
```

dhyper

`dhyper` — hypergeometric PMF. Args: `k`, `m` (white), `n` (black), `nn` (draws).

```
p dhyper(2, 5, 5, 3) # P(2 white balls in 3 draws from 5W+5B)
```

diag

`diag` — operation `diag`. Alias for `diagonal`. Category: **extended stdlib**.

```
p diag $x
```


diagonal

`diagonal` — operation `diagonal`. Category: **extended stdlib**.

```
p diagonal $x
```

diagonal_dominance_q

`diagonal_dominance_q` — operation `diagonal dominance q`. Category: **more extensions (2)**.

```
p diagonal_dominance_q $x
```

diameter_bfs

`diameter_bfs` — operation `diameter bfs`. Category: **graph algorithms**.

```
p diameter_bfs $x
```

diameter_unweighted

`diameter_unweighted` — operation `diameter unweighted`. Category: **more extensions**.

```
p diameter_unweighted $x
```

dice_coeff

`dice_coeff(a, b)` → number — $2 \cdot |A \cap B| / (|A| + |B|)$.

dice_coefficient

`dice_coefficient` — operation `dice coefficient`. Category: **extended stdlib**.

```
p dice_coefficient $x
```

dice_roll

`dice_roll` — operation `dice roll`. Category: **random**.

```
p dice_roll $x
```

dicecoef

`dicecoef` — operation `dicecoef`. Alias for `dice_coefficient`. Category: **extended stdlib**.

```
p dicecoef $x
```

dickey_fuller_critical

`dickey_fuller_critical` — operation `dickey fuller critical`. Category: **econometrics**.

```
p dickey_fuller_critical $x
```

dictator_game_share

`dictator_game_share` — operation `dictator game share`. Category: **climate, fluids, atmospheric**.

```
p dictator_game_share $x
```

did_estimator

`did_estimator` — operation `did_estimator`. Category: *economics + game theory*.

```
p did_estimator $x
```

die_if

`die_if` — operation `die_if`. Category: *conversion / utility*.

```
p die_if $x
```

die_unless

`die_unless` — operation `die_unless`. Category: *conversion / utility*.

```
p die_unless $x
```

diesel_efficiency

`diesel_efficiency` — operation `diesel_efficiency`. Category: *misc*.

```
p diesel_efficiency $x
```

diff

`diff` — operation `diff`. Category: *extended stdlib*.

```
p diff $x
```

diff_array

`numerical_diff` (aliases `numdiff`, `diff_array`) computes the numerical first derivative via central differences. Returns an array.

```
val @y = map { _ ** 2 } 0:10  
var @dy = @{numdiff(\@y)} # ≈ [0, 2, 4, 6, ...]
```

diff_coeff_aqueous_estimate

`diff_coeff_aqueous_estimate` — operation `diff_coeff_aqueous_estimate`. Category: *electrochemistry, batteries, fuel cells*.

```
p diff_coeff_aqueous_estimate $x
```

diff_days

`diff_days` — operation `diff_days`. Category: *extended stdlib*.

```
p diff_days $x
```

diff_hours

`diff_hours` — operation `diff_hours`. Category: *extended stdlib*.

```
p diff_hours $x
```

diff_in_diff

`diff_in_diff` — operation `diff in diff`. Category: *economics + game theory*.

```
p diff_in_diff $x
```

diff_lag

`diff_lag` (alias `diff_ts`) — lagged differences. Args: `vec` [, `lag`, `differences`]. Like R's `diff()`.

```
p diff_lag([1,3,6,10])      # [2,3,4] (lag=1)
p diff_lag([1,3,6,10], 2)   # [5,7] (lag=2)
p diff_lag([1,3,6,10], 1, 2) # [1,1] (second differences)
```

diff_myers

`myers_diff(\@A, \@B)` → list of [`op`, `value`] arrayrefs (`op` = "=", "+", "-"). Eugene Myers' $O((N+M) \cdot D)$ longest-common-subsequence diff. Foundational text-diff primitive.

```
val @a = ("alpha", "beta", "gamma")
var @b = ("alpha", "BETA", "gamma")
myers_diff(\@a, \@b) |> e { p "$_->[0] $_->[1]" }
# = alpha
# - beta
# + BETA
# = gamma
```

diff_patience

`patience_diff(\@A, \@B)` → list of [`op`, `value`] arrayrefs. Bram Cohen's patience diff: finds the longest common subsequence of *unique* lines as anchors, then runs Myers on the gaps between anchors. Produces more human-readable diffs than Myers on code-like inputs where repeated short lines (braces, blank lines) make Myers pick noisy alignments.

diff_pct

`diff_pct(series)` → `array` — percentage change between consecutive points: $(x[i]/x[i-1]) - 1$.

```
p diff_pct([100, 110, 99])
```

diff_series

`diff_series(series)` → `array` — first-order differences $x[i+1] - x[i]$. Used to make a series stationary.

```
p diff_series([1,3,6,10,15]) # [2,3,4,5]
```

diffd

`diffd` — operation `diffd`. Alias for `diff_days`. Category: *extended stdlib*.

```
p diffd $x
```

diffh

`diffh` — operation `diffh`. Alias for `diff_hours`. Category: *extended stdlib*.

```
p diffh $x
```

diffraction_grating_angle

`diffraction_grating_angle` — operation `diffraction grating angle`. Category: **em / optics / relativity**.

```
p diffraction_grating_angle $x
```

diffuse_layer_thickness

`diffuse_layer_thickness` — operation `diffuse layer thickness`. Category: **electrochemistry, batteries, fuel cells**.

```
p diffuse_layer_thickness $x
```

diffusion_layer_thickness

`diffusion_layer_thickness` — operation `diffusion layer thickness`. Category: **electrochemistry, batteries, fuel cells**.

```
p diffusion_layer_thickness $x
```

diffusion_stability

`diffusion_stability` — operation `diffusion stability`. Category: **ode advanced**.

```
p diffusion_stability $x
```

dig

`dig` — operation `dig`. Category: **python/ruby stdlib**.

```
p dig $x
```

digest_auth_quote

`digest_auth_quote` — operation `digest auth quote`. Category: **b81-misc-utility**.

```
p digest_auth_quote $x
```

digital_call

`digital_call` — operation `digital call`. Category: **financial pricing models**.

```
p digital_call $x
```

digital_root

`digital_root` — operation `digital root`. Category: **extended stdlib**.

```
p digital_root $x
```

digitize

`digitize` — operation `digitize`. Category: **numpy + scipy.special**.

```
p digitize $x
```

digits_of

`digits_of` — operation `digits of`. Category: **list helpers**.

```
p digits_of $x
```

digroot

digroot — operation **digroot**. Alias for **digital_root**. Category: *extended stdlib*.

```
p digroot $x
```

dijkstra

dijkstra (alias **shortest_path**) computes shortest paths from a source node. Graph is a hash of {node => [[neighbor, weight], ...]}. Returns {node => distance}.

```
val $g = {A => [{"B",1},{"C",4}], B => [{"C",2}], C => []}  
var $d = dijkstra($g, "A") # {A=>0, B=>1, C=>3}
```

dijkstra_relax

dijkstra_relax — operation **dijkstra relax**. Category: *networkx graph algorithms*.

```
p dijkstra_relax $x
```

dilation_2d

dilation_2d(image, size=3) → matrix — morphological dilation.

dilution_v2

dilution_v2 — operation **dilution v2**. Category: *chemistry*.

```
p dilution_v2 $x
```

dim

dim — operation **dim**. Alias for **red**. Category: *color / ansi*.

```
p dim $x
```

dinic_blocking_flow

dinic_blocking_flow — operation **dinic blocking flow**. Category: *combinatorial optimization, scheduling*.

```
p dinic_blocking_flow $x
```

dinic_step

dinic_step — single-step update of **dinic**. Category: *networkx graph algorithms*.

```
p dinic_step $x
```

diopter

diopter — operation **diopter**. Alias for **lens_power**. Category: *physics formulas*.

```
p diopter $x
```

dir_stack

`dir_stack` → list the saved directory stack (oldest first, most-recent last). Read-only view; use `pushd` / `popd` to mutate.

direct_age_adjusted

`direct_age_adjusted` — operation `direct age adjusted`. Category: *epidemiology / public health*.

```
p direct_age_adjusted $x
```

dirs

`dirs` — operation `dirs`. Category: *filesystem extensions*.

```
p dirs $x
```

dirty_price

`dirty_price` — operation `dirty price`. Category: *excel/sheets + bond/loan financial*.

```
p dirty_price $x
```

disc_excel

`disc_excel` — operation `disc excel`. Category: *b81-misc-utility*.

```
p disc_excel $x
```

discount

`discount` — operation `discount`. Category: *math / numeric extras*.

```
p discount $x
```

discount_amount

`discount_amount` — operation `discount amount`. Category: *physics formulas*.

```
p discount_amount $x
```

discount_continuous

`discount_continuous` — operation `discount continuous`. Category: *financial pricing models*.

```
p discount_continuous $x
```

discount_factor_step

`discount_factor_step` — single-step update of `discount factor`. Category: *climate, fluids, atmospheric*.

```
p discount_factor_step $x
```

discount_pct

`discount_pct(original, sale)` → number — $(\text{original} - \text{sale}) / \text{original} \cdot 100$.

```
p discount_pct(200, 150) # 25
```

discounted_payback

`discounted_payback($initial, $rate, @cashflows)` — computes payback period using discounted cash flows. Accounts for time value of money.

```
p discounted_payback(1000, 0.1, 400, 400, 400, 400)
```

discrete_haar_step

`discrete_haar_step` — single-step update of `discrete_haar`. Category: **electrochemistry, batteries, fuel cells**.

```
p discrete_haar_step $x
```

discrete_log

`discrete_log` — operation `discrete log`. Category: **math / number theory extras**.

```
p discrete_log $x
```

disk_avail

`disk_avail PATH` — available disk space in bytes for non-root users. Uses `f_bavail` from `statvfs`, which excludes blocks reserved for the superuser.

```
p format_bytes(disk_avail)           # 76.3 GiB
gauge(1 - disk_avail / disk_total) |> p # usage gauge
```

disk_free

`disk_free PATH` — free disk space in bytes (superuser view). Uses `f_bfree` from `statvfs`.

```
p format_bytes(disk_free)           # 76.3 GiB
p format_bytes(disk_free("/tmp"))
```

disk_total

`disk_total PATH` — total disk space in bytes for the filesystem containing `PATH` (default `/`). Uses `statvfs` on unix. Returns `undef` on unsupported platforms.

```
p format_bytes(disk_total)           # 1.8 TiB
p format_bytes(disk_total("/home"))  # specific mount
```

disk_used

`disk_used PATH` — used disk space in bytes (total - free).

```
p format_bytes(disk_used)           # 1.7 TiB
var $pct = int(disk_used / disk_total * 100)
p "$pct% disk used"
```

disso

`disso` — operation `disso`. Category: **algebraic match**.

```
p disso $x
```

dist

`dist` — operation `dist`. Alias for `distance`. Category: **extended stdlib**.

```
p dist $x
```

dist_mat

`dist_matrix` (alias `dist_mat`) — pairwise distance matrix. Supports 'euclidean' (default), 'manhattan', 'maximum'. Like R's `dist()`.

```
val $D = dist_mat([[0,0],[1,0],[0,1]]) # 3x3 distance matrix
var $D2 = dist_mat([[0,0],[1,1]], "manhattan") # Manhattan
```

dist_point_line_2d

`dist_point_line_2d` — operation `dist point line 2d`. Category: *geometry / topology*.

```
p dist_point_line_2d $x
```

dist_point_plane_3d

`dist_point_plane_3d` — operation `dist point plane 3d`. Category: *geometry / topology*.

```
p dist_point_plane_3d $x
```

distance

`distance` — operation `distance`. Category: *extended stdlib*.

```
p distance $x
```

distb

`distb` — operation `distb`. Alias for `distinct_by`. Category: *additional missing stdlib functions*.

```
p distb $x
```

distinct_by

`distinct_by` — operation `distinct by`. Category: *additional missing stdlib functions*.

```
p distinct_by $x
```

distinct_count

`distinct_count` — count of `distinct`. Category: *collection*.

```
p distinct_count $x
```

distinct_sample

`distinct_sample` — operation `distinct sample`. Category: *iterator + string-distance extras*.

```
p distinct_sample $x
```

divergence_omega_step

`divergence_omega_step` — single-step update of `divergence omega`. Category: *climate, fluids, atmospheric*.

```
p divergence_omega_step $x
```


divisible_goods_proportional

`divisible_goods_proportional` — operation `divisible_goods_proportional`. Category: **climate, fluids, atmospheric**.

```
p divisible_goods_proportional $x
```

divisor_count

`divisor_count` — count of `divisor`. Category: **math / number theory extras**.

```
p divisor_count $x
```

divisor_sum

`divisor_sum` — sum of `divisor`. Category: **math / number theory extras**.

```
p divisor_sum @xs
```

divisors

`divisors` — operation `divisors`. Category: **math / numeric extras**.

```
p divisors $x
```

divmod

`divmod` — operation `divmod`. Category: **python/ruby stdlib**.

```
p divmod $x
```

divs

`divs` — operation `divs`. Alias for `divisors`. Category: **math / numeric extras**.

```
p divs $x
```

djb2

`djb2` — operation `djb2`. Category: **extended stdlib**.

```
p djb2 $x
```

djb2_hash

`djb2_hash` — hash function of `djb2`. Category: **misc**.

```
p djb2_hash $x
```

dlnorm

`dlnorm` — operation `dlnorm`. Category: **extras**.

```
p dlnorm $x
```

dlogis

`dlogis` — operation `dlogis`. Category: **extras**.

```
p dlogis $x
```

dm

dm — operation `dm`. Alias for `divmod`. Category: **python/ruby stdlib**.

```
p dm $x
```

dmetph

dmetph — operation `dmetph`. Alias for `double_metaphone`. Category: **encoding / phonetics**.

```
p dmetph $x
```

dna_at_content

`dna_at_content(seq)` → `number` — fraction of A, T, or U bases.

dna_complement

`dna_complement(seq)` → `string` — Watson-Crick complement (A ↔ T, G ↔ C). Case-preserving; non-ACGT chars pass through.

```
p dna_complement("ATGC") # "TACG"
```

dna_gc_content

`dna_gc_content(seq)` → `number` — fraction of G or C bases in [0,1].

```
p dna_gc_content("ATGCGCATAT") # 0.4
```

dna_kmer_count

`dna_kmer_count(seq, k=3)` → `hash` — frequency map of all k-mers in the sequence.

```
p dna_kmer_count("ATGATC", 2)
```

dna_kmer_index

`dna_kmer_index(seq, k=3)` → `hash` — for each k-mer, the list of start positions where it occurs.

dna_melting_temp

`dna_melting_temp(seq)` → `number` — T_m in °C. Uses Wallace rule for length <14, Marmur for longer.

```
p dna_melting_temp("ATGC") # 12
```

dna_reverse_complement

`dna_reverse_complement(seq)` → `string` — reverse-then-complement. Equivalent to the antisense strand read 5' → 3'.

```
p dna_reverse_complement("AAATGGC") # "GCCATTT"
```

`dna_transcribe`

`dna_transcribe(dna)` → `string` — replace T with U to produce mRNA.

```
p dna_transcribe("ATGC") # "AUGC"
```

`dna_translate`

`dna_translate(dna)` → `string` — translate DNA to protein (single-letter codes); stops at stop codon.

```
p dna_translate("ATGGCAGAATAA") # MAE
```

`dnbinom`

`dnbinom` — negative binomial PMF. Args: k, size, prob.

```
p dnbinom(3, 5, 0.5) # P(3 failures before 5 successes)
```

`dnds_ratio`

`dnds_ratio` — ratio of `dnds`. Category: **bioinformatics deep**.

```
p dnds_ratio $x
```

`dnorm`

`dnorm` — operation `dnorm`. Category: **r / scipy distributions and tests**.

```
p dnorm $x
```

`dnorm2`

`dnorm2` — operation `dnorm2`. Category: **blas / lapack**.

```
p dnorm2 $x
```

`do_call`

`do_call` — call a function with args from a list. Like R's `do.call()`.

```
val $result = docall(fn { _0 + _1 }, [3, 4]) # 7
```

`docs`

`docs(TOPIC)` — return the raw hover-doc string for `TOPIC` (the same source the `stryke docs` browser renders). `TOPIC` may be any primary builtin name, alias spelling, keyword, operator, or sigil-prefixed reflection-hash name (`%stryke::builtins`, `~>`, `$_`, ...). Returns `undef` when no doc page exists for the topic.

With multiple args, returns a hashref of `{ name => doc }` so a list of topics round-trips in one call. With no args, returns an arrayref of every documented topic name (sorted, deduped) — the in-language equivalent of `stryke docs --list`.

```
p docs("pmap")           # raw markdown doc string for pmap
p docs("tj")              # alias spellings work too
var %d = %{docs(qw(pmap pgrep psort))}
p scalar @{docs()}        # total count of documented topics
docs("foo") // p "no such topic"
```

See also: `stryke docs` CLI subcommand (interactive TUI pager), `%stryke::descriptions` (one-line summaries), `%stryke::builtins` (name → category).

doctor

doctor (alias **health**) — runtime diagnostic report. Prints version + binary path, runtime flags (`--compat`, `--no-interop`, `bigint`, `strict_vars`, `strict_refs`), environment overrides (`STRYKE_NO_TTY`, `STRYKE_NO_JIT`, `RUST_LOG`, ...), reflection counts (`%b` primaries, `%all` spellings, categories, list-builtin extras), concurrency (logical CPUs + rayon pool size), home/cache/config dirs, toolchain availability (`rustc/cargo/perl/git/ig`), and a battery of sanity checks. Returns the warning count as an integer (0 = healthy).

```
doctor          # full report
var $w = doctor
exit($w == 0 ? 0 : 1) # CI-friendly health gate
```

dodecahedral

dodecahedral(*n*) → int — $n \cdot (3n-1) \cdot (3n-2) / 2$.

dodecahedral_number

dodecahedral_number — operation **dodecahedral number**. Category: **misc**.

```
p dodecahedral_number $x
```

dodgson_swap_count

dodgson_swap_count — count of **dodgson swap**. Category: **climate, fluids, atmospheric**.

```
p dodgson_swap_count $x
```

dogleg_step

dogleg_step — single-step update of **dogleg**. Category: **more extensions (2)**.

```
p dogleg_step $x
```

dollarde

dollarde — operation **dollarde**. Category: **b81-misc-utility**.

```
p dollarde $x
```

dollarfr

dollarfr — operation **dollarfr**. Category: **b81-misc-utility**.

```
p dollarfr $x
```

dom_tree_idom

dom_tree_idom — operation **dom tree idom**. Category: **compilers / parsing**.

```
p dom_tree_idom $x
```

dominance_frontier

dominance_frontier — operation **dominance frontier**. Category: **compilers / parsing**.

```
p dominance_frontier $x
```

dominating_set_greedy

`dominating_set_greedy` — operation `dominating set greedy`. Category: **networkx graph algorithms**.

```
p dominating_set_greedy $x
```

dominating_set_greedy_step

`dominating_set_greedy_step` — single-step update of `dominating set greedy`. Category: **combinatorial optimization, scheduling**.

```
p dominating_set_greedy_step $x
```

dominating_set_lp_bound

`dominating_set_lp_bound` — operation `dominating set lp bound`. Category: **combinatorial optimization, scheduling**.

```
p dominating_set_lp_bound $x
```

donaldson_invariant

`donaldson_invariant` — operation `donaldson invariant`. Category: **econometrics**.

```
p donaldson_invariant $x
```

donchian_lower

`donchian_lower`(`lows`, `period=20`) → `array` — rolling min of `lows` over `period`.

```
p donchian_lower(\@lo, 20)
```

donchian_upper

`donchian_upper`(`highs`, `period=20`) → `array` — rolling max of `highs` over `period`. Used for breakout systems (Turtle traders).

```
p donchian_upper(\@hi, 20)
```

doppler

`doppler_frequency`(`$source_freq`, `$source_vel`, `$observer_vel`) (alias `doppler`) — observed frequency with Doppler effect. Uses speed of sound (343 m/s).

```
p doppler(440, -30, 0) # ~482 Hz (ambulance approaching at 30 m/s)
p doppler(440, 30, 0)  # ~405 Hz (ambulance receding)
```

doppler_classical

`doppler_classical` — operation `doppler classical`. Category: **em / optics / relativity**.

```
p doppler_classical $x
```

doppler_relativistic

`doppler_relativistic` — operation `doppler relativistic`. Category: **misc**.

```
p doppler_relativistic $x
```

dopri5_combine

dopri5_combine — operation `dopri5 combine`. Category: **ode advanced**.

```
p dopri5_combine $x
```

dorglq

dorglq — operation `dorglq`. Category: **blas / lapack**.

```
p dorglq $x
```

dorgqr

dorgqr — operation `dorgqr`. Category: **blas / lapack**.

```
p dorgqr $x
```

dot_case

dot_case — operation `dot case`. Category: **string**.

```
p dot_case $x
```

dot_product

dot_product(*a*, *b*) → *number* — $\sum a_i \cdot b_i$ (alias `dotp`).

dotp

dotp — operation `dotp`. Alias for `dot_product`. Category: **extras**.

```
p dotp $x
```

double

double — operation `double`. Category: **trivial numeric / predicate builtins**.

```
p double $x
```

double_auction_step

double_auction_step — single-step update of `double auction`. Category: **climate, fluids, atmospheric**.

```
p double_auction_step $x
```

double_each

double_each — operation `double each`. Category: **trivial numeric helpers**.

```
p double_each $x
```

double_exponential_smoothing

double_exponential_smoothing(*series*, *alpha*=0.3, *beta*=0.1) → *array* — Holt's linear trend method: level + trend components updated each step.

```
p double_exponential_smoothing(@xs, 0.3, 0.1)
```

double_factorial

`double_factorial` — operation `double factorial`. Category: **math formulas**.

```
p double_factorial $x
```

double_layer_capacitance

`double_layer_capacitance` — operation `double layer capacitance`. Category: **electrochemistry, batteries, fuel cells**.

```
p double_layer_capacitance $x
```

double_metaphone

`double_metaphone` — operation `double metaphone`. Category: **encoding / phonetics**.

```
p double_metaphone $x
```

double_metaphone_primary

`double_metaphone_primary` — operation `double metaphone primary`. Category: **test runner**.

```
p double_metaphone_primary $x
```

double_metaphone_secondary

`double_metaphone_secondary` — operation `double metaphone secondary`. Category: **test runner**.

```
p double_metaphone_secondary $x
```

double_q_learning_step

`double_q_learning_step` — single-step update of `double q learning`. Category: **electrochemistry, batteries, fuel cells**.

```
p double_q_learning_step $x
```

doubling_time_growth

`doubling_time_growth` — operation `doubling time growth`. Category: **epidemiology / public health**.

```
p doubling_time_growth $x
```

downcase_each

`downcase_each` — operation `downcase each`. Category: **trivial numeric helpers**.

```
p downcase_each $x
```

downto

`downto` — operation `downto`. Category: **python/ruby stdlib**.

```
p downto $x
```

doy

`doy` — operation `doy`. Alias for `day_of_year`. Category: **extended stdlib**.

```
p doy $x
```

dpayback

`dpayback` — operation `dpayback`. Alias for `discounted_payback`. Category: **finance (extended)**.

```
p dpayback $x
```

dpbsv

`dpbsv` — operation `dpbsv`. Category: **blas / lapack**.

```
p dpbsv $x
```

dpll_clause_learning

`dpll_clause_learning` — operation `dpll clause learning`. Category: **logic, proof, sat/smt, type theory**.

```
p dpll_clause_learning $x
```

dpois

`dpois` — operation `dpois`. Category: **extras**.

```
p dpois $x
```

dpotrf

`dpotrf` — operation `dpotrf`. Category: **blas / lapack**.

```
p dpotrf $x
```

dpss_window

`dpss_window` — operation `dpss window`. Category: **economics + game theory**.

```
p dpss_window $x
```

dr

`dr` — operation `dr`. Category: **filesystem extensions**.

```
p dr $x
```

drag_force

`drag_force` (alias `fdrag`) computes aerodynamic drag: $F = 0.5 \cdot C_d \cdot \rho \cdot A \cdot v^2$. Args: `drag_coeff`, `air_density`, `area`, `velocity`.

```
p fdrag(0.47, 1.225, 0.01, 30) # drag on a ball at 30 m/s
```

drag_force_quadratic

`drag_force_quadratic` — operation `drag force quadratic`. Category: **misc**.

```
p drag_force_quadratic $x
```


drnk

`drnk` — operation `drnk`. Alias for `dense_rank`. Category: **extended stdlib**.

```
p drnk $x
```

drop_at

`drop_at` — operation `drop at`. Category: **misc**.

```
p drop_at $x
```

drop_every

`drop_every` — operation `drop every`. Category: **list helpers**.

```
p drop_every $x
```

drop_last

`drop_last` — operation `drop last`. Category: **file stat / path**.

```
p drop_last $x
```

drop_n

`drop_n` — operation `drop n`. Category: **collection helpers (trivial)**.

```
p drop_n $x
```

drot

`drot` — operation `drot`. Category: **blas / lapack**.

```
p drot $x
```

drotg

`drotg` — operation `drotg`. Category: **blas / lapack**.

```
p drotg $x
```

dsamp

`dsamp` — operation `dsamp`. Alias for `downsample`. Category: **dsp / signal (extended)**.

```
p dsamp $x
```

dscal

`dscal` — operation `dscal`. Category: **blas / lapack**.

```
p dscal $x
```

dsyevd

`dsyevd` — operation `dsyevd`. Category: **blas / lapack**.

```
p dsyevd $x
```

dsyrk

`dsyrk` — operation `dsyrk`. Category: **blas / lapack**.

```
p dsyrk $x
```

dt

`dt` — operation `dt`. Category: **r / scipy distributions and tests**.

```
p dt $x
```

dtbsv

`dtbsv` — operation `dtbsv`. Category: **blas / lapack**.

```
p dtbsv $x
```

dte

`dte` — operation `dte`. Alias for `datetime_from_epoch`. Category: **date / time**.

```
p dte $x
```

dtf

`dtf` — operation `dtf`. Alias for `datetime_strftime`. Category: **date / time**.

```
p dtf $x
```

dtrsm

`dtrsm` — operation `dtrsm`. Category: **blas / lapack**.

```
p dtrsm $x
```

dtrsv

`dtrsv` — operation `dtrsv`. Category: **blas / lapack**.

```
p dtrsv $x
```

dump

`dump` — operation `dump`. Category: **process / system**.

```
p dump $x
```

dunif

`dunif` — uniform PDF. Args: `x`, `a`, `b`.

```
p dunif(0.5, 0, 1) # 1.0
```

duped

`uplicated` — boolean array: 1 if element appeared earlier in the vector, 0 otherwise. Like R's `uplicated()`.

```
p duplicated([1,2,3,2,1]) # [0,0,0,1,1]
```

duplicate_count

`duplicate_count` — count of `duplicate`. Category: **list helpers**.

```
p duplicate_count $x
```

durand_kerner

`durand_kerner` — operation `durand kerner`. Alias for `polynomial_roots_dk`. Category: **misc**.

```
p durand_kerner $x
```

duration_macaulay

`duration_macaulay(cashflows, rate)` → `number` — weighted average time to receive cashflows, weighted by present value. Measured in periods.

```
p duration_macaulay([100,100,1100], 0.05)
```

duration_modified

`duration_modified(cashflows, rate)` → `number` — `macaulay / (1 + rate)`. Direct linear sensitivity to yield change.

```
p duration_modified([100,100,1100], 0.05)
```

durbin_watson_d

`durbin_watson_d` — operation `durbin watson d`. Category: **statsmodels**.

```
p durbin_watson_d $x
```

dutch_auction

`dutch_auction` — operation `dutch auction`. Category: **economics + game theory**.

```
p dutch_auction $x
```

dutch_auction_step

`dutch_auction_step` — single-step update of `dutch auction`. Category: **climate, fluids, atmospheric**.

```
p dutch_auction_step $x
```

dweibull

`dweibull` — operation `dweibull`. Category: **extras**.

```
p dweibull $x
```

dynamics_db_level

`dynamics_db_level` — operation `dynamics db level`. Category: **music theory**.

```
p dynamics_db_level $x
```

each_cons

`each_cons` — operation `each_cons`. Category: *python/ruby stdlib*.

```
p each_cons $x
```

each_slice

`each_slice` — operation `each_slice`. Category: *python/ruby stdlib*.

```
p each_slice $x
```

each_with_index

`each_with_index` — index of `each_with`. Category: *functional / iterator*.

```
p each_with_index $x
```

each_with_object

`each_with_object` — operation `each_with_object`. Category: *additional missing stdlib functions*.

```
p each_with_object $x
```

eadie_hofstee_y

`eadie_hofstee_y` — operation `eadie_hofstee_y`. Category: *misc*.

```
p eadie_hofstee_y $x
```

ean13_check

`ean13_check` — operation `ean13_check`. Category: *b82-misc-utility*.

```
p ean13_check $x
```

earley_complete

`earley_complete` — operation `earley_complete`. Category: *compilers / parsing*.

```
p earley_complete $x
```

earley_predict

`earley_predict` — operation `earley_predict`. Category: *compilers / parsing*.

```
p earley_predict $x
```

earley_scan

`earley_scan` — operation `earley_scan`. Category: *compilers / parsing*.

```
p earley_scan $x
```

earth_mass

`earth_mass` (alias `mearth`) — $M_{\oplus} \approx 5.972 \times 10^{24}$ kg. Mass of Earth.

```
p mearth      # 5.972167867e24 kg
```

earth_mover_1d

`earth_mover_1d(a, b)` → `number` — Wasserstein-1 distance between two histograms via cumulative sum.

earth_radius

`earth_radius` — operation `earth radius`. Category: **physics constants**.

```
p earth_radius $x
```

easter_gregorian_year

`easter_gregorian_year` — operation `easter gregorian year`. Category: **calendrical algorithms**.

```
p easter_gregorian_year $x
```

easter_julian_year

`easter_julian_year` — operation `easter julian year`. Category: **calendrical algorithms**.

```
p easter_julian_year $x
```

easter_orthodox_year

`easter_orthodox_year` — operation `easter orthodox year`. Category: **calendrical algorithms**.

```
p easter_orthodox_year $x
```

easter_orthodox_year_2

`easter_orthodox_year_2` — operation `easter orthodox year 2`. Category: **b82-misc-utility**.

```
p easter_orthodox_year_2 $x
```

easter_sunday

`easter_sunday` — operation `easter sunday`. Category: **misc**.

```
p easter_sunday $x
```

easter_western

`easter_western` — operation `easter western`. Category: **b82-misc-utility**.

```
p easter_western $x
```

ec_point_add

`ec_point_add(p1, p2, a, prime)` → `[x, y]` — affine elliptic-curve point addition (placeholder uses Euclidean addition).

ec_point_double

`ec_point_double(p)` → `[x, y]` — point doubling (simplified).

`ecc_point_double`

`ecc_point_double` — operation `ecc point double`. Category: *cryptanalysis & number theory deep*.

```
p ecc_point_double $x
```

`ecdf`

`ecdf` — empirical CDF: proportion of data $\leq x$. Like R's `ecdf()`.

```
p ecdf([1,2,3,4,5], 3) # 0.6 (3 of 5 values  $\leq 3$ )
```

`echarge`

`echarge` — operation `echarge`. Alias for `elementary_charge`. Category: *constants*.

```
p echarge $x
```

`eclipse_magnitude`

`eclipse_magnitude` — operation `eclipse magnitude`. Category: *astronomy / astrometry*.

```
p eclipse_magnitude $x
```

`ecliptic_obliquity`

`ecliptic_obliquity` — operation `ecliptic obliquity`. Category: *misc*.

```
p ecliptic_obliquity $x
```

`ecliptic_to_equatorial`

`ecliptic_to_equatorial` — convert from `ecliptic` to `equatorial`. Category: *more extensions*.

```
p ecliptic_to_equatorial $value
```

`econs`

`econs` — operation `econs`. Alias for `each_cons`. Category: *python/ruby stdlib*.

```
p econs $x
```

`ed25519_keypair_simple`

`ed25519_keypair_simple(seed) → {private, public}` — SHA-512-derived keypair (demo only, not a real Ed25519 implementation).

`ed25519_sign_simple`

`ed25519_sign_simple(msg, private_key) → string` — SHA-512 of `private_key || msg`, base64-encoded.

`ed25519_verify_simple`

`ed25519_verify_simple(msg, public_key, sig) → 0|1` — companion verify.

eddington_luminosity

`eddington_luminosity` — operation `eddington luminosity`. Category: **misc**.

```
p eddington_luminosity $x
```

edge_coloring_vizing_step

`edge_coloring_vizing_step` — single-step update of `edge coloring vizing`. Category: **combinatorial optimization, scheduling**.

```
p edge_coloring_vizing_step $x
```

edge_connectivity

`edge_connectivity` — operation `edge connectivity`. Category: **networkx graph algorithms**.

```
p edge_connectivity $x
```

edge_detect_kernel

`edge_detect_kernel` — convolution kernel of `edge detect`. Category: **more extensions**.

```
p edge_detect_kernel $x
```

edgeworth_box_alloc

`edgeworth_box_alloc` — operation `edgeworth box alloc`. Category: **economics + game theory**.

```
p edgeworth_box_alloc $x
```

edmonds_karp_bfs

`edmonds_karp_bfs` — operation `edmonds karp bfs`. Category: **networkx graph algorithms**.

```
p edmonds_karp_bfs $x
```

edmonds_karp_max_flow

`edmonds_karp_max_flow(capacity_matrix, source, sink) → number` — max-flow via Edmonds-Karp (BFS-based Ford-Fulkerson).

```
p edmonds_karp_max_flow($cap, 0, 5)
```

edmonds_karp_step

`edmonds_karp_step` — single-step update of `edmonds karp`. Category: **combinatorial optimization, scheduling**.

```
p edmonds_karp_step $x
```

effect

`effect` — operation `effect`. Category: **b81-misc-utility**.

```
p effect $x
```

effective_pop_size

`effective_pop_size` — operation `effective pop size`. Category: **biology / ecology**.

```
p effective_pop_size $x
```

efficiency_ratio

`efficiency_ratio` — ratio of `efficiency`. Category: **misc**.

```
p efficiency_ratio $x
```

efield

`efield` — operation `efield`. Alias for `electric_field`. Category: **physics formulas**.

```
p efield $x
```

efield_point

`efield_point` — operation `efield point`. Category: **em / optics / relativity**.

```
p efield_point $x
```

egalitarian_solution

`egalitarian_solution` — operation `egalitarian solution`. Category: **climate, fluids, atmospheric**.

```
p egalitarian_solution $x
```

egalitarian_split

`egalitarian_split` — operation `egalitarian split`. Category: **climate, fluids, atmospheric**.

```
p egalitarian_split $x
```

egarch_step

`egarch_step` — single-step update of `egarch`. Category: **more extensions (2)**.

```
p egarch_step $x
```

ei

`ei(x) → number` — exponential integral $Ei(x) = -\int_{-x}^{\infty} e^{-t} / t \, dt$. Uses series + log expansion.

```
p ei(1) # -1.895
```

eig

`matrix_eigenvalues` (aliases `meig`, `eig`) computes eigenvalues of a square matrix via QR iteration. Returns an array of eigenvalues.

```
val @eigs = @{eig([[2,1],[1,2]])} # [3, 1]
```


einstein_dilaton_step

`einstein_dilaton_step` — single-step update of `einstein dilaton`. Category: *electrochemistry, batteries, fuel cells*.

```
p einstein_dilaton_step $x
```

einstein_ring_radius

`einstein_ring_radius` — operation `einstein ring radius`. Category: *cosmology / gr / flrw*.

```
p einstein_ring_radius $x
```

einstein_tensor_step

`einstein_tensor_step` — single-step update of `einstein tensor`. Category: *electrochemistry, batteries, fuel cells*.

```
p einstein_tensor_step $x
```

either

`either` — operation `either`. Category: *boolean combinators*.

```
p either $x
```

ekf_jacobian_step

`ekf_jacobian_step` — single-step update of `ekf jacobian`. Category: *electrochemistry, batteries, fuel cells*.

```
p ekf_jacobian_step $x
```

ekman_layer_depth

`ekman_layer_depth` — operation `ekman layer depth`. Category: *climate, fluids, atmospheric*.

```
p ekman_layer_depth $x
```

ekman_pumping_step

`ekman_pumping_step` — single-step update of `ekman pumping`. Category: *climate, fluids, atmospheric*.

```
p ekman_pumping_step $x
```

elastic_collision_1d

`elastic_collision_1d(m1, v1, m2, v2) → [v1', v2']` — 1D elastic collision (kinetic energy preserved).

```
p elastic_collision_1d(1, 2, 1, 0) # [0, 2]
```

electric_field

`electric_field` — operation `electric field`. Category: *physics formulas*.

```
p electric_field $x
```

electrochem_impedance_z

`electrochem_impedance_z` — operation `electrochem impedance z`. Category: *electrochemistry, batteries, fuel cells*.

```
p electrochem_impedance_z $x
```

electrochemiluminescence_yield

`electrochemiluminescence_yield` — operation `electrochemiluminescence yield`. Category: **electrochemistry, batteries, fuel cells**.

```
p electrochemiluminescence_yield $x
```

electrocrystallization_step

`electrocrystallization_step` — single-step update of `electrocrystallization`. Category: **electrochemistry, batteries, fuel cells**.

```
p electrocrystallization_step $x
```

electrode_potential_step

`electrode_potential_step` — single-step update of `electrode potential`. Category: **electrochemistry, batteries, fuel cells**.

```
p electrode_potential_step $x
```

electrolysis_mass

`electrolysis_mass` — operation `electrolysis mass`. Category: **chemistry & biochemistry**.

```
p electrolysis_mass $x
```

electrolyte_decomposition_temp

`electrolyte_decomposition_temp` — operation `electrolyte decomposition temp`. Category: **electrochemistry, batteries, fuel cells**.

```
p electrolyte_decomposition_temp $x
```

electrolyzer_voltage

`electrolyzer_voltage` — operation `electrolyzer voltage`. Category: **electrochemistry, batteries, fuel cells**.

```
p electrolyzer_voltage $x
```

electron_mass

`electron_mass` — operation `electron mass`. Category: **constants**.

```
p electron_mass $x
```

electroosmotic_velocity

`electroosmotic_velocity` — operation `electroosmotic velocity`. Category: **electrochemistry, batteries, fuel cells**.

```
p electroosmotic_velocity $x
```

elem_at

`elem_at` — operation `elem at`. Category: **list helpers**.

```
p elem_at $x
```

elem_index

`elem_index` — index of `elem`. Category: *additional missing stdlib functions*.

```
p elem_index $x
```

elem_indices

`elem_indices` — operation `elem indices`. Category: *additional missing stdlib functions*.

```
p elem_indices $x
```

elem_of

`elem_of` — operation `elem of`. Category: *haskell list functions*.

```
p elem_of $x
```

elementary_charge

`elementary_charge` — operation `elementary charge`. Category: *constants*.

```
p elementary_charge $x
```

elf_header_read

`elf_header_read(hex_bytes) → hash` — parse ELF `e_ident` + machine → {`class`, `endianness`, `machine`}.

elgamal_encrypt

`elgamal_encrypt` — encrypt of `elgamal`. Category: *cryptanalysis & number theory deep*.

```
p elgamal_encrypt $x
```

elias_delta_code

`elias_delta_code` — operation `elias delta code`. Category: *electrochemistry, batteries, fuel cells*.

```
p elias_delta_code $x
```

elias_gamma_code

`elias_gamma_code` — operation `elias gamma code`. Category: *electrochemistry, batteries, fuel cells*.

```
p elias_gamma_code $x
```

elidx

`elidx` — operation `elidx`. Alias for `elem_index`. Category: *additional missing stdlib functions*.

```
p elidx $x
```

elidxs

`elidxs` — operation `elidxs`. Alias for `elem_indices`. Category: *additional missing stdlib functions*.

```
p elidxs $x
```

ellip_lp

`ellip_lp` — operation `ellip lp`. Category: *economics + game theory*.

```
p ellip_lp $x
```

ellipse

`ellipse` — operation `ellipse`. Category: *numpy + scipy.special*.

```
p ellipse $x
```

ellipk

`ellipk` — operation `ellipk`. Category: *numpy + scipy.special*.

```
p ellipk $x
```

ellipperim

`ellipperim` — operation `ellipperim`. Alias for `ellipse_perimeter`. Category: *geometry (extended)*.

```
p ellipperim $x
```

ellipse_area

`ellipse_area` — operation `ellipse area`. Category: *complex / geom / color / trig*.

```
p ellipse_area $x
```

ellipse_perimeter

`ellipse_perimeter($x, $y)` (alias `ellper`) — approximates the perimeter of an ellipse with semi-axes a and b. Uses Ramanujan's approximation.

```
p ellper(5, 3)    # ~25.5
p ellper(10, 10)  # ~62.8 (circle)
```

ellipsis

`ellipsis` — operation `ellipsis`. Category: *string quote / escape*.

```
p ellipsis $x
```

ellipsoid_surface_approx

`ellipsoid_surface_approx` — operation `ellipsoid surface approx`. Category: *geometry / topology*.

```
p ellipsoid_surface_approx $x
```

ellipsoid_volume

`ellipsoid_volume` — operation `ellipsoid volume`. Category: *geometry / topology*.

```
p ellipsoid_volume $x
```

elo_expected

`elo_expected` — operation `elo expected`. Category: **astronomy / astrometry**.

```
p elo_expected $x
```

elo_update

`elo_update` — update step of `elo`. Category: **astronomy / astrometry**.

```
p elo_update $x
```

elof

`elof` — operation `elof`. Alias for `elem_of`. Category: **haskell list functions**.

```
p elof $x
```

elu

`elu` applies the Exponential Linear Unit: x if $x \geq 0$, $\alpha \cdot (e^x - 1)$ otherwise.

```
p elu(1)      # 1
p elu(-1)     # -0.632
```

em_frequency

`em_frequency` — operation `em frequency`. Category: **em / optics / relativity**.

```
p em_frequency $x
```

em_intensity

`em_intensity` — operation `em intensity`. Category: **em / optics / relativity**.

```
p em_intensity $x
```

em_step

`em_step` — single-step update of `em`. Category: **ode advanced**.

```
p em_step $x
```

em_wavelength

`em_wavelength` — operation `em wavelength`. Category: **em / optics / relativity**.

```
p em_wavelength $x
```

ema

`ema(prices, period=10) → array` — exponential moving average with smoothing factor $\alpha = 2 / (\text{period} + 1)$. Seeded with the first sample so output length equals input length.

```
p ema([10,11,12,13,14], 3)
```

email_domain

`email_domain` — operation `email domain`. Category: **url / email parts**.

```
p email_domain $x
```

email_local

`email_local` — operation `email local`. Category: **url / email parts**.

```
p email_local $x
```

emass

`emass` — operation `emass`. Alias for `electron_mass`. Category: **constants**.

```
p emass $x
```

emavg

`emavg` — operation `emavg`. Alias for `exponential_moving_average`. Category: **math / numeric extras**.

```
p emavg $x
```

embed_text

`embed_text(text, dim=128) → array` — deterministic hashing-trick text embedding (testing/non-ML use).

embed_ts

`embed_ts` — time-delay embedding. Converts a time series into a matrix of lagged values. Like R's `embed()`.

```
p embed_ts([1,2,3,4,5], 3) # [[3,2,1],[4,3,2],[5,4,3]]
```

emboss_kernel

`emboss_kernel` — convolution kernel of `emboss`. Category: **more extensions**.

```
p emboss_kernel $x
```

emf_from_half_cells

`emf_from_half_cells` — operation `emf from half cells`. Category: **chemistry**.

```
p emf_from_half_cells $x
```

emissivity_grey_body

`emissivity_grey_body` — operation `emissivity grey body`. Category: **climate, fluids, atmospheric**.

```
p emissivity_grey_body $x
```

emitted_long_wave

`emitted_long_wave` — operation `emitted long wave`. Category: **climate, fluids, atmospheric**.

```
p emitted_long_wave $x
```

emojinum

`emojinum` — operation `emojinum`. Alias for `to_emoji_num`. Category: **encoding / phonetics**.

```
p emojinum $x
```

empc

`empc` — operation `empc`. Alias for `empty_clj`. Category: **algebraic match**.

```
p empc $x
```

emptyArray

`emptyArray` — operation `emptyArray`. Alias for `listOf`. Category: **kotlin collections**.

```
p emptyArray $x
```

emptyList

`emptyList` — operation `emptyList`. Alias for `listOf`. Category: **kotlin collections**.

```
p emptyList $x
```

emptyMap

`emptyMap` — operation `emptyMap`. Alias for `mapOf`. Category: **kotlin collections**.

```
p emptyMap $x
```

emptySequence

`emptySequence` — operation `emptySequence`. Alias for `listOf`. Category: **kotlin collections**.

```
p emptySequence $x
```

emptySet

`emptySet` — operation `emptySet`. Alias for `setOf`. Category: **kotlin collections**.

```
p emptySet $x
```

empty_clj

`empty_clj` — operation `empty_clj`. Category: **algebraic match**.

```
p empty_clj $x
```

empty_count

`empty_count` — count of `empty`. Category: **counters**.

```
p empty_count $x
```

encode_base

`encode_base` — operation `encode_base`. Category: **apl/j/k array primitives**.

```
p encode_base $x
```

encode_pair

`encode_pair` — operation `encode pair`. Category: *logic, proof, sat/smt, type theory*.

```
p encode_pair $x
```

encode_succ

`encode_succ` — operation `encode succ`. Category: *logic, proof, sat/smt, type theory*.

```
p encode_succ $x
```

end_of_day

`end_of_day` — operation `end of day`. Category: *extended stdlib*.

```
p end_of_day $x
```

endgrent

`endgrent` — operation `endgrent`. Category: *posix metadata*.

```
p endgrent $x
```

endhostent

`endhostent` — operation `endhostent`. Category: *posix metadata*.

```
p endhostent $x
```

endianness

`endianness` — byte order of the platform: "little" or "big". Compile-time constant.

```
p endianness # little
```

endnetent

`endnetent` — operation `endnetent`. Category: *posix metadata*.

```
p endnetent $x
```

endowment_combined_a

`endowment_combined_a` — operation `endowment combined a`. Category: *actuarial science*.

```
p endowment_combined_a $x
```

endowment_pure_e

`endowment_pure_e` — operation `endowment pure e`. Category: *actuarial science*.

```
p endowment_pure_e $x
```

endprotoent

`endprotoent` — operation `endprotoent`. Category: *posix metadata*.

```
p endprotoent $x
```


endpwent

`endpwent` — operation `endpwent`. Category: **posix metadata**.

```
p endpwent $x
```

ends_with

`ends_with` — operation `ends with`. Category: **trivial string ops**.

```
p ends_with $x
```

ends_with_any

`ends_with_any` — operation `ends with any`. Category: **string**.

```
p ends_with_any $x
```

endservent

`endservent` — operation `endservent`. Category: **posix metadata**.

```
p endservent $x
```

energy

`energy(\@signal)` — computes the total energy of a signal as the sum of squared samples. Useful for audio analysis and feature extraction.

```
p energy(\@signal) # total signal energy
```

energy_density_battery

`energy_density_battery` — operation `energy density battery`. Category: **electrochemistry, batteries, fuel cells**.

```
p energy_density_battery $x
```

energy_efficiency_cell

`energy_efficiency_cell` — operation `energy efficiency cell`. Category: **electrochemistry, batteries, fuel cells**.

```
p energy_efficiency_cell $x
```

engel_curve

`engel_curve` — operation `engel curve`. Category: **economics + game theory**.

```
p engel_curve $x
```

engle_granger_step

`engle_granger_step` — single-step update of `engle granger`. Category: **econometrics**.

```
p engle_granger_step $x
```

english_auction

`english_auction` — operation `english auction`. Category: **economics + game theory**.

```
p english_auction $x
```

english_auction_step

`english_auction_step` — single-step update of `english_auction`. Category: **climate, fluids, atmospheric**.

```
p english_auction_step $x
```

english_chi2

`english_chi2` — operation `english_chi2`. Category: **cryptography deep**.

```
p english_chi2 $x
```

english_likeness

`english_likeness` — operation `english_likeness`. Category: **cryptanalysis & number theory deep**.

```
p english_likeness $x
```

enso_index_step

`enso_index_step` — single-step update of `enso_index`. Category: **climate, fluids, atmospheric**.

```
p enso_index_step $x
```

ensure_prefix

`ensure_prefix` — operation `ensure_prefix`. Category: **path helpers**.

```
p ensure_prefix $x
```

ensure_suffix

`ensure_suffix` — operation `ensure_suffix`. Category: **path helpers**.

```
p ensure_suffix $x
```

entab

`entab(s, width=8) → string` — compress leading runs of spaces into tabs at every multiple of `width`.

```
p entab("    hello", 4) # "\thello"
```

entanglement_entropy

`entanglement_entropy` — operation `entanglement_entropy`. Category: **quantum**.

```
p entanglement_entropy $x
```

enthalpy_reaction

`enthalpy_reaction` — operation `enthalpy_reaction`. Category: **chemistry**.

```
p enthalpy_reaction $x
```

entropy

`entropy` — operation `entropy`. Category: **math functions**.

```
p entropy $x
```

entropy_bits

`entropy_bits` — operation `entropy bits`. Category: **ml extensions**.

```
p entropy_bits $x
```

enum

`enum` declares a tagged-union / algebraic data type — a finite set of named variants, each optionally carrying a typed payload. Stryke enums pair with `match` for exhaustive pattern dispatch (the compiler errors on missing arms unless you write `_ =>`).

Declaration:

```
enum Sig { Hup, Int, Term, Kill }           # tag-only variants
enum Color { Red, Green, Blue }
enum ResultStatus { Ok => Int, Err => Str }   # variants with payload
enum Shape {                                # variants with payload + methods
  Circle => Float,
  Square => Float,
  fn area($self) {
    match ($self) {
      Shape::Circle($r) => 3.14159 * $r * $r,
      Shape::Square($s) => $s * $s,
    }
  }
}
```

Constructing variants:

```
my $s = Sig::Term           # tag-only construction
my $ok = ResultStatus::Ok(42) # payload construction
my $err = ResultStatus::Err("boom")
p $ok                       # ResultStatus::Ok(42)
```

Pattern matching:

```
fn handle($sig) {
  match ($sig) {
    Sig::Hup => "reload config",
    Sig::Int => "graceful shutdown",
    Sig::Term => "drain + stop",
    Sig::Kill => "(no handler - kernel reaps)",
  }
}
# Bind the payload in the arm:
match ($result) {
  ResultStatus::Ok($value) => p "got $value",
  ResultStatus::Err($msg) => die $msg,
}
```

Iteration / reflection:

```
for my $v (Sig::Hup, Sig::Int, Sig::Term, Sig::Kill) { p $v }
p Sig::values |> ep           # every variant
p ResultStatus::Ok(1)->variant # "Ok"
p ResultStatus::Ok(1)->payload # 1
```

Note: `EnumName::Variant` namespaces the variant tag; `EnumName::Variant(payload)` constructs one with data. The compiler tracks payload types per variant — `match` arms must bind a compatible pattern.

See also: `match`, `struct` (single-shape records), `class` (full OO).

env

`env` — operation `env`. Alias for `envelope`. Category: *dsp / signal (extended)*.

```
p env $x
```

env_get

`env_get` — environment-variable helper `get`. Category: *process / env*.

```
p env_get $x
```

env_has

`env_has` — environment-variable helper `has`. Category: *process / env*.

```
p env_has $x
```

env_keys

`env_keys` — environment-variable helper `keys`. Category: *process / env*.

```
p env_keys $x
```

env_pairs

`env_pairs` — environment-variable helper `pairs`. Category: *more process / system*.

```
p env_pairs $x
```

env_remove

`env_remove` — environment-variable helper `remove`. Category: *more process / system*.

```
p env_remove $x
```

env_set

`env_set` — environment-variable helper `set`. Category: *more process / system*.

```
p env_set $x
```

envelope

`envelope(\@signal)` (alias `hilbert_env`) — computes the amplitude envelope using Hilbert transform magnitude. Useful for amplitude modulation analysis.

```
val $env = envelope(\@signal)
# $env traces the amplitude of oscillations
```

envelope_amplitude

`envelope_amplitude` — envelope generator (ADSR-style) `amplitude`. Category: *economics + game theory*.

```
p envelope_amplitude $x
```

eod

`eod` — operation `eod`. Alias for `end_of_day`. Category: *extended stdlib*.

```
p eod $x
```

eof_from_concurrence

`eof_from_concurrence` — operation `eof from concurrence`. Category: *quantum mechanics deep*.

```
p eof_from_concurrence $x
```

epa_per_play

`epa_per_play` — operation `epa per play`. Category: *astronomy / astrometry*.

```
p epa_per_play $x
```

epoch

`epoch` — operation `epoch`. Alias for `unix_epoch`. Category: *now / timestamp*.

```
p epoch $x
```

epoch_ms

`epoch_ms` — operation `epoch ms`. Alias for `unix_epoch_ms`. Category: *now / timestamp*.

```
p epoch_ms $x
```

epotential_point

`epotential_point` — operation `epotential point`. Category: *em / optics / relativity*.

```
p epotential_point $x
```

eps

`eps` — operation `eps`. Alias for `epsilon`. Category: *math / numeric extras*.

```
p eps $x
```

epsilon

`epsilon` — operation `epsilon`. Category: *math / numeric extras*.

```
p epsilon $x
```

epsilon0

`vacuum_permittivity` (alias `epsilon0`) — $\varepsilon_0 \approx 8.854 \times 10^{-12}$ F/m. Electric constant, permittivity of free space.

```
p epsilon0 # 8.8541878128e-12
```

epsilon_extinction

`epsilon_extinction` — operation `epsilon extinction`. Category: *chemistry & biochemistry*.

```
p epsilon_extinction $x
```

epsilon_greedy_choose

`epsilon_greedy_choose(values, epsilon=0.1, seed=0) → int` — index of greedy action with probability $1-\epsilon$, else random.

eqidx

`eqidx` — operation `eqidx`. Alias for `equilibrium_index`. Category: *extended stdlib*.

```
p eqidx $x
```

eqrng

`eqrng` — operation `eqrng`. Alias for `equal_range`. Category: *extended stdlib*.

```
p eqrng $x
```

equal_range

`equal_range` — operation `equal range`. Category: *extended stdlib*.

```
p equal_range $x
```

equal_tempered_freq

`equal_tempered_freq` — operation `equal tempered freq`. Category: *music theory*.

```
p equal_tempered_freq $x
```

equation_of_time

`equation_of_time` — operation `equation of time`. Category: *misc*.

```
p equation_of_time $x
```

equatorial_to_galactic

`equatorial_to_galactic` — convert from `equatorial` to `galactic`. Category: *more extensions*.

```
p equatorial_to_galactic $value
```

equilibrium_index

`equilibrium_index` — index of `equilibrium`. Category: *extended stdlib*.

```
p equilibrium_index $x
```

equinox_spring

`equinox_spring` — operation `equinox spring`. Category: *b82-misc-utility*.

```
p equinox_spring $x
```

equirectangular_project

`equirectangular_project` — operation `equirectangular project`. Category: *more extensions*.

```
p equirectangular_project $x
```

equivalent_potential_temp

`equivalent_potential_temp` — operation `equivalent potential temp`. Category: **climate, fluids, atmospheric**.

```
p equivalent_potential_temp $x
```

era_plus

`era_plus` — operation `era plus`. Category: **astronomy / astrometry**.

```
p era_plus $x
```

erf_approx

`erf_approx($x)` — error function. Used in probability, statistics, and partial differential equations.

```
p erf(1)      # ~0.843
p erf(2)      # ~0.995
```

erfcx

`erfcx` — operation `erfcx`. Category: **numpy + scipy.special**.

```
p erfcx $x
```

erfi

`erfi` — operation `erfi`. Category: **numpy + scipy.special**.

```
p erfi $x
```

erosion_2d

`erosion_2d(image, size=3)` → `matrix` — morphological erosion with `size*size` structuring element.

```
p erosion_2d($binary_img, 3)
```

error_correction_step

`error_correction_step` — single-step update of `error correction`. Category: **econometrics**.

```
p error_correction_step $x
```

ertel_pv_step

`ertel_pv_step` — single-step update of `ertel pv`. Category: **climate, fluids, atmospheric**.

```
p ertel_pv_step $x
```

escape_json

`escape_json` — operation `escape json`. Category: **json helpers**.

```
p escape_json $x
```

escape_velocity

`escape_velocity($mass, $radius)` (alias `escvel`) — minimum velocity to escape gravitational field: $v = \sqrt{2GM/r}$. Defaults to Earth values.

```
p escvel                # ~11186 m/s (Earth)
p escvel(1.989e30, 6.96e8) # ~617 km/s (Sun)
```

escape_velocity_body

`escape_velocity_body` — operation `escape velocity body`. Category: **astronomy / astrometry**.

```
p escape_velocity_body $x
```

eslice

`eslice` — operation `eslice`. Alias for `each_slice`. Category: **python/ruby stdlib**.

```
p eslice $x
```

et_freq_ratio

`et_freq_ratio` — ratio of `et freq`. Category: **astronomy / music / color / units**.

```
p et_freq_ratio $x
```

etag_validate

`etag_validate` — operation `etag validate`. Category: **b81-misc-utility**.

```
p etag_validate $x
```

etd_rk2

`etd_rk2` — operation `etd rk2`. Category: **ode advanced**.

```
p etd_rk2 $x
```

eth_addr_check

`eth_addr_check` — operation `eth addr check`. Category: **b82-misc-utility**.

```
p eth_addr_check $x
```

ethiopic_from_fixed

`ethiopic_from_fixed` — operation `ethiopic from fixed`. Category: **calendrical algorithms**.

```
p ethiopic_from_fixed $x
```

etot

`etot` — operation `etot`. Alias for `euler_totient`. Category: **extended stdlib**.

```
p etot $x
```


euclidist

`euclidist` — operation `euclidist`. Alias for `euclidean_distance`. Category: **extras**.

```
p euclidist $x
```

euclidean_distance

`euclidean_distance(a, b) → number` — L2 distance.

euclidean_distance_nd

`euclidean_distance_nd` — operation `euclidean distance nd`. Category: **geometry / topology**.

```
p euclidean_distance_nd $x
```

eui48_to_eui64

`eui48_to_eui64` — convert from `eui48` to `eui64`. Category: **network / ip / cidr**.

```
p eui48_to_eui64 $value
```

eui64_from_mac

`eui64_from_mac` — operation `eui64 from mac`. Category: **network / ip / cidr**.

```
p eui64_from_mac $x
```

eui64_to_eui48

`eui64_to_eui48` — convert from `eui64` to `eui48`. Category: **network / ip / cidr**.

```
p eui64_to_eui48 $value
```

euler_char_complex

`euler_char_complex` — operation `euler char complex`. Category: **econometrics**.

```
p euler_char_complex $x
```

euler_characteristic

`euler_characteristic` — operation `euler characteristic`. Category: **geometry / topology**.

```
p euler_characteristic $x
```

euler_e

`euler_e` — operation `euler e`. Alias for `euler_number`. Category: **math / numeric extras**.

```
p euler_e $x
```

euler_lte

`euler_lte` — operation `euler lte`. Category: **ode advanced**.

```
p euler_lte $x
```

euler_mascheroni

`euler_mascheroni` (alias `gamma_const`) — Euler-Mascheroni constant $\gamma \approx 0.5772$. Appears in number theory and analysis.

```
p euler_mascheroni # 0.5772156649015329
```

euler_method

`euler_ode` (alias `euler_method`) solves an ODE $dy/dt = f(t,y)$ using the Euler method. Returns $[[t,y], \dots]$.

```
val $sol = euler_ode(fn { _1 }, 0, 1, 0.01, 100) # exponential growth
```

euler_number

`euler_number` — operation `euler number`. Category: **math / numeric extras**.

```
p euler_number $x
```

euler_totient

`euler_totient` — operation `euler totient`. Category: **extended stdlib**.

```
p euler_totient $x
```

eulerian_path_q

`eulerian_path_q` — operation `eulerian path q`. Category: **misc**.

```
p eulerian_path_q $x
```

ev_to_joules

`ev_to_joules` — convert from `ev` to `joules`. Category: **astronomy / music / color / units**.

```
p ev_to_joules $value
```

eval_rpn

`eval_rpn` — operation `eval rpn`. Category: **algorithms / puzzles**.

```
p eval_rpn $x
```

even

`even` — operation `even`. Category: **trivial numeric / predicate builtins**.

```
p even $x
```

event_horizon_check

`event_horizon_check` — operation `event horizon check`. Category: **electrochemistry, batteries, fuel cells**.

```
p event_horizon_check $x
```

every_nth

`every_nth` — operation `every nth`. Category: **collection helpers (trivial)**.

```
p every_nth $x
```

evolutionary_stable_strategy

`evolutionary_stable_strategy` — operation `evolutionary stable strategy`. Category: **climate, fluids, atmospheric**.

```
p evolutionary_stable_strategy $x
```

ew

`ew` — operation `ew`. Alias for `ends_with`. Category: **trivial string ops**.

```
p ew $x
```

ewma_smooth

`ewma_smooth` — operation `ewma smooth`. Category: **statsmodels**.

```
p ewma_smooth $x
```

ewo

`ewo` — operation `ewo`. Alias for `each_with_object`. Category: **additional missing stdlib functions**.

```
p ewo $x
```

ex_ante_value_check

`ex_ante_value_check` — operation `ex ante value check`. Category: **climate, fluids, atmospheric**.

```
p ex_ante_value_check $x
```

ex_post_value_check

`ex_post_value_check` — operation `ex post value check`. Category: **climate, fluids, atmospheric**.

```
p ex_post_value_check $x
```

exchange_current_density

`exchange_current_density` — operation `exchange current density`. Category: **electrochemistry, batteries, fuel cells**.

```
p exchange_current_density $x
```

exclude

`exclude` — operation `exclude`. Category: **iterator + string-distance extras**.

```
p exclude $x
```

exclude_first

`exclude_first` — operation `exclude first`. Category: **iterator + string-distance extras**.

```
p exclude_first $x
```

exclude_last

`exclude_last` — operation `exclude last`. Category: **iterator + string-distance extras**.

```
p exclude_last $x
```

exe

`exe` — operation `exe`. Category: **filesystem extensions**.

```
p exe $x
```

exec_script

`run(TARGET, ...args)` → exit code. Spawn a stryke script or external binary as a subprocess. Inherits stdin/stdout/stderr so output streams to the parent. Returns the child's exit code (`-1` on spawn failure).

Routing:

- `TARGET` ending in `.stk` → spawned via the current stryke binary (`$STRYKE_EXE script.stk ARGS...`)
- `TARGET` containing `/` → spawned as a direct path
- bare name → resolved via `PATH`

```
val $rc = run("echo", "hello") # 0
var $rc = run("./build.stk", "--release") # spawns isolated stryke proc
run("git", "status") == 0 or die "git failed"
```

Isolation: state defined in a script run via `run` does NOT leak into the caller (subprocess). Use `source` if you want the script's `our` vars / `fn` defs in your REPL.

executables

`executables` — operation `executables`. Category: **filesystem extensions**.

```
p executables $x
```

exists_key

`exists_key` — operation `exists key`. Category: **redis-flavour primitives**.

```
p exists_key $x
```

exp2

`exp2` — operation `exp2`. Category: **trivial numeric / predicate builtins**.

```
p exp2 $x
```

exp_euler_step

`exp_euler_step` — exponential helper `euler step`. Category: **ode advanced**.

```
p exp_euler_step $x
```

exp_golomb_code

`exp_golomb_code` — exponential helper `golomb code`. Category: **electrochemistry, batteries, fuel cells**.

```
p exp_golomb_code $x
```

exp_integral_e1

`exp_integral_e1` — exponential helper `integral e1`. Category: **special functions extra**.

```
p exp_integral_e1 $x
```

exp_smooth_double

`exp_smooth_double` — exponential helper `smooth double`. Category: **statsmodels**.

```
p exp_smooth_double $x
```

exp_smooth_simple

`exp_smooth_simple` — exponential helper `smooth simple`. Category: **statsmodels**.

```
p exp_smooth_simple $x
```

expanding_mean

`expanding_mean(series)` → `array` — cumulative running mean; output[i] is mean of `x[0..i]`.

```
p expanding_mean([1,2,3,4]) # [1.0, 1.5, 2.0, 2.5]
```

expanding_sum

`expanding_sum(series)` → `array` — cumulative running sum.

expansion_scalar_step

`expansion_scalar_step` — single-step update of `expansion scalar`. Category: **electrochemistry, batteries, fuel cells**.

```
p expansion_scalar_step $x
```

expected_coverage

`expected_coverage` — operation `expected coverage`. Category: **bioinformatics deep**.

```
p expected_coverage $x
```

expectiminimax_value

`expectiminimax_value(leaves, depth, chance_prob=0.5)` → `number` — minimax with intervening chance nodes (max → min → chance, repeating).

expenditure_function

`expenditure_function` — operation `expenditure function`. Category: **economics + game theory**.

```
p expenditure_function $x
```

experience_factor

`experience_factor` — factor of `experience`. Category: **actuarial science**.

```
p experience_factor $x
```

expint

`expint(n, x) → number` — generalised exponential integral $E_n(x) = \int_1^\infty e^{-xt}/t^n dt$.

```
p expint(1, 1)
```

expire

`expire` — operation `expire`. Category: *redis-flavour primitives*.

```
p expire $x
```

explore_exploit_epsilon

`explore_exploit_epsilon` — operation `explore` `exploit` `epsilon`. Category: *electrochemistry, batteries, fuel cells*.

```
p explore_exploit_epsilon $x
```

exponential_moving_average

`exponential_moving_average` — operation `exponential` `moving` `average`. Category: *math / numeric extras*.

```
p exponential_moving_average $x
```

exponential_pdf

`exponential_pdf` — operation `exponential` `pdf`. Category: *math formulas*.

```
p exponential_pdf $x
```

exponential_search

`exponential_search(sorted, target) → int` — doubles the range until target is bracketed, then binary-search. Good for unbounded streams.

exposure_value

`exposure_value(aperture, shutter, iso=100) → number` — $EV = \log_2(N^2 / t \cdot 100/ISO)$.

```
p exposure_value(8, 1/250, 100)
```

exppdf

`exppdf` — operation `exppdf`. Alias for `exponential_pdf`. Category: *math formulas*.

```
p exppdf $x
```

extended_gcd

`extended_gcd` — operation `extended` `gcd`. Category: *math / number theory extras*.

```
p extended_gcd $x
```

extends

`extends` specifies parent classes for inheritance. Child classes inherit all fields and methods from parents.

```
class Animal { name: Str }
class Dog extends Animal { breed: Str }
class Hybrid extends Dog, Cat { } # multiple inheritance
```

Inherited fields appear first in construction order. Methods are resolved child-first (override pattern).

exterior_derivative_one_form

`exterior_derivative_one_form` — operation `exterior derivative one form`. Category: **electrochemistry, batteries, fuel cells**.

```
p exterior_derivative_one_form $x
```

extract_between

`extract_between` — operation `extract between`. Category: **string quote / escape**.

```
p extract_between $x
```

extrinsic_principal_curv

`extrinsic_principal_curv` — operation `extrinsic principal curv`. Category: **electrochemistry, batteries, fuel cells**.

```
p extrinsic_principal_curv $x
```

eye

`eye` — operation `eye`. Alias for `identity_matrix`. Category: **matrix / linear algebra**.

```
p eye $x
```

eyring_k

`eyring_k` — operation `eyring k`. Category: **chemistry**.

```
p eyring_k $x
```

eyring_rate

`eyring_rate` — rate of `eyring`. Category: **chemistry & biochemistry**.

```
p eyring_rate $x
```

f

`f` — operation `f`. Category: **filesystem extensions**.

```
p f $x
```

f64_max

`f64_max` — maximum of `f64`. Category: **math / numeric extras**.

```
p f64_max $x
```

f64_min

`f64_min` — minimum of `f64`. Category: **math / numeric extras**.

```
p f64_min $x
```

f_beta

`f_beta` — operation `f beta`. Category: **ml extensions**.

```
p f_beta $x
```

f_pdf

`f_pdf` (alias `fpdf`) evaluates the F-distribution PDF at `x` with `d1` and `d2` degrees of freedom.

```
p fpdf(1.0, 5, 10)
```

f_statistic_pooled

`f_statistic_pooled` — operation `f statistic pooled`. Category: **econometrics**.

```
p f_statistic_pooled $x
```

f_to_c

`f_to_c` — convert from `f` to `c`. Category: **unit conversions**.

```
p f_to_c $value
```

f_to_k

`f_to_k` — convert from `f` to `k`. Category: **unit conversions**.

```
p f_to_k $value
```

face_detect_haar

`face_detect_haar` — operation `face detect haar`. Category: **pil/opencv image processing**.

```
p face_detect_haar $x
```

fact

`fact` — operation `fact`. Alias for `factorial`. Category: **trivial numeric / predicate builtins**.

```
p fact $x
```

factorial

`factorial` — operation `factorial`. Category: **trivial numeric / predicate builtins**.

```
p factorial $x
```

factorial2

`factorial2` — operation `factorial2`. Category: **numpy + scipy.special**.

```
p factorial2 $x
```


factorialk

`factorialk` — operation `factorialk`. Category: **numpy + scipy.special**.

```
p factorialk $x
```

fade_curve_exponential

`fade_curve_exponential` — operation `fade curve exponential`. Category: **extras**.

```
p fade_curve_exponential $x
```

fade_curve_linear

`fade_curve_linear` — operation `fade curve linear`. Category: **extras**.

```
p fade_curve_linear $x
```

fade_curve_logarithmic

`fade_curve_logarithmic` — operation `fade curve logarithmic`. Category: **extras**.

```
p fade_curve_logarithmic $x
```

faf

`faf` — operation `faf`. Alias for `fire_and_forget`. Category: **stress testing**.

```
p faf $x
```

fair_division_envy_free

`fair_division_envy_free` — release / destroy of `fair division envy`. Category: **climate, fluids, atmospheric**.

```
p fair_division_envy_free $x
```

falf

`falf` — operation `falf`. Alias for `filterfalse`. Category: **python/ruby stdlib**.

```
p falf $x
```

fallback

`fallback` — operation `fallback`. Category: **functional combinators**.

```
p fallback $x
```

falling_factorial

`falling_factorial` — operation `falling factorial`. Category: **math formulas**.

```
p falling_factorial $x
```

false_position

`false_position` — operation `false position`. Alias for `regula_falsi`. Category: **more extensions (2)**.

```
p false_position $x
```

falsy_count

`falsy_count` — count of `falsy`. Category: **counters**.

```
p falsy_count $x
```

fano_inequality_bound

`fano_inequality_bound` — operation `fano inequality bound`. Category: **electrochemistry, batteries, fuel cells**.

```
p fano_inequality_bound $x
```

faradaic_efficiency

`faradaic_efficiency` — operation `faradaic efficiency`. Category: **electrochemistry, batteries, fuel cells**.

```
p faradaic_efficiency $x
```

faraday

`faraday_constant` (alias `faraday`) — $F \approx 96485$ C/mol. Charge per mole of electrons.

```
p faraday # 96485.33212
```

faraday_efficiency_h2

`faraday_efficiency_h2` — operation `faraday efficiency h2`. Category: **electrochemistry, batteries, fuel cells**.

```
p faraday_efficiency_h2 $x
```

faraday_emf

`faraday_emf` — operation `faraday emf`. Category: **em / optics / relativity**.

```
p faraday_emf $x
```

faraday_mass_deposited

`faraday_mass_deposited` — operation `faraday mass deposited`. Category: **chemistry**.

```
p faraday_mass_deposited $x
```

farmhash_64

`farmhash_64` — operation `farmhash 64`. Category: **b81-misc-utility**.

```
p farmhash_64 $x
```

farthest_insertion_step

`farthest_insertion_step` — single-step update of `farthest insertion`. Category: **combinatorial optimization, scheduling**.

```
p farthest_insertion_step $x
```

fb

`fb` — operation `fb`. Alias for `fizzbuzz`. Category: **algorithms / puzzles**.

```
p fb $x
```

fbm_noise_2d

`fbm_noise_2d(x, y, octaves=4, lacunarity=2, gain=0.5)` → `number` — fractional Brownian motion: sum of octaves of Perlin noise.

fcntl

`fcntl` — operation `fcntl`. Category: `*ipc*`.

```
p fcntl $x
```

fcp

`fcp` — operation `fcp`. Alias for `from_code_point`. Category: `*extended stdlib*`.

```
p fcp $x
```

fdivop

`fdivop` — operation `fdivop`. Alias for `floor_div_op`. Category: `*extended stdlib*`.

```
p fdivop $x
```

feet_to_m

`feet_to_m` — convert from `feet` to `m`. Category: `*unit conversions*`.

```
p feet_to_m $value
```

feet_to_meters

`feet_to_meters` — convert from `feet` to `meters`. Category: `*astronomy / music / color / units*`.

```
p feet_to_meters $value
```

feiga

`feiga` — operation `feiga`. Alias for `feigenbaum_alpha`. Category: `*math constants*`.

```
p feiga $x
```

feigd

`feigenbaum_delta` (alias `feigd`) — Feigenbaum constant $\delta \approx 4.669$. Universal constant in chaos theory.

```
p feigd # 4.669201609102990
```

feigenbaum_alpha

`feigenbaum_alpha` — operation `feigenbaum_alpha`. Category: `*math constants*`.

```
p feigenbaum_alpha $x
```

feldspar_classify

`feldspar_classify` — operation `feldspar classify`. Category: *geology, seismology, mineralogy*.

```
p feldspar_classify $x
```

felsenstein_step

`felsenstein_step` — single-step update of `felsenstein`. Category: *bioinformatics deep*.

```
p felsenstein_step $x
```

felzenszwalb_segment

`felzenszwalb_segment` — operation `felzenszwalb segment`. Category: *pil/opencv image processing*.

```
p felzenszwalb_segment $x
```

fenwick_build

`fenwick_build` — Fenwick (binary indexed) tree op `build`. Category: *misc*.

```
p fenwick_build $x
```

fenwick_for

`fenwick_for` — Fenwick (binary indexed) tree op `for`. Category: *astronomy / astrometry*.

```
p fenwick_for $x
```

fenwick_new

`fenwick_new(n) → array` — Fenwick (BIT) tree of size `n`, all zeros. 1-indexed.

fenwick_query

`fenwick_query` — Fenwick (binary indexed) tree op `query`. Category: *misc*.

```
p fenwick_query $x
```

fenwick_query_prefix

`fenwick_query_prefix(tree, idx) → int` — sum of [1..idx].

fenwick_query_range

`fenwick_query_range(tree, l, r) → int` — sum of [l..r].

fenwick_update

`fenwick_update(tree, idx, delta) → array` — add `delta` to position `idx` (1-indexed).

```
my $f = fenwick_new(10)
$f = fenwick_update($f, 3, 5)
```

fermat_factor

`fermat_factor` — factor of `fermat`. Category: *cryptanalysis & number theory deep*.

```
p fermat_factor $x
```

fermi_dirac_int

`fermi_dirac_int` — operation `fermi dirac int`. Category: *special functions extra*.

```
p fermi_dirac_int $x
```

fermi_golden_rule

`fermi_golden_rule` — operation `fermi golden rule`. Category: *quantum mechanics deep*.

```
p fermi_golden_rule $x
```

fermi_normal_step

`fermi_normal_step` — single-step update of `fermi normal`. Category: *electrochemistry, batteries, fuel cells*.

```
p fermi_normal_step $x
```

ferrel_cell_step

`ferrel_cell_step` — single-step update of `ferrel cell`. Category: *climate, fluids, atmospheric*.

```
p ferrel_cell_step $x
```

fetch_val

`fetch_val` — operation `fetch val`. Category: *python/ruby stdlib*.

```
p fetch_val $x
```

ffact

`ffact` — operation `ffact`. Alias for `falling_factorial`. Category: *math formulas*.

```
p ffact $x
```

ffirst

`ffirst` — operation `ffirst`. Category: *algebraic match*.

```
p ffirst $x
```

ffs

`ffs` — operation `ffs`. Alias for `ffirst`. Category: *algebraic match*.

```
p ffs $x
```

fft_magnitude

`fft_magnitude` — Fast Fourier Transform helper `magnitude`. Category: *math / numeric extras*.

```
p fft_magnitude $x
```

fftconvolve_step

`fftconvolve_step` — single-step update of `fftconvolve`. Category: *economics + game theory*.

```
p fftconvolve_step $x
```

fftmag

`fftmag` — operation `fftmag`. Alias for `fft_magnitude`. Category: *math / numeric extras*.

```
p fftmag $x
```

fib

`fib` — operation `fib`. Alias for `fibonacci`. Category: *trivial numeric / predicate builtins*.

```
p fib $x
```

fibonacci

`fibonacci` — operation `fibonacci`. Category: *trivial numeric / predicate builtins*.

```
p fibonacci $x
```

fibonacci_code

`fibonacci_code` — operation `fibonacci code`. Category: *electrochemistry, batteries, fuel cells*.

```
p fibonacci_code $x
```

fibonacci_extension

`fibonacci_extension(high, low) → array` — extension targets at 127.2%, 161.8%, 261.8% beyond the retracement zone.

```
p fibonacci_extension(20, 10)
```

fibonacci_matrix

`fibonacci_matrix(n) → int` — n-th Fibonacci number via matrix exponentiation. $O(\log n)$.

```
p fibonacci_matrix(10) # 55
```

fibonacci_nth_fast

`fibonacci_nth_fast` — operation `fibonacci nth fast`. Category: *test runner*.

```
p fibonacci_nth_fast $x
```

fibonacci_retracement

`fibonacci_retracement(high, low) → array` — seven retracement prices [high, 76.4%, 61.8%, 50%, 38.2%, 23.6%, low].

```
p fibonacci_retracement(20, 10)
```

fibonacci_seq

`fibonacci_seq` — operation `fibonacci seq`. Category: **sequences**.

```
p fibonacci_seq $x
```

fibonacci_word

`fibonacci_word` — operation `fibonacci word`. Category: **more extensions**.

```
p fibonacci_word $x
```

fictitious_play_step

`fictitious_play_step` — single-step update of `fictitious play`. Category: **climate, fluids, atmospheric**.

```
p fictitious_play_step $x
```

fidelity_pure_real

`fidelity_pure_real` — operation `fidelity pure real`. Category: **quantum mechanics deep**.

```
p fidelity_pure_real $x
```

fidx

`fidx` — operation `fidx`. Alias for `find_index_fn`. Category: **python/ruby stdlib**.

```
p fidx $x
```

field_of_view

`field_of_view(focal_mm, sensor_dim_mm=36) → number` — angular FOV in degrees.

```
p field_of_view(50, 36) # ~39.6°
```

file

`file PATH` — one-line description in the spirit of `file(1)`: symlink targets, `directory`, FIFO/socket/device nodes (Unix), and magic-byte sniff for regular files (`PNG`, `ELF`, `gzip`, shebang scripts, `ASCII text`, etc.). With no arguments, inspects `$_`.

```
p file("/bin/sh")
p file("Cargo.toml")
```

file_acl_get

`file_acl_get` — filesystem op `acl get`. Category: **extras**.

```
p file_acl_get $x
```

file_acl_set

`file_acl_set` — filesystem op `acl set`. Category: **extras**.

```
p file_acl_set $x
```

file_atime

`file_atime` — filesystem op `atime`. Category: `*file stat / path*`.

```
p file_atime $x
```

file_attr_get

`file_attr_get(path)` → `hash` — filesystem attributes (size, mtime, mode, ...).

file_attr_set

`file_attr_set` — filesystem op `attr set`. Category: `*extras*`.

```
p file_attr_set $x
```

file_chmod_octal

`file_chmod_octal(path, mode)` → `0|1` — chmod with octal mode (e.g., 0o755).

file_chmod_string

`file_chmod_string(path, spec)` → `0|1` — chmod with symbolic spec ("u+x").

file_ctime

`file_ctime` — filesystem op `ctime`. Category: `*file stat / path*`.

```
p file_ctime $x
```

file_kind

`file_kind(path)` → `string` — "file", "dir", "symlink", "socket", "fifo", "block", "char".

file_locked

`file_locked(path)` → `0|1` — 1 if an advisory lock is held.

file_mime

`file_mime(path)` → `string` — guess MIME type by content sniffing + extension.

```
p file_mime("photo.jpg") # "image/jpeg"
```

file_mtime

`file_mtime` — filesystem op `mtime`. Category: `*file stat / path*`.

```
p file_mtime $x
```

file_size

`file_size` — filesystem op `size`. Category: `*file stat / path*`.

```
p file_size $x
```


files

`files` — operation `files`. Category: **filesystem extensions**.

```
p files $x
```

filesf

`filesf` — operation `filesf`. Category: **filesystem extensions**.

```
p filesf $x
```

fill_arr

`fill_arr` — operation `fill arr`. Category: **javascript array/object methods**.

```
p fill_arr $x
```

filla

`filla` — operation `filla`. Alias for `fill_arr`. Category: **javascript array/object methods**.

```
p filla $x
```

filter_bilateral

`filter_bilateral` — signal-filtering op `bilateral`. Category: **pil/opencv image processing**.

```
p filter_bilateral $x
```

filter_chars

`filter_chars` — signal-filtering op `chars`. Category: **string helpers**.

```
p filter_chars $x
```

filter_indexed

`filter_indexed` — signal-filtering op `indexed`. Category: **go/general functional utilities**.

```
p filter_indexed $x
```

filter_map

`filter_map` — signal-filtering op `map`. Category: **rust iterator methods**.

```
p filter_map $x
```

filter_median

`filter_median` — signal-filtering op `median`. Category: **pil/opencv image processing**.

```
p filter_median @xs
```

filter_nlmeans

`filter_nlmeans` — signal-filtering op `nlmeans`. Category: **pil/opencv image processing**.

```
p filter_nlmeans $x
```

filter_ts

`ts_filter` — linear convolution filter. Like R's `filter(method='convolution')`.

```
val @smooth = @{ts_filter([1,5,2,8,3], [0.25,0.5,0.25])}
```

filterfalse

`filterfalse` — operation `filterfalse`. Category: **python/ruby stdlib**.

```
p filterfalse $x
```

filtfilt_pad

`filtfilt_pad` — operation `filtfilt pad`. Category: **economics + game theory**.

```
p filtfilt_pad $x
```

find

`find` — operation `find`. Category: **functional / iterator**.

```
p find $x
```

findInterval

`find_interval` — for each `x`, find which interval in `breaks` it falls into. Like R's `findInterval()`.

```
p findInterval([1.5, 3.5, 7.5], [0, 2, 5, 10]) # [1, 2, 3]
```

find_all

`find_all` — operation `find all`. Category: **functional / iterator**.

```
p find_all $x
```

find_all_indices

`find_all_indices` — operation `find all indices`. Category: **extended stdlib**.

```
p find_all_indices $x
```

find_first

`find_first` — operation `find first`. Category: **list helpers**.

```
p find_first $x
```

find_index

`find_index` — index of `find`. Category: **functional / iterator**.

```
p find_index $x
```

find_index_fn

`find_index_fn` — operation `find index fn`. Category: **python/ruby stdlib**.

```
p find_index_fn $x
```

find_indices

`find_indices` — operation `find indices`. Category: *additional missing stdlib functions*.

```
p find_indices $x
```

find_last

`find_last` — operation `find last`. Category: *javascript array/object methods*.

```
p find_last $x
```

find_last_index

`find_last_index` — index of `find last`. Category: *javascript array/object methods*.

```
p find_last_index $x
```

find_map

`find_map` — operation `find map`. Category: *rust iterator methods*.

```
p find_map $x
```

finite_difference_central

`finite_difference_central(ys, h=1) → array` — central difference: $(y[i+1] - y[i-1]) / (2h)$. Second-order accurate.

```
p finite_difference_central(@ys, 0.1)
```

finite_difference_forward

`finite_difference_forward(ys, h=1) → array` — forward difference: $(y[i+1] - y[i]) / h$.

```
p finite_difference_forward([1,3,6,10,15], 1)
```

finite_rate_lambda

`finite_rate_lambda` — operation `finite rate lambda`. Category: *biology / ecology*.

```
p finite_rate_lambda $x
```

fintushel_stern_step

`fintushel_stern_step` — single-step update of `fintushel stern`. Category: *econometrics*.

```
p fintushel_stern_step $x
```

fip

`fip` — operation `fip`. Category: *astronomy / astrometry*.

```
p fip $x
```

fir_filter

`fir_filter(signal, taps) → array` — finite impulse response: $y[n] = \sum taps[k] \cdot x[n-k]$.

`fir_lowpass_design`

`fir_lowpass_design` — operation `fir lowpass design`. Category: **signal processing**.

```
p fir_lowpass_design $x
```

`fir_moving_average`

`fir_moving_average` — operation `fir moving average`. Category: **signal processing**.

```
p fir_moving_average $x
```

`fire`

`fire` — operation `fire`. Category: **stress testing**.

```
p fire $x
```

`fire_and_forget`

`fire_and_forget` — operation `fire and forget`. Category: **stress testing**.

```
p fire_and_forget $x
```

`first_arg`

`first_arg` — operation `first arg`. Category: **functional primitives**.

```
p first_arg $x
```

`first_elem`

`first_elem` — operation `first elem`. Category: **list helpers**.

```
p first_elem $x
```

`first_eq`

`first_eq` — operation `first eq`. Category: **collection**.

```
p first_eq $x
```

`first_index`

`first_index` — index of `first`. Category: **functional / iterator**.

```
p first_index $x
```

`first_of_month`

`first_of_month` — operation `first of month`. Category: **misc**.

```
p first_of_month $x
```

`first_order_concentration`

`first_order_concentration` — operation `first order concentration`. Category: **chemistry**.

```
p first_order_concentration $x
```

first_order_half_life

`first_order_half_life` — operation `first_order_half_life`. Category: **chemistry**.

```
p first_order_half_life $x
```

first_price_seal

`first_price_seal` — operation `first_price_seal`. Category: **economics + game theory**.

```
p first_price_seal $x
```

first_word

`first_word` — operation `first_word`. Category: **string**.

```
p first_word $x
```

firstidx

`firstidx` — operation `firstidx`. Category: **functional / iterator**.

```
p firstidx $x
```

firwin2_freq

`firwin2_freq` — operation `firwin2_freq`. Category: **economics + game theory**.

```
p firwin2_freq $x
```

firwin_bandpass

`firwin_bandpass` — operation `firwin_bandpass`. Category: **economics + game theory**.

```
p firwin_bandpass $x
```

firwin_bandstop

`firwin_bandstop` — operation `firwin_bandstop`. Category: **economics + game theory**.

```
p firwin_bandstop $x
```

firwin_highpass

`firwin_highpass` — operation `firwin_highpass`. Category: **economics + game theory**.

```
p firwin_highpass $x
```

firwin_lowpass

`firwin_lowpass` — operation `firwin_lowpass`. Category: **economics + game theory**.

```
p firwin_lowpass $x
```

fiscal_year_us

`fiscal_year_us` — operation `fiscal_year_us`. Category: **b82-misc-utility**.

```
p fiscal_year_us $x
```

fisher_exact_2x2

`fisher_exact_2x2(a, b, c, d) → {statistic, df, p_value}` — Fisher's exact test for a 2×2 table; two-sided p-value.

```
p fisher_exact_2x2(8, 2, 1, 5)
```

fisher_info_metric

`fisher_info_metric` — metric of `fisher info`. Category: *electrochemistry, batteries, fuel cells*.

```
p fisher_info_metric $x
```

fisher_yates_shuffle

`fisher_yates_shuffle` — operation `fisher yates shuffle`. Category: *misc*.

```
p fisher_yates_shuffle $x
```

fista_step

`fista_step` — single-step update of `fista`. Category: *combinatorial optimization, scheduling*.

```
p fista_step $x
```

fit_curve_least_squares

`fit_curve_least_squares(xs, ys) → hash` — OLS linear regression $y = a + b \cdot x$. Returns `{a, b}`.

```
my $fit = fit_curve_least_squares([0,1,2,3], [1,3,5,7])  
p "$fit->{a} + $fit->{b}*x"
```

five_number_summary

`five_number_summary(@data)` (alias `fivenum`) — returns [min, Q1, median, Q3, max], the classic five-number summary used for box plots. Provides a quick view of data distribution.

```
val @data = 1:100  
var $f = fivenum(@data)  
p "min=$f->[0] Q1=$f->[1] med=$f->[2] Q3=$f->[3] max=$f->[4]"
```

fixed_from_french

`fixed_from_french` — operation `fixed from french`. Category: *calendrical algorithms*.

```
p fixed_from_french $x
```

fixed_from_gregorian

`fixed_from_gregorian` — operation `fixed from gregorian`. Category: *calendrical algorithms*.

```
p fixed_from_gregorian $x
```

fixed_from_hebrew

`fixed_from_hebrew` — operation `fixed from hebrew`. Category: *calendrical algorithms*.

```
p fixed_from_hebrew $x
```

fixed_from_islamic

`fixed_from_islamic` — operation `fixed from islamic`. Category: **calendrical algorithms**.

```
p fixed_from_islamic $x
```

fixed_from_julian

`fixed_from_julian` — operation `fixed from julian`. Category: **calendrical algorithms**.

```
p fixed_from_julian $x
```

fixed_from_persian

`fixed_from_persian` — operation `fixed from persian`. Category: **calendrical algorithms**.

```
p fixed_from_persian $x
```

fixed_quad

`fixed_quad` — operation `fixed quad`. Category: **numpy + scipy.special**.

```
p fixed_quad $x
```

fizzbuzz

`fizzbuzz` — operation `fizzbuzz`. Category: **algorithms / puzzles**.

```
p fizzbuzz $x
```

flanger_simple

`flanger_simple` — operation `flanger simple`. Category: **test runner**.

```
p flanger_simple $x
```

flat_band_potential

`flat_band_potential` — operation `flat band potential`. Category: **electrochemistry, batteries, fuel cells**.

```
p flat_band_potential $x
```

flat_depth

`flat_depth` — operation `flat depth`. Category: **javascript array/object methods**.

```
p flat_depth $x
```

flat_map

`flat_map` — operation `flat map`. Category: **functional / iterator**.

```
p flat_map $x
```

flat_map_fn

`flat_map_fn` — operation `flat map fn`. Category: **python/ruby stdlib**.

```
p flat_map_fn $x
```

flat_maps

`flat_maps { BLOCK } LIST` — a lazy streaming flat-map that evaluates the block for each element and flattens the resulting lists into a single iterator. Where `maps` expects the block to return one value, `flat_maps` handles blocks that return zero or more values per element and concatenates them seamlessly. This is the streaming equivalent of calling `map` with a multi-value block. Use it in `|>` chains when each input element fans out into multiple outputs and you want lazy evaluation.

```
1:3 |> flat_maps { (_, _ * 10) } |> e p
# 1 10 2 20 3 30
@nested |> flat_maps { @_ } |> e p # flatten array-of-arrays
```

flatnonzero

`flatnonzero` — operation `flatnonzero`. Category: `*numpy + scipy.special*`.

```
p flatnonzero $x
```

flatten_deep

`flatten_deep` — operation `flatten deep`. Category: `*collection more*`.

```
p flatten_deep $x
```

flatten_once

`flatten_once` — operation `flatten once`. Category: `*list helpers*`.

```
p flatten_once $x
```

flattop_w

`flattop_w` — operation `flattop w`. Category: `*economics + game theory*`.

```
p flattop_w $x
```

fldr

`fldr` — operation `fldr`. Alias for `fold_right`. Category: `*rust iterator methods*`.

```
p fldr $x
```

flesch_kincaid_grade

`flesch_kincaid_grade` — operation `flesch kincaid grade`. Category: `*misc*`.

```
p flesch_kincaid_grade $x
```

flesch_reading_ease

`flesch_reading_ease` — operation `flesch reading ease`. Category: `*misc*`.

```
p flesch_reading_ease $x
```

fletcher16

`fletcher16(s) → int` — Fletcher-16 checksum (two 8-bit sums mod 255).

```
p fletcher16("abcde") # 0xC8F0
```


fletcher32

`fletcher32(s)` → `int` — Fletcher-32 (two 16-bit sums mod 65535).

fletcher64

`fletcher64(s)` → `int` — Fletcher-64 (two 32-bit sums mod $2^{32}-1$).

flip_args

`flip_args` — operation `flip args`. Category: *functional primitives*.

```
p flip_args $x
```

float_bits

`float_bits` — floating-point helper `bits`. Category: *math functions*.

```
p float_bits $x
```

floer_homology_rank

`floer_homology_rank` — operation `floer homology rank`. Category: *econometrics*.

```
p floer_homology_rank $x
```

floor

`floor` — operation `floor`. Category: *trivial numeric / predicate builtins*.

```
p floor $x
```

floor_div

`floor_div` — operation `floor div`. Category: *trig / math*.

```
p floor_div $x
```

floor_div_op

`floor_div_op` — operation `floor div op`. Category: *extended stdlib*.

```
p floor_div_op $x
```

floor_each

`floor_each` — operation `floor each`. Category: *trivial numeric helpers*.

```
p floor_each $x
```

floor_mod

`floor_mod` — operation `floor mod`. Category: *extended stdlib*.

```
p floor_mod $x
```

flow_shop_johnson_step

`flow_shop_johnson_step` — single-step update of `flow_shop_johnson`. Category: *combinatorial optimization, scheduling*.

```
p flow_shop_johnson_step $x
```

floyd_cycle_detect

`floyd_cycle_detect(seq) → 0|1` — Floyd's tortoise-and-hare cycle detection on a functional graph.

floyd_warshall_step

`floyd_warshall_step` — single-step update of `floyd_warshall`. Category: *networkx graph algorithms*.

```
p floyd_warshall_step $x
```

fltd

`fltd` — operation `fltd`. Alias for `flat_depth`. Category: *javascript array/object methods*.

```
p fltd $x
```

flti

`flti` — operation `flti`. Alias for `filter_indexed`. Category: *go/general functional utilities*.

```
p flti $x
```

fltm

`fltm` — operation `fltm`. Alias for `filter_map`. Category: *rust iterator methods*.

```
p fltm $x
```

flux_richardson_full

`flux_richardson_full` — operation `flux_richardson_full`. Category: *climate, fluids, atmospheric*.

```
p flux_richardson_full $x
```

fm_synth_2op

`fm_synth_2op(carrier, mod_freq, mod_idx, sr=44100, dur=1) → array` — 2-operator FM synthesis: $\sin(2\pi \cdot fc \cdot t + I \cdot \sin(2\pi \cdot fm \cdot t))$.

fma

`fma` — operation `fma`. Category: *math functions*.

```
p fma $x
```

fmadd

`fmadd` — operation `fmadd`. Alias for `fused_mul_add`. Category: *extended stdlib*.

```
p fmadd $x
```

fmf

`fmf` — operation `fmf`. Alias for `flat_map_fn`. Category: **python/ruby stdlib**.

```
p fmf $x
```

fmin_cobyla

`fmin_cobyla` — operation `fmin_cobyla`. Category: **numpy + scipy.special**.

```
p fmin_cobyla $x
```

fmin_powell

`fmin_powell` — operation `fmin_powell`. Category: **numpy + scipy.special**.

```
p fmin_powell $x
```

fmod

`fmod` — operation `fmod`. Alias for `floor_mod`. Category: **extended stdlib**.

```
p fmod $x
```

fndalli

`fndalli` — operation `fndalli`. Alias for `find_all_indices`. Category: **extended stdlib**.

```
p fndalli $x
```

fndidxs

`fndidxs` — operation `fndidxs`. Alias for `find_indices`. Category: **additional missing stdlib functions**.

```
p fndidxs $x
```

fndl

`fndl` — operation `fndl`. Alias for `find_last`. Category: **javascript array/object methods**.

```
p fndl $x
```

fndli

`fndli` — operation `fndli`. Alias for `find_last_index`. Category: **javascript array/object methods**.

```
p fndli $x
```

fndm

`fndm` — operation `fndm`. Alias for `find_map`. Category: **rust iterator methods**.

```
p fndm $x
```

fne

`fne` — operation `fne`. Alias for `fnext`. Category: **algebraic match**.

```
p fne $x
```

fnext

`fnext` — operation `fnext`. Category: **algebraic match**.

```
p fnext $x
```

fnv0_32

`fnv0_32` — operation `fnv0 32`. Category: **b81-misc-utility**.

```
p fnv0_32 $x
```

fnv1a

`fnv1a` — operation `fnv1a`. Category: **extended stdlib**.

```
p fnv1a $x
```

fnv1a_32

`fnv1a_32` — operation `fnv1a 32`. Category: **cryptography deep**.

```
p fnv1a_32 $x
```

fnv1a_64

`fnv1a_64` — operation `fnv1a 64`. Category: **cryptography deep**.

```
p fnv1a_64 $x
```

fnv1a_hash

`fnv1a_hash` — hash function of `fnv1a`. Category: **misc**.

```
p fnv1a_hash $x
```

focal_length_35mm_equiv

`focal_length_35mm_equiv(focal_mm, crop_factor=1) → number` — `focal` · `crop`.

fold_axis

`fold_axis` — operation `fold axis`. Category: **apl/j/k array primitives**.

```
p fold_axis $x
```

fold_case

`fold_case` — operation `fold case`. Category: **iterator + string-distance extras**.

```
p fold_case $x
```

fold_left

`fold_left` — operation `fold left`. Category: **list helpers**.

```
p fold_left $x
```

fold_right

`fold_right` — operation `fold right`. Category: **rust iterator methods**.

```
p fold_right $x
```

foldl1_iter

`foldl1_iter` — operation `foldl1 iter`. Category: **iterator + string-distance extras**.

```
p foldl1_iter $x
```

foldr1_iter

`foldr1_iter` — operation `foldr1 iter`. Category: **iterator + string-distance extras**.

```
p foldr1_iter $x
```

folk_theorem_value

`folk_theorem_value` — value of `folk theorem`. Category: **climate, fluids, atmospheric**.

```
p folk_theorem_value $x
```

force_mass_acc

`force_mass_acc` — operation `force mass acc`. Category: **physics formulas**.

```
p force_mass_acc $x
```

force_of_mortality

`force_of_mortality` — operation `force of mortality`. Category: **actuarial science**.

```
p force_of_mortality $x
```

ford_fulkerson_max_flow

`ford_fulkerson_max_flow` — operation `ford fulkerson max flow`. Category: **test runner**.

```
p ford_fulkerson_max_flow $x
```

ford_fulkerson_step

`ford_fulkerson_step` — single-step update of `ford fulkerson`. Category: **networkx graph algorithms**.

```
p ford_fulkerson_step $x
```

formal_charge

`formal_charge` — operation `formal charge`. Category: **chemistry & biochemistry**.

```
p formal_charge $x
```

format_bytes

`format_bytes` — operation `format bytes`. Category: **file stat / path**.

```
p format_bytes $x
```

format_duration

`format_duration` — operation `format duration`. Category: `*file stat / path*`.

```
p format_duration $x
```

format_number

`format_number` — operation `format number`. Category: `*file stat / path*`.

```
p format_number $x
```

format_percent

`format_percent` — operation `format percent`. Category: `*file stat / path*`.

```
p format_percent $x
```

format_strftime

`strftime(FORMAT, EPOCH_SECS?)` → format a Unix timestamp using `strftime` directives (`%Y %m %d %H %M %S %a %A` etc.). Default timestamp is `Local::now()`. Wraps the existing `chrono` dep.

```
p strftime("%Y-%m-%d %H:%M:%S")      # 2026-05-14 17:19:48
p strftime("%FT%TZ", 1700000000)
```

format_table_simple

`format_table_simple` — operation `format table simple`. Category: `*misc*`.

```
p format_table_simple $x
```

forward_algorithm

`forward_algorithm(obs, init, trans, emit)` → `number` — total probability `P(obs)` summed over all state paths.

forward_backward_pos

`forward_backward_pos` — operation `forward backward pos`. Category: `*computational linguistics*`.

```
p forward_backward_pos $x
```

forward_kinematics_dh

`forward_kinematics_dh` — operation `forward kinematics dh`. Category: `*robotics & control*`.

```
p forward_kinematics_dh $x
```

forward_rate

`forward_rate` — rate of `forward`. Category: `*financial pricing models*`.

```
p forward_rate $x
```

fourier_number_step

`fourier_number_step` — single-step update of `fourier number`. Category: `*climate, fluids, atmospheric*`.

```
p fourier_number_step $x
```

fp_depression

`fp_depression` — operation `fp depression`. Category: **chemistry**.

```
p fp_depression $x
```

fr

`fr` — operation `fr`. Category: **filesystem extensions**.

```
p fr $x
```

frac_part

`frac_part` — operation `frac part`. Category: **extended stdlib**.

```
p frac_part $x
```

fracp

`fracp` — operation `fracp`. Alias for `frac_part`. Category: **extended stdlib**.

```
p fracp $x
```

fractional_chromatic_lower

`fractional_chromatic_lower` — operation `fractional chromatic lower`. Category: **combinatorial optimization, scheduling**.

```
p fractional_chromatic_lower $x
```

frank_wolfe_step

`frank_wolfe_step` — single-step update of `frank wolfe`. Category: **combinatorial optimization, scheduling**.

```
p frank_wolfe_step $x
```

fredkin_gate

`fredkin_gate` — operation `fredkin gate`. Category: **more extensions**.

```
p fredkin_gate $x
```

free_energy_principle

`free_energy_principle` — operation `free energy principle`. Category: **electrochemistry, batteries, fuel cells**.

```
p free_energy_principle $x
```

free_group_rank_lower

`free_group_rank_lower` — operation `free group rank lower`. Category: **econometrics**.

```
p free_group_rank_lower $x
```

free_particle_energy

`free_particle_energy` — operation `free particle energy`. Category: **quantum mechanics deep**.

```
p free_particle_energy $x
```

freefall_time

freefall_time — operation **freefall time**. Category: **misc**.

```
p freefall_time $x
```

freefall_velocity_schwarzschild

freefall_velocity_schwarzschild — operation **freefall velocity schwarzschild**. Category: **cosmology / gr / flrw**.

```
p freefall_velocity_schwarzschild $x
```

freeverb_lite

freeverb_lite — operation **freeverb lite**. Category: **test runner**.

```
p freeverb_lite $x
```

french_revolutionary_leap

french_revolutionary_leap — operation **french revolutionary leap**. Category: **calendrical algorithms**.

```
p french_revolutionary_leap $x
```

freq_to_note

freq_to_note — convert from **freq** to **note**. Category: **extras**.

```
p freq_to_note $value
```

freq_wavelength

freq_wavelength — frequency-domain helper **wavelength**. Category: **physics formulas**.

```
p freq_wavelength $x
```

freqs_eval

freqs_eval — operation **freqs eval**. Category: **economics + game theory**.

```
p freqs_eval $x
```

freqz_eval

freqz_eval — operation **freqz eval**. Category: **economics + game theory**.

```
p freqz_eval $x
```

fresnel_reflection_normal

fresnel_reflection_normal — operation **fresnel reflection normal**. Category: **em / optics / relativity**.

```
p fresnel_reflection_normal $x
```


fresnel_rp

`fresnel_rp` — operation `fresnel rp`. Category: **em / optics / relativity**.

```
p fresnel_rp $x
```

fresnel_rs

`fresnel_rs` — operation `fresnel rs`. Category: **em / optics / relativity**.

```
p fresnel_rs $x
```

friction_factor_laminar

`friction_factor_laminar` — operation `friction factor laminar`. Category: **misc**.

```
p friction_factor_laminar $x
```

friedmann_density_total

`friedmann_density_total` — operation `friedmann density total`. Category: **cosmology / gr / flrw**.

```
p friedmann_density_total $x
```

from_base

`from_base` — construct from base representation. Category: **base conversion**.

```
p from_base $x
```

from_bin

`from_bin` — construct from bin representation. Category: **base conversion**.

```
p from_bin $x
```

from_code_point

`from_code_point` — construct from code point representation. Category: **extended stdlib**.

```
p from_code_point $x
```

from_csv_line

`from_csv_line` — construct from csv line representation. Category: **string helpers**.

```
p from_csv_line $x
```

from_digits

`from_digits` — construct from digits representation. Category: **list helpers**.

```
p from_digits $x
```

from_hex

`from_hex` — construct from hex representation. Category: **base conversion**.

```
p from_hex $x
```

from_ini

Parse an INI document into a hashref. Pre-section keys land at the top level; each `[section]` becomes a nested hashref. `;` and `#` start comments; keys split on the first `=` or `..`. Inverse of `to_ini`.

```
val %cfg = %{ from_ini(read_file("app.ini")) }
p $cfg{database}{host}
p $cfg{debug}           # a global (pre-section) key
```

from_jsonl

Parse newline-delimited JSON (JSON Lines / NDJSON — one value per line) into an array. Blank lines are skipped. The streaming format used by logs, `jq`, and data pipelines. Inverse of `to_jsonl`.

```
val @rows = from_jsonl(read_file("events.ndjson"))
p scalar @rows
p $rows[0]{user}
```

from_oct

`from_oct` — construct from oct representation. Category: **base conversion**.

```
p from_oct $x
```

from_pairs

`from_pairs` — construct from pairs representation. Category: **list helpers**.

```
p from_pairs $x
```

froude_number

`froude_number` — operation `froude number`. Category: **misc**.

```
p froude_number $x
```

froude_number_step

`froude_number_step` — single-step update of `froude number`. Category: **climate, fluids, atmospheric**.

```
p froude_number_step $x
```

frustum_volume

`frustum_volume($r1, $r2, $h)` — computes the volume of a conical frustum (truncated cone) with base radii `r1` and `r2` and height `h`.

```
p frustum_volume(5, 3, 10)  # ~513
```

frustvol

`frustvol` — operation `frustvol`. Alias for `frustum_volume`. Category: **geometry (extended)**.

```
p frustvol $x
```

fsize

`fsize` — operation `fsize`. Alias for `file_size`. Category: **file stat / path**.

```
p fsize $x
```

fst

`fst` — operation `fst`. Alias for `first`. Category: **functional / iterator**.

```
p fst $x
```

ft

`ft` — operation `ft`. Alias for `fetch`. Category: **data / network**.

```
p ft $x
```

fta

`fta` — operation `fta`. Alias for `fetch_async`. Category: **data / network**.

```
p fta $x
```

ftaj

`ftaj` — operation `ftaj`. Alias for `fetch_async_json`. Category: **data / network**.

```
p ftaj $x
```

ftj

`ftj` — operation `ftj`. Alias for `fetch_json`. Category: **data / network**.

```
p ftj $x
```

fuel_cell_polarization

`fuel_cell_polarization` — operation `fuel cell polarization`. Category: **electrochemistry, batteries, fuel cells**.

```
p fuel_cell_polarization $x
```

full_moon_julian

`full_moon_julian(cycle)` → `number` — Julian Date of `cycle`-th full moon.

```
p full_moon_julian(0)
```

fundamental_group_zn

`fundamental_group_zn` — operation `fundamental group zn`. Category: **econometrics**.

```
p fundamental_group_zn $x
```

fused_mul_add

`fused_mul_add` — operation `fused mul add`. Category: **extended stdlib**.

```
p fused_mul_add $x
```

fusion_rule_su2_step

`fusion_rule_su2_step` — single-step update of `fusion rule su2`. Category: **econometrics**.

```
p fusion_rule_su2_step $x
```

future_value

`future_value(present_value, rate, periods) → number` — $PV \cdot (1 + rate)^{periods}$. Compound a single cashflow forward.

```
p future_value(1000, 0.05, 10) # 1628.89
```

fuzzy_substring_match

`fuzzy_substring_match(needle, haystack, max_dist=2) → int` — earliest start index where `needle` matches a substring of `haystack` within Levenshtein distance `max_dist`; -1 if no match.

```
p fuzzy_substring_match("hello", "yellow helo world", 1)
```

fv

`fv` — operation `fv`. Alias for `fetch_val`. Category: **python/ruby stdlib**.

```
p fv $x
```

fx_forward

`fx_forward` — operation `fx forward`. Category: **financial pricing models**.

```
p fx_forward $x
```

g2_zero

`g2_zero` — operation `g2 zero`. Category: **quantum mechanics deep**.

```
p g2_zero $x
```

g_to_oz

`g_to_oz` — convert from `g` to `oz`. Category: **unit conversions**.

```
p g_to_oz $value
```

gabor_filter

`gabor_filter` — filter op of `gabor`. Category: **pil/opencv image processing**.

```
p gabor_filter $x
```

gabor_kernel

`gabor_kernel` — convolution kernel of `gabor`. Category: **more extensions**.

```
p gabor_kernel $x
```

gae_advantage_step

`gae_advantage_step` — single-step update of `gae advantage`. Category: *electrochemistry, batteries, fuel cells*.

```
p gae_advantage_step $x
```

gain_ratio

`gain_ratio` — ratio of `gain`. Category: *ml extensions*.

```
p gain_ratio $x
```

gale_optimal

`gale_optimal` — operation `gale optimal`. Category: *economics + game theory*.

```
p gale_optimal $x
```

gale_shapley_step

`gale_shapley_step` — single-step update of `gale shapley`. Category: *economics + game theory*.

```
p gale_shapley_step $x
```

gallons_to_liters

`gallons_to_liters` — convert from `gallons` to `liters`. Category: *file stat / path*.

```
p gallons_to_liters $value
```

game_of_life_step

`game_of_life_step` — single-step update of `game of life`. Category: *algorithms / puzzles*.

```
p game_of_life_step $x
```

game_two_player_value

`game_two_player_value` — value of `game two player`. Category: *climate, fluids, atmospheric*.

```
p game_two_player_value $x
```

gamma_apply

`gamma_apply` — operation `gamma apply`. Category: *complex / geom / color / trig*.

```
p gamma_apply $x
```

gamma_approx

`gamma_approx($z)` — Gamma function $\Gamma(z)$ using Lanczos approximation. Extends factorial: $\Gamma(n) = (n-1)!$

```
p gamma(5)      # 24 (same as 4!)
p gamma(0.5)     # ~1.772 ( $\sqrt{\pi}$ )
```

gamma_correct

`gamma_correct(v, gamma=2.2) → number` — apply gamma correction $v^{(1/\gamma)}$ to a [0,1] value.

```
p gamma_correct(0.5, 2.2)
```

gamma_pdf

`gamma_pdf` (alias `gammapdf`) evaluates the Gamma distribution PDF at `x` with shape `k` and scale `theta`.

```
p gammapdf(2.0, 2, 1) # Gamma(2,1) at x=2
```

gamma_regularized_p

`gamma_regularized_p(a, x) → number` — lower regularised incomplete gamma $P(a, x) = \gamma(a, x) / \Gamma(a)$.

```
p gamma_regularized_p(2, 1)
```

gamma_regularized_q

`gamma_regularized_q(a, x) → number` — upper regularised incomplete gamma $Q(a, x) = 1 - P(a, x)$.

```
p gamma_regularized_q(2, 1)
```

gamma_remove

`gamma_remove` — operation `gamma remove`. Category: **complex / geom / color / trig**.

```
p gamma_remove $x
```

gamma_uncorrect

`gamma_uncorrect(v, gamma=2.2) → number` — undo gamma: v^γ .

gamow_factor

`gamow_factor` — factor of `gamow`. Category: **quantum mechanics deep**.

```
p gamow_factor $x
```

garch_fit

`garch_fit` — operation `garch fit`. Category: **statsmodels**.

```
p garch_fit $x
```

garch_step

`garch_step` — single-step update of `garch`. Category: **more extensions (2)**.

```
p garch_step $x
```

garman_kohlhagen_call

`garman_kohlhagen_call` — operation `garman kohlhagen call`. Category: **financial pricing models**.

```
p garman_kohlhagen_call $x
```

gas_constant

`gas_constant` (alias `rgas`) — $R \approx 8.314 \text{ J}/(\text{mol}\cdot\text{K})$. Universal gas constant.

```
p rgas # 8.314462618
```

gas_constant_r

`gas_constant_r` — operation `gas constant r`. Category: **misc**.

```
p gas_constant_r $x
```

gas_constant_value

`gas_constant_value` — value of `gas constant`. Category: **chemistry & biochemistry**.

```
p gas_constant_value $x
```

gasp

`ideal_gas_pressure($n, $V, $T)` (alias `gasp`) — pressure $P = nRT/V$. n moles, V volume (m^3), T temperature (K).

```
p gasp(1, 0.0224, 273.15) # ~101325 Pa (1 mol at STP)
```

gasv

`gasv` — operation `gasv`. Alias for `ideal_gas_volume`. Category: **physics formulas**.

```
p gasv $x
```

gate_decompose

`gate_decompose` — operation `gate decompose`. Category: **quantum**.

```
p gate_decompose $x
```

gauss_bonnet_term_2d

`gauss_bonnet_term_2d` — operation `gauss bonnet term 2d`. Category: **electrochemistry, batteries, fuel cells**.

```
p gauss_bonnet_term_2d $x
```

gauss_bonnet_total

`gauss_bonnet_total` — operation `gauss bonnet total`. Category: **econometrics**.

```
p gauss_bonnet_total $x
```

gauss_codazzi_step

`gauss_codazzi_step` — single-step update of `gauss codazzi`. Category: **electrochemistry, batteries, fuel cells**.

```
p gauss_codazzi_step $x
```

gauss_irk_2_stage

`gauss_irk_2_stage` — operation `gauss irk 2 stage`. Category: **ode advanced**.

```
p gauss_irk_2_stage $x
```

gauss_kronrod_15

`gauss_kronrod_15` — operation `gauss kronrod 15`. Category: **more extensions (2)**.

```
p gauss_kronrod_15 $x
```

gauss_legendre_5

`gauss_legendre_5` — operation `gauss legendre 5`. Category: **more extensions (2)**.

```
p gauss_legendre_5 $x
```

gaussian_curvature_step

`gaussian_curvature_step` — single-step update of `gaussian curvature`. Category: **electrochemistry, batteries, fuel cells**.

```
p gaussian_curvature_step $x
```

gaussian_filter_window

`gaussian_filter_window` — operation `gaussian filter window`. Category: **electrochemistry, batteries, fuel cells**.

```
p gaussian_filter_window $x
```

gaussian_kernel

`gaussian_kernel(size=5, sigma=1)` → `matrix` — normalised 2D Gaussian; sums to 1.

```
p gaussian_kernel(5, 1.4)
```

gaussian_window

`gaussian_window` — operation `gaussian window`. Category: **signal processing**.

```
p gaussian_window $x
```

gb_to_bytes

`gb_to_bytes` — convert from `gb` to `bytes`. Category: **unit conversions**.

```
p gb_to_bytes $value
```

gbf

`gbf` — operation `gbf`. Alias for `group_by_fn`. Category: **go/general functional utilities**.

```
p gbf $x
```

gbi

`gbi` — operation `gbi`. Alias for `groupby_iter`. Category: **python/ruby stdlib**.

```
p gbi $x
```

gcby

`gcby` — operation `gcby`. Alias for `group_consecutive_by`. Category: **array / list operations extras**.


```
p gcby $x
```

gcd

gcd — operation **gcd**. Category: *trivial numeric / predicate builtins*.

```
p gcd $x
```

gconst

gconst — operation **gconst**. Alias for **gravitational_constant**. Category: *constants*.

```
p gconst $x
```

gelu

gelu applies the Gaussian Error Linear Unit, used in BERT/GPT transformers.

```
p gelu(1)      # 0.8412
p gelu(-1)     # -0.1588
```

generalized_advantage

generalized_advantage — operation **generalized advantage**. Category: *electrochemistry, batteries, fuel cells*.

```
p generalized_advantage $x
```

generation_time

generation_time — operation **generation time**. Category: *biology / ecology*.

```
p generation_time $x
```

generation_time_demo

generation_time_demo — operation **generation time demo**. Category: *biology / ecology*.

```
p generation_time_demo $x
```

generation_time_step

generation_time_step — single-step update of **generation time**. Category: *epidemiology / public health*.

```
p generation_time_step $x
```

genetic_crossover_one_point

genetic_crossover_one_point — operation **genetic crossover one point**. Category: *combinatorial optimization, scheduling*.

```
p genetic_crossover_one_point $x
```

genus_curve_arith

genus_curve_arith — operation **genus curve arith**. Category: *econometrics*.

```
p genus_curve_arith $x
```

genus_curve_geo

`genus_curve_geo` — operation `genus curve geo`. Category: **econometrics**.

```
p genus_curve_geo $x
```

genus_from_euler

`genus_from_euler` — operation `genus from euler`. Category: **geometry / topology**.

```
p genus_from_euler $x
```

genus_surface

`genus_surface` — operation `genus surface`. Category: **econometrics**.

```
p genus_surface $x
```

geoadd

`geoadd` — operation `geoadd`. Category: **redis-flavour primitives**.

```
p geoadd $x
```

geodesic_curvature_step

`geodesic_curvature_step` — single-step update of `geodesic curvature`. Category: **electrochemistry, batteries, fuel cells**.

```
p geodesic_curvature_step $x
```

geodesic_equation_step_zero

`geodesic_equation_step_zero` — operation `geodesic equation step zero`. Category: **electrochemistry, batteries, fuel cells**.

```
p geodesic_equation_step_zero $x
```

geodist

`geodist` — operation `geodist`. Category: **redis-flavour primitives**.

```
p geodist $x
```

geohash

`geohash` — operation `geohash`. Category: **redis-flavour primitives**.

```
p geohash $x
```

geohash_bbox

`geohash_bbox` — operation `geohash bbox`. Category: **more extensions**.

```
p geohash_bbox $x
```

geohash_decode

`geohash_decode` — decode of `geohash`. Category: **more extensions**.

```
p geohash_decode $x
```

geohash_encode

`geohash_encode` — encode of `geohash`. Category: **more extensions**.

```
p geohash_encode $x
```

geohash_neighbor

`geohash_neighbor` — operation `geohash neighbor`. Category: **more extensions**.

```
p geohash_neighbor $x
```

geohash_neighbors

`geohash_neighbors` — operation `geohash neighbors`. Category: **excel/sheets + bond/loan financial**.

```
p geohash_neighbors $x
```

geohash_precision

`geohash_precision` — operation `geohash precision`. Category: **extras**.

```
p geohash_precision $x
```

geomag_declination

`geomag_declination(lat, lon) → number` — magnetic declination in degrees using a simple dipole model (not WMM-accurate).

```
p geomag_declination(40, -74)
```

geomagnetic_kp_index

`geomagnetic_kp_index` — index of `geomagnetic kp`. Category: **climate, fluids, atmospheric**.

```
p geomagnetic_kp_index $x
```

geometric_height_full

`geometric_height_full` — operation `geometric height full`. Category: **climate, fluids, atmospheric**.

```
p geometric_height_full $x
```

geometric_intersection_number

`geometric_intersection_number` — operation `geometric intersection number`. Category: **econometrics**.

```
p geometric_intersection_number $x
```

geometric_mean

`geometric_mean` — arithmetic mean of `geometric`. Category: *file stat / path*.

```
p geometric_mean @xs
```

geometric_mean_arr

`geometric_mean_arr` — operation `geometric mean arr`. Category: *misc*.

```
p geometric_mean_arr $x
```

geometric_sequence

`geometric_sequence` — operation `geometric sequence`. Category: *misc*.

```
p geometric_sequence $x
```

geometric_series

`geometric_series($a1, $r, $n)` (alias `geomser`) — sum of n terms of geometric sequence. Formula: $a1 \times (1-r^n)/(1-r)$. If $r=1$, returns $a1 \times n$.

```
p geomser(1, 2, 10)      # 1023 (1+2+4+...+512)
p geomser(1, 0.5, 10)    # ~1.998 (converging to 2)
```

geopotential_height_full

`geopotential_height_full` — operation `geopotential height full`. Category: *climate, fluids, atmospheric*.

```
p geopotential_height_full $x
```

geostrophic_wind_step

`geostrophic_wind_step` — single-step update of `geostrophic wind`. Category: *climate, fluids, atmospheric*.

```
p geostrophic_wind_step $x
```

get_in

`get_in` — operation `get in`. Category: *algebraic match*.

```
p get_in $x
```

gethostent

`gethostent` — operation `gethostent`. Category: *posix metadata*.

```
p gethostent $x
```

getnetbyname

`getnetbyname` — operation `getnetbyname`. Category: *posix metadata*.

```
p getnetbyname $x
```

getnetent

`getnetent` — operation `getnetent`. Category: *posix metadata*.

```
p getnetent $x
```

getprotoent

`getprotoent` — operation `getprotoent`. Category: *posix metadata*.

```
p getprotoent $x
```

getrange

`getrange` — operation `getrange`. Category: *redis-flavour primitives*.

```
p getrange $x
```

getservent

`getservent` — operation `getservent`. Category: *posix metadata*.

```
p getservent $x
```

getset

`getset` — operation `getset`. Category: *redis-flavour primitives*.

```
p getset $x
```

gf256_multiply

`gf256_multiply` — operation `gf256 multiply`. Category: *cryptanalysis & number theory deep*.

```
p gf256_multiply $x
```

gforce

`gravitational_force($m1, $m2, $r)` (alias `gforce`) — Newton's law of gravitation: $F = Gm_1m_2/r^2$.

```
p gforce(5.972e24, 1000, 6.371e6) # ~9820 N (1 ton at Earth surface)
```

gfx_aabb_intersect_check

`gfx_aabb_intersect_check` — operation `gfx aabb intersect check`. Category: *climate, fluids, atmospheric*.

```
p gfx_aabb_intersect_check $x
```

gfx_axis_angle_to_quat

`gfx_axis_angle_to_quat` — convert from `gfx axis angle` to `quat`. Category: *climate, fluids, atmospheric*.

```
p gfx_axis_angle_to_quat $value
```

gfx_barycentric_alpha

`gfx_barycentric_alpha` — operation `gfx barycentric alpha`. Category: *climate, fluids, atmospheric*.

```
p gfx_barycentric_alpha $x
```

gfx_barycentric_beta

gfx_barycentric_beta — operation `gfx barycentric beta`. Category: *climate, fluids, atmospheric*.

```
p gfx_barycentric_beta $x
```

gfx_barycentric_gamma

gfx_barycentric_gamma — operation `gfx barycentric gamma`. Category: *climate, fluids, atmospheric*.

```
p gfx_barycentric_gamma $x
```

gfx_blinn_specular_step

gfx_blinn_specular_step — single-step update of `gfx blinn specular`. Category: *climate, fluids, atmospheric*.

```
p gfx_blinn_specular_step $x
```

gfx_blue_noise_value

gfx_blue_noise_value — value of `gfx blue noise`. Category: *climate, fluids, atmospheric*.

```
p gfx_blue_noise_value $x
```

gfx_bresenham_step_x

gfx_bresenham_step_x — operation `gfx bresenham step x`. Category: *climate, fluids, atmospheric*.

```
p gfx_bresenham_step_x $x
```

gfx_bresenham_step_y

gfx_bresenham_step_y — operation `gfx bresenham step y`. Category: *climate, fluids, atmospheric*.

```
p gfx_bresenham_step_y $x
```

gfx_clip_polygon_step

gfx_clip_polygon_step — single-step update of `gfx clip polygon`. Category: *climate, fluids, atmospheric*.

```
p gfx_clip_polygon_step $x
```

gfx_cohen_sutherland_code

gfx_cohen_sutherland_code — operation `gfx cohen sutherland code`. Category: *climate, fluids, atmospheric*.

```
p gfx_cohen_sutherland_code $x
```

gfx_cook_torrance_d_ggx

gfx_cook_torrance_d_ggx — operation `gfx cook torrance d ggx`. Category: *climate, fluids, atmospheric*.

```
p gfx_cook_torrance_d_ggx $x
```

gfx_cook_torrance_f_schlick

gfx_cook_torrance_f_schlick — operation `gfx cook torrance f schlick`. Category: *climate, fluids, atmospheric*.

```
p gfx_cook_torrance_f_schlick $x
```

gfx_cook_torrance_g_smith

gfx_cook_torrance_g_smith — operation `gfx cook torrance g smith`. Category: *climate, fluids, atmospheric*.

```
p gfx_cook_torrance_g_smith $x
```

gfx_cube_map_face_index

gfx_cube_map_face_index — index of `gfx cube map face`. Category: *climate, fluids, atmospheric*.

```
p gfx_cube_map_face_index $x
```

gfx_curl_noise_step

gfx_curl_noise_step — single-step update of `gfx curl noise`. Category: *climate, fluids, atmospheric*.

```
p gfx_curl_noise_step $x
```

gfx_disney_principled_d

gfx_disney_principled_d — operation `gfx disney principled d`. Category: *climate, fluids, atmospheric*.

```
p gfx_disney_principled_d $x
```

gfx_dither_bayer_4x4

gfx_dither_bayer_4x4 — operation `gfx dither bayer 4x4`. Category: *climate, fluids, atmospheric*.

```
p gfx_dither_bayer_4x4 $x
```

gfx_dither_floyd_steinberg

gfx_dither_floyd_steinberg — operation `gfx dither floyd steinberg`. Category: *climate, fluids, atmospheric*.

```
p gfx_dither_floyd_steinberg $x
```

gfx_environment_map_uv_u

gfx_environment_map_uv_u — operation `gfx environment map uv u`. Category: *climate, fluids, atmospheric*.

```
p gfx_environment_map_uv_u $x
```

gfx_environment_map_uv_v

gfx_environment_map_uv_v — operation `gfx environment map uv v`. Category: *climate, fluids, atmospheric*.

```
p gfx_environment_map_uv_v $x
```

gfx_euler_to_quat_w

gfx_euler_to_quat_w — convert from `gfx euler` to `quat w`. Category: *climate, fluids, atmospheric*.

```
p gfx_euler_to_quat_w $value
```

gfx_euler_to_quat_x

gfx_euler_to_quat_x — convert from `gfx euler` to `quat x`. Category: *climate, fluids, atmospheric*.

```
p gfx_euler_to_quat_x $value
```

gfx_euler_to_quat_y

`gfx_euler_to_quat_y` — convert from `gfx euler` to `quat y`. Category: *climate, fluids, atmospheric*.

```
p gfx_euler_to_quat_y $value
```

gfx_euler_to_quat_z

`gfx_euler_to_quat_z` — convert from `gfx euler` to `quat z`. Category: *climate, fluids, atmospheric*.

```
p gfx_euler_to_quat_z $value
```

gfx_fbm_noise_step

`gfx_fbm_noise_step` — single-step update of `gfx fbm noise`. Category: *climate, fluids, atmospheric*.

```
p gfx_fbm_noise_step $x
```

gfx_fresnel_conductor_step

`gfx_fresnel_conductor_step` — single-step update of `gfx fresnel conductor`. Category: *climate, fluids, atmospheric*.

```
p gfx_fresnel_conductor_step $x
```

gfx_fresnel_dielectric_step

`gfx_fresnel_dielectric_step` — single-step update of `gfx fresnel dielectric`. Category: *climate, fluids, atmospheric*.

```
p gfx_fresnel_dielectric_step $x
```

gfx_gamma_correct_step

`gfx_gamma_correct_step` — single-step update of `gfx gamma correct`. Category: *climate, fluids, atmospheric*.

```
p gfx_gamma_correct_step $x
```

gfx_geometric_attenuation_smith

`gfx_geometric_attenuation_smith` — operation `gfx geometric attenuation smith`. Category: *climate, fluids, atmospheric*.

```
p gfx_geometric_attenuation_smith $x
```

gfx_gradient_noise_step

`gfx_gradient_noise_step` — single-step update of `gfx gradient noise`. Category: *climate, fluids, atmospheric*.

```
p gfx_gradient_noise_step $x
```

gfx_halton_step

`gfx_halton_step` — single-step update of `gfx halton`. Category: *climate, fluids, atmospheric*.

```
p gfx_halton_step $x
```


gfx_homogeneous_divide

`gfx_homogeneous_divide` — operation `gfx homogeneous divide`. Category: **climate, fluids, atmospheric**.

```
p gfx_homogeneous_divide $x
```

gfx_index_of_refraction

`gfx_index_of_refraction` — operation `gfx index of refraction`. Category: **climate, fluids, atmospheric**.

```
p gfx_index_of_refraction $x
```

gfx_irradiance_sh_eval

`gfx_irradiance_sh_eval` — operation `gfx irradiance sh eval`. Category: **climate, fluids, atmospheric**.

```
p gfx_irradiance_sh_eval $x
```

gfx_lambert_term

`gfx_lambert_term` — operation `gfx lambert term`. Category: **climate, fluids, atmospheric**.

```
p gfx_lambert_term $x
```

gfx_liang_barsky_t

`gfx_liang_barsky_t` — operation `gfx liang barsky t`. Category: **climate, fluids, atmospheric**.

```
p gfx_liang_barsky_t $x
```

gfx_linear_to_srgb

`gfx_linear_to_srgb` — convert from `gfx linear` to `srgb`. Category: **climate, fluids, atmospheric**.

```
p gfx_linear_to_srgb $value
```

gfx_lookat_forward

`gfx_lookat_forward` — operation `gfx lookat forward`. Category: **climate, fluids, atmospheric**.

```
p gfx_lookat_forward $x
```

gfx_lookat_right

`gfx_lookat_right` — operation `gfx lookat right`. Category: **climate, fluids, atmospheric**.

```
p gfx_lookat_right $x
```

gfx_lookat_up

`gfx_lookat_up` — operation `gfx lookat up`. Category: **climate, fluids, atmospheric**.

```
p gfx_lookat_up $x
```

gfx_low_discrepancy_step

`gfx_low_discrepancy_step` — single-step update of `gfx low discrepancy`. Category: **climate, fluids, atmospheric**.

```
p gfx_low_discrepancy_step $x
```

gfx_microfacet_brdf_step

`gfx_microfacet_brdf_step` — single-step update of `gfx microfacet brdf`. Category: *climate, fluids, atmospheric*.

```
p gfx_microfacet_brdf_step $x
```

gfx_ndc_to_screen_x

`gfx_ndc_to_screen_x` — convert from `gfx ndc` to `screen x`. Category: *climate, fluids, atmospheric*.

```
p gfx_ndc_to_screen_x $value
```

gfx_ndc_to_screen_y

`gfx_ndc_to_screen_y` — convert from `gfx ndc` to `screen y`. Category: *climate, fluids, atmospheric*.

```
p gfx_ndc_to_screen_y $value
```

gfx_normal_distribution_ggx

`gfx_normal_distribution_ggx` — operation `gfx normal distribution ggx`. Category: *climate, fluids, atmospheric*.

```
p gfx_normal_distribution_ggx $x
```

gfx_obb_overlap_step

`gfx_obb_overlap_step` — single-step update of `gfx obb overlap`. Category: *climate, fluids, atmospheric*.

```
p gfx_obb_overlap_step $x
```

gfx_octahedral_encode_x

`gfx_octahedral_encode_x` — operation `gfx octahedral encode x`. Category: *climate, fluids, atmospheric*.

```
p gfx_octahedral_encode_x $x
```

gfx_octahedral_encode_y

`gfx_octahedral_encode_y` — operation `gfx octahedral encode y`. Category: *climate, fluids, atmospheric*.

```
p gfx_octahedral_encode_y $x
```

gfx_oklab_a_step

`gfx_oklab_a_step` — single-step update of `gfx oklab a`. Category: *climate, fluids, atmospheric*.

```
p gfx_oklab_a_step $x
```

gfx_oklab_b_step

`gfx_oklab_b_step` — single-step update of `gfx oklab b`. Category: *climate, fluids, atmospheric*.

```
p gfx_oklab_b_step $x
```

gfx_oklab_l_step

`gfx_oklab_l_step` — single-step update of `gfx oklab l`. Category: *climate, fluids, atmospheric*.

```
p gfx_oklab_l_step $x
```

gfx_oklch_chroma

`gfx_oklch_chroma` — operation `gfx oklch chroma`. Category: *climate, fluids, atmospheric*.

```
p gfx_oklch_chroma $x
```

gfx_oklch_hue

`gfx_oklch_hue` — operation `gfx oklch hue`. Category: *climate, fluids, atmospheric*.

```
p gfx_oklch_hue $x
```

gfx_oren_nayar_term

`gfx_oren_nayar_term` — operation `gfx oren nayar term`. Category: *climate, fluids, atmospheric*.

```
p gfx_oren_nayar_term $x
```

gfx_orthographic_proj

`gfx_orthographic_proj` — operation `gfx orthographic proj`. Category: *climate, fluids, atmospheric*.

```
p gfx_orthographic_proj $x
```

gfx_pcg_hash_step

`gfx_pcg_hash_step` — single-step update of `gfx pcg hash`. Category: *climate, fluids, atmospheric*.

```
p gfx_pcg_hash_step $x
```

gfx_perlin_noise_step

`gfx_perlin_noise_step` — single-step update of `gfx perlin noise`. Category: *climate, fluids, atmospheric*.

```
p gfx_perlin_noise_step $x
```

gfx_perspective_proj_x

`gfx_perspective_proj_x` — operation `gfx perspective proj x`. Category: *climate, fluids, atmospheric*.

```
p gfx_perspective_proj_x $x
```

gfx_perspective_proj_y

`gfx_perspective_proj_y` — operation `gfx perspective proj y`. Category: *climate, fluids, atmospheric*.

```
p gfx_perspective_proj_y $x
```

gfx_phong_ambient_step

`gfx_phong_ambient_step` — single-step update of `gfx phong ambient`. Category: *climate, fluids, atmospheric*.

```
p gfx_phong_ambient_step $x
```

gfx_phong_diffuse_step

`gfx_phong_diffuse_step` — single-step update of `gfx phong diffuse`. Category: *climate, fluids, atmospheric*.

```
p gfx_phong_diffuse_step $x
```

gfx_phong_specular_step

`gfx_phong_specular_step` — single-step update of `gfx phong specular`. Category: *climate, fluids, atmospheric*.

```
p gfx_phong_specular_step $x
```

gfx_quat_dot_two

`gfx_quat_dot_two` — operation `gfx quat dot two`. Category: *climate, fluids, atmospheric*.

```
p gfx_quat_dot_two $x
```

gfx_quat_inverse_step

`gfx_quat_inverse_step` — single-step update of `gfx quat inverse`. Category: *climate, fluids, atmospheric*.

```
p gfx_quat_inverse_step $x
```

gfx_quat_nlerp_step

`gfx_quat_nlerp_step` — single-step update of `gfx quat nlerp`. Category: *climate, fluids, atmospheric*.

```
p gfx_quat_nlerp_step $x
```

gfx_quat_slerp_step

`gfx_quat_slerp_step` — single-step update of `gfx quat slerp`. Category: *climate, fluids, atmospheric*.

```
p gfx_quat_slerp_step $x
```

gfx_quat_to_axis_angle

`gfx_quat_to_axis_angle` — convert from `gfx quat` to `axis angle`. Category: *climate, fluids, atmospheric*.

```
p gfx_quat_to_axis_angle $value
```

gfx_quat_to_euler_pitch

`gfx_quat_to_euler_pitch` — convert from `gfx quat` to `euler pitch`. Category: *climate, fluids, atmospheric*.

```
p gfx_quat_to_euler_pitch $value
```

gfx_quat_to_euler_roll

`gfx_quat_to_euler_roll` — convert from `gfx quat` to `euler roll`. Category: *climate, fluids, atmospheric*.

```
p gfx_quat_to_euler_roll $value
```

gfx_quat_to_euler_yaw

`gfx_quat_to_euler_yaw` — convert from `gfx quat` to `euler yaw`. Category: *climate, fluids, atmospheric*.

```
p gfx_quat_to_euler_yaw $value
```

gfx_radiance_sh_eval

`gfx_radiance_sh_eval` — operation `gfx radiance sh eval`. Category: *climate, fluids, atmospheric*.

```
p gfx_radiance_sh_eval $x
```

gfx_ray_box_t

gfx_ray_box_t — operation `gfx ray box t`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_box_t $x
```

gfx_ray_cone_t

gfx_ray_cone_t — operation `gfx ray cone t`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_cone_t $x
```

gfx_ray_cylinder_t

gfx_ray_cylinder_t — operation `gfx ray cylinder t`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_cylinder_t $x
```

gfx_ray_disk_t

gfx_ray_disk_t — operation `gfx ray disk t`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_disk_t $x
```

gfx_ray_ellipsoid_t

gfx_ray_ellipsoid_t — operation `gfx ray ellipsoid t`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_ellipsoid_t $x
```

gfx_ray_plane_t

gfx_ray_plane_t — operation `gfx ray plane t`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_plane_t $x
```

gfx_ray_sphere_t

gfx_ray_sphere_t — operation `gfx ray sphere t`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_sphere_t $x
```

gfx_ray_torus_t_approx

gfx_ray_torus_t_approx — operation `gfx ray torus t approx`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_torus_t_approx $x
```

gfx_ray_triangle_t

gfx_ray_triangle_t — operation `gfx ray triangle t`. Category: *climate, fluids, atmospheric*.

```
p gfx_ray_triangle_t $x
```

gfx_reflect_direction_x

gfx_reflect_direction_x — operation `gfx reflect direction x`. Category: *climate, fluids, atmospheric*.

```
p gfx_reflect_direction_x $x
```

gfx_refract_direction_x

gfx_refract_direction_x — operation `gfx refract direction x`. Category: *climate, fluids, atmospheric*.

```
p gfx_refract_direction_x $x
```

gfx_rotation_matrix_xx

gfx_rotation_matrix_xx — operation `gfx rotation matrix xx`. Category: *climate, fluids, atmospheric*.

```
p gfx_rotation_matrix_xx $x
```

gfx_rotation_matrix_yy

gfx_rotation_matrix_yy — operation `gfx rotation matrix yy`. Category: *climate, fluids, atmospheric*.

```
p gfx_rotation_matrix_yy $x
```

gfx_rotation_matrix_zz

gfx_rotation_matrix_zz — operation `gfx rotation matrix zz`. Category: *climate, fluids, atmospheric*.

```
p gfx_rotation_matrix_zz $x
```

gfx_scale_matrix_step

gfx_scale_matrix_step — single-step update of `gfx scale matrix`. Category: *climate, fluids, atmospheric*.

```
p gfx_scale_matrix_step $x
```

gfx_screen_space_x

gfx_screen_space_x — operation `gfx screen space x`. Category: *climate, fluids, atmospheric*.

```
p gfx_screen_space_x $x
```

gfx_screen_space_y

gfx_screen_space_y — operation `gfx screen space y`. Category: *climate, fluids, atmospheric*.

```
p gfx_screen_space_y $x
```

gfx_screen_to_ndc_x

gfx_screen_to_ndc_x — convert from `gfx screen` to `ndc x`. Category: *climate, fluids, atmospheric*.

```
p gfx_screen_to_ndc_x $value
```

gfx_screen_to_ndc_y

gfx_screen_to_ndc_y — convert from `gfx screen` to `ndc y`. Category: *climate, fluids, atmospheric*.

```
p gfx_screen_to_ndc_y $value
```

gfx_shear_matrix_xy

gfx_shear_matrix_xy — operation `gfx shear matrix xy`. Category: *climate, fluids, atmospheric*.

```
p gfx_shear_matrix_xy $x
```

gfx_signed_distance_box

`gfx_signed_distance_box` — operation `gfx signed distance box`. Category: *climate, fluids, atmospheric*.

```
p gfx_signed_distance_box $x
```

gfx_signed_distance_capsule

`gfx_signed_distance_capsule` — operation `gfx signed distance capsule`. Category: *climate, fluids, atmospheric*.

```
p gfx_signed_distance_capsule $x
```

gfx_signed_distance_sphere

`gfx_signed_distance_sphere` — operation `gfx signed distance sphere`. Category: *climate, fluids, atmospheric*.

```
p gfx_signed_distance_sphere $x
```

gfx_simplex_noise_step

`gfx_simplex_noise_step` — single-step update of `gfx simplex noise`. Category: *climate, fluids, atmospheric*.

```
p gfx_simplex_noise_step $x
```

gfx_skybox_uv_u

`gfx_skybox_uv_u` — operation `gfx skybox uv u`. Category: *climate, fluids, atmospheric*.

```
p gfx_skybox_uv_u $x
```

gfx_skybox_uv_v

`gfx_skybox_uv_v` — operation `gfx skybox uv v`. Category: *climate, fluids, atmospheric*.

```
p gfx_skybox_uv_v $x
```

gfx_snells_law_angle

`gfx_snells_law_angle` — operation `gfx snells law angle`. Category: *climate, fluids, atmospheric*.

```
p gfx_snells_law_angle $x
```

gfx_sobol_step

`gfx_sobol_step` — single-step update of `gfx sobol`. Category: *climate, fluids, atmospheric*.

```
p gfx_sobol_step $x
```

gfx_sphere_intersect_t

`gfx_sphere_intersect_t` — operation `gfx sphere intersect t`. Category: *climate, fluids, atmospheric*.

```
p gfx_sphere_intersect_t $x
```

gfx_spherical_harmonic_y00

`gfx_spherical_harmonic_y00` — operation `gfx spherical harmonic y00`. Category: *climate, fluids, atmospheric*.

```
p gfx_spherical_harmonic_y00 $x
```

gfx_spherical_harmonic_y10

gfx_spherical_harmonic_y10 — operation `gfx spherical harmonic y10`. Category: *climate, fluids, atmospheric*.

```
p gfx_spherical_harmonic_y10 $x
```

gfx_spherical_harmonic_y11

gfx_spherical_harmonic_y11 — operation `gfx spherical harmonic y11`. Category: *climate, fluids, atmospheric*.

```
p gfx_spherical_harmonic_y11 $x
```

gfx_spherical_harmonic_y20

gfx_spherical_harmonic_y20 — operation `gfx spherical harmonic y20`. Category: *climate, fluids, atmospheric*.

```
p gfx_spherical_harmonic_y20 $x
```

gfx_srgb_to_linear

gfx_srgb_to_linear — convert from `gfx srgb` to `linear`. Category: *climate, fluids, atmospheric*.

```
p gfx_srgb_to_linear $value
```

gfx_subsurface_scattering_term

gfx_subsurface_scattering_term — operation `gfx subsurface scattering term`. Category: *climate, fluids, atmospheric*.

```
p gfx_subsurface_scattering_term $x
```

gfx_sutherland_hodgman

gfx_sutherland_hodgman — operation `gfx sutherland hodgman`. Category: *climate, fluids, atmospheric*.

```
p gfx_sutherland_hodgman $x
```

gfx_tonemap_aces

gfx_tonemap_aces — operation `gfx tonemap aces`. Category: *climate, fluids, atmospheric*.

```
p gfx_tonemap_aces $x
```

gfx_tonemap_filmic

gfx_tonemap_filmic — operation `gfx tonemap filmic`. Category: *climate, fluids, atmospheric*.

```
p gfx_tonemap_filmic $x
```

gfx_tonemap_reinhard

gfx_tonemap_reinhard — operation `gfx tonemap reinhard`. Category: *climate, fluids, atmospheric*.

```
p gfx_tonemap_reinhard $x
```

gfx_tonemap_uncharted2

gfx_tonemap_uncharted2 — operation `gfx tonemap uncharted2`. Category: *climate, fluids, atmospheric*.


```
p gfx_tonemap_uncharted2 $x
```

gfx_total_internal_reflection

`gfx_total_internal_reflection` — operation `gfx total internal reflection`. Category: *climate, fluids, atmospheric*.

```
p gfx_total_internal_reflection $x
```

gfx_translation_matrix_step

`gfx_translation_matrix_step` — single-step update of `gfx translation matrix`. Category: *climate, fluids, atmospheric*.

```
p gfx_translation_matrix_step $x
```

gfx_translucent_falloff

`gfx_translucent_falloff` — operation `gfx translucent falloff`. Category: *climate, fluids, atmospheric*.

```
p gfx_translucent_falloff $x
```

gfx_value_noise_step

`gfx_value_noise_step` — single-step update of `gfx value noise`. Category: *climate, fluids, atmospheric*.

```
p gfx_value_noise_step $x
```

gfx_van_der_corput

`gfx_van_der_corput` — operation `gfx van der corput`. Category: *climate, fluids, atmospheric*.

```
p gfx_van_der_corput $x
```

gfx_view_matrix_step

`gfx_view_matrix_step` — single-step update of `gfx view matrix`. Category: *climate, fluids, atmospheric*.

```
p gfx_view_matrix_step $x
```

gfx_voronoi_distance

`gfx_voronoi_distance` — distance / dissimilarity metric of `gfx voronoi`. Category: *climate, fluids, atmospheric*.

```
p gfx_voronoi_distance $x
```

gfx_worley_noise_step

`gfx_worley_noise_step` — single-step update of `gfx worley noise`. Category: *climate, fluids, atmospheric*.

```
p gfx_worley_noise_step $x
```

gfx_xiaolin_wu_intensity

`gfx_xiaolin_wu_intensity` — operation `gfx xiaolin wu intensity`. Category: *climate, fluids, atmospheric*.

```
p gfx_xiaolin_wu_intensity $x
```

gfx_xorshift_step

`gfx_xorshift_step` — single-step update of `gfx xorshift`. Category: *climate, fluids, atmospheric*.

```
p gfx_xorshift_step $x
```

gfx_zonal_harmonic_step

`gfx_zonal_harmonic_step` — single-step update of `gfx zonal harmonic`. Category: *climate, fluids, atmospheric*.

```
p gfx_zonal_harmonic_step $x
```

gh_branches

`gh_branches(OWNER, REPO)` — paginated list of branches.

```
val @br = gh_branches("MenkeTechnologies", "zpwr")
p pluck(@br, "name")
```

gh_commits

`gh_commits(OWNER, REPO)` — paginated list of commits.

```
val @c = gh_commits("MenkeTechnologies", "zpwr")
p pluck(@c, "sha")
```

gh_contributors

`gh_contributors(OWNER, REPO)` — paginated list of contributors.

```
val @c = gh_contributors("MenkeTechnologies", "zpwr")
p pluck(@c, "login")
```

gh_emojis

`gh_emojis()` — { name => url } hashref of GitHub's emoji set.

gh_followers

`gh_followers(USER)` — paginated list of a user's followers (flat list of hashrefs).

```
val @f = gh_followers("MenkeTechnologies")
p "followers: " . len(@f)
```

gh_following

`gh_following(USER)` — paginated list of users a user is following.

```
val @f = gh_following("MenkeTechnologies")
p pluck(@f, "login")
```

gh_forks

`gh_forks(OWNER, REPO)` — paginated list of forks.

```
val @f = gh_forks("MenkeTechnologies", "zpwr")
```

gh_get

`gh_get(PATH)` — GET an arbitrary GitHub REST endpoint. `PATH` is a relative path (`/users/octocat`) or full URL. Returns parsed JSON as a stryke value (hashref/arrayref/scalar); 404 → `undef`. Authenticates via `GITHUB_TOKEN` env var when present (5000 req/hour); otherwise unauthenticated (60 req/hour).

```
val $u = gh_get("/users/MenkeTechnologies")
p $u->{public_repos}
```

gh_gist

`gh_gist(ID)` — fetch a single gist by ID.

```
val $g = gh_gist("aa5a315d61ae9438b18d")
```

gh_gists

`gh_gists(USER)` — paginated list of a user's public gists.

```
val @g = gh_gists("MenkeTechnologies")
```

gh_issues

`gh_issues(OWNER, REPO)` — paginated list of issues for a repo (includes PRs by GitHub convention; filter `_->{pull_request}` to separate).

```
val @issues = gh_issues("MenkeTechnologies", "zpwr")
var @real = grep { !exists _->{pull_request} } @issues
```

gh_langs

`gh_languages(OWNER, REPO)` (alias `gh_langs`) — `{ language => bytes }` hashref of language breakdown.

```
val $h = gh_languages("MenkeTechnologies", "zpwr")
for val $k (sort { $h->{_1} <=> $h->{_0} } keys %$h) { p "#{$k}: #{$h->{$k}}" }
```

gh_meta

`gh_meta()` — GitHub service metadata hashref (IP ranges, hooks, importer).

gh_org

`gh_org(ORG)` — fetch an organization's profile as a hashref.

```
val $o = gh_org("github")
p "#{$o->{login}}: #{$o->{public_repos}} repos"
```

gh_org_repos

`gh_org_repos(ORG)` — paginated list of an org's repos.

```
val @r = gh_org_repos("github")
p "#{len(@r)} repos"
```

gh_prs

`gh_prs(OWNER, REPO)` (alias `gh_pulls`) — paginated list of pull requests.

```
val @prs = gh_prs("MenkeTechnologies", "zpwr")
```

gh_rate_limit

`gh_rate_limit()` — current rate-limit status as a hashref. Use to check budget before launching a `pmap`.

```
val $rl = gh_rate_limit()
p $rl->{rate}{remaining}
```

gh_readme

`gh_readme(OWNER, REPO)` — README content as a decoded UTF-8 string (handles base64 transparently).

```
val $md = gh_readme("MenkeTechnologies", "zpwr")
p "#{len($md)} bytes"
```

gh_releases

`gh_releases(OWNER, REPO)` — paginated list of releases.

```
val @r = gh_releases("MenkeTechnologies", "zpwr")
p pluck(@r, "tag_name")
```

gh_repo

`gh_repo(OWNER, REPO)` or `gh_repo("OWNER/REPO")` — fetch a single repo as a hashref. 404 → `undef`.

```
val $r = gh_repo("MenkeTechnologies/zpwr")
p "★ #{ $r->{stargazers_count} } ♡ #{ $r->{forks_count} } lang #{ $r->{language} }"
```

gh_repos

`gh_repos(USER)` — paginated list of a user's public repos (flat list of hashrefs). Pairs naturally with `pmap` for parallel per-repo work.

```
val @repos = gh_repos("MenkeTechnologies")
var @top = sort { _1->{stargazers_count} <=> _0->{stargazers_count} } @repos
p pluck(@top[0:9], "full_name")
```

gh_runs

`gh_runs(OWNER, REPO)` — array of workflow-run hashrefs (auto-extracts `workflow_runs`).

```
val @r = gh_runs("MenkeTechnologies", "zpwr")
p pluck(@r[0:9], "status")
```

gh_search_code

`gh_search_code(QUERY)` — paginated code search results (requires auth; pass `GITHUB_TOKEN`).

```
val @hits = gh_search_code("org:MenkeTechnologies fn main")
```

gh_search_issues

`gh_search_issues(QUERY)` — paginated issue / PR search results.

```
val @i = gh_search_issues("author:MenkeTechnologies is:pr is:merged")
```

gh_search_repos

gh_search_repos(QUERY) — paginated repo search results.

```
val @r = gh_search_repos("language:rust stars:>10000")
p pluck(\@r[0:9], "full_name")
```

gh_search_users

gh_search_users(QUERY) — paginated user search results.

```
val @u = gh_search_users("location:berlin language:rust")
```

gh_stargazers

gh_stargazers(OWNER, REPO) — paginated list of users who starred a repo.

```
val @s = gh_stargazers("MenkeTechnologies", "zpwir")
p pluck(\@s, "login")
```

gh_starred

gh_starred(USER) — paginated list of repos starred by USER.

```
val @s = gh_starred("MenkeTechnologies")
p pluck(\@s, "full_name")
```

gh_tags

gh_tags(OWNER, REPO) — paginated list of tags.

```
val @t = gh_tags("MenkeTechnologies", "zpwir")
p pluck(\@t, "name")
```

gh_topics

gh_topics(OWNER, REPO) — array of topic name strings (auto-extracts `names` key from the API response).

```
p gh_topics("MenkeTechnologies", "zpwir")
```

gh_user

gh_user(USERNAME) — fetch a user's profile as a hashref. 404 → `undef`.

```
val $u = gh_user("MenkeTechnologies")
p "#{$u->{login}}: #{ $u->{public_repos} } repos, #{ $u->{followers} } followers"
```

gh_workflows

gh_workflows(OWNER, REPO) — array of workflow hashrefs (auto-extracts `workflows` key).

```
val @w = gh_workflows("MenkeTechnologies", "zpwir")
p pluck(\@w, "name")
```

gh_zen

gh_zen() — fetch GitHub's design-philosophy random aphorism as a plain string.

```
p gh_zen()
```

ghz_state

`ghz_state` — operation `ghz state`. Category: **quantum**.

```
p ghz_state $x
```

ghz_state_n

`ghz_state_n` — operation `ghz state n`. Category: **more extensions**.

```
p ghz_state_n $x
```

gibbard_satterthwaite_check

`gibbard_satterthwaite_check` — operation `gibbard satterthwaite check`. Category: **climate, fluids, atmospheric**.

```
p gibbard_satterthwaite_check $x
```

gibbs_thomson_undercooling

`gibbs_thomson_undercooling` — operation `gibbs thomson undercooling`. Category: **electrochemistry, batteries, fuel cells**.

```
p gibbs_thomson_undercooling $x
```

gid

`gid` — operation `gid`. Category: **system introspection**.

```
p gid $x
```

gif_header_read

`gif_header_read(hex_bytes) → hash` — parse GIF magic+lsd → {format, version, width, height}.

gin

`gin` — operation `gin`. Alias for `get_in`. Category: **algebraic match**.

```
p gin $x
```

gini

`gini(@data)` (alias `gini_coefficient`) — computes the Gini coefficient measuring inequality in a distribution. Returns a value from 0 (perfect equality) to 1 (perfect inequality). Commonly used for income distribution analysis.

```
p gini(1, 1, 1, 1)      # 0 (perfect equality)
p gini(0, 0, 0, 100)    # ~0.75 (high inequality)
```

gini_impurity

`gini_impurity` — operation `gini impurity`. Category: **ml extensions**.

```
p gini_impurity $x
```

gini_inequality_health

`gini_inequality_health` — operation `gini inequality health`. Category: *epidemiology / public health*.

```
p gini_inequality_health $x
```

gini_market

`gini_market` — operation `gini market`. Category: *economics + game theory*.

```
p gini_market $x
```

girth

`girth` — operation `girth`. Category: *graph algorithms*.

```
p girth $x
```

girvan_newman

`girvan_newman` — operation `girvan newman`. Category: *networkx graph algorithms*.

```
p girvan_newman $x
```

gk15

`gk15` — operation `gk15`. Alias for `gauss_kronrod_15`. Category: *more extensions (2)*.

```
p gk15 $x
```

gl5

`gl5` — operation `gl5`. Alias for `gauss_legendre_5`. Category: *more extensions (2)*.

```
p gl5 $x
```

glaisher

`glaisher` — operation `glaisher`. Alias for `glaisher_constant`. Category: *math constants*.

```
p glaisher $x
```

glaisher_constant

`glaisher_constant` — operation `glaisher constant`. Category: *math constants*.

```
p glaisher_constant $x
```

glicko_rating

`glicko_rating` — operation `glicko rating`. Category: *astronomy / astrometry*.

```
p glicko_rating $x
```

glicko_rd_update

`glicko_rd_update(rd, c=34.6, t=1) → number` — Glicko RD growth over inactive periods: $\sqrt{(rd^2 + c^2 \cdot t)}$, capped at 350.

glicko_volatility

`glicko_volatility(sigma, delta, phi, v, tau=0.5)` → `number` — Glicko-2 volatility update (Illinois iteration on $f(x) = 0$).

glm

`glm` — operation `glm`. Category: **r / scipy distributions and tests**.

```
p glm $x
```

glob_to_regex

`glob_to_regex` — convert from `glob` to `regex`. Category: **more regex**.

```
p glob_to_regex $value
```

global_efficiency

`global_efficiency` — operation `global efficiency`. Category: **more extensions**.

```
p global_efficiency $x
```

glushkov_construction

`glushkov_construction` — operation `glushkov construction`. Category: **compilers / parsing**.

```
p glushkov_construction $x
```

gmm_moment_condition

`gmm_moment_condition` — operation `gmm moment condition`. Category: **econometrics**.

```
p gmm_moment_condition $x
```

gmm_moment_function

`gmm_moment_function` — operation `gmm moment function`. Category: **econometrics**.

```
p gmm_moment_function $x
```

gnomonic_number

`gnomonic_number(n)` → `int` — $2n - 1$. n -th odd number.

go_enrichment_p

`go_enrichment_p` — operation `go enrichment p`. Category: **bioinformatics deep**.

```
p go_enrichment_p $x
```

goals_above_avg

`goals_above_avg` — operation `goals above avg`. Category: **astronomy / astrometry**.

```
p goals_above_avg $x
```


god

god *EXPR* — omniscient runtime introspection. Returns a structured multi-line dump string showing information that **pp/ddump** do not surface: the canonical type tag, the heap pointer (so two refs to the same heap object are visibly aliases), the **Arc** strong / weak counts, a byte hex preview for byte buffers, generator state (**pc**, **exhausted**), pipeline state (queued ops, source length), and closure captures.

Cycle-safe. Every heap pointer descended through is tracked in a per-call **HashSet**; a back-edge to an already-visited pointer is annotated **...cycle** instead of recursing, so **god** terminates on self-referential structures.

Pure — returns the dump as a string. Pipe through **p / warn / die** when you want output. Cleanly composable with assertions: **god(\$x) =~ /strong=2/ |> aok**.

Sibling reflectors:

| Builtin | What it shows | |---|---| | **pp** | Human-friendly value formatting | | **ddump** | Deep structural dump (recursive value structure) | | **god** | Runtime metadata (type tag, heap ptr, refcounts, internals) |

Reach for **god** when debugging refcount surprises, aliasing bugs, closure-capture leaks, generator-pc state, or pipeline-op accumulation.

```
val $h = { a => 1, b => [10, 20], c => "hi" };
p god $h;                                     # HASH @ 0xXXXX (strong=1, ..., entries=3) + recursive

# Detect aliasing - two refs to the same Arc show identical heap pointers.
var $shared = [];
var $ref1 = \ $shared;
var $ref2 = \ $shared;
warn god $ref1;
warn god $ref2;                               # same 0x... for the inner ARRAY

# Cycle handling - self-referential hash terminates with an annotation.
var %cyc;
$cyc{self} = \%cyc;
p god \%cyc;                                  # ...cycle at the back-edge, no stack overflow

# Generator state mid-iteration.
var $g = gen { yield 1; yield 2; yield 3 };
$g->next;
p god $g;                                     # Generator @ 0xXXXX pc=1/3 exhausted=false
```

See also: **pp** (pretty), **ddump** (deep structure), **ref** (just the type tag).

goertzel

goertzel computes the magnitude of a single DFT frequency bin using the Goertzel algorithm. Much faster than full FFT when you need one frequency.

```
val $mag = goertzel(\@signal, 440, 44100) # 440 Hz component
```

goldb

goldb — operation **goldb**. Alias for **goldbach**. Category: **math / numeric extras**.

```
p goldb $x
```

goldbach

goldbach — operation **goldbach**. Category: **math / numeric extras**.

```
p goldbach $x
```

goldbach_pair

goldbach_pair — operation **goldbach** **pair**. Category: **cryptanalysis & number theory deep**.

```
p goldbach_pair $x
```

golden

`golden_section` (aliases `golden`, `gss`) finds the minimum of `f` on `[a,b]` via golden-section search.

```
val $xmin = golden(fn { (_-3)**2 }, 0, 10) # 3.0
```

golden_ratio

`golden_ratio` (alias `phi`) — $\varphi = (1+\sqrt{5})/2 \approx 1.618$. The golden ratio, appears throughout nature and art.

```
p phi # 1.618033988749895
# Fibonacci limit: fib(n)/fib(n-1) → φ
```

goldfeld_quandt

`goldfeld_quandt` — operation `goldfeld quandt`. Category: **econometrics**.

```
p goldfeld_quandt $x
```

golomb_rice_code

`golomb_rice_code` — operation `golomb rice code`. Category: **electrochemistry, batteries, fuel cells**.

```
p golomb_rice_code $x
```

golstep

`golstep` — operation `golstep`. Alias for `game_of_life_step`. Category: **algorithms / puzzles**.

```
p golstep $x
```

gomory_cut_step

`gomory_cut_step` — single-step update of `gomory cut`. Category: **combinatorial optimization, scheduling**.

```
p gomory_cut_step $x
```

gomory_hu_step

`gomory_hu_step` — single-step update of `gomory hu`. Category: **combinatorial optimization, scheduling**.

```
p gomory_hu_step $x
```

gompertz_growth_step

`gompertz_growth_step` — single-step update of `gompertz growth`. Category: **biology / ecology**.

```
p gompertz_growth_step $x
```

gompertz_pdf

`gompertz_pdf` — operation `gompertz pdf`. Category: **more extensions (2)**.

```
p gompertz_pdf $x
```

goto

`goto` — operation `goto`. Category: **control flow**.

```
p goto $x
```

gouy_chapman_potential

`gouy_chapman_potential` — operation `gouy chapman potential`. Category: **electrochemistry, batteries, fuel cells**.

```
p gouy_chapman_potential $x
```

gradient_descent_step

`gradient_descent_step` — single-step update of `gradient descent`. Category: **more extensions (2)**.

```
p gradient_descent_step $x
```

gradient_geothermal

`gradient_geothermal` — operation `gradient geothermal`. Category: **geology, seismology, mineralogy**.

```
p gradient_geothermal $x
```

gradient_magnitude_2d

`gradient_magnitude_2d(image)` → `matrix` — $\sqrt{(G_x^2 + G_y^2)}$ using Sobel kernels.

```
p gradient_magnitude_2d($img)
```

gradient_richardson_full

`gradient_richardson_full` — operation `gradient richardson full`. Category: **climate, fluids, atmospheric**.

```
p gradient_richardson_full $x
```

gradient_temporal_diff

`gradient_temporal_diff` — operation `gradient temporal diff`. Category: **electrochemistry, batteries, fuel cells**.

```
p gradient_temporal_diff $x
```

gradient_wind_step

`gradient_wind_step` — single-step update of `gradient wind`. Category: **climate, fluids, atmospheric**.

```
p gradient_wind_step $x
```

graham_2approx_bound

`graham_2approx_bound` — operation `graham 2approx bound`. Category: **combinatorial optimization, scheduling**.

```
p graham_2approx_bound $x
```

graham_scan_hull

`graham_scan_hull(points)` → `array` — convex hull via Graham scan. $O(n \log n)$.

```
p graham_scan_hull([[0,0],[1,0],[1,1],[0,1],[0.5,0.5]])
```

gram_schmidt

`gram_schmidt` — operation `gram_schmidt`. Alias for `orthogonalize_vectors`. Category: **misc**.

```
p gram_schmidt $x
```

granger_causality

`granger_causality` — operation `granger_causality`. Category: **statsmodels**.

```
p granger_causality $x
```

granger_causality_chi2

`granger_causality_chi2` — operation `granger_causality_chi2`. Category: **econometrics**.

```
p granger_causality_chi2 $x
```

graph_articulation_points

`graph_articulation_points(adj_list)` → `array` — cut vertices whose removal disconnects the graph (DFS with discovery/low values).

```
p graph_articulation_points($g)
```

graph_average_degree

`graph_average_degree` — graph algorithm `average_degree`. Category: **misc**.

```
p graph_average_degree $x
```

graph_avg_clustering

`graph_avg_clustering` — graph algorithm `avg_clustering`. Category: **more extensions (2)**.

```
p graph_avg_clustering $x
```

graph_bellman_ford

`graph_bellman_ford(weighted_adj, start)` → `array` — shortest-path distances; handles negative edges (no negative-cycle detection in simplified form).

```
p graph_bellman_ford($g, 0)
```

graph_betweenness

`graph_betweenness(adj_list)` → `array` — betweenness centrality (Brandes' algorithm) for unweighted graphs.

```
p graph_betweenness($g)
```

graph_bfs

`graph_bfs(adj_list, start)` → `array` — breadth-first traversal order starting at `start`.

```
p graph_bfs([[1,2],[3],[3],[ ]], 0) # [0,1,2,3]
```

graph_bipartite_match_simple

`graph_bipartite_match_simple` — graph algorithm `bipartite match simple`. Category: **more extensions (2)**.

```
p graph_bipartite_match_simple $x
```

graph_bridges

`graph_bridges(adj_list)` → `array` — bridge edges as `[[u,v], ...]`; removal increases component count.

```
p graph_bridges($g)
```

graph_closeness

`graph_closeness(adj_list, v)` → `number` — closeness centrality: $(n-1) / \sum \text{dist}(v, u)$.

```
p graph_closeness($g, 0)
```

graph_clustering_coefficient

`graph_clustering_coefficient(adj_list, v)` → `number` — local clustering: fraction of v's neighbour pairs that are themselves connected. $\in [0,1]$.

```
p graph_clustering_coefficient($g, 1)
```

graph_color_greedy

`graph_color_greedy(adj_list)` → `array` — first-fit graph colouring; returns a colour id per node. Number of colours used is at most `max_degree + 1`.

```
p graph_color_greedy($g)
```

graph_coloring_brooks_bound

`graph_coloring_brooks_bound` — graph algorithm `coloring brooks bound`. Category: **combinatorial optimization, scheduling**.

```
p graph_coloring_brooks_bound $x
```

graph_coloring_dsatur_step

`graph_coloring_dsatur_step` — graph algorithm `coloring dsatur step`. Category: **combinatorial optimization, scheduling**.

```
p graph_coloring_dsatur_step $x
```

graph_coloring_greedy

`graph_coloring_greedy` — graph algorithm `coloring greedy`. Category: **networkx graph algorithms**.

```
p graph_coloring_greedy $x
```

graph_coloring_lp_bound

`graph_coloring_lp_bound` — graph algorithm `coloring lp bound`. Category: **combinatorial optimization, scheduling**.

```
p graph_coloring_lp_bound $x
```

graph_coloring_welsh_powell

`graph_coloring_welsh_powell` — graph algorithm `coloring welsh powell`. Category: **combinatorial optimization, scheduling**.

```
p graph_coloring_welsh_powell $x
```

graph_complement

`graph_complement` — graph algorithm `complement`. Category: **misc**.

```
p graph_complement $x
```

graph_connected_components

`graph_connected_components(adj_list)` → `array` — component id per node (BFS over undirected graph).

```
p graph_connected_components([[1],[0],[3],[2]]) # [0,0,1,1]
```

graph_count_edges

`graph_count_edges` — graph algorithm `count edges`. Category: **more extensions (2)**.

```
p graph_count_edges $x
```

graph_count_paths_length_k

`graph_count_paths_length_k` — graph algorithm `count paths length k`. Category: **more extensions (2)**.

```
p graph_count_paths_length_k $x
```

graph_count_triangles

`graph_count_triangles` — graph algorithm `count triangles`. Category: **more extensions (2)**.

```
p graph_count_triangles $x
```

graph_cut_segment

`graph_cut_segment` — graph algorithm `cut segment`. Category: **pil/opencv image processing**.

```
p graph_cut_segment $x
```

graph_cycle_detect

`graph_cycle_detect(adj_list)` → `0|1` — 1 if the directed graph has a cycle (white/gray/black DFS).

```
p graph_cycle_detect([[1],[2],[0]]) # 1
```

graph_degree

`graph_degree(adj_list, v)` → `int` — number of outgoing neighbours.

```
p graph_degree([[1,2],[3],[],[1]], 0) # 2
```

graph_degree_distribution

`graph_degree_distribution` — graph algorithm `degree distribution`. Category: **more extensions (2)**.

```
p graph_degree_distribution $x
```

graph_density

`graph_density` — graph algorithm `density`. Category: **misc**.

```
p graph_density $x
```

graph_dfs

`graph_dfs(adj_list, start) → array` — depth-first traversal order (iterative).

```
p graph_dfs([[1,2],[3],[3],[1]], 0)
```

graph_dijkstra

`graph_dijkstra(weighted_adj, start) → array` — shortest-path distances from `start` (Dijkstra with binary heap). Returns $+\infty$ for unreachable nodes.

```
p graph_dijkstra($g, 0)
```

graph_eccentricity

`graph_eccentricity(weighted_adj, v) → number` — max shortest-path distance from `v` to any reachable node.

```
p graph_eccentricity($g, 0)
```

graph_eccentricity_all

`graph_eccentricity_all` — graph algorithm `eccentricity all`. Category: **misc**.

```
p graph_eccentricity_all $x
```

graph_eigenvector_centrality

`graph_eigenvector_centrality(adj_list, iters=100) → array` — power-iteration eigenvector centrality on the (possibly weighted) adjacency matrix: $x \leftarrow A \cdot x / \|A \cdot x\|$. Early-exits when $\|x_{\text{new}} - x_{\text{old}}\|_1 < 1e-12$. Pass `iters` to cap iterations.

```
p graph_eigenvector_centrality($g, 100)
```

graph_floyd_warshall

`graph_floyd_warshall(matrix) → matrix` — all-pairs shortest paths via Floyd-Warshall ($O(n^3)$). Input is an adjacency matrix where `0` indicates no edge and any non-zero value is the edge weight. Output cells are `Infinity` for unreachable pairs.

```
p graph_floyd_warshall($adj_matrix)
```

graph_from_edges

`graph_from_edges(edges, directed=0) → adj_list` — build adjacency list from `[[u, v, w?], ...]`. Edge weight defaults to 1.

```
p graph_from_edges([[0,1,4],[0,2,1],[1,3,1]], 0)
```

graph_has_path

graph_has_path(adj_list, s, t) → 0|1 — BFS reachability check.

```
p graph_has_path([[1],[2],[3],[ ]], 0, 3) # 1
```

graph_in_degree

graph_in_degree(adj_list, v) → int — incoming edge count (scan all adjacency lists).

```
p graph_in_degree($g, 3)
```

graph_independent_set_brute

graph_independent_set_brute — graph algorithm `independent set brute`. Category: *more extensions (2)*.

```
p graph_independent_set_brute $x
```

graph_is_bipartite

graph_is_bipartite(adj_list) → 0|1 — 2-colouring BFS; 1 if no odd-length cycle.

```
p graph_is_bipartite([[1,3],[0,2],[1,3],[0,2]]) # 1
```

graph_is_connected

graph_is_connected(adj_list) → 0|1 — BFS-checks that all nodes are reachable from node 0.

```
p graph_is_connected([[1],[0,2],[1]]) # 1
```

graph_is_tree

graph_is_tree — graph algorithm `is tree`. Category: *misc*.

```
p graph_is_tree $x
```

graph_kosaraju

graph_kosaraju(adj_list) → array — SCC labelling via Kosaraju (alias of `graph_strongly_connected_components`).

```
p graph_kosaraju($g)
```

graph_kruskal_mst

graph_kruskal_mst(weighted_adj) → array — minimum spanning tree as list of [u, v, w] triples (Kruskal with union-find).

```
p graph_kruskal_mst($g)
```

graph_max_clique_brute

graph_max_clique_brute — graph algorithm `max clique brute`. Category: *more extensions (2)*.

```
p graph_max_clique_brute $x
```


graph_max_degree

graph_max_degree — graph algorithm `max degree`. Category: **misc**.

```
p graph_max_degree $x
```

graph_min_degree

graph_min_degree — graph algorithm `min degree`. Category: **misc**.

```
p graph_min_degree $x
```

graph_out_degree

graph_out_degree(adj_list, v) → int — alias for `graph_degree`.

```
p graph_out_degree($g, 0)
```

graph_pagerank

graph_pagerank(adj_list, damping=0.85, iters=50) → array — PageRank score per node via power iteration. Dangling nodes redistribute uniformly.

```
p graph_pagerank($g, 0.85, 100)
```

graph_pagerank_simple

graph_pagerank_simple — graph algorithm `pagerank simple`. Category: **more extensions (2)**.

```
p graph_pagerank_simple $x
```

graph_prim_mst

graph_prim_mst(weighted_adj) → array — minimum spanning tree as list of [u, v, w] (Prim with binary heap).

```
p graph_prim_mst($g)
```

graph_shortest_path

graph_shortest_path(adj_list, s, t) → array — BFS shortest path in **unweighted** graph; returns the node sequence.

```
p graph_shortest_path($g, 0, 5)
```

graph_strongly_connected_components

graph_strongly_connected_components(adj_list) → array — SCC id per node via Kosaraju's algorithm.

```
p graph_strongly_connected_components($g)
```

graph_tarjan

graph_tarjan(adj_list) → array — SCC labelling via Tarjan's lowlink algorithm; single DFS pass.

```
p graph_tarjan($g)
```

graph_to_adj_list

`graph_to_adj_list(matrix)` → `adj_list` — sparsify a weighted matrix to adjacency-list form.

```
p graph_to_adj_list([[0,1,0],[1,0,2],[0,2,0]])
```

graph_to_adj_matrix

`graph_to_adj_matrix(adj_list)` → `matrix` — densify adjacency list to `n*n` weighted matrix (0 = no edge).

```
p graph_to_adj_matrix($g)
```

graph_topological_sort

`graph_topological_sort(adj_list)` → `array` — Kahn's algorithm; returns empty list if a cycle exists.

```
p graph_topological_sort([[1,2],[3],[3],[]])
```

graph_transitivity

`graph_transitivity` — graph algorithm `transitivity`. Category: **more extensions (2)**.

```
p graph_transitivity $x
```

grashof_number

`grashof_number` — operation `grashof number`. Category: **misc**.

```
p grashof_number $x
```

grashof_number_step

`grashof_number_step` — single-step update of `grashof number`. Category: **climate, fluids, atmospheric**.

```
p grashof_number_step $x
```

grav_binding_energy

`grav_binding_energy` — operation `grav binding energy`. Category: **cosmology / gr / flrw**.

```
p grav_binding_energy $x
```

grav_dilation_factor

`grav_dilation_factor` — factor of `grav dilation`. Category: **cosmology / gr / flrw**.

```
p grav_dilation_factor $x
```

grav_redshift

`grav_redshift` — operation `grav redshift`. Category: **em / optics / relativity**.

```
p grav_redshift $x
```

grav_time_dilation

`grav_time_dilation` — operation `grav time dilation`. Category: **em / optics / relativity**.

```
p grav_time_dilation $x
```

gravitational_constant

`gravitational_constant` — operation `gravitational constant`. Category: **constants**.

```
p gravitational_constant $x
```

gravitational_pe

`gravitational_pe` — operation `gravitational pe`. Category: **misc**.

```
p gravitational_pe $x
```

gravitational_wave_quadrupole

`gravitational_wave_quadrupole` — operation `gravitational wave quadrupole`. Category: **electrochemistry, batteries, fuel cells**.

```
p gravitational_wave_quadrupole $x
```

gravity

`gravity` — operation `gravity`. Category: **math / physics constants**.

```
p gravity $x
```

gravity_at_radius

`gravity_at_radius` — operation `gravity at radius`. Category: **misc**.

```
p gravity_at_radius $x
```

gravity_constant

`gravity_constant` — operation `gravity constant`. Category: **extras**.

```
p gravity_constant $x
```

gray

`gray` — operation `gray`. Alias for `red`. Category: **color / ansi**.

```
p gray $x
```

gray2b

`gray2b` — operation `gray2b`. Alias for `gray_to_binary`. Category: **base / gray code**.

```
p gray2b $x
```

gray_code_decode

`gray_code_decode(g) → int` — reflected Gray → binary inverse.

gray_code_encode

`gray_code_encode(x) → int` — binary → reflected Gray code: $x \wedge (x \gg 1)$.

```
p gray_code_encode(7) # 4
```

gray_code_sequence

`gray_code_sequence` — operation `gray code sequence`. Category: **base / gray code**.

```
p gray_code_sequence $x
```

gray_to_bin

`gray_to_bin` — convert from `gray` to `bin`. Category: **misc**.

```
p gray_to_bin $value
```

gray_to_binary

`gray_to_binary` — convert from `gray` to `binary`. Category: **base / gray code**.

```
p gray_to_binary $value
```

grayseq

`grayseq` — operation `grayseq`. Alias for `gray_code_sequence`. Category: **base / gray code**.

```
p grayseq $x
```

great_circle_law_of_cos

`great_circle_law_of_cos` — operation `great circle law of cos`. Category: **geometry / topology**.

```
p great_circle_law_of_cos $x
```

greedy_best_first

`greedy_best_first(adj, s, t, heuristic) → array` — pure-heuristic search; not guaranteed optimal but fast.

greedy_coloring

`greedy_coloring` — operation `greedy coloring`. Category: **graph algorithms**.

```
p greedy_coloring $x
```

greedy_edge_tour

`greedy_edge_tour` — operation `greedy edge tour`. Category: **combinatorial optimization, scheduling**.

```
p greedy_edge_tour $x
```

greedy_set_cover_round

`greedy_set_cover_round` — operation `greedy set cover round`. Category: **combinatorial optimization, scheduling**.

```
p greedy_set_cover_round $x
```

green

green — operation `green`. Alias for `red`. Category: **color / ansi**.

```
p green $x
```

green_bold

green_bold — operation `green bold`. Alias for `red`. Category: **color / ansi**.

```
p green_bold $x
```

greenhouse_forcing_step

greenhouse_forcing_step — single-step update of `greenhouse forcing`. Category: **climate, fluids, atmospheric**.

```
p greenhouse_forcing_step $x
```

gregorian_from_fixed

gregorian_from_fixed — operation `gregorian from fixed`. Category: **calendrical algorithms**.

```
p gregorian_from_fixed $x
```

grepv

grepv — operation `grepv`. Alias for `take_while`. Category: **functional / iterator**.

```
p grepv $x
```

grey

grey — operation `grey`. Alias for `red`. Category: **color / ansi**.

```
p grey $x
```

grim_trigger_step

grim_trigger_step — single-step update of `grim trigger`. Category: **climate, fluids, atmospheric**.

```
p grim_trigger_step $x
```

gromov_wasserstein_step

gromov_wasserstein_step — single-step update of `gromov wasserstein`. Category: **electrochemistry, batteries, fuel cells**.

```
p gromov_wasserstein_step $x
```

gromov_witten_invariant

gromov_witten_invariant — operation `gromov witten invariant`. Category: **econometrics**.

```
p gromov_witten_invariant $x
```

gross_premium_load

gross_premium_load — operation `gross premium load`. Category: **actuarial science**.

```
p gross_premium_load $x
```

group_by

`group_by` — operation `group by`. Category: **functional / iterator**.

```
p group_by $x
```

group_by_fn

`group_by_fn` — operation `group by fn`. Category: **go/general functional utilities**.

```
p group_by_fn $x
```

group_by_size

`group_by_size` — operation `group by size`. Category: **list helpers**.

```
p group_by_size $x
```

group_cohomology_dim

`group_cohomology_dim` — operation `group cohomology dim`. Category: **econometrics**.

```
p group_cohomology_dim $x
```

group_consecutive

`group_consecutive` — operation `group consecutive`. Category: **functional combinators**.

```
p group_consecutive $x
```

group_consecutive_by

`group_consecutive_by` — operation `group consecutive by`. Category: **array / list operations extras**.

```
p group_consecutive_by $x
```

group_delay_eval

`group_delay_eval` — operation `group delay eval`. Category: **economics + game theory**.

```
p group_delay_eval $x
```

group_homology_dim

`group_homology_dim` — operation `group homology dim`. Category: **econometrics**.

```
p group_homology_dim $x
```

group_map

`group_map` — operation `group map`. Category: **iterator + string-distance extras**.

```
p group_map $x
```

group_number

`group_number` — operation `group number`. Alias for `format_number`. Category: **file stat / path**.

```
p group_number $x
```

group_of_n

`group_of_n` — operation `group of n`. Category: **collection**.

```
p group_of_n $x
```

group_runs

`group_runs` — operation `group runs`. Category: **haskell list functions**.

```
p group_runs $x
```

groupby_iter

`groupby_iter` — operation `groupby iter`. Category: **python/ruby stdlib**.

```
p groupby_iter $x
```

groups

`groups` → list of supplementary group names for the current process. Wraps `libc::getgroups + libc::getgrgid_r` for each GID. Returns an array of strings; empty on Windows.

```
p "in groups: " . join(", ", groups)
```

grover_iter

`grover_iter` — operation `grover iter`. Category: **quantum**.

```
p grover_iter $x
```

growing_perpetuity

`growing_perpetuity` — operation `growing perpetuity`. Category: **misc**.

```
p growing_perpetuity $x
```

growth_rate_from_ratio

`growth_rate_from_ratio` — ratio of `growth rate from`. Category: **biology / ecology**.

```
p growth_rate_from_ratio $x
```

gruns

`gruns` — operation `gruns`. Alias for `group_runs`. Category: **haskell list functions**.

```
p gruns $x
```

gullstrand_painleve_step

`gullstrand_painleve_step` — single-step update of `gullstrand painleve`. Category: **electrochemistry, batteries, fuel cells**.

```
p gullstrand_painleve_step $x
```

gunning_fog

`gunning_fog` — operation `gunning fog`. Category: **misc**.

```
p gunning_fog $x
```

gutenberg_richter_b

`gutenberg_richter_b` — operation `gutenberg richter b`. Category: **geology, seismology, mineralogy**.

```
p gutenberg_richter_b $x
```

gw_strain_amplitude

`gw_strain_amplitude` — operation `gw strain amplitude`. Category: **cosmology / gr / flrw**.

```
p gw_strain_amplitude $x
```

gx_diameter

`gx_diameter` — operation `gx diameter`. Category: **networkx graph algorithms**.

```
p gx_diameter $x
```

gx_eccentricity

`gx_eccentricity` — operation `gx eccentricity`. Category: **networkx graph algorithms**.

```
p gx_eccentricity $x
```

gx_radius

`gx_radius` — operation `gx radius`. Category: **networkx graph algorithms**.

```
p gx_radius $x
```

gz

`gz` — operation `gz`. Alias for `gzip`. Category: **steganography**.

```
p gz $x
```

gzip_crc32_init

`gzip_crc32_init` — initialise of `gzip crc32`. Category: **archive/encoding format primitives**.

```
p gzip_crc32_init $x
```

gzip_encode_step

`gzip_encode_step` — single-step update of `gzip encode`. Category: **b81-misc-utility**.

```
p gzip_encode_step $x
```


gzip_isize

`gzip_isize` — operation `gzip isize`. Category: *archive/encoding format primitives*.

```
p gzip_isize $x
```

gzip_member_step

`gzip_member_step` — single-step update of `gzip member`. Category: *archive/encoding format primitives*.

```
p gzip_member_step $x
```

h

`h` — operation `h`. Alias for `docs`. Category: *system info*.

```
p h $x
```

h0

`h0` — operation `h0`. Alias for `hubble_constant`. Category: *physics constants*.

```
p h0 $x
```

h3_geo_to_h3

`h3_geo_to_h3` — convert from `h3 geo` to `h3`. Category: *excel/sheets + bond/loan financial*.

```
p h3_geo_to_h3 $value
```

h3_h3_to_geo

`h3_h3_to_geo` — convert from `h3 h3` to `geo`. Category: *excel/sheets + bond/loan financial*.

```
p h3_h3_to_geo $value
```

h3_index

`h3_index` — index of `h3`. Category: *excel/sheets + bond/loan financial*.

```
p h3_index $x
```

h3_k_ring

`h3_k_ring` — operation `h3 k ring`. Category: *excel/sheets + bond/loan financial*.

```
p h3_k_ring $x
```

h3_neighbor

`h3_neighbor` — operation `h3 neighbor`. Category: *excel/sheets + bond/loan financial*.

```
p h3_neighbor $x
```

h3_resolution

`h3_resolution` — operation `h3 resolution`. Category: *excel/sheets + bond/loan financial*.

```
p h3_resolution $x
```

h_infinity_norm

`h_infinity_norm` — norm (vector length-like) of `h_infinity`. Category: *robotics & control*.

```
p h_infinity_norm $x
```

haar_2d_step

`haar_2d_step` — single-step update of `haar_2d`. Category: *more extensions*.

```
p haar_2d_step $x
```

hadamard_walsh_transform_step

`hadamard_walsh_transform_step` — single-step update of `hadamard_walsh_transform`. Category: *electrochemistry, batteries, fuel cells*.

```
p hadamard_walsh_transform_step $x
```

hadley_cell_max_lat

`hadley_cell_max_lat` — operation `hadley_cell_max_lat`. Category: *climate, fluids, atmospheric*.

```
p hadley_cell_max_lat $x
```

hagen_poiseuille_eo

`hagen_poiseuille_eo` — operation `hagen_poiseuille_eo`. Category: *electrochemistry, batteries, fuel cells*.

```
p hagen_poiseuille_eo $x
```

half

`half` — operation `half`. Category: *trivial numeric / predicate builtins*.

```
p half $x
```

half_cauchy_pdf

`half_cauchy_pdf` — operation `half_cauchy_pdf`. Category: *more extensions (2)*.

```
p half_cauchy_pdf $x
```

half_each

`half_each` — operation `half_each`. Category: *trivial numeric helpers*.

```
p half_each $x
```

half_logistic_pdf

`half_logistic_pdf` — operation `half_logistic_pdf`. Category: *more extensions (2)*.

```
p half_logistic_pdf $x
```

half_reaction_balance

`half_reaction_balance` — operation `half_reaction_balance`. Category: *chemistry & biochemistry*.

```
p half_reaction_balance $x
```

hall_inner_product_two

hall_inner_product_two — operation `hall inner product two`. Category: **econometrics**.

```
p hall_inner_product_two $x
```

halley

halley — operation `halley`. Alias for `halley_root`. Category: **more extensions (2)**.

```
p halley $x
```

halley_root

halley_root — operation `halley root`. Category: **more extensions (2)**.

```
p halley_root $x
```

hamdist

hamdist — operation `hamdist`. Category: **extended stdlib**.

```
p hamdist $x
```

hamdist_int

hamdist_int — operation `hamdist int`. Alias for `hamming_distance_int`. Category: **misc**.

```
p hamdist_int $x
```

hamiltonian_brute

hamiltonian_brute — operation `hamiltonian brute`. Category: **misc**.

```
p hamiltonian_brute $x
```

hamiltonian_path

hamiltonian_path — operation `hamiltonian path`. Category: **networkx graph algorithms**.

```
p hamiltonian_path $x
```

hamming

`window_hamming($n)` (alias `hamming`) — generates a Hamming window of length `n`. Similar to Hann but with slightly different coefficients, optimized for speech processing.

```
val $w = hamming(1024)
```

hamming_distance

hamming_distance — distance / dissimilarity metric of `hamming`. Category: **string**.

```
p hamming_distance $x
```

hamming_distance_int

hamming_distance_int — operation `hamming distance int`. Category: **misc**.

```
p hamming_distance_int $x
```

hamming_distance_str

hamming_distance_str — operation `hamming distance str`. Category: **geometry / topology**.

```
p hamming_distance_str $x
```

hamming_protein

hamming_protein — operation `hamming protein`. Category: **bioinformatics deep**.

```
p hamming_protein $x
```

hamming_w

hamming_w — operation `hamming w`. Category: **economics + game theory**.

```
p hamming_w $x
```

hamming_weight

hamming_weight — operation `hamming weight`. Category: **misc**.

```
p hamming_weight $x
```

hamming_window

hamming_window — operation `hamming window`. Category: **signal processing**.

```
p hamming_window $x
```

hann

`window_hann($n)` (alias `hann`) — generates a Hann (raised cosine) window of length `n`. Common window function for spectral analysis that reduces spectral leakage.

```
val $w = hann(1024)
```

hann_w

hann_w — operation `hann w`. Category: **economics + game theory**.

```
p hann_w $x
```

hann_window

hann_window — operation `hann window`. Category: **signal processing**.

```
p hann_window $x
```

hannan_quinn_ic

`hannan_quinn_ic` — operation `hannan quinn ic`. Category: **econometrics**.

```
p hannan_quinn_ic $x
```

hanoi

`hanoi` — operation `hanoi`. Alias for `tower_of_hanoi`. Category: **algorithms / puzzles**.

```
p hanoi $x
```

hansen_j_test

`hansen_j_test` — statistical test of `hansen j`. Category: **econometrics**.

```
p hansen_j_test $x
```

hard_sigmoid

`hard_sigmoid` — operation `hard sigmoid`. Category: **ml activation functions**.

```
p hard_sigmoid $x
```

hard_swish

`hard_swish` — operation `hard swish`. Category: **ml activation functions**.

```
p hard_swish $x
```

hardsigmoid

`hardsigmoid` — operation `hardsigmoid`. Alias for `hard_sigmoid`. Category: **ml activation functions**.

```
p hardsigmoid $x
```

hardswish

`hardswish` — operation `hardswish`. Alias for `hard_swish`. Category: **ml activation functions**.

```
p hardswish $x
```

harmonic

`harmonic_number` (alias `harmonic`) computes the n th harmonic number $H_n = 1 + 1/2 + 1/3 + \dots + 1/n$.

```
p harmonic(1)      # 1.0
p harmonic(10)     # 2.9290
p harmonic(100)    # 5.1874
```

harmonic centrality

`harmonic centrality` — operation `harmonic centrality`. Category: **networkx graph algorithms**.

```
p harmonic centrality $x
```

harmonic_mean

`harmonic_mean` — arithmetic mean of `harmonic`. Category: **file stat / path**.

```
p harmonic_mean @xs
```

harmonic_mean_arr

`harmonic_mean_arr` — operation `harmonic mean arr`. Category: **misc**.

```
p harmonic_mean_arr $x
```

harmonic_oscillator_energy

`harmonic_oscillator_energy` — operation `harmonic oscillator energy`. Category: **quantum mechanics deep**.

```
p harmonic_oscillator_energy $x
```

harmonic_seq_sum

`harmonic_seq_sum` — sum of `harmonic seq`. Category: **more extensions**.

```
p harmonic_seq_sum @xs
```

harmonics_partial

`harmonics_partial` — operation `harmonics partial`. Category: **music theory**.

```
p harmonics_partial $x
```

harris_corners

`harris_corners` — operation `harris corners`. Category: **pill/opencv image processing**.

```
p harris_corners $x
```

harris_response

`harris_response(image, k=0.04) → matrix` — Harris corner response $\det(M) - k \cdot \text{trace}(M)^2$ per pixel. M is the 2×2 structure tensor $\Sigma_w \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$ summed over a 3×3 window (without the window sum, $\det(M)$ would be identically zero per pixel and the response could not distinguish corners from edges).

hartle_thorne_metric

`hartle_thorne_metric` — metric of `hartle thorne`. Category: **electrochemistry, batteries, fuel cells**.

```
p hartle_thorne_metric $x
```

has_all_keys

`has_all_keys` — predicate: 1 if input has all keys, 0 otherwise. Category: **hash helpers**.

```
p has_all_keys $x
```

has_any_key

`has_any_key` — predicate: 1 if input has any key, 0 otherwise. Category: **hash helpers**.

```
p has_any_key $x
```

has_cycle_directed

`has_cycle_directed` — predicate: 1 if input has cycle directed, 0 otherwise. Category: **graph algorithms**.

```
p has_cycle_directed $x
```

has_cycle_graph

`has_cycle_graph` — predicate: 1 if input has cycle graph, 0 otherwise. Category: **extended stdlib**.

```
p has_cycle_graph $x
```

has_cycle_undirected

`has_cycle_undirected` — predicate: 1 if input has cycle undirected, 0 otherwise. Category: **graph algorithms**.

```
p has_cycle_undirected $x
```

has_duplicates

`has_duplicates` — predicate: 1 if input has duplicates, 0 otherwise. Category: **more list helpers**.

```
p has_duplicates $x
```

has_key

`has_key` — predicate: 1 if input has key, 0 otherwise. Category: **hash helpers**.

```
p has_key $x
```

has_stderr_tty

`has_stderr_tty` — predicate: 1 if input has stderr tty, 0 otherwise. Category: **process / env**.

```
p has_stderr_tty $x
```

has_stdin_tty

`has_stdin_tty` — predicate: 1 if input has stdin tty, 0 otherwise. Category: **process / env**.

```
p has_stdin_tty $x
```

has_stdout_tty

`has_stdout_tty` — predicate: 1 if input has stdout tty, 0 otherwise. Category: **process / env**.

```
p has_stdout_tty $x
```

hascyc

`hascyc` — operation `hascyc`. Alias for `has_cycle_graph`. Category: **extended stdlib**.

```
p hascyc $x
```

hashMapOf

`hashMapOf` — operation `hashMapOf`. Alias for `mapOf`. Category: **kotlin collections**.

```
p hashMapOf $x
```

hashSetOf

`hashSetOf` — operation `hashSetOf`. Alias for `setOf`. Category: **kotlin collections**.

```
p hashSetOf $x
```

hash_2d_int

`hash_2d_int(x, y) → int` — fast deterministic spatial hash on 32-bit integer coordinates.

hash_combine

`hash_combine` — hash-table / hashing helper `combine`. Category: **cryptanalysis & number theory deep**.

```
p hash_combine $x
```

hash_delete

`hash_delete` — hash-table / hashing helper `delete`. Category: **file stat / path**.

```
p hash_delete $x
```

hash_eq

`hash_eq` — hash-table / hashing helper `eq`. Category: **hash ops**.

```
p hash_eq $x
```

hash_filter_keys

`hash_filter_keys` — hash-table / hashing helper `filter keys`. Category: **hash helpers**.

```
p hash_filter_keys $x
```

hash_filter_values

`hash_filter_values` — hash-table / hashing helper `filter values`. Category: **hash helpers**.

```
p hash_filter_values $x
```

hash_from_list

`hash_from_list` — hash-table / hashing helper `from list`. Category: **list helpers**.

```
p hash_from_list $x
```

hash_from_pairs

`hash_from_pairs` — hash-table / hashing helper `from pairs`. Category: **hash ops**.

```
p hash_from_pairs $x
```

hash_insert

`hash_insert` — hash-table / hashing helper `insert`. Category: **file stat / path**.

```
p hash_insert $x
```


hash_map_values

`hash_map_values` — hash-table / hashing helper `map values`. Category: **hash helpers**.

```
p hash_map_values $x
```

hash_merge_deep

`hash_merge_deep` — hash-table / hashing helper `merge deep`. Category: **list helpers**.

```
p hash_merge_deep $x
```

hash_ring_add

`hr_add(HR, NAME...)` → number of new nodes inserted (duplicates skipped).

hash_ring_get

`hr_get(HR, KEY)` → owner node name, or `undef` if the ring is empty.

hash_ring_nodes

`hr_nodes(HR)` → list of currently-registered node names.

hash_ring_remove

`hr_remove(HR, NAME...)` → number of nodes removed.

hash_size

`hash_size` — hash-table / hashing helper `size`. Category: **hash ops**.

```
p hash_size $x
```

hash_to_list

`hash_to_list` — convert from `hash` to `list`. Category: **list helpers**.

```
p hash_to_list $value
```

hash_update

`hash_update` — hash-table / hashing helper `update`. Category: **file stat / path**.

```
p hash_update $x
```

hash_zip

`hash_zip` — hash-table / hashing helper `zip`. Category: **list helpers**.

```
p hash_zip $x
```

hausdorff_distance_2d

`hausdorff_distance_2d` — operation `hausdorff distance 2d`. Category: **more extensions (2)**.

```
p hausdorff_distance_2d $x
```

hausman_test

`hausman_test` — statistical test of `hausman`. Category: *econometrics*.

```
p hausman_test $x
```

havdist

`havdist` — operation `havdist`. Alias for `haversine_distance`. Category: *geometry (extended)*.

```
p havdist $x
```

haversine

`haversine_distance($lat1, $lon1, $lat2, $lon2)` (alias `haversine`) — computes the great-circle distance between two points on Earth given latitude/longitude in degrees. Returns distance in kilometers.

```
# NYC to LA
p haversine(40.7128, -74.0060, 34.0522, -118.2437)
# ~3940 km
```

haversine_dist

`haversine_dist` — operation `haversine dist`. Category: *excel/sheets + bond/loan financial*.

```
p haversine_dist $x
```

hawk_dove_payoff

`hawk_dove_payoff` — operation `hawk dove payoff`. Category: *climate, fluids, atmospheric*.

```
p hawk_dove_payoff $x
```

hawking_area_increase

`hawking_area_increase` — operation `hawking area increase`. Category: *electrochemistry, batteries, fuel cells*.

```
p hawking_area_increase $x
```

hclust

`hclust` — hierarchical clustering (average linkage). Takes a distance matrix. Returns merge list `[[i,j,height],...]`. Like R's `hclust()`.

```
val $merges = hclust(dist_mat([[0,0],[1,0],[10,10]]))
```

head_finding_collins

`head_finding_collins` — operation `head finding collins`. Category: *computational linguistics*.

```
p head_finding_collins $x
```

head_lines

`head_lines` — operation `head lines`. Category: *file stat / path*.

```
p head_lines $x
```

head_n

`head_n` — operation `head n`. Category: **list helpers**.

```
p head_n $x
```

head_str

`head_str` — operation `head str`. Alias for `left_str`. Category: **string**.

```
p head_str $x
```

heap_sift_down

`heap_sift_down` — binary-heap op `sift down`. Category: **misc**.

```
p heap_sift_down $x
```

heat_capacity_q

`heat_capacity_q` — operation `heat capacity q`. Category: **chemistry**.

```
p heat_capacity_q $x
```

heat_flow_radiogenic

`heat_flow_radiogenic` — operation `heat flow radiogenic`. Category: **geology, seismology, mineralogy**.

```
p heat_flow_radiogenic $x
```

heat_index

`heat_index` — index of `heat`. Category: **physics formulas**.

```
p heat_index $x
```

heat_index_celsius

`heat_index_celsius` — operation `heat index celsius`. Category: **misc**.

```
p heat_index_celsius $x
```

hebrew_leap_year

`hebrew_leap_year` — operation `hebrew leap year`. Category: **calendrical algorithms**.

```
p hebrew_leap_year $x
```

hebrew_year_length

`hebrew_year_length` — operation `hebrew year length`. Category: **calendrical algorithms**.

```
p hebrew_year_length $x
```

heckman_correction

`heckman_correction` — operation `heckman correction`. Category: **econometrics**.

```
p heckman_correction $x
```

heegaard_genus_lower

`heegaard_genus_lower` — operation `heegaard genus lower`. Category: **econometrics**.

```
p heegaard_genus_lower $x
```

heisenberg_min

`heisenberg_min` — minimum of `heisenberg`. Category: **quantum mechanics deep**.

```
p heisenberg_min $x
```

hellinger_distance_step

`hellinger_distance_step` — single-step update of `hellinger distance`. Category: **electrochemistry, batteries, fuel cells**.

```
p hellinger_distance_step $x
```

hellinger_kernel

`hellinger_kernel` — convolution kernel of `hellinger`. Category: **electrochemistry, batteries, fuel cells**.

```
p hellinger_kernel $x
```

helmholtz_capacitance

`helmholtz_capacitance` — operation `helmholtz capacitance`. Category: **electrochemistry, batteries, fuel cells**.

```
p helmholtz_capacitance $x
```

helmholtz_decomp_step

`helmholtz_decomp_step` — single-step update of `helmholtz decomp`. Category: **climate, fluids, atmospheric**.

```
p helmholtz_decomp_step $x
```

help

`help` — operation `help`. Alias for `docs`. Category: **system info**.

```
p help $x
```

henderson_base

`henderson_base` — operation `henderson base`. Category: **chemistry**.

```
p henderson_base $x
```

henderson_buffer

`henderson_buffer` — operation `henderson buffer`. Category: **chemistry & biochemistry**.

```
p henderson_buffer $x
```

henderson_hasselbalch_solve

`henderson_hasselbalch_solve` — operation `henderson hasselbalch solve`. Category: **electrochemistry, batteries, fuel cells**.

```
p henderson_hasselbalch_solve $x
```

henikoff_weight

henikoff_weight — operation [henikoff_weight](#). Category: *bioinformatics deep*.

```
p henikoff_weight $x
```

heptagonal_number

heptagonal_number(n) $\rightarrow$ int — $n \cdot (5n-3) / 2$.

herd_immunity_threshold

herd_immunity_threshold — operation [herd immunity threshold](#). Category: *biology / ecology*.

```
p herd_immunity_threshold $x
```

heron

heron — operation [heron](#). Alias for [heron_area](#). Category: *geometry / physics*.

```
p heron $x
```

heron_area

heron_area — operation [heron area](#). Category: *geometry / physics*.

```
p heron_area $x
```

heston_price_simple

heston_price_simple — operation [heston price simple](#). Category: *financial pricing models*.

```
p heston_price_simple $x
```

heteroskedasticity_test

heteroskedasticity_test — statistical test of [heteroskedasticity](#). Category: *econometrics*.

```
p heteroskedasticity_test $x
```

heterozygosity

heterozygosity — operation [heterozygosity](#). Category: *bioinformatics deep*.

```
p heterozygosity $x
```

heun_euler_step

heun_euler_step — single-step update of [heun euler](#). Category: *ode advanced*.

```
p heun_euler_step $x
```

heun_method

`heun_method` — operation `heun method`. Category: **more extensions (2)**.

```
p heun_method $x
```

heun_sde_step

`heun_sde_step` — single-step update of `heun sde`. Category: **ode advanced**.

```
p heun_sde_step $x
```

hex_lower

`hex_lower` — hex helper `lower`. Category: **file stat / path**.

```
p hex_lower $x
```

hex_of

`hex_of` — hex helper `of`. Alias for `to_hex`. Category: **base conversion**.

```
p hex_of $x
```

hex_to_bytes

`hex_to_bytes` — convert from `hex` to `bytes`. Category: **string helpers**.

```
p hex_to_bytes $value
```

hex_to_rgb

`hex_to_rgb` — convert from `hex` to `rgb`. Category: **color / ansi**.

```
p hex_to_rgb $value
```

hex_upper

`hex_upper` — hex helper `upper`. Category: **file stat / path**.

```
p hex_upper $x
```

hexagonal_number

`hexagonal_number(n) → int` — $n \cdot (2n-1)$.

hget

`hget` — operation `hget`. Category: **redis-flavour primitives**.

```
p hget $x
```

hgetall

`hgetall` — operation `hgetall`. Category: **redis-flavour primitives**.

```
p hgetall $x
```

hhi_concentration

`hhi_concentration` — operation `hhi concentration`. Category: **economics + game theory**.

```
p hhi_concentration $x
```

hicksian_demand

`hicksian_demand` — operation `hicksian demand`. Category: **economics + game theory**.

```
p hicksian_demand $x
```

hidden

`hidden` — operation `hidden`. Alias for `red`. Category: **color / ansi**.

```
p hidden $x
```

highpass_filter

`highpass_filter(@signal, $cutoff)` (alias `hpf`) — applies a simple high-pass filter to remove low-frequency components (DC offset, drift). Signal = original - lowpass.

```
val @with_dc = map { 5 + sin(_/10) } 1:100  
var $ac_only = hpf(@with_dc, 0.1)
```

highway_hash

`highway_hash` — hash function of `highway`. Category: **b81-misc-utility**.

```
p highway_hash $x
```

hilbert_envelope

`hilbert_envelope` — operation `hilbert envelope`. Category: **signal processing**.

```
p hilbert_envelope $x
```

hilbert_signal

`hilbert_signal` — operation `hilbert signal`. Category: **economics + game theory**.

```
p hilbert_signal $x
```

hill_equation

`hill_equation` — operation `hill equation`. Category: **misc**.

```
p hill_equation $x
```

hill_radius

`hill_radius` — operation `hill radius`. Category: **misc**.

```
p hill_radius $x
```

hill_sphere_radius

`hill_sphere_radius` — operation `hill sphere radius`. Category: **astronomy / astrometry**.

```
p hill_sphere_radius $x
```

hill_velocity

`hill_velocity` — operation `hill velocity`. Category: **cosmology / gr / flrw**.

```
p hill_velocity $x
```

hincrby

`hincrby` — operation `hincrby`. Category: **redis-flavour primitives**.

```
p hincrby $x
```

hindley_milner_step

`hindley_milner_step` — single-step update of `hindley milner`. Category: **logic, proof, sat/smt, type theory**.

```
p hindley_milner_step $x
```

hindu_lunisolar_month

`hindu_lunisolar_month` — operation `hindu lunisolar month`. Category: **calendrical algorithms**.

```
p hindu_lunisolar_month $x
```

hindu_solar_year

`hindu_solar_year` — operation `hindu solar year`. Category: **calendrical algorithms**.

```
p hindu_solar_year $x
```

hinge_loss

`hinge_loss` — operation `hinge loss`. Category: **ml extensions**.

```
p hinge_loss $x
```

hirschberg_lcs_length

`hirschberg_lcs_length` — operation `hirschberg lcs length`. Category: **bioinformatics deep**.

```
p hirschberg_lcs_length $x
```

hirzebruch_signature

`hirzebruch_signature` — operation `hirzebruch signature`. Category: **econometrics**.

```
p hirzebruch_signature $x
```

hist

`hist` — operation `hist`. Alias for `histogram`. Category: **array / list operations extras**.

```
p hist $x
```


histogram

`histogram` — operation `histogram`. Category: **array / list operations extras**.

```
p histogram $x
```

histogram_bin_edges

`histogram_bin_edges` — operation `histogram bin edges`. Category: **numpy + scipy.special**.

```
p histogram_bin_edges $x
```

histogram_bins

`histogram_bins` — operation `histogram bins`. Category: **list helpers**.

```
p histogram_bins $x
```

history

`history` / `history(N)` / `history(-N)` — REPL history access. No args: full history buffer as a list. Positive N: the last N lines. Negative N: the N-th-from-last line as a scalar. The REPL frontend pushes accepted lines into this buffer via `repl_history_push()`; from a non-REPL script, history is whatever the embedding application wires.

```
val @last10 = history(10)
p "prev: " . history(-1)
```

hits_authority

`hits_authority` — operation `hits authority`. Category: **networkx graph algorithms**.

```
p hits_authority $x
```

hits_hub

`hits_hub` — operation `hits hub`. Category: **networkx graph algorithms**.

```
p hits_hub $x
```

hits_simple

`hits_simple` — operation `hits simple`. Category: **graph algorithms**.

```
p hits_simple $x
```

hitting_set_greedy

`hitting_set_greedy` — operation `hitting set greedy`. Category: **combinatorial optimization, scheduling**.

```
p hitting_set_greedy $x
```

hk

`hk` — operation `hk`. Alias for `has_key`. Category: **hash helpers**.

```
p hk $x
```

hkdf_expand_step

`hkdf_expand_step` — single-step update of `hkdf_expand`. Category: *cryptography deep*.

```
p hkdf_expand_step $x
```

hkeys

`hkeys` — operation `hkeys`. Category: *redis-flavour primitives*.

```
p hkeys $x
```

hlen

`hlen` — operation `hlen`. Category: *redis-flavour primitives*.

```
p hlen $x
```

hll

`hll(PRECISION=14)` — HyperLogLog cardinality (distinct-count) sketch with $2^{\text{precision}}$ 8-bit registers. Precision clamped to `[4, 18]`; typical `14` gives ~1.3% relative error in 16 KB. Replaces the older slow `hyperloglog_pp_*` family with a HeapObject-backed primitive that is ~1000x faster on `_add` (no allocation per insert).

```
val $h = hll(14)
hll_add($h, "user_$") for 1..100_000
printf "%d\n", hll_count($h)          # ~99672 - ~0.33% error
```

Family: `hll_add`, `hll_count`, `hll_merge`, `hll_clear`, `hll_precision`, `hll_serialize`, `hll_deserialize`. Legacy aliases (`hyperloglog_pp_new`, `hyperloglog_pp_add`, `hyperloglog_pp_estimate`, `hyperloglog_new`, etc.) all route here.

hll_add

`hll_add(HLL, KEY)` — fold `KEY` into the sketch (one xxh3-64 + one register update). Returns the same HLL for chaining.

hll_clear

`hll_clear(HLL)` — zero every register.

hll_count

`hll_count(HLL)` → estimated number of distinct items seen. Uses alpha-corrected standard HLL with linear-counting fallback for small ranges.

hll_deserialize

`hll_deserialize(BYTES)` → `HllSketch` | `undef` — round-trip from `hll_serialize`.

hll_merge

`hll_merge(HLL, OTHER)` — register-wise max into the receiver (union of two distinct sets). Returns `1` on success, `0` on precision mismatch.

hll_precision

`hll_precision(HLL)` → integer precision parameter (4-18) — the sketch holds $2^{\text{precision}}$ registers.

hll_serialize

`hll_serialize(HLL)` → BYTES — `5TKHLL\x01` magic + 4-byte precision + register vector.

hlookup

`hlookup` — operation `hlookup`. Category: *excel/sheets + bond/loan financial*.

```
p hlookup $x
```

hma

`hma(prices, period=10)` → array — Hull moving average. Reduces lag relative to SMA/EMA via `WMA(2·WMA(half) - WMA(full), √period)`.

```
p hma(\@prices, 14)
```

hmac_md5_hex

`hmac_md5_hex(key, msg)` → string — HMAC-MD5 as lowercase hex.

hmac_sha1_hex

`hmac_sha1_hex(key, msg)` → string — HMAC-SHA1 as lowercase hex.

hmac_sha256_hex

`hmac_sha256_hex(key, msg)` → string — HMAC-SHA256 as lowercase hex (RFC 4231 compliant).

```
p hmac_sha256_hex("key", "hello")
```

hmac_sha384_hex

`hmac_sha384_hex(key, msg)` → string — HMAC-SHA384 as lowercase hex.

hmac_sha512_hex

`hmac_sha512_hex(key, msg)` → string — HMAC-SHA512 as lowercase hex.

hmac_step_xor

`hmac_step_xor` — HMAC keyed-hash variant `step xor`. Category: *more extensions (2)*.

```
p hmac_step_xor $x
```

hmset

`hmset` — operation `hmset`. Category: *redis-flavour primitives*.

```
p hmset $x
```

hochschild_zero

`hochschild_zero` — operation `hochschild zero`. Category: *econometrics*.

```
p hochschild_zero $x
```

`hodge_diamond_value`

`hodge_diamond_value` — value of `hodge diamond`. Category: **econometrics**.

```
p hodge_diamond_value $x
```

`hodge_inner_product_one`

`hodge_inner_product_one` — operation `hodge inner product one`. Category: **electrochemistry, batteries, fuel cells**.

```
p hodge_inner_product_one $x
```

`hodge_polynomial_eval`

`hodge_polynomial_eval` — operation `hodge polynomial eval`. Category: **econometrics**.

```
p hodge_polynomial_eval $x
```

`hodge_star_one_form`

`hodge_star_one_form` — operation `hodge star one form`. Category: **electrochemistry, batteries, fuel cells**.

```
p hodge_star_one_form $x
```

`hodes_lehmann`

`hodes_lehmann` — operation `hodes lehmann`. Category: **more extensions**.

```
p hodes_lehmann $x
```

`hog_features`

`hog_features` — operation `hog features`. Category: **pil/opencv image processing**.

```
p hog_features $x
```

`hohmann_dv1`

`hohmann_dv1` — operation `hohmann dv1`. Category: **more extensions**.

```
p hohmann_dv1 $x
```

`hohmann_dv2`

`hohmann_dv2` — operation `hohmann dv2`. Category: **more extensions**.

```
p hohmann_dv2 $x
```

`hohmann_total`

`hohmann_total` — operation `hohmann total`. Category: **more extensions**.

```
p hohmann_total $x
```

`holling_type1`

`holling_type1` — operation `holling type1`. Category: **biology / ecology**.

```
p holling_type1 $x
```

holling_type2

`holling_type2` — operation `holling type2`. Category: **biology / ecology**.

```
p holling_type2 $x
```

holling_type3

`holling_type3` — operation `holling type3`. Category: **biology / ecology**.

```
p holling_type3 $x
```

holographic_entanglement_step

`holographic_entanglement_step` — single-step update of `holographic entanglement`. Category: **electrochemistry, batteries, fuel cells**.

```
p holographic_entanglement_step $x
```

holt_winters_additive

`holt_winters_additive` — operation `holt winters additive`. Category: **statsmodels**.

```
p holt_winters_additive $x
```

holt_winters_multiplicative

`holt_winters_multiplicative` — operation `holt winters multiplicative`. Category: **statsmodels**.

```
p holt_winters_multiplicative $x
```

home_dir

`home_dir` — operation `home dir`. Category: **system introspection**.

```
p home_dir $x
```

homfly_evaluation

`homfly_evaluation` — operation `homfly evaluation`. Category: **econometrics**.

```
p homfly_evaluation $x
```

homology_rank

`homology_rank` — operation `homology rank`. Category: **econometrics**.

```
p homology_rank $x
```

homotopy_group_sphere_pi

`homotopy_group_sphere_pi` — operation `homotopy group sphere pi`. Category: **econometrics**.

```
p homotopy_group_sphere_pi $x
```

hooke

`hooke` — operation `hooke`. Alias for `spring_force`. Category: **physics formulas**.

```
p hooke $x
```

hopcroft_karp

hopcroft_karp — operation [hopcroft karp](#). Category: *graph algorithms*.

```
p hopcroft_karp $x
```

hopcroft_karp_phase

hopcroft_karp_phase — operation [hopcroft karp phase](#). Category: *combinatorial optimization, scheduling*.

```
p hopcroft_karp_phase $x
```

hopcroft_karp_step

hopcroft_karp_step — single-step update of [hopcroft karp](#). Category: *networkx graph algorithms*.

```
p hopcroft_karp_step $x
```

horizon_distance

horizon_distance — distance / dissimilarity metric of [horizon](#). Category: *more extensions*.

```
p horizon_distance $x
```

horn_schunck_step

horn_schunck_step — single-step update of [horn schunck](#). Category: *electrochemistry, batteries, fuel cells*.

```
p horn_schunck_step $x
```

horndeski_step

horndeski_step — single-step update of [horndeski](#). Category: *electrochemistry, batteries, fuel cells*.

```
p horndeski_step $x
```

horspool_search

horspool_search — operation [horspool search](#). Category: *misc*.

```
p horspool_search $x
```

hostname_str

hostname_str — operation [hostname str](#). Category: *more process / system*.

```
p hostname_str $x
```

hotelling_location_step

hotelling_location_step — single-step update of [hotelling location](#). Category: *climate, fluids, atmospheric*.

```
p hotelling_location_step $x
```

hotelling_price_step

`hotelling_price_step` — single-step update of `hotelling price`. Category: **climate, fluids, atmospheric**.

```
p hotelling_price_step $x
```

hough_accumulator_step

`hough_accumulator_step` — single-step update of `hough accumulator`. Category: **electrochemistry, batteries, fuel cells**.

```
p hough_accumulator_step $x
```

hough_circles

`hough_circles` — operation `hough circles`. Category: **pil/opencv image processing**.

```
p hough_circles $x
```

hough_lines

`hough_lines` — operation `hough lines`. Category: **pil/opencv image processing**.

```
p hough_lines $x
```

hours_to_days

`hours_to_days` — convert from `hours` to `days`. Category: **unit conversions**.

```
p hours_to_days $value
```

hours_to_minutes

`hours_to_minutes` — convert from `hours` to `minutes`. Category: **unit conversions**.

```
p hours_to_minutes $value
```

hours_to_seconds

`hours_to_seconds` — convert from `hours` to `seconds`. Category: **unit conversions**.

```
p hours_to_seconds $value
```

householder_root

`householder_root` — operation `householder root`. Category: **more extensions (2)**.

```
p householder_root $x
```

hp_to_watts

`hp_to_watts` — convert from `hp` to `watts`. Category: **file stat / path**.

```
p hp_to_watts $value
```

hr

`hr` — operation `hr`. Alias for `http_request`. Category: **data / network**.

```
p hr $x
```

hset

hset — operation **hset**. Category: *redis-flavour primitives*.

```
p hset $x
```

hsl2rgb

hsl2rgb — operation **hsl2rgb**. Alias for **hsl_to_rgb**. Category: *color operations*.

```
p hsl2rgb $x
```

hsl_to_rgb

hsl_to_rgb — convert from **hsl** to **rgb**. Category: *color operations*.

```
p hsl_to_rgb $value
```

hstore_to_array

hstore_to_array — convert from **hstore** to **array**. Category: *postgres sql strings, json, aggregates*.

```
p hstore_to_array $value
```

hsv2rgb

hsv2rgb — operation **hsv2rgb**. Alias for **hsv_to_rgb**. Category: *color operations*.

```
p hsv2rgb $x
```

hsv_to_rgb

hsv_to_rgb — convert from **hsv** to **rgb**. Category: *color operations*.

```
p hsv_to_rgb $value
```

html_canonical_url

html_canonical_url — HTML helper **canonical url**. Category: *extras*.

```
p html_canonical_url $x
```

html_decode

html_decode — HTML helper **decode**. Category: *extended stdlib*.

```
p html_decode $x
```

html_encode

html_encode — HTML helper **encode**. Category: *extended stdlib*.

```
p html_encode $x
```


html_extract_headings

html_extract_headings — HTML helper `extract headings`. Category: `*extras*`.

```
p html_extract_headings $x
```

html_extract_images

html_extract_images — HTML helper `extract images`. Category: `*extras*`.

```
p html_extract_images $x
```

html_extract_links

html_extract_links — HTML helper `extract links`. Category: `*extras*`.

```
p html_extract_links $x
```

html_extract_meta

html_extract_meta — HTML helper `extract meta`. Category: `*extras*`.

```
p html_extract_meta $x
```

html_extract_tables

html_extract_tables — HTML helper `extract tables`. Category: `*extras*`.

```
p html_extract_tables $x
```

html_extract_text

html_extract_text — HTML helper `extract text`. Category: `*extras*`.

```
p html_extract_text $x
```

html_extract_title

html_extract_title — HTML helper `extract title`. Category: `*extras*`.

```
p html_extract_title $x
```

html_inner_text

html_inner_text — HTML helper `inner text`. Category: `*extras*`.

```
p html_inner_text $x
```

html_meta_charset

html_meta_charset — HTML helper `meta charset`. Category: `*extras*`.

```
p html_meta_charset $x
```

html_meta_description

html_meta_description — HTML helper `meta description`. Category: `*extras*`.

```
p html_meta_description $x
```

html_meta_keywords

html_meta_keywords — HTML helper `meta keywords`. Category: `*extras*`.

```
p html_meta_keywords $x
```

html_meta_og

html_meta_og — HTML helper `meta og`. Category: `*extras*`.

```
p html_meta_og $x
```

html_meta_twitter

html_meta_twitter — HTML helper `meta twitter`. Category: `*extras*`.

```
p html_meta_twitter $x
```

html_minify

html_minify — HTML helper `minify`. Category: `*extras*`.

```
p html_minify $x
```

html_pretty

html_pretty — HTML helper `pretty`. Category: `*extras*`.

```
p html_pretty $x
```

html_sanitize

html_sanitize — HTML helper `sanitize`. Category: `*extras*`.

```
p html_sanitize $x
```

html_strip_scripts

html_strip_scripts — HTML helper `strip scripts`. Category: `*extras*`.

```
p html_strip_scripts $x
```

html_strip_styles

html_strip_styles — HTML helper `strip styles`. Category: `*extras*`.

```
p html_strip_styles $x
```

html_strip_tags

html_strip_tags — HTML helper `strip tags`. Category: `*extras*`.

```
p html_strip_tags $x
```

html_to_markdown

html_to_markdown — convert from `html` to `markdown`. Category: `*extras*`.

```
p html_to_markdown $value
```

html_to_text

`html_to_text` — convert from `html` to `text`. Category: **extras**.

```
p html_to_text $value
```

htmld

`htmld` — operation `htmld`. Alias for `html_decode`. Category: **extended stdlib**.

```
p htmld $x
```

htmlē

`htmlē` — operation `htmlē`. Alias for `html_encode`. Category: **extended stdlib**.

```
p htmlē $x
```

http_date_format

`http_date_format` — HTTP helper `date format`. Category: **network / ip / cidr**.

```
p http_date_format $x
```

http_date_parse

`http_date_parse` — HTTP helper `date parse`. Category: **network / ip / cidr**.

```
p http_date_parse $x
```

http_method_connect

`http_method_connect` — HTTP helper `method connect`. Category: **extras**.

```
p http_method_connect $x
```

http_method_delete

`http_method_delete` — HTTP helper `method delete`. Category: **extras**.

```
p http_method_delete $x
```

http_method_get

`http_method_get` — HTTP helper `method get`. Category: **extras**.

```
p http_method_get $x
```

http_method_has_body

`http_method_has_body` — HTTP helper `method has body`. Category: **network / ip / cidr**.

```
p http_method_has_body $x
```

http_method_head

`http_method_head` — HTTP helper `method head`. Category: **extras**.

```
p http_method_head $x
```

http_method_is_idempotent

http_method_is_idempotent — HTTP helper `method is idempotent`. Category: `*network / ip / cidr*`.

```
p http_method_is_idempotent $x
```

http_method_is_safe

http_method_is_safe — HTTP helper `method is safe`. Category: `*network / ip / cidr*`.

```
p http_method_is_safe $x
```

http_method_options

http_method_options — HTTP helper `method options`. Category: `*extras*`.

```
p http_method_options $x
```

http_method_patch

http_method_patch — HTTP helper `method patch`. Category: `*extras*`.

```
p http_method_patch $x
```

http_method_post

http_method_post — HTTP helper `method post`. Category: `*extras*`.

```
p http_method_post $x
```

http_method_put

http_method_put — HTTP helper `method put`. Category: `*extras*`.

```
p http_method_put $x
```

http_method_trace

http_method_trace — HTTP helper `method trace`. Category: `*extras*`.

```
p http_method_trace $x
```

http_status_accepted

http_status_accepted — HTTP helper `status accepted`. Category: `*extras*`.

```
p http_status_accepted $x
```

http_status_bad_gateway

http_status_bad_gateway — HTTP helper `status bad gateway`. Category: `*extras*`.

```
p http_status_bad_gateway $x
```

http_status_bad_request

http_status_bad_request — HTTP helper `status bad request`. Category: `*extras*`.

```
p http_status_bad_request $x
```

http_status_class

http_status_class — HTTP helper `status class`. Category: `*network / ip / cidr*`.

```
p http_status_class $x
```

http_status_conflict

http_status_conflict — HTTP helper `status conflict`. Category: `*extras*`.

```
p http_status_conflict $x
```

http_status_continue

http_status_continue — HTTP helper `status continue`. Category: `*extras*`.

```
p http_status_continue $x
```

http_status_created

http_status_created — HTTP helper `status created`. Category: `*extras*`.

```
p http_status_created $x
```

http_status_expectation_failed

http_status_expectation_failed — HTTP helper `status expectation failed`. Category: `*extras*`.

```
p http_status_expectation_failed $x
```

http_status_forbidden

http_status_forbidden — HTTP helper `status forbidden`. Category: `*extras*`.

```
p http_status_forbidden $x
```

http_status_found

http_status_found — HTTP helper `status found`. Category: `*extras*`.

```
p http_status_found $x
```

http_status_gateway_timeout

http_status_gateway_timeout — HTTP helper `status gateway timeout`. Category: `*extras*`.

```
p http_status_gateway_timeout $x
```

http_status_gone

http_status_gone — HTTP helper `status gone`. Category: `*extras*`.

```
p http_status_gone $x
```

http_status_http_version_not_supported

http_status_http_version_not_supported — HTTP helper `status http version not supported`. Category: `*extras*`.

```
p http_status_http_version_not_supported $x
```

http_status_im_a_teapot

http_status_im_a_teapot — HTTP helper status im a teapot. Category: *extras*.

```
p http_status_im_a_teapot $x
```

http_status_internal_server_error

http_status_internal_server_error — HTTP helper status internal server error. Category: *extras*.

```
p http_status_internal_server_error $x
```

http_status_is_client_error

http_status_is_client_error — HTTP helper status is client error. Category: *network / ip / cidr*.

```
p http_status_is_client_error $x
```

http_status_is_informational

http_status_is_informational — HTTP helper status is informational. Category: *network / ip / cidr*.

```
p http_status_is_informational $x
```

http_status_is_redirect

http_status_is_redirect — HTTP helper status is redirect. Category: *network / ip / cidr*.

```
p http_status_is_redirect $x
```

http_status_is_server_error

http_status_is_server_error — HTTP helper status is server error. Category: *network / ip / cidr*.

```
p http_status_is_server_error $x
```

http_status_is_success

http_status_is_success — HTTP helper status is success. Category: *network / ip / cidr*.

```
p http_status_is_success $x
```

http_status_length_required

http_status_length_required — HTTP helper status length required. Category: *extras*.

```
p http_status_length_required $x
```

http_status_method_not_allowed

http_status_method_not_allowed — HTTP helper status method not allowed. Category: *extras*.

```
p http_status_method_not_allowed $x
```

http_status_moved_permanently

http_status_moved_permanently — HTTP helper status moved permanently. Category: *extras*.

```
p http_status_moved_permanently $x
```

http_status_multiple_choices

http_status_multiple_choices — HTTP helper status multiple choices. Category: *extras*.

```
p http_status_multiple_choices $x
```

http_status_no_content

http_status_no_content — HTTP helper status no content. Category: *extras*.

```
p http_status_no_content $x
```

http_status_not_acceptable

http_status_not_acceptable — HTTP helper status not acceptable. Category: *extras*.

```
p http_status_not_acceptable $x
```

http_status_not_found

http_status_not_found — HTTP helper status not found. Category: *extras*.

```
p http_status_not_found $x
```

http_status_not_implemented

http_status_not_implemented — HTTP helper status not implemented. Category: *extras*.

```
p http_status_not_implemented $x
```

http_status_not_modified

http_status_not_modified — HTTP helper status not modified. Category: *extras*.

```
p http_status_not_modified $x
```

http_status_ok

http_status_ok — HTTP helper status ok. Category: *extras*.

```
p http_status_ok $x
```

http_status_partial_content

http_status_partial_content — HTTP helper status partial content. Category: *extras*.

```
p http_status_partial_content $x
```

http_status_payload_too_large

http_status_payload_too_large — HTTP helper status payload too large. Category: *extras*.

```
p http_status_payload_too_large $x
```

http_status_payment_required

http_status_payment_required — HTTP helper status payment required. Category: *extras*.

```
p http_status_payment_required $x
```

http_status_permanent_redirect

http_status_permanent_redirect — HTTP helper status permanent redirect. Category: *extras*.

```
p http_status_permanent_redirect $x
```

http_status_precondition_failed

http_status_precondition_failed — HTTP helper status precondition failed. Category: *extras*.

```
p http_status_precondition_failed $x
```

http_status_range_not_satisfiable

http_status_range_not_satisfiable — HTTP helper status range not satisfiable. Category: *extras*.

```
p http_status_range_not_satisfiable $x
```

http_status_see_other

http_status_see_other — HTTP helper status see other. Category: *extras*.

```
p http_status_see_other $x
```

http_status_service_unavailable

http_status_service_unavailable — HTTP helper status service unavailable. Category: *extras*.

```
p http_status_service_unavailable $x
```

http_status_switching_protocols

http_status_switching_protocols — HTTP helper status switching protocols. Category: *extras*.

```
p http_status_switching_protocols $x
```

http_status_temporary_redirect

http_status_temporary_redirect — HTTP helper status temporary redirect. Category: *extras*.

```
p http_status_temporary_redirect $x
```

http_status_text

http_status_text — HTTP helper status text. Category: *network / ip / cidr*.

```
p http_status_text $x
```

http_status_too_many_requests

http_status_too_many_requests — HTTP helper status too many requests. Category: *extras*.

```
p http_status_too_many_requests $x
```


http_status_unauthorized

http_status_unauthorized — HTTP helper `status unauthorized`. Category: **extras**.

```
p http_status_unauthorized $x
```

http_status_unprocessable_entity

http_status_unprocessable_entity — HTTP helper `status unprocessable entity`. Category: **extras**.

```
p http_status_unprocessable_entity $x
```

http_status_unsupported_media_type

http_status_unsupported_media_type — HTTP helper `status unsupported media type`. Category: **extras**.

```
p http_status_unsupported_media_type $x
```

http_status_uri_too_long

http_status_uri_too_long — HTTP helper `status uri too long`. Category: **extras**.

```
p http_status_uri_too_long $x
```

hu_moments

hu_moments(image) → array — all 7 Hu moment invariants (translation, scale, rotation invariant shape features). `h7` flips sign under reflection.

hubble_constant

hubble_constant — operation `hubble constant`. Category: **physics constants**.

```
p hubble_constant $x
```

hubble_distance_mpc

hubble_distance_mpc — operation `hubble distance mpc`. Category: **astronomy / music / color / units**.

```
p hubble_distance_mpc $x
```

hubble_distance_si

hubble_distance_si — operation `hubble distance si`. Category: **cosmology / gr / flrw**.

```
p hubble_distance_si $x
```

hubble_lcdm

hubble_lcdm — operation `hubble lcdm`. Category: **cosmology / gr / flrw**.

```
p hubble_lcdm $x
```

hubble_time

hubble_time — operation `hubble time`. Category: **cosmology / gr / flrw**.

```
p hubble_time $x
```

huber_m_estimator

huber_m_estimator — operation `huber m estimator`. Category: **more extensions**.

```
p huber_m_estimator $x
```

huffman_balanced_step

huffman_balanced_step — Huffman encoding `balanced step`. Category: **electrochemistry, batteries, fuel cells**.

```
p huffman_balanced_step $x
```

huffman_decode

huffman_decode(bits, table) → string — inverse Huffman decode using the same table.

huffman_encode

huffman_encode(s) → {bits, table} — Huffman encode a string. `bits` is a 0/1 string; `table` is `[[char, code], ...]`.

```
my $r = huffman_encode("hello")
```

hull_white_drift

hull_white_drift — operation `hull white drift`. Category: **financial pricing models**.

```
p hull_white_drift $x
```

human_bytes

human_bytes — operation `human bytes`. Alias for `format_bytes`. Category: **file stat / path**.

```
p human_bytes $x
```

human_duration

human_duration — operation `human duration`. Alias for `format_duration`. Category: **file stat / path**.

```
p human_duration $x
```

humidex

humidex — operation `humidex`. Category: **misc**.

```
p humidex $x
```

hungarian_method_step

hungarian_method_step — single-step update of `hungarian method`. Category: **combinatorial optimization, scheduling**.

```
p hungarian_method_step $x
```

hungarian_step

hungarian_step — single-step update of `hungarian`. Category: **networkx graph algorithms**.

```
p hungarian_step $x
```

hurst_exponent

`hurst_exponent(series)` → `number` — multi-scale R/S analysis: subdivides the series into dyadic chunk sizes (8, 16, 32, ..., n/2), averages rescaled-range R/S per size, then returns the OLS slope of $\log(R/S)$ vs $\log(n)$. $H \approx 0.5$ random walk; >0.5 trending; <0.5 mean-reverting. Returns UNDEF if $n < 16$ or fewer than 2 dyadic scales fit.

```
p hurst_exponent(\@returns)
```

hurst_exponent_estimate

`hurst_exponent_estimate` — operation `hurst exponent estimate`. Category: **climate, fluids, atmospheric**.

```
p hurst_exponent_estimate $x
```

hvals

`hvals` — operation `hvals`. Category: **redis-flavour primitives**.

```
p hvals $x
```

hw_expected_counts

`hw_expected_counts` — operation `hw expected counts`. Category: **bioinformatics deep**.

```
p hw_expected_counts $x
```

hxd

`hxd` — operation `hxd`. Alias for `hex_decode`. Category: **crypto / encoding**.

```
p hxd $x
```

hxe

`hxe` — operation `hxe`. Alias for `hex_encode`. Category: **crypto / encoding**.

```
p hxe $x
```

hydrogen_energy_n

`hydrogen_energy_n` — operation `hydrogen energy n`. Category: **quantum mechanics deep**.

```
p hydrogen_energy_n $x
```

hyp

`hyp` — operation `hyp`. Alias for `triangle_hypotenuse`. Category: **geometry / physics**.

```
p hyp $x
```

hyp1f1

`hyp1f1` — operation `hyp1f1`. Category: **numpy + scipy.special**.

```
p hyp1f1 $x
```

hyp2f1

`hyp2f1` — operation `hyp2f1`. Category: *numpy + scipy.special*.

```
p hyp2f1 $x
```

hyper0f1

`hyper0f1` — operation `hyper0f1`. Category: *special functions extra*.

```
p hyper0f1 $x
```

hyper1f1

`hyper1f1` — operation `hyper1f1`. Category: *special functions extra*.

```
p hyper1f1 $x
```

hyper2f1

`hyper2f1` — operation `hyper2f1`. Category: *special functions extra*.

```
p hyper2f1 $x
```

hyperfactorial

`hyperfactorial(n)` → int — $\prod i^i$ for $i=1..n$.

hyperfocal_distance

`hyperfocal_distance(focal_mm, aperture, coc_mm=0.03)` → number — focus distance where DoF extends to infinity.

hypergeom_1f1

`hypergeom_1f1(a, c, z)` → number — Kummer's confluent hypergeometric ${}_1F_1$.

```
p hypergeom_1f1(1, 2, 0.5)
```

hypergeom_2f1

`hypergeom_2f1(a, b, c, z)` → number — Gauss hypergeometric ${}_2F_1$ via power series. Requires $|z| < 1$.

```
p hypergeom_2f1(1, 1, 2, 0.5)
```

hyperoperation

`hyperoperation` — operation `hyperoperation`. Category: *math / number theory extras*.

```
p hyperoperation $x
```

hypot

`hypot` — operation `hypot`. Category: *trivial numeric / predicate builtins*.

```
p hypot $x
```

hz_to_midi

`hz_to_midi` — convert from `hz` to `midi`. Category: **astronomy / music / color / units**.

```
p hz_to_midi $value
```

i64_max

`i64_max` — maximum of `i64`. Category: **math / numeric extras**.

```
p i64_max $x
```

i64_min

`i64_min` — minimum of `i64`. Category: **math / numeric extras**.

```
p i64_min $x
```

iban_check

`iban_check` — operation `iban check`. Category: **b82-misc-utility**.

```
p iban_check $x
```

iban_country

`iban_country` — operation `iban country`. Category: **validation / input checks**.

```
p iban_country $x
```

iban_format

`iban_format` — operation `iban format`. Category: **validation / input checks**.

```
p iban_format $x
```

ibnr_estimate

`ibnr_estimate` — operation `ibnr estimate`. Category: **actuarial science**.

```
p ibnr_estimate $x
```

ical

`ical` — operation `ical`. Alias for `intercalate`. Category: **haskell list functions**.

```
p ical $x
```

ice_age_milankovitch

`ice_age_milankovitch` — operation `ice age milankovitch`. Category: **climate, fluids, atmospheric**.

```
p ice_age_milankovitch $x
```

ico_header_read

`ico_header_read(hex_bytes)` → `hash` — parse ICO/CUR header → `{type, image_count}`.

icosahedral

`icosahedral(n)` $\rightarrow$ `int` — $n \cdot (5n^2 - 5n + 2) / 2$.

icosahedral_number

`icosahedral_number` — operation `icosahedral number`. Category: **misc**.

```
p icosahedral_number $x
```

id

`id` — operation `id`. Alias for `identity`. Category: **trivial numeric / predicate builtins**.

```
p id $x
```

ida_star

`ida_star` — operation `ida star`. Category: **networkx graph algorithms**.

```
p ida_star $x
```

ida_star_search

`ida_star_search(weighted_adj, start, goal, heuristic)` $\rightarrow$ `array` — iterative deepening A*. Memory-efficient; revisits nodes.

idamax

`idamax` — operation `idamax`. Category: **blas / lapack**.

```
p idamax $x
```

idct

`idct` computes the inverse DCT (Type-III). Reconstructs a signal from DCT coefficients.

```
val @signal = @{idct(dct([1,2,3,4]))}
```

ideal_gas

`ideal_gas` (alias `pv_nrt`) solves $PV=nRT$ for the unknown (pass 0 for the value to solve). Args: P, V, n, T.

```
p pv_nrt(0, 0.0224, 1, 273.15) # pressure at STP
p pv_nrt(101325, 0, 1, 273.15) # volume at STP
```

ideal_gas_n

`ideal_gas_n` — operation `ideal gas n`. Category: **chemistry**.

```
p ideal_gas_n $x
```

ideal_gas_volume

`ideal_gas_volume` — operation `ideal gas volume`. Category: **physics formulas**.

```
p ideal_gas_volume $x
```

identity

`identity` — operation `identity`. Category: **trivial numeric / predicate builtins**.

```
p identity $x
```

identity_matrix

`identity_matrix` — operation `identity matrix`. Category: **matrix / linear algebra**.

```
p identity_matrix $x
```

identity_permutation

`identity_permutation` — operation `identity permutation`. Category: **misc**.

```
p identity_permutation $x
```

idft

`idft(\@spectrum)` — computes the Inverse Discrete Fourier Transform. Takes arrayref of [re, im] pairs and returns the time-domain signal.

```
val $spectrum = dft(\@signal)
var $reconstructed = idft($spectrum)
```

idn_normalize

`idn_normalize` — operation `idn normalize`. Category: **b81-misc-utility**.

```
p idn_normalize $x
```

idn_to_ascii

`idn_to_ascii` — convert from `idn` to `ascii`. Category: **archive/encoding format primitives**.

```
p idn_to_ascii $value
```

idn_to_unicode

`idn_to_unicode` — convert from `idn` to `unicode`. Category: **archive/encoding format primitives**.

```
p idn_to_unicode $value
```

idt

`idt` — operation `idt`. Alias for `indent`. Category: **extended stdlib**.

```
p idt $x
```

idxb

`idxb` — operation `idxb`. Alias for `index_by`. Category: **go/general functional utilities**.

```
p idxb $x
```

if_else

`if_else` — operation `if else`. Category: **functional combinators**.

```
p if_else $x
```

if_match_check

`if_match_check` — operation `if match check`. Category: **b81-misc-utility**.

```
p if_match_check $x
```

if_none_match_check

`if_none_match_check` — operation `if none match check`. Category: **b81-misc-utility**.

```
p if_none_match_check $x
```

ifr_infection_fatality

`ifr_infection_fatality` — operation `ifr infection fatality`. Category: **epidemiology / public health**.

```
p ifr_infection_fatality $x
```

igneous_qapf

`igneous_qapf` — operation `igneous qapf`. Category: **geology, seismology, mineralogy**.

```
p igneous_qapf $x
```

ilerp

`ilerp` — operation `ilerp`. Alias for `inv_lerp`. Category: **extended stdlib**.

```
p ilerp $x
```

image_blur_box

`image_blur_box` — operation `image blur box`. Category: **pil/opencv image processing**.

```
p image_blur_box $x
```

image_blur_gaussian

`image_blur_gaussian` — operation `image blur gaussian`. Category: **pil/opencv image processing**.

```
p image_blur_gaussian $x
```

image_brightness

`image_brightness` — operation `image brightness`. Category: **pil/opencv image processing**.

```
p image_brightness $x
```

image_clahe

`image_clahe` — operation `image clahe`. Category: **pil/opencv image processing**.

```
p image_clahe $x
```


image_contrast

`image_contrast` — operation `image contrast`. Category: `*pil/opencv image processing*`.

```
p image_contrast $x
```

image_dilate

`image_dilate` — operation `image dilate`. Category: `*pil/opencv image processing*`.

```
p image_dilate $x
```

image_edge_canny

`image_edge_canny` — operation `image edge canny`. Category: `*pil/opencv image processing*`.

```
p image_edge_canny $x
```

image_edge_laplacian

`image_edge_laplacian` — operation `image edge laplacian`. Category: `*pil/opencv image processing*`.

```
p image_edge_laplacian $x
```

image_edge_sobel

`image_edge_sobel` — operation `image edge sobel`. Category: `*pil/opencv image processing*`.

```
p image_edge_sobel $x
```

image_emboss

`image_emboss` — operation `image emboss`. Category: `*pil/opencv image processing*`.

```
p image_emboss $x
```

image_equalize

`image_equalize` — operation `image equalize`. Category: `*pil/opencv image processing*`.

```
p image_equalize $x
```

image_erode

`image_erode` — operation `image erode`. Category: `*pil/opencv image processing*`.

```
p image_erode $x
```

image_flip_h

`image_flip_h` — operation `image flip h`. Category: `*pil/opencv image processing*`.

```
p image_flip_h $x
```

image_flip_v

`image_flip_v` — operation `image flip v`. Category: `*pil/opencv image processing*`.

```
p image_flip_v $x
```

image_gamma

image_gamma — operation `image gamma`. Category: `*pil/opencv image processing*`.

```
p image_gamma $x
```

image_grayscale

image_grayscale — operation `image grayscale`. Category: `*pil/opencv image processing*`.

```
p image_grayscale $x
```

image_histogram

image_histogram — operation `image histogram`. Category: `*pil/opencv image processing*`.

```
p image_histogram $x
```

image_invert

image_invert — operation `image invert`. Category: `*pil/opencv image processing*`.

```
p image_invert $x
```

image_morphology_close

image_morphology_close — close handle of `image morphology`. Category: `*pil/opencv image processing*`.

```
p image_morphology_close $x
```

image_morphology_open

image_morphology_open — open handle of `image morphology`. Category: `*pil/opencv image processing*`.

```
p image_morphology_open $x
```

image_motion_blur

image_motion_blur — operation `image motion blur`. Category: `*pil/opencv image processing*`.

```
p image_motion_blur $x
```

image_posterize

image_posterize — operation `image posterize`. Category: `*pil/opencv image processing*`.

```
p image_posterize $x
```

image_resize

image_resize — operation `image resize`. Category: `*pil/opencv image processing*`.

```
p image_resize $x
```

image_rotate

image_rotate — operation `image rotate`. Category: `*pil/opencv image processing*`.

```
p image_rotate $x
```

image_sepia

`image_sepia` — operation `image sepia`. Category: **pil/opencv image processing**.

```
p image_sepia $x
```

image_sharpen

`image_sharpen` — operation `image sharpen`. Category: **pil/opencv image processing**.

```
p image_sharpen $x
```

image_solarize

`image_solarize` — operation `image solarize`. Category: **pil/opencv image processing**.

```
p image_solarize $x
```

image_threshold

`image_threshold` — operation `image threshold`. Category: **pil/opencv image processing**.

```
p image_threshold $x
```

imei_check

`imei_check` — operation `imei check`. Category: **b82-misc-utility**.

```
p imei_check $x
```

imex_euler_step

`imex_euler_step` — single-step update of `imex euler`. Category: **ode advanced**.

```
p imex_euler_step $x
```

imex_theta_split

`imex_theta_split` — operation `imex theta split`. Category: **ode advanced**.

```
p imex_theta_split $x
```

impedance_rlc

`impedance_rlc` — operation `impedance rlc`. Category: **physics formulas**.

```
p impedance_rlc $x
```

impl

`impl` declares which traits a class implements.

```
trait Printable { fn to_str }
class Item impl Printable {
  name: Str
  fn to_str { $self->name }
}
```

Multiple traits: `class X impl A, B, C { }`

implicit_euler

`implicit_euler` — operation `implicit euler`. Alias for `backward_euler`. Category: **more extensions (2)**.

```
p implicit_euler $x
```

impulse

`impulse($force, $time)` — compute impulse $J = F \times t$ (N·s). Change in momentum.

```
p impulse(100, 0.5) # 50 N·s
```

impulse_response_n

`impulse_response_n` — operation `impulse response n`. Category: **economics + game theory**.

```
p impulse_response_n $x
```

impulse_response_step

`impulse_response_step` — single-step update of `impulse response`. Category: **econometrics**.

```
p impulse_response_step $x
```

imputation_efficient_check

`imputation_efficient_check` — operation `imputation efficient check`. Category: **climate, fluids, atmospheric**.

```
p imputation_efficient_check $x
```

imputation_individual_rational

`imputation_individual_rational` — operation `imputation individual rational`. Category: **climate, fluids, atmospheric**.

```
p imputation_individual_rational $x
```

imu_madgwick_step

`imu_madgwick_step` — single-step update of `imu madgwick`. Category: **robotics & control**.

```
p imu_madgwick_step $x
```

imu_mahony_step

`imu_mahony_step` — single-step update of `imu mahony`. Category: **robotics & control**.

```
p imu_mahony_step $x
```

in_degree_directed

`in_degree_directed` — operation `in degree directed`. Category: **misc**.

```
p in_degree_directed $x
```

in_range

`in_range` — operation `in range`. Category: **extended stdlib**.

```
p in_range $x
```

inc

`inc` — operation `inc`. Category: **pipeline / string helpers**.

```
p inc $x
```

inc_each

`inc_each` — operation `inc each`. Category: **trivial numeric helpers**.

```
p inc_each $x
```

incentive_compatibility_check

`incentive_compatibility_check` — operation `incentive compatibility check`. Category: **climate, fluids, atmospheric**.

```
p incentive_compatibility_check $x
```

inches_to_cm

`inches_to_cm` — convert from `inches` to `cm`. Category: **unit conversions**.

```
p inches_to_cm $value
```

incidence_algebra_dim

`incidence_algebra_dim` — operation `incidence algebra dim`. Category: **econometrics**.

```
p incidence_algebra_dim $x
```

includes_val

`includes_val` — operation `includes val`. Category: **javascript array/object methods**.

```
p includes_val $x
```

income_elasticity

`income_elasticity` — operation `income elasticity`. Category: **economics + game theory**.

```
p income_elasticity $x
```

incr

`incr` — operation `incr`. Category: **redis-flavour primitives**.

```
p incr $x
```

incrby

`incrby` — operation `incrby`. Category: **redis-flavour primitives**.

```
p incrby $x
```

incv

`incv` — operation `incv`. Alias for `includes_val`. Category: **javascript array/object methods**.

```
p incv $x
```

indenergy

`indenergy` — operation `indenergy`. Alias for `inductor_energy`. Category: **physics formulas**.

```
p indenergy $x
```

indent

`indent` — operation `indent`. Category: **extended stdlib**.

```
p indent $x
```

indent_block

`indent_block(s, prefix=" ")` → `string` — prepend `prefix` to every line.

```
p indent_block("line1\nline2", "> ")
```

indent_text

`indent_text` — operation `indent text`. Category: **string processing extras**.

```
p indent_text $x
```

independence_number_upper

`independence_number_upper` — operation `independence number upper`. Category: **combinatorial optimization, scheduling**.

```
p independence_number_upper $x
```

index_by

`index_by` — operation `index by`. Category: **go/general functional utilities**.

```
p index_by $x
```

index_calculus_naive

`index_calculus_naive` — operation `index calculus naive`. Category: **cryptanalysis & number theory deep**.

```
p index_calculus_naive $x
```

index_from_speed

`index_from_speed` — operation `index from speed`. Category: **em / optics / relativity**.

```
p index_from_speed $x
```

index_match

`index_match` — operation `index match`. Category: **excel/sheets + bond/loan financial**.

```
p index_match $x
```

index_of

`index_of` — operation `index of`. Category: **collection**.

```
p index_of $x
```

index_of_coincidence

`index_of_coincidence` — operation `index of coincidence`. Category: **cryptography deep**.

```
p index_of_coincidence $x
```

index_of_elem

`index_of_elem` — operation `index of elem`. Category: **list helpers**.

```
p index_of_elem $x
```

indexes_of

`indexes_of` — operation `indexes of`. Category: **string**.

```
p indexes_of $x
```

indian_monsoon_index

`indian_monsoon_index` — index of `indian monsoon`. Category: **climate, fluids, atmospheric**.

```
p indian_monsoon_index $x
```

indirect

`indirect` — operation `indirect`. Category: **excel/sheets + bond/loan financial**.

```
p indirect $x
```

indirect_age_adjusted

`indirect_age_adjusted` — operation `indirect age adjusted`. Category: **epidemiology / public health**.

```
p indirect_age_adjusted $x
```

indirect_utility

`indirect_utility` — operation `indirect utility`. Category: **economics + game theory**.

```
p indirect_utility $x
```

indivisible_envy_free_check

`indivisible_envy_free_check` — operation `indivisible envy free check`. Category: **climate, fluids, atmospheric**.

```
p indivisible_envy_free_check $x
```

inductance_parallel

`inductance_parallel` — operation `inductance_parallel`. Category: **misc**.

```
p inductance_parallel $x
```

inductance_series

`inductance_series` — operation `inductance_series`. Category: **misc**.

```
p inductance_series $x
```

inductor_energy

`inductor_energy` — operation `inductor_energy`. Category: **physics formulas**.

```
p inductor_energy $x
```

inelastic_collision_1d

`inelastic_collision_1d(m1, v1, m2, v2) → number` — final common velocity for a perfectly inelastic collision.

infinite_well_energy

`infinite_well_energy` — operation `infinite_well_energy`. Category: **quantum mechanics deep**.

```
p infinite_well_energy $x
```

information_bottleneck_step

`information_bottleneck_step` — single-step update of `information_bottleneck`. Category: **electrochemistry, batteries, fuel cells**.

```
p information_bottleneck_step $x
```

information_gain

`information_gain` — operation `information_gain`. Category: **ml extensions**.

```
p information_gain $x
```

information_ratio

`information_ratio(portfolio_returns, benchmark_returns) → number` — $\text{mean}(\text{active}) / \text{stddev}(\text{active})$ where $\text{active} = \text{portfolio} - \text{benchmark}$.

```
p information_ratio(\@port, \@bench)
```

init_list

`init_list` — operation `init_list`. Category: **list helpers**.

```
p init_list $x
```

initial_bearing

`initial_bearing` — operation `initial_bearing`. Category: **geometry / topology**.

```
p initial_bearing $x
```


initials

`initials` — operation `initials`. Category: **string processing extras**.

```
p initials $x
```

inits

`inits` — operation `inits`. Category: **additional missing stdlib functions**.

```
p inits $x
```

inits_str

`inits_str` — operation `inits str`. Alias for `initials`. Category: **string processing extras**.

```
p inits_str $x
```

ining

`ining` — operation `ining`. Alias for `in_range`. Category: **extended stdlib**.

```
p ining $x
```

insa

`insa` — operation `insa`. Alias for `insert_at`. Category: **go/general functional utilities**.

```
p insa $x
```

insert_at

`insert_at` — operation `insert at`. Category: **go/general functional utilities**.

```
p insert_at $x
```

insert_at_idx

`insert_at_idx` — operation `insert at idx`. Category: **misc**.

```
p insert_at_idx $x
```

insert_sorted

`insert_sorted` — operation `insert sorted`. Category: **additional missing stdlib functions**.

```
p insert_sorted $x
```

insert_str

`insert_str` — operation `insert str`. Category: **string helpers**.

```
p insert_str $x
```

insp

`insp` — operation `insp`. Alias for `inspect`. Category: **rust iterator methods**.

```
p insp $x
```

inspect

`inspect` — operation `inspect`. Category: **rust iterator methods**.

```
p inspect $x
```

insrt

`insrt` — operation `insrt`. Alias for `insert_sorted`. Category: **additional missing stdlib functions**.

```
p insrt $x
```

int_bits

`int_bits` — integer helper `bits`. Category: **math functions**.

```
p int_bits $x
```

int_to_ip

`int_to_ip` — convert from `int` to `ip`. Category: **network / ip / cidr**.

```
p int_to_ip $value
```

int_to_ipv4

`int_to_ipv4` — convert from `int` to `ipv4`. Category: **network / validation**.

```
p int_to_ipv4 $value
```

int_to_mac

`int_to_mac` — convert from `int` to `mac`. Category: **network / ip / cidr**.

```
p int_to_mac $value
```

int_to_roman

`int_to_roman` — convert from `int` to `roman`. Category: **roman numerals**.

```
p int_to_roman $value
```

integral_image

`integral_image(image) → matrix` — summed-area table for O(1) box-sum queries.

```
p integral_image([[1,1,1],[1,1,1],[1,1,1]]) # bottom-right = 9
```

integral_length_scale

`integral_length_scale` — operation `integral length scale`. Category: **climate, fluids, atmospheric**.

```
p integral_length_scale $x
```

intercalate

`intercalate` — operation `intercalate`. Category: **haskell list functions**.

```
p intercalate $x
```

interleave_lists

`interleave_lists` — operation `interleave_lists`. Category: **list helpers**.

```
p interleave_lists $x
```

internal_rate_of_return

`internal_rate_of_return(cashflows)` → `number` — IRR via Newton iteration; the discount rate that makes NPV = 0.

```
p internal_rate_of_return([-1000, 300, 400, 500])
```

interp1d

`interp1d` — operation `interp1d`. Category: **numpy + scipy.special**.

```
p interp1d $x
```

interp_lagrange

`interp_lagrange(xs, ys, x)` → `number` — Lagrange polynomial interpolation through all sample points.

```
p interp_lagrange([0,1,2], [1,4,9], 1.5)
```

interp_linear

`interp_linear(xs, ys, x)` → `number` — linear interpolation. Binary-searches `xs` to find the bracketing interval.

```
p interp_linear([0,1,2], [0,10,20], 0.5) # 5
```

interpolate

`upsample(\@signal, $factor)` (alias `interpolate`) — increases sample rate by inserting zeros and filtering. Factor must be positive integer.

```
val $upsampled = interpolate(\@signal, 4) # 4x sample rate
```

interpolation_search

`interpolation_search(sorted, target)` → `int` — estimate position by linear interpolation. $O(\log \log n)$ on uniform data.

interpose

`interpose` — operation `interpose`. Category: **go/general functional utilities**.

```
p interpose $x
```

intersect_list

`intersect_list` — operation `intersect_list`. Category: **additional missing stdlib functions**.

```
p intersect_list $x
```

intersperse

`intersperse` — operation `intersperse`. Alias for `riffle`. Category: **collection helpers (trivial)**.

```
p intersperse $x
```

intersperse_char

`intersperse_char` — operation `intersperse char`. Category: **string helpers**.

```
p intersperse_char $x
```

intersperse_val

`intersperse_val` — operation `intersperse val`. Category: **haskell list functions**.

```
p intersperse_val $x
```

intersperse_with

`intersperse_with` — operation `intersperse with`. Category: **additional missing stdlib functions**.

```
p intersperse_with $x
```

interval_at

`it_query_point(IT, P)` → list of `[start, end, payload]` arrayrefs containing point `P`.

interval_count

`it_len(IT)` → number of intervals stored.

interval_insert

`it_insert(IT, START, END, PAYLOAD=undef)` — append an interval. Endpoints normalized so `START ≤ END`.

interval_name

`interval_name(semitones)` → `string` — Western music interval label: P1, m2, M2, m3, M3, P4, TT, P5, m6, M6, m7, M7.

```
p interval_name(7) # P5
```

interval_overlap

`it_query_range(IT, LO, HI)` → list of `[start, end, payload]` arrayrefs overlapping range `[LO, HI]`.

interval_quality_size

`interval_quality_size` — operation `interval quality size`. Category: **music theory**.

```
p interval_quality_size $x
```

interval_remove

`it_remove(IT, START, END)` → number of removed intervals with exact endpoints.

interval_semitones

`interval_semitones` — operation `interval_semitones`. Category: **misc**.

```
p interval_semitones $x
```

interval_tree

`interval_tree()` — store `[start, end]` intervals (inclusive) with associated payloads; query for all intervals overlapping a point or another range.

```
val $it = interval_tree()
it_insert($it, 1, 10, "meeting-a")
it_insert($it, 8, 15, "meeting-b")
var @hits = it_query_point($it, 9)      # both overlap
```

intl

`intl` — operation `intl`. Alias for `intersect_list`. Category: **additional missing stdlib functions**.

```
p intl $x
```

into

`into` — operation `into`. Category: **algebraic match**.

```
p into $x
```

intrate

`intrate` — operation `intrate`. Category: **b81-misc-utility**.

```
p intrate $x
```

intrinsic_growth_rate

`intrinsic_growth_rate` — rate of `intrinsic growth`. Category: **biology / ecology**.

```
p intrinsic_growth_rate $x
```

intrinsic_principal_curv

`intrinsic_principal_curv` — operation `intrinsic_principal_curv`. Category: **electrochemistry, batteries, fuel cells**.

```
p intrinsic_principal_curv $x
```

inv_lerp

`inv_lerp` — operation `inv_lerp`. Category: **extended stdlib**.

```
p inv_lerp $x
```

inverse

`inverse` — operation `inverse`. Category: **trivial numeric / predicate builtins**.

```
p inverse $x
```

inverse_chi2_pdf

`inverse_chi2_pdf` — operation `inverse_chi2_pdf`. Category: **more extensions (2)**.

```
p inverse_chi2_pdf $x
```

inverse_kinematics_2link

`inverse_kinematics_2link` — operation `inverse_kinematics_2link`. Category: **robotics & control**.

```
p inverse_kinematics_2link $x
```

inverse_lerp

`inverse_lerp` — operation `inverse_lerp`. Category: **math formulas**.

```
p inverse_lerp $x
```

inverse_qft

`inverse_qft` — operation `inverse_qft`. Category: **quantum**.

```
p inverse_qft $x
```

inverse_quad_interp

`inverse_quad_interp` — operation `inverse_quad_interp`. Category: **more extensions (2)**.

```
p inverse_quad_interp $x
```

inverse_simpson

`inverse_simpson` — operation `inverse_simpson`. Category: **biology / ecology**.

```
p inverse_simpson $x
```

inverse_weibull_pdf

`inverse_weibull_pdf` — operation `inverse_weibull_pdf`. Category: **more extensions (2)**.

```
p inverse_weibull_pdf $x
```

invert

`invert` — operation `invert`. Category: **hash helpers**.

```
p invert $x
```

invert_perm

`invert_perm` — operation `invert_perm`. Category: **apl/j/k array primitives**.

```
p invert_perm $x
```

ioctl

`ioctl` — operation `ioctl`. Category: **ipc**.

```
p ioctl $x
```

ionic_conductivity_arrhenius

`ionic_conductivity_arrhenius` — operation `ionic conductivity arrhenius`. Category: **electrochemistry, batteries, fuel cells**.

```
p ionic_conductivity_arrhenius $x
```

ionic_liquid_viscosity_step

`ionic_liquid_viscosity_step` — single-step update of `ionic liquid viscosity`. Category: **electrochemistry, batteries, fuel cells**.

```
p ionic_liquid_viscosity_step $x
```

ionic_strength

`ionic_strength` — operation `ionic strength`. Category: **misc**.

```
p ionic_strength $x
```

iota_n

`iota_n` — operation `iota n`. Category: **apl/j/k array primitives**.

```
p iota_n $x
```

iou_2d_axis_aligned

`iou_2d_axis_aligned` — operation `iou 2d axis aligned`. Category: **more extensions (2)**.

```
p iou_2d_axis_aligned $x
```

ip_arpa

`ip_arpa` — IP address op `arpa`. Category: **network / ip / cidr**.

```
p ip_arpa $x
```

ip_canonical

`ip_canonical` — IP address op `canonical`. Category: **network / ip / cidr**.

```
p ip_canonical $x
```

ip_compare

`ip_compare` — IP address op `compare`. Category: **network / ip / cidr**.

```
p ip_compare $x
```

ip_family

`ip_family` — IP address op `family`. Category: **network / ip / cidr**.

```
p ip_family $x
```

ip_in_cidr

ip_in_cidr — IP address op in cidr. Category: *network / ip / cidr*.

```
p ip_in_cidr $x
```

ip_in_subnet

ip_in_subnet — IP address op in subnet. Category: *network / ip / cidr*.

```
p ip_in_subnet $x
```

ip_is_benchmarking

ip_is_benchmarking — IP address op is benchmarking. Category: *network / ip / cidr*.

```
p ip_is_benchmarking $x
```

ip_is_broadcast

ip_is_broadcast — IP address op is broadcast. Category: *network / ip / cidr*.

```
p ip_is_broadcast $x
```

ip_is_documentation

ip_is_documentation — IP address op is documentation. Category: *network / ip / cidr*.

```
p ip_is_documentation $x
```

ip_is_global

ip_is_global — IP address op is global. Category: *network / ip / cidr*.

```
p ip_is_global $x
```

ip_is_link_local

ip_is_link_local — IP address op is link local. Category: *network / ip / cidr*.

```
p ip_is_link_local $x
```

ip_is_loopback

ip_is_loopback — IP address op is loopback. Category: *network / ip / cidr*.

```
p ip_is_loopback $x
```

ip_is_multicast

ip_is_multicast — IP address op is multicast. Category: *network / ip / cidr*.

```
p ip_is_multicast $x
```

ip_is_private

ip_is_private — IP address op is private. Category: *network / ip / cidr*.

```
p ip_is_private $x
```


ip_is_reserved

`ip_is_reserved` — IP address op `is reserved`. Category: `*network / ip / cidr*`.

```
p ip_is_reserved $x
```

ip_is_shared

`ip_is_shared` — IP address op `is shared`. Category: `*network / ip / cidr*`.

```
p ip_is_shared $x
```

ip_is_unspecified

`ip_is_unspecified` — IP address op `is unspecified`. Category: `*network / ip / cidr*`.

```
p ip_is_unspecified $x
```

ip_is_valid

`ip_is_valid` — IP address op `is valid`. Category: `*network / ip / cidr*`.

```
p ip_is_valid $x
```

ip_parse

`ip_parse` — IP address op `parse`. Category: `*network / ip / cidr*`.

```
p ip_parse $x
```

ip_random

`ip_random` — IP address op `random`. Category: `*network / ip / cidr*`.

```
p ip_random $x
```

ip_random_in_cidr

`ip_random_in_cidr` — IP address op `random in cidr`. Category: `*network / ip / cidr*`.

```
p ip_random_in_cidr $x
```

ip_reverse

`ip_reverse` — IP address op `reverse`. Category: `*network / ip / cidr*`.

```
p ip_reverse $x
```

ip_sort

`ip_sort` — IP address op `sort`. Category: `*network / ip / cidr*`.

```
p ip_sort $x
```

ip_subnet_split

`ip_subnet_split(cidr, new_prefix) → array` — divide a CIDR into $2^{(\text{new_prefix} - \text{prefix})}$ smaller blocks.

```
p ip_subnet_split("10.0.0.0/24", 26) # 4 /26 blocks
```

ip_to_bits

`ip_to_bits` — convert from `ip` to `bits`. Category: `*network / ip / cidr*`.

```
p ip_to_bits $value
```

ip_to_bytes

`ip_to_bytes` — convert from `ip` to `bytes`. Category: `*network / ip / cidr*`.

```
p ip_to_bytes $value
```

ip_to_int

`ip_to_int` — convert from `ip` to `int`. Category: `*network / ip / cidr*`.

```
p ip_to_int $value
```

ip_version

`ip_version` — IP address op `version`. Category: `*network / ip / cidr*`.

```
p ip_version $x
```

ipmt

`ipmt` — operation `ipmt`. Category: `*excel/sheets + bond/loan financial*`.

```
p ipmt $x
```

ipos

`ipos` — operation `ipos`. Alias for `interpose`. Category: `*go/general functional utilities*`.

```
p ipos $x
```

ipv4_classful_class

`ipv4_classful_class` — operation `ipv4 classful class`. Category: `*network / ip / cidr*`.

```
p ipv4_classful_class $x
```

ipv4_is_valid

`ipv4_is_valid` — operation `ipv4 is valid`. Category: `*network / ip / cidr*`.

```
p ipv4_is_valid $x
```

ipv4_parse

`ipv4_parse` — operation `ipv4 parse`. Category: `*network / ip / cidr*`.

```
p ipv4_parse $x
```

ipv4_to_int

`ipv4_to_int` — convert from `ipv4` to `int`. Category: `*network / validation*`.

```
p ipv4_to_int $value
```

ipv4_to_ipv6_6to4

ipv4_to_ipv6_6to4 — convert from `ipv4` to `ipv6 6to4`. Category: `*network / ip / cidr*`.

```
p ipv4_to_ipv6_6to4 $value
```

ipv4_to_ipv6_mapped

ipv4_to_ipv6_mapped — convert from `ipv4` to `ipv6 mapped`. Category: `*network / ip / cidr*`.

```
p ipv4_to_ipv6_mapped $value
```

ipv6_6to4_extract

ipv6_6to4_extract — IPv6 address op `6to4 extract`. Category: `*network / ip / cidr*`.

```
p ipv6_6to4_extract $x
```

ipv6_canonical

ipv6_canonical — IPv6 address op `canonical`. Category: `*network / ip / cidr*`.

```
p ipv6_canonical $x
```

ipv6_compress

ipv6_compress — IPv6 address op `compress`. Category: `*network / ip / cidr*`.

```
p ipv6_compress $x
```

ipv6_eui64_addr

ipv6_eui64_addr — IPv6 address op `eui64 addr`. Category: `*network / ip / cidr*`.

```
p ipv6_eui64_addr $x
```

ipv6_expand

ipv6_expand — IPv6 address op `expand`. Category: `*network / ip / cidr*`.

```
p ipv6_expand $x
```

ipv6_global_unicast

ipv6_global_unicast(ipv6) → 0|1 — 1 if address is in 2000::/3 (global unicast).

ipv6_is_6to4

ipv6_is_6to4 — IPv6 address op `is 6to4`. Category: `*network / ip / cidr*`.

```
p ipv6_is_6to4 $x
```

ipv6_is_isatap

ipv6_is_isatap — IPv6 address op `is isatap`. Category: `*network / ip / cidr*`.

```
p ipv6_is_isatap $x
```

ipv6_is_teredo

ipv6_is_teredo — IPv6 address op **is_teredo**. Category: *network / ip / cidr*.

```
p ipv6_is_teredo $x
```

ipv6_is_valid

ipv6_is_valid — IPv6 address op **is_valid**. Category: *network / ip / cidr*.

```
p ipv6_is_valid $x
```

ipv6_isatap_extract

ipv6_isatap_extract — IPv6 address op **isatap_extract**. Category: *network / ip / cidr*.

```
p ipv6_isatap_extract $x
```

ipv6_link_local

ipv6_link_local — IPv6 address op **link_local**. Category: *network / ip / cidr*.

```
p ipv6_link_local $x
```

ipv6_link_local_from_mac

ipv6_link_local_from_mac — IPv6 address op **link_local_from_mac**. Category: *network / ip / cidr*.

```
p ipv6_link_local_from_mac $x
```

ipv6_parse

ipv6_parse — IPv6 address op **parse**. Category: *network / ip / cidr*.

```
p ipv6_parse $x
```

ipv6_solicited_node

ipv6_solicited_node — IPv6 address op **solicited_node**. Category: *network / ip / cidr*.

```
p ipv6_solicited_node $x
```

ipv6_strip_zone

ipv6_strip_zone — IPv6 address op **strip_zone**. Category: *network / ip / cidr*.

```
p ipv6_strip_zone $x
```

ipv6_teredo_extract

ipv6_teredo_extract — IPv6 address op **teredo_extract**. Category: *network / ip / cidr*.

```
p ipv6_teredo_extract $x
```

ipv6_to_ipv4_compat

ipv6_to_ipv4_compat — convert from **ipv6** to **ipv4_compat**. Category: *network / ip / cidr*.

```
p ipv6_to_ipv4_compat $value
```

ipv6_unique_local

`ipv6_unique_local` — IPv6 address op `unique local`. Category: **network / ip / cidr**.

```
p ipv6_unique_local $x
```

ipv6_zone_id

`ipv6_zone_id` — IPv6 address op `zone id`. Category: **network / ip / cidr**.

```
p ipv6_zone_id $x
```

ipv_potential_vorticity

`ipv_potential_vorticity` — operation `ipv potential vorticity`. Category: **climate, fluids, atmospheric**.

```
p ipv_potential_vorticity $x
```

iqr

`iqr` — operation `iqr`. Category: **extended stdlib**.

```
p iqr $x
```

irr

`irr(@cashflows)` — computes the Internal Rate of Return for a series of cash flows. First value is typically negative (initial investment). Uses Newton-Raphson iteration.

```
val @cf = (-1000, 300, 420, 680)
p irr(@cf) # -0.166 (16.6%)
```

is_7_smooth

`is_7_smooth` — predicate: 1 if input is 7 smooth, 0 otherwise. Alias for `seven_smooth_q`. Category: **misc**.

```
p is_7_smooth $x
```

is_abundant

`is_abundant` — predicate: 1 if input is abundant, 0 otherwise. Category: **number theory / primes**.

```
p is_abundant $x
```

is_alnum

`is_alnum` — predicate: 1 if input is alnum, 0 otherwise. Category: **trivial string ops**.

```
p is_alnum $x
```

is_alpha

`is_alpha` — predicate: 1 if input is alpha, 0 otherwise. Category: **trivial string ops**.

```
p is_alpha $x
```

is_alpha_only

`is_alpha_only` — predicate: 1 if input is alpha only, 0 otherwise. Category: **validation / input checks**.

```
p is_alpha_only $x
```

is_alphanumeric_only

`is_alphanumeric_only` — predicate: 1 if input is alphanumeric only, 0 otherwise. Category: **validation / input checks**.

```
p is_alphanumeric_only $x
```

is_anagram

`is_anagram` — predicate: 1 if input is anagram, 0 otherwise. Category: **validation predicates extras**.

```
p is_anagram $x
```

is_anagram_q

`is_anagram_q` — predicate: 1 if input is anagram q, 0 otherwise. Alias for `anagram_q`. Category: **misc**.

```
p is_anagram_q $x
```

is_armstrong

`is_armstrong` — predicate: 1 if input is armstrong, 0 otherwise. Alias for `armstrong_q`. Category: **misc**.

```
p is_armstrong $x
```

is_array

`is_array` — predicate: 1 if input is array, 0 otherwise. Category: **trivial type predicates**.

```
p is_array $x
```

is_arrayref

`is_arrayref` — predicate: 1 if input is arrayref, 0 otherwise. Alias for `is_array`. Category: **trivial type predicates**.

```
p is_arrayref $x
```

is_asc

`is_asc` — predicate: 1 if input is asc, 0 otherwise. Alias for `is_sorted`. Category: **predicates**.

```
p is_asc $x
```

is_ascii

`is_ascii` — predicate: 1 if input is ascii, 0 otherwise. Category: **extended stdlib**.

```
p is_ascii $x
```

is_ascii_only

`is_ascii_only` — predicate: 1 if input is ascii only, 0 otherwise. Category: **validation / input checks**.

```
p is_ascii_only $x
```

is_b_smooth

`is_b_smooth` — predicate: 1 if input is b smooth, 0 otherwise. Category: *cryptanalysis & number theory deep*.

```
p is_b_smooth $x
```

is_balanced_parens

`is_balanced_parens` — predicate: 1 if input is balanced parens, 0 otherwise. Category: *validation predicates extras*.

```
p is_balanced_parens $x
```

is_base32

`is_base32` — predicate: 1 if input is base32, 0 otherwise. Category: *validation / input checks*.

```
p is_base32 $x
```

is_base64

`is_base64` — predicate: 1 if input is base64, 0 otherwise. Category: *extended stdlib*.

```
p is_base64 $x
```

is_between

`is_between` — predicate: 1 if input is between, 0 otherwise. Category: *predicates*.

```
p is_between $x
```

is_bic

`is_bic` — predicate: 1 if input is bic, 0 otherwise. Category: *validation / input checks*.

```
p is_bic $x
```

is_binary

`is_binary` — predicate: 1 if input is binary, 0 otherwise. Category: *validation / input checks*.

```
p is_binary $x
```

is_binary_str

`is_binary_str` — predicate: 1 if input is binary str, 0 otherwise. Category: *extended stdlib*.

```
p is_binary_str $x
```

is_bipartite_graph

`is_bipartite_graph` — predicate: 1 if input is bipartite graph, 0 otherwise. Category: *extended stdlib*.

```
p is_bipartite_graph $x
```

is_blank

`is_blank` — predicate: 1 if input is blank, 0 otherwise. Category: *trivial string ops*.

```
p is_blank $x
```

is_blank_or_nil

`is_blank_or_nil` — predicate: 1 if input is blank or nil, 0 otherwise. Category: **predicates**.

```
p is_blank_or_nil $x
```

is_callable

`is_callable` — predicate: 1 if input is callable, 0 otherwise. Category: **functional combinators**.

```
p is_callable $x
```

is_carmichael

`is_carmichael` — predicate: 1 if input is carmichael, 0 otherwise. Alias for `carmichael_q`. Category: **misc**.

```
p is_carmichael $x
```

is_cidr

`is_cidr` — predicate: 1 if input is cidr, 0 otherwise. Category: **validation / input checks**.

```
p is_cidr $x
```

is_code

`is_code` — predicate: 1 if input is code, 0 otherwise. Category: **trivial type predicates**.

```
p is_code $x
```

is_coderef

`is_coderef` — predicate: 1 if input is coderef, 0 otherwise. Alias for `is_code`. Category: **trivial type predicates**.

```
p is_coderef $x
```

is_connected

`is_connected` — predicate: 1 if input is connected, 0 otherwise. Category: **misc**.

```
p is_connected $x
```

is_consonant

`is_consonant` — predicate: 1 if input is consonant, 0 otherwise. Category: **string**.

```
p is_consonant $x
```

is_control

`is_control` — predicate: 1 if input is control, 0 otherwise. Category: **validation predicates extras**.

```
p is_control $x
```

is_coprime

`is_coprime` — predicate: 1 if input is coprime, 0 otherwise. Category: **extended stdlib**.

```
p is_coprime $x
```


is_credit_card

`is_credit_card` — predicate: 1 if input is credit card, 0 otherwise. Category: **extended stdlib**.

```
p is_credit_card $x
```

is_def

`is_def` — predicate: 1 if input is def, 0 otherwise. Alias for `is_defined`. Category: **trivial type predicates**.

```
p is_def $x
```

is_deficient

`is_deficient` — predicate: 1 if input is deficient, 0 otherwise. Category: **number theory / primes**.

```
p is_deficient $x
```

is_defined

`is_defined` — predicate: 1 if input is defined, 0 otherwise. Category: **trivial type predicates**.

```
p is_defined $x
```

is_desc

`is_desc` — predicate: 1 if input is desc, 0 otherwise. Alias for `is_sorted_desc`. Category: **predicates**.

```
p is_desc $x
```

is_digit

`is_digit` — predicate: 1 if input is digit, 0 otherwise. Category: **trivial string ops**.

```
p is_digit $x
```

is_divisible_by

`is_divisible_by` — predicate: 1 if input is divisible by, 0 otherwise. Category: **predicates**.

```
p is_divisible_by $x
```

is_ean13

`is_ean13` — predicate: 1 if input is ean13, 0 otherwise. Category: **validation / input checks**.

```
p is_ean13 $x
```

is_email

`is_email` — predicate: 1 if input is email, 0 otherwise. Category: **predicates**.

```
p is_email $x
```

is_email_strict

`is_email_strict` — predicate: 1 if input is email strict, 0 otherwise. Category: **validation / input checks**.

```
p is_email_strict $x
```

is_empty

`is_empty` — predicate: 1 if input is empty, 0 otherwise. Category: **trivial string ops**.

```
p is_empty $x
```

is_empty_arr

`is_empty_arr` — predicate: 1 if input is empty arr, 0 otherwise. Category: **predicates**.

```
p is_empty_arr $x
```

is_empty_hash

`is_empty_hash` — predicate: 1 if input is empty hash, 0 otherwise. Category: **predicates**.

```
p is_empty_hash $x
```

is_even

`is_even` — predicate: 1 if input is even, 0 otherwise. Category: **predicates**.

```
p is_even $x
```

is_even_num

`is_even_num` — predicate: 1 if input is even num, 0 otherwise. Category: **extended stdlib**.

```
p is_even_num $x
```

is_executable

`is_executable` — predicate: 1 if input is executable, 0 otherwise. Category: **file stat / path**.

```
p is_executable $x
```

is_falsy

`is_falsy` — predicate: 1 if input is falsy, 0 otherwise. Category: **predicates**.

```
p is_falsy $x
```

is_fibonacci

`is_fibonacci` — predicate: 1 if input is fibonacci, 0 otherwise. Category: **predicates**.

```
p is_fibonacci $x
```

is_finite

`is_finite` — predicate: 1 if input is finite, 0 otherwise. Category: **trig / math**.

```
p is_finite $x
```

is_float

`is_float` — predicate: 1 if input is float, 0 otherwise. Category: **trivial type predicates**.

```
p is_float $x
```

is_harshad

`is_harshad` — predicate: 1 if input is harshad, 0 otherwise. Category: **extended stdlib**.

```
p is_harshad $x
```

is_hash

`is_hash` — predicate: 1 if input is hash, 0 otherwise. Category: **trivial type predicates**.

```
p is_hash $x
```

is_hashref

`is_hashref` — predicate: 1 if input is hashref, 0 otherwise. Alias for `is_hash`. Category: **trivial type predicates**.

```
p is_hashref $x
```

is_heterogram

`is_heterogram` — predicate: 1 if input is heterogram, 0 otherwise. Category: **extended stdlib**.

```
p is_heterogram $x
```

is_hex

`is_hex` — predicate: 1 if input is hex, 0 otherwise. Category: **validation / input checks**.

```
p is_hex $x
```

is_hex_color

`is_hex_color` — predicate: 1 if input is hex color, 0 otherwise. Category: **predicates**.

```
p is_hex_color $x
```

is_hex_str

`is_hex_str` — predicate: 1 if input is hex str, 0 otherwise. Category: **extended stdlib**.

```
p is_hex_str $x
```

is_hostname_valid

`is_hostname_valid` — predicate: 1 if input is hostname valid, 0 otherwise. Category: **extended stdlib**.

```
p is_hostname_valid $x
```

is_iban

`is_iban` — predicate: 1 if input is iban, 0 otherwise. Category: **extended stdlib**.

```
p is_iban $x
```

is_imei

`is_imei` — predicate: 1 if input is imei, 0 otherwise. Category: **validation / input checks**.

```
p is_imei $x
```

is_imsi

`is_imsi` — predicate: 1 if input is imsi, 0 otherwise. Category: *validation / input checks*.

```
p is_imsi $x
```

is_in_range

`is_in_range` — predicate: 1 if input is in range, 0 otherwise. Category: *predicates*.

```
p is_in_range $x
```

is_inf

`is_inf` — predicate: 1 if input is inf, 0 otherwise. Alias for `is_infinite`. Category: *trig / math*.

```
p is_inf $x
```

is_infinite

`is_infinite` — predicate: 1 if input is infinite, 0 otherwise. Category: *trig / math*.

```
p is_infinite $x
```

is_int

`is_int` — predicate: 1 if input is int, 0 otherwise. Category: *trivial type predicates*.

```
p is_int $x
```

is_integer

`is_integer` — predicate: 1 if input is integer, 0 otherwise. Alias for `is_int`. Category: *trivial type predicates*.

```
p is_integer $x
```

is_ipv4

`is_ipv4` — predicate: 1 if input is ipv4, 0 otherwise. Category: *predicates*.

```
p is_ipv4 $x
```

is_ipv4_addr

`is_ipv4_addr` — predicate: 1 if input is ipv4 addr, 0 otherwise. Category: *extended stdlib*.

```
p is_ipv4_addr $x
```

is_ipv6

`is_ipv6` — predicate: 1 if input is ipv6, 0 otherwise. Category: *validation / input checks*.

```
p is_ipv6 $x
```

is_ipv6_addr

`is_ipv6_addr` — predicate: 1 if input is ipv6 addr, 0 otherwise. Category: *extended stdlib*.

```
p is_ipv6_addr $x
```

is_isbn

`is_isbn` — predicate: 1 if input is isbn, 0 otherwise. Category: *validation / input checks*.

```
p is_isbn $x
```

is_isbn10

`is_isbn10` — predicate: 1 if input is isbn10, 0 otherwise. Category: *extended stdlib*.

```
p is_isbn10 $x
```

is_isbn13

`is_isbn13` — predicate: 1 if input is isbn13, 0 otherwise. Category: *extended stdlib*.

```
p is_isbn13 $x
```

is_iso_date

`is_iso_date` — predicate: 1 if input is iso date, 0 otherwise. Category: *extended stdlib*.

```
p is_iso_date $x
```

is_iso_datetime

`is_iso_datetime` — predicate: 1 if input is iso datetime, 0 otherwise. Category: *extended stdlib*.

```
p is_iso_datetime $x
```

is_iso_time

`is_iso_time` — predicate: 1 if input is iso time, 0 otherwise. Category: *extended stdlib*.

```
p is_iso_time $x
```

is_isogram

`is_isogram` — predicate: 1 if input is isogram, 0 otherwise. Category: *extended stdlib*.

```
p is_isogram $x
```

is_json

`is_json` — predicate: 1 if input is json, 0 otherwise. Category: *extended stdlib*.

```
p is_json $x
```

is_jwt

`is_jwt` — predicate: 1 if input is jwt, 0 otherwise. Category: *validation / input checks*.

```
p is_jwt $x
```

is_kaprekar

`is_kaprekar` — predicate: 1 if input is kaprekar, 0 otherwise. Category: *extended stdlib*.

```
p is_kaprekar $x
```

is_keith

`is_keith` — predicate: 1 if input is keith, 0 otherwise. Alias for `keith_q`. Category: *misc*.

```
p is_keith $x
```

is_leap

`is_leap` — predicate: 1 if input is leap, 0 otherwise. Alias for `is_leap_year`. Category: *date helpers*.

```
p is_leap $x
```

is_leap_year

`is_leap_year` — predicate: 1 if input is leap year, 0 otherwise. Category: *date helpers*.

```
p is_leap_year $x
```

is_lower

`is_lower` — predicate: 1 if input is lower, 0 otherwise. Category: *trivial string ops*.

```
p is_lower $x
```

is_lowercase

`is_lowercase` — predicate: 1 if input is lowercase, 0 otherwise. Category: *validation / input checks*.

```
p is_lowercase $x
```

is_mac

`is_mac` — predicate: 1 if input is mac, 0 otherwise. Category: *validation / input checks*.

```
p is_mac $x
```

is_mac_addr

`is_mac_addr` — predicate: 1 if input is mac addr, 0 otherwise. Category: *extended stdlib*.

```
p is_mac_addr $x
```

is_match

`is_match` — predicate: 1 if input is match, 0 otherwise. Category: *extended stdlib*.

```
p is_match $x
```

is_md5_hash

`is_md5_hash` — predicate: 1 if input is md5 hash, 0 otherwise. Category: *validation / input checks*.

```
p is_md5_hash $x
```

is_monotonic

`is_monotonic` — predicate: 1 if input is monotonic, 0 otherwise. Category: *extended stdlib*.

```
p is_monotonic $x
```

is_multiple_of

`is_multiple_of` — predicate: 1 if input is multiple of, 0 otherwise. Category: **predicates**.

```
p is_multiple_of $x
```

is_nan

`is_nan` — predicate: 1 if input is nan, 0 otherwise. Category: **trig / math**.

```
p is_nan $x
```

is_narcissistic

`is_narcissistic` — predicate: 1 if input is narcissistic, 0 otherwise. Category: **extended stdlib**.

```
p is_narcissistic $x
```

is_negative

`is_negative` — predicate: 1 if input is negative, 0 otherwise. Category: **predicates**.

```
p is_negative $x
```

is_negative_num

`is_negative_num` — predicate: 1 if input is negative num, 0 otherwise. Category: **extended stdlib**.

```
p is_negative_num $x
```

is_nil

`is_nil` — predicate: 1 if input is nil, 0 otherwise. Category: **predicates**.

```
p is_nil $x
```

is_nonzero

`is_nonzero` — predicate: 1 if input is nonzero, 0 otherwise. Category: **predicates**.

```
p is_nonzero $x
```

is_numeric

`is_numeric` — predicate: 1 if input is numeric, 0 otherwise. Category: **trivial string ops**.

```
p is_numeric $x
```

is_numeric_only

`is_numeric_only` — predicate: 1 if input is numeric only, 0 otherwise. Category: **validation / input checks**.

```
p is_numeric_only $x
```

is_numeric_string

`is_numeric_string` — predicate: 1 if input is numeric string, 0 otherwise. Category: **validation predicates extras**.

```
p is_numeric_string $x
```

is_octal

`is_octal` — predicate: 1 if input is octal, 0 otherwise. Category: **validation / input checks**.

```
p is_octal $x
```

is_octal_str

`is_octal_str` — predicate: 1 if input is octal str, 0 otherwise. Category: **extended stdlib**.

```
p is_octal_str $x
```

is_odd

`is_odd` — predicate: 1 if input is odd, 0 otherwise. Category: **predicates**.

```
p is_odd $x
```

is_odd_num

`is_odd_num` — predicate: 1 if input is odd num, 0 otherwise. Category: **extended stdlib**.

```
p is_odd_num $x
```

is_pair

`is_pair` — predicate: 1 if input is pair, 0 otherwise. Category: **predicates**.

```
p is_pair $x
```

is_palindrome

`is_palindrome` — predicate: 1 if input is palindrome, 0 otherwise. Category: **string**.

```
p is_palindrome $x
```

is_palindrome_str

`is_palindrome_str` — predicate: 1 if input is palindrome str, 0 otherwise. Category: **validation / input checks**.

```
p is_palindrome_str $x
```

is_pangram

`is_pangram` — predicate: 1 if input is pangram, 0 otherwise. Category: **validation predicates extras**.

```
p is_pangram $x
```

is_pentagonal

`is_pentagonal` — predicate: 1 if input is pentagonal, 0 otherwise. Category: **number theory / primes**.

```
p is_pentagonal $x
```

is_perfect

`is_perfect` — predicate: 1 if input is perfect, 0 otherwise. Category: **number theory / primes**.

```
p is_perfect $x
```


is_perfect_square

`is_perfect_square` — predicate: 1 if input is perfect square, 0 otherwise. Category: **predicates**.

```
p is_perfect_square $x
```

is_permutation

`is_permutation` — predicate: 1 if input is permutation, 0 otherwise. Category: **predicates**.

```
p is_permutation $x
```

is_phone

`is_phone` — predicate: 1 if input is phone, 0 otherwise. Category: **validation / input checks**.

```
p is_phone $x
```

is_phone_e164

`is_phone_e164` — predicate: 1 if input is phone e164, 0 otherwise. Category: **validation / input checks**.

```
p is_phone_e164 $x
```

is_phone_num

`is_phone_num` — predicate: 1 if input is phone num, 0 otherwise. Category: **extended stdlib**.

```
p is_phone_num $x
```

is_port_num

`is_port_num` — predicate: 1 if input is port num, 0 otherwise. Category: **extended stdlib**.

```
p is_port_num $x
```

is_positive

`is_positive` — predicate: 1 if input is positive, 0 otherwise. Category: **predicates**.

```
p is_positive $x
```

is_positive_num

`is_positive_num` — predicate: 1 if input is positive num, 0 otherwise. Category: **extended stdlib**.

```
p is_positive_num $x
```

is_postal_code

`is_postal_code` — predicate: 1 if input is postal code, 0 otherwise. Category: **validation / input checks**.

```
p is_postal_code $x
```

is_pow2

`is_pow2` — predicate: 1 if input is pow2, 0 otherwise. Alias for `is_power_of_two`. Category: **trivial numeric / predicate builtins**.

```
p is_pow2 $x
```

is_power_of

`is_power_of` — predicate: 1 if input is power of, 0 otherwise. Category: **predicates**.

```
p is_power_of $x
```

is_power_of_two

`is_power_of_two` — predicate: 1 if input is power of two, 0 otherwise. Category: **trivial numeric / predicate builtins**.

```
p is_power_of_two $x
```

is_prefix

`is_prefix` — predicate: 1 if input is prefix, 0 otherwise. Category: **predicates**.

```
p is_prefix $x
```

is_present

`is_present` — predicate: 1 if input is present, 0 otherwise. Category: **predicates**.

```
p is_present $x
```

is_prime

`is_prime` — predicate: 1 if input is prime, 0 otherwise. Category: **trivial numeric / predicate builtins**.

```
p is_prime $x
```

is_printable

`is_printable` — predicate: 1 if input is printable, 0 otherwise. Category: **validation predicates extras**.

```
p is_printable $x
```

is_printable_ascii

`is_printable_ascii` — predicate: 1 if input is printable ascii, 0 otherwise. Category: **validation / input checks**.

```
p is_printable_ascii $x
```

is_probable_prime

`miller_rabin` (aliases `millerrabin`, `is_probable_prime`) performs a probabilistic primality test with `k` rounds (default 20).

```
p millerrabin(104729)    # 1 (prime)
p millerrabin(104730)    # 0 (composite)
```

is_quadratic_residue

`is_quadratic_residue` — predicate: 1 if input is quadratic residue, 0 otherwise. Category: **math / number theory extras**.

```
p is_quadratic_residue $x
```

is_readable

`is_readable` — predicate: 1 if input is readable, 0 otherwise. Category: *file stat / path*.

```
p is_readable $x
```

is_ref

`is_ref` — predicate: 1 if input is ref, 0 otherwise. Category: *trivial type predicates*.

```
p is_ref $x
```

is_regex_valid

`is_regex_valid` — predicate: 1 if input is regex valid, 0 otherwise. Category: *more regex*.

```
p is_regex_valid $x
```

is_root

`is_root` — predicate: 1 if input is root, 0 otherwise. Category: *more process / system*.

```
p is_root $x
```

is_semver

`is_semver` — predicate: 1 if input is semver, 0 otherwise. Category: *extended stdlib*.

```
p is_semver $x
```

is_sha1_hash

`is_sha1_hash` — predicate: 1 if input is sha1 hash, 0 otherwise. Category: *validation / input checks*.

```
p is_sha1_hash $x
```

is_sha256_hash

`is_sha256_hash` — predicate: 1 if input is sha256 hash, 0 otherwise. Category: *validation / input checks*.

```
p is_sha256_hash $x
```

is_slug

`is_slug` — predicate: 1 if input is slug, 0 otherwise. Category: *extended stdlib*.

```
p is_slug $x
```

is_smith

`is_smith` — predicate: 1 if input is smith, 0 otherwise. Category: *number theory / primes*.

```
p is_smith $x
```

is_sorted

`is_sorted` — predicate: 1 if input is sorted, 0 otherwise. Category: *predicates*.

```
p is_sorted $x
```

is_sorted_by

`is_sorted_by` — predicate: 1 if input is sorted by, 0 otherwise. Category: **additional missing stdlib functions**.

```
p is_sorted_by $x
```

is_sorted_desc

`is_sorted_desc` — predicate: 1 if input is sorted desc, 0 otherwise. Category: **predicates**.

```
p is_sorted_desc $x
```

is_space

`is_space` — predicate: 1 if input is space, 0 otherwise. Category: **trivial string ops**.

```
p is_space $x
```

is_sphenic

`is_sphenic` — predicate: 1 if input is sphenic, 0 otherwise. Alias for `sphenic_q`. Category: **misc**.

```
p is_sphenic $x
```

is_square

`is_square` — predicate: 1 if input is square, 0 otherwise. Category: **trivial numeric / predicate builtins**.

```
p is_square $x
```

is_squarefree

`is_squarefree` — predicate: 1 if input is squarefree, 0 otherwise. Category: **extended stdlib**.

```
p is_squarefree $x
```

is_ssn_us

`is_ssn_us` — predicate: 1 if input is ssn us, 0 otherwise. Category: **validation / input checks**.

```
p is_ssn_us $x
```

is_str

`is_str` — predicate: 1 if input is str, 0 otherwise. Alias for `is_string`. Category: **trivial type predicates**.

```
p is_str $x
```

is_strictly_decreasing

`is_strictly_decreasing` — predicate: 1 if input is strictly decreasing, 0 otherwise. Category: **predicates**.

```
p is_strictly_decreasing $x
```

is_strictly_increasing

`is_strictly_increasing` — predicate: 1 if input is strictly increasing, 0 otherwise. Category: **predicates**.

```
p is_strictly_increasing $x
```

is_string

`is_string` — predicate: 1 if input is string, 0 otherwise. Category: **trivial type predicates**.

```
p is_string $x
```

is_subset

`is_subset` — predicate: 1 if input is subset, 0 otherwise. Category: **predicates**.

```
p is_subset $x
```

is_suffix

`is_suffix` — predicate: 1 if input is suffix, 0 otherwise. Category: **predicates**.

```
p is_suffix $x
```

is_superset

`is_superset` — predicate: 1 if input is superset, 0 otherwise. Category: **predicates**.

```
p is_superset $x
```

is_swift

`is_swift` — predicate: 1 if input is swift, 0 otherwise. Category: **validation / input checks**.

```
p is_swift $x
```

is_symlink

`is_symlink` — predicate: 1 if input is symlink, 0 otherwise. Category: **file stat / path**.

```
p is_symlink $x
```

is_titlecase

`is_titlecase` — predicate: 1 if input is titlecase, 0 otherwise. Category: **validation / input checks**.

```
p is_titlecase $x
```

is_tree

`is_tree` — predicate: 1 if input is tree, 0 otherwise. Category: **graph algorithms**.

```
p is_tree $x
```

is_triangular

`is_triangular` — predicate: 1 if input is triangular, 0 otherwise. Category: **predicates**.

```
p is_triangular $x
```

is_triple

`is_triple` — predicate: 1 if input is triple, 0 otherwise. Category: **predicates**.

```
p is_triple $x
```

is_truthy

`is_truthy` — predicate: 1 if input is truthy, 0 otherwise. Category: **predicates**.

```
p is_truthy $x
```

is_tty

`is_tty` — predicate: 1 if input is tty, 0 otherwise. Category: **more process / system**.

```
p is_tty $x
```

is_ulid

`is_ulid` *SCALAR* → 1/0 per the 26-char Crockford-Base32 format (`[0-9A-HJKMNP-TV-Z]{26}`). Excludes `I`, `L`, `O`, `U` per Crockford spec.

is_undef

`is_undef` — predicate: 1 if input is undef, 0 otherwise. Category: **trivial type predicates**.

```
p is_undef $x
```

is_upc

`is_upc` — predicate: 1 if input is upc, 0 otherwise. Category: **validation / input checks**.

```
p is_upc $x
```

is_upper

`is_upper` — predicate: 1 if input is upper, 0 otherwise. Category: **trivial string ops**.

```
p is_upper $x
```

is_uppercase

`is_uppercase` — predicate: 1 if input is uppercase, 0 otherwise. Category: **validation / input checks**.

```
p is_uppercase $x
```

is_url

`is_url` — predicate: 1 if input is url, 0 otherwise. Category: **predicates**.

```
p is_url $x
```

is_url_http

`is_url_http` — predicate: 1 if input is url http, 0 otherwise. Category: **validation / input checks**.

```
p is_url_http $x
```

is_url_https

`is_url_https` — predicate: 1 if input is url https, 0 otherwise. Category: **validation / input checks**.

```
p is_url_https $x
```

is_us_zip

`is_us_zip` — predicate: 1 if input is us zip, 0 otherwise. Category: **extended stdlib**.

```
p is_us_zip $x
```

is_utf8

`is_utf8` — predicate: 1 if input is utf8, 0 otherwise. Category: **validation / input checks**.

```
p is_utf8 $x
```

is_uuid

`is_uuid` — predicate: 1 if input is uuid, 0 otherwise. Category: **id helpers**.

```
p is_uuid $x
```

is_uuid_v4

`is_uuid_v4` — predicate: 1 if input is uuid v4, 0 otherwise. Category: **validation / input checks**.

```
p is_uuid_v4 $x
```

is_uuid_v7

`is_uuid_v7` — predicate: 1 if input is uuid v7, 0 otherwise. Category: **validation / input checks**.

```
p is_uuid_v7 $x
```

is_valid_cidr

`is_valid_cidr` — predicate: 1 if input is valid cidr, 0 otherwise. Category: **validation predicates extras**.

```
p is_valid_cidr $x
```

is_valid_cron

`is_valid_cron` — predicate: 1 if input is valid cron, 0 otherwise. Category: **validation predicates extras**.

```
p is_valid_cron $x
```

is_valid_date

`is_valid_date` — predicate: 1 if input is valid date, 0 otherwise. Category: **extended stdlib**.

```
p is_valid_date $x
```

is_valid_email

`is_valid_email` — predicate: 1 if input is valid email, 0 otherwise. Category: **network / validation**.

```
p is_valid_email $x
```

is_valid_hex

`is_valid_hex` — predicate: 1 if input is valid hex, 0 otherwise. Category: **file stat / path**.

```
p is_valid_hex $x
```

is_valid_hex_color

`is_valid_hex_color` — predicate: 1 if input is valid hex color, 0 otherwise. Category: **validation predicates extras**.

```
p is_valid_hex_color $x
```

is_valid_ipv4

`is_valid_ipv4` — predicate: 1 if input is valid ipv4, 0 otherwise. Category: **network / validation**.

```
p is_valid_ipv4 $x
```

is_valid_ipv6

`is_valid_ipv6` — predicate: 1 if input is valid ipv6, 0 otherwise. Category: **network / validation**.

```
p is_valid_ipv6 $x
```

is_valid_latitude

`is_valid_latitude` — predicate: 1 if input is valid latitude, 0 otherwise. Category: **validation predicates extras**.

```
p is_valid_latitude $x
```

is_valid_longitude

`is_valid_longitude` — predicate: 1 if input is valid longitude, 0 otherwise. Category: **validation predicates extras**.

```
p is_valid_longitude $x
```

is_valid_mime

`is_valid_mime` — predicate: 1 if input is valid mime, 0 otherwise. Category: **validation predicates extras**.

```
p is_valid_mime $x
```

is_valid_url

`is_valid_url` — predicate: 1 if input is valid url, 0 otherwise. Category: **network / validation**.

```
p is_valid_url $x
```

is_vin

`is_vin` — predicate: 1 if input is vin, 0 otherwise. Category: **validation / input checks**.

```
p is_vin $x
```

is_vowel

`is_vowel` — predicate: 1 if input is vowel, 0 otherwise. Category: **string**.

```
p is_vowel $x
```

is_weekday

`is_weekday` — predicate: 1 if input is weekday, 0 otherwise. Category: **date**.

```
p is_weekday $x
```


is_weekend

`is_weekend` — predicate: 1 if input is weekend, 0 otherwise. Category: **date**.

```
p is_weekend $x
```

is_whitespace

`is_whitespace` — predicate: 1 if input is whitespace, 0 otherwise. Alias for `is_space`. Category: **trivial string ops**.

```
p is_whitespace $x
```

is_whole

`is_whole` — predicate: 1 if input is whole, 0 otherwise. Category: **predicates**.

```
p is_whole $x
```

is_whole_num

`is_whole_num` — predicate: 1 if input is whole num, 0 otherwise. Category: **extended stdlib**.

```
p is_whole_num $x
```

is_writable

`is_writable` — predicate: 1 if input is writable, 0 otherwise. Category: **file stat / path**.

```
p is_writable $x
```

is_zero

`is_zero` — predicate: 1 if input is zero, 0 otherwise. Category: **predicates**.

```
p is_zero $x
```

is_zero_num

`is_zero_num` — predicate: 1 if input is zero num, 0 otherwise. Category: **extended stdlib**.

```
p is_zero_num $x
```

is_zip_plus4

`is_zip_plus4` — predicate: 1 if input is zip plus4, 0 otherwise. Category: **validation / input checks**.

```
p is_zip_plus4 $x
```

is_zip_us

`is_zip_us` — predicate: 1 if input is zip us, 0 otherwise. Category: **validation / input checks**.

```
p is_zip_us $x
```

isabun

`isabun` — operation `isabun`. Alias for `is_abundant`. Category: **number theory / primes**.

```
p isabun $x
```

isanag

isanag — operation isanag. Alias for is_anagram. Category: *validation predicates extras*.

```
p isanag $x
```

isasc

isasc — operation isasc. Alias for is_ascii. Category: *extended stdlib*.

```
p isasc $x
```

isb64

isb64 — operation isb64. Alias for is_base64. Category: *extended stdlib*.

```
p isb64 $x
```

isbalp

isbalp — operation isbalp. Alias for is_balanced_parens. Category: *validation predicates extras*.

```
p isbalp $x
```

isbin

isbin — operation isbin. Alias for is_binary_str. Category: *extended stdlib*.

```
p isbin $x
```

isbip

isbip — operation isbip. Alias for is_bipartite_graph. Category: *extended stdlib*.

```
p isbip $x
```

isbn10

isbn10 — operation isbn10. Alias for is_isbn10. Category: *extended stdlib*.

```
p isbn10 $x
```

isbn10_check

isbn10_check — operation isbn10 check. Category: *b82-misc-utility*.

```
p isbn10_check $x
```

isbn10_to_isbn13

isbn10_to_isbn13 — convert from isbn10 to isbn13. Category: *validation / input checks*.

```
p isbn10_to_isbn13 $value
```

isbn13

isbn13 — operation isbn13. Alias for is_isbn13. Category: *extended stdlib*.

```
p isbn13 $x
```

isbn13_check

`isbn13_check` — operation `isbn13_check`. Category: *b82-misc-utility*.

```
p isbn13_check $x
```

isbn13_to_isbn10

`isbn13_to_isbn10` — convert from `isbn13` to `isbn10`. Category: *validation / input checks*.

```
p isbn13_to_isbn10 $value
```

iscc

`iscc` — operation `iscc`. Alias for `is_credit_card`. Category: *extended stdlib*.

```
p iscc $x
```

iscidr

`iscidr` — operation `iscidr`. Alias for `is_valid_cidr`. Category: *validation predicates extras*.

```
p iscidr $x
```

isco_radius_kerr_step

`isco_radius_kerr_step` — single-step update of `isco radius kerr`. Category: *electrochemistry, batteries, fuel cells*.

```
p isco_radius_kerr_step $x
```

iscopr

`iscopr` — operation `iscopr`. Alias for `is_coprime`. Category: *extended stdlib*.

```
p iscopr $x
```

iscron

`iscron` — operation `iscron`. Alias for `is_valid_cron`. Category: *validation predicates extras*.

```
p iscron $x
```

isctrl

`isctrl` — operation `isctrl`. Alias for `is_control`. Category: *validation predicates extras*.

```
p isctrl $x
```

isdefi

`isdefi` — operation `isdefi`. Alias for `is_deficient`. Category: *number theory / primes*.

```
p isdefi $x
```

iseven

`iseven` — operation `iseven`. Alias for `is_even_num`. Category: *extended stdlib*.

```
p iseven $x
```

isharsh

`isharsh` — operation `isharsh`. Alias for `is_harshad`. Category: **extended stdlib**.

```
p isharsh $x
```

ishet

`ishet` — operation `ishet`. Alias for `is_heterogram`. Category: **extended stdlib**.

```
p ishet $x
```

ishex

`ishex` — operation `ishex`. Alias for `is_hex_str`. Category: **extended stdlib**.

```
p ishex $x
```

ishexc

`ishexc` — operation `ishexc`. Alias for `is_valid_hex_color`. Category: **validation predicates extras**.

```
p ishexc $x
```

ishost

`ishost` — operation `ishost`. Alias for `is_hostname_valid`. Category: **extended stdlib**.

```
p ishost $x
```

isiban

`isiban` — operation `isiban`. Alias for `is_iban`. Category: **extended stdlib**.

```
p isiban $x
```

isip4

`isip4` — operation `isip4`. Alias for `is_ipv4_addr`. Category: **extended stdlib**.

```
p isip4 $x
```

isip6

`isip6` — operation `isip6`. Alias for `is_ipv6_addr`. Category: **extended stdlib**.

```
p isip6 $x
```

isiso

`isiso` — operation `isiso`. Alias for `is_isogram`. Category: **extended stdlib**.

```
p isiso $x
```

isisodt

`isisodt` — operation `isisodt`. Alias for `is_iso_date`. Category: **extended stdlib**.

```
p isisodt $x
```

isisodtm

`isisodtm` — operation `isisodtm`. Alias for `is_iso_datetime`. Category: **extended stdlib**.

```
p isisodtm $x
```

isisotm

`isisotm` — operation `isisotm`. Alias for `is_iso_time`. Category: **extended stdlib**.

```
p isisotm $x
```

isjson

`isjson` — operation `isjson`. Alias for `is_json`. Category: **extended stdlib**.

```
p isjson $x
```

iskap

`iskap` — operation `iskap`. Alias for `is_kaprekar`. Category: **extended stdlib**.

```
p iskap $x
```

isl

`isl` — operation `isl`. Alias for `islice`. Category: **python/ruby stdlib**.

```
p isl $x
```

islamic_leap_year

`islamic_leap_year` — operation `islamic leap year`. Category: **calendrical algorithms**.

```
p islamic_leap_year $x
```

island_equilibrium

`island_equilibrium` — operation `island equilibrium`. Category: **biology / ecology**.

```
p island_equilibrium $x
```

islat

`islat` — operation `islat`. Alias for `is_valid_latitude`. Category: **validation predicates extras**.

```
p islat $x
```

islice

`islice` — operation `islice`. Category: **python/ruby stdlib**.

```
p islice $x
```

islon

`islon` — operation `islon`. Alias for `is_valid_longitude`. Category: **validation predicates extras**.

```
p islon $x
```

ism

`ism` — operation `ism`. Alias for `is_match`. Category: **extended stdlib**.

```
p ism $x
```

ismac

`ismac` — operation `ismac`. Alias for `is_mac_addr`. Category: **extended stdlib**.

```
p ismac $x
```

ismime

`ismime` — operation `ismime`. Alias for `is_valid_mime`. Category: **validation predicates extras**.

```
p ismime $x
```

ismono

`ismono` — operation `ismono`. Alias for `is_monotonic`. Category: **extended stdlib**.

```
p ismono $x
```

isnarc

`isnarc` — operation `isnarc`. Alias for `is_narcissistic`. Category: **extended stdlib**.

```
p isnarc $x
```

isneg

`isneg` — operation `isneg`. Alias for `is_negative_num`. Category: **extended stdlib**.

```
p isneg $x
```

isnumstr

`isnumstr` — operation `isnumstr`. Alias for `is_numeric_string`. Category: **validation predicates extras**.

```
p isnumstr $x
```

iso226_phon_adjustment

`iso226_phon_adjustment` — operation `iso226 phon adjustment`. Category: **astronomy / music / color / units**.

```
p iso226_phon_adjustment $x
```

iso8601_duration_parse

`iso8601_duration_parse` — operation `iso8601 duration parse`. Category: **b82-misc-utility**.

```
p iso8601_duration_parse $x
```

iso8601_duration_to_seconds

`iso8601_duration_to_seconds` — convert from `iso8601 duration` to `seconds`. Category: **b82-misc-utility**.

```
p iso8601_duration_to_seconds $value
```

iso_day_number

`iso_day_number` — operation `iso day number`. Category: **calendrical algorithms**.

```
p iso_day_number $x
```

iso_dow

`iso_dow` — operation `iso dow`. Alias for `day_of_week_iso`. Category: **misc**.

```
p iso_dow $x
```

iso_ordinal_date

`iso_ordinal_date` — operation `iso ordinal date`. Category: **misc**.

```
p iso_ordinal_date $x
```

iso_week

`iso_week` — operation `iso week`. Alias for `iso_week_number`. Category: **misc**.

```
p iso_week $x
```

iso_week_date

`iso_week_date` — operation `iso week date`. Category: **calendrical algorithms**.

```
p iso_week_date $x
```

iso_week_number

`iso_week_number` — operation `iso week number`. Category: **misc**.

```
p iso_week_number $x
```

isoct

`isoct` — operation `isoct`. Alias for `is_octal_str`. Category: **extended stdlib**.

```
p isoct $x
```

isodd

`isodd` — operation `isodd`. Alias for `is_odd_num`. Category: **extended stdlib**.

```
p isodd $x
```

isoelectric_point_protein

`isoelectric_point_protein` — operation `isoelectric point protein`. Category: **chemistry & biochemistry**.

```
p isoelectric_point_protein $x
```

isolated_horizon_charge

`isolated_horizon_charge` — operation `isolated horizon charge`. Category: **electrochemistry, batteries, fuel cells**.

```
p isolated_horizon_charge $x
```

isotropic_relation_check

`isotropic_relation_check` — operation `isotropic_relation_check`. Category: **climate, fluids, atmospheric**.

```
p isotropic_relation_check $x
```

isp

`isp` — operation `isp`. Alias for `intersperse_val`. Category: **haskell list functions**.

```
p isp $x
```

ispang

`ispang` — operation `ispang`. Alias for `is_pangram`. Category: **validation predicates extras**.

```
p ispang $x
```

ispent

`ispent` — operation `ispent`. Alias for `is_pentagonal`. Category: **number theory / primes**.

```
p ispent $x
```

isperf

`isperf` — operation `isperf`. Alias for `is_perfect`. Category: **number theory / primes**.

```
p isperf $x
```

isphone

`isphone` — operation `isphone`. Alias for `is_phone_num`. Category: **extended stdlib**.

```
p isphone $x
```

isport

`isport` — operation `isport`. Alias for `is_port_num`. Category: **extended stdlib**.

```
p isport $x
```

ispos

`ispos` — operation `ispos`. Alias for `is_positive_num`. Category: **extended stdlib**.

```
p ispos $x
```

isprint

`isprint` — operation `isprint`. Alias for `is_printable`. Category: **validation predicates extras**.

```
p isprint $x
```

ispw

`ispw` — operation `ispw`. Alias for `intersperse_with`. Category: **additional missing stdlib functions**.

```
p ispw $x
```


isslug

`isslug` — operation `isslug`. Alias for `is_slug`. Category: *extended stdlib*.

```
p isslug $x
```

issmith

`issmith` — operation `issmith`. Alias for `is_smith`. Category: *number theory / primes*.

```
p issmith $x
```

issqfr

`issqfr` — operation `issqfr`. Alias for `is_squarefree`. Category: *extended stdlib*.

```
p issqfr $x
```

issrtb

`issrtb` — operation `issrtb`. Alias for `is_sorted_by`. Category: *additional missing stdlib functions*.

```
p issrtb $x
```

issv

`issv` — operation `issv`. Alias for `is_semver`. Category: *extended stdlib*.

```
p issv $x
```

ista_step

`ista_step` — single-step update of `ista`. Category: *combinatorial optimization, scheduling*.

```
p ista_step $x
```

istft_step

`istft_step` — single-step update of `istft`. Category: *economics + game theory*.

```
p istft_step $x
```

isvdate

`isvdate` — operation `isvdate`. Alias for `is_valid_date`. Category: *extended stdlib*.

```
p isvdate $x
```

iswap_gate

`iswap_gate` — operation `iswap_gate`. Category: *more extensions*.

```
p iswap_gate $x
```

isweak

`isweak` — operation `isweak`. Category: *list / aggregate*.

```
p isweak $x
```

iswhole

`iswhole` — operation `iswhole`. Alias for `is_whole_num`. Category: **extended stdlib**.

```
p iswhole $x
```

iszero

`iszero` — operation `iszero`. Alias for `is_zero_num`. Category: **extended stdlib**.

```
p iszero $x
```

iszip

`iszip` — operation `iszip`. Alias for `is_us_zip`. Category: **extended stdlib**.

```
p iszip $x
```

italic

`italic` — operation `italic`. Alias for `red`. Category: **color / ansi**.

```
p italic $x
```

itcz_position_lat

`itcz_position_lat` — operation `itcz position lat`. Category: **climate, fluids, atmospheric**.

```
p itcz_position_lat $x
```

iter

`iter` — operation `iter`. Alias for `iterate`. Category: **algebraic match**.

```
p iter $x
```

iterate

`iterate` — operation `iterate`. Category: **algebraic match**.

```
p iterate $x
```

iterate_n

`iterate_n` — operation `iterate n`. Category: **functional combinators**.

```
p iterate_n $x
```

iv

`iv` — operation `iv`. Category: **numpy + scipy.special**.

```
p iv $x
```

iv_estimator

`iv_estimator` — operation `iv estimator`. Category: **econometrics**.

```
p iv_estimator $x
```

jaccard

`jaccard` — operation `jaccard`. Alias for `jaccard_index`. Category: *extended stdlib*.

```
p jaccard $x
```

jaccard_index

`jaccard_index` — index of `jaccard`. Category: *extended stdlib*.

```
p jaccard_index $x
```

jaccard_sim

`jaccard_sim(set_a, set_b) → number` — $|A \cap B| / |A \cup B|$. Sets are passed as arrays.

jaccard_similarity

`jaccard_similarity` — similarity metric of `jaccard`. Category: *math functions*.

```
p jaccard_similarity $x
```

jacobi_cn_small_q

`jacobi_cn_small_q` — operation `jacobi cn small q`. Category: *special functions extra*.

```
p jacobi_cn_small_q $x
```

jacobi_dn_small_q

`jacobi_dn_small_q` — operation `jacobi dn small q`. Category: *special functions extra*.

```
p jacobi_dn_small_q $x
```

jacobi_sn_small_q

`jacobi_sn_small_q` — operation `jacobi sn small q`. Category: *special functions extra*.

```
p jacobi_sn_small_q $x
```

jacobian_2dof

`jacobian_2dof` — operation `jacobian 2dof`. Category: *robotics & control*.

```
p jacobian_2dof $x
```

jahn_teller_check

`jahn_teller_check` — operation `jahn teller check`. Category: *chemistry & biochemistry*.

```
p jahn_teller_check $x
```

jaro_similarity

`jaro_similarity` — similarity metric of `jaro`. Category: *extended stdlib*.

```
p jaro_similarity $x
```

jarosim

`jarosim` — operation `jarosim`. Alias for `jaro_similarity`. Category: **extended stdlib**.

```
p jarosim $x
```

jd

`jd` — operation `jd`. Alias for `json_decode`. Category: **data / network**.

```
p jd $x
```

jd_to_calendar

`jd_to_calendar` — convert from `jd` to `calendar`. Category: **astronomy / astrometry**.

```
p jd_to_calendar $value
```

je

`je` — operation `je`. Alias for `json_encode`. Category: **data / network**.

```
p je $x
```

jeans_length

`jeans_length` — operation `jeans length`. Category: **misc**.

```
p jeans_length $x
```

jenkins_hash

`jenkins_hash(s) → int` — Bob Jenkins' one-at-a-time hash; classic non-crypto 32-bit hash.

```
p jenkins_hash("hello")
```

jenkins_oat

`jenkins_oat` — Jenkins hash variant `oat`. Alias for `jenkins_one_at_a_time`. Category: **misc**.

```
p jenkins_oat $x
```

jenkins_one_at_a_time

`jenkins_one_at_a_time` — Jenkins hash variant `one at a time`. Category: **misc**.

```
p jenkins_one_at_a_time $x
```

jensen_alpha

`jensen_alpha(portfolio_return, beta, market_return, rf=0) → number` — alpha vs CAPM expectation: $r - (r_f + \beta \cdot (r_m - r_f))$.

```
p jensen_alpha(0.12, 1.1, 0.08, 0.02)
```

jensen_shannon_div

jensen_shannon_div — operation `jensen_shannon_div`. Category: **electrochemistry, batteries, fuel cells**.

```
p jensen_shannon_div $x
```

jensens_alpha

jensens_alpha — operation `jensens_alpha`. Category: **misc**.

```
p jensens_alpha $x
```

jitter_ms

jitter_ms — operation `jitter_ms`. Category: **network / ip / cidr**.

```
p jitter_ms $x
```

job_schedule_ljf

`job_schedule_ljf(durations)` → array — Longest Job First.

job_schedule_spt

`job_schedule_spt(durations)` → array — Shortest Processing Time first: indices sorted by ascending duration.

```
p job_schedule_spt([8, 1, 3, 5]) # [1, 2, 3, 0]
```

job_shop_makespan_lower

job_shop_makespan_lower — operation `job_shop_makespan_lower`. Category: **combinatorial optimization, scheduling**.

```
p job_shop_makespan_lower $x
```

johansen_test

johansen_test — statistical test of `johansen`. Category: **statsmodels**.

```
p johansen_test $x
```

johansen_trace_step

johansen_trace_step — single-step update of `johansen_trace`. Category: **econometrics**.

```
p johansen_trace_step $x
```

johnson_2m

johnson_2m — operation `johnson_2m`. Alias for `johnsons_rule`. Category: **misc**.

```
p johnson_2m $x
```

johnson_reweight

johnson_reweight — operation `johnson_reweight`. Category: **networkx graph algorithms**.

```
p johnson_reweight $x
```

johnsons_rule

johnsons_rule — operation `johnsons rule`. Category: **misc**.

```
p johnsons_rule $x
```

join_colons

join_colons — operation `join colons`. Category: **conversion / utility**.

```
p join_colons $x
```

join_commas

join_commas — operation `join commas`. Category: **conversion / utility**.

```
p join_commas $x
```

join_dashes

join_dashes — operation `join dashes`. Category: **conversion / utility**.

```
p join_dashes $x
```

join_dots

join_dots — operation `join dots`. Category: **conversion / utility**.

```
p join_dots $x
```

join_lines

join_lines — operation `join lines`. Category: **conversion / utility**.

```
p join_lines $x
```

join_pipes

join_pipes — operation `join pipes`. Category: **conversion / utility**.

```
p join_pipes $x
```

join_slashes

join_slashes — operation `join slashes`. Category: **conversion / utility**.

```
p join_slashes $x
```

join_spaces

join_spaces — operation `join spaces`. Category: **conversion / utility**.

```
p join_spaces $x
```

join_tabs

join_tabs — operation `join tabs`. Category: **conversion / utility**.

```
p join_tabs $x
```

joint_entropy

`joint_entropy(joint_matrix)` → `number` — $H(X,Y) = -\sum p \cdot \ln p$.

joint_entropy_step

`joint_entropy_step` — single-step update of `joint_entropy`. Category: *electrochemistry, batteries, fuel cells*.

```
p joint_entropy_step $x
```

jones_polynomial_at_i

`jones_polynomial_at_i` — operation `jones polynomial at i`. Category: *econometrics*.

```
p jones_polynomial_at_i $x
```

jones_polynomial_at_minus_one

`jones_polynomial_at_minus_one` — operation `jones polynomial at minus one`. Category: *econometrics*.

```
p jones_polynomial_at_minus_one $x
```

joule_heating_battery

`joule_heating_battery` — operation `joule heating battery`. Category: *electrochemistry, batteries, fuel cells*.

```
p joule_heating_battery $x
```

joules_to_cal

`joules_to_cal` — convert from `joules` to `cal`. Category: *file stat / path*.

```
p joules_to_cal $value
```

joules_to_ev

`joules_to_ev` — convert from `joules` to `ev`. Category: *astronomy / music / color / units*.

```
p joules_to_ev $value
```

jpeg_dct_8x8_quant

`jpeg_dct_8x8_quant` — operation `jpeg dct 8x8 quant`. Category: *electrochemistry, batteries, fuel cells*.

```
p jpeg_dct_8x8_quant $x
```

jpeg_markers

`jpeg_markers(hex_bytes)` → `array` — list of JPEG segment markers ("FFD8", "FFE0", ...) in order.

jpeg_zig_zag_index

`jpeg_zig_zag_index` — index of `jpeg zig zag`. Category: *electrochemistry, batteries, fuel cells*.

```
p jpeg_zig_zag_index $x
```

jq_all

jq_all — operation `jq all`. Category: *extras*.

```
p jq_all $x
```

jq_any

jq_any — operation `jq any`. Category: *extras*.

```
p jq_any $x
```

jq_chunks

jq_chunks — operation `jq chunks`. Category: *extras*.

```
p jq_chunks $x
```

jq_combinations

jq_combinations — operation `jq combinations`. Category: *extras*.

```
p jq_combinations $x
```

jq_delete

jq_delete — operation `jq delete`. Category: *extras*.

```
p jq_delete $x
```

jq_filter

jq_filter — filter op of `jq`. Category: *extras*.

```
p jq_filter $x
```

jq_first

jq_first — operation `jq first`. Category: *extras*.

```
p jq_first $x
```

jq_flatten

jq_flatten — operation `jq flatten`. Category: *extras*.

```
p jq_flatten $x
```

jq_from_entries

jq_from_entries — operation `jq from entries`. Category: *extras*.

```
p jq_from_entries $x
```

jq_get

jq_get — operation `jq get`. Category: *extras*.

```
p jq_get $x
```


jq_group_by

jq_group_by — operation `jq group by`. Category: *extras*.

```
p jq_group_by $x
```

jq_has

jq_has — operation `jq has`. Category: *extras*.

```
p jq_has $x
```

jq_index

jq_index — index of `jq`. Category: *extras*.

```
p jq_index $x
```

jq_indices

jq_indices — operation `jq indices`. Category: *extras*.

```
p jq_indices $x
```

jq_keys_at

jq_keys_at — operation `jq keys at`. Category: *extras*.

```
p jq_keys_at $x
```

jq_last

jq_last — operation `jq last`. Category: *extras*.

```
p jq_last $x
```

jq_leaf_paths

jq_leaf_paths — operation `jq leaf paths`. Category: *extras*.

```
p jq_leaf_paths $x
```

jq_length_at

jq_length_at — operation `jq length at`. Category: *extras*.

```
p jq_length_at $x
```

jq_map_values

jq_map_values — operation `jq map values`. Category: *extras*.

```
p jq_map_values $x
```

jq_max_by

jq_max_by — operation `jq max by`. Category: *extras*.

```
p jq_max_by $x
```

jq_min_by

jq_min_by — operation `jq min by`. Category: *extras*.

```
p jq_min_by $x
```

jq_paths

jq_paths — operation `jq paths`. Category: *extras*.

```
p jq_paths $x
```

jq_recurse

jq_recurse — operation `jq recurse`. Category: *extras*.

```
p jq_recurse $x
```

jq_select

jq_select — operation `jq select`. Category: *extras*.

```
p jq_select $x
```

jq_set

jq_set — operation `jq set`. Category: *extras*.

```
p jq_set $x
```

jq_sort_by

jq_sort_by — operation `jq sort by`. Category: *extras*.

```
p jq_sort_by $x
```

jq_split_at

jq_split_at — operation `jq split at`. Category: *extras*.

```
p jq_split_at $x
```

jq_to_entries

jq_to_entries — convert from `jq` to `entries`. Category: *extras*.

```
p jq_to_entries $value
```

jq_type

jq_type — operation `jq type`. Category: *extras*.

```
p jq_type $x
```

jq_unique_by

jq_unique_by — operation `jq unique by`. Category: *extras*.

```
p jq_unique_by $x
```

jq_values_at

jq_values_at — operation `jq values at`. Category: *extras*.

```
p jq_values_at $x
```

jq_walk

jq_walk — operation `jq walk`. Category: *extras*.

```
p jq_walk $x
```

jq_with_entries

jq_with_entries — operation `jq with entries`. Category: *extras*.

```
p jq_with_entries $x
```

jq_zip

jq_zip — operation `jq zip`. Category: *extras*.

```
p jq_zip $x
```

jr_european_call

jr_european_call — operation `jr european call`. Category: *financial pricing models*.

```
p jr_european_call $x
```

js_divergence_distributions

js_divergence_distributions(p, q) → number — Jensen-Shannon divergence; symmetric, bounded by $\ln 2$.

```
p js_divergence_distributions([0.5,0.5], [0.7,0.3])
```

json_array_length

json_array_length — JSON helper `array length`. Category: *postgres sql strings, json, aggregates*.

```
p json_array_length $x
```

json_build_object

json_build_object — JSON helper `build object`. Category: *postgres sql strings, json, aggregates*.

```
p json_build_object $x
```

json_diff

json_diff — JSON helper `diff`. Category: *extras*.

```
p json_diff $x
```

json_each

json_each — JSON helper `each`. Category: *postgres sql strings, json, aggregates*.

```
p json_each $x
```

json_escape

`json_escape` — JSON helper `escape`. Alias for `escape_json`. Category: *json helpers*.

```
p json_escape $x
```

json_extract_path

`json_extract_path` — JSON helper `extract path`. Category: *postgres sql strings, json, aggregates*.

```
p json_extract_path $x
```

json_merge_patch

`json_merge_patch` — JSON helper `merge patch`. Category: *extras*.

```
p json_merge_patch $x
```

json_minify

`json_minify` — JSON helper `minify`. Category: *json helpers*.

```
p json_minify $x
```

json_patch

`json_patch` — JSON helper `patch`. Category: *extras*.

```
p json_patch $x
```

json_pointer_resolve

`json_pointer_resolve` — JSON helper `pointer resolve`. Category: *extras*.

```
p json_pointer_resolve $x
```

json_pointer_set

`json_pointer_set` — JSON helper `pointer set`. Category: *extras*.

```
p json_pointer_set $x
```

json_pretty

`json_pretty` — JSON helper `pretty`. Category: *json helpers*.

```
p json_pretty $x
```

json_strip_nulls

`json_strip_nulls` — JSON helper `strip nulls`. Category: *postgres sql strings, json, aggregates*.

```
p json_strip_nulls $x
```

json_to_xml

`json_to_xml` — convert from `json` to `xml`. Category: *extras*.

```
p json_to_xml $value
```

jsonb_array_length

`jsonb_array_length` — operation `jsonb array length`. Category: *postgres sql strings, json, aggregates*.

```
p jsonb_array_length $x
```

jsonb_object_keys

`jsonb_object_keys` — operation `jsonb object keys`. Category: *postgres sql strings, json, aggregates*.

```
p jsonb_object_keys $x
```

jsonb_path_query

`jsonb_path_query` — operation `jsonb path query`. Category: *postgres sql strings, json, aggregates*.

```
p jsonb_path_query $x
```

jsonb_pretty

`jsonb_pretty` — operation `jsonb pretty`. Category: *postgres sql strings, json, aggregates*.

```
p jsonb_pretty $x
```

jsonb_set

`jsonb_set` — operation `jsonb set`. Category: *postgres sql strings, json, aggregates*.

```
p jsonb_set $x
```

jsonb_typeof

`jsonb_typeof` — operation `jsonb typeof`. Category: *postgres sql strings, json, aggregates*.

```
p jsonb_typeof $x
```

jukes_cantor_distance

`jukes_cantor_distance` — distance / dissimilarity metric of `jukes cantor`. Category: *bioinformatics deep*.

```
p jukes_cantor_distance $x
```

julian_centuries_j2000

`julian_centuries_j2000` — Julian-date conversion `centuries j2000`. Category: *more extensions*.

```
p julian_centuries_j2000 $x
```

julian_day

`julian_day` — Julian-date conversion `day`. Category: *astronomy / astrometry*.

```
p julian_day $x
```

julian_from_fixed

`julian_from_fixed` — Julian-date conversion `from fixed`. Category: *calendrical algorithms*.

```
p julian_from_fixed $x
```

julian_to_unix

`julian_to_unix(jd)` → `int` — convert Julian Date to unix timestamp.

```
p julian_to_unix(2459945.5)
```

just_intonation_freq

`just_intonation_freq` — operation `just intonation freq`. Category: **music theory**.

```
p just_intonation_freq $x
```

just_intonation_ratio

`just_intonation_ratio` — ratio of `just intonation`. Category: **astronomy / music / color / units**.

```
p just_intonation_ratio $x
```

justify_center

`justify_center(s, width)` → `string` — pad symmetrically. Extra space goes to the right when uneven.

justify_left

`justify_left(s, width)` → `string` — pad right with spaces to reach `width`.

```
p justify_left("hi", 5) # "hi  "
```

justify_right

`justify_right(s, width)` → `string` — pad left with spaces to reach `width`.

juxt2

`juxt2` — operation `juxt2`. Category: **functional combinators**.

```
p juxt2 $x
```

juxt3

`juxt3` — operation `juxt3`. Category: **functional combinators**.

```
p juxt3 $x
```

jv

`jv` — operation `jv`. Category: **numpy + scipy.special**.

```
p jv $x
```

k_coreness

`k_coreness` — operation `k coreness`. Category: **graph algorithms**.

```
p k_coreness $x
```

k_from_dg

`k_from_dg` — operation `k from dg`. Category: **chemistry**.

```
p k_from_dg $x
```

k_shortest_spanning

`k_shortest_spanning` — operation `k shortest spanning`. Category: **networkx graph algorithms**.

```
p k_shortest_spanning $x
```

k_to_c

`k_to_c` — convert from `k` to `c`. Category: **unit conversions**.

```
p k_to_c $value
```

k_to_f

`k_to_f` — convert from `k` to `f`. Category: **unit conversions**.

```
p k_to_f $value
```

ka_to_pka

`ka_to_pka` — convert from `ka` to `pka`. Category: **chemistry & biochemistry**.

```
p ka_to_pka $value
```

kahan

`kahan_sum LIST` — Neumaier-variant compensated summation. Recovers ~16-17 digits of precision on long float sums where naive `sum` loses precision to cancellation.

```
p sum(1e20, 1, -1e20, 1, -1e20, 1e20)      # 0  (precision lost)
p kahan_sum(1e20, 1, -1e20, 1, -1e20, 1e20)  # 2  (recovered)
```

World's-first as scripting-lang stdlib pipe-friendly builtin: Python's `math.fsum` is closest but not pipeable; Ruby / Node / Perl have no stdlib equivalent.

kaiser

`window_kaiser($n, $beta)` (alias `kaiser`) — generates a Kaiser window with parameter `beta` controlling the tradeoff between main lobe width and sidelobe level. `Beta=0` is rectangular, `beta~5` is similar to Hamming.

```
val $w = kaiser(1024, 5.0)
```

kaiser_window

`kaiser_window` — operation `kaiser window`. Category: **signal processing**.

```
p kaiser_window $x
```

kaiserord_step

`kaiserord_step` — single-step update of `kaiserord`. Category: **economics + game theory**.

```
p kaiserord_step $x
```

kalai_smorodinsky_step

`kalai_smorodinsky_step` — single-step update of `kalai smorodinsky`. Category: **climate, fluids, atmospheric**.

```
p kalai_smorodinsky_step $x
```

kalman_filter_step

`kalman_filter_step` — single-step update of `kalman filter`. Category: **statsmodels**.

```
p kalman_filter_step $x
```

kalman_predict_state

`kalman_predict_state` — operation `kalman predict state`. Category: **electrochemistry, batteries, fuel cells**.

```
p kalman_predict_state $x
```

kalman_smoother_step

`kalman_smoother_step` — single-step update of `kalman smoother`. Category: **statsmodels**.

```
p kalman_smoother_step $x
```

kalman_update_state

`kalman_update_state` — operation `kalman update state`. Category: **electrochemistry, batteries, fuel cells**.

```
p kalman_update_state $x
```

kaluza_klein_step

`kaluza_klein_step` — single-step update of `kaluza klein`. Category: **electrochemistry, batteries, fuel cells**.

```
p kaluza_klein_step $x
```

kama

`kama(prices, period=10) → array` — Kaufman adaptive moving average. Smoothing constant tracks volatility: $sc = (er \cdot (fast - slow) + slow)^2$, so the average accelerates in trends and idles in chop.

```
p kama(\@prices, 10)
```

kaprekar_routine_step

`kaprekar_routine_step` — single-step update of `kaprekar routine`. Category: **misc**.

```
p kaprekar_routine_step $x
```

karger_contract_edge

`karger_contract_edge` — operation `karger contract edge`. Category: **combinatorial optimization, scheduling**.

```
p karger_contract_edge $x
```


karger_min_cut_count

`karger_min_cut_count` — count of `karger min cut`. Category: *combinatorial optimization, scheduling*.

```
p karger_min_cut_count $x
```

karger_step

`karger_step` — single-step update of `karger`. Category: *networkx graph algorithms*.

```
p karger_step $x
```

karhunen_loeve_step

`karhunen_loeve_step` — single-step update of `karhunen loeve`. Category: *electrochemistry, batteries, fuel cells*.

```
p karhunen_loeve_step $x
```

kasiski_repeats

`kasiski_repeats` — operation `kasiski repeats`. Category: *cryptography deep*.

```
p kasiski_repeats $x
```

katz centrality

`katz centrality` — operation `katz centrality`. Category: *graph algorithms*.

```
p katz Centrality $x
```

kauffman_bracket_eval

`kauffman_bracket_eval` — operation `kauffman bracket eval`. Category: *econometrics*.

```
p kauffman_bracket_eval $x
```

kb_to_b

`kb_to_b` — convert from `kb` to `b`. Alias for `kb_to_bytes`. Category: *unit conversions*.

```
p kb_to_b $value
```

kb_to_bytes

`kb_to_bytes` — convert from `kb` to `bytes`. Category: *unit conversions*.

```
p kb_to_bytes $value
```

kb_to_mb

`kb_to_mb` — convert from `kb` to `mb`. Category: *unit conversions*.

```
p kb_to_mb $value
```

kc_from_rates

`kc_from_rates` — operation `kc from rates`. Category: *chemistry*.

```
p kc_from_rates $x
```

kcoulomb

`kcoulomb` — operation `kcoulomb`. Alias for `coulomb_constant`. Category: **physics constants**.

```
p kcoulomb $x
```

kde_bandwidth_lscv

`kde_bandwidth_lscv` — kernel density estimator `bandwidth lscv`. Category: **b82-misc-utility**.

```
p kde_bandwidth_lscv $x
```

kde_biweight

`kde_biweight` — kernel density estimator `biweight`. Category: **b82-misc-utility**.

```
p kde_biweight $x
```

kde_cosine

`kde_cosine` — kernel density estimator `cosine`. Category: **b82-misc-utility**.

```
p kde_cosine $x
```

kde_epanechnikov

`kde_epanechnikov` — kernel density estimator `epanechnikov`. Category: **b82-misc-utility**.

```
p kde_epanechnikov $x
```

kde_gaussian_2d

`kde_gaussian_2d` — kernel density estimator `gaussian 2d`. Category: **b82-misc-utility**.

```
p kde_gaussian_2d $x
```

kde_logistic_kernel

`kde_logistic_kernel` — kernel density estimator `logistic kernel`. Category: **b82-misc-utility**.

```
p kde_logistic_kernel $x
```

kde_scott_bw

`kde_scott_bw` — kernel density estimator `scott bw`. Category: **b82-misc-utility**.

```
p kde_scott_bw $x
```

kde_silverman_bw

`kde_silverman_bw` — kernel density estimator `silverman bw`. Category: **b82-misc-utility**.

```
p kde_silverman_bw $x
```

kde_triangular

`kde_triangular` — kernel density estimator `triangular`. Category: **b82-misc-utility**.

```
p kde_triangular $x
```

kde_triweight

`kde_triweight` — kernel density estimator `triweight`. Category: **b82-misc-utility**.

```
p kde_triweight $x
```

kde_uniform

`kde_uniform` — kernel density estimator `uniform`. Category: **b82-misc-utility**.

```
p kde_uniform $x
```

keep

`keep` — operation `keep`. Category: **algebraic match**.

```
p keep $x
```

keep_if

`keep_if` — operation `keep if`. Category: **functional combinators**.

```
p keep_if $x
```

keith_q

`keith_q` — operation `keith q`. Category: **misc**.

```
p keith_q $x
```

kelly_criterion

`kelly_criterion` — operation `kelly criterion`. Category: **more extensions (2)**.

```
p kelly_criterion $x
```

keltner_lower

`keltner_lower(highs, lows, closes, period=20, mult=2) → array` — `ema - mult · atr`.

```
p keltner_lower(\@hi, \@lo, \@cl, 20, 2)
```

keltner_upper

`keltner_upper(highs, lows, closes, period=20, mult=2) → array` — `ema + mult · atr`.

```
p keltner_upper(\@hi, \@lo, \@cl, 20, 2)
```

kemeny_score_step

`kemeny_score_step` — single-step update of `kemeny score`. Category: **climate, fluids, atmospheric**.

```
p kemeny_score_step $x
```

kendall

`kendall_tau` (aliases `kendall`, `ktau`) computes Kendall's rank correlation coefficient (`tau-b`).

```
p ktau([1,2,3,4], [1,2,4,3]) # 0.67
```

kendalltau

kendalltau — operation `kendalltau`. Category: **r / scipy distributions and tests**.

```
p kendalltau $x
```

kepler_hyperbolic

kepler_hyperbolic — operation `kepler hyperbolic`. Category: **more extensions**.

```
p kepler_hyperbolic $x
```

kepler_period_au

kepler_period_au — operation `kepler period au`. Category: **misc**.

```
p kepler_period_au $x
```

kerr_ergosphere_eq

kerr_ergosphere_eq — operation `kerr ergosphere eq`. Category: **cosmology / gr / flrw**.

```
p kerr_ergosphere_eq $x
```

kerr_horizon

kerr_horizon — operation `kerr horizon`. Category: **cosmology / gr / flrw**.

```
p kerr_horizon $x
```

kerr_newman_charge_term

kerr_newman_charge_term — operation `kerr newman charge term`. Category: **electrochemistry, batteries, fuel cells**.

```
p kerr_newman_charge_term $x
```

key_signature_flats

key_signature_flats — operation `key signature flats`. Category: **music theory**.

```
p key_signature_flats $x
```

key_signature_for

key_signature_for — operation `key signature for`. Category: **misc**.

```
p key_signature_for $x
```

key_signature_sharps

key_signature_sharps — operation `key signature sharps`. Category: **music theory**.

```
p key_signature_sharps $x
```

keys_sorted

keys_sorted — operation `keys sorted`. Category: **hash ops**.

```
p keys_sorted $x
```

kg_to_lb

`kg_to_lb` — convert from `kg` to `lb`. Category: **astronomy / music / color / units**.

```
p kg_to_lb $value
```

kg_to_lbs

`kg_to_lbs` — convert from `kg` to `lbs`. Category: **unit conversions**.

```
p kg_to_lbs $value
```

kg_to_stone

`kg_to_stone` — convert from `kg` to `stone`. Category: **unit conversions**.

```
p kg_to_stone $value
```

khinchin

`khinchin` — operation `khinchin`. Alias for `khinchin_constant`. Category: **math constants**.

```
p khinchin $x
```

khinchin_constant

`khinchin_constant` — operation `khinchin constant`. Category: **math constants**.

```
p khinchin_constant $x
```

khovanov_q_grading

`khovanov_q_grading` — operation `khovanov q grading`. Category: **econometrics**.

```
p khovanov_q_grading $x
```

khovanov_rasmussen_s

`khovanov_rasmussen_s` — operation `khovanov rasmussen s`. Category: **econometrics**.

```
p khovanov_rasmussen_s $x
```

killing_form_su2

`killing_form_su2` — operation `killing form su2`. Category: **econometrics**.

```
p killing_form_su2 $x
```

killing_vector_lie_step

`killing_vector_lie_step` — single-step update of `killing vector lie`. Category: **electrochemistry, batteries, fuel cells**.

```
p killing_vector_lie_step $x
```

kimura_2p_distance

`kimura_2p_distance` — distance / dissimilarity metric of `kimura 2p`. Category: **bioinformatics deep**.

```
p kimura_2p_distance $x
```

kinetic_energy

kinetic_energy — operation `kinetic_energy`. Category: *physics formulas*.

```
p kinetic_energy $x
```

kirnberger_iii

kirnberger_iii — operation `kirnberger_iii`. Category: *music theory*.

```
p kirnberger_iii $x
```

kl_divergence_distributions

kl_divergence_distributions(p, q) → number — discrete KL divergence $\sum p \cdot \ln(p/q)$.

kleibers_law

kleibers_law — operation `kleibers_law`. Category: *biology / ecology*.

```
p kleibers_law $x
```

km_to_miles

km_to_miles — convert from `km` to `miles`. Category: *unit conversions*.

```
p km_to_miles $value
```

kmeans

kmeans — k-means clustering (Lloyd's algorithm). Takes array of points and k. Returns cluster assignments. Like R's `kmeans()`.

```
val @clusters = @{kmeans([[0,0],[1,0],[10,10],[11,10]], 2)}  
# [0,0,1,1] - two clusters
```

kmer_jaccard

kmer_jaccard — operation `kmer_jaccard`. Category: *bioinformatics deep*.

```
p kmer_jaccard $x
```

kmh_to_mph

kmh_to_mph — convert from `kmh` to `mph`. Category: *astronomy / music / color / units*.

```
p kmh_to_mph $value
```

kmh_to_mps

kmh_to_mps — convert from `kmh` to `mps`. Category: *astronomy / music / color / units*.

```
p kmh_to_mps $value
```

kmp

`kmp` — operation `kmp`. Alias for `kmp_failure`. Category: **misc**.

```
p kmp $x
```

kmp_failure

`kmp_failure` — operation `kmp failure`. Category: **misc**.

```
p kmp_failure $x
```

knapsack_01_dp_value

`knapsack_01_dp_value` — knapsack DP variant `01 dp value`. Category: **combinatorial optimization, scheduling**.

```
p knapsack_01_dp_value $x
```

knapsack_branch_bound

`knapsack_branch_bound` — knapsack DP variant `branch bound`. Category: **combinatorial optimization, scheduling**.

```
p knapsack_branch_bound $x
```

knapsack_fractional

`knapsack_fractional(values, weights, capacity) → number` — greedy fractional knapsack by value-density.

```
p knapsack_fractional([60,100,120], [10,20,30], 50) # 240
```

knapsack_fractional_step

`knapsack_fractional_step` — knapsack DP variant `fractional step`. Category: **combinatorial optimization, scheduling**.

```
p knapsack_fractional_step $x
```

knapsack_lp_relaxation

`knapsack_lp_relaxation` — knapsack DP variant `lp relaxation`. Category: **combinatorial optimization, scheduling**.

```
p knapsack_lp_relaxation $x
```

knapsack_unbounded

`knapsack_unbounded(values, weights, capacity) → int` — unbounded 0/∞ knapsack DP.

```
p knapsack_unbounded([60,100,120], [10,20,30], 50) # 300
```

knapsack_unbounded_dp

`knapsack_unbounded_dp` — knapsack DP variant `unbounded dp`. Category: **combinatorial optimization, scheduling**.

```
p knapsack_unbounded_dp $x
```

knots_to_kmh

`knots_to_kmh` — convert from `knots` to `kmh`. Category: **astronomy / music / color / units**.

```
p knots_to_kmh $value
```

kohlrausch_law

`kohlrausch_law` — operation `kohlrausch law`. Category: **electrochemistry, batteries, fuel cells**.

```
p kohlrausch_law $x
```

kolmogorov_microscale

`kolmogorov_microscale` — operation `kolmogorov microscale`. Category: **climate, fluids, atmospheric**.

```
p kolmogorov_microscale $x
```

kolmogorov_smirnov

`kolmogorov_smirnov` — operation `kolmogorov smirnov`. Category: **r / scipy distributions and tests**.

```
p kolmogorov_smirnov $x
```

komar_mass_step

`komar_mass_step` — single-step update of `komar mass`. Category: **electrochemistry, batteries, fuel cells**.

```
p komar_mass_step $x
```

kosaraju_scc

`kosaraju_scc` — operation `kosaraju scc`. Category: **graph algorithms**.

```
p kosaraju_scc $x
```

kosaraju_step

`kosaraju_step` — single-step update of `kosaraju`. Category: **networkx graph algorithms**.

```
p kosaraju_step $x
```

koutecky_levich_intercept

`koutecky_levich_intercept` — operation `koutecky levich intercept`. Category: **electrochemistry, batteries, fuel cells**.

```
p koutecky_levich_intercept $x
```

kp

`kp` — operation `kp`. Alias for `keep`. Category: **algebraic match**.

```
p kp $x
```

kp_from_kc

`kp_from_kc` — operation `kp from kc`. Category: **chemistry**.


```
p kp_from_kc $x
```

kpss_test

`kpss_test` — statistical test of `kpss`. Category: **statsmodels**.

```
p kpss_test $x
```

kpss_test_stat

`kpss_test_stat` — operation `kpss test stat`. Category: **econometrics**.

```
p kpss_test_stat $x
```

kraus_apply

`kraus_apply` — operation `kraus apply`. Category: **quantum mechanics deep**.

```
p kraus_apply $x
```

kretschmann_schwarzschild

`kretschmann_schwarzschild` — operation `kretschmann schwarzschild`. Category: **cosmology / gr / flrw**.

```
p kretschmann_schwarzschild $x
```

kron_product

`kron_product` — operation `kron product`. Category: **numpy + scipy.special**.

```
p kron_product $x
```

kronecker_four

`kronecker_four` — operation `kronecker four`. Category: **electrochemistry, batteries, fuel cells**.

```
p kronecker_four $x
```

kronecker_three

`kronecker_three` — operation `kronecker three`. Category: **electrochemistry, batteries, fuel cells**.

```
p kronecker_three $x
```

kruskal_h

`kruskal_h` — operation `kruskal h`. Category: **r / scipy distributions and tests**.

```
p kruskal_h $x
```

kruskal_step

`kruskal_step` — single-step update of `kruskal`. Category: **networkx graph algorithms**.

```
p kruskal_step $x
```

kruskal_wallis

`kruskal_wallis(group1, group2, ...)` → {*statistic*, *df*, *p_value*} — Kruskal-Wallis H non-parametric multi-group test (rank-based ANOVA).

```
p kruskal_wallis(\@g1, \@g2, \@g3)
```

ks

`ks_test` (alias `ks`) — two-sample Kolmogorov-Smirnov test. Returns D statistic (max CDF difference). Like R's `ks.test()`.

```
val $D = ks([1,2,3,4,5], [2,3,4,5,6]) # small D = similar
```

ks_test_one_sample

`ks_test_one_sample(xs, mu=0, sigma=1)` → {*statistic*, *df*, *p_value*} — Kolmogorov-Smirnov one-sample test against Normal(μ , σ).

```
p ks_test_one_sample(\@xs, 0, 1)
```

ks_test_two_sample

`ks_test_two_sample(a, b)` → {*statistic*, *df*, *p_value*} — two-sample KS test on empirical CDFs.

```
p ks_test_two_sample(\@x, \@y)
```

ksp_from_concs

`ksp_from_concs` — operation `ksp from concs`. Category: **chemistry**.

```
p ksp_from_concs $x
```

kth_largest

`kth_largest` — operation `kth largest`. Category: **extended stdlib**.

```
p kth_largest $x
```

kth_smallest

`kth_smallest` — operation `kth smallest`. Category: **extended stdlib**.

```
p kth_smallest $x
```

kthl

`kthl` — operation `kthl`. Alias for `kth_largest`. Category: **extended stdlib**.

```
p kthl $x
```

kths

`kths` — operation `kths`. Alias for `kth_smallest`. Category: **extended stdlib**.

```
p kths $x
```

kullback_jensen_div

`kullback_jensen_div` — operation `kullback jensen div`. Category: *electrochemistry, batteries, fuel cells*.

```
p kullback_jensen_div $x
```

kurt

`kurt` — operation `kurt`. Alias for `kurtosis`. Category: *math / numeric extras*.

```
p kurt $x
```

kurtosis

`kurtosis` — operation `kurtosis`. Category: *math / numeric extras*.

```
p kurtosis $x
```

kv

`kv` — operation `kv`. Category: *numpy + scipy.special*.

```
p kv $x
```

kv_batch

`kv_batch(STORE, [{"put", k, v}, {"del", k}, ...])` → number of ops applied. All-or-nothing on the in-memory state: if any op rejects (unknown kind, etc.), the entries map is rolled back to the snapshot taken at entry. Caller invokes `kv_commit` afterwards for durability.

```
kv_batch($db, [
  ["put", "counter:a", 1],
  ["put", "counter:b", 2],
  ["del", "old_key"],
]);
kv_commit($db);
```

kv_close

`kv_close(STORE)` → 1. Auto-commits if dirty, then drops the handle (Arc decref). Safe to omit if `kv_commit` was already called.

kv_commit

`kv_commit(STORE)` → 1. Atomic rewrite of the whole archive to disk via tmp + rename + fsync. No-op if not dirty. Versioned header (STKV magic + `format_version`) lets future stryke versions reject mismatched archives instead of silently mis-deserialising.

kv_count

`kv_len(STORE)` → entry count.

kv_del

`kv_del(STORE, KEY)` → 1 if the key existed, 0 otherwise. Marks store dirty so the next `kv_commit` persists the removal.

kv_exists

`kv_exists(STORE, KEY)` → 1 / 0. Constant-time hash lookup.

kv_get

`kv_get(STORE, KEY)` → value or `undef`. Compound values come back as refs (`->[i]` / `->{k}` arrow deref works directly). Reads are zero-copy validation on the mmap'd archive when no write has occurred since open.

kv_info

`kv_stats(STORE)` → hashref with `path`, `entries`, `dirty`, `format_version`, `created_at_secs`, `last_commit_secs`, `commit_count`, `stryke_version`.

kv_keys

`kv_keys(STORE [, PREFIX])` → sorted array of keys, optionally filtered by prefix. Use prefixes like `users:` / `log:` as namespaces.

```
for val $k (kv_keys($db, "users:")) { p kv_get($db, $k) }
```

kv_new

`kv_open(PATH)` → `KvStore` handle. Open or create a rkyv-archived KV store at `PATH`. Missing file → empty store, no I/O until first commit. Existing file → mmap + `check_archived_root` + deserialize into the in-memory mirror.

```
val $db = kv_open("/tmp/cache.rkyv");
kv_put($db, "users:alice", { name => "Alice", age => 30 });
kv_commit($db);
var $u = kv_get($db, "users:alice");
p $u->{name}                                # Alice
```

World's-first as scripting-lang stdlib: zero-copy archive KV with SQLite-shaped ergonomics. Reads are `mmap + validate + cast`; no parse, no allocate per row. Family: `kv_put`, `kv_get`, `kv_del`, `kv_exists`, `kv_keys`, `kv_scan`, `kv_len`, `kv_commit`, `kv_batch`, `kv_close`, `kv_stats`.

kv_put

`kv_put(STORE, KEY, VALUE)` → previous value or `undef`. Scalars round-trip lossless; arrayrefs become `Array`, hashrefs become `Hash`. Mutation is in-memory until `kv_commit` rewrites the archive atomically (tmp + rename).

kv_scan

`kv_scan(STORE, PREFIX)` → array of `[KEY, VALUE]` arrayref pairs (sorted by key). For lazy streaming over network connections see Phase 2 `kv_connect` / `WireFrame::StreamChunk`.

```
for val $row (kv_scan($db, "users:")) {
    var ($key, $val) = @$row;
    p "$key -> $val->{name}";
}
```

kwh_to_joules

`kwh_to_joules` — convert from `kwh` to `joules`. Category: `*astronomy / music / color / units*`.

```
p kwh_to_joules $value
```

l

`l` — operation `l`. Alias for `count`. Category: `*functional / iterator*`.

```
p l $x
```

`l1_coherence`

`l1_coherence` — operation `l1 coherence`. Category: **quantum mechanics deep**.

```
p l1_coherence $x
```

`l1_norm`

`l1_norm` — norm (vector length-like) of `l1`. Category: **misc**.

```
p l1_norm $x
```

`l2_norm`

`l2_norm` — norm (vector length-like) of `l2`. Category: **misc**.

```
p l2_norm $x
```

`l_parallel`

`l_parallel` — operation `l parallel`. Alias for `inductance_parallel`. Category: **misc**.

```
p l_parallel $x
```

`l_series`

`l_series` — operation `l series`. Alias for `inductance_series`. Category: **misc**.

```
p l_series $x
```

`label_propagation`

`label_propagation` — operation `label propagation`. Category: **networkx graph algorithms**.

```
p label_propagation $x
```

`label_propagation_step`

`label_propagation_step` — single-step update of `label propagation`. Category: **more extensions**.

```
p label_propagation_step $x
```

`lag_series`

`lag_series` — operation `lag series`. Category: **test runner**.

```
p lag_series $x
```

`lagrange`

`lagrange_interp` (aliases `lagrange`, `linterp`) performs Lagrange polynomial interpolation. Takes `xs`, `ys`, and a query point `x`.

```
p lagrange([0,1,2], [0,1,4], 1.5) # 2.25
```

`lagrange_l1`

`lagrange_l1` — operation `lagrange l1`. Category: **cosmology / gr / flrw**.

```
p lagrange_ll $x
```

lagrangian_dual_step

`lagrangian_dual_step` — single-step update of `lagrangian dual`. Category: *combinatorial optimization, scheduling*.

```
p lagrangian_dual_step $x
```

lagrangian_relax_step

`lagrangian_relax_step` — single-step update of `lagrangian relax`. Category: *combinatorial optimization, scheduling*.

```
p lagrangian_relax_step $x
```

lalr_lookahead_compute

`lalr_lookahead_compute` — operation `lalr lookahead compute`. Category: *compilers / parsing*.

```
p lalr_lookahead_compute $x
```

lambert_azimuthal_project

`lambert_azimuthal_project` — operation `lambert azimuthal project`. Category: *more extensions*.

```
p lambert_azimuthal_project $x
```

lambert_project

`lambert_project(lat, lon, lat0, lon0) → [x, y]` — Lambert Conformal Conic on a sphere with one standard parallel `lat0` and central meridian `lon0`. Degenerates to equirectangular when `lat0 = 0`.

```
p lambert_project(37, -122, 40, -100)
```

lambert_simple

`lambert_simple` — operation `lambert simple`. Category: *more extensions*.

```
p lambert_simple $x
```

lambert_w

`lambert_w` (aliases `lambertw`, `productlog`) computes the Lambert W function (principal branch): the inverse of $f(w) = w \cdot e^w$.

```
p lambertw(1)      # 0.5671 (Omega constant)
p lambertw(exp(1)) # 1.0
```

lambert_w0

`lambert_w0` — operation `lambert w0`. Category: *special functions extra*.

```
p lambert_w0 $x
```

lander_waterman_gaps

`lander_waterman_gaps` — operation `lander waterman gaps`. Category: *bioinformatics deep*.

```
p lander_waterman_gaps $x
```

language_iso_639_1

language_iso_639_1(name_or_code) → string — ISO 639-1 two-letter code.

```
p language_iso_639_1("English") # "en"
```

language_iso_639_2

language_iso_639_2(name_or_code) → string — ISO 639-2 three-letter code.

language_iso_639_3

language_iso_639_3(name_or_code) → string — ISO 639-3 three-letter code (covers more languages than 639-2).

language_name

language_name(code) → string — human-readable language name for an ISO code.

language_tag_match

language_tag_match(tag, pattern) → 0|1 — RFC 4647 basic filter match.

language_tag_subtags

language_tag_subtags(tag) → array — list of subtags split on '-'.
Note: This function is deprecated in favor of `language_tag_split`.

laplace_de_rham_step

laplace_de_rham_step — single-step update of `laplace de rham`. Category: *electrochemistry, batteries, fuel cells*.

```
p laplace_de_rham_step $x
```

laplace_pdf

laplace_pdf (alias `laplacepdf`) evaluates the Laplace distribution PDF at x with location mu and scale b.

```
p laplacepdf(0, 0, 1) # 0.5 (peak)
```

laplacian_kernel

laplacian_kernel() → matrix — 4-connected discrete Laplacian.

laplacian_kernel_value

laplacian_kernel_value — value of `laplacian kernel`. Category: *electrochemistry, batteries, fuel cells*.

```
p laplacian_kernel_value $x
```

larmor_frequency

larmor_frequency — operation `larmor frequency`. Category: *quantum mechanics deep*.

```
p larmor_frequency $x
```

last_arg

last_arg — operation last arg. Category: *functional primitives*.

```
p last_arg $x
```

last_clj

last_clj — operation last clj. Category: *algebraic match*.

```
p last_clj $x
```

last_elem

last_elem — operation last elem. Category: *list helpers*.

```
p last_elem $x
```

last_eq

last_eq — operation last eq. Category: *collection*.

```
p last_eq $x
```

last_index_of

last_index_of — operation last index of. Category: *collection*.

```
p last_index_of $x
```

last_of_month

last_of_month — operation last of month. Category: *misc*.

```
p last_of_month $x
```

last_word

last_word — operation last word. Category: *string*.

```
p last_word $x
```

lastc

lastc — operation lastc. Alias for last_clj. Category: *algebraic match*.

```
p lastc $x
```

lat_lng_to_tile_xyz

lat_lng_to_tile_xyz — convert from lat lng to tile xyz. Category: *excel/sheets + bond/loan financial*.

```
p lat_lng_to_tile_xyz $value
```

lat_lng_to_xy_lambert

lat_lng_to_xy_lambert — convert from lat lng to xy lambert. Category: *excel/sheets + bond/loan financial*.

```
p lat_lng_to_xy_lambert $value
```


lat_lng_to_xy_mercator

lat_lng_to_xy_mercator — convert from lat lng to xy mercator. Category: *excel/sheets + bond/loan financial*.

```
p lat_lng_to_xy_mercator $value
```

lat_lon_to_utm

lat_lon_to_utm(lat, lon) → [zone, easting_m, northing_m, hemi] — convert geographic to UTM coordinates (WGS84 ellipsoid).

```
p lat_lon_to_utm(40.7128, -74.006) # [18, 583959, 4507351, "N"]
```

latency_ms

latency_ms — operation latency ms. Category: *network / ip / cidr*.

```
p latency_ms $x
```

lattice_paths

lattice_paths(m, n) → int — monotonic lattice paths from (0,0) to (m,n) = $C(m+n, m)$.

```
p lattice_paths(3, 3) # 20
```

lax_friedrichs_flux

lax_friedrichs_flux — operation lax friedrichs flux. Category: *ode advanced*.

```
p lax_friedrichs_flux $x
```

lax_wendroff_flux

lax_wendroff_flux — operation lax wendroff flux. Category: *ode advanced*.

```
p lax_wendroff_flux $x
```

lazy_caterer

lazy_caterer — operation lazy caterer. Category: *misc*.

```
p lazy_caterer $x
```

lb_to_kg

lb_to_kg — convert from lb to kg. Category: *astronomy / music / color / units*.

```
p lb_to_kg $value
```

lbf_to_newtons

lbf_to_newtons — convert from lbf to newtons. Category: *file stat / path*.

```
p lbf_to_newtons $value
```

lbound

`lbound` — operation `lbound`. Alias for `lower_bound`. Category: **extended stdlib**.

```
p lbound $x
```

lbs_to_kg

`lbs_to_kg` — convert from `lbs` to `kg`. Category: **unit conversions**.

```
p lbs_to_kg $value
```

lc_frequency

`lc_frequency` — operation `lc frequency`. Category: **em / optics / relativity**.

```
p lc_frequency $x
```

lc_lines

`lc_lines` — operation `lc lines`. Alias for `line_count`. Category: **trivial string ops**.

```
p lc_lines $x
```

lc_omega

`lc_omega` — operation `lc omega`. Category: **em / optics / relativity**.

```
p lc_omega $x
```

lc_resonant

`lc_resonant` — operation `lc resonant`. Category: **misc**.

```
p lc_resonant $x
```

lcdm_eos

`lcdm_eos` — operation `lcdm eos`. Category: **cosmology / gr / flrw**.

```
p lcdm_eos $x
```

lcg_next_u32

`lcg_next_u32` — operation `lcg next u32`. Category: **misc**.

```
p lcg_next_u32 $x
```

lcg_numrec_step

`lcg_numrec_step` — single-step update of `lcg numrec`. Category: **cryptography deep**.

```
p lcg_numrec_step $x
```

lch_to_rgb

`lch_to_rgb([L, C, H])` → `[r, g, b]` — inverse of `rgb_to_lch`. Output is clamped sRGB 0-255.

lcm

`lcm` — operation `lcm`. Category: *trivial numeric / predicate builtins*.

```
p lcm $x
```

lcontract

`length_contraction($proper_length, $velocity)` (alias `lcontract`) — contracted length $L = L_0 \times \sqrt{(1-v^2/c^2)}$.

```
p lcontract(1, 0.99 * 3e8) # ~0.14 m (1m at 99% c)
```

lcount

`lcount` — operation `lcount`. Alias for `count_lines`. Category: *extended stdlib*.

```
p lcount $x
```

lcp

`lcp` — operation `lcp`. Alias for `longest_common_prefix`. Category: *string*.

```
p lcp $x
```

lcp_array

`lcp_array` — operation `lcp array`. Category: *misc*.

```
p lcp_array $x
```

lcs_length

`lcs_length` — operation `lcs length`. Category: *bioinformatics deep*.

```
p lcs_length $x
```

lcseq

`lcseq` — operation `lcseq`. Alias for `longest_common_subsequence`. Category: *extended stdlib*.

```
p lcseq $x
```

lcsb

`lcsb` — operation `lcsb`. Alias for `longest_common_substring`. Category: *extended stdlib*.

```
p lcsb $x
```

ld_d

`ld_d` — operation `ld d`. Category: *bioinformatics deep*.

```
p ld_d $x
```

ld_r_squared

`ld_r_squared` — operation `ld r squared`. Category: *bioinformatics deep*.

```
p ld_r_squared $x
```

ldec

`ldec` — operation `ldec`. Alias for `longest_decreasing`. Category: **extended stdlib**.

```
p ldec $x
```

le_chatelier_dir

`le_chatelier_dir` — operation `le_chatelier_dir`. Category: **chemistry**.

```
p le_chatelier_dir $x
```

leading_zeros

`leading_zeros` — operation `leading_zeros`. Category: **base conversion**.

```
p leading_zeros $x
```

leaky_bucket

`leaky_bucket(CAPACITY, DRAIN_PER_SEC)` — leaky-bucket rate limiter (drains over time, fills as requests arrive). `rl_try_take(cost)` succeeds if the fill plus `cost` doesn't overflow.

leaky_relu

`leaky_relu` (alias `lrelu`) applies Leaky ReLU: x if $x \geq 0$, αx otherwise (default $\alpha=0.01$).

```
p lrelu(5)      # 5
p lrelu(-5)     # -0.05
p lrelu(-5, 0.1) # -0.5
```

leap_year_gregorian

`leap_year_gregorian` — operation `leap_year_gregorian`. Category: **calendrical algorithms**.

```
p leap_year_gregorian $x
```

leapfrog_step

`leapfrog_step` — single-step update of `leapfrog`. Category: **ode advanced**.

```
p leapfrog_step $x
```

least_common

`least_common` — operation `least_common`. Category: **list helpers**.

```
p least_common $x
```

leet

`leet` — operation `leet`. Alias for `leetspeak`. Category: **string processing extras**.

```
p leet $x
```

leetspeak

`leetspeak` — operation `leetspeak`. Category: **string processing extras**.

```
p leetspeak $x
```

left_str

left_str — operation `left str`. Category: **string**.

```
p left_str $x
```

lehmer_mean

lehmer_mean — arithmetic mean of `lehmer`. Category: **misc**.

```
p lehmer_mean @xs
```

lemmatize_lemmy

lemmatize_lemmy — operation `lemmatize lemmy`. Category: **computational linguistics**.

```
p lemmatize_lemmy $x
```

lemmatize_wordnet

lemmatize_wordnet — operation `lemmatize wordnet`. Category: **computational linguistics**.

```
p lemmatize_wordnet $x
```

length_each

length_each — operation `length each`. Category: **trivial numeric helpers**.

```
p length_each $x
```

lens_focal_length

lens_focal_length — operation `lens focal length`. Category: **misc**.

```
p lens_focal_length $x
```

lens_magnification

lens_magnification — operation `lens magnification`. Category: **em / optics / relativity**.

```
p lens_magnification $x
```

lens_power

lens_power — operation `lens power`. Category: **physics formulas**.

```
p lens_power $x
```

lense_thirring_omega

lense_thirring_omega — operation `lense thirring omega`. Category: **cosmology / gr / flrw**.

```
p lense_thirring_omega $x
```

lensing_convergence

`lensing_convergence` — operation `lensing convergence`. Category: *cosmology / gr / flrw*.

```
p lensing_convergence $x
```

lensmaker

`lensmaker` — operation `lensmaker`. Category: *em / optics / relativity*.

```
p lensmaker $x
```

leontief_input

`leontief_input` — operation `leontief input`. Category: *economics + game theory*.

```
p leontief_input $x
```

leontief_output

`leontief_output` — operation `leontief output`. Category: *economics + game theory*.

```
p leontief_output $x
```

lerner_index

`lerner_index` — index of `lerner`. Category: *economics + game theory*.

```
p lerner_index $x
```

lerp

`lerp($x, $y, $t)` — linear interpolation between a and b. When t=0 returns a, t=1 returns b, t=0.5 returns midpoint.

```
p lerp(0, 100, 0.5) # 50
p lerp(10, 20, 0.25) # 12.5
# Animation: smoothly transition between values
for val $t (0:10) { p lerp($start, $end, $t/10) }
```

lerp_color

`lerp_color` — operation `lerp color`. Alias for `color_blend_t`. Category: *misc*.

```
p lerp_color $x
```

leslie_step

`leslie_step` — single-step update of `leslie`. Category: *biology / ecology*.

```
p leslie_step $x
```

less

`LIST |> pager / pager LIST` — pipe each element (one per line) into the user's `$PAGER` (default `less -R`; falls back to `more`, then plain stdout). Bypasses the pager when stdout isn't a TTY so pipelines like `stryke '...' |> pager | grep` still compose.

Blocks until the user quits the pager; returns `undef`.

Aliases: `pager`, `pg`, `less`.

```
# browse every callable spelling interactively
keys %all |> sort |> pager

# filter the reference for parallel ops
keys %stryke::builtins |> grep { $stryke::builtins{$_} eq "parallel" } |> pager

# whole file, one screen at a time
slurp("README.md") |> pager
```

letter_frequency

`letter_frequency` — operation `letter frequency`. Category: **cryptography deep**.

```
p letter_frequency $x
```

letters_lc

`letters_lc` extracts only lowercase letters from a string.

```
p join "", letters_lc("Hello World 123") # elloorld
```

letters_uc

`letters_uc` extracts only uppercase letters from a string.

```
p join "", letters_uc("Hello World 123") # HW
```

lev

`lev` — operation `lev`. Alias for `levenshtein`. Category: **extended stdlib**.

```
p lev $x
```

level_k_step

`level_k_step` — single-step update of `level k`. Category: **climate, fluids, atmospheric**.

```
p level_k_step $x
```

level_premium

`level_premium` — operation `level premium`. Category: **actuarial science**.

```
p level_premium $x
```

levenshtein

`levenshtein` — operation `levenshtein`. Category: **extended stdlib**.

```
p levenshtein $x
```

levenshtein_dist

`levenshtein_dist` — operation `levenshtein dist`. Category: **postgres sql strings, json, aggregates**.

```
p levenshtein_dist $x
```

levenshtein_edit_path

`levenshtein_edit_path(a, b) → array` — edit operations: "match:x->x", "sub:x->y", "ins:y", "del:x".

```
p levenshtein_edit_path("kitten", "sitting")
```

levenshtein_normalized

`levenshtein_normalized` — operation `levenshtein normalized`. Category: *iterator + string-distance extras*.

```
p levenshtein_normalized $x
```

leverage_index

`leverage_index` — index of `leverage`. Category: *astronomy / astrometry*.

```
p leverage_index $x
```

levi_civita_four

`levi_civita_four` — operation `levi civita four`. Category: *electrochemistry, batteries, fuel cells*.

```
p levi_civita_four $x
```

levi_civita_three

`levi_civita_three` — operation `levi civita three`. Category: *electrochemistry, batteries, fuel cells*.

```
p levi_civita_three $x
```

levich_current_rde

`levich_current_rde` — operation `levich current rde`. Category: *electrochemistry, batteries, fuel cells*.

```
p levich_current_rde $x
```

levin_t_transform

`levin_t_transform` — transform of `levin t`. Category: *more extensions*.

```
p levin_t_transform $x
```

levy_pdf

`levy_pdf` — operation `levy pdf`. Category: *more extensions (2)*.

```
p levy_pdf $x
```

lewis_dot_count

`lewis_dot_count` — count of `lewis dot`. Category: *chemistry & biochemistry*.

```
p lewis_dot_count $x
```

lex_keyword_classify

`lex_keyword_classify` — operation `lex keyword classify`. Category: *compilers / parsing*.

```
p lex_keyword_classify $x
```


lexicalized_parse

`lexicalized_parse` — operation `lexicalized_parse`. Category: *computational linguistics*.

```
p lexicalized_parse $x
```

lfilter_zi_init

`lfilter_zi_init` — initialise of `lfilter_zi`. Category: *economics + game theory*.

```
p lfilter_zi_init $x
```

lfrm

`lfrm` — operation `lfrm`. Alias for `lines_from`. Category: *go/general functional utilities*.

```
p lfrm $x
```

lfsr_galois_step

`lfsr_galois_step` — single-step update of `lfsr_galois`. Category: *cryptography deep*.

```
p lfsr_galois_step $x
```

li

$\text{li}(x) \rightarrow \text{number}$ — logarithmic integral $\text{li}(x) = \text{Ei}(\ln x)$. Approximates the prime-counting function $\pi(x)$.

```
p li(100)
```

liang_barsky_clip

`liang_barsky_clip(p1, p2, [xmin, ymin, xmax, ymax]) → [p1', p2']` — line-segment clip against rectangle. Empty result if segment is outside.

```
p liang_barsky_clip([0.5,0.5], [0.8,0.8], [0,0,1,1])
```

lie_derivative_scalar_step

`lie_derivative_scalar_step` — single-step update of `lie_derivative_scalar`. Category: *electrochemistry, batteries, fuel cells*.

```
p lie_derivative_scalar_step $x
```

lie_derivative_vector_step

`lie_derivative_vector_step` — single-step update of `lie_derivative_vector`. Category: *electrochemistry, batteries, fuel cells*.

```
p lie_derivative_vector_step $x
```

lie_split

`lie_split` — operation `lie_split`. Category: *ode advanced*.

```
p lie_split $x
```

life_expectancy_e0

`life_expectancy_e0` — operation `life expectancy e0`. Category: **actuarial science**.

```
p life_expectancy_e0 $x
```

lift_force

`lift_force(Cl, density=1.225, velocity, area) → number` — $\frac{1}{2} \cdot Cl \cdot \rho \cdot v^2 \cdot A$.

light_deflection_angle

`light_deflection_angle` — operation `light deflection angle`. Category: **cosmology / gr / flrw**.

```
p light_deflection_angle $x
```

light_year

`light_year` (alias `ly`) — $ly \approx 9.461 \times 10^{15}$ m. Distance light travels in one year.

```
p ly          # 9.4607304725808e15 m
p ly / au     # ~63241 AU
```

likelihood_ratio_test

`likelihood_ratio_test` — statistical test of `likelihood ratio`. Category: **econometrics**.

```
p likelihood_ratio_test $x
```

limiting_current_density

`limiting_current_density` — operation `limiting current density`. Category: **electrochemistry, batteries, fuel cells**.

```
p limiting_current_density $x
```

lin_bairstow_step

`lin_bairstow_step` — single-step update of `lin bairstow`. Category: **misc**.

```
p lin_bairstow_step $x
```

lin_kernighan_step

`lin_kernighan_step` — single-step update of `lin kernighan`. Category: **combinatorial optimization, scheduling**.

```
p lin_kernighan_step $x
```

linc

`linc` — operation `linc`. Alias for `longest_increasing`. Category: **extended stdlib**.

```
p linc $x
```

line_count

`line_count` — count of `line`. Category: **trivial string ops**.

```
p line_count $x
```

line_count_simple

`line_count_simple` — operation `line count simple`. Alias for `string_lines_count`. Category: **misc**.

```
p line_count_simple $x
```

line_distance_point

`line_distance_point` — operation `line distance point`. Category: **complex / geom / color / trig**.

```
p line_distance_point $x
```

line_intersect

`line_intersect` — operation `line intersect`. Category: **complex / geom / color / trig**.

```
p line_intersect $x
```

line_intersection

`line_intersection($x1, $y1, $x2, $y2, $x3, $y3, $x4, $y4)` — finds the intersection point of two lines defined by points (x1,y1)-(x2,y2) and (x3,y3)-(x4,y4). Returns [\$x, \$y] or undef if parallel.

```
val $pt = line_intersection(0, 0, 1, 1, 0, 1, 1, 0)
p @$pt      # (0.5, 0.5)
```

line_segment_intersect

`line_segment_intersect` — operation `line segment intersect`. Category: **complex / geom / color / trig**.

```
p line_segment_intersect $x
```

linear_entropy

`linear_entropy` — operation `linear entropy`. Category: **quantum mechanics deep**.

```
p linear_entropy $x
```

linear_regression

`linear_regression` — operation `linear regression`. Category: **math / numeric extras**.

```
p linear_regression $x
```

linear_search

`linear_search` — operation `linear search`. Category: **collection**.

```
p linear_search $x
```

linearized_pb_step

`linearized_pb_step` — single-step update of `linearized pb`. Category: **electrochemistry, batteries, fuel cells**.

```
p linearized_pb_step $x
```

lineintr

`lineintr` — operation `lineintr`. Alias for `line_intersection`. Category: **geometry (extended)**.

```
p lineintr $x
```

lines_from

`lines_from` — operation `lines from`. Category: **go/general functional utilities**.

```
p lines_from $x
```

lineweaver_burk

`lineweaver_burk` — operation `lineweaver burk`. Category: **misc**.

```
p lineweaver_burk $x
```

linf_norm

`linf_norm` — norm (vector length-like) of `linf`. Category: **misc**.

```
p linf_norm $x
```

linkedMapOf

`linkedMapOf` — operation `linkedMapOf`. Alias for `mapOf`. Category: **kotlin collections**.

```
p linkedMapOf $x
```

linkedSetOf

`linkedSetOf` — operation `linkedSetOf`. Alias for `setOf`. Category: **kotlin collections**.

```
p linkedSetOf $x
```

linking_number_two

`linking_number_two` — operation `linking number two`. Category: **econometrics**.

```
p linking_number_two $x
```

linreg

`linreg` — operation `linreg`. Alias for `linear_regression`. Category: **math / numeric extras**.

```
p linreg $x
```

linspace

`linspace` — operation `linspace`. Category: **matrix / linear algebra**.

```
p linspace $x
```

liouville

`liouville` — operation `liouville`. Category: **cryptanalysis & number theory deep**.

```
p liouville $x
```

liquefaction_potential_index

`liquefaction_potential_index` — index of `liquefaction potential`. Category: **geology, seismology, mineralogy**.

```
p liquefaction_potential_index $x
```

listOf

`listOf` — operation `listOf`. Category: **kotlin collections**.

```
p listOf $x
```

listOfNotNull

`listOfNotNull` — operation `listOfNotNull`. Category: **kotlin collections**.

```
p listOfNotNull $x
```

list_coloring_step

`list_coloring_step` — list helper `coloring step`. Category: **combinatorial optimization, scheduling**.

```
p list_coloring_step $x
```

list_compact

`list_compact` — list helper `compact`. Category: **list helpers**.

```
p list_compact $x
```

list_count

`list_count` (alias `list_size`) returns the total number of elements after flattening a nested list structure. Unlike `count` which returns the top-level element count, `list_count` recursively descends into array references and counts only leaf values. This is useful when you have a list of lists and need to know the total number of individual items rather than the number of sublists.

```
val @nested = ([1, 2], [3, 4, 5], [6])
p list_count @nested      # 6
var @deep = ([1, [2, 3]], [4])
p list_size @deep         # 4
p list_count 1, 2, 3      # 3 (flat list works too)
```

list_difference

`list_difference` — list helper `difference`. Alias for `array_difference`. Category: **collection**.

```
p list_difference $x
```

list_eq

`list_eq` — list helper `eq`. Category: **list helpers**.

```
p list_eq $x
```

list_flatten_deep

`list_flatten_deep` — list helper `flatten deep`. Category: **list helpers**.

```
p list_flatten_deep $x
```

list_intersection

`list_intersection` — list helper `intersection`. Alias for `array_intersection`. Category: *collection*.

```
p list_intersection $x
```

list_reverse

`list_reverse` — list helper `reverse`. Alias for `reverse_list`. Category: *file stat / path*.

```
p list_reverse $x
```

list_scheduling_step

`list_scheduling_step` — list helper `scheduling step`. Category: *combinatorial optimization, scheduling*.

```
p list_scheduling_step $x
```

list_union

`list_union` — list helper `union`. Alias for `array_union`. Category: *collection*.

```
p list_union $x
```

liters_to_gallons

`liters_to_gallons` — convert from `liters` to `gallons`. Category: *file stat / path*.

```
p liters_to_gallons $value
```

liters_to_ml

`liters_to_ml` — convert from `liters` to `ml`. Category: *file stat / path*.

```
p liters_to_ml $value
```

lithium_ion_diffusivity

`lithium_ion_diffusivity` — operation `lithium ion diffusivity`. Category: *electrochemistry, batteries, fuel cells*.

```
p lithium_ion_diffusivity $x
```

lix

`lix` — operation `lix`. Category: *misc*.

```
p lix $x
```

ljt

`ljt` — operation `ljt`. Alias for `ljust_text`. Category: *extended stdlib*.

```
p lj_t $x
```

ljung_box_q

`ljung_box_q` — operation `ljung box q`. Category: *statsmodels*.

```
p ljung_box_q $x
```

ljust

`ljust` — operation `ljust`. Category: **string helpers**.

```
p ljust $x
```

ljust_text

`ljust_text` — operation `ljust text`. Category: **extended stdlib**.

```
p ljust_text $x
```

lkpa

`lkpa` — operation `lkpa`. Alias for `lookup_assoc`. Category: **haskell list functions**.

```
p lkpa $x
```

ll1_first_set

`ll1_first_set` — operation `ll1 first set`. Category: **compilers / parsing**.

```
p ll1_first_set $x
```

ll1_follow_set

`ll1_follow_set` — operation `ll1 follow set`. Category: **compilers / parsing**.

```
p ll1_follow_set $x
```

ll1_predict_table

`ll1_predict_table` — lookup table of `ll1 predict`. Category: **compilers / parsing**.

```
p ll1_predict_table $x
```

lll_2x2_step

`lll_2x2_step` — single-step update of `lll 2x2`. Category: **cryptanalysis & number theory deep**.

```
p lll_2x2_step $x
```

lm

`lm_fit` (alias `lm`) — simple linear regression. Returns hash with intercept, slope, `r_squared`, residuals, fitted. Like R's `lm()`.

```
val %m = %{lm([1,2,3,4,5], [2,4,5,4,5])}
p $m{slope}      # slope
p $m{r_squared}  # R2
```

lm_step

`lm_step` — single-step update of `lm`. Category: **more extensions (2)**.

```
p lm_step $x
```

ln10

`ln10` — operation `ln10`. Category: **math / physics constants**.

```
p ln10 $x
```

ln2

`ln2` — operation `ln2`. Category: **math / physics constants**.

```
p ln2 $x
```

lnormpdf

`lognormal_pdf` (alias `lnormpdf`) evaluates the log-normal distribution PDF at `x` with parameters `mu` and `sigma`.

```
p lnormpdf(1.0, 0, 1) # LogN(0,1) at x=1
```

lns

`lns` — operation `lns`. Alias for `look_and_say`. Category: **math / numeric extras**.

```
p lns $x
```

load_avg

`load_avg` — system load averages as a 3-element arrayref [`1min`, `5min`, `15min`]. Uses `getloadavg()` on unix. Returns `undef` on unsupported platforms.

```
val $la = load_avg
p "1m=$la->[0] 5m=$la->[1] 15m=$la->[2]"
load_avg |> spark |> p # sparkline of load
```

load centrality

`load centrality` — operation `load centrality`. Category: **networkx graph algorithms**.

```
p load centrality $x
```

loan_balance

`loan_balance` — operation `loan balance`. Category: **misc**.

```
p loan_balance $x
```

loan_interest_total

`loan_interest_total(principal, rate, n_periods) → number` — total interest paid over the life of an amortising loan.

```
p loan_interest_total(100000, 0.05/12, 360)
```

loan_payment

`loan_payment(principal, rate_per_period, n_periods) → number` — amortising loan payment formula:
$$P \cdot r \cdot (1+r)^n / ((1+r)^n - 1).$$

```
p loan_payment(100000, 0.05/12, 360) # 536.82
```


loan_payment_pmt

`loan_payment_pmt` — operation `loan payment pmt`. Category: **misc**.

```
p loan_payment_pmt $x
```

loan_remaining

`loan_remaining(principal, rate, n_total, n_paid) → number` — remaining balance after `n_paid` payments under fixed-rate amortisation.

```
p loan_remaining(100000, 0.05/12, 360, 60)
```

lobatto_iiia_2

`lobatto_iiia_2` — operation `lobatto iiia 2`. Category: **ode advanced**.

```
p lobatto_iiia_2 $x
```

lobatto_iiic_3

`lobatto_iiic_3` — operation `lobatto iiic 3`. Category: **ode advanced**.

```
p lobatto_iiic_3 $x
```

local_efficiency

`local_efficiency` — operation `local efficiency`. Category: **more extensions**.

```
p local_efficiency $x
```

local_search_swap_step

`local_search_swap_step` — single-step update of `local search swap`. Category: **combinatorial optimization, scheduling**.

```
p local_search_swap_step $x
```

locale_calendar

`locale_calendar(locale) → string` — preferred calendar system (gregorian, persian, hebrew, ...).

locale_canonical

`locale_canonical(name) → string` — canonicalised CLDR locale identifier.

locale_collation

`locale_collation(locale) → string` — collation tag (e.g. "standard", "phonebk").

locale_currency

`locale_currency(locale) → string` — ISO 4217 currency code for region.

locale_date_format

`locale_date_format(locale) → string` — date pattern (e.g. "MM/dd/yyyy").

locale_decimal_separator

`locale_decimal_separator(locale)` → `string` — decimal point character ("," or ".").

locale_first_day_of_week

`locale_first_day_of_week(locale)` → `int` — 0=Sunday, 1=Monday.

locale_format

`locale_format(template, locale, vars)` → `string` — locale-aware ICU-like message format.

locale_group_separator

`locale_group_separator(locale)` → `string` — thousands separator.

locale_language

`locale_language(locale)` → `string` — language portion.

locale_likely_subtags

`locale_likely_subtags(locale)` → `string` — fill in likely script/region per CLDR likely-subtags table.

locale_measurement_system

`locale_measurement_system(locale)` → `string` — "metric" or "US".

locale_minimize

`locale_minimize` — locale / CLDR helper `minimize`. Category: *extras*.

```
p locale_minimize $x
```

locale_number_format

`locale_number_format(n, locale)` → `string` — render number with locale separators.

locale_parse

`locale_parse(name)` → `hash` — parse POSIX/CLDR locale id into language/region/script/variant.

locale_region

`locale_region(locale)` → `string` — region portion.

locale_script

`locale_script(locale)` → `string` — script portion.

locale_time_format

`locale_time_format(locale)` → `string` — time pattern.

locale_variant

`locale_variant(locale)` → `string` — variant portion.

loess

`lowess` (alias `loess`) — locally-weighted scatterplot smoothing (LOWESS/LOESS). Returns smoothed Y values with tricube weighting.

```
val @smooth = @{lowess([1,2,3,4,5], [2,1,4,3,5])}
```

log10

`log10` — operation `log10`. Category: **trivial numeric / predicate builtins**.

```
p log10 $x
```

log1p_exp

`log1p_exp` — operation `log1p exp`. Category: **misc**.

```
p log1p_exp $x
```

log2

`log2` — operation `log2`. Category: **trivial numeric / predicate builtins**.

```
p log2 $x
```

log_base

`log_base` — logarithm / logging helper `base`. Category: **math functions**.

```
p log_base $x
```

log_gamma_simple

`log_gamma_simple` — logarithm / logging helper `gamma simple`. Category: **more extensions (2)**.

```
p log_gamma_simple $x
```

log_law_wind_profile

`log_law_wind_profile` — logarithm / logging helper `law wind profile`. Category: **climate, fluids, atmospheric**.

```
p log_law_wind_profile $x
```

log_loss

`log_loss` — logarithm / logging helper `loss`. Category: **ml extensions**.

```
p log_loss $x
```

log_returns

`log_returns(prices)` → `array` — natural log of consecutive ratios: $\ln(p[i]/p[i-1])$. Additive across time.

```
p log_returns([100, 105, 102])
```

log_sigmoid

`log_sigmoid` — logarithm / logging helper `sigmoid`. Category: **misc**.

```
p log_sigmoid $x
```

log_softmax

`log_softmax(xs)` → array — $xs - \log(\sum \exp(xs))$ in a numerically stable way.

log_sum_exp

`log_sum_exp` — logarithm / logging helper `sum exp`. Category: **misc**.

```
p log_sum_exp $x
```

log_with_base

`log_with_base` — logarithm / logging helper `with base`. Category: **extended stdlib**.

```
p log_with_base $x
```

logb

`logb` — operation `logb`. Alias for `log_with_base`. Category: **extended stdlib**.

```
p logb $x
```

loggamma

`loggamma` — operation `loggamma`. Category: **numpy + scipy.special**.

```
p loggamma $x
```

logistic_growth_analytic

`logistic_growth_analytic` — operation `logistic growth analytic`. Category: **biology / ecology**.

```
p logistic_growth_analytic $x
```

logistic_growth_step

`logistic_growth_step` — single-step update of `logistic growth`. Category: **biology / ecology**.

```
p logistic_growth_step $x
```

logistic_loss

`logistic_loss` — operation `logistic loss`. Category: **ml extensions**.

```
p logistic_loss $x
```

logit_log_likelihood

`logit_log_likelihood` — operation `logit log likelihood`. Category: **econometrics**.

```
p logit_log_likelihood $x
```

lomax_pdf

lomax_pdf — operation lomax pdf. Category: *more extensions (2)*.

```
p lomax_pdf $x
```

lombscargle_freq

lombscargle_freq — operation lombscargle freq. Category: *economics + game theory*.

```
p lombscargle_freq $x
```

longest

longest — operation longest. Category: *collection*.

```
p longest $x
```

longest_common_prefix

longest_common_prefix — operation longest common prefix. Category: *string*.

```
p longest_common_prefix $x
```

longest_common_subsequence

longest_common_subsequence — operation longest common subsequence. Category: *extended stdlib*.

```
p longest_common_subsequence $x
```

longest_common_substring

longest_common_substring — operation longest common substring. Category: *extended stdlib*.

```
p longest_common_substring $x
```

longest_decreasing

longest_decreasing — operation longest decreasing. Category: *extended stdlib*.

```
p longest_decreasing $x
```

longest_increasing

longest_increasing — operation longest increasing. Category: *extended stdlib*.

```
p longest_increasing $x
```

longest_run

longest_run — operation longest run. Category: *extended stdlib*.

```
p longest_run $x
```

look_and_say

look_and_say — operation look and say. Category: *math / numeric extras*.

```
p look_and_say $x
```

lookahead_n

`lookahead_n` — operation `lookahead n`. Category: **iterator + string-distance extras**.

```
p lookahead_n $x
```

lookback_call

`lookback_call` — operation `lookback call`. Category: **financial pricing models**.

```
p lookback_call $x
```

lookback_time

`lookback_time` — operation `lookback time`. Category: **cosmology / gr / flrw**.

```
p lookback_time $x
```

looks_like_number

`looks_like_number` — operation `looks like number`. Category: **list / aggregate**.

```
p looks_like_number $x
```

lookup_assoc

`lookup_assoc` — operation `lookup assoc`. Category: **haskell list functions**.

```
p lookup_assoc $x
```

loose_hash

`loose_hash(s) → int` — simple multiplicative hash $h = h \cdot 31 + \text{byte}$ (Java's `String.hashCode` shape).

```
p loose_hash("hello")
```

lorentz

`lorentz_factor($velocity)` (alias `lorentz`) — relativistic gamma factor $\gamma = 1/\sqrt{1-v^2/c^2}$. Returns infinity at $v \geq c$.

```
p lorentz(0)           # 1 (at rest)
p lorentz(0.9 * 3e8)    # ~2.29 (at 90% speed of light)
```

lorentz_factor_v

`lorentz_factor_v` — operation `lorentz factor v`. Category: **misc**.

```
p lorentz_factor_v $x
```

lorentz_force_mag

`lorentz_force_mag` — operation `lorentz force mag`. Category: **em / optics / relativity**.

```
p lorentz_force_mag $x
```

lorentz_gamma

`lorentz_gamma` — operation `lorentz gamma`. Category: **em / optics / relativity**.

```
p lorentz_gamma $x
```

lorenz

`lorenz_curve(@data)` (alias `lorenz`) — computes the Lorenz curve as an arrayref of [cumulative_share_of_population, cumulative_share_of_wealth] pairs. Used to visualize inequality. The diagonal represents perfect equality.

```
val $curve = lorenz(10, 20, 30, 40)
for val $pt (@$curve) {
  p "$pt->[0], $pt->[1]"
}
```

lose_lose

`lose_lose` — operation `lose lose`. Category: **b81-misc-utility**.

```
p lose_lose $x
```

loss_frequency_poisson

`loss_frequency_poisson` — operation `loss frequency poisson`. Category: **actuarial science**.

```
p loss_frequency_poisson $x
```

loss_severity_lognormal

`loss_severity_lognormal` — operation `loss severity lognormal`. Category: **actuarial science**.

```
p loss_severity_lognormal $x
```

lotka_volterra_step

`lotka_volterra_step` — single-step update of `lotka volterra`. Category: **misc**.

```
p lotka_volterra_step $x
```

louvain_gain

`louvain_gain` — operation `louvain gain`. Category: **networkx graph algorithms**.

```
p louvain_gain $x
```

lower_bound

`lower_bound` — operation `lower bound`. Category: **extended stdlib**.

```
p lower_bound $x
```

lower_triangular_q

`lower_triangular_q` — operation `lower triangular q`. Category: **more extensions (2)**.

```
p lower_triangular_q $x
```

lowercase

`lowercase` — operation `lowercase`. Category: **string**.

```
p lowercase $x
```

lowpass_filter

`lowpass_filter(\@signal, $cutoff)` (alias `lpf`) — applies a simple low-pass filter to remove high-frequency components. Cutoff is normalized (0-1). Uses exponential moving average.

```
val @noisy = map { sin(_/10) + rand(0.2) } 1:100  
var $smooth = lpf(\@noisy, 0.1)
```

lp_norm

`lp_norm` — Linear Programming solver `norm`. Category: **misc**.

```
p lp_norm $x
```

lp_relax_round

`lp_relax_round` — Linear Programming solver `relax round`. Category: **combinatorial optimization, scheduling**.

```
p lp_relax_round $x
```

lp_simplex_max

`lp_simplex_max(c, A, b) → number` — maximise $c \cdot x$ subject to $A \cdot x \leq b$, $x \geq 0$ via revised simplex.

```
p lp_simplex_max([1,1], [[1,1]], [10]) # 10
```

lp_simplex_min

`lp_simplex_min(c, A, b) → number` — minimise $c \cdot x$ (negates `c` and re-uses `simplex_max`).

lpad

`lpad` — operation `lpad`. Alias for `pad_left`. Category: **trivial string ops**.

```
p lpad $x
```

lplanck

`planck_length` (alias `lplanck`) — $IP \approx 1.616 \times 10^{-35}$ m. Smallest meaningful length in quantum mechanics.

```
p lplanck # 1.616255e-35 m
```

lpt

`lpt` — operation `lpt`. Alias for `lpt_schedule`. Category: **misc**.

```
p lpt $x
```

lpt_schedule

`lpt_schedule` — operation `lpt schedule`. Category: **misc**.

```
p lpt_schedule $x
```


lpush

`lpush` — operation `lpush`. Category: **redis-flavour primitives**.

```
p lpush $x
```

lqg_step

`lqg_step` — single-step update of `lqg`. Category: **robotics & control**.

```
p lqg_step $x
```

lqr_gain_continuous

`lqr_gain_continuous` — operation `lqr gain continuous`. Category: **robotics & control**.

```
p lqr_gain_continuous $x
```

lqr_gain_discrete

`lqr_gain_discrete` — operation `lqr gain discrete`. Category: **robotics & control**.

```
p lqr_gain_discrete $x
```

lr0_items_step

`lr0_items_step` — single-step update of `lr0 items`. Category: **compilers / parsing**.

```
p lr0_items_step $x
```

lr1_canonical_collection

`lr1_canonical_collection` — operation `lr1 canonical collection`. Category: **compilers / parsing**.

```
p lr1_canonical_collection $x
```

lrange

`lrange` — operation `lrange`. Category: **redis-flavour primitives**.

```
p lrange $x
```

lrem

`lrem` — operation `lrem`. Category: **redis-flavour primitives**.

```
p lrem $x
```

lru_new

`lru_new` — constructor of `lru`. Category: **data structure helpers**.

```
p lru_new $x
```

lrun

`lrun` — operation `lrun`. Alias for `longest_run`. Category: **extended stdlib**.

```
p lrun $x
```

ls

`ls / ls(DIR)` — long directory listing in the style of `ls -l`. With no arguments, lists the interpreter working directory (same base `cd` uses; not `$_` and not necessarily the OS `getcwd`). With one or more directory paths, lists each directory; if more than one path is given, each block is prefixed with `DIR:` like GNU `ls`. Returns a single multiline string (suitable for `p` or `spurt`). On Unix, includes a `total` line (1024-byte block units), mode bits, link count, numeric uid/gid, size, mtime, and name; symbolic links show `-> target`. On Windows, lines use file attributes, size, and a simplified timestamp.

```
p ls()           # logical cwd (see `cd`)
p ls("/tmp")
```

lse

`lse` — operation `lse`. Alias for `log_sum_exp`. Category: `*misc*`.

```
p lse $x
```

lsearch

`lsearch` — operation `lsearch`. Alias for `linear_search`. Category: `*collection*`.

```
p lsearch $x
```

lsp_completion_words

`lsp_completion_words` (alias `lsp_words`) — emits every name LSP tab-complete should know about, sorted ASCII. Three sources merged: (1) every callable bare-name spelling from `%all` (~10.6k names, primaries + aliases), (2) sigil-prefixed live bindings (`%a`, `%all`, `%b`, `%parameters`, `@ARGV`, `@INC`, `@fpath`, `@path`, `$_`, `$stryke::VERSION`, ...), (3) `CORE:::/main::` qualified spellings of every callable / unqualified binding, plus stryke language keywords (`fn`, `match`, `class`, `struct`, `enum`, `varsync`, `frozen`, ...). Drives the canonical `strykelang/lsp_completion_words.txt` snapshot consumed by both the static analyzer (linter) and LSP completion provider.

```
lsp_words |> ep           # browse everything
p len(lsp_words)         # ~19,000 entries
# Regenerate the on-disk snapshot:
# stryke -e 'lsp_words |> ep' > strykelang/lsp_completion_words.txt
```

lst_to_solar

`lst_to_solar` — convert from `lst` to `solar`. Category: `*more extensions*`.

```
p lst_to_solar $value
```

lstat

`lstat` — operation `lstat`. Category: `*filesystem*`.

```
p lstat $x
```

lucas

`lucas` — operation `lucas`. Category: `*math / numeric extras*`.

```
p lucas $x
```

lucas_n

`lucas_n` — operation `lucas n`. Category: `*math / number theory extras*`.

```
p lucas_n $x
```

lucas_nth

`lucas_nth(n)` → int — n-th Lucas number $L(n) = L(n-1) + L(n-2)$, $L(0)=2$, $L(1)=1$.

```
p lucas_nth(10) # 123
```

luhn

`luhn` — operation `luhn`. Alias for `luhn_check`. Category: *math / numeric extras*.

```
p luhn $x
```

luhn_check

`luhn_check` — operation `luhn check`. Category: *math / numeric extras*.

```
p luhn_check $x
```

luhn_digit

`luhn_digit` — operation `luhn digit`. Category: *validation / input checks*.

```
p luhn_digit $x
```

luminance_relative

`luminance_relative` — operation `luminance relative`. Category: *b82-misc-utility*.

```
p luminance_relative $x
```

lunation_index

`lunation_index` — index of `lunation`. Category: *astronomy / astrometry*.

```
p lunation_index $x
```

lv_competition_step

`lv_competition_step` — single-step update of `lv competition`. Category: *biology / ecology*.

```
p lv_competition_step $x
```

ly_to_pc

`ly_to_pc` — convert from `ly` to `pc`. Category: *astronomy / music / color / units*.

```
p ly_to_pc $value
```

lyndon_factorize

`lyndon_factorize` — operation `lyndon factorize`. Category: *more extensions*.

```
p lyndon_factorize $x
```

lz

`lz` — operation `lz`. Alias for `leading_zeros`. Category: *base conversion*.

```
p lz $x
```

lz4_block_step

lz4_block_step — single-step update of lz4 block. Category: *archive/encoding format primitives*.

```
p lz4_block_step $x
```

lz4_encode_block

lz4_encode_block — operation lz4 encode block. Category: *b81-misc-utility*.

```
p lz4_encode_block $x
```

lz4_match_offset

lz4_match_offset — operation lz4 match offset. Category: *archive/encoding format primitives*.

```
p lz4_match_offset $x
```

lz77_match_length

lz77_match_length — operation lz77 match length. Category: *electrochemistry, batteries, fuel cells*.

```
p lz77_match_length $x
```

lz78_dictionary_growth

lz78_dictionary_growth — operation lz78 dictionary growth. Category: *electrochemistry, batteries, fuel cells*.

```
p lz78_dictionary_growth $x
```

lzma_encode_step

lzma_encode_step — single-step update of lzma encode. Category: *b81-misc-utility*.

```
p lzma_encode_step $x
```

lzma_range_step

lzma_range_step — single-step update of lzma range. Category: *archive/encoding format primitives*.

```
p lzma_range_step $x
```

lzo_encode_step

lzo_encode_step — single-step update of lzo encode. Category: *b81-misc-utility*.

```
p lzo_encode_step $x
```

lzw_encode

lzw_encode — encode of lzw. Category: *b81-misc-utility*.

```
p lzw_encode $x
```

lzw_step_dict

lzw_step_dict — operation lzw step dict. Category: *electrochemistry, batteries, fuel cells*.

```
p lzw_step_dict $x
```

m5

`m5` — operation `m5`. Alias for `md5`. Category: **crypto / encoding**.

```
p m5 $x
```

m_to_feet

`m_to_feet` — convert from `m` to `feet`. Category: **unit conversions**.

```
p m_to_feet $value
```

m_to_miles

`m_to_miles` — convert from `m` to `miles`. Category: **unit conversions**.

```
p m_to_miles $value
```

m_to_s

`m_to_s` — convert from `m` to `s`. Alias for `minutes_to_seconds`. Category: **unit conversions**.

```
p m_to_s $value
```

m_to_yards

`m_to_yards` — convert from `m` to `yards`. Category: **unit conversions**.

```
p m_to_yards $value
```

ma_model_likelihood

`ma_model_likelihood` — operation `ma model likelihood`. Category: **econometrics**.

```
p ma_model_likelihood $x
```

mac_compare

`mac_compare` — MAC address op `compare`. Category: **network / ip / cidr**.

```
p mac_compare $x
```

mac_duration

`bond_duration` (alias `mac_duration`) computes Macaulay duration — the weighted-average time to receive cash flows.

```
val $dur = bond_duration([5,5,5,105], 0.05) # ≈ 3.72 years
```

mac_format

`mac_format` — MAC address op `format`. Category: **network / ip / cidr**.

```
p mac_format $x
```

mac_is_broadcast

`mac_is_broadcast` — MAC address op `is broadcast`. Category: **network / ip / cidr**.

```
p mac_is_broadcast $x
```

mac_is_locally_administered

`mac_is_locally_administered` — MAC address op `is locally administered`. Category: `*network / ip / cidr*`.

```
p mac_is_locally_administered $x
```

mac_is_multicast

`mac_is_multicast` — MAC address op `is multicast`. Category: `*network / ip / cidr*`.

```
p mac_is_multicast $x
```

mac_is_unicast

`mac_is_unicast` — MAC address op `is unicast`. Category: `*network / ip / cidr*`.

```
p mac_is_unicast $x
```

mac_is_universally_administered

`mac_is_universally_administered` — MAC address op `is universally administered`. Category: `*network / ip / cidr*`.

```
p mac_is_universally_administered $x
```

mac_is_valid

`mac_is_valid` — MAC address op `is valid`. Category: `*network / ip / cidr*`.

```
p mac_is_valid $x
```

mac_lookup_vendor

`mac_lookup_vendor` — MAC address op `lookup vendor`. Category: `*network / ip / cidr*`.

```
p mac_lookup_vendor $x
```

mac_normalize

`mac_normalize` — MAC address op `normalize`. Category: `*network / ip / cidr*`.

```
p mac_normalize $x
```

mac_oui

`mac_oui` — MAC address op `oui`. Category: `*network / ip / cidr*`.

```
p mac_oui $x
```

mac_parse

`mac_parse` — MAC address op `parse`. Category: `*network / ip / cidr*`.

```
p mac_parse $x
```

mac_random

`mac_random` — MAC address op `random`. Category: `*network / ip / cidr*`.

```
p mac_random $x
```

mac_random_local

mac_random_local — MAC address op random local. Category: *network / ip / cidr*.

```
p mac_random_local $x
```

mac_to_bytes

mac_to_bytes — convert from mac to bytes. Category: *network / ip / cidr*.

```
p mac_to_bytes $value
```

mac_to_int

mac_to_int — convert from mac to int. Category: *network / ip / cidr*.

```
p mac_to_int $value
```

mac_vendor_lookup

mac_vendor_lookup — MAC address op vendor lookup. Category: *network / ip / cidr*.

```
p mac_vendor_lookup $x
```

macarthur_wilson_extinction

macarthur_wilson_extinction — operation macarthur wilson extinction. Category: *biology / ecology*.

```
p macarthur_wilson_extinction $x
```

macarthur_wilson_immigration

macarthur_wilson_immigration — operation macarthur wilson immigration. Category: *biology / ecology*.

```
p macarthur_wilson_immigration $x
```

macaulay_duration

macaulay_duration — operation macaulay duration. Category: *financial pricing models*.

```
p macaulay_duration $x
```

macauley_duration

macauley_duration — operation macauley duration. Category: *excel/sheets + bond/loan financial*.

```
p macauley_duration $x
```

macd

macd(prices, fast=12, slow=26) → array — MACD line: EMA(fast) - EMA(slow). Pair with macd_signal and macd_histogram for full divergence analysis.

```
p macd(\@closes, 12, 26)
```

macd_histogram

`macd_histogram(prices, fast=12, slow=26, sig=9)` → array — `macd - macd_signal`. Positive bars = bullish momentum; bar height tracks divergence size.

```
p macd_histogram(\@closes)
```

macd_signal

`macd_signal(prices, fast=12, slow=26, sig=9)` → array — signal line for MACD: `EMA(macd_line, sig)`. Crossover with `macd` gives buy/sell ticks.

```
p macd_signal(\@closes, 12, 26, 9)
```

macdur

`macdur` — operation `macdur`. Alias for `duration`. Category: **finance (extended)**.

```
p macdur $x
```

mach_full_step

`mach_full_step` — single-step update of `mach full`. Category: **climate, fluids, atmospheric**.

```
p mach_full_step $x
```

mach_o_header_read

`mach_o_header_read(hex_bytes)` → hash — parse Mach-O magic → {`magic`, `cputype`, `filetype`}. Supports both endians and 32/64-bit.

mad

`mad` — operation `mad`. Alias for `median_absolute_deviation`. Category: **math / numeric extras**.

```
p mad $x
```

madd

`madd` — operation `madd`. Alias for `matrix_add`. Category: **extended stdlib**.

```
p madd $x
```

mae

`mae` — operation `mae`. Category: **math functions**.

```
p mae $x
```

mag

`mag` — operation `mag`. Alias for `magnitude`. Category: **extended stdlib**.

```
p mag $x
```


magenta

`magenta` — operation `magenta`. Alias for `red`. Category: **color / ansi**.

```
p magenta $x
```

magenta_bold

`magenta_bold` — operation `magenta bold`. Alias for `red`. Category: **color / ansi**.

```
p magenta_bold $x
```

maglens

`maglens` — operation `maglens`. Alias for `magnification_lens`. Category: **physics formulas**.

```
p maglens $x
```

magnification_lens

`magnification_lens` — operation `magnification lens`. Category: **physics formulas**.

```
p magnification_lens $x
```

magnitude

`magnitude` — operation `magnitude`. Category: **extended stdlib**.

```
p magnitude $x
```

magnus_1st

`magnus_1st` — operation `magnus 1st`. Category: **ode advanced**.

```
p magnus_1st $x
```

mahal

`mahalanobis` — Mahalanobis distance. Args: `data`, `center`, `inverse_covariance`. Like R's `mahalanobis()`.

```
val @d = @[mahal([[1,2],[3,4]], [2,3], [[1,0],[0,1]])]
```

mahalanobis_1d

`mahalanobis_1d` — operation `mahalanobis 1d`. Category: **ml extensions**.

```
p mahalanobis_1d $x
```

mahalanobis_sq

`mahalanobis_sq(x, mean, cov_inv) → number` — squared Mahalanobis distance with precision (inverse-covariance) matrix.

main

`main::` — the default package every script starts in. `our` variables and `fn` declarations without an explicit `package` directive land in `main::`; bare references like `$x` / `@a` / `%h` resolve to `$main::x` / `@main::a` / `%main::h` when looked up via the symbol table.

Use `main::` qualifier when you need to disambiguate inside a nested package, or when introspecting a script from a library context.

```
our $verbose = 1
p $main::verbose           # 1 - same scalar

# Inside a class, the bare name resolves to the class - main:: gets you out:
class Foo {
  fn check { p $main::verbose }      # reads top-level our
}

# %main:: is the symbol-table stash. Lists every name declared
# at the top level (subs, our, our @, our %).
p scalar keys %main::           # symbol-table size
keys %main:: |> sort |> p       # browse every top-level name
```

Related namespaces: `CORE::` (every builtin), `stryke::` (reflection hashes — `%stryke::builtins` etc.), and `Pkg::` for any user-declared package `Pkg`.

main_sequence_lifetime

`main_sequence_lifetime` — operation `main sequence lifetime`. Category: **cosmology / gr / flrw**.

```
p main_sequence_lifetime $x
```

mainardi_codazzi_step

`mainardi_codazzi_step` — single-step update of `mainardi codazzi`. Category: **electrochemistry, batteries, fuel cells**.

```
p mainardi_codazzi_step $x
```

majority

`majority` — operation `majority`. Alias for `majority_element`. Category: **extended stdlib**.

```
p majority $x
```

majority_element

`majority_element` — operation `majority element`. Category: **extended stdlib**.

```
p majority_element $x
```

mall

`mall` — operation `mall`. Alias for `match_all`. Category: **extended stdlib**.

```
p mall $x
```

mallat_pyramid_step

`mallat_pyramid_step` — single-step update of `mallat pyramid`. Category: **electrochemistry, batteries, fuel cells**.

```
p mallat_pyramid_step $x
```

man

`man` — operation `man`. Category: **shell-like repl (tier s)**.

```
p man $x
```

manacher

`manacher` — operation `manacher`. Alias for `manacher_radII`. Category: **misc**.

```
p manacher $x
```

manacher_radII

`manacher_radII` — operation `manacher radII`. Category: **misc**.

```
p manacher_radII $x
```

mandel

`mandel` — operation `mandel`. Alias for `mandelbrot_char`. Category: **algorithms / puzzles**.

```
p mandel $x
```

mandel_q

`mandel_q` — operation `mandel q`. Category: **quantum mechanics deep**.

```
p mandel_q $x
```

mandelbrot_char

`mandelbrot_char` — operation `mandelbrot char`. Category: **algorithms / puzzles**.

```
p mandelbrot_char $x
```

manhattan_distance

`manhattan_distance(a, b) → number` — L1 distance.

manhattan_norm

`manhattan_norm(v) → number` — L1 norm $\sum |v_i|$.

manipulability_yoshikawa

`manipulability_yoshikawa` — operation `manipulability yoshikawa`. Category: **robotics & control**.

```
p manipulability_yoshikawa $x
```

mann_kendall_tau

`mann_kendall_tau` — operation `mann kendall tau`. Category: **more extensions**.

```
p mann_kendall_tau $x
```

mann_whitney

`wilcox_test` (aliases `wilcox`, `mann_whitney`) — Wilcoxon rank-sum test (Mann-Whitney U). Returns U statistic. Like R's `wilcox.test()`.

```
val $U = wilcox([1,2,3], [4,5,6]) # 0 (no overlap)
```

mann_whitney_u

`mann_whitney_u(a, b)` → {`statistic`, `df`, `p_value`} — Mann-Whitney U (Wilcoxon rank-sum) non-parametric two-sample test.

```
p mann_whitney_u(\@group_a, \@group_b)
```

manning_velocity

`manning_velocity` — operation `manning velocity`. Category: **misc**.

```
p manning_velocity $x
```

mannwhitneyu

`mannwhitneyu` — operation `mannwhitneyu`. Category: **r / scipy distributions and tests**.

```
p mannwhitneyu $x
```

manpage

`manpage` — operation `manpage`. Alias for `man`. Category: **shell-like repl (tier s)**.

```
p manpage $x
```

mapOf

`mapOf` — operation `mapOf`. Category: **kotlin collections**.

```
p mapOf $x
```

map_bp

`map_bp` — operation `map bp`. Alias for `mean_arterial_pressure`. Category: **misc**.

```
p map_bp $x
```

map_chars

`map_chars` — operation `map chars`. Category: **string helpers**.

```
p map_chars $x
```

map_indexed

`map_indexed` — operation `map indexed`. Category: **go/general functional utilities**.

```
p map_indexed $x
```

map_keys_fn

`map_keys_fn` — operation `map keys fn`. Category: **hash ops**.

```
p map_keys_fn $x
```

map_range

`map_range($value, $in_min, $in_max, $out_min, $out_max)` (alias `remap`) — remap a value from one range to another.

```
p remap(50, 0, 100, 0, 1)      # 0.5
p remap(75, 0, 100, 0, 255)    # 191.25 (percent to byte)
```

map_values_fn

`map_values_fn` — operation `map values fn`. Category: **hash ops**.

```
p map_values_fn $x
```

map_while

`map_while` — operation `map while`. Category: **rust iterator methods**.

```
p map_while $x
```

mapcat

`mapcat` — operation `mapcat`. Category: **algebraic match**.

```
p mapcat $x
```

mapi

`mapi` — operation `mapi`. Alias for `map_indexed`. Category: **go/general functional utilities**.

```
p mapi $x
```

mapping_class_torus

`mapping_class_torus` — operation `mapping class torus`. Category: **econometrics**.

```
p mapping_class_torus $x
```

mapw

`mapw` — operation `mapw`. Alias for `map_while`. Category: **rust iterator methods**.

```
p mapw $x
```

marcum_q

`marcum_q` — operation `marcum q`. Category: **numpy + scipy.special**.

```
p marcum_q $x
```

margalef_richness

`margalef_richness` — operation `margalef richness`. Category: **biology / ecology**.

```
p margalef_richness $x
```

margin

`margin` — operation `margin`. Category: **finance**.

```
p margin $x
```

margin_price

`margin_price` — operation `margin price`. Category: **physics formulas**.

```
p margin_price $x
```

margrabe

`margrabe` — operation `margrabe`. Category: **financial pricing models**.

```
p margrabe $x
```

markdown_render

`markdown_render` — operation `markdown render`. Category: **extras**.

```
p markdown_render $x
```

markdown_to_html

`markdown_to_html` — convert from `markdown` to `html`. Category: **extras**.

```
p markdown_to_html $value
```

markov_decision_value

`markov_decision_value` — Markov chain op `decision value`. Category: **electrochemistry, batteries, fuel cells**.

```
p markov_decision_value $x
```

markov_stationary

`markov_stationary(transition_matrix, iters=200) → array` — power-iteration stationary distribution π .

```
p markov_stationary([[0.7,0.3],[0.4,0.6]]) # ≈ [0.571, 0.429]
```

markov_switching_ar

`markov_switching_ar` — Markov chain op `switching ar`. Category: **statsmodels**.

```
p markov_switching_ar $x
```

markov_switching_mr

`markov_switching_mr` — Markov chain op `switching mr`. Category: **statsmodels**.

```
p markov_switching_mr $x
```

markov_transition_matrix

`markov_transition_matrix(state_sequence) → matrix` — row-stochastic transition matrix learned from a sequence of integer states.

```
p markov_transition_matrix([0,1,1,0,1,0])
```

markup

markup — operation `markup`. Category: **finance**.

```
p markup $x
```

markup_pct

markup_pct(cost, price) → number — $(\text{price} - \text{cost}) / \text{cost} \cdot 100$.

```
p markup_pct(100, 150)
```

markup_price

markup_price — operation `markup price`. Category: **physics formulas**.

```
p markup_price $x
```

marshallian_demand

marshallian_demand — operation `marshallian demand`. Category: **economics + game theory**.

```
p marshallian_demand $x
```

mask

mask — operation `mask`. Alias for `mask_string`. Category: **string processing extras**.

```
p mask $x
```

mask_string

mask_string — operation `mask string`. Category: **string processing extras**.

```
p mask_string $x
```

mass_transport_overpotential

mass_transport_overpotential — operation `mass transport overpotential`. Category: **electrochemistry, batteries, fuel cells**.

```
p mass_transport_overpotential $x
```

mat4_determinant

mat4_determinant(m) → number — explicit 4×4 determinant.

mat4_identity

mat4_identity() → matrix — 4×4 identity matrix.

mat4_inverse

mat4_inverse(m) → matrix — explicit 4×4 inverse via cofactor expansion. Returns identity if singular.

mat4_look_at

mat4_look_at(eye, center, up) → matrix — right-handed view matrix.

```
p mat4_look_at([0,0,5], [0,0,0], [0,1,0])
```

mat4_multiply

mat4_multiply(a, b) → matrix — $a \cdot b$ matrix product.

mat4_orthographic

mat4_orthographic(l, r, b, t, n, f) → matrix — orthographic projection.

mat4_perspective

mat4_perspective(fov_y_rad, aspect, near, far) → matrix — symmetric perspective projection (right-handed, z to $-\infty$).

```
p mat4_perspective(1.5708, 16/9, 0.1, 100)
```

mat4_rotate_axis

mat4_rotate_axis([x,y,z], angle_rad) → matrix — Rodrigues axis-angle rotation.

mat4_rotate_x

mat4_rotate_x(angle_rad) → matrix — rotation about X axis.

mat4_rotate_y

mat4_rotate_y(angle_rad) → matrix — rotation about Y axis.

mat4_rotate_z

mat4_rotate_z(angle_rad) → matrix — rotation about Z axis.

mat4_scale

mat4_scale([sx,sy,sz]) → matrix — non-uniform scale matrix.

mat4_translate

mat4_translate([x,y,z]) → matrix — translation matrix.

```
p mat4_translate([1,2,3])
```

mat4_transpose

mat4_transpose(m) → matrix — transpose.

mat_mul

mat_mul — operation `mat mul`. Alias for `matrix_multiply`. Category: *matrix / linear algebra*.

```
p mat_mul $x
```


mat_scale

`mat_scale` — operation `mat scale`. Alias for `matrix_scale`. Category: **extended stdlib**.

```
p mat_scale $x
```

match_all

`match_all` — operation `match all`. Category: **extended stdlib**.

```
p match_all $x
```

match_rating

`match_rating` — operation `match rating`. Category: **iterator + string-distance extras**.

```
p match_rating $x
```

match_rating_codex

`match_rating_codex` — operation `match rating codex`. Category: **computational linguistics**.

```
p match_rating_codex $x
```

match_rating_compare

`match_rating_compare(s1, s2) → 0|1` — Match Rating Approach (Moore et al.); 1 if the two names are likely the same.

```
p match_rating_compare("Catherine", "Kathryn") # 1
```

matches_regex

`matches_regex` — operation `matches regex`. Category: **file stat / path**.

```
p matches_regex $x
```

matching_bipartite_greedy

`matching_bipartite_greedy(edges) → array` — greedy matching: first edge to an unmatched pair wins.

```
p matching_bipartite_greedy([[0,1],[0,2],[1,2]])
```

matching_bipartite_hungarian

`matching_bipartite_hungarian(cost_matrix) → array` — minimum-cost assignment via Hungarian algorithm. Returns `[[i,j], ...]`.

```
p matching_bipartite_hungarian([[4,1,3],[2,0,5],[3,2,2]])
```

matching_market_step

`matching_market_step` — single-step update of `matching market`. Category: **climate, fluids, atmospheric**.

```
p matching_market_step $x
```

matching_pennies_payoff

`matching_pennies_payoff` — operation `matching_pennies_payoff`. Category: **climate, fluids, atmospheric**.

```
p matching_pennies_payoff $x
```

mathieu_b

`mathieu_b` — operation `mathieu_b`. Category: **numpy + scipy.special**.

```
p mathieu_b $x
```

mathieu_ce0

`mathieu_ce0` — operation `mathieu_ce0`. Category: **special functions extra**.

```
p mathieu_ce0 $x
```

mathieu_se1

`mathieu_se1` — operation `mathieu_se1`. Category: **special functions extra**.

```
p mathieu_se1 $x
```

matmul

`matmul` — operation `matmul`. Alias for `matrix_mul`. Category: **extended stdlib**.

```
p matmul $x
```

matrix_add

`matrix_add` — matrix op (dense `Vec<Vec<f64>>`) `add`. Category: **extended stdlib**.

```
p matrix_add $x
```

matrix_adjugate

`matrix_adjugate(m)` → `matrix` — adjugate (classical adjoint): transpose of the cofactor matrix.

```
p matrix_adjugate([[1,2],[3,4]])
```

matrix_anti_diagonal

`matrix_anti_diagonal` — matrix op (dense `Vec<Vec<f64>>`) `anti_diagonal`. Category: **misc**.

```
p matrix_anti_diagonal $x
```

matrix_cholesky_decompose

`matrix_cholesky_decompose(m)` → `matrix` — Cholesky `L` such that $L \cdot L^T = m$. Requires `m` symmetric positive-definite.

```
p matrix_cholesky_decompose([[4,12,-16],[12,37,-43],[-16,-43,98]])
```

matrix_cofactor

`matrix_cofactor(m)` → `matrix` — cofactor matrix: each entry $(-1)^{(i+j)} \cdot |\text{minor}(i,j)|$.

```
p matrix_cofactor([[1,2],[3,4]])
```

matrix_col

`matrix_col` — matrix op (dense `Vec<Vec<f64>>`) `col`. Category: `*extended stdlib*`.

```
p matrix_col $x
```

matrix_cols

`matrix_cols(m)` → `int` — number of columns (length of first row).

```
p matrix_cols([[1,2,3],[4,5,6]]) # 3
```

matrix_concat_h

`matrix_concat_h(a, b)` → `matrix` — horizontal concatenation (`[A|B]`); rows must match.

```
p matrix_concat_h([[1,2]], [[3,4]])
```

matrix_concat_v

`matrix_concat_v(a, b)` → `matrix` — vertical concatenation (`[A;B]`); columns must match.

```
p matrix_concat_v([[1,2]], [[3,4]])
```

matrix_det

`matrix_det` — matrix op (dense `Vec<Vec<f64>>`) `det`. Category: `*extended stdlib*`.

```
p matrix_det $x
```

matrix_determinant

`matrix_determinant(m)` → `number` — determinant via Gaussian elimination with partial pivoting. $O(n^3)$.

```
p matrix_determinant([[1,2],[3,4]]) # -2
```

matrix_diag

`matrix_diag` — matrix op (dense `Vec<Vec<f64>>`) `diag`. Category: `*extended stdlib*`.

```
p matrix_diag $x
```

matrix_diagonal

`matrix_diagonal` — matrix op (dense `Vec<Vec<f64>>`) `diagonal`. Category: `*misc*`.

```
p matrix_diagonal $x
```

matrix_flatten

`matrix_flatten` — matrix op (dense `Vec<Vec<f64>>`) `flatten`. Category: `*matrix operations extras*`.

```
p matrix_flatten $x
```

matrix_from_cols

`matrix_from_cols(col1, col2, ...)` → `matrix` — assemble a matrix column-by-column.

```
p matrix_from_cols([1,2,3], [4,5,6])
```

matrix_from_rows

`matrix_from_rows` — matrix op (dense `Vec<Vec<f64>>`) `from` `rows`. Category: `*matrix operations extras*`.

```
p matrix_from_rows $x
```

matrix_get

`matrix_get(m, r, c)` → `number` — element at (`row`, `col`); returns `undef` if out of bounds.

```
p matrix_get([[1,2],[3,4]], 1, 0) # 3
```

matrix_hadamard

`matrix_hadamard` — matrix op (dense `Vec<Vec<f64>>`) `hadamard`. Category: `*matrix operations extras*`.

```
p matrix_hadamard $x
```

matrix_identity

`matrix_identity` — matrix op (dense `Vec<Vec<f64>>`) `identity`. Category: `*extended stdlib*`.

```
p matrix_identity $x
```

matrix_identity_q

`matrix_identity_q` — matrix op (dense `Vec<Vec<f64>>`) `identity q`. Category: `*more extensions (2)*`.

```
p matrix_identity_q $x
```

matrix_inverse

`matrix_inverse` — matrix op (dense `Vec<Vec<f64>>`) `inverse`. Category: `*matrix operations extras*`.

```
p matrix_inverse $x
```

matrix_kronecker

`matrix_kronecker(a, b)` → `matrix` — Kronecker (tensor) product. Result is $(\mathbf{ra} \cdot \mathbf{rb}) \times (\mathbf{ca} \cdot \mathbf{cb})$.

```
p matrix_kronecker([[1,2],[3,4]], [[0,5],[6,7]])
```

matrix_lu

`matrix_lu` (alias `mlu`) computes the LU decomposition with partial pivoting. Returns `[L, U, P]` where $\mathbf{PA} = \mathbf{LU}$.

```
var ($L, $U, $P) = @{mlu([[4,3],[6,3]])}
```

matrix_lu_decompose

`matrix_lu_decompose(m)` → `[L, U]` — LU decomposition without pivoting (Doolittle). Returns lower-triangular `L` (unit diag) and upper-triangular `U`.

```
my ($L, $U) = @{matrix_lu_decompose([[4,3],[6,3]])}
```

matrix_map

`matrix_map` — matrix op (dense `Vec<Vec<f64>`) `map`. Category: **matrix operations extras**.

```
p matrix_map $x
```

matrix_max

`matrix_max` — matrix op (dense `Vec<Vec<f64>`) `max`. Category: **matrix operations extras**.

```
p matrix_max $x
```

matrix_min

`matrix_min` — matrix op (dense `Vec<Vec<f64>`) `min`. Category: **matrix operations extras**.

```
p matrix_min $x
```

matrix_minor

`matrix_minor(m, skip_r, skip_c) → matrix` — submatrix with row `skip_r` and column `skip_c` removed.

```
p matrix_minor([[1,2,3],[4,5,6],[7,8,9]], 1, 1)
```

matrix_mul

`matrix_mul` — matrix op (dense `Vec<Vec<f64>`) `mul`. Category: **extended stdlib**.

```
p matrix_mul $x
```

matrix_mult

`matrix_mult` — matrix op (dense `Vec<Vec<f64>`) `mult`. Category: **extended stdlib**.

```
p matrix_mult $x
```

matrix_multiply

`matrix_multiply` — matrix op (dense `Vec<Vec<f64>`) `multiply`. Category: **matrix / linear algebra**.

```
p matrix_multiply $x
```

matrix_new

`matrix_new(rows, cols=rows, fill=0) → matrix` — allocate an `rows×cols` matrix filled with `fill`.

```
p matrix_new(3, 4, 0) # 3x4 zero matrix
```

matrix_norm

`matrix_norm` (alias `mnorm`) computes a matrix norm. Default is Frobenius; pass 1 for max-column-sum, Inf for max-row-sum.

```
p mnorm([[3,4]]) # 5 (Frobenius)
p mnorm([[1,2],[3,4]], 1) # 6 (1-norm)
```

matrix_norm_frobenius

`matrix_norm_frobenius(m)` → `number` — $\sqrt{(\sum |m[i][j]|^2)}$. Matrix L2 in the entry-wise sense.

```
p matrix_norm_frobenius([[1,2],[3,4]]) # sqrt(30) ≈ 5.477
```

matrix_norm_l1

`matrix_norm_l1(m)` → `number` — max absolute column sum.

```
p matrix_norm_l1([[1,-2],[3,4]])
```

matrix_norm_linf

`matrix_norm_linf(m)` → `number` — max absolute row sum.

```
p matrix_norm_linf([[1,-2],[3,4]])
```

matrix_ones

`matrix_ones` — matrix op (dense `Vec<Vec<f64>>`) `ones`. Category: `*extended stdlib*`.

```
p matrix_ones $x
```

matrix_orthogonal_q

`matrix_orthogonal_q` — matrix op (dense `Vec<Vec<f64>>`) `orthogonal q`. Category: `*misc*`.

```
p matrix_orthogonal_q $x
```

matrix_outer_product

`matrix_outer_product(u, v)` → `matrix` — outer product $u \cdot v^T$; result is `len(u) * len(v)`.

```
p matrix_outer_product([1,2,3], [4,5])
```

matrix_pinv

`matrix_pinv` (aliases `mpinv`, `pinv`) computes the Moore-Penrose pseudo-inverse via $(A^T A)^{-1} A^T$.

```
val $A_plus = pinv([[1,2],[3,4],[5,6]])
```

matrix_power

`matrix_power` — matrix op (dense `Vec<Vec<f64>>`) `power`. Category: `*matrix operations extras*`.

```
p matrix_power $x
```

matrix_qr

`matrix_qr` (alias `mqr`) computes the QR decomposition via Gram-Schmidt orthogonalization. Returns `[Q, R]` where $A = QR$.

```
var ($Q, $R) = @{mqr([[1,1],[1,-1]])}
```

matrix_qr_decompose

`matrix_qr_decompose(m)` → `[Q, R]` — Gram-Schmidt QR decomposition. `Q` has orthonormal columns; `R` is upper-triangular.

```
my ($Q, $R) = @{matrix_qr_decompose($m)}
```

matrix_random_normal

`matrix_random_normal` — matrix op (dense Vec<Vec<f64>>) `random normal`. Category: *more extensions (2)*.

```
p matrix_random_normal $x
```

matrix_random_uniform

`matrix_random_uniform` — matrix op (dense Vec<Vec<f64>>) `random uniform`. Category: *more extensions (2)*.

```
p matrix_random_uniform $x
```

matrix_reshape

`matrix_reshape(m, new_rows, new_cols) → matrix` — reshape preserving element order (row-major flatten then refill).

```
p matrix_reshape([[1,2,3,4]], 2, 2)
```

matrix_row

`matrix_row` — matrix op (dense Vec<Vec<f64>>) `row`. Category: *extended stdlib*.

```
p matrix_row $x
```

matrix_rows

`matrix_rows(m) → int` — number of rows.

```
p matrix_rows([[1,2],[3,4],[5,6]]) # 3
```

matrix_scalar

`matrix_scalar` — matrix op (dense Vec<Vec<f64>>) `scalar`. Category: *extended stdlib*.

```
p matrix_scalar $x
```

matrix_scale

`matrix_scale` — matrix op (dense Vec<Vec<f64>>) `scale`. Category: *extended stdlib*.

```
p matrix_scale $x
```

matrix_set

`matrix_set(m, r, c, v) → matrix` — returns a new matrix with `m[r][c] = v`.

```
p matrix_set([[1,2],[3,4]], 0, 1, 99)
```

matrix_shape

`matrix_shape` — matrix op (dense Vec<Vec<f64>>) `shape`. Category: *extended stdlib*.

```
p matrix_shape $x
```

matrix_solve

`matrix_solve` (aliases `msolve`, `solve`) solves the linear system $Ax=b$ via Gaussian elimination with partial pivoting. Returns the solution vector x .

```
val $A = [[2,1],[-1,1]]
var $y = [5,2]
var $x = solve($A, $y) # [1, 3]
```

matrix_sub

`matrix_sub` — matrix op (dense `Vec<Vec<f64>`) `sub`. Category: **extended stdlib**.

```
p matrix_sub $x
```

matrix_submatrix

`matrix_submatrix(m, r0, c0, nr, nc) → matrix` — `nr×nc` block starting at `(r0,c0)`.

```
p matrix_submatrix([[1,2,3],[4,5,6],[7,8,9]], 1, 1, 2, 2)
```

matrix_sum

`matrix_sum` — matrix op (dense `Vec<Vec<f64>`) `sum`. Category: **matrix operations extras**.

```
p matrix_sum @xs
```

matrix_swap_cols

`matrix_swap_cols(m, i, j) → matrix` — swap two columns across all rows.

matrix_swap_rows

`matrix_swap_rows(m, i, j) → matrix` — swap two rows.

matrix_symmetric_q

`matrix_symmetric_q` — matrix op (dense `Vec<Vec<f64>`) `symmetric q`. Category: **misc**.

```
p matrix_symmetric_q $x
```

matrix_to_string

`matrix_to_string(m) → string` — pretty-print rows joined by newlines, each row comma-separated with 4 decimal places.

```
print matrix_to_string([[1.5,2.7],[3.14,4.2]])
```

matrix_trace

`matrix_trace` — matrix op (dense `Vec<Vec<f64>`) `trace`. Category: **extended stdlib**.

```
p matrix_trace $x
```

matrix_transpose

`matrix_transpose` — matrix op (dense `Vec<Vec<f64>`) `transpose`. Category: **matrix operations extras**.

```
p matrix_transpose $x
```


matrix_vec_mul

`matrix_vec_mul(m, v) → array` — matrix-vector product $m \cdot v$.

```
p matrix_vec_mul([[1,2],[3,4]], [5,6]) # [17, 39]
```

matrix_zero_q

`matrix_zero_q` — matrix op (dense `Vec<Vec<f64>>`) `zero q`. Category: **more extensions (2)**.

```
p matrix_zero_q $x
```

matrix_zeros

`matrix_zeros` — matrix op (dense `Vec<Vec<f64>>`) `zeros`. Category: **extended stdlib**.

```
p matrix_zeros $x
```

matroid_greedy_step

`matroid_greedy_step` — single-step update of `matroid greedy`. Category: **combinatorial optimization, scheduling**.

```
p matroid_greedy_step $x
```

matroid_intersection_step

`matroid_intersection_step` — single-step update of `matroid intersection`. Category: **combinatorial optimization, scheduling**.

```
p matroid_intersection_step $x
```

max2

`max2` — operation `max2`. Category: **trivial numeric / predicate builtins**.

```
p max2 $x
```

max_abs

`max_abs` — operation `max abs`. Category: **trig / math**.

```
p max_abs $x
```

max_by

`max_by` — operation `max by`. Alias for `take_while`. Category: **functional / iterator**.

```
p max_by $x
```

max_degree

`max_degree` — operation `max degree`. Category: **graph algorithms**.

```
p max_degree $x
```

max_drawdown

`max_drawdown(@equity_curve)` (alias `mdd`) — computes the maximum peak-to-trough decline in a series of values. Returns as a decimal (0.2 = 20% drawdown). Key risk metric.

```
val @equity = (100, 120, 90, 110, 85, 130)
p mdd(@equity)    # -0.29 (29% max drawdown)
```

max_drawdown_arr

`max_drawdown_arr` — operation `max drawdown arr`. Category: **more extensions (2)**.

```
p max_drawdown_arr $x
```

max_float

`max_float` — operation `max float`. Category: **conversion / utility**.

```
p max_float $x
```

max_flow_ek

`max_flow_ek` — operation `max flow ek`. Category: **graph algorithms**.

```
p max_flow_ek $x
```

max_flow_ford_fulkerson_step

`max_flow_ford_fulkerson_step` — single-step update of `max flow ford fulkerson`. Category: **combinatorial optimization, scheduling**.

```
p max_flow_ford_fulkerson_step $x
```

max_heart_rate

`max_heart_rate` — rate of `max heart`. Category: **misc**.

```
p max_heart_rate $x
```

max_independent_set

`max_independent_set` — operation `max independent set`. Category: **networkx graph algorithms**.

```
p max_independent_set $x
```

max_list

`max_list` — operation `max list`. Category: **list helpers**.

```
p max_list $x
```

max_norm

`max_norm` — norm (vector length-like) of `max`. Alias for `linf_norm`. Category: **misc**.

```
p max_norm $x
```

max_of

`max_of` — operation `max of`. Category: **more list helpers**.

```
p max_of $x
```

max_sum_subarray

`max_sum_subarray` — operation `max sum subarray`. Category: **extended stdlib**.

```
p max_sum_subarray $x
```

maxby

`maxby` — operation `maxby`. Alias for `maximum_by`. Category: **additional missing stdlib functions**.

```
p maxby $x
```

maxdd

`maxdd` — operation `maxdd`. Alias for `max_drawdown`. Category: **finance (extended)**.

```
p maxdd $x
```

maximin_share

`maximin_share` — operation `maximin share`. Category: **climate, fluids, atmospheric**.

```
p maximin_share $x
```

maximum_a_posteriori

`maximum_a_posteriori(posterior) → int` — argmax of a posterior probability array.

maximum_bipartite_matching

`maximum_bipartite_matching` — operation `maximum bipartite matching`. Category: **combinatorial optimization, scheduling**.

```
p maximum_bipartite_matching $x
```

maximum_by

`maximum_by` — operation `maximum by`. Category: **additional missing stdlib functions**.

```
p maximum_by $x
```

maxstr

`maxstr` — operation `maxstr`. Category: **list / aggregate**.

```
p maxstr $x
```

maxsub

`maxsub` — operation `maxsub`. Alias for `max_sum_subarray`. Category: **extended stdlib**.

```
p maxsub $x
```

maya_long_count_from_fixed

`maya_long_count_from_fixed` — operation `maya long count from fixed`. Category: **calendrical algorithms**.

```
p maya_long_count_from_fixed $x
```

mayan_haab_from_fixed

`mayan_haab_from_fixed` — operation `mayan haab from fixed`. Category: **calendrical algorithms**.

```
p mayan_haab_from_fixed $x
```

mayan_tzolkin_from_fixed

`mayan_tzolkin_from_fixed` — operation `mayan tzolkin from fixed`. Category: **calendrical algorithms**.

```
p mayan_tzolkin_from_fixed $x
```

mb_to_bytes

`mb_to_bytes` — convert from `mb` to `bytes`. Category: **unit conversions**.

```
p mb_to_bytes $value
```

mb_to_gb

`mb_to_gb` — convert from `mb` to `gb`. Category: **unit conversions**.

```
p mb_to_gb $value
```

mc_limiter

`mc_limiter` — operation `mc limiter`. Category: **ode advanced**.

```
p mc_limiter $x
```

mcat

`mcat` — operation `mcat`. Alias for `mapcat`. Category: **algebraic match**.

```
p mcat $x
```

mcc

`mcc` — operation `mcc`. Category: **ml extensions**.

```
p mcc $x
```

mcol

`mcol` — operation `mcol`. Alias for `matrix_col`. Category: **extended stdlib**.

```
p mcol $x
```

mcp_serve_registered_tools

`mcp_serve_registered_tools("server-name")` — convert every tool registered via `tool fn / ai_register_tool` into MCP server schema and run the stdio JSON-RPC loop. Used by `stryke build --mcp-server`.

md5

`md5` — operation `md5`. Category: **crypto / encoding**.

```
p md5 $x
```

mdet

`mdet` — operation `mdet`. Alias for `matrix_det`. Category: **extended stdlib**.

```
p mdet $x
```

mdia

`mdia` — operation `mdia`. Alias for `matrix_diag`. Category: **extended stdlib**.

```
p mdia $x
```

mdiagvec

`mdiagvec` — operation `mdiagvec`. Alias for `matrix_diagonal`. Category: **misc**.

```
p mdiagvec $x
```

mdist

`mdist` — operation `mdist`. Alias for `manhattan_distance`. Category: **extras**.

```
p mdist $x
```

mduration

`mduration` — operation `mduration`. Category: **b81-misc-utility**.

```
p mduration $x
```

mean_absolute_error

`mean_absolute_error` — operation `mean absolute error`. Category: **math / numeric extras**.

```
p mean_absolute_error $x
```

mean_activity_coeff_pitzer

`mean_activity_coeff_pitzer` — operation `mean activity coeff pitzer`. Category: **electrochemistry, batteries, fuel cells**.

```
p mean_activity_coeff_pitzer $x
```

mean_arterial_pressure

`mean_arterial_pressure` — operation `mean arterial pressure`. Category: **misc**.

```
p mean_arterial_pressure $x
```

mean_curvature_step

`mean_curvature_step` — single-step update of `mean curvature`. Category: **electrochemistry, batteries, fuel cells**.

```
p mean_curvature_step $x
```

mean_filter_window

`mean_filter_window` — operation `mean filter window`. Category: **electrochemistry, batteries, fuel cells**.

```
p mean_filter_window $x
```

mean_list

`mean_list` — operation `mean list`. Category: **list helpers**.

```
p mean_list $x
```

mean_solar_anomaly

`mean_solar_anomaly` — operation `mean solar anomaly`. Category: **more extensions**.

```
p mean_solar_anomaly $x
```

mean_solar_longitude

`mean_solar_longitude` — operation `mean solar longitude`. Category: **more extensions**.

```
p mean_solar_longitude $x
```

mean_squared_error

`mean_squared_error` — operation `mean squared error`. Category: **math / numeric extras**.

```
p mean_squared_error $x
```

mean_tone_freq

`mean_tone_freq` — operation `mean tone freq`. Category: **music theory**.

```
p mean_tone_freq $x
```

measure

`measure` — operation `measure`. Category: **conversion / utility**.

```
p measure $x
```

mechanism_design_obj

`mechanism_design_obj` — operation `mechanism design obj`. Category: **climate, fluids, atmospheric**.

```
p mechanism_design_obj $x
```

medfilt

`median_filter(\@signal, $window_size)` (alias `medfilt`) — applies a median filter for noise removal. Each output is the median of the surrounding window. Excellent for removing impulse noise while preserving edges.

```
val @spiky = (1, 2, 100, 3, 4) # 100 is spike
var $clean = medfilt(\@spiky, 3)
p @$clean # spike removed
```

medfilt_1d

`medfilt_1d` — operation `medfilt 1d`. Category: **economics + game theory**.

```
p medfilt_1d $x
```

median_absolute_deviation

`median_absolute_deviation` — operation `median absolute deviation`. Category: **math / numeric extras**.

```
p median_absolute_deviation $x
```

median_filter_window

`median_filter_window` — operation `median filter window`. Category: **electrochemistry, batteries, fuel cells**.

```
p median_filter_window $x
```

median_voter_step

`median_voter_step` — single-step update of `median voter`. Category: **climate, fluids, atmospheric**.

```
p median_voter_step $x
```

mem_free

`mem_free` — free/available physical RAM in bytes. On Linux reads `MemAvailable` from `/proc/meminfo` (falls back to `MemFree`). On macOS uses `vm.page_free_count` via `sysctl`. Returns `undef` on unsupported platforms.

```
p mem_free                                # 3821666304
p format_bytes(mem_free)                  # 3.6 GiB
var $pct = int(mem_free / mem_total * 100)
p "$pct% free"
```

mem_total

`mem_total` — total physical RAM in bytes. Uses `/proc/meminfo` on Linux, `sysctlbyname("hw.memsize")` on macOS. Returns `undef` on unsupported platforms.

```
p mem_total                                # 68719476736
p format_bytes(mem_total)                  # 64.0 GiB
```

mem_used

`mem_used` — used physical RAM in bytes (total - free). Returns `undef` if either `mem_total` or `mem_free` is unavailable.

```
p format_bytes(mem_used)                  # 60.4 GiB
gauge(mem_used / mem_total) |> p          # memory usage gauge
```

membership_idx

`membership_idx` — operation `membership idx`. Category: **apl/j/k array primitives**.

```
p membership_idx $x
```

menhinick_richness

`menhinick_richness` — operation `menhinick richness`. Category: **biology / ecology**.

```
p menhinick_richness $x
```

mercator_project_x

`mercator_project_x(lon_deg)` → `number` — spherical Mercator x in meters using WGS84 radius.

```
p mercator_project_x(-122.4194)
```

mercator_project_y

`mercator_project_y(lat_deg)` → `number` — spherical Mercator y in meters; clamps lat to $\pm 85.05^\circ$ to stay finite.

```
p mercator_project_y(37.7749)
```

mercator_unproject

`mercator_unproject` — operation `mercator unproject`. Category: **extras**.

```
p mercator_unproject $x
```

mercator_unproject_lat

`mercator_unproject_lat(y)` → `number` — inverse Mercator latitude in degrees.

mercator_unproject_lon

`mercator_unproject_lon(x)` → `number` — inverse Mercator longitude in degrees.

mercury_perihelion_advance

`mercury_perihelion_advance` — operation `mercury perihelion advance`. Category: **electrochemistry, batteries, fuel cells**.

```
p mercury_perihelion_advance $x
```

merge_hash

`merge_hash` — hash function of `merge`. Category: **hash helpers**.

```
p merge_hash $x
```

merge_sorted

`merge_sorted` — operation `merge sorted`. Category: **array / list operations extras**.

```
p merge_sorted $x
```

mertens

`mertens` — operation `mertens`. Category: **cryptanalysis & number theory deep**.

```
p mertens $x
```

mertens_zamir_step

`mertens_zamir_step` — single-step update of `mertens zamir`. Category: **climate, fluids, atmospheric**.

```
p mertens_zamir_step $x
```


merton_jump_call

`merton_jump_call` — operation `merton_jump_call`. Category: **financial pricing models**.

```
p merton_jump_call $x
```

metal_corrosion_rate

`metal_corrosion_rate` — rate of `metal_corrosion`. Category: **electrochemistry, batteries, fuel cells**.

```
p metal_corrosion_rate $x
```

metamorphic_grade

`metamorphic_grade` — operation `metamorphic_grade`. Category: **geology, seismology, mineralogy**.

```
p metamorphic_grade $x
```

metaphone

`metaphone` — operation `metaphone`. Category: **encoding / phonetics**.

```
p metaphone $x
```

metaphone_phonetic

`metaphone_phonetic` — operation `metaphone_phonetic`. Category: **computational linguistics**.

```
p metaphone_phonetic $x
```

meters_to_feet

`meters_to_feet` — convert from `meters` to `feet`. Category: **astronomy / music / color / units**.

```
p meters_to_feet $value
```

methane_growth_rate

`methane_growth_rate` — rate of `methane_growth`. Category: **climate, fluids, atmospheric**.

```
p methane_growth_rate $x
```

metonic_cycle

`metonic_cycle` — operation `metonic_cycle`. Category: **astronomy / astrometry**.

```
p metonic_cycle $x
```

metph

`metph` — operation `metph`. Alias for `metaphone`. Category: **encoding / phonetics**.

```
p metph $x
```

metric_frw_lapse

`metric_frw_lapse` — operation `metric_frw_lapse`. Category: **electrochemistry, batteries, fuel cells**.

```
p metric_frw_lapse $x
```

metric_kerr_step_simple

`metric_kerr_step_simple` — operation `metric kerr step simple`. Category: **electrochemistry, batteries, fuel cells**.

```
p metric_kerr_step_simple $x
```

metric_minkowski_eta_step

`metric_minkowski_eta_step` — single-step update of `metric minkowski eta`. Category: **electrochemistry, batteries, fuel cells**.

```
p metric_minkowski_eta_step $x
```

metric_schwarzschild_step

`metric_schwarzschild_step` — single-step update of `metric schwarzschild`. Category: **electrochemistry, batteries, fuel cells**.

```
p metric_schwarzschild_step $x
```

metro_hash_64

`metro_hash_64` — operation `metro hash 64`. Category: **b81-misc-utility**.

```
p metro_hash_64 $x
```

mexican_hat_wavelet

`mexican_hat_wavelet` — operation `mexican hat wavelet`. Category: **economics + game theory**.

```
p mexican_hat_wavelet $x
```

mfcc_coeff_step

`mfcc_coeff_step` — single-step update of `mfcc coeff`. Category: **electrochemistry, batteries, fuel cells**.

```
p mfcc_coeff_step $x
```

mflat

`mflat` — operation `mflat`. Alias for `matrix_flatten`. Category: **matrix operations extras**.

```
p mflat $x
```

mfromr

`mfromr` — operation `mfromr`. Alias for `matrix_from_rows`. Category: **matrix operations extras**.

```
p mfromr $x
```

mget

`mget` — operation `mget`. Category: **redis-flavour primitives**.

```
p mget $x
```

mgrs_decode

`mgrs_decode` — decode of `mgrs`. Category: **excel/sheets + bond/loan financial**.

```
p mgrs_decode $x
```

mgrs_encode

`mgrs_encode` — encode of `mgrs`. Category: **excel/sheets + bond/loan financial**.

```
p mgrs_encode $x
```

mh_add

`mh_add(MH, ITEM)` — fold an element into the signature.

mh_jaccard

`mh_jaccard(A, B)` → estimated Jaccard similarity.

mh_merge

`mh_merge(A, B)` — register-wise min into A (union of two sets). Returns `1` on success.

mhad

`mhad` — operation `mhad`. Alias for `matrix_hadamard`. Category: **matrix operations extras**.

```
p mhad $x
```

michaelis_menten

`michaelis_menten` — operation `michaelis menten`. Category: **misc**.

```
p michaelis_menten $x
```

microlensing_magnification

`microlensing_magnification` — operation `microlensing magnification`. Category: **cosmology / gr / flrw**.

```
p microlensing_magnification $x
```

mid_str

`mid_str` — operation `mid str`. Category: **string**.

```
p mid_str $x
```

mident

`mident` — operation `mident`. Alias for `matrix_identity`. Category: **extended stdlib**.

```
p mident $x
```

midi_note_to_name

`midi_note_to_name` — convert from `midi note` to `name`. Category: **extras**.

```
p midi_note_to_name $value
```

midi_to_hz

`midi_to_hz` — convert from `midi` to `hz`. Category: **astronomy / music / color / units**.

```
p midi_to_hz $value
```

midi_to_note_name

`midi_to_note_name(midi) → string` — convert MIDI note number (0-127) to scientific pitch notation.

```
p midi_to_note_name(60) # C4
p midi_to_note_name(69) # A4
```

midi_to_pitch_class

`midi_to_pitch_class` — convert from `midi` to `pitch class`. Category: **misc**.

```
p midi_to_pitch_class $value
```

midpnt

`midpnt` — operation `midpnt`. Alias for `midpoint`. Category: **geometry / physics**.

```
p midpnt $x
```

midpoint

`midpoint` — operation `midpoint`. Category: **geometry / physics**.

```
p midpoint $x
```

midpoint_great_circle

`midpoint_great_circle` — operation `midpoint great circle`. Category: **geometry / topology**.

```
p midpoint_great_circle $x
```

midpoint_of

`midpoint_of` — operation `midpoint of`. Category: **extended stdlib**.

```
p midpoint_of $x
```

midpoint_rule

`midpoint_rule` — operation `midpoint rule`. Category: **more extensions (2)**.

```
p midpoint_rule $x
```

midpt

`midpt` — operation `midpt`. Alias for `midpoint_of`. Category: **extended stdlib**.

```
p midpt $x
```

miles_to_km

`miles_to_km` — convert from `miles` to `km`. Category: **unit conversions**.

```
p miles_to_km $value
```

miles_to_m

`miles_to_m` — convert from `miles` to `m`. Category: **unit conversions**.

```
p miles_to_m $value
```

milne

`milne` — operation `milne`. Alias for `milne_pc`. Category: **more extensions (2)**.

```
p milne $x
```

milne_pc

`milne_pc` — operation `milne pc`. Category: **more extensions (2)**.

```
p milne_pc $x
```

milstein_step

`milstein_step` — single-step update of `milstein`. Category: **ode advanced**.

```
p milstein_step $x
```

mime_extension_for_type

`mime_extension_for_type` — operation `mime extension for type`. Category: **network / ip / cidr**.

```
p mime_extension_for_type $x
```

mime_is_application

`mime_is_application` — operation `mime is application`. Category: **network / ip / cidr**.

```
p mime_is_application $x
```

mime_is_audio

`mime_is_audio` — operation `mime is audio`. Category: **network / ip / cidr**.

```
p mime_is_audio $x
```

mime_is_image

`mime_is_image` — operation `mime is image`. Category: **network / ip / cidr**.

```
p mime_is_image $x
```

mime_is_text

`mime_is_text` — operation `mime is text`. Category: **network / ip / cidr**.

```
p mime_is_text $x
```

mime_is_video

`mime_is_video` — operation `mime is video`. Category: **network / ip / cidr**.

```
p mime_is_video $x
```

mime_type_for_extension

`mime_type_for_extension` — operation `mime type for extension`. Category: **network / ip / cidr**.

```
p mime_type_for_extension $x
```

min2

`min2` — operation `min2`. Category: **trivial numeric / predicate builtins**.

```
p min2 $x
```

min_abs

`min_abs` — operation `min abs`. Category: **trig / math**.

```
p min_abs $x
```

min_by

`min_by` — operation `min by`. Alias for `take_while`. Category: **functional / iterator**.

```
p min_by $x
```

min_cut_value

`min_cut_value` — value of `min cut`. Category: **graph algorithms**.

```
p min_cut_value $x
```

min_float

`min_float` — operation `min float`. Category: **conversion / utility**.

```
p min_float $x
```

min_list

`min_list` — operation `min list`. Category: **list helpers**.

```
p min_list $x
```

min_max

`min_max` — maximum of `min`. Category: **file stat / path**.

```
p min_max $x
```

min_of

`min_of` — operation `min of`. Category: **more list helpers**.

```
p min_of $x
```

min_steiner_tree

`min_steiner_tree` — operation `min steiner tree`. Category: **networkx graph algorithms**.

```
p min_steiner_tree $x
```

min_vertex_cover

`min_vertex_cover` — operation `min vertex cover`. Category: **networkx graph algorithms**.

```
p min_vertex_cover $x
```

minby

`minby` — operation `minby`. Alias for `minimum_by`. Category: **additional missing stdlib functions**.

```
p minby $x
```

mincut_stoer_wagner

`mincut_stoer_wagner` — operation `mincut stoer wagner`. Category: **combinatorial optimization, scheduling**.

```
p mincut_stoer_wagner $x
```

mineral_mohs_hardness

`mineral_mohs_hardness` — operation `mineral mohs hardness`. Category: **geology, seismology, mineralogy**.

```
p mineral_mohs_hardness $x
```

minhash

`minhash(K=128)` — MinHash Jaccard-similarity signature with `K` independent hash functions (Kirsch-Mitzenmacher double-hashing over xxh3-128). `mh_jaccard(a, b) ≈ true` Jaccard with error $O(1/\sqrt{K})$.

```
val $ma = minhash(256); val $mb = minhash(256)
mh_add($ma, _) for @shingles_a
mh_add($mb, _) for @shingles_b
p mh_jaccard($ma, $mb)           # ~|A∩B|/|A∪B|
```

minimax_value

`minimax_value(leaves, depth) → number` — minimax over a complete binary tree of leaf payoffs. Even depth = max layer.

```
p minimax_value([3,5,2,9], 2) # 3
```

minimum_by

`minimum_by` — operation `minimum by`. Category: **additional missing stdlib functions**.

```
p minimum_by $x
```

minkdist

`minkdist` — operation `minkdist`. Alias for `minkowski_distance`. Category: **math / numeric extras**.

```
p minkdist $x
```

minkowski_distance

`minkowski_distance` — distance / dissimilarity metric of `minkowski`. Category: **math / numeric extras**.

```
p minkowski_distance $x
```

minkowski_norm

`minkowski_norm(v, p=2)` → number — $(\sum |v_i|^p)^{1/p}$.

minkowski_sum_2d

`minkowski_sum_2d(a, b)` → array — Minkowski sum point set $A \oplus B = \{a+b : a \in A, b \in B\}$.

minkowski_sum_simple

`minkowski_sum_simple` — operation `minkowski sum simple`. Category: **more extensions (2)**.

```
p minkowski_sum_simple $x
```

minmax

`minmax` — operation `minmax`. Category: **python/ruby stdlib**.

```
p minmax $x
```

minmax_by

`minmax_by` — operation `minmax by`. Category: **python/ruby stdlib**.

```
p minmax_by $x
```

minmax_scale

`minmax_scale` — operation `minmax scale`. Category: **ml extensions**.

```
p minmax_scale $x
```

minmod_limiter

`minmod_limiter` — operation `minmod limiter`. Category: **ode advanced**.

```
p minmod_limiter $x
```

minstr

`minstr` — operation `minstr`. Category: **list / aggregate**.

```
p minstr $x
```


minutes_to_hours

`minutes_to_hours` — convert from `minutes` to `hours`. Category: **unit conversions**.

```
p minutes_to_hours $value
```

minutes_to_seconds

`minutes_to_seconds` — convert from `minutes` to `seconds`. Category: **unit conversions**.

```
p minutes_to_seconds $value
```

minv

`minv` — operation `minv`. Alias for `matrix_inverse`. Category: **matrix operations extras**.

```
p minv $x
```

mirr

`mirr` — operation `mirr`. Alias for `modified_irr`. Category: **misc**.

```
p mirr $x
```

mirr_excel

`mirr_excel` — operation `mirr excel`. Category: **b81-misc-utility**.

```
p mirr_excel $x
```

mirror_descent_step

`mirror_descent_step` — single-step update of `mirror descent`. Category: **combinatorial optimization, scheduling**.

```
p mirror_descent_step $x
```

mirror_equation_v

`mirror_equation_v` — operation `mirror equation v`. Category: **em / optics / relativity**.

```
p mirror_equation_v $x
```

mirror_string

`mirror_string` — operation `mirror string`. Category: **string helpers**.

```
p mirror_string $x
```

mirror_symmetry_check

`mirror_symmetry_check` — operation `mirror symmetry check`. Category: **econometrics**.

```
p mirror_symmetry_check $x
```

mish

`mish` applies the Mish activation: $x \cdot \tanh(\text{softplus}(x))$. Often outperforms ReLU/Swish.

```
p mish(1)      # 0.8651
p mish(-1)     # -0.3034
```

mixed_integer_round_down

`mixed_integer_round_down` — operation `mixed integer round down`. Category: **combinatorial optimization, scheduling**.

```
p mixed_integer_round_down $x
```

mixed_integer_round_up

`mixed_integer_round_up` — operation `mixed integer round up`. Category: **combinatorial optimization, scheduling**.

```
p mixed_integer_round_up $x
```

mixed_potential_step

`mixed_potential_step` — single-step update of `mixed potential`. Category: **electrochemistry, batteries, fuel cells**.

```
p mixed_potential_step $x
```

mixed_strategy_2x2

`mixed_strategy_2x2(payload) → {row_strategy, col_strategy}` — Nash equilibrium probabilities for a 2×2 **zero-sum** game given the row player's payoff matrix `[[a,b],[c,d]]`. Returns row's prob of row 1 and col's prob of col 1. For non-zero-sum, both players' payoff matrices are required and this function does not apply.

```
p mixed_strategy_2x2([[1,-1],[-1,1]]) # 0.5/0.5
```

mixed_strategy_value

`mixed_strategy_value` — value of `mixed strategy`. Category: **climate, fluids, atmospheric**.

```
p mixed_strategy_value $x
```

mixing_length_prandtl

`mixing_length_prandtl` — operation `mixing length prandtl`. Category: **climate, fluids, atmospheric**.

```
p mixing_length_prandtl $x
```

mjo_phase_step

`mjo_phase_step` — single-step update of `mjo phase`. Category: **climate, fluids, atmospheric**.

```
p mjo_phase_step $x
```

mkdtemp

`mktempdir(PREFIX="stryke")` — create a fresh temp directory and return its absolute path. NOT auto-deleted.

mktemp

`mktemp(PREFIX="stryke")` — create a fresh temp file under `$TMPDIR` and return its absolute path as a string. File is NOT auto-deleted (caller must `rm` it); use the existing `tempfile()` builtin for auto-cleanup semantics. Wraps the `tempfile` crate.

```
val $p = mktemp("build")          # /tmp/build-AbCdEf
spurt($p, "...")
```

ml_adabelief_step

ml_adabelief_step — ML inference primitive [adabelief step](#). Category: *climate, fluids, atmospheric*.

```
p ml_adabelief_step $x
```

ml_adagrad_step

ml_adagrad_step — ML inference primitive [adagrad step](#). Category: *climate, fluids, atmospheric*.

```
p ml_adagrad_step $x
```

ml_adam_step

ml_adam_step — ML inference primitive [adam step](#). Category: *climate, fluids, atmospheric*.

```
p ml_adam_step $x
```

ml_adamax_step

ml_adamax_step — ML inference primitive [adamax step](#). Category: *climate, fluids, atmospheric*.

```
p ml_adamax_step $x
```

ml_adamw_step

ml_adamw_step — ML inference primitive [adamw step](#). Category: *climate, fluids, atmospheric*.

```
p ml_adamw_step $x
```

ml_amsgrad_step

ml_amsgrad_step — ML inference primitive [amsgrad step](#). Category: *climate, fluids, atmospheric*.

```
p ml_amsgrad_step $x
```

ml_arcface_loss_step

ml_arcface_loss_step — ML inference primitive [arcface loss step](#). Category: *climate, fluids, atmospheric*.

```
p ml_arcface_loss_step $x
```

ml_attention_score

ml_attention_score(q, k) → number — $q \cdot k / \sqrt{d}$. Scaled dot product score for a single QK pair.

ml_attention_score_step

ml_attention_score_step — ML inference primitive [attention score step](#). Category: *climate, fluids, atmospheric*.

```
p ml_attention_score_step $x
```

ml_batch_norm

ml_batch_norm(xs, eps=1e-5) → array — zero-mean unit-variance normalisation.

ml_batch_norm_step

ml_batch_norm_step — ML inference primitive **batch norm step**. Category: *climate, fluids, atmospheric*.

```
p ml_batch_norm_step $x
```

ml_binary_cross_entropy

ml_binary_cross_entropy — ML inference primitive **binary cross entropy**. Category: *climate, fluids, atmospheric*.

```
p ml_binary_cross_entropy $x
```

ml_celu_step

ml_celu_step — ML inference primitive **celu step**. Category: *climate, fluids, atmospheric*.

```
p ml_celu_step $x
```

ml_center_loss_step

ml_center_loss_step — ML inference primitive **center loss step**. Category: *climate, fluids, atmospheric*.

```
p ml_center_loss_step $x
```

ml_ciou_loss_step

ml_ciou_loss_step — ML inference primitive **ciou loss step**. Category: *climate, fluids, atmospheric*.

```
p ml_ciou_loss_step $x
```

ml_constant_init

ml_constant_init — ML inference primitive **constant init**. Category: *climate, fluids, atmospheric*.

```
p ml_constant_init $x
```

ml_contrastive_loss

ml_contrastive_loss — ML inference primitive **contrastive loss**. Category: *climate, fluids, atmospheric*.

```
p ml_contrastive_loss $x
```

ml_cosine_lr_schedule

ml_cosine_lr_schedule — ML inference primitive **cosine lr schedule**. Category: *climate, fluids, atmospheric*.

```
p ml_cosine_lr_schedule $x
```

ml_cross_entropy_loss

ml_cross_entropy_loss — ML inference primitive **cross entropy loss**. Category: *climate, fluids, atmospheric*.

```
p ml_cross_entropy_loss $x
```

ml_cutmix_box_iou

ml_cutmix_box_iou — ML inference primitive **cutmix box iou**. Category: *climate, fluids, atmospheric*.

```
p ml_cutmix_box_iou $x
```

ml_cyclic_lr_step

ml_cyclic_lr_step — ML inference primitive `cyclic lr step`. Category: *climate, fluids, atmospheric*.

```
p ml_cyclic_lr_step $x
```

ml_dice_loss_step

ml_dice_loss_step — ML inference primitive `dice loss step`. Category: *climate, fluids, atmospheric*.

```
p ml_dice_loss_step $x
```

ml_diou_loss_step

ml_diou_loss_step — ML inference primitive `diou loss step`. Category: *climate, fluids, atmospheric*.

```
p ml_diou_loss_step $x
```

ml_dot_product_attention

ml_dot_product_attention(`q`, `keys`, `values`) → `array` — softmax(scores)·values where scores = $q \cdot k_i / \sqrt{d}$ for each key.

```
p ml_dot_product_attention([1,0], [[1,0],[0,1]], [[10,20],[30,40]])
```

ml_dropout_mask

ml_dropout_mask(`n`, `p=0.5`, `seed=0`) → `array` — Bernoulli mask of length `n`; live values scaled by $1/(1-p)$ for inverted dropout.

ml_dropout_mask_prob

ml_dropout_mask_prob — ML inference primitive `dropout mask prob`. Category: *climate, fluids, atmospheric*.

```
p ml_dropout_mask_prob $x
```

ml_elu_layer

ml_elu_layer(`xs`, `alpha=1`) → `array` — ELU: x if $x > 0$ else $\alpha \cdot (e^x - 1)$.

ml_elu_step

ml_elu_step — ML inference primitive `elu step`. Category: *climate, fluids, atmospheric*.

```
p ml_elu_step $x
```

ml_eos_logit_boost

ml_eos_logit_boost — ML inference primitive `eos logit boost`. Category: *climate, fluids, atmospheric*.

```
p ml_eos_logit_boost $x
```

ml_exponential_lr

ml_exponential_lr — ML inference primitive `exponential lr`. Category: *climate, fluids, atmospheric*.

```
p ml_exponential_lr $x
```

ml_focal_loss_step

ml_focal_loss_step — ML inference primitive **focal loss step**. Category: *climate, fluids, atmospheric*.

```
p ml_focal_loss_step $x
```

ml_geglu_step

ml_geglu_step — ML inference primitive **geglu step**. Category: *climate, fluids, atmospheric*.

```
p ml_geglu_step $x
```

ml_gelu_layer

ml_gelu_layer(xs) → array — Gaussian Error Linear Unit using the tanh approximation.

ml_gelu_step

ml_gelu_step — ML inference primitive **gelu step**. Category: *climate, fluids, atmospheric*.

```
p ml_gelu_step $x
```

ml_giou_loss_step

ml_giou_loss_step — ML inference primitive **giou loss step**. Category: *climate, fluids, atmospheric*.

```
p ml_giou_loss_step $x
```

ml_glorot_init_value

ml_glorot_init_value — ML inference primitive **glorot init value**. Category: *climate, fluids, atmospheric*.

```
p ml_glorot_init_value $x
```

ml_glu_step

ml_glu_step — ML inference primitive **glu step**. Category: *climate, fluids, atmospheric*.

```
p ml_glu_step $x
```

ml_gradient_accumulate

ml_gradient_accumulate — ML inference primitive **gradient accumulate**. Category: *climate, fluids, atmospheric*.

```
p ml_gradient_accumulate $x
```

ml_gradient.centralize

ml_gradient.centralize — ML inference primitive **gradient centralize**. Category: *climate, fluids, atmospheric*.

```
p ml_gradient.centralize $x
```

ml_gradient_clip_norm

ml_gradient_clip_norm — ML inference primitive **gradient clip norm**. Category: *climate, fluids, atmospheric*.

```
p ml_gradient_clip_norm $x
```

ml_gradient_clip_value

ml_gradient_clip_value — ML inference primitive **gradient clip value**. Category: *climate, fluids, atmospheric*.

```
p ml_gradient_clip_value $x
```

ml_group_norm_step

ml_group_norm_step — ML inference primitive **group norm step**. Category: *climate, fluids, atmospheric*.

```
p ml_group_norm_step $x
```

ml_hard_sigmoid

ml_hard_sigmoid — ML inference primitive **hard sigmoid**. Category: *climate, fluids, atmospheric*.

```
p ml_hard_sigmoid $x
```

ml_hard_tanh

ml_hard_tanh — ML inference primitive **hard tanh**. Category: *climate, fluids, atmospheric*.

```
p ml_hard_tanh $x
```

ml_he_init_value

ml_he_init_value — ML inference primitive **he init value**. Category: *climate, fluids, atmospheric*.

```
p ml_he_init_value $x
```

ml_hinge_loss

ml_hinge_loss(pred, target) → number — $\text{mean}(\max(0, 1 - t \cdot p))$. For SVM-style classifiers.

ml_huber_loss

ml_huber_loss(pred, target, delta=1) → number — smooth quadratic-then-linear loss with breakpoint **delta**.

ml_huber_loss_step

ml_huber_loss_step — ML inference primitive **huber loss step**. Category: *climate, fluids, atmospheric*.

```
p ml_huber_loss_step $x
```

ml_id_to_label_step

ml_id_to_label_step — convert from **ml id** to **label step**. Category: *climate, fluids, atmospheric*.

```
p ml_id_to_label_step $value
```

ml_instance_norm_step

ml_instance_norm_step — ML inference primitive **instance norm step**. Category: *climate, fluids, atmospheric*.

```
p ml_instance_norm_step $x
```

ml_inverse_sqrt_lr

ml_inverse_sqrt_lr — ML inference primitive `inverse_sqrt_lr`. Category: *climate, fluids, atmospheric*.

```
p ml_inverse_sqrt_lr $x
```

ml_iou_loss_step

ml_iou_loss_step — ML inference primitive `iou_loss_step`. Category: *climate, fluids, atmospheric*.

```
p ml_iou_loss_step $x
```

ml_kaiming_init

ml_kaiming_init — ML inference primitive `kaiming_init`. Category: *climate, fluids, atmospheric*.

```
p ml_kaiming_init $x
```

ml_kl_div_loss

ml_kl_div_loss(*p*, *q*) → *number* — discrete KL divergence $\sum p \cdot \ln(p/q)$. Always ≥ 0 ; zero when $p == q$.

ml_kl_divergence_loss

ml_kl_divergence_loss — ML inference primitive `kl_divergence_loss`. Category: *climate, fluids, atmospheric*.

```
p ml_kl_divergence_loss $x
```

ml_l2_normalize_step

ml_l2_normalize_step — ML inference primitive `l2_normalize_step`. Category: *climate, fluids, atmospheric*.

```
p ml_l2_normalize_step $x
```

ml_label_smooth

ml_label_smooth(*probs*, *smoothing*=0.1) → *array* — $(1-\alpha) \cdot p + \alpha/K$ smoothing for classification targets.

ml_label_smoothing

ml_label_smoothing — ML inference primitive `label_smoothing`. Category: *climate, fluids, atmospheric*.

```
p ml_label_smoothing $x
```

ml_label_to_id

ml_label_to_id — convert from `ml_label` to `id`. Category: *climate, fluids, atmospheric*.

```
p ml_label_to_id $value
```

ml_lamb_step

ml_lamb_step — ML inference primitive `lamb_step`. Category: *climate, fluids, atmospheric*.

```
p ml_lamb_step $x
```


ml_lars_step

ml_lars_step — ML inference primitive **lars step**. Category: *climate, fluids, atmospheric*.

```
p ml_lars_step $x
```

ml_layer_norm

ml_layer_norm — ML inference primitive **layer norm**. Category: *test runner*.

```
p ml_layer_norm $x
```

ml_layer_norm_step

ml_layer_norm_step — ML inference primitive **layer norm step**. Category: *climate, fluids, atmospheric*.

```
p ml_layer_norm_step $x
```

ml_leaky_relu_layer

ml_leaky_relu_layer(xs, slope=0.01) → array — x if $x > 0$ else $\text{slope} \cdot x$.

ml_leaky_relu_step

ml_leaky_relu_step — ML inference primitive **leaky relu step**. Category: *climate, fluids, atmospheric*.

```
p ml_leaky_relu_step $x
```

ml_lecun_init_value

ml_lecun_init_value — ML inference primitive **lecun init value**. Category: *climate, fluids, atmospheric*.

```
p ml_lecun_init_value $x
```

ml_lion_step

ml_lion_step — ML inference primitive **lion step**. Category: *climate, fluids, atmospheric*.

```
p ml_lion_step $x
```

ml_log_sigmoid

ml_log_sigmoid — ML inference primitive **log sigmoid**. Category: *climate, fluids, atmospheric*.

```
p ml_log_sigmoid $x
```

ml_log_softmax_step

ml_log_softmax_step — ML inference primitive **log softmax step**. Category: *climate, fluids, atmospheric*.

```
p ml_log_softmax_step $x
```

ml_logsumexp_step

ml_logsumexp_step — ML inference primitive **logsumexp step**. Category: *climate, fluids, atmospheric*.

```
p ml_logsumexp_step $x
```

ml_lookahead_step

ml_lookahead_step — ML inference primitive [lookahead_step](#). Category: *climate, fluids, atmospheric*.

```
p ml_lookahead_step $x
```

ml_mae_loss

ml_mae_loss(pred, target) → number — mean absolute error.

ml_mish_layer

ml_mish_layer(xs) → array — $x \cdot \tanh(\text{softplus}(x))$.

ml_mish_step

ml_mish_step — ML inference primitive [mish_step](#). Category: *climate, fluids, atmospheric*.

```
p ml_mish_step $x
```

ml_mixup_lambda

ml_mixup_lambda — ML inference primitive [mixup_lambda](#). Category: *climate, fluids, atmospheric*.

```
p ml_mixup_lambda $x
```

ml_momentum_step

ml_momentum_step — ML inference primitive [momentum_step](#). Category: *climate, fluids, atmospheric*.

```
p ml_momentum_step $x
```

ml_mse_loss

ml_mse_loss(pred, target) → number — mean squared error.

ml_multihead_avg

ml_multihead_avg — ML inference primitive [multihead_avg](#). Category: *climate, fluids, atmospheric*.

```
p ml_multihead_avg $x
```

ml_nadam_step

ml_nadam_step — ML inference primitive [nadam_step](#). Category: *climate, fluids, atmospheric*.

```
p ml_nadam_step $x
```

ml_nesterov_momentum

ml_nesterov_momentum — ML inference primitive [nesterov_momentum](#). Category: *climate, fluids, atmospheric*.

```
p ml_nesterov_momentum $x
```

ml_nucleus_sample_p

ml_nucleus_sample_p — ML inference primitive **nucleus sample p**. Category: *climate, fluids, atmospheric*.

```
p ml_nucleus_sample_p $x
```

ml_one_cycle_lr

ml_one_cycle_lr — ML inference primitive **one cycle lr**. Category: *climate, fluids, atmospheric*.

```
p ml_one_cycle_lr $x
```

ml_one_hot_encode

ml_one_hot_encode(idx, n_classes=10) → array — vector of zeros with 1 at position **idx**.

```
p ml_one_hot_encode(3, 5) # [0,0,0,1,0]
```

ml_one_hot_index

ml_one_hot_index — ML inference primitive **one hot index**. Category: *climate, fluids, atmospheric*.

```
p ml_one_hot_index $x
```

ml_orthogonal_init

ml_orthogonal_init — ML inference primitive **orthogonal init**. Category: *climate, fluids, atmospheric*.

```
p ml_orthogonal_init $x
```

ml_polynomial_lr

ml_polynomial_lr — ML inference primitive **polynomial lr**. Category: *climate, fluids, atmospheric*.

```
p ml_polynomial_lr $x
```

ml_position_encoding

ml_position_encoding(seq_len, dim) → matrix — Transformer sinusoidal position encoding.

ml_prelu_step

ml_prelu_step — ML inference primitive **prelu step**. Category: *climate, fluids, atmospheric*.

```
p ml_prelu_step $x
```

ml_radam_step

ml_radam_step — ML inference primitive **radam step**. Category: *climate, fluids, atmospheric*.

```
p ml_radam_step $x
```

ml_random_erasing_step

ml_random_erasing_step — ML inference primitive **random erasing step**. Category: *climate, fluids, atmospheric*.

```
p ml_random_erasing_step $x
```

ml_relu_layer

`ml_relu_layer(xs)` → array — $\max(0, x)$.

ml_relu_step

`ml_relu_step` — ML inference primitive `relu step`. Category: *climate, fluids, atmospheric*.

```
p ml_relu_step $x
```

ml_repetition_penalty

`ml_repetition_penalty` — ML inference primitive `repetition penalty`. Category: *climate, fluids, atmospheric*.

```
p ml_repetition_penalty $x
```

ml_rms_norm_step

`ml_rms_norm_step` — ML inference primitive `rms norm step`. Category: *climate, fluids, atmospheric*.

```
p ml_rms_norm_step $x
```

ml_rmsprop_step

`ml_rmsprop_step` — ML inference primitive `rmsprop step`. Category: *climate, fluids, atmospheric*.

```
p ml_rmsprop_step $x
```

ml_scaled_dot_product

`ml_scaled_dot_product` — ML inference primitive `scaled dot product`. Category: *climate, fluids, atmospheric*.

```
p ml_scaled_dot_product $x
```

ml_self_attention

`ml_self_attention(seq_matrix)` → matrix — single-head self-attention $\text{softmax}(X \cdot X^T / \sqrt{d}) \cdot X$ (no learned weights).

ml_selu_step

`ml_selu_step` — ML inference primitive `selu step`. Category: *climate, fluids, atmospheric*.

```
p ml_selu_step $x
```

ml_sgd_step

`ml_sgd_step` — ML inference primitive `sgd step`. Category: *climate, fluids, atmospheric*.

```
p ml_sgd_step $x
```

ml_shampoo_step

`ml_shampoo_step` — ML inference primitive `shampoo step`. Category: *climate, fluids, atmospheric*.

```
p ml_shampoo_step $x
```

ml_sigmoid_layer

`ml_sigmoid_layer(xs)` → array — elementwise $1/(1+e^{-x})$.

```
p ml_sigmoid_layer([0, 1, -1])
```

ml_silu_step

`ml_silu_step` — ML inference primitive `silu step`. Category: *climate, fluids, atmospheric*.

```
p ml_silu_step $x
```

ml_smooth_l1_loss

`ml_smooth_l1_loss` — ML inference primitive `smooth l1 loss`. Category: *climate, fluids, atmospheric*.

```
p ml_smooth_l1_loss $x
```

ml_softmax_layer

`ml_softmax_layer(xs)` → array — max-stable softmax (subtracts max before exp). Sums to 1.

```
p ml_softmax_layer([1,2,3]) # sums to 1
```

ml_softmax_temperature

`ml_softmax_temperature` — ML inference primitive `softmax temperature`. Category: *climate, fluids, atmospheric*.

```
p ml_softmax_temperature $x
```

ml_softplus_layer

`ml_softplus_layer(xs)` → array — $\ln(1+e^x)$. Smooth relu.

ml_softplus_step

`ml_softplus_step` — ML inference primitive `softplus step`. Category: *climate, fluids, atmospheric*.

```
p ml_softplus_step $x
```

ml_softsign_step

`ml_softsign_step` — ML inference primitive `softsign step`. Category: *climate, fluids, atmospheric*.

```
p ml_softsign_step $x
```

ml_sophia_step

`ml_sophia_step` — ML inference primitive `sophia step`. Category: *climate, fluids, atmospheric*.

```
p ml_sophia_step $x
```

ml_spectral_norm_step

`ml_spectral_norm_step` — ML inference primitive `spectral norm step`. Category: *climate, fluids, atmospheric*.

```
p ml_spectral_norm_step $x
```

ml_step_lr_schedule

ml_step_lr_schedule — ML inference primitive `step lr schedule`. Category: **climate, fluids, atmospheric**.

```
p ml_step_lr_schedule $x
```

ml_swiglu_step

ml_swiglu_step — ML inference primitive `swiglu step`. Category: **climate, fluids, atmospheric**.

```
p ml_swiglu_step $x
```

ml_swish_layer

ml_swish_layer(xs) → array — $x \cdot \text{sigmoid}(x)$.

ml_swish_step

ml_swish_step — ML inference primitive `swish step`. Category: **climate, fluids, atmospheric**.

```
p ml_swish_step $x
```

ml_tanh_layer

ml_tanh_layer(xs) → array — elementwise `tanh`.

ml_temperature_decay

ml_temperature_decay — ML inference primitive `temperature decay`. Category: **climate, fluids, atmospheric**.

```
p ml_temperature_decay $x
```

ml_to_cups

ml_to_cups — convert from `ml` to `cups`. Category: **file stat / path**.

```
p ml_to_cups $value
```

ml_to_liters

ml_to_liters — convert from `ml` to `liters`. Category: **file stat / path**.

```
p ml_to_liters $value
```

ml_token_logit_top_k

ml_token_logit_top_k — ML inference primitive `token logit top k`. Category: **climate, fluids, atmospheric**.

```
p ml_token_logit_top_k $x
```

ml_topk_argmax

ml_topk_argmax — ML inference primitive `topk argmax`. Category: **climate, fluids, atmospheric**.

```
p ml_topk_argmax $x
```

ml_triplet_loss_step

ml_triplet_loss_step — ML inference primitive `triplet_loss_step`. Category: *climate, fluids, atmospheric*.

```
p ml_triplet_loss_step $x
```

ml_truncnormal_init

ml_truncnormal_init — ML inference primitive `truncnormal_init`. Category: *climate, fluids, atmospheric*.

```
p ml_truncnormal_init $x
```

ml_uniform_init

ml_uniform_init — ML inference primitive `uniform_init`. Category: *climate, fluids, atmospheric*.

```
p ml_uniform_init $x
```

ml_warmup_lr_step

ml_warmup_lr_step — ML inference primitive `warmup_lr_step`. Category: *climate, fluids, atmospheric*.

```
p ml_warmup_lr_step $x
```

ml_weight_decay_step

ml_weight_decay_step — ML inference primitive `weight_decay_step`. Category: *climate, fluids, atmospheric*.

```
p ml_weight_decay_step $x
```

ml_weight_norm_step

ml_weight_norm_step — ML inference primitive `weight_norm_step`. Category: *climate, fluids, atmospheric*.

```
p ml_weight_norm_step $x
```

ml_xavier_init_value

ml_xavier_init_value — ML inference primitive `xavier_init_value`. Category: *climate, fluids, atmospheric*.

```
p ml_xavier_init_value $x
```

ml_yogi_step

ml_yogi_step — ML inference primitive `yogi_step`. Category: *climate, fluids, atmospheric*.

```
p ml_yogi_step $x
```

ml_zero_init

ml_zero_init — ML inference primitive `zero_init`. Category: *climate, fluids, atmospheric*.

```
p ml_zero_init $x
```

mle_exponential_log_lik

mle_exponential_log_lik — operation `mle_exponential_log_lik`. Category: *econometrics*.

```
p mle_exponential_log_lik $x
```

mle_normal_log_lik

mle_normal_log_lik — operation `mle normal log lik`. Category: **econometrics**.

```
p mle_normal_log_lik $x
```

mle_poisson_log_lik

mle_poisson_log_lik — operation `mle poisson log lik`. Category: **econometrics**.

```
p mle_poisson_log_lik $x
```

mm

mm — operation `mm`. Alias for `matrix_mul`. Category: **extended stdlib**.

```
p mm $x
```

mmap

mmap — operation `mmap`. Alias for `matrix_map`. Category: **matrix operations extras**.

```
p mmap $x
```

mmax

mmax — operation `mmax`. Alias for `matrix_max`. Category: **matrix operations extras**.

```
p mmax $x
```

mmhg_to_pa

mmhg_to_pa — convert from `mmhg` to `pa`. Category: **astronomy / music / color / units**.

```
p mmhg_to_pa $value
```

mmin

mmin — operation `mmin`. Alias for `matrix_min`. Category: **matrix operations extras**.

```
p mmin $x
```

mmult

mmult — operation `mmult`. Alias for `matrix_mult`. Category: **extended stdlib**.

```
p mmult $x
```

mmx

mmx — operation `mmx`. Alias for `minmax`. Category: **python/ruby stdlib**.

```
p mmx $x
```

mmxb

mmxb — operation `mmxb`. Alias for `minmax_by`. Category: **python/ruby stdlib**.

```
p mmxb $x
```


mob

mob — operation **mob**. Alias for **mobius**. Category: **extended stdlib**.

```
p mob $x
```

mobius

mobius — operation **mobius**. Category: **extended stdlib**.

```
p mobius $x
```

mobius_function_pair

mobius_function_pair — operation **mobius function pair**. Category: **econometrics**.

```
p mobius_function_pair $x
```

mobius_inversion_step

mobius_inversion_step — single-step update of **mobius inversion**. Category: **econometrics**.

```
p mobius_inversion_step $x
```

mobius_poset_two

mobius_poset_two — operation **mobius poset two**. Category: **econometrics**.

```
p mobius_poset_two $x
```

mod_dur

modified_duration(\$face, \$coupon_rate, \$ytm, \$periods) (alias **mod_dur**) — computes modified duration, which directly measures price sensitivity to yield changes. Equal to Macaulay duration / (1 + ytm).

```
p mod_dur(1000, 0.05, 0.05, 10) # ~7.3
```

mod_exp

mod_exp (aliases **modexp**, **powmod**) computes modular exponentiation: $\text{base}^{\text{exp}} \bmod m$, using fast binary exponentiation.

```
p powmod(2, 10, 1000) # 24 (2^10 mod 1000)
p powmod(3, 13, 50)   # 7
```

mod_inv

mod_inv (alias **modinv**) computes the modular multiplicative inverse via extended Euclidean algorithm. Errors if no inverse exists.

```
p modinv(3, 7) # 5 (3*5 ≡ 1 mod 7)
```

mod_op

mod_op — modular-arithmetic helper **op**. Category: **trig / math**.

```
p mod_op $x
```

mod_sph_bessel_i0

`mod_sph_bessel_i0` — modular-arithmetic helper `sph_bessel i0`. Category: *special functions extra*.

```
p mod_sph_bessel_i0 $x
```

mod_sph_bessel_i1

`mod_sph_bessel_i1` — modular-arithmetic helper `sph_bessel i1`. Category: *special functions extra*.

```
p mod_sph_bessel_i1 $x
```

mod_sph_bessel_k0

`mod_sph_bessel_k0` — modular-arithmetic helper `sph_bessel k0`. Category: *special functions extra*.

```
p mod_sph_bessel_k0 $x
```

moddur

`moddur` — operation `moddur`. Alias for `modified_duration`. Category: *finance (extended)*.

```
p moddur $x
```

mode_agg

`mode_agg` — operation `mode_agg`. Category: *postgres sql strings, json, aggregates*.

```
p mode_agg $x
```

mode_iter

`mode_iter` — operation `mode_iter`. Category: *iterator + string-distance extras*.

```
p mode_iter $x
```

mode_list

`mode_list` — operation `mode_list`. Category: *list helpers*.

```
p mode_list $x
```

mode_pitches_dorian

`mode_pitches_dorian` — operation `mode_pitches_dorian`. Category: *music theory*.

```
p mode_pitches_dorian $x
```

mode_pitches_lydian

`mode_pitches_lydian` — operation `mode_pitches_lydian`. Category: *music theory*.

```
p mode_pitches_lydian $x
```

mode_pitches_phrygian

`mode_pitches_phrygian` — operation `mode_pitches_phrygian`. Category: *music theory*.

```
p mode_pitches_phrygian $x
```

mode_stat

`mode_stat` — operation `mode stat`. Category: **extended stdlib**.

```
p mode_stat $x
```

mode_val

`mode_val` — operation `mode val`. Category: **trivial numeric / predicate builtins**.

```
p mode_val $x
```

model_checking_ctl

`model_checking_ctl` — operation `model checking ctl`. Category: **logic, proof, sat/smt, type theory**.

```
p model_checking_ctl $x
```

model_checking_ltl

`model_checking_ltl` — operation `model checking ltl`. Category: **logic, proof, sat/smt, type theory**.

```
p model_checking_ltl $x
```

modhex_encode

`modhex_encode` — encode of `modhex`. Category: **archive/encoding format primitives**.

```
p modhex_encode $x
```

modified_irr

`modified_irr` — operation `modified irr`. Category: **misc**.

```
p modified_irr $x
```

modified_julian_date

`modified_julian_date(unix) → number` — $MJD = 40587 + unix/86400$. MJD epoch is 1858-11-17.

```
p modified_julian_date(0) # 40587
```

modified_midpoint_ode

`modified_midpoint_ode` — operation `modified midpoint ode`. Category: **more extensions (2)**.

```
p modified_midpoint_ode $x
```

modified_sharpe

`modified_sharpe` — operation `modified sharpe`. Category: **financial pricing models**.

```
p modified_sharpe $x
```

modinverse

`modinverse` — operation `modinverse`. Category: **math / number theory extras**.

```
p modinverse $x
```

modmidpoint

`modmidpoint` — operation `modmidpoint`. Alias for `modified_midpoint_ode`. Category: **more extensions (2)**.

```
p modmidpoint $x
```

modpow

`modpow` — operation `modpow`. Category: **math / number theory extras**.

```
p modpow $x
```

modular_data_s_value

`modular_data_s_value` — value of `modular data s`. Category: **econometrics**.

```
p modular_data_s_value $x
```

modular_data_t_value

`modular_data_t_value` — value of `modular data t`. Category: **econometrics**.

```
p modular_data_t_value $x
```

modular_sqrt

`modular_sqrt` — operation `modular sqrt`. Category: **math / number theory extras**.

```
p modular_sqrt $x
```

modularity_q

`modularity_q` — operation `modularity q`. Category: **more extensions**.

```
p modularity_q $x
```

modularity_score

`modularity_score` — score of `modularity`. Category: **networkx graph algorithms**.

```
p modularity_score $x
```

moduli_dimension_curves

`moduli_dimension_curves` — operation `moduli dimension curves`. Category: **econometrics**.

```
p moduli_dimension_curves $x
```

molality

`molality` — operation `molality`. Category: **misc**.

```
p molality $x
```

molarity

`molarity` — operation `molarity`. Category: **misc**.

```
p molarity $x
```

molarity_dilution

molarity_dilution — operation `molarity dilution`. Category: **chemistry & biochemistry**.

```
p molarity_dilution $x
```

mole_fraction

mole_fraction — operation `mole fraction`. Category: **misc**.

```
p mole_fraction $x
```

molecular_weight_compound

molecular_weight_compound — operation `molecular weight compound`. Category: **chemistry & biochemistry**.

```
p molecular_weight_compound $x
```

moles_from_mass

moles_from_mass — operation `moles from mass`. Category: **chemistry**.

```
p moles_from_mass $x
```

mollweide_project

mollweide_project — operation `mollweide project`. Category: **more extensions**.

```
p mollweide_project $x
```

moment_image

moment_image(image, p, q) → number — $\sum x^p \cdot y^q \cdot I(x,y)$. Raw image moments.

moment_magnitude_mw

moment_magnitude_mw — operation `moment magnitude mw`. Category: **geology, seismology, mineralogy**.

```
p moment_magnitude_mw $x
```

moment_of_inertia_cylinder

moment_of_inertia_cylinder(mass, radius) → number — $\frac{1}{2} \cdot m \cdot r^2$ about cylinder axis.

moment_of_inertia_disc

moment_of_inertia_disc(mass, radius) → number — $\frac{1}{2} \cdot m \cdot r^2$ about disc axis.

moment_of_inertia_rod

moment_of_inertia_rod(mass, length) → number — $m \cdot L^2 / 12$ about centroid perpendicular axis.

moment_of_inertia_sphere

moment_of_inertia_sphere(mass, radius) → number — $(2/5) \cdot m \cdot r^2$ for solid sphere.

momentum

`momentum($mass, $velocity)` — compute momentum $p = mv$ (kg·m/s).

```
p momentum(10, 5) # 50 kg·m/s
```

mones

`mones` — operation `mones`. Alias for `matrix_ones`. Category: *extended stdlib*.

```
p mones $x
```

money_add

`money_add(a, b) → number` — add two dollar amounts by converting each to integer cents via banker's rounding, summing in i64 (saturating), then dividing by 100. Avoids accumulated float drift.

money_compare

`money_compare(a, b) → -1|0|1` — three-way compare after converting both to integer cents (banker's rounding). Avoids float-equality pitfalls.

money_div

`money_div(amount, divisor) → number` — divide amount (in cents) by `divisor`, then round half-to-even back to cents. UNDEF on `divisor == 0`.

money_mul

`money_mul(amount, factor) → number` — multiply amount (in cents) by `factor`, then round half-to-even (banker's rounding) back to cents.

money_sub

`money_sub(a, b) → number` — subtract dollar amounts in integer cents with banker's rounding on input.

monin_obukhov_length

`monin_obukhov_length` — operation `monin obukhov length`. Category: *climate, fluids, atmospheric*.

```
p monin_obukhov_length $x
```

monobit_test

`monobit_test` — statistical test of `monobit`. Category: *cryptography deep*.

```
p monobit_test $x
```

monopoly_lerner

`monopoly_lerner` — operation `monopoly lerner`. Category: *economics + game theory*.

```
p monopoly_lerner $x
```

montecarlo_returns_step

`montecarlo_returns_step` — single-step update of `montecarlo_returns`. Category: *electrochemistry, batteries, fuel cells*.

```
p montecarlo_returns_step $x
```

month_name

`month_name` — operation `month_name`. Category: *date helpers*.

```
p month_name $x
```

month_short

`month_short` — operation `month_short`. Category: *date helpers*.

```
p month_short $x
```

months_between

`months_between` — operation `months_between`. Category: *misc*.

```
p months_between $x
```

montreal_protocol_track

`montreal_protocol_track` — operation `montreal_protocol_track`. Category: *climate, fluids, atmospheric*.

```
p montreal_protocol_track $x
```

moon_age_days

`moon_age_days(unix)` → `number` — days since the most recent new moon (synodic period 29.53 days).

```
p moon_age_days(time())
```

moon_distance_km

`moon_distance_km(unix)` → `number` — approximate Earth-Moon distance in km (sinusoidal around mean).

```
p moon_distance_km(time())
```

moon_phase

`moon_phase(unix)` → `number` — 0..1 cycle position: 0 new, 0.25 first quarter, 0.5 full, 0.75 last quarter.

```
p moon_phase(time())
```

moon_phase_age

`moon_phase_age` — lunar astronomy helper `phase_age`. Category: *astronomy / astrometry*.

```
p moon_phase_age $x
```

moon_position_low

`moon_position_low` — lunar astronomy helper `position low`. Category: **astronomy / astrometry**.

```
p moon_position_low $x
```

morphological_gradient

`morphological_gradient(image, size=3)` → `matrix` — dilation - erosion. Highlights edges.

morse

`morse` — operation `morse`. Alias for `morse_encode`. Category: **encoding / phonetics**.

```
p morse $x
```

morse_decode

`morse_decode` — decode of `morse`. Category: **encoding / phonetics**.

```
p morse_decode $x
```

morse_encode

`morse_encode` — encode of `morse`. Category: **encoding / phonetics**.

```
p morse_encode $x
```

mortality_table_q

`mortality_table_q` — operation `mortality table q`. Category: **actuarial science**.

```
p mortality_table_q $x
```

mortgage_payment

`mortgage_payment` — operation `mortgage payment`. Category: **physics formulas**.

```
p mortgage_payment $x
```

most_common

`most_common` — operation `most common`. Category: **list helpers**.

```
p most_common $x
```

motion_blur_kernel

`motion_blur_kernel` — convolution kernel of `motion blur`. Category: **more extensions**.

```
p motion_blur_kernel $x
```

motor_torque

`motor_torque` — operation `motor torque`. Category: **misc**.

```
p motor_torque $x
```


mott_schottky_capacitance

`mott_schottky_capacitance` — operation `mott schottky capacitance`. Category: **electrochemistry, batteries, fuel cells**.

```
p mott_schottky_capacitance $x
```

movavg

`movavg` — operation `movavg`. Alias for `moving_average`. Category: **math / numeric extras**.

```
p movavg $x
```

moving_average

`moving_average` — operation `moving average`. Category: **math / numeric extras**.

```
p moving_average $x
```

moving_average_filter

`moving_average_filter` — filter op of `moving average`. Category: **statsmodels**.

```
p moving_average_filter $x
```

mpeg_quant_value

`mpeg_quant_value` — value of `mpeg quant`. Category: **electrochemistry, batteries, fuel cells**.

```
p mpeg_quant_value $x
```

mph_to_kmh

`mph_to_kmh` — convert from `mph` to `kmh`. Category: **astronomy / music / color / units**.

```
p mph_to_kmh $value
```

mplanck

`mplanck` — operation `mplanck`. Alias for `planck_mass`. Category: **physics constants**.

```
p mplanck $x
```

mpmc_new

`mpmc_new` — constructor of `mpmc`. Category: **extras**.

```
p mpmc_new $x
```

mpow

`mpow` — operation `mpow`. Alias for `matrix_power`. Category: **matrix operations extras**.

```
p mpow $x
```

mps_to_kmh

`mps_to_kmh` — convert from `mps` to `kmh`. Category: **astronomy / music / color / units**.

```
p mps_to_kmh $value
```

mpsc_new

`mpsc_new` — constructor of `mpsc`. Category: **extras**.

```
p mpsc_new $x
```

mrow

`mrow` — operation `mrow`. Alias for `matrix_row`. Category: **extended stdlib**.

```
p mrow $x
```

ms_to_ns

`ms_to_ns` — convert from `ms` to `ns`. Category: **file stat / path**.

```
p ms_to_ns $value
```

ms_to_s

`ms_to_s` — convert from `ms` to `s`. Category: **file stat / path**.

```
p ms_to_s $value
```

msa_column_entropy

`msa_column_entropy` — operation `msa column entropy`. Category: **bioinformatics deep**.

```
p msa_column_entropy $x
```

mscal

`mscal` — operation `mscal`. Alias for `matrix_scalar`. Category: **extended stdlib**.

```
p mscal $x
```

mse

`mse` — operation `mse`. Category: **math functions**.

```
p mse $x
```

mset

`mset` — operation `mset`. Category: **redis-flavour primitives**.

```
p mset $x
```

msgctl

`msgctl` — operation `msgctl`. Category: **sysv ipc**.

```
p msgctl $x
```

msgget

`msgget` — operation `msgget`. Category: **sysv ipc**.

```
p msgget $x
```

msgpack_pack_int

`msgpack_pack_int` — operation `msgpack pack int`. Category: **archive/encoding format primitives**.

```
p msgpack_pack_int $x
```

msgpack_pack_str

`msgpack_pack_str` — operation `msgpack pack str`. Category: **archive/encoding format primitives**.

```
p msgpack_pack_str $x
```

msgrcv

`msgrcv` — operation `msgrcv`. Category: **sysv ipc**.

```
p msgrcv $x
```

msgsnd

`msgsnd` — operation `msgsnd`. Category: **sysv ipc**.

```
p msgsnd $x
```

mshape

`mshape` — operation `mshape`. Alias for `matrix_shape`. Category: **extended stdlib**.

```
p mshape $x
```

msorted

`msorted` — operation `msorted`. Alias for `merge_sorted`. Category: **array / list operations extras**.

```
p msorted $x
```

mst

`prim_mst` (aliases `mst`, `prim`) computes the minimum spanning tree weight via Prim's algorithm. Takes an adjacency matrix.

```
p mst([[0,2,0],[2,0,3],[0,3,0]]) # 5
```

mstat

`mstat` — operation `mstat`. Alias for `mode_stat`. Category: **extended stdlib**.

```
p mstat $x
```

msub

`msub` — operation `msub`. Alias for `matrix_sub`. Category: **extended stdlib**.

```
p msub $x
```

msum

`msum` — operation `msum`. Alias for `matrix_sum`. Category: **matrix operations extras**.

```
p msum $x
```

msun

`sun_mass` (alias `msun`) — $M_{\odot} \approx 1.989 \times 10^{30}$ kg. Mass of Sun (solar mass).

```
p msun # 1.98892e30 kg
```

mt19937_temper

`mt19937_temper` — operation `mt19937_temper`. Category: **cryptography deep**.

```
p mt19937_temper $x
```

mtf_decode

`mtf_decode` — decode of `mtf`. Category: **more extensions**.

```
p mtf_decode $x
```

mtf_encode

`mtf_encode` — encode of `mtf`. Category: **more extensions**.

```
p mtf_encode $x
```

mtime

`mtime` — operation `mtime`. Alias for `file_mtime`. Category: **file stat / path**.

```
p mtime $x
```

mtrace

`mtrace` — operation `mtrace`. Alias for `matrix_trace`. Category: **extended stdlib**.

```
p mtrace $x
```

mtrans

`mtrans` — operation `mtrans`. Alias for `matrix_transpose`. Category: **matrix operations extras**.

```
p mtrans $x
```

mu0

`mu0` — operation `mu0`. Alias for `vacuum_permeability`. Category: **physics constants**.

```
p mu0 $x
```

mu_low_decode

`mu_low_decode` — decode of `mu_low`. Category: **signal processing**.

```
p mu_low_decode $x
```

mu_low_encode

`mu_low_encode` — encode of `mu_low`. Category: *signal processing*.

```
p mu_low_encode $x
```

mub

`mub` — operation `mub`. Alias for `bohr_magneton`. Category: *physics constants*.

```
p mub $x
```

mulberry32_next

`mulberry32_next(seed) → int` — Mulberry32 PRNG (Tommy Ettinger 2017). Advances state by `+0x602B79F5` then permutes to 32 bits. To iterate, caller tracks state and adds `0x602B79F5` between calls.

muller

`muller` — operation `muller`. Alias for `muller_root`. Category: *more extensions (2)*.

```
p muller $x
```

muller_root

`muller_root` — operation `muller root`. Category: *more extensions (2)*.

```
p muller_root $x
```

multi_decrement_q

`multi_decrement_q` — operation `multi decrement q`. Category: *actuarial science*.

```
p multi_decrement_q $x
```

multi_knapsack_step

`multi_knapsack_step` — single-step update of `multi knapsack`. Category: *combinatorial optimization, scheduling*.

```
p multi_knapsack_step $x
```

multi_state_pij

`multi_state_pij` — operation `multi state pij`. Category: *actuarial science*.

```
p multi_state_pij $x
```

multidim_packing_step

`multidim_packing_step` — single-step update of `multidim packing`. Category: *combinatorial optimization, scheduling*.

```
p multidim_packing_step $x
```

multinom

`multinom` — operation `multinom`. Alias for `multinomial`. Category: *math / numeric extras*.

```
p multinom $x
```

multinomial

`multinomial` — operation `multinomial`. Category: **math / numeric extras**.

```
p multinomial $x
```

multinomial_coefficient

`multinomial_coefficient([k1, k2, ...])` → `number` — $n! / (k1! \cdot k2! \cdot \dots)$. Returns rounded float for large values.

multinomial_count

`multinomial_count` — count of `multinomial`. Alias for `multiset_permutations_count`. Category: **misc**.

```
p multinomial_count $x
```

multinomial_logit_prob

`multinomial_logit_prob` — operation `multinomial logit prob`. Category: **econometrics**.

```
p multinomial_logit_prob $x
```

multipeek

`multipeek` — operation `multipeek`. Category: **iterator + string-distance extras**.

```
p multipeek $x
```

multiset_difference

`multiset_difference` — operation `multiset difference`. Category: **misc**.

```
p multiset_difference $x
```

multiset_intersection

`multiset_intersection` — operation `multiset intersection`. Category: **misc**.

```
p multiset_intersection $x
```

multiset_permutations_count

`multiset_permutations_count` — count of `multiset permutations`. Category: **misc**.

```
p multiset_permutations_count $x
```

multiset_union

`multiset_union` — operation `multiset union`. Category: **misc**.

```
p multiset_union $x
```

mun

`mun` — operation `mun`. Alias for `nuclear_magneton`. Category: **physics constants**.

```
p mun $x
```

murmurhash3_x32

murmurhash3_x32 — operation `murmurhash3 x32`. Category: **misc**.

```
p murmurhash3_x32 $x
```

mutableListOf

mutableListOf — operation `mutableListOf`. Alias for `listOf`. Category: **kotlin collections**.

```
p mutableListOf $x
```

mutableMapOf

mutableMapOf — operation `mutableMapOf`. Alias for `mapOf`. Category: **kotlin collections**.

```
p mutableMapOf $x
```

mutableSetOf

mutableSetOf — operation `mutableSetOf`. Alias for `setOf`. Category: **kotlin collections**.

```
p mutableSetOf $x
```

mutation_bit_flip_prob

mutation_bit_flip_prob — operation `mutation bit flip prob`. Category: **combinatorial optimization, scheduling**.

```
p mutation_bit_flip_prob $x
```

mutation_rate

mutation_rate — rate of `mutation`. Category: **bioinformatics deep**.

```
p mutation_rate $x
```

mutex

`mutex()` → `mutex` — create a fresh, unlocked exclusive mutex. Pair with `mutex_lock` / `mutex_unlock` (or `mutex_try_lock`) to serialize access to shared state. Reference-shared across `async/spawn` blocks; safe under contention via `parking_lot::Condvar` (waiters park, no busy spin).

```
mysync $m = mutex()
mysync $counter = 0
my @t
for (1:10) {
    push @t, async {
        mutex_lock($m)
        defer { mutex_unlock($m) }
        $counter++
    }
}
await($_) for @t
p $counter # 10
```

mutex_is_locked

`mutex_is_locked($m)` → `1|0` — `1` when currently held, `0` when free. Inspection only; no acquire/release side effect.

```
p mutex_is_locked($m)    # 0
mutex_lock($m)
p mutex_is_locked($m)    # 1
mutex_unlock($m)
```

mutex_lock

`mutex_lock($m)` → `undef` — block until the mutex is acquired, marking it held. Use with `defer { mutex_unlock($m) }` for RAIL-style release on scope exit (covers early returns and exceptions).

```
mutex_lock($m)
defer { mutex_unlock($m) }
# critical section - exclusive access until block exit
update_shared_state()
```

mutex_try_lock

`mutex_try_lock($m)` → `1|0` — non-blocking acquire. Returns `1` if the mutex was free (and is now held); `0` if it was already held. Useful for opportunistic work that should be skipped under contention.

```
if (mutex_try_lock($m)) {
  defer { mutex_unlock($m) }
  do_work()
} else {
  p "someone else has it - skipping"
}
```

mutex_unlock

`mutex_unlock($m)` → `undef` — release the lock and wake one waiter (if any). Unlocking a free mutex is a tolerated no-op.

```
mutex_lock($m)
try {
  work()
} finally {
  mutex_unlock($m)
}
```

mutual_information_step

`mutual_information_step` — single-step update of `mutual information`. Category: **electrochemistry, batteries, fuel cells**.

```
p mutual_information_step $x
```

myerson_optimal

`myerson_optimal` — operation `myerson optimal`. Category: **economics + game theory**.

```
p myerson_optimal $x
```

mzeros

`mzeros` — operation `mzeros`. Alias for `matrix_zeros`. Category: **extended stdlib**.

```
p mzeros $x
```

n_ball_volume

`n_ball_volume` — operation `n ball volume`. Category: **geometry / topology**.


```
p n_ball_volume $x
```

n_tuples

`n_tuples` — operation `n_tuples`. Category: *iterator + string-distance extras*.

```
p n_tuples $x
```

nanoid

`nanoid` — operation `nanoid`. Category: *id helpers*.

```
p nanoid $x
```

nao_index_step

`nao_index_step` — single-step update of `nao_index`. Category: *climate, fluids, atmospheric*.

```
p nao_index_step $x
```

narayana_cow

`narayana_cow(n) → int` — Narayana's cow sequence: $a(n) = a(n-1) + a(n-3)$.

nash_bargaining_solution

`nash_bargaining_solution` — operation `nash bargaining solution`. Category: *climate, fluids, atmospheric*.

```
p nash_bargaining_solution $x
```

nash_equilibrium_pair

`nash_equilibrium_pair` — operation `nash equilibrium pair`. Category: *climate, fluids, atmospheric*.

```
p nash_equilibrium_pair $x
```

nash_social_welfare

`nash_social_welfare` — operation `nash social welfare`. Category: *climate, fluids, atmospheric*.

```
p nash_social_welfare $x
```

nato

`nato` — operation `nato`. Alias for `nato_phonetic`. Category: *encoding / phonetics*.

```
p nato $x
```

nato_phonetic

`nato_phonetic` — operation `nato phonetic`. Category: *encoding / phonetics*.

```
p nato_phonetic $x
```

nb_bernoulli_likelihood

`nb_bernoulli_likelihood` — operation `nb bernoulli likelihood`. Category: **ml extensions**.

```
p nb_bernoulli_likelihood $x
```

nb_gaussian_likelihood

`nb_gaussian_likelihood` — operation `nb gaussian likelihood`. Category: **ml extensions**.

```
p nb_gaussian_likelihood $x
```

nb_multinomial_log_likelihood

`nb_multinomial_log_likelihood` — operation `nb multinomial log likelihood`. Category: **ml extensions**.

```
p nb_multinomial_log_likelihood $x
```

nbe_normalize

`nbe_normalize` — operation `nbe normalize`. Category: **logic, proof, sat/smt, type theory**.

```
p nbe_normalize $x
```

ncol

`ncol` — number of columns in a matrix.

```
p ncol([[1,2,3],[4,5,6]]) # 3
```

ndivs

`ndivs` — operation `ndivs`. Alias for `num_divisors`. Category: **math / numeric extras**.

```
p ndivs $x
```

ne_from_variance

`ne_from_variance` — operation `ne from variance`. Category: **bioinformatics deep**.

```
p ne_from_variance $x
```

nearest_insertion_step

`nearest_insertion_step` — single-step update of `nearest insertion`. Category: **combinatorial optimization, scheduling**.

```
p nearest_insertion_step $x
```

nearest_neighbor_tour_step

`nearest_neighbor_tour_step` — single-step update of `nearest neighbor tour`. Category: **combinatorial optimization, scheduling**.

```
p nearest_neighbor_tour_step $x
```

necklace_count

`necklace_count` — count of `necklace`. Category: **misc**.

```
p necklace_count $x
```

needleman_wunsch_score

`needleman_wunsch_score` — score of `needleman` `wunsch`. Category: **bioinformatics deep**.

```
p needleman_wunsch_score $x
```

neg_inf

`neg_inf` — operation `neg inf`. Category: **math / numeric extras**.

```
p neg_inf $x
```

neg_n

`neg_n` — operation `neg n`. Alias for `negative`. Category: **trivial numeric / predicate builtins**.

```
p neg_n $x
```

negate

`negate` — operation `negate`. Category: **trivial numeric / predicate builtins**.

```
p negate $x
```

negate_each

`negate_each` — operation `negate each`. Category: **trivial numeric helpers**.

```
p negate_each $x
```

negative

`negative` — operation `negative`. Category: **trivial numeric / predicate builtins**.

```
p negative $x
```

nei_genetic_distance

`nei_genetic_distance` — distance / dissimilarity metric of `nei genetic`. Category: **biology / ecology**.

```
p nei_genetic_distance $x
```

neither

`neither` — operation `neither`. Category: **boolean combinators**.

```
p neither $x
```

nelder_mead_contract

`nelder_mead_contract` — operation `nelder mead contract`. Category: **more extensions (2)**.

```
p nelder_mead_contract $x
```

nelder_mead_expand

nelder_mead_expand — operation `nelder mead expand`. Category: **more extensions (2)**.

```
p nelder_mead_expand $x
```

nelder_mead_reflect

nelder_mead_reflect — operation `nelder mead reflect`. Category: **more extensions (2)**.

```
p nelder_mead_reflect $x
```

nemhauser_wolsey_bound

nemhauser_wolsey_bound — operation `nemhauser wolsey bound`. Category: **combinatorial optimization, scheduling**.

```
p nemhauser_wolsey_bound $x
```

ner_bilou_decode

ner_bilou_decode — decode of `ner bilou`. Category: **computational linguistics**.

```
p ner_bilou_decode $x
```

nernst_diffusion_layer

nernst_diffusion_layer — operation `nernst diffusion layer`. Category: **electrochemistry, batteries, fuel cells**.

```
p nernst_diffusion_layer $x
```

nernst_einstein_diffusivity

nernst_einstein_diffusivity — operation `nernst einstein diffusivity`. Category: **electrochemistry, batteries, fuel cells**.

```
p nernst_einstein_diffusivity $x
```

nernst_planck_flux

nernst_planck_flux — operation `nernst planck flux`. Category: **electrochemistry, batteries, fuel cells**.

```
p nernst_planck_flux $x
```

nernst_potential_full

nernst_potential_full — operation `nernst potential full`. Category: **electrochemistry, batteries, fuel cells**.

```
p nernst_potential_full $x
```

nerve_complex_count

nerve_complex_count — count of `nerve complex`. Category: **econometrics**.

```
p nerve_complex_count $x
```

nesterov_accelerate_step

nesterov_accelerate_step — single-step update of `nesterov accelerate`. Category: **combinatorial optimization, scheduling**.

```
p nesterov_accelerate_step $x
```

nesterov_step

nesterov_step — single-step update of **nesterov**. Category: *more extensions (2)*.

```
p nesterov_step $x
```

net_aiad_step

net_aiad_step — single-step update of **net aiad**. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aiad_step $x
```

net_aimd_step

net_aimd_step — single-step update of **net aimd**. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aimd_step $x
```

net_alohanet_throughput

net_alohanet_throughput — operation **net alohanet throughput**. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_alohanet_throughput $x
```

net_alpha_fair_step

net_alpha_fair_step — single-step update of **net alpha fair**. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_alpha_fair_step $x
```

net_amc_threshold_index

net_amc_threshold_index — index of **net amc threshold**. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_amc_threshold_index $x
```

net_aqm_blue_step

net_aqm_blue_step — single-step update of **net aqm blue**. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_blue_step $x
```

net_aqm_choke_step

net_aqm_choke_step — single-step update of **net aqm choke**. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_choke_step $x
```

net_aqm_codel_target

net_aqm_codel_target — operation **net aqm codel target**. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_codel_target $x
```

net_aqm_drr_step

`net_aqm_drr_step` — single-step update of `net aqm drr`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_drr_step $x
```

net_aqm_fq_codel_step

`net_aqm_fq_codel_step` — single-step update of `net aqm fq codel`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_fq_codel_step $x
```

net_aqm_pie_drop_rate

`net_aqm_pie_drop_rate` — rate of `net aqm pie drop`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_pie_drop_rate $x
```

net_aqm_red_drop_prob

`net_aqm_red_drop_prob` — operation `net aqm red drop prob`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_red_drop_prob $x
```

net_aqm_sfq_step

`net_aqm_sfq_step` — single-step update of `net aqm sfq`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_sfq_step $x
```

net_aqm_wrr_step

`net_aqm_wrr_step` — single-step update of `net aqm wrr`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_aqm_wrr_step $x
```

net_array_gain

`net_array_gain` — operation `net array gain`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_array_gain $x
```

net_backpressure_step

`net_backpressure_step` — single-step update of `net backpressure`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_backpressure_step $x
```

net_bandwidth_delay_product

`net_bandwidth_delay_product` — operation `net bandwidth delay product`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_bandwidth_delay_product $x
```

net_bcjr_step

`net_bcjr_step` — single-step update of `net bcjr`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_bcjr_step $x
```

net_blocking_probability

`net_blocking_probability` — operation `net blocking probability`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_blocking_probability $x
```

net_burst_size_compute

`net_burst_size_compute` — operation `net burst size compute`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_burst_size_compute $x
```

net_call_admission_check

`net_call_admission_check` — operation `net call admission check`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_call_admission_check $x
```

net_capacity_shannon

`net_capacity_shannon` — operation `net capacity shannon`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_capacity_shannon $x
```

net_coding_gain

`net_coding_gain` — operation `net coding gain`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_coding_gain $x
```

net_csma_back_off

`net_csma_back_off` — operation `net csma back off`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_csma_back_off $x
```

net_csma_efficiency

`net_csma_efficiency` — operation `net csma efficiency`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_csma_efficiency $x
```

net_diversity_gain

`net_diversity_gain` — operation `net diversity gain`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_diversity_gain $x
```

net_doppler_shift

`net_doppler_shift` — operation `net doppler shift`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_doppler_shift $x
```

net_drop_tail_check

`net_drop_tail_check` — operation `net drop tail check`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_drop_tail_check $x
```

net_engset_formula

`net_engset_formula` — operation `net engset formula`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_engset_formula $x
```

net_erlang_b_formula

`net_erlang_b_formula` — operation `net erlang b formula`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_erlang_b_formula $x
```

net_erlang_c_formula

`net_erlang_c_formula` — operation `net erlang c formula`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_erlang_c_formula $x
```

net_forced_flow_law

`net_forced_flow_law` — operation `net forced flow law`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_forced_flow_law $x
```

net_friis_received_power

`net_friis_received_power` — operation `net friis received power`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_friis_received_power $x
```

net_handoff_threshold

`net_handoff_threshold` — operation `net handoff threshold`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_handoff_threshold $x
```

net_harq_combining_gain

`net_harq_combining_gain` — operation `net harq combining gain`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_harq_combining_gain $x
```


net_jitter_estimate

`net_jitter_estimate` — operation `net jitter estimate`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_jitter_estimate $x
```

net_kelly_pricing_step

`net_kelly_pricing_step` — single-step update of `net kelly pricing`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_kelly_pricing_step $x
```

net_latency_avg

`net_latency_avg` — operation `net latency avg`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_latency_avg $x
```

net_ldpc_iteration_step

`net_ldpc_iteration_step` — single-step update of `net ldpc iteration`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_ldpc_iteration_step $x
```

net_link_capacity_share

`net_link_capacity_share` — operation `net link capacity share`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_link_capacity_share $x
```

net_little_law_l

`net_little_law_l` — operation `net little law l`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_little_law_l $x
```

net_log_distance_path

`net_log_distance_path` — operation `net log distance path`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_log_distance_path $x
```

net_loss_rate_to_throughput

`net_loss_rate_to_throughput` — convert from `net loss rate` to `throughput`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_loss_rate_to_throughput $value
```

net_lyapunov_drift_plus_penalty

`net_lyapunov_drift_plus_penalty` — operation `net lyapunov drift plus penalty`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_lyapunov_drift_plus_penalty $x
```

net_macro_diversity_step

`net_macro_diversity_step` — single-step update of `net macro diversity`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_macro_diversity_step $x
```

net_max_min_fair_step

`net_max_min_fair_step` — single-step update of `net max min fair`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_max_min_fair_step $x
```

net_max_weight_match

`net_max_weight_match` — operation `net max weight match`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_max_weight_match $x
```

net_miad_step

`net_miad_step` — single-step update of `net miad`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_miad_step $x
```

net_micro_diversity_step

`net_micro_diversity_step` — single-step update of `net micro diversity`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_micro_diversity_step $x
```

net_mimd_step

`net_mimd_step` — single-step update of `net mimd`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_mimd_step $x
```

net_mimo_capacity_step

`net_mimo_capacity_step` — single-step update of `net mimo capacity`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_mimo_capacity_step $x
```

net_mmse_beam_step

`net_mmse_beam_step` — single-step update of `net mmse beam`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_mmse_beam_step $x
```

net_multiplexing_gain

`net_multiplexing_gain` — operation `net multiplexing gain`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_multiplexing_gain $x
```

net_network_utility_max

`net_network_utility_max` — maximum of `net network utility`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_network_utility_max $x
```

net_okumura_hata_loss

`net_okumura_hata_loss` — operation `net okumura hata loss`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_okumura_hata_loss $x
```

net_outage_probability

`net_outage_probability` — operation `net outage probability`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_outage_probability $x
```

net_packet_loss_estimate

`net_packet_loss_estimate` — operation `net packet loss estimate`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_packet_loss_estimate $x
```

net_packet_pacing_step

`net_packet_pacing_step` — single-step update of `net packet pacing`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_packet_pacing_step $x
```

net_path_capacity_kleinrock

`net_path_capacity_kleinrock` — operation `net path capacity kleinrock`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_path_capacity_kleinrock $x
```

net_polar_decode_step

`net_polar_decode_step` — single-step update of `net polar decode`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_polar_decode_step $x
```

net_polling_efficiency

`net_polling_efficiency` — operation `net polling efficiency`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_polling_efficiency $x
```

net_present_value

`net_present_value(cashflows, rate)` → number — $\sum cf[t] / (1+rate)^t$. Standard NPV for project valuation; first cashflow is at $t=0$.

```
p net_present_value([-1000, 300, 400, 500], 0.1)
```

net_priority_queue_index

`net_priority_queue_index` — index of `net priority queue`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_priority_queue_index $x
```

net_proportional_fair_share

`net_proportional_fair_share` — operation `net proportional fair share`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_proportional_fair_share $x
```

net_pruning_gain

`net_pruning_gain` — operation `net pruning gain`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_pruning_gain $x
```

net_qcsma_propose

`net_qcsma_propose` — operation `net qcsma propose`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_qcsma_propose $x
```

net_radio_path_loss

`net_radio_path_loss` — operation `net radio path loss`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_radio_path_loss $x
```

net_rayleigh_envelope

`net_rayleigh_envelope` — operation `net rayleigh envelope`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_rayleigh_envelope $x
```

net_reproductive_rate

`net_reproductive_rate` — rate of `net reproductive`. Category: *biology / ecology*.

```
p net_reproductive_rate $x
```

net_response_time_law

`net_response_time_law` — operation `net response time law`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_response_time_law $x
```

net_rician_k_factor

`net_rician_k_factor` — factor of `net rician k`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_rician_k_factor $x
```

net_router_buffer_size

`net_router_buffer_size` — operation `net router buffer size`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_router_buffer_size $x
```

net_rto_compute

`net_rto_compute` — operation `net rto compute`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_rto_compute $x
```

net_rtt_smoothed

`net_rtt_smoothed` — operation `net rtt smoothed`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_rtt_smoothed $x
```

net_rtt_variation

`net_rtt_variation` — operation `net rtt variation`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_rtt_variation $x
```

net_shadowing_normal

`net_shadowing_normal` — operation `net shadowing normal`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_shadowing_normal $x
```

net_slotted_aloha_throughput

`net_slotted_aloha_throughput` — operation `net slotted aloha throughput`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_slotted_aloha_throughput $x
```

net_tcp_bbr_step

`net_tcp_bbr_step` — single-step update of `net tcp bbr`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_bbr_step $x
```

net_tcp_compound_step

`net_tcp_compound_step` — single-step update of `net tcp compound`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_compound_step $x
```

net_tcp_cubic_step

`net_tcp_cubic_step` — single-step update of `net tcp cubic`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_cubic_step $x
```

net_tcp_cwnd_step

`net_tcp_cwnd_step` — single-step update of `net tcp cwnd`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_cwnd_step $x
```

net_tcp_dctcp_step

`net_tcp_dctcp_step` — single-step update of `net tcp dctcp`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_dctcp_step $x
```

net_tcp_htcp_step

`net_tcp_htcp_step` — single-step update of `net tcp htcp`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_htcp_step $x
```

net_tcp_hybla_step

`net_tcp_hybla_step` — single-step update of `net tcp hybla`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_hybla_step $x
```

net_tcp_illinois_step

`net_tcp_illinois_step` — single-step update of `net tcp illinois`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_illinois_step $x
```

net_tcp_lp_step

`net_tcp_lp_step` — single-step update of `net tcp lp`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_lp_step $x
```

net_tcp_reno_step

`net_tcp_reno_step` — single-step update of `net tcp reno`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_reno_step $x
```

net_tcp_scalable_step

`net_tcp_scalable_step` — single-step update of `net tcp scalable`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_scalable_step $x
```

net_tcp_ssthresh_update

`net_tcp_ssthresh_update` — update step of `net tcp ssthresh`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_ssthresh_update $x
```

net_tcp_vegas_step

`net_tcp_vegas_step` — single-step update of `net tcp vegas`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_vegas_step $x
```

net_tcp_veno_step

`net_tcp_veno_step` — single-step update of `net tcp veno`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_veno_step $x
```

net_tcp_westwood_step

`net_tcp_westwood_step` — single-step update of `net tcp westwood`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_westwood_step $x
```

net_tcp_yeah_step

`net_tcp_yeah_step` — single-step update of `net tcp yeah`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_tcp_yeah_step $x
```

net_throughput_law

`net_throughput_law` — operation `net throughput law`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_throughput_law $x
```

net_throughput_mathis

`net_throughput_mathis` — operation `net throughput mathis`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_throughput_mathis $x
```

net_throughput_padhye

`net_throughput_padhye` — operation `net throughput padhye`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_throughput_padhye $x
```

net_throughput_response

`net_throughput_response` — operation `net throughput response`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_throughput_response $x
```

net_token_rate_limit

`net_token_rate_limit` — operation `net token rate limit`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_token_rate_limit $x
```

net_token_ring_efficiency

`net_token_ring_efficiency` — operation `net token ring efficiency`. Category: *networking — tcp, aqm, mimo, queueing*.

```
p net_token_ring_efficiency $x
```

net_traffic_shaper_step

`net_traffic_shaper_step` — single-step update of `net traffic shaper`. Category: **networking — tcp, aqm, mimo, queueing**.

```
p net_traffic_shaper_step $x
```

net_turbo_decode_iter

`net_turbo_decode_iter` — operation `net turbo decode iter`. Category: **networking — tcp, aqm, mimo, queueing**.

```
p net_turbo_decode_iter $x
```

net_two_ray_ground_loss

`net_two_ray_ground_loss` — operation `net two ray ground loss`. Category: **networking — tcp, aqm, mimo, queueing**.

```
p net_two_ray_ground_loss $x
```

net_utilization_law

`net_utilization_law` — operation `net utilization law`. Category: **networking — tcp, aqm, mimo, queueing**.

```
p net_utilization_law $x
```

net_viterbi_step

`net_viterbi_step` — single-step update of `net viterbi`. Category: **networking — tcp, aqm, mimo, queueing**.

```
p net_viterbi_step $x
```

net_water_filling_power

`net_water_filling_power` — operation `net water filling power`. Category: **networking — tcp, aqm, mimo, queueing**.

```
p net_water_filling_power $x
```

net_zero_forcing_beam

`net_zero_forcing_beam` — operation `net zero forcing beam`. Category: **networking — tcp, aqm, mimo, queueing**.

```
p net_zero_forcing_beam $x
```

netmask_to_prefix

`netmask_to_prefix` — convert from `netmask` to `prefix`. Category: **network / ip / cidr**.

```
p netmask_to_prefix $value
```

network_formation_step

`network_formation_step` — single-step update of `network formation`. Category: **climate, fluids, atmospheric**.

```
p network_formation_step $x
```


neutron_mass

`neutron_mass` — operation `neutron mass`. Category: **physics constants**.

```
p neutron_mass $x
```

new_moon_julian

`new_moon_julian(cycle)` → `number` — Julian Date of `cycle`-th new moon since the anchor (2000-01-06).

```
p new_moon_julian(0)
```

newey_west_se

`newey_west_se` — operation `newey west se`. Category: **econometrics**.

```
p newey_west_se $x
```

newmark_beta_step

`newmark_beta_step` — single-step update of `newmark beta`. Category: **ode advanced**.

```
p newmark_beta_step $x
```

newton

`newton_method` (aliases `newton`, `newton_raphson`) finds a root via Newton-Raphson. Takes `f`, `f'`, `x0`, and optional tolerance.

```
val $root = newton(fn { _0**2 - 2 }, fn { 2*_0 }, 1.5) # √2
```

newtons_to_lbf

`newtons_to_lbf` — convert from `newtons` to `lbf`. Category: **file stat / path**.

```
p newtons_to_lbf $value
```

next_permutation

`next_permutation` — operation `next permutation`. Category: **array / list operations extras**.

```
p next_permutation $x
```

next_prime

`next_prime` — operation `next prime`. Category: **number theory / primes**.

```
p next_prime $x
```

nfa_to_dfa

`nfa_to_dfa` — convert from `nfa` to `dfa`. Category: **compilers / parsing**.

```
p nfa_to_dfa $value
```

nfirfirst

`nfirfirst` — operation `nfirfirst`. Category: **algebraic match**.

```
p nfirfirst $x
```

nfs

`nfs` — operation `nfs`. Alias for `nfirst`. Category: **algebraic match**.

```
p nfs $x
```

ngram

`ngram` — operation `ngram`. Alias for `ngrams`. Category: **string processing extras**.

```
p ngram $x
```

ngram_perplexity

`ngram_perplexity(model, tokens, n=2)` → `number` — perplexity over a held-out sequence.

ngram_prob

`ngram_prob(model, context, next_token)` → `number` — conditional probability $P(\text{next} \mid \text{context})$. Returns 0 for unseen.

ngram_top_k_next

`ngram_top_k_next(model, context, k=5)` → `array` — top-k most likely next tokens with probabilities.

```
p ngram_top_k_next($model, "the", 5)
```

ngram_train

`ngram_train(tokens, n=2)` → `hash` — context → next-token probability hash. Maximum-likelihood estimate.

```
my $model = ngram_train(["the","cat","sat","the","cat"], 2)
```

ngrams

`ngrams` — operation `ngrams`. Category: **string processing extras**.

```
p ngrams $x
```

nice

`nice(PRIORITY=10)` — set this process's priority adjustment via `libc::setpriority`. Range `[-20, 19]` (negative is higher priority; requires `CAP_SYS_NICE` / root to set). Returns the new priority on success or `undef` on permission denial. Standard Unix `nice` semantics.

```
nice(5) # be polite
```

nichols_chart_step

`nichols_chart_step` — single-step update of `nichols chart`. Category: **robotics & control**.

```
p nichols_chart_step $x
```

nilpotency_class_lower

`nilpotency_class_lower` — operation `nilpotency class lower`. Category: **econometrics**.

```
p nilpotency_class_lower $x
```

nlp_alibi_position_bias

nlp_alibi_position_bias — operation `nlp alibi position bias`. Category: *climate, fluids, atmospheric*.

```
p nlp_alibi_position_bias $x
```

nlp_attention_pool_step

nlp_attention_pool_step — single-step update of `nlp attention pool`. Category: *climate, fluids, atmospheric*.

```
p nlp_attention_pool_step $x
```

nlp_avg_pool_step

nlp_avg_pool_step — single-step update of `nlp avg pool`. Category: *climate, fluids, atmospheric*.

```
p nlp_avg_pool_step $x
```

nlp_bigbird_step

nlp_bigbird_step — single-step update of `nlp bigbird`. Category: *climate, fluids, atmospheric*.

```
p nlp_bigbird_step $x
```

nlp_bits_per_character

nlp_bits_per_character — operation `nlp bits per character`. Category: *climate, fluids, atmospheric*.

```
p nlp_bits_per_character $x
```

nlp_bleu_score_n

nlp_bleu_score_n — operation `nlp bleu score n`. Category: *climate, fluids, atmospheric*.

```
p nlp_bleu_score_n $x
```

nlp_block_attn_step

nlp_block_attn_step — single-step update of `nlp block attn`. Category: *climate, fluids, atmospheric*.

```
p nlp_block_attn_step $x
```

nlp_bm25_score

nlp_bm25_score — score of `nlp bm25`. Category: *climate, fluids, atmospheric*.

```
p nlp_bm25_score $x
```

nlp_byte_frequency

nlp_byte_frequency — operation `nlp byte frequency`. Category: *climate, fluids, atmospheric*.

```
p nlp_byte_frequency $x
```

nlp_byte_level_bpe_step

nlp_byte_level_bpe_step — single-step update of `nlp byte level bpe`. Category: *climate, fluids, atmospheric*.

```
p nlp_byte_level_bpe_step $x
```

nlp_byte_pair_merge_step

`nlp_byte_pair_merge_step` — single-step update of `nlp byte pair merge`. Category: *climate, fluids, atmospheric*.

```
p nlp_byte_pair_merge_step $x
```

nlp_cer_score

`nlp_cer_score` — score of `nlp cer`. Category: *climate, fluids, atmospheric*.

```
p nlp_cer_score $x
```

nlp_char_frequency

`nlp_char_frequency` — operation `nlp char frequency`. Category: *climate, fluids, atmospheric*.

```
p nlp_char_frequency $x
```

nlp_char_ngram_count

`nlp_char_ngram_count` — count of `nlp char ngram`. Category: *climate, fluids, atmospheric*.

```
p nlp_char_ngram_count $x
```

nlp_chi2_collocation

`nlp_chi2_collocation` — operation `nlp chi2 collocation`. Category: *climate, fluids, atmospheric*.

```
p nlp_chi2_collocation $x
```

nlp_chrf_score

`nlp_chrf_score` — score of `nlp chrf`. Category: *climate, fluids, atmospheric*.

```
p nlp_chrf_score $x
```

nlp_cosine_similarity_two

`nlp_cosine_similarity_two` — operation `nlp cosine similarity two`. Category: *climate, fluids, atmospheric*.

```
p nlp_cosine_similarity_two $x
```

nlp_cross_attn_compute_step

`nlp_cross_attn_compute_step` — single-step update of `nlp cross attn compute`. Category: *climate, fluids, atmospheric*.

```
p nlp_cross_attn_compute_step $x
```

nlp_damerau_levenshtein

`nlp_damerau_levenshtein` — operation `nlp damerau levenshtein`. Category: *climate, fluids, atmospheric*.

```
p nlp_damerau_levenshtein $x
```

nlp_dice_coefficient_two

`nlp_dice_coefficient_two` — operation `nlp dice coefficient two`. Category: *climate, fluids, atmospheric*.

```
p nlp_dice_coefficient_two $x
```

nlp_digit_ratio

nlp_digit_ratio — ratio of `nlp digit`. Category: *climate, fluids, atmospheric*.

```
p nlp_digit_ratio $x
```

nlp_dilated_attn_step

nlp_dilated_attn_step — single-step update of `nlp dilated attn`. Category: *climate, fluids, atmospheric*.

```
p nlp_dilated_attn_step $x
```

nlp_dirichlet_smoothing

nlp_dirichlet_smoothing — operation `nlp dirichlet smoothing`. Category: *climate, fluids, atmospheric*.

```
p nlp_dirichlet_smoothing $x
```

nlp_doc_embedding_avg

nlp_doc_embedding_avg — operation `nlp doc embedding avg`. Category: *climate, fluids, atmospheric*.

```
p nlp_doc_embedding_avg $x
```

nlp_doc_freq_step

nlp_doc_freq_step — single-step update of `nlp doc freq`. Category: *climate, fluids, atmospheric*.

```
p nlp_doc_freq_step $x
```

nlp_dunning_log_likelihood

nlp_dunning_log_likelihood — operation `nlp dunning log likelihood`. Category: *climate, fluids, atmospheric*.

```
p nlp_dunning_log_likelihood $x
```

nlp_email_count

nlp_email_count — count of `nlp email`. Category: *climate, fluids, atmospheric*.

```
p nlp_email_count $x
```

nlp_emoji_ratio

nlp_emoji_ratio — ratio of `nlp emoji`. Category: *climate, fluids, atmospheric*.

```
p nlp_emoji_ratio $x
```

nlp_fasttext_subword_count

nlp_fasttext_subword_count — count of `nlp fasttext subword`. Category: *climate, fluids, atmospheric*.

```
p nlp_fasttext_subword_count $x
```

nlp_global_attn_step

nlp_global_attn_step — single-step update of `nlp_global_attn`. Category: *climate, fluids, atmospheric*.

```
p nlp_global_attn_step $x
```

nlp_glove_loss_step

nlp_glove_loss_step — single-step update of `nlp_glove_loss`. Category: *climate, fluids, atmospheric*.

```
p nlp_glove_loss_step $x
```

nlp_good_turing_count

nlp_good_turing_count — count of `nlp_good_turing`. Category: *climate, fluids, atmospheric*.

```
p nlp_good_turing_count $x
```

nlp_hamming_distance

nlp_hamming_distance — distance / dissimilarity metric of `nlp_hamming`. Category: *climate, fluids, atmospheric*.

```
p nlp_hamming_distance $x
```

nlp_hashtag_count

nlp_hashtag_count — count of `nlp_hashtag`. Category: *climate, fluids, atmospheric*.

```
p nlp_hashtag_count $x
```

nlp_inverse_doc_freq

nlp_inverse_doc_freq — operation `nlp_inverse_doc_freq`. Category: *climate, fluids, atmospheric*.

```
p nlp_inverse_doc_freq $x
```

nlp_jaccard_similarity_two

nlp_jaccard_similarity_two — operation `nlp_jaccard_similarity_two`. Category: *climate, fluids, atmospheric*.

```
p nlp_jaccard_similarity_two $x
```

nlp_jaro_distance

nlp_jaro_distance — distance / dissimilarity metric of `nlp_jaro`. Category: *climate, fluids, atmospheric*.

```
p nlp_jaro_distance $x
```

nlp_jaro_winkler

nlp_jaro_winkler — operation `nlp_jaro_winkler`. Category: *climate, fluids, atmospheric*.

```
p nlp_jaro_winkler $x
```

nlp_jelinek_merger

nlp_jelinek_merger — operation `nlp_jelinek_merger`. Category: *climate, fluids, atmospheric*.

```
p nlp_jelinek_merger $x
```

nlp_kl_lm_div

nlp_kl_lm_div — operation `nlp kl lm div`. Category: *climate, fluids, atmospheric*.

```
p nlp_kl_lm_div $x
```

nlp_kneser_ney_step

nlp_kneser_ney_step — single-step update of `nlp kneser ney`. Category: *climate, fluids, atmospheric*.

```
p nlp_kneser_ney_step $x
```

nlp_laplace_smoothing

nlp_laplace_smoothing — operation `nlp laplace smoothing`. Category: *climate, fluids, atmospheric*.

```
p nlp_laplace_smoothing $x
```

nlp_lcs_length

nlp_lcs_length — operation `nlp lcs length`. Category: *climate, fluids, atmospheric*.

```
p nlp_lcs_length $x
```

nlp_lcs_ratio

nlp_lcs_ratio — ratio of `nlp lcs`. Category: *climate, fluids, atmospheric*.

```
p nlp_lcs_ratio $x
```

nlp_lda_alpha_step

nlp_lda_alpha_step — single-step update of `nlp lda alpha`. Category: *climate, fluids, atmospheric*.

```
p nlp_lda_alpha_step $x
```

nlp_lda_beta_step

nlp_lda_beta_step — single-step update of `nlp lda beta`. Category: *climate, fluids, atmospheric*.

```
p nlp_lda_beta_step $x
```

nlp_lda_topic_dist

nlp_lda_topic_dist — operation `nlp lda topic dist`. Category: *climate, fluids, atmospheric*.

```
p nlp_lda_topic_dist $x
```

nlp_levenshtein_dist

nlp_levenshtein_dist — operation `nlp levenshtein dist`. Category: *climate, fluids, atmospheric*.

```
p nlp_levenshtein_dist $x
```

nlp_lidstone_smoothing

nlp_lidstone_smoothing — operation `nlp lidstone smoothing`. Category: *climate, fluids, atmospheric*.

```
p nlp_lidstone_smoothing $x
```

nlp_linformer_step

`nlp_linformer_step` — single-step update of `nlp linformer`. Category: *climate, fluids, atmospheric*.

```
p nlp_linformer_step $x
```

nlp_local_attn_step

`nlp_local_attn_step` — single-step update of `nlp local attn`. Category: *climate, fluids, atmospheric*.

```
p nlp_local_attn_step $x
```

nlp_loglikelihood_collocation

`nlp_loglikelihood_collocation` — operation `nlp loglikelihood collocation`. Category: *climate, fluids, atmospheric*.

```
p nlp_loglikelihood_collocation $x
```

nlp_longformer_step

`nlp_longformer_step` — single-step update of `nlp longformer`. Category: *climate, fluids, atmospheric*.

```
p nlp_longformer_step $x
```

nlp_max_pool_step

`nlp_max_pool_step` — single-step update of `nlp max pool`. Category: *climate, fluids, atmospheric*.

```
p nlp_max_pool_step $x
```

nlp_max_seq_len_check

`nlp_max_seq_len_check` — operation `nlp max seq len check`. Category: *climate, fluids, atmospheric*.

```
p nlp_max_seq_len_check $x
```

nlp_mention_count

`nlp_mention_count` — count of `nlp mention`. Category: *climate, fluids, atmospheric*.

```
p nlp_mention_count $x
```

nlp_meteor_score

`nlp_meteor_score` — score of `nlp meteor`. Category: *climate, fluids, atmospheric*.

```
p nlp_meteor_score $x
```

nlp_npmi_score

`nlp_npmi_score` — score of `nlp npmi`. Category: *climate, fluids, atmospheric*.

```
p nlp_npmi_score $x
```

nlp_okapi_score

`nlp_okapi_score` — score of `nlp okapi`. Category: *climate, fluids, atmospheric*.


```
p nlp_okapi_score $x
```

nlp_overlap_coefficient

nlp_overlap_coefficient — operation `nlp overlap coefficient`. Category: *climate, fluids, atmospheric*.

```
p nlp_overlap_coefficient $x
```

nlp_pe_freq_band

nlp_pe_freq_band — operation `nlp pe freq band`. Category: *climate, fluids, atmospheric*.

```
p nlp_pe_freq_band $x
```

nlp_performer_step

nlp_performer_step — single-step update of `nlp performer`. Category: *climate, fluids, atmospheric*.

```
p nlp_performer_step $x
```

nlp_perplexity_value

nlp_perplexity_value — value of `nlp perplexity`. Category: *climate, fluids, atmospheric*.

```
p nlp_perplexity_value $x
```

nlp_phone_count

nlp_phone_count — count of `nlp phone`. Category: *climate, fluids, atmospheric*.

```
p nlp_phone_count $x
```

nlp_plsa_step

nlp_plsa_step — single-step update of `nlp plsa`. Category: *climate, fluids, atmospheric*.

```
p nlp_plsa_step $x
```

nlp_pmi_score

nlp_pmi_score — score of `nlp pmi`. Category: *climate, fluids, atmospheric*.

```
p nlp_pmi_score $x
```

nlp_pointwise_attn_score

nlp_pointwise_attn_score — score of `nlp pointwise attn`. Category: *climate, fluids, atmospheric*.

```
p nlp_pointwise_attn_score $x
```

nlp_position_encoding_cos

nlp_position_encoding_cos — operation `nlp position encoding cos`. Category: *climate, fluids, atmospheric*.

```
p nlp_position_encoding_cos $x
```

nlp_position_encoding_sin

nlp_position_encoding_sin — operation `nlp position encoding sin`. Category: *climate, fluids, atmospheric*.

```
p nlp_position_encoding_sin $x
```

nlp_punct_ratio

nlp_punct_ratio — ratio of `nlp punct`. Category: *climate, fluids, atmospheric*.

```
p nlp_punct_ratio $x
```

nlp_query_likelihood_step

nlp_query_likelihood_step — single-step update of `nlp query likelihood`. Category: *climate, fluids, atmospheric*.

```
p nlp_query_likelihood_step $x
```

nlp_reformer_step

nlp_reformer_step — single-step update of `nlp reformer`. Category: *climate, fluids, atmospheric*.

```
p nlp_reformer_step $x
```

nlp_relative_position_bias

nlp_relative_position_bias — operation `nlp relative position bias`. Category: *climate, fluids, atmospheric*.

```
p nlp_relative_position_bias $x
```

nlp_rope_apply_step

nlp_rope_apply_step — single-step update of `nlp rope apply`. Category: *climate, fluids, atmospheric*.

```
p nlp_rope_apply_step $x
```

nlp_rope_rotary_angle

nlp_rope_rotary_angle — operation `nlp rope rotary angle`. Category: *climate, fluids, atmospheric*.

```
p nlp_rope_rotary_angle $x
```

nlp_rouge_score_n

nlp_rouge_score_n — operation `nlp rouge score n`. Category: *climate, fluids, atmospheric*.

```
p nlp_rouge_score_n $x
```

nlp_routing_attn_step

nlp_routing_attn_step — single-step update of `nlp routing attn`. Category: *climate, fluids, atmospheric*.

```
p nlp_routing_attn_step $x
```

nlp_self_attn_compute_step

nlp_self_attn_compute_step — single-step update of `nlp self attn compute`. Category: *climate, fluids, atmospheric*.

```
p nlp_self_attn_compute_step $x
```

nlp_sentencepiece_score

nlp_sentencepiece_score — score of nlp sentencepiece. Category: *climate, fluids, atmospheric*.

```
p nlp_sentencepiece_score $x
```

nlp_sif_weight_step

nlp_sif_weight_step — single-step update of nlp sif weight. Category: *climate, fluids, atmospheric*.

```
p nlp_sif_weight_step $x
```

nlp_simpson_coefficient

nlp_simpson_coefficient — operation nlp simpson coefficient. Category: *climate, fluids, atmospheric*.

```
p nlp_simpson_coefficient $x
```

nlp_skip_gram_count

nlp_skip_gram_count — count of nlp skip gram. Category: *climate, fluids, atmospheric*.

```
p nlp_skip_gram_count $x
```

nlp_sliding_window_step

nlp_sliding_window_step — single-step update of nlp sliding window. Category: *climate, fluids, atmospheric*.

```
p nlp_sliding_window_step $x
```

nlp_sparse_attn_score

nlp_sparse_attn_score — score of nlp sparse attn. Category: *climate, fluids, atmospheric*.

```
p nlp_sparse_attn_score $x
```

nlp_strided_attn_step

nlp_strided_attn_step — single-step update of nlp strided attn. Category: *climate, fluids, atmospheric*.

```
p nlp_strided_attn_step $x
```

nlp_subword_regularization

nlp_subword_regularization — operation nlp subword regularization. Category: *climate, fluids, atmospheric*.

```
p nlp_subword_regularization $x
```

nlp_sum_pool_step

nlp_sum_pool_step — single-step update of nlp sum pool. Category: *climate, fluids, atmospheric*.

```
p nlp_sum_pool_step $x
```

nlp_t_score_collocation

nlp_t_score_collocation — operation `nlp t score collocation`. Category: *climate, fluids, atmospheric*.

```
p nlp_t_score_collocation $x
```

nlp_ter_score

nlp_ter_score — score of `nlp ter`. Category: *climate, fluids, atmospheric*.

```
p nlp_ter_score $x
```

nlp_tf_idf_step

nlp_tf_idf_step — single-step update of `nlp tf idf`. Category: *climate, fluids, atmospheric*.

```
p nlp_tf_idf_step $x
```

nlp_token_drop_rate

nlp_token_drop_rate — rate of `nlp token drop`. Category: *climate, fluids, atmospheric*.

```
p nlp_token_drop_rate $x
```

nlp_token_overlap_two

nlp_token_overlap_two — operation `nlp token overlap two`. Category: *climate, fluids, atmospheric*.

```
p nlp_token_overlap_two $x
```

nlp_unigram_lm_score

nlp_unigram_lm_score — score of `nlp unigram lm`. Category: *climate, fluids, atmospheric*.

```
p nlp_unigram_lm_score $x
```

nlp_unigram_subword_loss

nlp_unigram_subword_loss — operation `nlp unigram subword loss`. Category: *climate, fluids, atmospheric*.

```
p nlp_unigram_subword_loss $x
```

nlp_uppercase_ratio

nlp_uppercase_ratio — ratio of `nlp uppercase`. Category: *climate, fluids, atmospheric*.

```
p nlp_uppercase_ratio $x
```

nlp_url_count

nlp_url_count — count of `nlp url`. Category: *climate, fluids, atmospheric*.

```
p nlp_url_count $x
```

nlp_wer_score

nlp_wer_score — score of `nlp wer`. Category: *climate, fluids, atmospheric*.

```
p nlp_wer_score $x
```

nlp_window_attn_step

nlp_window_attn_step — single-step update of nlp window attn. Category: *climate, fluids, atmospheric*.

```
p nlp_window_attn_step $x
```

nlp_witten_bell_step

nlp_witten_bell_step — single-step update of nlp witten bell. Category: *climate, fluids, atmospheric*.

```
p nlp_witten_bell_step $x
```

nlp_word2vec_cbow_loss

nlp_word2vec_cbow_loss — operation nlp word2vec cbow loss. Category: *climate, fluids, atmospheric*.

```
p nlp_word2vec_cbow_loss $x
```

nlp_word2vec_skipgram_loss

nlp_word2vec_skipgram_loss — operation nlp word2vec skipgram loss. Category: *climate, fluids, atmospheric*.

```
p nlp_word2vec_skipgram_loss $x
```

nlp_word_freq_value

nlp_word_freq_value — value of nlp word freq. Category: *climate, fluids, atmospheric*.

```
p nlp_word_freq_value $x
```

nlp_word_mover_dist

nlp_word_mover_dist — operation nlp word mover dist. Category: *climate, fluids, atmospheric*.

```
p nlp_word_mover_dist $x
```

nlp_word_ngram_count

nlp_word_ngram_count — count of nlp word ngram. Category: *climate, fluids, atmospheric*.

```
p nlp_word_ngram_count $x
```

nlp_wordpiece_score

nlp_wordpiece_score — score of nlp wordpiece. Category: *climate, fluids, atmospheric*.

```
p nlp_wordpiece_score $x
```

nm_from_fst

nm_from_fst — operation nm from fst. Category: *bioinformatics deep*.

```
p nm_from_fst $x
```

nmass

nmass — operation nmass. Alias for neutron_mass. Category: *physics constants*.

```
p nmass $x
```

nne

`nne` — operation `nne`. Alias for `nnext`. Category: **algebraic match**.

```
p nne $x
```

nnext

`nnext` — operation `nnext`. Category: **algebraic match**.

```
p nnext $x
```

nohtml

`nohtml` — operation `nohtml`. Alias for `strip_html`. Category: **string processing extras**.

```
p nohtml $x
```

noise_model_depolarize

`noise_model_depolarize` — procedural noise generator `model depolarize`. Category: **quantum**.

```
p noise_model_depolarize $x
```

nominal

`nominal` — operation `nominal`. Category: **b81-misc-utility**.

```
p nominal $x
```

non_max_suppression

`non_max_suppression(image) → matrix` — suppress non-maximal pixels in each 3×3 window. Used in Canny edge thinning.

nonagonal_number

`nonagonal_number(n) → int` — $n \cdot (7n - 5) / 2$.

none_match

`none_match` — operation `none match`. Category: **python/ruby stdlib**.

```
p none_match $x
```

nonem

`nonem` — operation `nonem`. Alias for `none_match`. Category: **python/ruby stdlib**.

```
p nonem $x
```

nonempty_count

`nonempty_count` — count of `nonempty`. Category: **counters**.

```
p nonempty_count $x
```

nonlinear_dual_step

`nonlinear_dual_step` — single-step update of `nonlinear dual`. Category: *combinatorial optimization, scheduling*.

```
p nonlinear_dual_step $x
```

nonzero

`nonzero` — operation `nonzero`. Category: *trivial numeric / predicate builtins*.

```
p nonzero $x
```

nonzero_count

`nonzero_count` — count of `nonzero`. Category: *numpy + scipy.special*.

```
p nonzero_count $x
```

noop_val

`noop_val` — operation `noop val`. Category: *conversion / utility*.

```
p noop_val $x
```

nop

`nop` — operation `nop`. Category: *conversion / utility*.

```
p nop $x
```

normal_cdf

`normal_cdf($x, $mu, $sigma)` (alias `normcdf`) — cumulative distribution function. Returns probability that a value is $\leq x$.

```
p normcdf(0)           # 0.5 (half below mean)
p normcdf(1.96)        # ~0.975 (95% confidence bound)
```

normal_pdf

`normal_pdf($x, $mu, $sigma)` (alias `normpdf`) — normal/Gaussian distribution probability density function. Default: $\mu=0$, $\sigma=1$ (standard normal).

```
p normpdf(0)           # ~0.399 (peak of standard normal)
p normpdf(0, 100, 15)  # PDF at IQ=100, mean=100, stddev=15
```

normality_chem

`normality_chem` — operation `normality chem`. Category: *misc*.

```
p normality_chem $x
```

normalize_array

`normalize_array` — operation `normalize array`. Category: *array / list operations extras*.

```
p normalize_array $x
```

normalize_list

`normalize_list` — operation `normalize list`. Category: **functional combinators**.

```
p normalize_list $x
```

normalize_range

`normalize_range` — operation `normalize range`. Category: **array / list operations extras**.

```
p normalize_range $x
```

normalize_signal

`normalize_signal(\@signal)` (alias `normsig`) — scales signal to range `[-1, 1]` based on its peak absolute value. Useful for audio processing.

```
val $norm = normsig(\@audio)
p max(map { abs } @$norm) # 1
```

normalize_spaces

`normalize_spaces` — operation `normalize spaces`. Category: **extended stdlib**.

```
p normalize_spaces $x
```

normalize_vec

`normalize_vec` — operation `normalize vec`. Category: **extended stdlib**.

```
p normalize_vec $x
```

normalize_vector

`normalize_vector(v) → array` — divide by L2 norm. Zero vector returns zero vector.

normalize_whitespace

`normalize_whitespace` — operation `normalize whitespace`. Category: **string helpers**.

```
p normalize_whitespace $x
```

normarr

`normarr` — operation `normarr`. Alias for `normalize_array`. Category: **array / list operations extras**.

```
p normarr $x
```

not_any

`not_any` — operation `not any`. Category: **algebraic match**.

```
p not_any $x
```

not_each

`not_each` — operation `not each`. Category: **trivial numeric helpers**.

```
p not_each $x
```


not_elem

`not_elem` — operation `not elem`. Category: **haskell list functions**.

```
p not_elem $x
```

not_every

`not_every` — operation `not every`. Category: **algebraic match**.

```
p not_every $x
```

notall

`notall` — operation `notall`. Category: **list / aggregate**.

```
p notall $x
```

notch_filter

`notch_filter` — filter op of `notch`. Category: **economics + game theory**.

```
p notch_filter $x
```

note_name_from_midi

`note_name_from_midi` — musical-note helper `name from midi`. Category: **music theory**.

```
p note_name_from_midi $x
```

note_name_to_midi

`note_name_to_midi` — convert from `note name` to `midi`. Category: **astronomy / music / color / units**.

```
p note_name_to_midi $value
```

note_to_freq

`note_to_freq` — convert from `note` to `freq`. Category: **extras**.

```
p note_to_freq $value
```

now

`now` — Unix epoch seconds. Returns the current time as the number of seconds since `1970-01-01T00:00:00Z`, like Perl's `time()`. Pass a timezone string (e.g. `now("UTC")` or `now("America/New_York")`) to get an ISO-8601 datetime string in that zone instead — that path forwards to `datetime_now_tz`.

```
p now                # 1778262882
p now("UTC")         # 2026-05-08T17:54:42.577935Z
var $start = now
# ... do work ...
printf "elapsed: %d s\n", now - $start
```

now_ms

`now_ms` — operation `now ms`. Category: **now / timestamp**.

```
p now_ms $x
```

now_ns

`now_ns` — operation `now ns`. Category: **now / timestamp**.

```
p now_ns $x
```

now_us

`now_us` — operation `now us`. Category: **now / timestamp**.

```
p now_us $x
```

nper

`nper($rate, $pmt, $pv)` (alias `num_periods`) — computes the number of periods needed to pay off a loan given rate, payment, and principal.

```
p nper(0.05/12, 500, 50000) # ~127 months
```

npv

`npv` — operation `npv`. Category: **math / numeric extras**.

```
p npv $x
```

nrmng

`nrmng` — operation `nrmng`. Alias for `normalize_range`. Category: **array / list operations extras**.

```
p nrmng $x
```

nrmsp

`nrmsp` — operation `nrmsp`. Alias for `normalize_spaces`. Category: **extended stdlib**.

```
p nrmsp $x
```

nrmv

`nrmv` — operation `nrmv`. Alias for `normalize_vec`. Category: **extended stdlib**.

```
p nrmv $x
```

nroot

`nroot` — operation `nroot`. Alias for `nth_root_of`. Category: **extended stdlib**.

```
p nroot $x
```

nrow

`nrow` — number of rows in a matrix.

```
p nrow([[1,2],[3,4],[5,6]]) # 3
```

ns_to_ms

`ns_to_ms` — convert from `ns` to `ms`. Category: **file stat / path**.

```
p ns_to_ms $value
```

ns_to_us

`ns_to_us` — convert from `ns` to `us`. Category: **file stat / path**.

```
p ns_to_us $value
```

ntelm

`ntelm` — operation `ntelm`. Alias for `not_elem`. Category: **haskell list functions**.

```
p ntelm $x
```

nth_largest

`nth_largest` — operation `nth largest`. Category: **misc**.

```
p nth_largest $x
```

nth_prime

`nth_prime` — operation `nth prime`. Category: **number theory / primes**.

```
p nth_prime $x
```

nth_root

`nth_root` — operation `nth root`. Category: **math functions**.

```
p nth_root $x
```

nth_root_of

`nth_root_of` — operation `nth root of`. Category: **extended stdlib**.

```
p nth_root_of $x
```

nth_smallest

`nth_smallest` — operation `nth smallest`. Category: **misc**.

```
p nth_smallest $x
```

nth_word

`nth_word` — operation `nth word`. Category: **file stat / path**.

```
p nth_word $x
```

nthp

`nthp` — operation `nthp`. Alias for `nth_prime`. Category: **number theory / primes**.

```
p nthp $x
```

nub

`nub` — operation `nub`. Category: **haskell list functions**.

```
p nub $x
```

nub_by

`nub_by` — operation `nub by`. Category: **additional missing stdlib functions**.

```
p nub_by $x
```

nub_count

`nub_count` — count of `nub`. Category: **apl/j/k array primitives**.

```
p nub_count $x
```

nub_list

`nub_list` — operation `nub list`. Category: **apl/j/k array primitives**.

```
p nub_list $x
```

nubb

`nubb` — operation `nubb`. Alias for `nub_by`. Category: **additional missing stdlib functions**.

```
p nubb $x
```

nuclear_magneton

`nuclear_magneton` — operation `nuclear magneton`. Category: **physics constants**.

```
p nuclear_magneton $x
```

nucleation_rate_constant

`nucleation_rate_constant` — operation `nucleation rate constant`. Category: **electrochemistry, batteries, fuel cells**.

```
p nucleation_rate_constant $x
```

nucleolus_lp_step

`nucleolus_lp_step` — single-step update of `nucleolus lp`. Category: **climate, fluids, atmospheric**.

```
p nucleolus_lp_step $x
```

num_cpus

`num_cpus` — operation `num cpus`. Category: **system introspection**.

```
p num_cpus $x
```

num_divisors

`num_divisors` — operation `num divisors`. Category: **math / numeric extras**.

```
p num_divisors $x
```

num_edges

num_edges — operation `num edges`. Category: **graph algorithms**.

```
p num_edges $x
```

num_unrooted_trees

num_unrooted_trees — operation `num unrooted trees`. Category: **bioinformatics deep**.

```
p num_unrooted_trees $x
```

number_needed_to_treat

number_needed_to_treat — convert from `number needed` to `treat`. Category: **epidemiology / public health**.

```
p number_needed_to_treat $value
```

number_to_english

number_to_english — convert from `number` to `english`. Category: **astronomy / music / color / units**.

```
p number_to_english $value
```

numeric_count

numeric_count — count of `numeric`. Category: **counters**.

```
p numeric_count $x
```

numerical_jacobian_col

numerical_jacobian_col — operation `numerical jacobian col`. Category: **ode advanced**.

```
p numerical_jacobian_col $x
```

nusselt_dittus_boelter

nusselt_dittus_boelter — operation `nusselt dittus boelter`. Category: **misc**.

```
p nusselt_dittus_boelter $x
```

nusselt_full_number

nusselt_full_number — operation `nusselt full number`. Category: **climate, fluids, atmospheric**.

```
p nusselt_full_number $x
```

nutatation_iau2000a

nutatation_iau2000a — operation `nutatation iau2000a`. Category: **astronomy / astrometry**.

```
p nutatation_iau2000a $x
```

nuttall_w

`nuttall_w` — operation `nuttall w`. Category: *economics + game theory*.

```
p nuttall_w $x
```

nw_score

`nw_score(a, b, match=1, mismatch=-1, gap=-2)` → `number` — Needleman-Wunsch global alignment score.

```
p nw_score("ACGT", "ACCT")
```

nxtp

`nxtp` — operation `nxtp`. Alias for `next_prime`. Category: *number theory / primes*.

```
p nxtp $x
```

nxtperm

`nxtperm` — operation `nxtperm`. Alias for `next_permutation`. Category: *array / list operations extras*.

```
p nxtperm $x
```

nyquist_encirclement

`nyquist_encirclement` — operation `nyquist encirclement`. Category: *robotics & control*.

```
p nyquist_encirclement $x
```

nyquist_phase

`nyquist_phase` — operation `nyquist phase`. Category: *electrochemistry, batteries, fuel cells*.

```
p nyquist_phase $x
```

nysiis

`nysiis(name)` → `string` — New York State Identification and Intelligence System phonetic encoding. Better than Soundex for non-Anglo names.

```
p nysiis("Smith") # SNAD
```

nysiis_phonetic

`nysiis_phonetic` — operation `nysiis phonetic`. Category: *computational linguistics*.

```
p nysiis_phonetic $x
```

oacconvolve_step

`oacconvolve_step` — single-step update of `oacconvolve`. Category: *economics + game theory*.

```
p oacconvolve_step $x
```

oat_hash

`oat_hash` — hash function of `oat`. Category: *b81-misc-utility*.

```
p oat_hash $x
```

object_encoding

`object_encoding` — operation `object encoding`. Category: *redis-flavour primitives*.

```
p object_encoding $x
```

object_entries

`object_entries` — operation `object entries`. Category: *javascript array/object methods*.

```
p object_entries $x
```

object_from_entries

`object_from_entries` — operation `object from entries`. Category: *javascript array/object methods*.

```
p object_from_entries $x
```

object_keys

`object_keys` — operation `object keys`. Category: *javascript array/object methods*.

```
p object_keys $x
```

object_values

`object_values` — operation `object values`. Category: *javascript array/object methods*.

```
p object_values $x
```

obv

`obv(closes, volumes)` → array — On-Balance Volume (Granville). Cumulative sum that adds `volume` on up-bars and subtracts on down-bars.

```
p obv(\@cl, \@vol)
```

oct_of

`oct_of` — octal helper `of`. Alias for `to_oct`. Category: *base conversion*.

```
p oct_of $x
```

octagonal_number

`octagonal_number(n)` → int — $n \cdot (3n - 2)$.

octahedral_number

`octahedral_number` — operation `octahedral number`. Category: *misc*.

```
p octahedral_number $x
```

ocv_lithium_ion_step

`ocv_lithium_ion_step` — single-step update of `ocv_lithium_ion`. Category: *electrochemistry, batteries, fuel cells*.

```
p ocv_lithium_ion_step $x
```

odd

`odd` — operation `odd`. Category: *trivial numeric / predicate builtins*.

```
p odd $x
```

odddyield

`odddyield` — operation `odddyield`. Category: *b81-misc-utility*.

```
p odddyield $x
```

odds_ratio_2x2

`odds_ratio_2x2` — operation `odds_ratio 2x2`. Category: *epidemiology / public health*.

```
p odds_ratio_2x2 $x
```

ode45_step

`ode45_step` — single-step update of `ode45`. Category: *numpy + scipy.special*.

```
p ode45_step $x
```

ode_euler

`ode_euler(derivatives, dt=0.01, y0=0) → array` — explicit Euler integration: $y[i+1] = y[i] + dt \cdot f[i]$ with initial condition `y0`. Returns a trajectory of length `len(derivatives) + 1`.

```
p ode_euler(\@dx_dt, 0.1, 1.0)
```

ode_lsoda

`ode_lsoda` — ODE integrator `lsoda`. Category: *numpy + scipy.special*.

```
p ode_lsoda $x
```

oents

`oents` — operation `oents`. Alias for `object_entries`. Category: *javascript array/object methods*.

```
p oents $x
```

ofents

`ofents` — operation `ofents`. Alias for `object_from_entries`. Category: *javascript array/object methods*.

```
p ofents $x
```


off

off — operation **off**. Alias for **red**. Category: **color / ansi**.

```
p off $x
```

offset

offset — operation **offset**. Category: **excel/sheets + bond/loan financial**.

```
p offset $x
```

offset_each

offset_each — operation **offset** **each**. Category: **trivial numeric helpers**.

```
p offset_each $x
```

ohm_voltage

ohm_voltage — operation **ohm** **voltage**. Category: **em / optics / relativity**.

```
p ohm_voltage $x
```

ohms_law_i

ohms_law_i — operation **ohms** **law** **i**. Category: **physics formulas**.

```
p ohms_law_i $x
```

ohms_law_r

ohms_law_r — operation **ohms** **law** **r**. Category: **physics formulas**.

```
p ohms_law_r $x
```

ohms_law_v

ohms_law_v — operation **ohms** **law** **v**. Category: **physics formulas**.

```
p ohms_law_v $x
```

okeys

okeys — operation **okeys**. Alias for **object_keys**. Category: **javascript array/object methods**.

```
p okeys $x
```

oklab_to_rgb

oklab_to_rgb — convert from **oklab** to **rgb**. Category: **b82-misc-utility**.

```
p oklab_to_rgb $value
```

oklch_to_rgb

oklch_to_rgb([L, C, H]) → [r, g, b] — inverse OKLCH → sRGB.

ols_adjusted_r2

`ols_adjusted_r2` — operation `ols adjusted r2`. Category: **econometrics**.

```
p ols_adjusted_r2 $x
```

ols_estimator

`ols_estimator` — operation `ols estimator`. Category: **econometrics**.

```
p ols_estimator $x
```

ols_r_squared

`ols_r_squared` — operation `ols r squared`. Category: **econometrics**.

```
p ols_r_squared $x
```

ols_residual_variance

`ols_residual_variance` — operation `ols residual variance`. Category: **econometrics**.

```
p ols_residual_variance $x
```

omega_equation_step

`omega_equation_step` — single-step update of `omega equation`. Category: **climate, fluids, atmospheric**.

```
p omega_equation_step $x
```

omega_m_at_z

`omega_m_at_z` — operation `omega m at z`. Category: **cosmology / gr / flrw**.

```
p omega_m_at_z $x
```

omega_ratio

`omega_ratio(returns, threshold=0) → number` — $\Sigma(\text{returns above threshold}) / \Sigma(|\text{returns below threshold}|)$. Captures full return distribution.

```
p omega_ratio(\@returns, 0)
```

omit

`omit` — operation `omit`. Alias for `omit_keys`. Category: **hash ops**.

```
p omit $x
```

omit_keys

`omit_keys` — operation `omit keys`. Category: **hash ops**.

```
p omit_keys $x
```

omori_aftershock

`omori_aftershock` — operation `omori aftershock`. Category: **geology, seismology, mineralogy**.

```
p omori_aftershock $x
```

one

`one` — operation `one`. Category: **python/ruby stdlib**.

```
p one $x
```

one_hot

`one_hot` — operation `one hot`. Category: **ml extensions**.

```
p one_hot $x
```

ones

`ones` — operation `ones`. Category: **matrix / linear algebra**.

```
p ones $x
```

ones_mat

`ones_mat` — operation `ones mat`. Alias for `ones_matrix`. Category: **matrix / linear algebra**.

```
p ones_mat $x
```

ones_matrix

`ones_matrix` — operation `ones matrix`. Category: **matrix / linear algebra**.

```
p ones_matrix $x
```

oneshot_new

`oneshot_new()` → `channel` — single-value rendezvous; receiver blocks until the one send.

only_alnum

`only_alnum` — operation `only alnum`. Category: **string helpers**.

```
p only_alnum $x
```

only_alpha

`only_alpha` — operation `only alpha`. Category: **string helpers**.

```
p only_alpha $x
```

only_ascii

`only_ascii` — operation `only ascii`. Category: **string helpers**.

```
p only_ascii $x
```

only_digits

`only_digits` — operation `only digits`. Category: **string helpers**.

```
p only_digits $x
```

onsager_relation_two_species

`onsager_relation_two_species` — operation `onsager relation two species`. Category: **electrochemistry, batteries, fuel cells**.

```
p onsager_relation_two_species $x
```

open_pipe_harmonic

`open_pipe_harmonic` — operation `open pipe harmonic`. Category: **em / optics / relativity**.

```
p open_pipe_harmonic $x
```

open_url

`openurl(URL_OR_PATH)` — launch the OS default handler for `URL_OR_PATH`. macOS → `open`, Linux → `xdg-open`, Windows → `cmd /c start`. Returns `1` on launch success, `0` otherwise.

```
openurl("https://stryke.menketech.com")  
openurl("./report.pdf")
```

opening_2d

`opening_2d(image, size=3)` → `matrix` — erosion followed by dilation. Removes small bright specks.

operad_count_two

`operad_count_two` — operation `operad count two`. Category: **econometrics**.

```
p operad_count_two $x
```

oppenheimer_volkoff_step

`oppenheimer_volkoff_step` — single-step update of `oppenheimer volkoff`. Category: **electrochemistry, batteries, fuel cells**.

```
p oppenheimer_volkoff_step $x
```

ops_plus

`ops_plus` — operation `ops plus`. Category: **astronomy / astrometry**.

```
p ops_plus $x
```

optical_flow_farneback

`optical_flow_farneback` — operation `optical flow farneback`. Category: **pil/opencv image processing**.

```
p optical_flow_farneback $x
```

optical_flow_lk

`optical_flow_lk` — operation `optical flow lk`. Category: *pil/opencv image processing*.

```
p optical_flow_lk $x
```

optical_flow_lk_step

`optical_flow_lk_step` — single-step update of `optical flow lk`. Category: *electrochemistry, batteries, fuel cells*.

```
p optical_flow_lk_step $x
```

optical_scalars_step

`optical_scalars_step` — single-step update of `optical scalars`. Category: *electrochemistry, batteries, fuel cells*.

```
p optical_scalars_step $x
```

or_list

`or_list` — operation `or list`. Category: *additional missing stdlib functions*.

```
p or_list $x
```

or_opt_delta

`or_opt_delta` — operation `or opt delta`. Category: *combinatorial optimization, scheduling*.

```
p or_opt_delta $x
```

orb_keypoints

`orb_keypoints` — operation `orb keypoints`. Category: *pil/opencv image processing*.

```
p orb_keypoints $x
```

orbit_eccentric_anomaly

`orbit_eccentric_anomaly` — operation `orbit eccentric anomaly`. Category: *astronomy / astrometry*.

```
p orbit_eccentric_anomaly $x
```

orbit_kepler3

`orbit_kepler3` — operation `orbit kepler3`. Category: *astronomy / astrometry*.

```
p orbit_kepler3 $x
```

orbital_eccentricity

`orbital_eccentricity` — operation `orbital eccentricity`. Category: *more extensions*.

```
p orbital_eccentricity $x
```

orbital_period

`orbital_period($mass, $radius)` (alias `orbper`) — period for circular orbit: $T = 2\pi\sqrt{(r^3/GM)}$. Returns seconds.

```
val $t = orbper      # ~5560 s (ISS orbital period)
p $t / 60            # ~93 minutes
```

orbital_period_au

`orbital_period_au` — operation `orbital period au`. Category: **astronomy / astrometry**.

```
p orbital_period_au $x
```

orbital_velocity

`orbital_velocity($mass, $radius)` (alias `orbvel`) — velocity for circular orbit: $v = \sqrt{GM/r}$. Defaults to LEO around Earth.

```
p orbvel      # ~7672 m/s (ISS orbital speed)
```

order_modulo

`order_modulo` — operation `order modulo`. Category: **math / number theory extras**.

```
p order_modulo $x
```

ordered_probit_threshold

`ordered_probit_threshold` — operation `ordered probit threshold`. Category: **econometrics**.

```
p ordered_probit_threshold $x
```

ordered_set

`ordered_set` — operation `ordered set`. Category: **data structure helpers**.

```
p ordered_set $x
```

ordinal_date

`ordinal_date` — operation `ordinal date`. Alias for `iso_ordinal_date`. Category: **misc**.

```
p ordinal_date $x
```

ordinalize

`ordinalize` — operation `ordinalize`. Category: **extended stdlib**.

```
p ordinalize $x
```

ordn

`ordn` — operation `ordn`. Alias for `ordinalize`. Category: **extended stdlib**.

```
p ordn $x
```

orifice_velocity

`orifice_velocity` — operation `orifice velocity`. Category: **misc**.

```
p orifice_velocity $x
```

orl

`orl` — operation `orl`. Alias for `or_list`. Category: *additional missing stdlib functions*.

```
p orl $x
```

orthogonalize_vectors

`orthogonalize_vectors` — operation `orthogonalize vectors`. Category: *misc*.

```
p orthogonalize_vectors $x
```

os_acpi_state_transition

`os_acpi_state_transition` — operating-system helper `acpi state transition`. Category: *os internals — schedulers, i/o, memory*.

```
p os_acpi_state_transition $x
```

os_amd_pstate_target

`os_amd_pstate_target` — operating-system helper `amd pstate target`. Category: *os internals — schedulers, i/o, memory*.

```
p os_amd_pstate_target $x
```

os_anticipation_window

`os_anticipation_window` — operating-system helper `anticipation window`. Category: *os internals — schedulers, i/o, memory*.

```
p os_anticipation_window $x
```

os_apic_irq_priority

`os_apic_irq_priority` — operating-system helper `apic irq priority`. Category: *os internals — schedulers, i/o, memory*.

```
p os_apic_irq_priority $x
```

os_apparmor_profile_active

`os_apparmor_profile_active` — operating-system helper `apparmor profile active`. Category: *os internals — schedulers, i/o, memory*.

```
p os_apparmor_profile_active $x
```

os_arch

`os_arch` — operating-system helper `arch`. Category: *system introspection*.

```
p os_arch $x
```

os_audit_event_priority

`os_audit_event_priority` — operating-system helper `audit event priority`. Category: *os internals — schedulers, i/o, memory*.

```
p os_audit_event_priority $x
```

os_battery_capacity_pct

`os_battery_capacity_pct` — operating-system helper `battery capacity pct`. Category: *os internals — schedulers, i/o, memory*.

```
p os_battery_capacity_pct $x
```

os_buddy_order_pick

`os_buddy_order_pick` — operating-system helper `buddy order pick`. Category: *os internals — schedulers, i/o, memory*.

```
p os_buddy_order_pick $x
```

os_buffer_cache_pages

`os_buffer_cache_pages` — operating-system helper `buffer cache pages`. Category: *os internals — schedulers, i/o, memory*.

```
p os_buffer_cache_pages $x
```

os_c_state_residency

`os_c_state_residency` — operating-system helper `c state residency`. Category: *os internals — schedulers, i/o, memory*.

```
p os_c_state_residency $x
```

os_capability_check

`os_capability_check` — operating-system helper `capability check`. Category: *os internals — schedulers, i/o, memory*.

```
p os_capability_check $x
```

os_cfs_load_balance_step

`os_cfs_load_balance_step` — operating-system helper `cfs load balance step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_cfs_load_balance_step $x
```

os_cgroup_v1_count

`os_cgroup_v1_count` — operating-system helper `cgroup v1 count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_cgroup_v1_count $x
```

os_cgroup_v2_count

`os_cgroup_v2_count` — operating-system helper `cgroup v2 count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_cgroup_v2_count $x
```

os_compact_memory_step

`os_compact_memory_step` — operating-system helper `compact memory step`. Category: *os internals — schedulers, i/o, memory*.


```
p os_compact_memory_step $x
```

os_completely_fair_vruntime

`os_completely_fair_vruntime` — operating-system helper `completely fair vruntime`. Category: *os internals — schedulers, i/o, memory*.

```
p os_completely_fair_vruntime $x
```

os_copy_on_write_check

`os_copy_on_write_check` — operating-system helper `copy on write check`. Category: *os internals — schedulers, i/o, memory*.

```
p os_copy_on_write_check $x
```

os_cpu_affinity_score

`os_cpu_affinity_score` — operating-system helper `cpu affinity score`. Category: *os internals — schedulers, i/o, memory*.

```
p os_cpu_affinity_score $x
```

os_cpufreq_governor_step

`os_cpufreq_governor_step` — operating-system helper `cpufreq governor step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_cpufreq_governor_step $x
```

os_demand_paging_step

`os_demand_paging_step` — operating-system helper `demand paging step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_demand_paging_step $x
```

os_dirty_page_threshold

`os_dirty_page_threshold` — operating-system helper `dirty page threshold`. Category: *os internals — schedulers, i/o, memory*.

```
p os_dirty_page_threshold $x
```

os_disk_rotational_lat

`os_disk_rotational_lat` — operating-system helper `disk rotational lat`. Category: *os internals — schedulers, i/o, memory*.

```
p os_disk_rotational_lat $x
```

os_disk_seek_time

`os_disk_seek_time` — operating-system helper `disk seek time`. Category: *os internals — schedulers, i/o, memory*.

```
p os_disk_seek_time $x
```

os_disk_transfer_time

`os_disk_transfer_time` — operating-system helper `disk transfer time`. Category: *os internals — schedulers, i/o, memory*.

```
p os_disk_transfer_time $x
```

os_dmesg_severity_level

`os_dmesg_severity_level` — operating-system helper `dmesg severity level`. Category: *os internals — schedulers, i/o, memory*.

```
p os_dmesg_severity_level $x
```

os_dvfs_step

`os_dvfs_step` — operating-system helper `dvfs step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_dvfs_step $x
```

os_eas_energy_estimate

`os_eas_energy_estimate` — operating-system helper `eas energy estimate`. Category: *os internals — schedulers, i/o, memory*.

```
p os_eas_energy_estimate $x
```

os_eevdf_eligible

`os_eevdf_eligible` — operating-system helper `eevdf eligible`. Category: *os internals — schedulers, i/o, memory*.

```
p os_eevdf_eligible $x
```

os_elevator_step

`os_elevator_step` — operating-system helper `elevator step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_elevator_step $x
```

os_epoll_ctl_count

`os_epoll_ctl_count` — operating-system helper `epoll ctl count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_epoll_ctl_count $x
```

os_family

`os_family` — OS family string: "unix" or "windows". From `std::env::consts::FAMILY`.

```
p os_family # unix
die "unix only" unless os_family eq "unix"
```

os_file_max_value

`os_file_max_value` — operating-system helper `file max value`. Category: *os internals — schedulers, i/o, memory*.

```
p os_file_max_value $x
```

os_frequency_scaling_step

`os_frequency_scaling_step` — operating-system helper `frequency scaling step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_frequency_scaling_step $x
```

os_huge_page_threshold

`os_huge_page_threshold` — operating-system helper `huge page threshold`. Category: *os internals — schedulers, i/o, memory*.

```
p os_huge_page_threshold $x
```

os_idle_state_select

`os_idle_state_select` — operating-system helper `idle state select`. Category: *os internals — schedulers, i/o, memory*.

```
p os_idle_state_select $x
```

os_inotify_event_count

`os_inotify_event_count` — operating-system helper `inotify event count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_inotify_event_count $x
```

os_inotify_max_watches

`os_inotify_max_watches` — operating-system helper `inotify max watches`. Category: *os internals — schedulers, i/o, memory*.

```
p os_inotify_max_watches $x
```

os_intel_pstate_target

`os_intel_pstate_target` — operating-system helper `intel pstate target`. Category: *os internals — schedulers, i/o, memory*.

```
p os_intel_pstate_target $x
```

os_io_scheduler_bfq_step

`os_io_scheduler_bfq_step` — operating-system helper `io scheduler bfq step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_io_scheduler_bfq_step $x
```

os_io_scheduler_cfq_step

`os_io_scheduler_cfq_step` — operating-system helper `io scheduler cfq step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_io_scheduler_cfq_step $x
```

os_io_scheduler_deadline

`os_io_scheduler_deadline` — operating-system helper `io scheduler deadline`. Category: *os internals — schedulers, i/o, memory*.

```
p os_io_scheduler_deadline $x
```

`os_io_scheduler_kyber_step`

`os_io_scheduler_kyber_step` — operating-system helper `io scheduler kyber step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_io_scheduler_kyber_step $x
```

`os_io_scheduler_mq_deadline`

`os_io_scheduler_mq_deadline` — operating-system helper `io scheduler mq deadline`. Category: *os internals — schedulers, i/o, memory*.

```
p os_io_scheduler_mq_deadline $x
```

`os_io_scheduler_noop_step`

`os_io_scheduler_noop_step` — operating-system helper `io scheduler noop step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_io_scheduler_noop_step $x
```

`os_io_uring_cqe_count`

`os_io_uring_cqe_count` — operating-system helper `io uring cqe count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_io_uring_cqe_count $x
```

`os_io_uring_sqe_count`

`os_io_uring_sqe_count` — operating-system helper `io uring sqe count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_io_uring_sqe_count $x
```

`os_iommu_domain_step`

`os_iommu_domain_step` — operating-system helper `iommu domain step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_iommu_domain_step $x
```

`os_kasan_shadow_offset`

`os_kasan_shadow_offset` — operating-system helper `kasan shadow offset`. Category: *os internals — schedulers, i/o, memory*.

```
p os_kasan_shadow_offset $x
```

`os_kfence_alloc_index`

`os_kfence_alloc_index` — operating-system helper `kfence alloc index`. Category: *os internals — schedulers, i/o, memory*.

```
p os_kfence_alloc_index $x
```

os_kfence_check

`os_kfence_check` — operating-system helper `kfence check`. Category: *os internals — schedulers, i/o, memory*.

```
p os_kfence_check $x
```

os_kqueue_event_count

`os_kqueue_event_count` — operating-system helper `kqueue event count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_kqueue_event_count $x
```

os_kswapd_wake_threshold

`os_kswapd_wake_threshold` — operating-system helper `kswapd wake threshold`. Category: *os internals — schedulers, i/o, memory*.

```
p os_kswapd_wake_threshold $x
```

os_kvm_vmcs_field_offset

`os_kvm_vmcs_field_offset` — operating-system helper `kvm vmcs field offset`. Category: *os internals — schedulers, i/o, memory*.

```
p os_kvm_vmcs_field_offset $x
```

os_load_average_decay

`os_load_average_decay` — operating-system helper `load average decay`. Category: *os internals — schedulers, i/o, memory*.

```
p os_load_average_decay $x
```

os_load_iowait_avg

`os_load_iowait_avg` — operating-system helper `load iowait avg`. Category: *os internals — schedulers, i/o, memory*.

```
p os_load_iowait_avg $x
```

os_load_proc_avg

`os_load_proc_avg` — operating-system helper `load proc avg`. Category: *os internals — schedulers, i/o, memory*.

```
p os_load_proc_avg $x
```

os_load_user_avg

`os_load_user_avg` — operating-system helper `load user avg`. Category: *os internals — schedulers, i/o, memory*.

```
p os_load_user_avg $x
```

os_lottery_ticket_count

`os_lottery_ticket_count` — operating-system helper `lottery ticket count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_lottery_ticket_count $x
```

os_mlfq_demote_step

`os_mlfq_demote_step` — operating-system helper `mlfq demote step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_mlfq_demote_step $x
```

os_mlfq_promote_step

`os_mlfq_promote_step` — operating-system helper `mlfq promote step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_mlfq_promote_step $x
```

os_msi_x_vector_count

`os_msi_x_vector_count` — operating-system helper `msi x vector count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_msi_x_vector_count $x
```

os_name

`os_name` — operating-system helper `name`. Category: *system introspection*.

```
p os_name $x
```

os_namespace_isolation

`os_namespace_isolation` — operating-system helper `namespace isolation`. Category: *os internals — schedulers, i/o, memory*.

```
p os_namespace_isolation $x
```

os_numa_node_distance

`os_numa_node_distance` — operating-system helper `numa node distance`. Category: *os internals — schedulers, i/o, memory*.

```
p os_numa_node_distance $x
```

os_oom_kill_score

`os_oom_kill_score` — operating-system helper `oom kill score`. Category: *os internals — schedulers, i/o, memory*.

```
p os_oom_kill_score $x
```

os_oom_score_step

`os_oom_score_step` — operating-system helper `oom score step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_oom_score_step $x
```

os_open_files_count

`os_open_files_count` — operating-system helper `open files count`. Category: *os internals — schedulers, i/o, memory*.

```
p os_open_files_count $x
```

os_p_state_voltage

`os_p_state_voltage` — operating-system helper `p state voltage`. Category: *os internals — schedulers, i/o, memory*.

```
p os_p_state_voltage $x
```

os_page_replacement_2q

`os_page_replacement_2q` — operating-system helper `page replacement 2q`. Category: *os internals — schedulers, i/o, memory*.

```
p os_page_replacement_2q $x
```

os_page_replacement_clock

`os_page_replacement_clock` — operating-system helper `page replacement clock`. Category: *os internals — schedulers, i/o, memory*.

```
p os_page_replacement_clock $x
```

os_page_replacement_lru

`os_page_replacement_lru` — operating-system helper `page replacement lru`. Category: *os internals — schedulers, i/o, memory*.

```
p os_page_replacement_lru $x
```

os_pci_bus_address

`os_pci_bus_address` — operating-system helper `pci bus address`. Category: *os internals — schedulers, i/o, memory*.

```
p os_pci_bus_address $x
```

os_per_cpu_cache_size

`os_per_cpu_cache_size` — operating-system helper `per cpu cache size`. Category: *os internals — schedulers, i/o, memory*.

```
p os_per_cpu_cache_size $x
```

os_pid_max_value

`os_pid_max_value` — operating-system helper `pid max value`. Category: *os internals — schedulers, i/o, memory*.

```
p os_pid_max_value $x
```

os_powertop_score

`os_powertop_score` — operating-system helper `powertop score`. Category: *os internals — schedulers, i/o, memory*.

```
p os_powertop_score $x
```

os_pre_fetch_window

`os_pre_fetch_window` — operating-system helper `pre fetch window`. Category: *os internals — schedulers, i/o, memory*.

```
p os_pre_fetch_window $x
```

os_pressure_stall_step

`os_pressure_stall_step` — operating-system helper `pressure stall step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_pressure_stall_step $x
```

os_priority_aging_step

`os_priority_aging_step` — operating-system helper `priority aging step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_priority_aging_step $x
```

os_psi_avg10_step

`os_psi_avg10_step` — operating-system helper `psi avg10 step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_psi_avg10_step $x
```

os_psi_avg300_step

`os_psi_avg300_step` — operating-system helper `psi avg300 step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_psi_avg300_step $x
```

os_psi_avg60_step

`os_psi_avg60_step` — operating-system helper `psi avg60 step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_psi_avg60_step $x
```

os_round_robin_quantum

`os_round_robin_quantum` — operating-system helper `round robin quantum`. Category: *os internals — schedulers, i/o, memory*.

```
p os_round_robin_quantum $x
```

os_runqueue_depth

`os_runqueue_depth` — operating-system helper `runqueue depth`. Category: *os internals — schedulers, i/o, memory*.

```
p os_runqueue_depth $x
```

os_seccomp_filter_step

`os_seccomp_filter_step` — operating-system helper `seccomp filter step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_seccomp_filter_step $x
```


os_selinux_context_match

`os_selinux_context_match` — operating-system helper `selinux context match`. Category: *os internals — schedulers, i/o, memory*.

```
p os_selinux_context_match $x
```

os_slab_color_offset

`os_slab_color_offset` — operating-system helper `slab color offset`. Category: *os internals — schedulers, i/o, memory*.

```
p os_slab_color_offset $x
```

os_slub_object_size_round

`os_slub_object_size_round` — operating-system helper `slub object size round`. Category: *os internals — schedulers, i/o, memory*.

```
p os_slub_object_size_round $x
```

os_smack_label_compare

`os_smack_label_compare` — operating-system helper `smack label compare`. Category: *os internals — schedulers, i/o, memory*.

```
p os_smack_label_compare $x
```

os_smt_threading_share

`os_smt_threading_share` — operating-system helper `smt threading share`. Category: *os internals — schedulers, i/o, memory*.

```
p os_smt_threading_share $x
```

os_socket_max_value

`os_socket_max_value` — operating-system helper `socket max value`. Category: *os internals — schedulers, i/o, memory*.

```
p os_socket_max_value $x
```

os_stride_pass_step

`os_stride_pass_step` — operating-system helper `stride pass step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_stride_pass_step $x
```

os_swap_pressure_score

`os_swap_pressure_score` — operating-system helper `swap pressure score`. Category: *os internals — schedulers, i/o, memory*.

```
p os_swap_pressure_score $x
```

os_swappiness_factor

`os_swappiness_factor` — operating-system helper `swappiness factor`. Category: *os internals — schedulers, i/o, memory*.

```
p os_swappiness_factor $x
```

`os_systemd_journal_size`

`os_systemd_journal_size` — operating-system helper `systemd journal size`. Category: *os internals — schedulers, i/o, memory*.

```
p os_systemd_journal_size $x
```

`os_thermal_zone_trip`

`os_thermal_zone_trip` — operating-system helper `thermal zone trip`. Category: *os internals — schedulers, i/o, memory*.

```
p os_thermal_zone_trip $x
```

`os_thrashing_threshold`

`os_thrashing_threshold` — operating-system helper `thrashing threshold`. Category: *os internals — schedulers, i/o, memory*.

```
p os_thrashing_threshold $x
```

`os_thread_max_value`

`os_thread_max_value` — operating-system helper `thread max value`. Category: *os internals — schedulers, i/o, memory*.

```
p os_thread_max_value $x
```

`os_thread_migration_cost`

`os_thread_migration_cost` — operating-system helper `thread migration cost`. Category: *os internals — schedulers, i/o, memory*.

```
p os_thread_migration_cost $x
```

`os_throttle_temperature`

`os_throttle_temperature` — operating-system helper `throttle temperature`. Category: *os internals — schedulers, i/o, memory*.

```
p os_throttle_temperature $x
```

`os_transparent_hugepage`

`os_transparent_hugepage` — operating-system helper `transparent hugepage`. Category: *os internals — schedulers, i/o, memory*.

```
p os_transparent_hugepage $x
```

`os_version`

`os_version` — OS kernel version/release string from `uname`. Returns `undef` on non-unix.

```
p os_version # 25.4.0
p "kernel: " . os_version
```

os_voltage_scaling_step

`os_voltage_scaling_step` — operating-system helper `voltage_scaling_step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_voltage_scaling_step $x
```

os_working_set_size

`os_working_set_size` — operating-system helper `working_set_size`. Category: *os internals — schedulers, i/o, memory*.

```
p os_working_set_size $x
```

os_writeback_step

`os_writeback_step` — operating-system helper `writeback_step`. Category: *os internals — schedulers, i/o, memory*.

```
p os_writeback_step $x
```

os_zero_page_optimization

`os_zero_page_optimization` — operating-system helper `zero_page_optimization`. Category: *os internals — schedulers, i/o, memory*.

```
p os_zero_page_optimization $x
```

os_zram_compress_ratio

`os_zram_compress_ratio` — operating-system helper `zram_compress_ratio`. Category: *os internals — schedulers, i/o, memory*.

```
p os_zram_compress_ratio $x
```

os_zswap_compress_ratio

`os_zswap_compress_ratio` — operating-system helper `zswap_compress_ratio`. Category: *os internals — schedulers, i/o, memory*.

```
p os_zswap_compress_ratio $x
```

oscillator_sawtooth

`oscillator_sawtooth` — oscillator (waveform generator) `sawtooth`. Category: *extras*.

```
p oscillator_sawtooth $x
```

oscillator_sine

`oscillator_sine` — oscillator (waveform generator) `sine`. Category: *extras*.

```
p oscillator_sine $x
```

oscillator_square

`oscillator_square` — oscillator (waveform generator) `square`. Category: **extras**.

```
p oscillator_square $x
```

oscillator_triangle

`oscillator_triangle` — oscillator (waveform generator) `triangle`. Category: **extras**.

```
p oscillator_triangle $x
```

osmotic_coefficient_pitzer

`osmotic_coefficient_pitzer` — operation `osmotic coefficient pitzer`. Category: **electrochemistry, batteries, fuel cells**.

```
p osmotic_coefficient_pitzer $x
```

osmotic_pressure

`osmotic_pressure` — operation `osmotic pressure`. Category: **chemistry**.

```
p osmotic_pressure $x
```

otsu_threshold

`otsu_threshold(image)` → `number` — Otsu's method: best intensity threshold maximising between-class variance.

```
my $t = otsu_threshold($img)
```

otto_efficiency

`otto_efficiency` — operation `otto efficiency`. Category: **misc**.

```
p otto_efficiency $x
```

out_degree_directed

`out_degree_directed` — operation `out degree directed`. Category: **misc**.

```
p out_degree_directed $x
```

outer

`outer` — outer product of two vectors. Returns matrix where $M[i][j] = v1[i] * v2[j]$.

```
val $m = outer([1,2,3], [10,20]) # [[10,20],[20,40],[30,60]]
```

outlier_iqr_q

`outlier_iqr_q` — operation `outlier iqr q`. Category: **misc**.

```
p outlier_iqr_q $x
```

outliers

`outliers` — operation `outliers`. Alias for `outliers_iqr`. Category: **statistics (extended)**.

```
p outliers $x
```

outliers_iqr

`outliers_iqr(@data)` — identifies outliers using the IQR method ($1.5 \times \text{IQR}$ beyond $Q1/Q3$). Returns an arrayref of outlier values. Standard approach for detecting unusual data points.

```
val @data = (1, 2, 3, 4, 5, 100)
var $out = outliers_iqr(@data)
p @$out # (100)
```

ovals

`ovals` — operation `ovals`. Alias for `object_values`. Category: **javascript array/object methods**.

```
p ovals $x
```

overall_efficiency_pec

`overall_efficiency_pec` — operation `overall efficiency pec`. Category: **electrochemistry, batteries, fuel cells**.

```
p overall_efficiency_pec $x
```

overlap_coeff

`overlap_coeff(a, b) → number` — $|A \cap B| / \min(|A|, |B|)$.

overlap_coefficient

`overlap_coefficient` — operation `overlap coefficient`. Category: **extended stdlib**.

```
p overlap_coefficient $x
```

overlapcoef

`overlapcoef` — operation `overlapcoef`. Alias for `overlap_coefficient`. Category: **extended stdlib**.

```
p overlapcoef $x
```

overline

`overline` — operation `overline`. Alias for `red`. Category: **color / ansi**.

```
p overline $x
```

overpotential_her

`overpotential_her` — operation `overpotential her`. Category: **electrochemistry, batteries, fuel cells**.

```
p overpotential_her $x
```

overpotential_oer

`overpotential_oer` — operation `overpotential oer`. Category: **electrochemistry, batteries, fuel cells**.

```
p overpotential_oer $x
```

owen_t

`owen_t` — operation `owen t`. Category: **special functions extra**.

```
p owen_t $x
```

owens_t

`owens_t` — operation `owens t`. Category: **numpy + scipy.special**.

```
p owens_t $x
```

oxidation_number

`oxidation_number` — operation `oxidation number`. Category: **chemistry & biochemistry**.

```
p oxidation_number $x
```

oz_to_g

`oz_to_g` — convert from `oz` to `g`. Category: **unit conversions**.

```
p oz_to_g $value
```

ozone_dobson_total

`ozone_dobson_total` — operation `ozone dobson total`. Category: **climate, fluids, atmospheric**.

```
p ozone_dobson_total $x
```

ozsvath_szabo_tau

`ozsvath_szabo_tau` — operation `ozsvath szabo tau`. Category: **econometrics**.

```
p ozsvath_szabo_tau $x
```

p3_to_rgb

`p3_to_rgb` — convert from `p3` to `rgb`. Category: **complex / geom / color / trig**.

```
p p3_to_rgb $value
```

pa_to_atm

`pa_to_atm` — convert from `pa` to `atm`. Category: **astronomy / music / color / units**.

```
p pa_to_atm $value
```

pacf

`pacf_fn` (alias `pacf`) — partial autocorrelation function via Durbin-Levinson. Like R's `pacf()`.

```
val @pa = @{pacf([1,3,2,4,3,5], 3)}
```

pacf_basis

`pacf_basis` — operation `pacf basis`. Category: **statsmodels**.

```
p pacf_basis $x
```

packet_loss

`packet_loss` — operation `packet loss`. Category: **network / ip / cidr**.

```
p packet_loss $x
```

packrat_parse_step

`packrat_parse_step` — single-step update of `packrat parse`. Category: **compilers / parsing**.

```
p packrat_parse_step $x
```

pad_left

`pad_left` — operation `pad left`. Category: **trivial string ops**.

```
p pad_left $x
```

pad_left_n

`pad_left_n` — operation `pad left n`. Category: **iterator + string-distance extras**.

```
p pad_left_n $x
```

pad_number

`pad_number` — operation `pad number`. Category: **file stat / path**.

```
p pad_number $x
```

pad_right

`pad_right` — operation `pad right`. Category: **trivial string ops**.

```
p pad_right $x
```

pad_right_n

`pad_right_n` — operation `pad right n`. Category: **iterator + string-distance extras**.

```
p pad_right_n $x
```

page_size

`page_size` — memory page size in bytes. Uses `sysconf(_SC_PAGESIZE)` on unix.

```
p page_size # 16384 (Apple Silicon)
p page_size # 4096 (x86_64)
```

pagerank_damped

`pagerank_damped` — operation `pagerank damped`. Category: **graph algorithms**.

```
p pagerank_damped $x
```

pagerank_iter

`pagerank_iter` — operation `pagerank iter`. Category: **networkx graph algorithms**.

```
p pagerank_iter $x
```

pagerank_tournament

`pagerank_tournament(wins_matrix, damping=0.85, iters=50)` → array — PageRank-style ratings from a wins matrix.

paired_ttest

`paired_ttest` (alias `pairedt`) performs a paired t-test on two matched samples. Returns [t-statistic, degrees of freedom].

```
var ($t, $df) = @{pairedt([85,90,78], [88,92,80])}
```

pairs_from_hash

`pairs_from_hash` — hash function of `pairs from`. Category: *hash ops*.

```
p pairs_from_hash $x
```

pairwise

`pairwise` — operation `pairwise`. Category: *file stat / path*.

```
p pairwise $x
```

pairwise_iter

`pairwise_iter` — operation `pairwise iter`. Category: *python/ruby stdlib*.

```
p pairwise_iter $x
```

palindromic_q

`palindromic_q` — operation `palindromic q`. Category: *misc*.

```
p palindromic_q $x
```

pall

`pall` — operation `pall`. Alias for `partition_all`. Category: *algebraic match*.

```
p pall $x
```

pam250_score

`pam250_score` — score of `pam250`. Category: *bioinformatics deep*.

```
p pam250_score $x
```

panel_between_estimator

`panel_between_estimator` — operation `panel between estimator`. Category: *econometrics*.

```
p panel_between_estimator $x
```


panel_random_effects

`panel_random_effects` — operation `panel random effects`. Category: **econometrics**.

```
p panel_random_effects $x
```

panel_var_step

`panel_var_step` — single-step update of `panel var`. Category: **econometrics**.

```
p panel_var_step $x
```

panel_within_estimator

`panel_within_estimator` — operation `panel within estimator`. Category: **econometrics**.

```
p panel_within_estimator $x
```

parabolic_d0

`parabolic_d0` — operation `parabolic d0`. Category: **special functions extra**.

```
p parabolic_d0 $x
```

parabolic_d1

`parabolic_d1` — operation `parabolic d1`. Category: **special functions extra**.

```
p parabolic_d1 $x
```

parabolic_sar

`parabolic_sar(highs, lows, af_step=0.02, af_max=0.2) → array` — Wilder's stop-and-reverse trailing stop. Flips when price crosses the SAR line; acceleration factor grows on new extremes.

```
p parabolic_sar(\@hi, \@lo)
```

parallax_correction

`parallax_correction` — operation `parallax correction`. Category: **astronomy / astrometry**.

```
p parallax_correction $x
```

parallel_machine_lpt

`parallel_machine_lpt` — operation `parallel machine lpt`. Category: **combinatorial optimization, scheduling**.

```
p parallel_machine_lpt $x
```

parallel_machine_spt

`parallel_machine_spt` — operation `parallel machine spt`. Category: **combinatorial optimization, scheduling**.

```
p parallel_machine_spt $x
```

parallel_transport_step

`parallel_transport_step` — single-step update of `parallel_transport`. Category: *electrochemistry, batteries, fuel cells*.

```
p parallel_transport_step $x
```

parenthesize

`parenthesize` — operation `parenthesize`. Category: *string helpers*.

```
p parenthesize $x
```

pareto_dominance

`pareto_dominance` — operation `pareto_dominance`. Category: *economics + game theory*.

```
p pareto_dominance $x
```

pareto_efficiency

`pareto_efficiency` — operation `pareto_efficiency`. Category: *economics + game theory*.

```
p pareto_efficiency $x
```

pareto_pdf

`pareto_pdf` (alias `paretopdf`) evaluates the Pareto distribution PDF at `x` with minimum `xm` and shape `alpha`.

```
p paretopdf(2, 1, 3) # Pareto(1,3) at x=2
```

parse_bool

`parse_bool` — operation `parse_bool`. Category: *extended stdlib*.

```
p parse_bool $x
```

parse_float

`parse_float` — operation `parse_float`. Category: *extended stdlib*.

```
p parse_float $x
```

parse_int

`parse_int` — operation `parse_int`. Category: *extended stdlib*.

```
p parse_int $x
```

parsec

`parsec` (alias `pc`) — $\text{pc} \approx 3.086 \times 10^{16} \text{ m} \approx 3.26 \text{ ly}$. Common astronomical distance unit.

```
p pc # 3.0856775814913673e16 m
p pc / ly # ~3.26 light years
```

partial_pressure

`partial_pressure` — operation `partial pressure`. Category: **misc**.

```
p partial_pressure $x
```

particle_emit

`particle_emit` — operation `particle emit`. Category: **extras**.

```
p particle_emit $x
```

particle_filter_resample

`particle_filter_resample` — operation `particle filter resample`. Category: **electrochemistry, batteries, fuel cells**.

```
p particle_filter_resample $x
```

particle_update

`particle_update` — update step of `particle`. Category: **extras**.

```
p particle_update $x
```

partition_all

`partition_all` — operation `partition all`. Category: **algebraic match**.

```
p partition_all $x
```

partition_at

`partition_at` — operation `partition at`. Category: **misc**.

```
p partition_at $x
```

partition_by

`partition_by` — operation `partition by`. Category: **algebraic match**.

```
p partition_by $x
```

partition_either

`partition_either` — operation `partition either`. Category: **rust iterator methods**.

```
p partition_either $x
```

partition_n

`partition_n` — operation `partition n`. Category: **go/general functional utilities**.

```
p partition_n $x
```

partition_number

`partition_number` — operation `partition number`. Category: **math / numeric extras**.

```
p partition_number $x
```

partition_point

`partition_point` — operation `partition point`. Category: **extended stdlib**.

```
p partition_point $x
```

partition_two

`partition_two` — operation `partition two`. Category: **list helpers**.

```
p partition_two $x
```

partitions_count

`partitions_count(n) → int` — Euler's partition function $p(n)$.

```
p partitions_count(10) # 42
```

partn

`partn` — operation `partn`. Alias for `partition_n`. Category: **go/general functional utilities**.

```
p partn $x
```

partn_num

`partn_num` — operation `partn num`. Alias for `partition_number`. Category: **math / numeric extras**.

```
p partn_num $x
```

parzen_window

`parzen_window` — operation `parzen window`. Category: **economics + game theory**.

```
p parzen_window $x
```

pascal_case

`pascal_case` — operation `pascal case`. Category: **string**.

```
p pascal_case $x
```

pascal_row

`pascal_row` — operation `pascal row`. Category: **extended stdlib**.

```
p pascal_row $x
```

pascals_to_bar

`pascals_to_bar` — convert from `pascals` to `bar`. Category: **file stat / path**.

```
p pascals_to_bar $value
```

pascals_to_psi

`pascals_to_psi` — convert from `pascals` to `psi`. Category: **file stat / path**.

```
p pascals_to_psi $value
```

pascals_triangle

`pascals_triangle` — operation `pascals triangle`. Category: **math / numeric extras**.

```
p pascals_triangle $x
```

pascrow

`pascrow` — operation `pascrow`. Alias for `pascal_row`. Category: **extended stdlib**.

```
p pascrow $x
```

pasctri

`pasctri` — operation `pasctri`. Alias for `pascals_triangle`. Category: **math / numeric extras**.

```
p pasctri $x
```

pass

`pass` — operation `pass`. Category: **conversion / utility**.

```
p pass $x
```

path_canonical

`path_canonical(path)` → `string` — absolute path with symlinks resolved (canonicalize).

path_case

`path_case` — path manipulation `case`. Category: **string**.

```
p path_case $x
```

path_common_ancestor

`path_common_ancestor(paths)` → `string` — longest common directory prefix.

path_components

`path_components(path)` → `array` — split into segments.

```
p path_components("/usr/local/bin") # ["usr","local","bin"]
```

path_dubins_lsl

`path_dubins_lsl` — path manipulation `dubins lsl`. Category: **robotics & control**.

```
p path_dubins_lsl $x
```

path_dubins_rsr

path_dubins_rsr — path manipulation [dubins](#) [rsr](#). Category: *robotics & control*.

```
p path_dubins_rsr $x
```

path_ext

path_ext — path manipulation [ext](#). Category: *path helpers*.

```
p path_ext $x
```

path_extension

path_extension(path) → [string](#) — final extension without leading dot, empty if none.

path_filename

path_filename(path) → [string](#) — last segment with extension.

path_glob_match_regex

path_glob_match_regex(glob) → [regex](#) — convert glob [*.txt](#) into anchored regex.

path_is_abs

path_is_abs — path manipulation [is](#) [abs](#). Category: *file stat / path*.

```
p path_is_abs $x
```

path_is_rel

path_is_rel — path manipulation [is](#) [rel](#). Category: *file stat / path*.

```
p path_is_rel $x
```

path_is_subdirectory

path_is_subdirectory(child, parent) → [0|1](#) — 1 if [child](#) is inside [parent](#) after canonicalisation.

path_join

path_join — path manipulation [join](#). Category: *path helpers*.

```
p path_join $x
```

path_join_many

path_join_many(parts) → [string](#) — join multiple path segments with the OS separator.

path_parent

path_parent — path manipulation [parent](#). Category: *path helpers*.

```
p path_parent $x
```

path_reeds_shepp

`path_reeds_shepp` — path manipulation `reeds shepp`. Category: **robotics & control**.

```
p path_reeds_shepp $x
```

path_relative_to

`path_relative_to(path, base) → string` — make `path` relative to `base`.

path_split

`path_split` — path manipulation `split`. Category: **path helpers**.

```
p path_split $x
```

path_stem

`path_stem(path) → string` — filename without final extension.

```
p path_stem("foo/bar/baz.tar.gz") # "baz.tar"
```

path_strip_prefix

`path_strip_prefix` — path manipulation `strip prefix`. Category: **extras**.

```
p path_strip_prefix $x
```

path_with_extension

`path_with_extension(path, ext) → string` — replace (or add) the extension.

path_with_filename

`path_with_filename(path, name) → string` — replace the final filename component.

pauli_decomposition_2x2

`pauli_decomposition_2x2` — operation `pauli decomposition 2x2`. Category: **more extensions**.

```
p pauli_decomposition_2x2 $x
```

pauli_real_part

`pauli_real_part` — operation `pauli real part`. Category: **quantum mechanics deep**.

```
p pauli_real_part $x
```

pauli_string_expect

`pauli_string_expect` — operation `pauli string expect`. Category: **quantum**.

```
p pauli_string_expect $x
```

pauli_y_imag

`pauli_y_imag` — operation `pauli y imag`. Category: **quantum mechanics deep**.

```
p pauli_y_imag $x
```

payback

`payback_period($initial, @cashflows)` (alias `payback`) — computes the number of periods to recover initial investment. Returns fractional periods.

```
p payback(1000, 300, 400, 500) # 2.6 periods
```

payback_period_simple

`payback_period_simple` — operation `payback period simple`. Category: **misc**.

```
p payback_period_simple $x
```

payback_simple

`payback_simple` — operation `payback simple`. Alias for `payback_period_simple`. Category: **misc**.

```
p payback_simple $x
```

payoff_matrix

`payoff_matrix` — operation `payoff matrix`. Category: **test runner**.

```
p payoff_matrix $x
```

pbeta

`pbeta` — beta CDF (regularized incomplete beta). Args: `x`, `a`, `b`.

```
p pbeta(0.5, 2, 5) # P(X ≤ 0.5) for Beta(2,5)
```

pbinom

`pbinom` — binomial CDF. $P(X \leq k)$ for $\text{Binom}(n, p)$. Args: `k`, `n`, `p`.

```
p pbinom(5, 10, 0.5) # P(≤5 heads in 10 flips)
```

pbkdf2_hmac_step

`pbkdf2_hmac_step` — single-step update of `pbkdf2 hmac`. Category: **cryptography deep**.

```
p pbkdf2_hmac_step $x
```

pbool

`pbool` — operation `pbool`. Alias for `parse_bool`. Category: **extended stdlib**.

```
p pbool $x
```


pby

pby — operation **pby**. Alias for **partition_by**. Category: **algebraic match**.

```
p pby $x
```

pc_case

pc_case — operation **pc case**. Alias for **pascal_case**. Category: **string**.

```
p pc_case $x
```

pc_to_au

pc_to_au — convert from **pc** to **au**. Category: **astronomy / music / color / units**.

```
p pc_to_au $value
```

pc_to_ly

pc_to_ly — convert from **pc** to **ly**. Category: **astronomy / music / color / units**.

```
p pc_to_ly $value
```

pca

prcomp (alias **pca**) — Principal Component Analysis via eigendecomposition of covariance matrix. Returns eigenvalues (variance explained). Like R's **prcomp()**.

```
val @var = @[pca([[1,2],[3,4],[5,6],[7,8]])]
# variance explained by each component
```

pcauchy

pcauchy — Cauchy CDF. $P(X \leq x)$. Args: *x* [, location, scale].

```
p pcauchy(0, 0, 1) # 0.5
```

pcg32_next

pcg32_next(state, inc) → int — PCG32 output for the current state (O'Neill 2014). To iterate, compute the next state as **state**·6364136223846793005 + **inc** and call again. **inc** must be odd; it identifies the PCG stream and does not affect this call's output.

pcg32_step

pcg32_step — single-step update of **pcg32**. Category: **cryptography deep**.

```
p pcg32_step $x
```

pchisq

pchisq — chi-squared CDF. Args: *x*, *df*.

```
p pchisq(3.84, 1) # 0.95 (critical value for p=0.05)
```

pct

`pct` — operation `pct`. Alias for `par_csv_read`. Category: **data / network**.

```
p pct $x
```

pct

`pct` — operation `pct`. Alias for `percent`. Category: **trivial numeric / predicate builtins**.

```
p pct $x
```

pctrank

`pctrank` — operation `pctrank`. Alias for `percentile_rank`. Category: **statistics (extended)**.

```
p pctrank $x
```

pdo_index_step

`pdo_index_step` — single-step update of `pdo index`. Category: **climate, fluids, atmospheric**.

```
p pdo_index_step $x
```

peak_db

`peak_db` — operation `peak db`. Category: **extras**.

```
p peak_db $x
```

peak_detect

`peak_detect` — operation `peak detect`. Category: **array / list operations extras**.

```
p peak_detect $x
```

peak_widths_at

`peak_widths_at` — operation `peak widths at`. Category: **economics + game theory**.

```
p peak_widths_at $x
```

peaks

`peaks` — operation `peaks`. Alias for `peak_detect`. Category: **array / list operations extras**.

```
p peaks $x
```

pearson_hash

`pearson_hash` — hash function of `pearson`. Alias for `pearson_hash_byte`. Category: **misc**.

```
p pearson_hash $x
```

pearson_hash_byte

`pearson_hash_byte` — operation `pearson hash byte`. Category: **misc**.

```
p pearson_hash_byte $x
```

pearson_skewness_2

`pearson_skewness_2` — operation `pearson skewness 2`. Category: **more extensions**.

```
p pearson_skewness_2 $x
```

pearsonr

`pearsonr` — operation `pearsonr`. Category: **r / scipy distributions and tests**.

```
p pearsonr $x
```

peclet_number_step

`peclet_number_step` — single-step update of `peclet number`. Category: **climate, fluids, atmospheric**.

```
p peclet_number_step $x
```

peek_clj

`peek_clj` — operation `peek clj`. Category: **algebraic match**.

```
p peek_clj $x
```

peekable

`peekable` — operation `peekable`. Category: **iterator + string-distance extras**.

```
p peekable $x
```

peeling_step_psi4

`peeling_step_psi4` — operation `peeling step psi4`. Category: **electrochemistry, batteries, fuel cells**.

```
p peeling_step_psi4 $x
```

peg_choice

`peg_choice` — operation `peg choice`. Category: **compilers / parsing**.

```
p peg_choice $x
```

peg_lookahead

`peg_lookahead` — operation `peg lookahead`. Category: **compilers / parsing**.

```
p peg_lookahead $x
```

peg_repeat

`peg_repeat` — operation `peg repeat`. Category: **compilers / parsing**.

```
p peg_repeat $x
```

peg_seq

`peg_seq` — operation `peg seq`. Category: **compilers / parsing**.

```
p peg_seq $x
```

peith

`peith` — operation `peith`. Alias for `partition_either`. Category: **rust iterator methods**.

```
p peith $x
```

pell_fundamental

`pell_fundamental` — operation `pell fundamental`. Category: **cryptanalysis & number theory deep**.

```
p pell_fundamental $x
```

pell_nth

`pell_nth(n) → int` — n-th Pell number $P(n) = 2 \cdot P(n-1) + P(n-2)$.

pendper

`pendulum_period($length)` (alias `pendper`) — period $T = 2\pi\sqrt{L/g}$ of a simple pendulum.

```
p pendper(1)      # ~2.01 s (1 meter pendulum)
p pendper(0.25)   # ~1.00 s (grandfather clock)
```

pendulum_freq

`pendulum_freq` — operation `pendulum freq`. Category: **misc**.

```
p pendulum_freq $x
```

penrose_diagram_factor

`penrose_diagram_factor` — factor of `penrose diagram`. Category: **electrochemistry, batteries, fuel cells**.

```
p penrose_diagram_factor $x
```

pentagonal_number

`pentagonal_number` — operation `pentagonal number`. Category: **number theory / primes**.

```
p pentagonal_number $x
```

pentagonal_pyramidal_number

`pentagonal_pyramidal_number` — operation `pentagonal pyramidal number`. Category: **misc**.

```
p pentagonal_pyramidal_number $x
```

pentnum

`pentnum` — operation `pentnum`. Alias for `pentagonal_number`. Category: **number theory / primes**.

```
p pentnum $x
```

percent

`percent` — operation `percent`. Category: **trivial numeric / predicate builtins**.

```
p percent $x
```

percent_decode_url

`percent_decode_url` — operation `percent decode url`. Category: **b81-misc-utility**.

```
p percent_decode_url $x
```

percent_encode_full

`percent_encode_full` — operation `percent encode full`. Category: **archive/encoding format primitives**.

```
p percent_encode_full $x
```

percentile

`percentile` — operation `percentile`. Category: **file stat / path**.

```
p percentile $x
```

percentile_cont

`percentile_cont` — operation `percentile cont`. Category: **postgres sql strings, json, aggregates**.

```
p percentile_cont $x
```

percentile_disc

`percentile_disc` — operation `percentile disc`. Category: **postgres sql strings, json, aggregates**.

```
p percentile_disc $x
```

percentile_rank

`percentile_rank($value, @data)` — returns the percentile rank (0-100) of a value within a dataset. Indicates what percentage of the data falls below the given value. Useful for ranking and normalization.

```
val @scores = 1:100
p percentile_rank(50, @scores)    # ~50
p percentile_rank(90, @scores)   # ~90
```

percentrank

`percentrank` — operation `percentrank`. Category: **excel/sheets + bond/loan financial**.

```
p percentrank $x
```

perfect_number

`perfect_number` — operation `perfect number`. Category: **math / number theory extras**.

```
p perfect_number $x
```

perfect_numbers

`perfect_numbers` — operation `perfect numbers`. Category: **number theory / primes**.

```
p perfect_numbers $x
```

perfnums

perfnums — operation **perfnums**. Alias for **perfect_numbers**. Category: *number theory / primes*.

```
p perfnums $x
```

perfview

perfview (alias **pfv**) — read the wall-clock perf log written by stryke runs launched with the **--record** CLI flag (or with **STRYKE_RECORD=1** in the environment). Every recorded invocation writes one row to **~/.stryke/perf.sqlite** (path, argv, started_ns, duration_ns, exit_code, version, host, pid, parent_pid). **perfview** queries that table and returns an arrayref of hashrefs.

No args = top 20 slowest. Options pass as **key => value** pairs:

top => N — limit to N rows (default 20; pass **all => 1** for no limit) **name => "SUB"** — substring filter on **path** **re => "REGEX"** — regex filter on **path** (alias: **regex**) **path => "/abs"** — exact-match filter on **path** **since => "7d"** — only rows newer than 7d / 24h / 30m / 90s / 2w **slowest => 1** — order by duration desc (default) **fastest => 1** — order by duration asc **recent => 1** — order by id desc (most recent first)

From the CLI, **s perfview** wraps the result in **ddump** so the table prints directly; from stryke code it composes with **grep** / **dd** / **to_csv** naturally.

```
# CLI quick view: top 20 slowest recorded runs
# $ s --record t examples/rosetta/t          # record one row per test file
# $ s perfview                               # show top 20 slowest
# $ s perfview top 5                         # top 5
# $ s perfview name rosetta                 # filter by path substring

# From stryke code:
perfview(top => 10) |> e { p "$_->{duration_ms}ms $_->{path}" }
perfview(name => "rosetta", since => "24h") |> dd
perfview(path => "/abs/path/to/script.stk") |> e { p $_->{duration_ns} }
perfview(all => 1) |> grep { $_->{exit_code} } |> e p # all failed runs
perfview(fastest => 1, top => 5) |> to_csv |> p
```

Schema columns returned: **id**, **path**, **argv**, **started_ns**, **duration_ns**, **duration_ms**, **exit_code**, **version**, **host**, **pid**, **parent_pid**. Auto-prunes rows older than 90 days every ~1000 inserts. SQLite is in WAL mode so **perfview** reads don't block concurrent parallel-test inserts.

See also: **--record** CLI flag, **~/.stryke/perf.sqlite**, **--profile** / **--flame** (per-function profiling within one script — orthogonal to wall-clock per-script recording).

perihelion_precession

perihelion_precession — operation **perihelion precession**. Category: *cosmology / gr / flrw*.

```
p perihelion_precession $x
```

perimeter_rectangle

perimeter_rectangle — operation **perimeter rectangle**. Category: *geometry / physics*.

```
p perimeter_rectangle $x
```

perimeter_triangle

perimeter_triangle — operation **perimeter triangle**. Category: *geometry / physics*.

```
p perimeter_triangle $x
```

period_of_string

period_of_string — operation **period of string**. Category: *more extensions*.

```
p period_of_string $x
```

periodogram_basic

`periodogram_basic` — operation `periodogram basic`. Category: *economics + game theory*.

```
p periodogram_basic $x
```

perlin_2d

`perlin_2d(x, y)` → `number` — classic Perlin gradient noise; deterministic per (x,y).

```
p perlin_2d(0.5, 0.5)
```

perlin_3d

`perlin_3d(x, y, z)` → `number` — Ken Perlin's 3D gradient noise: looks up 8 corner gradients of the unit cube, computes dot products with offset vectors, trilinearly interpolates through `fade(xf, yf, zf)`. Range ~[-1, 1].

perm

`permutations($n, $r)` (alias `perm`) — number of ways to arrange r items from n. Formula: $n!/(n-r)!$. Order matters.

```
p perm(5, 3)      # 60 (ways to arrange 3 items from 5)
p perm(10, 2)     # 90
```

perm_mul

`perm_mul` — operation `perm mul`. Alias for `permutation_compose`. Category: *misc*.

```
p perm_mul $x
```

perm_sign

`perm_sign` — sign with private key of `perm`. Alias for `permutation_parity`. Category: *misc*.

```
p perm_sign $x
```

permutation_compose

`permutation_compose` — operation `permutation compose`. Category: *misc*.

```
p permutation_compose $x
```

permutation_order

`permutation_order` — operation `permutation order`. Category: *misc*.

```
p permutation_order $x
```

permutation_parity

`permutation_parity` — operation `permutation parity`. Category: *misc*.

```
p permutation_parity $x
```

permutation_test

`permutation_test(a, b, n_perms=1000, seed=0)` → `number` — two-sided p-value: fraction of label permutations producing a mean-difference $\geq$ observed.

```
p permutation_test(\@group_a, \@group_b, 10000, 1)
```

permute_idx

`permute_idx` — operation `permute_idx`. Category: **apl/j/k array primitives**.

```
p permute_idx $x
```

perpetuity_value

`perpetuity_value` — value of `perpetuity`. Category: **misc**.

```
p perpetuity_value $x
```

persian_arithmetic_leap

`persian_arithmetic_leap` — operation `persian arithmetic leap`. Category: **calendrical algorithms**.

```
p persian_arithmetic_leap $x
```

persist

`persist` — operation `persist`. Category: **redis-flavour primitives**.

```
p persist $x
```

personalized_pagerank

`personalized_pagerank` — operation `personalized pagerank`. Category: **networkx graph algorithms**.

```
p personalized_pagerank $x
```

petersen_estimator

`petersen_estimator` — operation `petersen estimator`. Category: **biology / ecology**.

```
p petersen_estimator $x
```

peukert_capacity

`peukert_capacity` — operation `peukert capacity`. Category: **electrochemistry, batteries, fuel cells**.

```
p peukert_capacity $x
```

peukert_exponent_fit

`peukert_exponent_fit` — operation `peukert exponent fit`. Category: **electrochemistry, batteries, fuel cells**.

```
p peukert_exponent_fit $x
```


pexp

pexp — exponential CDF. $P(X \leq x)$ for $\text{Exp}(\text{rate})$. Args: x, rate.

```
p pexp(1, 1)      # 0.6321 (1 - e^-1)
p pexp(5, 0.5)    # P(wait ≤ 5 with avg wait 2)
```

pf

pf — F-distribution CDF. Args: x, d1, d2.

```
p pf(4.0, 5, 10)  # P(F ≤ 4) for F(5,10)
```

pfact

pfact — operation **pfact**. Alias for **prime_factors**. Category: *number theory / primes*.

```
p pfact $x
```

pfadd

pfadd — operation **pfadd**. Category: *redis-flavour primitives*.

```
p pfadd $x
```

pfcount

pfcount — operation **pfcount**. Category: *redis-flavour primitives*.

```
p pfcount $x
```

pflt

pflt — operation **pflt**. Alias for **parse_float**. Category: *extended stdlib*.

```
p pflt $x
```

pfreq

pfreq — operation **pfreq**. Alias for **pfrequencies**. Category: *pipeline / string helpers*.

```
p pfreq $x
```

pfrequencies

pfrequencies — operation **pfrequencies**. Category: *pipeline / string helpers*.

```
p pfrequencies $x
```

pfrq

pfrq — operation **pfrq**. Alias for **pfrequencies**. Category: *pipeline / string helpers*.

```
p pfrq $x
```

pft

pft — operation **pft**. Alias for **par_fetch**. Category: *data / network*.

```
p pft $x
```

pga_attenuation

`pga_attenuation` — operation `pga attenuation`. Category: *geology, seismology, mineralogy*.

```
p pga_attenuation $x
```

pgamma

`pgamma` — gamma CDF. Args: `x`, `shape` [, `scale`].

```
p pgamma(2, 2, 1) # P(X ≤ 2) for Gamma(2,1)
```

ph_at_iseoelectric

`ph_at_iseoelectric` — operation `ph at isoelectric`. Category: *electrochemistry, batteries, fuel cells*.

```
p ph_at_iseoelectric $x
```

ph_from_h

`ph_from_h` — operation `ph from h`. Category: *chemistry*.

```
p ph_from_h $x
```

phase_flip_prob

`phase_flip_prob` — operation `phase flip prob`. Category: *quantum mechanics deep*.

```
p phase_flip_prob $x
```

phase_spectrum

`phase_spectrum(\@signal)` — computes the phase angle (in radians) at each frequency bin from the DFT.

```
val $phases = phase_spectrum(\@signal)
```

phaser_simple

`phaser_simple(signal, rate=0.5, depth=1, sr=44100) → array` — 4-stage all-pass cascade with LFO-modulated coefficient. Each stage runs $y[n] = -a \cdot x[n] + x[n-1] + a \cdot y[n-1]$ where $a = \text{depth} \cdot \sin(2\pi \cdot \text{rate} \cdot t)$. Output is 50/50 wet/dry mix.

phasespec

`phasespec` — operation `phasespec`. Alias for `phase_spectrum`. Category: *dsp / signal (extended)*.

```
p phasespec $x
```

phillips_perron

`phillips_perron` — operation `phillips perron`. Category: *statsmodels*.

```
p phillips_perron $x
```

phonedigit

`phonedigit` — operation `phonedigit`. Alias for `phonetic_digit`. Category: **encoding / phonetics**.

```
p phonedigit $x
```

phonetic_digit

`phonetic_digit` — operation `phonetic digit`. Category: **encoding / phonetics**.

```
p phonetic_digit $x
```

phonex

`phonex(name) → string` — Phonex (Lait & Randell) phonetic key; improves Soundex by stripping silent leads.

```
p phonex("Robinson")
```

photocurrent_density

`photocurrent_density` — operation `photocurrent density`. Category: **electrochemistry, batteries, fuel cells**.

```
p photocurrent_density $x
```

photon_energy

`photon_energy($frequency)` (alias `photonenergy`) — energy $E = hf$ (Joules) of a photon.

```
p photonenergy(5e14) # ~3.3e-19 J (visible light)
```

photon_energy_freq

`photon_energy_freq` — operation `photon energy freq`. Category: **chemistry**.

```
p photon_energy_freq $x
```

photon_energy_lambda

`photon_energy_lambda` — operation `photon energy lambda`. Category: **chemistry**.

```
p photon_energy_lambda $x
```

photon_energy_wavelength

`photon_energy_wavelength` — operation `photon energy wavelength`. Category: **physics formulas**.

```
p photon_energy_wavelength $x
```

photon_sphere_radius

`photon_sphere_radius` — operation `photon sphere radius`. Category: **cosmology / gr / flrw**.

```
p photon_sphere_radius $x
```

photonlambda

`photonlambda` — operation `photonlambda`. Alias for `photon_energy_wavelength`. Category: **physics formulas**.

```
p photonlambda $x
```

physics_apply_force

`physics_apply_force` — operation `physics apply force`. Category: **extras**.

```
p physics_apply_force $x
```

physics_apply_impulse

`physics_apply_impulse` — operation `physics apply impulse`. Category: **extras**.

```
p physics_apply_impulse $x
```

physics_collide_aabb

`physics_collide_aabb` — operation `physics collide aabb`. Category: **extras**.

```
p physics_collide_aabb $x
```

physics_collide_sphere

`physics_collide_sphere` — operation `physics collide sphere`. Category: **extras**.

```
p physics_collide_sphere $x
```

physics_raycast

`physics_raycast` — operation `physics raycast`. Category: **extras**.

```
p physics_raycast $x
```

physics_step

`physics_step` — single-step update of `physics`. Category: **extras**.

```
p physics_step $x
```

pi

`pi` — operation `pi`. Category: **constants**.

```
p pi $x
```

pi_step_size

`pi_step_size` — operation `pi step size`. Category: **ode advanced**.

```
p pi_step_size $x
```

pick

`pick` — operation `pick`. Alias for `pick_keys`. Category: **hash ops**.

```
p pick $x
```

pick_keys

`pick_keys` — operation `pick keys`. Category: **hash ops**.

```
p pick_keys $x
```

picks_theorem

`picks_theorem` — operation `picks theorem`. Category: *geometry / topology*.

```
p picks_theorem $x
```

pid

`pid` — operation `pid`. Category: *system introspection*.

```
p pid $x
```

pid_anti_windup

`pid_anti_windup` — operation `pid anti windup`. Category: *robotics & control*.

```
p pid_anti_windup $x
```

pid_ziegler_nichols

`pid_ziegler_nichols` — operation `pid ziegler nichols`. Category: *robotics & control*.

```
p pid_ziegler_nichols $x
```

pielou_evenness

`pielou_evenness` — operation `pielou evenness`. Category: *biology / ecology*.

```
p pielou_evenness $x
```

pig_latin

`pig_latin` — operation `pig latin`. Category: *string processing extras*.

```
p pig_latin $x
```

piglat

`piglat` — operation `piglat`. Alias for `pig_latin`. Category: *string processing extras*.

```
p piglat $x
```

pin

`pin` — operation `pin`. Alias for `fire_and_forget`. Category: *stress testing*.

```
p pin $x
```

pint

`pint` — operation `pint`. Alias for `parse_int`. Category: *extended stdlib*.

```
p pint $x
```

pip

`point_in_polygon($x, $y, @polygon)` (alias `pip`) — tests if a point lies inside a polygon. Polygon is given as flat list `[x1,y1,x2,y2,...]`. Returns 1 if inside, 0 if outside. Uses ray-casting algorithm.

```
val @square = (0,0, 4,0, 4,4, 0,4)
p pip(2, 2, @square)    # 1 (inside)
p pip(5, 5, @square)    # 0 (outside)
```

pipe_pressure_drop

pipe_pressure_drop — operation `pipe pressure drop`. Category: **misc**.

```
p pipe_pressure_drop $x
```

pipes

pipes — operation `pipes`. Category: **filesystem extensions**.

```
p pipes $x
```

pivot_points

pivot_points(high, low, close) → hash — classic floor-trader pivots. Returns {pivot, r1, r2, s1, s2} where pivot = (h+l+c)/3.

```
p pivot_points(110, 95, 100)
```

pivoted_cholesky_var

pivoted_cholesky_var — variance of `pivoted cholesky`. Category: **statsmodels**.

```
p pivoted_cholesky_var $x
```

pka_from_ka

pka_from_ka — operation `pka from ka`. Category: **chemistry**.

```
p pka_from_ka $x
```

pkb_to_kb

pkb_to_kb — convert from `pkb` to `kb`. Category: **chemistry & biochemistry**.

```
p pkb_to_kb $value
```

pkc

pkc — operation `pkc`. Alias for `peek_clj`. Category: **algebraic match**.

```
p pkc $x
```

pkcs7_pad

pkcs7_pad — operation `pkcs7 pad`. Category: **more extensions (2)**.

```
p pkcs7_pad $x
```

pkcs7_unpad

pkcs7_unpad — operation `pkcs7 unpad`. Category: **more extensions (2)**.

```
p pkcs7_unpad $x
```

plactic_class_size

`plactic_class_size` — operation `plactic class size`. Category: **econometrics**.

```
p plactic_class_size $x
```

planck

`planck` — operation `planck`. Category: **math / physics constants**.

```
p planck $x
```

planck_const_h

`planck_const_h` — operation `planck const h`. Category: **misc**.

```
p planck_const_h $x
```

planck_constant

`planck_constant` — operation `planck constant`. Category: **constants**.

```
p planck_constant $x
```

planck_energy

`planck_energy` — operation `planck energy`. Category: **cosmology / gr / flrw**.

```
p planck_energy $x
```

planck_mass

`planck_mass` — operation `planck mass`. Category: **physics constants**.

```
p planck_mass $x
```

planck_spectral_radiance

`planck_spectral_radiance` — operation `planck spectral radiance`. Category: **cosmology / gr / flrw**.

```
p planck_spectral_radiance $x
```

planck_temperature

`planck_temperature` — operation `planck temperature`. Category: **physics constants**.

```
p planck_temperature $x
```

planck_time

`planck_time` — operation `planck time`. Category: **physics constants**.

```
p planck_time $x
```

plane_distance_to_point

`plane_distance_to_point(point, normal, plane_point)` → `number` — signed perpendicular distance.

plane_normalize

`plane_normalize(normal)` → `[x,y,z]` — unit-length plane normal.

plastic

`plastic` — operation `plastic`. Alias for `plastic_number`. Category: **math constants**.

```
p plastic $x
```

plastic_number

`plastic_number` — operation `plastic number`. Category: **math constants**.

```
p plastic_number $x
```

plate_reverb_simple

`plate_reverb_simple` — operation `plate reverb simple`. Category: **test runner**.

```
p plate_reverb_simple $x
```

plethystic_substitution_value

`plethystic_substitution_value` — value of `plethystic substitution`. Category: **econometrics**.

```
p plethystic_substitution_value $x
```

plnorm

`plnorm` — log-normal CDF. $P(X \leq x)$ for $\text{LogN}(\mu, \sigma)$. Args: `x`, `mu`, `sigma`.

```
p plnorm(1, 0, 1) # 0.5 (median of LogN(0,1))
```

plur

`plur` — operation `plur`. Alias for `pluralize`. Category: **extended stdlib**.

```
p plur $x
```

plurality_winner_step

`plurality_winner_step` — single-step update of `plurality winner`. Category: **climate, fluids, atmospheric**.

```
p plurality_winner_step $x
```

pluralize

`pluralize` — operation `pluralize`. Category: **extended stdlib**.

```
p pluralize $x
```

pluralize_simple

`pluralize_simple` — operation `pluralize simple`. Category: **misc**.

```
p pluralize_simple $x
```


plus_polarization_amp

`plus_polarization_amp` — operation `plus polarization amp`. Category: **electrochemistry, batteries, fuel cells**.

```
p plus_polarization_amp $x
```

pmass

`pmass` — operation `pmass`. Alias for `proton_mass`. Category: **constants**.

```
p pmass $x
```

pmt

`pmt($rate, $nper, $pv)` — computes the periodic payment for a loan given interest rate, number of periods, and present value. Standard amortization formula.

```
# $100k loan, 5% annual rate, 30 years
p pmt(0.05/12, 360, 100000) # ~$537/month
```

png_header_read

`png_header_read(hex_bytes) → hash` — parse PNG IHDR → {format, width, height, bit_depth, color_type}.

pnorm

`pnorm` — normal CDF. $P(X \leq x)$ for $N(\mu, \sigma)$. Args: `x` [, `mu`, `sigma`]. Default standard normal.

```
p pnorm(0)           # 0.5
p pnorm(1.96)        # 0.975
p pnorm(100, 90, 15) # z-score for IQ 100
```

pochhammer

`pochhammer` — operation `pochhammer`. Category: **special functions extra**.

```
p pochhammer $x
```

poh_from_oh

`poh_from_oh` — operation `poh from oh`. Category: **chemistry**.

```
p poh_from_oh $x
```

poincare_duality

`poincare_duality` — operation `poincare duality`. Category: **econometrics**.

```
p poincare_duality $x
```

poincare_polynomial_eval

`poincare_polynomial_eval` — operation `poincare polynomial eval`. Category: **econometrics**.

```
p poincare_polynomial_eval $x
```

point_angle

`point_angle` — operation `point angle`. Category: *complex / geom / color / trig*.

```
p point_angle $x
```

point_distance

`point_distance` — distance / dissimilarity metric of `point`. Category: *geometry / physics*.

```
p point_distance $x
```

point_in_polygon_ray

`point_in_polygon_ray` — operation `point in polygon ray`. Category: *excel/sheets + bond/loan financial*.

```
p point_in_polygon_ray $x
```

point_in_polygon_winding

`point_in_polygon_winding` — operation `point in polygon winding`. Category: *excel/sheets + bond/loan financial*.

```
p point_in_polygon_winding $x
```

pointer_width

`pointer_width` — pointer width in bits: `32` or `64`. Compile-time constant.

```
p pointer_width # 64
```

poisson

`poisson_pmf($k, $lambda)` (alias `poisson`) — Poisson probability mass function. $P(X=k)$ for events with rate λ .

```
p poisson(3, 5) # P(X=3) when avg rate is 5
```

poisson_boltzmann_step

`poisson_boltzmann_step` — single-step update of `poisson boltzmann`. Category: *electrochemistry, batteries, fuel cells*.

```
p poisson_boltzmann_step $x
```

poisson_brackets

`poisson_brackets(df_dq, df_dp, dg_dq, dg_dp) → number` — 1D $\{f, g\} = \partial f / \partial q \cdot \partial g / \partial p - \partial f / \partial p \cdot \partial g / \partial q$.

poisson_photon_pmf

`poisson_photon_pmf` — operation `poisson photon pmf`. Category: *quantum mechanics deep*.

```
p poisson_photon_pmf $x
```

polar_to_cartesian

`polar_to_cartesian` — convert from `polar` to `cartesian`. Category: *complex / geom / color / trig*.

```
p polar_to_cartesian $value
```

polar_vortex_radius

`polar_vortex_radius` — operation `polar vortex radius`. Category: **climate, fluids, atmospheric**.

```
p polar_vortex_radius $x
```

policy_iteration_step

`policy_iteration_step` — single-step update of `policy iteration`. Category: **electrochemistry, batteries, fuel cells**.

```
p policy_iteration_step $x
```

pollard_p_minus_1

`pollard_p_minus_1` — operation `pollard p minus 1`. Category: **cryptanalysis & number theory deep**.

```
p pollard_p_minus_1 $x
```

poly1305_block_step

`poly1305_block_step` — single-step update of `poly1305 block`. Category: **more extensions (2)**.

```
p poly1305_block_step $x
```

poly_eval

`poly_eval` (alias `polyval`) evaluates a polynomial $c_0 + c_1 \cdot x + c_2 \cdot x^2 + \dots$ using Horner's method.

```
p polyval([1, 0, 1], 3) # 1 + 0*3 + 1*9 = 10
```

polyarea

`polyarea` — operation `polyarea`. Alias for `polygon_area`. Category: **geometry / physics**.

```
p polyarea $x
```

polyfit

`polynomial_fit` (alias `polyfit`) performs least-squares polynomial fitting. Returns coefficients $[c_0, c_1, \dots, c_n]$.

```
val $c = polyfit([0,1,2,3], [1,3,5,7], 1) # linear fit
```

polyfit_rmse

`polyfit_rmse` — operation `polyfit rmse`. Category: **numpy + scipy.special**.

```
p polyfit_rmse $x
```

polygamma

`polygamma` — operation `polygamma`. Category: **numpy + scipy.special**.

```
p polygamma $x
```

polygon_area

`polygon_area` — 2D polygon op `area`. Category: **geometry / physics**.

```
p polygon_area $x
```

polygon_area_signed

`polygon_area_signed` — 2D polygon op `area signed`. Category: *more extensions (2)*.

```
p polygon_area_signed $x
```

polygon_contains_point

`polygon_contains_point` — 2D polygon op `contains point`. Category: *complex / geom / color / trig*.

```
p polygon_contains_point $x
```

polygon_convex

`polygon_convex` — 2D polygon op `convex`. Category: *complex / geom / color / trig*.

```
p polygon_convex $x
```

polygon_convex_hull_2d

`polygon_convex_hull_2d` — 2D polygon op `convex hull 2d`. Category: *complex / geom / color / trig*.

```
p polygon_convex_hull_2d $x
```

polygon_convex_q

`polygon_convex_q` — 2D polygon op `convex q`. Category: *more extensions (2)*.

```
p polygon_convex_q $x
```

polygon_inflate

`polygon_inflate` — 2D polygon op `inflate`. Category: *test runner*.

```
p polygon_inflate $x
```

polygon_is_convex

`polygon_is_convex` — 2D polygon op `is convex`. Category: *geometry / topology*.

```
p polygon_is_convex $x
```

polygon_offset

`polygon_offset(points, d=1) → array` — perpendicular-distance offset using the corner-bisector formula $\text{vertex} + d \cdot (n1 + n2) / (1 + n1 \cdot n2)$, where `n1`, `n2` are the outward unit normals of the adjacent edges. Gives an exact perpendicular offset (not just `d`-bisector_dir); falls back to bisector direction when $1 + n1 \cdot n2 \approx 0$.

polygon_orientation

`polygon_orientation` — 2D polygon op `orientation`. Category: *complex / geom / color / trig*.

```
p polygon_orientation $x
```

polygon_perimeter

`polygon_perimeter(@points)` (alias `polyper`) — computes the perimeter of a polygon. Points as flat list [x1,y1,x2,y2,...]. Assumes closed polygon.

```
val @square = (0,0, 1,0, 1,1, 0,1)
p polyper(@square)    # 4
```

polygon_reverse

`polygon_reverse` — 2D polygon op `reverse`. Category: *complex / geom / color / trig*.

```
p polygon_reverse $x
```

polygon_self_intersects

`polygon_self_intersects(points)` → 0|1 — inverse of `polygon_simple_check`.

polygon_shrink

`polygon_shrink(points, d=1)` → array — `polygon_offset` with negative `d`.

polygon_signed_area

`polygon_signed_area` — 2D polygon op `signed area`. Category: *complex / geom / color / trig*.

```
p polygon_signed_area $x
```

polygon_simple_check

`polygon_simple_check(points)` → 0|1 — 1 if polygon edges don't self-intersect.

polygon_simplify_dp

`polygon_simplify_dp` — 2D polygon op `simplify dp`. Category: *complex / geom / color / trig*.

```
p polygon_simplify_dp $x
```

polygon_winding

`polygon_winding(points)` → -1|0|1 — sign of polygon orientation: 1 CCW, -1 CW.

polygon_winding_number

`polygon_winding_number` — 2D polygon op `winding number`. Category: *more extensions (2)*.

```
p polygon_winding_number $x
```

polygon_winding_order

`polygon_winding_order` — 2D polygon op `winding order`. Category: *excel/sheets + bond/loan financial*.

```
p polygon_winding_order $x
```

polygon_with_holes_area

`polygon_with_holes_area` — 2D polygon op `with holes area`. Category: *geometry / topology*.

```
p polygon_with_holes_area $x
```

polylog_li2

`polylog_li2` — operation `polylog li2`. Category: **special functions extra**.

```
p polylog_li2 $x
```

polylog_n

`polylog_n` — operation `polylog n`. Category: **special functions extra**.

```
p polylog_n $x
```

polynomial_jones_skein

`polynomial_jones_skein` — operation `polynomial jones skein`. Category: **econometrics**.

```
p polynomial_jones_skein $x
```

polynomial_roots_dk

`polynomial_roots_dk` — operation `polynomial roots dk`. Category: **misc**.

```
p polynomial_roots_dk $x
```

polyperim

`polyperim` — operation `polyperim`. Alias for `polygon_perimeter`. Category: **geometry (extended)**.

```
p polyperim $x
```

pooling_test_stat

`pooling_test_stat` — operation `pooling test stat`. Category: **econometrics**.

```
p pooling_test_stat $x
```

pop_clj

`pop_clj` — operation `pop clj`. Category: **algebraic match**.

```
p pop_clj $x
```

popc

`popc` — operation `popc`. Alias for `pop_clj`. Category: **algebraic match**.

```
p popc $x
```

popcnt

`popcnt` — operation `popcnt`. Alias for `hamming_weight`. Category: **misc**.

```
p popcnt $x
```

popcount

`popcount` — operation `popcount`. Alias for `bits_count`. Category: **base conversion**.

```
p popcount $x
```

popcount_u32

`popcount_u32(x)` → `int` — population count of the 32 lowest bits.

popcount_u64

`popcount_u64(x)` → `int` — population count over all 64 bits (alias for `bit_count_ones`).

popd

`popd` — pop the top entry off the directory stack and `chdir` to it. Returns the new cwd. Errors when the stack is empty (`popd: directory stack empty`). Pair with `pushd`.

porosity_active_layer

`porosity_active_layer` — operation `porosity active layer`. Category: **electrochemistry, batteries, fuel cells**.

```
p porosity_active_layer $x
```

port_is_assigned

`port_is_assigned` — operation `port is assigned`. Category: **network / ip / cidr**.

```
p port_is_assigned $x
```

port_is_dynamic

`port_is_dynamic` — operation `port is dynamic`. Category: **network / ip / cidr**.

```
p port_is_dynamic $x
```

port_is_ephemeral

`port_is_ephemeral` — operation `port is ephemeral`. Category: **network / ip / cidr**.

```
p port_is_ephemeral $x
```

port_is_registered

`port_is_registered` — operation `port is registered`. Category: **network / ip / cidr**.

```
p port_is_registered $x
```

port_is_well_known

`port_is_well_known` — operation `port is well known`. Category: **network / ip / cidr**.

```
p port_is_well_known $x
```

port_name

`port_name` — operation `port name`. Category: **network / ip / cidr**.

```
p port_name $x
```

port_parse_range

`port_parse_range` — operation `port parse range`. Category: **network / ip / cidr**.

```
p port_parse_range $x
```

port_random_ephemeral

`port_random_ephemeral` — operation `port random ephemeral`. Category: **network / ip / cidr**.

```
p port_random_ephemeral $x
```

port_service_lookup

`port_service_lookup` — operation `port service lookup`. Category: **network / ip / cidr**.

```
p port_service_lookup $x
```

port_to_service

`port_to_service` — convert from `port` to `service`. Category: **network / ip / cidr**.

```
p port_to_service $value
```

porter_stem_step

`porter_stem_step` — single-step update of `porter stem`. Category: **computational linguistics**.

```
p porter_stem_step $x
```

pos_n

`pos_n` — operation `pos n`. Alias for `positive`. Category: **trivial numeric / predicate builtins**.

```
p pos_n $x
```

poset_zeta_two

`poset_zeta_two` — operation `poset zeta two`. Category: **econometrics**.

```
p poset_zeta_two $x
```

position_max

`position_max` — maximum of `position`. Category: **iterator + string-distance extras**.

```
p position_max $x
```

position_max_by

`position_max_by` — operation `position max by`. Category: **iterator + string-distance extras**.

```
p position_max_by $x
```

position_min

`position_min` — minimum of `position`. Category: **iterator + string-distance extras**.

```
p position_min $x
```


position_min_by

`position_min_by` — operation `position min by`. Category: **iterator + string-distance extras**.

```
p position_min_by $x
```

position_of_all_matching

`position_of_all_matching` — operation `position of all matching`. Category: **misc**.

```
p position_of_all_matching $x
```

positions_of

`positions_of` — operation `positions of`. Category: **collection**.

```
p positions_of $x
```

positions_of_all

`positions_of_all` — operation `positions of all`. Alias for `position_of_all_matching`. Category: **misc**.

```
p positions_of_all $x
```

positive

`positive` — operation `positive`. Category: **trivial numeric / predicate builtins**.

```
p positive $x
```

post_newtonian_step

`post_newtonian_step` — single-step update of `post newtonian`. Category: **electrochemistry, batteries, fuel cells**.

```
p post_newtonian_step $x
```

posted_price_offer_accept

`posted_price_offer_accept` — operation `posted price offer accept`. Category: **climate, fluids, atmospheric**.

```
p posted_price_offer_accept $x
```

posterior_predictive_beta

`posterior_predictive_beta(alpha, beta) → number` — posterior predictive probability of success = $\alpha/(\alpha+\beta)$.

posterior_predictive_normal

`posterior_predictive_normal(mean, variance) → {mean, variance}` — posterior predictive parameters (identity for the conjugate Normal case).

potential_energy

`potential_energy` — operation `potential energy`. Category: **physics formulas**.

```
p potential_energy $x
```

potential_temperature_step

`potential_temperature_step` — single-step update of `potential temperature`. Category: **climate, fluids, atmospheric**.

```
p potential_temperature_step $x
```

pourbaix_line_value

`pourbaix_line_value` — value of `pourbaix line`. Category: **electrochemistry, batteries, fuel cells**.

```
p pourbaix_line_value $x
```

pow2

`pow2` — operation `pow2`. Category: **trivial numeric / predicate builtins**.

```
p pow2 $x
```

power_density_battery

`power_density_battery` — operation `power density battery`. Category: **electrochemistry, batteries, fuel cells**.

```
p power_density_battery $x
```

power_i2r

`power_i2r` — operation `power i2r`. Category: **em / optics / relativity**.

```
p power_i2r $x
```

power_law_wind_profile

`power_law_wind_profile` — operation `power law wind profile`. Category: **climate, fluids, atmospheric**.

```
p power_law_wind_profile $x
```

power_phys

`power_phys` — operation `power phys`. Category: **physics formulas**.

```
p power_phys $x
```

power_residue_check

`power_residue_check` — operation `power residue check`. Category: **cryptanalysis & number theory deep**.

```
p power_residue_check $x
```

power_set

`power_set` — operation `power set`. Category: **extended stdlib**.

```
p power_set $x
```

power_spectrum

`power_spectrum(@signal)` (alias `psd`) — computes the power spectral density (magnitude squared of DFT). Returns real values representing power at each frequency bin.

```
val $psd = psd(\@signal)
@$psd |> e p
```

power_spectrum_slope

`power_spectrum_slope` — operation `power spectrum slope`. Category: **climate, fluids, atmospheric**.

```
p power_spectrum_slope $x
```

power_vi

`power_vi` — operation `power vi`. Category: **em / optics / relativity**.

```
p power_vi $x
```

powers_of_seq

`powers_of_seq` — operation `powers of seq`. Category: **sequences**.

```
p powers_of_seq $x
```

powerset

`powerset` — operation `powerset`. Alias for `power_set`. Category: **extended stdlib**.

```
p powerset $x
```

powspec

`powspec` — operation `powspec`. Alias for `power_spectrum`. Category: **dsp / signal (extended)**.

```
p powspec $x
```

poynting_magnitude

`poynting_magnitude` — operation `poynting magnitude`. Category: **em / optics / relativity**.

```
p poynting_magnitude $x
```

pp_test_stat

`pp_test_stat` — operation `pp test stat`. Category: **econometrics**.

```
p pp_test_stat $x
```

ppi

`ppi` — operation `ppi`. Alias for `prime_pi`. Category: **number theory / primes**.

```
p ppi $x
```

ppid

`ppid` — operation `ppid`. Category: **system introspection**.

```
p ppid $x
```

ppm_predict_prob

ppm_predict_prob — operation ppm predict prob. Category: *electrochemistry, batteries, fuel cells*.

```
p ppm_predict_prob $x
```

ppmt

ppmt — operation ppmt. Category: *excel/sheets + bond/loan financial*.

```
p ppmt $x
```

ppo_advantage_step

ppo_advantage_step — single-step update of ppo advantage. Category: *electrochemistry, batteries, fuel cells*.

```
p ppo_advantage_step $x
```

ppo_clip_term

ppo_clip_term — operation ppo clip term. Category: *electrochemistry, batteries, fuel cells*.

```
p ppo_clip_term $x
```

ppois

ppois — Poisson CDF. $P(X \leq k)$ for Poisson(lambda). Args: k, lambda.

```
p ppois(3, 2.5) # P(≤3 events when avg is 2.5)
```

ppt

ppt — operation ppt. Alias for partition_point. Category: *extended stdlib*.

```
p ppt $x
```

prandtl_number_step

prandtl_number_step — single-step update of prandtl number. Category: *climate, fluids, atmospheric*.

```
p prandtl_number_step $x
```

pratt_parse_step

pratt_parse_step — single-step update of pratt parse. Category: *compilers / parsing*.

```
p pratt_parse_step $x
```

precession_iau2006

precession_iau2006 — operation precession iau2006. Category: *astronomy / astrometry*.

```
p precession_iau2006 $x
```

prect

prect — operation prect. Alias for perimeter_rectangle. Category: *geometry / physics*.

```
p prect $x
```

pred

`pred` — operation `pred`. Category: *conversion / utility*.

```
p pred $x
```

predict

`predict_lm` (alias `predict`) — predict from a linear model at new x values.

```
val $model = lm([1,2,3], [2,4,6])
var @pred = @{predict($model, [4,5,6])} # [8,10,12]
```

predictor_corrector

`predictor_corrector` — operation `predictor corrector`. Category: *ode advanced*.

```
p predictor_corrector $x
```

prefix_sums

`prefix_sums` — operation `prefix sums`. Category: *list helpers*.

```
p prefix_sums $x
```

prelu

`prelu` — operation `prelu`. Category: *ml extensions*.

```
p prelu $x
```

premium_net

`premium_net` — operation `premium net`. Category: *actuarial science*.

```
p premium_net $x
```

prepend

`prepend` — operation `prepend`. Category: *list helpers*.

```
p prepend $x
```

present_value

`present_value(future_value, rate, periods) → number` — $FV / (1 + rate)^{periods}$. Discount a single future cashflow to today.

```
p present_value(1000, 0.05, 10) # 613.91
```

pressure_altitude_m

`pressure_altitude_m` — operation `pressure altitude m`. Category: *misc*.

```
p pressure_altitude_m $x
```

prev_prime

`prev_prime` — operation `prev prime`. Category: *number theory / primes*.

```
p prev_prime $x
```

preventive_fraction

`preventive_fraction` — operation `preventive fraction`. Category: *epidemiology / public health*.

```
p preventive_fraction $x
```

prewitt_kernel_value

`prewitt_kernel_value` — value of `prewitt kernel`. Category: *electrochemistry, batteries, fuel cells*.

```
p prewitt_kernel_value $x
```

prewitt_x_kernel

`prewitt_x_kernel()` → `matrix` — 3×3 Prewitt horizontal-gradient kernel (lighter than Sobel).

prewitt_y_kernel

`prewitt_y_kernel()` → `matrix` — 3×3 Prewitt vertical-gradient kernel.

price_disc

`price_disc` — operation `price disc`. Category: *b81-misc-utility*.

```
p price_disc $x
```

price_elasticity

`price_elasticity` — operation `price elasticity`. Category: *economics + game theory*.

```
p price_elasticity $x
```

prim_step

`prim_step` — single-step update of `prim`. Category: *networkx graph algorithms*.

```
p prim_step $x
```

prime_count_below

`prime_count_below` — prime-number helper `count below`. Category: *math / number theory extras*.

```
p prime_count_below $x
```

prime_factors

`prime_factors` — prime-number helper `factors`. Category: *number theory / primes*.

```
p prime_factors $x
```

prime_pi

`prime_pi` — prime-number helper `pi`. Category: *number theory / primes*.

```
p prime_pi $x
```

primes_seq

`primes_seq` — operation `primes seq`. Category: *sequences*.

```
p primes_seq $x
```

primes_up_to

`primes_up_to` — operation `primes up to`. Category: *number theory / primes*.

```
p primes_up_to $x
```

primorial

`primorial(n) → int` — product of primes $\leq n$.

```
p primorial(10) # 2·3·5·7 = 210
```

primorial_n

`primorial_n` — operation `primorial n`. Category: *cryptanalysis & number theory deep*.

```
p primorial_n $x
```

prior_jeffreys_uniform

`prior_jeffreys_uniform() → {alpha, beta}` — Jeffreys prior for Bernoulli `Beta(0.5, 0.5)`.

prisoners_dilemma_payoff

`prisoners_dilemma_payoff` — operation `prisoners dilemma payoff`. Category: *climate, fluids, atmospheric*.

```
p prisoners_dilemma_payoff $x
```

prisoners_repeated_eq

`prisoners_repeated_eq` — operation `prisoners repeated eq`. Category: *climate, fluids, atmospheric*.

```
p prisoners_repeated_eq $x
```

priv

`priv` marks a field or method as private. Private members can only be accessed from within the class.

```
class Secret {
  priv hidden: Int = 42
  priv fn internal { }
  pub fn reveal { $self->hidden } # internal access ok
}
# $obj->hidden # ERROR: field is private
```

prim_node_connect

`prim_node_connect` — operation `prim node connect`. Category: **robotics & control**.

```
p prim_node_connect $x
```

probability_density

`probability_density` — operation `probability density`. Category: **quantum mechanics deep**.

```
p probability_density $x
```

probit_log_likelihood

`probit_log_likelihood` — operation `probit log likelihood`. Category: **econometrics**.

```
p probit_log_likelihood $x
```

proc_mem

`proc_mem` (alias `rss`) — current process resident set size (RSS) in bytes. On Linux reads `VmRSS` from `/proc/self/status`. On macOS uses `getrusage(RUSAGE_SELF)`. Returns `undef` on unsupported platforms.

```
p format_bytes(rss)                # 18.5 MiB
var $before = rss
do_work()
p format_bytes(rss - $before) . " allocated"
```

producer_surplus

`producer_surplus` — operation `producer surplus`. Category: **economics + game theory**.

```
p producer_surplus $x
```

product_digits

`product_digits` — operation `product digits`. Category: **math / number theory extras**.

```
p product_digits $x
```

product_list

`product_list` — operation `product list`. Category: **list helpers**.

```
p product_list $x
```

product_of

`product_of` — operation `product of`. Category: **more list helpers**.

```
p product_of $x
```

profile_hmm_score

`profile_hmm_score` — score of `profile hmm`. Category: **test runner**.

```
p profile_hmm_score $x
```


profit_margin_pct

`profit_margin_pct(revenue, cost) → number` — $(\text{revenue} - \text{cost}) / \text{revenue} \cdot 100$.

```
p profit_margin_pct(150, 100)
```

proj

`proj` — operation `proj`. Alias for `vector_project`. Category: **misc**.

```
p proj $x
```

projectile_max_height

`projectile_max_height($velocity, $angle)` (alias `projheight`) — maximum height $H = v^2 \sin^2(\theta) / (2g)$.

```
p projheight(100, 45) # ~255 m
p projheight(100, 90) # ~510 m (straight up)
```

projectile_position

`projectile_position(v0, angle_deg, g=9.81, t) → [x, y]` — position at time `t` for given launch velocity/angle.

```
p projectile_position(10, 45, 9.81, 0.5)
```

projectile_range

`projectile_range($velocity, $angle)` (alias `projrange`) — horizontal range $R = v^2 \sin(2\theta) / g$. Angle in degrees.

```
p projrange(100, 45) # ~1020 m (optimal angle for range)
p projrange(100, 30) # ~884 m
```

projectile_time

`projectile_time` — operation `projectile time`. Category: **physics formulas**.

```
p projectile_time $x
```

projectile_velocity

`projectile_velocity(v0, angle_deg, g=9.81, t) → [vx, vy]` — velocity components at time `t`.

projtime

`projtime` — operation `projtime`. Alias for `projectile_time`. Category: **physics formulas**.

```
p projtime $x
```

pronic_number

`pronic_number` — operation `pronic number`. Category: **misc**.

```
p pronic_number $x
```

prop_table

`prop_table` — convert a count matrix to proportions (each cell / total). Like R's `prop.table()`.

```
p prop_table([[10,20],[30,40]]) # [[0.1,0.2],[0.3,0.4]]
```

prop_test

`prop_test` — one-sample proportion z-test. Returns [z-statistic, p-value]. Like R's `prop.test()`.

```
var ($z, $pval) = @{prop_test(55, 100, 0.5)} # test if 55/100 differs from 50%%
```

proper_motion_apply

`proper_motion_apply` — operation `proper motion apply`. Category: **astronomy / astrometry**.

```
p proper_motion_apply $x
```

proportion_test

`proportion_test(x, n, p0=0.5)` → {`statistic`, `df`, `p_value`} — one-sample z-test for a proportion.

```
p proportion_test(45, 100, 0.5)
```

proportional_share

`proportional_share` — operation `proportional share`. Category: **climate, fluids, atmospheric**.

```
p proportional_share $x
```

protein_charge_at_ph

`protein_charge_at_ph(seq, ph=7)` → `number` — net charge at given pH using Henderson-Hasselbalch on each ionisable residue.

protein_hydrophobicity

`protein_hydrophobicity(seq)` → `number` — average Kyte-Doolittle hydrophathy.

```
p protein_hydrophobicity("GAVIL") # 2.78
```

protein_molecular_weight

`protein_molecular_weight(seq)` → `number` — molecular weight in Da. Sums per-residue weights, subtracts (n-1)·18.02 (water lost in peptide bonds).

```
p protein_molecular_weight("GAVIL")
```

protein_pI

`protein_pI(seq)` → `number` — isoelectric point via bisection on net-charge curve using standard pKa table.

```
p protein_pI("MKVL")
```

proton_mass

`proton_mass` — operation `proton mass`. Category: **constants**.

```
p proton_mass $x
```

proximal_gradient_step

`proximal_gradient_step` — single-step update of `proximal_gradient`. Category: *combinatorial optimization, scheduling*.

```
p proximal_gradient_step $x
```

prvp

`prvp` — operation `prvp`. Alias for `prev_prime`. Category: *number theory / primes*.

```
p prvp $x
```

pseudoprime_base2

`pseudoprime_base2` — operation `pseudoprime_base2`. Category: *cryptanalysis & number theory deep*.

```
p pseudoprime_base2 $x
```

psi_to_pascals

`psi_to_pascals` — convert from `psi` to `pascals`. Category: *file stat / path*.

```
p psi_to_pascals $value
```

pt

`pt` — Student's t CDF. Args: x, df.

```
p pt(1.96, 100) # ≈ 0.975
```

ptdist

`ptdist` — operation `ptdist`. Alias for `point_distance`. Category: *geometry / physics*.

```
p ptdist $x
```

ptinpoly

`ptinpoly` — operation `ptinpoly`. Alias for `point_in_polygon`. Category: *geometry (extended)*.

```
p ptinpoly $x
```

ptri

`ptri` — operation `ptri`. Alias for `perimeter_triangle`. Category: *geometry / physics*.

```
p ptri $x
```

pttl

`pttl` — operation `pttl`. Category: *redis-flavour primitives*.

```
p pttl $x
```

pub

`pub` marks a field or method as public (default visibility).

```
class Example {
  pub name: Str      # explicitly public
  pub fn greet { }   # explicitly public
}
```

public_goods_game_payoff

`public_goods_game_payoff` — operation `public_goods_game_payoff`. Category: **climate, fluids, atmospheric**.

```
p public_goods_game_payoff $x
```

pulse_pressure

`pulse_pressure` — operation `pulse_pressure`. Category: **misc**.

```
p pulse_pressure $x
```

punct

`punctuation` (alias `punct`) extracts all ASCII punctuation characters from a string and returns them as a list. Filters out letters, digits, whitespace — keeps only punctuation.

```
p join "", punctuation("Hello, world!") # ,!
"a.b-c" |> punct |> cnt |> p              # 2
```

punif

`punif` — uniform CDF. $P(X \leq x)$ for `Uniform(a, b)`. Args: `x`, `a`, `b`.

```
p punif(0.5, 0, 1) # 0.5
p punif(3, 0, 10) # 0.3
```

punycode_decode_step

`punycode_decode_step` — single-step update of `punycode_decode`. Category: **b81-misc-utility**.

```
p punycode_decode_step $x
```

punycode_encode

`punycode_encode` — encode of `punycode`. Category: **archive/encoding format primitives**.

```
p punycode_encode $x
```

pure_state_density

`pure_state_density` — operation `pure_state_density`. Category: **quantum mechanics deep**.

```
p pure_state_density $x
```

purity

`purity` — operation `purity`. Category: **quantum mechanics deep**.

```
p purity $x
```

purple

purple — operation **purple**. Alias for **red**. Category: **color / ansi**.

```
p purple $x
```

push_relabel_relabel

push_relabel_relabel — operation **push** **relabel** **relabel**. Category: **networkx graph algorithms**.

```
p push_relabel_relabel $x
```

push_relabel_step

push_relabel_step — single-step update of **push** **relabel**. Category: **combinatorial optimization, scheduling**.

```
p push_relabel_step $x
```

pushd

pushd(DIR) — save current cwd onto a process-wide directory stack, then **chdir(DIR)**. Returns the new cwd. Bare **pushd()** with no argument swaps the top of the stack with cwd (zsh convention). Also updates **stryke_pwd** so relative-path builtins (**glob**, **slurp**, **slurp_self**, ...) follow the new directory.

```
pushd("/tmp")           # save cwd, chdir /tmp
# ... work in /tmp ...
popd()                  # restore previous cwd
```

pv

pv — operation **pv**. Category: **finance (extended)**.

```
p pv $x
```

pwave_velocity_depth

pwave_velocity_depth — operation **pwave** **velocity** **depth**. Category: **geology, seismology, mineralogy**.

```
p pwave_velocity_depth $x
```

pwd_str

pwd_str — operation **pwd** **str**. Alias for **cwd**. Category: **more process / system**.

```
p pwd_str $x
```

pweibull

pweibull — Weibull CDF. $P(X \leq x)$ for Weibull(shape, scale). Args: x, shape, scale.

```
p pweibull(1, 1, 1) # same as exponential
```

pwi

pwi — operation **pwi**. Alias for **pairwise_iter**. Category: **python/ruby stdlib**.

```
p pwi $x
```

pwm_score

`pwm_score` — score of `pwm`. Category: *bioinformatics deep*.

```
p pwm_score $x
```

pyramid_volume

`pyramid_volume($base_area, $height)` — computes the volume of a pyramid. Returns $(1/3) * \text{base_area} * \text{height}$.

```
p pyramid_volume(100, 15) # 500
```

pyrvol

`pyrvol` — operation `pyrvol`. Alias for `pyramid_volume`. Category: *geometry (extended)*.

```
p pyrvol $x
```

pythagorean_expectation

`pythagorean_expectation` — operation `pythagorean expectation`. Category: *astronomy / astrometry*.

```
p pythagorean_expectation $x
```

pythagorean_freq

`pythagorean_freq` — operation `pythagorean freq`. Category: *music theory*.

```
p pythagorean_freq $x
```

pythagorean_ratio

`pythagorean_ratio` — ratio of `pythagorean`. Category: *astronomy / music / color / units*.

```
p pythagorean_ratio $x
```

q

`q` — operation `q`. Category: *quoting*.

```
p q $x
```

q10

`q10` — operation `q10`. Category: *biology / ecology*.

```
p q10 $x
```

q_factor_rlc

`q_factor_rlc` — operation `q factor rlc`. Category: *misc*.

```
p q_factor_rlc $x
```

q_learning_update

`q_learning_update` — update step of `q_learning`. Category: *electrochemistry, batteries, fuel cells*.

```
p q_learning_update $x
```

qaly_lifetime

`qaly_lifetime` — operation `qaly_lifetime`. Category: *epidemiology / public health*.

```
p qaly_lifetime $x
```

qaoa_step

`qaoa_step` — single-step update of `qaoa`. Category: *quantum*.

```
p qaoa_step $x
```

qap_lower_bound

`qap_lower_bound` — operation `qap_lower_bound`. Category: *combinatorial optimization, scheduling*.

```
p qap_lower_bound $x
```

qbeta

`qbeta` — beta quantile. Args: p, alpha, beta.

```
p qbeta(0.5, 2, 5) # median of Beta(2,5)
```

qbinom

`qbinom` — binomial quantile (smallest k where $P(X \leq k) \geq p$). Args: p, n, prob.

```
p qbinom(0.5, 10, 0.5) # median of Binom(10, 0.5) = 5
```

qbo_oscillation_step

`qbo_oscillation_step` — single-step update of `qbo_oscillation`. Category: *climate, fluids, atmospheric*.

```
p qbo_oscillation_step $x
```

qbr_metric

`qbr_metric` — metric of `qbr`. Category: *astronomy / astrometry*.

```
p qbr_metric $x
```

qcauchy

`qcauchy` — Cauchy quantile. Args: p [, location, scale].

```
p qcauchy(0.75, 0, 1) # 1.0
```

qchisq

`qchisq` — chi-squared quantile. Args: p, df.

```
p qchisq(0.95, 1) # 3.84 (critical value)
p qchisq(0.95, 5) # 11.07
```

qdisc

`quadratic_discriminant($x, $y, $c)` (alias `qdisc`) — computes $b^2 - 4ac$. Positive = two real roots, zero = one repeated root, negative = complex roots.

```
p qdisc(1, -5, 6) # 1 (two distinct real roots)
p qdisc(1, 2, 1)  # 0 (one repeated root)
p qdisc(1, 2, 5)  # -16 (complex roots)
```

qexp

`qexp` — exponential quantile. Args: `p` [, `rate`].

```
p qexp(0.5, 1) # 0.693 (median of Exp(1))
```

qf

`qf` — F-distribution quantile. Args: `p`, `d1`, `d2`.

```
p qf(0.95, 5, 10) # critical F value
```

qft

`qft` — operation `qft`. Category: `*quantum*`.

```
p qft $x
```

qft_4_real

`qft_4_real` — operation `qft 4 real`. Category: `*more extensions*`.

```
p qft_4_real $x
```

qgamma

`qgamma` — gamma quantile (inverse CDF). Args: `p`, `shape` [, `scale`].

```
p qgamma(0.95, 2, 1) # 95th percentile of Gamma(2,1)
```

qgeom

`qgeom` — operation `qgeom`. Category: `*extras*`.

```
p qgeom $x
```

qho_ground_state

`qho_ground_state` — operation `qho ground state`. Category: `*quantum mechanics deep*`.

```
p qho_ground_state $x
```

qhyper

`qhyper` — operation `qhyper`. Category: `*extras*`.


```
p qhyper $x
```

qlearning_step

`qlearning_step(q, alpha, r, gamma, max_next_q) → number` — $q \leftarrow q + \alpha \cdot (r + \gamma \cdot \text{max_next_q} - q)$.

```
p qlearning_step(0, 0.1, 10, 0.9, 5) # 1.45
```

qlnorm

`qlnorm` — log-normal quantile. Args: p [, mu, sigma].

```
p qlnorm(0.5, 0, 1) # 1.0 (median of LogN(0,1))
```

qlogis

`qlogis` — operation `qlogis`. Category: *extras*.

```
p qlogis $x
```

qm

`qm` — operation `qm`. Alias for `quotemeta`. Category: *string*.

```
p qm $x
```

qnorm

`qnorm` — normal quantile (inverse CDF). Returns x such that $P(X \leq x) = p$. Args: p [, mu, sigma].

```
p qnorm(0.975) # 1.96
p qnorm(0.5) # 0.0
p qnorm(0.95, 100, 15) # 90th percentile IQ
```

qntl

`qntl` — operation `qntl`. Alias for `quantile`. Category: *extended stdlib*.

```
p qntl $x
```

qntls

`quantiles \@DATA, \@PROBS` — batch quantile lookup. Returns one element per probability in `\@PROBS`, computed via the same linear interpolation as `quantile`. Sorts the data exactly once across all probability points so callers don't pay an $O(n \log n)$ sort per quantile. Probabilities outside [0,1] are clamped.

```
val @d = (1..1000)
var @qs = quantiles(\@d, [0.25, 0.5, 0.75])
printf "q25=%.2f q50=%.2f q75=%.2f\n", @qs
# q25=250.75 q50=500.50 q75=750.25
```

qpe_iteration

`qpe_iteration` — operation `qpe_iteration`. Category: *quantum*.

```
p qpe_iteration $x
```

qpois

qpois — Poisson quantile (smallest k where $P(X \leq k) \geq p$). Args: p , λ .

```
p qpois(0.5, 5) # median of Poisson(5)
```

qq

qq — operation **qq**. Category: **quoting**.

```
p qq $x
```

qroots

quadratic_roots(\$x, \$y, \$c) (alias **qroots**) — solves $ax^2 + bx + c = 0$. Returns $[x_1, x_2]$ for real roots, `undef` if no real solutions (discriminant < 0).

```
val @roots = quadratic_roots(1, -5, 6) # [3, 2] (x^2 - 5x + 6 = 0)
p qroots(1, 0, -4)                    # [2, -2]
p qroots(1, 2, 5)                     # undef (complex roots)
```

qs_relation

qs_relation — operation **qs relation**. Category: **cryptanalysis & number theory deep**.

```
p qs_relation $x
```

qt

qt — Student's t quantile. Args: p , df .

```
p qt(0.975, 10) # ≈ 2.228 (two-tailed 5%)
```

quadratic_assignment_step

quadratic_assignment_step — single-step update of **quadratic assignment**. Category: **combinatorial optimization, scheduling**.

```
p quadratic_assignment_step $x
```

quadratic_mean_arr

quadratic_mean_arr — operation **quadratic mean arr**. Category: **misc**.

```
p quadratic_mean_arr $x
```

quadratic_residue

quadratic_residue — operation **quadratic residue**. Category: **math / number theory extras**.

```
p quadratic_residue $x
```

quantal_response_logit

quantal_response_logit — operation **quantal response logit**. Category: **climate, fluids, atmospheric**.

```
p quantal_response_logit $x
```

quantile

`quantile` — operation `quantile`. Category: **extended stdlib**.

```
p quantile $x
```

quantile_p

`quantile_p` — quantile helper `p`. Category: **more extensions**.

```
p quantile_p $x
```

quantize

`quantize` — operation `quantize`. Category: **signal processing**.

```
p quantize $x
```

quanto_adjustment

`quanto_adjustment` — operation `quanto adjustment`. Category: **financial pricing models**.

```
p quanto_adjustment $x
```

quantum_dimension_q

`quantum_dimension_q` — operation `quantum dimension q`. Category: **econometrics**.

```
p quantum_dimension_q $x
```

quantum_dimension_su2

`quantum_dimension_su2` — operation `quantum dimension su2`. Category: **econometrics**.

```
p quantum_dimension_su2 $x
```

quantum_efficiency_photo

`quantum_efficiency_photo` — operation `quantum efficiency photo`. Category: **electrochemistry, batteries, fuel cells**.

```
p quantum_efficiency_photo $x
```

quantum_fidelity_pure

`quantum_fidelity_pure` — operation `quantum fidelity pure`. Category: **more extensions**.

```
p quantum_fidelity_pure $x
```

quantum_fisher_info

`quantum_fisher_info` — operation `quantum fisher info`. Category: **quantum mechanics deep**.

```
p quantum_fisher_info $x
```

quantum_invariant_eval

`quantum_invariant_eval` — operation `quantum invariant eval`. Category: **econometrics**.

```
p quantum_invariant_eval $x
```

quantum_mutual_info

quantum_mutual_info — operation `quantum mutual info`. Category: **quantum mechanics deep**.

```
p quantum_mutual_info $x
```

quantum_relative_entropy

quantum_relative_entropy — operation `quantum relative entropy`. Category: **more extensions**.

```
p quantum_relative_entropy $x
```

quantum_teleportation

quantum_teleportation — operation `quantum teleportation`. Category: **quantum**.

```
p quantum_teleportation $x
```

quantum_variance

quantum_variance — operation `quantum variance`. Category: **quantum mechanics deep**.

```
p quantum_variance $x
```

quarter_of

quarter_of — operation `quarter of`. Category: **date helpers**.

```
p quarter_of $x
```

quarter_of_year

quarter_of_year — operation `quarter of year`. Category: **misc**.

```
p quarter_of_year $x
```

quartile_exc

quartile_exc — operation `quartile exc`. Category: **excel/sheets + bond/loan financial**.

```
p quartile_exc $x
```

quartile_inc

quartile_inc — operation `quartile inc`. Category: **excel/sheets + bond/loan financial**.

```
p quartile_inc $x
```

quartiles

`quartiles(@data)` — returns the three quartile values [Q1, Q2, Q3] of a dataset. Q1 is the 25th percentile, Q2 is the median (50th), Q3 is the 75th percentile. Useful for understanding data distribution and computing IQR.

```
val @data = 1:100
var $q = quartiles(@data)
p "Q1=$q->[0] Q2=$q->[1] Q3=$q->[2]"
```

quarts

`quarts` — operation `quarts`. Alias for `quartiles`. Category: *statistics (extended)*.

```
p quarts $x
```

quasi_geostrophic_omega

`quasi_geostrophic_omega` — operation `quasi geostrophic omega`. Category: *climate, fluids, atmospheric*.

```
p quasi_geostrophic_omega $x
```

quat_conjugate

`quat_conjugate(q)` → `[w,-x,-y,-z]`.

quat_dot

`quat_dot(a, b)` → `number` — 4-component dot product.

quat_from_euler

`quat_from_euler(roll, pitch, yaw)` → `[w,x,y,z]` — quaternion from XYZ Euler angles (radians).

```
p quat_from_euler(0, 0, 3.14159/2)
```

quat_identity

`quat_identity()` → `[w,x,y,z]` — identity quaternion `[1,0,0,0]`.

quat_inverse

`quat_inverse(q)` → `[w,x,y,z]` — `conjugate(q) / |q|2`.

quat_multiply

`quat_multiply(a, b)` → `[w,x,y,z]` — Hamilton product `a ⊗ b`.

quat_normalize

`quat_normalize(q)` → `[w,x,y,z]` — divide by magnitude. Required after iterative composition.

quat_to_euler

`quat_to_euler(q)` → `[roll, pitch, yaw]` — inverse of `quat_from_euler`.

quat_to_mat4

`quat_to_mat4(q)` → `matrix` — convert quaternion to 4×4 rotation matrix.

quaternion_from_axis_angle

`quaternion_from_axis_angle` — operation `quaternion from axis angle`. Category: *extras*.

```
p quaternion_from_axis_angle $x
```

quaternion_from_imu

`quaternion_from_imu` — operation `quaternion from imu`. Category: **robotics & control**.

```
p quaternion_from_imu $x
```

quaternion_multiply

`quaternion_multiply` — operation `quaternion multiply`. Category: **extras**.

```
p quaternion_multiply $x
```

quaternion_new

`quaternion_new` — constructor of `quaternion`. Category: **extras**.

```
p quaternion_new $x
```

quaternion_normalize

`quaternion_normalize` — operation `quaternion normalize`. Category: **extras**.

```
p quaternion_normalize $x
```

quaternion_to_matrix

`quaternion_to_matrix` — convert from `quaternion` to `matrix`. Category: **extras**.

```
p quaternion_to_matrix $value
```

qubit_ccx

`qubit_ccx` — operation `qubit ccx`. Category: **quantum**.

```
p qubit_ccx $x
```

qubit_cnot

`qubit_cnot` — operation `qubit cnot`. Category: **quantum**.

```
p qubit_cnot $x
```

qubit_cz

`qubit_cz` — operation `qubit cz`. Category: **quantum**.

```
p qubit_cz $x
```

qubit_h

`qubit_h` — operation `qubit h`. Category: **quantum**.

```
p qubit_h $x
```

qubit_measure

`qubit_measure` — operation `qubit measure`. Category: **quantum**.

```
p qubit_measure $x
```

qubit_phase

qubit_phase — operation `qubit phase`. Category: `*quantum*`.

```
p qubit_phase $x
```

qubit_reset

qubit_reset — operation `qubit reset`. Category: `*quantum*`.

```
p qubit_reset $x
```

qubit_rx

qubit_rx — operation `qubit rx`. Category: `*quantum*`.

```
p qubit_rx $x
```

qubit_ry

qubit_ry — operation `qubit ry`. Category: `*quantum*`.

```
p qubit_ry $x
```

qubit_rz

qubit_rz — operation `qubit rz`. Category: `*quantum*`.

```
p qubit_rz $x
```

qubit_s

qubit_s — operation `qubit s`. Category: `*quantum*`.

```
p qubit_s $x
```

qubit_swap

qubit_swap — operation `qubit swap`. Category: `*quantum*`.

```
p qubit_swap $x
```

qubit_t

qubit_t — operation `qubit t`. Category: `*quantum*`.

```
p qubit_t $x
```

qubit_u1

qubit_u1 — operation `qubit u1`. Category: `*quantum*`.

```
p qubit_u1 $x
```

qubit_u2

qubit_u2 — operation `qubit u2`. Category: `*quantum*`.

```
p qubit_u2 $x
```

qubit_u3

`qubit_u3` — operation `qubit u3`. Category: **quantum**.

```
p qubit_u3 $x
```

qubit_x

`qubit_x` — operation `qubit x`. Category: **quantum**.

```
p qubit_x $x
```

qubit_y

`qubit_y` — operation `qubit y`. Category: **quantum**.

```
p qubit_y $x
```

qubit_z

`qubit_z` — operation `qubit z`. Category: **quantum**.

```
p qubit_z $x
```

queue_new

`queue_new` — queue op `new`. Category: **data structure helpers**.

```
p queue_new $x
```

quickselect_median

`quickselect_median` — median of `quickselect`. Category: **iterator + string-distance extras**.

```
p quickselect_median @xs
```

quickselect_nth

`quickselect_nth` — operation `quickselect nth`. Category: **iterator + string-distance extras**.

```
p quickselect_nth $x
```

quiver_path_count

`quiver_path_count` — count of `quiver path`. Category: **econometrics**.

```
p quiver_path_count $x
```

qunif

`qunif` — uniform quantile. Args: `p` [, `min`, `max`].

```
p qunif(0.5, 0, 10) # 5.0 (median)
```

quote

`quote` — operation `quote`. Category: **string quote / escape**.

```
p quote $x
```


quoted_printable_encode

`quoted_printable_encode` — encode of `quoted printable`. Category: **archive/encoding format primitives**.

```
p quoted_printable_encode $x
```

qw

`qw` — operation `qw`. Category: **quoting**.

```
p qw $x
```

qweibull

`qweibull` — Weibull quantile. Args: `p`, shape `[, scale]`.

```
p qweibull(0.5, 1, 1) # same as qexp
```

r0_basic

`r0_basic` — operation `r0 basic`. Category: **biology / ecology**.

```
p r0_basic $x
```

r2d

`r2d` — operation `r2d`. Alias for `rad_to_deg`. Category: **trivial numeric / predicate builtins**.

```
p r2d $x
```

r72

`rule_of_72($rate)` (alias `r72`) — estimates years to double investment using the Rule of 72: $72 / (\text{rate} \times 100)$. Quick mental math approximation.

```
p r72(0.06) # 12 years to double at 6%
p r72(0.10) # 7.2 years at 10%
```

r_effective_t

`r_effective_t` — operation `r effective t`. Category: **epidemiology / public health**.

```
p r_effective_t $x
```

r_naught_basic

`r_naught_basic` — operation `r naught basic`. Category: **epidemiology / public health**.

```
p r_naught_basic $x
```

r_parallel

`r_parallel` — operation `r parallel`. Alias for `resistance_parallel`. Category: **misc**.

```
p r_parallel $x
```

r_series

`r_series` — operation `r_series`. Alias for `resistance_series`. Category: **misc**.

```
p r_series $x
```

r_squared

`r_squared` — operation `r_squared`. Category: **math functions**.

```
p r_squared $x
```

ra_dec_to_alt_az

`ra_dec_to_alt_az` — convert from `ra_dec` to `alt_az`. Category: **astronomy / astrometry**.

```
p ra_dec_to_alt_az $value
```

ra_dec_to_az_alt

`ra_dec_to_az_alt` — convert from `ra_dec` to `az_alt`. Category: **more extensions**.

```
p ra_dec_to_az_alt $value
```

rabi_frequency

`rabi_frequency` — operation `rabi_frequency`. Category: **quantum mechanics deep**.

```
p rabi_frequency $x
```

rabin_karp_hash

`rabin_karp_hash` — hash function of `rabin_karp`. Category: **misc**.

```
p rabin_karp_hash $x
```

rad_to_deg

`rad_to_deg` — convert from `rad` to `deg`. Category: **trivial numeric / predicate builtins**.

```
p rad_to_deg $value
```

radd

`radd` — operation `radd`. Alias for `roman_add`. Category: **roman numerals**.

```
p radd $x
```

radians

`radians` — operation `radians`. Category: **trig / math**.

```
p radians $x
```

radiation_pressure

`radiation_pressure` — operation `radiation pressure`. Category: **em / optics / relativity**.

```
p radiation_pressure $x
```

radius_bfs

`radius_bfs` — operation `radius bfs`. Category: *graph algorithms*.

```
p radius_bfs $x
```

ragone_point

`ragone_point` — operation `ragone point`. Category: *electrochemistry, batteries, fuel cells*.

```
p ragone_point $x
```

railfence_encrypt

`railfence_encrypt` — encrypt of `railfence`. Category: *cryptography deep*.

```
p railfence_encrypt $x
```

ramachandran_phi_psi

`ramachandran_phi_psi` — operation `ramachandran phi psi`. Category: *chemistry & biochemistry*.

```
p ramachandran_phi_psi $x
```

ramsey_reset_test

`ramsey_reset_test` — statistical test of `ramsey reset`. Category: *econometrics*.

```
p ramsey_reset_test $x
```

ramsey_visibility

`ramsey_visibility` — operation `ramsey visibility`. Category: *quantum mechanics deep*.

```
p ramsey_visibility $x
```

randles_circuit_z

`randles_circuit_z` — operation `randles circuit z`. Category: *electrochemistry, batteries, fuel cells*.

```
p randles_circuit_z $x
```

randles_sevcik_peak

`randles_sevcik_peak` — operation `randles sevcik peak`. Category: *electrochemistry, batteries, fuel cells*.

```
p randles_sevcik_peak $x
```

random_alpha

`random_alpha` — random sampling helper `alpha`. Category: *random*.

```
p random_alpha $x
```

random_alphabetic

`random_alphabetic` — random sampling helper `alphabetic`. Category: *random / sampling extras*.

```
p random_alphabetic $x
```

random_alphanumeric

`random_alphanumeric` — random sampling helper `alphanumeric`. Category: **random / sampling extras**.

```
p random_alphanumeric $x
```

random_bernoulli

`random_bernoulli` — random sampling helper `bernoulli`. Category: **random / sampling extras**.

```
p random_bernoulli $x
```

random_beta

`random_beta` — random sampling helper `beta`. Category: **random / sampling extras**.

```
p random_beta $x
```

random_between

`random_between` — random sampling helper `between`. Category: **random**.

```
p random_between $x
```

random_bool

`random_bool` — random sampling helper `bool`. Category: **random**.

```
p random_bool $x
```

random_char

`random_char` — random sampling helper `char`. Category: **file stat / path**.

```
p random_char $x
```

random_choice

`random_choice` — random sampling helper `choice`. Category: **random**.

```
p random_choice $x
```

random_choices_weighted

`random_choices_weighted` — random sampling helper `choices weighted`. Category: **random / sampling extras**.

```
p random_choices_weighted $x
```

random_color

`random_color` — random sampling helper `color`. Category: **color operations**.

```
p random_color $x
```

random_digit

`random_digit` — random sampling helper `digit`. Category: **random**.

```
p random_digit $x
```

random_exponential

`random_exponential` — random sampling helper `exponential`. Category: **random / sampling extras**.

```
p random_exponential $x
```

random_float

`random_float` — random sampling helper `float`. Category: **random**.

```
p random_float $x
```

random_gamma

`random_gamma` — random sampling helper `gamma`. Category: **random / sampling extras**.

```
p random_gamma $x
```

random_int

`random_int` — random sampling helper `int`. Category: **random**.

```
p random_int $x
```

random_lognormal

`random_lognormal` — random sampling helper `lognormal`. Category: **random / sampling extras**.

```
p random_lognormal $x
```

random_normal

`random_normal` — random sampling helper `normal`. Category: **random / sampling extras**.

```
p random_normal $x
```

random_password

`random_password` — random sampling helper `password`. Category: **random / sampling extras**.

```
p random_password $x
```

random_poisson

`random_poisson` — random sampling helper `poisson`. Category: **random / sampling extras**.

```
p random_poisson $x
```

random_sample

`random_sample` — random sampling helper `sample`. Category: **collection**.

```
p random_sample $x
```

random_string

`random_string` — random sampling helper `string`. Category: **random**.

```
p random_string $x
```

random_walk_drift_step

`random_walk_drift_step` — random sampling helper `walk drift_step`. Category: **econometrics**.

```
p random_walk_drift_step $x
```

random_walk_hitting

`random_walk_hitting` — random sampling helper `walk hitting`. Category: **networkx graph algorithms**.

```
p random_walk_hitting $x
```

random_walk_innovation

`random_walk_innovation` — random sampling helper `walk innovation`. Category: **econometrics**.

```
p random_walk_innovation $x
```

range_coding_step

`range_coding_step` — single-step update of `range coding`. Category: **electrochemistry, batteries, fuel cells**.

```
p range_coding_step $x
```

range_compress

`range_compress` — compress of `range`. Category: **array / list operations extras**.

```
p range_compress $x
```

range_decode_step

`range_decode_step` — single-step update of `range decode`. Category: **electrochemistry, batteries, fuel cells**.

```
p range_decode_step $x
```

range_exclusive

`range_exclusive` — operation `range exclusive`. Category: **conversion / utility**.

```
p range_exclusive $x
```

range_expand

`range_expand` — operation `range expand`. Category: **array / list operations extras**.

```
p range_expand $x
```

range_header_parse

`range_header_parse` — operation `range header parse`. Category: **b81-misc-utility**.

```
p range_header_parse $x
```

range_inclusive

`range_inclusive` — operation `range inclusive`. Category: **conversion / utility**.

```
p range_inclusive $x
```

range_of

`range_of` — operation `range of`. Category: *more list helpers*.

```
p range_of $x
```

range_to_cidr

`range_to_cidr(start, end) → array` — minimum set of CIDR blocks covering the range.

range_voting_score

`range_voting_score` — score of `range voting`. Category: *climate, fluids, atmospheric*.

```
p range_voting_score $x
```

rank

`rank` — operation `rank`. Category: *extended stdlib*.

```
p rank $x
```

rank_avg

`rank_avg` — operation `rank avg`. Category: *excel/sheets + bond/loan financial*.

```
p rank_avg $x
```

rank_data

`rank_data(xs) → array` — assign ranks 1..n with ties handled by average rank.

```
p rank_data([10, 20, 20, 30]) # [1, 2.5, 2.5, 4]
```

rank_eq

`rank_eq` — operation `rank eq`. Category: *excel/sheets + bond/loan financial*.

```
p rank_eq $x
```

ranked_choice

`ranked_choice` — operation `ranked choice`. Category: *iterator + string-distance extras*.

```
p ranked_choice $x
```

ranking_average

`ranking_average(lists) → array` — element-wise average across multiple ranking lists (Borda-style).

ranking_kendall_tau

`ranking_kendall_tau(a, b) → number` — Kendall's τ rank correlation in [-1, 1].

```
p ranking_kendall_tau([1,2,3], [3,2,1]) # -1
```

ranking_spearman_rho

`ranking_spearman_rho(a, b) → number` — Spearman's ρ rank correlation.

ransac_homography

`ransac_homography` — operation `ransac homography`. Category: *pil/opencv image processing*.

```
p ransac_homography $x
```

ransac_iteration_count

`ransac_iteration_count` — count of `ransac iteration`. Category: *electrochemistry, batteries, fuel cells*.

```
p ransac_iteration_count $x
```

rao_quadratic_entropy

`rao_quadratic_entropy` — operation `rao quadratic entropy`. Category: *biology / ecology*.

```
p rao_quadratic_entropy $x
```

raoult_law

`raoult_law` — operation `raoult law`. Category: *chemistry*.

```
p raoult_law $x
```

rapid_blink

`rapid_blink` — operation `rapid blink`. Alias for `red`. Category: *color / ansi*.

```
p rapid_blink $x
```

ratcliff_overshelp

`ratcliff_overshelp` — operation `ratcliff overshelp`. Category: *iterator + string-distance extras*.

```
p ratcliff_overshelp $x
```

rate

`rate` — operation `rate`. Category: *excel/sheets + bond/loan financial*.

```
p rate $x
```

ray_aabb_intersect

`ray_aabb_intersect(origin, dir, aabb) → number` — t along ray where it enters; `-1` if miss.

```
p ray_aabb_intersect([-1,0.5,0.5], [1,0,0], aabb_new([0,0,0],[1,1,1])) # 1.0
```

ray_plane_intersect

`ray_plane_intersect(origin, dir, point, normal) → number` — distance along ray; `-1` for parallel or behind ray.

ray_plane_intersect_2

`ray_plane_intersect_2` — ray-geometry intersection op `plane intersect 2`. Category: *test runner*.

```
p ray_plane_intersect_2 $x
```

raychaudhuri_step

`raychaudhuri_step` — single-step update of `raychaudhuri`. Category: *electrochemistry, batteries, fuel cells*.

```
p raychaudhuri_step $x
```

rayleigh_criterion

`rayleigh_criterion` — operation `rayleigh criterion`. Category: *astronomy / astrometry*.

```
p rayleigh_criterion $x
```

rayleigh_number_step

`rayleigh_number_step` — single-step update of `rayleigh number`. Category: *climate, fluids, atmospheric*.

```
p rayleigh_number_step $x
```

rayleigh_resolution

`rayleigh_resolution` — operation `rayleigh resolution`. Category: *em / optics / relativity*.

```
p rayleigh_resolution $x
```

rb

`rb` — operation `rb`. Alias for `read_bytes`. Category: *filesystem extensions*.

```
p rb $x
```

rb_add

`rb_add(RB, U32...)` — insert one or more u32 values (clamped to `[0, 232-1]`); accepts ARRAYREFs as well. Returns the count newly inserted.

rb_and

`rb_and(RB, OTHER)` — in-place intersect.

rb_andnot

`rb_andnot(RB, OTHER)` — in-place `RB - OTHER`.

rb_clear

`rb_clear` — operation `rb clear`. Category: *probabilistic data structures*.

```
p rb_clear $x
```

rb_contains

`rb_contains(RB, U32) → 1/0.`

rb_count

`rb_count` — count of `rb`. Alias for `rb_len`. Category: *probabilistic data structures*.

```
p rb_count $x
```

rb_del

`rb_del` — operation `rb del`. Alias for `rb_remove`. Category: *probabilistic data structures*.

```
p rb_del $x
```

rb_deserialize

`rb_deserialize` — operation `rb deserialize`. Category: *probabilistic data structures*.

```
p rb_deserialize $x
```

rb_from_bytes

`rb_from_bytes` — operation `rb from bytes`. Alias for `rb_deserialize`. Category: *probabilistic data structures*.

```
p rb_from_bytes $x
```

rb_len

`rb_len` — operation `rb len`. Category: *probabilistic data structures*.

```
p rb_len $x
```

rb_max

`rb_max` — maximum of `rb`. Category: *probabilistic data structures*.

```
p rb_max $x
```

rb_min

`rb_min` — minimum of `rb`. Category: *probabilistic data structures*.

```
p rb_min $x
```

rb_or

`rb_or(RB, OTHER)` — in-place set union into RB.

rb_rank

`rb_rank(RB, V)` → number of elements $\leq V$.

rb_remove

`rb_remove` — operation `rb remove`. Category: *probabilistic data structures*.

```
p rb_remove $x
```

rb_reset

`rb_reset` — operation `rb reset`. Alias for `rb_clear`. Category: *probabilistic data structures*.

```
p rb_reset $x
```

rb_serialize

`rb_serialize` — operation `rb serialize`. Category: *probabilistic data structures*.

```
p rb_serialize $x
```

rb_symdiff

`rb_xor(RB, OTHER)` — in-place symmetric difference.

rb_to_array

`rb_to_array(RB)` → sorted ascending list of contained `u32`s.

rb_to_bytes

`rb_to_bytes` — convert from `rb` to `bytes`. Alias for `rb_serialize`. Category: *probabilistic data structures*.

```
p rb_to_bytes $value
```

rbeta

`rbeta` — generate `n` random beta variates. Args: `n`, `alpha`, `beta`. Like R's `rbeta()`.

```
val @x = @{rbeta(100, 2, 5)} # Beta(2,5)
```

rbind

`rbind` — bind matrices/vectors by rows (vertical stack). Like R's `rbind()`.

```
val $m = rbind([1,2,3], [4,5,6]) # [[1,2,3],[4,5,6]]
```

rbinom

`rbinom` — generate `n` random binomial variates. Args: `n`, `size`, `prob`. Like R's `rbinom()`.

```
val @x = @{rbinom(100, 10, 0.3)} # 100 draws from Binom(10,0.3)
```

rbm

`roaring(LIST | \@ARR | ...)` — Roaring Bitmap (compressed `u32` set). Apache-licensed `roaring` crate; the canonical impl used by quickwit/tantivy/lucene-rs. $O(1)$ `add/contains`, $O(n/\text{run-length})$ set ops; typically 10-100× smaller than `HashSet<u32>` on natural datasets. `rb` short alias reserved for `read_bytes`; use `rbm`.

```
val $ra = roaring(1, 2, 3, 5, 8)
var $rb = roaring([2, 4, 6, 8, 10])
rb_and($ra, $rb) # in-place intersect
p join(", ", rb_to_array($ra)) # 2,8
```

Family: `rb_add`, `rb_remove`, `rb_contains`, `rb_or` (alias `rb_union`), `rb_and` (`rb_intersect`), `rb_xor` (`rb_symdiff`), `rb_andnot` (`rb_diff`, `rb_minus`), `rb_len`, `rb_min`, `rb_max`, `rb_rank`, `rb_to_array`, `rb_clear`, `rb_serialize`, `rb_deserialize`.

rc_tau

`rc_tau` — operation `rc tau`. Category: **em / optics / relativity**.

```
p rc_tau $x
```

rc_time_constant

`rc_time_constant` — operation `rc time constant`. Category: **physics formulas**.

```
p rc_time_constant $x
```

rcauchy

`rcauchy` — generate n random Cauchy variates. Args: n [, location, scale].

```
val @x = @{rcauchy(100, 0, 1)} # standard Cauchy
```

rchisq

`rchisq` — generate n random chi-squared variates. Args: n, df. Like R's `rchisq()`.

```
val @x = @{rchisq(100, 5)} # Chi-sq with 5 df
```

rcolor

`rcolor` — operation `rcolor`. Alias for `random_color`. Category: **color operations**.

```
p rcolor $x
```

rctau

`rctau` — operation `rctau`. Alias for `rc_time_constant`. Category: **physics formulas**.

```
p rctau $x
```

rdcs

`rdcs` — operation `rdcs`. Alias for `reductions`. Category: **algebraic match**.

```
p rdcs $x
```

rdd_estimate

`rdd_estimate` — operation `rdd estimate`. Category: **economics + game theory**.

```
p rdd_estimate $x
```

re_escape

`re_escape` — operation `re escape`. Category: **more regex**.

```
p re_escape $x
```

re_find_all

`re_find_all` — operation `re find all`. Category: **more regex**.

```
p re_find_all $x
```

re_groups

`re_groups` — operation `re groups`. Category: **more regex**.

```
p re_groups $x
```

re_match

`re_match` — operation `re match`. Alias for `matches_regex`. Category: **file stat / path**.

```
p re_match $x
```

re_split_limit

`re_split_limit` — operation `re split limit`. Category: **more regex**.

```
p re_split_limit $x
```

re_test

`re_test` — statistical test of `re`. Category: **more regex**.

```
p re_test $x
```

reaction_quotient

`reaction_quotient` — operation `reaction quotient`. Category: **chemistry**.

```
p reaction_quotient $x
```

realized_volatility

`realized_volatility` — operation `realized volatility`. Category: **more extensions (2)**.

```
p realized_volatility $x
```

rearth

`rearth` — operation `rearth`. Alias for `earth_radius`. Category: **physics constants**.

```
p rearth $x
```

received

`received` — operation `received`. Category: **b81-misc-utility**.

```
p received $x
```

recip

`recip` — operation `recip`. Alias for `reciprocal_of`. Category: **extended stdlib**.

```
p recip $x
```

reciprocal

`reciprocal` — operation `reciprocal`. Category: **math functions**.

```
p reciprocal $x
```

reciprocal_of

`reciprocal_of` — operation `reciprocal of`. Category: **extended stdlib**.

```
p reciprocal_of $x
```

reciprocal_pdf

`reciprocal_pdf` — operation `reciprocal pdf`. Category: **more extensions (2)**.

```
p reciprocal_pdf $x
```

rect_area

`rect_area` — rectangle geometry op `area`. Category: **complex / geom / color / trig**.

```
p rect_area $x
```

rect_contains_point

`rect_contains_point` — rectangle geometry op `contains point`. Category: **complex / geom / color / trig**.

```
p rect_contains_point $x
```

rect_intersect

`rect_intersect` — rectangle geometry op `intersect`. Category: **complex / geom / color / trig**.

```
p rect_intersect $x
```

rect_perimeter

`rect_perimeter` — rectangle geometry op `perimeter`. Category: **complex / geom / color / trig**.

```
p rect_perimeter $x
```

rect_union

`rect_union` — rectangle geometry op `union`. Category: **complex / geom / color / trig**.

```
p rect_union $x
```

red

`red` — operation `red`. Category: **color / ansi**.

```
p red $x
```

red_bold

`red_bold` — operation `red bold`. Alias for `red`. Category: **color / ansi**.

```
p red_bold $x
```

redi

`redi` — operation `redi`. Alias for `reduce_indexed`. Category: **go/general functional utilities**.

```
p redi $x
```

redlich_kwong_p

redlich_kwong_p — operation `redlich kwong p`. Category: **chemistry**.

```
p redlich_kwong_p $x
```

redox_potential_cell

redox_potential_cell — operation `redox potential cell`. Category: **chemistry & biochemistry**.

```
p redox_potential_cell $x
```

redr

redr — operation `redr`. Alias for `reduce_right`. Category: **additional missing stdlib functions**.

```
p redr $x
```

redshift_from_a

redshift_from_a — operation `redshift from a`. Category: **cosmology / gr / flrw**.

```
p redshift_from_a $x
```

reduce_axis

reduce_axis — operation `reduce axis`. Category: **apl/j/k array primitives**.

```
p reduce_axis $x
```

reduce_indexed

reduce_indexed — operation `reduce indexed`. Category: **go/general functional utilities**.

```
p reduce_indexed $x
```

reduce_right

reduce_right — operation `reduce right`. Category: **additional missing stdlib functions**.

```
p reduce_right $x
```

refaddr

refaddr — operation `refaddr`. Category: **list / aggregate**.

```
p refaddr $x
```

refl_check

refl_check — operation `refl check`. Category: **logic, proof, sat/smt, type theory**.

```
p refl_check $x
```

reflect_point

`reflect_point($x, $y, $axis)` — reflects a point across an axis. Axis can be 'x', 'y', or 'origin'. Returns the reflected coordinates.

```
p reflect_point(3, 4, 'x')      # [3, -4]
p reflect_point(3, 4, 'y')      # [-3, 4]
p reflect_point(3, 4, 'origin') # [-3, -4]
```

reflpt

reflpt — operation **reflpt**. Alias for **reflect_point**. Category: *geometry (extended)*.

```
p reflpt $x
```

refresh_stashes

refresh_stashes — operation **refresh stashes**. Category: *symbol table*.

```
p refresh_stashes $x
```

reftype

reftype — operation **reftype**. Category: *list / aggregate*.

```
p reftype $x
```

regex_compile_thompson

regex_compile_thompson — regular-expression op **compile thompson**. Category: *compilers / parsing*.

```
p regex_compile_thompson $x
```

regex_escape_simple

regex_escape_simple — regular-expression op **escape simple**. Category: *misc*.

```
p regex_escape_simple $x
```

regex_extract

regex_extract — regular-expression op **extract**. Category: *file stat / path*.

```
p regex_extract $x
```

regex_match_dfa

regex_match_dfa — regular-expression op **match dfa**. Category: *compilers / parsing*.

```
p regex_match_dfa $x
```

regex_replace_str

regex_replace_str — regular-expression op **replace str**. Category: *file stat / path*.

```
p regex_replace_str $x
```

regex_split_str

regex_split_str — regular-expression op **split str**. Category: *file stat / path*.

```
p regex_split_str $x
```


regex_to_nfa_thompson

`regex_to_nfa_thompson` — convert from `regex` to `nfa thompson`. Category: *compilers / parsing*.

```
p regex_to_nfa_thompson $value
```

regexp_matches

`regexp_matches` — operation `regexp matches`. Category: *postgres sql strings, json, aggregates*.

```
p regexp_matches $x
```

regexp_replace

`regexp_replace` — operation `regexp replace`. Category: *postgres sql strings, json, aggregates*.

```
p regexp_replace $x
```

regexp_split

`regexp_split` — operation `regexp split`. Category: *postgres sql strings, json, aggregates*.

```
p regexp_split $x
```

regr_intercept

`regr_intercept` — operation `regr intercept`. Category: *postgres sql strings, json, aggregates*.

```
p regr_intercept $x
```

regr_r2

`regr_r2` — operation `regr r2`. Category: *postgres sql strings, json, aggregates*.

```
p regr_r2 $x
```

regr_slope

`regr_slope` — operation `regr slope`. Category: *postgres sql strings, json, aggregates*.

```
p regr_slope $x
```

regula_falsi

`regula_falsi` — operation `regula falsi`. Category: *more extensions (2)*.

```
p regula_falsi $x
```

regular_ngon_area

`regular_ngon_area` — operation `regular ngon area`. Category: *geometry / topology*.

```
p regular_ngon_area $x
```

regular_ngon_circumradius

`regular_ngon_circumradius` — operation `regular ngon circumradius`. Category: *geometry / topology*.

```
p regular_ngon_circumradius $x
```

regular_ngon_inradius

`regular_ngon_inradius` — operation `regular_ngon_inradius`. Category: *geometry / topology*.

```
p regular_ngon_inradius $x
```

regulator_naive

`regulator_naive` — operation `regulator_naive`. Category: *cryptanalysis & number theory deep*.

```
p regulator_naive $x
```

reject_if

`reject_if` — operation `reject_if`. Category: *functional combinators*.

```
p reject_if $x
```

rel_energy_pm

`rel_energy_pm` — operation `rel_energy_pm`. Category: *em / optics / relativity*.

```
p rel_energy_pm $x
```

rel_ke

`rel_ke` — operation `rel_ke`. Category: *em / optics / relativity*.

```
p rel_ke $x
```

rel_momentum

`rel_momentum` — operation `rel_momentum`. Category: *em / optics / relativity*.

```
p rel_momentum $x
```

rel_total_energy

`rel_total_energy` — operation `rel_total_energy`. Category: *em / optics / relativity*.

```
p rel_total_energy $x
```

rel_velocity_add

`rel_velocity_add` — operation `rel_velocity_add`. Category: *em / optics / relativity*.

```
p rel_velocity_add $x
```

relative_entropy

`relative_entropy(p, q) → number` — alias for `kl_divergence_distributions`.

relative_entropy_coherence

`relative_entropy_coherence` — operation `relative_entropy_coherence`. Category: *quantum mechanics deep*.

```
p relative_entropy_coherence $x
```

relative_entropy_kl

`relative_entropy_kl` — operation `relative_entropy kl`. Category: **electrochemistry, batteries, fuel cells**.

```
p relative_entropy_kl $x
```

relative_humidity_step

`relative_humidity_step` — single-step update of `relative_humidity`. Category: **climate, fluids, atmospheric**.

```
p relative_humidity_step $x
```

relative_luminance

`relative_luminance` — operation `relative_luminance`. Category: **misc**.

```
p relative_luminance $x
```

relative_vorticity_step

`relative_vorticity_step` — single-step update of `relative_vorticity`. Category: **climate, fluids, atmospheric**.

```
p relative_vorticity_step $x
```

relativistic_doppler

`relativistic_doppler` — operation `relativistic_doppler`. Category: **em / optics / relativity**.

```
p relativistic_doppler $x
```

relativistic_energy

`relativistic_energy` — operation `relativistic_energy`. Category: **physics formulas**.

```
p relativistic_energy $x
```

relativistic_kinetic

`relativistic_kinetic` — operation `relativistic_kinetic`. Category: **misc**.

```
p relativistic_kinetic $x
```

relativistic_mass

`relativistic_mass` — operation `relativistic_mass`. Category: **physics formulas**.

```
p relativistic_mass $x
```

relenergy

`relenergy` — operation `relenergy`. Alias for `relativistic_energy`. Category: **physics formulas**.

```
p relenergy $x
```

reliability_diagram

`reliability_diagram` — operation `reliability_diagram`. Category: **excel/sheets + bond/loan financial**.

```
p reliability_diagram $x
```

relmass

relmass — operation `relmass`. Alias for `relativistic_mass`. Category: **physics formulas**.

```
p relmass $x
```

relu

relu — operation `relu`. Category: **math functions**.

```
p relu $x
```

relu_grad

relu_grad — operation `relu_grad`. Category: **ml extensions**.

```
p relu_grad $x
```

remc

remc — operation `remc`. Alias for `remove_clj`. Category: **algebraic match**.

```
p remc $x
```

remez_design

remez_design — operation `remez design`. Category: **economics + game theory**.

```
p remez_design $x
```

remove_all_str

remove_all_str — operation `remove all str`. Category: **extended stdlib**.

```
p remove_all_str $x
```

remove_at

remove_at — operation `remove at`. Category: **list helpers**.

```
p remove_at $x
```

remove_clj

remove_clj — operation `remove clj`. Category: **algebraic match**.

```
p remove_clj $x
```

remove_consonants

remove_consonants — operation `remove consonants`. Category: **string processing extras**.

```
p remove_consonants $x
```

remove_elem

remove_elem — operation `remove elem`. Category: **list helpers**.

```
p remove_elem $x
```

remove_first_elem

`remove_first_elem` — operation `remove first elem`. Category: **list helpers**.

```
p remove_first_elem $x
```

remove_keys

`remove_keys` — operation `remove keys`. Category: **hash ops**.

```
p remove_keys $x
```

remove_seasonality

`remove_seasonality(series, period=12) → array` — subtract per-position mean over the period to deseasonalise.

```
p remove_seasonality(\@monthly, 12)
```

remove_str

`remove_str` — operation `remove str`. Category: **string helpers**.

```
p remove_str $x
```

remove_vowels

`remove_vowels` — operation `remove vowels`. Category: **string processing extras**.

```
p remove_vowels $x
```

remove_whitespace

`remove_whitespace` — operation `remove whitespace`. Category: **extended stdlib**.

```
p remove_whitespace $x
```

renamenx

`renamenx` — operation `renamenx`. Category: **redis-flavour primitives**.

```
p renamenx $x
```

renice

`renice(PID, PRIORITY)` — adjust another process's priority. Same `[-20, 19]` range; requires permission. Returns the new priority or `undef`.

renyi_divergence_step

`renyi_divergence_step` — single-step update of `renyi divergence`. Category: **electrochemistry, batteries, fuel cells**.

```
p renyi_divergence_step $x
```

rep

`rep` — operation `rep`. Alias for `rep_fn`. Category: **r base**.

```
p rep $x
```

rep_fn

`rep_fn` — repeat a value N times. Like R's `rep()`.

```
val @r = @{rep_fn(0, 10)} # ten zeros
var @s = @{rep_fn("x", 5)} # five "x"s
```

repa

`repa` — operation `repa`. Alias for `replace_at`. Category: **javascript array/object methods**.

```
p repa $x
```

repdigit

`repdigit` — operation `repdigit`. Category: **misc**.

```
p repdigit $x
```

repeat

`repeat` — operation `repeat`. Category: **trivial string ops**.

```
p repeat $x
```

repeat_elem

`repeat_elem` — operation `repeat elem`. Category: **list helpers**.

```
p repeat_elem $x
```

repeat_list

`repeat_list` — operation `repeat list`. Category: **collection**.

```
p repeat_list $x
```

repeat_string

`repeat_string` — operation `repeat string`. Category: **string helpers**.

```
p repeat_string $x
```

repeated_game_avg_payoff

`repeated_game_avg_payoff` — operation `repeated game avg payoff`. Category: **climate, fluids, atmospheric**.

```
p repeated_game_avg_payoff $x
```

repeatedly

`repeatedly` — operation `repeatedly`. Category: **algebraic match**.

```
p repeatedly $x
```

repl_alias

`repl_alias(NAME, EXPANSION)` / `repl_alias("NAME=EXPANSION")` — register a REPL alias. With one arg as a query (`repl_alias("ll")`), returns the expansion or `undef`. With no args, returns the full alias table as `name=expansion` strings. The REPL prompt rewriter consults this table before each parse.

Renamed from bare `alias` so Perl typeglob assignment (`*alias = \&sub`) keeps working.

```
repl_alias("ll", "ls -la")
repl_alias("g=git")          # equals-form
p repl_alias("ll")           # ls -la
```

repl_unalias

`repl_unalias(NAME)` — drop one REPL alias. Returns `1` if removed, `0` if not present.

replace_all_str

`replace_all_str` — operation `replace all str`. Category: `*string*`.

```
p replace_all_str $x
```

replace_at

`replace_at` — operation `replace at`. Category: `*javascript array/object methods*`.

```
p replace_at $x
```

replace_at_index

`replace_at_index` — index of `replace at`. Category: `*misc*`.

```
p replace_at_index $x
```

replace_first

`replace_first` — operation `replace first`. Category: `*string*`.

```
p replace_first $x
```

replace_n_times

`replace_n_times` — operation `replace n times`. Category: `*extended stdlib*`.

```
p replace_n_times $x
```

replace_regex

`replace_regex` — operation `replace regex`. Category: `*extended stdlib*`.

```
p replace_regex $x
```

replicate_val

`replicate_val` — operation `replicate val`. Category: `*haskell list functions*`.

```
p replicate_val $x
```

replicator_dynamics_step

`replicator_dynamics_step` — single-step update of `replicator_dynamics`. Category: **climate, fluids, atmospheric**.

```
p replicator_dynamics_step $x
```

repln

`repln` — operation `repln`. Alias for `replace_n_times`. Category: **extended stdlib**.

```
p repln $x
```

replre

`replre` — operation `replre`. Alias for `replace_regex`. Category: **extended stdlib**.

```
p replre $x
```

representation_dim_step

`representation_dim_step` — single-step update of `representation_dim`. Category: **econometrics**.

```
p representation_dim_step $x
```

repunit

`repunit` — operation `repunit`. Category: **misc**.

```
p repunit $x
```

repv

`repv` — operation `repv`. Alias for `replicate_val`. Category: **haskell list functions**.

```
p repv $x
```

resample

`resample(\@signal, $factor)` — resamples signal by a rational factor. Factor > 1 upsamples, factor < 1 downsamples. Uses linear interpolation.

```
val $up = resample(\@signal, 2)      # double sample rate
var $down = resample(\@signal, 0.5)  # halve sample rate
```

resample_linear

`resample_linear` — operation `resample linear`. Category: **signal processing**.

```
p resample_linear $x
```

resample_nearest

`resample_nearest` — operation `resample nearest`. Category: **signal processing**.

```
p resample_nearest $x
```


resample_poly_step

`resample_poly_step` — single-step update of `resample poly`. Category: *economics + game theory*.

```
p resample_poly_step $x
```

reserve_prospective

`reserve_prospective` — operation `reserve prospective`. Category: *actuarial science*.

```
p reserve_prospective $x
```

reserve_retrospective

`reserve_retrospective` — operation `reserve retrospective`. Category: *actuarial science*.

```
p reserve_retrospective $x
```

reservoir_sample

`reservoir_sample` — operation `reservoir sample`. Category: *array / list operations extras*.

```
p reservoir_sample $x
```

reservoir_sample_weighted

`reservoir_sample_weighted` — operation `reservoir sample weighted`. Category: *random / sampling extras*.

```
p reservoir_sample_weighted $x
```

reset

`reset` — operation `reset`. Category: *process / system*.

```
p reset $x
```

resfreq

`resfreq` — operation `resfreq`. Alias for `resonant_frequency`. Category: *physics formulas*.

```
p resfreq $x
```

reshape_dim

`reshape_dim` — operation `reshape dim`. Category: *apl/j/k array primitives*.

```
p reshape_dim $x
```

residuals_compute

`residuals_compute` — operation `residuals compute`. Category: *misc*.

```
p residuals_compute $x
```

resistance_level

`resistance_level(prices, period=20) → number` — rolling max over `period`; the most recent local ceiling.

```
p resistance_level(\@closes, 20)
```

resistance_parallel

`resistance_parallel` — operation `resistance parallel`. Category: **misc**.

```
p resistance_parallel $x
```

resistance_series

`resistance_series` — operation `resistance series`. Category: **misc**.

```
p resistance_series $x
```

resolution_step

`resolution_step` — single-step update of `resolution`. Category: **logic, proof, sat/smt, type theory**.

```
p resolution_step $x
```

resonance_count

`resonance_count` — count of `resonance`. Category: **chemistry & biochemistry**.

```
p resonance_count $x
```

resonant_frequency

`resonant_frequency` — operation `resonant frequency`. Category: **physics formulas**.

```
p resonant_frequency $x
```

rest

`rest` — operation `rest`. Category: **algebraic match**.

```
p rest $x
```

rest_energy

`rest_energy` — operation `rest energy`. Category: **physics formulas**.

```
p rest_energy $x
```

restenergy

`restenergy` — operation `restenergy`. Alias for `rest_energy`. Category: **physics formulas**.

```
p restenergy $x
```

restore_random_state

`restore_random_state` — operation `restore random state`. Category: **random / sampling extras**.

```
p restore_random_state $x
```

rev

`rev` — operation `rev`. Category: **short aliases**.

```
p rev $x
```

rev_lines

`rev_lines(STRING)` → reverse a multi-line string line-by-line (the LINE order, not chars within each line). Pairs with `tac` for the list-form variant.

```
p rev_lines(slurp("log.txt"))           # newest entries on top
```

rev_str

`rev_str` — operation `rev str`. Alias for `reverse_str`. Category: **trivial string ops**.

```
p rev_str $x
```

rev_vec

`rev_vec` — operation `rev vec`. Category: **r base**.

```
p rev_vec $x
```

revenue_equivalence_check

`revenue_equivalence_check` — operation `revenue equivalence check`. Category: **climate, fluids, atmospheric**.

```
p revenue_equivalence_check $x
```

reverse_delete_step

`reverse_delete_step` — single-step update of `reverse delete`. Category: **networkx graph algorithms**.

```
p reverse_delete_step $x
```

reverse_each

`reverse_each` — operation `reverse each`. Category: **trivial numeric helpers**.

```
p reverse_each $x
```

reverse_each_word

`reverse_each_word` — operation `reverse each word`. Category: **string processing extras**.

```
p reverse_each_word $x
```

reverse_lines

`tac(\@LIST)` → reverse the order of a list. Mirrors POSIX `tac`. Composes with `cat` / pipes for traditional shell idioms.

```
val @rev = tac([sort @items])
var @recent_first = tac([read_lines("history.log")])
```

reverse_list

`reverse_list` — operation `reverse list`. Category: **file stat / path**.

```
p reverse_list $x
```

reverse_str

`reverse_str` — operation `reverse str`. Category: **trivial string ops**.

```
p reverse_str $x
```

reverse_transcribe

`reverse_transcribe` — operation `reverse transcribe`. Category: **bioinformatics deep**.

```
p reverse_transcribe $x
```

reverse_words

`reverse_words` — operation `reverse words`. Category: **string**.

```
p reverse_words $x
```

revwords

`revwords` — operation `revwords`. Alias for `reverse_each_word`. Category: **string processing extras**.

```
p revwords $x
```

rexp

`rexp` — generate n random exponential variates. Args: n [, rate]. Like R's `rexp()`.

```
val @x = @{rexp(100, 0.5)} # rate=0.5 (mean=2)
```

reynolds_full_number

`reynolds_full_number` — operation `reynolds full number`. Category: **climate, fluids, atmospheric**.

```
p reynolds_full_number $x
```

rf

`rf` — generate n random F variates. Args: n, d1, d2. Like R's `rf()`.

```
val @x = @{rf(100, 5, 10)} # F(5,10)
```

rfact

`rfact` — operation `rfact`. Alias for `rising_factorial`. Category: **math formulas**.

```
p rfact $x
```

rfc3339

`rfc3339` — operation `rfc3339`. Alias for `rfc3339_format`. Category: **misc**.

```
p rfc3339 $x
```

rfc3339_format

`rfc3339_format` — operation `rfc3339 format`. Category: **misc**.

```
p rfc3339_format $x
```

rfc3339_parse

`rfc3339_parse` — operation `rfc3339 parse`. Category: **misc**.

```
p rfc3339_parse $x
```

rg_radius_of_gyration

`rg_radius_of_gyration` — operation `rg radius of gyration`. Category: **chemistry & biochemistry**.

```
p rg_radius_of_gyration $x
```

rgamma

`rgamma` — generate `n` random gamma variates. Args: `n`, `shape` [, `scale`]. Like R's `rgamma()`.

```
val @x = @{rgamma(100, 2, 1)} # Gamma(2,1)
```

rgb

`rgb` — operation `rgb`. Category: **color / ansi**.

```
p rgb $x
```

rgb2hsl

`rgb2hsl` — operation `rgb2hsl`. Alias for `rgb_to_hsl`. Category: **color operations**.

```
p rgb2hsl $x
```

rgb2hsv

`rgb2hsv` — operation `rgb2hsv`. Alias for `rgb_to_hsv`. Category: **color operations**.

```
p rgb2hsv $x
```

rgb_blend_color_burn

`rgb_blend_color_burn(a, b) → [r,g,b]` — color burn: $255 - (255-a) \cdot 255/b$, clamped.

rgb_blend_color_dodge

`rgb_blend_color_dodge(a, b) → [r,g,b]` — color dodge: $a \cdot 255 / (255 - b)$, clamped.

rgb_blend_darken

`rgb_blend_darken(a, b) → [r,g,b]` — channelwise min.

rgb_blend_lighten

`rgb_blend_lighten(a, b) → [r,g,b]` — channelwise max.

rgb_blend_multiply

`rgb_blend_multiply([r,g,b], [r,g,b]) → [r,g,b]` — Photoshop-style multiply: $a \cdot b / 255$.

```
p rgb_blend_multiply([255,128,0], [128,255,0])
```

rgb_blend_normal

`rgb_blend_normal([r,g,b], [r,g,b], alpha=1) → [r,g,b]` — alpha-blended overlay of two RGB triples (0-255).

```
p rgb_blend_normal([255,0,0], [0,255,0], 0.5)
```

rgb_blend_overlay

`rgb_blend_overlay(a, b) → [r,g,b]` — overlay = multiply for dark base, screen for light.

rgb_blend_screen

`rgb_blend_screen(a, b) → [r,g,b]` — Photoshop screen: $255 - (255-a) \cdot (255-b) / 255$.

rgb_to_adobe_rgb

`rgb_to_adobe_rgb` — convert from `rgb` to `adobe rgb`. Category: *complex / geom / color / trig*.

```
p rgb_to_adobe_rgb $value
```

rgb_to_cmyk

`rgb_to_cmyk` — convert from `rgb` to `cmyk`. Category: *b82-misc-utility*.

```
p rgb_to_cmyk $value
```

rgb_to_color_temperature

`rgb_to_color_temperature` — convert from `rgb` to `color temperature`. Category: *complex / geom / color / trig*.

```
p rgb_to_color_temperature $value
```

rgb_to_hex

`rgb_to_hex` — convert from `rgb` to `hex`. Category: *color / ansi*.

```
p rgb_to_hex $value
```

rgb_to_hsl

`rgb_to_hsl` — convert from `rgb` to `hsl`. Category: *color operations*.

```
p rgb_to_hsl $value
```

rgb_to_hsv

`rgb_to_hsv` — convert from `rgb` to `hsv`. Category: *color operations*.

```
p rgb_to_hsv $value
```

rgb_to_lch

`rgb_to_lch([r,g,b]) → [L, C, H]` — convert sRGB to CIELAB-then-LCH (cylindrical CIELAB).

```
p rgb_to_lch([128, 64, 200])
```

rgb_to_oklab

`rgb_to_oklab` — convert from `rgb` to `oklab`. Category: `*b82-misc-utility*`.

```
p rgb_to_oklab $value
```

rgb_to_oklch

`rgb_to_oklch([r,g,b])` → `[L, C, H]` — convert to Björn Ottosson's OKLab in cylindrical form. Perceptually uniform.

```
p rgb_to_oklch([128, 64, 200])
```

rgb_to_p3

`rgb_to_p3` — convert from `rgb` to `p3`. Category: `*complex / geom / color / trig*`.

```
p rgb_to_p3 $value
```

rgb_to_srgb

`rgb_to_srgb` — convert from `rgb` to `srgb`. Category: `*complex / geom / color / trig*`.

```
p rgb_to_srgb $value
```

rgb_to_xyz

`rgb_to_xyz` — convert from `rgb` to `xyz`. Category: `*b82-misc-utility*`.

```
p rgb_to_xyz $value
```

rgb_to_yiq

`rgb_to_yiq` — convert from `rgb` to `yiq`. Category: `*astronomy / music / color / units*`.

```
p rgb_to_yiq $value
```

rgb_to_yuv

`rgb_to_yuv` — convert from `rgb` to `yuv`. Category: `*b82-misc-utility*`.

```
p rgb_to_yuv $value
```

rgb_to_yuv601

`rgb_to_yuv601` — convert from `rgb` to `yuv601`. Category: `*astronomy / music / color / units*`.

```
p rgb_to_yuv601 $value
```

rgeom

`rgeom` — generate n random geometric variates. Args: n, prob.

```
val @x = @{rgeom(100, 0.3)} # trials until first success
```

rho

`spearman` (aliases `spearman_corr`, `rho`) computes Spearman's rank correlation coefficient between two samples.

```
p rho([1,2,3,4,5], [5,6,7,8,7]) # ~0.82
```

rhumb_line_bearing

`rhumb_line_bearing` — operation `rhumb line bearing`. Category: *excel/sheets + bond/loan financial*.

```
p rhumb_line_bearing $x
```

rhyper

`rhyper` — operation `rhyper`. Category: *extras*.

```
p rhyper $x
```

ricci_identity_step

`ricci_identity_step` — single-step update of `ricci identity`. Category: *electrochemistry, batteries, fuel cells*.

```
p ricci_identity_step $x
```

ricci_tensor_step_zero

`ricci_tensor_step_zero` — operation `ricci tensor step zero`. Category: *electrochemistry, batteries, fuel cells*.

```
p ricci_tensor_step_zero $x
```

richardson_number_step

`richardson_number_step` — single-step update of `richardson number`. Category: *climate, fluids, atmospheric*.

```
p richardson_number_step $x
```

richter_local_ml

`richter_local_ml` — operation `richter local ml`. Category: *geology, seismology, mineralogy*.

```
p richter_local_ml $x
```

ricker_wavelet

`ricker_wavelet` — operation `ricker wavelet`. Category: *economics + game theory*.

```
p ricker_wavelet $x
```

ridders

`ridders` — operation `ridders`. Alias for `ridders_root`. Category: *more extensions (2)*.

```
p ridders $x
```

ridders_root

`ridders_root` — operation `ridders root`. Category: *more extensions (2)*.

```
p ridders_root $x
```

ridge_noise_2d

`ridge_noise_2d(x, y)` → `number` — `1 - |perlin|`. Sharp ridges for mountains.

ridx

`ridx` — operation `ridx`. Alias for `rindex_fn`. Category: **python/ruby stdlib**.

```
p ridx $x
```

riemann_curvature_normal_form

`riemann_curvature_normal_form` — operation `riemann curvature normal form`. Category: **electrochemistry, batteries, fuel cells**.

```
p riemann_curvature_normal_form $x
```

riemann_tensor_step_zero

`riemann_tensor_step_zero` — operation `riemann tensor step zero`. Category: **electrochemistry, batteries, fuel cells**.

```
p riemann_tensor_step_zero $x
```

riemann_xi

`riemann_xi` — operation `riemann xi`. Category: **special functions extra**.

```
p riemann_xi $x
```

riffle

`riffle` — operation `riffle`. Category: **collection helpers (trivial)**.

```
p riffle $x
```

right_str

`right_str` — operation `right str`. Category: **string**.

```
p right_str $x
```

rindex_fn

`rindex_fn` — operation `rindex fn`. Category: **python/ruby stdlib**.

```
p rindex_fn $x
```

ring_modulate

`ring_modulate(a, b) → array` — elementwise product of two signals. Bell-like inharmonic spectra.

rising_factorial

`rising_factorial` — operation `rising factorial`. Category: **math formulas**.

```
p rising_factorial $x
```

risk_ratio_2x2

`risk_ratio_2x2` — operation `risk ratio 2x2`. Category: **epidemiology / public health**.

```
p risk_ratio_2x2 $x
```

rjt

`rjt` — operation `rjt`. Alias for `rjust_text`. Category: **extended stdlib**.

```
p rjt $x
```

rjust

`rjust` — operation `rjust`. Category: **string helpers**.

```
p rjust $x
```

rjust_text

`rjust_text` — operation `rjust text`. Category: **extended stdlib**.

```
p rjust_text $x
```

rk4

`rk4` (aliases `runge_kutta`, `rk4_ode`) solves an ODE $dy/dt = f(t,y)$ using 4th-order Runge-Kutta. Returns $[[t,y], \dots]$.

```
# dy/dt = -y, y(0) = 1 → y = e^(-t)
var $sol = rk4(fn { -$_[1] }, 0, 1, 0.1, 100)
```

rk45_cash_karp

`rk45_cash_karp` — operation `rk45 cash karp`. Category: **more extensions (2)**.

```
p rk45_cash_karp $x
```

rk4_single

`rk4_single` — operation `rk4 single`. Category: **ode advanced**.

```
p rk4_single $x
```

rk_combine

`rk_combine` — operation `rk combine`. Category: **ode advanced**.

```
p rk_combine $x
```

rkck

`rkck` — operation `rkck`. Alias for `rk45_cash_karp`. Category: **more extensions (2)**.

```
p rkck $x
```

rkf45_error

`rkf45_error` — operation `rkf45 error`. Category: **ode advanced**.

```
p rkf45_error $x
```

rl_discount_returns

`rl_discount_returns(rewards, gamma=0.9) → array` — backward-cumulative discounted returns $G_t = r_t + \gamma \cdot G_{t+1}$.

rl_n_step_return

`rl_n_step_return(rewards, gamma=0.9) → number` — $\sum \gamma^k \cdot r_k$. Bootstrap-free n-step return.

rl_tau

`rl_tau` — operation `rl tau`. Category: *em / optics / relativity*.

```
p rl_tau $x
```

rl_td_error

`rl_td_error(r, v_next, v, gamma=0.9) → number` — $r + \gamma \cdot V(s') - V(s)$. TD(0) error term.

rl_time_constant

`rl_time_constant` — operation `rl time constant`. Category: *physics formulas*.

```
p rl_time_constant $x
```

rld

`rld` — operation `rld`. Alias for `run_length_decode`. Category: *collection more*.

```
p rld $x
```

rld_str

`rld_str` — operation `rld str`. Alias for `run_length_decode_str`. Category: *array / list operations extras*.

```
p rld_str $x
```

rle

`rle` — operation `rle`. Alias for `run_length_encode`. Category: *collection more*.

```
p rle $x
```

rle_compress

`rle_compress(s) → string` — run-length encode runs of 1-9 chars: "aaabbc" → "3a2b1c".

```
p rle_compress("aaabbc") # "3a2b1c"
```

rle_decompress

`rle_decompress(s) → string` — inverse of `rle_compress`.

rle_str

`rle_str` — run-length encoding `str`. Alias for `run_length_encode_str`. Category: *array / list operations extras*.

```
p rle_str $x
```

rlnorm

rlnorm — generate n random log-normal variates. Args: n [, mu, sigma].

```
val @x = @{rlnorm(100, 0, 1)}    # LogN(0,1)
```

rlogis

rlogis — operation **rlogis**. Category: *extras*.

```
p rlogis $x
```

rltau

rltau — operation **rltau**. Alias for **rl_time_constant**. Category: *physics formulas*.

```
p rltau $x
```

rm

rm PATH... — remove one or more files. Returns the count actually removed. Silently skips paths that don't exist (Perl **unlink** semantics applied to the shell-name).

```
rm("/tmp/foo")  
rm(@stale_files)    # bulk
```

rmall

rmall — operation **rmall**. Alias for **remove_all_str**. Category: *extended stdlib*.

```
p rmall $x
```

rmcons

rmcons — operation **rmcons**. Alias for **remove_consonants**. Category: *string processing extras*.

```
p rmcons $x
```

rms

rms — operation **rms**. Category: *extended stdlib*.

```
p rms $x
```

rmse

rmse — operation **rmse**. Category: *math functions*.

```
p rmse $x
```

rmvowel

rmvowel — operation **rmvowel**. Alias for **remove_vowels**. Category: *string processing extras*.

```
p rmvowel $x
```

rmws

`rmws` — operation `rmws`. Alias for `remove_whitespace`. Category: **extended stdlib**.

```
p rmws $x
```

rna_gc_content

`rna_gc_content` — RNA sequence op `gc_content`. Category: **test runner**.

```
p rna_gc_content $x
```

rna_hamming

`rna_hamming(a, b) → int` — Hamming distance between two equal-length sequences.

rna_reverse_complement

`rna_reverse_complement(rna) → string` — reverse complement using RNA pairing ($A \leftrightarrow U$, $G \leftrightarrow C$).

rna_to_dna

`rna_to_dna(rna) → string` — reverse transcription: $U \rightarrow T$.

rngcmp

`rngcmp` — operation `rngcmp`. Alias for `range_compress`. Category: **array / list operations extras**.

```
p rngcmp $x
```

rngexp

`rngexp` — operation `rngexp`. Alias for `range_expand`. Category: **array / list operations extras**.

```
p rngexp $x
```

rnk

`rnk` — operation `rnk`. Alias for `rank`. Category: **extended stdlib**.

```
p rnk $x
```

rnorm

`rnorm` — generate n random normal variates. Args: n [, mu, sigma]. Like R's `rnorm()`.

```
val @x = @{rnorm(1000)}      # 1000 standard normal
var @y = @{rnorm(100, 50, 10)} # mean=50, sd=10
```

roberts_cross_kernel

`roberts_cross_kernel() → matrix` — 2×2 Roberts cross-gradient kernel.

roberts_kernel_value

`roberts_kernel_value` — value of `roberts_kernel`. Category: **electrochemistry, batteries, fuel cells**.

```
p roberts_kernel_value $x
```

robinson_project

`robinson_project` — operation `robinson project`. Category: **more extensions**.

```
p robinson_project $x
```

robinson_schensted_pair

`robinson_schensted_pair` — operation `robinson schensted pair`. Category: **econometrics**.

```
p robinson_schensted_pair $x
```

robust_scale

`robust_scale` — operation `robust scale`. Category: **ml extensions**.

```
p robust_scale $x
```

robust_se_huber_white

`robust_se_huber_white` — operation `robust se huber white`. Category: **econometrics**.

```
p robust_se_huber_white $x
```

roc

`roc(prices, period=10) → array` — rate of change: $100 \cdot (\text{price}[i] - \text{price}[i\text{-period}]) / \text{price}[i\text{-period}]$. Pure momentum oscillator centred at 0.

```
p roc(\@closes, 12)
```

roche_limit_fluid

`roche_limit_fluid` — operation `roche limit fluid`. Category: **cosmology / gr / flrw**.

```
p roche_limit_fluid $x
```

roche_limit_rigid

`roche_limit_rigid` — operation `roche limit rigid`. Category: **cosmology / gr / flrw**.

```
p roche_limit_rigid $x
```

roi

`roi(gain, cost) → number` — Return on Investment as percent: $(\text{gain} - \text{cost}) / \text{cost} \cdot 100$.

```
p roi(1500, 1000) # 50
```

roll_n

`roll_n` — operation `roll n`. Category: **apl/j/k array primitives**.

```
p roll_n $x
```

rolling_kurtosis

`rolling_kurtosis(series, period=10) → array` — sliding excess kurtosis.

rolling_max

`rolling_max(series, period=10)` → array — sliding maximum.

rolling_mean

`rolling_mean(series, period=10)` → array — sliding mean. Output length is `len - period + 1`.

```
p rolling_mean([1,2,3,4,5], 3) # [2.0, 3.0, 4.0]
```

rolling_median

`rolling_median(series, period=10)` → array — sliding median; robust to outliers.

```
p rolling_median(\@xs, 5)
```

rolling_min

`rolling_min(series, period=10)` → array — sliding minimum.

rolling_skew

`rolling_skew(series, period=10)` → array — sliding skewness (third central moment / σ^3).

rolling_std

`rolling_std(series, period=10)` → array — sliding (population) standard deviation.

rolling_sum

`rolling_sum(series, period=10)` → array — sliding sum.

```
p rolling_sum([1,2,3,4,5], 3) # [6, 9, 12]
```

rolling_var

`rolling_var(series, period=10)` → array — sliding (population) variance.

roman

`roman` — operation `roman`. Alias for `int_to_roman`. Category: **roman numerals**.

```
p roman $x
```

roman_add

`roman_add` — operation `roman` `add`. Category: **roman numerals**.

```
p roman_add $x
```

roman_decode

`roman_decode` — decode of `roman`. Category: **astronomy / music / color / units**.

```
p roman_decode $x
```

roman_encode

`roman_encode` — encode of `roman`. Category: **astronomy / music / color / units**.

```
p roman_encode $x
```

roman_numeral_list

`roman_numeral_list` — operation `roman numeral list`. Category: **roman numerals**.

```
p roman_numeral_list $x
```

roman_to_int

`roman_to_int` — convert from `roman` to `int`. Category: **roman numerals**.

```
p roman_to_int $value
```

romanlist

`romanlist` — operation `romanlist`. Alias for `roman_numeral_list`. Category: **roman numerals**.

```
p romanlist $x
```

romberg_quad

`romberg_quad` — operation `romberg quad`. Category: **numpy + scipy.special**.

```
p romberg_quad $x
```

roommate_match_step

`roommate_match_step` — single-step update of `roommate match`. Category: **climate, fluids, atmospheric**.

```
p roommate_match_step $x
```

root_brentq

`root_brentq` — operation `root brentq`. Category: **numpy + scipy.special**.

```
p root_brentq $x
```

root_newton

`root_newton` — operation `root newton`. Category: **numpy + scipy.special**.

```
p root_newton $x
```

root_secant

`root_secant` — operation `root secant`. Category: **numpy + scipy.special**.

```
p root_secant $x
```

root_system_count

`root_system_count` — count of `root system`. Category: **econometrics**.

```
p root_system_count $x
```


rope

`rope(INITIAL_STRING="")` — text data structure for fast insert/delete in long strings. Chunk-balanced (1 KB max per chunk); amortized $O(\log n)$ edits. Codepoint-safe across UTF-8 boundaries.

```
val $r = rope("Hello, world!")
rope_insert($r, 7, "beautiful ")
p rope_to_string($r)           # "Hello, beautiful world!"
```

rope_delete

`rope_delete(R, START, END)` — delete codepoints `[START, END)`.

rope_insert

`rope_insert(R, POS, TEXT)` — insert `TEXT` at codepoint position `POS` (clamped to `[0, len]`).

rope_len

`rope_len(R)` → codepoint count.

rope_str

`rope_to_string(R)` → materialize as a contiguous String.

rope_substr

`rope_substring(R, START, END)` → substring at codepoint range `[START, END)`.

ros2_step

`ros2_step` — single-step update of `ros2`. Category: **ode advanced**.

```
p ros2_step $x
```

rot13

`rot13` — operation `rot13`. Category: **string**.

```
p rot13 $x
```

rot47

`rot47` — operation `rot47`. Category: **string**.

```
p rot47 $x
```

rotate

`rotate` — operation `rotate`. Category: **collection helpers (trivial)**.

```
p rotate $x
```

rotate_axis

`rotate_axis` — operation `rotate axis`. Category: **apl/j/k array primitives**.

```
p rotate_axis $x
```

rotate_point

`rotate_point` — operation `rotate point`. Category: **math functions**.

```
p rotate_point $x
```

roulette_wheel_select_index

`roulette_wheel_select_index` — index of `roulette wheel select`. Category: **combinatorial optimization, scheduling**.

```
p roulette_wheel_select_index $x
```

round

`round` — operation `round`. Category: **trivial numeric / predicate builtins**.

```
p round $x
```

round_each

`round_each` — operation `round each`. Category: **trivial numeric helpers**.

```
p round_each $x
```

round_to

`round_to` — operation `round to`. Category: **math functions**.

```
p round_to $x
```

roundrobin

`roundrobin` — operation `roundrobin`. Category: **iterator + string-distance extras**.

```
p roundrobin $x
```

rowMeans

`row_means` (alias `rowMeans`) — mean of each row. Like R's `rowMeans()`.

```
p rowMeans([[2,4],[6,8]]) # [3, 7]
```

rowSums

`row_sums` (alias `rowSums`) — sum of each row in a matrix. Like R's `rowSums()`.

```
p row_sums([[1,2,3],[4,5,6]]) # [6, 15]
```

rpadd

`rpadd` — operation `rpadd`. Alias for `pad_right`. Category: **trivial string ops**.

```
p rpadd $x
```

rpn

`rpn` — operation `rpn`. Alias for `eval_rpn`. Category: **algorithms / puzzles**.

```
p rpn $x
```

rpois

rpois — generate *n* random Poisson variates. Args: *n*, *lambda*. Like R's `rpois()`.

```
val @x = @{rpois(100, 5)} # Poisson with mean 5
```

rptd

rptd — operation **rptd**. Alias for **repeatedly**. Category: **algebraic match**.

```
p rptd $x
```

rpush

rpush — operation **rpush**. Category: **redis-flavour primitives**.

```
p rpush $x
```

rrt_extend

rrt_extend — operation **rrt extend**. Category: **robotics & control**.

```
p rrt_extend $x
```

rrt_star_rewire

rrt_star_rewire — operation **rrt star rewire**. Category: **robotics & control**.

```
p rrt_star_rewire $x
```

rrule_next_occurrence

rrule_next_occurrence — operation **rrule next occurrence**. Category: **b82-misc-utility**.

```
p rrule_next_occurrence $x
```

rsa_d_from_e

rsa_d_from_e — RSA cryptographic primitive **d from e**. Category: **cryptanalysis & number theory deep**.

```
p rsa_d_from_e $x
```

rsa_decrypt_pkcs1

rsa_decrypt_pkcs1 decrypts RSA-PKCS1v15 ciphertext.

```
val $plain = rsa_decrypt_pkcs1($priv, $cipher)
```

rsa_encrypt_pkcs1

rsa_encrypt_pkcs1 encrypts with legacy RSA-PKCS1v15 padding. Only for compatibility with old systems — prefer OAEP.

```
val $cipher = rsa_encrypt_pkcs1($pub, "data")
```

rsa_encrypt_simple

`rsa_encrypt_simple` — RSA cryptographic primitive `encrypt simple`. Category: *cryptography deep*.

```
p rsa_encrypt_simple $x
```

rsa_keypair_simple

`rsa_keypair_simple(p, q, e=17) → {n, e, d}` — textbook RSA from two primes. NOT cryptographically secure.

```
p rsa_keypair_simple(61, 53) # {n:3233, e:17, d:2753}
```

rsa_modular_exp

`rsa_modular_exp(base, exp, modulus) → string` — `base^exp mod modulus` on arbitrary-precision BigUints (decimal strings).

```
p rsa_modular_exp("2", "10", "100") # "24"
```

rsamp

`rsamp` — operation `rsamp`. Alias for `reservoir_sample`. Category: *array / list operations extras*.

```
p rsamp $x
```

rsi

`rsi(prices, period=14) → array` — Relative Strength Index (Wilder). Computes $100 - 100 / (1 + \text{avg_gain} / \text{avg_loss})$ over a moving window; values >70 commonly mark overbought, <30 oversold.

```
p rsi(\@closes, 14)
```

rsk_size

`rsk_size` — operation `rsk size`. Category: *econometrics*.

```
p rsk_size $x
```

rst

`rst` — operation `rst`. Alias for `rest`. Category: *algebraic match*.

```
p rst $x
```

rsun

`rsun` — operation `rsun`. Alias for `sun_radius`. Category: *physics constants*.

```
p rsun $x
```

rt

`rt` — generate n random Student's t variates. Args: n, df. Like R's `rt()`.

```
val @x = @{rt(100, 10)} # t with 10 df
```

rt_effective

`rt_effective` — operation `rt effective`. Category: **biology / ecology**.

```
p rt_effective $x
```

rt_serial_interval

`rt_serial_interval` — operation `rt serial interval`. Category: **epidemiology / public health**.

```
p rt_serial_interval $x
```

rtt_avg

`rtt_avg` — operation `rtt avg`. Category: **network / ip / cidr**.

```
p rtt_avg $x
```

rtt_max

`rtt_max` — maximum of `rtt`. Category: **network / ip / cidr**.

```
p rtt_max $x
```

rtt_min

`rtt_min` — minimum of `rtt`. Category: **network / ip / cidr**.

```
p rtt_min $x
```

ruin_probability_lundberg

`ruin_probability_lundberg` — operation `ruin probability lundberg`. Category: **actuarial science**.

```
p ruin_probability_lundberg $x
```

run_length_decode

`run_length_decode` — decode of `run length`. Category: **collection more**.

```
p run_length_decode $x
```

run_length_decode_str

`run_length_decode_str` — operation `run length decode str`. Category: **array / list operations extras**.

```
p run_length_decode_str $x
```

run_length_encode

`run_length_encode` — encode of `run length`. Category: **collection more**.

```
p run_length_encode $x
```

run_length_encode_str

`run_length_encode_str` — operation `run length encode str`. Category: **array / list operations extras**.

```
p run_length_encode_str $x
```

run_off_triangle_step

`run_off_triangle_step` — single-step update of `run off triangle`. Category: **actuarial science**.

```
p run_off_triangle_step $x
```

runif

`runif` — generate n random uniform variates. Args: n [, min, max]. Like R's `runif()`.

```
val @x = @{runif(100, 0, 1)} # 100 uniform [0,1]
```

running_max

`running_max` — maximum of `running`. Category: **list helpers**.

```
p running_max $x
```

running_mean

`running_mean` — arithmetic mean of `running`. Category: **misc**.

```
p running_mean @xs
```

running_min

`running_min` — minimum of `running`. Category: **list helpers**.

```
p running_min $x
```

running_reduce

`running_reduce` — operation `running reduce`. Category: **additional missing stdlib functions**.

```
p running_reduce $x
```

running_variance

`running_variance` — operation `running variance`. Category: **misc**.

```
p running_variance $x
```

runred

`runred` — operation `runred`. Alias for `running_reduce`. Category: **additional missing stdlib functions**.

```
p runred $x
```

runs

`runs` — operation `runs`. Category: **iterator + string-distance extras**.

```
p runs $x
```

rweibull

`rweibull` — generate n random Weibull variates. Args: n, shape [, scale].

```
val @x = @{rweibull(100, 2, 1)} # Weibull(2,1)
```

rx_gate

`rx_gate` — operation `rx gate`. Category: **more extensions**.

```
p rx_gate $x
```

rxn_q

`rxn_q` — operation `rxn q`. Alias for `reaction_quotient`. Category: **chemistry**.

```
p rxn_q $x
```

ry_gate

`ry_gate` — operation `ry gate`. Category: **more extensions**.

```
p ry_gate $x
```

rydberg

`rydberg` — operation `rydberg`. Alias for `rydberg_constant`. Category: **physics constants**.

```
p rydberg $x
```

rydberg_constant

`rydberg_constant` — operation `rydberg constant`. Category: **physics constants**.

```
p rydberg_constant $x
```

rydberg_lambda

`rydberg_lambda` — operation `rydberg lambda`. Category: **chemistry**.

```
p rydberg_lambda $x
```

ryu_takayanagi_step

`ryu_takayanagi_step` — single-step update of `ryu takayanagi`. Category: **electrochemistry, batteries, fuel cells**.

```
p ryu_takayanagi_step $x
```

rz_gate

`rz_gate` — operation `rz gate`. Category: **more extensions**.

```
p rz_gate $x
```

s1

`s1` — operation `s1`. Alias for `sha1`. Category: **crypto / encoding**.

```
p s1 $x
```

s224

`s224` — operation `s224`. Alias for `sha224`. Category: **crypto / encoding**.

```
p s224 $x
```

s256

s256 — operation `s256`. Alias for `sha256`. Category: **crypto / encoding**.

```
p s256 $x
```

s2_cell_at_lat_lng

s2_cell_at_lat_lng — operation `s2 cell at lat lng`. Category: **excel/sheets + bond/loan financial**.

```
p s2_cell_at_lat_lng $x
```

s2_cell_id

s2_cell_id — operation `s2 cell id`. Category: **excel/sheets + bond/loan financial**.

```
p s2_cell_id $x
```

s2_cell_neighbors

s2_cell_neighbors — operation `s2 cell neighbors`. Category: **excel/sheets + bond/loan financial**.

```
p s2_cell_neighbors $x
```

s2c

s2c — operation `s2c`. Alias for `snake_to_camel`. Category: **string processing extras**.

```
p s2c $x
```

s384

s384 — operation `s384`. Alias for `sha384`. Category: **crypto / encoding**.

```
p s384 $x
```

s512

s512 — operation `s512`. Alias for `sha512`. Category: **crypto / encoding**.

```
p s512 $x
```

s_to_m

s_to_m — convert from `s` to `m`. Alias for `seconds_to_minutes`. Category: **unit conversions**.

```
p s_to_m $value
```

s_to_ms

s_to_ms — convert from `s` to `ms`. Category: **file stat / path**.

```
p s_to_ms $value
```

sa_accept_prob

sa_accept_prob — operation `sa accept prob`. Category: **more extensions (2)**.

```
p sa_accept_prob $x
```


sa_boltzmann_temp

sa_boltzmann_temp — operation `sa boltzmann temp`. Category: **more extensions (2)**.

```
p sa_boltzmann_temp $x
```

sa_cauchy_temp

sa_cauchy_temp — operation `sa cauchy temp`. Category: **more extensions (2)**.

```
p sa_cauchy_temp $x
```

sa_geometric_temp

sa_geometric_temp — operation `sa geometric temp`. Category: **more extensions (2)**.

```
p sa_geometric_temp $x
```

sabr_implied_vol

sabr_implied_vol — operation `sabr implied vol`. Category: **financial pricing models**.

```
p sabr_implied_vol $x
```

sadd

sadd — operation `sadd`. Category: **redis-flavour primitives**.

```
p sadd $x
```

saddle_point_check

saddle_point_check — operation `saddle point check`. Category: **climate, fluids, atmospheric**.

```
p saddle_point_check $x
```

safe_div

safe_div — operation `safe div`. Category: **functional combinators**.

```
p safe_div $x
```

safe_log

safe_log — operation `safe log`. Category: **functional combinators**.

```
p safe_log $x
```

safe_mod

safe_mod — operation `safe mod`. Category: **functional combinators**.

```
p safe_mod $x
```

safe_sqrt

safe_sqrt — operation `safe sqrt`. Category: **functional combinators**.

```
p safe_sqrt $x
```

salloc

`salloc` — operation `salloc`. Alias for `stress_alloc`. Category: **stress / telemetry extensions**.

```
p salloc $x
```

salt_activity_coefficient

`salt_activity_coefficient` — operation `salt activity coefficient`. Category: **electrochemistry, batteries, fuel cells**.

```
p salt_activity_coefficient $x
```

sample_n

`sample_n` — operation `sample n`. Alias for `random_sample`. Category: **collection**.

```
p sample_n $x
```

sample_one

`sample_one` — operation `sample one`. Category: **list helpers**.

```
p sample_one $x
```

sample_stddev

`sample_stddev` — operation `sample stddev`. Category: **statistics (extended)**.

```
p sample_stddev $x
```

sample_variance

`sample_variance` — operation `sample variance`. Category: **statistics (extended)**.

```
p sample_variance $x
```

sample_weighted_unique

`sample_weighted_unique` — operation `sample weighted unique`. Category: **random / sampling extras**.

```
p sample_weighted_unique $x
```

sampn

`sampn` — operation `sampn`. Category: **extended stdlib**.

```
p sampn $x
```

sapply

`sapply` — apply a function to each element, return a vector. Like R's `sapply()`.

```
val @sq = @{sapply([1,2,3,4], fn { _0 ** 2 })} # [1,4,9,16]
```

sarimax_fit

`sarimax_fit` — operation `sarimax fit`. Category: **statsmodels**.

```
p sarimax_fit $x
```

saros_cycle

saros_cycle — operation `saros cycle`. Category: **astronomy / astrometry**.

```
p saros_cycle $x
```

sarsa_step

sarsa_step(`q`, `alpha`, `r`, `gamma`, `next_q`) → `number` — $q \leftarrow q + \alpha \cdot (r + \gamma \cdot \text{next_q} - q)$. On-policy TD(0).

sarsa_update

sarsa_update — update step of `sarsa`. Category: **electrochemistry, batteries, fuel cells**.

```
p sarsa_update $x
```

sat01

sat01 — operation `sat01`. Alias for `saturate`. Category: **trig / math**.

```
p sat01 $x
```

saturate

saturate — operation `saturate`. Category: **trig / math**.

```
p saturate $x
```

saturation_equivalent_pt

saturation_equivalent_pt — operation `saturation equivalent pt`. Category: **climate, fluids, atmospheric**.

```
p saturation_equivalent_pt $x
```

saturation_vapor_pressure

saturation_vapor_pressure — operation `saturation vapor pressure`. Category: **misc**.

```
p saturation_vapor_pressure $x
```

save_pct

save_pct — percentage of `save`. Category: **astronomy / astrometry**.

```
p save_pct $x
```

save_random_state

save_random_state — operation `save random state`. Category: **random / sampling extras**.

```
p save_random_state $x
```

savgol_coef

savgol_coef — operation `savgol_coef`. Category: *economics + game theory*.

```
p savgol_coef $x
```

sbranch

sbranch — operation `sbranch`. Alias for `stress_branch`. Category: *stress / telemetry extensions*.

```
p sbranch $x
```

scalar_curvature_step

scalar_curvature_step — single-step update of `scalar_curvature`. Category: *electrochemistry, batteries, fuel cells*.

```
p scalar_curvature_step $x
```

scale

scale — standardize a vector: $(x - \text{mean}) / \text{sd}$. Like R's `scale()`.

```
val @z = @{scale([10,20,30])} # [-1, 0, 1]
```

scale_blues

scale_blues(root) → array — blues scale: minor pentatonic + b5 'blue note'.

scale_chromatic

scale_chromatic(root) → array — all 12 semitones from root.

scale_dorian

scale_dorian(root) → array — Dorian mode (W-H-W-W-W-H-W).

scale_each

scale_each — music-theory scale constructor `each`. Category: *trivial numeric helpers*.

```
p scale_each $x
```

scale_factor

scale_factor — music-theory scale constructor `factor`. Category: *cosmology / gr / flrw*.

```
p scale_factor $x
```

scale_harmonic_minor

scale_harmonic_minor(root) → array — natural minor with raised 7th (W-H-W-W-H-3-H).

scale_locrian

scale_locrian(root) → array — Locrian mode (H-W-W-H-W-W-W).

scale_lydian

`scale_lydian(root)` → `array` — Lydian mode (W-W-W-H-W-W-H).

scale_major

`scale_major(root)` → `array` — major scale (8 notes, intervals W-W-H-W-W-W-H).

```
p scale_major(60) # [60,62,64,65,67,69,71,72]
```

scale_melodic_minor

`scale_melodic_minor(root)` → `array` — ascending melodic minor (W-H-W-W-W-W-H).

scale_minor

`scale_minor(root)` → `array` — natural minor (W-H-W-W-H-W-W).

scale_mixolydian

`scale_mixolydian(root)` → `array` — Mixolydian mode (W-W-H-W-W-H-W).

scale_notes

`scale_notes` — music-theory scale constructor `notes`. Category: **extras**.

```
p scale_notes $x
```

scale_pentatonic

`scale_pentatonic(root)` → `array` — major pentatonic scale (5 notes per octave).

scale_phrygian

`scale_phrygian(root)` → `array` — Phrygian mode (H-W-W-W-H-W-W).

scale_pitches_major

`scale_pitches_major` — music-theory scale constructor `pitches major`. Category: **music theory**.

```
p scale_pitches_major $x
```

scale_pitches_minor

`scale_pitches_minor` — music-theory scale constructor `pitches minor`. Category: **music theory**.

```
p scale_pitches_minor $x
```

scale_point

`scale_point($x, $y, $sx, $sy)` — scales a 2D point by factors `sx` and `sy` from the origin. Returns `[$x*$sx, $y*$sy]`. Use for geometric transformations.

```
val $p = scale_point(2, 3, 2, 2)
p @$p # (4, 6)
```

scale_to_intervals

`scale_to_intervals` — convert from `scale` to `intervals`. Category: **misc**.

```
p scale_to_intervals $value
```

scalept

`scalept` — operation `scalept`. Alias for `scale_point`. Category: **geometry (extended)**.

```
p scalept $x
```

scan

`scan` — operation `scan`. Category: **functional combinators**.

```
p scan $x
```

scan_axis

`scan_axis` — operation `scan axis`. Category: **apl/j/k array primitives**.

```
p scan_axis $x
```

scan_left

`scan_left` — operation `scan left`. Category: **list helpers**.

```
p scan_left $x
```

scanl

`scanl` — operation `scanl`. Category: **haskell list functions**.

```
p scanl $x
```

scanr

`scanr` — operation `scanr`. Category: **haskell list functions**.

```
p scanr $x
```

scard

`scard` — operation `scard`. Category: **redis-flavour primitives**.

```
p scard $x
```

scentroid

`spectral_centroid(@spectrum)` (alias `scentroid`) — computes the spectral centroid (center of mass) of a spectrum. Indicates the 'brightness' of a sound. Returns frequency bin index.

```
val $psd = psd(@signal)
p scentroid($psd) # brightness measure
```

scharr_x_kernel

`scharr_x_kernel()` → `matrix` — Scharr horizontal kernel; better rotational symmetry than Sobel.

`scharr_y_kernel`

`scharr_y_kernel()` → `matrix` — Scharr vertical kernel.

`schmidt_number_step`

`schmidt_number_step` — single-step update of `schmidt number`. Category: **climate, fluids, atmospheric**.

```
p schmidt_number_step $x
```

`schnorr_sign_simple`

`schnorr_sign_simple(msg, private_key)` → `string` — Schnorr-like signature using SHA-256 hash chaining. Demo only.

`schnorr_verify_simple`

`schnorr_verify_simple(msg, public_key, sig)` → `0|1` — companion verify.

`school_choice_match`

`school_choice_match` — operation `school choice match`. Category: **climate, fluids, atmospheric**.

```
p school_choice_match $x
```

`schottky_barrier_height`

`schottky_barrier_height` — operation `schottky barrier height`. Category: **electrochemistry, batteries, fuel cells**.

```
p schottky_barrier_height $x
```

`schouten_tensor_step`

`schouten_tensor_step` — single-step update of `schouten tensor`. Category: **electrochemistry, batteries, fuel cells**.

```
p schouten_tensor_step $x
```

`schreier_index`

`schreier_index` — index of `schreier`. Category: **econometrics**.

```
p schreier_index $x
```

`schrodinger_step_real`

`schrodinger_step_real` — operation `schrodinger step real`. Category: **quantum mechanics deep**.

```
p schrodinger_step_real $x
```

`schroeder_reverb`

`schroeder_reverb(signal)` → `array` — Schroeder's 4-comb + 2-all-pass classical reverb topology.

`schulze_method_step`

`schulze_method_step` — single-step update of `schulze method`. Category: **climate, fluids, atmospheric**.

```
p schulze_method_step $x
```

schur_polynomial_eval

`schur_polynomial_eval` — operation `schur polynomial eval`. Category: **econometrics**.

```
p schur_polynomial_eval $x
```

schwarz

`schwarzschild_radius($mass)` (alias `schwarz`) — event horizon radius $r_s = 2GM/c^2$ of a black hole.

```
p schwarz(1.989e30) # ~2954 m (Sun as black hole)
p schwarz(5.972e24) # ~0.009 m (Earth as black hole)
```

schwarzschild_freefall_time

`schwarzschild_freefall_time` — operation `schwarzschild freefall time`. Category: **cosmology / gr / flrw**.

```
p schwarzschild_freefall_time $x
```

schwarzschild_g_rr

`schwarzschild_g_rr` — operation `schwarzschild g rr`. Category: **cosmology / gr / flrw**.

```
p schwarzschild_g_rr $x
```

schwarzschild_g_tt

`schwarzschild_g_tt` — operation `schwarzschild g tt`. Category: **cosmology / gr / flrw**.

```
p schwarzschild_g_tt $x
```

schwarzschild_isco

`schwarzschild_isco` — operation `schwarzschild isco`. Category: **cosmology / gr / flrw**.

```
p schwarzschild_isco $x
```

schwarzschild_kruskal_step

`schwarzschild_kruskal_step` — single-step update of `schwarzschild kruskal`. Category: **electrochemistry, batteries, fuel cells**.

```
p schwarzschild_kruskal_step $x
```

schwarzschild_radius_kg

`schwarzschild_radius_kg` — operation `schwarzschild radius kg`. Category: **cosmology / gr / flrw**.

```
p schwarzschild_radius_kg $x
```

schwarzschild_radius_m

`schwarzschild_radius_m` — operation `schwarzschild radius m`. Category: **misc**.

```
p schwarzschild_radius_m $x
```


score_test_stat

`score_test_stat` — operation `score test stat`. Category: **econometrics**.

```
p score_test_stat $x
```

script_name

`script_name` — operation `script name`. Category: **process / env**.

```
p script_name $x
```

scrypt_round

`scrypt_round` — operation `scrypt round`. Category: **cryptography deep**.

```
p scrypt_round $x
```

sdbm_hash

`sdbm_hash` — hash function of `sdbm`. Category: **cryptography deep**.

```
p sdbm_hash $x
```

sdiff

`sdiff` — operation `sdiff`. Category: **redis-flavour primitives**.

```
p sdiff $x
```

sdivs

`sdivs` — operation `sdivs`. Alias for `sum_divisors`. Category: **math / numeric extras**.

```
p sdivs $x
```

sdns

`sdns` — operation `sdns`. Alias for `stress_dns`. Category: **stress / telemetry extensions**.

```
p sdns $x
```

sdx

`sdx` — operation `sdx`. Alias for `soundex`. Category: **extended stdlib**.

```
p sdx $x
```

sealed_bid_first_price

`sealed_bid_first_price` — operation `sealed bid first price`. Category: **climate, fluids, atmospheric**.

```
p sealed_bid_first_price $x
```

sealed_bid_second_price

`sealed_bid_second_price` — operation `sealed bid second price`. Category: **climate, fluids, atmospheric**.

```
p sealed_bid_second_price $x
```

searchsorted

`searchsorted` — operation `searchsorted`. Category: **numpy + scipy.special**.

```
p searchsorted $x
```

season_of_year

`season_of_year(month, day, hemisphere="N")` → `string` — Winter/Spring/Summer/Autumn. South-hemisphere flips seasons.

```
p season_of_year(3, 25, "N") # Spring
```

seasonal_decompose

`seasonal_decompose` — operation `seasonal decompose`. Category: **statsmodels**.

```
p seasonal_decompose $x
```

seasonal_diff

`seasonal_diff` — operation `seasonal diff`. Category: **more extensions (2)**.

```
p seasonal_diff $x
```

sec

`sec` returns the secant ($1/\cos$) of an angle in radians.

```
p sec(0) # 1.0
```

sec_2fa_totp_window

`sec_2fa_totp_window` — operation `sec 2fa totp window`. Category: **os internals — schedulers, i/o, memory**.

```
p sec_2fa_totp_window $x
```

sec_account_lockout_step

`sec_account_lockout_step` — single-step update of `sec account lockout`. Category: **os internals — schedulers, i/o, memory**.

```
p sec_account_lockout_step $x
```

sec_acme_dns_challenge

`sec_acme_dns_challenge` — operation `sec acme dns challenge`. Category: **os internals — schedulers, i/o, memory**.

```
p sec_acme_dns_challenge $x
```

sec_aes_keyschedule_step

`sec_aes_keyschedule_step` — single-step update of `sec aes keyschedule`. Category: **os internals — schedulers, i/o, memory**.

```
p sec_aes_keyschedule_step $x
```

sec_aes_round_step

`sec_aes_round_step` — single-step update of `sec aes round`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_aes_round_step $x
```

sec_arc_chain_step

`sec_arc_chain_step` — single-step update of `sec arc chain`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_arc_chain_step $x
```

sec_argon2_block_step

`sec_argon2_block_step` — single-step update of `sec argon2 block`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_argon2_block_step $x
```

sec_argon2_memcost

`sec_argon2_memcost` — operation `sec argon2 memcost`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_argon2_memcost $x
```

sec_argon2_parallelism

`sec_argon2_parallelism` — operation `sec argon2 parallelism`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_argon2_parallelism $x
```

sec_argon2_state_advance

`sec_argon2_state_advance` — operation `sec argon2 state advance`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_argon2_state_advance $x
```

sec_argon2_timecost

`sec_argon2_timecost` — operation `sec argon2 timecost`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_argon2_timecost $x
```

sec_balloon_hash_step

`sec_balloon_hash_step` — single-step update of `sec balloon hash`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_balloon_hash_step $x
```

sec_basic_constraints_ca

`sec_basic_constraints_ca` — operation `sec basic constraints ca`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_basic_constraints_ca $x
```

sec_bcrypt_cost_factor

`sec_bcrypt_cost_factor` — factor of `sec bcrypt cost`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_bcrypt_cost_factor $x
```

sec_bcrypt_round_step

`sec_bcrypt_round_step` — single-step update of `sec bcrypt round`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_bcrypt_round_step $x
```

sec_blake3_chunk_step

`sec_blake3_chunk_step` — single-step update of `sec blake3 chunk`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_blake3_chunk_step $x
```

sec_blowfish_round_step

`sec_blowfish_round_step` — single-step update of `sec blowfish round`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_blowfish_round_step $x
```

sec_brute_force_attempts

`sec_brute_force_attempts` — operation `sec brute force attempts`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_brute_force_attempts $x
```

sec_cbc_mac_block_count

`sec_cbc_mac_block_count` — count of `sec cbc mac block`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_cbc_mac_block_count $x
```

sec_certificate_chain_depth

`sec_certificate_chain_depth` — operation `sec certificate chain depth`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_certificate_chain_depth $x
```

sec_certificate_transparency

`sec_certificate_transparency` — operation `sec certificate transparency`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_certificate_transparency $x
```

sec_chacha20_quarterround

`sec_chacha20_quarterround` — operation `sec chacha20 quarterround`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_chacha20_quarterround $x
```

sec_chachapoly_nonce_check

`sec_chachapoly_nonce_check` — operation `sec chachapoly nonce check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_chachapoly_nonce_check $x
```

sec_chosen_charset_strength

`sec_chosen_charset_strength` — operation `sec chosen charset strength`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_chosen_charset_strength $x
```

sec_cipher_suite_strength

`sec_cipher_suite_strength` — operation `sec cipher suite strength`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_cipher_suite_strength $x
```

sec_command_injection_score

`sec_command_injection_score` — score of `sec command injection`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_command_injection_score $x
```

sec_complexity_policy_score

`sec_complexity_policy_score` — score of `sec complexity policy`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_complexity_policy_score $x
```

sec_cors_origin_match

`sec_cors_origin_match` — operation `sec cors origin match`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_cors_origin_match $x
```

sec_credential_stuffing_score

`sec_credential_stuffing_score` — score of `sec credential stuffing`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_credential_stuffing_score $x
```

sec_crl_age_seconds

`sec_crl_age_seconds` — operation `sec crl age seconds`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_crl_age_seconds $x
```

sec_csp_directive_match

`sec_csp_directive_match` — operation `sec csp directive match`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_csp_directive_match $x
```

sec_csrf_token_match

sec_csrf_token_match — operation `sec csrf token match`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_csrf_token_match $x
```

sec_dane_tlsa_match

sec_dane_tlsa_match — operation `sec dane tlsa match`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_dane_tlsa_match $x
```

sec_des_round_step

sec_des_round_step — single-step update of `sec des round`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_des_round_step $x
```

sec_diameter_avp_step

sec_diameter_avp_step — single-step update of `sec diameter avp`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_diameter_avp_step $x
```

sec_diceware_word_index

sec_diceware_word_index — index of `sec diceware word`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_diceware_word_index $x
```

sec_dictionary_attack_check

sec_dictionary_attack_check — operation `sec dictionary attack check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_dictionary_attack_check $x
```

sec_dkim_signature_check

sec_dkim_signature_check — operation `sec dkim signature check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_dkim_signature_check $x
```

sec_dmarc_policy_check

sec_dmarc_policy_check — operation `sec dmarc policy check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_dmarc_policy_check $x
```

sec_dnssec_signature_check

sec_dnssec_signature_check — operation `sec dnssec signature check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_dnssec_signature_check $x
```

sec_ed25519_signature_step

`sec_ed25519_signature_step` — single-step update of `sec_ed25519_signature`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_ed25519_signature_step $x
```

sec_ed448_signature_step

`sec_ed448_signature_step` — single-step update of `sec_ed448_signature`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_ed448_signature_step $x
```

sec_fido2_assertion_check

`sec_fido2_assertion_check` — operation `sec_fido2_assertion_check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_fido2_assertion_check $x
```

sec_gcm_iv_unique_check

`sec_gcm_iv_unique_check` — operation `sec_gcm_iv_unique_check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_gcm_iv_unique_check $x
```

sec_haveibeenpwned_check

`sec_haveibeenpwned_check` — operation `sec_haveibeenpwned_check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_haveibeenpwned_check $x
```

sec_hotp_counter_step

`sec_hotp_counter_step` — single-step update of `sec_hotp_counter`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_hotp_counter_step $x
```

sec_hpkp_pin_match

`sec_hpkp_pin_match` — operation `sec_hpkp_pin_match`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_hpkp_pin_match $x
```

sec_html_escape_check

`sec_html_escape_check` — operation `sec_html_escape_check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_html_escape_check $x
```

sec_idle_timeout_step

`sec_idle_timeout_step` — single-step update of `sec_idle_timeout`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_idle_timeout_step $x
```

sec_idor_check

`sec_idor_check` — operation `sec idor check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_idor_check $x
```

sec_imap_starttls_check

`sec_imap_starttls_check` — operation `sec imap starttls check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_imap_starttls_check $x
```

sec_jwt_alg_safe

`sec_jwt_alg_safe` — operation `sec jwt alg safe`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_jwt_alg_safe $x
```

sec_jwt_kid_match

`sec_jwt_kid_match` — operation `sec jwt kid match`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_jwt_kid_match $x
```

sec_jwt_signature_verify

`sec_jwt_signature_verify` — verify with public key of `sec jwt signature`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_jwt_signature_verify $x
```

sec_keccak_round_step

`sec_keccak_round_step` — single-step update of `sec keccak round`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_keccak_round_step $x
```

sec_kerberos_pac_check

`sec_kerberos_pac_check` — operation `sec kerberos pac check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_kerberos_pac_check $x
```

sec_kerberos_pre_auth

`sec_kerberos_pre_auth` — operation `sec kerberos pre auth`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_kerberos_pre_auth $x
```

sec_kerberos_ticket_age

`sec_kerberos_ticket_age` — operation `sec kerberos ticket age`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_kerberos_ticket_age $x
```


sec_keystroke_timing_var

sec_keystroke_timing_var — variance of `sec keystroke timing`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_keystroke_timing_var $x
```

sec_ldap_bind_step

sec_ldap_bind_step — single-step update of `sec ldap bind`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_ldap_bind_step $x
```

sec_login_throttle_step

sec_login_throttle_step — single-step update of `sec login throttle`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_login_throttle_step $x
```

sec_oauth2_pkce_step

sec_oauth2_pkce_step — single-step update of `sec oauth2 pkce`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_oauth2_pkce_step $x
```

sec_oauth2_state_validate

sec_oauth2_state_validate — operation `sec oauth2 state validate`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_oauth2_state_validate $x
```

sec_oauth_nonce_check

sec_oauth_nonce_check — operation `sec oauth nonce check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_oauth_nonce_check $x
```

sec_oidc_id_token_age

sec_oidc_id_token_age — operation `sec oidc id token age`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_oidc_id_token_age $x
```

sec_p384_curve_step

sec_p384_curve_step — single-step update of `sec p384 curve`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_p384_curve_step $x
```

sec_passphrase_entropy

sec_passphrase_entropy — operation `sec passphrase entropy`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_passphrase_entropy $x
```

sec_password_history_check

`sec_password_history_check` — operation `sec password history check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_password_history_check $x
```

sec_password_strength_zxcvbn

`sec_password_strength_zxcvbn` — operation `sec password strength zxcvbn`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_password_strength_zxcvbn $x
```

sec_path_traversal_detect

`sec_path_traversal_detect` — operation `sec path traversal detect`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_path_traversal_detect $x
```

sec_pbkdf2_iter

`sec_pbkdf2_iter` — operation `sec pbkdf2 iter`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_pbkdf2_iter $x
```

sec_pinning_compare

`sec_pinning_compare` — operation `sec pinning compare`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_pinning_compare $x
```

sec_pki_path_validate

`sec_pki_path_validate` — operation `sec pki path validate`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_pki_path_validate $x
```

sec_pop3_security_step

`sec_pop3_security_step` — single-step update of `sec pop3 security`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_pop3_security_step $x
```

sec_radius_auth_step

`sec_radius_auth_step` — single-step update of `sec radius auth`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_radius_auth_step $x
```

sec_revocation_ocsp_check

`sec_revocation_ocsp_check` — operation `sec revocation ocsp check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_revocation_ocsp_check $x
```

sec_saml_assertion_age

sec_saml_assertion_age — operation `sec saml assertion age`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_saml_assertion_age $x
```

sec_san_match_count

sec_san_match_count — count of `sec san match`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_san_match_count $x
```

sec_scrypt_n_param

sec_scrypt_n_param — operation `sec scrypt n param`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_scrypt_n_param $x
```

sec_scrypt_p_param

sec_scrypt_p_param — operation `sec scrypt p param`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_scrypt_p_param $x
```

sec_scrypt_r_param

sec_scrypt_r_param — operation `sec scrypt r param`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_scrypt_r_param $x
```

sec_secp256k1_step

sec_secp256k1_step — single-step update of `sec secp256k1`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_secp256k1_step $x
```

sec_serpent_round_step

sec_serpent_round_step — single-step update of `sec serpent round`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_serpent_round_step $x
```

sec_session_lifetime

sec_session_lifetime — operation `sec session lifetime`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_session_lifetime $x
```

sec_sha3_padding_step

sec_sha3_padding_step — single-step update of `sec sha3 padding`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_sha3_padding_step $x
```

sec_smtp_ssl_check

sec_smtp_ssl_check — operation `sec smtp ssl check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_smtp_ssl_check $x
```

sec_spf_pass_check

sec_spf_pass_check — operation `sec spf pass check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_spf_pass_check $x
```

sec_sql_i_pattern_score

sec_sql_i_pattern_score — score of `sec sql_i pattern`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_sql_i_pattern_score $x
```

sec_ssl3_disabled_check

sec_ssl3_disabled_check — operation `sec ssl3 disabled check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_ssl3_disabled_check $x
```

sec_tls11_deprecation_check

sec_tls11_deprecation_check — operation `sec tls11 deprecation check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_tls11_deprecation_check $x
```

sec_tls12_handshake_step

sec_tls12_handshake_step — single-step update of `sec tls12 handshake`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_tls12_handshake_step $x
```

sec_tls13_handshake_step

sec_tls13_handshake_step — single-step update of `sec tls13 handshake`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_tls13_handshake_step $x
```

sec_tls_alert_severity

sec_tls_alert_severity — operation `sec tls alert severity`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_tls_alert_severity $x
```

sec_totp_drift_check

sec_totp_drift_check — operation `sec totp drift check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_totp_drift_check $x
```

sec_twofish_round_step

sec_twofish_round_step — single-step update of `sec twofish round`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_twofish_round_step $x
```

sec_url_safe_encode_check

`sec_url_safe_encode_check` — operation `sec url safe encode check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_url_safe_encode_check $x
```

sec_webauthn_attestation_check

`sec_webauthn_attestation_check` — operation `sec webauthn attestation check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_webauthn_attestation_check $x
```

sec_x25519_clamping_step

`sec_x25519_clamping_step` — single-step update of `sec x25519 clamping`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_x25519_clamping_step $x
```

sec_x509_subject_match

`sec_x509_subject_match` — operation `sec x509 subject match`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_x509_subject_match $x
```

sec_xkcd_passphrase_score

`sec_xkcd_passphrase_score` — score of `sec xkcd passphrase`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_xkcd_passphrase_score $x
```

sec_xss_filter_score

`sec_xss_filter_score` — score of `sec xss filter`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_xss_filter_score $x
```

sec_xxe_dtd_check

`sec_xxe_dtd_check` — operation `sec xxe dtd check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_xxe_dtd_check $x
```

sec_xxe_pattern_score

`sec_xxe_pattern_score` — score of `sec xxe pattern`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_xxe_pattern_score $x
```

sec_yescript_step

`sec_yescript_step` — single-step update of `sec yescript`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_yescript_step $x
```

sec_yubikey_otp_check

`sec_yubikey_otp_check` — operation `sec yubikey otp check`. Category: *os internals — schedulers, i/o, memory*.

```
p sec_yubikey_otp_check $x
```

secant

`secant` — operation `secant`. Alias for `secant_root`. Category: *more extensions (2)*.

```
p secant $x
```

secant_root

`secant_root` — operation `secant root`. Category: *more extensions (2)*.

```
p secant_root $x
```

second

`second` — operation `second`. Category: *algebraic match*.

```
p second $x
```

second_arg

`second_arg` — operation `second arg`. Category: *functional primitives*.

```
p second_arg $x
```

second_elem

`second_elem` — operation `second elem`. Category: *list helpers*.

```
p second_elem $x
```

second_order_concentration

`second_order_concentration` — operation `second order concentration`. Category: *chemistry*.

```
p second_order_concentration $x
```

second_order_half_life

`second_order_half_life` — operation `second order half life`. Category: *chemistry*.

```
p second_order_half_life $x
```

seconds_per_beat

`seconds_per_beat(bpm)` → `number` — `60 / bpm`.

```
p seconds_per_beat(120) # 0.5
```

seconds_to_days

`seconds_to_days` — convert from `seconds` to `days`. Category: *unit conversions*.

```
p seconds_to_days $value
```

seconds_to_hours

`seconds_to_hours` — convert from `seconds` to `hours`. Category: **unit conversions**.

```
p seconds_to_hours $value
```

seconds_to_minutes

`seconds_to_minutes` — convert from `seconds` to `minutes`. Category: **unit conversions**.

```
p seconds_to_minutes $value
```

secp256k1_y_recover

`secp256k1_y_recover` — operation `secp256k1 y recover`. Category: **more extensions (2)**.

```
p secp256k1_y_recover $x
```

sectarea

`sectarea` — operation `sectarea`. Alias for `sector_area`. Category: **geometry (extended)**.

```
p sectarea $x
```

sectional_curvature_two_plane

`sectional_curvature_two_plane` — operation `sectional curvature two plane`. Category: **electrochemistry, batteries, fuel cells**.

```
p sectional_curvature_two_plane $x
```

sector_area

`sector_area($radius, $theta)` — computes the area of a circular sector. Theta is the central angle in radians. Returns $0.5 * r^2 * \theta$.

```
p sector_area(10, 3.14159) # ~157 (half circle)
p sector_area(5, 1.57)    # ~19.6
```

seeded_rng

`seeded_rng` — operation `seeded rng`. Category: **random / sampling extras**.

```
p seeded_rng $x
```

seg_sum

`seg_sum` — sum of `seg`. Alias for `segment_tree_sum`. Category: **misc**.

```
p seg_sum @xs
```

segment_distance_point

`segment_distance_point` — operation `segment distance point`. Category: **excel/sheets + bond/loan financial**.

```
p segment_distance_point $x
```

segment_intersection

`segment_intersection` — operation `segment intersection`. Category: **excel/sheets + bond/loan financial**.

```
p segment_intersection $x
```

segment_length

`segment_length` — operation `segment length`. Category: **more extensions (2)**.

```
p segment_length $x
```

segment_tree_sum

`segment_tree_sum` — sum of `segment tree`. Category: **misc**.

```
p segment_tree_sum @xs
```

segments_parallel_q

`segments_parallel_q` — operation `segments parallel q`. Category: **more extensions (2)**.

```
p segments_parallel_q $x
```

segments_perpendicular_q

`segments_perpendicular_q` — operation `segments perpendicular q`. Category: **more extensions (2)**.

```
p segments_perpendicular_q $x
```

sei_resistance_growth

`sei_resistance_growth` — operation `sei resistance growth`. Category: **electrochemistry, batteries, fuel cells**.

```
p sei_resistance_growth $x
```

seiberg_witten_value

`seiberg_witten_value` — value of `seiberg witten`. Category: **econometrics**.

```
p seiberg_witten_value $x
```

seifert_form_2x2

`seifert_form_2x2` — operation `seifert form 2x2`. Category: **econometrics**.

```
p seifert_form_2x2 $x
```

seifert_genus_lower

`seifert_genus_lower` — operation `seifert genus lower`. Category: **econometrics**.

```
p seifert_genus_lower $x
```

seir_step

`seir_step` — single-step update of `seir`. Category: **biology / ecology**.

```
p seir_step $x
```


seird_step

`seird_step` — single-step update of `seird`. Category: *biology / ecology*.

```
p seird_step $x
```

seirs_step

`seirs_step` — single-step update of `seirs`. Category: *epidemiology / public health*.

```
p seirs_step $x
```

select_period_step

`select_period_step` — single-step update of `select period`. Category: *actuarial science*.

```
p select_period_step $x
```

select_ultimate

`select_ultimate` — operation `select ultimate`. Category: *actuarial science*.

```
p select_ultimate $x
```

selection_step

`selection_step` — single-step update of `selection`. Category: *biology / ecology*.

```
p selection_step $x
```

selector_to_xpath

`selector_to_xpath` — convert from `selector` to `xpath`. Category: *extras*.

```
p selector_to_xpath $value
```

selu

`selu` applies the Scaled ELU with fixed $\lambda=1.0507$, $\alpha=1.6733$ for self-normalizing networks.

```
p selu(1)      # 1.0507
p selu(-1)     # -1.1113
```

sem

`semaphore(N) → semaphore` (alias `sem`) — counting semaphore with `N` permits (must be ≥ 0). Use to bound concurrent access: `semaphore(K)` lets at most `K` threads pass `semaphore_acquire` at once.

```
mysync $s = semaphore(3)  # at most 3 concurrent workers
my @t
for (1:10) {
  push @t, async {
    semaphore_acquire($s)
    defer { semaphore_release($s) }
    do_work()  # at most 3 in flight at any moment
  }
}
await($_) for @t
```

sem_acquire

`semaphore_acquire($s)` → `undef` (alias `sem_acquire`) — block until a permit is available, then take one. Pair with `defer { semaphore_release($s) }`.

```
semaphore_acquire($s)
defer { semaphore_release($s) }
# bounded critical section
```

sem_limit

`semaphore_limit($s)` → `integer` (alias `sem_limit`) — initial permit count from the original `semaphore(N)`.

```
my $s = semaphore(8)
p semaphore_limit($s)    # 8 (constant for life of $s)
```

sem_permits

`semaphore_permits($s)` → `integer` (alias `sem_permits`) — currently available permits (race-free snapshot).

```
my $s = semaphore(5)
p semaphore_permits($s) # 5
semaphore_acquire($s)
p semaphore_permits($s) # 4
```

sem_release

`semaphore_release($s)` → `undef` (alias `sem_release`) — release one permit and wake one waiter.

```
semaphore_acquire($s)
defer { semaphore_release($s) }
work()
```

sem_try_acquire

`semaphore_try_acquire($s)` → `1|0` (alias `sem_try_acquire`) — non-blocking. Returns `1` if a permit was taken, `0` if none available.

```
if (semaphore_try_acquire($s)) {
  defer { semaphore_release($s) }
  burst_work()
} else {
  queue_for_later()
}
```

semctl

`semctl` — operation `semctl`. Category: `*sysv ipc*`.

```
p semctl $x
```

semget

`semget` — operation `semget`. Category: `*sysv ipc*`.

```
p semget $x
```

semi_major_axis

`semi_major_axis` — operation `semi major axis`. Category: `*more extensions*`.

```
p semi_major_axis $x
```

semop

`semop` — operation `semop`. Category: **sysv ipc**.

```
p semop $x
```

semver_compare

`semver_compare` — operation `semver compare`. Category: **validation / input checks**.

```
p semver_compare $x
```

semver_increment_major

`semver_increment_major` — operation `semver increment major`. Category: **validation / input checks**.

```
p semver_increment_major $x
```

semver_increment_minor

`semver_increment_minor` — operation `semver increment minor`. Category: **validation / input checks**.

```
p semver_increment_minor $x
```

semver_increment_patch

`semver_increment_patch` — operation `semver increment patch`. Category: **validation / input checks**.

```
p semver_increment_patch $x
```

semver_satisfies

`semver_satisfies` — operation `semver satisfies`. Category: **validation / input checks**.

```
p semver_satisfies $x
```

sentence_case

`sentence_case` — operation `sentence case`. Category: **string helpers**.

```
p sentence_case $x
```

seq

`seq` — operation `seq`. Category: **algebraic match**.

```
p seq $x
```

seq_fn

`seq_fn` — generate a numeric sequence from, to, by. Like R's `seq()`.

```
val @s = @{seq_fn(1, 10, 2)} # [1,3,5,7,9]  
var @r = @{seq_fn(5, 1, -1)} # [5,4,3,2,1]
```

seq_logo_information

seq_logo_information — sequence helper `logo information`. Category: **bioinformatics deep**.

```
p seq_logo_information $x
```

sequenceOf

sequenceOf — operation `sequenceOf`. Alias for `listOf`. Category: **kotlin collections**.

```
p sequenceOf $x
```

sequence_identity_pct

sequence_identity_pct(a, b) → `number` — percent of position-matched characters in aligned sequences.

```
p sequence_identity_pct("ACGT", "ACCT") # 75
```

sequence_shannon_info

sequence_shannon_info — operation `sequence shannon info`. Category: **bioinformatics deep**.

```
p sequence_shannon_info $x
```

sequence_similarity_pct

sequence_similarity_pct(a, b) → `number` — percent of positions that are identical or chemically similar (aliphatic/aromatic/polar/etc. groups).

sequent_left_intro

sequent_left_intro — operation `sequent left intro`. Category: **logic, proof, sat/smt, type theory**.

```
p sequent_left_intro $x
```

sequent_right_intro

sequent_right_intro — operation `sequent right intro`. Category: **logic, proof, sat/smt, type theory**.

```
p sequent_right_intro $x
```

sequential_eq_check

sequential_eq_check — operation `sequential eq check`. Category: **climate, fluids, atmospheric**.

```
p sequential_eq_check $x
```

servo_position_velocity

servo_position_velocity — operation `servo position velocity`. Category: **robotics & control**.

```
p servo_position_velocity $x
```

servo_torque_step

servo_torque_step — single-step update of `servo torque`. Category: **robotics & control**.

```
p servo_torque_step $x
```

setOf

setOf — operation `setOf`. Category: *kotlin collections*.

```
p setOf $x
```

set_at

set_at — set-theoretic op `at`. Alias for `replace_at_index`. Category: *misc*.

```
p set_at $x
```

set_cover_greedy_step

set_cover_greedy_step — set-theoretic op `cover greedy step`. Category: *combinatorial optimization, scheduling*.

```
p set_cover_greedy_step $x
```

set_cover_lp_round

set_cover_lp_round — set-theoretic op `cover lp round`. Category: *combinatorial optimization, scheduling*.

```
p set_cover_lp_round $x
```

set_subname

set_subname — set-theoretic op `subname`. Category: *list / aggregate*.

```
p set_subname $x
```

setex

setex — operation `setex`. Category: *redis-flavour primitives*.

```
p setex $x
```

setgrent

setgrent — operation `setgrent`. Category: *posix metadata*.

```
p setgrent $x
```

sethostent

sethostent — operation `sethostent`. Category: *posix metadata*.

```
p sethostent $x
```

setnetent

setnetent — operation `setnetent`. Category: *posix metadata*.

```
p setnetent $x
```

setnx

setnx — operation `setnx`. Category: *redis-flavour primitives*.

```
p setnx $x
```

setprotoent

`setprotoent` — operation `setprotoent`. Category: *posix metadata*.

```
p setprotoent $x
```

setpwent

`setpwent` — operation `setpwent`. Category: *posix metadata*.

```
p setpwent $x
```

setrange

`setrange` — operation `setrange`. Category: *redis-flavour primitives*.

```
p setrange $x
```

setservernt

`setservernt` — operation `setservernt`. Category: *posix metadata*.

```
p setservernt $x
```

seven_smooth_q

`seven_smooth_q` — operation `seven smooth q`. Category: *misc*.

```
p seven_smooth_q $x
```

sfork

`sfork` — operation `sfork`. Alias for `stress_fork`. Category: *stress / telemetry extensions*.

```
p sfork $x
```

sgemm

`sgemm` — operation `sgemm`. Category: *blas / lapack*.

```
p sgemm $x
```

sgemv

`sgemv` — operation `sgemv`. Category: *blas / lapack*.

```
p sgemv $x
```

sgnum

`sgnum` — operation `sgnum`. Alias for `signum_of`. Category: *extended stdlib*.

```
p sgnum $x
```

sgp4_propagate_step

`sgp4_propagate_step` — single-step update of `sgp4 propagate`. Category: *astronomy / astrometry*.

```
p sgp4_propagate_step $x
```

sh_add

`sh_add(SH, FEATURE, WEIGHT=1)` — fold a token into the sketch.

sh_digest

`sh_digest(SH)` → 64-bit integer fingerprint.

sh_similarity

`sh_similarity(A, B)` → approximate cosine similarity $\in [0, 1]$.

shake_map_pga

`shake_map_pga` — operation `shake map pga`. Category: *geology, seismology, mineralogy*.

```
p shake_map_pga $x
```

shanks_transform

`shanks_transform` — transform of `shanks`. Category: *more extensions*.

```
p shanks_transform $x
```

shannon_fano_elias_code

`shannon_fano_elias_code` — operation `shannon fano elias code`. Category: *electrochemistry, batteries, fuel cells*.

```
p shannon_fano_elias_code $x
```

shape_operator_eig

`shape_operator_eig` — operation `shape operator eig`. Category: *electrochemistry, batteries, fuel cells*.

```
p shape_operator_eig $x
```

shapiro

`shapiro_test` (alias `shapiro`) — Shapiro-Wilk normality test. Returns W statistic (close to 1 = normal). Like R's `shapiro.test()`.

```
p shapiro([rnorm(100)]) # W ≈ 0.99 for normal data
p shapiro([1,1,1,2,10]) # W < 0.9 for non-normal
```

shapiro_delay

`shapiro_delay` — operation `shapiro delay`. Category: *cosmology / gr / flrw*.

```
p shapiro_delay $x
```

shapiro_delay_step

`shapiro_delay_step` — single-step update of `shapiro delay`. Category: *electrochemistry, batteries, fuel cells*.

```
p shapiro_delay_step $x
```

shapiro_wilk

`shapiro_wilk` — operation `shapiro_wilk`. Category: *r / scipy distributions and tests*.

```
p shapiro_wilk $x
```

shapley_value_two_step

`shapley_value_two_step` — single-step update of `shapley_value_two`. Category: *climate, fluids, atmospheric*.

```
p shapley_value_two_step $x
```

sharpe

`sharpe_ratio(@returns, $risk_free_rate)` (alias `sharpe`) — computes the Sharpe ratio measuring risk-adjusted returns. Higher is better. Returns (mean_return - rf) / stddev.

```
val @returns = (0.05, -0.02, 0.08, 0.03)
p sharpe(@returns, 0.02) # risk-adjusted performance
```

sharpe_annualized

`sharpe_annualized` — operation `sharpe_annualized`. Category: *financial pricing models*.

```
p sharpe_annualized $x
```

sharpen_kernel

`sharpen_kernel` — convolution kernel of `sharpen`. Category: *more extensions*.

```
p sharpen_kernel $x
```

shear_tensor_step

`shear_tensor_step` — single-step update of `shear_tensor`. Category: *electrochemistry, batteries, fuel cells*.

```
p shear_tensor_step $x
```

shepherd_voltage_step

`shepherd_voltage_step` — single-step update of `shepherd_voltage`. Category: *electrochemistry, batteries, fuel cells*.

```
p shepherd_voltage_step $x
```

sherwood_number_step

`sherwood_number_step` — single-step update of `sherwood_number`. Category: *climate, fluids, atmospheric*.

```
p sherwood_number_step $x
```

shi_tomasi_corners

`shi_tomasi_corners` — operation `shi_tomasi_corners`. Category: *pill/opencv image processing*.

```
p shi_tomasi_corners $x
```


shift_left

`shift_left` — operation `shift left`. Category: **bit ops**.

```
p shift_left $x
```

shift_right

`shift_right` — operation `shift right`. Category: **bit ops**.

```
p shift_right $x
```

shift_series

`shift_series(series, n=1) → array` — shifts elements by `n` positions; negative shifts left, positive shifts right.

```
p shift_series([1,2,3,4,5], 2)
```

shl

`shl` — operation `shl`. Alias for `shift_left`. Category: **bit ops**.

```
p shl $x
```

shmctl

`shmctl` — operation `shmctl`. Category: **sysv ipc**.

```
p shmctl $x
```

shmget

`shmget` — operation `shmget`. Category: **sysv ipc**.

```
p shmget $x
```

shmread

`shmread` — operation `shmread`. Category: **sysv ipc**.

```
p shmread $x
```

shmwrite

`shmwrite` — operation `shmwrite`. Category: **sysv ipc**.

```
p shmwrite $x
```

shoelace_area

`shoelace_area` — operation `shoelace area`. Category: **geometry / topology**.

```
p shoelace_area $x
```

shooting_pct

`shooting_pct` — percentage of `shooting`. Category: **astronomy / astrometry**.

```
p shooting_pct $x
```

shor_period

`shor_period` — operation `shor period`. Category: **quantum**.

```
p shor_period $x
```

shor_period_prob

`shor_period_prob` — operation `shor period prob`. Category: **cryptanalysis & number theory deep**.

```
p shor_period_prob $x
```

short_id

`short_id` — operation `short id`. Category: **id helpers**.

```
p short_id $x
```

shorten

`shorten` — operation `shorten`. Alias for `truncate_at`. Category: **trivial string ops**.

```
p shorten $x
```

shortest

`shortest` — operation `shortest`. Category: **collection**.

```
p shortest $x
```

shortest_path_bfs

`shortest_path_bfs` — operation `shortest path bfs`. Category: **extended stdlib**.

```
p shortest_path_bfs $x
```

shr

`shr` — operation `shr`. Alias for `shift_right`. Category: **bit ops**.

```
p shr $x
```

shuf

`shuf` — operation `shuf`. Alias for `shuffle_arr`. Category: **extended stdlib**.

```
p shuf $x
```

shuffle_arr

`shuffle_arr` — operation `shuffle arr`. Category: **extended stdlib**.

```
p shuffle_arr $x
```

shuffle_chars

`shuffle_chars` — operation `shuffle chars`. Category: **file stat / path**.

```
p shuffle_chars $x
```

shuffle_resample

`shuffle_resample(xs, seed=0)` → `array` — in-place Fisher-Yates shuffle returning a new permutation.

shuffled

`shuffled` — operation `shuffled`. Category: **functional / iterator**.

```
p shuffled $x
```

shunting_yard_step

`shunting_yard_step` — single-step update of `shunting_yard`. Category: **compilers / parsing**.

```
p shunting_yard_step $x
```

shutter_speed_reciprocal

`shutter_speed_reciprocal(focal_mm, crop=1)` → `number` — $1/(\text{focal} \cdot \text{crop})$. Heuristic minimum hand-held shutter.

si

`si(x)` → `number` — sine integral $Si(x) = \int_0^x \sin(t)/t \, dt$. Used in signal-processing windowing.

```
p si(3.14159) # ~1.852
```

sidereal_day_period

`sidereal_day_period` — sidereal-time computation `day_period`. Category: **misc**.

```
p sidereal_day_period $x
```

sidereal_time_greenwich

`sidereal_time_greenwich(unix)` → `number` — Greenwich Mean Sidereal Time in hours, [0, 24).

```
p sidereal_time_greenwich(time())
```

sidereal_time_local

`sidereal_time_local(unix, longitude_deg)` → `number` — local sidereal time = GMST + lon/15.

```
p sidereal_time_local(time(), -122.4)
```

siera

`siera` — operation `siera`. Category: **astronomy / astrometry**.

```
p siera $x
```

sierp

`sierp` — operation `sierp`. Alias for `sierpinski`. Category: **algorithms / puzzles**.

```
p sierp $x
```

sierpinski

`sierpinski` — operation `sierpinski`. Category: *algorithms / puzzles*.

```
p sierpinski $x
```

sieve

`sieve` — operation `sieve`. Alias for `primes_up_to`. Category: *number theory / primes*.

```
p sieve $x
```

sift_keypoints

`sift_keypoints` — operation `sift keypoints`. Category: *pil/opencv image processing*.

```
p sift_keypoints $x
```

sigma8_default

`sigma8_default` — operation `sigma8 default`. Category: *cosmology / gr / flrw*.

```
p sigma8_default $x
```

sigma_crit

`sigma_crit` — operation `sigma crit`. Category: *cosmology / gr / flrw*.

```
p sigma_crit $x
```

sigma_divisors

`sigma_divisors` — operation `sigma divisors`. Category: *math / number theory extras*.

```
p sigma_divisors $x
```

sigma_sb

`sigma_sb` — operation `sigma sb`. Alias for `stefan_boltzmann_constant`. Category: *physics constants*.

```
p sigma_sb $x
```

sigmoid

`sigmoid(x)` $\rightarrow$ number $\frac{1}{1 + e^{-x}}$.

sigmoid_grad

`sigmoid_grad` — operation `sigmoid grad`. Category: *ml extensions*.

```
p sigmoid_grad $x
```

sign

`sign` — operation `sign`. Category: *trivial numeric / predicate builtins*.

```
p sign $x
```

signal_name

`signal_name` — operation `signal name`. Category: **more process / system**.

```
p signal_name $x
```

signum

`signum` — operation `signum`. Category: **math functions**.

```
p signum $x
```

signum_of

`signum_of` — operation `signum of`. Category: **extended stdlib**.

```
p signum_of $x
```

silicate_classify

`silicate_classify` — operation `silicate classify`. Category: **geology, seismology, mineralogy**.

```
p silicate_classify $x
```

silu

`silu` (alias `swish`) applies SiLU/Swish: $x \cdot \text{sigmoid}(x)$. Smooth approximation to ReLU.

```
p silu(1)      # 0.7311
p swish(-2)    # -0.2384
```

silver

`silver_ratio` (alias `silver`) — $\delta S = 1 + \sqrt{2} \approx 2.414$. Related to Pell numbers.

```
p silver      # 2.414213562373095
```

sim

`sim` — operation `sim`. Alias for `similarity`. Category: **extended stdlib**.

```
p sim $x
```

simhash

`simhash()` — Charikar 64-bit SimHash sketch. Each feature flips +/- on 64 signed accumulators; final `digest()` is the per-bit sign. Cosine similarity $\approx (64 - \text{hamming}(a, b)) / 64$. Stateful upgrade to the deleted `db_simhash_bit` spec primitive.

```
val $sa = simhash(); sh_add($sa, _) for @tokens_a
val $sb = simhash(); sh_add($sb, _) for @tokens_b
p sh_similarity($sa, $sb)      # 0..1
```

similarity

`similarity` — operation `similarity`. Category: **extended stdlib**.

```
p similarity $x
```

similarity_function_phi

`similarity_function_phi` — operation `similarity function phi`. Category: **climate, fluids, atmospheric**.

```
p similarity_function_phi $x
```

simon_round

`simon_round` — operation `simon round`. Category: **more extensions**.

```
p simon_round $x
```

simple_interest

`simple_interest` — operation `simple interest`. Category: **physics formulas**.

```
p simple_interest $x
```

simple_interest_compute

`simple_interest_compute` — operation `simple interest compute`. Category: **misc**.

```
p simple_interest_compute $x
```

simple_returns

`simple_returns` — operation `simple returns`. Category: **test runner**.

```
p simple_returns $x
```

simplex_2d

`simplex_2d(x, y) → number` — 2D simplex noise (Perlin's improvement). Better isotropy than classic Perlin.

simplex_volume_3d

`simplex_volume_3d` — simplex noise `volume 3d`. Category: **geometry / topology**.

```
p simplex_volume_3d $x
```

simplex_volume_n

`simplex_volume_n` — simplex noise `volume n`. Category: **econometrics**.

```
p simplex_volume_n $x
```

simplicial_volume

`simplicial_volume` — operation `simplicial volume`. Category: **econometrics**.

```
p simplicial_volume $x
```

simply_typed_check

`simply_typed_check` — operation `simply typed check`. Category: **logic, proof, sat/smt, type theory**.

```
p simply_typed_check $x
```

simps

`simpson` (alias `simps`) integrates evenly-spaced samples using Simpson's rule. More accurate than `trapz` for smooth functions.

```
val @y = map { sin(_ * 0.01) } 0:314
p simps(@y, 0.01) # ≈ 2.0
```

simpson_integrate

`simpson_integrate(ys, dx=1) → number` — composite Simpson's rule. Uses Simpson's 1/3 over even intervals; falls back to Simpson's 3/8 on the trailing 3 intervals when `n-1` is odd, so the rule is exact for any `n ≥ 3`. Falls back to trapezoidal for `n < 3`.

```
p simpson_integrate([0,1,4,9,16,25,36,49,64,81,100], 1)
```

simpson_rule

`simpson_rule` — operation `simpson rule`. Category: `*numpy + scipy.special*`.

```
p simpson_rule $x
```

simrank

`simrank` — operation `simrank`. Category: `*networkx graph algorithms*`.

```
p simrank $x
```

simulated_annealing_step

`simulated_annealing_step` — single-step update of `simulated annealing`. Category: `*combinatorial optimization, scheduling*`.

```
p simulated_annealing_step $x
```

sinc

`sinc` returns the unnormalized sinc function: $\sin(x)/x$, with $\text{sinc}(0)=1$.

```
p sinc(0) # 1.0
p sinc(3.14159) # ≈ 0
```

sine_integral_si

`sine_integral_si` — operation `sine integral si`. Category: `*special functions extra*`.

```
p sine_integral_si $x
```

single_quote

`single_quote` — operation `single quote`. Category: `*string quote / escape*`.

```
p single_quote $x
```

single_slit_min

`single_slit_min` — minimum of `single slit`. Category: `*em / optics / relativity*`.

```
p single_slit_min $x
```

single_transferable_vote

`single_transferable_vote` — operation `single transferable vote`. Category: **climate, fluids, atmospheric**.

```
p single_transferable_vote $x
```

singularity_check_2link

`singularity_check_2link` — operation `singularity check 2link`. Category: **robotics & control**.

```
p singularity_check_2link $x
```

singularize

`singularize` — operation `singularize`. Alias for `singularize_simple`. Category: **misc**.

```
p singularize $x
```

singularize_simple

`singularize_simple` — operation `singularize simple`. Category: **misc**.

```
p singularize_simple $x
```

sinh

`sinh` — operation `sinh`. Category: **trig / math**.

```
p sinh $x
```

sinhc

`sinhc` — operation `sinhc`. Category: **special functions extra**.

```
p sinhc $x
```

sinkhorn_iteration_step

`sinkhorn_iteration_step` — single-step update of `sinkhorn iteration`. Category: **electrochemistry, batteries, fuel cells**.

```
p sinkhorn_iteration_step $x
```

sinter

`sinter` — operation `sinter`. Category: **redis-flavour primitives**.

```
p sinter $x
```

sinusoidal_project

`sinusoidal_project` — operation `sinusoidal project`. Category: **more extensions**.

```
p sinusoidal_project $x
```

siphash24

`siphash24` — operation `siphash24`. Category: **cryptography deep**.


```
p siphash24 $x
```

siphash_keyed

`siphash_keyed` computes SipHash-2-4 with a custom 128-bit key (two 64-bit integers). The key should be random per application to prevent collision attacks.

```
p siphash_keyed("data", 0x123456789, 0xabcdef012) # keyed hash
```

sirs_step

`sirs_step` — single-step update of `sirs`. Category: *epidemiology / public health*.

```
p sirs_step $x
```

sis_step

`sis_step` — single-step update of `sis`. Category: *biology / ecology*.

```
p sis_step $x
```

sismember

`sismember` — operation `sismember`. Category: *redis-flavour primitives*.

```
p sismember $x
```

size

`size` returns the byte size of a file on disk — equivalent to Perl's `-s FILE` file test. With no arguments, it operates on `_`; with one argument, it stats the given path; with multiple arguments (or a flattened list), it returns an array of sizes. Paths that can't be stat'd return `undef`. This is a stryke extension that makes pipelines over filenames concise.

```
p size "Cargo.toml" # 2013
f |> map +{ _ => size } |> tj |> p # [{name => bytes}, ...]
f |> filter { size > 1024 } |> e p # files larger than 1 KiB
```

sjson

`sjson` — operation `sjson`. Alias for `stress_json`. Category: *stress / telemetry extensions*.

```
p sjson $x
```

sk_adjusted_rand

`sk_adjusted_rand` — operation `sk adjusted rand`. Category: *sklearn*.

```
p sk_adjusted_rand $x
```

sk_agglomerative_fit

`sk_agglomerative_fit` — operation `sk agglomerative fit`. Category: *sklearn*.

```
p sk_agglomerative_fit $x
```

sk_bayes_search

sk_bayes_search — operation sk bayes search. Category: *sklearn*.

```
p sk_bayes_search $x
```

sk_calinski_harabasz

sk_calinski_harabasz — operation sk calinski harabasz. Category: *sklearn*.

```
p sk_calinski_harabasz $x
```

sk_count_vectorize

sk_count_vectorize — operation sk count vectorize. Category: *sklearn*.

```
p sk_count_vectorize $x
```

sk_cross_val_score

sk_cross_val_score — score of sk cross val. Category: *sklearn*.

```
p sk_cross_val_score $x
```

sk_davies_bouldin

sk_davies_bouldin — operation sk davies bouldin. Category: *sklearn*.

```
p sk_davies_bouldin $x
```

sk_dbscan_fit

sk_dbscan_fit — operation sk dbscan fit. Category: *sklearn*.

```
p sk_dbscan_fit $x
```

sk_decision_tree_split

sk_decision_tree_split — operation sk decision tree split. Category: *sklearn*.

```
p sk_decision_tree_split $x
```

sk_doc2vec_train

sk_doc2vec_train — operation sk doc2vec train. Category: *sklearn*.

```
p sk_doc2vec_train $x
```

sk_gbt_fit

sk_gbt_fit — operation sk gbt fit. Category: *sklearn*.

```
p sk_gbt_fit $x
```

sk_grid_search

sk_grid_search — operation sk grid search. Category: *sklearn*.

```
p sk_grid_search $x
```

sk_isolation_forest_fit

sk_isolation_forest_fit — operation `sk isolation forest fit`. Category: *sklearn*.

```
p sk_isolation_forest_fit $x
```

sk_kfold_split

sk_kfold_split — operation `sk kfold split`. Category: *sklearn*.

```
p sk_kfold_split $x
```

sk_kmeans_fit

sk_kmeans_fit — operation `sk kmeans fit`. Category: *sklearn*.

```
p sk_kmeans_fit $x
```

sk_knn_predict

sk_knn_predict — operation `sk knn predict`. Category: *sklearn*.

```
p sk_knn_predict $x
```

sk_label_encode

sk_label_encode — encode of `sk label`. Category: *sklearn*.

```
p sk_label_encode $x
```

sk_lda_topic

sk_lda_topic — operation `sk lda topic`. Category: *sklearn*.

```
p sk_lda_topic $x
```

sk_lightgbm_fit

sk_lightgbm_fit — operation `sk lightgbm fit`. Category: *sklearn*.

```
p sk_lightgbm_fit $x
```

sk_lof_fit

sk_lof_fit — operation `sk lof fit`. Category: *sklearn*.

```
p sk_lof_fit $x
```

sk_logistic_fit

sk_logistic_fit — operation `sk logistic fit`. Category: *sklearn*.

```
p sk_logistic_fit $x
```

sk_logistic_predict

sk_logistic_predict — operation `sk logistic predict`. Category: *sklearn*.

```
p sk_logistic_predict $x
```

sk_min_max_scaler

sk_min_max_scaler — operation `sk min max scaler`. Category: *sklearn*.

```
p sk_min_max_scaler $x
```

sk_mutual_info

sk_mutual_info — operation `sk mutual info`. Category: *sklearn*.

```
p sk_mutual_info $x
```

sk_naive_bayes_predict

sk_naive_bayes_predict — operation `sk naive bayes predict`. Category: *sklearn*.

```
p sk_naive_bayes_predict $x
```

sk_nmf_topic

sk_nmf_topic — operation `sk nmf topic`. Category: *sklearn*.

```
p sk_nmf_topic $x
```

sk_one_hot

sk_one_hot — operation `sk one hot`. Category: *sklearn*.

```
p sk_one_hot $x
```

sk_ordinal_encode

sk_ordinal_encode — encode of `sk ordinal`. Category: *sklearn*.

```
p sk_ordinal_encode $x
```

sk_pca_fit

sk_pca_fit — operation `sk pca fit`. Category: *sklearn*.

```
p sk_pca_fit $x
```

sk_pipeline_fit

sk_pipeline_fit — operation `sk pipeline fit`. Category: *sklearn*.

```
p sk_pipeline_fit $x
```

sk_power_transform

sk_power_transform — transform of `sk power`. Category: *sklearn*.

```
p sk_power_transform $x
```

sk_quantile_transform

sk_quantile_transform — transform of `sk quantile`. Category: *sklearn*.

```
p sk_quantile_transform $x
```

sk_random_forest_fit

sk_random_forest_fit — operation `sk random forest fit`. Category: *sklearn*.

```
p sk_random_forest_fit $x
```

sk_random_search

sk_random_search — operation `sk random search`. Category: *sklearn*.

```
p sk_random_search $x
```

sk_robust_scaler

sk_robust_scaler — operation `sk robust scaler`. Category: *sklearn*.

```
p sk_robust_scaler $x
```

sk_silhouette

sk_silhouette — operation `sk silhouette`. Category: *sklearn*.

```
p sk_silhouette $x
```

sk_standard_scaler

sk_standard_scaler — operation `sk standard scaler`. Category: *sklearn*.

```
p sk_standard_scaler $x
```

sk_stratified_kfold

sk_stratified_kfold — operation `sk stratified kfold`. Category: *sklearn*.

```
p sk_stratified_kfold $x
```

sk_svm_fit

sk_svm_fit — operation `sk svm fit`. Category: *sklearn*.

```
p sk_svm_fit $x
```

sk_tfidf

sk_tfidf — operation `sk tfidf`. Category: *sklearn*.

```
p sk_tfidf $x
```

sk_tsne_fit

sk_tsne_fit — operation `sk tsne fit`. Category: *sklearn*.

```
p sk_tsne_fit $x
```

sk_umap_fit

sk_umap_fit — operation `sk umap fit`. Category: *sklearn*.

```
p sk_umap_fit $x
```

sk_word2vec_train

sk_word2vec_train — operation `sk word2vec train`. Category: **sklearn**.

```
p sk_word2vec_train $x
```

sk_xgb_fit

sk_xgb_fit — operation `sk xgb fit`. Category: **sklearn**.

```
p sk_xgb_fit $x
```

skew

skew — operation `skew`. Alias for `skewness`. Category: **math / numeric extras**.

```
p skew $x
```

skewness

skewness — operation `skewness`. Category: **math / numeric extras**.

```
p skewness $x
```

skill_score

skill_score — score of `skill`. Category: **excel/sheets + bond/loan financial**.

```
p skill_score $x
```

skin_depth

skin_depth — operation `skin depth`. Category: **misc**.

```
p skin_depth $x
```

skip_assert

skip_assert — operation `skip assert`. Alias for `test_skip`. Category: **testing framework**.

```
p skip_assert $x
```

skip_test

skip_test — statistical test of `skip`. Alias for `test_skip`. Category: **testing framework**.

```
p skip_test $x
```

skip_while

skip_while { COND } LIST — skip leading elements while the predicate is true (alias for `drop_while`).

Behavior is identical to `drop_while`: once the predicate returns false, that element and all subsequent elements are emitted. The `skip_while` name is provided for users coming from Rust or Kotlin where this is the conventional name. Both compile to the same streaming operation internally.

```
1:10 |> skip_while { _ < 5 } |> e p # 5 6 7 8 9 10
@sorted |> skip_while { _ le "m" } |> e p
```

sla

`sla` — operation `sla`. Alias for `slice_arr`. Category: *python/ruby stdlib*.

```
p sla $x
```

slca

`slca` — operation `slca`. Alias for `slice_after`. Category: *additional missing stdlib functions*.

```
p slca $x
```

slcb

`slcb` — operation `slcb`. Alias for `slice_before`. Category: *additional missing stdlib functions*.

```
p slcb $x
```

slcw

`slcw` — operation `slcw`. Alias for `slice_when`. Category: *additional missing stdlib functions*.

```
p slcw $x
```

slic_superpixels

`slic_superpixels` — operation `slic superpixels`. Category: *pil/opencv image processing*.

```
p slic_superpixels $x
```

slice_after

`slice_after` — operation `slice after`. Category: *additional missing stdlib functions*.

```
p slice_after $x
```

slice_arr

`slice_arr` — operation `slice arr`. Category: *python/ruby stdlib*.

```
p slice_arr $x
```

slice_before

`slice_before` — operation `slice before`. Category: *additional missing stdlib functions*.

```
p slice_before $x
```

slice_when

`slice_when` — operation `slice when`. Category: *additional missing stdlib functions*.

```
p slice_when $x
```

sliced_wasserstein

`sliced_wasserstein` — operation `sliced wasserstein`. Category: *electrochemistry, batteries, fuel cells*.

```
p sliced_wasserstein $x
```

sliding_average

`sliding_average` — operation `sliding average`. Category: **iterator + string-distance extras**.

```
p sliding_average $x
```

sliding_dot_product

`sliding_dot_product(signal, kernel)` → `array` — valid-mode 1D correlation (no padding).

sliding_max

`sliding_max` — maximum of `sliding`. Category: **iterator + string-distance extras**.

```
p sliding_max $x
```

sliding_min

`sliding_min` — minimum of `sliding`. Category: **iterator + string-distance extras**.

```
p sliding_min $x
```

sliding_pairs

`sliding_pairs` — operation `sliding pairs`. Category: **collection more**.

```
p sliding_pairs $x
```

sliding_sum

`sliding_sum` — sum of `sliding`. Category: **iterator + string-distance extras**.

```
p sliding_sum @xs
```

sliding_window

`sliding_window` — operation `sliding window`. Category: **functional / iterator**.

```
p sliding_window $x
```

slope

`slope` — operation `slope`. Category: **geometry / physics**.

```
p slope $x
```

slug

`slug` — operation `slug`. Alias for `slugify`. Category: **extended stdlib**.

```
p slug $x
```

slugify

`slugify` — operation `slugify`. Category: **extended stdlib**.

```
p slugify $x
```


slutsky_decompose

`slutsky_decompose` — operation `slutsky_decompose`. Category: *economics + game theory*.

```
p slutsky_decompose $x
```

sma

`sma(prices, period=10) → array` — simple moving average over a price series. Output length is `len(prices) - period + 1`; each value is the arithmetic mean of the trailing `period` window.

```
p sma([10,11,12,13,14], 3) # [11.0, 12.0, 13.0]
```

smap

`smap` — operation `smap`. Alias for `starmap`. Category: *python/ruby stdlib*.

```
p smap $x
```

smembers

`smembers` — operation `smembers`. Category: *redis-flavour primitives*.

```
p smembers $x
```

smith_normal_2x2_step

`smith_normal_2x2_step` — single-step update of `smith normal 2x2`. Category: *cryptanalysis & number theory deep*.

```
p smith_normal_2x2_step $x
```

smith_predictor_step

`smith_predictor_step` — single-step update of `smith predictor`. Category: *robotics & control*.

```
p smith_predictor_step $x
```

smith_q

`smith_q` — operation `smith q`. Category: *misc*.

```
p smith_q $x
```

smith_waterman_score

`smith_waterman_score` — score of `smith waterman`. Category: *bioinformatics deep*.

```
p smith_waterman_score $x
```

s mmap

`s mmap` — operation `s mmap`. Alias for `stress_mmap`. Category: *stress / telemetry extensions*.

```
p s mmap $x
```

smoothstep

`smoothstep($edge0, $edge1, $x)` — smooth Hermite interpolation. Returns 0 if $x \leq \text{edge0}$, 1 if $x \geq \text{edge1}$, otherwise smooth S-curve. Great for animations.

```
p smoothstep(0, 1, 0.5)    # 0.5 (but with smooth acceleration/deceleration)
p smoothstep(10, 20, 15)   # 0.5
```

smst

`smst` — operation `smst`. Alias for `smoothstep`. Category: **extended stdlib**.

```
p smst $x
```

smt_qf_lia_solve_step

`smt_qf_lia_solve_step` — single-step update of `smt qf lia solve`. Category: **logic, proof, sat/smt, type theory**.

```
p smt_qf_lia_solve_step $x
```

smt_qf_uf_combine

`smt_qf_uf_combine` — operation `smt qf uf combine`. Category: **logic, proof, sat/smt, type theory**.

```
p smt_qf_uf_combine $x
```

snake_to_camel

`snake_to_camel` — convert from `snake` to `camel`. Category: **string processing extras**.

```
p snake_to_camel $value
```

snappy_encode

`snappy_encode` — encode of `snappy`. Category: **b81-misc-utility**.

```
p snappy_encode $x
```

snd

`snd` — operation `snd`. Alias for `second`. Category: **algebraic match**.

```
p snd $x
```

snell

`snells_law($n1, $n2, $theta1)` (alias `snell`) — refraction angle using Snell's law. Angles in degrees. Returns undef for total internal reflection.

```
p snell(1, 1.5, 30)        # ~19.5° (air to glass)
p snell(1.5, 1, 45)        # undef (total internal reflection)
```

snell_theta2

`snell_theta2` — operation `snell theta2`. Category: **em / optics / relativity**.

```
p snell_theta2 $x
```

snowball_stem_english

snowball_stem_english — operation `snowball stem english`. Category: *computational linguistics*.

```
p snowball_stem_english $x
```

snowball_stem_french

snowball_stem_french — operation `snowball stem french`. Category: *computational linguistics*.

```
p snowball_stem_french $x
```

sobel_diagonal_kernel

sobel_diagonal_kernel — convolution kernel of `sobel diagonal`. Category: *more extensions*.

```
p sobel_diagonal_kernel $x
```

sobel_kernel_value

sobel_kernel_value — value of `sobel kernel`. Category: *electrochemistry, batteries, fuel cells*.

```
p sobel_kernel_value $x
```

sobel_magnitude

sobel_magnitude(image) → matrix — alias for `gradient_magnitude_2d`.

sobel_x_kernel

sobel_x_kernel() → matrix — 3×3 Sobel horizontal-gradient kernel.

sobel_y_kernel

sobel_y_kernel() → matrix — 3×3 Sobel vertical-gradient kernel.

soc_estimate_coulomb

soc_estimate_coulomb — operation `soc estimate coulomb`. Category: *electrochemistry, batteries, fuel cells*.

```
p soc_estimate_coulomb $x
```

social_welfare_nash

social_welfare_nash — operation `social welfare nash`. Category: *economics + game theory*.

```
p social_welfare_nash $x
```

social_welfare_rawls

social_welfare_rawls — operation `social welfare rawls`. Category: *economics + game theory*.

```
p social_welfare_rawls $x
```

social_welfare_sum

`social_welfare_sum` — sum of `social_welfare`. Category: *climate, fluids, atmospheric*.

```
p social_welfare_sum @xs
```

social_welfare_utilitarian

`social_welfare_utilitarian` — operation `social_welfare_utilitarian`. Category: *economics + game theory*.

```
p social_welfare_utilitarian $x
```

socketpair

`socketpair` — operation `socketpair`. Category: *socket*.

```
p socketpair $x
```

sockets

`sockets` — operation `sockets`. Category: *filesystem extensions*.

```
p sockets $x
```

sod

`sod` — operation `sod`. Alias for `start_of_day`. Category: *extended stdlib*.

```
p sod $x
```

softmax

`softmax(xs) → array` — $\exp(x) / \sum \exp(x)$. Returns probabilities summing to 1.

softmax_choose

`softmax_choose(logits, temperature=1, seed=0) → int` — sample from softmax over logits.

softplus

`softplus` applies the Softplus activation: $\ln(1 + e^x)$. Smooth approximation to ReLU.

```
p softplus(0)    # 0.6931 (ln 2)
p softplus(10)   # ≈ 10
```

softsign

`softsign` — operation `softsign`. Category: *ml extensions*.

```
p softsign $x
```

soh

`soh` — operation `soh`. Alias for `start_of_hour`. Category: *extended stdlib*.

```
p soh $x
```

soh_capacity_fade

`soh_capacity_fade` — operation `soh capacity fade`. Category: **electrochemistry, batteries, fuel cells**.

```
p soh_capacity_fade $x
```

soi_oscillation_index

`soi_oscillation_index` — index of `soi oscillation`. Category: **climate, fluids, atmospheric**.

```
p soi_oscillation_index $x
```

sol

`sol` — operation `sol`. Category: **constants**.

```
p sol $x
```

solar_constant

`solar_constant` — operation `solar constant`. Category: **more extensions**.

```
p solar_constant $x
```

solar_constant_at_distance

`solar_constant_at_distance` — distance / dissimilarity metric of `solar constant at`. Category: **climate, fluids, atmospheric**.

```
p solar_constant_at_distance $x
```

solar_cycle_phase

`solar_cycle_phase` — operation `solar cycle phase`. Category: **climate, fluids, atmospheric**.

```
p solar_cycle_phase $x
```

solar_declination

`solar_declination` — operation `solar declination`. Category: **misc**.

```
p solar_declination $x
```

solar_luminosity_to_w

`solar_luminosity_to_w` — convert from `solar luminosity` to `w`. Category: **astronomy / music / color / units**.

```
p solar_luminosity_to_w $value
```

solar_mass_to_kg

`solar_mass_to_kg` — convert from `solar mass` to `kg`. Category: **astronomy / music / color / units**.

```
p solar_mass_to_kg $value
```

solar_noon_unix

`solar_noon_unix(longitude_deg, unix) → int` — unix timestamp of apparent solar noon at given longitude for that date. Applies the equation-of-time correction ($9.873 \cdot \sin(2B) - 7.655 \cdot \sin(B)$ minutes, B from day-of-year).

```
p solar_noon_unix(0, time()) # -12:00 UTC for Greenwich
```

solar_zenith_angle

`solar_zenith_angle` — operation `solar zenith angle`. Category: **more extensions**.

```
p solar_zenith_angle $x
```

sole

`sole` — operation `sole`. Category: **ruby enumerable extras**.

```
p sole $x
```

solid_electrolyte_capacity

`solid_electrolyte_capacity` — operation `solid electrolyte capacity`. Category: **electrochemistry, batteries, fuel cells**.

```
p solid_electrolyte_capacity $x
```

solstice_winter

`solstice_winter` — operation `solstice winter`. Category: **b82-misc-utility**.

```
p solstice_winter $x
```

solution_resistance_estimate

`solution_resistance_estimate` — operation `solution resistance estimate`. Category: **electrochemistry, batteries, fuel cells**.

```
p solution_resistance_estimate $x
```

solvable_length_upper

`solvable_length_upper` — operation `solvable length upper`. Category: **econometrics**.

```
p solvable_length_upper $x
```

solve_ivp_step

`solve_ivp_step` — single-step update of `solve ivp`. Category: **numpy + scipy.special**.

```
p solve_ivp_step $x
```

som

`som` — operation `som`. Alias for `start_of_minute`. Category: **extended stdlib**.

```
p som $x
```

some

`some` — operation `some`. Category: **algebraic match**.

```
p some $x
```

sorensen_dice

sorensen_dice — operation `sorensen dice`. Category: **biology / ecology**.

```
p sorensen_dice $x
```

sort_by

sort_by — operation `sort by`. Category: **functional / iterator**.

```
p sort_by $x
```

sort_by_cached_key

sort_by_cached_key — operation `sort by cached key`. Category: **iterator + string-distance extras**.

```
p sort_by_cached_key $x
```

sort_on

sort_on — operation `sort on`. Category: **haskell list functions**.

```
p sort_on $x
```

sort_words

sort_words — operation `sort words`. Category: **string processing extras**.

```
p sort_words $x
```

sorted

sorted — operation `sorted`. Category: **file stat / path**.

```
p sorted $x
```

sortedMapOf

sortedMapOf — operation `sortedMapOf`. Category: **kotlin collections**.

```
p sortedMapOf $x
```

sortedSetOf

sortedSetOf — operation `sortedSetOf`. Category: **kotlin collections**.

```
p sortedSetOf $x
```

sorted_by_length

sorted_by_length — operation `sorted by length`. Category: **file stat / path**.

```
p sorted_by_length $x
```

sorted_desc

sorted_desc — operation `sorted desc`. Category: **file stat / path**.

```
p sorted_desc $x
```

sorted_nums

`sorted_nums` — operation `sorted_nums`. Category: **file stat / path**.

```
p sorted_nums $x
```

sortino

`sortino_ratio(\@returns, $target)` (alias `sortino`) — computes the Sortino ratio, similar to Sharpe but only penalizes downside volatility. Better for asymmetric return distributions.

```
val @returns = (0.05, -0.02, 0.08, -0.01)
p sortino(\@returns, 0.0) # penalizes only negative returns
```

sortw

`sortw` — operation `sortw`. Alias for `sort_words`. Category: **string processing extras**.

```
p sortw $x
```

sos2tf_step

`sos2tf_step` — single-step update of `sos2tf`. Category: **economics + game theory**.

```
p sos2tf_step $x
```

sos_constraint_check

`sos_constraint_check` — operation `sos_constraint_check`. Category: **combinatorial optimization, scheduling**.

```
p sos_constraint_check $x
```

sosfilt_step

`sosfilt_step` — single-step update of `sosfilt`. Category: **economics + game theory**.

```
p sosfilt_step $x
```

sound_db

`sound_db` — operation `sound_db`. Category: **em / optics / relativity**.

```
p sound_db $x
```

sound_gas

`sound_gas` — operation `sound_gas`. Category: **em / optics / relativity**.

```
p sound_gas $x
```

sound_horizon_recomb

`sound_horizon_recomb` — operation `sound_horizon_recomb`. Category: **cosmology / gr / flrw**.

```
p sound_horizon_recomb $x
```


sound_solid

`sound_solid` — operation `sound solid`. Category: **em / optics / relativity**.

```
p sound_solid $x
```

soundex

`soundex` — operation `soundex`. Category: **extended stdlib**.

```
p soundex $x
```

soundex_phonetic

`soundex_phonetic` — operation `soundex phonetic`. Category: **computational linguistics**.

```
p soundex_phonetic $x
```

soundex_v1

`soundex_v1(name) → string` — classic American Soundex (4-char: leading letter + 3 digits).

```
p soundex_v1("Robert") # R163
```

soundex_v2

`soundex_v2(name) → string` — Apache Refined Soundex (longer code, no length cap, 9 finer-grained groups: BP, FV, CKS, GJ, QXZ, DT, L, MN, R).

```
p soundex_v2("Robert")
```

SOURCE

`source(PATH) → last expression value`. Read a stryke script from `PATH` and execute it in the **current** interpreter. Variables (*our/my*), subroutines, classes, enums, and pragmas defined in the file become visible in the calling scope — same semantics as `do "file"` (Perl 5) or `source FILE` (POSIX shell).

```
source("config.stk")
p $LIB_VERSION           # leaked from config.stk
Demo::Lib::greet("alice") # leaked fn defined in config.stk
```

Designed for REPL workflows: `source` a working environment without forking. Returns `undef` if the file can't be parsed, dies on file-read failure. For isolated execution (no state-leak), use `run` instead.

spaceship

`spaceship` — operation `spaceship`. Category: **file stat / path**.

```
p spaceship $x
```

span

`span` — operation `span`. Category: **list helpers**.

```
p span $x
```

span_fn

`span_fn` — operation `span fn`. Category: **haskell list functions**.

```
p span_fn $x
```

spanf

spanf — operation **spanf**. Alias for **span_fn**. Category: *haskell list functions*.

```
p spanf $x
```

sparse_csr_build

sparse_csr_build — operation **sparse csr build**. Category: *more extensions (2)*.

```
p sparse_csr_build $x
```

sparse_csr_mul_vec

sparse_csr_mul_vec — operation **sparse csr mul vec**. Category: *more extensions (2)*.

```
p sparse_csr_mul_vec $x
```

sparse_density

sparse_density — operation **sparse density**. Category: *more extensions (2)*.

```
p sparse_density $x
```

spat

spat — operation **spat**. Alias for **split_at**. Category: *algebraic match*.

```
p spat $x
```

spbfs

spbfs — operation **spbfs**. Alias for **shortest_path_bfs**. Category: *extended stdlib*.

```
p spbfs $x
```

spearman

spearman_correlation(\@x, \@y) (alias **spearman**) — computes Spearman's rank correlation coefficient between two datasets. Measures monotonic relationships, robust to outliers. Returns a value from -1 to 1.

```
val @x = (1, 2, 3, 4, 5)
var @y = (5, 6, 7, 8, 7)
p spearman(\@x, \@y) # high positive correlation
```

spearmanr

spearmanr — operation **spearmanr**. Category: *r / scipy distributions and tests*.

```
p spearmanr $x
```

species_area

species_area — operation **species area**. Category: *biology / ecology*.

```
p species_area $x
```

specific_angular_momentum

`specific_angular_momentum` — operation `specific angular momentum`. Category: **more extensions**.

```
p specific_angular_momentum $x
```

specific_capacity_active

`specific_capacity_active` — operation `specific capacity active`. Category: **electrochemistry, batteries, fuel cells**.

```
p specific_capacity_active $x
```

specific_gravity_water

`specific_gravity_water` — operation `specific gravity water`. Category: **geology, seismology, mineralogy**.

```
p specific_gravity_water $x
```

specific_heat_const_v

`specific_heat_const_v` — operation `specific heat const v`. Category: **misc**.

```
p specific_heat_const_v $x
```

specific_orbital_energy

`specific_orbital_energy` — operation `specific orbital energy`. Category: **more extensions**.

```
p specific_orbital_energy $x
```

specificity

`specificity` — operation `specificity`. Category: **ml extensions**.

```
p specificity $x
```

speck_round

`speck_round` — operation `speck round`. Category: **more extensions**.

```
p speck_round $x
```

spectral_density_estimate

`spectral_density_estimate` — operation `spectral density estimate`. Category: **statsmodels**.

```
p spectral_density_estimate $x
```

spectral_kappa_minus53

`spectral_kappa_minus53` — operation `spectral kappa minus53`. Category: **climate, fluids, atmospheric**.

```
p spectral_kappa_minus53 $x
```

spectral_radius

`spectral_radius` — operation `spectral radius`. Category: **ode advanced**.

```
p spectral_radius $x
```

spectral_signature_match

`spectral_signature_match` — operation `spectral signature match`. Category: **electrochemistry, batteries, fuel cells**.

```
p spectral_signature_match $x
```

spectro

`spectro` — operation `spectro`. Alias for `spectrogram`. Category: **dsp / signal (extended)**.

```
p spectro $x
```

spectrogram

`spectrogram(\@signal, $window_size, $hop_size)` (alias `stft`) — computes Short-Time Fourier Transform. Returns a 2D arrayref where each row is the spectrum of a windowed segment. Useful for time-frequency analysis.

```
val $spec = stft(\@audio, 1024, 512)
# $spec->[t][f] is magnitude at time t, frequency f
```

spectrogram_simple

`spectrogram_simple` — operation `spectrogram simple`. Category: **signal processing**.

```
p spectrogram_simple $x
```

spectrophotometer_beer_lambert

`spectrophotometer_beer_lambert` — operation `spectrophotometer beer lambert`. Category: **chemistry & biochemistry**.

```
p spectrophotometer_beer_lambert $x
```

spectroscopic_factor

`spectroscopic_factor` — factor of `spectroscopic`. Category: **chemistry & biochemistry**.

```
p spectroscopic_factor $x
```

speed_distance_time

`speed_distance_time` — operation `speed distance time`. Category: **physics formulas**.

```
p speed_distance_time $x
```

speed_of_light

`speed_of_light` — operation `speed of light`. Category: **math / physics constants**.

```
p speed_of_light $x
```

speed_of_sound_ideal

`speed_of_sound_ideal` — operation `speed of sound ideal`. Category: **misc**.

```
p speed_of_sound_ideal $x
```

spew

spew — operation `spew`. Alias for `spurt`. Category: *filesystem extensions*.

```
p spew $x
```

sph_harm

sph_harm — operation `sph harm`. Category: *numpy + scipy.special*.

```
p sph_harm $x
```

sphenic_q

sphenic_q — operation `sphenic q`. Category: *misc*.

```
p sphenic_q $x
```

sphere_aabb_intersect

sphere_aabb_intersect(center, radius, aabb) → 0|1 — 1 if the sphere overlaps the AABB.

sphere_of_influence

sphere_of_influence — operation `sphere of influence`. Category: *cosmology / gr / flrw*.

```
p sphere_of_influence $x
```

sphere_sphere_intersect

sphere_sphere_intersect(c1, r1, c2, r2) → 0|1 — 1 if $|c1-c2| \leq r1+r2$.

sphere_surface

sphere_surface — operation `sphere surface`. Category: *geometry / physics*.

```
p sphere_surface $x
```

sphere_surface_area

sphere_surface_area — operation `sphere surface area`. Category: *complex / geom / color / trig*.

```
p sphere_surface_area $x
```

sphere_volume

sphere_volume — operation `sphere volume`. Category: *geometry / physics*.

```
p sphere_volume $x
```

spherical_bessel_j0

spherical_bessel_j0 — operation `spherical bessel j0`. Category: *special functions extra*.

```
p spherical_bessel_j0 $x
```

spherical_bessel_j1

spherical_bessel_j1 — operation `spherical_bessel_j1`. Category: **special functions extra**.

```
p spherical_bessel_j1 $x
```

spherical_bessel_y0

spherical_bessel_y0 — operation `spherical_bessel_y0`. Category: **special functions extra**.

```
p spherical_bessel_y0 $x
```

spherical_bessel_y1

spherical_bessel_y1 — operation `spherical_bessel_y1`. Category: **special functions extra**.

```
p spherical_bessel_y1 $x
```

spherical_jn

spherical_jn — operation `spherical_jn`. Category: **numpy + scipy.special**.

```
p spherical_jn $x
```

spherical_to_cartesian

spherical_to_cartesian — convert from `spherical` to `cartesian`. Category: **complex / geom / color / trig**.

```
p spherical_to_cartesian $value
```

spherical_triangle_area

spherical_triangle_area — operation `spherical_triangle_area`. Category: **geometry / topology**.

```
p spherical_triangle_area $x
```

spherical_yn

spherical_yn — operation `spherical_yn`. Category: **numpy + scipy.special**.

```
p spherical_yn $x
```

sphsurf

sphsurf — operation `sphsurf`. Alias for `sphere_surface`. Category: **geometry / physics**.

```
p sphsurf $x
```

sphvol

sphvol — operation `sphvol`. Alias for `sphere_volume`. Category: **geometry / physics**.

```
p sphvol $x
```

spin_casimir

spin_casimir — operation `spin_casimir`. Category: **quantum mechanics deep**.

```
p spin_casimir $x
```

spin_orbit_coupling_term

`spin_orbit_coupling_term` — operation `spin orbit coupling term`. Category: **electrochemistry, batteries, fuel cells**.

```
p spin_orbit_coupling_term $x
```

spin_spin_coupling_term

`spin_spin_coupling_term` — operation `spin spin coupling term`. Category: **electrochemistry, batteries, fuel cells**.

```
p spin_spin_coupling_term $x
```

spinner_start

`spinner_start` — operation `spinner start`. Category: **pipeline / string helpers**.

```
p spinner_start $x
```

spinner_stop

`spinner_stop` — operation `spinner stop`. Category: **pipeline / string helpers**.

```
p spinner_stop $x
```

split_at

`split_at` — operation `split at`. Category: **algebraic match**.

```
p split_at $x
```

split_on

`split_on` — operation `split on`. Category: **go/general functional utilities**.

```
p split_on $x
```

split_regex

`split_regex` — operation `split regex`. Category: **extended stdlib**.

```
p split_regex $x
```

split_with

`split_with` — operation `split with`. Category: **algebraic match**.

```
p split_with $x
```

splitmix64_next

`splitmix64_next(seed) → int` — SplitMix64 step. Used for seeding other PRNGs.

splitmix64_step

`splitmix64_step` — single-step update of `splitmix64`. Category: **cryptography deep**.

```
p splitmix64_step $x
```

splre

`splre` — operation `splre`. Alias for `split_regex`. Category: **extended stdlib**.

```
p splre $x
```

spmc_new

`spmc_new` — constructor of `spmc`. Category: **extras**.

```
p spmc_new $x
```

spon

`spon` — operation `spon`. Alias for `split_on`. Category: **go/general functional utilities**.

```
p spon $x
```

spookyhash_128

`spookyhash_128` — operation `spookyhash 128`. Category: **b81-misc-utility**.

```
p spookyhash_128 $x
```

spop

`spop` — operation `spop`. Category: **redis-flavour primitives**.

```
p spop $x
```

spring_energy

`spring_energy` — operation `spring energy`. Category: **physics formulas**.

```
p spring_energy $x
```

spring_force

`spring_force` — operation `spring force`. Category: **physics formulas**.

```
p spring_force $x
```

spring_oscillator_pos

`spring_oscillator_pos(amplitude, omega, phase=0, t) → number` — $A \cdot \cos(\omega t + \phi)$.

spring_period

`spring_period` — operation `spring period`. Category: **misc**.

```
p spring_period $x
```

springpe

`springpe` — operation `springpe`. Alias for `spring_energy`. Category: **physics formulas**.

```
p springpe $x
```


spt_n_correction

`spt_n_correction` — operation `spt n correction`. Category: **geology, seismology, mineralogy**.

```
p spt_n_correction $x
```

spw

`spw` — operation `spw`. Alias for `split_with`. Category: **algebraic match**.

```
p spw $x
```

sql

`sql` — operation `sql`. Alias for `sqlite`. Category: **data / network**.

```
p sql $x
```

sqr

`sqr` — operation `sqr`. Category: **trig / math**.

```
p sqr $x
```

sqr2

`sqr2` — operation `sqr2`. Category: **math / physics constants**.

```
p sqr2 $x
```

sqr_each

`sqr_each` — square-root helper `each`. Category: **trivial numeric helpers**.

```
p sqr_each $x
```

sqr_swap_gate

`sqr_swap_gate` — square-root helper `swap gate`. Category: **more extensions**.

```
p sqr_swap_gate $x
```

square_each

`square_each` — operation `square each`. Category: **trivial numeric helpers**.

```
p square_each $x
```

square_free

`square_free` — release / destroy of `square`. Category: **math / number theory extras**.

```
p square_free $x
```

square_pyramidal

`square_pyramidal(n) → int` — $n \cdot (n+1) \cdot (2n+1) / 6$.

square_pyramidal_number

`square_pyramidal_number` — operation `square pyramidal number`. Category: **misc**.

```
p square_pyramidal_number $x
```

square_root

`square_root` — operation `square root`. Category: **math functions**.

```
p square_root $x
```

squared_hinge

`squared_hinge` — operation `squared hinge`. Category: **ml extensions**.

```
p squared_hinge $x
```

squared_triangular

`squared_triangular` — operation `squared triangular`. Category: **misc**.

```
p squared_triangular $x
```

squares_seq

`squares_seq` — operation `squares seq`. Category: **sequences**.

```
p squares_seq $x
```

squeezing_db

`squeezing_db` — operation `squeezing db`. Category: **quantum mechanics deep**.

```
p squeezing_db $x
```

squish

`squish` — operation `squish`. Category: **trivial string ops**.

```
p squish $x
```

sregex

`sregex` — operation `sregex`. Alias for `stress_regex`. Category: **stress / telemetry extensions**.

```
p sregex $x
```

srem

`srem` — operation `srem`. Category: **redis-flavour primitives**.

```
p srem $x
```

srgb_to_rgb

`srgb_to_rgb` — convert from `srgb` to `rgb`. Category: **complex / geom / color / trig**.

```
p srgb_to_rgb $value
```

srgb_to_xyz

srgb_to_xyz — convert from [srgb](#) to [xyz](#). Category: *astronomy / music / color / units*.

```
p srgb_to_xyz $value
```

srton

srton — operation [srton](#). Alias for [sort_on](#). Category: *haskell list functions*.

```
p srton $x
```

ssa_phi_insert

ssa_phi_insert — operation [ssa phi insert](#). Category: *compilers / parsing*.

```
p ssa_phi_insert $x
```

ssn_check

ssn_check — operation [ssn check](#). Category: *b82-misc-utility*.

```
p ssn_check $x
```

ssort

ssort — operation [ssort](#). Alias for [stress_sort](#). Category: *stress / telemetry extensions*.

```
p ssort $x
```

sst_anomaly_step

sst_anomaly_step — single-step update of [sst anomaly](#). Category: *climate, fluids, atmospheric*.

```
p sst_anomaly_step $x
```

stable_matching_count

stable_matching_count — count of [stable matching](#). Category: *economics + game theory*.

```
p stable_matching_count $x
```

stack_new

stack_new — stack op [new](#). Category: *data structure helpers*.

```
p stack_new $x
```

stackelberg_eq

stackelberg_eq — operation [stackelberg eq](#). Category: *economics + game theory*.

```
p stackelberg_eq $x
```

stackelberg_step

stackelberg_step — single-step update of [stackelberg](#). Category: *climate, fluids, atmospheric*.

```
p stackelberg_step $x
```

stag_hunt_payoff

stag_hunt_payoff — operation `stag_hunt_payoff`. Category: **climate, fluids, atmospheric**.

```
p stag_hunt_payoff $x
```

standard_error

standard_error — operation `standard_error`. Category: **math / numeric extras**.

```
p standard_error $x
```

standardized_mortality_smr

standardized_mortality_smr — operation `standardized_mortality_smr`. Category: **epidemiology / public health**.

```
p standardized_mortality_smr $x
```

standing_wave_fundamental

standing_wave_fundamental — operation `standing_wave_fundamental`. Category: **em / optics / relativity**.

```
p standing_wave_fundamental $x
```

star_number

star_number — operation `star_number`. Category: **misc**.

```
p star_number $x
```

stark_shift_linear

stark_shift_linear — operation `stark_shift_linear`. Category: **quantum mechanics deep**.

```
p stark_shift_linear $x
```

starmap

starmap — operation `starmap`. Category: **python/ruby stdlib**.

```
p starmap $x
```

start_of_day

start_of_day — operation `start_of_day`. Category: **extended stdlib**.

```
p start_of_day $x
```

start_of_hour

start_of_hour — operation `start_of_hour`. Category: **extended stdlib**.

```
p start_of_hour $x
```

start_of_minute

start_of_minute — operation `start_of_minute`. Category: **extended stdlib**.

```
p start_of_minute $x
```

starts_with

`starts_with` — operation `starts with`. Category: **trivial string ops**.

```
p starts_with $x
```

starts_with_any

`starts_with_any` — operation `starts with any`. Category: **string**.

```
p starts_with_any $x
```

state_norm

`state_norm` — norm (vector length-like) of `state`. Category: **quantum mechanics deep**.

```
p state_norm $x
```

state_normalize

`state_normalize` — operation `state normalize`. Category: **quantum mechanics deep**.

```
p state_normalize $x
```

state_of_charge_kalman

`state_of_charge_kalman` — operation `state of charge kalman`. Category: **electrochemistry, batteries, fuel cells**.

```
p state_of_charge_kalman $x
```

state_space_kalman

`state_space_kalman` — operation `state space kalman`. Category: **statsmodels**.

```
p state_space_kalman $x
```

stats_describe

`stats_describe` — operation `stats describe`. Alias for `describe`. Category: **statistics (extended)**.

```
p stats_describe $x
```

stderr_stat

`stderr_stat` — operation `stderr stat`. Alias for `standard_error`. Category: **math / numeric extras**.

```
p stderr_stat $x
```

stefan_boltzmann

`stefan_boltzmann` — operation `stefan boltzmann`. Category: **physics formulas**.

```
p stefan_boltzmann $x
```

stefan_boltzmann_constant

`stefan_boltzmann_constant` — operation `stefan boltzmann constant`. Category: **physics constants**.

```
p stefan_boltzmann_constant $x
```

stefan_boltzmann_radiation

`stefan_boltzmann_radiation` — operation `stefan boltzmann radiation`. Category: **climate, fluids, atmospheric**.

```
p stefan_boltzmann_radiation $x
```

stefan_boltzmann_si

`stefan_boltzmann_si` — operation `stefan boltzmann si`. Category: **cosmology / gr / flrw**.

```
p stefan_boltzmann_si $x
```

stefboltz

`stefboltz` — operation `stefboltz`. Alias for `stefan_boltzmann`. Category: **physics formulas**.

```
p stefboltz $x
```

steffensen

`steffensen` — operation `steffensen`. Alias for `steffensen_root`. Category: **more extensions (2)**.

```
p steffensen $x
```

steffensen_root

`steffensen_root` — operation `steffensen root`. Category: **more extensions (2)**.

```
p steffensen_root $x
```

stem_lancaster

`stem_lancaster` — operation `stem lancaster`. Category: **computational linguistics**.

```
p stem_lancaster $x
```

step

`step` — operation `step`. Category: **python/ruby stdlib**.

```
p step $x
```

stern_layer_capacitance

`stern_layer_capacitance` — operation `stern layer capacitance`. Category: **electrochemistry, batteries, fuel cells**.

```
p stern_layer_capacitance $x
```

stft_step

`stft_step` — single-step update of `stft`. Category: **economics + game theory**.

```
p stft_step $x
```

sthread

`sthread` — operation `sthread`. Alias for `stress_thread`. Category: **stress / telemetry extensions**.

```
p sthread $x
```

stiffness_ratio

`stiffness_ratio` — ratio of `stiffness`. Category: **ode advanced**.

```
p stiffness_ratio $x
```

stirling

`stirling` — operation `stirling`. Alias for `stirling_approx`. Category: **math formulas**.

```
p stirling $x
```

stirling2

`stirling2` (alias `stirling_second`) computes the Stirling number of the second kind $S(n,k)$ — the number of ways to partition n elements into k non-empty subsets.

```
p stirling2(4, 2) # 7
p stirling2(5, 3) # 25
```

stirling_1

`stirling_1` — operation `stirling 1`. Category: **math / number theory extras**.

```
p stirling_1 $x
```

stirling_2

`stirling_2` — operation `stirling 2`. Category: **math / number theory extras**.

```
p stirling_2 $x
```

stirling_approx

`stirling_approx` — operation `stirling approx`. Category: **math formulas**.

```
p stirling_approx $x
```

stl_decompose

`stl_decompose` — operation `stl decompose`. Category: **statsmodels**.

```
p stl_decompose $x
```

stoch_rsi

`stoch_rsi(prices, period=14) → array` — stochastic RSI: normalises RSI to $[0,1]$ by $(rsi - min) / (max - min)$ over its own window.

```
p stoch_rsi(\@closes, 14)
```

stoer_wagner_step

`stoer_wagner_step` — single-step update of `stoer wagner`. Category: **networkx graph algorithms**.

```
p stoer_wagner_step $x
```

stone_to_kg

`stone_to_kg` — convert from `stone` to `kg`. Category: **unit conversions**.

```
p stone_to_kg $value
```

stormer_verlet_step

`stormer_verlet_step` — single-step update of `stormer verlet`. Category: **ode advanced**.

```
p stormer_verlet_step $x
```

str_aho_corasick

`str_aho_corasick` — operation `str aho corasick`. Category: **iterator + string-distance extras**.

```
p str_aho_corasick $x
```

str_boyer_moore

`str_boyer_moore` — operation `str boyer moore`. Category: **iterator + string-distance extras**.

```
p str_boyer_moore $x
```

str_compress_lzss

`str_compress_lzss` — operation `str compress lzss`. Category: **iterator + string-distance extras**.

```
p str_compress_lzss $x
```

str_compress_rle

`str_compress_rle` — operation `str compress rle`. Category: **iterator + string-distance extras**.

```
p str_compress_rle $x
```

str_decompress_lzss

`str_decompress_lzss` — operation `str decompress lzss`. Category: **iterator + string-distance extras**.

```
p str_decompress_lzss $x
```

str_decompress_rle

`str_decompress_rle` — operation `str decompress rle`. Category: **iterator + string-distance extras**.

```
p str_decompress_rle $x
```

str_huffman_decode

`str_huffman_decode` — decode of `str huffman`. Category: **iterator + string-distance extras**.

```
p str_huffman_decode $x
```

str_huffman_encode

`str_huffman_encode` — encode of `str huffman`. Category: **iterator + string-distance extras**.

```
p str_huffman_encode $x
```


str_isogram

str_isogram — operation `str isogram`. Category: *iterator + string-distance extras*.

```
p str_isogram $x
```

str_kmp

str_kmp — operation `str kmp`. Category: *iterator + string-distance extras*.

```
p str_kmp $x
```

str_lcs

str_lcs — operation `str lcs`. Category: *iterator + string-distance extras*.

```
p str_lcs $x
```

str_lcs_length

str_lcs_length — operation `str lcs length`. Category: *iterator + string-distance extras*.

```
p str_lcs_length $x
```

str_longest_common_substring

str_longest_common_substring — operation `str longest common substring`. Category: *iterator + string-distance extras*.

```
p str_longest_common_substring $x
```

str_rabin_karp

str_rabin_karp — operation `str rabin karp`. Category: *iterator + string-distance extras*.

```
p str_rabin_karp $x
```

str_rotations

str_rotations — operation `str rotations`. Category: *iterator + string-distance extras*.

```
p str_rotations $x
```

str_suffix_array

str_suffix_array — operation `str suffix array`. Category: *iterator + string-distance extras*.

```
p str_suffix_array $x
```

str_z_array

str_z_array — operation `str z array`. Category: *iterator + string-distance extras*.

```
p str_z_array $x
```

strang_split

strang_split — operation `strang split`. Category: *ode advanced*.

```
p strang_split $x
```

stratonovich_correction

`stratonovich_correction` — operation `stratonovich correction`. Category: **ode advanced**.

```
p stratonovich_correction $x
```

strdist

`strdist` — operation `strdist`. Alias for `string_distance`. Category: **string processing extras**.

```
p strdist $x
```

streak_color_index

`streak_color_index` — index of `streak color`. Category: **geology, seismology, mineralogy**.

```
p streak_color_index $x
```

stream_prompt_cb

`stream_prompt_cb` — operation `stream prompt cb`. Category: **ai primitives (docs/ai_primitives.md)**.

```
p stream_prompt_cb $x
```

streamfunction_step

`streamfunction_step` — single-step update of `streamfunction`. Category: **climate, fluids, atmospheric**.

```
p streamfunction_step $x
```

strehl_ratio

`strehl_ratio` — ratio of `strehl`. Category: **astronomy / astrometry**.

```
p strehl_ratio $x
```

strict_word_similarity

`strict_word_similarity` — similarity metric of `strict word`. Category: **postgres sql strings, json, aggregates**.

```
p strict_word_similarity $x
```

strikethrough

`strikethrough` — operation `strikethrough`. Alias for `red`. Category: **color / ansi**.

```
p strikethrough $x
```

string_agg

`string_agg` — string helper `agg`. Category: **postgres sql strings, json, aggregates**.

```
p string_agg $x
```

string_camel_to_snake

`string_camel_to_snake` — convert from `string camel` to `snake`. Category: **misc**.

```
p string_camel_to_snake $value
```

string_chars

`string_chars` — string helper `chars`. Category: **misc**.

```
p string_chars $x
```

string_count

`string_count` — string helper `count`. Category: **string helpers**.

```
p string_count $x
```

string_distance

`string_distance` — string helper `distance`. Category: **string processing extras**.

```
p string_distance $x
```

string_drop_first

`string_drop_first` — string helper `drop first`. Category: **misc**.

```
p string_drop_first $x
```

string_drop_last

`string_drop_last` — string helper `drop last`. Category: **misc**.

```
p string_drop_last $x
```

string_drop_while

`string_drop_while` — string helper `drop while`. Category: **misc**.

```
p string_drop_while $x
```

string_expand_tabs

`string_expand_tabs` — string helper `expand tabs`. Category: **misc**.

```
p string_expand_tabs $x
```

string_interperse

`string_interperse` — string helper `interperse`. Category: **misc**.

```
p string_interperse $x
```

string_kebab_to_snake

`string_kebab_to_snake` — convert from `string kebab` to `snake`. Category: **misc**.

```
p string_kebab_to_snake $value
```

string_letter_frequency

string_letter_frequency — string helper `letter frequency`. Category: **misc**.

```
p string_letter_frequency $x
```

string_lines_count

string_lines_count — string helper `lines count`. Category: **misc**.

```
p string_lines_count $x
```

string_multiply

string_multiply — string helper `multiply`. Category: **string processing extras**.

```
p string_multiply $x
```

string_normalize_spaces

string_normalize_spaces — string helper `normalize spaces`. Category: **misc**.

```
p string_normalize_spaces $x
```

string_partition_at_word

string_partition_at_word — string helper `partition at word`. Category: **misc**.

```
p string_partition_at_word $x
```

string_replicate

string_replicate — string helper `replicate`. Category: **misc**.

```
p string_replicate $x
```

string_snake_to_camel

string_snake_to_camel — convert from `string snake` to `camel`. Category: **misc**.

```
p string_snake_to_camel $value
```

string_snake_to_kebab

string_snake_to_kebab — convert from `string snake` to `kebab`. Category: **misc**.

```
p string_snake_to_kebab $value
```

string_sort

string_sort — string helper `sort`. Category: **string helpers**.

```
p string_sort $x
```

string_split_at_first

string_split_at_first — string helper `split at first`. Category: **misc**.

```
p string_split_at_first $x
```

string_take_first

`string_take_first` — string helper `take first`. Category: **misc**.

```
p string_take_first $x
```

string_take_last

`string_take_last` — string helper `take last`. Category: **misc**.

```
p string_take_last $x
```

string_take_while

`string_take_while` — string helper `take while`. Category: **misc**.

```
p string_take_while $x
```

string_to_charcodes

`string_to_charcodes` — convert from `string` to `charcodes`. Category: **misc**.

```
p string_to_charcodes $value
```

string_to_chars

`string_to_chars` — convert from `string` to `chars`. Category: **string helpers**.

```
p string_to_chars $value
```

string_truncate_ellipsis

`string_truncate_ellipsis` — string helper `truncate ellipsis`. Category: **misc**.

```
p string_truncate_ellipsis $x
```

string_uniq_chars

`string_uniq_chars` — string helper `uniq chars`. Category: **misc**.

```
p string_uniq_chars $x
```

string_unique_chars

`string_unique_chars` — string helper `unique chars`. Category: **string helpers**.

```
p string_unique_chars $x
```

string_words_count

`string_words_count` — string helper `words count`. Category: **misc**.

```
p string_words_count $x
```

string_xor

`string_xor` — string helper `xor`. Category: **misc**.

```
p string_xor $x
```

strip_ansi

`strip_ansi` — operation `strip ansi`. Category: **color / ansi**.

```
p strip_ansi $x
```

strip_html

`strip_html` — operation `strip html`. Category: **string processing extras**.

```
p strip_html $x
```

strip_indent

`strip_indent(s) → string` — remove the common leading whitespace from every non-empty line.

```
p strip_indent("  a\n    b\n")
```

strip_prefix

`strip_prefix` — operation `strip prefix`. Category: **path helpers**.

```
p strip_prefix $x
```

strip_suffix

`strip_suffix` — operation `strip suffix`. Category: **path helpers**.

```
p strip_suffix $x
```

strlen

`strlen` — operation `strlen`. Category: **redis-flavour primitives**.

```
p strlen $x
```

stimul

`stimul` — operation `stimul`. Alias for `string_multiply`. Category: **string processing extras**.

```
p stimul $x
```

strong_pseudoprime

`strong_pseudoprime` — operation `strong pseudoprime`. Category: **cryptanalysis & number theory deep**.

```
p strong_pseudoprime $x
```

strouhal_full

`strouhal_full` — operation `strouhal full`. Category: **climate, fluids, atmospheric**.

```
p strouhal_full $x
```

strsm

`strsm` — operation `strsm`. Category: **blas / lapack**.

```
p strsm $x
```

struve_h0

`struve_h0` — operation `struve h0`. Category: **special functions extra**.

```
p struve_h0 $x
```

struve_h1

`struve_h1` — operation `struve h1`. Category: **special functions extra**.

```
p struve_h1 $x
```

student_pdf

`t_pdf` (alias `tpdf`) evaluates Student's t-distribution PDF at `x` with `nu` degrees of freedom.

```
p tpdf(0, 10)      # peak of t(10)
p tpdf(2.228, 10)
```

stulz_min_call

`stulz_min_call` — operation `stulz min call`. Category: **financial pricing models**.

```
p stulz_min_call $x
```

sturmian_word

`sturmian_word` — operation `sturmian word`. Category: **more extensions**.

```
p sturmian_word $x
```

sub

`sub` is a Perl 5 idiom that `stryke` rejects under `--no-interop`. Use `fn` instead — it opens the `stryke` topic-variable protocol (`_`, `_0`, `_1`, `_<`) and supports expression-bodied one-liners (`fn name = expr`).

```
# WRONG (legacy Perl 5)
sub double { val $x = shift; $x * 2 }
sub area($w, $h) { return $w * $h }

# RIGHT (stryke)
fn double = _ * 2                # expression-bodied
fn area($w, $h) = $w * $h        # named params, one-liner
fn shout { val $s = _; uc($s) . "!" } # block body, multi-statement
```

In `--compat` mode `sub` still parses for Perl-5 ports, but **all new `stryke` code should use `fn`**.

See also: `fn`, `_`, `_0`, `_1`, style-guide §0 rule 1.

sub_script

`sub_script` — operation `sub script`. Alias for `subscript`. Category: **encoding / phonetics**.

```
p sub_script $x
```

subfact

`subfact` — operation `subfact`. Alias for `subfactorial`. Category: **math / numeric extras**.

```
p subfact $x
```

subfactorial

`subfactorial` — operation `subfactorial`. Category: **math / numeric extras**.

```
p subfactorial $x
```

subgame_perfect_eq

`subgame_perfect_eq` — operation `subgame perfect eq`. Category: **climate, fluids, atmospheric**.

```
p subgame_perfect_eq $x
```

subgradient_step_size

`subgradient_step_size` — operation `subgradient step size`. Category: **combinatorial optimization, scheduling**.

```
p subgradient_step_size $x
```

subgraph_isomorphism

`subgraph_isomorphism` — operation `subgraph isomorphism`. Category: **networkx graph algorithms**.

```
p subgraph_isomorphism $x
```

submodular_curvature_bound

`submodular_curvature_bound` — operation `submodular curvature bound`. Category: **combinatorial optimization, scheduling**.

```
p submodular_curvature_bound $x
```

submodular_greedy_step

`submodular_greedy_step` — single-step update of `submodular greedy`. Category: **combinatorial optimization, scheduling**.

```
p submodular_greedy_step $x
```

subname

`subname` — operation `subname`. Category: **list / aggregate**.

```
p subname $x
```

subscript

`subscript` — operation `subscript`. Category: **encoding / phonetics**.

```
p subscript $x
```

subseqs

`subseqs` — operation `subseqs`. Alias for `subsequences`. Category: **additional missing stdlib functions**.

```
p subseqs $x
```


subsequences

`subsequences` — operation `subsequences`. Category: *additional missing stdlib functions*.

```
p subsequences $x
```

subset_construction

`subset_construction` — operation `subset construction`. Category: *compilers / parsing*.

```
p subset_construction $x
```

substitution_encrypt

`substitution_encrypt` — encrypt of `substitution`. Category: *cryptography deep*.

```
p substitution_encrypt $x
```

substring

`substring` — operation `substring`. Category: *string helpers*.

```
p substring $x
```

substring_count

`substring_count` — count of `substring`. Category: *misc*.

```
p substring_count $x
```

substring_similarity

`substring_similarity` — similarity metric of `substring`. Category: *postgres sql strings, json, aggregates*.

```
p substring_similarity $x
```

subsumption_check

`subsumption_check` — operation `subsumption check`. Category: *logic, proof, sat/smt, type theory*.

```
p subsumption_check $x
```

succ

`succ` — operation `succ`. Category: *conversion / utility*.

```
p succ $x
```

suffix_array_naive

`suffix_array_naive` — operation `suffix array naive`. Category: *misc*.

```
p suffix_array_naive $x
```

suffix_sums

`suffix_sums` — operation `suffix sums`. Category: *list helpers*.

```
p suffix_sums $x
```

sum_by

`sum_by` — operation `sum by`. Category: **python/ruby stdlib**.

```
p sum_by $x
```

sum_degrees

`sum_degrees` — operation `sum degrees`. Category: **graph algorithms**.

```
p sum_degrees $x
```

sum_digits

`sum_digits` — operation `sum digits`. Category: **math / number theory extras**.

```
p sum_digits $x
```

sum_divisors

`sum_divisors` — operation `sum divisors`. Category: **math / numeric extras**.

```
p sum_divisors $x
```

sum_list

`sum_list` — operation `sum list`. Category: **list helpers**.

```
p sum_list $x
```

sum_of

`sum_of` — operation `sum of`. Category: **more list helpers**.

```
p sum_of $x
```

sum_squares

`sum_squares` — operation `sum squares`. Category: **extended stdlib**.

```
p sum_squares $x
```

sum_two_squares

`sum_two_squares` — operation `sum two squares`. Category: **cryptanalysis & number theory deep**.

```
p sum_two_squares $x
```

sumb

`sumb` — operation `sumb`. Alias for `sum_by`. Category: **python/ruby stdlib**.

```
p sumb $x
```

sumif

`sumif` — operation `sumif`. Category: **excel/sheets + bond/loan financial**.

```
p sumif $x
```

sumifs

`sumifs` — operation `sumifs`. Category: *excel/sheets + bond/loan financial*.

```
p sumifs $x
```

sumproduct

`sumproduct` — operation `sumproduct`. Category: *excel/sheets + bond/loan financial*.

```
p sumproduct $x
```

sumsq

`sumsq` — operation `sumsq`. Alias for `sum_squares`. Category: *extended stdlib*.

```
p sumsq $x
```

sun_distance_au

`sun_distance_au` — solar astronomy helper `distance au`. Category: *astronomy / astrometry*.

```
p sun_distance_au $x
```

sun_position_low

`sun_position_low` — solar astronomy helper `position low`. Category: *astronomy / astrometry*.

```
p sun_position_low $x
```

sun_radius

`sun_radius` — solar astronomy helper `radius`. Category: *physics constants*.

```
p sun_radius $x
```

sun_rise_unix

`sun_rise_unix(lat, lon, unix_today) → int` — sunrise unix timestamp at given coords for that date. Uses solar declination from day-of-year, hour angle from $\text{acos}(-\tan(\phi) \cdot \tan(\delta))$, and equation-of-time correction. Returns UNDEF for polar day/night ($|\tan(\phi) \cdot \tan(\delta)| > 1$).

```
p sun_rise_unix(37.7749, -122.4194, time())
```

sun_set_unix

`sun_set_unix(lat, lon, unix_today) → int` — sunset unix timestamp at given coords for that date. Same model as `sun_rise_unix` with EoT correction; returns UNDEF for polar day/night.

```
p sun_set_unix(37.7749, -122.4194, time())
```

sunion

`sunion` — operation `sunion`. Category: *redis-flavour primitives*.

```
p sunion $x
```

sunny_16_rule

`sunny_16_rule(iso=100)` → {aperture, shutter, iso} — Sunny 16: f/16 at 1/ISO seconds in bright sun.

```
p sunny_16_rule(100) # {aperture: 16, shutter: 0.01, iso: 100}
```

sunspot_relative_number

`sunspot_relative_number` — operation `sunspot relative number`. Category: *climate, fluids, atmospheric*.

```
p sunspot_relative_number $x
```

sup

`sup` — operation `sup`. Alias for `superscript`. Category: *encoding / phonetics*.

```
p sup $x
```

super_factorial

`super_factorial(n)` → int — $1! \cdot 2! \cdot \dots \cdot n!$.

superbee_limiter

`superbee_limiter` — operation `superbee limiter`. Category: *ode advanced*.

```
p superbee_limiter $x
```

superdense_coding

`superdense_coding` — operation `superdense coding`. Category: *quantum*.

```
p superdense_coding $x
```

supergolden

`supergolden` — operation `supergolden`. Alias for `supergolden_ratio`. Category: *math constants*.

```
p supergolden $x
```

supergolden_ratio

`supergolden_ratio` — ratio of `supergolden`. Category: *math constants*.

```
p supergolden_ratio $x
```

superscript

`superscript` — operation `superscript`. Category: *encoding / phonetics*.

```
p superscript $x
```

supply_elasticity

`supply_elasticity` — operation `supply elasticity`. Category: *economics + game theory*.

```
p supply_elasticity $x
```

support_level

`support_level(prices, period=20)` → `number` — rolling min over `period`; the most recent local floor.

```
p support_level(\@closes, 20)
```

surf_keypoints

`surf_keypoints` — operation `surf keypoints`. Category: **pil/opencv image processing**.

```
p surf_keypoints $x
```

surface_wave_ms

`surface_wave_ms` — operation `surface wave ms`. Category: **geology, seismology, mineralogy**.

```
p surface_wave_ms $x
```

susceptible_to_infected

`susceptible_to_infected` — convert from `susceptible` to `infected`. Category: **epidemiology / public health**.

```
p susceptible_to_infected $value
```

SW

`sw` — operation `sw`. Alias for `starts_with`. Category: **trivial string ops**.

```
p sw $x
```

SW_SCORE

`sw_score(a, b, match=2, mismatch=-1, gap=-2)` → `number` — Smith-Waterman local alignment score (max non-negative cell in DP matrix).

```
p sw_score("XXACGTTY", "ZZACGTWW")
```

swamee_jain_factor

`swamee_jain_factor` — factor of `swamee jain`. Category: **misc**.

```
p swamee_jain_factor $x
```

swampland_distance_check

`swampland_distance_check` — operation `swampland distance check`. Category: **electrochemistry, batteries, fuel cells**.

```
p swampland_distance_check $x
```

swap_bits

`swap_bits` — operation `swap bits`. Alias for `swap_bits_pos`. Category: **misc**.

```
p swap_bits $x
```

swap_bits_pos

`swap_bits_pos` — operation `swap bits pos`. Category: **misc**.

```
p swap_bits_pos $x
```

swap_case

`swap_case` — operation `swap case`. Category: **trivial string ops**.

```
p swap_case $x
```

swap_free

`swap_free` — free swap space in bytes.

```
p format_bytes(swap_free)
var $pct = swap_total > 0 ? int(swap_free / swap_total * 100) : 100
p "$pct% swap free"
```

swap_indices

`swap_indices` — operation `swap indices`. Category: **misc**.

```
p swap_indices $x
```

swap_pairs

`swap_pairs` — operation `swap pairs`. Category: **collection helpers (trivial)**.

```
p swap_pairs $x
```

swap_total

`swap_total` — total swap space in bytes. Linux: `/proc/meminfo SwapTotal`. macOS: `vm.swapusage` sysctl. Returns 0 if swap is disabled, `undef` on unsupported platforms.

```
p swap_total                # 8589934592
p format_bytes(swap_total)   # 8.0 GiB
```

swap_used

`swap_used` — used swap space in bytes (total - free).

```
p format_bytes(swap_used)
p "swap pressure" if swap_used > swap_total * 0.8
```

swave_velocity_depth

`swave_velocity_depth` — operation `swave velocity depth`. Category: **geology, seismology, mineralogy**.

```
p swave_velocity_depth $x
```

swelling_strain_step

`swelling_strain_step` — single-step update of `swelling strain`. Category: **electrochemistry, batteries, fuel cells**.

```
p swelling_strain_step $x
```

swiss_pairing

`swiss_pairing(scores, played) → array` — greedy Swiss-system pairings; avoids rematches.

```
p swiss_pairing([3, 3, 2, 2, 1, 1], $played)
```

sym4_coeffs

`sym4_coeffs` — operation `sym4 coeffs`. Category: **more extensions**.

```
p sym4_coeffs $x
```

sym_check

`sym_check` — operation `sym check`. Category: **logic, proof, sat/smt, type theory**.

```
p sym_check $x
```

sym_links

`sym_links` — operation `sym links`. Category: **filesystem extensions**.

```
p sym_links $x
```

symmetric_diff

`symmetric_diff` — operation `symmetric diff`. Category: **collection**.

```
p symmetric_diff $x
```

symplectic_euler_step

`symplectic_euler_step` — single-step update of `symplectic euler`. Category: **ode advanced**.

```
p symplectic_euler_step $x
```

synge_world_function

`synge_world_function` — operation `synge world function`. Category: **electrochemistry, batteries, fuel cells**.

```
p synge_world_function $x
```

synodic_period

`synodic_period` — operation `synodic period`. Category: **misc**.

```
p synodic_period $x
```

sys_uptime

`sys_uptime` — system uptime in seconds (float). On Linux reads `/proc/uptime`. On macOS uses `kern.boottime` sysctl. This is the wall-clock uptime of the machine, not the stryke process (see `uptime_secs` for process uptime).

```
p sys_uptime # 147951.5
var $days = int(sys_uptime / 86400)
p "up $days days"
```

t1ha

`t1ha` — operation `t1ha`. Category: *b81-misc-utility*.

```
p t1ha $x
```

t_digest

`t_digest(COMPRESSSION=100)` — streaming-quantile sketch (Dunning). Larger compression → more centroids, more accuracy, linear extra memory. Default `100` gives ~1% error at p50, ~5% at p99 in O(100) bytes. Pending-buffer + flush-on-query so per-`td_add` is amortized O(1).

```
val $td = t_digest(200)
td_add($td, $_) for @latencies_us
printf "p99: %.1f us\n", td_quantile($td, 0.99)
```

Family: `td_add`, `td_quantile`, `td_count`, `td_min`, `td_max`, `td_sum`, `td_mean`, `td_merge`, `td_clear`, `td_serialize`, `td_deserialize`. `td` short alias is reserved for `toml_decode`; use `tdg`.

t_test_one_sample

`t_test_one_sample($mu, @sample)` (alias `ttest1`) — performs a one-sample t-test comparing a sample mean against a hypothesized population mean `mu`. Returns [`t_statistic`, `degrees_of_freedom`]. Use t-distribution tables for p-value lookup.

```
val @sample = (5.1, 4.9, 5.2, 5.0, 4.8)
var $result = ttest1(5.0, @sample)
p "t=$result->[0] df=$result->[1]"
```

t_test_paird

`t_test_paird(a, b) → {statistic, df, p_value}` — paired t-test on equal-length samples; tests $H_0: \text{mean}(a-b) = 0$.

```
p t_test_paird(\@before, \@after)
```

t_test_two_sample

`t_test_two_sample(\@a, \@b)` (alias `ttest2`) — performs an independent two-sample t-test comparing means of two samples. Assumes equal variances. Returns [`t_statistic`, `degrees_of_freedom`].

```
val @a = (5.1, 5.3, 4.9)
var @b = (4.2, 4.5, 4.1)
var $r = ttest2(\@a, \@b)
p "t=$r->[0] df=$r->[1]"
```

tableau_branch_close

`tableau_branch_close` — close handle of `tableau branch`. Category: *logic, proof, sat/smt, type theory*.

```
p tableau_branch_close $x
```

tabu_search_move_score

`tabu_search_move_score` — score of `tabu search move`. Category: *combinatorial optimization, scheduling*.

```
p tabu_search_move_score $x
```

tabulate

`tabulate` — frequency table of values. Returns hash of value => count. Like R's `table()`.


```
val %t = %{tabulate([qw(a b a c b a)])}  
p ${t{a}} # 3
```

tackle_efficiency

tackle_efficiency — operation [tackle efficiency](#). Category: **astronomy / astrometry**.

```
p tackle_efficiency $x
```

tafel_anodic_current

tafel_anodic_current — operation [tafel anodic current](#). Category: **electrochemistry, batteries, fuel cells**.

```
p tafel_anodic_current $x
```

tafel_cathodic_current

tafel_cathodic_current — operation [tafel cathodic current](#). Category: **electrochemistry, batteries, fuel cells**.

```
p tafel_cathodic_current $x
```

tail_lines

tail_lines — operation [tail lines](#). Category: **file stat / path**.

```
p tail_lines $x
```

tail_n

tail_n — operation [tail n](#). Category: **list helpers**.

```
p tail_n $x
```

tail_str

tail_str — operation [tail str](#). Alias for [right_str](#). Category: **string**.

```
p tail_str $x
```

tails

tails — operation [tails](#). Category: **additional missing stdlib functions**.

```
p tails $x
```

tajimas_d

tajimas_d — operation [tajimas d](#). Category: **bioinformatics deep**.

```
p tajimas_d $x
```

take_every

take_every — operation [take every](#). Category: **list helpers**.

```
p take_every $x
```

take_last

`take_last` — operation `take last`. Category: **file stat / path**.

```
p take_last $x
```

take_n

`take_n` — operation `take n`. Category: **collection helpers (trivial)**.

```
p take_n $x
```

take_n_random

`take_n_random` — operation `take n random`. Category: **iterator + string-distance extras**.

```
p take_n_random $x
```

talb

`talb` — operation `talb`. Alias for `tally_by`. Category: **ruby enumerable extras**.

```
p talb $x
```

tally_by

`tally_by` — operation `tally by`. Category: **ruby enumerable extras**.

```
p tally_by $x
```

tan

`tan` — operation `tan`. Category: **trig / math**.

```
p tan $x
```

tanh

`tanh` — operation `tanh`. Category: **trig / math**.

```
p tanh $x
```

tanh_grad

`tanh_grad` — operation `tanh grad`. Category: **ml extensions**.

```
p tanh_grad $x
```

tanimoto_bits

`tanimoto_bits` — operation `tanimoto bits`. Category: **bioinformatics deep**.

```
p tanimoto_bits $x
```

tanimoto_coeff

`tanimoto_coeff(a, b) → number` — Tanimoto on numeric vectors: $a \cdot b / (|a|^2 + |b|^2 - a \cdot b)$.

tap_debug

tap_debug — operation `tap debug`. Category: **conversion / utility**.

```
p tap_debug $x
```

tap_val

tap_val — operation `tap val`. Category: **functional combinators**.

```
p tap_val $x
```

tapply

tapply — apply a function by group. Takes data, group labels, and function. Returns hash. Like R's `tapply()`.

```
val %means = %{tapply([1,2,3,4], ["a","a","b","b"], fn { avg(@{_0}) })}
p $means{a}  # 1.5
p $means{b}  # 3.5
```

tar_header_checksum

tar_header_checksum — operation `tar header checksum`. Category: **archive/encoding format primitives**.

```
p tar_header_checksum $x
```

tar_header_read

tar_header_read(hex_bytes) → hash — parse ustar header → {name, size} (octal size decoded).

tar_member_record

tar_member_record — operation `tar member record`. Category: **archive/encoding format primitives**.

```
p tar_member_record $x
```

tar_pad_512

tar_pad_512 — operation `tar pad 512`. Category: **archive/encoding format primitives**.

```
p tar_pad_512 $x
```

target_heart_rate

target_heart_rate — rate of `target heart`. Category: **misc**.

```
p target_heart_rate $x
```

tarjan_scc_step

tarjan_scc_step — single-step update of `tarjan scc`. Category: **networkx graph algorithms**.

```
p tarjan_scc_step $x
```

tau

tau — operation `tau`. Category: **constants**.

```
p tau $x
```

tax

`tax` — operation `tax`. Category: *finance*.

```
p tax $x
```

tax_amount

`tax_amount` — operation `tax amount`. Category: *physics formulas*.

```
p tax_amount $x
```

tax_incidence

`tax_incidence` — operation `tax incidence`. Category: *economics + game theory*.

```
p tax_incidence $x
```

taylor_diagram_score

`taylor_diagram_score` — score of `taylor diagram`. Category: *excel/sheets + bond/loan financial*.

```
p taylor_diagram_score $x
```

taylor_microscale_step

`taylor_microscale_step` — single-step update of `taylor microscale`. Category: *climate, fluids, atmospheric*.

```
p taylor_microscale_step $x
```

taylor_window

`taylor_window` — operation `taylor window`. Category: *economics + game theory*.

```
p taylor_window $x
```

tcrossprod

`tcrossprod` — $M * t(M)$ (R's `tcrossprod`).

```
val $aat = tcrossprod([[1,2],[3,4]]) # [[5,11],[11,25]]
```

td

`td` — operation `td`. Alias for `toml_decode`. Category: *serialization (stryke-only encoders)*.

```
p td $x
```

td_add

`td_add(TD, VALUE)` — insert a single sample. Non-finite values silently dropped.

td_clear

`td_clear` — operation `td clear`. Category: *probabilistic data structures*.

```
p td_clear $x
```

td_count

`td_count(TD)` → number of samples inserted so far.

td_deserialize

`td_deserialize` — operation `td deserialize`. Category: *probabilistic data structures*.

```
p td_deserialize $x
```

td_from_bytes

`td_from_bytes` — operation `td from bytes`. Alias for `td_deserialize`. Category: *probabilistic data structures*.

```
p td_from_bytes $x
```

td_lambda_update

`td_lambda_update` — update step of `td lambda`. Category: *electrochemistry, batteries, fuel cells*.

```
p td_lambda_update $x
```

td_max

`td_max(TD)` — exact maximum sample seen.

td_mean

`td_mean(TD)` — exact mean of all inserted samples.

td_merge

`td_merge(TD, OTHER)` — combine two t-digests via `tdigest::TDigest::merge_digests`.

td_min

`td_min(TD) / td_max(TD)` — exact min/max (t-digest tracks both).

td_quantile

`td_quantile(TD, Q)` → estimated Q -th quantile ($0 \leq Q \leq 1$).

td_reset

`td_reset` — operation `td reset`. Alias for `td_clear`. Category: *probabilistic data structures*.

```
p td_reset $x
```

td_serialize

`td_serialize` — operation `td serialize`. Category: *probabilistic data structures*.

```
p td_serialize $x
```

td_sum

`td_sum(TD)` — exact sum of all inserted samples.

td_to_bytes

`td_to_bytes` — convert from `td` to `bytes`. Alias for `td_serialize`. Category: **probabilistic data structures**.

```
p td_to_bytes $value
```

td_zero_update

`td_zero_update` — update step of `td zero`. Category: **electrochemistry, batteries, fuel cells**.

```
p td_zero_update $x
```

tdilate

`time_dilation($proper_time, $velocity)` (alias `tdilate`) — dilated time $\Delta t = \Delta t_0 \times \gamma$. Time appears to slow for moving objects.

```
p tdilate(1, 0.99 * 3e8) # ~7.09 s (1 second at 99% c)
```

te

`te` — operation `te`. Alias for `toml_encode`. Category: **serialization (stryke-only encoders)**.

```
p te $x
```

tee_iter

`tee_iter` — operation `tee iter`. Category: **python/ruby stdlib**.

```
p tee_iter $x
```

teei

`teei` — operation `teei`. Alias for `tee_iter`. Category: **python/ruby stdlib**.

```
p teei $x
```

tema

`tema(prices, period=10) → array` — triple EMA: $3 \cdot \text{EMA} - 3 \cdot \text{EMA}(\text{EMA}) + \text{EMA}(\text{EMA}(\text{EMA}))$. Designed to remove EMA lag with no extra phase shift.

```
p tema(@prices, 21)
```

temp_dir

`temp_dir` — operation `temp dir`. Category: **system introspection**.

```
p temp_dir $x
```

template_match

`template_match` — operation `template match`. Category: **pill/opencv image processing**.

```
p template_match $x
```

tempo_to_ms

`tempo_to_ms` — convert from `tempo` to `ms`. Category: **music theory**.

```
p tempo_to_ms $value
```

tempo_to_ms_per_beat

`tempo_to_ms_per_beat(bpm)` → `number` — $60000 / \text{bpm}$. Common for sequencer math.

```
p tempo_to_ms_per_beat(120) # 500
```

tempplanck

`tempplanck` — operation `tempplanck`. Alias for `planck_temperature`. Category: **physics constants**.

```
p tempplanck $x
```

tensor_antisymmetrize_two

`tensor_antisymmetrize_two` — operation `tensor antisymmetrize two`. Category: **electrochemistry, batteries, fuel cells**.

```
p tensor_antisymmetrize_two $x
```

tensor_contract_two

`tensor_contract_two` — operation `tensor contract two`. Category: **electrochemistry, batteries, fuel cells**.

```
p tensor_contract_two $x
```

tensor_outer_two

`tensor_outer_two` — operation `tensor outer two`. Category: **electrochemistry, batteries, fuel cells**.

```
p tensor_outer_two $x
```

tensor_symmetrize_two

`tensor_symmetrize_two` — operation `tensor symmetrize two`. Category: **electrochemistry, batteries, fuel cells**.

```
p tensor_symmetrize_two $x
```

tensor_trace_index

`tensor_trace_index` — index of `tensor trace`. Category: **electrochemistry, batteries, fuel cells**.

```
p tensor_trace_index $x
```

term_life_a_n_t

`term_life_a_n_t` — operation `term life a n t`. Category: **actuarial science**.

```
p term_life_a_n_t $x
```

terminal_velocity

`terminal_velocity` — operation `terminal velocity`. Category: **misc**.

```
p terminal_velocity $x
```

ternary_search

`ternary_search(sorted, target) → int` — divides search range into thirds; $O(\log n)$ like binary search.

```
p ternary_search([1,3,5,7,9,11,13], 9) # 4
```

test_skip

`test_skip` — testing helper `skip`. Category: *testing framework*.

```
p test_skip $x
```

tetrahedral

`tetrahedral(n) → int` — $n \cdot (n+1) \cdot (n+2) / 6$.

tetrahedral_number

`tetrahedral_number` — operation `tetrahedral number`. Category: *misc*.

```
p tetrahedral_number $x
```

tetranacci

`tetranacci(n) → int` — like Fibonacci but summing the previous four terms.

text_after

`text_after` — text helper `after`. Category: *extended stdlib*.

```
p text_after $x
```

text_after_last

`text_after_last` — text helper `after last`. Category: *extended stdlib*.

```
p text_after_last $x
```

text_before

`text_before` — text helper `before`. Category: *extended stdlib*.

```
p text_before $x
```

text_before_last

`text_before_last` — text helper `before last`. Category: *extended stdlib*.

```
p text_before_last $x
```

text_between

`text_between` — text helper `between`. Category: *extended stdlib*.

```
p text_between $x
```


tf2sos_step

`tf2sos_step` — single-step update of `tf2sos`. Category: **economics + game theory**.

```
p tf2sos_step $x
```

tf2zpk_step

`tf2zpk_step` — single-step update of `tf2zpk`. Category: **economics + game theory**.

```
p tf2zpk_step $x
```

tfidf_compute

`tfidf_compute(docs) → {vocab, matrix}` — TF-IDF matrix `[doc][term]`. Each row is a document's TF-IDF vector.

```
my $r = tfidf_compute([["the","cat"], ["a","dog"]])
```

tfld

`tfld` — operation `tfld`. Alias for `try_fold`. Category: **rust iterator methods**.

```
p tfld $x
```

theil_sen_slope

`theil_sen_slope` — operation `theil sen slope`. Category: **more extensions**.

```
p theil_sen_slope $x
```

then_fn

`then_fn` — operation `then fn`. Category: **python/ruby stdlib**.

```
p then_fn $x
```

thermal_mean_photons

`thermal_mean_photons` — operation `thermal mean photons`. Category: **quantum mechanics deep**.

```
p thermal_mean_photons $x
```

thermal_runaway_threshold

`thermal_runaway_threshold` — operation `thermal runaway threshold`. Category: **electrochemistry, batteries, fuel cells**.

```
p thermal_runaway_threshold $x
```

thermal_wind_step

`thermal_wind_step` — single-step update of `thermal wind`. Category: **climate, fluids, atmospheric**.

```
p thermal_wind_step $x
```

theta2

`theta2` — operation `theta2`. Category: **special functions extra**.

```
p theta2 $x
```

theta3

theta3 — operation `theta3`. Category: **special functions extra**.

```
p theta3 $x
```

thfn

thfn — operation `thfn`. Alias for `then_fn`. Category: **python/ruby stdlib**.

```
p thfn $x
```

thin_lens

`thin_lens($object_dist, $focal_length)` (alias `thinlens`) — image distance using $1/f = 1/do + 1/di$.

```
p thinlens(30, 20) # 60 cm (real image)
p thinlens(10, 20) # -20 cm (virtual image)
```

thin_lens_v

thin_lens_v — operation `thin lens v`. Category: **em / optics / relativity**.

```
p thin_lens_v $x
```

third_elem

third_elem — operation `third elem`. Category: **list helpers**.

```
p third_elem $x
```

thompson_beta_choose

`thompson_beta_choose(alphas, betas, seed=0) → int` — Thompson sampling from per-arm Beta posteriors.

thompson_sample_beta

thompson_sample_beta — operation `thompson sample beta`. Category: **electrochemistry, batteries, fuel cells**.

```
p thompson_sample_beta $x
```

three_opt_delta

three_opt_delta — operation `three opt delta`. Category: **combinatorial optimization, scheduling**.

```
p three_opt_delta $x
```

threshold_act

threshold_act — operation `threshold act`. Category: **ml extensions**.

```
p threshold_act $x
```

threshold_hard_value

`threshold_hard_value` — value of `threshold hard`. Category: *electrochemistry, batteries, fuel cells*.

```
p threshold_hard_value $x
```

threshold_soft_value

`threshold_soft_value` — value of `threshold soft`. Category: *electrochemistry, batteries, fuel cells*.

```
p threshold_soft_value $x
```

thue_morse_string

`thue_morse_string` — operation `thue morse string`. Category: *more extensions*.

```
p thue_morse_string $x
```

ti2

`ti2` — operation `ti2`. Category: *special functions extra*.

```
p ti2 $x
```

tidal_force

`tidal_force` — operation `tidal force`. Category: *cosmology / gr / flrw*.

```
p tidal_force $x
```

tied

`tied` — operation `tied`. Category: *type / reflection*.

```
p tied $x
```

tile_xyz_to_lat_lng

`tile_xyz_to_lat_lng` — convert from `tile xyz` to `lat lng`. Category: *excel/sheets + bond/loan financial*.

```
p tile_xyz_to_lat_lng $value
```

time_sig_subdivision

`time_sig_subdivision` — operation `time sig subdivision`. Category: *music theory*.

```
p time_sig_subdivision $x
```

times_fn

`times_fn` — operation `times fn`. Category: *python/ruby stdlib*.

```
p times_fn $x
```

timf

`timf` — operation `timf`. Alias for `times_fn`. Category: *python/ruby stdlib*.

```
p timf $x
```

tip

`tip` — operation `tip`. Category: *finance*.

```
p tip $x
```

tip_amount

`tip_amount` — operation `tip amount`. Category: *physics formulas*.

```
p tip_amount $x
```

tisserand_param

`tisserand_param` — operation `tisserand param`. Category: *astronomy / astrometry*.

```
p tisserand_param $x
```

tit_for_tat_step

`tit_for_tat_step` — single-step update of `tit for tat`. Category: *climate, fluids, atmospheric*.

```
p tit_for_tat_step $x
```

title

`title` — operation `title`. Alias for `title_case`. Category: *trivial string ops*.

```
p title $x
```

title_case

`title_case` — operation `title case`. Category: *trivial string ops*.

```
p title_case $x
```

title_words

`title_words` — operation `title words`. Alias for `capitalize_words`. Category: *misc*.

```
p title_words $x
```

titration_endpoint_index

`titration_endpoint_index` — index of `titration endpoint`. Category: *electrochemistry, batteries, fuel cells*.

```
p titration_endpoint_index $x
```

titration_ph_endpoint

`titration_ph_endpoint` — operation `titration ph endpoint`. Category: *chemistry & biochemistry*.

```
p titration_ph_endpoint $x
```

titration_volume

`titration_volume` — operation `titration volume`. Category: *misc*.

```
p titration_volume $x
```

tkeys

`tkeys` — operation `tkeys`. Alias for `transform_keys`. Category: **python/ruby stdlib**.

```
p tkeys $x
```

tle_mean_motion

`tle_mean_motion` — operation `tle mean motion`. Category: **astronomy / astrometry**.

```
p tle_mean_motion $x
```

tm_marmur

`tm_marmur` — operation `tm marmur`. Category: **bioinformatics deep**.

```
p tm_marmur $x
```

tm_wallace

`tm_wallace` — operation `tm wallace`. Category: **bioinformatics deep**.

```
p tm_wallace $x
```

to_array

`to_array` — convert input to array form. Category: **list helpers**.

```
p to_array $x
```

to_ascii

`to_ascii` — convert input to ascii form. Category: **extended stdlib**.

```
p to_ascii $x
```

to_base

`to_base` — convert input to base form. Category: **base conversion**.

```
p to_base $x
```

to_bin

`to_bin` — convert input to bin form. Category: **base conversion**.

```
p to_bin $x
```

to_bool

`to_bool` — convert input to bool form. Category: **conversion / utility**.

```
p to_bool $x
```

to_csv_line

`to_csv_line` — convert input to csv line form. Category: **string helpers**.

```
p to_csv_line $x
```

to_emoji_num

`to_emoji_num` — convert input to emoji num form. Category: *encoding / phonetics*.

```
p to_emoji_num $x
```

to_float

`to_float` — convert input to float form. Category: *conversion / utility*.

```
p to_float $x
```

to_float_each

`to_float_each` — convert input to float each form. Category: *trivial numeric helpers*.

```
p to_float_each $x
```

to_hash

`to_hash LIST` — collect a flat list of key-value pairs into a Perl hash.

The list is consumed two elements at a time: odd-positioned elements become keys and even-positioned elements become values. If there is an odd number of elements, the last key maps to `undef`. This is a terminal operation that materializes the full stream. Useful for converting pipeline output into a lookup table.

```
val %h = qw(a 1 b 2) |> to_hash
@pairs |> to_hash
var %freq = @words |> map { _, 1 } |> to_hash
```

to_hex

`to_hex` — convert input to hex form. Category: *base conversion*.

```
p to_hex $x
```

to_ini

Serialize a hashref to INI text. Scalar top-level entries become global `key=value` lines (emitted first); nested hashref entries become `[section]` blocks. Inverse of `from_ini`.

```
print to_ini({debug => 1, server => {port => 8080}})
# debug=1
# [server]
# port=8080
```

to_int

`to_int` — convert input to int form. Category: *conversion / utility*.

```
p to_int $x
```

to_int_each

`to_int_each` — convert input to int each form. Category: *trivial numeric helpers*.

```
p to_int_each $x
```

to_jsonl

Serialize a list to newline-delimited JSON — one compact value per line, trailing newline included. Accepts a flattened list or a single arrayref. Inverse of `from_jsonl`.

```
val @rows = ({id => 1}, {id => 2})
print to_jsonl(@rows)
# {"id":1}
# {"id":2}
```

to_oct

`to_oct` — convert input to oct form. Category: **base conversion**.

```
p to_oct $x
```

to_pairs

`to_pairs` — convert input to pairs form. Category: **list helpers**.

```
p to_pairs $x
```

to_reversed

`to_reversed` — convert input to reversed form. Category: **javascript array/object methods**.

```
p to_reversed $x
```

to_set

`to_set LIST` — collect a list or iterator into a `set` object with O(1) membership testing.

The resulting set contains only unique elements (duplicates are discarded). This is a terminal operation that materializes the full stream. The returned set supports `->contains($val)`, `->union($other)`, `->intersection($other)`, and `->difference($other)`. Use this when you need fast repeated lookups against a collection of values.

```
val $s = 1:5 |> to_set
@words |> to_set           # deduplicated set
var $allowed = to_set @whitelist
p $allowed->contains("foo")
```

to_sorted

`to_sorted` — convert input to sorted form. Category: **javascript array/object methods**.

```
p to_sorted $x
```

to_spliced

`to_spliced` — convert input to spliced form. Category: **javascript array/object methods**.

```
p to_spliced $x
```

to_string

`to_string` — convert input to string form. Category: **conversion / utility**.

```
p to_string $x
```

to_string_val

`to_string_val` — convert input to string val form. Category: *misc / utility*.

```
p to_string_val $x
```

toasc

`toasc` — operation `toasc`. Alias for `to_ascii`. Category: *extended stdlib*.

```
p toasc $x
```

tobit_log_likelihood

`tobit_log_likelihood` — operation `tobit log likelihood`. Category: *econometrics*.

```
p tobit_log_likelihood $x
```

today

`today` — operation `today`. Category: *date*.

```
p today $x
```

todd_genus_eval

`todd_genus_eval` — operation `todd genus eval`. Category: *econometrics*.

```
p todd_genus_eval $x
```

toffoli_gate

`toffoli_gate` — operation `toffoli gate`. Category: *more extensions*.

```
p toffoli_gate $x
```

token

`token` — operation `token`. Category: *id helpers*.

```
p token $x
```

token_bucket

`token_bucket(CAPACITY=100, RATE=10)` — token-bucket rate limiter. `CAPACITY` is the max burst; `RATE` (tokens/sec) is the steady-state allowance. Refills lazily on each `rl_try_take` based on wall-clock — no background thread. Stateful upgrade over the deleted `db_token_bucket_step` spec primitive.

```
val $rl = token_bucket(100, 50)      # 50 req/s, burst 100
if (rl_try_take($rl, 1)) { ... } else { sleep(0.02) }
```

tokenize_bpe

`tokenize_bpe(s, merges) → array` — Byte-Pair Encoding tokenisation against a merge table.

tokenize_sentencepiece

`tokenize_sentencepiece(s) → array` — SentencePiece-like tokenisation; prepends `_` to word starts.


```
p tokenize_sentencepiece("hello world") # ["_hello", "_world"]
```

tokenize_simple

`tokenize_simple(s)` → `array` — split on whitespace and basic punctuation.

tokenize_subword

`tokenize_subword(s, max_len=4)` → `array` — naive subword tokenisation: splits on non-alphanumeric boundaries, then chunks long words into pieces of at most `max_len` characters.

tokenize_word

`tokenize_word(s)` → `array` — whitespace tokeniser.

tomorrow

`tomorrow` — operation `tomorrow`. Category: `*date*`.

```
p tomorrow $x
```

top

`top` — operation `top`. Category: `*pipeline / string helpers*`.

```
p top $x
```

top_hat_transform

`top_hat_transform(image, size=3)` → `matrix` — `image - opening`. Extracts bright details smaller than the structuring element.

top_k

`topk(K=10)` — SpaceSaving top-K heavy-hitters sketch. Tracks exactly `K` slots; on overflow the lowest-count slot is replaced with a new key inheriting `min+1` count and the evicted count as an error floor. `topk_heavies(N)` returns `[key, count, err]` rows sorted by count desc; truth lies in `[count - err, count]`.

```
val $t = topk(10)
topk_add($t, $_) for @clickstream
var @hot = topk_heavies($t)           # top 10 by frequency
```

top_k_indices

`top_k_indices(values, k=5)` → `array` — indices of the `k` largest values, sorted descending.

```
p top_k_indices([3,1,4,1,5,9,2,6], 3) # [5, 7, 4]
```

top_k_min_heap

`top_k_min_heap` — operation `top k min heap`. Category: `*iterator + string-distance extras*`.

```
p top_k_min_heap $x
```

top_n_by

`top_n_by` — operation `top n by`. Category: *iterator + string-distance extras*.

```
p top_n_by $x
```

top_trading_cycle

`top_trading_cycle` — operation `top trading cycle`. Category: *economics + game theory*.

```
p top_trading_cycle $x
```

top_trading_cycles_step

`top_trading_cycles_step` — single-step update of `top trading cycles`. Category: *climate, fluids, atmospheric*.

```
p top_trading_cycles_step $x
```

topk_add

`topk_add(TOPK, KEY)` — observe one occurrence of `KEY`.

topk_clear

`topk_clear` — top-K sketch op `clear`. Category: *probabilistic data structures*.

```
p topk_clear $x
```

topk_count

`topk_count(TOPK, KEY)` → count of `KEY` if currently tracked; 0 if displaced.

topk_deserialize

`topk_deserialize` — top-K sketch op `deserialize`. Category: *probabilistic data structures*.

```
p topk_deserialize $x
```

topk_from_bytes

`topk_from_bytes` — top-K sketch op `from bytes`. Alias for `topk_deserialize`. Category: *probabilistic data structures*.

```
p topk_from_bytes $x
```

topk_heavies

`topk_heavies(TOPK, N=K)` → list of `[key, count, error_floor]` rows sorted by count desc.

topk_indices

`topk_indices` — top-K sketch op `indices`. Category: *ml extensions*.

```
p topk_indices $x
```

topk_merge

`topk_merge(TOPK, OTHER)` — replay `OTHER`'s observations through the standard online update. Cost is $O(\text{sum_of_counts})$.

topk_reset

`topk_reset` — top-K sketch op `reset`. Alias for `topk_clear`. Category: *probabilistic data structures*.

```
p topk_reset $x
```

topk_serialize

`topk_serialize` — top-K sketch op `serialize`. Category: *probabilistic data structures*.

```
p topk_serialize $x
```

topk_size

`topk_size(TOPK)` → current entries ($\leq K$).

topk_to_bytes

`topk_to_bytes` — convert from `topk` to `bytes`. Alias for `topk_serialize`. Category: *probabilistic data structures*.

```
p topk_to_bytes $value
```

topo_kahn_step

`topo_kahn_step` — single-step update of `topo_kahn`. Category: *networkx graph algorithms*.

```
p topo_kahn_step $x
```

topological_sort

`topological_sort` — operation `topological sort`. Category: *extended stdlib*.

```
p topological_sort $x
```

topological_sort_kahn

`topological_sort_kahn` — operation `topological sort kahn`. Category: *graph algorithms*.

```
p topological_sort_kahn $x
```

toposort

`toposort` — operation `toposort`. Alias for `topological_sort`. Category: *extended stdlib*.

```
p toposort $x
```

torque

`torque($force, $lever_arm, $angle)` — compute torque $\tau = r \times F \times \sin(\theta)$ (N·m). Angle in degrees, default 90.

```
p torque(50, 0.5)      # 25 N·m (perpendicular force)
p torque(50, 0.5, 30)  # 12.5 N·m
```

torque_arm

`torque_arm(force, radius, angle_deg=90)` → number — $F \cdot r \cdot \sin(\theta)$.

torsion_coefficient

`torsion_coefficient` — operation `torsion coefficient`. Category: **econometrics**.

```
p torsion_coefficient $x
```

tortuosity_estimate_bruggeman

`tortuosity_estimate_bruggeman` — operation `tortuosity estimate bruggeman`. Category: **electrochemistry, batteries, fuel cells**.

```
p tortuosity_estimate_bruggeman $x
```

torus_surface

`torus_surface($major_r, $minor_r)` — computes the surface area of a torus. Returns $4\pi^2 Rr$.

```
p torus_surface(10, 2) # ~789.6
```

torus_surface_area

`torus_surface_area` — operation `torus surface area`. Category: **complex / geom / color / trig**.

```
p torus_surface_area $x
```

torus_volume

`torus_volume($major_r, $minor_r)` — computes the volume of a torus with major radius R (center to tube center) and minor radius r (tube radius). Returns $2\pi^2 Rr^2$.

```
p torus_volume(10, 2) # ~789.6
```

torussurf

`torussurf` — operation `torussurf`. Alias for `torus_surface`. Category: **geometry (extended)**.

```
p torussurf $x
```

torusvol

`torusvol` — operation `torusvol`. Alias for `torus_volume`. Category: **geometry (extended)**.

```
p torusvol $x
```

total_solar_irradiance_step

`total_solar_irradiance_step` — single-step update of `total solar irradiance`. Category: **climate, fluids, atmospheric**.

```
p total_solar_irradiance_step $x
```

total_variation_distance

`total_variation_distance` — distance / dissimilarity metric of `total variation`. Category: **electrochemistry, batteries, fuel cells**.

```
p total_variation_distance $x
```

totient_sum

`totient_sum` — Euler totient helper `sum`. Category: **math / numeric extras**.

```
p totient_sum @xs
```

totsum

`totsum` — operation `totsum`. Alias for `totient_sum`. Category: **math / numeric extras**.

```
p totsum $x
```

touch

`touch` — operation `touch`. Category: **filesystem extensions**.

```
p touch $x
```

tournament_score

`tournament_score(wins, draws, losses) → number` — `wins + 0.5·draws`.

tov_mass_limit

`tov_mass_limit` — operation `tov mass limit`. Category: **cosmology / gr / flrw**.

```
p tov_mass_limit $x
```

tower_of_hanoi

`tower_of_hanoi` — operation `tower of hanoi`. Category: **algorithms / puzzles**.

```
p tower_of_hanoi $x
```

tplanck

`tplanck` — operation `tplanck`. Alias for `planck_time`. Category: **physics constants**.

```
p tplanck $x
```

trace_distance

`trace_distance` — distance / dissimilarity metric of `trace`. Category: **more extensions**.

```
p trace_distance $x
```

trade_wind_speed

`trade_wind_speed` — operation `trade wind speed`. Category: **climate, fluids, atmospheric**.

```
p trade_wind_speed $x
```

tragedy_commons_metric

`tragedy_commons_metric` — metric of `tragedy commons`. Category: **climate, fluids, atmospheric**.

```
p tragedy_commons_metric $x
```

trailing_zeros

`trailing_zeros` — operation `trailing zeros`. Category: **base conversion**.

```
p trailing_zeros $x
```

trait

`trait` declares an interface that classes can implement via `impl`. Traits define method signatures that implementing classes must provide.

Declaration:

```
trait Printable {  
  fn to_str      # required (no body)  
  fn debug { ... } # optional default impl  
}
```

Implementation:

```
class Item impl Printable {  
  name: Str  
  fn to_str { $self->name }  
}
```

Multiple traits:

```
class Widget impl Printable, Comparable {  
  ...  
}
```

Note: trait method bodies provide default implementations; classes can override them.

trans_check

`trans_check` — operation `trans check`. Category: **logic, proof, sat/smt, type theory**.

```
p trans_check $x
```

transcribe_dna_rna

`transcribe_dna_rna` — operation `transcribe dna rna`. Category: **bioinformatics deep**.

```
p transcribe_dna_rna $x
```

transform_keys

`transform_keys` — operation `transform keys`. Category: **python/ruby stdlib**.

```
p transform_keys $x
```

transform_values

`transform_values` — operation `transform values`. Category: **python/ruby stdlib**.

```
p transform_values $x
```

transition_arc_eager

`transition_arc_eager` — operation `transition arc eager`. Category: **computational linguistics**.

```
p transition_arc_eager $x
```

transition_arc_standard

`transition_arc_standard` — operation `transition arc standard`. Category: *computational linguistics*.

```
p transition_arc_standard $x
```

transitivity

`transitivity` — operation `transitivity`. Category: *networkx graph algorithms*.

```
p transitivity $x
```

translate

`translate` — operation `translate`. Category: *postgres sql strings, json, aggregates*.

```
p translate $x
```

translate_dna

`translate_dna` — operation `translate dna`. Category: *bioinformatics deep*.

```
p translate_dna $x
```

translate_point

`translate_point($x, $y, $dx, $dy)` — translates a 2D point by offset (dx, dy). Returns [\$x+\$dx, \$y+\$dy]. Basic vector addition.

```
val $p = translate_point(1, 2, 3, 4)
p @$p # (4, 6)
```

transmission_pair_index

`transmission_pair_index` — index of `transmission pair`. Category: *epidemiology / public health*.

```
p transmission_pair_index $x
```

transmittance

`transmittance` — operation `transmittance`. Category: *chemistry*.

```
p transmittance $x
```

transmittance_to_a

`transmittance_to_a` — convert from `transmittance` to `a`. Category: *chemistry & biochemistry*.

```
p transmittance_to_a $value
```

transpose

`transpose` — operation `transpose`. Category: *collection more*.

```
p transpose $x
```

transpose_axis

`transpose_axis` — operation `transpose axis`. Category: **apl/j/k array primitives**.

```
p transpose_axis $x
```

transpose_freq_semitones

`transpose_freq_semitones` — operation `transpose freq semitones`. Category: **misc**.

```
p transpose_freq_semitones $x
```

transpose_note

`transpose_note` — operation `transpose note`. Category: **extras**.

```
p transpose_note $x
```

transpose_semi

`transpose_semi` — operation `transpose semi`. Alias for `transpose_freq_semitones`. Category: **misc**.

```
p transpose_semi $x
```

transpt

`transpt` — operation `transpt`. Alias for `translate_point`. Category: **geometry (extended)**.

```
p transpt $x
```

trapezoid

`trapz` (alias `trapezoid`) integrates evenly-spaced samples using the trapezoidal rule. Optional `dx` (default 1).

```
val @y = map { _ ** 2 } 0:100  
p trapz(\@y, 0.01) # ≈ 0.3333
```

trapezoidal_integrate

`trapezoidal_integrate(ys, dx=1) → number` — composite trapezoidal rule on pre-computed `y` samples spaced `dx` apart.

```
p trapezoidal_integrate([0,1,4,9,16], 1) # 16
```

trapezoidal_lte

`trapezoidal_lte` — operation `trapezoidal lte`. Category: **ode advanced**.

```
p trapezoidal_lte $x
```

trapped_surface_check

`trapped_surface_check` — operation `trapped surface check`. Category: **electrochemistry, batteries, fuel cells**.

```
p trapped_surface_check $x
```


trasatti_voltammetry_charge

`trasatti_voltammetry_charge` — operation `trasatti voltammetry charge`. Category: **electrochemistry, batteries, fuel cells**.

```
p trasatti_voltammetry_charge $x
```

tree

`tree(PATH=".", max_depth=5)` → recursive directory listing as a single multi-line string with box-drawing characters (├, └, ─, │). Skips hidden entries (leading `.`). Sorted alphabetically per directory.

```
p tree(".", 3) # 3-level deep tree of cwd
spurt("struct.txt", tree("/etc", 2))
```

tree_kernel_collins

`tree_kernel_collins` — operation `tree kernel collins`. Category: **computational linguistics**.

```
p tree_kernel_collins $x
```

trend_line

`trend_line(prices)` → `hash` — OLS linear regression through index (`0..n`) vs `prices`. Returns `{slope, intercept}`.

```
my $t = trend_line(@closes); p $t->{slope}
```

trev

`trev` — operation `trev`. Alias for `to_reversed`. Category: **javascript array/object methods**.

```
p trev $x
```

treynor

`treynor(portfolio_return, beta, rf=0)` → `number` — $(\text{return} - \text{rf}) / \text{beta}$. Excess return per unit of market risk.

```
p treynor(0.12, 1.2, 0.03)
```

treynor_ratio

`treynor_ratio` — ratio of `treynor`. Category: **misc**.

```
p treynor_ratio $x
```

trial_smallest_factor

`trial_smallest_factor` — factor of `trial smallest`. Category: **cryptanalysis & number theory deep**.

```
p trial_smallest_factor $x
```

triangle_area

`triangle_area` — triangle geometry op `area`. Category: **complex / geom / color / trig**.

```
p triangle_area $x
```

triangle_area_3d

`triangle_area_3d(a, b, c) → number` — area of triangle in 3D = $0.5 \cdot |AB \times AC|$.

triangle_area_heron

`triangle_area_heron` — triangle geometry op `area heron`. Category: **geometry / topology**.

```
p triangle_area_heron $x
```

triangle_area_pts

`triangle_area_pts` — triangle geometry op `area pts`. Category: **geometry / topology**.

```
p triangle_area_pts $x
```

triangle_centroid

`triangle_centroid` — triangle geometry op `centroid`. Category: **complex / geom / color / trig**.

```
p triangle_centroid $x
```

triangle_circumcircle

`triangle_circumcircle` — triangle geometry op `circumcircle`. Category: **complex / geom / color / trig**.

```
p triangle_circumcircle $x
```

triangle_circumradius

`triangle_circumradius` — triangle geometry op `circumradius`. Category: **geometry / topology**.

```
p triangle_circumradius $x
```

triangle_contains_point

`triangle_contains_point` — triangle geometry op `contains point`. Category: **complex / geom / color / trig**.

```
p triangle_contains_point $x
```

triangle_hypotenuse

`triangle_hypotenuse` — triangle geometry op `hypotenuse`. Category: **geometry / physics**.

```
p triangle_hypotenuse $x
```

triangle_incircle

`triangle_incircle` — triangle geometry op `incircle`. Category: **complex / geom / color / trig**.

```
p triangle_incircle $x
```

triangle_inradius

`triangle_inradius` — triangle geometry op `inradius`. Category: **geometry / topology**.

```
p triangle_inradius $x
```

triangle_normal

`triangle_normal(a, b, c) → [x,y,z]` — unit normal of triangle ABC via cross product.

triangles_count

`triangles_count` — count of `triangles`. Category: **networkx graph algorithms**.

```
p triangles_count $x
```

triangular_number

`triangular_number` — operation `triangular number`. Category: **number theory / primes**.

```
p triangular_number $x
```

triangular_seq

`triangular_seq` — operation `triangular seq`. Category: **sequences**.

```
p triangular_seq $x
```

trib

`trib` — operation `trib`. Alias for `tribonacci`. Category: **math / numeric extras**.

```
p trib $x
```

tribonacci

`tribonacci` — operation `tribonacci`. Category: **math / numeric extras**.

```
p tribonacci $x
```

trie_count

`trie_count(trie) → int` — number of stored words.

trie_insert

`trie_insert(trie, word) → trie` — insert word; returns trie (mutates in place but lets you chain).

```
my $t = trie_new()  
$t = trie_insert($t, "hello")
```

trie_keys

`trie_keys(trie) → array` — all words stored in the trie (empty prefix search).

trie_lookup

`trie_lookup(trie, word) → 0|1` — 1 if word is in the trie.

trie_new

`trie_new() → trie` — empty trie root: `{end: 0, children: {}}`.

trie_prefix_search

`trie_prefix_search(trie, prefix) → array` — all stored words sharing the given prefix.

```
p trie_prefix_search($t, "hel")
```

trie_remove

`trie_remove(trie, word) → trie` — clear the `end` flag for the given path; keeps structure for shared prefixes.

trigamma

`trigamma` — operation `trigamma`. Category: `*numpy + scipy.special*`.

```
p trigamma $x
```

trigger_strategy_payoff

`trigger_strategy_payoff` — operation `trigger strategy payoff`. Category: `*climate, fluids, atmospheric*`.

```
p trigger_strategy_payoff $x
```

trigram

`trigram` — operation `trigram`. Alias for `trigrams`. Category: `*string processing extras*`.

```
p trigram $x
```

trigram_perplexity

`trigram_perplexity` — operation `trigram perplexity`. Category: `*computational linguistics*`.

```
p trigram_perplexity $x
```

trigrams

`trigrams` — operation `trigrams`. Category: `*string processing extras*`.

```
p trigrams $x
```

trim_each

`trim_each` — operation `trim each`. Category: `*trivial numeric helpers*`.

```
p trim_each $x
```

trim_left

`trim_left` — operation `trim left`. Category: `*string helpers*`.

```
p trim_left $x
```

trim_right

`trim_right` — operation `trim right`. Category: `*string helpers*`.

```
p trim_right $x
```

trimmed_mean

`trimmed_mean` — arithmetic mean of `trimmed`. Category: *list helpers*.

```
p trimmed_mean @xs
```

trinomial_call

`trinomial_call` — operation `trinomial call`. Category: *financial pricing models*.

```
p trinomial_call $x
```

trinum

`trinum` — operation `trinum`. Alias for `triangular_number`. Category: *number theory / primes*.

```
p trinum $x
```

triple

`triple` — operation `triple`. Category: *trivial numeric / predicate builtins*.

```
p triple $x
```

triples

`triples` — operation `triples`. Category: *iterator + string-distance extras*.

```
p triples $x
```

trix

`trix(prices, period=15) → array` — 1-period rate-of-change of a triple EMA. Returns percent change between consecutive triple-smoothed values.

```
p trix(\@prices, 15)
```

trpo_kl_constraint

`trpo_kl_constraint` — operation `trpo kl constraint`. Category: *electrochemistry, batteries, fuel cells*.

```
p trpo_kl_constraint $x
```

true_range

`true_range(highs, lows, closes) → array` — $\max(\text{high-low}, |\text{high-prev_close}|, |\text{low-prev_close}|)$ per bar. Input to `atr`.

```
p true_range([12,13], [9,10], [10,11])
```

trueskill_match_quality

`trueskill_match_quality` — operation `trueskill match quality`. Category: *astronomy / astrometry*.

```
p trueskill_match_quality $x
```

trueskill_simple

`trueskill_simple(mu_w, sigma_w, mu_l, sigma_l) → {winner_mu, winner_sigma, loser_mu, loser_sigma}` — 1v1 TrueSkill update.

trueskill_update

`trueskill_update` — update step of `trueskill`. Category: **astronomy / astrometry**.

```
p trueskill_update $x
```

trunc

`trunc` — operation `trunc`. Category: **trivial numeric / predicate builtins**.

```
p trunc $x
```

truncate_at

`truncate_at` — operation `truncate at`. Category: **trivial string ops**.

```
p truncate_at $x
```

truncate_middle

`truncate_middle(s, width=20) → string` — truncate keeping head and tail, replacing the middle with Preserves visual ends.

```
p truncate_middle("averyverylongstring", 10)
```

truncn

`truncn` — operation `truncn`. Alias for `trunc`. Category: **trivial numeric / predicate builtins**.

```
p truncn $x
```

trust_game_repayment

`trust_game_repayment` — operation `trust game repayment`. Category: **climate, fluids, atmospheric**.

```
p trust_game_repayment $x
```

truth_table

`truth_table` — lookup table of `truth`. Category: **algorithms / puzzles**.

```
p truth_table $x
```

truthful_mechanism_check

`truthful_mechanism_check` — operation `truthful mechanism check`. Category: **climate, fluids, atmospheric**.

```
p truthful_mechanism_check $x
```

truthy_count

`truthy_count` — count of `truthy`. Category: **counters**.

```
p truthy_count $x
```

try_fn

`try_fn` — operation `try fn`. Category: **functional combinators**.

```
p try_fn $x
```

try_fold

`try_fold` — operation `try fold`. Category: **rust iterator methods**.

```
p try_fold $x
```

ts_headline

`ts_headline` — operation `ts headline`. Category: **postgres sql strings, json, aggregates**.

```
p ts_headline $x
```

ts_match

`ts_match` — operation `ts match`. Category: **postgres sql strings, json, aggregates**.

```
p ts_match $x
```

ts_rank

`ts_rank` — operation `ts rank`. Category: **postgres sql strings, json, aggregates**.

```
p ts_rank $x
```

tsirelson_bound

`tsirelson_bound` — operation `tsirelson bound`. Category: **quantum mechanics deep**.

```
p tsirelson_bound $x
```

tsp_2opt

`tsp_2opt(distance_matrix) → array` — improve a TSP tour using iterative 2-opt swaps until stable.

```
p tsp_2opt([[0,1,5,7],[1,0,3,4],[5,3,0,2],[7,4,2,0]])
```

tsp_christofides

`tsp_christofides` — Travelling Salesman heuristic `christofides`. Category: **networkx graph algorithms**.

```
p tsp_christofides $x
```

tsp_held_karp

`tsp_held_karp` — Travelling Salesman heuristic `held karp`. Category: **networkx graph algorithms**.

```
p tsp_held_karp $x
```

tsp_held_karp_step

`tsp_held_karp_step` — Travelling Salesman heuristic `held karp step`. Category: **combinatorial optimization, scheduling**.

```
p tsp_held_karp_step $x
```

tsp_lower_bound_mst

`tsp_lower_bound_mst` — Travelling Salesman heuristic `lower bound mst`. Category: **combinatorial optimization, scheduling**.

```
p tsp_lower_bound_mst $x
```

tsp_nn_step

`tsp_nn_step` — Travelling Salesman heuristic `nn step`. Category: **networkx graph algorithms**.

```
p tsp_nn_step $x
```

tspl

`tspl` — operation `tspl`. Alias for `to_spliced`. Category: **javascript array/object methods**.

```
p tspl $x
```

tsrt

`tsrt` — operation `tsrt`. Alias for `to_sorted`. Category: **javascript array/object methods**.

```
p tsrt $x
```

tstr

`tstr` — operation `tstr`. Alias for `to_string_val`. Category: **misc / utility**.

```
p tstr $x
```

tt_to_tdb

`tt_to_tdb` — convert from `tt` to `tdb`. Category: **astronomy / astrometry**.

```
p tt_to_tdb $value
```

ttable

`ttable` — operation `ttable`. Alias for `truth_table`. Category: **algorithms / puzzles**.

```
p ttable $x
```

ttl

`ttl` — operation `ttl`. Category: **redis-flavour primitives**.

```
p ttl $x
```

tty_cooked

`tty_cooked` — flip the controlling TTY back into canonical (line-buffered, echoing) mode. The default state for interactive shells.

tty_raw

`tty_raw` — flip the controlling TTY into raw mode: ICANON and ECHO off, no line buffering, no input echo. Used for interactive selectors and key-by-key input. Returns `1` on success, `0` otherwise. Pair with `tty_cooked` to restore.

```
tty_raw();
var $key = sysread(STDIN, val $buf, 1) ? $buf : undef;
tty_cooked();
```

tukey_lambda_quantile

`tukey_lambda_quantile` — operation `tukey lambda quantile`. Category: **more extensions (2)**.

```
p tukey_lambda_quantile $x
```

tukey_w

`tukey_w` — operation `tukey w`. Category: **economics + game theory**.

```
p tukey_w $x
```

tukey_window

`tukey_window` — operation `tukey window`. Category: **signal processing**.

```
p tukey_window $x
```

tunneling_prob

`tunneling_prob` — operation `tunneling prob`. Category: **quantum mechanics deep**.

```
p tunneling_prob $x
```

turaev_alexander_step

`turaev_alexander_step` — single-step update of `turaev alexander`. Category: **econometrics**.

```
p turaev_alexander_step $x
```

turbulence_intensity_step

`turbulence_intensity_step` — single-step update of `turbulence intensity`. Category: **climate, fluids, atmospheric**.

```
p turbulence_intensity_step $x
```

turbulence_noise_2d

`turbulence_noise_2d(x, y, octaves=4)` → `number` — sum of `|perlin|` octaves with halving amplitude.

turbulent_dissipation_eps

`turbulent_dissipation_eps` — operation `turbulent dissipation eps`. Category: **climate, fluids, atmospheric**.

```
p turbulent_dissipation_eps $x
```

turbulent_kinetic_energy_step

`turbulent_kinetic_energy_step` — single-step update of `turbulent kinetic energy`. Category: **climate, fluids, atmospheric**.

```
p turbulent_kinetic_energy_step $x
```

tvals

tvals — operation `tvals`. Alias for `transform_values`. Category: *python/ruby stdlib*.

```
p tvals $x
```

tversky

tversky — operation `tversky`. Category: *ml extensions*.

```
p tversky $x
```

tversky_index

tversky_index(a, b, alpha=1, beta=1) → number — asymmetric similarity; α/β weight the differences.

twap

twap(prices) → array — Time-Weighted Average Price: cumulative arithmetic mean across the series.

```
p twap(\@prices)
```

twin_primes

twin_primes — operation `twin primes`. Category: *number theory / primes*.

```
p twin_primes $x
```

twinp

twinp — operation `twinp`. Alias for `twin_primes`. Category: *number theory / primes*.

```
p twinp $x
```

twist_tensor_step

twist_tensor_step — single-step update of `twist tensor`. Category: *electrochemistry, batteries, fuel cells*.

```
p twist_tensor_step $x
```

two_opt_swap_delta

two_opt_swap_delta — operation `two opt swap delta`. Category: *combinatorial optimization, scheduling*.

```
p two_opt_swap_delta $x
```

two_sls_iv

two_sls_iv — operation `two sls iv`. Category: *econometrics*.

```
p two_sls_iv $x
```

two_watched_literals

`two_watched_literals` — operation `two watched literals`. Category: **logic, proof, sat/smt, type theory**.

```
p two_watched_literals $x
```

txaft

`txaft` — operation `txaft`. Alias for `text_after`. Category: **extended stdlib**.

```
p txaft $x
```

txaftl

`txaftl` — operation `txaftl`. Alias for `text_after_last`. Category: **extended stdlib**.

```
p txaftl $x
```

txbef

`txbef` — operation `txbef`. Alias for `text_before`. Category: **extended stdlib**.

```
p txbef $x
```

txbefl

`txbefl` — operation `txbefl`. Alias for `text_before_last`. Category: **extended stdlib**.

```
p txbefl $x
```

txbtwn

`txbtwn` — operation `txbtwn`. Alias for `text_between`. Category: **extended stdlib**.

```
p txbtwn $x
```

type

`type` — operation `type`. Alias for `type_of`. Category: **misc / utility**.

```
p type $x
```

type_each

`type_each` — operation `type each`. Category: **trivial numeric helpers**.

```
p type_each $x
```

type_of

`type_of` — operation `type of`. Category: **misc / utility**.

```
p type_of $x
```

type_redis

`type_redis` — operation `type redis`. Category: **redis-flavour primitives**.

```
p type_redis $x
```

typeof

`typeof` — operation `typeof`. Alias for `type_of`. Category: **misc / utility**.

```
p typeof $x
```

tz

`tz` — operation `tz`. Alias for `trailing_zeros`. Category: **base conversion**.

```
p tz $x
```

ubound

`ubound` — operation `ubound`. Alias for `upper_bound`. Category: **extended stdlib**.

```
p ubound $x
```

ucbl_action_value

`ucbl_action_value` — value of `ucbl action`. Category: **electrochemistry, batteries, fuel cells**.

```
p ucbl_action_value $x
```

ucbl_choose

`ucbl_choose(q_values, counts, c=√2) → int` — UCB1 selection: $Q + c \cdot \sqrt{(\ln(N)/n_i)}$. Unvisited arms get $+\infty$.

ucm_unobserved_components

`ucm_unobserved_components` — operation `ucm unobserved components`. Category: **statsmodels**.

```
p ucm_unobserved_components $x
```

ud

`ud` — operation `ud`. Alias for `url_decode`. Category: **steganography**.

```
p ud $x
```

ue

`ue` — operation `ue`. Alias for `url_encode`. Category: **steganography**.

```
p ue $x
```

ugz

`ugz` — operation `ugz`. Alias for `gunzip`. Category: **steganography**.

```
p ugz $x
```

uid

`uid` — operation `uid`. Category: **system introspection**.

```
p uid $x
```

uin

`uin` — operation `uin`. Alias for `update_in`. Category: **algebraic match**.

```
p uin $x
```

ulcer_index

`ulcer_index(prices) → number` — Peter Martin's measure of downside risk: $\sqrt{\text{mean}(\text{drawdown_pct}^2)}$. Heavier penalty on deep, lasting drawdowns than stddev.

```
p ulcer_index(\@equity_curve)
```

ulid

`ulid()` → 26-char Crockford-Base32 ULID — 48-bit ms timestamp + 80-bit random. Lexicographically sortable; monotonic within the same millisecond via a process-wide `Mutex<(last_ms, last_rand)>`. World's-first as scripting-language stdlib builtin: Python / Ruby / Node all need third-party packages. Use for trace IDs in `ai/prompt` workflows where ordered insertion matters.

```
val $trace_id = ulid()           # 01KRJ8YC8AGW5XHM2BKHJ23ZRR
is_ulid($trace_id) ? "ok" : die
p ulid_timestamp($trace_id)      # ms-since-epoch
```

ulid_timestamp

`ulid_timestamp(ULID) → 48-bit ms-since-Unix-epoch` extracted from the timestamp prefix. Returns `undef` on malformed input.

ultimatum_acceptance_prob

`ultimatum_acceptance_prob` — operation `ultimatum acceptance prob`. Category: **climate, fluids, atmospheric**.

```
p ultimatum_acceptance_prob $x
```

uncertainty_position

`uncertainty_position` — operation `uncertainty position`. Category: **quantum mechanics deep**.

```
p uncertainty_position $x
```

undef_count

`undef_count` — count of `undef`. Category: **counters**.

```
p undef_count $x
```

underline

`underline` — operation `underline`. Alias for `red`. Category: **color / ansi**.

```
p underline $x
```

unfold

`unfold` — operation `unfold`. Category: **functional combinators**.

```
p unfold $x
```

unfoldr

`unfoldr` — operation `unfoldr`. Category: *haskell list functions*.

```
p unfoldr $x
```

unfr

`unfr` — operation `unfr`. Alias for `unfoldr`. Category: *haskell list functions*.

```
p unfr $x
```

unicode_codepoints

`unicode_codepoints(s)` → `array` — array of Unicode code points (integers).

```
p unicode_codepoints("hi") # [104, 105]
```

unicode_to_native

`unicode_to_native` — convert from `unicode` to `native`. Category: *list / aggregate*.

```
p unicode_to_native $value
```

unification_robinson

`unification_robinson` — operation `unification robinson`. Category: *logic, proof, sat/smt, type theory*.

```
p unification_robinson $x
```

union_find_components

`union_find_components(uf)` → `int` — number of disjoint components.

union_find_find

`union_find_find(uf, x)` → `int` — root of x with path compression.

union_find_new

`union_find_new(n)` → {`parent`, `rank`} — disjoint-set forest of n singletons.

union_find_union

`union_find_union(uf, x, y)` → `uf` — union by rank.

```
my $uf = union_find_new(5)
$uf = union_find_union($uf, 0, 1)
```

union_list

`union_list` — operation `union list`. Category: *additional missing stdlib functions*.

```
p union_list $x
```

unionl

`unionl` — operation `unionl`. Alias for `union_list`. Category: *additional missing stdlib functions*.

```
p unionl $x
```

uniq_by

`uniq_by` — operation `uniq by`. Category: *python/ruby stdlib*.

```
p uniq_by $x
```

unique_by

`unique_by` — operation `unique by`. Category: *iterator + string-distance extras*.

```
p unique_by $x
```

unique_consecutive

`unique_consecutive` — operation `unique consecutive`. Category: *iterator + string-distance extras*.

```
p unique_consecutive $x
```

unique_count

`unique_count` — count of `unique`. Category: *numpy + scipy.special*.

```
p unique_count $x
```

unique_count_of

`unique_count_of` — operation `unique count of`. Category: *list helpers*.

```
p unique_count_of $x
```

unique_words

`unique_words` — operation `unique words`. Category: *string processing extras*.

```
p unique_words $x
```

uniqw

`uniqw` — operation `uniqw`. Alias for `unique_words`. Category: *string processing extras*.

```
p uniqw $x
```

unit_energy

`unit_energy(value, from, to) → number` — energy conversion. Units: J/joule, kJ, cal, kcal, Wh, kWh, eV, BTU.

```
p unit_energy(1, "kwh", "j") # 3.6e6
```

unit_pressure

`unit_pressure(value, from, to) → number` — pressure conversion. Units: Pa, kPa, MPa, bar, atm, torr/mmHg, psi.

unit_temperature

`unit_temperature(value, from, to)` → `number` — temperature conversion. Units: C/celsius, F/fahrenheit, K/kelvin, R/rankine.

```
p unit_temperature(100, "c", "f") # 212
```

unit_vec

`unit_vec` — unit conversion `vec`. Alias for `vec_normalize`. Category: **matrix / linear algebra**.

```
p unit_vec $x
```

unit_vector

`unit_vector` — unit conversion `vector`. Category: **misc**.

```
p unit_vector $x
```

unit_volume_metric_to_us

`unit_volume_metric_to_us(ml, unit)` → `number` — inverse: milliliters to specified US unit.

unit_volume_us_to_metric

`unit_volume_us_to_metric(value, unit)` → `number` — convert US volume unit to milliliters. Units: tsp, tbsp, fl_oz, cup, pint, quart, gallon.

```
p unit_volume_us_to_metric(1, "cup") # 236.588
```

universal_code_length

`universal_code_length` — operation `universal code length`. Category: **electrochemistry, batteries, fuel cells**.

```
p universal_code_length $x
```

universal_enveloping_dim

`universal_enveloping_dim` — operation `universal enveloping dim`. Category: **econometrics**.

```
p universal_enveloping_dim $x
```

unix_epoch

`unix_epoch` — operation `unix epoch`. Category: **now / timestamp**.

```
p unix_epoch $x
```

unix_epoch_ms

`unix_epoch_ms` — operation `unix epoch ms`. Category: **now / timestamp**.

```
p unix_epoch_ms $x
```

unix_epoch_to_iso

`unix_epoch_to_iso` — convert from `unix epoch` to `iso`. Category: **misc**.

```
p unix_epoch_to_iso $value
```


unix_to_julian

`unix_to_julian(unix)` → `number` — convert unix timestamp to Julian Date. JD 2440587.5 is unix 0.

```
p unix_to_julian(0) # 2440587.5
```

unlines

`unlines` — operation `unlines`. Category: **go/general functional utilities**.

```
p unlines $x
```

unlns

`unlns` — operation `unlns`. Alias for `unlines`. Category: **go/general functional utilities**.

```
p unlns $x
```

unquote

`unquote` — operation `unquote`. Category: **string quote / escape**.

```
p unquote $x
```

unruh_temperature_full

`unruh_temperature_full` — operation `unruh temperature full`. Category: **electrochemistry, batteries, fuel cells**.

```
p unruh_temperature_full $x
```

unscented_sigma_point

`unscented_sigma_point` — operation `unscented sigma point`. Category: **electrochemistry, batteries, fuel cells**.

```
p unscented_sigma_point $x
```

unsharp_mask_kernel

`unsharp_mask_kernel` — convolution kernel of `unsharp mask`. Category: **more extensions**.

```
p unsharp_mask_kernel $x
```

unsharp_mask_step

`unsharp_mask_step` — single-step update of `unsharp mask`. Category: **electrochemistry, batteries, fuel cells**.

```
p unsharp_mask_step $x
```

untie

`untie` — operation `untie`. Category: **type / reflection**.

```
p untie $x
```

unwds

`unwds` — operation `unwds`. Alias for `unwords`. Category: **go/general functional utilities**.

```
p unwds $x
```

unweaken

`unweaken` — operation `unweaken`. Category: **list / aggregate**.

```
p unweaken $x
```

unwords

`unwords` — operation `unwords`. Category: **go/general functional utilities**.

```
p unwords $x
```

unzip

`unzip` — operation `unzip`. Category: **collection more**.

```
p unzip $x
```

unzip3

`unzip3` — operation `unzip3`. Category: **iterator + string-distance extras**.

```
p unzip3 $x
```

unzip_archive

`unzip_archive` — operation `unzip archive`. Alias for `zip_extract`. Category: **steganography**.

```
p unzip_archive $x
```

unzip_pairs

`unzip_pairs` — operation `unzip pairs`. Category: **go/general functional utilities**.

```
p unzip_pairs $x
```

upc_check

`upc_check` — operation `upc check`. Category: **b82-misc-utility**.

```
p upc_check $x
```

upcase_each

`upcase_each` — operation `upcase each`. Category: **trivial numeric helpers**.

```
p upcase_each $x
```

upda

`upda` — operation `upda`. Alias for `update_at`. Category: **go/general functional utilities**.

```
p upda $x
```

update_at

`update_at` — operation `update at`. Category: **go/general functional utilities**.

```
p update_at $x
```

update_in

`update_in` — operation `update in`. Category: **algebraic match**.

```
p update_in $x
```

upfirdn_step

`upfirdn_step` — single-step update of `upfirdn`. Category: **economics + game theory**.

```
p upfirdn_step $x
```

upper_bound

`upper_bound` — operation `upper bound`. Category: **extended stdlib**.

```
p upper_bound $x
```

upper_snake

`upper_snake` — operation `upper snake`. Alias for `constant_case`. Category: **string**.

```
p upper_snake $x
```

upper_triangular_q

`upper_triangular_q` — operation `upper triangular q`. Category: **more extensions (2)**.

```
p upper_triangular_q $x
```

uppercase

`uppercase` — operation `uppercase`. Category: **string**.

```
p uppercase $x
```

uptime_secs

`uptime_secs` — operation `uptime secs`. Category: **more process / system**.

```
p uptime_secs $x
```

upto

`upto` — operation `upto`. Category: **python/ruby stdlib**.

```
p upto $x
```

uqb

`uqb` — operation `uqb`. Alias for `uniq_by`. Category: **python/ruby stdlib**.

```
p uqb $x
```

uri_normalize

`uri_normalize` — URI helper `normalize`. Category: **b81-misc-utility**.

```
p uri_normalize $x
```

uri_resolve

uri_resolve — URI helper [resolve](#). Category: *b81-misc-utility*.

```
p uri_resolve $x
```

uri_template_expand

uri_template_expand — URI helper [template](#) [expand](#). Category: *b81-misc-utility*.

```
p uri_template_expand $x
```

url_decode_form

url_decode_form — URL helper [decode](#) [form](#). Category: *b81-misc-utility*.

```
p url_decode_form $x
```

url_encode_form

url_encode_form — URL helper [encode](#) [form](#). Category: *b81-misc-utility*.

```
p url_encode_form $x
```

url_host

url_host — URL helper [host](#). Category: *url / email parts*.

```
p url_host $x
```

url_origin

url_origin — URL helper [origin](#). Category: *b81-misc-utility*.

```
p url_origin $x
```

url_path

url_path — URL helper [path](#). Category: *url / email parts*.

```
p url_path $x
```

url_query

url_query — URL helper [query](#). Category: *url / email parts*.

```
p url_query $x
```

url_scheme

url_scheme — URL helper [scheme](#). Category: *url / email parts*.

```
p url_scheme $x
```

urld

urld — operation [urld](#). Category: *extended stdlib*.

```
p urld $x
```

urle

`urle` — operation `urle`. Category: **extended stdlib**.

```
p urle $x
```

us_to_ns

`us_to_ns` — convert from `us` to `ns`. Category: **file stat / path**.

```
p us_to_ns $value
```

usamp

`usamp` — operation `usamp`. Alias for `upsample`. Category: **dsp / signal (extended)**.

```
p usamp $x
```

username

`username` — operation `username`. Category: **system introspection**.

```
p username $x
```

utc

`utc` — operation `utc`. Alias for `datetime_utc`. Category: **date / time**.

```
p utc $x
```

utci_simple

`utci_simple` — operation `utci simple`. Category: **misc**.

```
p utci_simple $x
```

utilitarian_solution

`utilitarian_solution` — operation `utilitarian solution`. Category: **climate, fluids, atmospheric**.

```
p utilitarian_solution $x
```

utm_from_lat_lng

`utm_from_lat_lng` — operation `utm from lat lng`. Category: **excel/sheets + bond/loan financial**.

```
p utm_from_lat_lng $x
```

utm_to_lat_lng

`utm_to_lat_lng` — convert from `utm` to `lat lng`. Category: **excel/sheets + bond/loan financial**.

```
p utm_to_lat_lng $value
```

utm_to_lat_lon

`utm_to_lat_lon(zone, easting, northing, hemi) → [lat, lon]` — UTM → geographic inverse.

```
p utm_to_lat_lon(18, 583959, 4507351, "N")
```

utm_zone

`utm_zone(lon_deg) → int` — UTM zone number 1-60 for given longitude.

```
p utm_zone(-122.4) # 10
```

uuencode_step

`uuencode_step` — single-step update of `uuencode`. Category: **archive/encoding format primitives**.

```
p uuencode_step $x
```

uuid_v4

`uuid_v4` — operation `uuid v4`. Category: **id helpers**.

```
p uuid_v4 $x
```

uzp

`uzp` — operation `uzp`. Alias for `unzip_pairs`. Category: **go/general functional utilities**.

```
p uzp $x
```

uzst

`uzst` — operation `uzst`. Alias for `zstd_decode`. Category: **steganography**.

```
p uzst $x
```

v_polynomial_eval

`v_polynomial_eval` — operation `v polynomial eval`. Category: **econometrics**.

```
p v_polynomial_eval $x
```

vaccination_coverage_required

`vaccination_coverage_required` — operation `vaccination coverage required`. Category: **epidemiology / public health**.

```
p vaccination_coverage_required $x
```

vacuum_energy_density

`vacuum_energy_density` — operation `vacuum energy density`. Category: **cosmology / gr / flrw**.

```
p vacuum_energy_density $x
```

vacuum_permeability

`vacuum_permeability` — operation `vacuum permeability`. Category: **physics constants**.

```
p vacuum_permeability $x
```

value_iteration_step

`value_iteration_step` — single-step update of `value iteration`. Category: **electrochemistry, batteries, fuel cells**.

```
p value_iteration_step $x
```

value_noise_2d

`value_noise_2d(x, y) → number` — interpolated random-value lattice. Cheaper than Perlin, more grid-aligned.

values_at

`values_at` — operation `values at`. Category: **python/ruby stdlib**.

```
p values_at $x
```

values_sorted

`values_sorted` — operation `values sorted`. Category: **hash ops**.

```
p values_sorted $x
```

van_leer_limiter

`van_leer_limiter` — operation `van leer limiter`. Category: **ode advanced**.

```
p van_leer_limiter $x
```

van_t_hoff_kp

`van_t_hoff_kp` — operation `van t hoff kp`. Category: **chemistry & biochemistry**.

```
p van_t_hoff_kp $x
```

vang

`vang` — operation `vang`. Alias for `vector_angle`. Category: **geometry (extended)**.

```
p vang $x
```

vant_hoff

`vant_hoff` — operation `vant hoff`. Category: **chemistry**.

```
p vant_hoff $x
```

var_fit

`var_fit` — operation `var fit`. Category: **statsmodels**.

```
p var_fit $x
```

var_historical

`var_historical` — operation `var historical`. Category: **more extensions (2)**.

```
p var_historical $x
```

variance_decomposition

`variance_decomposition` — operation `variance decomposition`. Category: **econometrics**.

```
p variance_decomposition $x
```

varint_decode

`varint_decode(bytes)` → `int` — inverse varint decode.

varint_encode

`varint_encode(n)` → `array` — Protobuf-style variable-length 7-bit-per-byte encoding with high-bit continuation flag.

```
p varint_encode(300) # [0xAC, 0x02]
```

vary_match

`vary_match` — operation `vary match`. Category: **b81-misc-utility**.

```
p vary_match $x
```

vasicek_bond

`vasicek_bond` — operation `vasicek bond`. Category: **financial pricing models**.

```
p vasicek_bond $x
```

vat

`vat` — operation `vat`. Alias for `values_at`. Category: **python/ruby stdlib**.

```
p vat $x
```

vcg_payment

`vcg_payment` — operation `vcg payment`. Category: **economics + game theory**.

```
p vcg_payment $x
```

vcg_payment_step

`vcg_payment_step` — single-step update of `vcg payment`. Category: **climate, fluids, atmospheric**.

```
p vcg_payment_step $x
```

VCROSS

`vector_cross(\@a, \@b)` (alias `vcross`) — computes the cross product of two 3D vectors. Returns a 3-element arrayref perpendicular to both inputs.

```
val $c = vcross([1,0,0], [0,1,0])
p @$c # (0, 0, 1)
```

vdot

`vector_dot(\@a, \@b)` (alias `vdot`) — computes the dot product of two vectors. Returns the scalar sum of element-wise products.


```
p vdot([1,2,3], [4,5,6]) # 32
```

vec3_add

`vec3_add(a, b) → [x,y,z]` — componentwise vector addition.

```
p vec3_add([1,2,3], [4,5,6]) # [5,7,9]
```

vec3_cross

`vec3_cross(a, b) → [x,y,z]` — cross product $\mathbf{a} \times \mathbf{b}$.

```
p vec3_cross([1,0,0], [0,1,0]) # [0,0,1]
```

vec3_distance

`vec3_distance(a, b) → number` — Euclidean distance between two points.

vec3_dot

`vec3_dot(a, b) → number` — dot product $\sum a_i \cdot b_i$.

```
p vec3_dot([1,0,0], [0,1,0]) # 0
```

vec3_length

`vec3_length(v) → number` — Euclidean magnitude.

```
p vec3_length([3,0,4]) # 5
```

vec3_lerp

`vec3_lerp(a, b, t) → [x,y,z]` — linear interpolation with `t` clamped to [0,1].

vec3_normalize

`vec3_normalize(v) → [x,y,z]` — unit-length direction; returns [0,0,0] for zero vector.

vec3_project

`vec3_project(v, u) → [x,y,z]` — projection of `v` onto direction `u`: $(\mathbf{v} \cdot \mathbf{u} / |\mathbf{u}|^2) \cdot \mathbf{u}$.

vec3_reflect

`vec3_reflect(v, n) → [x,y,z]` — reflect `v` about unit-normal `n`.

vec3_refract

`vec3_refract(v, n, eta) → [x,y,z]` — Snell's law refraction with index ratio `eta`; returns 0 vector for total internal reflection.

vec3_scale

`vec3_scale(v, s) → [x,y,z]` — scalar multiplication.

vec3_sub

`vec3_sub(a, b)` → `[x,y,z]` — componentwise subtraction.

vec4_add

`vec4_add(a, b)` → `[x,y,z,w]` — 4-component componentwise addition.

vec4_dot

`vec4_dot(a, b)` → `number` — 4-component dot product.

vec4_length

`vec4_length(v)` → `number` — 4-component magnitude.

vec4_scale

`vec4_scale(v, s)` → `[x,y,z,w]` — scalar multiply.

vec4_sub

`vec4_sub(a, b)` → `[x,y,z,w]` — 4-component subtraction.

vec_add

`vec_add` — operation `vec add`. Category: **matrix / linear algebra**.

```
p vec_add $x
```

vec_clj

`vec_clj` — operation `vec clj`. Category: **algebraic match**.

```
p vec_clj $x
```

vec_dot

`vec_dot` — operation `vec dot`. Category: **misc**.

```
p vec_dot $x
```

vec_l2

`vec_l2` — operation `vec l2`. Alias for `l2_norm`. Category: **misc**.

```
p vec_l2 $x
```

vec_normalize

`vec_normalize` — operation `vec normalize`. Category: **matrix / linear algebra**.

```
p vec_normalize $x
```

vec_outer

`vec_outer` — operation `vec outer`. Alias for `outer_product`. Category: **misc**.

```
p vec_outer $x
```

vec_scale

`vec_scale` — operation `vec scale`. Category: **matrix / linear algebra**.

```
p vec_scale $x
```

vec_set_value

`vec_set_value` — value of `vec set`. Category: **internal**.

```
p vec_set_value $x
```

vec_sub

`vec_sub` — operation `vec sub`. Category: **matrix / linear algebra**.

```
p vec_sub $x
```

vecc

`vecc` — operation `vecc`. Alias for `vec_clj`. Category: **algebraic match**.

```
p vecc $x
```

vecm_alpha_beta

`vecm_alpha_beta` — operation `vecm alpha beta`. Category: **econometrics**.

```
p vecm_alpha_beta $x
```

vecm_fit

`vecm_fit` — operation `vecm fit`. Category: **statsmodels**.

```
p vecm_fit $x
```

vector2_add

`vector2_add` — operation `vector2 add`. Category: **extras**.

```
p vector2_add $x
```

vector2_cross

`vector2_cross` — operation `vector2 cross`. Category: **extras**.

```
p vector2_cross $x
```

vector2_distance

`vector2_distance` — distance / dissimilarity metric of `vector2`. Category: **extras**.

```
p vector2_distance $x
```

vector2_dot

`vector2_dot` — operation `vector2 dot`. Category: *extras*.

```
p vector2_dot $x
```

vector2_length

`vector2_length` — operation `vector2 length`. Category: *extras*.

```
p vector2_length $x
```

vector2_new

`vector2_new` — constructor of `vector2`. Category: *extras*.

```
p vector2_new $x
```

vector2_normalize

`vector2_normalize` — operation `vector2 normalize`. Category: *extras*.

```
p vector2_normalize $x
```

vector2_rotate

`vector2_rotate` — operation `vector2 rotate`. Category: *extras*.

```
p vector2_rotate $x
```

vector2_scale

`vector2_scale` — operation `vector2 scale`. Category: *extras*.

```
p vector2_scale $x
```

vector2_sub

`vector2_sub` — operation `vector2 sub`. Category: *extras*.

```
p vector2_sub $x
```

vector_add

`vector_add(a, b) → array` — componentwise addition.

vector_angle

`vector_angle(\@a, \@b)` (alias `vangle`) — computes the angle in radians between two vectors using the dot product formula.

```
p vangle([[1,0], [0,1]]) # ~1.5708 (π/2)
p vangle([[1,0], [1,0]]) # 0
```

vector_magnitude

`vector_magnitude(\@v)` (alias `vmag`) — computes the Euclidean length (L2 norm) of a vector.

```
p vmag([3, 4])    # 5
p vmag([1, 2, 2]) # 3
```

vector_mean

`vector_mean(v)` → `number` — arithmetic mean.

vector_normalize

`vector_normalize(\@v)` (alias `vnorm`) — returns a unit vector in the same direction. Divides by magnitude.

```
val $u = vnorm([3, 4])
p @$u    # (0.6, 0.8)
```

vector_project

`vector_project` — operation `vector project`. Category: `*misc*`.

```
p vector_project $x
```

vector_reject

`vector_reject` — operation `vector reject`. Category: `*misc*`.

```
p vector_reject $x
```

vector_scale

`vector_scale(v, s)` → `array` — scalar multiply.

vector_sub

`vector_sub(a, b)` → `array` — componentwise subtraction.

velocity_potential_step

`velocity_potential_step` — single-step update of `velocity potential`. Category: `*climate, fluids, atmospheric*`.

```
p velocity_potential_step $x
```

verlinde_count_step

`verlinde_count_step` — single-step update of `verlinde count`. Category: `*econometrics*`.

```
p verlinde_count_step $x
```

verma_character_step

`verma_character_step` — single-step update of `verma character`. Category: `*econometrics*`.

```
p verma_character_step $x
```

verner_8_combine

`verner_8_combine` — operation `verner 8 combine`. Category: `*ode advanced*`.

```
p verner_8_combine $x
```

versin

`versin` — operation `versin`. Category: **trig extensions**.

```
p versin $x
```

versine

`versine` — operation `versine`. Alias for `versin`. Category: **trig extensions**.

```
p versine $x
```

versine_fn

`versine_fn` — operation `versine fn`. Category: **complex / geom / color / trig**.

```
p versine_fn $x
```

version_cmp

`version_cmp` — operation `version cmp`. Alias for `compare_versions`. Category: **file stat / path**.

```
p version_cmp $x
```

vertex_connectivity

`vertex_connectivity` — operation `vertex connectivity`. Category: **networkx graph algorithms**.

```
p vertex_connectivity $x
```

vertex_cover_lp_round

`vertex_cover_lp_round` — operation `vertex cover lp round`. Category: **combinatorial optimization, scheduling**.

```
p vertex_cover_lp_round $x
```

vf2_consistent

`vf2_consistent` — operation `vf2 consistent`. Category: **networkx graph algorithms**.

```
p vf2_consistent $x
```

vickrey_auction

`vickrey_auction` — operation `vickrey auction`. Category: **economics + game theory**.

```
p vickrey_auction $x
```

vigenere_decrypt

`vigenere_decrypt` — decrypt of `vigenere`. Category: **more extensions (2)**.

```
p vigenere_decrypt $x
```

vigenere_encrypt

`vigenere_encrypt` — encrypt of `vigenere`. Category: **more extensions (2)**.

```
p vigenere_encrypt $x
```

vin_check

`vin_check` — operation `vin check`. Category: *b82-misc-utility*.

```
p vin_check $x
```

vin_decode

`vin_decode` — decode of `vin`. Category: *validation / input checks*.

```
p vin_decode $x
```

vincenty_bearing

`vincenty_bearing(lat1, lon1, lat2, lon2) → number` — initial great-circle bearing in degrees (0-360, north = 0).

```
p vincenty_bearing(40.71, -74.00, 51.51, -0.13) # ~51°
```

vincenty_dist

`vincenty_dist` — operation `vincenty dist`. Category: *excel/sheets + bond/loan financial*.

```
p vincenty_dist $x
```

virtual_temperature_full

`virtual_temperature_full` — operation `virtual temperature full`. Category: *climate, fluids, atmospheric*.

```
p virtual_temperature_full $x
```

viterbi_decode

`viterbi_decode(obs, init, trans, emit) → array` — most likely HMM state path; log-space Viterbi.

```
p viterbi_decode([0,1,0], \@pi, \@A, \@B)
```

viterbi_pos_tag

`viterbi_pos_tag` — operation `viterbi pos tag`. Category: *computational linguistics*.

```
p viterbi_pos_tag $x
```

vlookup

`vlookup` — operation `vlookup`. Category: *excel/sheets + bond/loan financial*.

```
p vlookup $x
```

vo2_max_estimate

`vo2_max_estimate` — operation `vo2 max estimate`. Category: *misc*.

```
p vo2_max_estimate $x
```

void

`void` — operation `void`. Category: *conversion / utility*.

```
p void $x
```

voigt_profile

`voigt_profile` — operation `voigt profile`. Category: **numpy + scipy.special**.

```
p voigt_profile $x
```

voigt_profile_simple

`voigt_profile_simple` — operation `voigt profile simple`. Category: **more extensions (2)**.

```
p voigt_profile_simple $x
```

volatility_annualized

`volatility_annualized(returns, periods_per_year=252) → number` — `stddev(returns) · √periods_per_year`. Standard equity-return annualisation.

```
p volatility_annualized(\@daily_returns)
```

volatility_realized

`volatility_realized` — operation `volatility realized`. Category: **test runner**.

```
p volatility_realized $x
```

voltage_divider

`voltage_divider` — operation `voltage divider`. Category: **misc**.

```
p voltage_divider $x
```

voltaic_efficiency

`voltaic_efficiency` — operation `voltaic efficiency`. Category: **electrochemistry, batteries, fuel cells**.

```
p voltaic_efficiency $x
```

volume_form_riemannian

`volume_form_riemannian` — operation `volume form riemannian`. Category: **electrochemistry, batteries, fuel cells**.

```
p volume_form_riemannian $x
```

voronoi_cell_2d

`voronoi_cell_2d(point, others, bbox=[xmin,ymin,xmax,ymax]) → array` — Voronoi cell of `point` as intersection of half-planes within bounding box.

vqe_step

`vqe_step` — single-step update of `vqe`. Category: **quantum**.

```
p vqe_step $x
```

vsepr_geometry

`vsepr_geometry` — operation `vsepr geometry`. Category: **chemistry & biochemistry**.

```
p vsepr_geometry $x
```


vwap

`vwap(highs, lows, closes, volumes) → array` — Volume-Weighted Average Price: cumulative $\Sigma(\text{typical} \cdot \text{vol}) / \Sigma(\text{vol})$.

```
p vwap(\@hi, \@lo, \@cl, \@vol)
```

w_state

`w_state` — operation `w state`. Category: **quantum**.

```
p w_state $x
```

w_state_n

`w_state_n` — operation `w state n`. Category: **more extensions**.

```
p w_state_n $x
```

wacc

`wacc($equity, $debt, $cost_equity, $cost_debt, $tax_rate)` — computes Weighted Average Cost of Capital. Returns the blended cost of capital accounting for the tax shield on debt.

```
p wacc(60, 40, 0.10, 0.05, 0.25) # ~7.5%
```

wald_test_stat

`wald_test_stat` — operation `wald test stat`. Category: **econometrics**.

```
p wald_test_stat $x
```

walden_product

`walden_product` — operation `walden product`. Category: **electrochemistry, batteries, fuel cells**.

```
p walden_product $x
```

walker_circulation_step

`walker_circulation_step` — single-step update of `walker circulation`. Category: **climate, fluids, atmospheric**.

```
p walker_circulation_step $x
```

walksat_step

`walksat_step` — single-step update of `walksat`. Category: **logic, proof, sat/smt, type theory**.

```
p walksat_step $x
```

war_above_replacement

`war_above_replacement` — operation `war above replacement`. Category: **astronomy / astrometry**.

```
p war_above_replacement $x
```

warburg_impedance

`warburg_impedance` — operation `warburg impedance`. Category: **electrochemistry, batteries, fuel cells**.

```
p warburg_impedance $x
```

warped_product_step_zero

`warped_product_step_zero` — operation `warped product step zero`. Category: **electrochemistry, batteries, fuel cells**.

```
p warped_product_step_zero $x
```

warshall_step

`warshall_step` — single-step update of `warshall`. Category: **networkx graph algorithms**.

```
p warshall_step $x
```

wasserstein_dist_emp

`wasserstein_dist_emp` — operation `wasserstein dist emp`. Category: **electrochemistry, batteries, fuel cells**.

```
p wasserstein_dist_emp $x
```

watershed_segment

`watershed_segment` — operation `watershed segment`. Category: **pil/opencv image processing**.

```
p watershed_segment $x
```

wattersons_theta

`wattersons_theta` — operation `wattersons theta`. Category: **bioinformatics deep**.

```
p wattersons_theta $x
```

watts_to_hp

`watts_to_hp` — convert from `watts` to `hp`. Category: **file stat / path**.

```
p watts_to_hp $value
```

wav_header_read

`wav_header_read(hex_bytes) → hash` — parse WAV RIFF/fmt → `{format, channels, sample_rate, bits_per_sample}`.

wave_string_speed

`wave_string_speed` — operation `wave string speed`. Category: **em / optics / relativity**.

```
p wave_string_speed $x
```

wavelength_freq

`wavelength_freq` — operation `wavelength freq`. Category: **physics formulas**.

```
p wavelength_freq $x
```

wavetable_synth

`wavetable_synth(table, freq=440, sr=44100, dur=1) → array` — wavetable oscillator with linear interpolation; returns float samples.

```
my @sine = map { sin(2*3.14159*$_/256) } 0..255
p wavetable_synth(\@sine, 440, 44100, 0.1)
```

wc

`wc` — operation `wc`. Alias for `word_count`. Category: *trivial string ops*.

```
p wc $x
```

wcag_contrast

`wcag_contrast` — operation `wcag contrast`. Alias for `contrast_ratio_wcag`. Category: *misc*.

```
p wcag_contrast $x
```

wcag_luminance

`wcag_luminance` — operation `wcag luminance`. Alias for `relative_luminance`. Category: *misc*.

```
p wcag_luminance $x
```

wcag_pass

`wcag_pass` — operation `wcag pass`. Category: *b82-misc-utility*.

```
p wcag_pass $x
```

wcount

`wcount` — operation `wcount`. Alias for `count_words`. Category: *extended stdlib*.

```
p wcount $x
```

weak_composition_count

`weak_composition_count` — count of `weak composition`. Category: *misc*.

```
p weak_composition_count $x
```

weaken

`weaken` — operation `weaken`. Category: *list / aggregate*.

```
p weaken $x
```

weave_n

`weave_n` — operation `weave n`. Category: *iterator + string-distance extras*.

```
p weave_n $x
```

web_tmpl_empty

`web_tmpl_empty` — operation `web_tmpl_empty`. Category: **pipeline / string helpers**.

```
p web_tmpl_empty $x
```

web_tmpl_get

`web_tmpl_get` — operation `web_tmpl_get`. Category: **pipeline / string helpers**.

```
p web_tmpl_get $x
```

web_tmpl_section

`web_tmpl_section` — operation `web_tmpl_section`. Category: **pipeline / string helpers**.

```
p web_tmpl_section $x
```

weber_number

`weber_number` — operation `weber_number`. Category: **misc**.

```
p weber_number $x
```

weber_number_step

`weber_number_step` — single-step update of `weber_number`. Category: **climate, fluids, atmospheric**.

```
p weber_number_step $x
```

week_number_iso

`week_number_iso` — operation `week_number_iso`. Category: **b82-misc-utility**.

```
p week_number_iso $x
```

week_of_year

`week_of_year` — operation `week_of_year`. Category: **extended stdlib**.

```
p week_of_year $x
```

weekday_name

`weekday_name` — operation `weekday_name`. Category: **date helpers**.

```
p weekday_name $x
```

weekday_name_short

`weekday_name_short` — operation `weekday_name_short`. Category: **calendrical algorithms**.

```
p weekday_name_short $x
```

weekday_short

`weekday_short` — operation `weekday_short`. Category: **date helpers**.

```
p weekday_short $x
```

weeks_between

`weeks_between` — operation `weeks between`. Category: **misc**.

```
p weeks_between $x
```

weibpdf

`weibull_pdf` (alias `weibpdf`) evaluates the Weibull distribution PDF at `x` with shape `k` and scale `lambda`.

```
p weibpdf(1.0, 1.5, 1.0)
```

weighted_match_kuhn_step

`weighted_match_kuhn_step` — single-step update of `weighted match kuhn`. Category: **combinatorial optimization, scheduling**.

```
p weighted_match_kuhn_step $x
```

weighted_mean

`weighted_mean` — arithmetic mean of `weighted`. Category: **math / numeric extras**.

```
p weighted_mean @xs
```

weighted_sample

`weighted_sample` — operation `weighted sample`. Category: **extended stdlib**.

```
p weighted_sample $x
```

weighted_set_cover_step

`weighted_set_cover_step` — single-step update of `weighted set cover`. Category: **combinatorial optimization, scheduling**.

```
p weighted_set_cover_step $x
```

weighted_var

`weighted_var` — weighted variance. Args: values, weights.

```
p wvar([1,2,3,4], [1,1,1,1]) # same as var
```

weiler_atherton_clip

`weiler_atherton_clip` — operation `weiler atherton clip`. Category: **test runner**.

```
p weiler_atherton_clip $x
```

weingarten_map_step

`weingarten_map_step` — single-step update of `weingarten map`. Category: **electrochemistry, batteries, fuel cells**.

```
p weingarten_map_step $x
```

welch_psd_avg

welch_psd_avg — operation `welch psd avg`. Category: **economics + game theory**.

```
p welch_psd_avg $x
```

welch_ttest

welch_ttest (alias `welcht`) performs Welch's t-test for two independent samples with unequal variances. Returns [t-statistic, degrees of freedom].

```
var ($t, $df) = @{welcht([1,2,3,4,5], [3,4,5,6,7])}
p "t=$t df=$df"
```

welch_window

welch_window — operation `welch window`. Category: **signal processing**.

```
p welch_window $x
```

welford_mean

welford_mean `LIST` — single-pass online mean via Welford's algorithm. Numerically stable for long streams where naive `sum / len` overflows or loses precision.

welford_pop_variance

welford_pop_variance `LIST` — population variance (n denominator), counterpart to `welford_variance` (sample, n-1).

welford_sd

welford_stddev `LIST` — sqrt of `welford_variance`; numerically-stable sample standard deviation.

welford_var

welford_variance `LIST` — single-pass sample variance (n-1 denominator) via Welford's algorithm.

welsh_powell

welsh_powell — operation `welsh powell`. Category: **networkx graph algorithms**.

```
p welsh_powell $x
```

werckmeister_iii

werckmeister_iii — operation `werckmeister iii`. Category: **music theory**.

```
p werckmeister_iii $x
```

westerlies_jet_speed

westerlies_jet_speed — operation `westerlies jet speed`. Category: **climate, fluids, atmospheric**.

```
p westerlies_jet_speed $x
```

wet_bulb_potential

wet_bulb_potential — operation `wet bulb potential`. Category: **climate, fluids, atmospheric**.

```
p wet_bulb_potential $x
```

weyl_group_order

weyl_group_order — operation `weyl group order`. Category: **econometrics**.

```
p weyl_group_order $x
```

weyl_tensor_step_zero

weyl_tensor_step_zero — operation `weyl tensor step zero`. Category: **electrochemistry, batteries, fuel cells**.

```
p weyl_tensor_step_zero $x
```

wfirm

wfirm — operation `wfirm`. Alias for `words_from`. Category: **go/general functional utilities**.

```
p wfirm $x
```

wh

wh — operation `wh`. Alias for `which`. Category: **filesystem extensions**.

```
p wh $x
```

wha

Return a list of all absolute paths matching the given command name across every directory in `PATH`, not just the first match. The short alias `wha` keeps things concise. This is useful for detecting shadowed executables or auditing which versions of a tool are installed.

```
val @all = which_all("python3")
@all |> e p          # /usr/local/bin/python3
                   # /usr/bin/python3
p scalar wha("perl") # number of perls on PATH
```

when_false

when_false — operation `when false`. Category: **functional combinators**.

```
p when_false $x
```

when_true

when_true — operation `when true`. Category: **functional combinators**.

```
p when_true $x
```

whereis

`whereis(CMD)` → list of every match for `CMD` on `$PATH`, then every man-page file in `$MANPATH` (or `/usr/share/man` if unset). Returns an array of absolute paths.

```
val @hits = whereis("git")
# /usr/bin/git, /usr/share/man/man1/git.1, ...
```

which_fn

`which_fn` — return indices where a predicate is true. Like R's `which()`.

```
val @idx = @{which_fn([1,5,3,8,2], fn { _0 > 3 })} # [1, 3]
```

white

`white` — operation `white`. Alias for `red`. Category: **color / ansi**.

```
p white $x
```

white_bold

`white_bold` — operation `white bold`. Alias for `red`. Category: **color / ansi**.

```
p white_bold $x
```

white_point_d50

`white_point_d50` — operation `white point d50`. Category: **complex / geom / color / trig**.

```
p white_point_d50 $x
```

white_point_d65

`white_point_d65` — operation `white point d65`. Category: **complex / geom / color / trig**.

```
p white_point_d65 $x
```

white_robust_se

`white_robust_se` — operation `white robust se`. Category: **econometrics**.

```
p white_robust_se $x
```

white_test_stat

`white_test_stat` — operation `white test stat`. Category: **econometrics**.

```
p white_test_stat $x
```

whittaker_m

`whittaker_m` — operation `whittaker m`. Category: **special functions extra**.

```
p whittaker_m $x
```

whoami

`whoami` → current user name. Wraps `libc::getuid()` + the existing `getpwuid` lookup. Returns the bare string username (NOT the passwd-list array that `getpwuid` returns in list context).

```
p "running as: " . whoami
```


whole_life_a

`whole_life_a` — operation `whole life a`. Category: **actuarial science**.

```
p whole_life_a $x
```

wieferich_check

`wieferich_check` — operation `wieferich check`. Category: **cryptanalysis & number theory deep**.

```
p wieferich_check $x
```

wien

`wien` — operation `wien`. Alias for `wien_displacement`. Category: **physics formulas**.

```
p wien $x
```

wien_constant

`wien_constant` — operation `wien constant`. Category: **physics constants**.

```
p wien_constant $x
```

wien_displacement

`wien_displacement` — operation `wien displacement`. Category: **physics formulas**.

```
p wien_displacement $x
```

wiener_filter

`wiener_filter` — filter op of `wiener`. Category: **economics + game theory**.

```
p wiener_filter $x
```

wigner_3j_bound

`wigner_3j_bound` — operation `wigner 3j bound`. Category: **quantum mechanics deep**.

```
p wigner_3j_bound $x
```

wilcoxon

`wilcoxon` — operation `wilcoxon`. Category: **r / scipy distributions and tests**.

```
p wilcoxon $x
```

wilcoxon_signed_rank

`wilcoxon_signed_rank(a, b) → {statistic, df, p_value}` — paired non-parametric test on ranks of `|a-b|`.

```
p wilcoxon_signed_rank(\@before, \@after)
```

williams_r

`williams_r(highs, lows, closes, period=14) → array` — Williams %R: $-100 \cdot (\text{highest_high} - \text{close}) / (\text{highest_high} - \text{lowest_low})$. Mirrors stochastic-K in [-100, 0].

```
p williams_r(\@hi, \@lo, \@cl, 14)
```

wilson_test

`wilson_test` — statistical test of `wilson`. Category: *cryptanalysis & number theory deep*.

```
p wilson_test $x
```

wilson_theta_step

`wilson_theta_step` — single-step update of `wilson theta`. Category: *ode advanced*.

```
p wilson_theta_step $x
```

win_probability

`win_probability` — operation `win probability`. Category: *astronomy / astrometry*.

```
p win_probability $x
```

wincirc

`wincirc` — operation `wincirc`. Alias for `windowed_circular`. Category: *additional missing stdlib functions*.

```
p wincirc $x
```

wind_chill

`wind_chill` — operation `wind chill`. Category: *physics formulas*.

```
p wind_chill $x
```

wind_chill_celsius

`wind_chill_celsius` — operation `wind chill celsius`. Category: *misc*.

```
p wind_chill_celsius $x
```

window_bartlett

`window_bartlett(n)` → `array` — triangular (Bartlett) window; zero at endpoints.

window_blackman_harris

`window_blackman_harris(n)` → `array` — 4-term Blackman-Harris window. Very low side-lobes (~-92 dB).

window_flat_top

`window_flat_top(n)` → `array` — flat-top window. Trades wider main-lobe for flat passband (amplitude calibration).

window_gaussian

`window_gaussian(n, sigma=0.4)` → `array` — Gaussian window of length `n`.

window_n

`window_n` — windowing function for DSP / FFT `n`. Category: **go/general functional utilities**.

```
p window_n $x
```

window_tukey

`window_tukey` — windowing function for DSP / FFT `tukey`. Category: **extras**.

```
p window_tukey $x
```

window_welch

`window_welch(n)` → `array` — Welch (parabolic) window.

windowed_circular

`windowed_circular` — operation `windowed circular`. Category: **additional missing stdlib functions**.

```
p windowed_circular $x
```

winn

`winn` — operation `winn`. Alias for `window_n`. Category: **go/general functional utilities**.

```
p winn $x
```

winsor

`winsor` — operation `winsor`. Alias for `winsorize`. Category: **math / numeric extras**.

```
p winsor $x
```

winsorize

`winsorize` — operation `winsorize`. Category: **math / numeric extras**.

```
p winsorize $x
```

winsorized_variance_arr

`winsorized_variance_arr` — operation `winsorized variance arr`. Category: **more extensions**.

```
p winsorized_variance_arr $x
```

wire_resistance

`wire_resistance` — operation `wire resistance`. Category: **misc**.

```
p wire_resistance $x
```

without

`without` — operation `without`. Category: **file stat / path**.

```
p without $x
```

without_nth

`without_nth` — operation `without_nth`. Category: **file stat / path**.

```
p without_nth $x
```

wma

`wma(prices, period=10) → array` — weighted moving average. Newer values get higher linear weights (1, 2, ..., period); divides by triangular-number total.

```
p wma([10,11,12,13,14], 3)
```

wmean

`wmean` — operation `wmean`. Alias for `weighted_mean`. Category: **math / numeric extras**.

```
p wmean $x
```

woba_weight

`woba_weight` — operation `woba_weight`. Category: **astronomy / astrometry**.

```
p woba_weight $x
```

wofz

`wofz` — operation `wofz`. Category: **numpy + scipy.special**.

```
p wofz $x
```

wolfe_strong_q

`wolfe_strong_q` — operation `wolfe_strong_q`. Category: **more extensions (2)**.

```
p wolfe_strong_q $x
```

woodall_number

`woodall_number` — operation `woodall_number`. Category: **misc**.

```
p woodall_number $x
```

word_alignment_ibm1

`word_alignment_ibm1` — operation `word_alignment_ibm1`. Category: **computational linguistics**.

```
p word_alignment_ibm1 $x
```

word_alignment_ibm2

`word_alignment_ibm2` — operation `word_alignment_ibm2`. Category: **computational linguistics**.

```
p word_alignment_ibm2 $x
```

word_count

`word_count` — count of `word`. Category: *trivial string ops*.

```
p word_count $x
```

word_count_simple

`word_count_simple` — operation `word count simple`. Alias for `string_words_count`. Category: *misc*.

```
p word_count_simple $x
```

word_frequencies

`word_frequencies` — operation `word frequencies`. Category: *string processing extras*.

```
p word_frequencies $x
```

word_similarity

`word_similarity` — similarity metric of `word`. Category: *postgres sql strings, json, aggregates*.

```
p word_similarity $x
```

word_wrap

`word_wrap(s, width=80) → string` — break a string into lines no longer than `width`, splitting on whitespace.

```
p word_wrap("the quick brown fox", 10)
```

word_wrap_text

`word_wrap_text` — operation `word wrap text`. Category: *extended stdlib*.

```
p word_wrap_text $x
```

wordfreq

`wordfreq` — operation `wordfreq`. Alias for `word_frequencies`. Category: *string processing extras*.

```
p wordfreq $x
```

words_from

`words_from` — operation `words from`. Category: *go/general functional utilities*.

```
p words_from $x
```

work

`work($force, $distance, $angle)` — compute work $W = F \times d \times \cos(\theta)$ (Joules). Angle in degrees, default 0.

```
p work(100, 10)      # 1000 J (force parallel to motion)
p work(100, 10, 60)  # 500 J (force at 60°)
```

worley_2d

`worley_2d(x, y) → number` — F1 cellular (Worley) noise: distance to nearest random feature point.

woy

`woy` — operation `woy`. Alias for `week_of_year`. Category: **extended stdlib**.

```
p woy $x
```

wpa

`wpa` — operation `wpa`. Category: **astronomy / astrometry**.

```
p wpa $x
```

wrap_around

`wrap_around` — operation `wrap around`. Category: **trig / math**.

```
p wrap_around $x
```

wrap_index

`wrap_index` — index of `wrap`. Category: **list helpers**.

```
p wrap_index $x
```

wrap_range

`wrap_range` — operation `wrap range`. Category: **extended stdlib**.

```
p wrap_range $x
```

wrap_text

`wrap_text` — operation `wrap text`. Category: **extended stdlib**.

```
p wrap_text $x
```

wrc_plus

`wrc_plus` — operation `wrc plus`. Category: **astronomy / astrometry**.

```
p wrc_plus $x
```

wright_omega

`wright_omega` — operation `wright omega`. Category: **special functions extra**.

```
p wright_omega $x
```

writhe_polygon

`writhe_polygon` — operation `writhe polygon`. Category: **econometrics**.

```
p writhe_polygon $x
```

wiping

`wiping` — operation `wiping`. Alias for `wrap_range`. Category: **extended stdlib**.

```
p wiping $x
```

wrpt

`wrpt` — operation `wrpt`. Alias for `wrap_text`. Category: **extended stdlib**.

```
p wrpt $x
```

ws_close_frame

`ws_close_frame` — WebSocket helper `close frame`. Category: **network / ip / cidr**.

```
p ws_close_frame $x
```

ws_frame_decode

`ws_frame_decode` — WebSocket helper `frame decode`. Category: **network / ip / cidr**.

```
p ws_frame_decode $x
```

ws_frame_encode

`ws_frame_encode` — WebSocket helper `frame encode`. Category: **network / ip / cidr**.

```
p ws_frame_encode $x
```

ws_handshake_accept

`ws_handshake_accept` — WebSocket helper `handshake accept`. Category: **network / ip / cidr**.

```
p ws_handshake_accept $x
```

ws_handshake_key

`ws_handshake_key` — WebSocket helper `handshake key`. Category: **network / ip / cidr**.

```
p ws_handshake_key $x
```

ws_mask

`ws_mask` — WebSocket helper `mask`. Category: **network / ip / cidr**.

```
p ws_mask $x
```

ws_unmask

`ws_unmask` — WebSocket helper `unmask`. Category: **network / ip / cidr**.

```
p ws_unmask $x
```

wsamp

`wsamp` — operation `wsamp`. Alias for `weighted_sample`. Category: **extended stdlib**.

```
p wsamp $x
```

wwrap

`wwrap` — operation `wwrap`. Alias for `word_wrap_text`. Category: **extended stdlib**.

```
p wwrap $x
```

www_auth_parse

`www_auth_parse` — operation `www_auth_parse`. Category: **b81-misc-utility**.

```
p www_auth_parse $x
```

wyhash_mix

`wyhash_mix` — operation `wyhash_mix`. Category: **cryptography deep**.

```
p wyhash_mix $x
```

wynn_epsilon

`wynn_epsilon` — operation `wynn_epsilon`. Category: **more extensions**.

```
p wynn_epsilon $x
```

x25519_field_mul

`x25519_field_mul` — operation `x25519_field_mul`. Category: **more extensions (2)**.

```
p x25519_field_mul $x
```

xadd

`xadd` — operation `xadd`. Category: **redis-flavour primitives**.

```
p xadd $x
```

xargs

`xargs(\&CALLABLE, \@ARGS)` → invoke `CALLABLE` once per element of `\@ARGS`, returning a list of results. Sequential; for parallel use `pmap` directly. Composes naturally with `tac` / `comm` / pipe forwarding.

```
val @hashes = xargs(\&sha256, \@filenames)
xargs(\&open_url, ["https://a.com", "https://b.com"])
```

xattr_get

`xattr_get(path, name)` → `string` — read an extended attribute value.

xattr_list

`xattr_list(path)` → `array` — names of all extended attributes.

xattr_set

`xattr_set(path, name, value)` → `0|1` — write an extended attribute.

xd

`xd` — operation `xd`. Alias for `xml_decode`. Category: **serialization (stryke-only encoders)**.

```
p xd $x
```


xe

`xe` — operation `xe`. Alias for `xml_encode`. Category: *serialization (stryke-only encoders)*.

```
p xe $x
```

xentropy

`xentropy` — operation `xentropy`. Alias for `cross_entropy`. Category: *extras*.

```
p xentropy $x
```

xfip

`xfip` — operation `xfip`. Category: *astronomy / astrometry*.

```
p xfip $x
```

xirr

`xirr(\@cashflows, \@dates)` — computes IRR for irregularly-spaced cash flows. Dates can be epoch timestamps or date strings. More accurate than `irr` for real-world investments.

```
val @cf = (-1000, 500, 600)
var @dates = ('2020-01-01', '2020-07-01', '2021-01-01')
p xirr(\@cf, \@dates)
```

xlen

`xlen` — operation `xlen`. Category: *redis-flavour primitives*.

```
p xlen $x
```

xlookup

`xlookup` — operation `xlookup`. Category: *excel/sheets + bond/loan financial*.

```
p xlookup $x
```

xml_attrs

`xml_attrs` — XML helper `attrs`. Category: *extras*.

```
p xml_attrs $x
```

xml_canonicalize

`xml_canonicalize` — XML helper `canonicalize`. Category: *extras*.

```
p xml_canonicalize $x
```

xml_children_by_tag

`xml_children_by_tag` — XML helper `children by tag`. Category: *extras*.

```
p xml_children_by_tag $x
```

xml_minify

xml_minify — XML helper [minify](#). Category: *extras*.

```
p xml_minify $x
```

xml_namespace

xml_namespace — XML helper [namespace](#). Category: *extras*.

```
p xml_namespace $x
```

xml_pretty

xml_pretty — XML helper [pretty](#). Category: *extras*.

```
p xml_pretty $x
```

xml_root

xml_root — XML helper [root](#). Category: *extras*.

```
p xml_root $x
```

xml_text

xml_text — XML helper [text](#). Category: *extras*.

```
p xml_text $x
```

xml_to_json

xml_to_json — convert from [xml](#) to [json](#). Category: *extras*.

```
p xml_to_json $value
```

xmlagg

xmlagg — operation [xmlagg](#). Category: *postgres sql strings, json, aggregates*.

```
p xmlagg $x
```

xmlforest

xmlforest — operation [xmlforest](#). Category: *postgres sql strings, json, aggregates*.

```
p xmlforest $x
```

xnpv

xnpv — operation [xnpv](#). Category: *excel/sheets + bond/loan financial*.

```
p xnpv $x
```

xor_bool

xor_bool — operation [xor](#) [bool](#). Category: *boolean combinators*.

```
p xor_bool $x
```

xor_break_singlebyte

`xor_break_singlebyte` — operation `xor break singlebyte`. Category: *cryptanalysis & number theory deep*.

```
p xor_break_singlebyte $x
```

xor_brute_keylen

`xor_brute_keylen` — operation `xor brute keylen`. Category: *more extensions (2)*.

```
p xor_brute_keylen $x
```

xor_byte_string

`xor_byte_string` — operation `xor byte string`. Category: *more extensions (2)*.

```
p xor_byte_string $x
```

xor_cipher_byte

`xor_cipher_byte` — operation `xor cipher byte`. Category: *cryptography deep*.

```
p xor_cipher_byte $x
```

xor_strings

`xor_strings` — operation `xor strings`. Category: *string helpers*.

```
p xor_strings $x
```

xorshift32

`xorshift32` — operation `xorshift32`. Category: *cryptography deep*.

```
p xorshift32 $x
```

xorshift32_next

`xorshift32_next(seed)` → `int` — Marsaglia's xorshift32 step.

xorshift32_step

`xorshift32_step` — single-step update of `xorshift32`. Category: *misc*.

```
p xorshift32_step $x
```

xorshift64

`xorshift64` — operation `xorshift64`. Category: *cryptography deep*.

```
p xorshift64 $x
```

xpath_attribute

`xpath_attribute` — operation `xpath attribute`. Category: *extras*.

```
p xpath_attribute $x
```

xpath_select_one

`xpath_select_one` — operation `xpath select one`. Category: *extras*.

```
p xpath_select_one $x
```

xpath_text

`xpath_text` — operation `xpath text`. Category: *extras*.

```
p xpath_text $x
```

xpath_to_selector

`xpath_to_selector` — convert from `xpath` to `selector`. Category: *extras*.

```
p xpath_to_selector $value
```

xrange

`xrange` — operation `xrange`. Category: *redis-flavour primitives*.

```
p xrange $x
```

xtea_round

`xtea_round` — operation `xtea round`. Category: *more extensions*.

```
p xtea_round $x
```

xxd

`xxd PATH` — hex dump (`xxd(1)` layout: offset, paired hex, ASCII). Optional second positional sets bytes per line (default 16, max 32). Trailing hash: `cols => N, max => BYTES` (default 1_048_576 bytes read). No path uses `$_`.

```
p xxd("Cargo.toml", 8)
```

xyz_d50_to_d65

`xyz_d50_to_d65` — convert from `xyz d50` to `d65`. Category: *complex / geom / color / trig*.

```
p xyz_d50_to_d65 $value
```

xyz_d65_to_d50

`xyz_d65_to_d50` — convert from `xyz d65` to `d50`. Category: *complex / geom / color / trig*.

```
p xyz_d65_to_d50 $value
```

xyz_to_lab

`xyz_to_lab` — convert from `xyz` to `lab`. Category: *astronomy / music / color / units*.

```
p xyz_to_lab $value
```

xyz_to_rgb

`xyz_to_rgb` — convert from `xyz` to `rgb`. Category: **b82-misc-utility**.

```
p xyz_to_rgb $value
```

yamanouchi_word_count

`yamanouchi_word_count` — count of `yamanouchi` `word`. Category: **econometrics**.

```
p yamanouchi_word_count $x
```

yards_per_attempt

`yards_per_attempt` — operation `yards` `per` `attempt`. Category: **astronomy / astrometry**.

```
p yards_per_attempt $x
```

yards_to_m

`yards_to_m` — convert from `yards` to `m`. Category: **unit conversions**.

```
p yards_to_m $value
```

yd

`yd` — operation `yd`. Alias for `yaml_decode`. Category: **serialization (stryke-only encoders)**.

```
p yd $x
```

ye

`ye` — operation `ye`. Alias for `yaml_encode`. Category: **serialization (stryke-only encoders)**.

```
p ye $x
```

years_between

`years_between` — operation `years` `between`. Category: **misc**.

```
p years_between $x
```

yellow

`yellow` — operation `yellow`. Alias for `red`. Category: **color / ansi**.

```
p yellow $x
```

yellow_bold

`yellow_bold` — operation `yellow` `bold`. Alias for `red`. Category: **color / ansi**.

```
p yellow_bold $x
```

yen_k_shortest

`yen_k_shortest` — operation `yen` `k` `shortest`. Category: **networkx graph algorithms**.

```
p yen_k_shortest $x
```

yesterday

`yesterday` — operation `yesterday`. Category: `*date*`.

```
p yesterday $x
```

yield_to_maturity

`yield_to_maturity(coupons, face_value, price) → number` — discount rate that equates bond price to PV of coupon + face cashflows (Newton solver).

```
p yield_to_maturity([50,50,50], 1000, 950)
```

yieldddisc

`yieldddisc` — operation `yieldddisc`. Category: `*b81-misc-utility*`.

```
p yieldddisc $x
```

yieldmat

`yieldmat` — operation `yieldmat`. Category: `*b81-misc-utility*`.

```
p yieldmat $x
```

ylll_pml

`ylll_pml` — operation `ylll pml`. Category: `*epidemiology / public health*`.

```
p ylll_pml $x
```

yn

`yn` — operation `yn`. Category: `*numpy + scipy.special*`.

```
p yn $x
```

ytm_newton

`ytm_newton` — operation `ytm newton`. Category: `*financial pricing models*`.

```
p ytm_newton $x
```

yuv_to_rgb

`yuv_to_rgb` — convert from `yuv` to `rgb`. Category: `*b82-misc-utility*`.

```
p yuv_to_rgb $value
```

z85_decode

`z85_decode(s) → string` — ZeroMQ Base85 decode; input length must be a multiple of 5.

z85_encode

`z85_encode(s) → string` — ZeroMQ Base85 (RFC 32/Z85). Input length must be a multiple of 4.

```
p z85_encode("12345678")
```

z_array

`z_array` — operation `z array`. Category: **misc**.

```
p z_array $x
```

z_func

`z_func` — operation `z func`. Alias for `z_array`. Category: **misc**.

```
p z_func $x
```

z_function_alt

`z_function_alt` — operation `z function alt`. Category: **more extensions**.

```
p z_function_alt $x
```

z_score

`z_score($value, $mean, $stddev)` — computes the z-score (standard score) of a value given the population mean and standard deviation. Returns how many standard deviations the value is from the mean. Useful for standardizing data and detecting outliers.

```
val $z = z_score(85, 75, 10)
p $z    # 1 (one stddev above mean)
p z_score(50, 60, 5)    # -2
```

z_score_robust

`z_score_robust` — operation `z score robust`. Category: **misc**.

```
p z_score_robust $x
```

z_scores

`z_scores(@data)` — computes z-scores for all values in a list, returning an arrayref of standardized values. Each z-score indicates how many standard deviations that value is from the mean of the dataset. Useful for comparing values across different scales.

```
val @grades = (70, 80, 90)
var $zs = z_scores(@grades)
p @$zs    # [-1, 0, 1]
@data |> z_scores |> e p
```

zadd

`zadd` — operation `zadd`. Category: **redis-flavour primitives**.

```
p zadd $x
```

zalgo

`zalgo` — operation `zalgo`. Category: **string processing extras**.

```
p zalgo $x
```

zall

`zall` — operation `zall`. Alias for `zip_all`. Category: **go/general functional utilities**.

```
p zall $x
```

zcard

`zcard` — operation `zcard`. Category: **redis-flavour primitives**.

```
p zcard $x
```

zcount

`zcount` — operation `zcount`. Category: **redis-flavour primitives**.

```
p zcount $x
```

zcross

`zcross` — operation `zcross`. Alias for `zero_crossings`. Category: **array / list operations extras**.

```
p zcross $x
```

zeeman_energy

`zeeman_energy` — operation `zeeman energy`. Category: **quantum mechanics deep**.

```
p zeeman_energy $x
```

zeller_day_of_week

`zeller_day_of_week` — operation `zeller day of week`. Category: **misc**.

```
p zeller_day_of_week $x
```

zeno_survival

`zeno_survival` — operation `zeno survival`. Category: **quantum mechanics deep**.

```
p zeno_survival $x
```

zernike_radial

`zernike_radial(n, m, rho) → number` — Zernike radial polynomial $R_n^{|m|}(\rho)$. Returns 0 if $(n-|m|)$ is odd or $|m| > n$. Negative `m` is accepted and uses $|m|$ per the convention $R_n^m = R_n^{-m}$.

zero

`zero` — operation `zero`. Category: **trivial numeric / predicate builtins**.

```
p zero $x
```

zero_crossing_rate

`zero_crossing_rate` — rate of `zero crossing`. Category: **extras**.

```
p zero_crossing_rate $x
```


zero_crossings

`zero_crossings` — operation `zero crossings`. Category: *array / list operations extras*.

```
p zero_crossings $x
```

zero_order_concentration

`zero_order_concentration` — operation `zero order concentration`. Category: *chemistry*.

```
p zero_order_concentration $x
```

zero_pad

`zero_pad` — operation `zero pad`. Category: *signal processing*.

```
p zero_pad $x
```

zero_sum_minmax

`zero_sum_minmax` — operation `zero sum minmax`. Category: *climate, fluids, atmospheric*.

```
p zero_sum_minmax $x
```

zero_sum_value

`zero_sum_value(payoff)` → `number` — value of a two-player zero-sum game. Returns the saddle-point value when `maximin == minmax`, the 2×2 mixed-strategy value $(ad - bc)/(a + d - b - c)$ otherwise, or UNDEF for larger matrices with no saddle (caller should use LP).

zeros

`zeros` — operation `zeros`. Category: *matrix / linear algebra*.

```
p zeros $x
```

zeros_mat

`zeros_mat` — operation `zeros mat`. Alias for `zeros_matrix`. Category: *matrix / linear algebra*.

```
p zeros_mat $x
```

zeros_matrix

`zeros_matrix` — operation `zeros matrix`. Category: *matrix / linear algebra*.

```
p zeros_matrix $x
```

zeta_potential_estimate

`zeta_potential_estimate` — operation `zeta potential estimate`. Category: *electrochemistry, batteries, fuel cells*.

```
p zeta_potential_estimate $x
```

zeta_riemann

`zeta_riemann(s)` → `number` — Riemann zeta $\zeta(s) = \sum 1/n^s$ for $n=1..∞$ (truncated series).

```
p zeta_riemann(2) # π²/6 ≈ 1.645
```

zfill

`zfill` — operation `zfill`. Category: **extended stdlib**.

```
p zfill $x
```

zfill_num

`zfill_num` — operation `zfill num`. Category: **extended stdlib**.

```
p zfill_num $x
```

zgemm

`zgemm` — operation `zgemm`. Category: **blas / lapack**.

```
p zgemm $x
```

zigzag_decode

`zigzag_decode(n) → int` — inverse ZigZag.

zigzag_encode

`zigzag_encode(n) → int` — map signed int to unsigned via Protobuf ZigZag: $(n \ll 1) \wedge (n \gg 63)$. Lets small magnitudes pack into few bytes.

```
p zigzag_encode(-1) # 1
p zigzag_encode(1)  # 2
```

zincrby

`zincrby` — operation `zincrby`. Category: **redis-flavour primitives**.

```
p zincrby $x
```

zip_all

`zip_all` — operation `zip all`. Category: **go/general functional utilities**.

```
p zip_all $x
```

zip_apply

`zip_apply` — operation `zip apply`. Category: **functional combinators**.

```
p zip_apply $x
```

zip_archive

`zip_archive` — operation `zip archive`. Alias for `zip_create`. Category: **steganography**.

```
p zip_archive $x
```

zip_central_dir

`zip_central_dir` — operation `zip central dir`. Category: **archive/encoding format primitives**.

```
p zip_central_dir $x
```

zip_central_directory

`zip_central_directory(hex_bytes)` → `hash` — scan for EOCD signature, return {`entries`, `cd_size`, `cd_offset`}.

zip_eocd

`zip_eocd` — operation `zip eocd`. Category: **archive/encoding format primitives**.

```
p zip_eocd $x
```

zip_fill

`zip_fill` — operation `zip fill`. Category: **python/ruby stdlib**.

```
p zip_fill $x
```

zip_local_file_header

`zip_local_file_header(hex_bytes)` → `hash` — parse local file header → {`compression`, `crc32`, `compressed_size`, `uncompressed_size`}.

zip_local_header

`zip_local_header` — operation `zip local header`. Category: **archive/encoding format primitives**.

```
p zip_local_header $x
```

zip_with

`zip_with` — operation `zip with`. Alias for `take_while`. Category: **functional / iterator**.

```
p zip_with $x
```

zipf

`zipf` — operation `zipf`. Alias for `zip_fill`. Category: **python/ruby stdlib**.

```
p zipf $x
```

zipl

`zipl` — operation `zipl`. Category: **python/ruby stdlib**.

```
p zipl $x
```

zipmap

`zipmap` — operation `zipmap`. Category: **file stat / path**.

```
p zipmap $x
```

ziv_lempel_estimate

`ziv_lempel_estimate` — operation `ziv lempel estimate`. Category: **electrochemistry, batteries, fuel cells**.

```
p ziv_lempel_estimate $x
```

zlexcount

zlexcount — operation **zlexcount**. Category: *redis-flavour primitives*.

```
p zlexcount $x
```

zodiac_sign

zodiac_sign(month, day) → string — Western tropical zodiac sign for a given month/day.

```
p zodiac_sign(7, 15) # Cancer
```

zpexpand

zpexpand(TEMPLATE, name, value, ...) — run a zsh `${ ... }` **parameter expansion** and return the result. World-first: zsh parameter-expansion semantics outside zsh, backed by `zsh::ported::subst::params` from zshrs — single source of truth, zero stryke-side reimplementation, so every operator and flag from `man zshexpn` works.

Normally you don't call this directly: stryke's double-quoted strings expand `${ ... }` natively (`"${(U)x}"`, `"${f##*}"`, `"${a[2]}"`), lowering to **zpexpand** for you. Call it explicitly for the cases the string sugar can't disambiguate at parse time, supplying each referenced variable's **real type** so zshrs sees pure zsh semantics:

```
var @arr = (1, 2, 3, 4)
p zpexpand( "${#arr}"', "arr", \@arr)      # 4 - array element count (not string length)
p zpexpand( "${arr[1]}"', "arr", \@arr)     # 1 - pure zsh 1-indexing
val $f = "deploy.sh"
p zpexpand( "${1:-stage}"', "1", $f)       # deploy.sh - explicit positional, not capture $1
```

Argument convention: the first argument is the raw `${ ... }` template (single-quote it so stryke doesn't pre-interpolate); the rest are **name**, **value** pairs bridged into zshrs's parameter table. **Pass arrays by reference** (`\@arr`, not `@arr`) so the list does not flatten into the pairs — `setaparam` then makes it a zsh array; scalars go in via `setsparam`. Supports defaults (`:-/:=/:?/:+`), trim (`#/##/%/%%`), substitution (`//`), **length** (`${#x}`), **flags** (`(U)/(L)/...`), and **subscripts + subscript flags** (`${a[2,4]}`, `${a[(i)v]}`, `${a[(r)pat]}`).

See also: `glob` (the same zshrs-delegation model for filename generation).

zpk2sos_step

zpk2sos_step — single-step update of **zpk2sos**. Category: *economics + game theory*.

```
p zpk2sos_step $x
```

zpk2tf_step

zpk2tf_step — single-step update of **zpk2tf**. Category: *economics + game theory*.

```
p zpk2tf_step $x
```

zrangebyscore

zrangebyscore — operation **zrangebyscore**. Category: *redis-flavour primitives*.

```
p zrangebyscore $x
```

zrank

zrank — operation **zrank**. Category: *redis-flavour primitives*.

```
p zrank $x
```

zrem

`zrem` — operation `zrem`. Category: *redis-flavour primitives*.

```
p zrem $x
```

zrevrank

`zrevrank` — operation `zrevrank`. Category: *redis-flavour primitives*.

```
p zrevrank $x
```

zrlc

`zrlc` — operation `zrlc`. Alias for `impedance_rlc`. Category: *physics formulas*.

```
p zrlc $x
```

zscore

`zscore` — operation `zscore`. Category: *file stat / path*.

```
p zscore $x
```

zscore_count

`zscore_count` — count of `zscore`. Category: *bioinformatics deep*.

```
p zscore_count $x
```

zscore_norm

`zscore_norm` — norm (vector length-like) of `zscore`. Category: *ml extensions*.

```
p zscore_norm $x
```

zscores

`zscores` — operation `zscores`. Alias for `z_scores`. Category: *statistics (extended)*.

```
p zscores $x
```

zst

`zst` — operation `zst`. Alias for `zstd`. Category: *steganography*.

```
p zst $x
```

zstd_encode_step

`zstd_encode_step` — single-step update of `zstd encode`. Category: *b81-misc-utility*.

```
p zstd_encode_step $x
```

zstd_frame_header

`zstd_frame_header` — operation `zstd frame header`. Category: *archive/encoding format primitives*.

```
p zstd_frame_header $x
```

zstd_window_size_log

`zstd_window_size_log` — operation `zstd window size log`. Category: *electrochemistry, batteries, fuel cells*.

```
p zstd_window_size_log $x
```